

Sentaurus Device User Guide

Version G-2012.06, June 2012

SYNOPSYS®

Copyright Notice and Proprietary Information

Copyright © 2012 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Right to Copy Documentation

The license agreement with Synopsys permits licensee to make copies of the documentation for its internal use only. Each copy shall include all copyrights, trademarks, service marks, and proprietary rights notices, if any. Licensee must assign sequential numbers to all copies. These copies shall contain the following legend on the cover page:

"This document is duplicated with the permission of Synopsys, Inc., for the exclusive use of _____ and its employees. This is copy number _____."

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Registered Trademarks (®)

Synopsys, AEON, AMPS, ARC, Astro, Behavior Extracting Synthesis Technology, Cadabra, CATS, Certify, CHIPit, CODE V, CoMET, Confirma, Design Compiler, DesignSphere, DesignWare, Eclipse, Formality, Galaxy Custom Designer, Global Synthesis, HAPS, HapsTrak, HDL Analyst, HSiM, HSPICE, Identify, Leda, LightTools, MAST, MaVeric, METeor, ModelTools, NanoSim, NOVeA, OpenVera, ORA, PathMill, Physical Compiler, PrimeTime, SCOPE, SiVL, SNUG, SolvNet, Sonic Focus, STAR Memory System, SVP Café, Syndicated, Synplicity, the Synplicity logo, Synplify, Synplify Pro, Synthesis Constraints Optimization Environment, TetraMAX, UMRBus, VCS, Vera, and YieldExplorer are registered trademarks of Synopsys, Inc.

Trademarks (™)

AFGen, Apollo, ASAP, Astro-Rail, Astro-Xtalk, Aurora, AvanWaves, BEST, Columbia, Columbia-CE, Cosmos, CosmosLE, CosmosScope, CRITIC, Custom WaveView, CustomExplorer, CustomSim, DC Expert, DC Professional, DC Ultra, Design Analyzer, Design Vision, DesignerHDL, DesignPower, DFTMAX, Direct Silicon Access, Discovery, Encore, EPIC, Galaxy, HANEX, HDL Compiler, Hercules, Hierarchical Optimization Technology, High-performance ASIC Prototyping System, HSiM^{plus}, i-Virtual Stepper, IC Compiler, ICE, in-Sync, iN-Tandem, Intelli, Jupiter, Jupiter-DP, JupiterXT, JupiterXT-ASIC, Liberty, Libra-Passport, Library Compiler, Macro-PLUS, Magellan, Mars, Mars-Rail, Mars-Xtalk, Milkyway, ModelSource, Module Compiler, MultiPoint, ORAengineering, Physical Analyst, Planet, Planet-PL, Platform Architect, Polaris, Power Compiler, Processor Designer, Raphael, RippledMixer, Saturn, Scirocco, Scirocco-i, SiWare, SPW, StarRC, Star-RCXT, Star-SimXT, Symphony Model Compiler, System Compiler, System Designer, System Studio, Taurus, TotalRecall, TSUPREM-4, VCSi, VHDL Compiler, Virtualizer, VMC, and Worksheet Buffer are trademarks of Synopsys, Inc.

Service Marks (SM)

MAP-in and TAP-in are service marks of Synopsys, Inc.

SystemC is a trademark of the Open SystemC Initiative and is used under license.

ARM and AMBA are registered trademarks of ARM Limited.

Saber is a registered trademark of SabreMark Limited Partnership and is used under license.

Entrust is a registered trademark of Entrust Inc. in the United States and in certain other countries. In Canada Entrust is a trademark or registered trademark of Entrust Technologies Limited. Used by Entrust.net Inc. under license.

All other product or company names may be trademarks of their respective owners.

Contents

About This Guide	xxxvii
Audience	xxxvii
Related Publications	xxxviii
Typographic Conventions	xxxviii
Customer Support	xxxix
Accessing SolvNet	xxxix
Contacting Synopsys Support	xxxix
Contacting Your Local TCAD Support Team Directly	xl
Acknowledgments	xl

Part I Getting Started **1**

Chapter 1 Overview	3
About Sentaurus Device	3
Creating and Meshing Device Structures	5
Tool Flow	6
Starting Sentaurus Device	6
From Command Line	6
From Sentaurus Workbench	8
Simulation Examples	8
Example: Simple MOSFET Id–Vg Simulation	8
Command File	9
File Section	10
Electrode Section	11
Physics Section	12
Plot Section	13
Math Section	13
Solve Section	14
Simulated Id–Vg Characteristic	15
Analysis of 2D Output Data	17
Example: Advanced Hydrodynamic Id–Vd Simulation	18
Command File	18
File Section	21
Main Options	22
Parameter File	22
Listing of mos.par	23
Report in Protocol File n3_des.log	23

Contents

Electrode Section	23
Main Options	23
Physics Section	24
Main Options	25
Interface Physics	26
Main Options	26
Plot Section	27
CurrentPlot Section	27
Main Options	27
Math Section	28
Example	28
Solve Section	29
Two-dimensional Output Data.	33
Example: Mixed-Mode CMOS Inverter Simulation.	33
Command File	34
Device Section.	36
System Section.	37
File Section	38
Plot Section	38
Math Section	39
Solve Section	39
Results of Inverter Transient Simulation.	40
Example: Small-Signal AC Extraction	41
Command File	41
Device Section.	43
File Section	44
System Section.	44
Solve Section	44
Results of AC Simulation	46
 Chapter 2 Defining Devices	 47
Reading a Structure.	47
Abrupt and Graded Heterojunctions	48
Doping Specification.	49
User-Defined Species	50
Material Specification.	52
User-Defined Materials	52
Mole-Fraction Materials	53
Mole-Fraction Specification	55
Physical Models and the Hierarchy of Their Specification	57
Region-specific and Material-specific Models	57

Interface-specific Models	59
Electrode-specific Models	60
Physical Model Parameters	60
Parameters for Composition-dependent Materials	62
Ternary Semiconductor Composition	63
Quaternary Semiconductor Composition	66
Default Model Parameters for Compound Semiconductors	67
Combining Parameter Specifications	68
Materialwise Parameters	68
Regionwise Parameters	69
Material Interface-wise Parameters	69
Region Interface-wise Parameters	69
Electrode-wise Parameters	70
Generating a Copy of Parameter File	70
Undefined Physical Models	71
Default Parameters	73
Named Parameter Sets	74
Auto-Orientation Framework	75
Changing Orientations Used with Auto-Orientation	76
Auto-Orientation Smoothing	76
References	77

Chapter 3 Mixed-Mode Sentaurus Device	79
Overview	79
Compact Models	80
Hierarchical Description of Compact Models	80
Example: Compact Models	83
SPICE Circuit Files	85
Device Section	86
System Section	87
Physical Devices	88
Circuit Devices	89
Electrical and Thermal Netlist	90
Set, Unset, Initialize, and Hint	92
System Plot	93
AC System Plot	93
File Section	94
SPICE Circuit Models	95
User-Defined Circuit Models	95
Mixed-Mode Math Section	96
Using Mixed-Mode Simulation	96

Contents

From Single-Device File to Multidevice File	96
File-naming Convention: Mixed-Mode Extension	98

Chapter 4 Performing Numeric Experiments	99
Specifying Electrical Boundary Conditions	99
Changing Boundary Condition Type During Simulation	100
Mixed-Mode Electrical Boundary Conditions	101
Specifying Thermal Boundary Conditions	101
Break Criteria: Conditionally Stopping the Simulation	102
Global Contact Break Criteria	103
Global Device Break Criteria	103
Sweep-specific Break Criteria	104
Mixed-Mode Break Criteria	105
Quasistationary Ramps	106
Ramping Boundary Conditions	106
Ramping Quasi-Fermi Potentials in Doping Wells	107
Ramping Physical Parameter Values	109
Quasistationary in Mixed Mode	111
Saving and Plotting During a Quasistationary	112
Extrapolation	113
Additional Features	115
Relaxed Newton Parameters	115
Continuation Command	116
Transient Command	119
Numeric Control of Transient Analysis	121
Time-stepping	122
Ramping Physical Parameter Values	123
Extrapolation	123
Additional Features	124
Relaxed Newton Parameters	124
Large-Signal Cyclic Analysis	124
Description of Method	125
Using Cyclic Analysis	127
Small-Signal AC Analysis	127
Example: AC Analysis of Simple Device	129
Optical AC Analysis	130
Harmonic Balance	130
Modes of Harmonic Balance Analysis	131
MDFT Mode	131
SDFT Mode	132
Performing Harmonic Balance Analysis	132

Solve Spectrum	133
Convergence Parameters	134
Harmonic Balance Analysis Output	134
Device Instance Currents, Voltages, Temperatures, and Heat Components	135
Circuit Currents and Voltages	135
Solution Variables	135
Application Notes	135
References	136

Chapter 5 Simulation Results	137
-------------------------------------	------------

Current File	137
When to Write to the Current File	137
Example: CurrentPlot Statements	138
NewCurrentPrefix Statement	139
Tracking Additional Data in the Current File	140
Example: Node Numbers	142
Example: Mixed Mode	142
Example: Advanced Options	142
Example: Plotting Parameter Values	143
CurrentPlot Options	143
Device Plots	144
What to Plot	144
When to Plot	145
Snapshots	147
Interface Plots	147
Log File	148
Extraction File	149
Extraction File Format	149
Analysis Modes	150
File Section	151
Electrode Section	151
Extraction Section	152
Solve Section	152

Chapter 6 Numeric and Software-related Issues	155
--	------------

Structure of Command File	155
Inserting Files	156
Solve Section: How the Simulation Proceeds	157
Nonlinear Iterations	158
Coupled Command	158

Contents

Coupled Error Control	159
Damped Newton Iterations.	161
Derivatives	161
Incomplete Newton Algorithm.	161
Additional Equations Available in Mixed Mode	162
Selecting Individual Devices in Mixed Mode	163
Plugin Command	164
Linear Solvers	165
Nonlocal Meshes.	166
Specifying Nonlocal Meshes	166
Visualizing Nonlocal Meshes	167
Visualizing Data Defined on Nonlocal Meshes.	167
Constructing Nonlocal Meshes	168
Specification Using Barrier	169
Specification Using a Reference Surface	169
Special Handling of 1D Schrödinger Equation	171
Special Handling of Nonlocal Tunneling Model.	171
Unnamed Meshes.	172
Performance Suggestions.	172
Monitoring Convergence Behavior.	172
CNormPrint	173
NewtonPlot	173
Save and Load.	174
Tcl Command File	176
Overview	176
sdevice Command	178
sdevice_init Command	179
sdevice_solve Command	179
sdevice_finish Command.	179
sdevice_parameters Command	179
Flowchart	180
Extraction.	180
List of Available Inspect Tcl Commands.	181
Output Redirection.	182
Known Restrictions	182
Parallelization	183
Extended Precision	185
System Command	187

Part II Physics in Sentaurus Device**189**

Chapter 7 Electrostatic Potential and Quasi-Fermi Potentials	191
Electrostatic Potential	191
Dipole Layer	192
Quasi-Fermi Potential with Boltzmann Statistics	193
Fermi Statistics	194
Using Fermi Statistics	195
Initial Guess for Electrostatic Potential and Quasi-Fermi Potentials in Doping Wells .	195
Regionwise Specification of Initial Quasi-Fermi Potentials	196
Chapter 8 Carrier Transport in Semiconductors	197
Introduction to Carrier Transport Models.	197
Drift-Diffusion Model.	198
Thermodynamic Model for Current Densities	199
Hydrodynamic Model for Current Densities	200
Numeric Parameters for Continuity Equation.	200
Numeric Approaches for Contact Current Computation	201
Current Potential	201
References.	202
Chapter 9 Temperature Equations	205
Introduction to Temperature Equations	205
Uniform Self-Heating	206
Using Uniform Self-Heating	207
Default Model for Lattice Temperature	208
Thermodynamic Model for Lattice Temperature	209
Using the Thermodynamic Model	209
Hydrodynamic Model for Temperatures.	211
Hydrodynamic Model Parameters	214
Using the Hydrodynamic Model	214
Numeric Parameters for Temperature Equations	215
References.	215
Chapter 10 Boundary Conditions	217
Electrical Boundary Conditions	217
Ohmic Contacts	217
Contacts on Insulators	218

Contents

Schottky Contacts	219
Barrier Lowering at Schottky Contacts	220
Resistive Contacts	222
Boundaries Without Contacts	225
Floating Contacts	225
Floating Metal Contacts.....	225
Floating Semiconductor Contacts	227
Thermal Boundary Conditions	228
Boundary Conditions for Lattice Temperature	228
Boundary Conditions for Carrier Temperatures	229
Periodic Boundary Conditions	230
Robin PBC Approach	230
Mortar PBC Approach.....	231
Specifying Periodic Boundary Conditions	231
Specifying Robin Periodic Boundary Conditions	231
Specifying Mortar Periodic Boundary Conditions.....	232
Application Notes.....	232
References.....	233
Chapter 11 Transport in Metals, Organic Materials, and Disordered Media	235
Singlet Exciton Equation	235
Boundary and Continuity Conditions for Singlet Exciton Equation.....	236
Using the Singlet Exciton Equation.....	237
Transport in Metals.....	239
Electric Boundary Conditions for Metals	240
Metal Workfunction	242
Metal Workfunction Randomization	243
Temperature in Metals.....	244
Conductive Insulators	244
Chapter 12 Semiconductor Band Structure	249
Intrinsic Density	249
Band Gap and Electron Affinity	249
Selecting the Bandgap Model	250
Bandgap and Electron-Affinity Models.....	250
Bandgap Narrowing for Bennett–Wilson Model	251
Bandgap Narrowing for Slotboom Model	252
Bandgap Narrowing for del Alamo Model.....	252
Bandgap Narrowing for Jain–Roulston Model.....	252
Table Specification of Bandgap Narrowing.....	254

Schenk Bandgap Narrowing Model	255
Bandgap Narrowing with Fermi Statistics	259
Bandgap Parameters	260
Effective Masses and Effective Density-of-States	261
Electron Effective Mass and DOS	261
Formula 1	261
Formula 2	262
Electron Effective Mass and Conduction Band DOS Parameters	262
Hole Effective Mass and DOS	263
Formula 1	263
Formula 2	263
Hole Effective Mass and Valence Band DOS Parameters	264
Gaussian Density-of-States for Organic Semiconductors	264
Multivalley Band Structure	267
Nonparabolic Band Structure	268
Using Multivalley Band Structure	269
References	271

Chapter 13 Incomplete Ionization	273
---	------------

Overview	273
Using Incomplete Ionization	274
Incomplete Ionization Model	274
Physical Model Parameters	276
References	277

Chapter 14 Quantization Models	279
---------------------------------------	------------

Overview	279
van Dort Quantization Model	280
van Dort Model	280
Using the van Dort Model	281
1D Schrödinger Solver	281
Nonlocal Mesh for 1D Schrödinger	282
Using 1D Schrödinger	283
1D Schrödinger Parameters	283
Explicit Ladder Specification	284
Automatic Extraction of Ladder Parameters	284
Visualizing Schrödinger Solutions	286
1D Schrödinger Model	286
1D Schrödinger Application Notes	287
Density Gradient Quantization Model	288

Contents

Density Gradient Model	288
Using the Density Gradient Model	289
Named Parameter Sets for Density Gradient	291
Auto-Orientation for Density Gradient	291
Density Gradient Application Notes	292
Modified Local-Density Approximation	293
MLDA Model	293
Interface Orientation and Stress Dependencies	293
Using MLDA	295
MLDA Application Notes	298
References	299

Chapter 15 Mobility Models	301
How Mobility Models Combine	301
Mobility due to Phonon Scattering	302
Doping-dependent Mobility Degradation	302
Using Doping-dependent Mobility	303
Masetti Model	304
Arora Model	304
University of Bologna Bulk Mobility Model	305
The pmi_msc_mobility Model	307
PMIs for Bulk Mobility	308
Carrier–Carrier Scattering	309
Using Carrier–Carrier Scattering	309
Conwell–Weisskopf Model	309
Brooks–Herring Model	310
Physical Model Parameters	310
Philips Unified Mobility Model	310
Using Philips Model	311
Using an Alternative Philips Model	311
Philips Model Description	311
Screening Parameter	313
Philips Model Parameters	314
Mobility Degradation at Interfaces	315
Using Mobility Degradation at Interfaces	315
Enhanced Lombardi Model	316
Named Parameter Sets for Lombardi Model	319
Orientation-dependent Lombardi Parameters	319
Enhanced Lombardi Model with High-k Degradation	319
Inversion and Accumulation Layer Mobility Model	323
Using Inversion and Accumulation Layer Mobility Model	328

Named Parameter Sets for IALMob	329
Auto-Orientation for IALMob	330
University of Bologna Surface Mobility Model	330
Mobility Degradation Components due to Coulomb Scattering	332
Using Mobility Degradation Components	334
Computing Transverse Field	335
Normal to Interface	336
Normal to Current Flow	336
Field Correction on Interface	337
Thin-Layer Mobility Model	337
Using the Thin-Layer Mobility Model	338
Thickness Extraction	340
High-Field Saturation	342
Using High-Field Saturation	342
Named Parameter Sets for High-Field Saturation	343
Extended Canali Model	343
Transferred Electron Model	344
Basic Model	345
Meinerzhagen–Engl Model	345
Physical Model Interface	346
Lucent Model	346
Velocity Saturation Models	347
Selecting Velocity Saturation Models	347
Driving Force Models	348
Field Correction Close to Interfaces	349
Non-Einstein Diffusivity	350
Monte Carlo–computed Mobility for Strained Silicon	351
Monte Carlo–computed Mobility for Strained SiGe in npn-SiGe HBTs	351
Incomplete Ionization–dependent Mobility Models	352
Poole–Frenkel Mobility (Organic Material Mobility)	353
Mobility Averaging	354
Mobility Doping File	354
Effective Mobility	355
References	356

Chapter 16 Generation–Recombination 359

Shockley–Read–Hall Recombination	359
Using SRH Recombination	360
SRH Doping Dependence	361
Lifetime Profiles from Files	361
SRH Temperature Dependence	362

Contents

SRH Doping- and Temperature-dependent Parameters	363
SRH Field Enhancement	363
Using Field Enhancement	364
Schenk Trap-assisted Tunneling (TAT) Model	364
Schenk TAT Density Correction	366
Hurkx TAT Model	366
Field-Enhancement Parameters	367
Dynamic Nonlocal Path Trap-assisted Tunneling	367
Recombination Rate	368
Using Dynamic Nonlocal Path TAT Model	369
Trap-assisted Auger Recombination	371
Surface SRH Recombination	372
Coupled Defect Level (CDL) Recombination	373
Using CDL	373
CDL Model	374
Radiative Recombination	375
Using Radiative Recombination	375
Radiative Model	375
Auger Recombination	376
Avalanche Generation	377
Using Avalanche Generation	377
van Overstraeten – de Man Model	378
Okuto–Crowell Model	379
Lackner Model	380
University of Bologna Impact Ionization Model	380
New University of Bologna Impact Ionization Model	382
Hatakeyama Avalanche Model	384
Driving Force	386
Anisotropic Coordinate System	386
Usage	386
Driving Force	387
Avalanche Generation with Hydrodynamic Transport	387
Approximate Breakdown Analysis: Poisson Equation Approach	389
Using Breakdown Analysis	389
Band-to-Band Tunneling Models	391
Using Band-to-Band Tunneling	391
Schenk Model	392
Schenk Density Correction	393
Simple Band-to-Band Models	393
Hurkx Band-to-Band Model	394
Tunneling Near Interfaces and Equilibrium Regions	395

Dynamic Nonlocal Path Band-to-Band Model	395
Band-to-Band Generation Rate	396
Using Nonlocal Path Band-to-Band Model	399
Visualizing Nonlocal Band-to-Band Generation Rate	400
Bimolecular Recombination	401
Physical Model	401
Using Bimolecular Recombination	401
Exciton Dissociation Model	402
Physical Model	402
Using Exciton Dissociation	402
References.....	403
 Chapter 17 Traps and Fixed Charges	 407
Basic Syntax for Traps	407
Trap Types	408
Energetic and Spatial Distribution of Traps	408
Specifying Single Traps.....	410
Trap Randomization	411
Trap Models and Parameters.....	412
Trap Occupation Dynamics	412
Local Trap Capture and Emission	414
J-Model Cross Sections	415
Hurkx Model for Cross Sections	415
Poole–Frenkel Model for Cross Sections.....	415
Local Capture and Emission Rates Based on Makram-Ebeid–Lannoo Phonon-assisted Tunnel Ionization Model	416
Local Capture and Emission Rates from PMI	417
Tunneling and Traps	417
Trap Numeric Parameters	419
Visualizing Traps	419
Explicit Trap Occupation	421
Trap Examples	422
References.....	423
 Chapter 18 Phase and State Transitions	 425
Multistate Configurations and Their Dynamic	425
Specifying Multistate Configurations.....	427
Transition Models	428
The pmi_ce_msc Model.....	428
States.....	429

Contents

Transitions	430
Model Parameters	432
Interaction of Multistate Configurations with Transport	435
Apparent Band-Edge Shift	435
The pmi_msc_abes Model	436
Thermal Conductivity, Heat Capacity, and Mobility	437
Manipulating MSCs During Solve	437
Explicit State Occupations	437
Manipulating Transition Dynamics	438
Example: Two-State Phase-Change Memory Model	439
References	440
Chapter 19 Degradation Model	441
Overview	441
Trap Degradation Model	441
Trap Formation Kinetics	441
Power Law and Kinetic Equation	442
Si-H Density-dependent Activation Energy	442
Diffusion of Hydrogen in Oxide	443
Syntax and Parameterized Equations	444
Device Lifetime and Simulation	446
MSC-Hydrogen Transport Degradation Model	449
Hydrogen Transport	450
Reactions Between Mobile Elements	450
Reactions with Multistate Configurations	453
The CEModel_Depassivation Model	454
Using MSC-Hydrogen Transport Degradation Model	456
Two-Stage NBTI Degradation Model	457
Formulation	458
Using Two-Stage NBTI Model	461
References	462
Chapter 20 Organic Devices	465
Introduction to Organic Device Simulation	465
References	466
Chapter 21 Optical Generation	469
Overview	469
Specifying the Type of Optical Generation Computation	470

Optical Generation from Monochromatic Source	472
Illumination Spectrum	472
Multidimensional Illumination Spectra	473
Loading and Saving Optical Generation from and to File	474
Constant Optical Generation	475
Quantum Yield Models	475
Optical Absorption Heat	477
Specifying Time Dependency for Transient Simulations	478
Solving the Optical Problem	480
Specifying the Optical Solver	480
Transfer Matrix Method	481
Finite-Difference Time-Domain Method	481
Raytracing	483
Beam Propagation Method	484
Loading Solution of Optical Problem from File	485
Optical Beam Absorption Method	485
Setting the Excitation Parameters	486
Illumination Window	487
Choosing Refractive Index Model	491
Extracting the Layer Stack	491
Controlling Computation of Optical Problem in Solve Section	493
Parameter Ramping	494
Complex Refractive Index Model	496
Physical Model	496
Wavelength Dependency	497
Temperature Dependency	497
Carrier Dependency	497
Gain Dependency	499
Using Complex Refractive Index	499
Complex Refractive Index Model Interface	504
C++ Application Programming Interface (API)	504
Shared Object Code	509
Command File of Sentaurus Device	509
Raytracing	510
Raytracer	510
Ray Photon Absorption and Optical Generation	513
Using the Raytracer	514
Terminating Raytracing	515
Monte Carlo Raytracing	515
Multithreading for Raytracer	516
Compact Memory Model for Raytracer	517

Contents

Window of Starting Rays	517
Rectangular Window of Rays	518
User-Defined Window of Rays	518
Distribution Window of Rays	518
Boundary Condition for Raytracing	520
Fresnel Boundary Condition	520
Constant Reflectivity and Transmittivity Boundary Condition	521
Raytrace PMI Boundary Condition	522
Thin-Layer-Stack Boundary Condition	523
TMM Optical Generation in Raytracer	524
Diffuse Surface Boundary Condition	526
Virtual Regions in Raytracer	528
Additional Options for Raytracing	529
Redistributing Power of Stopped Rays	529
Visualizing Raytracing	530
Reporting Various Powers in Raytracing	530
Plotting Interface Flux	531
Far Field and Sensors for Raytracing	533
Dual-Grid Setup for Raytracing	536
Transfer Matrix Method	538
Physical Model	538
Rough Surface Scattering	540
Using Transfer Matrix Method	543
Using Scattering Solver	545
Loading Solution of Optical Problem from File	553
Importing 1D Profiles into Higher-dimensional Grids	555
Ramping Profile Index	556
Optical Beam Absorption Method	557
Physical Model	557
Using Optical Beam Absorption Method	558
Beam Propagation Method	559
Physical Model	560
Bidirectional BPM	560
Boundary Conditions	561
Using Beam Propagation Method	561
General	561
Bidirectional BPM	563
Excitation	563
Boundary	566
Ramping Input Parameters	567
Visualizing Results on Native Tensor Grid	567

Optical AC Analysis	568
References	569

Chapter 22 Radiation Models	571
------------------------------------	------------

Generation by Gamma Radiation	571
Using Gamma Radiation Model	571
Yield Function	572
Alpha Particles	572
Using Alpha Particle Model	572
Alpha Particle Model	573
Heavy Ions	574
Using Heavy Ion Model	574
Heavy Ion Model	575
Examples: Heavy Ions	577
Example 1	577
Example 2	578
Example 3	578
Improved Alpha Particle/Heavy Ion Generation Rate Integration	579
References	580

Chapter 23 Noise, Fluctuations, and Sensitivity	581
--	------------

Using the Impedance Field Method	581
Noise Sources	585
Diffusion Noise	585
Equivalent Monopolar Generation–Recombination Noise	586
Bulk Flicker Noise	586
Trapping Noise	586
Random Dopant Fluctuations	587
Random Geometric Fluctuations	588
Random Trap Concentration Fluctuations	590
Random Workfunction Fluctuations	590
Noise from SPICE Circuit Elements	591
Statistical Impedance Field Method	592
Doping Variations	593
Trap Concentration Variations	594
Workfunction Variations	594
Deterministic Variations	595
Deterministic Doping Variations	595
Deterministic Geometric Variations	596
Parameter Variations	597

Contents

Impedance Field Method	598
Noise Output Data.	601
References.....	603
Chapter 24 Tunneling	605
Tunneling Model Overview	605
Fowler–Nordheim Tunneling	606
Using Fowler–Nordheim	606
Fowler–Nordheim Model	607
Fowler–Nordheim Parameters	608
Direct Tunneling	608
Using Direct Tunneling	608
Direct Tunneling Model	609
Image Force Effect	610
Direct Tunneling Parameters	611
Nonlocal Tunneling at Interfaces, Contacts, and Junctions	612
Defining Nonlocal Meshes	612
Specifying Nonlocal Tunneling Model	614
Nonlocal Tunneling Parameters	615
Visualizing Nonlocal Tunneling	617
Physics of Nonlocal Tunneling Model	618
WKB Tunneling Probability	618
Schrödinger Equation–based Tunneling Probability	620
Density Gradient Quantization Correction	620
Nonlocal Tunneling Current	621
Band-to-Band Contributions to Nonlocal Tunneling Current	621
Carrier Heating	622
References	623
Chapter 25 Hot-Carrier Injection Models	625
Overview	625
Destination of Injected Current	626
Injection Barrier and Image Potential	628
Effective Field	630
Classical Lucky Electron Injection	630
Fiegna Hot-Carrier Injection	631
SHE Distribution Hot-Carrier Injection	633
Spherical Harmonics Expansion Method	634
Using Spherical Harmonics Expansion Method	638
Visualizing Spherical Harmonics Expansion Method	645

Carrier Injection with Explicitly Evaluated Boundary Conditions for Continuity Equations 647	
References.....	648
Chapter 26 Heterostructure Device Simulation	651
Thermionic Emission Current.....	651
Using Thermionic Emission Current.....	651
Thermionic Emission Model.....	652
Gaussian Transport Across Organic Heterointerfaces.....	653
Using Gaussian Transport at Organic Heterointerfaces.....	653
Gaussian Transport at Organic Heterointerface Model.....	654
References.....	654
Chapter 27 Energy-dependent Parameters	655
Overview.....	655
Energy-dependent Energy Relaxation Time.....	655
Spline Interpolation.....	657
Energy-dependent Mobility.....	658
Spline Interpolation.....	660
Energy-dependent Peltier Coefficient.....	661
Spline Interpolation.....	662
Chapter 28 Anisotropic Properties	665
Overview.....	665
Coordinate Systems.....	666
Cylindrical Symmetry.....	666
Anisotropic Direction.....	667
Orthogonal Matrix from Eigenvectors Q.....	668
Anisotropic Mobility.....	669
Crystal Reference System.....	669
Anisotropy Factor.....	669
Current Densities.....	670
Driving Forces.....	671
Total Anisotropic Mobility.....	672
Total Direction-dependent Anisotropic Mobility.....	673
Self-Consistent Anisotropic Mobility.....	673
Plot Section.....	675
Anisotropic Avalanche Generation.....	675
Anisotropic Electrical Permittivity.....	677

Contents

Anisotropic Thermal Conductivity	678
Anisotropic Density Gradient Model	679
Chapter 29 Ferroelectric Materials	681
Using Ferroelectrics	681
Ferroelectrics Model	683
References	685
Chapter 30 Modeling Mechanical Stress Effect	687
Overview	687
Stress and Strain in Semiconductors	687
Using Stress and Strain	689
Deformation of Band Structure	691
Using Deformation Potential Model	693
Strained Effective Masses and Density-of-States	695
Strained Electron Effective Mass and DOS	695
Strained Hole Effective Mass and DOS	697
Using Strained Effective Masses and DOS	698
Multivalley Band Structure	699
Using Multivalley Band Structure	699
Stress-dependent Mobility Models	700
Stress-induced Electron Mobility Model	700
Intervalley Scattering	702
Effective Mass	704
Inversion Layer	704
Using Stress-induced Electron Mobility Model	706
Stress-induced Hole Mobility Model	708
Effective Mass	708
Scattering	709
Using Stress-induced Hole Mobility Model	711
Intel Stress-induced Hole Mobility Model	713
Stress Dependencies	714
Generalization of Model	715
Using Intel Mobility Model	716
Piezoresistance Mobility Model	717
Doping and Temperature Dependency	719
Isotropic Mobility Factors	720
Using Piezoresistance Mobility Model	720
Named Parameter Sets for Piezoresistance	722
Auto-Orientation for Piezoresistance	722

Additional Factor Model Options	723
Enormal- and MoleFraction-dependent Piezo Coefficients	723
Using Piezoresistive Prefactors Model	723
Stress Mobility Model for Minority Carriers	729
Dependency of Saturation Velocity on Stress	731
Mobility Enhancement Limits	732
Plotting Mobility Enhancement Factors	733
Numeric Approximations for Tensor Mobility	733
Tensor Grid Option	733
Stress Tensor Applied to Low-Field Mobility	734
Piezoelectric Polarization	735
Strain Model	735
Stress Model	736
Poisson Equation	737
Parameter File	737
Coordinate Systems	738
Converse Piezoelectric Field	739
Gate-dependent Polarization in GaN Devices	739
Two-dimensional Simulations	740
References	740

Chapter 31 Galvanic Transport Model	743
--	------------

Model Description	743
Using Galvanic Transport Model	744
Discretization Scheme for Continuity Equations	744
References	744

Chapter 32 Thermal Properties	745
--------------------------------------	------------

Heat Capacity	745
The pmi_msc_heatcapacity Model	746
Thermal Conductivity	747
The pmi_msc_thermalconductivity Model	748
Thermoelectric Power (TEP)	749
Physical Model	749
Using Thermoelectric Power	751
Heating at Contacts, Metal–Semiconductor and Conductive Insulator–Semiconductor Interfaces	752
References	753

Part III Physics of Lasers and Light-Emitting Diodes**755**

Chapter 33 Introduction to Lasers and Light-Emitting Diodes	757
Overview	757
Command File Syntax	759
Simulating Single-Grid Edge-Emitting Laser	760
Simulating Dual-Grid Edge-Emitting Laser	763
Simulating Vertical-Cavity Surface-Emitting Laser	768
Investigating Simulation Results	772
Current File and Plot Variables for Laser Simulation	773
 Chapter 34 Lasers	 777
Overview	777
Coupling Between Optics and Electronics	778
Algorithm for Coupling Electrical and Optical Problems	779
Photon Rate and Photon Phase Equations	780
Laser Small-Signal AC Analysis and Modulation Response	783
Intensity and Photon Phase Modulation Response	783
Syntax for Small-Signal AC Extraction	784
Modeling Relative Intensity Noise and Frequency Noise	785
Relative Intensity Noise and Frequency Noise	785
Small-Signal Noise Modeling	786
Syntax for Relative Intensity Noise and Frequency Noise Model	787
Plotting the Laser Power Spectrum	788
Refractive Index, Dispersion, and Optical Loss	789
Temperature Dependence of Refractive Index	789
Carrier-Density Dependence of Refractive Index	790
Wavelength Dependence and Absorption of Refractive Index	791
Free Carrier Loss	792
Edge-Emitting Lasers	793
Waveguide Optical Modes and Fabry–Perot Cavity	793
Lasing Wavelength in Fabry–Perot Cavity	795
Output Power from a Fabry–Perot Laser	795
Symmetry Considerations	796
Multiple Transverse Modes	796
Multiple Longitudinal Modes	797
Specifying Fixed Optical Confinement Factor	798
Simple Distributed Feedback Model	798
Bulk Active-Region Edge-Emitting Lasers	799
Leaky Waveguide Lasers	799

Device Physics and Parameter Tuning	800
Vertical-Cavity Surface-Emitting Lasers	801
Cavity Optical Modes in VCSELs.....	801
VCSEL Output Power	804
Cylindrical Symmetry	805
Different Grid and Structure for Electrical and Optical Problems	806
Aligning Resonant Wavelength Within Gain Spectrum	807
Approximate Methods for VCSEL Cavity Problem	808
Device Physics and Parameter Tuning	808
References.....	809

Chapter 35 Light-Emitting Diodes	811
Modeling Light-Emitting Diodes	811
Coupling Electronics and Optics in LED Simulations	812
Single-Grid Versus Dual-Grid LED Simulation	812
Electrical Transport in LEDs	813
Spontaneous Emission Rate and Power.....	813
Spontaneous Emission Power Spectrum	814
Current File and Plot Variables for LED Simulation	815
LED Wavelength	817
Optical Absorption Heat	818
QW Physics	819
Localized QW Model	820
Accelerating Gain Calculations and LED Simulations	821
Discussion of LED Physics	822
LED Raytracing	823
Compact Memory Raytracing	824
Isotropic Starting Rays from Spontaneous Emission Sources.....	824
Anisotropic Starting Rays from Spontaneous Emission Sources	825
Randomizing Starting Rays.....	826
Pseudorandom Starting Rays	827
Reading Starting Rays from File	827
Moving Starting Rays on Boundaries	828
Clustering Active Vertices.....	828
Plane Area Cluster	829
Nodal Clustering.....	830
Optical Grid Element Clustering	830
Using the Clustering Feature	830
Debugging Raytracing.....	831
Print Options in Raytracing	832
Interfacing LED Starting Rays to LightTools®.....	833

Contents

Example: n99_000000_des_lighttools.txt	834
LED Radiation Pattern	835
Two-dimensional LED Radiation Pattern and Output Files	837
Three-dimensional LED Radiation Pattern and Output Files	838
Staggered 3D Grid LED Radiation Pattern	839
Spectrum-dependent LED Radiation Pattern.....	841
Tracing Source of Output Rays	842
Nonactive Region Absorption (Photon Recycling)	843
Interface to LightTools	843
Example: orainterface_rays.txt	845
Example: orainterface_spectrum.txt	845
Device Physics and Tuning Parameters	845
Example of 3D GaN LED Simulation.....	846
References.....	852

Chapter 36 Optical Mode Solver for Lasers	853
Overview.....	853
Finite-Element Formulation	854
Syntax of FE Scalar and FE Vectorial Optical Solvers	855
FE Scalar Solver	856
FE Vectorial Solver	857
FE Vectorial Solver for Waveguides	857
FE Vectorial Solver for VCSEL Cavity	858
Specifying Multiple Entries for Parameters in FEScalar and FEVectorial	859
Multiple Waveguide Modes	860
Multiple Cavity Modes.....	860
Boundary Conditions and Symmetry for Optical Solvers	861
Symmetric FEScalar Waveguide Mode in Cartesian Coordinates	862
Symmetric FEVectorial Waveguide Modes in Cartesian Coordinates	863
Symmetric FEVectorial VCSEL Cavity Modes in Cartesian Coordinates	863
Symmetric FEVectorial VCSEL Cavity Modes in Cylindrical Coordinates.....	864
Perfectly Matched Layers	865
Transfer Matrix Method for VCSELs.....	867
Effective Index Method for VCSELs	869
Formulation of Effective Index Method	870
Transverse Mode Pattern of VCSELs	872
Syntax for Effective Index Method	873
Cylindrical Modes	873
Cartesian Modes.....	874
Saving and Loading Optical Modes	875
Saving Optical Modes on Optical or Electrical Mesh	875

Loading Optical Modes	876
Loading Optical Fields for Edge-emitting Laser	877
Loading Optical Fields for VCSEL	877
Far Field	878
Far-Field Observation Angle	880
Syntax for Far Field	881
Far-Field Output Files	882
Scalar1D Far Field	882
Scalar2D and Vector2D Far Fields	883
Far Field from Loaded Optical Field File	885
VCSEL Near Field and Far Field	885
Optics Stand-alone Option	887
Automatic Optical Mode Searching	889
Syntax for Automatic Mode Searching	890
Searching for Cavity Resonances	891
Searching for Waveguide Modes	892
References	893

Chapter 37 Modeling Quantum Wells	895
Overview	895
Carrier Capture in Quantum Wells	896
Special Meshing Requirements for Quantum Wells	896
Thermionic Emission	897
Radiative Recombination and Gain Coefficients	898
Stimulated and Spontaneous Emission Coefficients	898
Active Bulk Material Gain	900
Stimulated Recombination Rate	900
Spontaneous Recombination Rate	901
Fitting Stimulated and Spontaneous Emission Spectra	901
Gain-broadening Models	902
Lorentzian Broadening	902
Landsberg Broadening	902
Hyperbolic-Cosine Broadening	903
Syntax to Activate Broadening	903
Nonlinear Gain Saturation Effects	903
Simple Quantum-Well Subband Model	905
Syntax for Simple Quantum-Well Model	908
Strain Effects	908
Syntax for Quantum-Well Strain	909
Polarization-dependent Optical Matrix Element	910
Loading Tabulated Stimulated and Spontaneous Emission	913

Contents

Importing Gain and Spontaneous Emission Data with PMI	914
Implementing Gain PMI	915
References	918

Chapter 38 Additional Features of Laser or LED Simulations	919
---	------------

Plotting Gain and Power Spectra	919
Modal Gain as Function of Bias	919
Material Gain in Active Region	920
Modal Gain as Function of Energy/Wavelength	920
Transient Simulation	921
Syntax for Laser Transient Simulation	922
Performing Temperature Simulation	923
Lattice Temperature Simulation	924
Carrier Temperature Simulation	926
Switching from Voltage to Current Ramping	927
Robust Wavelength Search Algorithm	929
Reducing Number of Result Files	929
Scripts	930

Part IV Mesh and Numeric Methods	931
---	------------

Chapter 39 Automatic Grid Generation and Adaptation Module AGM	933
---	------------

Overview	933
General Adaptation Procedure	934
Adaptation Criteria	935
Refinement on Local-Field Variation	935
Refinement on Residual Error Estimation	936
Solution Recomputation	936
Device-Level Data Smoothing	936
System-Level Data Smoothing	937
Specifying Grid Adaptations	937
Adaptive Device Instances	938
Device Structure	938
Device Adaptation Parameters	939
Parameters Affecting Grid Generation	939
Parameters Affecting Smoothing	939
Adaptation Criteria	940
Parameters Common to All Refinement Criteria	940
Element Criterion	940
Residual Criterion	941

Mesh Domains	942
Adaptive Solve Statements	943
General Adaptive Solve Statements	943
Adaptive Coupled Solve Statements	943
Adaptive Quasistationary Solve Statements	943
Performing Adaptive Simulations.	944
Rampable Adaptation Parameters	944
Command File Example	945
Limitations and Recommendations.	946
Limitations	946
Recommendations	946
Initial Grid Construction.	946
Accuracy of Terminal Currents as Adaptation Goal	946
AGM Simulation Times	946
Large Grid Sizes	947
Convergence Problems After Adaptation.	947
AGM and Extrapolation	947
References	948

Chapter 40 Numeric Methods	949
Discretization	949
Box Method Coefficients	950
Basic Definitions	950
Variation of Box Method Algorithms	952
Truncated and Non-Delaunay Elements	953
Math Parameters for Box Method Coefficients	954
Weighted Box Method Coefficients	955
Weighted Points	955
Weighted Voronoï Diagram	956
Saving and Restoring Box Method Coefficients	957
Log File	959
AC Simulation.	960
AC Response	960
AC Current Density Responses	962
Harmonic Balance Analysis	963
Harmonic Balance Equation	963
Multitone Harmonic Balance Analysis	963
Multidimensional Fourier Transformation	964
Quasi-Periodic Functions	965
Multidimensional Frequency Domain Problem	965
One-Tone Harmonic Balance Analysis	965

Contents

Solving HB Equation	966
Solving HB Newton Step Equation	967
Restarted GMRES Method	967
Direct Solver Method	968
Transient Simulation	968
Backward Euler Method	968
TRBDF Composite Method	969
Controlling Transient Simulations	970
Floating Gates	971
Nonlinear Solvers	972
Fully Coupled Solution	972
‘Plugin’ Iterations	975
References	975

Part V Physical Model Interface

977

Chapter 41 Physical Model Interface	979
Overview	979
Standard C++ Interface	981
Simplified C++ Interface	985
Shared Object Code	988
Command File of Sentaurus Device	989
Vertex-based Run-Time Support	991
Vertex-based Run-Time Support for Multistate Configuration-dependent Models .	995
Mesh-based Run-Time Support	996
Device Mesh	998
Vertex	998
Edge	999
Element	999
Region	1001
Region Interface	1002
Mesh	1003
Device Data	1004
Parameter File of Sentaurus Device	1007
Parallelization	1009
Thread-Local Storage	1009
Debugging	1011
Generation-Recombination Model	1012
Dependencies	1012
Standard C++ Interface	1013
Simplified C++ Interface	1014

Example: Auger Recombination	1015
Avalanche Generation Model	1015
Dependencies	1015
Standard C++ Interface	1016
Simplified C++ Interface	1018
Example: Okuto Model	1019
Mobility Models	1022
Doping-dependent Mobility	1023
Dependencies	1023
Standard C++ Interface	1024
Simplified C++ Interface	1025
Example: Masetti Model	1026
Multistate Configuration-dependent Bulk Mobility	1029
Command File	1029
Dependencies	1029
Standard C++ Interface	1030
Simplified C++ Interface	1031
Mobility Degradation at Interfaces	1032
Dependencies	1032
Standard C++ Interface	1033
Simplified C++ Interface	1035
Example: Lombardi Model	1036
High-Field Saturation Model	1041
Dependencies	1041
Standard C++ Interface	1042
Simplified C++ Interface	1044
Example: Canali Model	1045
High-Field Saturation with Two Driving Forces	1050
Command File	1050
Dependencies	1050
Standard C++ Interface	1052
Simplified C++ Interface	1053
Band Gap	1054
Dependencies	1055
Standard C++ Interface	1055
Simplified C++ Interface	1056
Example: Default Bandgap Model	1056
Bandgap Narrowing	1058
Dependencies	1058
Standard C++ Interface	1058
Simplified C++ Interface	1059

Contents

Example: Default Model	1059
Apparent Band-Edge Shift	1061
Dependencies	1061
Standard C++ Interface	1062
Simplified C++ Interface	1063
Multistate Configuration–dependent Apparent Band-Edge Shift	1064
Dependencies	1064
Additional Functionality	1065
Using Dependencies	1065
Updating Actual Status	1065
Standard C++ Interface	1065
Simplified C++ Interface	1067
Electron Affinity	1068
Dependencies	1068
Standard C++ Interface	1068
Simplified C++ Interface	1069
Example: Default Affinity Model	1070
Effective Mass	1071
Dependencies	1071
Standard C++ Interface	1072
Simplified C++ Interface	1072
Example: Linear Effective Mass Model	1073
Energy Relaxation Times	1075
Dependencies	1076
Standard C++ Interface	1076
Simplified C++ Interface	1077
Example: Constant Energy Relaxation Times	1077
Lifetimes	1079
Dependencies	1080
Standard C++ Interface	1080
Simplified C++ Interface	1081
Example: Doping- and Temperature-dependent Lifetimes	1082
Thermal Conductivity	1084
Dependencies	1084
Standard C++ Interface	1085
Simplified C++ Interface	1086
Example: Temperature-dependent Thermal Conductivity	1087
Multistate Configuration–dependent Thermal Conductivity	1088
Command File	1088
Dependencies	1089
Standard C++ Interface	1089

Simplified C++ Interface	1091
Heat Capacity	1092
Dependencies	1092
Standard C++ Interface	1093
Simplified C++ Interface	1093
Example: Constant Heat Capacity	1094
Multistate Configuration–dependent Heat Capacity	1095
Command File	1095
Dependencies	1095
Standard C++ Interface	1096
Simplified C++ Interface	1097
Optical Quantum Yield	1098
Dependencies	1098
Standard C++ Interface	1099
Simplified C++ Interface	1100
Stress	1101
Dependencies	1101
Standard C++ Interface	1102
Simplified C++ Interface	1103
Example: Constant Stress Model	1103
Space Factor	1105
Dependencies	1106
Standard C++ Interface	1106
Simplified C++ Interface	1106
Example: PMI User Field as Space Factor	1107
Trap Capture and Emission Rates	1108
Traps	1108
Multistate Configurations	1108
Dependencies	1109
Standard C++ Interface	1110
Simplified C++ Interface	1111
Example: CEModel_ArrheniusLaw	1112
Trap Energy Shift	1112
Command File	1112
Dependencies	1113
Standard C++ Interface	1113
Simplified C++ Interface	1114
Piezoelectric Polarization	1114
Dependencies	1115
Standard C++ Interface	1115
Simplified C++ Interface	1116

Contents

Example: Gaussian Polarization Model	1116
Incomplete Ionization	1117
Dependencies	1118
Standard C++ Interface	1118
Simplified C++ Interface	1119
Example: Matsuura Incomplete Ionization Model	1120
Hot-Carrier Injection	1125
Dependencies	1125
Standard C++ Interface	1126
Simplified C++ Interface	1126
Example: Lucky Model	1127
Piezoresistive Coefficients	1130
Dependencies	1130
Standard C++ Interface	1131
Simplified C++ Interface	1131
Current Plot File of Sentaurus Device	1132
Structure of Current Plot File	1132
Standard C++ Interface	1133
Simplified C++ Interface	1134
Example: Average Electrostatic Potential	1134
Postprocess for Transient Simulation	1137
Standard C++ Interface	1137
Simplified C++ Interface	1137
Example: Postprocess User-Field	1138
Special Contact PMI for Raytracing	1139
Dependencies	1140
Standard C++ Interface	1142
Example: Assessing and Modifying a Ray	1143
Spatial Distribution Function	1146
Dependencies	1146
Standard C++ Interface	1147
Simplified C++ Interface	1147
Example: Gaussian Spatial Distribution Function	1148
Metal Resistivity	1149
Dependencies	1150
Standard C++ Interface	1150
Simplified C++ Interface	1151
Example: Linear Metal Resistivity	1152
Heat Generation Rate	1153
Dependencies	1154
Standard C++ Interface	1154

Simplified C++ Interface	1156
Example: Dependency on Electric Field and Gradient of Temperature	1157
Thermoelectric Power	1158
Dependencies	1158
Standard C++ Interface	1159
Simplified C++ Interface	1159
Example: Analytic TEP	1160
Metal Thermoelectric Power	1163
Dependencies	1163
Standard C++ Interface	1164
Simplified C++ Interface	1165
Example: Linear Field Dependency of Metal TEP	1165
References	1168

Part VI Appendices 1169

Appendix A Mathematical Symbols 1171

Appendix B Syntax 1175

Appendix C File-naming Convention 1177

Overview	1177
Compatibility with Old File-naming Convention	1178

Appendix D Command-Line Options 1179

Starting Sentaurus Device	1179
Command-Line Options	1179

Appendix E Run-Time Statistics 1183

The sdevicestat Command	1183
-------------------------------	------

Appendix F Data and Plot Names 1185

Overview	1185
Scalar Data	1186
Vector Data	1212
Special Vector Data	1216

Appendix G Command File Overview	1217
Organization of Command File Overview	1217
Top Level of Command File	1219
Device	1219
File	1221
Solve	1224
System	1239
Boundary Conditions	1240
Math	1243
Physics	1259
Generation and Recombination	1266
Laser and LED	1282
Mobility	1293
Radiation Models	1297
Various	1299
Plotting	1320
Various	1324

About This Guide

Sentaurus Device is a fully featured 2D and 3D device simulator that provides you with the ability to simulate a broad range of devices.

Sentaurus Device is a multidimensional, electrothermal, mixed-mode device and circuit simulator for one-dimensional, two-dimensional, and three-dimensional semiconductor devices. It incorporates advanced physical models and robust numeric methods for the simulation of most types of semiconductor device ranging from very deep-submicron silicon MOSFETs to large bipolar power structures. In addition, SiC and III–V compound homostructure and heterostructure devices are fully supported.

The user guide is divided into parts:

- Part I contains information about how to start Sentaurus Device and how it interacts with other Synopsys tools.
- Part II describes the physics in Sentaurus Device.
- Part III describes the physical models used in laser and light-emitting diode simulations.
- Part IV presents the automatic grid generation facility and provides background information on the numeric methods used in Sentaurus Device.
- Part V describes the physical model interface, which provides direct access to certain models in the semiconductor transport equations and the numeric methods in Sentaurus Device.
- Part VI contains the appendices.

Audience

This guide is intended for users of the Sentaurus Device software package. It assumes familiarity with working on UNIX-like systems and requires a basic understanding of semiconductor device physics and its terminology.

Related Publications

For additional information about Sentaurus Device, see:

- The TCAD Sentaurus release notes, available on SolvNet (see [Accessing SolvNet on page xxxix](#)).
- Documentation available through SolvNet at <https://solvnet.synopsys.com/DocsOnWeb>.

Typographic Conventions

Convention	Explanation
< >	Angle brackets
{ }	Braces
[]	Brackets
()	Parentheses
Blue text	Identifies a cross-reference (only on the screen).
Bold text	Identifies a selectable icon, button, menu, or tab. It also indicates the name of a field or an option.
<code>Courier font</code>	Identifies text that is displayed on the screen or that the user must type. It identifies the names of files, directories, paths, parameters, keywords, and variables.
<i>Italicized text</i>	Used for emphasis, the titles of books and journals, and non-English words. It also identifies components of an equation or a formula, a placeholder, or an identifier.
Menu > Command	Indicates a menu command, for example, File > New (from the File menu, select New).
NOTE	Identifies important information.

Customer Support

Customer support is available through SolvNet online customer support and through contacting the Synopsys support center.

Accessing SolvNet

SolvNet includes an electronic knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. SolvNet also gives you access to a wide range of Synopsys online services, which include downloading software, viewing documentation, and entering a call to the Synopsys support center.

To access SolvNet:

1. Go to the SolvNet Web page at <https://solvnet.synopsys.com>.
2. If prompted, enter your user name and password. (If you do not have a Synopsys user name and password, follow the instructions to register with SolvNet.)

If you need help using SolvNet, click Help on the SolvNet menu bar.

Contacting Synopsys Support

If you have problems, questions, or suggestions, you can contact Synopsys support in the following ways:

- Go to the Synopsys [Global Support Centers](#) site on www.synopsys.com. There you can find e-mail addresses and telephone numbers for Synopsys support centers throughout the world.
- Go to either the Synopsys SolvNet site or the Synopsys Global Support Centers site and [open a case online](#) (Synopsys user name and password required).

Contacting Your Local TCAD Support Team Directly

Send an e-mail message to:

- support-tcad-us@synopsys.com from within North America and South America.
- support-tcad-eu@synopsys.com from within Europe.
- support-tcad-ap@synopsys.com from within Asia Pacific (China, Taiwan, Singapore, Malaysia, India, Australia).
- support-tcad-kr@synopsys.com from Korea.
- support-tcad-jp@synopsys.com from Japan.

Acknowledgments

Parts of Sentaurs Device were codeveloped by Integrated Systems Laboratory of ETH Zurich in the joint research project LASER with financial support by the Swiss funding agency CTI and in the joint research project VCSEL with financial support by the Swiss funding agency TOP NANO 21. The GEBAS Library was codeveloped by Integrated Systems Laboratory of ETH Zurich in the joint research project MQW with financial support by the Swiss funding agency TOP NANO 21.

The third-party software ARPACK (ARnoldi PACKage) by R. Lehoucq, K. Maschhoff, D. Sorensen, and C. Yang is used in Sentaurs Device (<http://www.caam.rice.edu/software/ARPACK>).

Sentaurs Device contains the QD library for double-double and quad-double floating-point arithmetic (<http://crd.lbl.gov/~dhbailey/mpdist>). The QD library requires the following copyright notice:

This work was supported by the Director, Office of Science, Division of Mathematical, Information, and Computational Sciences of the U.S. Department of Energy under contract number DE-AC03-76SF00098.

Copyright © 2003, The Regents of the University of California, through Lawrence Berkeley National Laboratory (subject to receipt of any required approvals from U.S. Dept. of Energy)

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- (1) Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

(2) Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

(3) Neither the name of Lawrence Berkeley National Laboratory, U.S. Dept. of Energy nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Sentaurus Device contains the ARPREC library for arbitrary floating-point arithmetic (<http://crd.lbl.gov/~dhbailey/mpdist>). The ARPREC library requires the following copyright notice:

This work was supported by the Director, Office of Science, Division of Mathematical, Information, and Computational Sciences of the U.S. Department of Energy under contract numbers DE-AC03-76SF00098 and DE-AC02-05CH11231.

Copyright © 2003-2009, The Regents of the University of California, through Lawrence Berkeley National Laboratory (subject to receipt of any required approvals from U.S. Dept. of Energy)

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

(1) Redistributions of source code must retain the copyright notice, this list of conditions and the following disclaimer.

(2) Redistributions in binary form must reproduce the copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

(3) Neither the name of the University of California, Lawrence Berkeley National Laboratory, U.S. Dept. of Energy nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

About This Guide

Acknowledgments

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

You are under no obligation whatsoever to provide any bug fixes, patches, or upgrades to the features, functionality or performance of the source code ("Enhancements") to anyone; however, if you choose to make your Enhancements available either publicly, or directly to Lawrence Berkeley National Laboratory, without imposing a separate written license agreement for such Enhancements, then you hereby grant the following license: a non-exclusive, royalty-free perpetual license to install, use, modify, prepare derivative works, incorporate into other computer software, distribute, and sublicense such enhancements or derivative works thereof, in binary and source code form.

Part I Getting Started

This part of the *Sentaurus Device User Guide* contains the following chapters:

[Chapter 1 Overview on page 3](#)

[Chapter 2 Defining Devices on page 47](#)

[Chapter 3 Mixed-Mode Sentaurus Device on page 79](#)

[Chapter 4 Performing Numeric Experiments on page 99](#)

[Chapter 5 Simulation Results on page 137](#)

[Chapter 6 Numeric and Software-related Issues on page 155](#)

This chapter describes how Sentaurus Device integrates into the TCAD tool suite and presents some complete simulation examples.

About Sentaurus Device

Sentaurus Device simulates numerically the electrical behavior of a single semiconductor device in isolation or several physical devices combined in a circuit. Terminal currents, voltages, and charges are computed based on a set of physical device equations that describes the carrier distribution and conduction mechanisms. A real semiconductor device, such as a transistor, is represented in the simulator as a ‘virtual’ device whose physical properties are discretized onto a nonuniform ‘grid’ (or ‘mesh’) of nodes.

Therefore, a virtual device is an approximation of a real device. Continuous properties such as doping profiles are represented on a sparse mesh and, therefore, are only defined at a finite number of discrete points in space. The doping at any point between nodes (or any physical quantity calculated by Sentaurus Device) can be obtained by interpolation. Each virtual device structure is described in the Synopsys TCAD tool suite by a TDR file containing the following information:

- The grid (or geometry) of the device contains a description of the various regions, that is, boundaries, material types, and the locations of any electrical contacts. It also contains the locations of all the discrete nodes and their connectivity.
- The data fields contain the properties of the device, such as the doping profiles, in the form of data associated with the discrete nodes. [Figure 1 on page 4](#) shows a typical example: the doping profile of a MOSFET structure discretized by a mixed-element grid. By default, a device simulated in 2D is assumed to have a ‘thickness’ in the third dimension of 1 μm .

1: Overview

About Sentaurus Device

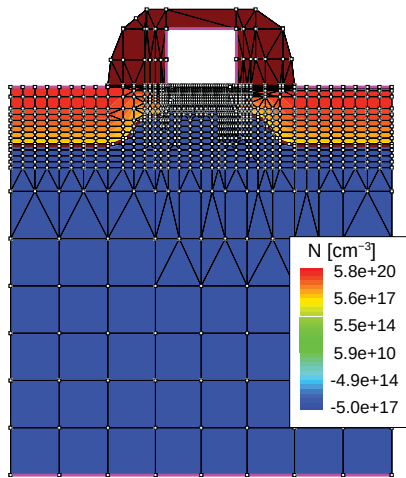


Figure 1 Two-dimensional doping profile that is discretized on the nodes of simulation grid

The features of Sentaurus Device are many and varied. They can be summarized as:

- An extensive set of models for device physics and effects in semiconductor devices (drift-diffusion, thermodynamic, and hydrodynamic models).
- General support for different device geometries (1D, 2D, 3D, and 2D cylindrical).
- Mixed-mode support of electrothermal netlists with mesh-based device models and SPICE circuit models.

Nonvolatile memory simulations are accommodated by robust treatment of floating electrodes in combination with Fowler–Nordheim and direct tunneling, and hot-carrier injection mechanisms.

Hydrodynamic (energy balance) transport is simulated rigorously to provide a more physically accurate alternative to conventional drift-diffusion formulations of carrier conduction in advanced devices.

Floating semiconductor regions in devices such as thyristors and silicon-on-insulator (SOI) transistors (floating body) are handled robustly. This allows hydrodynamic breakdown simulations in such devices to be achieved with good convergence.

The mixed device and circuit capabilities give Sentaurus Device the ability to solve three basic types of simulation: single device, single device with a circuit netlist, and multiple devices with a circuit netlist (see [Figure 2 on page 5](#)).

Multiple-device simulations can combine devices of different mesh dimensionality, and different physical models can be applied in individual devices, providing greater flexibility. In all cases, the circuit netlists can contain an electrical and a thermal section.

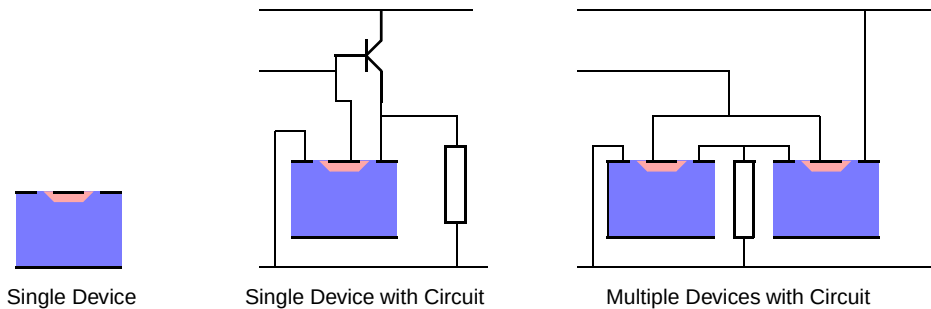


Figure 2 Three types of simulation

Creating and Meshing Device Structures

Device structures can be created in various ways, including 1D, 2D, or 3D process simulation (Sentaurus Process), 2D or 3D process emulation (Sentaurus Structure Editor), and 2D or 3D structure editors (Sentaurus Structure Editor).

Regardless of the means used to generate a virtual device structure, it is recommended that the structure be remeshed using Sentaurus Structure Editor (2D and 3D meshing with an interactive graphical user interface (GUI)) or Sentaurus Mesh (1D, 2D, and 3D meshing without a GUI) to optimize the grid for efficiency and robustness.

For maximum efficiency of a simulation, a mesh must be created with a minimum number of vertices to achieve the required level of accuracy. For any given device structure, the optimal mesh varies depending on the type of simulation.

It is recommended that to create the most suitable mesh, the mesh must be densest in those regions of the device where the following are expected:

- High current density (MOSFET channels, bipolar base regions)
- High electric fields (MOSFET channels, MOSFET drains, depletion regions in general)
- High charge generation (single event upset (SEU) alpha particle, optical beam)

For example, accurate drain current modeling in a MOSFET requires very fine, vertical, mesh spacing in the channel at the oxide interface (of the order $1\text{ \AA}$) when using advanced mobility models. For reliable simulation of breakdown at a drain junction, the mesh must be more concentrated inside the junction depletion region for good resolution of avalanche multiplication.

Generally, a total node count of 2000 to 4000 is reasonable for most 2D simulations. Large power devices and 3D structures require a considerably larger number of elements.

Tool Flow

In a typical device tool flow, the creation of a device structure by process simulation (Sentaurus Process) is followed by remeshing using Sentaurus Structure Editor or Sentaurus Mesh. In this scheme, control of mesh refinement is handled automatically through the file `_dvs.cmd`.

Sentaurus Device is used to simulate the electrical characteristics of the device. Finally, Tecplot SV is used to visualize the output from the simulation in 2D and 3D, and Inspect is used to plot the electrical characteristics.

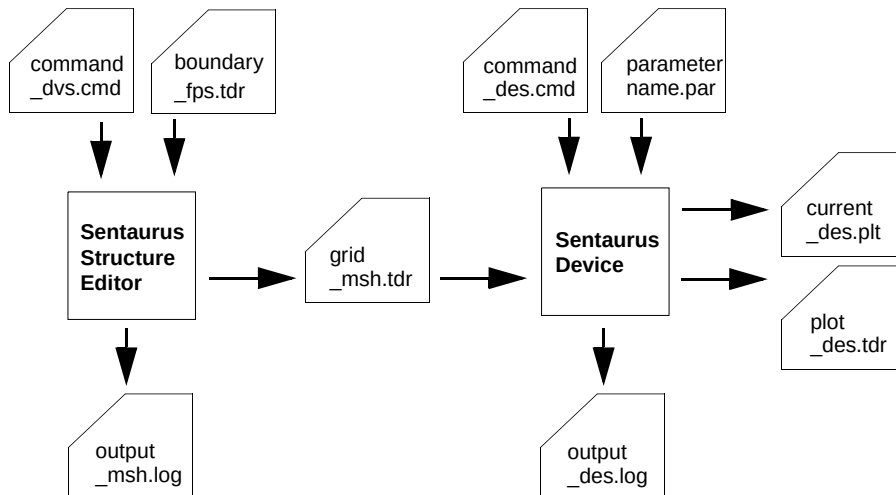


Figure 3 Typical tool flow with device simulation using Sentaurus Device

Starting Sentaurus Device

There are two ways to start Sentaurus Device: from the command line or Sentaurus Workbench.

From Command Line

Sentaurus Device is driven by a command file and run by the command:

```
sdevice <command_filename>
```

Various options exist at start-up and are listed by using:

```
sdevice -h
```

By default, `sdevice` runs the latest version in the current release of Sentaurus Device. To run a particular version in the current release, use the command-line option `-ver`. For example:

```
sdevice -ver 1.4 nmos_des.cmd
```

starts version 1.4 of the latest release of Sentaurus Device. To run the latest version in a particular release, use the command-line option `-rel`. For example:

```
sdevice -rel G-2012.06 nmos_des.cmd
```

starts the latest version available in release G-2012.06. To run a particular version in a particular release, combine `-ver` and `-rel`. For example:

```
sdevice -rel F-2011.09 -ver 1.4 nmos_des.cmd
```

starts Sentaurus Device, release F-2011.09, version 1.4.

When the release or the version requested is not installed, the available releases or versions are listed, and the program aborts.

[Appendix D on page 1179](#) lists the command options of Sentaurus Device, which include:

`sdevice -versions`

Checks which versions are in the installation path.

`sdevice -P` and `sdevice -L`

Extract model parameter files (see [Generating a Copy of Parameter File on page 70](#)).

`sdevice --parameter-names`

Prints names of parameters that can be ramped (see [Ramping Physical Parameter Values on page 109](#)).

When Sentaurus Device starts, the command file is checked for correct syntax, and the commands are executed in sequence. Character strings starting with `*` or `#` are ignored by Sentaurus Device, so that these characters can be used to insert comments in the simulation command file.

From Sentaurus Workbench

Sentaurus Device is launched automatically through the Scheduler when working inside Sentaurus Workbench.

Sentaurus Workbench interprets `#` as a special marker for conditional statements (for example, `#if...`, `#elif...`, and `#endif...`).

To access the tutorial examples for Sentaurus Device, open Sentaurus Workbench and either select **Help** > **Training** or click the corresponding toolbar button. The TCAD Sentaurus Tutorial opens in a separate window.

Simulation Examples

In the following sections, many of the widely used Sentaurus Device commands are introduced in the context of a series of typical MOSFET device simulations. The examples are available as Sentaurus Workbench projects, in the directory `$STR00T/tcad/$STRELEASE/lib/sdevice/GettingStarted`.

First, a very simple example of an I_d - V_g simulation is presented using default models and methods (`simple_Id-Vg`). Second, a more advanced approach to an I_d - V_d simulation (`advanced_Id-Vd`) is presented, in which more complex models are introduced and some important options are indicated.

Subsequently, a mixed-mode simulation of a CMOS inverter is presented (`advanced_Inverter`), and the small-signal AC response of a MOSFET is obtained (`advanced_AC`). The intention is to introduce some of the most widely used Sentaurus Device features in a realistic context.

Example: Simple MOSFET Id-Vg Simulation

Simulation of the drain current–gate voltage characteristic of a MOSFET is a typical Sentaurus Device application. It allows important device properties, such as threshold voltage, off-current, subthreshold slope, on-state drive current, and transconductance to be extracted. In this example, only the most essential commands are used for a reasonable simulation.

Command File

The Sentaurus Device command file is organized in command or statement sections that can be in any order (except in mixed-mode simulations). Sentaurus Device keywords are not case sensitive and most can be abbreviated. However, Sentaurus Device is syntax sensitive, for example, parentheses must be consistent and character strings for variable names must be delimited by quotation marks (" ").

An example of a complete command file (sdevice_des.cmd in the Sentaurus Workbench project simple_Id-Vg) is presented. Each statement section is explained individually.

```
File {
  * input files:
  Grid      = "nmos_msh.tdr"
  * output files:
  Plot      = "n1_des.tdr"
  Current   = "n1_des.plt"
  Output    = "n1_des.log"
}

Electrode {
  { Name="source" Voltage=0.0 }
  { Name="drain"   Voltage=0.1 }
  { Name="gate"    Voltage=0.0 Barrier=-0.55 }
  { Name="substrate" Voltage=0.0 }
}

Physics {
  Mobility (DopingDependence HighFieldSat Enormal)
  EffectiveIntrinsicDensity (BandGapNarrowing (OldSlotboom))
}

Plot {
  eDensity hDensity eCurrent hCurrent
  Potential SpaceCharge ElectricField
  eMobility hMobility eVelocity hVelocity
  Doping DonorConcentration AcceptorConcentration
}

Math {
  Extrapolate
  RelErrControl
}

Solve {
  #-initial solution:
  Poisson
}
```

1: Overview

Example: Simple MOSFET Id-Vg Simulation

```
Coupled { Poisson Electron }
#-ramp gate:
Quasistationary ( MaxStep=0.05
  Goal{ Name="gate" Voltage=2 } )
  { Coupled { Poisson Electron } }
}

$STR00T/tcad/$STRELEASE/lib/sdevice/GettingStarted/simple_Id-Vd/
sdevice_des.cmd
```

File Section

First, the input files that define the device structure and the output files for the simulation results must be specified. The device to be simulated is the one plotted in [Figure 1 on page 4](#). The device is defined by the file `nmos_msh.tdr`:

```
File {
  * input files:
  Grid    = "nmos_msh.tdr"
  * output files:
  Plot    = "n1_des.tdr"
  Current = "n1_des.plt"
  Output  = "n1_des.log"
}
```

The `File` section specifies the input and output files necessary to perform the simulation.

`* input files:`

This is a comment line.

`Grid = "nmos_msh.tdr"`

This essential input file (default extension `.tdr`) defines the mesh and various regions of the device structure, including contacts. Sentaurus Device automatically determines the dimensionality of the problem from this file. It also contains the doping profiles data for the device structure.

`* output files:`

This is a comment line.

`Plot = "n1_des.tdr"`

This is the file name for the final spatial solution variables on the structure mesh (extension `_des.tdr`).

`Current = "n1_des.plt"`

This is the file name for electrical output data (such as currents, voltages, charges at electrodes). Its standard extension is `_des.plt`.

`Output = "n1_des.log"`

This is an alternate file name for the output log or protocol file (default name `output_des.log`) that is automatically created whenever Sentaurus Device is run. This file contains the redirected standard output, which is generated by Sentaurus Device as it runs.

NOTE Only the root file names are necessary. Sentaurus Device automatically appends the appropriate file name extensions, for example, `Plot="n1"` is sufficient.

The device in this example is two-dimensional. By default, Sentaurus Device assumes a ‘thickness’ (effective gate width along the z-axis) of 1 μm . This effective width is adjusted by specifying an `AreaFactor` in the `Physics` section, or an `AreaFactor` for each electrode individually. An `AreaFactor` is a multiplier for the electrode currents and charges.

Electrode Section

Having loaded the device structure into Sentaurus Device, it is necessary to specify which of the contacts are to be treated as electrodes. Electrodes in Sentaurus Device are defined by electrical boundary conditions and contain no mesh. For example, in [Figure 7 on page 17](#), the ‘polysilicon’ gate is empty; it is not a region.

The `Electrode` section defines all the electrodes to be used in the Sentaurus Device simulation, with their respective boundary conditions and initial biases. Any contacts that are not defined as electrodes are ignored by Sentaurus Device. The polysilicon gate of a MOS transistor can be treated in two ways:

- As a metal, in which case, it is simply an electrode.
- As a region of doped polysilicon, in which case, the gate electrode must be a contact on top of the polysilicon region.

In the former case, an important property of the gate electrode is the ‘metal’–semiconductor work function difference. In Sentaurus Device, this is defined by the parameter `barrier`, which equals the difference in energy [eV] between the polysilicon extrinsic Fermi level and the intrinsic Fermi level in the silicon. The value of `barrier` must, therefore, be specified to be consistent with the doping in the polysilicon. This is the gate definition used in this example and is valid for most applications. However, it totally neglects any polysilicon depletion effects.

1: Overview

Example: Simple MOSFET Id–Vg Simulation

In the latter case, where the gate is modeled as an appropriately doped polysilicon region, the contact must be on top of the polysilicon and Ohmic (the default condition). In this case, depletion of the polysilicon is modeled correctly.

```
Electrode{
  { Name="source" Voltage=0.0 }
  { Name="drain" Voltage=0.1 }
  { Name="gate" Voltage=0.0 Barrier=-0.55 }
  { Name="substrate" Voltage=0.0 }
}
```

Name="string"

Each electrode is specified by a case-sensitive name that must match exactly an existing contact name in the structure file. Only those contacts that are named in the `Electrode` section are included in the simulation.

Voltage=0.0

This defines a voltage boundary condition with an initial value. One or more boundary conditions must be defined for each electrode, and any value given to a boundary condition applies in the initial solution. In this example, the simulation commences with a 100 mV bias on the drain.

Barrier=-0.55

This is the metal–semiconductor work function difference or `barrier` value for a polysilicon electrode that is treated as a metal. This is defined, in general, as the difference between the metal Fermi level in the electrode and the intrinsic Fermi level in the semiconductor. This `barrier` value is consistent with n^+ -polysilicon doping.

Physics Section

The `Physics` section allows a selection of the physical models to be applied in the device simulation. In this example, it is sufficient to include basic mobility models and a definition of the band gap (and, therefore, the intrinsic carrier concentration).

Potentially important effects, such as impact ionization (avalanche breakdown at the drain), are ignored at this stage.

```
Physics {
  Mobility (DopingDependence HighFieldSat Enormal)
  EffectiveIntrinsicDensity (BandGapNarrowing (OldSlotboom))
}
```

Mobility (DopingDependence HighFieldSat Enormal)

Mobility models including doping dependence, high-field saturation (velocity saturation), and transverse field dependence are specified for this simulation.

NOTE HighFieldSaturation can be specified for a specific carrier (for example, eHighFieldSaturation for electrons) and is a function of the effective field experienced by the carrier in its direction of motion. Sentaurus Device provides a choice of effective field computation: GradQuasiFermi (the default), Eparallel, or CarrierTempDrive (in hydrodynamic simulations only).

EffectiveIntrinsicDensity (BandGapNarrowing (OldSlotboom))

This is the silicon bandgap narrowing model that determines the intrinsic carrier concentration.

Plot Section

The Plot section specifies all of the solution variables that are saved in the output plot files (.tdr). Only data that Sentaurus Device is able to compute, based on the selected physics models, is saved to a plot file.

```
Plot {
    eDensity hDensity eCurrent hCurrent
    Potential SpaceCharge ElectricField
    eMobility hMobility eVelocity hVelocity
    Doping DonorConcentration AcceptorConcentration
}
```

An extensive list of optional plot variables is in [Appendix F on page 1185](#).

To save a variable as a vector, append /Vector to the keyword:

```
Plot { eCurrent/Vector ElectricField/v }
```

Math Section

Sentaurus Device solves the device equations (which are essentially a set of partial differential equations) self-consistently, on the discrete mesh, in an iterative fashion. For each iteration, an error is calculated and Sentaurus Device attempts to converge on a solution that has an acceptably small error. For this example, it is only necessary to define a few settings for the numeric solver. Other options, including selection of solver type and user definition of error criteria, are outlined in [Chapter 6 on page 155](#).

1: Overview

Example: Simple MOSFET Id–Vg Simulation

```
Math {  
    Extrapolate  
    RelErrControl  
}
```

Extrapolate

In quasistationary bias ramps, the initial guess for a given step is obtained by extrapolation from the solutions of the previous two steps (if they exist).

RelErrControl

Switches error control during iterations from using internal error parameters to more physically meaningful parameters (ErrRef) (see [Coupled Error Control on page 159](#)).

Solve Section

The `Solve` section defines a sequence of solutions to be obtained by the solver. The drain has a fixed initial bias of 100 mV, and the source and substrate are at 0 V. To simulate the $I_d V_g$ characteristic, it is necessary to ramp the gate bias from 0 V to 2 V, and obtain solutions at a number of points in-between. By default, the size of the step between solution points is determined by Sentaurus Device internally, see [Quasistationary Ramps on page 106](#).

As the simulation proceeds, output data for each of the electrodes (currents, voltages, and charges) is saved to the current file `n1_des.plt` after each step and, therefore, the electrical characteristic is obtained. This can be plotted using Inspect, as shown in [Figure 4 on page 16](#) and [Figure 5 on page 16](#). The final 2D solution is saved in the plot file `n1_des.tdr`, which is plotted in [Figure 6 on page 17](#) and [Figure 7 on page 17](#).

```
Solve {  
    Poisson  
    Coupled {Poisson Electron}  
  
    Quasistationary (Goal { Name="gate" Voltage=2 })  
    { Coupled {Poisson Electron} }  
}
```

Poisson

This specifies that the initial solution is of the nonlinear Poisson equation only. Electrodes have initial electrical bias conditions as defined in the `Electrode` section. In this example, a 100 mV bias is applied to the drain.

Coupled {Poisson Electron}

The second step introduces the continuity equation for electrons, with the initial bias conditions applied. In this case, the electron current continuity equation is solved fully coupled to the Poisson equation, taking the solution from the previous step as the initial guess. The fully coupled or ‘Newton’ method is fast and converges in most cases. It is rarely necessary to use a ‘Plugin’ (or the so-called Gummel) approach.

```
Quasistationary (Goal { Name="gate" Voltage=2 })
{ Coupled { Poisson Electron } }
}
```

The Quasistationary statement specifies that quasistatic or steady state ‘equilibrium’ solutions are to be obtained. A set of **GOALS** for one or more electrodes is defined in parentheses. In this case, a sequence of solutions is obtained for increasing gate bias up to and including the goal of 2 V. A fully coupled (Newton) method for the self-consistent solution of the Poisson and electron continuity equations is specified in braces. Each bias step is solved by taking the solution from the previous step as its initial guess. If **Extrapolate** is specified in the **Math** section, the initial guess for each bias step is calculated by extrapolation from the previous two solutions.

Simulated Id–Vg Characteristic

In **Inspect**, the gate **OuterVoltage** is plotted to the x-axis, and the drain **eCurrent** is plotted to the y-axis. The gate **InnerVoltage** is an internally calculated voltage equal to the **OuterVoltage** and adjusted to account for the barrier, and it is not plotted.

NOTE Although the solution points obtained are equally spaced (0.1 V), this is not predetermined. Generally, Sentaurus Device selects step sizes according to an internal algorithm for maximum numeric robustness. If a step fails to converge, the step size is reduced until convergence is achieved. In this example, the simulation proceeded with the maximum step size from start to finish.

1: Overview

Example: Simple MOSFET Id-Vg Simulation

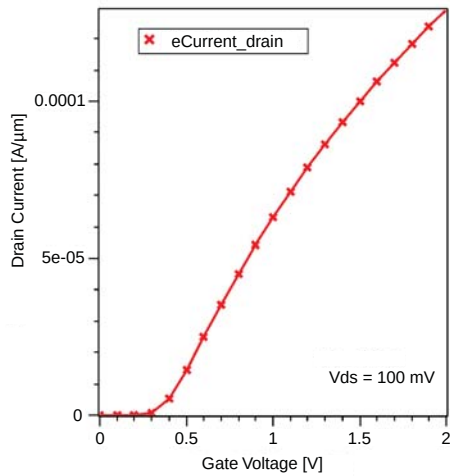


Figure 4 I_d - V_{gs} characteristic of 0.18 μm n-channel MOSFET

In Figure 5, a $\log(I_d)$ - $\ln(V_{gs})$ plot shows the subthreshold characteristic.

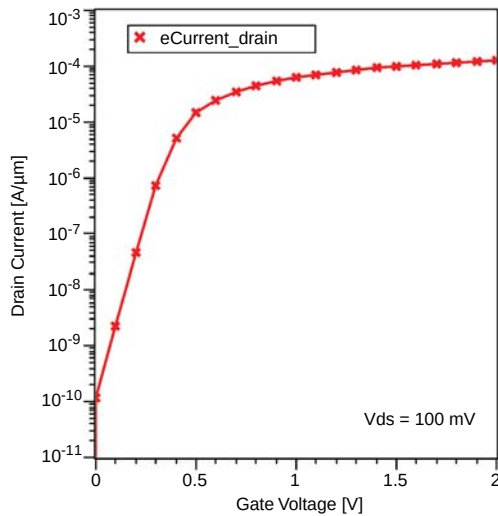


Figure 5 I_d - V_{gs} characteristic of 0.18 μm n-channel MOSFET replotted on semi-log scale

NOTE Successful completion of the simulation is confirmed by a message in the output file:
Finished, because of...
Curve trace finished.
Writing plot 'n1_des.tdr' (TDR format) ... done.

The plot data in `n1_des.tdr` can be analyzed using a graphics package such as Tecplot SV.

Analysis of 2D Output Data

The contents of the file `n1_des.tdr` is loaded into Tecplot SV. The command is:

```
> tecplot_sv n1_des.tdr &
```

Figure 6 shows the default display, which is electrostatic potential.

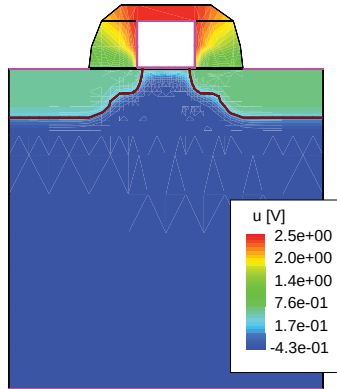


Figure 6 Default display of electrostatic potential and junctions

The electrostatic potential is relative to the intrinsic Fermi level in the silicon.

NOTE The potential at the gate electrode is 2.55 V (InnerVoltage), which equals the applied bias minus the barrier potential. In the substrate, which is tied to 0 V, the ‘inner’ electrostatic potential is -0.4253 V, which corresponds to the position of the hole quasi-Fermi level relative to the intrinsic level.

Figure 7 is a magnification of the channel region, created by selecting the dataset `eDensity`. The inversion layer is clearly visible.

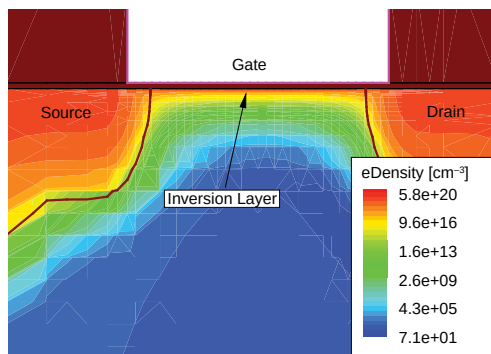


Figure 7 Magnification of channel region showing contours of electron concentration

1: Overview

Example: Advanced Hydrodynamic Id-Vd Simulation

Figure 8 shows the electron and hole densities along a vertical outline in the center of the channel.

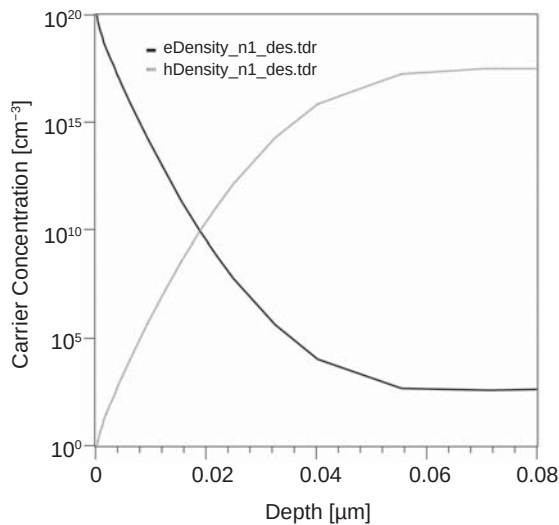


Figure 8 Vertical electron and hole profiles at center of channel

Example: Advanced Hydrodynamic Id-Vd Simulation

Command File

```
#-----#
#- Sentaurus Device command file for
#-
#- Id=f(Vd) for Vd=0-10V while Vg=[0.0V, 1.0V, 2.0V] and Vs=0V
#-
#- USING HYDRODYNAMIC MODEL & IMPACT IONIZATION - FAMILY OF CURVES
#-----#
File {
  Grid      = "@tdr@"
  Parameter = "mos"
  Plot      = "@tdrdat@"
  Current   = "@plot@"
  Output    = "@log@"
}

Electrode {
  { Name="source" Voltage=0.0 }
  { Name="drain"  Voltage=0.0 }
  { Name="gate"   Voltage=0.0 Barrier=-0.55 }
```



```

    { Name="substrate" Voltage=0.0 }
}

Physics {
    AreaFactor=0.4
    Hydrodynamic( eTemperature )
    Mobility ( DopingDependence Enormal
               eHighFieldsat(CarrierTempDrive)
               hHighFieldsat(GradQuasiFermi) )
    Recombination( SRH(DopingDependence)
                  eAvalanche(CarrierTempDrive)
                  hAvalanche(Eparallel) )
    EffectiveIntrinsicDensity (BandGapNarrowing (OldSlotboom))
}

Physics(
    MaterialInterface="Silicon/Oxide") {
    Traps((FixedCharge Conc=4.5e+10))
}

Plot {
    eDensity hDensity eCurrent hCurrent
    equasiFermi hquasiFermi
    eTemperature
    ElectricField eEparallel hEparallel
    Potential SpaceCharge
    SRHRecombination Auger AvalancheGeneration
    eMobility hMobility eVelocity hVelocity
    Doping DonorConcentration AcceptorConcentration
}

CurrentPlot {
    Potential (82, 530, 1009)
    eTemperature (82, 530, 1009)
}

Math {
    Extrapolate
    RelErrControl
    Iterations=20
    NotDamped=50
    BreakCriteria {Current(Contact="drain" Absval=3e-4)
    }
}

Solve {
    # initial gate voltage Vgs=0.0V
    Poisson

```

1: Overview

Example: Advanced Hydrodynamic Id-Vd Simulation

```
Coupled { Poisson Electron }
Coupled { Poisson Electron Hole eTemperature }
Save (FilePrefix="vg0")

# ramp gate and save solutions:

# second gate voltage Vgs=1.0V
Quasistationary
  (InitialStep=0.1 Maxstep=0.1 MinStep=0.01
   Goal { name="gate" voltage=1.0 } )
  { Coupled { Poisson Electron Hole eTemperature } }
  Save(FilePrefix="vg1")

# third gate voltage Vgs=2.0V
Quasistationary
  (InitialStep=0.1 Maxstep=0.1 MinStep=0.01
   Goal { name="gate" voltage=2.0 } )
  { Coupled { Poisson Electron Hole eTemperature } }
  Save(FilePrefix="vg2")

# Load saved structures and ramp drain to create family of curves:

# first curve
Load(FilePrefix="vg0")
NewCurrentPrefix="vg0_"
Quasistationary
  (InitialStep=0.01 Maxstep=0.1 MinStep=0.0001
   Goal{ name="drain" voltage=10.0 }
  )
  { Coupled {Poisson Electron Hole eTemperature}
  CurrentPlot (time=
    (range = (0 0.2) intervals=20;
    range = (0.2 1.0)))}

# second curve
Load(FilePrefix="vg1")
NewCurrentPrefix="vg1_"
Quasistationary
  (InitialStep=0.01 Maxstep=0.1 MinStep=0.0001
   Goal{ name="drain" voltage=10.0 }
  )
  {Coupled {Poisson Electron Hole eTemperature}
  CurrentPlot (time=
    (range = (0 0.2) intervals=20;
    range = (0.2 1.0)))}
```

```
# third curve
Load(FilePrefix="vg2")
NewCurrentPrefix="vg2_"
Quasistationary
  (InitialStep=0.01 Maxstep=0.1 MinStep=0.0001
   Goal{ name="drain" voltage=10.0 }
  )
  { Coupled {Poisson Electron Hole eTemperature }
    CurrentPlot (time=
      (range = (0 0.2) intervals=20;
       range = (0.2 1.0)))
  }
}
$STROOT/tcad/$STRELEASE/lib/sdevice/GettingStarted/advanced_Id-Vd/
sdevice_des.cmd
```

File Section

```
File {
  Grid      = "@tdr@"
  Parameter = "mos"
  Plot      = "@tdrdat@"
  Current   = "@plot@"
  Output    = "@log@"
}
```

The File section specifies the input and output files necessary to perform the simulation. In the example, all file names except mos are file references, which Sentaurus Workbench recognizes and replaces with real file names during preprocessing. For example, in the processed command file pp2_des.cmd:

```
File {
  Grid      = "n1_msh.tdr"
  Parameter = "mos"
  Plot      = "n2_des.tdr"
  Current   = "n2_des.plt"
  Output    = "n2_des.log"
}
```

Grid

The given file names relate to the appropriate node numbers in the Sentaurus Workbench Family Tree. In most projects, Grid is generated by Sentaurus Mesh and, therefore, has the characteristic name nX_msh.tdr.

1: Overview

Example: Advanced Hydrodynamic Id–Vd Simulation

Plot, Current, Output

See [File Section on page 10](#).

Parameter = "mos"

The optional input file `mos.par` contains user-defined values for model parameters (coefficients) (see [Parameter File](#)). Sentaurus Device adds the file extension `.par` automatically.

Main Options

`(e|h)Lifetime = "<name>"`

Loads a lifetime profile contained in a data file. The profile is generated using Sentaurus Structure Editor.

Parameter File

The parameter file contains user-defined values for model parameters (coefficients). The parameters in this file replace the values contained in a default parameter file `models.par`. All other parameter values remain equal to the defaults in `models.par`. Model coefficients can be specified separately for each region or material in the device structure (see [Physical Model Parameters on page 60](#)). Although this feature is intended for the simulation of heterostructure devices, it is useful in silicon devices as it allows materials, such as polysilicon, to have different mobilities.

The default parameter file contains the default parameters for all the physical models available in Sentaurus Device. A copy of the parameter file for silicon is extracted using the command:

```
sdevice -P
```

A list of the parameter files is printed to the command window and into a file `models.par`, which is created in the working directory. For other materials, such as gallium arsenide (GaAs) and silicon carbide (SiC), use:

```
sdevice -P:GaAs
sdevice -P:SiC
```

More options are described in [Generating a Copy of Parameter File on page 70](#).

Listing of mos.par

```
Scharfetter * SRH recombination lifetimes
{ * tau=taumin+(taumax-taumin) / ( 1+(N/Nref )^gamma)
  *
    electrons      holes
    taumin = 0.0000e+00, 0.0000e+00 # [s]
    taumax = 1.0000e-07, 1.0000e-07 # [s]
}
```

Report in Protocol File n3_des.log

```
Reading parameter file 'mos.par' ...
Differences compared with default parameters:
Scharfetter(elec): tau_max = 1.0000e-07, instead of: 1.0000e-05 [s]
Scharfetter(hole): tau_max = 1.0000e-07, instead of: 3.0000e-06 [s]
```

Electrode Section

```
Electrode{
  { Name="source" Voltage=0.0 }
  { Name="drain" Voltage=0.0 }
  { Name="gate" Voltage=0.0 Barrier=-0.55 }
  { Name="substrate" Voltage=0.0 }
}
```

In this example, all electrodes have voltage boundary conditions with the initial condition of zero bias.

Main Options

Current=

Defines a current boundary condition with initial value [A].

Charge=

Defines a floating electrode with a charge boundary condition and an initial charge value [C].

Resist=

Defines a series resistance [Ω] (AreaFactor-dependent).

1: Overview

Example: Advanced Hydrodynamic Id–Vd Simulation

eRecVelocity=

Defines a recombination velocity [cm/s] at a contact for electrons (**hRecVelocity** for holes).

Schottky=

Defines an electrode as a Schottky contact. The attributes of such a contact are specified as **Barrier** and **eRecVelocity** or **hRecVelocity**.

AreaFactor=

Specifies a multiplication factor for the current in or out of an electrode. It is preferable to define **AreaFactor** in the **Physics** section. In a 2D simulation, this can represent the size of the device in the third dimension (for example, gate width), which is 1 μm by default.

Barrier=-0.55

This is the metal–semiconductor work function difference or barrier value for an electrode that is treated as a metal. Defined, in general, as the difference between the metal Fermi level in the electrode and the intrinsic Fermi level in the semiconductor.

In this example, this corresponds to the difference between the quasi-Fermi level in the gate polysilicon and the intrinsic Fermi level in the silicon. The value of **barrier=-0.55** is approximately representative of n^+ -doped polysilicon.

Physics Section

```
Physics {  
  AreaFactor=0.4  
  Hydrodynamic(eTemperature)  
  Mobility(DopingDependence Enormal  
    hHighFieldSaturation(GradQuasiFermi  
      eHighFieldSaturation(CarrierTempDrive))  
  Recombination(SRH(DopingDependence)  
    eAvalanche(CarrierTempDrive)  
    hAvalanche(Eparallel))  
  EffectiveIntrinsicDensity(BandGapNarrowing (OldSlotboom))  
}
```

AreaFactor=0.4

Specifies that the electrode currents and charges are multiplied by a factor of 0.4 (equivalent to a simulated gate width of 0.4 μm).

Hydrodynamic(eTemperature)

Selects hydrodynamic transport models for electrons only. Hole transport is modeled using drift-diffusion.

Mobility(DopingDependence Enormal

See [Physics Section on page 12](#).

hHighFieldSaturation(GradQuasiFermi)

Velocity saturation for holes. It uses the default model after Canali (based on Caughey–Thomas) (see [High-Field Saturation on page 342](#)), and is driven by a field computed as the gradient of the hole quasi-Fermi level, which is the default.

eHighFieldSaturation(CarrierTempDrive)

Velocity saturation for electrons. It is based on an adaptation of the Canali model, driven by an effective field that is based on the electron temperature (kinetic energy) (see [High-Field Saturation on page 342](#)).

Recombination(...)

Defines the generation and recombination models.

SRH(DopingDependence)

Shockley–Read–Hall recombination with doping-dependent lifetime (Scharfetter coefficients modified in the parameter file `mos.par`).

eAvalanche(CarrierTempDrive)

Avalanche multiplication for electrons is driven by an effective field computed from the local carrier temperature.

hAvalanche(Eparallel)

Avalanche multiplication for holes is driven by the component of the field that is parallel to the hole current flow. The default impact ionization model is from van Overstraeten–de Man (see [Avalanche Generation on page 377](#)).

Main Options**Temperature**

Specifies the lattice temperature [K] (default 300 K).

1: Overview

Example: Advanced Hydrodynamic Id–Vd Simulation

IncompleteIonization

Incomplete ionization of individual species.

GateCurrent(<model>)

Selects a model for gate leakage or (dis)charging of floating gates (see [Chapter 24 on page 605](#)).

Recombination(Band2Band)

Simulates band-to-band tunneling.

NOTE Model coefficients can be specified independently for each region or material in the device structure. You can specify the model coefficients in the parameter file .par.

Interface Physics

```
Physics(MaterialInterface="Silicon/Oxide") {  
    Traps((FixedCharge Conc=4.5e+10))  
}
```

Special physical models are defined for the interfaces between specified regions or materials. In this example, an interface fixed charge is specified for all oxide–silicon interfaces with areal concentration defined in cm^{-2} .

Main Options

Interface traps can be specified and interfaces can be defined between materials or specific regions:

```
Physics(RegionInterface="region-name1/region-name2") {  
    <physics-body>  
}
```

Plot Section

```
Plot {  
    eDensity hDensity  
    eCurrent hCurrent  
    eQuasiFermi hQuasiFermi  
    eTemperature  
    ElectricField eEparallel hEparallel  
    Potential SpaceCharge  
    SRHRecombination Auger AvalancheGeneration  
    eMobility hMobility eVelocity hVelocity  
    Doping DonorConcentration AcceptorConcentration  
}
```

The variables `eTemperature` and `hEparallel` are added to the list of variables to be included in the plot file `_des.tdr`. The data is saved only if the variables are consistent with the specified physical models.

CurrentPlot Section

```
CurrentPlot {  
    Potential (82, 530, 1009)  
    eTemperature (82, 530, 1009)  
}
```

This feature allows solution variables at specified nodes to be saved to the current file `_des.plt`.

In this example, the electrostatic potential and electron temperature are saved at three nodes corresponding to selected locations in the source (# 82), drain extensions (# 530), and the center of the body (# 1009). Node numbers are identified using the `VertexIndex` variable in Tecplot SV (see [Tecplot SV User Guide, Environment Variables on page 48](#)). Any number of nodes is allowed.

Main Options

Any of the variables in [Table 140 on page 1186](#).

1: Overview

Example: Advanced Hydrodynamic Id-Vd Simulation

Math Section

```
Math {  
    Extrapolate  
    RelErrControl  
    NotDamped=50  
    Iterations=20  
    BreakCriteria {Current(Contact="drain" Absval=3e-4)}  
}
```

NotDamped=50

Specifies the number of Newton iterations over which the right-hand side (RHS)-norm is allowed to increase. With the default of 1, the error is allowed to increase for one step only. It is recommended that `NotDamped > Iterations` is set to allow a simulation to continue despite the RHS-norm increasing.

Iterations=20

Specifies the maximum number of Newton iterations allowed per bias step (default=50). If convergence is not achieved within this number of steps, for a quasistationary or transient simulation, the step size is reduced by the factor `Decrement` (see [Quasistationary Ramps on page 106](#)) and simulation continues.

BreakCriteria {Current(Contact="drain" Absval=3e-4)}

Break criteria are used to stop a simulation if a certain limit value is exceeded (see [Break Criteria: Conditionally Stopping the Simulation on page 102](#)). In this case, the simulation terminates when the drain current exceeds 3×10^{-4} A.

Example

Cylindrical (<float>)

This keyword forces a 2D device to be simulated using cylindrical coordinates, that is, it is rotated around the y-axis. The optional argument `<float>` is the x-location of the axis of symmetry (default=0).

Method=

Selects the linear solver to be used in the coupled command.

Break criteria based on bulk properties (not contact variables) can be defined in a material-specific or region-specific `Math` section:

```
Math (material="Silicon") {
    BreakCriteria { LatticeTemperature (Maxval = 1400)
                  CurrentDensity (Absval = 1e7) }
}
```

Solve Section

```
Solve {
    # initial gate voltage Vgs=0.0V
    Poisson
    Coupled { Poisson Electron }
    Coupled { Poisson Electron Hole eTemperature }
    Save(FilePrefix="vg0")
}
```

The `Solve` section defines the sequence of solutions to be obtained by the solver.

Poisson

Specifies the initial solution of the nonlinear Poisson equation. Electrodes will have initial electrical bias conditions as defined in the `Electrode` section.

In this example, all electrodes are at zero initial bias. However, the initial conditions can be nonzero. For example, it is reasonable to begin with a small bias applied to the gate or drain of a MOSFET.

Coupled { Poisson Electron }

The second step introduces carrier continuity for electrons, with the initial bias conditions still applied. In this case, the electron current continuity is solved fully coupled to the Poisson equation.

Coupled { Poisson Electron Hole eTemperature }

Solves the carrier continuity equations for both carriers and the electron temperature equations.

Save (FilePrefix="vg0")

The zero bias solution is saved to a file named with the default extension `_des.sav`, in this case, `vg0_des.sav`.

1: Overview

Example: Advanced Hydrodynamic Id-Vd Simulation

The save file contains all the information required to restart the simulation, the solution variables on the mesh, and the bias conditions on the electrodes. It can be reloaded within the same Solve section or in another simulation file. In the latter case, the model selection must be consistent.

Solve Continued

```
# ramp gate and save solutions:

# second gate voltage Vgs=1.0V
Quasistationary
( Goal { Name="gate" Voltage=1.0 }
  InitialStep=0.1 Maxstep=0.1 MinStep=0.01
)
{ Coupled { Poisson Electron Hole eTemperature } }
Save(FilePrefix="vg1"){
```

The `Quasistationary` statement implies that a series of quasistatic or steady state ‘equilibrium’ solutions can be obtained.

```
( Goal { Name="gate" Voltage=1.0 }
```

A `Goal` or set of `Goals` for one or more electrodes are defined in the parentheses. In this case, the gate bias is increased to and includes the goal of 1 V.

```
(InitialStep=0.1 Maxstep=0.1 MinStep=0.01
```

Specifies the constraints on the step size (Δt) as proportions of the normalized `Goal` ($t=1$). With the initial and maximum step sizes set to 0.1 ($t=0.1$), the gate voltage ramp is concluded in a total of ten steps, not counting the ‘zero’ step. It is assumed that convergence is achieved at each step.

If, at any step, there is a failure to converge, Sentaurus Device performs automatic step size reduction until convergence is again achieved, and then continues the simulation.

```
{ Coupled { Poisson Electron Hole eTemperature } }
```

At each step, the device equations are solved self-consistently (coupled or Newton method). Poisson, hole current continuity, electron flux and temperature continuity are specified in braces.

```
Save(FilePrefix="vg1") {
```

At the end of the ramp, another save file is created for the 1 V gate bias solution, `vg1_des.sav`. Next, the gate bias is ramped to 2 V to provide a solution file, `vg2_des.sav`.

Solve Continued

```
# Load solutions & ramp drain for family of curves
Load(FilePrefix="vg0")
NewCurrentPrefix="vg0_"
Quasistationary
  (Goal { Name="drain" Voltage=10.0 }
    InitialStep=0.01 MaxStep=0.1 MinStep=0.0001
  )
{ Coupled { Poisson Electron Hole eTemperature }
  CurrentPlot (Time =
    ( range = (0 0.2) intervals = 20;
      range = (0.2 1.0)))
}
```

Load(FilePrefix="vg0")

Loads the solution file for zero gate bias.

NewCurrentPrefix="vg0_"

Starts a new current file in which the following results are saved. The file is `vg0_n3_des.plt`.

Quasistationary

Implies that a series of quasistatic or steady state ‘equilibrium’ solutions are to be obtained.

(Goal { Name="drain" Voltage=10.0 }

In this case, the goal is to increase the drain bias up to and including the goal of 10 V. The goal is not reached if any of the break criteria are met.

InitialStep=0.01 MaxStep=0.1 MinStep=0.0001)

Specifies the constraints on the step size (Δt) as proportions of the normalized Goal ($t=1$). If, at any step, there is a failure to converge, Sentaurus Device performs automatic step size reduction until convergence is again achieved, and then continues the simulation.

{ Coupled { Poisson Electron Hole eTemperature }

At each step, the device equations are solved self-consistently (coupled or Newton method). Poisson, hole current continuity, electron flux, and temperature continuity are specified in braces.

1: Overview

Example: Advanced Hydrodynamic I_d - V_d Simulation

```
CurrentPlot (Time=(range=(0 0.2) intervals=20;  
range=(0.2 1.0)))
```

This statement ensures that solutions are saved in the current file only at certain specific values of the drain voltage in the range 0 V to 2.0 V. In this example, time implies the notional (normalized) time t whose full range is $t=0$ to $t=1$.

In this case, 21 equally spaced solutions are saved in the range $t=0$ to $t=0.2$, corresponding to 0 V and 2.0 V (that is, in steps of 0.1 V). This is in addition to any intermediate steps that may be solved but not saved. From $t=0.2$ to $t=1.0$ (2.0 V to 10.0 V), all solutions are saved in the file `vg0_n3_des.plt`.

Figure 9 shows the family of drain output characteristics (I_d - V_{ds}) in the drain bias range from 0 V to 2 V. The equal drain voltage steps of 0.1 V are suitable for SPICE parameter extraction.

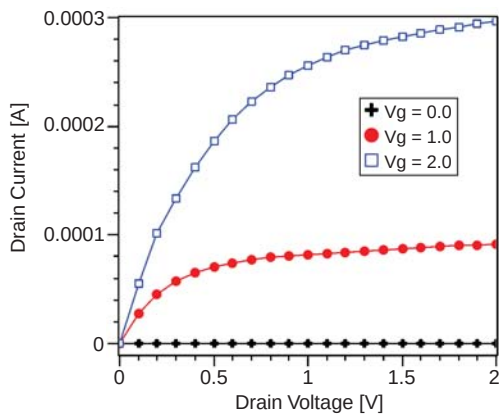


Figure 9 Family of drain output characteristics (I_d - V_{ds}) in drain bias range 0–2 V

Figure 10 shows the I_{ds} - V_{ds} characteristics plotted with the electron temperature at the drain end of the channel over the full range of the simulations.

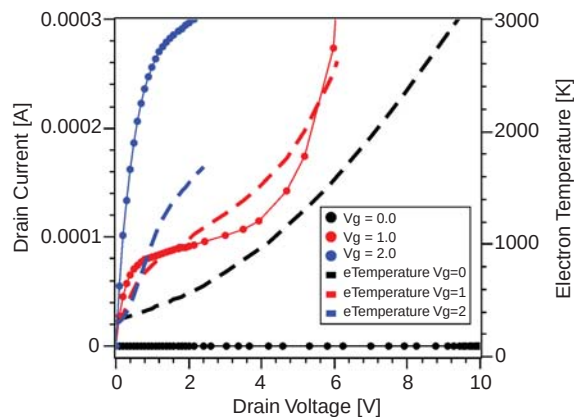


Figure 10 I_{ds} - V_{ds} characteristics plotted with electron temperature at drain end of channel over full range of simulations

Two-dimensional Output Data

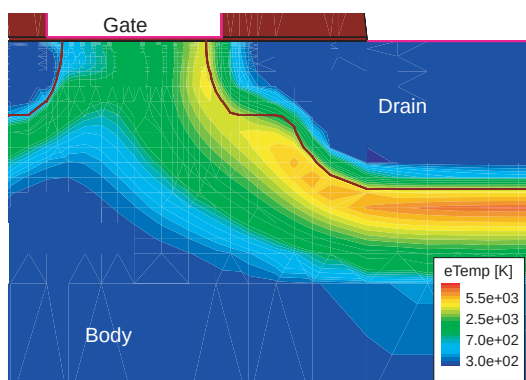


Figure 11 Contours of electron temperature computed at final solution ($V_{ds} = 2.425$ V, $I_d = 30$ mA)

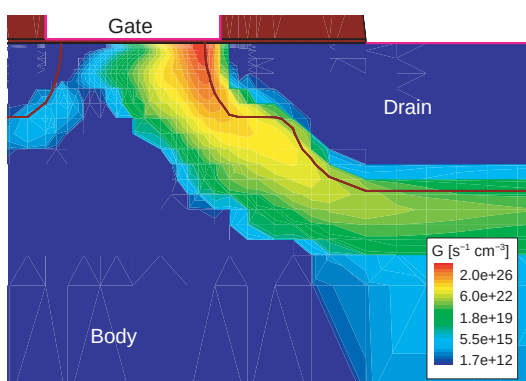


Figure 12 Contours of impact ionization rate at final solution

Example: Mixed-Mode CMOS Inverter Simulation

The mixed-mode capability of Sentaurus Device allows for the simulation of a circuit that combines any number of Sentaurus Device devices of arbitrary dimensionality (1D, 2D, or 3D) with other devices based on compact models (SPICE).

In this Sentaurus Workbench project (see `$STR00T/tcad/$STRELEASE/lib/sdevice/GettingStarted/advanced_Inverter`), a transient mixed-mode simulation is presented with two 2D physical devices; an n-channel and a p-channel MOSFET, combined with a capacitor and a voltage source to form a CMOS inverter circuit. Sentaurus Mesh is used to create 2D grids for the NMOSFET and PMOSFET devices. Sentaurus Device computes the transient response of the inverter to a voltage signal, which codes a 010 binary sequence.

1: Overview

Example: Mixed-Mode CMOS Inverter Simulation

Command File

```
#-----#
#- Sentaurus Device command file for a transient mixed-mode simulation of the
#- switching of an inverter build with a nMOSFET and a pMOSFET.
#-----#

Device NMOS {

  Electrode {
    { Name="source"      Voltage=0.0 Area=5 }
    { Name="drain"       Voltage=0.0 Area=5 }
    { Name="gate"        Voltage=0.0 Area=5 Barrier=-0.55 }
    { Name="substrate"   Voltage=0.0 Area=5 }
  }
  File {
    Grid    = "@tdr@"
    Plot    = "nmos"
    Current = "nmos"
    Param   = "mos"
  }
  Physics {
    Mobility( DopingDependence HighFieldSaturation Enormal )
    EffectiveIntrinsicDensity(BandGapNarrowing (OldSlotboom))
  }
}

Device PMOS{

  Electrode {
    { Name="source"      Voltage=0.0 Area=10 }
    { Name="drain"       Voltage=0.0 Area=10 }
    { Name="gate"        Voltage=0.0 Area=10 Barrier=0.55 }
    { Name="substrate"   Voltage=0.0 Area=10 }
  }
  File {Grid = "@tdr:+1@"
    Plot    = "pmos"
    Current = "pmos"
    Param   = "mos"
  }
  Physics {
    Mobility( DopingDependence HighFieldSaturation Enormal )
    EffectiveIntrinsicDensity(BandGapNarrowing (OldSlotboom))
  }
}

System {
  Vsource_pset v0 (n1 n0) { pwl = (0.0e+00 0.0
                                1.0e-11 0.0
```



```

1.5e-11 2.0
10.0e-11 2.0
10.5e-11 0.0
20.0e-11 0.0) }
NMOS nmos( "source"=n0 "drain"=n3 "gate"=n1 "substrate"=n0 )
PMOS pmos( "source"=n2 "drain"=n3 "gate"=n1 "substrate"=n2 )
Capacitor_pset c1 ( n3 n0 ){ capacitance = 3e-14 }
Set (n0 = 0)
Set (n2 = 0)
Set (n3 = 0)
Plot "nodes.plt" (time() n0 n1 n2 n3 )
}
File {
  Current= "inv"
  Output = "inv"
}
Plot {
  eDensity hDensity eCurrent hCurrent
  ElectricField eEnormal hEnormal
  eQuasiFermi hQuasiFermi
  Potential Doping SpaceCharge
  DonorConcentration AcceptorConcentration
}
Math {
  Extrapolate
  RelErrControl
  Digits=4
  Notdamped=50
  Iterations=12
  NoCheckTransientError
}
Solve {
  #-build up initial solution
  NewCurrentPrefix = "ignore_"
  Coupled { Poisson }
  Quasistationary ( InitialStep=0.1 MaxStep=0.1
                    Goal { Node="n2" Voltage=2 }
                    Goal { Node="n3" Voltage=2 }
                    )
    { Coupled { Poisson Electron Hole } }
  NewCurrentPrefix = ""
  Unset (n3)
  Transient (
    InitialTime=0 FinalTime=20e-11
    InitialStep=1e-12 MaxStep=1e-11 MinStep=1e-15
    Increment=1.3
  )
  { Coupled { nmos.poisson nmos.electron nmos.contact

```

1: Overview

Example: Mixed-Mode CMOS Inverter Simulation

```
        pmos.poisson pmos.hole pmos.contact circuit }
    }
}
$STROOT/tcad/$STRELEASE/lib/sdevice/GettingStarted/advanced_Inverter/
sdevice_des.cmd
```

Device Section

The sequence of command sections is different when comparing mixed-mode to single-device simulation. For mixed-mode simulations, the physical devices are defined in separate Device statement sections. The following is the section for an n-channel MOSFET:

```
Device NMOS {
  Electrode {
    { Name="source" Voltage=0.0 Area=5 }
    { Name="drain" Voltage=0.0 Area=5 }
    { Name="gate" Voltage=0.0 Area=5 Barrier=-0.55 }
    { Name="substrate" Voltage=0.0 Area=5 }
  }
  File {
    Grid    = "@tdr@"
    Plot    = "nmos"
    Current = "nmos"
    Param   = "mos"
  }
  Physics {
    Mobility( DopingDependence HighFieldSaturation Enormal)
    EffectiveIntrinsicDensity(BandGapNarrowing OldSlotboom )
  }
}
```

For a mixed-mode simulation (see [Device Section on page 86](#)), the physical devices are named NMOS and PMOS, and are defined in separate Device statements.

Inside the Device statements, the Electrode, Physics, and most of the File sections are defined in the same way as in command files for single device simulations.

NOTE The p-channel has twice the AreaFactor of the n-channel MOSFET, which is equivalent to setting twice the gate width.

Different physical models can be applied in each device type, as well as different coefficients if each device has a dedicated parameter file.

The equivalent section for the p-channel device is defined almost identically except that the source file for the p-channel structure is @tdr:+1@ (referring to the Sentaurus Workbench

node corresponding to the Sentaurus Mesh split for the p-channel structure), and the plot and current files have the prefix pmos. Further, Barrier=+0.55 for the p-channel MOSFET.

System Section

The circuit is defined in the System section, which uses a SPICE syntax. The two MOSFETs are connected to form a CMOS inverter with a capacitive load and voltage source for the input signal.

```
System {
  Vsource_pset v0 (n1 n0) {pwl = (0.0e+00 0.0
                                1.0e-11 0.0
                                1.5e-11 2.0
                                10.0e-11 2.0
                                10.5e-11 0.0
                                20.0e-11 0.0)}

  NMOS nmos( "source"=n0 "drain"=n3 "gate"=n1 "substrate"=n0 )
  PMOS pmos( "source"=n2 "drain"=n3 "gate"=n1 "substrate"=n2 )

  Capacitor_pset c1 ( n3 n0 ){ capacitance = 3e-14 }
  Set (n0 = 0)
  Set (n2 = 0)
  Set (n3 = 0)
  Plot "nodes.plt" (time() n0 n1 n2 n3 )
}
```

Vsource_pset v0 (n1 n0)...

A voltage source that generates a piecewise linear (pwl) voltage signal is connected between the input node (n1) and ground node (n0). The time-voltage sequence generates a low-high-low or 010 binary sequence over a 200 ps time period.

NMOS nmos (...)

The previously defined device named NMOS is instantiated with a tag nmos. Each of its electrodes is connected to a circuit node. (If an electrode is not connected to the circuit, it is driven by any bias conditions specified in the corresponding Electrode statement.)

NOTE A physical device can be instantiated any number of times in a mixed-mode circuit. Therefore, it is necessary to assign a name for each instance in the circuit. In this example, the name nmos is chosen.

PMOS pmos (...)

The PMOSFET is instantiated similarly.

1: Overview

Example: Mixed-Mode CMOS Inverter Simulation

```
Capacitor_pset c1 ( n3 n0 )...
```

Capacitive load (c1) is connected between the output node (n3) and ground node (n0).

```
Set ( n0 = 0 )
```

```
Set ( n2 = 0 )
```

```
Set ( n3 = 0 )
```

The **Set** command defines the nodal voltages at the beginning of the simulation. These definitions are kept until an **Unset** command is specified. Node n0 is tied to the ground, and n2 and n3 are initially set to 0 V.

File Section

```
File {  
    Current = "inv"  
    Output = "inv"  
}
```

Output file names, which are not device-specific, are defined outside of the **Device** sections.

Plot Section

```
Plot {  
    eDensity hDensity eCurrent hCurrent  
    ElectricField eEnormal hEnormal  
    eQuasiFermi hQuasiFermi  
    Potential Doping SpaceCharge  
    DonorConcentration AcceptorConcentration  
}
```

In this case, the **Plot** statement is global and applies to all physical devices. It can also be specified inside individual **Device** sections.

Math Section

```
Math {  
    ...  
    NoCheckTransientError  
}
```

NoCheckTransientError

This keyword disables the computation of error estimates based on time derivatives. This error estimation scheme is inappropriate when abrupt changes in time are enforced externally.

In this example, it is advantageous to specify this option because the inverter is driven by a voltage pulse with very steep rising and falling edges.

Solve Section

The circuit and physical device equations are solved self-consistently for the duration of the input pulse. A transient simulation is performed for the time duration specified in the Transient command.

```
Solve {  
    #-build up initial solution  
    NewCurrentPrefix = "ignore_"  
    Coupled { Poisson }  
    Quasistationary ( InitialStep=0.1 MaxStep=0.1  
                      Goal { Node="n2" Voltage=2 }  
                      Goal { Node="n3" Voltage=2 }  
                      )  
                      { Coupled { Poisson Electron Hole } }  
    NewCurrentPrefix = ""  
    Unset (n3)  
    Transient (  
        InitialTime=0 FinalTime=20e-11  
        InitialStep=1e-12 MaxStep=1e-11 MinStep=1e-15  
        Increment=1.3  
    )  
    { Coupled { nmos.poisson nmos.electron nmos.contact  
                pmos.poisson pmos.hole pmos.contact circuit }  
    }  
}
```

The initial solution is obtained in steps. It starts with the Poisson equation. Then, the electron and hole carrier continuity equations are introduced, and the nodes n2 and n3 are ramped to

1: Overview

Example: Mixed-Mode CMOS Inverter Simulation

the supply voltage of 2 V. The circuit and contact equations are included automatically with the continuity equations.

Unset (n3)

When the initial solution is established, the output node (n3) is released.

Transient (...)

In the Transient command, the start time, final time, and step size constraints (initial, maximum, minimum) are in seconds. Actual step sizes are determined internally, based on the rate of convergence of the solution at the previous step. Increment=1.3 determines the maximum step size increase.

```
{Coupled { nmos.poisson nmos.electron nmos.contact  
           pmos.poisson pmos.hole      pmos.contact  
           circuit } }
```

The Coupled statement illustrates how, in a mixed-mode environment, a specific set of equations can be selected. In this example, the electron continuity equation is solved in the NMOSFET. The hole continuity equation is solved for the PMOSFET. The corresponding contact equations for nmos and pmos and the circuit equations are added explicitly.

Results of Inverter Transient Simulation

The simulation results are plotted automatically using Inspect, driven by the command file pp3_ins.cmd, which is created (or preprocessed) by Sentaurus Workbench from the root command file inspect_ins.cmd. In [Figure 13](#), the transient response of the output voltage and drain current through the n-channel device is plotted, overlaying the input pulse.

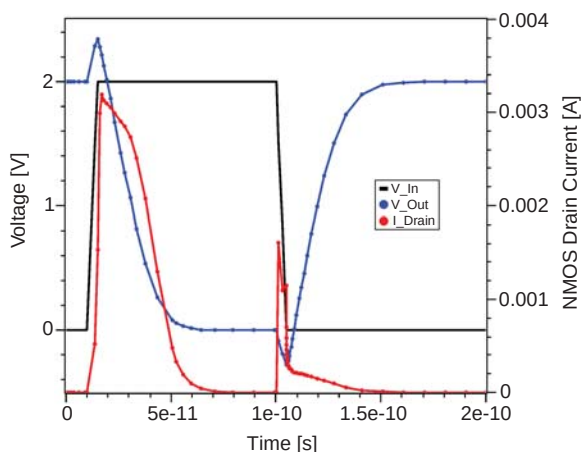


Figure 13 Inspect output from Sentaurus Workbench project (GettingStarted/advanced_Inverter)

Example: Small-Signal AC Extraction

This example demonstrates how to perform an AC analysis simulation for an NMOSFET. In Sentaurus Device, AC simulations are performed in mixed mode. In an AC simulation, Sentaurus Device computes the complex (small signal) admittance Y matrix. This matrix specifies the current response at a given node to a small voltage signal at another node:

$$j = Yu = Au + i\omega Cu \quad (1)$$

where j is the vector containing the small-signal currents at all nodes and u is the corresponding voltage vector.

Sentaurus Device output contains the components of the conductance matrix A and the capacitance matrix C (see [Small-Signal AC Analysis on page 127](#)). The conductances and capacitances are used to construct the small signal equivalent circuit or to compute the other AC parameters, such as H , Z , and S .

Command File

```
#-----#
#- Sentaurus Device command file for
#- AC analysis at 1 MHz while Vg=-2 to 3V and Vd=2V
#-----#
Device NMOS {
  Electrode {
    { Name="source"      Voltage=0.0 }
    { Name="drain"       Voltage=0.0 }
    { Name="gate"        Voltage=0.0 Barrier=-0.55 }
    { Name="substrate"   Voltage=0.0 }
  }
  File {
    Grid    = "@tdr@"
    Current = "@plot@"
    Plot    = "@tdrdat@"
    Param   = "mos"
  }
  Physics {
    Mobility ( DopingDependence HighFieldSaturation Enormal )
    EffectiveIntrinsicDensity(BandGapNarrowing (OldSlotboom))
  }
  Plot {
    eDensity hDensity eCurrent hCurrent
    ElectricField eParallel hParallel
    eQuasiFermi hQuasiFermi
    Potential Doping SpaceCharge
  }
}
```

1: Overview

Example: Small-Signal AC Extraction

```
        DonorConcentration AcceptorConcentration
    }
}
Math {
    Extrapolate
    RelErrControl
    Notdamped=50
    Iterations=20
}
File {
    Output      = "@log@"
    ACExtract   = "@acplot@"
}
System {
    NMOS trans (drain=d source=s gate=g substrate=b)
    Vsource_pset vd (d 0) {dc=0}
    Vsource_pset vs (s 0) {dc=0}
    Vsource_pset vg (g 0) {dc=0}
    Vsource_pset vb (b 0) {dc=0}
}
Solve (
    #-a) zero solution
    Poisson
    Coupled { Poisson Electron Hole }

    #-b) ramp drain to positive starting voltage
    Quasistationary (
        InitialStep=0.1 MaxStep=0.5 MinStep=1.e-5
        Goal { Parameter=vd.dc Voltage=2 }
    )
    { Coupled { Poisson Electron Hole } }

    #-c) ramp gate to negative starting voltage
    Quasistationary (
        InitialStep=0.1 MaxStep=0.5 MinStep=1.e-5
        Goal { Parameter=vg.dc Voltage=-2 }
    )
    { Coupled { Poisson Electron Hole } }

    #-d) ramp gate -2V to +3V
    Quasistationary (
        InitialStep=0.01 MaxStep=0.04 MinStep=1.e-5
        Goal { Parameter=vg.dc Voltage=3 }
    )
    { ACCoupled (
        StartFrequency=1e6 EndFrequency=1e6
        NumberOfPoints=1 Decade
        Node(d s g b) Exclude(vd vs vg vb)
    )
}
```



```
)  
  { Poisson Electron Hole }  
}  
}
```

```
$STROOT/tcad/$STRELEASE/lib/sdevice/GettingStarted/advanced_AC/sdevice_des.cmd
```

Device Section

```
Device NMOS {  
  
  Electrode {  
    { Name="source" Voltage=0.0 }  
    { Name="drain" Voltage=0.0 }  
    { Name="gate" Voltage=0.0 Barrier=-0.55 }  
    { Name="substrate" Voltage=0.0 }  
  }  
  File {  
    Grid      = "@tdr@"  
    Current   = "@plot@"  
    Plot      = "@tdrdat@"  
    Param     = "mos"  
  }  
  Physics {  
    Mobility (DopingDependence HighFieldSaturation Enormal)  
    EffectiveIntrinsicDensity (BandGapNarrowing (OldSlotboom))  
  }  
  Plot {  
    eDensity hDensity eCurrent hCurrent  
    ElectricField eEparallel hEparallel  
    eQuasiFermi hQuasiFermi  
    Potential Doping SpaceCharge  
    DonorConcentration AcceptorConcentration  
  }  
}
```

The AC analysis is performed in a mixed-mode environment (see [Small-Signal AC Analysis on page 127](#)). In this environment, the physical device named NMOS is defined using the Device statement.

The File section inside Device includes all device-specific files, but cannot contain the Output identifier. This identifier is outside the Device statement in a separate global File section, with the file identifier for the data file, which contains the extracted AC results (see [File Section on page 44](#)).

1: Overview

Example: Small-Signal AC Extraction

File Section

```
File {  
    Output = "@log@"  
    ACExtract = "@acplot@"  
}
```

Output files that are not device-specific are specified outside of the Device section(s).

The computed small-signal AC components are saved into a file defined by @acplot@, which, for a Sentaurus Device node number X, is replaced by the Sentaurus Workbench preprocessor with file name nX_ac_des.plt.

System Section

```
System {  
    NMOS trans (drain=d source=s gate=g substrate=b)  
    Vsource_pset vd (d 0) {dc=0}  
    Vsource_pset vs (s 0) {dc=0}  
    Vsource_pset vg (g 0) {dc=0}  
    Vsource_pset vb (b 0) {dc=0}  
}
```

A simple circuit is defined as a SPICE netlist in the System section. For a standard AC analysis, a voltage source is attached to each contact of the physical device. Each voltage source is given a different instance name.

Solve Section

```
Solve {  
    #-a) zero solution  
    Poisson  
    Coupled { Poisson Electron Hole }  
  
    #-b) ramp drain to positive starting voltage  
    Quasistationary (  
        InitialStep=0.1 MaxStep=0.5 Minstep=1.e-5  
        Goal { Parameter=vd.dc Voltage=2 }  
    )  
    { Coupled { Poisson Electron Hole } }  
  
    #-c) ramp gate to negative starting voltage  
    Quasistationary (  
        InitialStep=0.1 MaxStep=0.5 Minstep=1.e-5
```

```

Goal { Parameter=vg.dc Voltage=-2 }
)
{ Coupled { Poisson Electron Hole } }

#-d) ramp gate -2V..3V : AC analysis at each step.
Quasistationary (
  InitialStep=0.01 MaxStep=0.04 MinStep=1.e-5
  Goal { Parameter=vg.dc Voltage=3 }
)
{ ACCoupled (
  StartFrequency=1e6 EndFrequency=1e6
  NumberOfPoints=1 Decade
  Node(d s g b) Exclude(vd vs vg vb)
)
  { Poisson Electron Hole }
}
}

```

The initial solution is obtained in steps. It starts with the Poisson equation and introduces the electron and hole carrier continuity equations, and the circuit equations, which are included by default.

```

#-d) ramp gate -2V to +3V : AC analysis at each step.
Quasistationary (
  InitialStep=0.01 MaxStep=0.04 MinStep=1.e-5
  Goal { Parameter=vg.dc Voltage=3 }
)
{ ACCoupled (
  StartFrequency=1e6 EndFrequency=1e6
  NumberOfPoints=1 Decade
  Node(d s g b) Exclude(vd vs vg vb)
)
  { Poisson Electron Hole }
}

```

This third Quasistationary statement performs an AC analysis at a single frequency (1×10^6 Hz) at each DC bias step during a sweep of the voltage source vg, which is attached to the gate.

Analysis at multiple frequencies is possible by defining a value for StartFrequency and EndFrequency.

Node(d s g b)

The AC analysis is performed between the circuit nodes d, s, g, and b. The conductance and capacitance matrices contain 16 elements each: $a(d, d)$, $c(d, d)$, $a(d, s)$, $c(d, s)$, ..., $a(b, b)$, $c(b, b)$.

1: Overview

Example: Small-Signal AC Extraction

`Exclude(vd vs vg vb)`

Excludes all voltage sources from the AC analysis.

Results of AC Simulation

When Sentaurus Device is run inside the Sentaurus Workbench project `advanced_AC`, the simulation results are plotted automatically after completion by `Inspect`, which is driven by the command file `pp3_ins.cmd`. The total small-signal gate capacitance $C_g = c(g, g)$ is plotted against gate voltage. Two small-signal conductances are also plotted (see [Figure 14](#)), where $g_m = a(d, g)$ is the small-signal transconductance and $g_d = a(d, d)$ is the small-signal drain output conductance.

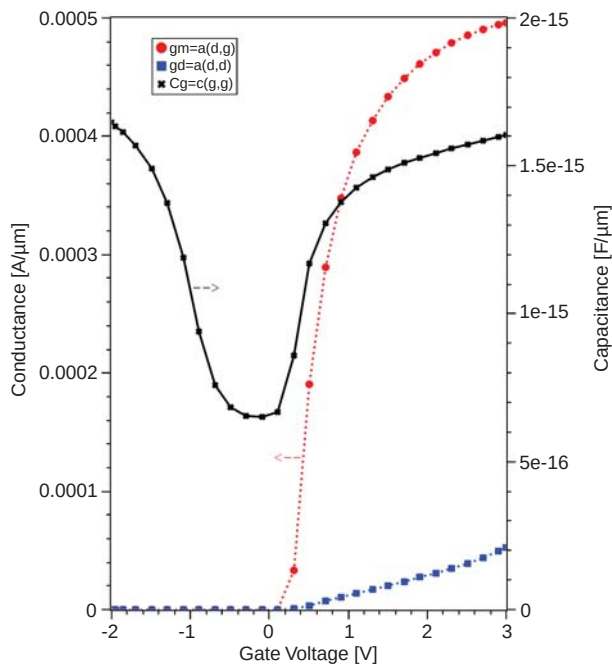


Figure 14 Inspect output from Sentaurus Workbench project AC simulation

CHAPTER 2 Defining Devices

This chapter describes how physical devices are specified.

Before performing a simulation, Sentaurus Device needs to know which device will actually be simulated. This chapter describes what Sentaurus Device needs to know and how to specify it. This includes obvious information, such as the size and shape of the device, the materials of which the device is made, and the doping profiles. It also includes less obvious aspects such as which physical effects must be taken into account, and by which models and model parameters this should be performed.

Reading a Structure

A device is defined by its shape, material composition, and doping. This information is defined on a grid and contained in a TDR file that you specify with the keyword `Grid` in the `File` section of the command file, for example:

```
File {  
    Grid = "mosfet.tdr"  
    ...  
}
```

The following keywords affect the interpretation of the geometry in the `Grid` file:

- `CoordinateSystem` in the global `Math` section specifies the coordinate system that is used for explicit coordinates in the Sentaurus Device command file. Many features such as current plot statements (see [Tracking Additional Data in the Current File on page 140](#)) or `LatticeParameters` in the parameter file (see [Crystal Reference System on page 669](#)) use explicit coordinates, and they are based on an implicit assumption regarding the orientation of the device. By specifying:

```
Math {  
    CoordinateSystem { <option> }  
}
```

you can indicate the required orientation of the device. [Table 1](#) lists the available options.

Table 1 Coordinate system options

Option	Description
AsIs	(Default) This option specifies that the coordinate system of the device is compatible with the explicit coordinates used in the Sentaurus Device command file. No transformation is applied to the structure.

Table 1 Coordinate system options

Option	Description
DFISE	In 2D, the x-axis points across the surface, and the y-axis points down into the device. In 3D, the x-axis and y-axis span the surface of the device, and the z-axis points upwards away from the device.
UCS	The unified coordinate system (UCS) uses the convention of the simulation coordinate system as established by Sentaurus Process. The x-axis always points down into the device. In 2D, the y-axis runs across the surface of the device. In 3D, the z-axis is added to obtain a right-handed coordinate system.

If the coordinate system of the device is incompatible with the specification in the **Math** section, Sentaurus Device automatically transforms the device into the required coordinate system.

- **Cylindrical** in the global **Math** section specifies that the device is simulated using cylindrical coordinates. In this case, a 3D device is specified by a 2D mesh and the vertical or horizontal axis around which the device is rotated.
- **AreaFactor** in the global **Physics** section specifies a multiplier for currents and charges. For 1D or 2D simulations, it typically specifies the extension of the device in the remaining one or two dimensions. In simulations that exploit the symmetry of the device, it can also be used to account for the reduction of the simulated device compared to the real device. **AreaFactor** is also available in the **Electrode** and **Thermode** sections, with the same meaning; if both **AreaFactors** are present, Sentaurus Device multiplies them.

TDR units are taken into account when loading from a .tdr file. Thereby, the units read from files are converted to the appropriate units used in Sentaurus Device. The TDR unit is ignored in the case of a conversion failure. Use the keyword **IgnoreTdrUnits** in the **Math** section to disregard TDR units during loading. This applies not only to the **Grid** file, but also to other loaded files (see [Save and Load on page 174](#)).

Abrupt and Graded Heterojunctions

Sentaurus Device supports both abrupt and graded heterojunctions, with an arbitrary mole fraction distribution. In the case of abrupt heterojunctions, Sentaurus Device treats discontinuous datasets properly by introducing double points at the heterointerfaces.

This option is switched on automatically when thermionic emission (see [Thermionic Emission Current](#)) is selected, or when the keyword **HeteroInterface** is specified in the **Physics** section of a selected heterointerface. By default, this double points option is switched off.

NOTE The keyword `HeteroInterface` provides equilibrium conditions (continuous quasi-Fermi potentials) for the double points. It does not provide realistic physics at the interface for high-current regimes. Using the `HeteroInterface` option without thermionic emission or a tunneling model is discouraged.

To illustrate the double points option, [Figure 15](#) shows the conduction band near an abrupt heterointerface. The wide line shows a case without double points, which requires a very fine mesh to avoid a large barrier error (δE_C).

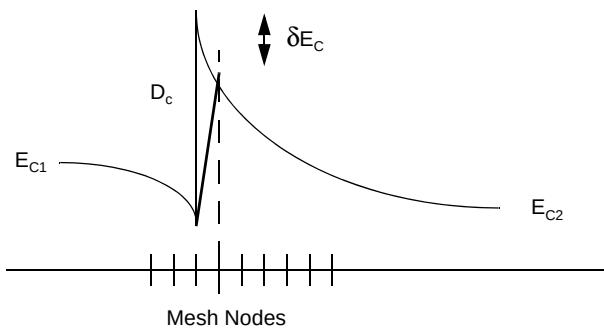


Figure 15 Band edge at a heterointerface with and without double points

Doping Specification

By default, Sentaurus Device supports all typical species of silicon technology (donors: As, P, and Sb, and acceptors: B and In) and allows user-defined ones (see [User-Defined Species on page 50](#)). Usually, nitrogen is not recognized as a doping species. To enable nitrogen as a donor, use the keyword `UseNitrogenAsDopant` in the global `Physics` section. SiC dopants (acceptor Al and donor N) are treated as special cases. In SiC regions, the two dopants, Al and N, are recognized automatically. In all other regions, N is recognized as a donor if the keyword `UseNitrogenAsDopant` is used in the global `Physics` section.

Sentaurus Device loads doping distributions from the `Grid` file specified in the `File` section (see [Reading a Structure on page 47](#)) and reads the following datasets:

- Net doping (`DopingConcentration` in the `Grid` file)
- Total doping (`TotalConcentration` in the `Grid` file)
- Concentrations of individual species

Sentaurus Device supports the following rules of doping specification:

- Sentaurus Device takes the net doping dataset from the file if this dataset is present. Otherwise, net doping is recomputed from the concentrations of the separate species.
- The same rule is applied for the total doping dataset.

2: Defining Devices

Doping Specification

NOTE Total concentration, which originates from process simulators (for example, Sentaurus Process), is the sum of the chemical concentrations of dopants. However, if the total concentration is recomputed inside Sentaurus Device, active concentrations are used.

- Sentaurus Device takes the active concentration of a dopant if it is in the Grid file (for example, `BoronActiveConcentration`). Otherwise, the chemical dopant concentration is used (for example, `BoronConcentration`).

To perform any simulation, Sentaurus Device must prepare four major doping arrays:

- The net doping concentration, $N_{\text{net}} = N_{\text{D},0} - N_{\text{A},0}$.
- $N_{\text{D},0}$ and $N_{\text{A},0}$, the donor and acceptor concentrations.
- The total doping concentration $N_{\text{D},0} + N_{\text{A},0}$.

After loading the doping file, Sentaurus Device uses the following scheme to compute the doping arrays:

1. If the Grid file does not have any species: N_{net} is initialized from `DopingConcentration`, N_{tot} is initialized from `TotalConcentration` or $|N_{\text{net}}|$ if `TotalConcentration` was not read, $N_{\text{D},0} = (N_{\text{tot}} + N_{\text{net}})/2$, and $N_{\text{A},0} = (N_{\text{tot}} - N_{\text{net}})/2$.
2. If the Grid file has individual species: $N_{\text{A},0}$ and $N_{\text{D},0}$ are computed as the sum of individual donor and acceptor concentrations, N_{net} is initialized from `DopingConcentration` or $N_{\text{D},0} - N_{\text{A},0}$ if `DopingConcentration` was not read, and N_{tot} is initialized from `TotalConcentration` or $N_{\text{A},0} + N_{\text{D},0}$ if `TotalConcentration` was not read.
3. If the command file contains trap specifications (see [Chapter 17 on page 407](#)) in the Physics section with the keyword `Add2TotalDoping` (see [Table 261 on page 1316](#)), the trap concentration is added to the acceptor or donor doping concentration (depending on the sign of the trap), and also to the total doping concentration. However, if instead the keyword `Add2TotalDoping(ChargedTraps)` is specified, the *charged* trap concentration is added to the acceptor or donor doping concentration for mobility calculations only.

User-Defined Species

For the simulation of other semiconductors such as III–V compounds, the actual dopants (such as Si and Be) are supported only through user-defined species. All such species used during a simulation must be declared in the `Variables` section of the file `datexcodes.txt` (see [Material Specification](#)). If the incomplete ionization model is activated (see [Chapter 13 on page 273](#)), the model parameters of user-defined species must be specified in the `Ionization` section of the material parameter file.

Each new dopant must have two subsections in the file `datexcodes.txt`. One subsection must have the field:

```
doping = type(ionized_code = <code value>)
```

where `type` can be a donor or an acceptor, and `ionized_code` is the code of another subsection that corresponds to the ionized dopant concentration.

User-defined species can be plotted by specifying the following keywords in the `Plot` section:

```
Plot {  
    UserSpeciesConcentration  
    UserSpeciesIncompleteConcentration  
}
```

If the option `UserSpeciesIncompleteConcentration` or `UserSpeciesConcentration` is activated, all user-defined species are saved.

The following example shows the definition of nitrogen as a donor-type species:

```
Variables {  
    ...  
    Nitrogen {  
        code = 960  
        label = "total (chemical) Nitrogen concentration"  
        symbol = "NitrogenTotal"  
        unit = "cm^-3"  
        factor = 1.0e+12  
        precision = 4  
        interpol = log  
        material = Semiconductor  
        doping = donor(ionized_code = 961)  
    }  
    NitrogenPlus {  
        code = 961  
        label = "Nitrogen+ concentration (incomplete ionization)"  
        symbol = "N-Nitrogen+"  
        unit = "cm^-3"  
        factor = 1.0e+12  
        precision = 4  
        interpol = log  
        material = Semiconductor  
    }  
    ...  
}
```

Material Specification

Sentaurus Device supports all materials that are declared in the files `datexcodes.txt` (see [Utilities User Guide, Chapter 1 on page 1](#)). The following search strategy is observed to locate the `datexcodes.txt` files:

- `$STROOT_LIB/datexcodes.txt` or `$STROOT/tcad/$STRELEASE/lib/datexcodes.txt` if the environment variable `STROOT_LIB` is not defined (lowest priority)
- `$HOME/datexcodes.txt` (medium priority)
- `datexcodes.txt` in local directory (highest priority)

Definitions in later files replace or add to the definitions in earlier files.

User-Defined Materials

New materials can be defined in a local `datexcodes.txt` file. To add a new material, add its description to the `Materials` section of `datexcodes.txt`:

```
Materials {  
  Silicon {  
    label = "Silicon"  
    group = Semiconductor  
    color = #ffb6c1  
  }  
  Oxide {  
    label = "SiO2"  
    group = Insulator  
    color = #7d0505  
  }  
  ...  
}
```

The `label` value is used as a legend in visualization tools such as Tecplot SV.

The `group` value identifies the type of new material. The available values are:

```
Conductor  
Insulator  
Semiconductor
```

The field `color` defines the color of the material in visualization tools. This field must have the syntax:

`color = #rrggbb`

where `rr`, `gg`, and `bb` denote hexadecimal numbers representing the intensity of red, green, and blue, respectively. The values of `rr`, `gg`, and `bb` must be in the range 00 to ff.

[Table 2](#) lists sample values for `color`.

Table 2 Sample color values

Color code	Color	Color code	Color
#000000	Black	#ffffff	White
#ff0000	Red	#40e0d0	Turquoise
#00ff00	Green	#7fff00	Chartreuse
#0000ff	Blue	#b03060	Maroon
#ffff00	Yellow	#ff7f50	Coral
#ff00ff	Magenta	#da70d6	Orchid
#00ffff	Cyan	#e6e6fa	Lavender

Mole-Fraction Materials

Sentaurus Device reads the file `Molefraction.txt` to determine mole fraction-dependent materials. The following search strategy is used to locate this file:

1. Sentaurus Device looks for `Molefraction.txt` in the current working directory.
2. Sentaurus Device checks if the environment variable `SDEVICEDB` is defined. This variable should contain a directory or a list of directories separated by spaces or colons, for example:

`SDEVICEDB="/home/usr/lib /home/tcad/lib"`

Sentaurus Device scans the directories in the given order until `Molefraction.txt` is found.

NOTE The environment variable `SDEVICEDB` is also used to locate included parameter files (see [Physical Model Parameters on page 60](#)).

2: Defining Devices

Material Specification

3. If the environment variables `STROOT` and `STRELEASE` are defined, Sentaurus Device tries to read the file:

`$STROOT/tcad/$STRELEASE/lib/sdevice/MaterialDB/Molefraction.txt`

4. If these previous strategies are unsuccessful, Sentaurus Device uses the built-in defaults that follow.

The default `Molefraction.txt` file has the following content:

```
# Ge(x)Si(1-x)
SiliconGermanium (x=0) = Silicon
SiliconGermanium (x=1) = Germanium
```

```
# Al(x)Ga(1-x)As
AlGaAs (x=0) = GaAs
AlGaAs (x=1) = AlAs
```

```
# In(1-x)Al(x)As
InAlAs (x=0) = InAs
InAlAs (x=1) = AlAs
```

```
# In(1-x)Ga(x)As
InGaAs (x=0) = InAs
InGaAs (x=1) = GaAs
```

```
# Ga(x)In(1-x)P
GaInP (x=0) = InP
GaInP (x=1) = GaP
```

```
# InAs(x)P(1-x)
InAsP (x=0) = InP
InAsP (x=1) = InAs
```

```
# GaAs(x)P(1-x)
GaAsP (x=0) = GaP
GaAsP (x=1) = GaAs
```

```
# Hg(1-x)Cd(x)Te
HgCdTe (x=0) = HgTe
HgCdTe (x=1) = CdTe
```

```
# In(1-x)Ga(x)As(y)P(1-y)
InGaAsP (x=0, y=0) = InP
InGaAsP (x=1, y=0) = GaP
InGaAsP (x=1, y=1) = GaAs
InGaAsP (x=0, y=1) = InAs
```

To add a new mole fraction–dependent material, the material (and its side and corner materials) must first be added to `datexcodes.txt`. Afterwards, `Molefraction.txt` can be updated.

Quaternary alloys are specified by their corner materials in the file `Molefraction.txt`. For example, the 2:2 III–V quaternary alloy $\text{In}_{1-x}\text{Ga}_x\text{As}_y\text{P}_{1-y}$ is given by:

```
InGaASP (x=0, y=0) = InP
InGaASP (x=1, y=0) = GaP
InGaASP (x=1, y=1) = GaAs
InGaASP (x=0, y=1) = InAs
```

and the 3:1 III–V quaternary alloy $\text{Al}_x\text{Ga}_y\text{In}_{1-x-y}\text{As}$ is defined by:

```
AlGaInAs (x=0, y=0) = InAs
AlGaInAs (x=1, y=0) = AlAs
AlGaInAs (x=0, y=1) = GaAs
```

When the corner materials of an alloy have been specified, Sentaurus Device determines the corresponding side materials automatically. In the case of $\text{In}_{1-x}\text{Ga}_x\text{As}_y\text{P}_{1-y}$, the four side materials are $\text{InAs}_x\text{P}_{1-x}$, $\text{GaAs}_x\text{P}_{1-x}$, $\text{GaIn}_{1-x}\text{P}$, and $\text{In}_{1-x}\text{Ga}_x\text{As}$. Similarly, for $\text{Al}_x\text{Ga}_y\text{In}_{1-x-y}\text{As}$, there are the three side materials $\text{In}_{1-x}\text{Al}_x\text{As}$, $\text{Al}_x\text{Ga}_{1-x}\text{As}$, and $\text{In}_{1-x}\text{Ga}_x\text{As}$.

NOTE All side and corner materials must appear in `datexcodes.txt`, and their mole dependencies must be specified in `Molefraction.txt` (see [Material Specification on page 52](#)).

NOTE If it cannot parse the file `Molefraction.txt`, Sentaurus Device reverts to the defaults shown above. This may lead to unexpected simulation results.

Mole-Fraction Specification

In Sentaurus Device, the mole fraction of a compound semiconductor or insulator is defined in two ways:

- In the Grid file (`<name>.tdr`) of the device structure
- Internally, in the Physics section of the command file

If the mole fraction is loaded from the `.tdr` file and an internal mole fraction specification is also applied, the loaded mole fraction values are overwritten in the regions specified in the `MoleFraction` sections of the command file.

2: Defining Devices

Mole-Fraction Specification

The internal mole fraction distribution is described in the `MoleFraction` statement inside the `Physics` section:

```
Physics { ...  
    MoleFraction(<MoleFraction parameters>)  
}
```

The parameters for the mole fraction specification and grading options are described in [Table 247 on page 1309](#).

The specification of an `xFraction` is mandatory in the `MoleFraction` statement for binary or ternary compounds; a `yFraction` is also mandatory for quaternary materials. If the `MoleFraction` statement is inside a default `Physics` section, the `RegionName` must be specified. If it is inside a region-specific `Physics` section, by default, it is applied only to that region. If a `MoleFraction` statement is inside a material-specific `Physics` section and the `RegionName` is not specified, this composition is applied to all regions containing the specified material. If `RegionName` is specified inside a region-specific and material-specific `Physics` section, this specification is used instead of the default regions.

NOTE Similar to all statements, only one `MoleFraction` statement is allowed inside each `Physics` section. By default, grading is not included.

An example of a mole fraction specification is:

```
Physics {  
    MoleFraction(RegionName = ["Region.3" "Region.4"]  
        xFraction=0.8  
        yFraction=0.7  
        Grading(  
            (xFraction=0.3 GrDistance=1  
                RegionInterface=("Region.0" "Region.3"))  
            (xFraction=0.2 yFraction=0.1 GrDistance=1  
                RegionInterface=("Region.0" "Region.5"))  
            (yFraction=0.4 GrDistance=1  
                RegionInterface=("Region.0" "Region.3"))  
        )  
    )  
}  
Physics (Region = "Region.6") {  
    MoleFraction(xFraction=0.1 yFraction=0.7 GrDistance=0.01)  
}
```

Physical Models and the Hierarchy of Their Specification

The `Physics` section is used to select the models that are used to simulate a device. [Table 182 on page 1259](#) lists the keywords that are available, and Part II and Part III discuss the models in detail.

Physical models can be specified globally, per region or material, per interface, or per electrode.

Some models (for example, the hydrodynamic transport model) can only be activated for the whole device. Regionwise or materialwise specifications are syntactically possible, but SENTAURUS Device silently ignores them. Likewise, some specifications are syntactically possible for all locations, but they are semantically valid only for interfaces or bulk regions. In the tables in [Appendix G on page 1217](#), the validity of specifications is indicated in the description column by characters in parentheses. For example, (g) denotes models that can be activated only for the entire device, but not for individual parts of it.

Some specifications are syntactically possible everywhere, but are valid for certain materials only. For example, `Mobility` is only valid in semiconductors.

Region-specific and Material-specific Models

In SENTAURUS Device, different physical models for different regions and materials within a device structure can be specified. The syntax for this feature is:

```
Physics (material="material") {  
    <physics-body>  
}
```

or:

```
Physics (region="region-name") {  
    <physics-body>  
}
```

This feature is also available for the `Math` section:

```
Math (material="material") {  
    <math-body>  
}
```

2: Defining Devices

Physical Models and the Hierarchy of Their Specification

or:

```
Math (region="region-name") {  
    <math-body>  
}
```

A `Physics` section without any region or material specifications is considered the default section.

NOTE Region names can be edited in Sentaurus Structure Editor (see [Sentaurus Structure Editor User Guide, Changing the Name of a Region on page 117](#)).

The `Physics` (and `Math`) sections for different locations are related as follows:

- Physical models defined in the global `Physics` section (that is, in the section without any region or material specifications) are applied in all regions of the device.
- Physical models defined in a material-specific `Physics` section are added to the default models for all regions containing the specified material.
- The same applies to the physical models defined in a region-specific `Physics` section: all regionwise defined models are added to the models defined in the default section.

NOTE If region-specific and material-specific `Physics` sections overlap, the region-specific declaration overrides the material-specific declaration.

For example:

```
Physics {<Default models>}  
Physics (Material="GaAs") {<GaAs models>}  
Physics (Region="Emitter"){<Emitter models>}
```

If the "Emitter" region is made of GaAs, the models in this region are `<Default models>` and `<Emitter models>`, that is, `<GaAs models>` is not added.

For some models, the model specification and numeric values of the parameters are defined in the `Physics` sections. Examples of such models are `Traps` and the `MoleFraction` specifications. For these models, the specifications in region or material `Physics` sections overwrite previously defined values of the corresponding parameters.

If in the default `Physics` section, `xMoleFraction` is defined for a given region and, afterward, is defined for the same region again, in a region or material `Physics` section, the default definition is overwritten. The hierarchy of the parameter specification is the same as discussed previously.

Interface-specific Models

A special set of models can be activated at the interface between two different materials or two different regions. In [Table 182 on page 1259](#), pure interface models are flagged with ‘(i)’ in the description column.

As physical phenomena at an interface are not the same as in the bulk of a device, not all models are allowed inside interface-specific `Physics` sections. For example, it is not possible to define any mobility models or bandgap narrowing at interfaces.

NOTE Although the `Recombination(surfaceSRH)` statement and the `GateCurrent` statement describe pure interface phenomena, they can be defined in a region-specific `Physics` section. In this case, the models are applied to all interfaces between this region and all adjacent insulator regions. If specified in the global `Physics` section, these models are applied to all semiconductor–insulator interfaces.

Interface models are specified in interface-specific `Physics` sections. Their respective parameters are accessible in the parameter file. The syntax for specification of an interface model is:

```
Physics (MaterialInterface="material-name1/material-name2") {  
    <physics-body>  
}
```

or:

```
Physics (RegionInterface="region-name1/region-name2") {  
    <physics-body>  
}
```

The following is an example illustrating the specification of fixed charges at the interface between the materials oxide and aluminum gallium arsenide (AlGaAs):

```
Physics(MaterialInterface="Oxide/AlGaAs") {  
    Traps(Conc=-1.e12 FixedCharge)  
}
```

Electrode-specific Models

Electrode-specific Physics sections can be defined, for example:

```
Physics(Electrode="Gate"){
  Schottky
  eRecVel = <float>
  hRecVel = <float>
  Workfunction = <float>
}
```

Physical Model Parameters

Most physical models depend on parameters that can be adjusted in a file given by `Parameter` in the `File` section:

```
File {
  Parameter = <string>
  ...
}
```

The name of the parameter file conventionally has the extension `.par`.

Model parameters are split into sets, where a particular set corresponds to a particular physical model. The available parameter sets and the individual parameters they contain are described along with the description of the related model (see Part II and Part III).

Parameters can be specified globally, materialwise, regionwise, material interface-wise, region interface-wise, and electrode-wise. The following example shows each of these possibilities:

```
LatticeHeatCapacity {
  cv = 1.1
}
Material = "Silicon" {
  LatticeHeatCapacity {
    cv = 1.63
  }
}
Region = "Oxide" {
  LatticeHeatCapacity {
    cv = 1.67
  }
}
MaterialInterface = "Silicon/Oxide" {
  LatticeHeatCapacity {
```

```

        cv = -1
    }
}
RegionInterface = "Oxide/Bulk" {
    LatticeHeatCapacity {
        cv = -2
    }
}
Electrode = "gate" {
    LatticeHeatCapacity {
        cv = -3
    }
}
}

```

As for the models themselves, not all parameters are valid in all locations, even though it is syntactically possible to specify them. For example, the heat capacity is not used for interfaces and electrodes. Therefore, it does not matter that the values provided in the example above are nonsensical.

To specify parameters for multiple parameter sets for the same location, put all these specifications together into one section for that location, as in the following example for dielectric permittivity and heat capacity:

```

Material = "Silicon" {
    Epsilon{
        epsilon= 11.6
    }
    LatticeHeatCapacity {
        cv = 1.63
    }
}

```

Parameter files support an `Insert` statement to insert other parameter files. Inserted files themselves can use `Insert`. You can change parameters after an `Insert` statement. This is useful to provide standard values through the inserted file and to make the changes to those standards explicit. `Insert` is used as in the following example:

```

Material = "Silicon" { Insert = "Silicon.par" }

```

Sentaurus Device uses the following strategy to search for inserted files. First, the current simulation directory is checked. If the file does not exist, definition of the environment variable `SDEVICEDB` is verified. If `SDEVICEDB` is defined, Sentaurus Device checks the directory associated with the variable. Otherwise, Sentaurus Device checks the default library location `$STROOT/tcad/$STRELEASE/lib/sdevice/MaterialDB`. In all cases, Sentaurus Device prints the path to the actual file and displays an error message if it cannot be found.

NOTE The insert directive is also available for command files (see [Inserting Files on page 156](#)).

Parameters for Composition-dependent Materials

The following parameter sets provide mole fraction dependencies. All models are available for compound semiconductors only, except where otherwise noted:

- Epsilon (also available for compound insulators)
- LatticeHeatCapacity (also available for compound insulators)
- Kappa (lattice thermal conductivity, also available for compound insulators)
- EnergyRelaxationTime
- Bandgap
- Bennett (bandgap narrowing)
- deLaLamo (bandgap narrowing)
- OldSlotboom (bandgap narrowing)
- Slotboom (bandgap narrowing)
- JainRoulston (bandgap narrowing)
- eDOSMass
- hDOSMass
- ConstantMobility
- DopingDependence (mobility model)
- HighFieldDependence (mobility model)
- Enormal (mobility model)
- ToCurrentEnormal (mobility model)
- PhuMob (mobility model)
- StressMobility (hSixBand model parameters)
- vanOverstraetendeMan (impact ionization model)
- MLDAQMModel
- SchroedingerParameters
- Band2BandTunneling
- DirectTunnelling (for semiconductor–insulator interfaces only)
- AbsorptionCoefficient
- QWStrain (see [Syntax for Quantum-Well Strain on page 909](#))
- RefractiveIndex (also available for compound insulators)
- Radiative recombination

- Shockley–Read–Hall recombination
- Auger recombination
- Piezoelectric polarization
- QuantumPotentialParameters
- Deformation potential (elasticity modulus, the parameters for the electron band: x_{is} , db_s , x_{iu} , x_{id} and, for hole band: ad_p , bd_p , dd_p , dso)
- SHEDistribution

Sentaurus Device supports the suppression of the mole fraction dependence of a given model and the use of a fixed (mole fraction–independent) parameter set instead. To suppress mole fraction dependency, specify the (fixed) values for the parameter (for example, $Eg0=1.53$) and omit all other coefficients associated with the interpolation over the mole fraction (for example, $Eg0(1)$, $B(Eg0(1))$, and $C(Eg0(1))$) from this section of the parameter file. It is not necessary to set them to zero individually.

NOTE When specifying a fixed value for one parameter of a given model, all other parameters for the same model must be fixed.

In summary, if the mole fraction dependence of a given model is suppressed, the parameter specification for this model is performed in exactly the same manner as for mole fraction–independent material.

Ternary Semiconductor Composition

To illustrate a calculation of mole fraction–dependent parameter values for ternary materials, consider one mole interval from x_{i-1} to x_i . For mole fraction value (x) of this interval, to compute the parameter value (P), Sentaurus Device uses the expression:

$$\begin{aligned}
 P &= P_{i-1} + A \cdot \Delta x + B_i \cdot \Delta x^2 + C_i \cdot \Delta x^3 \\
 A &= \frac{\Delta P_i}{\Delta x_i} - B_i \cdot \Delta x_i - C_i \cdot \Delta x_i^2 \\
 \Delta P_i &= P_i - P_{i-1} \\
 \Delta x_i &= x_i - x_{i-1} \\
 \Delta x &= x - x_{i-1}
 \end{aligned} \tag{2}$$

where P_i , B_i , C_i , x_i are values defined in the parameter file for each mole fraction interval, $x_0 = 0$, P_0 is the parameter value (at $x = 0$) specified using the same manner as for mole fraction–independent material (for example, $Eg0=1.53$). As in the formulas above, you are not required to specify coefficient A of the polynomial because it is easily recomputed inside Sentaurus Device.

2: Defining Devices

Physical Model Parameters

In the case of undefined parameters (these can be listed by printing the parameter file), Sentaurus Device uses linear interpolation using two parameter values of side materials (for $x = 0$ and $x = 1$):

$$P = (1 - x)P_{x0} + xP_{x1} \quad (3)$$

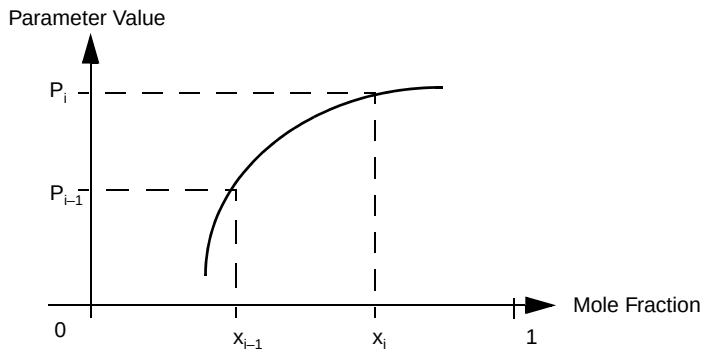


Figure 16 Parameter value as a function of mole fraction

Example 1: Specifying Electric Permittivity

This example provides the specification of dielectric permittivity for $\text{Al}_x\text{Ga}_{1-x}\text{As}$:

```
Epsilon
{ * Ratio of the permittivities of material and vacuum
  epsilon = 13.18      # [1]
  * Mole fraction dependent model.
  * The linear interpolation is used on interval [0,1].
  epsilon(1) = 10.06   # [1]
}
```

A linear interpolation is used for the dielectric permittivity, where `epsilon` specifies the value for the mole fraction $x = 0$, and `epsilon(1)` specifies the value for $x = 1$.

Example 2: Specifying Band Gap

This example provides a specification of the bandgap parameters for $\text{Al}_x\text{Ga}_{1-x}\text{As}$. A polynomial approximation, up to the third degree, describes the mole fraction–dependent band parameters on every mole fraction interval. In the following example, two intervals are used, namely, $[X_{\max}(0), X_{\max}(1)]$ and $[X_{\max}(1), X_{\max}(2)]$. The parameters `Eg0`, `Chi0`, ... correspond to the values for $X = X_{\max}(0)$; while the parameters `Eg0(1)`, `Chi0(1)`, ... correspond to the values for $X = X_{\max}(1)$ and, finally, `Eg0(2)`, `Chi0(2)`, ... correspond to the values for $X = X_{\max}(2)$.

The coefficients A and F of the polynomial:

$$F + A(X - X_{\min}(I)) + B(X - X_{\min}(I))^2 + C(X - X_{\min}(I))^3 \quad (4)$$

are determined from the values at both ends of the intervals, while the coefficients B and C must be specified explicitly. You can introduce additional intervals:

```
Bandgap *temperature dependent*
{ * Eg = Eg0 - alpha T^2 / (beta + T) + alpha Tpar^2 / (beta + Tpar)
  * Eg0 can be overwritten in below bandgap narrowing models,
  * if any of the BGN model is chosen in physics section.
  * Parameter 'Tpar' specifies the value of lattice
  * temperature, at which parameters below are defined.
    Eg0 = 1.42248      # [eV]
    Chi0 = 4.11826     # [eV]
    alpha = 5.4050e-04 # [eV K^-1]
    beta = 2.0400e+02  # [K]
    Tpar = 3.0000e+02  # [K]
  * Mole fraction dependent model.
  * The following interpolation polynomial can be used on interval
  [Xmin(I),Xmax(I)]:
  * F(X) = F(I-1)+A(I)*(X-Xmin(I))+B(I)*(X-Xmin(I))^2+C(I)*(X-Xmin(I))^3,
  * where Xmax(I), F(I), B(I), C(I) are defined below for each interval.
  * A(I) is calculated for a boundary condition F(Xmax(I)) = F(I).
  * Above parameters define values at the following mole fraction:
    Xmax(0) = 0.0000e+00 # [1]
  * Definition of mole fraction intervals, parameters, and coefficients:
    Xmax(1) = 0.45      # [1]
    Eg0(1) = 1.98515    # [eV]
    B(Eg0(1)) = 0.0000e+00 # [eV]
    C(Eg0(1)) = 0.0000e+00 # [eV]
    Chi0(1) = 3.575     # [eV]
    B(Chi0(1)) = 0.0000e+00 # [eV]
    C(Chi0(1)) = 0.0000e+00 # [eV]
    alpha(1) = 4.7727e-04 # [eV K^-1]
    B(alpha(1)) = 0.0000e+00 # [eV K^-1]
    C(alpha(1)) = 0.0000e+00 # [eV K^-1]
    beta(1) = 1.1220e+02 # [K]
    B(beta(1)) = 0.0000e+00 # [K]
    C(beta(1)) = 0.0000e+00 # [K]
    Xmax(2) = 1         # [1]
    Eg0(2) = 2.23       # [eV]
    B(Eg0(2)) = 0.143   # [eV]
    C(Eg0(2)) = 0.0000e+00 # [eV]
    Chi0(2) = 3.5       # [eV]
    B(Chi0(2)) = 0.0000e+00 # [eV]
    C(Chi0(2)) = 0.0000e+00 # [eV]
    alpha(2) = 4.0000e-04 # [eV K^-1]
```

2: Defining Devices

Physical Model Parameters

```

B(alpha(2)) = 0.0000e+00 # [eV K^-1]
C(alpha(2)) = 0.0000e+00 # [eV K^-1]
beta(2) = 0.0000e+00 # [K]
B(beta(2)) = 0.0000e+00 # [K]
C(beta(2)) = 0.0000e+00 # [K]
}

```

Quaternary Semiconductor Composition

Sentaurus Device supports 1:3, 2:2, and 3:1 III–V quaternary alloys. A 1:3 III–V quaternary alloy is given by:

$$AB_xC_yD_z \quad (5)$$

where A is a group III element, and B , C , and D are group V elements (usually listed according to increasing atomic number). Conversely, a 3:1 III–V quaternary alloy can be described as:

$$A_xB_yC_zD \quad (6)$$

where A , B , and C are group III elements, and D is a group V element.

The composition variables x , y , and z are nonnegative, and they are constrained by:

$$x + y + z = 1 \quad (7)$$

An example would be $\text{Al}_x\text{Ga}_y\text{In}_{1-x-y}\text{As}$, where $1 - x - y$ corresponds to z .

Sentaurus Device uses the symmetric interpolation scheme proposed by Williams *et al.* [1] to compute the parameter value $P(A_xB_yC_zD)$ of a 3:1 III–V quaternary alloy as a weighted sum of the corresponding ternary values:

$$P(A_xB_yC_zD) = \frac{xyP(A_{1-u}B_uD) + yzP(B_{1-v}C_vD) + xzP(A_{1-w}C_wD)}{xy + yz + xz} \quad (8)$$

where:

$$u = \frac{1-x+y}{2}, v = \frac{1-y+z}{2}, w = \frac{1-x+z}{2} \quad (9)$$

The parameter values $P(AB_xC_yD_z)$ for 1:3 III–V quaternary alloys are computed similarly. A general 2:2 III–V quaternary alloy is given by:

$$A_xB_{1-x}C_yD_{1-y} \quad (10)$$

where A and B are group III elements, and C and D are group V elements. The composition variables x and y satisfy the inequalities $0 \leq x \leq 1$ and $0 \leq y \leq 1$. As an example, the material $\text{In}_{1-x}\text{Ga}_x\text{As}_y\text{P}_{1-y}$ is mentioned.

The parameters $P(A_xB_{1-x}C_yD_{1-y})$ of a 2:2 III–V quaternary alloy are determined by interpolation between the four ternary side materials:

$$P(A_xB_{1-x}C_yD_{1-y}) = \frac{x(1-x)(yP(A_xB_{1-x}C) + (1-y)P(A_xB_{1-x}D)) + y(1-y)(xP(AC_yD_{1-y}) + (1-x)P(BC_yD_{1-y}))}{x(1-x) + y(1-y)} \quad (11)$$

The interpolation of model parameters for quaternary alloys is also discussed in the literature [2][3][4][5]. A comprehensive survey paper is available [6].

Default Model Parameters for Compound Semiconductors

It is important to understand how the default values for different physical models in different materials are determined. The approach used in Sentaurus Device is summarized here. For example, consider the material `Material`. Assume that no default parameters are defined for this material and a given physical model `Model`.

In this case, use the command `sdevice -P:Material` to see for which models specific default parameters are predefined in the material `Material`:

1. Silicon parameters are used, by default, in the model `Model` if the material `Material` is mole fraction independent.
2. If `Material` is a compound material and dependent on the mole fraction x , the default values of the parameters for the model `Model` are determined by a linear interpolation between the values of the respective parameters of the corresponding ‘pure’ materials (that is, materials corresponding to $x = 0$ and $x = 1$). For example, for $\text{Al}_x\text{Ga}_{1-x}\text{As}$, values of the parameters of GaAs and AlAs are used in the interpolation formula.
3. If `Material` is a quaternary material and dependent on x and y , an interpolation formula, which is based on the values of all corresponding ternary materials, is used. For example, for InGaAsP , the values of four materials (InAsP , GaAsP , GaInP , and InGaAs) are used in the interpolation procedure to obtain the default values of the parameters.

Additional details for each model and specific materials are found in the comments of the parameter file.

Combining Parameter Specifications

Sentaurus Device has built-in values for many parameters, can read parameters from files in a default location, and can read a user-supplied parameter file that can specify parameters for various locations. This section discusses the rules for combining all these specifications.

Parameter handling is affected by the presence of the flag `DefaultParametersFromFile` (specified in the global `Physics` section) and the value of `ParameterInheritance` (specified in the global `Math` section). By default, `ParameterInheritance=None`; by setting `ParameterInheritance=Flatten`, the rules for combining parameter specifications can be altered.

Materialwise Parameters

To determine materialwise parameters, follow these steps:

1. The parameters are initialized from built-in values. These values can depend on the material. For many materials, no appropriate built-in values exist, and silicon values are used instead.
2. If the `DefaultParametersFromFile` flag is present, Sentaurus Device reads the default parameter file for the material. This file can add parameters (for example, add intervals to a mole fraction specification) or overwrite built-in values (for details, see [Default Parameters on page 73](#)).
3. If `ParameterInheritance=Flatten` and specifications outside any location-specific section are present, they can add or overwrite the parameter values obtained through the previous two steps, and they also contribute to a separate, global, default parameter set.
4. If your parameter file contains a section for the material, specifications in this section can add to or overwrite the parameter values obtained through the previous three steps. If `ParameterInheritance=None`, specifications outside any location-specific section are handled as if they occurred in a section for the material silicon.

Materialwise specifications in your parameter file are read even when the material is not contained in the structure to be simulated. This is useful, for example, to provide parameters for corner and side materials for materials that are present in the structure.

Apart from corner and side materials, materialwise parameters are rarely used directly; physical bulk models access parameters regionwise, not materialwise. However, materialwise settings can affect regionwise parameters. The next section explains this in detail.

Regionwise Parameters

When your parameter file does not contain a section for a certain region, the parameter values for this region are the same as for the material of the region. In this case, if `ParameterInheritance=None`, the regionwise parameters are discarded entirely and the material parameters are accessed instead. If `ParameterInheritance=Flatten`, the region parameters are a copy of the material parameters (the subtle distinction of these two cases becomes important only when ramping parameters).

If your parameter file does contain a section for a certain region, the parameters for this region are initialized executing Steps 1 and 2 as for the materialwise parameters, using the material of the region to select the built-in values and the default parameter file. If `ParameterInheritance=Flatten`, and a section for this material is present in your parameter file, Steps 3 and 4 are taken as well. Finally, in any case, the section for the region is evaluated and can add to or overwrite the values previously obtained.

NOTE If `ParameterInheritance=None` and a section for a region is present in your parameter file, none of the settings for any parameter in the section for the material of the region (if such a section exists) has an effect on the parameters for that region. In other words, if `ParameterInheritance=None`, the region parameters do not ‘inherit’ user settings from the material parameters.

Material Interface–wise Parameters

Material interface–wise parameters are handled similarly to materialwise parameters, simply replace the term “material” by “material interface” in the description of the handling of materialwise parameters.

Region Interface–wise Parameters

Region interface–wise parameters are handled similarly to regionwise parameters, and the relation between region interface–wise parameters and material interface–wise parameters is analogous to the relation between regionwise and materialwise parameters. As an additional complication, if `ParameterInheritance=None` and neither a section for the region interface nor for its material interface is present in your parameter file, the region interface parameters are discarded, and the parameters for material silicon are accessed instead.

Electrode-wise Parameters

Electrode-wise parameters are handled similarly to materialwise parameters, simply replace the term “material” by “electrode” in the description of the handling of materialwise parameters.

Generating a Copy of Parameter File

To redefine a parameter value for a particular material, a copy of the default parameter file must be created. To do this, the command `sdevice -P` prints the parameter file for silicon, with insulator properties.

[Table 3](#) lists the principal options for the command `sdevice -P`.

Table 3 Principal options for generating parameter file

Option	Description
-P:All	Prints a copy of the parameter file for all materials. Materials are taken from the file <code>datexcodes.txt</code> .
-P:Material	Prints model parameters for the specified material.
-P:Material:x	Prints model parameters for the specified material for mole fraction x.
-P:Material:x:y	Prints model parameters for the specified material for mole fractions x and y.
-P filename	Prints model parameters for materials and interfaces used in the command file <code>filename</code> .
-r	Reads parameters from default location before printing parameters (analogous to using <code>DefaultParametersFromFile</code> flag, see Materialwise Parameters on page 68).

For regionwise and materialwise parameter specifications, any model and parameter from the default section is usable, even if it is not printed for the particular material.

For mole fraction–dependent parameters, for people, it is difficult to read a parameter file and to obtain the final values of parameters (for example, the band gap) for a particular composition mole fraction. By using the command `sdevice -M <inputfile.cmd>`, Sentaurus Device creates a `models-M.par` file that will contain regionwise parameters with only constant values (instead of the polynomial coefficients) for regions where the composition mole fraction is constant. For regions where the composition is not a constant, Sentaurus Device prints the default material parameters.

As an alternative to the command `sdevice -P`, Sentaurus Device provides the command `sdevice -L`, which supports the same options as described in [Table 3](#) for `sdevice -P`.

However, rather than generating a single file, it creates separate files for each material and each material interface. Typically, these files are edited and used to provide default parameters (see [Default Parameters on page 73](#)).

For example, assume the command file `pp1_des.cmd` is for a simple silicon MOSFET with four electrodes, one silicon region, and one oxide region. By using the command:

```
sdevice -L pp1_des.cmd
```

a parameter file is created in the current directory for each material, material interface, and electrode found in `nmos_mdr.tdr`. In this example, the appropriate files are `Silicon.par`, `Oxide.par`, `Oxide%Silicon.par`, and `Electrode.par`.

In addition, the following `models.par` file is created in the current directory:

```
Region = "Region0" { Insert = "Silicon.par" }  
Region = "Region1" { Insert = "Oxide.par" }  
RegionInterface = "Region1/Region0" { Insert = "Oxide%Silicon.par" }  
Electrode = "gate" { Insert = "Electrode.par" }  
Electrode = "source" { Insert = "Electrode.par" }  
Electrode = "drain" { Insert = "Electrode.par" }  
Electrode = "substrate" { Insert = "Electrode.par" }
```

This `models.par` file can be renamed. It is only used in the simulation if it is specified in the File section of the command file of Sentaurus Device:

```
File {...  
    Parameter = "models.par"  
    ...
```

Undefined Physical Models

For a nonsilicon simulation, the default behavior of Sentaurus Device is to use silicon parameters for models that are not defined in a material used in the simulation. It is useful for noncritical models, but it can lead to confusion, for example, if the semiconductor band gap is not defined, and Sentaurus Device uses that of silicon. Therefore, Sentaurus Device has a list of critical models and stops the simulation, with an error message, if these models are not defined.

NOTE The model is defined in a material if it is present in the default parameter file of Sentaurus Device for the material or it is specified in a user-defined parameter file.

2: Defining Devices

Physical Model Parameters

[Table 4](#) lists the critical models (with names from the parameter file of Sentaurus Device) with materials where these models are checked.

Table 4 List of critical models

Model	Insulator	Semiconductor	Conductor
Auger		x	
Bandgap		x	
ConstantMobility		x	
DopingDependence		x	
eDOSmass hDOSmass		x	
Epsilon	x	x	
Kappa	x	x	x
RadiativeRecombination		x	
RefractiveIndex	x	x	
Scharfetter		x	
SchroedingerParameters	x	x	

The models Bandgap, DOSmass, and Epsilon are checked always. For other models, this check is performed for each region, but only if appropriate models in the Physics section and equations in the Solve section are activated. The thermal conductivity model Kappa is checked only if the lattice temperature equation is included.

Drift-diffusion or hydrodynamic simulations activate the checking mobility models ConstantMobility and DopingDependence (with appropriate models in the Physics section).

NOTE This checking procedure can be switched off by the keyword
-CheckUndefinedModels in the Math section.

The models eDOSmass and hDOSmass also must be defined for insulators to support tunneling (see [Nonlocal Tunneling Parameters on page 615](#)). Only the following specifications are acceptable for insulators:

```
eDOSmass {  
  Formula = 1  
  a = 0  
  ml = 0  
  mm = <effective mass>    # default 0.42  
}
```

```
hDOSmass {
    Formula = 1
    a = 0
    b = 0
    c = 0
    d = 0
    e = 0
    f = 0
    g = 0
    h = 0
    i = 0
    mm = <effective mass>    # default 1
}
```

NOTE In general, the models eDOSmass and hDOSmass only need to be specified for user-specified insulators. The correct values are predefined for standard insulators.

Default Parameters

Sentaurus Device provides built-in default parameters for many models and materials. However, Sentaurus Device also offers the option to overwrite the built-in values with default parameters from files. This option is activated by `DefaultParametersFromFile` in the global Physics section:

```
Physics {
    DefaultParametersFromFile
    ...
}
```

[Table 5](#) lists the file names that are used to initialize default parameters.

Table 5 Filenames for default parameters

Location of parameters	Filename
Materials	<material>.par, for example, Silicon.par, GaAs.par
Material interfaces	<material1>%<material2>.par, for example, InAlAs%AlGaAs.par (Instead of %, a comma or space can also be used.)
Contacts	Contact.par or Electrode.par

Sentaurus Device uses the same search strategy as for inserted files (see [Physical Model Parameters on page 60](#)). Sentaurus Device ships with a selection of parameter files in the directory \$STR00T/tcad/\$STRELEASE/lib/sdevice/MaterialDB.

If a matching file is not found, the built-in default parameters are used unaltered. Check the log file of Sentaurus Device to see which files were actually used.

Named Parameter Sets

Some models in Sentaurus Device support the use of parameter sets that can be named. For example, `EnormalDependence` is the unnamed parameter set used with the Lombardi mobility model. In the parameter file, you can write:

```
EnormalDependence "myset" {  
    B = 3.6100e+07 , 1.5100e+07    # [cm/s]  
    C = 1.7000e+04 , 4.1800e+03    # [cm^(5/3)/(V^(2/3)s)]  
    ...  
}
```

to declare a parameter set for the Lombardi mobility model with the name `myset`.

Typically, named parameter sets are used to store alternative parameterizations for a model. They can be selected from the command file by specifying the name with `ParameterSetName` as an option to a model that supports this feature. For example:

```
Physics {  
    Mobility {  
        PhuMob  
        Enormal( Lombardi (ParameterSetName="myset") )  
        HighFieldSaturation  
    }  
}
```

If `ParameterSetName` is not specified, the unnamed parameter set associated with the model is used by default.

[Table 6 on page 75](#) lists the models that support named parameter sets.

Table 6 Models that support named parameter sets

Model name		Parameter set (unnamed)
eQuantumPotential hQuantumPotential		QuantumPotentialParameters
Mobility eMobility hMobility	Enormal (Lombardi) ToCurrentEnormal (Lombardi)	EnormalDependence
	Enormal(IALMob) ToCurrentEnormal(IALMob)	IALMob
	HighFieldSaturation eHighFieldSaturation hHighFieldSaturation Diffusivity eDiffusivity hDiffusivity	HighFieldDependence HydroHighFieldDependence
Piezo (Mobility)	Tensor eTensor hTensor Factor eFactor hFactor	Piezoresistance

Auto-Orientation Framework

Some models in Sentaurus Device support an auto-orientation framework that automatically switches between different named parameter sets for model evaluation, based on the surface orientation of the nearest interface.

When the `AutoOrientation` option is activated for a model, by default, Sentaurus Device looks for and uses parameter sets named "100", "110", and "111" corresponding to surface orientations of {100}, {110}, and {111}. If the nearest interface orientation is {110}, for example, the model is evaluated using the parameter set "110". For surface orientations that fall between {100}, {110}, and {111}, the named parameter set that most closely corresponds to the actual surface orientation is used.

Models that support the auto-orientation framework include:

- Density-gradient quantization model (eQuantumPotential, hQuantumPotential)
- Lombardi and IALMob mobility models
- Piezoresistance models (Tensor, eTensor, hTensor, Factor, eFactor, hFactor)

In the `Plot` section of the command file, the quantities `InterfaceOrientation` and `NearestInterfaceOrientation` can be specified to see the orientation that is used, at each

interface face vertex and at every vertex, respectively, when auto-orientation is enabled for a model.

Changing Orientations Used with Auto-Orientation

The auto-orientation framework can be modified to use surface orientations other than {100}, {110}, and {111}. This is accomplished by providing a definition for `AutoOrientation` in the `Math` section of the command file:

```
Math {  
    AutoOrientation=(ori1, ori2, ...)  
}
```

In the above specification, the `orii` values are three-digit integers that represent Miller indices for families of equivalent planes (a cubic lattice structure is assumed). For example, to modify `AutoOrientation` to use the surface orientations {100}, {110}, {111}, and {211}, specify:

```
Math {  
    AutoOrientation=(100, 110, 111, 211)  
}
```

In this case, models that support `AutoOrientation` will switch between the parameter sets named "100", "110", "111", and "211", depending on the surface orientation of the nearest interface.

Auto-Orientation Smoothing

By default, the switch from one parameter set to another when using auto-orientation occurs abruptly. To enable a smooth transition between different parameter sets, specify a nonzero value for the auto-orientation smoothing distance in the `Math` section:

```
Math {  
    AutoOrientationSmoothingDistance = 0.005    # [micrometers]  
}
```

The smoothing distance is an approximate measure of the distance over which the transition between different parameter sets will occur. For convenience, specifying `AutoOrientationSmoothingDistance < 0` uses the average interface vertex spacing as the smoothing distance. In many cases, this is a reasonable choice.

When auto-orientation smoothing is used, weights are calculated at each vertex for each orientation-dependent parameter set based on the proximity to the different supported surface orientations:

$$w_i(\text{vertex}) = \sum_{j=1}^{N_i} \left(\frac{A_j}{A_{\text{total}}} \right) \exp\left(-\frac{d_j - d_{\min}}{d_{\text{smooth}}}\right), i = 100, 110, \dots \quad (12)$$

The summation is over all interface vertices associated with orientation i :

- $d_{\min}$ is the minimum distance to the interface.
- d_j is the distance to the interface vertex j .
- d_{smooth} is the smoothing distance.
- A_j is the area associated with the interface vertex j .
- A_{total} is the total interface area.

At each vertex, very small weights are set to zero, and the remaining weights are normalized so that their sum is equal to one.

During model evaluation, the weights for each orientation at a vertex are used to obtain a weighted average of the quantity being calculated.

In the `Plot` section of the command file, the quantity `AutoOrientationSmoothing` can be specified to see the vertices where auto-orientation smoothing is used and the dominant orientation at those vertices.

References

- [1] C. K. Williams *et al.*, “Energy Bandgap and Lattice Constant Contours of III-V Quaternary Alloys of the Form $A_xB_yC_zD$ or $AB_xC_yD_z$,” *Journal of Electronic Materials*, vol. 7, no. 5, pp. 639–646, 1978.
- [2] T. H. Glisson *et al.*, “Energy Bandgap and Lattice Constant Contours of III-V Quaternary Alloys,” *Journal of Electronic Materials*, vol. 7, no. 1, pp. 1–16, 1978.
- [3] M. P. C. M. Krijn, “Heterojunction band offsets and effective masses in III–V quaternary alloys,” *Semiconductor Science and Technology*, vol. 6, no. 1, pp. 27–31, 1991.
- [4] S. Adachi, “Band gaps and refractive indices of AlGaAsSb, GaInAsSb, and InPAsSb: Key properties for a variety of the 2–4-mm optoelectronic device applications,” *Journal of Applied Physics*, vol. 61, no. 10, pp. 4869–4876, 1987.

2: Defining Devices

References

- [5] R. L. Moon, G. A. Antypas, and L. W. James, "Bandgap and Lattice Constant of GaInAsP as a Function of Alloy Composition," *Journal of Electronic Materials*, vol. 3, no. 3, pp. 635–644, 1974.
- [6] I. Vurgaftman, J. R. Meyer, and L. R. Ram-Mohan, "Band parameters for III–V compound semiconductors and their alloys," *Journal of Applied Physics*, vol. 89, no. 11, pp. 5815–5875, 2001.

This chapter describes the specification of circuits and compact devices.

Overview

Sentaurus Device is a single-device simulator, and a mixed-mode device and circuit simulator. A single-device command file is defined through the mesh, contacts, physical models, and solve command specifications.

For a multidevice simulation, the command file must include specifications of the mesh (File section), contacts (Electrode section), and physical models (Physics section) for each device. A circuit netlist must be defined to connect the devices (see [Figure 17](#)), and solve commands must be specified that solve the whole system of devices.

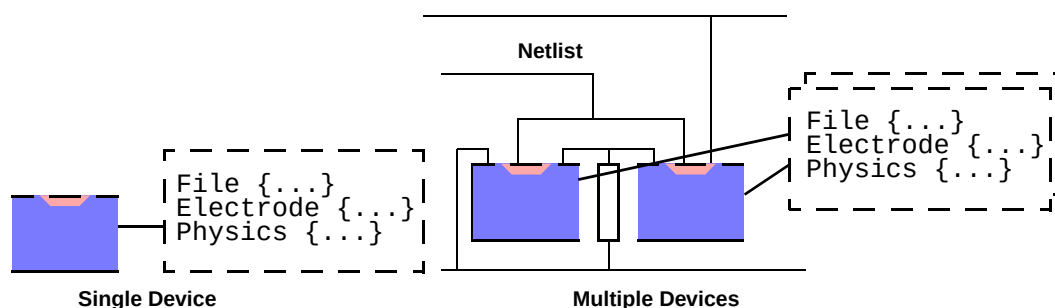


Figure 17 Each device in a multidevice simulation is connected with a circuit netlist

The Device command defines each physical device. A command file can have any number of Device sections. The Device section defines a device, but a System section is required to create and connect devices.

Sentaurus Device also provides a number of compact models for use in mixed-mode simulations.

Compact Models

Sentaurus Device provides four different types of model:

- **SPICE**

These include compact models from Berkeley SPICE 3 Version 3F5. The BSIM3v3.2, BSIM4.1.0, and BSIMPDv2.2.2 MOS models are also available.

- **HSPICE**

Several frequently used HSPICE models are provided.

- **Built-in**

There are several special-purpose models.

- **User-defined**

A compact model interface (CMI) is available for user-defined models. These models are implemented in C++ and linked to Sentaurus Device at run-time. No access to the source code of Sentaurus Device is necessary.

This section describes the incorporation of compact models in mixed-mode simulations. The *Compact Models User Guide* provides references for the different model types.

Hierarchical Description of Compact Models

In Sentaurus Device, the compact models comprise three levels:

- **Device**

This describes the basic properties of a compact model and includes the names of the model, electrodes, thermodes, and internal variables; and the names and types of the internal states and parameters. The devices are predefined for SPICE models and built-in models. For user-defined models, you must specify the devices.

- **Parameter set**

Each parameter set is derived from a device. It defines default values for the parameters of a compact model. Usually, a parameter set defines parameters that are shared among several instances. Most SPICE and built-in models provide a default parameter set, which can be directly referenced in a circuit description. For more complicated models, such as MOSFETs, you can introduce new parameter sets.

- **Instance**

Instances correspond to the elements in the Sentaurus Device circuit. Each instance is derived from a parameter set. If necessary, an instance can override the values of its parameters.

For SPICE and built-in models, you define parameter sets and instances. For user-defined models, it is possible (and required) to introduce new devices. This is described in the *Compact Models User Guide*.

The parameter sets and instances in a circuit simulation are specified in external SPICE circuit files (see [SPICE Circuit Files on page 85](#)). These files are recognized by the extension .scf and are parsed by Sentaurus Device at the beginning of a simulation.

[Table 7](#) lists the built-in models and [Table 8](#) presents an overview of the SPICE models.

Table 7 Built-in models in Sentaurus Device

Model	Device	Default parameter set
Electrothermal resistor	Ter	Ter_pset
Parameter interface	Param_Interface_Device	Param_Interface
SPICE temperature interface	Spice_Temperature_Interface_Device	Spice_Temperature_Interface

Table 8 SPICE models in Sentaurus Device

Model	Device	Default parameter set
Resistor	Resistor	Resistor_pset
Capacitor	Capacitor	Capacitor_pset
Inductor	Inductor	Inductor_pset
Coupled inductors	mutual	mutual_pset
Voltage-controlled switch	Switch	Switch_pset
Current-controlled switch	CSwitch	CSwitch_pset
Voltage source	Vsource	Vsource_pset
Current source	Isource	Isource_pset
Voltage-controlled current source	VCCS	VCCS_pset
Voltage-controlled voltage source	VCVS	VCVS_pset
Current-controlled current source	CCCS	CCCS_pset
Current-controlled voltage source	CCVS	CCVS_pset
Junction diode	Diode	Diode_pset
Bipolar junction transistor	BJT	BJT_pset
Junction field effect transistor	JFET	JFET_pset

3: Mixed-Mode Sentaurus Device

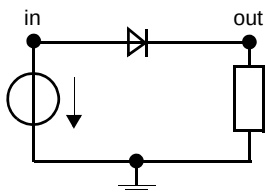
Overview

Table 8 SPICE models in Sentaurus Device

Model	Device	Default parameter set
MOSFET	Mos1	Mos1_pset
	Mos2	Mos2_pset
	Mos3	Mos3_pset
	Mos6	Mos6_pset
	BSIM1	BSIM1_pset
	BSIM2	BSIM2_pset
	BSIM3	BSIM3_pset
	BSIM4	BSIM4_pset
	B3SOI	B3SOI_pset
GaAs MESFET	MES	MES_pset
HSPICE Level 1	HMOS_L1	
HSPICE Level 2	HMOS_L2	
HSPICE Level 3	HMOS_L3	
HSPICE Level 28	HMOS_L28	
HSPICE Level 49	HMOS_L49	
HSPICE Level 53	HMOS_L53	
HSPICE Level 54	HMOS_L54	
HSPICE Level 57	HMOS_L57	
HSPICE Level 59	HMOS_L59	
HSPICE Level 61	HMOS_L61	
HSPICE Level 62	HMOS_L62	
HSPICE Level 64	HMOS_L64	
HSPICE Level 68	HMOS_L68	
HSPICE Level 69	HMOS_L69	
HSPICE Level 73	HMOS_L73	

Example: Compact Models

Consider the following simple rectifier circuit:



The circuit comprises three compact models and can be defined in the file `rectifier.scf` as:

```
PSET D1n4148
  DEVICE Diode
  PARAMETERS
    is = 0.1p // saturation current
    rs = 16   // Ohmic resistance
    cjo = 2p  // junction capacitance
    tt = 12n  // transit time
    bv = 100  // reverse breakdown voltage
    ibv = 0.1p // current at reverse breakdown voltage
END PSET

INSTANCE v
  PSET Vsource_pset
  ELECTRODES in 0
  PARAMETERS sine = [0 5 1meg 0 0]
END INSTANCE

INSTANCE d1
  PSET D1n4148
  ELECTRODES in out
  PARAMETERS temp = 30
END INSTANCE

INSTANCE r
  PSET Resistor_pset
  ELECTRODES out 0
  PARAMETERS resistance = 1000
END INSTANCE
```

The parameter set `D1n4148` defines the parameters shared by all diodes of type `1n4148`. Instance parameters are usually different for each diode, for example, their operating temperature.

3: Mixed-Mode Sentaurus Device

Overview

NOTE A parameter set must be declared before it can be referenced by an instance.

The *Compact Models User Guide* further explains the SPICE parameters in this example. The command file for this simulation can be:

```
File {
    SPICEPath = ". lib spice/lib"
}
System {
    Plot "rectifier" (time() v(in) v(out) i(r 0))
}
Solve {
    Circuit
    NewCurrentPrefix = "transient_"
    Transient (InitialTime = 0 FinalTime = 0.2e-5
               InitialStep = 1e-7 MaxStep = 1e-7) {Circuit}
}
```

The `SPICEPath` in the `File` section is assigned a list of directories. All directories are scanned for `.scf` files (SPICE circuit files).

Check the log file of Sentaurus Device to see which circuit files were found and used in the simulation.

The `System` section contains a `Plot` statement that produces the plot file `rectifier_des.plt`. The simulation time, the voltages of the nodes `in` and `out`, and the current from the resistor `r` into the node `0` are plotted. The `Solve` section describes the simulation. The keyword `Circuit` denotes the circuit equations to be solved.

The instances in a circuit can also appear directly in the `System` section of the command file, for example:

```
System {
    Vsource_pset v (in 0) {sine = (0 5 1meg 0 0)}
    D1n4148 d1 (in out) {temp = 30}
    Resistor_pset r (out 0) {resistance = 1000}

    Plot "rectifier" (time() v(in) v(out) i(r 0))
}
```

SPICE Circuit Files

Compact models can be specified in an external SPICE circuit file, which is recognized by the extension .Scf. The declaration of a parameter set can be:

```
PSET pset
  DEVICE dev
  PARAMETERS
    parameter0 = value0
    parameter1 = value1
    ...
END PSET
```

This declaration introduces the parameter set `pset` that is derived from the device `dev`. It assigns default values for the given parameters. The device `dev` should have already declared the parameter names. Furthermore, the values assigned to the parameters must be of the appropriate type.

[Table 9](#) lists the possible parameter types.

Table 9 Parameters in SPICE circuit files

Parameter type	Example	Parameter type	Example
char	c = 'n'	char[]	cc = ['a' 'b' 'c']
int	i = 7	int[]	ii = [1 2 3]
double	d = 3.14	double[]	dd = [1.2 -3.4 5e6]
string	s = "hello world"	string[]	ss = ["hello" "world"]

Similarly, the circuit instances can be declared as:

```
INSTANCE inst
  PSET pset
  ELECTRODES e0 e1 ...
  THERMODES t0 t1 ...
  PARAMETERS
    parameter0 = value0
    parameter1 = value1
    ...
END INSTANCE
```

According to this declaration, the instance `inst` is derived from the parameter set `pset`. Its electrodes are connected to the circuit nodes `e0`, `e1`, ..., and its thermodes are connected to `t0`, `t1`, ...

3: Mixed-Mode Sentaurus Device

Device Section

This instance also defines or overrides parameter values. The complete syntax of the input language for SPICE circuit files is given in [Compact Models User Guide, Syntax of Compact Circuit Files on page 140](#).

NOTE The tool `spice2sdevice` is available to convert Berkeley SPICE and HSPICE circuit files (extension `.cir`) to Sentaurus Device circuit files (extension `.scf`) (see [Utilities User Guide, Chapter 4 on page 23](#)).

Device Section

The Device sections of the command file define the different device types used in the system to be simulated. Each device type must have an identifier name that follows the keyword `Device`. Each Device section can include the `Electrode`, `Thermode`, `File`, `Plot`, `Physics`, and `Math` sections. For example:

```
Device "resist" {
  Electrode {
    ...
  }
  File {
    ...
  }
  Physics {
    ...
  }
  ...
}
```

If information is not specified in the Device section, information from the lowest level of the command file is taken (if defined there), for example:

```
# Default Physics section
Physics{
  ...
}
Device resist {
  # This device contains no Physics section
  # so it uses the default set above
  Electrode{
    ...
  }
  File{
    ...
  }
}
```

System Section

The System section defines the netlist of physical devices and circuit elements to be solved. The netlist is connected through circuit nodes. By default, a circuit node is electrical, but it can be declared to be electrical or thermal:

```
Electrical { enode0 enode1 ... }
Thermal { tnode0 tnode1 ... }
```

Each electrical node is associated with a voltage variable, and each thermal node is associated with a temperature variable. Node names are numeric or alphanumeric. The node 0 is predefined as the ground node (0 V).

Compact models can be defined in SPICE circuit files (see [SPICE Circuit Files on page 85](#)). However, instances of compact models can also appear directly in the System section of the command file:

```
parameter-set-name instance-name (node0 node1 ...) {
    <attributes>
}
```

The order of the nodes in the connectivity list corresponds to the electrodes and thermodes in the SPICE device definition (refer to the *Compact Models User Guide*).

The connectivity list is a list of contact-name=node-name connections, separated by white space. Contact-name is the name of the contact from the grid file of the given device, and node-name is the name of the circuit netlist node as previously defined in the definition of a circuit element.

Physical devices are defined as:

```
device-type instance-name (connectivity list) {<attributes>}
```

The connectivity list of a physical device explicitly establishes the connection between a contact and node. For example, the following defines a physical diode and circuit diode:

```
Diode_pset circuit_diode (1 2) {...}          # circuit diode
Diode243    device_diode (anode=1 cathode=2) {...}  # physical diode
```

Both diodes have their anode connected to node 1 and their cathode connected to node 2. In the case of the circuit diode, the device specification defines the order of the electrodes (see [Compact Models User Guide, Syntax of Compact Circuit Files on page 140](#)). Conversely, the connectivity for the physical diode must be given explicitly. The names anode and cathode of the contacts are defined in the grid file of the device. The device types of the physical devices must be defined elsewhere in the command file (with Device sections) or an external .device file.

The `System` section can contain the initial conditions of the nodes. Three types of initialization can be specified:

- Fixed values (`Set`)
- Fixed until transient (`Initialize`)
- Initialized only for the first solve (`Hint`)

In addition, the `Unset` command is available to free a node after a `Set` command. The fixed value set by a `Set` command remains fixed during all subsequent simulations until a `Set` or an `Unset` command is used in the `Solve` section or the node itself becomes a `Goal` of a `Quasistationary`.

The `System` section can also contain `Plot` statements to generate plot files of node values, currents through devices, and parameters of internal circuit elements. For a simple case of one device without circuit connections (besides resistive contacts), the keyword `System` is not required because the system is implicit and trivial given the information from the `Electrode`, `Thermode`, and `File` sections at the base level.

Physical Devices

A physical device is instantiated using a previously defined device type, name, connectivity list, and optional parameters, for example:

```
device_type instance-name (<connectivity list>) {  
    <optional device parameters>  
}
```

If no extra device parameters are required, the device is specified without braces, for example:

```
device-type instance-name (<connectivity list>)
```

The device parameters overload the parameters defined in the device type. As for a device type of `Sentaurus Device`, the parameters can include `Electrode`, `Thermode`, `File`, `Plot`, `Physics`, and `Math` sections.

NOTE Electrodes have `Voltage` statements that set the voltage of each electrode. If the electrodes are connected to nodes through the connectivity list, these values are only hints (as defined by the keyword `Hint`) for the first Newton solve, but do not set the electrodes to those values as with the keyword `Set`. By default, an electrode that is connected to a node is floating.

Electrodes must be connected to electrical nodes and thermodes to thermal nodes. This enables electrodes and thermodes to share the same contact name or number.

Circuit Devices

SPICE instances can be declared in SPICE circuit files as discussed in [SPICE Circuit Files on page 85](#). They can also appear directly in the `System` section of the command file, for example:

```
pset inst (e0 e1 ... t0 t1 ...) {  
    parameter0 = value0  
    parameter1 = value1  
    ...  
}
```

This declaration in the command file provides the same information as the equivalent declaration in the SPICE circuit file.

Array parameters must be specified with parentheses, not braces, for example:

```
dd = (1.2 -3.4 5e6)  
ss = ("hello" "world")
```

Certain SPICE models have internal nodes that are accessible through the form `instance_name.internal_node`. For example, a SPICE inductance creates an internal node `branch`, which represents the current through the instance. Therefore, the expression `v(l.branch)` can be used to gain access to the current through the inductor `l`. This is useful for plotting internal data or initializing currents through inductors (refer to the *Compact Models User Guide* for a list of the internal nodes for each model).

Electrical and Thermal Netlist

Sentaurus Device allows both electrical and thermal netlists to coexist in the same system, for example:

```
System {
  Thermal (ta tc t300)
  Set (t300 = 300)
  Hint (ta = 300 tc = 300)

  Isource_pset i (a 0) {dc = 0}
  PRES pres ("Anode"=a "Cathode"=0 "Anode"=ta "Cathode"=tc)
  Resistor_pset ra (ta t300) {resistance = 1}
  Resistor_pset rc (tc t300) {resistance = 1}

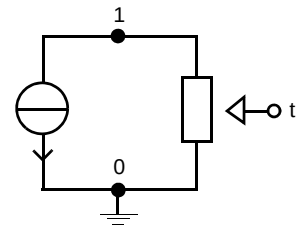
  Plot "pres" (v(a) t(ta) t(tc) h(pres ta) h(pres tc) i(pres 0))
}
```

The current source *i* drives a resistive physical device *pres*. This device has two contacts Anode and Cathode that are connected to the circuit nodes *a* and 0, and are also used as thermodes, which are connected to the heat sink *t300* through two thermal resistors *ra* and *rc*.

The **Plot** statement accesses the voltage of the node *a*, the temperature of the nodes *ta* and *tc*, the heat flux from *pres* into *ta* and *tc*, and the current from *pres* into the ground node 0.

Many SPICE models provide a temperature parameter for electrothermal simulations. In Sentaurus Device, a temperature parameter is connected to the thermal circuit by a parameter interface.

For example, in this simple circuit, the resistor *r* is a SPICE semiconductor resistor whose resistance depends on the value of the temperature parameter *temp*:



$$r_0 = r_{sh} \cdot \frac{l - narrow}{w - narrow} \quad (13)$$

$$r(temp) = r_0 \cdot (1 + tc_1 \cdot (temp - tnom) + tc_2 \cdot (temp - tnom)^2)$$

To feed the value of the thermal node *t* as a parameter into the resistor *r*, a parameter interface is required:

```
System {
  Thermal (t)
  Set (t = 300)

  Isource_pset i (1 0) {dc = 1}
```



```
cres_pset r (1 0) {temp = 27 l = 0.01 w = 0.001}
Param_Interface rt (t) {parameter = "r.temp" offset = -273.15}

Plot "cres" (t(t) p(r temp) i(r 0) v(1))
}
```

The parameter set `cres_pset` for the resistor `r` is defined in an external SPICE circuit file:

```
PSET cres_pset
  DEVICE Resistor
  PARAMETERS
    rsh = 1
    narrow = 0
    tc1 = 0.01
    tnom = 27
END PSET
```

The parameter interface `rt` updates the value of `temp` in `r` when the variable `t` is changed. This is the general mechanism in Sentaurus Device, which allows a circuit node to connect to a model parameter.

NOTE It is important to declare the parameter interface after the instance to which it refers. Otherwise, Sentaurus Device cannot establish the connection between the parameter interface and the instance.

Temperatures in Sentaurus Device are defined in kelvin. SPICE temperatures are measured in degree Celsius. Therefore, an offset of -273.15 must be used to convert kelvin to degree Celsius.

The parameter `Spice_Temperature` in the Math section is used to initialize the global SPICE circuit temperature at the beginning of a simulation. It cannot be used to change the SPICE temperature later. To modify the SPICE temperature during a simulation, a so-called SPICE temperature interface must be used.

A SPICE temperature interface has one contact that can be connected to an electrode or a thermode. When the value u of the electrode or thermode changes, the global SPICE temperature is updated according to:

$$\text{Spice temperature} = \text{offset} + c_1 u + c_2 u^2 + c_3 u^3 \quad (14)$$

By default, $\text{offset} = c_2 = c_3 = 0$ and $c_1 = 1$. Therefore, the SPICE temperature interface ensures that the global SPICE temperature is identical to the value u .

In the following example, a SPICE temperature interface is used to ramp the global SPICE temperature from 300 K to 400 K:

```
System {  
    Set (st = 300)  
    Spice_Temperature_Interface ti (st) { }  
}  
Solve {  
    Quasistationary (Goal {Node = st Value = 400} DoZero  
        InitialStep = 0.1 MaxStep = 0.1) {  
        Coupled {Circuit}  
    }  
}
```

Set, Unset, Initialize, and Hint

The keywords `Set`, `Initialize`, and `Hint` are used to set nodes to a specific value.

`Set` establishes the node value. This value remains fixed during all subsequent simulations until a `Set` or an `Unset` command is used in the `Solve` section (see [Mixed-Mode Electrical Boundary Conditions on page 101](#)), or the node becomes a `Goal` of a `Quasistationary`, which controls the node itself (see [Quasistationary in Mixed Mode on page 111](#)). For a set node, the corresponding equation (that is, the current balance equation for electrical nodes and the heat balance equation for thermal nodes) is not solved, unlike an unset node.

NOTE The `Set` and `Unset` commands exist in the `Solve` section. These act like the System-level `Set` but allow more flexibility (see [System Section on page 87](#)).

The `Set` command can also be used to change the value of a parameter in a compact model. For example, the resistance of the resistor `r1` changes to 1000 Ω :

```
Set (r1."resistance" = 1000)
```

`Initialize` is similar to the `Set` statement except that node values are kept only during nontransient simulations. When a transient simulation starts, the node is released with its actual value, that is, the node is unset.

NOTE The keyword `Initialize` can be used with an internal node to set a current through an inductor before a transient simulation. For example, the keyword:

```
Inductor_pset L2 (a b){ inductance = 1e-5 }  
Initialize (L2.branch = 1.0e-4)
```

Hint provides a guess value for an unset node for the first Newton step only. The numeric value is given in volt, ampere, or kelvin. A current value only makes sense for internal current nodes in circuit devices. The commands are used as follows:

```
Set ( <node> = <float> <node> = <float> ... )
Unset ( <node> <node> ... )
Initialize ( <node> = <float> <node> = <float> ... )
Hint ( <node> = <float> <node> = <float> ... )
```

For example:

```
Set ( anode = 5 )
```

System Plot

The **System** section can contain any number of **Plot** statements to print voltages at nodes, currents through devices, or circuit element parameters. The output is sent to a given file name. If no file name is provided, the output is sent to the standard output file and the log file. The **Plot** statement is:

```
Plot (<plot command list>)
Plot <filename> (<plot command list>)
```

where <plot command list> is a list of nodes or plot commands as defined in [Table 166 on page 1240](#).

NOTE Plot commands are case sensitive.

Two examples are:

```
Plot (a b i(r1 a) p(r1 rT) p(v0 "sine[0]"))
Plot "plotfile" (time() v(a b) i(d1 a))
```

NOTE The **Plot** command does not print the time by default. When plotting a transient simulation, the **time()** command must be added.

AC System Plot

An **ACPlot** statement in the **System** section can be used to modify the output in the Sentaurus Device AC plot file:

```
System {
    ACPlot (<plot command list>)
}
```

3: Mixed-Mode Sentaurus Device

File Section

The `<plot command list>` is the same as the system plot command discussed in [System Plot on page 93](#).

If an `ACPlot` statement is present, the given quantities are plotted in the Sentaurus Device AC plot file with the results from the AC analysis. Otherwise, only the voltages at the AC nodes are plotted with the results from the AC analysis.

File Section

In the File section, one of the following can be specified:

- Output file name and the small-signal AC extraction file name (keywords `Output` and `ACExtract`)
- Sentaurus Device directory path (keyword `DevicePath`)
- Search path for SPICE models and compact models (keywords `SPICEPath` and `CMIPath`)
- Default file names for the devices

The syntax for these keywords is shown in [Table 146 on page 1221](#).

The variables `Output` and `ACExtract` can be assigned a file name. Only a file name without an extension is required. Sentaurus Device automatically appends a predefined extension:

```
File {  
    Output = "mct"  
    ACExtract = "mct"  
}
```

The variables `DevicePath`, `SPICEPath`, and `CMIPath` represent search paths. They must be assigned a list of directories for which Sentaurus Device searches, for example:

```
File {  
    DevicePath = "../devices:/usr/local/tcad/devices:."  
    SPICEPath = ". lib lib/spice"  
    CMIPath = ". libcmi"  
}
```

Device files (extension `.device`) in `DevicePath` are loaded. The devices found can then be used in the System section. They are not overwritten by a definition with the same name in the command file.

SPICE circuit files (extension `.scf`) in `SPICEPath` are parsed and added to the System section of the command file.

The compact circuit files (extension .ccf) and the corresponding shared object files (extension .so.arch) in CMIPath are parsed and are added to the System section of the command file.

Device-specific keywords are defined under the file entry in the Device section.

SPICE Circuit Models

Sentaurus Device supports SPICE circuit models for mixed-mode simulations. These models are based on Berkeley SPICE 3 Version 3F5. Several frequently used HSPICE models are also available. A detailed description of the SPICE models can be found in the *Compact Models User Guide*.

User-Defined Circuit Models

Sentaurus Device provides a compact model interface (CMI) for user-defined circuit models. The models are implemented in C++ by you and linked to Sentaurus Device at run-time. Access to the source code of Sentaurus Device is not required.

To implement a new user-defined model:

1. Provide a corresponding equation for each variable in the compact model. For electrode voltages, compute the current flowing from the device into the electrode. For an internal model variable, use a model equation.
2. Write a formal description of the new compact model. This compact circuit file is read by Sentaurus Device before the model is loaded.
3. Implement a set of interface subroutines C++. Sentaurus Device provides a run-time environment.
4. Compile the model code into a shared object file, which is linked at run-time to Sentaurus Device. A cmi script executes this compilation.
5. Use the variable CMIPath in the File section of the command file to define a search path.
6. Reference user-defined compact models in compact circuit files (with the extension .ccf) or directly in the System section of the command file.

Mixed-Mode Math Section

The following SPICE circuit parameters can be specified in the global Math section:

```
Spice_Temperature = ...  
Spice_gmin = ...
```

The value of `Spice_Temperature` denotes the global SPICE circuit temperature. Its default value is 300.15 K. The parameter `Spice_gmin` refers to the minimum conductance in SPICE. The default value is $10^{-12} \Omega^{-1}$.

Using Mixed-Mode Simulation

In Sentaurus Device, mixed-mode simulations are handled as a direct extension of single device simulations.

From Single-Device File to Multidevice File

The command file of Sentaurus Device accepts both single-device and multidevice problems. Although the two forms of input look different, they fit in the same input syntax pattern. This is possible because the command file has multiple levels of definitions and there is a built-in default mechanism for the System section.

```
File {...}  
Electrode {...}  
...  
Device <type> {  
  File {...}  
  Electrode {...}  
  ...  
}  
System {  
  <type> <name> {  
    File {...}  
    Electrode {...}  
  }  
}
```

Global

Device

Instance

Figure 18 Three levels of device definition

The complete input syntax allows for three levels of device definition: global, device, and instance (see [Figure 18 on page 96](#)). The three levels are linked in that the global level is the default for the device level and instance level.

By default, if no Device section exists, a single device is created with the content of a global device. If no System section exists, one is created with this single device. In this way, single devices are converted to multidevice problems with a single device and no circuit.

NOTE The device that is created by default has the name " " (that is, an empty string).

This translation process can be performed manually by creating a Device and System section with a single entry (see [Figure 19](#)).

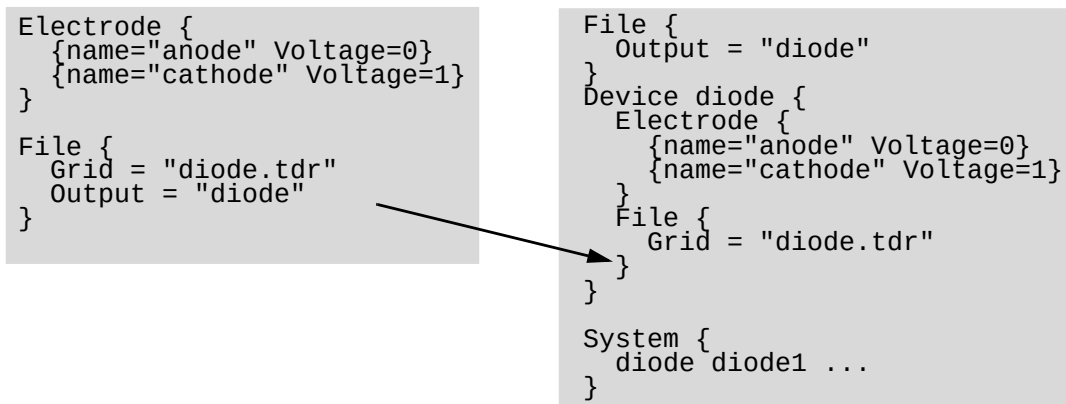


Figure 19 Translating a single device syntax to mixed-mode form

The Solve section can also be converted if the flag NoAutomaticCircuitContact is used (see [Figure 20](#)).

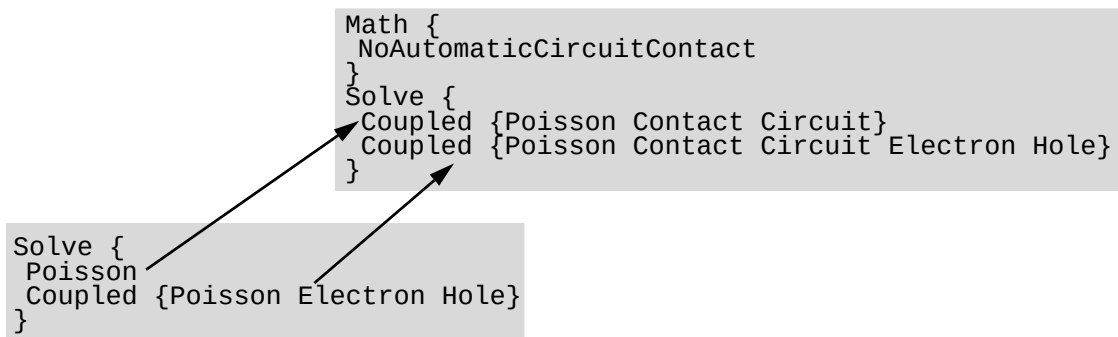


Figure 20 Translating a default solve syntax to a NoAutomaticCircuitContact form

In this case, all occurrences of the keyword Poisson must be expanded to the three keywords Poisson Contact Circuit.

File-naming Convention: Mixed-Mode Extension

A File section can be defined at all levels of a command file. Therefore, a default file name can be potentially included by more than one device, for example:

```
Device res {  
    File { Save="res" ... }  
    ...  
}  
System {  
    res r1  
    res r2  
}
```

Both r1 and r2 use the save device parameters set up in the definition of res. Therefore, they have in principle the same default for Save (that is, res). Since it is impractical to save both devices under the same name, the names of the device instances (that is, r1 and r2) are concatenated to the file name to produce the files res.r1 and res.r2. This process of file name extension is performed for the file parameters Save, Current, Path, and Plot, and is also performed when the file name is copied from the global default to a device type declaration.

In practice, three possibilities exist:

- The file name is defined at the instance level, in which case, it is unchanged.
- The file name is defined at the device type level, in which case, the instance name is concatenated to the original file name.
- The file name is defined at the global command file level, in which case, the device name and instance name are concatenated to the original file name.

[Table 10](#) summarizes these possibilities.

Table 10 File modification for Save, Current, Path, and Plot commands

Level	File name format
Instance	<given name>
Device	<given name>.<instance name>
Global	<given name>.<device type>.<instance name>

This chapter describes how to perform numeric experiments.

Performing a simulation is the virtual analog to performing an experiment in the real world. This chapter describes how to specify the parameters that constitute such an *experiment*, in particular, bias conditions and their variation. It also describes how to perform important types of experiment that involve periodic signals, such as small-signal analysis. Some experiments described here have no real-world analogy, for example, continuous change of physical parameters and device-internal quantities.

Specifying Electrical Boundary Conditions

Electrical boundary conditions are specified in the `Electrode` section. Only one `Electrode` section must be defined for each device. Each electrode is defined in a subsection enclosed by braces and must include a name and default voltage. For example, a complete `Electrode` section is:

```
Electrode {
  { name = "source" Voltage = 1.0 Current = 1e-3}
  { name = "drain" Voltage = 0.0 }
  { name = "gate" Voltage = 0.0 Material = "PolySi"(P=6.0e19) }
}
```

[Table 167 on page 1240](#) lists all keywords that can be specified for each electrode. Keywords that relate to the physical properties of electrodes are discussed in [Electrical Boundary Conditions on page 217](#).

You can specify time-dependent boundary conditions, by providing a list of voltage–time pairs. For example:

```
{ name = "source" voltage = (5 at 0, 10 at 1e-6) }
```

specifies the voltage at source to be 5 V at 0 s, increasing linearly to 10 V at 1 μ s. In addition, you can specify a separate ‘static’ voltage, for example:

```
{ name = "source" voltage = 0 voltage = (5 at 0, 10 at 1e-6) }
```

This combination is useful where an initial quasistationary command ramps source from 0 V to 5 V, before source increases to 10 V during a subsequent transient analysis.

The parameters `Charge` and `Current` determine the boundary condition type as well as the value for an electrode. By default, electrodes have a voltage boundary condition type. The keyword `Current` changes the boundary to current type, and the keyword `Charge` changes it to charge type. Note that for current boundary conditions, you still must specify `Voltage`; this value is used as an initial guess when the simulation begins, but it has no impact on the final results. `Charge` and `Current` support time-dependent specifications in the same manner as explained for `Voltage`.

Changing Boundary Condition Type During Simulation

The `Set` command in the `Solve` section changes the boundary condition type for an electrode. To change the boundary condition of a current contact `<name>` to voltage type, use `Set(<name> mode voltage)`. To change the boundary condition of a voltage contact `<name>` to current type or charge type, use `Set(<name> mode current)` or `Set(<name> mode charge)`, respectively. To change the boundary condition of a charge contact `<name>` to voltage type, use `Set(<name> mode voltage)`. Changing the boundary condition of a current contact `<name>` directly to charge type or from a charge contact `<name>` directly to current type is not allowed. For example, use:

```
Solve { ...  
    Set ("drain" mode current)  
    ...  
}
```

to change the boundary condition for the voltage contact `drain` from voltage type to current type. If the boundary condition for `drain` was of current type before the `Set` command is executed, nothing happens.

The `Set` command does not change the value of the voltages and currents at the electrodes. The boundary value for the new boundary condition type results from the solution previously obtained for the bias point at which the `Set` command appears.

An alternative method to change the boundary condition type of a contact is to use the `Quasistationary` statement (see [Quasistationary Ramps on page 106](#)). This alternative is usually more convenient. However, a goal value for the boundary condition must be specified, whereas the `Set` command allows you to fix the current, charge, or voltage at a contact to a value reached during the simulation, even if this value is not known beforehand.

Mixed-Mode Electrical Boundary Conditions

In mixed-mode simulations, an additional possibility to specify electrical boundary conditions is to connect electrodes to a circuit (see [Electrical and Thermal Netlist on page 90](#)).

The `Set` command in the `Solve` section can be used to determine the boundary conditions at circuit nodes (see [Set, Unset, Initialize, and Hint on page 92](#)). The `Set` command takes a list of nodes with optional values as parameters. If a value is given, the node is set to that value. If no value is given, the node is set to its current value. The nodes are set until the end of the Senterus Device run or the next `Unset` command with the specified node.

The `Unset` command takes a list of nodes and ‘frees’ them (that is, the nodes are floating). In practice, the `Set` command is used in the `Solve` section to establish a complex system of steps, circuit region by circuit region.

The SPICE voltage and current sources use vector parameters to define the properties of various source types. For example:

```
System {
  Vsource_pset v0 (n1 n0) { sine = (0.5 1 10 0 0) }
}
```

defines a voltage source with an offset $v_0 = \text{sine}[0] = 0.5\text{ V}$, an amplitude $v_a = \text{sine}[1] = 1\text{ V}$, a frequency $\text{freq} = \text{sine}[2] = 10\text{ Hz}$, a delay $t_d = \text{sine}[3] = 0\text{ s}$, and a damping factor $\theta = \text{sine}[4] = 0\text{ s}^{-1}$. Components of such vectors can be set in the `Solve` section. The following example sets the offset of the sine voltage source `v0` to 0.75 V :

```
Solve { ...
  Set (v0."sine[0]" = 0.75) ...
}
```

Specifying Thermal Boundary Conditions

The `Thermode` section defines the thermal contacts of a device. The `Thermode` section is defined in the same way as the `Electrode` section. By default, the temperature specified with `Temperature` is the temperature of the thermode. If, in addition, `Power` (in W/cm^2) is specified in the `Thermode` section, a heat flux boundary condition is imposed instead, and the specified temperature serves as an initial guess only, for example:

```
Thermode {
  { Name = "top"    Temperature = 350 }
  { Name = "bottom" Temperature = 300 Power = 1e6 }
}
```

4: Performing Numeric Experiments

Break Criteria: Conditionally Stopping the Simulation

Time-dependent thermal boundary conditions are specified using the same syntax as for electrical boundary conditions (see [Specifying Electrical Boundary Conditions on page 99](#)).

[Table 169 on page 1242](#) lists the keywords available for the Thermode section. [Thermal Boundary Conditions on page 228](#) discusses the physical options. In mixed-mode device simulation, an additional possibility to specify thermal boundary conditions is to connect thermodes to a thermal circuit (see [Electrical and Thermal Netlist on page 90](#)).

Table 11 Various thermode declarations

Command statement	Description
{ Name = "surface" Temperature = 310 SurfaceResistance = 0.1 }	This is a thermal resistive boundary condition with $0.1\text{cm}^2\text{K/W}$ thermal resistance, which is specified at the thermode 'surface.'
{ Name = "body" Temperature = 300 Power = 1e6 }	Heat flux boundary condition.
{ Name = "bulk" Temperature = 300 Power = 1e5 Power = (1e5 at 0, 1e6 at 1e-4, 1e3 at 2e-4) }	Thermode with time-dependent heat flux boundary condition.

Break Criteria: Conditionally Stopping the Simulation

Sentaurus Device prematurely terminates a simulation if certain values exceed a given limit. This feature is useful during a nonisothermal simulation to stop the calculations when the silicon starts to melt or to stop a breakdown simulation when the current through a contact exceeds a predefined value.

The following values can be monitored during a simulation:

- Contact voltage (inner voltage)
- Contact current
- Lattice temperature
- Current density
- Electric field (absolute value of field)
- Device power

It is possible to specify values to a lower bound and an upper bound. Similarly, a bound can be specified on the absolute value. The break criteria can have a global or sweep specification. If the break is in sweep, you can have multiple break criteria in one simulation. The break can be specified in a single device and in mixed mode.

Global Contact Break Criteria

The limits for contact voltages and contact currents can be specified in the global `Math` section:

```
Math {  
    BreakCriteria {  
        Voltage (Contact = "drain" absval = 10)  
        Current (Contact = "source" minval = -0.001 maxval = 0.002)  
    }  
    ...  
}
```

In this example, the stopping criterion is met if the absolute value of the inner voltage at the drain exceeds 10 V. In addition, Sentaurus Device terminates the simulation if the source current is less than $-0.001 \text{ A}/\mu\text{m}$ or greater than $0.002 \text{ A}/\mu\text{m}$.

NOTE The unit depends on the device dimension. It is $\text{A}/\mu\text{m}^2$ for 1D, $\text{A}/\mu\text{m}$ for 2D, and A for 3D devices.

Global Device Break Criteria

The device power is equal to $P = \sum I_k \cdot V_k$, where:

- k is the index of a device contact.
- I_k is the current.
- V_k is the inner or outer voltage at this contact.

Examples of the device power criteria are:

```
Math {  
    BreakCriteria { DevicePower( Absval=6e-5) }  
    BreakCriteria { OuterDevicePower( Absval=6e-5) }  
    BreakCriteria { InnerDevicePower( Absval=2e-6) }  
    ...  
}
```

The keywords `DevicePower` and `OuterDevicePower` are synonyms. In this example, the stopping criterion is met if the absolute value of the outer power exceeds $6 \times 10^{-5} \text{ W}/\mu\text{m}$ or the inner power exceeds $2 \times 10^{-6} \text{ W}/\mu\text{m}$.

4: Performing Numeric Experiments

Break Criteria: Conditionally Stopping the Simulation

The break criteria for lattice temperature, current density, and electric field can be specified by region and material. If no region or material is given, the stopping criteria apply to all regions. A sample specification is:

```
Math (material = "Silicon") {
  BreakCriteria {
    LatticeTemperature (maxval = 1000)
    CurrentDensity (maxval = 1e7)
  }
  ...
}
Math (region = "Region.1") {
  BreakCriteria {
    ElectricField (maxval = 1e6)
  }
  ...
}
```

Sentaurus Device terminates the simulation if the lattice temperature in silicon exceeds 1000 K, the current density in silicon exceeds 10^7 A/cm^2 , or the electrical field in the region Region.1 exceeds 10^6 V/cm .

An upper bound for the lattice temperature can also be specified in the Physics section, for example:

```
Physics {
  LatticeTemperatureLimit = 1693 # melting point of Si
  ...
}
```

NOTE The break criteria of the lattice temperature are only valid for nonisothermal simulations, that is, the keyword Temperature must appear in the corresponding Solve section.

Sweep-specific Break Criteria

Sweep-specific break criteria can be specified as an option of Quasistationary, Transient, and Continuation:

```
solve {
  Quasistationary(
    BreakCriteria {
      Current(Contact = "drain" Absval = 1e-8)
      DevicePower(Absval = 1e-5)
    }
    Goal { Name="drain" Voltage = 5 }
```

```
)  
{ coupled { poisson electron hole } }  
  
Quasistationary(  
  BreakCriteria { CurrentDensity( Absval = 0.1) }  
  Goal { Name = "gate" Voltage = 2 }  
)  
{ coupled { poisson electron hole } }  
  ...  
}
```

This example contains multiple break criteria. As soon as either the drain current or the device power exceeds the limit, Sentaurus Device stops the first Quasistationary computations and switches to the second Quasistationary. As soon as the current density exceeds its limit, Sentaurus Device exits this section and switches to the next section.

Mixed-Mode Break Criteria

All the abovementioned break criteria are also available in mixed mode. In this case, the BreakCriteria section must contain the circuit device name (an exception are voltage criteria on circuit nodes). Examples of the break criteria conditions in mixed mode are:

```
Quasistationary(  
  BreakCriteria {  
    # mixed-mode variables  
    Voltage( Node = a MaxValue = 10)  
    Current( DevName = resistor Node = b MinValue = -1e-5)  
  
    # device variables  
    Voltage(DevName = diode Contact = "anode" MaxValue=10)  
    LatticeTemperature(DevName = mos MaxValue = 1000)  
    ElectricField(DevName = mos MaxValue=1e6)  
    DevicePower(DevName = resistor AbsValue=1e-5)  
  }  
  ...  
)
```

Quasistationary Ramps

The Quasistationary command is used to ramp a device from one solution to another through the modification of its boundary conditions (such as contact voltages) or parameter values.

The simulation continues by iterating between the modification of the boundary conditions or parameter values, and re-solving the device (see [Figure 21](#)). The command to re-solve the device at each iteration is given with the Quasistationary command.

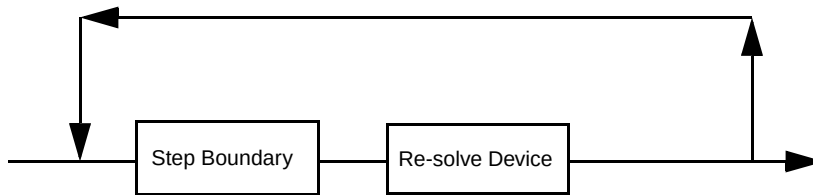


Figure 21 Quasistationary ramp

Ramping Boundary Conditions

To ramp boundary conditions, such as voltages on electrodes, the Quasistationary command is:

```
Quasistationary ( <parameter-list> ) { <solve-command> }
```

The possibilities for <parameter-list> are summarized in [Table 158 on page 1234](#). <solve-command> is Coupled, Plugin, ACCoupled, HBCoupled, or possibly another Quasistationary.

For example, ramping a drain voltage of a device to 5 V is performed by:

```
Quasistationary( Goal {Voltage=5 Name=Drain } ){  
    Coupled { Poisson Electron Hole }  
}
```

Internally, the Quasistationary command works by ramping a variable t from 0.0 to 1.0. The voltage at the contact changes according to the formula $V = V_0 + t(V_1 - V_0)$, where V_0 is the initial voltage and V_1 is the final voltage, which is specified in the Goal statement.

Control of the stepping is expressed in terms of the t variable. The control is not made over contact values because more than one contact can be ramped simultaneously.

The step control parameters are MaxStep, MinStep, InitialStep, Increment, and Decrement in <parameter-list>:

- MaxStep and MinStep limit the step size.
- InitialStep controls the size of the first step of the ramping. The step size is automatically augmented or reduced depending on the rate of success of the inner solve command.
- The rate of increase is controlled by the number of performed Newton iterations and by the factor Increment.
- The step size is reduced by the factor Decrement, when the inner solve fails. The ramping process aborts when the step becomes smaller than MinStep.

Each contact has a type, which can be voltage, current, or charge. Each Quasistationary command has a goal, which can also be voltage, current, or charge. If the goal and contact type do not match, Sentaurus Device changes the contact type to match the goal. However, Sentaurus Device cannot change a contact of charge type to current type, or a contact of current type to charge type.

The initial value (for $t = 0$) is the current or voltage computed for the contact before the Quasistationary command starts. Contacts keep their boundary condition type after the Quasistationary command finishes. To change the boundary condition type of a contact explicitly, use the Set command (see [Changing Boundary Condition Type During Simulation on page 100](#)).

If all equations to be solved in a Quasistationary also have been solved in the solve statement immediately before, Sentaurus Device omits the point $t = 0$, as the solution is already known. Otherwise, this point is computed at the beginning of the Quasistationary. You can explicitly force or prevent this point from being computed using DoZero or -DoZero in the <parameter-list>.

Ramping Quasi-Fermi Potentials in Doping Wells

Electron and hole quasi-Fermi potentials in specified doping wells can also be ramped in a Quasistationary statement. This is a quick and robust way to deplete doping wells without having to solve the continuity equation for the carrier to be depleted. Its main application is in CMOS image sensor (CIS) and charge-coupled device (CCD) simulations.

To ramp a quasi-Fermi potential, specify the keyword WellContactName or DopingWell followed by eQuasiFermi or hQuasiFermi in the Goal section of the Quasistationary command:

```
Goal { [ WellContactName = <contact name> | DopingWell(<point coordinates>)]  
      [eQuasiFermi = <value> | hQuasiFermi = <value>] }
```

4: Performing Numeric Experiments

Quasistationary Ramps

The `DopingWell` specification is more general and can be used for both semiconductor wells with or without contacts (buried wells). In this case, the coordinates of a point in the well must be given to select the well (see code above).

When the semiconductor well where the quasi-Fermi potential is ramped is connected to a contact, the `WellContactName` specification can be used. Now, the contact associated with the well must be specified to select the well (see code above).

The starting value of the `Goal` for the quasi-Fermi potential ramping in the specified well is taken as the well averaged value of the quasi-Fermi potential at the end of the previous `Quasistationary` command. This starting value can be changed using a `Set` statement:

```
Solve {
  Set(DopingWell(0.5 0.5) eQuasiFermi=5.0)
  Quasistationary(... Goal {DopingWell(0.5 0.5) eQuasiFermi=10.0})
    {Coupled {poisson hole}}
}
```

The `Set` statement has the same options and syntax as the `Goal` statement above.

In mixed-mode simulations, the feature still can be used for each device separately. In this case, the keywords `DopingWell` and `WellContactName` must be prefixed by the device name:

```
System {
  MOSCAP1 "mc1" (
    "gate_contact" = gate1
    "substrate_contact" = sub
  )
  ...
}

Solve {
  Set(mc1.DopingWell(0.5 0.5) eQuasiFermi=5.0)
  Quasistationary(...
    Goal {mc1.DopingWell(0.5 0.5) eQuasiFermi=10.0}

  ) {Coupled {poisson hole circuit}}
}
```

In addition, Sentaurus Device supports multiple-well ramping, activated by the keyword `DopingWells` with an option in parentheses. The syntax is:

```
Quasistationary(...
  Goal {DopingWells([Region=<region> | Material=<material> | Semiconductor])
    eQuasiFermi=<value>}
  ) {Coupled {poisson hole}}
```

As can be seen from this syntax description, wells in a region, wells having the same material, or all semiconductor wells in the device can be ramped to a specified value of the electron or hole quasi-Fermi potential. A well is considered to be in a region if it has one or more vertices in common with the region. A well belongs to a region even if it is not entirely in the region.

For multiple-well ramping, the `Set` command is used to change the quasi-Fermi potential in a group of wells before a Quasistationary ramping:

```
Set(DopingWells([Region=<region> | Material=<material> | Semiconductor])
    eQuasiFermi=5.0)
```

In the case of mixed-mode simulations, `DopingWells` must be prefixed with the device instance name and a "." similar to single-well syntax.

Electron and hole quasi-Fermi potentials can be simultaneously ramped using multiple `Goal` statements and solving for the Poisson equation only.

When the continuity equation has been solved for a carrier whose quasi-Fermi potential is to be ramped, the initial value of the quasi-Fermi potential for all vertices in the well is computed from the carrier density in the point specified when the well was defined. For multiple-well ramping, the same scheme applies well-wise.

Ramping Physical Parameter Values

A Quasistationary command allows parameters from the parameter file of Sentaurus Device to be ramped. The `Goal` statement has the form:

```
Goal { [ Device = <device> ]
      [ Material = <material> | MaterialInterface = <interface> |
        Region = <region> | RegionInterface = <interface> ]
      Model = <model> Parameter = <parameter> Value = <value> }
```

Specifying the device and location (material, material interface, region, or region interface) is optional. However, the model name and parameter name must always be specified.

A list of model names and parameter names is obtained by using:

```
sdevice --parameter-names
```

This list of parameters corresponds to those in the Sentaurus Device parameter file, which can be obtained by using `sdevice -P` (see [Generating a Copy of Parameter File on page 70](#)).

The following command produces a list of model names and parameter names that can be ramped in the command file:

```
sdevice --parameter-names <command file>
```

4: Performing Numeric Experiments

Quasistationary Ramps

Sentaurus Device reads the devices in the command file and reports all parameter names that can be ramped. However, no simulation is performed. The models in [Table 12](#) contain command file parameters that can be ramped.

Table 12 Command file parameters

Model name	Parameters
OptBeam(<index>) RayBeam(<index>)	RefractiveIndex, SemAbs, SemSurf, SemVelocity_Vx, SemVelocity_Vy, SemVelocity_Vz, SemWindow_xmax, SemWindow_xmin, SemWindow_ymax, SemWindow_ymin, SemWindow_zmax, SemWindow_zmin, WaveDir_x, WaveDir_y, WaveDir_z, WaveEnergy, WaveInt, WaveLength, WavePower, WaveTime_tmax, WaveTime_tmin, WaveTsigma, WaveXYSigma
OpticalGeneration(<index>)	WaveLength
Optics	Wavelength, Amplitude, WavePower, WaveIntensity, Theta, Phi, Polarization
RadiationBeam	Dose, DoseRate, DoseTSigma, DoseTime_end, DoseTime_start
Traps(<index>)	Conc, EnergyMid, EnergySig, eGfactor, eJfactor, eXsection, hGfactor, hJfactor, hXsection
DeviceTemperature	Temperature

NOTE Certain models such as optical beams and traps must be specified with an index:

Model="OptBeam(0)"

Model="Traps(1)"

The index denotes the exact model for which a parameter should be ramped. Usually, Sentaurus Device assigns an increasing index starting with zero for each optical beam, trap, and so on. However, the situation becomes more complex if material and region specifications are present. To confirm the value of the index, using the following command is recommended:

```
sdevice --parameter-names <command file>
```

Mole fraction–dependent parameters can be ramped. For example, if the parameter *p* is mole fraction–dependent, the parameter names listed in [Table 13](#) can appear in a `Goal` statement.

Table 13 Mole fraction–dependent parameters as Quasistationary Goals

Parameter in Goal statement	Description
Parameter= <i>p</i>	Parameter <i>p</i> in non-mole fraction–dependent materials.
Parameter="p(0)" Parameter="p(1)" ...	Interpolation value of <i>p</i> at Xmax(0), Xmax(1), ...

Table 13 Mole fraction–dependent parameters as Quasistationary Goals

Parameter in Goal statement	Description
Parameter="B(p(1))" Parameter="C(p(1))" Parameter="B(p(2))" Parameter="C(p(2))" ...	Quadratic and cubic interpolation coefficients of p in intervals $[X_{\max}(0), X_{\max}(1)]$, $[X_{\max}(1), X_{\max}(2)]$, ...

Mole fraction–dependent parameters can be ramped in all materials. In mole fraction–dependent materials, the interpolation values $p(\dots)$ and the interpolation coefficients $B(p(\dots))$ and $C(p(\dots))$ must be ramped. In a non-mole fraction–dependent material, only the parameter p can be ramped.

If a parameter is not found, Sentaurus Device issues a warning, and the corresponding goal statement is ignored.

Parameters in PMI models can also be ramped.

Quasistationary in Mixed Mode

The Quasistationary command is extended in mixed mode to include goals on nodes and circuit model parameters. [Table 153 on page 1230](#) shows the syntax of these goals.

The goal is usually used on a node that has been fixed with the Set or Initialize command in the System section. The keyword `Goal{Node=<string> Voltage=<float>}` assigns a new target voltage to a specified node. For example, the node `a` is set to 1 V and ramped to 10 V:

```
System {
    Resistor_pset r1(a 0){resistance = 1}
    Set(a=1)
}

Solve{
    Circuit
    Quasistationary( Goal{Node=a Voltage=10} ){ Circuit }
}
```

A goal on circuit model parameters can be used to change the configuration of a system. The keyword `Goal{Parameter=<i-name>.<p-name> value=<float>}` assigns a new target value to a specified parameter `p-name` of a device instance `i-name`. Any circuit model parameter can be changed.

4: Performing Numeric Experiments

Quasistationary Ramps

For example, the resistor *r1* is ramped from 1Ω to 0.1Ω , and the offset of the sine voltage source is ramped to 1.5 V:

```
System {
  Resistor_pset r1(a 0){resistance = 1}
  Vsource_pset v0 (n1 n0) { sine = (0.5 1 10 0 0) }
  Set(a=1)
}

Solve{
  Circuit
  Quasistationary(
    Goal { Parameter = r1."resistance" Value = 0.1 }
    Goal { Parameter = v0."sine[0]" Value = 1.5 })
  { Circuit }
}
```

NOTE When a node is used in a goal, it is set during the quasistationary simulation. At the end of the ramp, the node reverts to its previous ‘set’ status, that is, if it was not set before, it is not set afterward. This can cause unexpected behavior in the Solve statement following the Quasistationary statement. Therefore, it is better to set the node in the Quasistationary statement using the Set command.

Saving and Plotting During a Quasistationary

Data can be saved and plotted during a Quasistationary ramping process by using the Plot command. Plot is placed with the other Quasistationary parameters, for example:

```
Quasistationary(
  Goal {Voltage=5 Name=Drain}
  Plot {Range = (0 1) Intervals=5})
  {Coupled{ Poisson Electron Hole }}
```

In this example, six plot files are saved at five intervals: $t = 0, 0.2, 0.4, 0.6, 0.8$, and 1.0 .

Another way to plot data is with Plot in the Quasistationary body rather than inside the Quasistationary parameters (see [When to Plot on page 145](#)). Plot is added after the equations to solve, such as:

```
Quasistationary( Goal {Voltage=5 Name=Drain } ){
  Coupled { Poisson Electron Hole }
  Plot ( Time= ( 0.2; 0.4; 0.6; 0.8; 1.0 ) NoOverwrite )
}
```

Extrapolation

The Quasistationary command can use extrapolation to predict the next solution based on the values of the previous solutions. Extrapolation can be switched on globally in the Math section:

```
Math {
    Extrapolate
}
```

or it can be switched on for a specific Quasistationary command only:

```
Quasistationary (
    Goal { ... }
    Extrapolate
) { Coupled { Poisson Electron Hole } }
```

By default, Sentaurus Device uses linear extrapolation, but you also can request higher order extrapolation. For example, quadratic extrapolation is obtained by specifying:

```
Extrapolate (Order = 2)
```

The extrapolation information is preserved between Quasistationary commands, and it also is saved and loaded automatically by the Save and Load commands. A subsequent Quasistationary command can use this extrapolation information if the following conditions are met:

1. The previous and current Quasistationary commands have the same number of goals.
2. The previous and current Quasistationary commands ramp the same quantities.
3. The previous and current Quasistationary commands are contiguous, that is, for all goals, the current initial value is equal to the goal value in the previous command.
4. Multiple goals are ramped at the same rate in the current Quasistationary command compared to the previous Quasistationary command. As an example, assume that all contact voltages have zero initial values. Then, the following two Quasistationary commands satisfy this condition:

```
Quasistationary (
    Goal { Name = "gate"   Voltage = 1 }
    Goal { Name = "drain"  Voltage = 2 }
) { Coupled { Poisson Electron Hole } }
```

```
Quasistationary (
    Goal { Name = "gate"   Voltage = 3 }
    Goal { Name = "drain"  Voltage = 6 }
) { Coupled { Poisson Electron Hole } }
```

4: Performing Numeric Experiments

Quasistationary Ramps

On the other hand, the following two Quasistationary commands violate this condition:

```
Quasistationary (  
  Goal { Name = "gate"   Voltage = 1 }  
  Goal { Name = "drain"  Voltage = 2 }  
) { Coupled { Poisson Electron Hole } }
```

```
Quasistationary (  
  Goal { Name = "gate"   Voltage = 3 }  
  Goal { Name = "drain"  Voltage = 10 }  
) { Coupled { Poisson Electron Hole } }
```

In the second Quasistationary command, the drain voltage is ramped at a higher rate than the gate voltage. Therefore, the extrapolation information from the previous Quasistationary command cannot be used.

5. The values of the solution variables have not changed between the two Quasistationary commands, for example, by a Load command.

If the extrapolation information from a previous Quasistationary command can be used successfully, the following message appears in the log file:

Reusing extrapolation from a previous quasistationary

The Quasistationary options in [Table 14](#) control the handling of extrapolation information.

Table 14 Extrapolation options

Option	Description
ReadExtrapolation	Tries to use the extrapolation information from a previous command if it is available and compatible. This is the default.
-ReadExtrapolation	Do not use the extrapolation information from a previous command.
StoreExtrapolation	Stores the extrapolation information internally at the end of the command, so that it will be available for a subsequent command, or can be written to a save or plot file. This is the default.
-StoreExtrapolation	Do not store the extrapolation information internally at the end of the command.

Additional Features

Relaxed Newton Parameters

During a quasistationary ramp, the Newton parameters for the involved Coupled statements are fixed over the entire range. If you observe convergence problems for certain operating conditions during the ramp, which lead to a deterioration of the ramping step, you can use relaxed Newton convergence parameters if the ramping step falls below a certain threshold. for example:

```
Math { ...
  AcceptNewtonParameter (
    -RhsAndUpdateConvergence      * enable 'RHS OR Update' convergence
    RhsMin = 1.e-5                 * RHS converged if 'RHS < RhsMin'
    UpdateScale = 1.e-2            * scale actual update error
  )
}
Solve { ...
  Quasistationary ( ...
    AcceptNewtonParameter ( ReferenceStep = 1.e-3 )
  ) { Coupled { ... } }
}
```

In the global Math section, you specify with `AcceptNewtonParameter` some Newton parameters that are applied at the last Newton step if the Newton has neither diverged nor converged yet. In the `Quasistationary` statement, you add `AcceptNewtonParameter` and specify the ramping step threshold using `ReferenceStep`.

The relaxed Newton parameters become only effective now if the actual ramping step is smaller than the specified `ReferenceStep`, and the last Newton iteration of the Coupled statement is reached. Note that the relaxed Newton parameters are only passed to the next-level Coupled statements, that is, Coupled statements in Plugin statements do not use the relaxed Newton parameters. Only specified values are passed to the Coupled statement, that is, no default values are used.

The following parameters in `AcceptNewtonParameter` of the Math section are supported:

- `RhsAndUpdateConvergence`: Requires both RHS and update convergence.
- `RhsMin`: Newton steps with RHS norms smaller than this value are RHS converged.
- `UpdateScale`: Additional factor computing the update error.

Continuation Command

The `Continuation` command enables automated tracing of arbitrarily shaped $I(V)$ curves of complicated device phenomena such as breakdown or latchup. Simulation of these phenomena usually requires biasing conditions tracing a multivalued $I(V)$ curve with abrupt changes. The implementation is based on a dynamic load-line technique [1] adapting the boundary conditions along the traced $I(V)$ curve to ensure convergence. An external load resistor is connected to the device electrode at which the $I(V)$ curve is traced, the device being indirectly biased through the load resistance. The boundary condition consists of an external voltage applied to the other end of the load resistor not connected to device. By monitoring the slope of the traced $I(V)$ curve, an optimal boundary condition (external voltage) is determined by adjusting the load line so it is orthogonal to the local tangent of the $I(V)$ curve. The boundary conditions are generated automatically by the algorithm without prior knowledge of the $I(V)$ curve characteristics.

An important part of the continuation method consists of computing the slope in each point of the $I(V)$ curve. This is equivalent with computing the inverse of the device differential resistance when a small-voltage perturbation is applied to the contact undergoing continuation. The simulation advances to the next operating point if the solution has converged. Before moving to the next step, the load line is recalibrated so that it is orthogonal to the local tangent of the $I(V)$ curve. This ensures an optimal boundary condition. A user-defined window specifies the limits for curve tracing. The tracing window is specified by user-defined lower and upper values for the voltage and current of the operating point at the continuation electrode: `MinVoltage`, `MaxVoltage`, `MinCurrent`, and `MaxCurrent`. The simulation ends when the operating point is outside the tracing window.

The continuation method is activated by using the keyword `Continuation` in the `Solve` section. Some control parameters must be given in parentheses in the same way as for the `Plugin` statement, for example:

```
Continuation (<Control Parameters>) {  
    Coupled (iterations=15) { poisson electron hole }  
}
```

[Table 149 on page 1226](#) summarizes the control parameters of the `Continuation` command. The method works with both single-device and mixed-mode setups, with only one electrode undergoing continuation at a time.

The first step of the continuation is always a voltage-controlled step. For this, you must supply an initial voltage step in the control parameter list using the `InitialVstep` statement. The continuation proceeds automatically until the values given by any of the `MinVoltage`, `MaxVoltage`, `MinCurrent`, or `MaxCurrent` parameters are exceeded. In addition, the parameters `Increment`, `Decrement`, `Error` and `Digits` can be specified. Their definitions

are the same as for `Transient` and `Quasistationary` statements except that they measure the $I(V)$ curve arc length.

In the regions where the $I(V)$ curve becomes vertical (close to current boundary condition) and the current extends over several orders of magnitude for almost the same applied voltage, another parameter, `MinVoltageStep`, may need to be adjusted. `MinVoltageStep` is the minimum-allowed voltage difference between two adjacent operating points on the $I(V)$ curve. In such cases, increasing the parameter value produces a smoother curve over the vertical range.

Tracing successfully an $I(V)$ curve with the continuation method depends on how accurately the local slope of the traced $I(V)$ curve is computed. As slope computation involves using inverted Jacobian and current derivatives at the electrode, an accurate computation of the contact current and a low numeric noise in Jacobian are prerequisites for continuation.

At low-biasing voltages, current at the continuation electrode is very small and, in general, noisy. This leads to an inaccurate computation of the slope, which in turn causes the continuation method to backtrack. To overcome this problem, Sentaurus Device allows you to divide the continuation window into two regions separated by a threshold current with a value specified by the parameter `Iadapt`. The lower region is a low-current range where the slope computation is inaccurate, and the upper region is now the region where the continuation method is expected to work as designed.

From `MinCurrent` to `Iadapt` (the lower region of the simulation window), the adaptive algorithm is switched off and a fixed-value resistor is used instead. In this range, the simulation proceeds as a voltage ramping through a fixed-value resistor attached to the continuation electrode. Because no slope computation is necessary, the sensitivity of the simulation to current noise is eliminated. When the current increases to the value specified by `Iadapt`, the adaptive continuation is switched on and the curve tracing proceeds as previously described. The default value for the fixed resistor used in the lower region is $0.001\ \Omega$, and it can be changed by either using the continuation parameter `Rfixed` or specifying the `Resist` keyword in the `Electrode` section of the continuation electrode.

An example of a `Solve` entry for continuation is:

```
Electrode{
    ...
    {Name="collector" Voltage=0.0}
}

Solve{ ...
    Continuation ( Name="collector" InitialVstep=-0.001
                  MaxVoltage=0 MaxCurrent=0
                  MinVoltage=-10 MinCurrent=-1e-3
                  Iadapt=-1e-13) {
```

4: Performing Numeric Experiments

Continuation Command

```
        Coupled (iterations=10) { poisson electron hole }  
    }  
}
```

This specifies that the $I(V)$ curve must be traced at the electrode "collector". The initial voltage-controlled step is -0.001 V, the voltage range is -10 V to 0 V, and the current range is -1 mA to 0 A. The adaptive algorithm is switched on when the current reaches -1.0×10^{-13} A to avoid low-current regime noise.

The step along the traced $I(V)$ curve is controlled primarily by the convergence. The step increases by a factor `Increment` if the problem has converged. When the problem does not converge, the step is cut by a factor `Decrement` and the problem is re-solved. The step cut continues until the problem converges. The continuation parameters `Increment` and `Decrement` are available for adjustment in the `Continuation` section, and the default values are `2.0` for `Increment` and `1.5` for `Decrement`.

Sentaurus Device also allows a more complex step control based on both convergence and curve smoothness. This is activated by specifying the keyword `Normalized` in the `Continuation` section. The angle between the last two segments on the $I(V)$ is computed in a local scaled I - V plane, with the scaling factors depending on the current point on the $I(V)$ curve. If the angle is smaller than the continuation parameter `IncrementAngle`, the step increases by the factor `Increment`. If the angle is greater than `IncrementAngle` but smaller than `DecrementAngle`, the step is kept constant. Finally, if the angle is greater than `DecrementAngle`, the step decreases by the factor `Decrement`. Default values for `IncrementAngle` and `DecrementAngle` are `2.5` and `5` degrees, respectively.

A few more options are available for limiting the step (arclength) along the traced $I(V)$ curve. By specifying the continuation parameter `MaxVstep`, the step along the $I(V)$ curve is limited to an upper value such that the step projection on the V -axis is smaller in absolute value than `MaxVstep`. This option is particularly useful for a low-voltage range when a curve with more points is needed. In the higher-current range, one of the continuation parameters `MaxIstep`, `MaxIfactor`, or `MaxLogIfactor` can be used to limit the step along the curve. When `MaxIstep` is specified, the step is limited such that the step projection on the I -axis is smaller in absolute value than `MaxIstep`. `MaxIfactor` specifies how many times the current is allowed to increase for two adjacent points on the $I(V)$ curve. Similarly, `MaxLogIfactor` specifies by how many orders of magnitude the current is allowed to increase for two adjacent points on the $I(V)$ curve. `MaxVstep` can be combined with one of `MaxIstep`, `MaxIfactor`, or `MaxLogIfactor`.

In mixed mode, the continuation method allows you to have all other contacts except the continuation contact connected in a circuit. The continuation contact should not be connected to any circuit node. In this case, the `Name` keyword in the `Continuation` section specifying the continuation contact name is replaced by `dev_inst.Name`, where `dev_inst` represents the device instance of the respective contact. For example, for device `mos1` with electrodes

named source, drain, gate, and substrate undergoing continuation on the drain electrode, the possible syntax is:

```
Device mos1 {
  Electrode {
    {Name="source" Voltage=0.0}
    {Name="drain" Voltage=0.0}
    {Name="gate" Voltage=0.0}
    {Name="substrate" Voltage=0.0}
  }
  ...
}

System {
  mos1 d1 (source=s1 gate=g1 substrate=s1)
  set (s1=0 g1=0 b1=0)
}

Solve {
  ...
  Continuation(d1.Name="drain"
    ...
  ) {Coupled {Poisson Electron Hole}}
}
```

In mixed-mode simulations, sometimes the continuation electrode must be biased to a certain voltage using a Quasistationary simulation before the continuation starts. In mixed mode, because the continuation electrode is not allowed to be connected to any node, special syntax must be used to avoid biasing the contact as a node. For example, in the above syntax, the drain electrode needs to be biased to 3V before the continuation. Instead of Name, the Contact keyword is used to identify the drain contact:

```
Quasistationary ( InitialStep=1e-3
  ...
  Goal {Contact=d1."drain" Voltage=3}
) {Coupled {Poisson Electron Hole}}
```

Transient Command

The Transient command is used to perform a transient time simulation. The command must start with a device that has already been solved. The simulation continues by iterating between incrementing time and re-solving the device (see [Figure 22 on page 120](#)). The command to solve the device at each iteration is given with the Transient command.

4: Performing Numeric Experiments

Transient Command

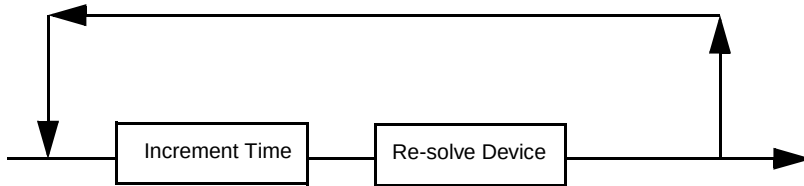


Figure 22 Transient simulation

The syntax of the Transient command is:

```
Transient ( <parameter-list> ) { <solve-command> }
```

[Table 162 on page 1237](#) lists the possible parameters.

In the above command, <solve-command> is Coupled or Plugin.

An example of performing a transient simulation for 10 μs is:

```
Transient( InitialTime = 0.0 FinalTime=1.0e-5 ){  
    Coupled { Poisson Electron Hole }  
}
```

The Transient command allows you to overwrite time-step control parameters, which have default values or are globally defined in the Math section. The error control parameters that are accepted by the Transient command are listed in [Table 180 on page 1258](#).

The parameters TransientError, TransientErrRef, and TransientDigits control the error over the transient integration method. This differs from the error control for the Coupled command, which only controls the error of each nonlinear solution. As with the error parameters for the Coupled command, the transient error controls can be both absolute and relative. Absolute values are parameterized according to equation–variable type.

The plot controls of the Transient command are the same as for the Quasistationary command, except the t is the real time (in seconds), and is not restricted to an interval from 0 to 1.

Plot can be given as a Transient parameter, for example:

```
Transient( InitialTime = 0.0 FinalTime = 1.0e-5  
    Plot { Range = (0 3.0e-6) Intervals=3 } )  
    { Coupled { Poisson Electron Hole } }
```

This example saves four plot files at $t = 0.0, 1.0\text{e-}6, 2.0\text{e-}6$, and $3.0\text{e-}6$. Alternatively, Plot can be put into the Transient solve command (see [When to Plot on page 145](#)):

```
Transient( InitialTime = 0.0 FinalTime = 1.0e-5 )
  { Coupled{ Poisson Electron Hole }
    Plot ( Time=( 1.0e-6; 2.0e-6; 3.0e-6 ) NoOverwrite )
  }
```

Numeric Control of Transient Analysis

A set of keywords is available in the Math section to control transient simulation. Sentaurus Device uses implicit discretization of nonstationary equations and supports two discretization schemes: the trapezoidal rule/backward differentiation formula (TRBDF), which is the default, and the simpler backward Euler (BE) method. To select a particular transient method, specify `Transient=<transient-method>`, where `<transient-method>` can be TRBDF or BE.

In transient simulations, in addition to numeric errors of the nonlinear equations, discretization errors due to the finite time-step occur (see [Transient Simulation on page 968](#)). By default, Sentaurus Device does not control the time step to limit this discretization error (that is, that time step depends only on convergence of Newton iterations).

To activate time-step control, `CheckTransientError` must be specified. For time-step control, Sentaurus Device uses a separate set of criteria, but as with Newton iterations, the control of both relative and absolute errors is performed. Relative transient error $\epsilon_{R,tr}$ is defined similarly to ϵ_R in [Eq. 31, p. 160](#):

$$\epsilon_{R,tr} = 10^{-\text{TransientDigits}} \quad (15)$$

Similarly, for $x_{ref,tr}$ and $\epsilon_{A,tr}$, the equation:

$$x_{ref,tr} = \frac{\epsilon_{A,tr}}{\epsilon_{R,tr}} x^* \quad (16)$$

is valid. The keyword `TransientDigits` must be used to specify relative error $\epsilon_{R,tr}$ in transient simulations. An absolute error in time-step control can be specified by either the keyword `TransientError` ($\epsilon_{A,tr}$) or `TransientErrRef` ($x_{ref,tr}$), and their values can be defined independently for each equation variable. The same flag as in Newton iteration control, `RelErrControl`, is used to switch to unscaled (`TransientErrRef`) absolute error specification.

For simulations with floating gates, Sentaurus Device can monitor the errors in the floating-gate charges as well. The details are described in [Floating Gates on page 971](#).

NOTE All transient parameters in the Math section, except `Transient`, can be overwritten in the `Transient` command of the `Solve` section (see [Transient Command on page 119](#)).

Time-stepping

A transient simulation computes the status of the system or device as a function of time for a finite time range, specified by `InitialTime` and `FinalTime`, by advancing from a status at a given time to the status at a later time point. The time step is chosen dynamically. The time step is bounded by `MinStep` and `MaxStep` from below and above, respectively; while `InitialStep` specifies the first time step at start time.

If the computation of the system status for a given time step fails, the time step is reduced iteratively until either the system status can be computed successfully or the minimum time step is reached. In the later case, the `Transient` terminates with an error. After a successful computation of the system status, the time step is typically increased, depending on the computational effort for the actual time step.

The effective time step can be smaller than the advancing time step, for example, if time points are specified for plotting, where a system status must be computed. However, these effective time steps do not reduce the advancing time step, that is, the subsequent effective time step may be as large as the advancing time step. You can bound (from above) the advancing time step at certain conditions by specifying turning points as a parameter in the `Transient`. These conditions can be a list of arbitrary time points or time ranges (see [Table 162 on page 1237](#)), for example:

```
Transient ( ...
  TurningPoints (
    ( Condition ( Time ( 1.e-7 ; 2.e-7 ; 3.e-7 ) ) Value = 1.1e-9 )
    ( Condition ( Time ( Range = (1.e-7 2.e-7 ) ) ) Value = 2.1.e-9 )
  )
)
```

The advancing time step is limited by the specified `Value` (in seconds) for the corresponding `Condition`. Here, the first condition limits the value for a list of time points. The second condition limits the time step for a whole range. Observe that, for time points, the computation of the transient is triggered; while for `Range`, this is not the case. `Time` takes the same options as the `Time` keyword in `Plot` in `Solve`. The specification:

```
TurningPoints ( ... (1.e-9 1.e-10) )
```

serves as shorthand for a single time-point condition:

```
TurningPoints ( ... ( Condition ( Time ( 1.e-9 ) ) Value = 1.e-10 ) )
```

Ramping Physical Parameter Values

A Transient command allows parameters from the parameter file of Sentaurus Device to be ramped linearly. The Bias statement (similar to the subsection Goal in a Quasistationary command, see [Ramping Physical Parameter Values on page 109](#)) has the form:

```
Bias { [ Device = <device> ]
      [ Material = <material> | MaterialInterface = <interface> |
        Region = <region> | RegionInterface = <interface> ]
      Model = <model> Parameter = <parameter> Value = <value_list>
    }
```

Specifying the device and location (material, material interface, region, or region interface) is optional. However, the model name and parameter name must always be specified. Example:

```
Transient (
  InitialTime = 0 FinalTime = 1
  Bias( Material = "Silicon"
    Model = DeviceTemperature Parameter = Temperature
    Value = (300 at 0.1, 400 at 0.2, 450 at 0.5)
  )
  Bias( Region = "Channel"
    Model = DeviceTemperature Parameter = Temperature
    Value = (550 at 0.6, 400 at 0.8, 300 at 0.9)
  )
) { Coupled { Poisson Electron Hole } }
```

For a list of parameters that can be ramped in a Transient command, see [Ramping Physical Parameter Values on page 109](#).

Extrapolation

If the Extrapolate option is present in the Math section, the Transient command uses the linear extrapolation of the last two solutions to predict the next solution. This extrapolation information is preserved between Transient commands, and it is also saved and loaded automatically by the Save and Load commands.

A Transient command can use this extrapolation information if the following conditions are met:

1. The previous and current Transient commands are contiguous, that is, the final time of the previous Transient is equal to the initial time of the current Transient.
2. The values of the solution variables have not changed between the two Transient commands, for example, by a Load command.

4: Performing Numeric Experiments

Large-Signal Cyclic Analysis

If the extrapolation information from a previous `Transient` command can be used successfully, the following message appears in the log file:

```
Reusing extrapolation from a previous transient
```

The options to control the handling of extrapolation information described in [Table 14 on page 114](#) are available for `Transient` as well.

Additional Features

Relaxed Newton Parameters

If the time step during a `Transient` becomes extremely small due to convergence problems of the next-level `Coupled`, you can avoid the deterioration of the time step by using relaxed Newton parameters for the corresponding `Coupled` statement. This can be performed in the same way as for `Quasistationary` statements (see [Relaxed Newton Parameters on page 115](#)) by using `AcceptNewtonParameter` in both the `Math` and the `Transient` sections:

```
Math { ... AcceptNewtonParameter ( RhsMin=1.e-5 ) }  
  
Solve { ...  
    Transient ( ...  
        AcceptNewtonParameter ( ReferenceStep=1.e-9 )  
    ) { Coupled {...} }  
}
```

Large-Signal Cyclic Analysis

For high-speed and high-frequency operations, devices are often evaluated by cyclic biases. After a time, device variables change periodically. This cyclic-bias steady state [\[2\]](#) is a condition that occurs when all parameters of a simulated system return to the initial values after one cycle bias is applied.

In fact, such a cyclic steady state is reached by using standard transient simulation. However, this is not always effective, especially if some processes in the system have very long characteristic times in comparison with the period of the signal.

For example, deep traps usually have relatively long characteristic times. A suggested approach [\[3\]](#) allows for significant acceleration of the process of reaching a cyclic steady state solution. The approach is based on iterative correction of the initial guess at the beginning of each period of transient simulation, using previous initial guesses and focusing on reaching a

cyclic steady state. This approach is implemented in Sentaurus Device. An alternative frequency-domain approach is harmonic balance (see [Harmonic Balance on page 130](#)).

Description of Method

The original method [3] is summarized. Transient simulation starts from some initial guess. A few periods of transient simulation are performed and, after each period, the change over the period of each independent variable of the simulated system is estimated (that is, the potential at each vertex of all devices, electron and hole concentrations, carrier temperatures, and lattice temperature if hydrodynamic or thermodynamic models are selected, trap occupation probabilities for each trap type and occupation level, and circuit node potentials in the case of mixed-mode simulation).

If x_n^I denotes the value of any variable in the beginning of the n -th period, and x_n^F denotes the same value at the end of the period, the cyclic steady state is reached when:

$$\Delta x_n = x_n^F - x_n^I \quad (17)$$

is equal to zero.

Considering linear extrapolation and that the goal is to achieve $\Delta x_{n+1} = 0$, the next initial guess can be estimated as:

$$x_{n+1}^I = x_n^I - \gamma \frac{x_n^I - x_{n-1}^I}{\Delta x_n - \Delta x_{n-1}} \Delta x_n \quad (18)$$

where γ is a user-defined relaxation factor to stabilize convergence.

As [Eq. 18](#) contains uncertainty such as 0/0, especially when Δx is close to zero (when the solution is close to the steady state), special precautions are necessary to provide robustness of the algorithm.

Consider the derivation of [Eq. 18](#) in a different fashion – near the cyclic steady state. If such a steady state exists, the initial guess $y = x^I$ is expected to behave with time as $y = a \exp(-\alpha t) + b$, where $\alpha > 0$. It is easy to show that [Eq. 18](#) gives $x^I = b$, that is, a desirable cyclic steady-state solution.

It follows that the ratio r :

$$r = -\frac{x_n^I - x_{n-1}^I}{\Delta x_n - \Delta x_{n-1}} \quad (19)$$

4: Performing Numeric Experiments

Large-Signal Cyclic Analysis

can be estimated as $r \approx 1/(1 - \exp(-\alpha t))$. From this, it is clear that because α is positive, the condition $r \geq 1$ must be valid. Moreover, r can be very large if some internal characteristic time (like the trap characteristic time) is much longer than the period of the cycle.

Using the definition of r from [Eq. 19](#), [Eq. 18](#) can be rewritten as:

$$x_{n+1}^I = x_n^I + \gamma r \Delta x_n \quad (20)$$

Although [Eq. 19](#) and [Eq. 20](#) are equivalent to [Eq. 18](#), it is more convenient inside Sentaurus Device to use [Eq. 19](#) and [Eq. 20](#). Sentaurus Device never allows r to be less than 1 because of the above arguments.

To provide convergence and robustness, it is reasonable also not to allow r to become very large. In Sentaurus Device, r cannot exceed a user-specified parameter $r_{\max}$. You can also specify the value of the parameter $r_{\min}$.

An extrapolation procedure, which is described by [Eq. 19](#) and [Eq. 20](#), is performed for every variable of all the devices, in each vertex of the mesh. Instead of densities, which can spatially vary over the device by many orders of magnitude, the extrapolation procedure is applied to the appropriate quasi-Fermi potentials.

For the trap equations, extrapolation can be applied either to the trap occupation probability f_T (the default) or, optionally, to the ‘trap quasi-Fermi level,’ $\Phi_T = -\ln((1 - f_T)/f_T)$. The cyclic steady state is supposedly reached if the following condition is satisfied:

$$\frac{\Delta x}{x + x_{\text{ref}}} < \epsilon_{\text{cyc}} \quad (21)$$

Values of x_{ref} are the same as ErrRef values of the Math section. For every object o of the simulated system (that is, every variable of all devices), an averaged value r_{av}^o of the ratio r is estimated and can be optionally printed. Estimation of r_{av}^o is performed only at such vertices of the object, where the condition:

$$\frac{\Delta x}{x + x_{\text{ref}}} < \frac{\epsilon_{\text{cyc}}}{f} \quad (22)$$

is fulfilled, that is, the same condition as [Eq. 21](#), but with a possibly different tolerance $\epsilon_{1_{\text{cyc}}} = \epsilon_{\text{cyc}}/f$.

The following extrapolation procedures are allowed:

1. Use of averaged extrapolation factors for every object. This is the default option.
2. Use of the factor r independently for every mesh vertex of all objects. If for some reason, the criterion in [Eq. 22](#) is already reached, the value of factor r is replaced by the user-

defined parameter $r_{\min}$. The option is activated by the keyword `-Average` in the `Extrapolate` statement inside the `Cyclic` specification.

3. The same as Step 2, but for the points where [Eq. 22](#) is fulfilled, the value of factor r is replaced by the averaged factor r_{av}^o . The option is activated by the keyword `-Average` in the `Extrapolate` statement inside the `Cyclic` specification, accompanied by the specification of the parameter $r_{\min} = 0$.

Using Cyclic Analysis

Cyclic analysis is activated by specifying the parameter `Cyclic` in the parameter list of the `Transient` statement. `Cyclic` is a complex structure-like parameter and contains cyclic options and parameters in parentheses:

```
Transient( InitialTime=0 FinalTime=2.e-7 InitialStep=2.e-14 MinStep=1.e-16
  Cyclic( <cyclic-parameters> )
) { ... }
```

With sub-options to the optional parameter `Extrapolate` to the `Cyclic` keyword, details of the cyclic extrapolation procedure are defined. [Table 151 on page 1228](#) lists all options for `Cyclic`.

An example of a `Transient` command with `Cyclic` specification is:

```
Transient( InitialTime=0 FinalTime=2.e-7 InitialStep=2.e-14 MinStep=1.e-16
  Cyclic( Period=8.e-10 StartingPeriod=4
    Accuracy=1.e-4 RelFactor=1
    Extrapolate (Average Print MaxVal=50) )
) { ... }
```

NOTE The value of the parameter `Period` in the `Cyclic` statement must be divisible by the period of the bias signal. A periodic bias signal must be specified elsewhere in the command file.

Small-Signal AC Analysis

An `ACCoupled` solve section is an extension of a `Coupled` section with an extra set of parameters allowing small-signal AC analysis. [Table 148 on page 1225](#) describes these parameters. `ACCoupled` can only be used in mixed mode. [AC Simulation on page 960](#) provides technical background information for the method.

4: Performing Numeric Experiments

Small-Signal AC Analysis

The options `StartFrequency`, `EndFrequency`, `NumberOfPoints`, `Linear`, and `Decade` are used to select the frequencies at which the analysis is performed and the frequency distribution.

A `Node` list must be given. With it, for each frequency $\nu = \omega/2\pi$, the compact equivalent small-signal model is computed:

$$\delta I = Y\delta V = A\delta V + i\omega C\delta V \quad (23)$$

This model describes the response of the node currents, δI , to a small periodic voltage signal δV . The response matrix is stored in an `ACExtract` file specified in the `File` or `ACCoupled` statement (default is `extraction_ac_des.plt`). This file contains the frequency, voltages at the nodes, and the coefficients of the matrices A (denoted by `a`) and C (denoted by `c`). If the file name prefix `ACPlot` is specified, for each node in the `Node` list and for each frequency, Sentaurus Device writes a file with the (complex-valued) derivatives of the solution variables and currents with respect to voltage at the node.

The `Exclude` list is used to remove a set of circuit or physical devices from the AC analysis. Typically, the power supply is removed so as not to short-circuit the AC analysis, but the list can also be used to isolate a single device from a whole circuit.

NOTE The system analyzed consists of the equations specified in the body of the `ACCoupled` statement, without the instances removed by the `Exclude` list. The `Exclude` list only specifies instances, therefore, all equations of these instances are removed.

The `ACCompute` option controls AC or noise analysis performances within a `Quasistationary` command. The parameters in `ACCompute` are identical to the parameters in the `Plot` and `Save` commands (see [Table 155 on page 1232](#)), for example:

```
Quasistationary (...) {  
  ACCoupled (...  
    ACCompute (Time = (0; 0.01; 0.02; 0.03; 0.04; 0.05)  
              Time = (Range = (0.9 1.0) Intervals = 4)  
              Time = (Range = (0.1 0.2); Range = (0.7 0.8))  
              When (Node = in Voltage = 1.5))  
    {...}  
  }  
}
```

In this example, an AC analysis is performed only for the time points:

```
t = 0, 0.01, 0.02, 0.03, 0.04, 0.05  
t = 0.9, 0.925, 0.95, 0.975, 1.0
```

and for all time points in the intervals [0.1, 0.2] and [0.7, 0.8]. Additionally, an AC analysis is triggered whenever the voltage at the node `in` reaches the 1.5 V threshold.

If the AC or noise analysis is suppressed by the ACCompute option, an ACCoupled command behaves like an ordinary Coupled command inside a Quasistationary.

Example: AC Analysis of Simple Device

This example illustrates AC analysis of a simple device. A 1D resistor is connected to ground (through resistor to_ground) and to a voltage source drive at reverse bias of -3 V . After calculating the initial voltage point at -3 V , the left voltage is ramped to 1 V in 0.1 V increments. The AC parameters between nodes left and right are calculated at one frequency $f = 10^3\text{ Hz}$. The circuit element drive and to_ground are excluded from the AC calculation.

By including circuits, complete Bode plots can be performed:

```
Device "Res" {
  Electrode {{Name=anode   Voltage=-3 resist=1}
             {Name=cathode Voltage= 0 resist=1}}

  File {Grid = "resist.tdr"}

  Physics {Mobility (DopingDependence)
            Recombination (SRH(DopingDependence) Auger)}
}

System {
  "Res" "1d" (anode="left" cathode="right")
  Vsource_pset drive("left" "right"){dc = -3}
  Resistor_pset to_ground ("right" 0){resistance=1}
}

Math {
  Method=Blocked SubMethod=Super      # default solvers for Coupled
  ACMethod=Blocked ACSubMethod=Super  # default solvers for AC analysis
  NoAutomaticCircuitContact
}

Solve {
  Poisson
  Coupled {Poisson Electron Hole}
  Coupled {Poisson Electron Hole Contact Circuit}
  ACCoupled (StartFrequency=1e3 EndFrequency=1e6
             NumberOfPoints=4 Decade
             Iterations=0
             Node("left" "right")
             Exclude(drive to_ground)
             ACMethod=Blocked ACSubMethod("1d")=ParDiSo)
```

4: Performing Numeric Experiments

Harmonic Balance

```
{Poisson Electron Hole Contact Circuit}
Quasistationary (MaxStep=0.025 InitialStep=0.025
    Goal{Parameter=drive.dc Value=1}) {
    ACCoupled(StartFrequency=1e3 EndFrequency=1e6
        NumberOfPoints=4 Decade
        Node("left" "right")
        Exclude(drive to_ground)
        ACMethod=Blocked ACSubMethod("1d")=ParDiSo)
    {Poisson Electron Hole Circuit Contact}
}
}
```

Optical AC Analysis

Optical AC analysis calculates the quantum efficiency as a function of the frequency of the optical signal. This technique is based on the AC analysis technique, and provides real and imaginary parts of the quantum efficiency versus the frequency.

To start optical AC analysis, add the keyword `Optical` in an `ACCoupled` statement, for example:

```
ACCoupled ( StartFrequency=1.e4 EndFrequency=1.e9
    NumberOfPoints=31 Decade Node(a c)
    Optical Exclude(v1 v2) )
{ poisson electron hole }
```

For more details, see [Optical AC Analysis on page 568](#).

Harmonic Balance

Harmonic balance (HB) analysis is a frequency domain method to solve periodic or quasi-periodic, time-dependent problems. Compared to transient analysis (see [Transient Command on page 119](#)), HB is computationally more efficient for problems with time constants that differ by many orders of magnitude. Compared to AC analysis (see [Small-Signal AC Analysis on page 127](#)), the periodic excitation is not restricted to infinitesimally small amplitudes. An alternative to HB for the periodic case is cyclic analysis (see [Large-Signal Cyclic Analysis on page 124](#)).

NOTE Harmonic balance is not supported for traps (see [Chapter 17 on page 407](#)) or ferroelectrics (see [Chapter 29 on page 681](#)).

The time-dependent simulation problems take the form:

$$\frac{d}{dt}q[r, u(t, r)] + f[r, u(t, r), w(t, r)] = 0 \quad (24)$$

where f and q are nonlinear functions that describe the circuit and the devices, u is the vector of solution variables, and w is a time-dependent excitation.

Assuming w is quasi-periodic with respect to $\underline{f} = (\hat{f}_1, \dots, \hat{f}_M)^T$, the vector of the positive base frequencies $\hat{f}_m$, and $\underline{H} = (H_1, \dots, H_M)^T$, the vector of nonnegative maximal numbers of harmonics H_m , the solution u is approximated by a truncated Fourier series:

$$u(t) = U_0 + \sum_{-\underline{H} \leq \underline{h} \leq \underline{H}} U_{\underline{h}} \exp(i\omega_{\underline{h}} t) \quad (25)$$

where $\omega_{\underline{h}} = 2\pi \underline{h} \cdot \underline{f}$.

In Fourier space, [Eq. 24](#) becomes:

$$L(U) := i\Omega Q(U) + F(U) = 0 \quad (26)$$

where Ω is the frequency matrix, and F and Q are the Fourier series of f and q . The function L depends nonlinearly on U and, therefore, solving [Eq. 26](#) requires a nonlinear (Newton) iteration. For more details on the numerics of harmonic balance, see [Harmonic Balance Analysis on page 963](#).

Modes of Harmonic Balance Analysis

Sentaurus Device supports two different modes to perform harmonic balance simulations: the MDFT mode and the SDFT mode.

MDFT Mode

The MDFT mode is suitable for multitone analysis and one-tone analysis, and is enabled by the MDFT option in the HB section of the global Math section. It uses the multidimensional Fourier transformation (MDFT) to switch between the frequency and time domain of the system. This mode requires compact models defined by the compact model interface (CMI), which support assembly routines in the frequency domain, that is, the CMI-HB-MDFT function set as described in [Compact Models User Guide, CMI Models and Sentaurus Device Analysis Methods on page 119](#).

Sentaurus Device provides a basic set of compact models that support this functional behavior (refer to [Compact Models User Guide, CMI Models with Frequency-Domain Assembly on page 166](#)). Standard SPICE models are not supported in this mode.

SDFT Mode

The SDFT mode supports only one-tone HB analysis. The mixed-mode circuit may contain SPICE models and CMI models that do not provide the CMI-HB-MDFT functional set. It is used if the MDFT mode is disabled.

Performing Harmonic Balance Analysis

Harmonic balance simulations are enabled through the keyword `HBCoupled` in the `Solve` section. The syntax of `HBCoupled` is the same as for `Coupled` (see [Coupled Command on page 158](#)). In addition, `HBCoupled` supports the options summarized in [Table 154 on page 1231](#).

NOTE Not all `HBCoupled` options are supported by both modes.

The `Tone` option is mandatory, as no default values are provided.

Specifying several `Tone` options in MDFT mode enables multitone analysis. The base frequencies for the analysis can be given by explicit numeric constants or can refer to the frequencies of time-dependent sources in the `System` section.

An example of a two-tone analysis is:

```
Math {
  HB { MDFT }    * enable MDFT mode
  ...
}
System {
  ...
  sd_hb_vsource2_pset "va" ( na 0 )          * two-tone voltage source
  { dc = 1.0 freq = 1.e9 mag = 5.e-3 phase = -90.
    freq2 = 1.e3 mag2 = 1.e-3 phase2 = -90.}

  HBPlot "hbplot" ( v(na) i(va na) ... )    * HB circuit output
}
Solve {
  * solve the DC problem
  ...
  Coupled { Poisson Electron Hole }
```

```
* solve the HB problem
HBCoupled ( * two-tone analysis
  Tone ( Frequency = "va"."freq" NumberOfHarmonics = 5 )
  Tone ( Frequency = "va"."freq2" NumberOfHarmonics = 1 )
  Initialize = DCMODE
  Method = ILS
  GMRES ( Tolerance = 1.e-2 MaxIterations = 30 Restart = 30 )
) { Poisson Electron Hole }
}
```

The base frequencies $\hat{f}_1$ and $\hat{f}_2$ in this example are taken from the time-dependent voltage source va.

HBCoupled can be used as a top-level Solve statement, or within Plugin, QuasiStationary.

NOTE If a QuasiStationary controls other Solve statements besides a single HBCoupled, step reduction due to convergence problems works correctly only when no HBCoupled in the failing step has converged before the statement that caused the failure is run.

Solve Spectrum

The spectrum to be solved is determined by the specified tones. For different HBCoupled statements, the number of tones and their number of harmonics are allowed to change (where the m -th tone is identified with the m -th tone of the next spectrum, regardless of the value of its frequency, that is, the order of tones is significant).

For some applications, you may be interested only in a few spectrum components or you may want to reduce the computational burden in the Newton process. For such situations, you can reduce the solve spectrum (only for the MDFT mode) by applying spectrum truncation. This is performed by specifying a list of spectrum (multi-)indices by using SolveSpectrum in the HB section of the global Math section, and referencing to this spectrum in the HBCoupled statement, for example:

```
Math { ...
  HB { ... SolveSpectrum ( Name = "sp1" ){ (0 0) (1 0) (0 1) (2 1) (2 -1) } }
}
Solve { ...
  HBCoupled ( ... SolveSpectrum = "sp1" ) { ... }
}
```

where, for example, the multi-index (2 -1) corresponds to the intermodulation frequency $2\hat{f}_1 - 1\hat{f}_2$.

Convergence Parameters

The convergence behavior of HBCoupled can be analyzed and influenced independently of specifications for the convergence of Coupled solve statements. Some parameters can be set exclusively in the HB section of the global Math section (see [Table 175 on page 1255](#)), others can be set exclusively in the HBCoupled statement (see [Table 154 on page 1231](#)), and some can be set in both places.

CNormPrint is used to print the residuum, the update error, and the number of corrections (the number of grid points where a correction of computed sample points is necessary) in each Newton step for all solved equations of all instances. This information can be used to adjust the numeric convergence parameters.

The Derivative flag in HBCoupled overwrites the Derivative flag of the Math section and determines whether all derivatives are included in the HB Newton process.

With RhsScale and UpdateScale, you can scale the residuum and the update error of individual equations, respectively, which are used as Newton convergence criteria. This is often necessary for the electron and hole continuity equations, and the values differ for different applications and devices.

The parameters ValueMin and ValueVariation are used for positive solution variables to specify the allowed minimum value in the time domain, and the corresponding ratio of maximum and minimum values, respectively. The number of corrections (given by CNormPrint) indicates how many grid points violate the specified bounds.

Harmonic Balance Analysis Output

All frequency-domain output data refers to the one-sided Fourier series representation for real-valued quantities, that is, it refers to $\tilde{U}_{\underline{h}}$ of:

$$\begin{aligned} u(t) &= \tilde{U}_0 + \sum_{\underline{h} \in K^+} \operatorname{Re}(\tilde{U}_{\underline{h}} \exp(i\omega_{\underline{h}} t)) \\ &= \tilde{U}_0 + \sum_{\underline{h} \in K^+} \left\{ \operatorname{Re}(\tilde{U}_{\underline{h}}) \cos(\omega_{\underline{h}} t) - \operatorname{Im}(\tilde{U}_{\underline{h}}) \sin(\omega_{\underline{h}} t) \right\} \end{aligned} \quad (27)$$

where the sum is taken over all multi-indices $\underline{h}$, which results in a positive frequency $\omega_{\underline{h}} = \underline{h} \cdot \hat{\omega}$.

Device Instance Currents, Voltages, Temperatures, and Heat Components

Each converged HBCoupled plots the results (contact currents and voltages, temperatures, and heat components) both in the time domain and frequency domain into a separate file. The names of the files for the time domain contain a component Tdom; those for the frequency domain contain a component Hdom.

Circuit Currents and Voltages

Additionally, the keyword HBPLOT in the System section allows you to plot circuit quantities. The syntax of HBPLOT is identical to that of PLOT in the System section (see [System Plot on page 93](#)). In MDFT mode, each converged HBCoupled plots its results in a Tdom and an Hdom file as for device instances, while in SDFT mode, it will write four files containing all Fourier coefficients (files with the name component Fdomain), the time domain data (Tdomain), the harmonic magnitude (Hmag), and the harmonic phase (Hphase).

Solution Variables

Plotting solution variables is only supported for one-tone HB simulations. This is implicitly done if the HB section in the Math section is present. Sentaurus Device will plot the coefficients $\tilde{U}_i$ of [Eq. 27](#) as a real-valued vector with components $U_0 = \tilde{U}_0$, $U_{2h-1} = \text{Re}(\tilde{U}_h)$, and $U_{2h} = \text{Im}(\tilde{U}_h)$ (for $1 \leq h \leq 3$), with names composed of a prefix HB, a suffix _C<i> with component <i>, and the names of the solution variables. Additionally, the magnitude and phase of $\tilde{U}_h$ (for $0 \leq h \leq 3$) are plotted, with the prefixes Mag and Phase to the variable names and suffixes _C<h>.

Application Notes

- Convergence: Typically, the nonlinear convergence improves with an increasing number of harmonics H for one-tone HB simulations.
- Linear solvers: Benefiting both memory requirements and simulation time, you can use the iterative linear solver GMRES for most simulations. Only for very small problems, in terms of grid size and number of harmonics, will the direct solver method be sufficient.

References

- [1] R. J. G. Goossens *et al.*, “An Automatic Biasing Scheme for Tracing Arbitrarily Shaped I-V Curves,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 13, no. 3, pp. 310–317, 1994.
- [2] K. S. Kundert, J. K. White, and A. Sangiovanni-Vincentelli, *Steady-State Methods for Simulating Analog and Microwave Circuits*, Boston: Kluwer Academic Publishers, 1990.
- [3] Y. Takahashi, K. Kunihiro, and Y. Ohno, “Two-Dimensional Cyclic Bias Device Simulator and Its Application to GaAs HJFET Pulse Pattern Effect Analysis,” *IEICE Transactions on Electronics*, vol. E82-C, no. 6, pp. 917–923, 1999.

CHAPTER 5 Simulation Results

This chapter describes the output of Sentaurus Device, which contains the simulation results.

Sentaurus Device provides several forms of output. Most importantly, the current file contains the terminal characteristics obtained during the numeric experiment. Plot files allow you to visualize device internal quantities, and thereby provide information not available in real experiments. Other output, such as the log file, allows you to investigate the simulation procedure itself and provides an important tool to understand and resolve problems with the simulation itself.

This chapter only describes the most important output that Sentaurus Device provides. Many more specific output files exist. They are described in context in other parts of the user guide.

Current File

When to Write to the Current File

By default, currents are output after each iteration in a `Plugin`, `Quasistationary`, or `Transient` command. This behavior can be modified by a `CurrentPlot` statement in the body of these commands. The `CurrentPlot` statement in the `Solve` section provides full control over the plotting of device currents and circuit currents. If a `CurrentPlot` statement is present, it determines exactly which points are written to the current file. Sentaurus Device can still perform computations for intermediate points, but they are not written to the file.

NOTE Do not confuse the `CurrentPlot` statement in the `Solve` section with the `CurrentPlot` section as described in [Tracking Additional Data in the Current File on page 140](#).

The syntax of the `CurrentPlot` statement is:

```
CurrentPlot (<parameters-opt>) {<system-opt>}
```

Both `<parameters-opt>` and `<system-opt>` are optional and can be omitted. `<parameters-opt>` is a space-separated list, which can consist of the following entries:

```
Time = (<entry> ; <entry> ; <entry> ;)
```

5: Simulation Results

Current File

The list of time entries enumerates the times for which a current plot is requested. The entries are separated by semicolons. A time entry can have these forms:

floating point number

The time value for which a current plot is requested.

Range = (a b)

This option specifies a free plot range between a and b. All the time points in this range are plotted.

Range = (a b) Intervals = n

This option specifies n intervals in the range between a and b. In other words, these plot points are generated:

$$t = a, t = a + \frac{b-a}{n}, \dots, t = b - \frac{b-a}{n}, t = b \quad (28)$$

Iterations = (<integer>; <integer>; <integer>;)

The list of integers specifies the iterations for which a plot is required. This option is available for the `Plugin` command.

IterationStep = <integer>

This option requests a current plot every n iterations. It is available for the `Plugin` command.

When (<when condition>)

A `When` option can be used to request a current plot whenever a condition has been met. This option works in the same way as in a `Plot` or `Save` command (see [Table 155 on page 1232](#)).

<system-opt> is a space-separated list of devices. If <system-opt> is not present, Sentaurs Device plots all device currents and the circuit (in mixed-mode simulations). If <system-opt> is present, only the currents of the given devices are plotted. The keyword `Circuit` can be used to request a circuit plot.

Example: CurrentPlot Statements

A `CurrentPlot` statement by itself creates a current plot for each iteration:

```
Quasistationary (InitialStep=0.2 MinStep=0.2 MaxStep=0.2
Goal { Name="drain" Voltage=0.5 })
  { Coupled { Poisson Electron Hole }
    CurrentPlot }
```


If no current plots are required, a current plot for the ‘impossible’ time $t = -1$ can be specified:

```
Quasistationary (InitialStep=0.2 MinStep=0.2 MaxStep=0.2
Goal { Name ="drain" Voltage=0.5 })
  { Coupled { Poisson Electron Hole }
    CurrentPlot ( Time = (-1)) }
```

In this example, the currents of the device nmos are plotted for $t = 0$, $t = 10^{-8}$, and $t = 10^{-7}$:

```
Transient ( MaxStep=1e-8 InitialTime=0 FinalTime=1e-6 )
  { Coupled { Poisson Circuit }
    CurrentPlot ( Time = (0 ; 1e-8; 1e-7)) { nmos } }
```

This CurrentPlot statement produces 11 equidistant plot points in the interval 0, 10^{-5} :

```
Transient ( MaxStep = 1e-8 InitialTime=0 FinalTime=1e-5 )
  { Coupled { Poisson Circuit }
    CurrentPlot ( Time = (range = (0 1e-5) intervals = 10)) }
```

In this example, a current plot for iteration 1, 2, 3, and for every tenth iteration is specified:

```
Plugin { Poisson Electron Hole
  CurrentPlot ( Iterations = (1; 2; 3) IterationStep = 10 ) }
```

A CurrentPlot statement can also appear at the top level in the Solve section. In this case, the currents are plotted when the flow of control reaches the statement.

A CurrentPlot statement is also recognized within a Continuation command. In this case, the time in the CurrentPlot statement corresponds to the arc length in the Continuation command. However, only free plot ranges are supported.

NewCurrentPrefix Statement

By default, Sentaurus Device saves all current plots in one file (as defined by the variable Current in the File section). This behavior can be modified by the NewCurrentPrefix statement in the Solve section:

```
NewCurrentPrefix = prefix
```

This statement appends the given prefix to the default, current file name, and all subsequent Plot statements are directed to the new file. Multiple NewCurrentPrefix statements can appear in the Solve section, for example:

```
Solve {
  Circuit
```

5: Simulation Results

Current File

```
Poisson
NewCurrentPrefix = "pre1"
Coupled {Poisson Electron Hole Contact Circuit}
NewCurrentPrefix = "pre2"
Transient (
    MaxStep= 2.5e-6 InitialStep=1.0e-6
    InitialTime=0.0 FinalTime=0.0001
    Plot {range=(10e-6,40e-6) Intervals=10}
)
{Coupled {Poisson Electron Hole Contact Circuit}}
```

In this example, the current files specified in the `File` and `System` sections contain the results of the `Circuit` and `Poisson` solves. The results of the `Coupled` solution are saved in a new current file with the same name but prefixed with `pre1`. The last current file contains the results of the `Transient` solve with the prefix `pre2`.

NOTE The file names for current plots defined in the `System` section, and the plot files of AC analyses are also modified by a `NewCurrentPrefix` statement.

Tracking Additional Data in the Current File

The `CurrentPlot` section on the top level of the command file is used to include selected parameter values and mesh data into the current plot file (.plt). The same mesh data can be selected as in the `Plot` section (see [Appendix F on page 1185](#)).

Data can be plotted according to node numbers or coordinates. The node numbers are obtained by probing the mesh using Tecplot SV (see [Tecplot SV User Guide, Environment Variables on page 48](#)).

Node numbers are given as plain integers. A coordinate is given as one to three (depending on device dimensions) numbers in parentheses. The parentheses distinguish coordinates from node numbers. When plotting according to coordinates, the plotted values are interpolated as required.

Furthermore, it is possible to output averages, integrals, maximum, and minimum of quantities over specified domains. To do this, specify the keyword `Average`, `Integrate`, `Maximum`, or `Minimum`, respectively, followed by the specification of the domain in parentheses. A domain specification consists of any number of the following:

- Region specification: `Region=<regionname>`
- Material specification: `Material=<materialname>`
- Region interface specification: `RegionInterface=<regioninterfacename>`

- Material interface specification: `MaterialInterface=<materialinterfacename>`
- Any of the keywords `Semiconductor`, `Insulator`, and `Everywhere`, which match all semiconductor regions, all insulator regions, or the entire device, respectively
- Window specification: `Window[(x1 y1 z1) (x2 y2 z2)]`
- Well specification: `DopingWell(x1 y1 z1)`

The average, integral, maximum, and minimum are applied to all of the specified parts of the device. Multiple specifications of the same part of the device are insignificant. In addition, `Name=<plotname>` is used to specify a name under which the average, integral, maximum, and minimum are written to the `.plt` file. (By default, the name is automatically obtained from a concatenation of the names in the domain specification, which yields impractically long names for complicated specifications.)

For maximum and minimum, Sentaurus Device can write coordinates where the maximum and minimum occur to the `.plt` file. In the case of average and integral, Sentaurus Device can write the coordinates ($\langle x \rangle$, $\langle y \rangle$, $\langle z \rangle$) of the centroid of a data field f defined:

$$\langle x \rangle = \frac{\int x f(x, y, z) dV}{\int f(x, y, z) dV} \quad (29)$$

where the integration covers the specified domain. The values of $\langle y \rangle$ and $\langle z \rangle$ are defined similarly. Output of coordinates is activated by adding the keyword `Coordinates` in the parentheses where the domain is specified.

The average, integral, maximum, and minimum can be confined to a window by using the window specification. A one-dimensional, 2D, or 3D window is defined by the coordinates (in micrometers) of two opposite corners of the window. In addition, it is possible to confine the domain in a well using the well specification. In this case, the well is defined by the coordinates of a point inside the well. When a list of domains is specified in addition to the window or well, the domains are neglected if the window or well is a valid one.

The length unit for integration and the number of digits used for names in the current plot file can be specified in the Math section (see [CurrentPlot Options on page 143](#) for details).

Parameters from the parameter file of Sentaurus Device can also be added to the current plot file. The general specification looks like:

```
[ Material = <material> | MaterialInterface = <interface> |
  Region = <region> | RegionInterface = <interface> ]
Model = <model> Parameter = <parameter>
```

Specifying the location (material, material interface, region, or region interface) is optional. However, the model name and parameter name must always be present. [Ramping Physical Parameter Values on page 109](#) describes how model names and parameter names can be

5: Simulation Results

Current File

determined. Finally, Sentaurus Device also provides a current plot PMI (see [Current Plot File of Sentaurus Device on page 1132](#)).

NOTE Do not confuse the `CurrentPlot` section with the `CurrentPlot` statement in the `Solve` section introduced in [When to Write to the Current File on page 137](#).

Example: Node Numbers

Consider this device declaration in a Sentaurus Device command file:

```
Device CAP {  
    ...  
    CurrentPlot { Potential (7, 8, 9)  
                  ElectricField (7, 8, 9) }  
}
```

In addition to the usual contact currents, the current file of the device `CAP` contains the electrostatic potential and electric field for the mesh vertices 7, 8, and 9.

Example: Mixed Mode

In mixed-mode simulations, the `CurrentPlot` section can appear in the body of a physical device within the `System` section (it is also possible to have a global `CurrentPlot` section), for example:

```
System {  
    Set (gnd = 0)  
    CAP Cm (Top=node2 Bot=gnd) { CurrentPlot { Potential (7, 8, 9) } }  
    ...  
}
```

Example: Advanced Options

This example is a 2D device that uses the more advanced `CurrentPlot` features:

```
CurrentPlot {  
    eDensity (0 1) * plot electron density at nodes 0 and 1  
    hDensity((0 1)) * hole density at position (0um, 1um)  
    ElectricField/Vector((0 1)) * Electric Field Vector  
    Potential (  
        (0.1 -0.2) * coordinates need not be integers  
        Average(Region="Channel") * average over a region  
        Average(Everywhere) * average over entire device  
        Maximum(Material="Oxide") * Maximum in a material  
        Maximum(Semiconductor) * in all semiconductors  
    )  
}
```

```

    * minimum in a material and a region, output under the name "x":
    Minimum(Name="x" Material="Oxide" Region="Channel")
  )
  eDensity(
    * average and coordinates of centroid
    Average(Semiconductor Coordinates)
    * maximum and coordinates of maximum in well
    Maximum(DopingWell(-0.1 0.3) Coordinates)
    * integral over the semiconductor regions
    Integrate(Semiconductor)
  )
  SpaceCharge(
    * maximum over 2D window
    Maximum(Window[(-0.2 0) (0.2 0.2)])
    * integral over well
    Integrate( DopingWell(-0.1 0.3) )
  )
}

```

Example: Plotting Parameter Values

The following example adds five curves to the current plot file:

```

CurrentPlot {
  Model = DeviceTemperature Parameter = "Temperature"
  Material = Silicon Model = Epsilon Parameter = epsilon
  MaterialInterface = "AlGaAs/InGaAs"
  Model = "SurfaceRecombination" Parameter = "S0_e"
  Region = "bulk" Model = LatticeHeatCapacity Parameter = cv
  RegionInterface = "Region.0/Region.1"
  Model = "SurfaceRecombination" Parameter = "S0_h"
}

```

CurrentPlot Options

The length unit used during a current plot integration (see [Tracking Additional Data in the Current File on page 140](#)) can be selected as follows:

```

Math {
  CurrentPlot (IntegrationUnit = um)
}

```

The possible choices are cm (centimeters) and um (micrometers). The default is IntegrationUnit=um. This option is useful when quantities such as densities (unit of cm^{-3}) are integrated. In a 2D simulation, the unit of the integral is either $\mu\text{m}^2\text{cm}^{-3}$ (IntegrationUnit=um) or cm^{-1} (IntegrationUnit=cm).

5: Simulation Results

Device Plots

The number of digits in the names of quantities in the current plot file can be specified as follows:

```
Math {  
    CurrentPlot (Digits = 6)  
}
```

The default is `Digits=6`.

This option is useful when the names of two quantities become equal due to rounding. In the following example, you want to monitor the valence band energy in two points:

```
CurrentPlot {  
    ValenceBandEnergy ((5.0, 199.011111) (5.0, 199.011112))  
}
```

However, with the default setting of `Digits=6`, both quantities are assigned the same name due to rounding:

```
Pos(5,199.011) ValenceBandEnergy
```

By increasing the number of digits, the names in the current plot file become distinct.

Specifying `Digits` affects the names of the following quantities in the current plot file:

- Coordinates
- Well specification
- Window specification

Device Plots

Device plots show spatial-dependent datasets in the device. They provide a view of the inside of the device.

What to Plot

The `Plot` section specifies the data that is saved at the end of the simulation to the `Plot` file specified in the `File` section or by the `Plot` command in the `Solve` section. See [Table 263 on page 1320](#) for all possible plot options.

Vector data can be plotted by appending `/Vector` to the corresponding keyword, for example:

```
Plot {
    ElectricField/Vector
}
```

Element-based scalar data can be plotted by appending `/Element` to the corresponding keyword, for example:

```
Plot {
    eMobility/Element
}
```

Some quantities (such as the carrier densities) are put into the `Plot` file even when they are not listed in the `Plot` section. The keyword `PlotExplicit` in the global `Math` section suppresses this behavior. By default, the plot file also contains additional information required by `Load` to restart a simulation (see [Save and Load on page 174](#)). To suppress writing this additional information, specify `-PlotLoadable` in the global `Math` section.

You can specify the keyword `DatasetsFromGrid` to copy quantities from the TDR grid file directly to the plot file. You can copy all datasets by specifying `DatasetsFromGrid` by itself:

```
Plot {
    DatasetsFromGrid
}
```

Alternatively, you can copy selected quantities by listing their names:

```
Plot {
    DatasetsFromGrid (dataset1 dataset2 dataset3)
}
```

A dataset is only copied from the TDR grid file if it is not computed by Sentaurus Device. Otherwise, the dataset computed by Sentaurus Device takes precedence.

NOTE The keyword `DatasetsFromGrid` only copies entire datasets from the TDR grid file to the plot file. You cannot copy partial datasets, for example, the components of tensor and vector datasets.

When to Plot

The simplest way to create a device plot is to define `Plot` in the `File` section. By default, the output to the `Plot` file will occur at the end of the simulation only.

The commands for `Quasistationary` and `Transient` analysis support an option `Plot` that allows you to write plots while these commands are executing. The plot file name is derived

5: Simulation Results

Device Plots

from the one specified with `Plot` in the `File` section. See [Saving and Plotting During a Quasistationary on page 112](#) and [Transient Command on page 119](#) for the syntax.

The most flexible way to create device plots is through the `Plot` statement in the `Solve` section. It provides full control of when to plot and limited control of the file names. The `Plot` statement can be used at any level of the `Solve` section. The command takes the form:

```
Plot (<parameters-opt>) <system-opt>
```

If no `<system-opt>` is specified, all physical devices and circuits are plotted. Use `<system-opt>` to specify an optional list of devices delimited by braces (see [Table 147 on page 1224](#)).

If `<parameters-opt>` is omitted, defaults are used. The keywords `Loadable` and `Explicit` provide plot-specific overrides for `PlotLoadable` and `PlotExplicit` in the global `Math` section, see [What to Plot on page 144](#). For a summary of all options, see [Table 155 on page 1232](#).

For example:

```
Solve {
  Plugin {
    Poisson
    Plot ( FilePrefix = "output/poisson")
    Coupled { Poisson Electron Hole }
    Plot (FilePrefix = "output/electric" noOverwrite)
  }
  Transient {
    Coupled { Poisson Electron Hole Temperature }
    Plot ( FilePrefix = "output/trans"
      Time = ( range = (0 1) ;
        range = (0 1) intervals = 4 ; 0.7 ;
        range = (1.e-3 1.e-1) intervals = 2 decade )
      NoOverwrite )
  }
  ...
}
```

The first `Plot` statement in `Plugin` writes (after the computation of the Poisson equation) to a file named `output/poisson_des.tdr`. The second `Plot` statement in `Plugin` writes to a file called `output/electric_0000_des.tdr` and increases the internal number for each call.

The `Plot` statement in the transient specifies three different types of time entries (separated by semicolons). The first entry indicates all the times within this range when a plot file must be written. The second time entry forces the transient simulation to compute solutions for the given times. In this example, the given times are 0.25, 0.5, 0.75, and 1.0. The third entry is for

the single time of 0.7. The fourth entry triggers the plotting at time points given by subdividing the range on the logarithmic scale, resulting for this example in plots at 1.e-3, 1.e-2, and 1.e-1.

Snapshots

Sentaurus Device offers the possibility to save snapshots interactively during a simulation. This can be undertaken by sending a POSIX signal to the Sentaurus Device process. Depending on whether `Plot` or `Save` or both is specified in the `File` section, the signal invokes a request to write a plot (`.tdr`) file or a save (`.sav`) file after the actual time step is finished.

The following signals are supported to save snapshots:

- **USR1:** The occurrence of the USR1 signal initiates Sentaurus Device to write a plot file or a save file after the simulation has finished its actual time step. The simulation will continue afterwards.
- **INT:** By default, sending the INT signal causes a process to exit. This behaviour changes if `Interrupt=BreakRequest` or `Interrupt=PlotRequest` is specified in the `Math` section (see [Table 170 on page 1243](#)). In both cases, the occurrence of the INT signal initiates Sentaurus Device to write a plot file or a save file. `Interrupt=BreakRequest` causes Sentaurus Device to abort the actual solve statement after the snapshot is saved. If `Interrupt=PlotRequest` is specified, the simulation continues.

A signal can be sent to a process by invoking the `kill` command (see the corresponding man page of your operating system). The process ID can be extracted from the `.log` file.

Interface Plots

Data fields defined on interfaces can be plotted by using the modifier `/RegionInterface`:

```
Plot {
    HotElectronInj/RegionInterface
}
```

The following fields are available for interface plots:

```
SurfaceRecombination
HotElectronInj
HotHoleInj
```

NOTE These fields can also be plotted on regions by omitting the qualifier `/RegionInterface`. However, they are zero inside bulk regions, and nonzero values are only produced for vertices along interfaces.

5: Simulation Results

Log File

Interface plots are only generated if interface regions appear in the grid file. For TDR files, the following command is available:

```
snmesh -u -AI input.tdr
```

In 2D, interface plots can be visualized as follows in Tecplot SV:

1. In Tecplot SV, **Plot > Zone/Mapping Style** for the required interfaces.
2. In the Zone Style dialog box:
 - a) Click **Edge** and set Show Edges to No.
 - b) Click **Mesh**, and set Mesh Show to Yes, set Mesh Color to Multi C1, and increase Line Thck as required to improve the visibility.
3. Click **Close**.

The mesh color will reflect the values of the field.

In 3D, interface plots can be visualized as follows in Tecplot SV:

1. In Tecplot SV, **Plot > Zone/Mapping Style** for the required interfaces.
2. In the Zone Style dialog box, click **Contour** and set Cont Show to Yes.
3. Click **Close**.

The surface color will reflect the values of the field.

Log File

The name of the log file is specified by **Output** in the **File** section. The log file contains a copy of the messages that Sentaurus Device prints while a simulation runs. The log file contains a variety of information, including:

- Technical information such as the version of Sentaurus Device that is running, the machine where it is running, and the process ID.
- Confirmation as to which structure is simulated, which physical models have been selected, and which parameters are used.
- Detailed information about how the simulation proceeds, including timing and convergence information.
- Warning messages.

The log file is of minor interest as long as your simulations are set up well, and no convergence problems occur. It is an indispensable diagnostic tool during simulation setup, or when convergence problems occur.

Extraction File

Sentaurus Device supports a special-purpose file format (extension `.xt r`) for the extraction of MOSFET compact model parameters.

Extraction File Format

The extraction file consists of the following parts:

1. A header identifying the file format.
2. A section containing the process information of the MOSFET such as channel length or channel width.
3. A collection of curves containing the values of a dependent variable as a function of an independent variable, for example, drain current versus gate voltage in an I_d - V_g ramp. In addition, each curve contains the corresponding bias conditions, for example, V_b , V_d , and V_s in an I_d - V_g ramp.

A sample extraction file is:

```
# Synopsys Extraction File Format Version 1.1
# Copyright (C) 2008 Synopsys, Inc.
# All rights reserved.

$ Process
AD 31.50f
AS 31.50f
D NMOS
L 90.00n
NF 1.000
NRD 0.000
NRS 0.000
PD 880.0n
PS 880.0n
SA 500.0n
SB 500.0n
SC 0.000
SCA 0.000
SCB 0.000
SCC 0.000
SD 0.000
W 90.00n
```

5: Simulation Results

Extraction File

```
$ Data
Curve: Id_Vg
Bias : Vb = 0 , Vd = 0.5 , Vs = 0
1.60000000000000E+00    7.95091490486384E-05
1.63750000000000E+00    8.27433425335473E-05
1.67500000000000E+00    8.59289596870642E-05
1.71250000000000E+00    8.90665609661286E-05
1.75000000000000E+00    9.21567959785304E-05

Curve: Is_Vg
Bias : Vb = 0 , Vd = 0.5 , Vs = 0
1.60000000000000E+00    -7.95091490484525E-05
1.63750000000000E+00    -8.27433425333610E-05
1.67500000000000E+00    -8.59289596868774E-05
1.71250000000000E+00    -8.90665609659414E-05
1.75000000000000E+00    -9.21567959783428E-05
```

Analysis Modes

Table 15 shows the supported analysis modes.

Table 15 Analysis modes

Analysis	Curve	Description
DC	Ii_Vj	The i -th terminal current versus j -th terminal voltage. $i,j=b$ (bulk), d (drain), g (gate), or s (source) Bias conditions: all terminal voltages other than j -th terminal voltage.
AC	Ai_j_Vk	Admittance A_{ij} versus k -th terminal voltage. $i,j,k=b$ (bulk), d (drain), g (gate), or s (source) Bias conditions: all terminal voltages other than k -th terminal voltage and frequency.
	Ai_j_F	Admittance A_{ij} versus frequency. $i,j=b$ (bulk), d (drain), g (gate), or s (source) Bias conditions: V_b , V_d , V_g , V_s .
	Ci_j_Vk	Capacitance C_{ij} versus k -th terminal voltage. where $i,j,k=b$ (bulk), d (drain), g (gate), or s (source) Bias conditions: all terminal voltages other than k -th terminal voltage and frequency.
	Ci_j_F	Capacitance C_{ij} versus frequency. $i,j=b$ (bulk), d (drain), g (gate), or s (source) Bias conditions: V_b , V_d , V_g , V_s .

Table 15 Analysis modes

Analysis	Curve	Description
Noise	noise_Vi_Vj	Autocorrelation noise voltage spectral density (NVSD) for electrode <i>i</i> versus <i>j</i> -th terminal voltage. <i>i,j</i> =b (bulk), d (drain), g (gate), or S (source) Bias conditions: all terminal voltages other than <i>j</i> -th terminal voltage and frequency.
	noise_Vi_F	Autocorrelation noise voltage spectral density (NVSD) for electrode <i>i</i> versus frequency. <i>i</i> =b (bulk), d (drain), g (gate), or S (source) Bias conditions: V_b , V_d , V_g , V_s .
	noise_Ii_Vj	Autocorrelation noise current spectral density (NISD) for electrode <i>i</i> versus <i>j</i> -th terminal voltage. <i>i,j</i> =b (bulk), d (drain), g (gate), or S (source) Bias conditions: all terminal voltages other than <i>j</i> -th terminal voltage and frequency.
	noise_Ii_F	Autocorrelation noise current spectral density (NISD) for electrode <i>i</i> versus frequency. <i>i</i> =b (bulk), d (drain), g (gate), or S (source) Bias conditions: V_b , V_d , V_g , V_s .

File Section

The name of the extraction file can be specified in the File section of the Sentaurus Device command file:

```
File {
    Extraction = "mosfet_des.xtr"
    ...
}
```

The default file name is `extraction_des.xtr`.

Electrode Section

Sentaurus Device automatically recognizes the four electrodes of a MOSFET if they are called "bulk", "drain", "gate", and "source", or "b", "d", "g", and "s" (case insensitive). For other contact names, it is possible to specify the mapping in the Electrode section:

```
Electrode {
    { Name = "contact_bulk"   Voltage = 0.0 Extraction {bulk}   }
    { Name = "contact_drain"  Voltage = 0.0 Extraction {drain}  }
    { Name = "contact_gate"   Voltage = 0.0 Extraction {gate}   }
    { Name = "contact_source" Voltage = 0.0 Extraction {source} }
}
```

5: Simulation Results

Extraction File

All four MOSFET electrodes (bulk, drain, gate, and source) must be present. Additional electrodes are allowed, but they will be ignored for extraction purposes.

Extraction Section

The process parameters can be part of the Sentaurus Device grid/doping file. It is also possible to specify the same information in an `Extraction` section:

```
Extraction {  
  AD 31.50f  
  AS 31.50f  
  D NMOS  
  L 90.00n  
  NF 1.000  
  NRD 0.000  
  NRS 0.000  
  PD 880.0n  
  PS 880.0n  
  SA 500.0n  
  SB 500.0n  
  SC 0.000  
  SCA 0.000  
  SCB 0.000  
  SCC 0.000  
  SD 0.000  
  W 90.00n  
}
```

Each line in the `Extraction` section consists of a name–value pair. All entries are copied to the extraction file as is.

NOTE The process parameters in the `Extraction` section of the Sentaurus Device command file take precedence over the information in the grid/doping file.

Solve Section

The `Quasistationary` command in the `Solve` section supports an `Extraction` option to request voltage-dependent extraction curves. Multiple curves can be generated during a single ramp. No extraction curves are produced if the `Extraction` option is missing:

```
Solve {  
  # ramp contributes to extraction  
  Quasistationary (  
    Extraction
```

```

    Goal { Name="contact_gate" Voltage=1.5 }
    Extraction { IdVg IsVg ... }
  )
  { Coupled { Poisson Electron }
    CurrentPlot (Time = (Range=(0 1) Intervals=10))
  }

# ramp does not contribute to extraction
Quasistationary (
  Goal { Name="contact_base" Voltage=0.5 }
)
  { Coupled { Poisson Electron } }

# ramp contributes to extraction
Quasistationary (
  Goal { Name="contact_drain" Voltage=1.5 }
  Extraction { IdVd IbVd ... }
)
  { Coupled { Poisson Electron } }
}

```

AC and noise simulations must be performed in mixed mode. Only one physical device is supported in the circuit. Voltage-dependent curves are specified as an option to the `Quasistationary` statement; whereas, frequency-dependent curves appear within the `ACCoupled` statement:

```

Solve {
  Quasistationary (
    ...
    Extraction { Ads_Vg Agg_Vg Cgd_Vg
                 noise_Id_Vg noise_Ig_Vg }
  )
  { ACCoupled (
    ...
    Extraction { Add_F Cgd_F Csd_F
                 noise_Vd_F noise_Vg_F noise_Is_F }
  )
  { Poisson Electron }
}
}

```

The recognized curves in the `Extraction` option are shown in [Table 15 on page 150](#).

5: Simulation Results

Extraction File

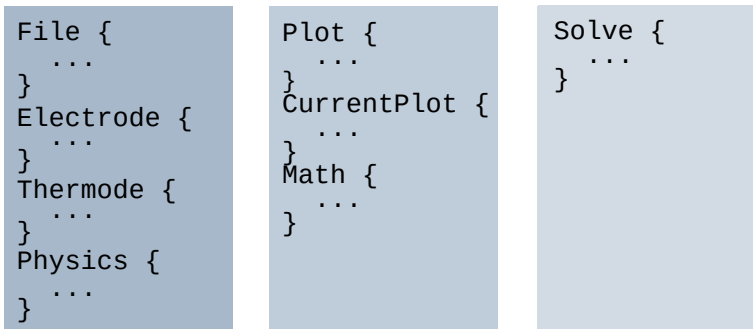
CHAPTER 6 Numeric and Software-related Issues

This chapter discusses technical issues of simulation that do not have real-world equivalents.

To successfully perform an experiment, apart from understanding the device under investigation, the experimenter must understand, control, and skillfully use the measurement equipment. The same holds for numeric *experiments*. This chapter describes the *equipment* available in Sentaurus Device for numeric experiments: linear and nonlinear solvers, control of accuracy, convergence monitors, and other software-related facilities. Additional background information on numerics is available in [Chapter 40 on page 949](#).

Structure of Command File

The Sentaurus Device command file is divided into sections that are defined by a keyword and braces (see [Figure 23](#)). A device is defined by the File, Electrode, Thermode, and Physics sections. The solve methods are defined by the Math and Solve sections. Two sections are used in mixed-mode circuit and device simulation, see [Chapter 3 on page 79](#). This part of the manual concentrates on the input to define single device simulations.



```
File {  
  ...  
}  
Electrode {  
  ...  
}  
Thermode {  
  ...  
}  
Physics {  
  ...  
}  
  
Plot {  
  ...  
} CurrentPlot {  
  ...  
}  
Math {  
  ...  
}  
  
Solve {  
  ...  
}
```

Figure 23 Different sections of a Sentaurus Device command file

Inserting Files

An insert directive is available in the command file of Sentaurus Device to incorporate other files:

```
Insert = "filename"
```

This directive can appear at the top level in the command file, or inside any of the following sections:

- CurrentPlot
- Device
- Electrode
- File
- Math
- MonteCarlo
- NoisePlot
- NonlocalPlot
- Physics
- Plot
- RayTraceBC
- Solve
- System
- Thermode

The following search strategy is used to locate a file:

- The current directory is checked first (highest priority).
- If the environment variable SDEVICEDB exists, the directory associated with the variable is checked (medium priority).
- Finally, the \$STR00T/tcad/\$STRELEASE/lib/sdevice/MaterialDB directory is checked (lowest priority).

NOTE The insert directive is also available for parameter files (see [Physical Model Parameters on page 60](#)).

Solve Section: How the Simulation Proceeds

The Solve section is the only section in which the order of commands are important. It consists of a series of simulation commands to be performed that are activated sequentially, according to the order of commands in the command file. Many Solve commands are high-level commands that have lower level commands as parameters. [Figure 24](#) shows an example of these different command levels:

- The Coupled command (the base command) is used to solve a set of equations.
- The Plugin command is used to iterate between a number of coupled equations.
- The Quasistationary command is used to ramp a solution from one boundary condition to another.
- The Transient command is used to run a transient simulation.

Furthermore, small-signal AC analysis can be performed with the ACCoupled command. An advanced ramping by continuation method can be performed with the command Continuation. (The ACCoupled and Continuation commands are presented in [Small-Signal AC Analysis on page 127](#) and [Continuation Command on page 116](#), respectively.)

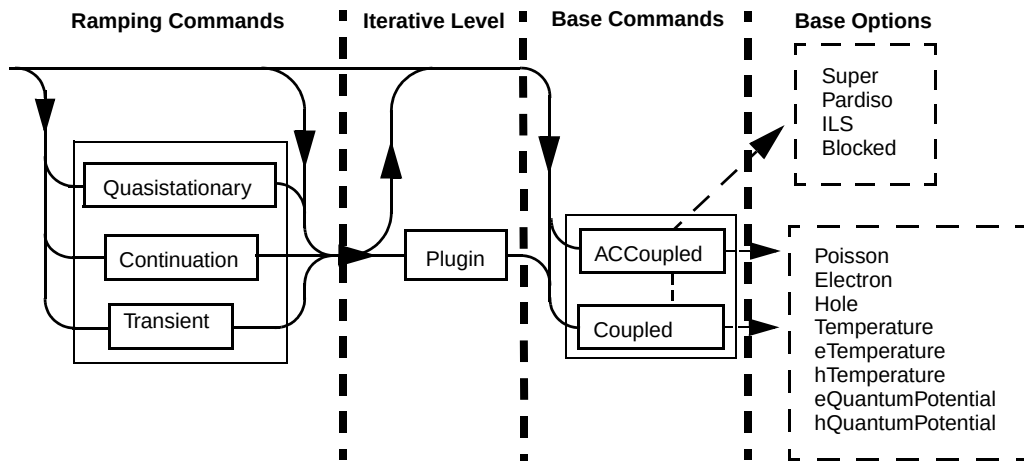


Figure 24 Different levels of Solve commands

Nonlinear Iterations

Coupled Command

The `Coupled` command activates a Newton-like solver over a set of equations. Available equations include the Poisson equation, continuity equations, and the different thermal and energy equations. The syntax of the `Coupled` command is:

```
Coupled ( <optional parameters> ){ <equation> }
```

or

```
<equation>
```

This last form uses only the keyword `equation`, which is equivalent to a `coupled` with default parameters and the single equation. For example, if the following command is used:

```
Coupled { Poisson Electron }
```

the electrostatic potential and electron density are computed from the resolution of the Poisson equation and electron continuity equation (using the default parameters).

If the following command is used:

```
Poisson
```

only the electrostatic potential is computed using the Poisson equation.

The `Coupled` command is based on a Newton solver. This is an iterative algorithm in which a linear system is solved at each step simulation. Parameters of the command determine:

- The maximum number of iterations allowed.
- The desired precision of the solution.
- The linear solver that must be used.
- Whether the solution is allowed to worsen over a number of iterations.

These parameters and others are summarized in [Table 150 on page 1227](#).

`Coupled` is controlled by both an absolute criterion and a relative error criterion. The relative error control can be specified with the optional parameter `Digits`. The absolute error control can be specified in the `Math` section (see [Coupled Error Control on page 159](#)).

The following example limits the previous `Coupled { Poisson Electron }` example to ten iterations and uses the ILS linear solver:

```
Coupled ( Iterations=10 Method=ILS ) { Poisson Electron }
```

NOTE Use a large number of iterations when the coupled iteration is not inside a ramping process. In this context, allow the Newton algorithm to proceed as far as possible. Inside a ramping command (for example, `Quasistationary`, `Transient`), the maximum number of iterations should be limited to approximately ten because if the Newton process does not converge rapidly, it is preferable to try again with a smaller step size than to pursue an iterative process that is not likely to converge.

The best linear method to use in a coupled iteration depends on the type and size of the problem solved. [Table 176 on page 1256](#) lists the solvers and the size of the problems for which they are designed.

Coupled Error Control

The `Coupled` command is sensitive to the `Math` parameters:

- `Iterations`
- `RhsAndUpdateConvergence`
- `RhsFactor`, `RhsMax`, `RhsMin`
- `Digits`
- `Error`
- `RelErrControl`
- `ErrRef`
- `UpdateIncrease`, `UpdateMax`

These parameters determine when the `Coupled` statement diverges or converges.

`Iterations` in the global `Math` section sets the defaults of the equivalent parameters of the `Coupled` command. `Iterations` limits the number of Newton iterations. If the equation being solved converges quadratically, the number of iterations can become larger. For `Iterations=0`, only one iteration is performed.

`RhsAndUpdateConvergence` requires that both the RHS and the update error have converged (are small enough) such that the Newton is considered to have converged. By default, only either the RHS or the update error has to converge.

`RhsMin` and `RhsFactor` add control to the size of the right-hand side (RHS, that is, the residual of the equations). `RhsMin` sets a maximum RHS value for the convergence to be

6: Numeric and Software-related Issues

Nonlinear Iterations

accepted, and `RhsFactor` sets a limit to the amount by which the RHS can augment during a single Newton step. If the RHS norm is larger than `RhsMax`, the Newton is diverged.

Apart from accepting a solution whenever the RHS falls below `RhsMin`, during a `Solve` statement, Sentauros Device tries to determine the value of an equation variable x , such that the computed update Δx (after k -th nonlinear iteration) is small enough:

$$\frac{\left| \frac{\Delta x}{x^*} \right|}{\varepsilon_R \left| \frac{x}{x^*} \right| + \varepsilon_A} < 1 \quad (30)$$

where:

$$\varepsilon_R = 10^{-\text{Digits}} \quad (31)$$

and x^* is a scaling constant. [Eq. 30](#) can be generalized to the case of a vector of unknowns (see [Fully Coupled Solution on page 972](#)).

For $x_{\text{ref}} = \varepsilon_A x^* / \varepsilon_R$, [Eq. 30](#) is equivalent to:

$$\frac{|\Delta x|}{|x| + x_{\text{ref}}} < \varepsilon_R \quad (32)$$

For large values of x ($|x| \rightarrow \infty$), the conditions [Eq. 30](#) and [Eq. 32](#) become relative error criteria:

$$\frac{|\Delta x|}{|x|} < \varepsilon_R \quad (33)$$

Conversely, for small values of x ($|x| \rightarrow 0$), they become absolute error criteria:

$$\left| \frac{\Delta x}{x^*} \right| < \varepsilon_A \quad \text{OR} \quad |\Delta x| < x_{\text{ref}} \cdot \varepsilon_R \quad (34)$$

Therefore, [Eq. 30](#) and [Eq. 32](#) ensure a smooth transition between absolute and relative error control.

The values of ε_A and x_{ref} can be defined commonly or independently for each equation with the keywords `Error` and `ErrRef`, respectively. The value of ε_R is defined by specifying the number of digits in [Eq. 31](#) with the keyword `Digits`.

By default, Sentauros Device uses the parameterization in terms of x_{ref} and ε_R (see [Eq. 32](#)). The keyword `-RelErrControl` switches to the parameterization in terms of ε_A and ε_R (see [Eq. 30](#)). [Table 152 on page 1229](#) lists the default values for the error control criteria.

UpdateMax and UpdateIncrease are used to detect divergence of the Newton algorithm. If the update error is larger than UpdateMax, or its increase from Newton step to Newton step exceeds UpdateIncrease, the Newton is regarded as diverged.

Damped Newton Iterations

Line search damping and Bank–Rose damping are two different damping methods. These approaches try to achieve convergence of the coupled iteration far from the final solution by changing the solution by smaller amounts than a normal Newton iteration would. Damping is useful to find an initial solution, but during Quasistationary or Transient ramps, using smaller time steps is usually preferable.

Line search damping is switched off by default. It is activated when the value of LineSearchDamping is set to a value smaller than one. This value determines the minimum factor by which the normal Newton update Δx can be damped. The damping method determines the actual damping factor automatically in each Newton step. If the actual factor falls below the minimum specified by LineSearchDamping, line search damping is considered to fail and is disabled for the following Newton iterations.

Bank–Rose damping becomes active after the number of Newton iterations set with the option NotDamped (default 1000).

LineSearchDamping and NotDamped are set in the global Math section and by parameters of the Coupled command. The former specification sets the defaults for the latter.

Derivatives

For most problems, Newton iterations converge best with full derivatives. Furthermore, for small-signal analysis, and noise and fluctuation analysis, using full derivatives is mandatory. Therefore, by default, Sentaurus Device takes full derivatives into account. For rare occasions where omission of derivatives improves convergence or performance significantly, use the keywords -AvalDerivatives and -Derivatives in the global Math section to switch off mobility and avalanche derivatives.

The derivatives are usually computed analytically, but a numeric computation can be used by specifying Numerically. This does not work with the method Blocked and, generally, is discouraged.

Incomplete Newton Algorithm

Certain simple simulations, such as I_d – V_g ramps, can be accelerated by using a modified Newton algorithm (see [Fully Coupled Solution on page 972](#) for a description of the standard Newton algorithm). The incomplete Newton algorithm, also known as the Newton–Richardson

6: Numeric and Software-related Issues

Nonlinear Iterations

method, tries to reuse the Jacobian matrix to avoid the expense of evaluating derivatives and computing matrix factorizations.

It can be switched on either in the global `Math` section:

```
Math {  
    IncompleteNewton  
    ...  
}
```

or it can be used as an option in a `Coupled` command:

```
Coupled (IncompleteNewton) {poisson electron hole}
```

The Jacobian matrix is reused if the following two conditions are satisfied:

$$\|Rhs_k\| < RhsFactor \cdot \|Rhs_{k-1}\| \quad (35)$$

$$\|Update_k\| < UpdateFactor \cdot \|Update_{k-1}\| \quad (36)$$

where k denotes the iteration index, `Rhs` denotes the right-hand side of the nonlinear equations, and `Update` denotes the Newton step. The values of $\|Rhs_k\|$ and $\|Update_k\|$ are displayed in the log file of Sentaurus Device during Newton iterations in the columns `|Rhs|` and `|step|`.

The parameters `RhsFactor` and `UpdateFactor` can be specified as arguments to the `IncompleteNewton` keyword:

```
IncompleteNewton (UpdateFactor=0.1 RhsFactor=10)
```

The default values are `UpdateFactor=0.1` and `RhsFactor=10`.

Additional Equations Available in Mixed Mode

The `Contact`, `Circuit`, `TContact` and `TCircuit` equations are introduced for mixed-mode problems. The keyword `Circuit` activates the solution of the electrical circuit models and nodes. `Contact` activates the solution of the electrical interface condition at the contacts. Similarly, `TContact` and `TCircuit` activate the solution of the thermal interface condition and thermal circuit, respectively.

By default, the keywords `Contact` and `Circuit` are not required because the Poisson equation also covers the contact and circuit domains. If only the Poisson equation is solved, no additional equations are added. Only if additional equations to `Poisson` appear in a `Coupled` statement, the circuit and contact equations are also added. Therefore:

```
Coupled { Poisson Electron Hole }
```


is equivalent to:

```
Coupled { Poisson Electron Hole Circuit Contact }
```

NOTE Sentaurus Device does not add the circuit and contact equations if Poisson is restricted to instances, for example:

```
Coupled {device1.Poisson device1.Electron
         device1.Hole
         device2.Poisson device2.Electron
         device2.Hole}
```

If the keyword `NoAutomaticCircuitContact` appears in the Math section, Sentaurus Device does not add the circuit and contact equations automatically (see [Figure 25](#)).

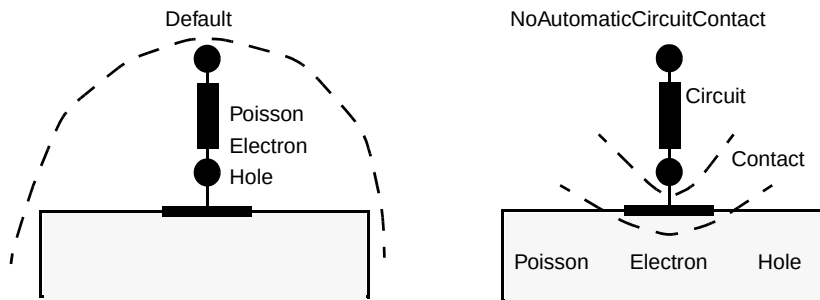


Figure 25 Range of equation keywords Circuit, Contact, Poisson, Electron, and Hole

Selecting Individual Devices in Mixed Mode

The default usage of an equation keyword such as `Poisson` activates the given equations for all devices. With complex multiple-device systems, such an action is not always desirable especially when a fully consistent solution has not yet been found. Sentaurus Device allows each equation to be restricted to a specific device by adding the name of the device instance to the equation keyword separated with a period. The syntax is:

```
<identifier>.<equation>
```

Device-specific solutions are used to obtain the initial solution for the whole system. For example, with two or more devices, it is often better to solve each device individually before coupling them all. Such a scheme can be written as:

```
System {
  ... device1 ...
  ... device2 ...
}
Solve {
  # Solve Circuit equation  Circuit
```

```

# Solve poisson and full coupled for each device
Coupled { device1.Poisson device1.Contact }
Coupled { device1.Poisson device1.Contact
          device1.Electron device1.Hole }
Coupled { device2.Poisson device2.Contact }
Coupled { device2.Poisson device2.Contact
          device2.Electron device2.Hole }

# Solve full coupled over all devices
Coupled { Circuit Poisson Contact Electron Hole }
}

```

Plugin Command

The **Plugin** command controls an iterative loop over two or more **Coupled** commands. It is used when a fully coupled method would use too many resources of a given machine, or when the problem is not yet solved and a full coupling of the equations would diverge. The **Plugin** syntax is defined as:

```
Plugin ( <options> ) ( <list-of-coupled-commands> )
```

Plugin commands can have any complexity but, usually, only a few combinations are effective. One standard form is the Gummel iteration in which each basic semiconductor equation is solved consecutively. With the **Plugin** command, this is written as:

```
Plugin { Poisson Electron Hole }
```

[Table 157 on page 1234](#) summarizes the options of **Plugin**.

Plugin commands can be used with other **Plugin** commands, such as:

```
Plugin{ Plugin{ ... } Plugin { ... } }
```

[Figure 26](#) illustrates the corresponding loop structure. A hierarchy of **Plugin** commands allows more complex iterative solve patterns to be created.

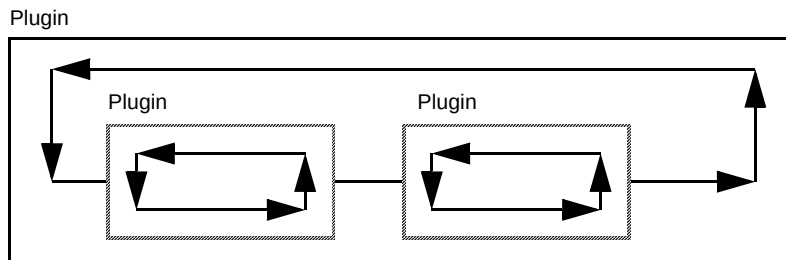


Figure 26 Example of hierarchy of **Plugin** commands

Linear Solvers

The Math parameters to the solution algorithms are device independent and must only appear in the base Math section. These can be grouped by solver type. The control parameters for the linear solvers are Method and SubMethod. The keyword Method selects the linear solver to be used, and the keyword SubMethod selects the inner method for block-decomposition methods (see [Table 176 on page 1256](#) for available linear solvers). The keywords ACMethod and ACSubMethod determine the linear solver used for AC analysis.

NOTE ACMethod=Blocked is the only valid choice for ACMethod. However, any of the available linear solvers can be selected for ACSubMethod.

[Table 176 on page 1256](#) lists the options that are available for the linear solver PARDISO. The options are specified in parentheses after the solver specification:

```
Method = ParDiSo (NonsymmetricPermutation IterativeRefinement)
```

The default options NonsymmetricPermutation, -IterativeRefinement, and -RecomputeNonsymmetricPermutation provide the best compromise between speed and accuracy. To improve speed, select -NonsymmetricPermutation. To improve accuracy, at the expense of speed, activate IterativeRefinement, or RecomputeNonsymmetricPermutation, or both.

All ILS options can be specified within an ILSrc statement in the global Math section:

```
Math {
  ILSrc = "
    set (...) {
      iterative (...);
      preconditioning (...);
      ordering (...);
      options (...);
    };
    ...
  "
  ...
}
```

Additional ILS options can be found in [Table 176 on page 1256](#).

The two linear solvers PARDISO and ILS support the option MultipleRHS to solve linear systems with multiple right-hand sides. This option is only appropriate for AC analysis. ILS may produce a small parallel speedup or slightly more accurate results if this option is selected.

Nonlocal Meshes

Nonlocal meshes are one-dimensional, special-purpose meshes that Sentaurus Device needs to implement one-dimensional, nonlocal physical models. A nonlocal mesh consists of nonlocal lines. Each nonlocal line is subdivided by nonlocal mesh points, to allow for the discretization of the equations that constitute the physical models. Sentaurus Device performs this subdivision automatically to obtain optimal interpolation between the nonlocal mesh and the normal mesh.

The 1D Schrödinger equation (see [1D Schrödinger Solver on page 281](#)), the nonlocal tunneling model (see [Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612](#)), and the trap tunneling model (see [Tunneling and Traps on page 417](#)) use nonlocal meshes. The documentation of the former two models introduces the use of nonlocal meshes in the context of the particular model and is restricted to typical cases. This section describes the construction of nonlocal meshes in detail.

Specifying Nonlocal Meshes

Nonlocal meshes are specified by the keyword `Nonlocal` in the global `Math` section. `Nonlocal` is followed by a string that gives the name of the nonlocal mesh and a list of options that control the construction of the nonlocal mesh. You can specify any number of nonlocal meshes; individual specifications are independent.

For example, with:

```
Math {
  Nonlocal "GateNonLocalMesh" (
    Electrode="Gate"
    Length=5e-7
  )
}
```

Sentaurus Device constructs a nonlocal mesh named `GateNonLocalMesh` that consists of nonlocal lines for semiconductor vertices up to a distance of 5 nm from the Gate electrode, and:

```
Math {
  Nonlocal "for_tunneling" (
    Barrier(Region="gateoxide")
  )
}
```

constructs a nonlocal mesh named `for_tunneling` that consists of lines that connect the different sides of the region `gateoxide`.

For a summary of available options, see [Table 179 on page 1257](#). For a comprehensive description of the construction of a nonlocal mesh, see [Constructing Nonlocal Meshes on page 168](#).

Visualizing Nonlocal Meshes

Sentaurus Device can visualize the nonlocal meshes it constructs. This feature is used to verify that the nonlocal mesh constructed is the one actually intended. For visualizing data defined on nonlocal meshes, see [Visualizing Data Defined on Nonlocal Meshes on page 167](#).

To visualize nonlocal meshes, use the keyword `NonLocal` in the `Plot` section (see [Device Plots on page 144](#)). The keyword causes Sentaurus Device to write two vector fields to the plot file that represent the nonlocal meshes constructed in the device.

For each vertex (of the normal mesh) for which a nonlocal line exists, the first vector field `NonLocalDirection` contains a vector that points from the vertex to the end of the nonlocal line in the direction of the reference surface for which the nonlocal line was constructed. The vector in the second field `NonLocalBackDirection` points from the vertex to the other end of the nonlocal line. The unit of both vectors is μm .

For vertices for which no nonlocal line exists, both vectors are zero. For vertices for which more than one nonlocal line exists, Sentaurus Device plots the vectors for one of these lines.

Visualizing Data Defined on Nonlocal Meshes

To visualize data defined on nonlocal meshes:

- In the `File` section, specify a file name using the `NonLocalPlot` keyword.
- On the top level of the command file, specify a `NonLocalPlot` section. There, `NonLocalPlot` is followed by a list of coordinates in parentheses and a list of datasets in braces.

Sentaurus Device writes nonlocal plots at the same time it writes normal plots. Nonlocal plot files have the extension `.plt` or `.tdr`.

Sentaurus Device picks nonlocal lines close to the coordinates specified in the `NonLocalPlot` section for output. The datasets given in the `NonLocalPlot` section are the datasets that can be used in the `Plot` section (see [Device Plots on page 144](#)). `NonLocalPlot` does not support

the `/Vector` option. Additionally, the Schrödinger equation provides special-purpose datasets available only for `NonLocalPlot` (see [Visualizing Schrödinger Solutions on page 286](#)).

In addition to the datasets explicitly specified, Sentaurus Device automatically includes the `Distance` dataset in the output. It provides the coordinate along the nonlocal line. The values in the `Distance` dataset are measured in μm . The interface or contact for which a nonlocal mesh line was constructed is located at zero, and its mesh vertex is located at positive coordinates.

For example:

```
NonLocalPlot(  
    (0 0) (0 1)  
) {  
    eDensity hDensity  
}
```

plots the electron and hole densities for the nonlocal lines close to the coordinates (0, 0, 0) and (0, 1, 0) in the device.

Constructing Nonlocal Meshes

Nonlocal meshes are specified in the global `Math` section:

```
Math {  
    NonLocal <string> (  
        Barrier(...)  
        RegionInterface=<string>  
        MaterialInterface=<string>  
        Electrode=<string>  
        ...  
    )  
}
```

The string following `NonLocal` is the name of the nonlocal mesh. The name relates the nonlocal mesh to the physical models defined on it.

Sentaurus Device supports two ways to specify nonlocal meshes:

- By specifying the regions and materials that form the tunneling barrier (keyword `Barrier`).
- By specifying a reference surface (keywords `RegionInterface`, `MaterialInterface`, and `Electrode`).

The `Barrier` specification is simpler but less general, and it is only suitable for nonlocal meshes used for direct tunneling through insulator barriers.

Specification Using Barrier

As an option to `Barrier`, specify all regions that belong to the tunneling barrier, using any number of `Region=<string>` or `Material=<string>` specifications. Sentaurus Device connects each side of the barrier to any other with nonlocal lines. Here, *side* means a conductively connected part of the surface of the barrier.

By default, semiconductor regions, metal regions, and electrodes are considered to be conductive. To enforce that a particular region (such as a wide-bandgap semiconductor region) is treated as not conductive, use the option `-Endpoint`, which accepts as an option a list of regions and materials, using the same syntax as for `Barrier`.

Use `Length=<float>` (in centimeters) to restrict the length on nonlocal lines (to suppress very long tunneling paths).

Specification Using a Reference Surface

`RegionInterface`, `MaterialInterface`, and `Electrode` specify a region interface name, material interface name, or electrode name. All interfaces and electrodes together form the reference surface that determines where in the device the nonlocal mesh is constructed.

The nonlocal lines link vertices of the normal mesh to the reference surface on the geometrically shortest path. The parameter `Length` of `NonLocal` determines the maximum distance of the vertex to the reference surface (in centimeters). Sentaurus Device provides no default value for `Length`; all nonlocal meshes must specify `Length` explicitly.

`Length` and `Permeation` are limited to the value of the parameter `NonLocalLengthLimit`, which is specified in centimeters in the global `Math` section and defaults to 10^{-4} . This parameter is used to capture cases where users accidentally specify lengths in micrometers rather than centimeters.

The property that nonlocal lines connect a vertex to the reference surface on the geometrically shortest path is fundamental. If any of the other rules described in this section inhibits the construction of a nonlocal line for this path, but a longer connection obeys all these restrictions, Sentaurus Device still does not use this connection to construct an alternative nonlocal line.

The parameter `Permeation` specifies a length by which Sentaurus Device extends the nonlocal lines, across the reference surface, towards the opposite of the side for which the line is constructed. `Permeation` defaults to zero. Sentaurus Device never extends the lines outside

6: Numeric and Software-related Issues

Nonlocal Meshes

the device or into regions flagged with `-Permeable` (see below). The extension is not affected by the `Transparent` and `Endpoint` options (see below).

The `Direction` parameter specifies a direction that the nonlocal lines approximately should have. Nonlocal lines with directions that deviate from the specified direction by an angle greater than `MaxAngle` are suppressed. If `Direction` is the zero vector or `MaxAngle` exceeds 90 (this is the default), nonlocal lines can have any direction. The length and sign of the direction vector are otherwise insignificant.

`Discretization` specifies the maximum spacing of the nonlocal mesh vertices. When necessary, Setaurus Device further refines the mesh created on a nonlocal line according to the built-in rules to yield a spacing no bigger than `Discretization` demands.

The flags `Transparent`, `Permeable`, `Endpoint`, and `Refined`, as well as their negation, specify flags for regions or materials that control nonlocal mesh construction. Each of the flags can be specified in two different ways:

- As an option to the main specification in the global `Math` section.
- As an option to `NonLocal` in a region-specific or material-specific `Math` section.

In the former case, the flag is followed by a list of region or material specifications that determine where the flag applies. In the latter case, the region or material of applicability is the location of the `Math` specification. Specifications of the latter style provide defaults that can be overwritten by specifications of the former style.

The part of a nonlocal line between the mesh vertex for which the line is constructed and the reference surface must not pass through regions flagged with `-Transparent`. By default, all regions are `Transparent`. For the exterior of the device, the flag `Outside` has the same meaning; this flag is an option to the main `NonLocal` specification.

The `-Permeable` flag limits extension on nonlocal lines across the reference surface. By default, all regions are `Permeable`.

For regions flagged with `-Endpoint`, Setaurus Device does not construct nonlocal lines that end in this region. The default is `-Endpoint` for insulator regions, and `Endpoint` for other regions.

Inside regions flagged as `-Refined`, the generation of nonlocal mesh points at element boundaries is suppressed. By default, all regions are `Refined`.

Special Handling of 1D Schrödinger Equation

For performance reasons, Sentaurus Device solves the 1D Schrödinger equation (see [1D Schrödinger Solver on page 281](#)) only on a reduced subset of nonlocal lines that still cover all vertices of the normal mesh for which nonlocal lines are constructed according to the rules outlined above.

To avoid artificial geometric quantization, Sentaurus Device extends nonlocal lines used for the 1D Schrödinger equation that are shorter than `Length` to reach full length. Sentaurus Device never extends the lines outside the device or into regions flagged by the `-Permeable` option. The extension is not affected by the `Transparent` and `Endpoint` options.

For nonlocal line segments in regions not marked by `-Refined` and `-Endpoint`, Sentaurus Device computes the intersections with the boxes of the normal mesh. Sentaurus Device needs this information to interpolate results from the 1D Schrödinger equation back to the normal mesh. Therefore, in regions where `-Refined` or `-Endpoint` is specified, 1D Schrödinger density corrections are not available, even when the regions are covered by nonlocal lines.

Special Handling of Nonlocal Tunneling Model

Sentaurus Device computes the intersections of the nonlocal lines with the boxes (see [Discretization on page 949](#)); to this end, some lines may become longer than `Length`. The nonlocal tunneling model (see [Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612](#)) uses the intersection points to limit the spatial range of the integrations Sentaurus Device must perform to compute the contribution to tunneling that comes from the particular vertex. For mesh points that border regions flagged with `-Endpoint`, the integration range is extended into that region, to pick up the Fowler–Nordheim current that enters it.

For nonlocal meshes used only for nonlocal tunneling, when `Permeation` is zero, Sentaurus Device assumes that the nonlocal lines cross the reference plane by an infinitesimal length. Therefore, if `-Endpoint` is specified for one of the regions that border the reference plane, Sentaurus Device suppresses nonlocal lines that point into that region.

If `Permeation` is positive, for nonlocal lines at nonmetal interfaces, Sentaurus Device switches to a nonlocal mesh construction mode similar to the one used for the Schrödinger equation: Sentaurus Device constructs only as many lines as needed to cover all vertices that must be covered, and extends all those lines to their maximum length. Furthermore, for nonlocal meshes not used for tunneling to traps, the integration range for tunneling covers the entire line. Tunneling to and from segments of the line marked by `-Endpoint` or `-Refined` will be assigned to the vertices for boxes crossed immediately outside those segments, to pick up Fowler–Nordheim currents in a similar manner as for the line construction mode with `Permeation=0`.

Unnamed Meshes

For backward compatibility, Sentaurus Device also allows you to specify nonlocal meshes in interface-specific or electrode-specific `Math` sections. For this kind of specification, the location of the mesh is the location of the `Math` section and, therefore, no interface or electrode specification must appear as an option to `NonLocal`.

Furthermore, the name of the nonlocal mesh must be omitted; physical models are associated with the nonlocal mesh by activating them in a `Physics` section specific to the same interface or electrode for which the nonlocal mesh was constructed.

For unnamed nonlocal meshes, vertices in regions with the trap tunneling model (see [Tunneling and Traps on page 417](#)) are connected irrespective of the material of the region or the `Endpoint` specification.

Performance Suggestions

To limit the negative performance impact of the nonlocal tunneling model, it is important to limit the number of nonlocal lines. To this end, most importantly, select `Length` and `Permeation` to be only as long as necessary. The option `-Endpoint` can be used to suppress the construction of lines to regions for which you know, in advance, that will not receive much tunneling current. The option `-Transparent` allows you to neglect tunneling through materials with comparatively high tunneling barrier, for example, oxides near a Schottky contact or heterointerface for which nonlocal tunneling has been activated.

Another use of the option `-Transparent` is at heterointerfaces, where there is no tunneling to the side of the material with the lower band edge (as there is no barrier to tunnel through). To restrict the construction of nonlocal lines to lines that go through the larger band-edge material, declare the lower band-edge material as not transparent by using `-Transparent`.

The option `-Refined` does not reduce the number of nonlocal lines, but it can reduce the size of the Jacobian matrix. The option `-Refined` is most useful for insulator regions, where the band-edge profile is approximately linear.

Monitoring Convergence Behavior

When Sentaurus Device has convergence problems, it can be helpful to know in which parts of the device and for which equations the errors are particularly large. With this information, it is easier to make adjustments to the mesh or the models used, to improve convergence.

Sentaurus Device can print the locations in the device where the largest errors occur (see [CNormPrint on page 173](#)). This provides limited information and has negligible performance impact. Sentaurus Device can also plot solution error information for the entire device after each Newton step (see [NewtonPlot on page 173](#)). This information is comprehensive, but can generate many files and can take significant time to write.

Both approaches provide access to the internal data of Sentaurus Device. Therefore, in both cases, the output is implementation dependent. Its proper interpretation can change between different Sentaurus Device releases.

CNormPrint

To obtain basic error information, specify the `CNormPrint` keyword in the global `Math` section. Then, after each Newton step and for each equation solved, Sentaurus Device prints to the standard output:

- The largest error according to [Eq. 32, p. 160](#) that occurs anywhere in the device for the equation.
- The vertex where the largest error occurs.
- The coordinates of the vertex.
- The current value of the solution variable for that vertex.

NewtonPlot

Sentaurus Device can plot the solution variable, the errors, the right-hand sides, and the solution updates after each Newton step. To use this feature:

- Use the `NewtonPlot` keyword in the `File` section to specify a file name for the plot. This name can contain up to two C-style integer format specifiers (for example, `%d`). If present, for the file name generation, the first one is replaced by the iteration number in the current Newton step and the second, by the number of Newton steps in the simulation so far. Sentaurus Device does not enforce any particular file name extension, but prepends the device instance name to the file name in mixed mode.
- Sentaurus Device plots Newton information if `StepSize` drops below a user-defined value. Use `NewtonPlotStep` to specify this limit. `NewtonPlotStep` is a valid option for `Quasistationary`, `Transient`, or `NewtonPlot`. By default, `NewtonPlot` files are saved only when the simulation fails (that is, `StepSize < MinStep`).
- By default, Sentaurus Device writes the current values of the solution variables only. Optionally, specify the data for output as options to `NewtonPlot` in the `Math` section.

- In addition, `MinError` can be specified as an option to `NewtonPlot`. If present, Sentaurus Device writes the file only if the error of the actual iteration is smaller than all previous errors within the current Newton step.

[Table 170 on page 1243](#) lists all of the options to `NewtonPlot`.

Save and Load

The `Save` and `Load` statements allow you to store the current simulation state of a device in a file and later (often, from another simulation run) to reload it, and to resume from where you stopped. By default, `Plot` files can be reloaded as well (see [Device Plots on page 144](#)). The `Save` statement generates files of type `.sav`, which only contain the information required to restart a simulation:

- Contact biases and currents
- The values of solution variables
- Occupation probability of traps
- Ferroelectric history (electric field and polarization) if the ferroelectric model is switched on

Loading traps is supported only if the specifications of traps for the saving and loading simulation are identical, that is, the set of traps and their order in the command file must be identical, as well as their types. A mismatch of trap specifications is silently ignored and could lead to partially correctly, incorrectly, or not loaded trap occupations. In this case, you must verify if the traps are initialized as intended.

When the file is loaded, all this information is restored.

NOTE Contact voltages stored in the loaded file overwrite the bias conditions that are specified in the command file.

When you specify a `Save` file in the `File` section, the simulation state is saved automatically to that file at the end of the simulation. When you specify a `Load` file in the `File` section, the state stored in that file is loaded at the beginning of the simulation. The geometry of a loaded file must match the geometry specified in the `Grid` file.

NOTE Interfaces regions are required in the grid file to save and load data on interfaces (see [Interface Plots on page 147](#)).

More control is possible with Save and Load commands in the Solve section. The commands are defined as:

```
Save(<parameters-opt>) <system-opt>
Load(<parameters-opt>) <system-opt>
```

The options are as for Plot, see [When to Plot on page 145](#). Save statements can be used at any level of the Solve section, the Load statement can only be used as a base level of the Solve statement. For example, in the case of a transient simulation, Load can only be used before or after the transient, not during it. Multiple Load and Save commands are allowed in a Solve statement. For example:

```
Solve {
  ...
  # Ramp the gate and save structures
  # First gate voltage
  Quasistationary (InitialStep=0.1 MaxStep=0.1 MinStep=0.01
    Goal {Name="gate" Voltage=1})
    {Coupled {Poisson Electron Hole}}
    Save(FilePrefix="vg1")
  # Second gate voltage
  Quasistationary (InitialStep=0.1 MaxStep=0.1 MinStep=0.01
    Goal {Name="gate" Voltage=3})
    {Coupled {Poisson Electron Hole}}
    Save(FilePrefix="vg2")
  # Load the saved structures and ramp the drain
  # First curve
  Load(FilePrefix="vg1")
  NewCurrentPrefix="Curve1"
  Quasistationary (InitialStep=0.1 MaxStep=0.5 MinStep=0.01
    Goal {Name="drain" Voltage=2.0})
    {Coupled {Poisson Electron Hole}}
  # Second curve
  Load(FilePrefix="vg2")
  NewCurrentPrefix="Curve2"
  Quasistationary (InitialStep=0.1 MaxStep=0.5 MinStep=0.01
    Goal {Name="drain" Voltage=2.0})
    {Coupled {Poisson Electron Hole}}
}
```

Tcl Command File

The Sentaurus Device command-line option `--tcl` invokes the Tcl interpreter to execute the command file. Tcl provides valuable extensions to the standard command file of Sentaurus Device, including:

- Variables
- Expressions
- Control structures

In addition, all of the Inspect Tcl commands are available (refer to the *Inspect User Guide* for more information). These commands are useful for the purpose of parameter extraction (see [Extraction on page 180](#)). However, the graphical functionality of the Inspect commands has been disabled.

Overview

An entire device simulation can be performed by the Tcl command `sdevice`. The following example shows a quasistationary ramp to computed contact values:

```
set vd 0.2
set vg [expr {10*$vd}]

sdevice "
  Electrode{
    { Name = \"source\" Voltage = 0.0 }
    { Name = \"gate\" Voltage = 0.0 }
    { Name = \"drain\" Voltage = $vd }
    { Name = \"bulk\" Voltage = 0.0 }
  }

  File{
    Grid    = \"mosfet.tdr\"
    Plot    = \"mosfet_des.tdr\"
    Current = \"mosfet_des.plt\"
    Output  = \"mosfet_des.log\"
  }

  Physics {
    Mobility (DopingDependence)
    Recombination (SRH (DopingDependence))
  }
}
```

```

Solve{
  Poisson
  QuasiStationary (Goal {Name=\\"gate\\" Voltage=$vg})
  { Coupled {Poisson Electron Hole} }
}
"

```

Sometimes, it is more useful to perform a device simulation in stages. In this case, the `sdevice_init` command is used for initialization, and multiple `sdevice_solve` commands are used to drive the simulation.

In the following example, an I_d - V_g sweep is performed, and the threshold voltage is extracted from the current plot file:

```

# initialize Sentaurus Device simulation
sdevice_init {
  Electrode{
    { Name = "source" Voltage = 0.0 }
    { Name = "gate" Voltage = 0.0 }
    { Name = "drain" Voltage = 0.2 }
    { Name = "bulk" Voltage = 0.0 }
  }

  File{
    Grid      = "mosfet.tdr"
    Plot      = "extract_VT_des.tdr"
    Current   = "extract_VT_des.plt"
    Output    = "extract_VT_des.log"
  }

  Physics{
    EffectiveIntrinsicDensity (Slotboom)
    Mobility (DopingDependence)
    Recombination (SRH (DopingDependence))
  }
}

# compute idvgs curve
sdevice_solve {
  Solve{
    NewCurrentPrefix = "initial_"

    Poisson
    Coupled {Poisson Electron Hole}

    Save (FilePrefix = "extract_VT_inital_save")

    NewCurrentPrefix = ""
  }
}

```

6: Numeric and Software-related Issues

Tcl Command File

```
        QuasiStationary (
            InitialStep=0.05 Increment=1.3
            MaxStep=0.05 MinStep=1e-4
            Goal {Name="gate" Voltage=2}
        )
        { Coupled {Poisson Electron Hole} }
    }
}

# extract threshold voltage
set VT [extract_VT extract_VT_des.plt gate drain]

# ramp gate voltage to threshold voltage
sdevice_solve "
    Solve{
        NewCurrentPrefix = \"ignore\_\"

        Load (FilePrefix = \"extract_VT_inital_save\")

        QuasiStationary (
            InitialStep=0.25 Increment=1.3
            MaxStep=0.25 MinStep=1e-4
            Goal {Name=\"gate\" Voltage=$VT}
        )
        { Coupled {Poisson Electron Hole} }
    }
"

# save final plot file
sdevice_finish
```

The final command `sdevice_finish` signals the end of a sequence of `sdevice_solve` statements, and Sentaurus Device saves the final plot files.

sdevice Command

The Tcl `sdevice` command expects a single argument. This argument describes an entire device simulation, that is, it includes a `File` section, `Physics` section, and `Solve` section.

The `sdevice` command returns 1 if the device simulation converges. Otherwise, the value 0 is returned.

sdevice_init Command

The Tcl `sdevice_init` command expects a single argument. This argument initializes a device simulation, that is, it can contain all of the sections of a standard command file of Sentaurus Device (except a `Solve` section).

No return value is computed by `sdevice_init`. If an error is encountered, Sentaurus Device will abort.

sdevice_solve Command

The Tcl `sdevice_solve` command expects a single argument. It can only be used after an `sdevice_init` command. The argument must contain a single `Solve` section. All the commands in the `Solve` section are executed.

The `sdevice_solve` command returns 1 if the device simulation converges. Otherwise, the value 0 is returned.

sdevice_finish Command

The Tcl `sdevice_finish` command expects no arguments, and it can be called after a series of `sdevice_solve` commands. It indicates that the device simulation performed by the `sdevice_solve` commands is finished, and the final plot file is generated. All open current plot files are closed.

sdevice_parameters Command

The Tcl `sdevice_parameters` command expects two arguments: a file name and the contents of a Sentaurus Device parameter file. It is an auxiliary function, which writes the parameter file to the given file name.

The following example generates the file `models.par` containing a permittivity parameter:

```
sdevice_parameters models.par {  
    Epsilon {  
        epsilon = 11.7  
    }  
}
```

Flowchart

Figure 27 shows the sequence in which the various Tcl sdevice commands can be invoked.

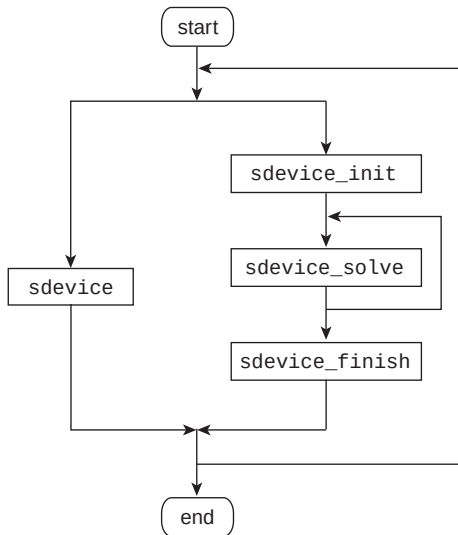


Figure 27 Flowchart of sequence for Tcl commands

Extraction

The Tcl interpreter in Sentaurus Device provides access to all of the Inspect commands. In this way, you can easily analyze .plt files generated by Sentaurus Device and extract parameters. The following example shows a gate ramp followed by the extraction of the threshold voltage:

```
sdevice {  
  Electrode {  
    { Name = "source" Voltage = 0.0 }  
    { Name = "gate"   Voltage = 0.0 }  
    { Name = "drain"  Voltage = 0.2 }  
    { Name = "bulk"   Voltage = 0.0 }  
  }  
  
  File {  
    Grid    = "mosfet.tdr"  
    Plot    = "extract_VT_des.tdr"  
    Current = "extract_VT_des.plt"  
    Output  = "extract_VT_des.log"  
  }  
  
  Physics { ... }  
}
```

```

Solve {
  QuasiStationary (
    InitialStep=0.05 MaxStep=0.05
    Goal {Name="gate" Voltage=2}
  )
  { Coupled {Poisson Electron Hole} }
}

proj_load extract_VT_des.plt proj
cv_create idvgs "proj gate OuterVoltage" "proj drain TotalCurrent"
set VT [f_VT idvgs]
puts stdout "threshold voltage VT = ${VT}V"

```

List of Available Inspect Tcl Commands

Reading and writing files:

```

graph_load, graph_write, load_library, param_load, param_write,
proj_getDataSet, proj_getList, proj_getNodeList, proj_load,
proj_unload, proj_write

```

Curve commands:

```

cv_abs, cv_compute, cv_create, cv_createFromScript,
cv_createWithFormula, cv_delete, cv_delPts, cv_getVals, cv_getValsX,
cv_getValsY, cv_getXaxis, cv_getYaxis, cv_getZero, cv_inv,
cv_log10Scale, cv_logScale, cv_printVals, cv_rename, cv_renameCurve,
cv_reset, cv_scaleCurve, cv_set_interpol, cv_split, cv_split_disc,
cv_write, macro_define

```

Parameter extraction:

```

f_Gamma, f_gm, f_IDSS, f_KP, f_Ron, f_Rout, f_TetaG, f_VT, f_VT1, f_VT2,
ft_scalar

```

Two-port network RF extraction:

```

tpnx_ac2h, tpxn_ac2s, tpxn_ac2y, tpxn_ac2z,
tpnx_characteristic_impedance, tpxn_DevWidth, tpxn_GetBias,
tpnx_GetParsAtPoint, tpxn_list_Bias, tpxn_list_fq, tpxn_load,
tpnx_setdBPoint, tpxn_startfq, tpxn_y2MSG, tpxn_y2MUG

```

Curve comparison library (load_library curvecomp):

```

cvcmp_CompareTwoCurves, cvcmp_DeltaTwoCurves

```

Extraction library (load_library EXTRACT):

```
cv_linTransCurve, cv_scaleCurve, ExtractBVi, ExtractBVv, ExtractEarlyV,
ExtractGm, ExtractGmb, ExtractIoff, ExtractMax, ExtractRon, ExtractSS,
ExtractValue, ExtractVtgm, ExtractVtgmb, ExtractVti
```

RF extraction library (load_library RFX):

```
rfx_abs_c, rfx_abs2_c, rfx_add_c, rfx_con_c, rfx_div_c, rfx_Export2csv,
rfx_ExtractVal, rfx_GetFK1, rfx_GetFmax, rfx_GetFt, rfx_GetK_MSG_MAG,
rfx_GetMUG, rfx_GetNearestIndex, rfx_GetRFCList, rfx_im_c, rfx_load,
rfx_mag_phase, rfx_mul_c, rfx_PolarBackdrop, rfx_re_c, rfx_ReIm2Abs,
rfx_ReIm2Phase, rfx_S2K, rfx_sign, rfx_SmithBackdrop, rfx_sub_c,
rfx_Y2H, rfx_Y2K, rfx_Y2S, rfx_Y2U, rfx_Y2Z, rfx_Z2U
```

IC-CAP model parameter extraction library (load_library ise2iccap):

```
iccap_Write
```

Output Redirection

The Tcl commands `sdevice`, `sdevice_init`, `sdevice_solve`, and `sdevice_finish` allow you to redirect standard output and standard errors. The syntax is shown in [Table 16](#).

Table 16 Output redirection

Syntax	Description
> filename	Writes standard output to <code>filename</code> .
2> filename	Writes standard error to <code>filename</code> .
>& filename	Writes both standard output and standard error to <code>filename</code> .
>> filename	Appends standard output to <code>filename</code> .
2>> filename	Appends standard error to <code>filename</code> .
>>& filename	Appends both standard output and standard error to <code>filename</code> .

Known Restrictions

The following solve commands only apply to the subsequent statements within a single `Solve` section. They do not apply beyond an `sdevice_solve` command:

- `NewCurrentPrefix` (see [NewCurrentPrefix Statement on page 139](#))
- `Set (TrapFilling=...)` (see [Explicit Trap Occupation on page 421](#))

If a transient simulation is performed using several `sdevice_solve` commands, the correct initial time must be specified. Sentaurus Device does not remember the last transient time from the previous `sdevice_solve` command.

Parallelization

Sentaurus Device uses thread parallelism to accelerate simulations on shared memory computers. [Table 17](#) gives an overview of the components of Sentaurus Device that have been parallelized.

Table 17 Areas of parallelization

Area	Description
Matrix assembly	Computation of mobility, avalanche, current density, energy flux; assembly of Poisson, continuity, lattice, and temperature equations; modified local-density approximation (MLDA).
Linear solver	Direct solver PARDISO; iterative solver ILS.

The number of threads can be specified in the global `Math` section of the Sentaurus Device command file:

```
Math {
    Number_of_Threads = number of threads
}
```

You can specify a constant for the number of threads, such as:

```
Number_of_Threads = 2
```

or:

```
Number_of_Threads = maximum
```

where `maximum` is equivalent to the number of processors available on the execution platform.

If required, the number of threads also can be specified separately for both the assembly and the linear solvers:

```
Math {
    Number_of_Assembly_Threads = number of assembly threads
    Number_of_Solver_Threads = number of solver threads
}
```

6: Numeric and Software-related Issues

Parallelization

In addition, the values of the following environment variables are checked:

```
SDEVICE_NUMBER_OF_THREADS, SNPS_NUMBER_OF_THREADS  
SDEVICE_NUMBER_OF_ASSEMBLY_THREADS  
SDEVICE_NUMBER_OF_SOLVER_THREADS  
OMP_NUM_THREADS
```

The priority of the various methods for specifying the number of threads is summarized in [Table 18](#).

Table 18 Specification of number of threads

Priority	Matrix assembly	Linear solver
Highest	Number_of_Assembly_Threads	Number_of_Solver_Threads
	Number_of_Threads	Number_of_Threads
	SDEVICE_NUMBER_OF_ASSEMBLY_THREADS	SDEVICE_NUMBER_OF_SOLVER_THREADS
	SDEVICE_NUMBER_OF_THREADS	SDEVICE_NUMBER_OF_THREADS
	SNPS_NUMBER_OF_THREADS	SNPS_NUMBER_OF_THREADS
Lowest		OMP_NUM_THREADS

The stack size per assembly thread can be specified in the global `Math` section of the Sentauros Device command file:

```
Math {  
    StackSize = stacksize in bytes  
}
```

Alternatively, the following UNIX environment variables are recognized:

```
SDEVICE_STACKSIZE, SNPS_STACKSIZE
```

By default, Sentauros Device uses only one thread and the stack size is 1 MB. For most simulations, the default stack size is adequate.

Sentauros Device requires one parallel license for every four threads. For example, a parallel simulation with 2–4 threads requires one parallel license, 5–8 threads require two licenses, 9–12 threads require three licenses, and so on. In the `Math` section, you can specify the behavior if there are not enough parallel licenses available:

```
Math {  
    ParallelLicense (option)  
}
```

The available options are shown as [Table 19](#).

Table 19 Options if not enough parallel licenses are available

Option	Description
Abort	Abort the simulation.
Serial	(Default) Continue the simulation in serial mode.
Wait	Wait until enough parallel licenses become available.

Observe the following recommendations to obtain the best results from a parallel Sentaurus Device run:

- Speedups are only obtained for sufficiently large problems. As a general rule, the device grid should have at least 5000 vertices. Three-dimensional problems are good candidates for parallelization.
- It is sensible to run a parallel Sentaurus Device job on an empty computer. As soon as multiple jobs compete for processors, performance decreases significantly.
- Use the keyword `Wallclock` in the Math section of the Sentaurus Device command file to display wallclock times rather than CPU times.
- The parallel execution produces different rounding errors. Therefore, the number of Newton iterations may change.

Extended Precision

Sentaurus Device allows you to perform simulations using extended precision floating-point arithmetic. This option is switched on in the global Math section by the keyword `ExtendedPrecision`. [Table 20](#) lists the available options, the size of a floating-point number, and the precision.

Table 20 Extended precision floating-point arithmetic

Option	Description	Size	Precision
<code>-ExtendedPrecision</code>	double (default)	64 bits	2.22×10^{-16}
<code>ExtendedPrecision</code>	long double	80 bits (AMD, Linux, SUSE)	1.08×10^{-19}
		128 bits (IBM)	2.47×10^{-32}
<code>ExtendedPrecision(128)</code>	double-double	128 bits	4.93×10^{-32}
<code>ExtendedPrecision(256)</code>	quad-double	256 bits	1.22×10^{-63}
<code>ExtendedPrecision(Digits=<digits>)</code>	arbitrary	$320 + 4.5 \times \text{digits}$ bits	$10^{-\text{digits}}$

6: Numeric and Software-related Issues

Extended Precision

On AMD, Linux, and SUSE, 80-bit extended precision arithmetic is supported in hardware with no noticeable degradation in performance. However, the gain in accuracy compared to normal precision (19 decimal digits versus 16 decimal digits) is limited. On IBM, the long double data type is implemented in software, and the run-time performance slows down by a factor of 10.

The data types double-double and quad-double are implemented in software on all platforms. They provide significant improvements in accuracy (32 and 63 decimal digits, respectively). However, the run-times increase by a factor of 10 and 100, respectively.

Arbitrary precision supports floating-point arithmetic with an arbitrary number of digits. It can be used for small research problems where even quad-double may not provide sufficient accuracy, for example, ultrawide-bandgap semiconductors.

The storage requirements of arbitrary precision increase linearly with the number of digits. The run-times, however, increase quadratically with the number of digits.

Extended precision can be a powerful tool to simulate wide-bandgap semiconductors, where it is necessary to resolve accurately small currents and very low carrier concentrations. It may or may not be beneficial for general convergence problems.

When using extended precision, the following points should be observed for the best results:

- In general, the direct linear solver SUPER provides the best accuracy. Additional choices include the direct linear solver PARDISO and the iterative linear solver ILS. Other linear solvers do not support extended precision.
- In the Math section:
 - Increase the value of `Digits`. Possible values are `Digits=15` for `ExtendedPrecision(128)` and `Digits=25` for `ExtendedPrecision(256)`. The value of `Digits` can be increased even further with arbitrary precision.
 - Decrease the value of `RhsMin`. Possible values are `RhsMin=1e-15` for `ExtendedPrecision(128)` and `RhsMin=1e-25` for `ExtendedPrecision(256)`. The value of `RhsMin` can be decreased even further with arbitrary precision.
 - Slightly increase `Iterations`, for example, from 15 to 20.
 - In the case of arbitrary precision, you may need to increase the maximum-allowed error (`UpdateMax` parameter) as well as to increase `Digits`. For example, `UpdateMax=1e220` is recommended for `Digits=200`.

Some features of Sentaurus Device do not take advantage of extended precision. They include:

- Input and output to files.
- Compact circuit models (refer to the *Compact Models User Guide*).
- CMI and the standard C++ interface in the PMI (see Part V of this manual).

- Monte Carlo (refer to the *Sentaurus Device Monte Carlo User Guide* for more information).
- Optical and laser simulations (see Part III of this manual).
- Integration of beam distribution for optical generation (see the keyword `RecBoxIntegr` in [Transfer Matrix Method on page 538](#)).

System Command

The `System` command allows UNIX commands to be executed during a Sentaurus Device simulation:

```
System ("UNIX command")
```

The `System` command can appear as an independent command in the `Solve` section, as well as within a `Transient`, `Plugin`, or `Quasistationary` command. The string argument of the `System` command is passed to a UNIX shell for evaluation.

By default, the return status of the UNIX command is ignored. If the variant:

```
+System ("UNIX command")
```

is used, Sentaurus Device examines the return status. The `System` command is considered to have converged if the return status is zero. Otherwise, it has not converged.

6: Numeric and Software-related Issues

System Command

Part II Physics in Sentaurus Device

This part of the *Sentaurus Device User Guide* contains the following chapters:

[Chapter 7 Electrostatic Potential and Quasi-Fermi Potentials on page 191](#)

[Chapter 8 Carrier Transport in Semiconductors on page 197](#)

[Chapter 9 Temperature Equations on page 205](#)

[Chapter 10 Boundary Conditions on page 217](#)

[Chapter 11 Transport in Metals, Organic Materials, and Disordered Media on page 235](#)

[Chapter 12 Semiconductor Band Structure on page 249](#)

[Chapter 13 Incomplete Ionization on page 273](#)

[Chapter 14 Quantization Models on page 279](#)

[Chapter 15 Mobility Models on page 301](#)

[Chapter 16 Generation–Recombination on page 359](#)

[Chapter 17 Traps and Fixed Charges on page 407](#)

[Chapter 18 Phase and State Transitions on page 425](#)

[Chapter 19 Degradation Model on page 441](#)

[Chapter 20 Organic Devices on page 465](#)

[Chapter 21 Optical Generation on page 469](#)

[Chapter 22 Radiation Models on page 571](#)

[Chapter 23 Noise, Fluctuations, and Sensitivity on page 581](#)

[Chapter 24 Tunneling on page 605](#)

[Chapter 25 Hot-Carrier Injection Models on page 625](#)

[Chapter 26 Heterostructure Device Simulation on page 651](#)

[Chapter 27 Energy-dependent Parameters on page 655](#)

[Chapter 28 Anisotropic Properties on page 665](#)

[Chapter 29 Ferroelectric Materials on page 681](#)
[Chapter 30 Modeling Mechanical Stress Effect on page 687](#)
[Chapter 31 Galvanic Transport Model on page 743](#)
[Chapter 32 Thermal Properties on page 745](#)

CHAPTER 7 Electrostatic Potential and Quasi-Fermi Potentials

This chapter discusses electrostatic potential and the quasi-Fermi potentials.

In all semiconductor devices, mobile charges (electrons and holes) and immobile charges (ionized dopants or traps) play a central role. The charges determine the electrostatic potential and, in turn, are themselves affected by the electrostatic potential. Therefore, each electrical device simulation at the very least must compute the electrostatic potential.

When all contacts of a device are biased to the same voltage, the device is in equilibrium, and the electron and hole densities are described by a constant quasi-Fermi potential. Therefore, together with the electrostatic potential, the relation between the quasi-Fermi potentials and the electron and hole densities is sufficient to perform the simplest possible device simulation.

Electrostatic Potential

The electrostatic potential is the solution of the Poisson equation, which is:

$$\nabla \cdot (\epsilon \nabla \phi + \vec{P}) = -q(p - n + N_D - N_A) - \rho_{\text{trap}} \quad (37)$$

where:

- ϵ is the electrical permittivity.
- $\vec{P}$ is the ferroelectric polarization (see [Chapter 29 on page 681](#)).
- q is the elementary electronic charge.
- n and p are the electron and hole densities.
- N_D is the concentration of ionized donors.
- N_A is the concentration of ionized acceptors.
- ρ_{trap} is the charge density contributed by traps and fixed charges (see [Chapter 17 on page 407](#)).

The dataset name for the electrostatic potential is `ElectrostaticPotential`. The right-hand side of [Eq. 37](#) (divided by q) is stored in the `SpaceCharge` dataset. The keyword for the Poisson equation in the `Solve` section is `Poisson`.

The dimensionless relative permittivity (that is, the permittivity in units of the vacuum permittivity ϵ_0) is given by the parameter `epsilon` in the `Epsilon` parameter set. The boundary conditions for the Poisson equation are discussed in [Chapter 10 on page 217](#).

Dipole Layer

At material interfaces, dipole layers of immobile charges can occur, leading to a potential jump across the interface. They are modeled by:

$$\phi_2 - \phi_1 = \frac{q\sigma_D}{\epsilon_r\epsilon_0} \quad (38)$$

where:

- ϕ_1 and ϕ_2 refer to the electrostatic potential at both sides of the interface (index 1 is the reference side).
- σ_D is the dipole surface density.
- ϵ_0 is the free space permittivity.
- ϵ_r is the relative interface permittivity.
- q is the elementary electronic charge.

The dipole interface model can be invoked for insulator–insulator interfaces by specifying:

`Dipole ( Reference = "R1" )`

in the `Physics` section of the interface. `Reference` denotes the reference side of the interface, and it can be either a region name or the material name.

The parameters of the model are given in the `Dipole` parameter set of the region or material interface section in the parameter file, and are listed in [Table 21](#).

Table 21 Dipole model parameters

Symbol	Parameter name	Default value	Unit
σ_D	<code>sigma_D</code>	0.	1/cm
ϵ_r	<code>epsilon</code>	3.9	1

NOTE At critical points, where heterointerfaces and dipole interfaces intersect, the semiconductor regions must be interface connected. Interface traps with tunneling cannot be located on dipole interfaces.

Quasi-Fermi Potential with Boltzmann Statistics

Electron and hole densities can be computed from the electron and hole quasi-Fermi potentials, and vice versa. If Boltzmann statistics is assumed, the formulas read:

$$n = N_C \exp\left(\frac{E_{F,n} - E_C}{kT}\right) \quad (39)$$

$$p = N_V \exp\left(\frac{E_V - E_{F,p}}{kT}\right) \quad (40)$$

where:

- N_C and N_V are the effective density-of-states.
- $E_{F,n} = -q\Phi_n$ and $E_{F,p} = -q\Phi_p$ are the quasi-Fermi energies for electrons and holes.
- Φ_n and Φ_p are electron and hole quasi-Fermi potentials, respectively.
- E_C and E_V are conduction and valence band edges, defined as:

$$E_C = -\chi - q(\phi - \phi_{\text{ref}}) \quad (41)$$

$$E_V = -\chi - E_{g,\text{eff}} - q(\phi - \phi_{\text{ref}}) \quad (42)$$

where χ denotes the electron affinity, $E_{g,\text{eff}}$ is the effective band gap, and ϕ_{ref} is a constant reference potential, see below. The zero level for the quasi-Fermi potential agrees with the zero level for the voltages applied at the contacts.

In unipolar devices, such as MOSFETs, it is sometimes possible to assume that the value of quasi-Fermi potential for the minority carrier is constant in certain regions. In this case, the concentration of the minority carrier can be directly computed from [Eq. 39](#) or [Eq. 40](#). This strategy is applied in Sentaurus Device if one of the carriers (electron or hole) is not specified inside the Coupled statement of the Solve section. Sentaurus Device uses an approximation scheme to determine the constant value of the quasi-Fermi potential.

NOTE In many cases if avalanche generation is important, the one carrier approximation cannot be applied even for unipolar devices.

The datasets for $E_{F,n}$, Φ_n , $E_{F,p}$, and Φ_p are named `eQuasiFermiEnergy`, `eQuasiFermiPotential`, `hQuasiFermiEnergy`, and `hQuasiFermiPotential`, respectively.

The `ConstRefPot` parameter allows you to specify ϕ_{ref} explicitly. Otherwise, Sentaurus Device computes ϕ_{ref} from the vacuum level, using the following rules:

7: Electrostatic Potential and Quasi-Fermi Potentials

Fermi Statistics

- If there is silicon in any simulated semiconductor structure, the intrinsic Fermi level of silicon is selected as reference, $\phi_{\text{ref}} = \Phi_{\text{intr}}(\text{Si})$.
- Otherwise, if any simulated device structure contains GaAs, $\phi_{\text{ref}} = \Phi_{\text{intr}}(\text{GaAs})$.
- In all other cases, Sentaurus Device selects the material with the smallest band gap (assuming a mole fraction of 0) and takes the value of its intrinsic Fermi level as ϕ_{ref} .

Fermi Statistics

For the equations presented in the previous section, Boltzmann statistics was assumed for electrons and holes. Physically more correct, Fermi (also called Fermi–Dirac) statistics can be used. Fermi statistics becomes important for high values of carrier densities, for example, $n > 1 \times 10^{19} \text{ cm}^{-3}$ in the active regions of a silicon device.

For Fermi statistics, [Eq. 39](#) and for [Eq. 40](#) are replaced by:

$$n = N_C F_{1/2} \left(\frac{E_{F,n} - E_C}{kT} \right) \quad (43)$$

$$p = N_V F_{1/2} \left(\frac{E_V - E_{F,p}}{kT} \right) \quad (44)$$

where $F_{1/2}$ is the Fermi integral of order 1/2.

Alternatively, you can write these expressions as:

$$n = \gamma_n N_C \exp \left(\frac{E_{F,n} - E_C}{kT} \right) \quad (45)$$

$$p = \gamma_p N_V \exp \left(\frac{E_V - E_{F,p}}{kT} \right) \quad (46)$$

where γ_n and γ_p are the functions of η_n and η_p :

$$\gamma_n = \frac{n}{N_C} \exp(-\eta_n) \quad (47)$$

$$\gamma_p = \frac{p}{N_V} \exp(-\eta_p) \quad (48)$$

$$\eta_n = \frac{E_{F,n} - E_C}{kT} \quad (49)$$

$$\eta_p = \frac{E_V - E_{F,p}}{kT} \quad (50)$$

Using Fermi Statistics

To activate Fermi statistics, the keyword `Fermi` must be specified in the global `Physics` section:

```
Physics {
    ...
    Fermi
}
```

NOTE Fermi statistics can be activated only for the whole device. Region-specific or material-specific activation is not possible, and the keyword `Fermi` is ignored in any `Physics` section other than the global one.

Initial Guess for Electrostatic Potential and Quasi-Fermi Potentials in Doping Wells

To determine an initial guess for the electrostatic potential and quasi-Fermi potentials, the device is divided into doping well regions, where a doping well region is a semiconductor region consisting of a set of connected semiconductor elements bounded by non-semiconductor elements or by vacuum (a doping well region may contain several mesh semiconductor regions). Then, each doping well region is further subdivided into wells of n-type and p-type doping, such that p-n junctions serve as dividers between wells. For doping wells with more than one contact, the wells are further subdivided, such that no well is associated with more than one contact. Every well is connected uniquely to a contact or it has no contact (floating well).

In wells with contacts, the quasi-Fermi potential of the majority carrier is set to the voltage on the contact associated with the well. For wells that have no contacts, the following equations define the quasi-Fermi potential for the majority carriers:

$$\Phi_p = k_{\text{float}} V_{\text{max}} + (1 - k_{\text{float}}) V_{\text{min}} \quad (51)$$

$$\Phi_n = (1 - k_{\text{float}}) V_{\text{max}} + k_{\text{float}} V_{\text{min}} \quad (52)$$

where V_{min} and V_{max} are the minimum and maximum of the contact voltages of the semiconductor wells with contacts in the doping well region, where the contactless well under

7: Electrostatic Potential and Quasi-Fermi Potentials

Initial Guess for Electrostatic Potential and Quasi-Fermi Potentials in Doping Wells

consideration is located. The coefficient k_{float} is an adjustable parameter with a default value of 0. To change the value of k_{float} , use the keyword `FloatCoef` in the `Physics` section.

If a well has a contact and it is the only well in the doping well region to which it belongs, then the quasi-Fermi potential of the minority carrier is set equal to the quasi-Fermi potential of the majority carrier. For all other wells, the quasi-Fermi potential of the minority carrier is set to V_{min} or V_{max} if the well is n-type or p-type, respectively, with V_{min} and V_{max} described above.

The electrostatic potential in a well is set to the value of the quasi-Fermi potential of the majority carrier adjusted by the built-in potential, that is, the initial solution will obey charge neutrality in all semiconductor regions.

Regionwise Specification of Initial Quasi-Fermi Potentials

You can set initial quasi-Fermi potentials regionwise. This is especially useful in CCD simulations, where a CCD cell or region must be initially in a certain state (usually, deep depletion). By specifying an initial quasi-Fermi potential, the region or set of regions can be brought to the proper state or states at the start of the simulation.

The initial quasi-Fermi potentials for electrons and holes can be specified regionwise in the `Physics` section using `eQuasiFermi` for electrons and `hQuasiFermi` for holes, followed by the value in volts:

```
Physics(Region="region_1") {  
    eQuasiFermi = 10  
}
```

The initial quasi-Fermi potential is recomputed when the continuity equation for the respective carrier is solved. If the continuity equation for the carrier whose quasi-Fermi potential has been specified is not solved, then the quasi-Fermi potential does not change. In this case, the device can be biased to an initial state through a quasistationary or transient simulation, while keeping the initial specified quasi-Fermi potential.

Carrier Transport in Semiconductors

This chapter discusses the carrier transport models for electrons and holes in inorganic semiconductors.

Introduction to Carrier Transport Models

Sentaurus Device supports several carrier transport models for semiconductors. They all can be written in the form of continuity equations, which describe charge conservation:

$$\nabla \cdot \vec{J}_n = qR_{\text{net}} + q\frac{\partial n}{\partial t} \quad -\nabla \cdot \vec{J}_p = qR_{\text{net}} + q\frac{\partial p}{\partial t} \quad (53)$$

where:

- R_{net} is the net recombination rate.
- $\vec{J}_n$ is the electron current density.
- $\vec{J}_p$ is the hole current density.
- n and p are the electron and hole density, respectively.

The transport models differ in the expressions used to compute $\vec{J}_n$ and $\vec{J}_p$. These expressions are the topic of this chapter. Additional equations to compute temperatures are usually also considered part of a transport model but are deferred to [Chapter 9 on page 205](#). Models for R_{net} are discussed in [Chapter 16 on page 359](#), [Chapter 17 on page 407](#), and [Chapter 24 on page 605](#). The boundary conditions for [Eq. 53](#) are discussed in [Chapter 10 on page 217](#).

Depending on the device under investigation and the level of modeling accuracy required, you can select four different transport models:

- Drift-diffusion
Isothermal simulation, suitable for low-power density devices with long active regions.
- Thermodynamic
Accounts for self-heating. Suitable for devices with low thermal exchange, particularly, high-power density devices with long active regions.

8: Carrier Transport in Semiconductors

Drift-Diffusion Model

- Hydrodynamic
Accounts for energy transport of the carriers. Suitable for devices with small active regions.
- Monte Carlo
Solves the Boltzmann equation for a full band structure.

The numeric approach for the Monte Carlo method (and the physical models usable with it) differs significantly from the other transport models. Therefore, the Monte Carlo method is described in a separate manual (refer to the [Sentaurus Device Monte Carlo User Guide](#)).

For all other transport models, the solution of [Eq. 53](#) is requested by the keywords `Electron` and `Hole` in the `Solve` section. When the equation for a density is not solved in a `Solve` statement, the quasi-Fermi potential for the respective carrier type remains fixed, unless the keyword `RecomputeQFP` is present in the `Math` section and the equation for the other density is solved.

The solutions of the densities n and p are stored in the datasets `eDensity` and `hDensity`, respectively. The current densities J_n and J_p are stored in the vector datasets `eCurrentDensity` and `hCurrentDensity`, their sum is stored in `ConductionCurrentDensity`, and the total current density (including displacement current) is stored in `TotalCurrentDensity`. An alternative representation of the total current density is described in [Current Potential on page 201](#).

Drift-Diffusion Model

The drift-diffusion model is the default carrier transport model in Sentaurus Device. For the drift-diffusion model, the current densities for electrons and holes are given by:

$$\vec{J}_n = \mu_n(n\nabla E_C - 1.5nkT\nabla \ln m_n) + D_n(\nabla n - n\nabla \ln \gamma_n) \quad (54)$$

$$\vec{J}_p = \mu_p(p\nabla E_V + 1.5pkT\nabla \ln m_p) - D_p(\nabla p - p\nabla \ln \gamma_p) \quad (55)$$

The first term takes into account the contribution due to the spatial variations of the electrostatic potential, the electron affinity, and the band gap. The remaining terms take into account the contribution due to the gradient of concentration, and the spatial variation of the effective masses m_n and m_p . For Fermi statistics, γ_n and γ_p are given by [Eq. 47](#) and [Eq. 48, p. 194](#). For Boltzmann statistics, $\gamma_n = \gamma_p = 1$.

By default, the diffusivities D_n and D_p are given through the mobilities by the Einstein relation, $D_n = kT\mu_n$ and $D_p = kT\mu_p$. However, it is possible to compute them independently (see [Non-Einstein Diffusivity on page 350](#)).

When the Einstein relation holds, the current equations can be simplified to:

$$\vec{J}_n = -nq\mu_n \nabla \Phi_n \quad (56)$$

$$\vec{J}_p = -pq\mu_p \nabla \Phi_p \quad (57)$$

where Φ_n and Φ_p are the electron and hole quasi-Fermi potentials, respectively (see [Quasi-Fermi Potential with Boltzmann Statistics on page 193](#)).

It is possible to use the drift-diffusion model together with a lattice temperature equation, but it is not mandatory. It is not possible to use the drift-diffusion model for a particular carrier type and to solve the carrier temperature for the same carrier type; the hydrodynamic model is required for that (see [Hydrodynamic Model for Current Densities on page 200](#)).

Thermodynamic Model for Current Densities

In the thermodynamic model [1], the relations [Eq. 56](#) and [Eq. 57](#) are generalized to include the temperature gradient as a driving term:

$$\vec{J}_n = -nq\mu_n (\nabla \Phi_n + P_n \nabla T) \quad (58)$$

$$\vec{J}_p = -pq\mu_p (\nabla \Phi_p + P_p \nabla T) \quad (59)$$

where P_n and P_p are the absolute thermoelectric powers [2] (see [Thermoelectric Power \(TEP\) on page 749](#)), and T is the lattice temperature.

The model differs from drift-diffusion when the lattice temperature equation is solved (see [Thermodynamic Model for Lattice Temperature on page 209](#)). However, it is possible to solve the lattice temperature equation even when using the drift-diffusion model.

To activate the thermodynamic model, specify the `Thermodynamic` keyword in the global `Physics` section.

Hydrodynamic Model for Current Densities

In the hydrodynamic model, current densities are defined as:

$$\vec{J}_n = \mu_n \left(n \nabla E_C + kT_n \nabla n - nkT_n \nabla \ln \gamma_n + \lambda_n f_n^{\text{td}} kn \nabla T_n - 1.5nkT_n \nabla \ln m_n \right) \quad (60)$$

$$\vec{J}_p = \mu_p \left(p \nabla E_V - kT_p \nabla p + pkT_p \nabla \ln \gamma_p - \lambda_p f_p^{\text{td}} kp \nabla T_p + 1.5pkT_p \nabla \ln m_p \right) \quad (61)$$

The first term takes into account the contribution due to the spatial variations of electrostatic potential, electron affinity, and the band gap. The remaining terms in Eq. 60 and Eq. 61 take into account the contribution due to the gradient of concentration, the carrier temperature gradients, and the spatial variation of the effective masses m_n and m_p . For Fermi statistics, γ_n and γ_p are given by Eq. 47 and Eq. 48, p. 194, and $\lambda_n = F_{1/2}(\eta_n)/F_{-1/2}(\eta_n)$ and $\lambda_p = F_{1/2}(\eta_p)/F_{-1/2}(\eta_p)$ (with η_n and η_p from Eq. 49 and Eq. 50, p. 195); for Boltzmann statistics, $\gamma_n = \gamma_p = \lambda_n = \lambda_p = 1$.

The thermal diffusion constants f_n^{td} and f_p^{td} are available in the ThermalDiffusion parameter set. They default to zero, which corresponds to the Stratton model [3][4].

Eq. 60 and Eq. 61 assume that the Einstein relation $D = \mu kT$ holds (D is the diffusion constant), which is true only near the equilibrium. Therefore, Sentaurus Device provides the option to replace the carrier temperatures in Eq. 60 and Eq. 61 by $gT_c + (1 - g)T$, an average of the carrier temperature (T_n or T_p) and the lattice temperature. This can be useful in devices where the carrier diffusion is important. The coefficient g for electrons and holes can be specified in the ThermalDiffusion parameter set. It defaults to one.

To activate the hydrodynamic model, the keyword Hydrodynamic must be specified in the global Physics section. If only one carrier temperature equation is to be solved, Hydrodynamic must be specified with an option, either Hydrodynamic(eTemperature) or Hydrodynamic(hTemperature).

Numeric Parameters for Continuity Equation

Carrier concentrations must never be negative. If during a Newton iteration a concentration erroneously becomes negative, Sentaurus Device applies damping procedures to make it positive. The concentration that finally results from a Newton iteration is limited to a value that can be specified (in cm^{-3}) by DensLowLimit=<float> in the global Math section; the default is 10^{-100}cm^{-3} .

Numeric Approaches for Contact Current Computation

The keywords `DirectCurrent` and `CurrentWeighting` in the global `Math` section determine the numeric approach to computing contact currents. The default method computes the contact current as the sum of the integral of the current density over the surface of the doping well associated with the contact and the integral of the charge generation rate over the volume of the doping well. `CurrentWeighting` activates a domain-integration method that uses a solution-dependent weighting function to minimize numeric errors (see [5] for details). With `DirectCurrent`, the current is computed as the surface integral of the current density over the contact area.

NOTE The current that flows into nodes in mixed-mode simulation is always computed with the `DirectCurrent` approach, no matter the specification in the `Math` section.

Current Potential

The total current density:

$$\vec{J} = \vec{J}_n + \vec{J}_p + \vec{J}_D \quad (62)$$

satisfies the conservation law:

$$\nabla \cdot \vec{J} = 0 \quad (63)$$

Consequently, $\vec{J}$ can be written as the curl of a vector potential $\vec{W}$:

$$\vec{J} = \nabla \times \vec{W} \quad (64)$$

In a 2D simulation, $\vec{W}$ only has a nonzero component along the z-axis, which is written simply as $W_z = W$. The total current density is then given by:

$$\vec{J} = \begin{bmatrix} \frac{\partial W}{\partial y} \\ -\frac{\partial W}{\partial x} \end{bmatrix} \quad (65)$$

The function W has the following important properties:

- The contour lines of W are the current lines of $\vec{J}$.
- The difference between the values of W at any two points equals the total current flowing across any line linking these points.

8: Carrier Transport in Semiconductors

References

Sentaurus Device computes the 2D current potential according to the approach proposed by Palm and Van de Wiele [6].

To visualize the results, add the keyword `CurrentPotential` to the `Plot` section:

```
Plot {  
    CurrentPotential  
}
```

Figure 28 displays plots of the total current density and current potential for a simple square device.

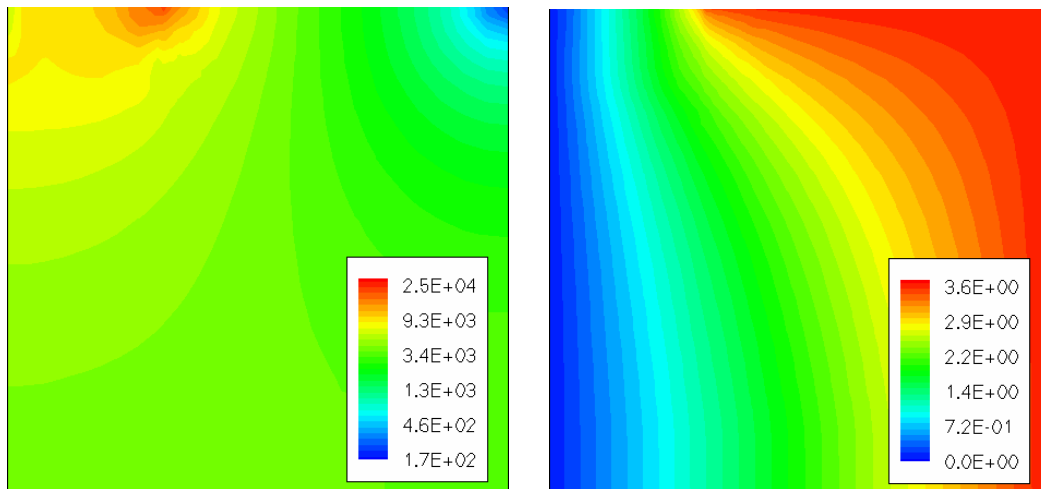


Figure 28 Total current density [Acm^{-1}] (left) and current potential [Acm^{-1}] (right)

References

- [1] G. Wachutka, “An Extended Thermodynamic Model for the Simultaneous Simulation of the Thermal and Electrical Behaviour of Semiconductor Devices,” in *Proceedings of the Sixth International Conference on the Numerical Analysis of Semiconductor Devices and Integrated Circuits (NASECODE VI)*, Dublin, Ireland, pp. 409–414, July 1989.
- [2] H. B. Callen, *Thermodynamics and an Introduction to Thermostatistics*, New York: John Wiley & Sons, 2nd ed., 1985.
- [3] R. Stratton, “Diffusion of Hot and Cold Electrons in Semiconductor Barriers,” *Physical Review*, vol. 126, no. 6, pp. 2002–2014, 1962.
- [4] Y. Apanovich *et al.*, “Numerical Simulation of Submicrometer Devices Including Coupled Nonlocal Transport and Nonisothermal Effects,” *IEEE Transactions on Electron Devices*, vol. 42, no. 5, pp. 890–898, 1995.

- [5] P. D. Yoder *et al.*, “Optimized Terminal Current Calculation for Monte Carlo Device Simulation,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 16, no. 10, pp. 1082–1087, 1997.
- [6] E. Palm and F. Van de Wiele, “Current Lines and Accurate Contact Current Evaluation in 2-D Numerical Simulation of Semiconductor Devices,” *IEEE Transactions on Electron Devices*, vol. ED-32, no. 10, pp. 2052–2059, 1985.

8: Carrier Transport in Semiconductors

References

This chapter describes the equations for lattice and carrier temperatures.

Introduction to Temperature Equations

Sentaurus Device can compute up to three different temperatures: lattice temperature, electron temperature, and hole temperature. Lattice temperature describes self-heating of devices. The electron and hole temperatures are required to model nonequilibrium effects in deep-submicron devices (in particular, velocity overshoot and impact ionization).

The following options are available:

- The lattice temperature can be computed from the total dissipated heat, assuming the temperature is constant throughout the device.
- The lattice temperature can be computed nonuniformly using the default model, or the thermodynamic model, or the hydrodynamic model.
- One or both carrier temperatures can be computed using the hydrodynamic model.

Irrespective of the model used, the lattice temperature is stored in the dataset `Temperature`, and the electron and hole temperatures are stored as `eTemperature` and `hTemperature`, respectively.

To solve the lattice temperature, it is necessary to specify a thermal boundary condition (see [Thermal Boundary Conditions on page 228](#)). To solve the carrier temperatures, no thermal boundary conditions are required.

By default (and as an initial guess), the lattice temperature is set to 300 K throughout the device, or to the value of `Temperature` set in the `Physics` section. If the device has one or more defined thermodes, an average temperature is calculated from the temperatures at the thermodes and is set throughout the device. As an initial guess, electron and hole temperatures are set to the lattice temperature. Carrier temperatures that are not solved are assumed to equal the lattice temperature throughout the simulation.

Uniform Self-Heating

A simple self-heating model is available to account for self-heating effects without solving the lattice heat flow equation. The self-heating effect can be captured with a uniform temperature, which is bias dependent or current dependent. The global temperature is computed from a global heat balance equation where the dissipated power P_{diss} is equal to the sum of the boundary heat fluxes (on thermodes with finite thermal resistance):

$$P_{\text{diss}} = \sum_i \frac{T - T_{\text{thermode}}^{(i)}}{R_{\text{th}}^{(i)}} + \sum_k \frac{T - T_{\text{circ}}^{(k)}}{R_{\text{th}}^{(k)}} \quad (66)$$

where:

- T is the device global temperature.
- $T_{\text{thermode}}^{(i)}$ and $R_{\text{th}}^{(i)}$ are the temperature and thermal resistivity of thermode i not connected in a thermal circuit.
- $T_{\text{circ}}^{(k)}$ and $R_{\text{th}}^{(k)}$ are the temperature and thermal resistivity of thermode k connected in a thermal circuit.

The second sum in Eq. 66 becomes zero when the device is not connected to any thermal circuit.

For a thermode k connected in a thermal circuit, an additional thermal circuit equation is solved for the temperature at the respective thermode $T_{\text{circ}}^{(k)}$. The thermal circuit equation solved is derived from the condition that the heat flux through the connected thermode is equal to the flux flowing through the thermal circuit element to which the thermode is connected.

In the case of thermode k connected to a thermal resistor, the thermal circuit equation becomes:

$$(T - T_{\text{circ}}^{(k)})/R_{\text{th}}^{(k)} = \nabla T^{\text{resistor}}/R_{\text{th}} \quad (67)$$

where:

- The left-hand side represents the thermal flux at thermode k .
- The right-hand side represents the heat flux flowing through the thermal resistor with thermal resistivity R_{th} .
- $\nabla T^{\text{resistor}} = T_{\text{resistor}} - T_{\text{circ}}^{(k)}$ is the temperature drop across the thermal resistor.

For a thermal capacitor, the thermal equation becomes $(T - T_{\text{circ}}^{(k)})/R_{\text{th}}^{(k)} = C_{\text{th}} \frac{dT}{dt}$ where $\nabla T^{\text{capacitor}} = T_{\text{capacitor}} - T_{\text{circ}}^{(k)}$ is the voltage drop across the thermal capacitor in a transient simulation.

If a thermode does not specify a thermal resistance or a thermal conductance, then by default, a zero surface conductance is assumed. In this case, the thermode is not accounted for in the sum on the right-hand side of Eq. 66 because the respective heat flux is zero. When none of the thermodes specifies any thermal resistance or conductance, Eq. 66 cannot be solved.

The dissipated power P_{diss} can be calculated as either the sum of all terminal currents multiplied by their respective terminal voltages ($\sum IV$), the sum of IV s from the user-specified node, or the total Joule heat $\int_V (J \cdot F) dv$.

The global temperature T_i for the point t_i (in transient or quasistationary simulations) is computed based on the estimation of T_i at the previous point t_{i-1} and the solution temperature T_{i-1} at the same point t_{i-1} .

NOTE For uniform self-heating, the temperature is obtained by postprocessing, rather than computed self-consistently with all other solution variables. Therefore, the simulation results may depend on the simulation time step used in quasistationary or transient simulations. In addition, uniform self-heating must not be used for AC, noise, or harmonic balance (HB) simulations.

Using Uniform Self-Heating

The uniform self-heating equation is activated in the global Physics section using the PostTemperature keyword:

```
Physics {
    PostTemperature
    ...
}
```

The heat fluxes at thermodes are defined in the Thermode sections by specifying Temperature and SurfaceResistance or SurfaceConductance:

```
Thermode {
    {Name= "top" Temperature=300 SurfaceConductance=0.1}
    ...
    {Name= "bottom" Temperature=300 SurfaceResistance=0.01}
}
```

9: Temperature Equations

Default Model for Lattice Temperature

The way P_{diss} is computed can be chosen by options to `PostTemperature` in the command file:

- As $\sum IV$ over all electrodes:

```
Physics {
    PostTemperature(IV_diss)
    ...
}
```
- As $\sum IV$ at user-selected electrodes:

```
Physics {
    PostTemperature(IV_diss("contact1" "contact2"))
    ...
}
```
- As the integral of Joule heat over the device volume:

```
Physics {
    PostTemperature
    ...
}
```

Uniform self-heating can be used during a quasistationary or transient simulation. As the feature replaces the lattice heat flow equation (Eq. 68, Eq. 69, and Eq. 73, p. 211), it cannot be used with the lattice temperature equation activated in the `Coupled` section.

Default Model for Lattice Temperature

Sentaurus Device can compute a spatially dependent lattice temperature even when neither the thermodynamic nor the hydrodynamic model is activated. The equation for the temperature is:

$$\frac{\partial W_L}{\partial t} + \nabla \cdot \vec{S}_L = \left. \frac{dW_L}{dt} \right|_{\text{coll}} + \vec{J}_n \cdot \nabla E_C + \left. \frac{dW_n}{dt} \right|_{\text{coll}} + \vec{J}_p \cdot \nabla E_V + \left. \frac{dW_p}{dt} \right|_{\text{coll}} \quad (68)$$

(See [Hydrodynamic Model for Current Densities on page 200](#) for an explanation of the symbols.) That is, the model is similar to the hydrodynamic model, but all temperatures merge into the lattice temperature, and all heating terms are accounted for in the lattice temperature equation.

To use this model, specify the keyword `Temperature` in the `Solve` section. You do not have to specify a model in the `Physics` section.

To account for Peltier heat at a contact, or a metal–semiconductor interface, or a conductive insulator–semiconductor interface, switch on the Peltier heating model explicitly. See [Heating](#)

at [Contacts, Metal–Semiconductor and Conductive Insulator–Semiconductor Interfaces](#) on [page 752](#) for more details.

Thermodynamic Model for Lattice Temperature

With the thermodynamic model, the lattice temperature T is computed from:

$$\begin{aligned} \frac{\partial}{\partial t} c_L T - \nabla \cdot \kappa \nabla T = & -\nabla \cdot [(P_n T + \Phi_n) \vec{J}_n + (P_p T + \Phi_p) \vec{J}_p] - \left(E_C + \frac{3}{2} kT\right) \nabla \cdot \vec{J}_n \\ & - \left(E_V - \frac{3}{2} kT\right) \nabla \cdot \vec{J}_p + q R_{\text{net}} (E_C - E_V + 3kT) + \hbar \omega G^{\text{opt}} \end{aligned} \quad (69)$$

where:

- κ is the thermal conductivity (see [Thermal Conductivity on page 747](#)).
- c_L is the lattice heat capacity (see [Heat Capacity on page 745](#)).
- E_C and E_V are the conduction and valence band energies, respectively.
- G^{opt} is the optical generation rate from photons with frequency ω (see [Chapter 21 on page 469](#)).
- R_{net} is the recombination rate (without contributions from G^{opt})
- The current densities J_n and J_p are computed as described in [Thermodynamic Model for Current Densities on page 199](#).

In metals, [Eq. 69](#) degenerates into:

$$\frac{\partial}{\partial t} c_L T - \nabla \cdot \kappa \nabla T = -\nabla \cdot [(PT + \Phi_M) \vec{J}_M] \quad (70)$$

where:

- P is the metal thermoelectric power.
- Φ_M is the Fermi potential in the metal.
- $\vec{J}_M$ is the metal current density as defined in [Eq. 129, p. 239](#).

Using the Thermodynamic Model

To use the thermodynamic model, the keyword `Temperature` must be specified inside the `Coupled` command of the `Solve` section. Use the keyword `Thermodynamic` in the `Physics` section to activate extra terms in the current density equations (due to the gradient of the temperature).

9: Temperature Equations

Thermodynamic Model for Lattice Temperature

To account for Peltier heat at a contact, or a metal–semiconductor interface, or a conductive insulator–semiconductor interface, switch on the Peltier heating model explicitly. See [Heating at Contacts, Metal–Semiconductor and Conductive Insulator–Semiconductor Interfaces on page 752](#) for more details.

Sentaurus Device allows the total heat generation rate to be plotted and the separate components of the total heat to be estimated and plotted. The total heat generation rate is the term on the right-hand side of [Eq. 69](#). It is plotted using the `TotalHeat` keyword in the `Plot` section. The total heat is calculated only when Sentaurus Device solves the temperature equation.

The formulas to estimate individual heating mechanisms and the appropriate keywords for use in the `Plot` section of the command file (see [Device Plots on page 144](#)) are shown in [Table 22 on page 210](#). The individual heat terms that are used for plotting and that are written to the `.log` file are less accurate than those Sentaurus Device uses to solve the transport equations. They serve as illustrations only.

Table 22 Terms and keywords used in Plot section of command file

Heat name	Keyword	Formula
Electron Joule heat	eJouleHeat	$\frac{ \vec{J}_n ^2}{qn\mu_n}$
Hole Joule heat	hJouleHeat	$\frac{ \vec{J}_p ^2}{qp\mu_p}$
Joule heat in conductive insulators	JouleHeat	$\frac{ \vec{J}_{CI} ^2}{\sigma}$
Recombination heat	RecombinationHeat	$qR_{\text{net}}(\Phi_p - \Phi_n + T(P_p - P_n))$
Thomson plus Peltier heat [1]	ThomsonHeat	$-\vec{J}_n \cdot T\nabla P_n - \vec{J}_p \cdot T\nabla P_p$
Peltier heat	PeltierHeat	$-T\left(\frac{\partial P_n}{\partial n}\vec{J}_n \cdot \nabla n + \frac{\partial P_p}{\partial p}\vec{J}_p \cdot \nabla p\right)$

To plot the lattice heat flux vector $\kappa\nabla T$, specify the keyword `lHeatFlux` in the `Plot` section (see [Device Plots on page 144](#)).

Hydrodynamic Model for Temperatures

Deep-submicron devices cannot be described properly using the conventional drift-diffusion transport model. In particular, the drift-diffusion approach cannot reproduce velocity overshoot and often overestimates the impact ionization generation rates. In this case, the hydrodynamic (or energy balance) model provides a good compromise between physical accuracy and computation time.

Since the work of Stratton [2] and Bløtekjær [3], there have been many variations of this model. The full formulation includes the so-called convective terms [4]. Sentaurus Device implements a simpler formulation without convective terms. In addition to the Poisson equation (Eq. 37, p. 191) and continuity equations (Eq. 53, p. 197), up to three additional equations for the lattice temperature T and the carrier temperatures T_n and T_p can be solved.

The energy balance equations read:

$$\frac{\partial W_n}{\partial t} + \nabla \cdot \vec{S}_n = \vec{J}_n \cdot \nabla E_C + \left. \frac{dW_n}{dt} \right|_{\text{coll}} \quad (71)$$

$$\frac{\partial W_p}{\partial t} + \nabla \cdot \vec{S}_p = \vec{J}_p \cdot \nabla E_V + \left. \frac{dW_p}{dt} \right|_{\text{coll}} \quad (72)$$

$$\frac{\partial W_L}{\partial t} + \nabla \cdot \vec{S}_L = \left. \frac{dW_L}{dt} \right|_{\text{coll}} \quad (73)$$

where the energy fluxes are:

$$\vec{S}_n = -\frac{5r_n\lambda_n}{2} \left(\frac{kT_n}{q} \vec{J}_n + f_n^{\text{hf}} \hat{\kappa}_n \nabla T_n \right) \quad (74)$$

$$\vec{S}_p = -\frac{5r_p\lambda_p}{2} \left(\frac{-kT_p}{q} \vec{J}_p + f_p^{\text{hf}} \hat{\kappa}_p \nabla T_p \right) \quad (75)$$

$$\vec{S}_L = -\kappa_L \nabla T_L \quad (76)$$

$$\hat{\kappa}_n = \frac{k^2}{q} n \mu_n T_n \quad (77)$$

$$\hat{\kappa}_p = \frac{k^2}{q} p \mu_p T_p \quad (78)$$

9: Temperature Equations

Hydrodynamic Model for Temperatures

and λ_n and λ_p are defined as for Eq. 60 and Eq. 61, p. 200. The parameters r_n , r_p , f_n^{hf} , and f_p^{hf} are accessible in the parameter file of Sentaurus Device. Different values of these parameters can significantly influence the physical results, such as velocity distribution and possible spurious velocity peaks. By changing these parameters, Sentaurus Device can be tuned to a very wide set of hydrodynamic/energy balance models as described in the literature. The default parameter values of Sentaurus Device are:

$$r_n = r_p = 0.6 \quad (79)$$

$$f_n^{\text{hf}} = f_p^{\text{hf}} = 1 \quad (80)$$

By changing the constants f_n^{hf} and r_n , the convective contribution and the diffusive contributions can be changed independently. With the default set of transport parameters of Sentaurus Device, the prefactor of the diffusive term has the form:

$$\kappa_n = \frac{3}{2} \cdot \frac{k^2 \lambda_n}{q} n \mu_n T_n \quad (81)$$

The collision terms are expressed by this set of equations:

$$\left. \frac{\partial W_n}{\partial t} \right|_{\text{coll}} = -H_n - \xi_n \frac{W_n - W_{n0}}{\tau_{en}} \quad (82)$$

$$\left. \frac{\partial W_p}{\partial t} \right|_{\text{coll}} = -H_p - \xi_p \frac{W_p - W_{p0}}{\tau_{ep}} \quad (83)$$

$$\left. \frac{\partial W_L}{\partial t} \right|_{\text{coll}} = H_L + \xi_n \frac{W_n - W_{n0}}{\tau_{en}} + \xi_p \frac{W_p - W_{p0}}{\tau_{ep}} \quad (84)$$

Here, H_n , H_p , and H_L are the energy gain/loss terms due to generation–recombination processes. The expressions used for these terms are based on approximations [5] and have the following form for the major generation–recombination phenomena:

$$H_n = 1.5kT_n(R_{\text{net}}^{\text{SRH}} + R_{\text{net}}^{\text{rad}} + R_{n,\text{net}}^{\text{trap}}) - E_{g,\text{eff}}(R_n^{\text{A}} - G_n^{\text{ii}}) - \alpha(\hbar\omega - E_{g,\text{eff}})G^{\text{opt}} \quad (85)$$

$$H_p = 1.5kT_p(R_{\text{net}}^{\text{SRH}} + R_{\text{net}}^{\text{rad}} + R_{p,\text{net}}^{\text{trap}}) - E_{g,\text{eff}}(R_p^{\text{A}} - G_p^{\text{ii}}) - (1 - \alpha)(\hbar\omega - E_{g,\text{eff}})G^{\text{opt}} \quad (86)$$

$$H_L = [R_{\text{net}}^{\text{SRH}} + 0.5(R_{n,\text{net}}^{\text{trap}} + R_{p,\text{net}}^{\text{trap}})](E_{g,\text{eff}} + 1.5kT_n + 1.5kT_p) \quad (87)$$

where:

- $R_{\text{net}}^{\text{SRH}}$ is the Shockley–Read–Hall (SRH) recombination rate (see [Shockley–Read–Hall Recombination on page 359](#)).
- $R_{\text{net}}^{\text{rad}}$ is the radiative recombination rate (see [Radiative Recombination on page 375](#)).
- R_n^{A} and R_p^{A} are Auger recombination rates related to electrons and holes (see [Auger Recombination on page 376](#)).
- $\hbar\omega$ is the photon energy (see [Hydrodynamic Model Parameters on page 214](#)).
- α is a dimensionless parameter that describes how the surplus energy of the photon splits between the bands (see [Hydrodynamic Model Parameters on page 214](#)).
- G_n^{ii} and G_p^{ii} are impact ionization rates (see [Avalanche Generation on page 377](#)).
- $R_{n,\text{net}}^{\text{trap}}$ and $R_{p,\text{net}}^{\text{trap}}$ are the recombination rates through trap levels (see [Chapter 17 on page 407](#)).
- G^{opt} is the optical generation rate (see [Chapter 21 on page 469](#)).

Surface recombination is taken into account in a way similar to bulk SRH recombination. Usually, the influence of H_n , H_p , and H_L is small, so they are not activated by default. To take these energy sources into account, the keyword `RecGenHeat` must be specified in the `Physics` section.

The energy densities W_n , W_p , and W_L are given by:

$$W_n = nw_n = n \left(\frac{3kT_n}{2} \right) \quad (88)$$

$$W_p = pw_p = p \left(\frac{3kT_p}{2} \right) \quad (89)$$

$$W_L = c_L T \quad (90)$$

and the corresponding equilibrium energy densities are:

$$W_{n0} = nw_0 = n \frac{3kT}{2} \quad (91)$$

$$W_{p0} = pw_0 = p \frac{3kT}{2} \quad (92)$$

The parameters ξ_n and ξ_p in [Eq. 82](#) to [Eq. 84](#) improve numeric stability.

They speed up relaxation for small densities and they approach 1 for large densities:

$$\xi_n = 1 + \frac{n_{\min}}{n} \left(\frac{n_0}{n_{\min}} \right)^{\max[0, (T - T_n)/100 \text{ K}]} \quad (93)$$

and likewise for ξ_p . Here, $n_{\min}$ and n_0 are adjustable small density parameters.

Hydrodynamic Model Parameters

The default set of transport coefficients (Eq. 79 and Eq. 80, p. 212) can be changed in the parameter file. The coefficients r and f^{hf} are specified in the EnergyFlux and HeatFlux parameter sets, respectively. Energy relaxation times τ_{en} and τ_{ep} can be modified in the EnergyRelaxationTime parameter set.

α in Eq. 85 and Eq. 86 divides the contribution of the optical generation rate into the energy gain or loss terms H_n and H_p . OptGenOffset specifies α , which can take values between 0 and 1 (default is 0.5). The angular frequency ω in Eq. 85 and Eq. 86 corresponds to the wavelength specified to compute the optical generation rate. This wavelength is defined by OptGenWavelength if the optical generation is loaded from a file (see Optical AC Analysis on page 568). OptGenOffset and OptGenWavelength are both options to RecGenHeat (see Table 182 on page 1259).

$n_{\min}$ and n_0 in Eq. 93 are set with the parameters RelTermMinDensity and RelTermMinDensityZero, respectively, in the global Math section. The default values for $n_{\min}$ and n_0 are 10^3 cm^{-3} and $2 \times 10^8 \text{ cm}^{-3}$, respectively.

Using the Hydrodynamic Model

To activate the hydrodynamic model, the keyword Hydrodynamic must be specified in the Physics section. If only one carrier temperature equation is to be solved, Hydrodynamic must be specified with the appropriate parameter, either Hydrodynamic(eTemperature) or Hydrodynamic(hTemperature). If the hydrodynamic model is not activated for a particular carrier type, Sentaurus Device merges the temperature for this carrier with the lattice temperature. That is, these temperatures are equal, and their heating terms (see the right-hand sides of Eq. 71, Eq. 72, and Eq. 73) are added.

In addition, the keywords eTemperature, hTemperature, and Temperature must be specified inside the Coupled command of the Solve section (see Coupled Command on page 158) to actually solve a carrier temperature equation or the lattice temperature equation. Temperatures remain fixed during Solve statements in which their equation is not solved.

By default, the energy conservation equations of Sentaurus Device do not include generation–recombination heat sources. To activate them, the keyword `RecGenHeat` must be specified in the `Physics` section.

To plot the electron, hole, and lattice heat flux vectors $\vec{S}_n$, $\vec{S}_p$, $\vec{S}_L$ (see [Eq. 74](#), [Eq. 75](#), and [Eq. 76](#)), specify the corresponding keywords `eHeatFlux`, `hHeatFlux`, and `lHeatFlux` in the `Plot` section (see [Device Plots on page 144](#)).

Numeric Parameters for Temperature Equations

Lower and upper limits for the lattice temperature and the carrier temperatures exist. The allowed temperature ranges are specified (in K) by `lT_Range=(<float> <float>)` (with defaults 50 K and 5000 K) and `cT_Range=(<float> <float>)` (with defaults 10 K and 80000 K). These ranges apply both to the temperatures during the Newton iterations and to the final results.

References

- [1] K. Kells, *General Electrothermal Semiconductor Device Simulation*, Series in Microelectronics, vol. 37, Konstanz, Germany: Hartung-Gorre, 1994.
- [2] R. Stratton, “Diffusion of Hot and Cold Electrons in Semiconductor Barriers,” *Physical Review*, vol. 126, no. 6, pp. 2002–2014, 1962.
- [3] K. Bløtekjær, “Transport Equations for Electrons in Two-Valley Semiconductors,” *IEEE Transactions on Electron Devices*, vol. ED-17, no. 1, pp. 38–47, 1970.
- [4] A. Benvenuti *et al.*, “Evaluation of the Influence of Convective Energy in HBTs Using a Fully Hydrodynamic Model,” in *IEDM Technical Digest*, Washington, DC, USA, pp. 499–502, December 1991.
- [5] D. Chen *et al.*, “Dual Energy Transport Model with Coupled Lattice and Carrier Temperatures,” in *Simulation of Semiconductor Devices and Processes (SISDEP)*, vol. 5, Vienna, Austria, pp. 157–160, September 1993.

9: Temperature Equations

References

CHAPTER 10 Boundary Conditions

This chapter describes the boundary conditions available in Sentaurus Device.

This chapter describe the properties of electrical contacts, thermal contacts, floating contacts, as well as boundary conditions at other borders of a domain where an equation is solved. Continuity conditions (how the solutions of one equation on two neighboring regions are matched at the region interface) are discussed elsewhere (see, for example, [Dipole Layer on page 192](#)). This chapter is restricted to the boundary conditions for the equations presented in [Chapter 7 on page 191](#), [Chapter 8 on page 197](#), and [Chapter 9 on page 205](#). Boundary conditions for less important equations are discussed with the equations themselves (see, for example, [Chapter 11 on page 235](#)).

Electrical Boundary Conditions

Ohmic Contacts

By default, contacts on semiconductors are Ohmic, with a resistance of $0.001\ \Omega$ when connected to a circuit node, and no resistance otherwise.

Charge neutrality and equilibrium are assumed at Ohmic contacts:

$$n_0 - p_0 = N_D - N_A \quad (94)$$

$$n_0 p_0 = n_{i,\text{eff}}^2 \quad (95)$$

For Boltzmann statistics, these conditions can be expressed analytically:

$$\phi = \phi_F + \frac{kT}{q} \operatorname{asinh}\left(\frac{N_D - N_A}{2n_{i,\text{eff}}}\right) \quad (96)$$

$$n_0 = \sqrt{\frac{(N_D - N_A)^2}{4} + n_{i,\text{eff}}^2} + \frac{N_D - N_A}{2}, \quad p_0 = \sqrt{\frac{(N_D - N_A)^2}{4} + n_{i,\text{eff}}^2} - \frac{N_D - N_A}{2} \quad (97)$$

where n_0, p_0 are the electron and hole equilibrium concentrations, and ϕ_F is the Fermi potential at the contact (which equals the applied voltage if it is not a resistive contact; see

[Resistive Contacts on page 222](#)). For Fermi statistics, Sentaurus Device computes the equilibrium solution numerically.

By default, $n = n_0, p = p_0$ are applied for concentrations at the Ohmic contacts. If the electron or hole recombination velocity is specified, Sentaurus Device uses the following current boundary conditions:

$$\vec{J}_n \cdot \hat{n} = qv_n(n - n_0) \quad \vec{J}_p \cdot \hat{n} = -qv_p(p - p_0) \quad (98)$$

where v_n, v_p are the electron and hole recombination velocities. In the command file, the recombination velocities can be specified as:

```
Electrode { ...
  { Name="Emitter" Voltage=0 eRecVelocity = v_n hRecVelocity = v_p }
}
```

NOTE If the values of the electron and hole recombination velocities are equal to zero ($v_n = 0, v_p = 0$), then $J_n \cdot \hat{n} = 0, J_p \cdot \hat{n} = 0$. That is, the electrode only defines the electrostatic potential (which is useful for SOI devices).

Contacts on Insulators

For contacts on insulators (for example, gate contacts), the electrostatic potential is:

$$\phi = \phi_F - \Phi_{MS} \quad (99)$$

where ϕ_F is the Fermi potential at the contact (which equals the applied voltage if it is not a resistive contact, see [Resistive Contacts on page 222](#)), and Φ_{MS} is the workfunction difference between the metal and an intrinsic reference semiconductor.

NOTE If an Ohmic contact touches both an insulator and a semiconductor, the electrostatic potential along the contact can be discontinuous because the boundary conditions ([Eq. 96](#) and [Eq. 99](#)) usually contradict. [Eq. 96](#) takes precedence over [Eq. 99](#).

Φ_{MS} can be specified in the `Electrode` section, either directly with `Barrier`, by specifying the metal workfunction with `WorkFunction`, or by specifying the electrode material:

```
Electrode { ...
  { Name="Gate" Voltage=0 Barrier = 0.55 }
}
Electrode { ...
  { Name="Gate" Voltage=0 WorkFunction = 5 }
}
```



```
Electrode { ...
  { Name="Gate" Voltage=0 Material="Gold" }
}
```

In the last case, the value for the workfunction is obtained from the parameter file:

```
Material = "Gold" {
  Bandgap { WorkFunction = 5 }
}
```

To emulate a poly gate, a semiconductor material and doping concentration can be specified in the `Electrode` section:

```
Electrode { ...
  { Name="Gate" Voltage=0 Material="Silicon" (N = 5e19) }
}
```

where N is used to specify the doping in an n-type semiconductor material. The built-in potential is approximated by $(kT/q)\ln(N/n_i)$. A p-type doping can be selected, similarly, by the letter P. If the value of the doping concentration is not specified (only N or P), the Fermi potential of the electrode is assumed to equal the conduction or valence band edge.

NOTE If the keyword `-MetalConductivity` is used in the `Math` section, the electrostatic potential ϕ in [Eq. 99](#) is computed as $\phi = \phi_F$. In this case, Φ_{MS} specified in the `Electrode` section as described above is neglected.

Schottky Contacts

The physics of Schottky contacts is considered in detail in [\[1\]](#) and [\[2\]](#). In this section, a typical model for Schottky contacts [\[3\]](#) is considered. The following boundary conditions hold:

$$\phi = \phi_F - \Phi_B + \frac{kT}{q} \ln\left(\frac{N_C}{n_{i,eff}}\right) \quad (100)$$

$$\vec{J}_n \cdot \hat{n} = qv_n(n - n_0^B) \quad \vec{J}_p \cdot \hat{n} = -qv_p(p - p_0^B) \quad (101)$$

$$n_0^B = N_C \exp\left(\frac{-q\Phi_B}{kT}\right) \quad p_0^B = N_V \exp\left(\frac{-E_{g,eff} + q\Phi_B}{kT}\right) \quad (102)$$

where:

- ϕ_F is the Fermi potential at the contact that is equal to an applied voltage $V_{applied}$ if it is not a resistive contact (see [Resistive Contacts on page 222](#)).

10: Boundary Conditions

Electrical Boundary Conditions

- Φ_B is the barrier height (the difference between the contact workfunction and the electron affinity of the semiconductor).
- v_n and v_p are the thermionic emission velocities.
- n_0^B and p_0^B are the equilibrium densities.

The default values for the thermionic emission velocities v_n and v_p are 2.573×10^6 cm/s and 1.93×10^6 cm/s, respectively.

The recombination velocities can be set in the `Electrode` section, for example:

```
Electrode { ...  
  { Name="Gate" Voltage=0 Schottky Barrier =  $\Phi_B$  eRecVelocity =  $v_n$   
    hRecVelocity =  $v_p$  }  
}
```

NOTE The `Barrier` specification can produce wrong results if the Schottky contact is connected to several different semiconductors. In such cases, use `WorkFunction` instead of `Barrier`. See [Contacts on Insulators on page 218](#) for more details.

Sentaurus Device also allows thermionic emission velocities v_n and v_p to be defined as functions of lattice temperature:

$$v_n(T) = v_n(300 \text{ K}) \sqrt{\frac{m_n(300 \text{ K})}{m_n(T)}} \sqrt{\frac{T}{300 \text{ K}}} \quad v_p(T) = v_p(300 \text{ K}) \sqrt{\frac{m_p(300 \text{ K})}{m_p(T)}} \sqrt{\frac{T}{300 \text{ K}}} \quad (103)$$

where:

- m_n and m_p are the temperature-dependent electron and hole DOS effective masses.
- $v_n(300 \text{ K})$ and $v_p(300 \text{ K})$ are the recombination velocities at 300 K specified in the `Electrode` section by `eRecVelocity` and `hRecVelocity`.

The default values for the thermionic emission velocities $v_n(300 \text{ K})$ and $v_p(300 \text{ K})$ are 2.573×10^6 cm/s and 1.93×10^6 cm/s, respectively. The temperature-dependent recombination velocity is activated by specifying `eRecVel(TempDep)`, `hRecVel(TempDep)` or `RecVel(TempDep)` in the electrode-specific `Physics` section:

```
Physics (Electrode = "cathode") { RecVel(TempDep) }
```

Barrier Lowering at Schottky Contacts

The barrier-lowering model for Schottky contacts can account for different physical mechanisms. The most important one is the image force [\[3\]](#), but it can also model tunneling and dipole effects.

The barrier lowering seen by the electron being emitted from metal into the conduction band and for the hole being emitted into the valence are given by:

$$\Delta\Phi_{B,e}(\vec{F}) = \begin{cases} aa_{1,e} \left(\frac{|\vec{F} \cdot \hat{n}|}{F_0} \right)^{pp_{1,e}} + aa_{2,e} \left(\frac{|\vec{F} \cdot \hat{n}|}{F_0} \right)^{pp_{2,e}} & \text{if } \vec{F} \cdot \hat{n} > 0 \\ 0 & \text{if } \vec{F} \cdot \hat{n} \leq 0 \end{cases} \quad (104)$$

$$\Delta\Phi_{B,h}(\vec{F}) = \begin{cases} aa_{1,h} \left(\frac{|\vec{F} \cdot \hat{n}|}{F_0} \right)^{pp_{1,h}} + aa_{2,h} \left(\frac{|\vec{F} \cdot \hat{n}|}{F_0} \right)^{pp_{2,h}} & \text{if } \vec{F} \cdot \hat{n} \leq 0 \\ 0 & \text{if } \vec{F} \cdot \hat{n} > 0 \end{cases} \quad (105)$$

where:

- $\vec{F} \cdot \hat{n}$ is the normal component of the electric field on the local exterior normal pointing from semiconductor region to metal contact, $F_0 = 1 \text{ Vcm}^{-1}$.
- $aa_{1,e}$, $aa_{2,e}$, $pp_{1,e}$, $pp_{2,e}$, $aa_{1,h}$, $aa_{2,h}$, $pp_{1,h}$, $pp_{2,h}$ are the model coefficients that can be specified in the parameter file of Sentaurus Device. Their default values are $aa_{1,e} = aa_{1,h} = 2.6 \times 10^{-4} \text{ eV}$, $pp_{1,e} = pp_{1,h} = 0.5$, $aa_{2,e} = aa_{2,h} = 0 \text{ eV}$, and $pp_{2,e} = pp_{2,h} = 1$.

The final value of the Schottky barrier is computed as $\Phi_B - \Delta\Phi_{B,e}(\vec{F})$ for the electrons in the conduction band and as $\Phi_B + \Delta\Phi_{B,h}(\vec{F})$ for the holes in the valence band. The barrier lowering also affects the equilibrium concentration of electrons n_0^B and holes p_0^B corresponding to its formula (Eq. 102), through the correction in barrier Φ_B described above.

In addition, Sentaurus Device implements a simplified model for barrier lowering. In this case, the barrier lowering is:

$$\Delta\Phi_B(F) = \begin{cases} a_1 \left[\left(\frac{F}{F_0} \right)^{p_1} - \left(\frac{F_{eq}}{F_0} \right)^{p_{1,eq}} \right] + a_2 \left[\left(\frac{F}{F_0} \right)^{p_2} - \left(\frac{F_{eq}}{F_0} \right)^{p_{2,eq}} \right] & \text{if } F > \eta F_{eq} \\ 0 & \text{if } F \leq \eta F_{eq} \end{cases} \quad (106)$$

where $F_0 = 1 \text{ Vcm}^{-1}$, and a_1 , a_2 , p_1 , $p_{1,eq}$, p_2 , and $p_{2,eq}$ are the model coefficients that can be specified in the parameter file of Sentaurus Device. Their default values are $a_1 = 2.6 \times 10^{-4} \text{ eV}$, $p_1 = p_{1,eq} = 0.5$, $a_2 = 0 \text{ eV}$, and $p_2 = p_{2,eq} = 1$. For fields smaller than ηF_{eq} (where η is one by default), barrier lowering is zero. The final value of the Schottky barrier is computed as $\Phi_B - \Delta\Phi_B(F)$ for n-doped contacts and $\Phi_B + \Delta\Phi_B(F)$ for p-doped contacts, because a difference between the metal workfunction and the valence band must be considered if holes are majority carriers. This model does not account for the direction of the electric field that determines in which band the barrier forms. For example, assume a Schottky

10: Boundary Conditions

Electrical Boundary Conditions

contact on an n-type semiconductor with Schottky barrier 0.8 V. The flat band condition is at a forward bias of approximately 0.8 V. For a reverse bias and forward bias less than 0.8 V, the barrier is in the conduction band, so the barrier lowering is applied correctly for electrons. On the other hand, for a forward bias greater than 0.8 V, the barrier is now in the valence band, so the barrier lowering should be applied to holes, not to electrons as the model does. In this case, this simplified model fails. This model is tuned to work properly for reverse biases, where the barrier lowering produces a sizable change in I–V characteristics of the Schottky device.

To activate the barrier-lowering model, specify `BarrierLowering` in the electrode-specific `Physics` section for the simplified model:

```
Physics(Electrode = "Gate") { BarrierLowering }
```

and for the complete model, specify:

```
Physics(Electrode = "Gate") { BarrierLowering(Full) }
```

To specify parameters of the model, create the following parameter set:

```
Electrode = "Gate"{
  BarrierLowering {
    a1 = a1           * [eV]
    p1 = p1           * [1]
    p1_eq = p1,eq      * [1]
    a2 = a2           * [eV]
    p2 = p2           * [1]
    p2_eq = p2,eq      * [1]
    eta = η            * [1]
    aa1 = aa1,e , aa1,h * [eV]
    pp1 = pp1,e , pp1,h * [1]
    aa2 = aa2,e , aa2,h * [eV]
    pp2 = pp2,e , pp2,h * [1]
  }
}
```

Resistive Contacts

By default, contacts have a resistance of 1 mΩ when they are connected to a circuit node, and no resistance otherwise. The resistance can be changed with `Resist` or `DistResist` or both in the `Electrode` section. For example:

```
Electrode { ...
  { name = "emitter" voltage = 2 Resist=1 }
}
```

defines an emitter as a contact with resistance 1 Ω (assuming that AreaFactor is one) and:

```
Electrode { ...
  { name = "emitter" voltage = 2 Resist=100 DistResist=2e-4 }
}
```

defines an emitter as a contact with a distributed resistance of 0.0002 Ωcm^2 in series with a 100 Ω lump resistor, with the 2 V external voltage applied to the lump resistor.

If V_{applied} is applied to the contact through a resistor R (Resist in the Electrode section) or distributed resistance R_d (DistResist in the Electrode section), there is an additional equation to compute ϕ_F in [Eq. 96](#), [Eq. 99](#), and [Eq. 100](#).

For a distributed resistance, ϕ_F is different for each mesh vertex of the contact and is computed as a solution of the following equation for each contact vertex, self-consistently with the system of all equations:

$$\hat{n} \cdot [J_p(\phi_F) + J_n(\phi_F) + J_D(\phi_F)] = \frac{(V_{\text{applied}} - \phi_F)}{R_d} \quad (107)$$

For a resistor, ϕ_F is a constant over the contact and only one equation per contact is solved self-consistently with the system of all equations:

$$\int_s \hat{n} \cdot [J_p(\phi_F) + J_n(\phi_F) + J_D(\phi_F)] ds = \frac{(V_{\text{applied}} - \phi_F)}{R} \quad (108)$$

where s is a contact area used in a Sentaurus Device simulation to compute a total current through the contact.

When both a lump resistor R and a distributed resistance R_d are specified, ϕ_F is computed for each mesh vertex of the contact as a solution of the following equations, self-consistently with the system of all equations:

$$\hat{n} \cdot [J_p(\phi_F) + J_n(\phi_F) + J_D(\phi_F)] = \frac{(V_{\text{internal}} - \phi_F)}{R_d} \quad (109)$$

$$\frac{V_{\text{applied}} - V_{\text{internal}}}{R} = \int_s \frac{V_{\text{internal}} - \phi_F}{R_d} ds \quad (110)$$

where V_{internal} depends on ϕ_F along the contact (on all contact vertices) and $\int_s \frac{V_{\text{internal}} - \phi_F}{R_d} ds$ is the total current on the contact.

NOTE In 2D simulations, R must be specified in units of $\Omega\mu\text{m}$. In 3D, R must be specified in units of Ω . R_d is given in Ωcm^2 (see [Table 167 on page 1240](#)).

10: Boundary Conditions

Electrical Boundary Conditions

To emulate the behavior of a Schottky contact [4], Schottky contact resistivity at zero bias was derived and a doping-dependent resistivity model of such contacts was obtained. This model is activated for Ohmic contacts by the keyword `DistResist=SchottkyResist` in the `Electrode` section. The model is expressed as:

$$\begin{aligned} R_d &= R_\infty \frac{300 \text{ K}}{T_0} \exp\left(\frac{q\Phi_B}{E_0}\right) \\ E_0 &= E_{00} \coth\left(\frac{E_{00}}{kT_0}\right) \\ E_{00} &= \frac{qh}{4\pi\epsilon_s} \sqrt{\frac{|N_{D,0} - N_{A,0}|}{m_t}} \end{aligned} \quad (111)$$

where:

- Φ_B is the Schottky barrier (in this model, for electrons, this is the difference between the metal workfunction and the electron affinity of the semiconductor; for holes, this is the difference between the valence band energy of the semiconductor and the metal workfunction).
- R_∞ is the Schottky resistance for an infinite doping concentration at the contact (or zero Schottky barrier).
- ϵ_s is the semiconductor permittivity.
- m_t is the tunneling mass.
- T_0 is the device lattice temperature defined in the `Physics` section (see [Table 182 on page 1259](#)).

The parameters of the model can be specified in an electrode-specific parameter set. The allowed syntax and recommended default values of these parameters [4] are:

```
SchottkyResistance {  
  Rinf = 2.4000e-09 , 5.2000e-09 # [Ohm*cm^2]  
  PhiB = 0.6 , 0.51 # [eV]  
  mt = 0.19 , 0.16 # [1]  
}
```

The model is applied to each contact vertex and checks the sign of the doping concentration $N_{D,0} - N_{A,0}$. If the sign is positive, the electron parameters are used to compute R_d for the vertex. If the sign is negative, the hole parameters are used.

Boundaries Without Contacts

Outer boundaries of the device that are not contacts are treated with ideal Neumann boundary conditions:

$$\epsilon \nabla \phi + \vec{P} = 0 \quad (112)$$

$$\vec{J}_n \cdot \hat{n} = 0 \quad \vec{J}_p \cdot \hat{n} = 0 \quad (113)$$

[Eq. 113](#) also applies to semiconductor–insulator interfaces.

Floating Contacts

Floating Metal Contacts

The charge on a floating contact (for example, a floating gate in an EEPROM cell) is specified in the `Electrode` section:

```
Electrode { ...
  { name="FloatGate" charge=1e-15 }
}
```

In the case of a floating metal contact (a contact not in touch with any semiconductor region), the electrostatic potential is determined by solving the Poisson equation with the following charge boundary condition:

$$\int \epsilon \hat{n} \cdot \nabla \phi dS = Q \quad (114)$$

where $\hat{n}$ is the normal vector on the floating contact surface S , and Q is the specified charge on the floating contact. The electrostatic potential on the floating-contact surface is assumed to be constant.

An additional capacitance between floating contact and control contact is necessary if, for example, EEPROM cells are simulated in a 2D approximation, to account for the additional influence on the capacitance from the real 3D layout. The additional capacitance can be specified in the `Electrode` section using the keyword `FGcap`.

10: Boundary Conditions

Floating Contacts

For example, if you have ContGate and FloatGate contacts, additional ContGate/FloatGate capacitance is specified as:

```
Electrode {  
  { name="ContGate" voltage=10 }  
  { name="FloatGate" charge=0 FGcap=(value=3e-15 name="ContGate") }  
}
```

where value is the capacitance value between FloatGate and ContGate. For the 1D case, the capacitance unit is $\text{F}/\mu\text{m}^2$; for the 2D case, $\text{F}/\mu\text{m}$; and for the 3D case, F.

If the floating-contact capacitance is specified, [Eq. 114](#) changes to:

$$\int \epsilon \hat{n} \cdot \nabla \phi dS + C_{\text{FC}}(\phi_{\text{CC}} - \phi_{\text{FC}} + \phi_{\text{B}}) = Q \quad (115)$$

where:

- C_{FC} is the specified floating-contact capacitance.
- ϕ_{FC} is the floating-contact potential.
- ϕ_{CC} is the potential on the control electrode (specified in the FGcap statement).
- ϕ_{B} is a barrier specified in the Electrode section for the floating contact, which can be used as a fitting parameter (default value is zero).

NOTE If the control contact is a metal, the value of ϕ_{CC} is defined by the applied voltage and the metal barrier/workfunction (see [Contacts on Insulators on page 218](#)). However, if the control contact touches a semiconductor region, ϕ_{CC} is equal to the applied voltage with an averaged built-in potential on the contact.

Floating contacts can have several capacitance values for different control contacts, for example:

```
Electrode {  
  { name="source" voltage=0 }  
  { name="ContGate" voltage=10 }  
  { name="FloatGate" charge=0 FGcap( (value=3e-15 name="ContGate")  
    (value=2e-15 name="source") ) }  
}
```

In transient simulations, Sentaurus Device takes the charge specified in the Electrode section as an initial condition. After each time step, the charge is updated due to tunneling and hot-carrier injection currents. For a description of tunneling and hot-carrier injection, see [Chapter 24 on page 605](#) and [Chapter 25 on page 625](#), respectively.

Floating Semiconductor Contacts

Sentaurus Device can simulate floating semiconductor contacts by connecting an electrode with charge boundary condition to an insulated semiconductor region. Within that floating-contact region, Sentaurus Device solves the Poisson equation with the following charge boundary condition:

$$\int_{FC} [q(p - n + N_D - N_A) + \rho_{\text{trap}}] dV = Q_{FC} \quad (116)$$

where Q_{FC} denotes the total charge on the floating gate and ρ_{trap} is the charge density contributed by traps and fixed charges (see [Chapter 17 on page 407](#)). The integral is calculated over all nodes of the floating-contact region [5].

The charge Q_{FC} is a boundary condition that must be specified in a Sentaurus Device `Electrode` statement in exactly the same way as for a floating metal contact:

```
Electrode {
  { name="floating_gate" Charge=1e-14 }
}
```

It is also possible to use the charge as a goal in a `Quasistationary` command:

```
Quasistationary (Goal {Name="floating_gate" Charge = 1e-13})
  { Coupled {Poisson Electron} }
```

Sentaurus Device automatically identifies a floating semiconductor contact based on information in the geometry (.tdr) file. It is not necessary for the charge contact to cover the entire boundary of the floating semiconductor region. A small contact is sufficient if the floating region has the same doping type throughout. However, if both n-type and p-type volumes exist in the floating region, separate contacts are required for each.

It is assumed that no current flows within the floating region. Therefore, the quasi-Fermi potential for electrons and holes is identical and constant within the floating region:

$$\Phi_n = \Phi_p = \Phi \quad (117)$$

Therefore, the electron and hole densities n and p are functions of the electrostatic potential ϕ and the quasi-Fermi potential Φ as discussed in [Quasi-Fermi Potential with Boltzmann Statistics on page 193](#) and [Fermi Statistics on page 194](#). Sentaurus Device does not solve the electron and hole continuity equations within a floating region.

In transient simulations, Sentaurus Device takes the charge specified in the `Electrode` section as an initial condition. After each time step, the charge is updated due to hot-carrier injection or tunneling currents including tunneling to traps. For a description of tunneling models and

how to enable them, see [Chapter 24 on page 605](#). For hot-carrier injection models, see [Chapter 25 on page 625](#).

The floating-contact capacitance can be specified in the `Electrode` statement in exactly the same way as for the floating metal contact (see [Floating Metal Contacts on page 225](#)). The only difference is that the potential on the semiconductor floating contact is not always a constant. Therefore, defining a constant capacitance may not be strictly correct, and it gives only approximate results. Sentaurus Device uses the semiconductor floating-contact Fermi level to compute the charge associated with the floating-contact capacitance and solves an equilibrium problem (zero voltages and zero charges) at the beginning to find a reference Fermi level.

For plotting purposes, floating semiconductor contacts are handled differently from other electrodes. Instead of the voltage, the value of the quasi-Fermi potential Φ is displayed.

Thermal Boundary Conditions

Boundary Conditions for Lattice Temperature

For the solution of [Eq. 67, p. 206](#), [Eq. 68, p. 208](#), [Eq. 69, p. 209](#), and [Eq. 73, p. 211](#), thermal boundary conditions must be applied. Wachutka [6] is followed, but the difference in thermopowers between the semiconductor and metal at Ohmic contacts is neglected. For free, thermally insulating, surfaces:

$$\kappa \hat{n} \cdot \nabla T = 0 \quad (118)$$

where $\hat{n}$ denotes a unit vector in the direction of the outer normal.

At thermally conducting interfaces, thermally resistive (nonhomogeneous Neumann) boundary conditions are imposed:

$$\kappa \hat{n} \cdot \nabla T = \frac{T_{\text{ext}} - T}{R_{\text{th}}} \quad (119)$$

where R_{th} is the external thermal resistance, which characterizes the thermal contact between the semiconductor and adjacent material. For the special case of an ideal heat sink ($R_{\text{th}} \rightarrow 0$), Dirichlet boundary conditions are imposed:

$$T = T_{\text{ext}} \quad (120)$$

By default, $R_{\text{th}} = 0$. Other values can be specified with the keyword `SurfaceResistance` in the specification of the `Thermode` (see [Table 169 on page 1242](#)). Alternatively, `SurfaceConductance` can be used to specify $1/R_{\text{th}}$.

When the lattice temperature equation is solved, the lattice temperature (Temperature) and lattice heat flux (lHeatFlux) at thermodes are added automatically to the .plt file in the corresponding Thermode section. The unit for heat flux is W for 3D, W/μm for 2D, and W/μm² for 1D.

At insulator–insulator, insulator–semiconductor, and semiconductor–semiconductor interfaces, a distributed thermal resistance is supported as well. This feature is activated by specifying the distributed resistance using the keyword `DistrThermalResist` in the Physics section of the respective interface. For example:

```
Physics (RegionInterface="reg1/reg2") {
    DistrThermalResist=1e-3
}
```

activates a distributed thermal resistance at the interface between region "reg1" and region "reg2" with the value 0.001 cm²K/W. As for thermionic emission, double points are used at the interfaces where distributed thermal resistance has been activated. In this case, double points allow a discontinuity in the lattice temperature on the two sides of the interface. Assuming an interface between materials 1 and 2 such that $\Delta T = T_2 - T_1 > 0$, if $S_{L,1}$ is the heat flux density entering material 1 and $S_{L,2}$ is the heat flux density leaving material 2, the thermal boundary condition at the interface between materials 1 and 2 can be written as:

$$\vec{S}_{L,2} = \vec{S}_{L,1} \quad (121)$$

$$\vec{S}_{L,1} = \frac{T_2 - T_1}{R_{d,th}} \quad (122)$$

where $R_{d,th}$ is the interface-distributed resistance defined with `DistrThermalResist` as described here.

Boundary Conditions for Carrier Temperatures

For the carrier temperatures T_n and T_p , at the thermal contacts, fast relaxation to the lattice temperature (boundary condition $T_n = T_p = T$) is assumed.

For other boundaries, adiabatic conditions for carrier temperatures are assumed:

$$\kappa_n \hat{n} \cdot \nabla T_n = \kappa_p \hat{n} \cdot \nabla T_p = 0 \quad (123)$$

When the hydrodynamic model is used, the lattice temperature (Temperature), the lattice heat flux (lHeatFlux), and the carrier heat fluxes (eHeatFlux and hHeatFlux) at thermodes are added automatically to the .plt file in the corresponding Thermode section. The unit for heat fluxes is W for 3D, W/μm for 2D, and W/μm² for 1D.

Periodic Boundary Conditions

The use of periodic boundary conditions (PBCs) can be helpful for simulations of periodic device structures, for example, lines or arrays of cells. Instead of building a periodic simulation domain, you can supply PBCs at the lower and upper boundary planes (perpendicular to the main axis of the coordinate system) of a single cell. Periodicity of the solution means that the two boundary planes are virtually identified, thereby forming a PBC interface: The solution is (weakly) continuous across this interface, and currents (or fluxes in the general case) can flow from one side to the other.

You can select between two different numeric PBC approaches, referred to as the Robin PBC (RPBC) approach and mortar PBC (MPBC) approach. Both approaches support:

- Several transport models, namely, drift-diffusion, thermodynamic, and hydrodynamic transport.
- Both conforming and nonconforming geometries and meshes.
- Two-dimensional and 3D device structures.
- The conservation of the fluxes (for example, current conservation for the continuity equations).
- Both one-fold and two-fold periodicity, that is, PBC can be applied in one axis direction and simultaneously in two directions for 3D structures.

Robin PBC Approach

The RPBC approach uses a current (or flux) model between both sides of the PBC interface of the form:

$$\alpha(u_1 - u_2) - j_{12} = 0 \quad (124)$$

where:

- u_1 and u_2 are the solution variables of an equation at both sides of the PBC interface.
- j_{12} is the flux density of the equation across the PBC interface.
- α represents a user-provided tuning parameter. The tuning parameter allows you to determine how strongly the local current density across the interface reduces the discontinuity of the solution variable. A large α reduces the jumps of the solution variable, while for $\alpha = 0$, no continuity is enforced at all and no current will flow across the interface.

Mortar PBC Approach

In the MPBC approach, both sides of the PBC interface are effectively glued together on one side (the mortar side), while on the other side (the nonmortar side), a weak continuity condition for the equation potential (mostly, the solution variable) is imposed. Therefore, the MPBC approach is not symmetric with respect to the selection of the mortar side if the mesh is nonconforming. The side where the accuracy requirements are larger (and, typically, the mesh is finer) should be selected as the mortar side. The equation potential for the carrier continuity equations are the corresponding quasi-Fermi potentials, while for the other equations, the solution variable is chosen. The MPBC approach does not require a current model across the PBC interface.

Specifying Periodic Boundary Conditions

Periodic boundary conditions are activated by using `PeriodicBC` in the global or device `Math` section. [Table 170 on page 1243](#) provides a complete list of possible options.

Specifying Robin Periodic Boundary Conditions

RPBCs can be activated individually for all supported equations (see [Electrostatic Potential on page 191](#), [Chapter 8 on page 197](#), and [Chapter 9 on page 205](#)). The solution variable u of the selected equation (for example, the electrostatic potential ϕ of the Poisson equation), and the corresponding flux density j are listed in [Table 23](#).

Table 23 Solution variables and fluxes used in periodic boundary conditions

u	j	Description
ϕ	$\epsilon \nabla \phi$	Electrostatic Potential on page 191
n	$\vec{J}_n$	Chapter 8 on page 197
p	$\vec{J}_p$	
T_n	$\vec{S}_n$	Chapter 9 on page 205
T_p	$\vec{S}_p$	
T	$\kappa \nabla T$	Chapter 9 on page 205

An RPBC is typically activated by `PeriodicBC(<options>)`, where `<options>` provides a selection of the equations and boundaries. Multiple specifications of `<options>` are possible to select several equations: `PeriodicBC( (<options1>) ... (<optionsN>) )`.

NOTE If the material parameters or the doping concentration differ on both sides of the interface, RPBC may result in nonphysical solutions.

An example of specifying RPBCs for different equations and boundaries is:

```
Math {  
    PeriodicBC(  
        (Direction=0 Coordinates=(-1.0 2.0))  
        (Poisson Direction=1 Coordinates=(-1e50 1e50))  
        (Electron Direction=1 Coordinates=(-1e50 1e50) Factor=2.e8)  
    )  
}
```

Here, the first option applies periodic boundary conditions to all equations in the direction of the x-axis and for the coordinates $-1.0\ \mu\text{m}$ and $2.0\ \mu\text{m}$. The next two options specify periodic boundary conditions for the electron and hole continuity equations in the y-direction and for the device side coordinates. **Factor** determines the tuning parameter α in the flux density model and defaults to 1.e8.

Specifying Mortar Periodic Boundary Conditions

Periodic boundary conditions using the MPBC approach are activated in the global or device Math section by:

```
PeriodicBC ( ( Type=MPBC Direction=1 MortarSide=Ymax ) )
```

Direction selects the coordinate axis where the PBC interface is created (that is, here the y-axis). **MortarSide** specifies that the plane at maximum coordinate values is taken as the mortar side (the default is the minimum coordinate plane). The maximum and minimum coordinate planes are extracted automatically. Using **MortarSide** implies the MPBC approach (making **Type** and **Direction** redundant), that means, the specification:

```
PeriodicBC( ( MortarSide=Xmin MortarSide=Ymax ) )
```

enables a two-fold MPBC simulation.

Application Notes

Note that:

- The RPBC approach supports, in contrast to the MPBC approach, some flexibility with respect to the involved equations.
- The MPBC approach treats the PBC interface essentially as a heterointerface. Conductor regions touching the PBC interface are not supported in MPBC. Heterointerfaces touching the PBC interface are allowed, but there are cases where the periodicity is further relaxed.

- If the device structure forms a heterointerface at the PBC interface, the MPBC approach must be used. The RPBC approach provides nonphysical results.
- Using MPBC in combination with iterative linear solvers (such as ILS), you might observe some convergence problems. If feasible, using a direct solver (such as PARDISO) should solve the problem. Otherwise, an improved preconditioner or the use of extended precision improves the convergence behavior in many cases.

References

- [1] A. Schenk and S. Müller, “Analytical Model of the Metal-Semiconductor Contact for Device Simulation,” in *Simulation of Semiconductor Devices and Processes (SISDEP)*, vol. 5, Vienna, Austria, pp. 441–444, September 1993.
- [2] A. Schenk, *Advanced Physical Models for Silicon Device Simulation*, Wien: Springer, 1998.
- [3] S. M. Sze, *Physics of Semiconductor Devices*, New York: John Wiley & Sons, 2nd ed., 1981.
- [4] K. Varahramyan and E. J. Verret, “A Model for Specific Contact Resistance Applicable for Titanium Silicide–Silicon Contacts,” *Solid-State Electronics*, vol. 39, no. 11, pp. 1601–1607, 1996.
- [5] S. Sugino *et al.*, “Analysis of Writing and Erasing Procedure of Flotox EEPROM Using the New Charge Balance Condition (CBC) Model,” in *Workshop on Numerical Modeling of Processes and Devices for Integrated Circuits (NUPAD IV)*, Seattle, WA, USA, pp. 65–69, May 1992.
- [6] G. K. Wachutka, “Rigorous Thermodynamic Treatment of Heat Generation and Conduction in Semiconductor Device Modeling,” *IEEE Transactions on Computer-Aided Design*, vol. 9, no. 11, pp. 1141–1149, 1990.

10: Boundary Conditions

References

CHAPTER 11 Transport in Metals, Organic Materials, and Disordered Media

This chapter describes transport in materials other than low-defect, inorganic semiconductors.

Singlet Exciton Equation

In organic semiconductor devices, besides the simulation of the electrical part consisting of solving the Poisson and continuity equations, the *singlet exciton equation* is introduced to account for optical properties of these materials related to Frenkel excitons.

This equation takes into account only singlet excitons because triplet excitons do not contribute directly to light emission. The equation models the dynamics of the generation, diffusion, recombination, and radiative decay of singlet excitons in organic semiconductors.

The singlet exciton equation, governing the transport of excitons in organic semiconductors, is given by:

$$\frac{\partial n_{se}}{\partial t} = R_{bimolec} + \nabla \cdot D_{se} \nabla n_{se} - \frac{n_{se} - n_{se}^{eq}}{\tau} - \frac{n_{se} - n_{se}^{eq}}{\tau_{trap}} - R_{se} \quad (125)$$

where:

- n_{se} is the singlet exciton density.
- $R_{bimolec}$ is the carrier bimolecular recombination rate acting as a singlet exciton generation term.
- D_{se} is the singlet exciton diffusion constant.
- τ, τ_{trap} are the singlet exciton lifetimes.
- R_{se} is the net singlet exciton recombination rate.

The first term on the right-hand side of [Eq. 125](#) accounts for the creation of excitons through bimolecular recombination (see [Bimolecular Recombination on page 401](#)). The second term describes exciton diffusion; while radiative decay associated with light emission is accounted for by the third and fourth terms.

Nonradiative exciton destruction and conversion of excitons to free electron and free hole populations are described by the net exciton recombination rate (fifth term):

$$\begin{aligned} R_{se} &= R_{se-nf} + R_{se-pf} \\ R_{se-nf} &= (v_{se} + v_n) \sigma_{se-n} n_{se} n \\ R_{se-pf} &= (v_{se} + v_p) \sigma_{se-p} n_{se} p \end{aligned} \quad (126)$$

where:

- R_{se-nf} , R_{se-pf} are the exciton recombination rate due to free electron and free hole populations, respectively.
- $v_{se} = v_{se,0}$ is the exciton velocity.
- $v_n = v_{n,0} \sqrt{T/300 \text{ K}}$ and $v_p = v_{p,0} \sqrt{T/300 \text{ K}}$ are the electron and hole velocities. $v_{se,0}$, $v_{n,0}$, and $v_{p,0}$ are set in the parameter file.
- σ_{se-n} and σ_{se-p} are the reaction cross sections between the singlet exciton, and the electron and hole, respectively.

In addition, the net exciton recombination rate R_{se} may contain exciton interface dissociation rates converted to volume terms if the exciton interface dissociation model is activated (see [Exciton Dissociation Model on page 402](#)).

Boundary and Continuity Conditions for Singlet Exciton Equation

At electrodes and thermodes, equilibrium is assumed for the singlet exciton population:

$$n_{se}(T) = n_{se}^{eq}(T) = \gamma g_{ex} (N_C(T) + N_V(T)) \exp\left(-\frac{E_{g,eff} - E_{ex}}{kT}\right) \quad (127)$$

where $\gamma = 1/4$, and g_{ex} and E_{ex} are the singlet exciton degeneracy factor and binding energy, respectively.

Conditions for the singlet exciton equation at interfaces depend on the interface type. The interface type is dictated by the two regions adjacent to the interface.

For interfaces where the singlet exciton equation is solved only in one of the regions of the interface, the boundary conditions imposed are either [Eq. 127](#) (default) or $\nabla n_{se} \cdot \hat{n} = 0$ (zero flux), depending on the user selection.

For a heterointerface where the singlet exciton equation is solved in both regions, the continuity conditions imposed are thermionic-like. In this case, you can select the barrier ΔE to be the

difference between the band gaps, the conduction bands, or the valence bands of the two regions.

Assuming that at heterointerfaces between materials 1 and 2 (that is, regions 1 and 2), $\Delta E > 0$ ($E_{g,2} > E_{g,1}$ for example), and $j_{se,2}$ is the singlet exciton flux leaving material 2 and $j_{se,1}$ is the singlet exciton flux entering material 1, the interface boundary condition can be written as:

$$\begin{aligned} j_{se,1} &= j_{se,2} \\ j_{se,2} &= v_{si} \left(n_{se,2} - n_{se,1} \exp\left(-\frac{\Delta E}{kT}\right) \right) \end{aligned} \quad (128)$$

where $n_{se,2}$ and $n_{se,1}$ are the singlet exciton densities of materials 2 and 1 at the heterointerface. v_{si} is the organic heterointerface emission velocity defined in the parameter file.

Using the Singlet Exciton Equation

To activate the singlet exciton equation, the keyword `SingletExciton` must be specified in the `Coupled` statement of the `Solve` section (see [Coupled Command on page 158](#)). Then, the regions where the equation will be solved are selected by specifying the keyword `SingletExciton` with different options in the `Physics` section of the respective regions (by default, none of the regions is selected for solving the singlet exciton equation, so you must select at least one region).

The singlet exciton generation and recombination terms in [Eq. 125](#) are switched off by default. They can be activated regionwise by specifying the corresponding keyword as an option for `Recombination` in the `SingletExciton` section of the respective region `Physics` section (see [Table 201 on page 1277](#)).

The keyword `Bimolecular` activates the bimolecular generation (first term). The keywords `Radiative` and `TrappedRadiative` switch on radiative decay of singlet excitons either directly or trap-assisted (third and fourth terms). Nonradiative exciton destruction (fifth term) is activated by `eQuenching` and `hQuenching`. An example is:

```
Physics(Region="EML-ETL") {
    SingletExciton(Recombination(Bimolecular eQuenching hQuenching))
}
```

For interfaces where the singlet exciton equation is solved only in one of the regions of the interface, the default boundary conditions are defined by [Eq. 127](#).

11: Transport in Metals, Organic Materials, and Disordered Media

Singlet Exciton Equation

To switch to zero flux boundary condition $\nabla n_{se} \cdot \hat{n} = 0$, the keyword `FluxBC` must be specified as an option for `SingletExciton` in the interface `Physics` section:

```
Physics(RegionInterface="Region_0/Region_2") {
    SingletExciton(FluxBC)
}
```

For heterointerfaces, the default barrier type for the singlet exciton flux injection across the interface (defined in [Eq. 128](#)) is the bandgap energy difference.

The barrier can be switched to conduction band or valence band type by specifying the keyword `BarrierType` with the option `CondBand` or `ValBand` in the `SingletExciton` section of the heterointerface `Physics` section:

```
Physics(RegionInterface="Region_0/Region_2") {
    SingletExciton(BarrierType(CondBand))
}
```

Parameters used with the singlet exciton equation must be specified in the `SingletExciton` section of the parameter file. [Table 24](#) lists these parameters with their default values.

Table 24 Singlet exciton equation parameters

Symbol	Parameter name	Default value	Location	Unit
γ	gamma	0.25	region	1
τ	tau	1×10^{-7}	region	s
τ_{trap}	tau_trap	1×10^{-8}	region	s
L_{diff}	l_diff	1×10^{-3}	region	cm
$D_{\text{se}} = L_{\text{diff}}^2 / \tau$			region	cm ² /s
$\sigma_{\text{se}-n}$	ex_cXsection	1×10^{-8}	region	cm ²
$\sigma_{\text{se}-p}$	ex_cXsection	1×10^{-8}	region	cm ²
E_{ex}	Eex	0.015	region	eV
g_{ex}	gex	4	region	1
$v_{\text{se},0}$	vth	1×10^7	region	cm/s
$v_{n,0}$	vth_car	1×10^3	region	cm/s
$v_{p,0}$	vth_car	1×10^3	region	cm/s
v_{si}	vel	1×10^8	interface	cm/s

Transport in Metals

The simulation of current transport in metals or semi-metals is important for interconnection problems in ICs. The current density in metals is given by:

$$\vec{J}_M = -\sigma(\nabla\Phi_M + P\nabla T) \quad (129)$$

where:

- σ is the metal conductivity.
- Φ_M is the Fermi potential in the metal.
- P is the metal thermoelectric power.
- T is the lattice temperature.

The second term in [Eq. 129](#) accounts for the Seebeck effect, and it is nonzero only when computation of metal thermoelectric power is enabled (see [Thermoelectric Power \(TEP\) on page 749](#)). For the steady-state case, $\nabla \cdot \vec{J}_M = 0$. Therefore, the equation for the Fermi potential inside of metals is:

$$\nabla \cdot (\sigma \nabla \Phi_M + P \nabla T) = 0 \quad (130)$$

The following temperature dependence is applied for $\rho = 1/\sigma$:

$$\rho = \rho_0(1 + \alpha_T(T - 273 \text{ K})) \quad (131)$$

All these resistivity parameters can be specified in the parameter file as:

```
Resistivity {
    * Resist(T) = Resist0 * ( 1 + TempCoef * ( T - 273 ) )
    Resist0 = <value>      # [Ohm*cm]
    TempCoef = <value>     # [1/K]
}
```

No specific keyword is required in the command file because Sentaurus Device recognizes all conductor regions and applies the appropriate equations to these regions and interfaces. The metal conductivity equation [Eq. 130](#) is a part of the Contact equation, which is solved automatically by default. If the keyword `NoAutomaticCircuitContact` is specified in the Math section or only the Poisson equation is solved, the Contact equation must be added explicitly in the Coupled statement to account for conductivity in metals.

To switch off current transport in metals, specify the keyword `-MetalConductivity` in the Math section.

Electric Boundary Conditions for Metals

The following boundary conditions are used:

- At contacts connected to metal regions, the Dirichlet condition $\Phi_M = V_{\text{applied}}$ is applied for the Fermi potential.
- Interface conditions always include the displacement current $\vec{J}_D$ to ensure conservation of current.
- For interfaces between metal and insulator, the equations are:

$$\begin{aligned}\vec{J}_M \cdot \hat{n} &= \vec{J}_D \cdot \hat{n} \\ \phi &= \Phi_M - \Phi_{MS}\end{aligned}\tag{132}$$

where Φ_{MS} is the workfunction difference between the metal and an intrinsic semiconductor selected (internally by Sentaurus Device) as a reference material, and $\hat{n}$ is the unit normal vector to the interface. The electrostatic potential inside metals is computed as $\phi = \Phi_M - \Phi_{MS}$.

- At metal–semiconductor interfaces, by default, there is an Ohmic boundary condition. Schottky boundary conditions can be selected as an option.

The Ohmic boundary condition at metal–semiconductor interfaces reads:

$$\begin{aligned}\vec{J}_M \cdot \hat{n} &= (\vec{J}_n + \vec{J}_p + \vec{J}_D) \cdot \hat{n} \\ \phi &= \Phi_M + \phi_0 \\ n &= n_0 \\ p &= p_0\end{aligned}\tag{133}$$

where:

- ϕ_0 is the equilibrium electrostatic potential (the built-in potential).
- n_0 and p_0 are the electron and hole equilibrium concentrations (see [Ohmic Contacts on page 217](#)).

There are several options for the Schottky interface (refer to the equations in [Schottky Contacts on page 219](#)). The potential barrier between metal and semiconductor is computed automatically and the barrier tunneling (see [Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612](#)) and barrier lowering (see [Barrier Lowering at Schottky Contacts on page 220](#)) models can be applied.

To select these models, use the following syntax in the command file:

```
Physics(MaterialInterface = "Metal/Silicon") { Schottky eRecVelocity=1e6
hRecVelocity=1e6 }
Physics(MaterialInterface = "Metal/Silicon") { Schottky BarrierLowering }
Physics(MaterialInterface = "Metal/Silicon") { Schottky }
```

The default values of the electron and hole surface recombination velocities are $eRecVel = 2.573e6$ and $hRecVel = 1.93e6$, respectively. Similarly to Electrode conditions, if Schottky is not specified but the SurfaceSRH keyword is specified, Sentaurus Device will apply Ohmic boundary conditions to the electrostatic potential and the surface recombination for both electron and hole carrier current densities on such interfaces.

NOTE At nonmetal interfaces, the keyword SurfaceSRH activates a different model from the one described in this section (see [Surface SRH Recombination on page 372](#)).

Both Ohmic and Schottky interfaces support a distributed resistance model similar to the one described in [Resistive Contacts on page 222](#). If a distributed resistance is specified at the interface, a distributed voltage drop $\Delta\phi(R_d) = R_d(J_n + J_p + J_D) \cdot \hat{n}$ is applied to the existing boundary condition for potential, where $\Delta\phi(R_d)$ is the voltage drop across the interface, and R_d is the distributed resistance at the interface. In addition, for Ohmic interfaces, emulation of a Schottky interface using a doping-dependent resistivity model is possible. These options are activated as following:

```
# Distributed resistance at Ohmic interface
Physics(MaterialInterface = "Metal/Silicon") {DistResist=1e-6}
# Distributed resistance at Schottky interface
Physics(MaterialInterface = "Metal/Silicon") {Schottky DistResist=1e-6}
# Distributed resistance at Ohmic interface emulating a Schottky interface
Physics(MaterialInterface = "Metal/Silicon")
{DistResist = SchottkyResist}
```

For Schottky interfaces, the distributed resistance model works with all other options previously described.

For all other metal boundaries, the Neumann condition $\vec{J}_M \cdot \hat{n} = 0$ is applied.

NOTE By default, Sentaurus Device computes output currents through a contact using integration over the associated wells. For metal contacts, the integration can be switched off and the local current can be used instead by using the keyword DirectCurrent in the Math section. This option may produce a wrong current at the metal contact.

Metal Workfunction

To specify the metal workfunction, use the Bandgap parameter set, for example:

```
Material = "Gold" {  
    Bandgap { WorkFunction = 5 # [eV] }  
}
```

It is also possible to account for workfunction variations within the metal.

If such variations can be characterized by mole-fraction variations and if the metal is treated as a mole fraction–dependent material (see [Mole-Fraction Materials on page 53](#)), then WorkFunction in the Bandgap parameter set can be specified as a mole fraction–dependent quantity, for example:

```
Material = "Metal" {  
    Bandgap {  
        Xmax(0)=0.0  
        Xmax(1)=0.2  
        WorkFunction(0) = 4.53# [eV]  
        WorkFunction(1) = 4.38# [eV]  
    }  
}
```

The positional dependency of the workfunction in metals also can be specified directly in the Physics section of the command file using one of three different methods:

```
Physics (Material = "<name>" | Region = "<name>") {  
    MetalWorkfunction (  
        [ Workfunction=<wf> ] |  
        [ Workfunction=(<wf1>, <x1>, <y1>, <z1>)  
          Workfunction=(<wf2>, <x2>, <y2>, <z2>)  
          ...  
          Workfunction=(<wfN>, <xN>, <yN>, <zN>) ] |  
        [ SFactor=<dataset-or-pmi_model> [Factor=<scale>] [Offset=<offset>] ]  
    )  
}
```

The first method (Workfunction=<wf>) simply specifies a constant workfunction value for the material or region that overrides any specification from the parameter file.

The next method allows the specification of an arbitrary number of workfunction–position quadruplets. The workfunction at a vertex in the metal is assigned the value of the workfunction from the quadruplet whose position is closest to the vertex.

Alternatively, the SFactor parameter can be used to specify a dataset name (which includes the PMI user fields PMIUserField0 through PMIUserField99) from which the vertex

workfunction values will be taken, or a space factor PMI model can be specified that calculates the vertex workfunction values directly (see [Space Factor on page 1105](#)). If Factor=<scale> is specified, the SFactor values are normalized and then are multiplied by this factor. If Offset=<offset> is specified, this value is added to the raw or scaled SFactor values.

Metal Workfunction Randomization

The workfunction in metal regions can be randomized using specifications in the command file:

```
Physics (Material = "<name>" | Region = "<name>") {
  MetalWorkfunction (
    Randomize (
      AverageGrainSize=<size>                # [micrometers]
      GrainProbability=(<P1>, <P2>, ...)
      GrainWorkfunction=(<wf1>, <wf2>, ...)  #[eV]
      RandomSeed=<seed>
      AtInsulatorInterface
      UniformDistribution
    )
  )
}
```

The approach taken assumes that the metal consists of randomized grains of varying size and shape that can be characterized with an average grain size. It also assumes that the grains in the metal occur with a finite number of orientations, and the number of grains for each orientation can be characterized with a probability of occurrence. All grains of the same orientation are assumed to have the same workfunction, but the workfunction can be different for each orientation.

AverageGrainSize is used to determine the average number of grains for a region, which then is used as the expectation value for a Poisson distribution, random number generator to determine the actual number of grains for the region. Each grain can have a different shape and size.

An arbitrary number of GrainProbability values can be specified, but their sum must be equal to one. In addition, there must be a one-to-one correspondence between GrainProbability values and GrainWorkfunction values.

By default, the seed for the random number generator used in the randomization process is different for every simulation. However, RandomSeed can be specified if required. This allows a particular randomization to be repeated if the same seed is used in a subsequent simulation of the same device on the same computer.

The `AtInsulatorInterface` option confines the randomization to the metal–insulator interface vertices. However, the resulting workfunction values at the interface will be exactly the same as those obtained when the entire metal region is randomized.

The `UniformDistribution` option *attempts* to use grains with a uniform size that are distributed uniformly. The success of this is highly dependent on the grid used in the metal region.

The workfunction in metal regions can be saved in a plot file for visualization by specifying `MetalWorkfunction` in the `Plot` section of the command file.

Temperature in Metals

In metals, only the lattice temperature is defined. If the lattice temperature is solved, in metals, it obeys the following equation:

$$c_L \frac{\partial T}{\partial t} - \nabla \cdot \kappa \nabla T = -\nabla \psi_M \cdot \vec{j}_M \quad (134)$$

where κ is the thermal conductivity (see [Thermal Conductivity on page 747](#)). If current transport in metals is switched off, the right-hand side of [Eq. 134](#) vanishes. However, a heat flow in metals can still be simulated.

Conductive Insulators

Leakage currents in dielectrics can be modeled by allowing dielectrics to have conductive properties. The conductive insulator (leaky insulator) model implements dielectrics that have nonzero (but typically low) conductivity (a metal-like property), besides the pure dielectric properties.

In a conductive insulator, the electrostatic potential is computed using the Poisson equation, similar to a pure dielectric. As in metals, conductivity is modeled by the Fermi potential (solving [Eq. 130](#)) and then the leakage current is computed using [Eq. 129](#). Since the model does not allow net charge in the conductive insulator, the Poisson equation remains unchanged by adding conductive properties to the insulator.

The equation for the Fermi potential inside a conductive insulator is the same used for metals (see [Eq. 130](#)), where σ is now the conductivity of the conductive insulator and $\Phi_M = \Phi_{CI}$ is the Fermi potential in the conductive insulator.

The following boundary conditions are used for this equation:

- At contacts connected to conductive insulator regions, the Dirichlet condition $\Phi_{CI} = V_{\text{applied}}$ is applied for the Fermi potential.
- Interface conditions always include the displacement current $\vec{J}_D$ in insulators, conductive insulators, and semiconductors to ensure conservation of current.
- For conductive insulator–semiconductor interfaces, Ohmic-like boundary conditions, thermionic-like boundary conditions, and boundary conditions for floating regions are available.

Leakage from or to a semiconductor to or from a conductive insulator is mainly determined by two factors: the conductive properties of the interface and the bulk resistivity of the conductive insulator. The bulk resistivity-limited conduction corresponds to the case when the current flow is limited by the high resistivity of the conductive insulator region, and the interface has zero resistivity. This case is modeled by the Ohmic-like boundary conditions:

$$\begin{aligned}\vec{J}_{CI} \cdot \hat{n} &= (\alpha \vec{J}_n + (1 - \alpha) \vec{J}_p) \cdot \hat{n} \\ \Phi_{CI} &= \phi - \phi_0\end{aligned}\tag{135}$$

where:

- Φ_{CI} and $\vec{J}_{CI}$ are the Fermi potential and current density on the interface.
- ϕ is the electrostatic potential (the solution of the Poisson equation in insulator and semiconductor regions).
- ϕ_0 is the built-in potential.
- $\hat{n}$ is the unit normal vector to the interface.
- $\vec{J}_n$ and $\vec{J}_p$ are the electron and hole currents on the semiconductor side of the interface.
- α is the fraction ($0 \leq \alpha \leq 1$) of the current density of the conductive insulator going to the electron current density.

Thermionic-like emission at the interface models the case where conduction is limited by both interface properties and the bulk resistivity of the conductive insulator. Thermionic boundary conditions read:

$$\begin{aligned}\vec{J}_{CI} \cdot \hat{n} &= (\vec{J}_{n, th} + \vec{J}_{p, th}) \cdot \hat{n} \\ \vec{J}_{n, th} &= aq \left[v_n(T_n) n - v_n(T) \left(\frac{T}{T_n} \right)^{1.5} N_C \exp \left(\frac{E_F^{CI} - E_C}{kT} \right) \right] \\ \vec{J}_{p, th} &= aq \left[v_p(T_p) p - v_p(T) \left(\frac{T}{T_p} \right)^{1.5} N_V \exp \left(\frac{E_V - E_F^{CI}}{kT} \right) \right]\end{aligned}\tag{136}$$

where:

- $\vec{J}_{n,th}$ and $\vec{J}_{p,th}$ are the electron and hole thermionic-emission current densities on the interface from the semiconductor side.
- $v_n(T) = (kT/2\pi m_n)^{0.5}$ and $v_p(T) = (kT/2\pi m_p)^{0.5}$ are ‘emission velocities’ at the interface.
- T , T_n , and T_p are the lattice, electron and hole temperatures, respectively.
- E_F^{CI} , E_C , and E_V are the conductive insulator Fermi-level energy, conduction band energy, and valence band energy, respectively.
- a is a constant set through the parameter file, which has the default value of 2.

By using a conductive insulator layer between a semiconductor floating region and a semiconductor region, leakage from a semiconductor floating-gate can be emulated. The boundary condition used in this case is:

$$\vec{J}_{CI} \cdot \hat{n} = -\sigma \nabla \Phi \quad (137)$$

where Φ is the Fermi potential close to the interface. In this case, during transient simulations, the charge update on the floating gate is computed as $J_{CI}\Delta t$, where J_{CI} is the total current at the conductive insulator–floating gate interface, and Δt is the current time step. The activation of this type of boundary condition is automatic if the boundary condition at the conductive insulator–semiconductor floating-gate interface is Ohmic and the simulation is transient (to allow for charging and discharging of the floating gate).

The model is activated by using the keyword `CondInsulator` in the `Physics` section to make the dielectric conductive and by adding the `CondInsulator` equation in the `Solve` section to actually solve the Fermi potential:

```
Physics(Region="insulator"){
    ...
    CondInsulator
    ...
}

Solve {
    ...
    Coupled {Poisson Electron Hole Contact CondInsulator}
    ...
}
```

The parameters for the resistivity of conductive insulators (see [Eq. 131](#)) are specified in the `Resistivity` parameter set:

```
Material = "Si3N4" {
    Resistivity {
        * Resist(T) = Resist0 * (1 + TempCoef * (T-273))
```

```

        Resist0 = 3.0e9      # [Ohm*cm]
        TempCoef = 43.0e-4   # [1/K]
    }
}

```

Boundary conditions according to [Eq. 135](#) are used by default. The parameter α in [Eq. 135](#) is given by the value of `curw` in the `CondInsCurr` parameter set:

```

MaterialInterface = "Si3N4/AlGaN" {
    CondInsCurr {
        curw = 1.0  # [1]
    }
}

```

Thermionic emission at the semiconductor–conductive insulator interface is enabled by adding `eThermionic` or `hThermionic` in the `Physics` section:

```

Physics(MaterialInterface="Si3N4/AlGaN") {
    *----- Insulator/AlGaN -----
    ...
    eThermionic
    hThermionic
    ...
}

```

where the parameter α (see [Eq. 136](#)) is specified in the parameter file:

```

MaterialInterface = "Si3N4/AlGaN" {
    ThermionicEmission {
        A = 2, 2
    }
}

```

For nonthermionic interfaces, an equilibrium solution is needed internally to solve the `CondInsulator` equation. The equilibrium solution is computed automatically by a nonlinear iteration. Sentaurus Device allows you to specify numeric parameters of the nonlinear iteration as options to the keyword `EquilibriumSolution` in the `Math` section (see [Table 173 on page 1254](#)). For example:

```

Math {
    ...
    EquilibriumSolution(Iterations=100)
    ...
}

```

11: Transport in Metals, Organic Materials, and Disordered Media
Conductive Insulators

CHAPTER 12 Semiconductor Band Structure

This chapter discusses the relevance of band structure to the simulation of semiconductor devices.

For device simulation, the most fundamental property of a semiconductor is its band structure. Realistic band structures are complex and can be fully accounted for only in Monte Carlo simulations (refer to the *Sentaurus Device Monte Carlo User Guide*). In Sentaurus Device, the band structure is simplified to four quantities: the energies of the conduction and valence band edges (or, in a different parameterization, band gap and electron affinity), and the density-of-states masses for electrons and holes (or, parameterized differently, the band edge density-of-states). The models for these quantities are discussed in [Band Gap and Electron Affinity](#) and [Effective Masses and Effective Density-of-States](#) on page 261.

Intrinsic Density

The band gap and band edge density-of-states are summarized in the intrinsic density $n_i(T)$ (for undoped semiconductors):

$$n_i(T) = \sqrt{N_C(T)N_V(T)} \exp\left(-\frac{E_g(T)}{2kT}\right) \quad (138)$$

and the effective intrinsic density (including doping-dependent bandgap narrowing):

$$n_{i, \text{eff}} = n_i \exp\left(\frac{E_{\text{bgn}}}{2kT}\right) \quad (139)$$

In devices that contain different materials, the electron affinity χ (see [Band Gap and Electron Affinity](#)) is also important. Along with the band gap, it determines the alignment of conduction and valence bands at material interfaces.

Band Gap and Electron Affinity

The band gap is the difference between the lowest energy in the conduction band and the highest energy in the valence band. The electron affinity is the difference between the lowest energy in the conduction band and the vacuum level.

Selecting the Bandgap Model

Sentaurus Device supports different bandgap models: BennettWilson, delAlamo, OldSlotboom, and Slotboom (the same model with different default parameters), JainRoulston, and TableBGN. The bandgap model can be selected in the EffectiveIntrinsicDensity statement in the Physics section, for example:

```
Physics {  
    EffectiveIntrinsicDensity(BandGapNarrowing (Slotboom))  
}
```

activates the Slotboom model. The default model is BennettWilson.

By default, bandgap narrowing is active. Bandgap narrowing can be switched off with the keyword NoBandGapNarrowing:

```
Physics {  
    EffectiveIntrinsicDensity(NoBandGapNarrowing)  
}
```

NOTE To plot the band gap including the bandgap narrowing, specify EffectiveBandGap in the Plot section of the command file. The quantity that Sentaurus Device plots when BandGap is specified does not include bandgap narrowing.

[Table 27 on page 260](#) and [Table 28 on page 261](#) list the model parameters available for calibration (see [Bandgap and Electron-Affinity Models](#)).

Bandgap and Electron-Affinity Models

Sentaurus Device models the lattice temperature-dependence of the band gap as [\[1\]](#):

$$E_g(T) = E_g(0) - \frac{\alpha T^2}{T + \beta} \quad (140)$$

where $E_g(0)$ is the bandgap energy at 0 K, and α and β are material parameters (see [Table 27 on page 260](#)).

To allow $E_g(0)$ to differ for different bandgap models, it is written as:

$$E_g(0) = E_{g,0} + \delta E_{g,0} \quad (141)$$

$E_{g,0}$ is an adjustable parameter common to all models. For the `TableBGN` and `JainRoulston` models, $\delta E_{g,0} = 0$. Each of the other models offers its own adjustable parameter for $\delta E_{g,0}$ (see [Table 27 on page 260](#)).

The effective band gap results from the band gap reduced by bandgap narrowing:

$$E_{g,\text{eff}}(T) = E_g(T) - E_{\text{bgn}} \quad (142)$$

The electron affinity χ is the energy separation between the conduction band and the vacuum. For `BennettWilson`, `deLamo`, `OldSlotboom`, `Slotboom`, and `TableBGN`, the affinity is temperature dependent and is affected by bandgap narrowing:

$$\chi(T) = \chi_0 + \frac{(\alpha + \alpha_2)T^2}{2(T + \beta + \beta_2)} + \text{Bgn2Chi} \cdot E_{\text{bgn}} \quad (143)$$

where χ_0 and `Bgn2Chi` are adjustable parameters (see [Table 27 on page 260](#)). `Bgn2Chi` defaults to 0.5 and, therefore, bandgap narrowing splits equally between conduction and valence bands.

In the case of the `JainRoulston` model, the electron affinity depends also on the doping concentration (especially at high dopings) because bandgap narrowing is doping dependent:

$$\chi(T, N_{A,0}, N_{D,0}) = \chi_0 + \frac{(\alpha + \alpha_2)T^2}{2(T + \beta + \beta_2)} + E_{\text{bgn}}^{\text{cond}} \quad (144)$$

where $E_{\text{bgn}}^{\text{cond}}$ is the shift in the conduction band due to the `JainRoulston` bandgap narrowing and χ_0 is an adjustable parameter as previously defined.

The main difference of the bandgap models is how they handle bandgap narrowing. Bandgap narrowing in Sentaurus Device has the form:

$$E_{\text{bgn}} = \Delta E_g^0 + \Delta E_g^{\text{Fermi}} \quad (145)$$

where ΔE_g^0 is determined by the particular bandgap narrowing model used, and $\Delta E_g^{\text{Fermi}}$ is an optional correction to account for carrier statistics (see [Eq. 149](#)).

Bandgap Narrowing for Bennett–Wilson Model

Bandgap narrowing for the Bennett–Wilson [\[2\]](#) model (keyword `BennettWilson`) in Sentaurus Device reads:

$$\Delta E_g^0 = \begin{cases} E_{\text{ref}} \left[\ln \left(\frac{N_{\text{tot}}}{N_{\text{ref}}} \right) \right]^2 & N_{\text{tot}} \geq N_{\text{ref}} \\ 0 & \text{otherwise} \end{cases} \quad (146)$$

12: Semiconductor Band Structure

Band Gap and Electron Affinity

The model was developed from absorption and luminescence data of heavily doped n-type materials. The material parameters E_{ref} and N_{ref} are accessible in the Bennett parameter set in the parameter file of Sentaurus Device (see [Table 28 on page 261](#)).

Bandgap Narrowing for Slotboom Model

Bandgap narrowing for the Slotboom model (keyword `Slotboom` or `OldSlotboom`) (the only difference is in the parameters) in Sentaurus Device reads:

$$\Delta E_g^0 = E_{\text{ref}} \left[\ln \left(\frac{N_{\text{tot}}}{N_{\text{ref}}} \right) + \sqrt{\left(\ln \left(\frac{N_{\text{tot}}}{N_{\text{ref}}} \right) \right)^2 + 0.5} \right] \quad (147)$$

The models are based on measurements of $\mu_n n_i^2$ in n-p-n transistors (or $\mu_p n_i^2$ in p-n-p transistors) with different base doping concentrations and a 1D model for the collector current [3]–[6]. The material parameters E_{ref} and N_{ref} are accessible in the `Slotboom` and `OldSlotboom` parameter sets in the parameter file (see [Table 28 on page 261](#)).

Bandgap Narrowing for del Alamo Model

Bandgap narrowing for the del Alamo model (keyword `delAlamo`) in Sentaurus Device reads:

$$\Delta E_g^0 = \begin{cases} E_{\text{ref}} \ln \left(\frac{N_{\text{tot}}}{N_{\text{ref}}} \right) & N_{\text{tot}} \geq N_{\text{ref}} \\ 0 & \text{otherwise} \end{cases} \quad (148)$$

This model was proposed [7]–[11] for n-type materials. The material parameters E_{ref} and N_{ref} are accessible in the `delAlamo` parameter set in the parameter file (see [Table 28 on page 261](#)).

Bandgap Narrowing for Jain–Roulston Model

Bandgap narrowing for the Jain–Roulston model (keyword `JainRoulston`) in Sentaurus Device is implemented based on the literature [12] and is given by:

$$\Delta E_g^0 = A \cdot N_{\text{tot}}^{1/3} + B \cdot N_{\text{tot}}^{1/4} + C \cdot N_{\text{tot}}^{1/2} + D \cdot N_{\text{tot}}^{1/2} \quad (149)$$

where A , B , C , and D are material-dependent coefficients that can be specified in the parameter file.

The coefficients A , B , C , and D are derived from the quantities defined and described in the literature [12]:

$$A = 1.83 \frac{\Lambda}{N_b^{1/3}} \frac{aR}{\left(\frac{3}{4\pi}\right)^{1/3}} \quad [\text{eV} \cdot \text{cm}] \quad (150)$$

$$B = \left((0.95Ra^{3/4}) / \left(\frac{3}{4\pi}\right)^{1/4} \right) \quad [\text{eV} \cdot \text{cm}^{3/4}] \quad (151)$$

$$C = \frac{1.57Ra^{3/2}}{N_b \left(\frac{3}{4\pi}\right)^{1/2}} \quad [\text{eV} \cdot \text{cm}^{3/2}] \quad (152)$$

$$D = \frac{1.57R_{(\text{mino})}a^{3/2}}{N_b \left(\frac{3}{4\pi}\right)^{1/2}} \quad [\text{eV} \cdot \text{cm}^{3/2}] \quad (153)$$

where a is the effective Bohr radius, R is the Rydberg energy, N_b is the number of valleys in the conduction or valence band, $R_{(\text{mino})}$ is the Rydberg energy for the minority carrier band, and Λ is a correction factor.

In general, papers on bandgap narrowing define the Jain–Roulston bandgap narrowing as $\Delta E_g^0 = C_1 \cdot N_{\text{tot}}^{1/3} + C_2 \cdot N_{\text{tot}}^{1/4} + C_3 \cdot N_{\text{tot}}^{1/2}$. Comparing this definition to Eq. 149, the coefficients C_1 , C_2 , and C_3 can be mapped to the ones of Sentaurus Device: $C_1 = A$, $C_2 = B$, and $C_3 = C + D$.

If, on the other hand, C_1 , C_2 , and C_3 are given, then computing the corresponding Sentaurus Device coefficients A , B , C , and D requires splitting C_3 into C and D using Eq. 152 and Eq. 153.

Sentaurus Device needs four coefficients instead of three to compute $E_{\text{bgn}}^{\text{cond}}$ and the doping-dependent affinity (Eq. 144) internally. The expression of $E_{\text{bgn}}^{\text{cond}}$ is given by:

$$E_{\text{bgn}}^{\text{cond}} = \begin{cases} A \cdot N_{\text{tot}}^{1/3} + C \cdot N_{\text{tot}}^{1/2} & N_{D,0} > N_{A,0} \\ B \cdot N_{\text{tot}}^{1/4} + D \cdot N_{\text{tot}}^{1/2} & \text{otherwise} \end{cases} \quad (154)$$

12: Semiconductor Band Structure

Band Gap and Electron Affinity

The coefficients A , B , C , and D are specified in the JainRoulston section of the parameter file:

```
Material = "Silicon" {  
  JainRoulston {  
    * n-type  
    A_n = 1.02e-8      # [eV cm]  
    B_n = 4.15e-7      # [eV cm^(3/4)]  
    C_n = 1.45e-12     # [eV cm^(3/2)]  
    D_n = 1.48e-12     # [eV cm^(3/2)]  
    * p-type  
    A_p = 1.11e-8      # [eV cm]  
    B_p = 4.79e-7      # [eV cm^(3/4)]  
    C_p = 3.23e-12     # [eV cm^(3/2)]  
    D_p = 1.81e-12     # [eV cm^(3/2)]  
  }  
}
```

The default values of the coefficients are listed in [Table 25](#).

Table 25 Default parameters for Jain–Roulston bandgap narrowing model

Symbol	Parameter name	Default value for material				Unit
		Si	Ge	GaAs	All others	
A	A_n	1.02e-8	7.30e-9	1.65e-8	0.0	eVcm
	A_p	1.11e-8	8.21e-9	9.77e-9	0.0	eVcm
B	B_n	4.15e-7	2.57e-7	2.38e-7	0.0	eVcm ^{3/4}
	B_p	4.79e-7	2.91e-7	3.87e-7	0.0	eVcm ^{3/4}
C	C_n	1.45e-12	2.29e-12	1.83e-11	0.0	eVcm ^{3/2}
	C_p	3.23e-12	3.58e-12	3.41e-12	0.0	eVcm ^{3/2}
D	D_n	1.48e-12	2.03e-12	7.25e-11	0.0	eVcm ^{3/2}
	D_p	1.81e-12	2.19e-12	4.84e-13	0.0	eVcm ^{3/2}

Table Specification of Bandgap Narrowing

It is possible to specify bandgap narrowing by using a table, which can be defined in the TableBGN parameter set. This table gives the value of bandgap narrowing as a function of donor or acceptor concentration, or total concentration (the sum of acceptor and donor concentrations).

When specifying acceptor and donor concentrations, the total bandgap narrowing is the sum of the contributions of the two dopant types. If only acceptor or only donor entries are present in the table, the bandgap narrowing contribution for the missing dopant type vanishes. Total concentration and donor or acceptor concentration must not be specified in the same table.

Each table entry is a line that specifies a concentration type (Donor, Acceptor, or Total) with a concentration in cm^{-3} , and the bandgap narrowing for this concentration in eV. The actual bandgap narrowing contribution for each concentration type is interpolated from the table, using a scheme that is piecewise linear in the logarithm of the concentration.

For concentrations below (or above) the range covered by table entries, the bandgap narrowing of the entry for the smallest (or greatest) concentration is assumed, for example:

```
TableBGN {
    Total 1e16, 0
    Total 1e20, 0.02
}
```

means that for total doping concentrations below 10^{16} cm^{-3} , the bandgap narrowing vanishes, then increases up to 20 meV at 10^{20} cm^{-3} concentration and maintains this value for even greater concentrations. The interpolation is such that, in this example, the bandgap narrowing at 10^{18} cm^{-3} is 10 meV.

Tabulated default parameters for bandgap narrowing are available only for GaAs. For all other materials, you can specify the tabulated data in the parameter file.

NOTE It is not possible to specify mole fraction–dependent bandgap narrowing tables. In particular, Sentaurus Device does not relate the parameters for ternary compound semiconductor materials to those of the related binary materials.

Schenk Bandgap Narrowing Model

BennettWilson, deAlamo, OldSlotboom, Slotboom, JainRoulston, and TableBGN are all doping-induced bandgap narrowing models. They do not depend on carrier concentrations. High carrier concentrations produced in optical excitation or high electric field injection can also cause bandgap narrowing. This effect is referred to as plasma-induced bandgap narrowing.

The Schenk bandgap narrowing model described in [13] also takes into account the plasma-induced narrowing effect in silicon. In this model, the bandgap narrowing is the sum of two parts:

- An exchange-correlation part, which is a function of plasma density and temperature.
- An ionic part, which is a function of activated doping concentration, plasma density, and temperature.

Sentaurus Device supports two versions of the Schenk model: a simplified version where bandgap narrowing is computed at $T = 0$ K and does not depend on temperature, and the full model where temperature dependence is accounted for. You can switch from the simplified model (the default) to the full model by setting `IsSimplified=-1` in the Schenk bandgap narrowing section of the parameter file.

The full-model exchange correlation and ionic terms are described by:

$$\Delta_a^{xc}(n, p, T) = -\frac{(4\pi)^3 n_\Sigma^2 \left[\left(\frac{48n_a}{\pi g_a} \right)^{1/3} + c_a \ln(1 + d_a n_p^{p_a}) \right] + \left(\frac{8\pi\alpha_a}{g_a} \right) n_a \Upsilon^2 + \sqrt{8\pi n_\Sigma} \Upsilon^{5/2}}{(4\pi)^3 n_\Sigma^2 + \Upsilon^3 + b_a \sqrt{n_\Sigma} \Upsilon^2 + 40 n_\Sigma^{3/2} \Upsilon} \quad (155)$$

$$\Delta_a^i(N_{tot}, n, p, T) = -\frac{N_{tot} [1 + U^i(n_\Sigma, \Upsilon)]}{\sqrt{\Upsilon n_\Sigma / (2\pi)} [1 + h_a \ln(1 + \sqrt{n_\Sigma} / \Upsilon) + j_a U^i(n_\Sigma, \Upsilon) n_p^{3/4} (1 + k_a n_p^{q_a})]} \quad (156)$$

and, for the simplified model by:

$$\Delta_a^{xc}(n, p, T=0) = -\left[\left(\frac{48n_a}{\pi g_a} \right)^{1/3} + c_a \ln(1 + d_a n_p^{p_a}) \right] \quad (157)$$

$$\Delta_a^i(N_{tot}, n, p, T=0) = -N_{tot} \frac{0.799\alpha_a}{n_p^{3/4}} \quad (158)$$

where $a = e, h$ is the index for the carrier type, $\Upsilon = kT/Ry_{ex}$, $n_\Sigma = n + p$, $n_p = \alpha_e n + \alpha_h p$, and $U^i(n_\Sigma, \Upsilon) = n_\Sigma^2 / \Upsilon^3$. The default values (silicon) and units for the parameters that can be changed in Sentaurus Device in this model are listed in [Table 26 on page 258](#).

Sentaurus Device implements the Schenk bandgap narrowing model using the framework of the density gradient model (see [Density Gradient Model on page 288](#)). If you want to suppress the quantization corrections contained in this model, set the density gradient model parameter $\gamma = 0$ (see [Eq. 209, p. 288](#)).

The Schenk model is switched on separately for electrons (conduction band correction) and holes (valence band correction). The complete bandgap narrowing correction is the sum of

conduction and valence band corrections, and it can be obtained by switching on the Schenk model for both electrons and holes.

First, in the Physics section of silicon regions, SchenkBGN_elec must be specified as the LocalModel (that is, the model for Λ_{PMI} in Eq. 209, p. 288) for eQuantumPotential (conduction bandgap correction), and SchenkBGN_hole must be specified as the LocalModel for hQuantumPotential (valence band correction). In addition, because the Schenk model is a bandgap narrowing model itself, all other models must be switched off by specifying the keyword NoBandGapNarrowing as the argument for EffectiveIntrinsicDensity:

```
Physics(Region = "Region1_Si") {
    ...
    EffectiveIntrinsicDensity (NoBandGapNarrowing)
    eQuantumPotential(LocalModel=SchenkBGN_elec)
    hQuantumPotential(LocalModel=SchenkBGN_hole)
    ...
}
```

Apart from switching on quantum corrections in the Physics section, the equations for quantum corrections must be solved to compute the corrections. This is performed by specifying eQuantumPotential (for electrons) or hQuantumPotential (for holes) or both in the Solve section:

```
Solve {
    Coupled (LineSearchDamping=0.01){Poisson eQuantumPotential}
    Coupled {Poisson eQuantumPotential hQuantumPotential}
    quasistationary ( Goal {name="base" voltage=0.4}
                     Goal {name="collector" voltage=2.0}
                     Initialstep=0.1 Maxstep=0.1 Minstep=1e-6
                     )
    {coupled {poisson electron hole eQuantumPotential hQuantumPotential}}
}
```

Finally, to ignore the quantization corrections by the density gradient model, in the parameter file, γ is set to zero. Then, the Schenk model parameter IsSimplified is set to 1 if the simplified model ($T = 0\text{ K}$) is used or -1 for the full model (temperature dependent):

```
Material = "Silicon" {
    ...
    *turn off everything in density gradient model except
    *apparent band-edge shift
    QuantumPotentialParameters {gamma = 0 , 0}
    ...
    SchenkBGN_elec {
        ...
        * Selects simplified model (1) or complete Schenk model (-1)
        IsSimplified = 1
    }
}
```

12: Semiconductor Band Structure

Band Gap and Electron Affinity

```
}
SchenkBGN_hole {
    ...
    * Selects simplified model (1) or complete Schenk model (-1)
    IsSimplified = 1
}
}
```

The Schenk bandgap narrowing model is specifically for silicon. Sentaurus Device permits the model parameters to be changed in the parameter file (the SchenkBGN_elec and SchenkBGN_hole sections) as an option for also using this bandgap narrowing model for other materials. A full description of the parameters is given in the literature [13] and their default values are summarized in Table 26.

The Schenk bandgap narrowing can be visualized by specifying the data entry eSchenkBGN or hSchenkBGN or both in the Plot section of the command file.

Table 26 Default parameters for Schenk bandgap narrowing model

Symbol	Parameter name	Default value (silicon)	Unit
SchenkBGN_elec			
α_e	alpha_e	0.5187	1
g_e	g_e	12	1
Ry_{ex}	Ry_ex	16.55	meV
a_{ex}	a_ex	3.719e-7	cm
b_e	b_e	8	1
c_e	c_e	1.3346	1
d_e	d_e	0.893	1
p_e	p_e	0.2333	1
h_e	h_e	3.91	1
j_e	j_e	2.8585	1
k_e	k_e	0.012	1
q_e	q_e	0.75	1
SchenkBGN_hole			
α_h	alpha_h	0.4813	1
g_h	g_h	4	1
Ry_{ex}	Ry_ex	16.55	meV

Table 26 Default parameters for Schenk bandgap narrowing model

Symbol	Parameter name	Default value (silicon)	Unit
a_{ex}	a_ex	3.719e-7	cm
b_h	b_h	1	1
c_h	c_h	1.2365	1
d_h	d_h	1.1530	1
p_h	p_h	0.2333	1
h_h	h_h	4.20	1
j_h	j_h	2.9307	1
k_h	k_h	0.19	1
q_h	q_h	0.25	1

Bandgap Narrowing with Fermi Statistics

Parameters for bandgap narrowing are often extracted from experimental data assuming Maxwell–Boltzmann statistics. However, in the high-doping regime for which bandgap narrowing is important, Maxwell–Boltzmann statistics differs significantly from the more realistic Fermi statistics.

The bandgap narrowing parameters are, therefore, systematically affected by using the wrong statistics to interpret the experiment. For use in simulations that do not use Fermi statistics, this ‘error’ in the parameters is desirable, as it partially compensates the error by using the ‘wrong’ statistics in the simulation. However, for simulations using Fermi statistics, this compensation does not occur.

Therefore, Sentaurus Device can apply a correction to the bandgap narrowing to reduce the errors introduced by using Maxwell–Boltzmann statistics for the interpretation of experiments on bandgap narrowing (see [Eq. 145](#)):

$$\Delta E_g^{\text{Fermi}} = k300\text{K} \left[\ln\left(\frac{N_V N_C}{N_{A,0} N_{D,0}}\right) + F_{1/2}^{-1}\left(\frac{N_{A,0}}{N_V}\right) + F_{1/2}^{-1}\left(\frac{N_{D,0}}{N_C}\right) \right] \quad (159)$$

where the right-hand side is evaluated at 300 K.

By default, correction [Eq. 159](#) is switched on for simulations using Fermi statistics and switched off (that is, $\Delta E_g^{\text{Fermi}} = 0$) for simulations using Maxwell–Boltzmann statistics. To switch off the correction in simulations using Fermi statistics, specify `EffectiveIntrinsicDensity(NoFermi)` in the `Physics` section of the command file. This is recommended if you use parameters for bandgap narrowing that have been extracted

assuming Fermi statistics. Sometimes for III–IV materials, the correction Eq. 159 is too large, and it is recommended to switch it off in these cases.

Bandgap Parameters

The band gap $E_{g,0}$ and values of $\delta E_{g,0}$ for each model are accessible in the BandGap section of the parameter file, in addition to the electron affinity χ_0 and the temperature coefficients α and β . As an extension to what Eq. 140 and Eq. 143 suggest, the parameters $E_{g,0}$ and χ_0 can be specified at any reference temperature T_{par} . By default, $T_{\text{par}} = 0\text{ K}$.

To prevent abnormally low (or negative) band gap, the calculated value of the band gap E_g (which may include bandgap narrowing and stress effects) is limited if $E_g < E_{g,\text{min}} + \delta E_{g,\text{min}}$:

$$E_{g,\text{limited}} = E_{g,\text{min}} + \delta E_{g,\text{min}} \exp[(E_g - E_{g,\text{min}} - \delta E_{g,\text{min}})/\delta E_{g,\text{min}}] \quad (160)$$

Table 27 summarizes the default parameter values for silicon.

Table 27 Bandgap models: Default parameters for silicon

Symbol	Parameter name	Default	Unit	Band gap at 0 K $E_{g,0} + \delta E_{g,0} + \frac{\alpha T_{\text{par}}^2}{\beta + T_{\text{par}}}$	Reference
$E_{g,0}$	Eg0	1.1696	eV	–	Eq. 141, p. 250
$\delta E_{g,0}$	dEg0(Bennett)	0.0	eV	1.1696	Eq. 141
	dEg0(Slotboom)	-4.795×10^{-3}	eV	1.1648	
	dEg0(OldSlotboom)	-1.595×10^{-2}	eV	1.1537	
	dEg0(delAlamo)	-1.407×10^{-2}	eV	1.1556	
α	alpha	4.73×10^{-4}	eV /K	–	Eq. 140, p. 250, Eq. 143, p. 251
α_2	alpha2	0	eV /K	–	Eq. 143
β	beta	636	K	–	Eq. 140, Eq. 143
β_2	beta2	0	K	–	Eq. 143
χ_0	Chi0	4.05	eV	–	Eq. 143
Bgn2Chi	Bgn2Chi	0.5	1	–	Eq. 143
T_{par}	Tpar	0	K	–	
$E_{g,\text{min}}$	EgMin	0	eV	–	Eq. 160
$\delta E_{g,\text{min}}$	dEgMin	0.01	eV	–	Eq. 160

[Table 28](#) summarizes the silicon default parameters for the analytic bandgap narrowing models available in Sentaurus Device.

Table 28 Bandgap narrowing models: Default parameters for silicon

Symbol	Parameter	Bennett	Slotboom	Old Slotboom	del Alamo	Unit
E_{ref}	Ebgn	6.84×10^{-3}	6.92×10^{-3}	9.0×10^{-3}	18.7×10^{-3}	eV
N_{ref}	Nref	3.162×10^{18}	1.3×10^{17}	1.0×10^{17}	7.0×10^{17}	cm^{-3}

Effective Masses and Effective Density-of-States

Sentaurus Device provides two options for computing carrier effective masses and densities of states. The first method, selected by specifying `Formula=1` in the parameter file, computes an effective density-of-states (DOS) as a function of carrier effective mass. The effective mass may be either independent of temperature or a function of the temperature-dependent band gap. The latter is the most appropriate model for carriers in silicon and is the default for simulations of silicon devices.

In the second method, selected by specifying `Formula=2` in the parameter file the effective carrier mass is computed as a function of a temperature-dependent density-of-states. The default for simulations of GaAs devices is `Formula=2`.

Electron Effective Mass and DOS

Formula 1

The lattice temperature-dependence of the DOS effective mass of electrons is modeled by:

$$m_n = 6^{2/3} (m_t^2 m_l)^{1/3} + m_m \quad (161)$$

where the temperature-dependent effective mass component $m_t(T)$ is best described in silicon by the temperature dependence of the energy gap [\[14\]](#):

$$\frac{m_t(T)}{m_0} = a \frac{E_g(0)}{E_g(T)} \quad (162)$$

The coefficient a and the mass m_l are defined in the parameter file with the default values provided in [Table 29 on page 262](#). The parameter m_m , which defaults to zero, allows m_n to be defined as a temperature-independent quantity if required.

The effective densities of states (DOS) in the conduction band N_C follows from:

$$N_C(m_n, T_n) = 2.5094 \times 10^{19} \left(\frac{m_n}{m_0} \right)^{\frac{3}{2}} \left(\frac{T_n}{300\text{K}} \right)^{\frac{3}{2}} \text{cm}^{-3} \quad (163)$$

Formula 2

If Formula=2 is specified in the parameter file, the value for the DOS is computed from $N_C(300\text{K})$, which is read from the parameter file:

$$N_C(T_n) = N_C(300\text{K}) \left(\frac{T_n}{300\text{K}} \right)^{\frac{3}{2}} \quad (164)$$

and the electron effective mass is simply a function of $N_C(300\text{K})$:

$$\frac{m_n}{m_0} = \left(\frac{N_C(300\text{K})}{2.5094 \times 10^{19} \text{cm}^{-3}} \right)^{\frac{2}{3}} \quad (165)$$

Electron Effective Mass and Conduction Band DOS Parameters

[Table 29](#) lists the default coefficients for the electron effective mass and conduction band DOS models. The values can be modified in the eDOSMass parameter set.

NOTE The default setting for the Formula parameter depends on the materials, for example, it is equal to 1 for silicon and 2 for GaAs.

Table 29 Default coefficients for effective electron mass and DOS models

Option	Symbol	Parameter name	Electrons	Unit
Formula=1	a	a	0.1905	1
	m_l	ml	0.9163	1
	m_m	mm	0	1
Formula=2	$N_C(300\text{K})$	Nc300	2.890×10^{19}	cm^{-3}

Hole Effective Mass and DOS

Formula 1

For the DOS effective mass of holes, the best fit in silicon is provided by the expression [15]:

$$\frac{m_p(T)}{m_0} = \left(\frac{a + bT + cT^2 + dT^3 + eT^4}{1 + fT + gT^2 + hT^3 + iT^4} \right)^{\frac{2}{3}} + m_m \quad (166)$$

where the coefficients are listed in [Table 30 on page 264](#). The parameter m_m , which defaults to zero, allows m_p to be defined as a temperature-independent quantity. The effective DOS for holes N_V follows from:

$$N_V(m_p, T_p) = 2.5094 \times 10^{19} \left(\frac{m_p}{m_0} \right)^{\frac{3}{2}} \left(\frac{T_p}{300 \text{ K}} \right)^{\frac{3}{2}} \text{ cm}^{-3} \quad (167)$$

Formula 2

If Formula=2 in the parameter file, the temperature-dependent DOS is computed from $N_V(300 \text{ K})$ as given in the parameter file:

$$N_V(T_p) = N_V(300 \text{ K}) \left(\frac{T_p}{300 \text{ K}} \right)^{\frac{3}{2}} \quad (168)$$

and the effective hole mass is given by:

$$\frac{m_p}{m_0} = \left(\frac{N_V(300 \text{ K})}{2.5094 \times 10^{19} \text{ cm}^{-3}} \right)^{\frac{2}{3}} \quad (169)$$

Hole Effective Mass and Valence Band DOS Parameters

The model coefficients for the hole effective mass and valence band DOS can be modified in the parameter set hDOSMass. [Table 30](#) lists the default parameter values.

NOTE The default setting for the Formula parameter depends on the materials, for example, it is equal to 1 for silicon and it is equal to 2 for GaAs.

Table 30 Default coefficients for hole effective mass and DOS models

Option	Symbol	Parameter name	Holes	Unit
Formula=1	a	a	0.4435870	1
	b	b	0.3609528×10^{-2}	K^{-1}
	c	c	0.1173515×10^{-3}	K^{-2}
	d	d	0.1263218×10^{-5}	K^{-3}
	e	e	0.3025581×10^{-8}	K^{-4}
	f	f	0.4683382×10^{-2}	K^{-1}
	g	g	0.2286895×10^{-3}	K^{-2}
	h	h	0.7469271×10^{-6}	K^{-3}
	i	i	0.1727481×10^{-8}	K^{-4}
	m_m	mm	0	1
Formula=2	$N_V(300\text{ K})$	Nv300	3.140×10^{19}	cm^{-3}

Gaussian Density-of-States for Organic Semiconductors

Gaussian density-of-states (DOS) has been introduced to better represent electron and hole effective DOS in disordered organic semiconductors. Within the Gaussian disorder model, electron and hole DOS are represented by:

$$\Gamma(E) = \frac{N_t}{\sqrt{2\pi}\sigma_{\text{DOS}}} \exp\left(-\frac{(E-E_0)^2}{2\sigma_{\text{DOS}}^2}\right) \quad (170)$$

where N_t is the total number of hopping sites with $N_t = N_{\text{LUMO}}$ for electrons (LUMO is the lowest unoccupied molecular orbital) and $N_t = N_{\text{HOMO}}$ for holes (HOMO is the highest occupied molecular orbital), E_0 and σ_{DOS} are the energy center and the width of the DOS distribution, respectively.

Electron and hole densities in the most general case (Fermi–Dirac statistics) then are computed using Gaussian distribution from [Eq. 170](#), p. 264 as:

$$\frac{n}{N_{\text{LUMO}}} = \frac{1}{\sqrt{2\pi}\sigma_n} \int_{-\infty}^{\infty} \exp\left(-\frac{(E-E_{0n})^2}{2\sigma_n^2}\right) \frac{1}{1 + e^{(E-E_{F,n})/(kT)}} dE = G_n(\zeta_n; \hat{s}_n) \quad (171)$$

$$\frac{p}{N_{\text{HOMO}}} = \frac{1}{\sqrt{2\pi}\sigma_p} \int_{-\infty}^{\infty} \exp\left(-\frac{(E-E_{0p})^2}{2\sigma_p^2}\right) \frac{1}{1 + e^{(E_{F,p}-E)/(kT)}} dE = G_p(\zeta_p; \hat{s}_p) \quad (172)$$

where $\zeta_n = \frac{E_{F,n} - E_{0n}}{kT}$, $\zeta_p = \frac{E_{0p} - E_{F,p}}{kT}$, $\hat{s}_n = \frac{\sigma_n}{kT}$, $\hat{s}_p = \frac{\sigma_p}{kT}$.

Sentaurus Device computes Gauss–Fermi integrals $G_n(\zeta_n; \hat{s}_n)$ and $G_p(\zeta_p; \hat{s}_p)$ using an analytic approximation [\[16\]](#). The approximation covers both the nondegenerate and degenerate cases:

$$G_c(\zeta_c; \hat{s}_c) = \begin{cases} \frac{\exp\left(\frac{\hat{s}_c^2}{2} + \zeta_c\right)}{1 + \exp(K(\hat{s}_c)(\zeta_c + \hat{s}_c^2))} & \zeta_c \leq -\hat{s}_c^2 \quad (\text{nondegenerate region}) \\ \frac{1}{2} \operatorname{erfc}\left(-\frac{\zeta_c}{\hat{s}_c \sqrt{2}} H(\hat{s}_c)\right) & \zeta_c > -\hat{s}_c^2 \quad (\text{degenerate region}) \end{cases} \quad (173)$$

where the index c is n or p , and the analytic functions H and K are given by:

$$H(s) = \frac{\sqrt{2}}{s} \operatorname{erfc}^{-1}\left(\exp\left(-\frac{s^2}{2}\right)\right) \quad (174)$$

$$K(s) = 2\left(1 - \frac{H(s)}{s} \sqrt{\frac{2}{\pi}} \exp\left[\frac{1}{2}s^2(1 - H^2(s))\right]\right) \quad (175)$$

The model is activated regionwise, materialwise, or globally by specifying the keyword `GaussianDOS_Full` in the respective `Physics` section together with the `Fermi` keyword in the global `Physics` section:

```
Physics(Region="Organic_sem1") {
    GaussianDOS_full
}

Physics {
    Fermi
}
```

The model parameters used in [Eq. 171](#) and [Eq. 172](#) are listed in [Table 31 on page 267](#). They can be specified in the `GaussianDOS_full` section of the parameter file.

12: Semiconductor Band Structure

Effective Masses and Effective Density-of-States

In the case of nondegenerate semiconductors (Maxwell–Boltzmann approximation can be used for carrier densities), a simplified version based on a correction of standard densities-of-states N_C and N_V is available as well. The densities-of-states are corrected so that equivalent carrier densities are obtained when Gaussian densities-of-states described by [Eq. 170, p. 264](#) are used instead of standard densities-of-states. Introducing the notations:

$$\begin{aligned} A_e &= \frac{\sigma_n^2}{2k^2} \\ B_e &= \frac{E_{0n} - E_C}{k} \\ C_e &= \frac{E_{0n} - E_C}{\sqrt{2}\sigma_n} \\ A_h &= \frac{\sigma_p^2}{2k^2} \\ B_h &= \frac{E_V - E_{0p}}{k} \\ C_h &= \frac{E_V - E_{0p}}{\sqrt{2}\sigma_p} \end{aligned} \tag{176}$$

the electron and hole effective DOS are computed in the Maxwell–Boltzmann approximation as:

$$N_C(T) = \frac{N_{\text{LUMO}}}{2} \exp\left(\frac{A_e}{T^2} - \frac{B_e}{T}\right) \operatorname{erfc}\left(\frac{\sqrt{A_e}}{T} - C_e\right) \tag{177}$$

and:

$$N_V(T) = \frac{N_{\text{HOMO}}}{2} \exp\left(\frac{A_h}{T^2} - \frac{B_h}{T}\right) \operatorname{erfc}\left(\frac{\sqrt{A_h}}{T} - C_h\right) \tag{178}$$

The simplified model can be activated regionwise or globally by specifying the keyword `GaussianDOS` for `EffectiveMass` in the `Physics` section of the command file:

```
Physics(Region="Organic_sem1") {  
    EffectiveMass(GaussianDOS)  
}
```


The model parameters used in [Eq. 170, p. 264](#) and [Eq. 176](#) and their default values are summarized in [Table 32](#).

Table 31 Gaussian DOS model parameters

Parameter symbol	Parameter name	Default value	Unit
N_t N_{LUMO} (electron) N_{HOMO} (hole)	Nt	1×10^{21} 1×10^{21}	cm^{-3}
σ_{DOS} σ_n σ_p	sigmaDOS	0.052 0.052	eV
$E_{0n} = E_0 - E_C$ (electron) $E_{0p} = E_V - E_0$ (hole)	E0	0.1 0.1	eV

Table 32 Simplified Gaussian DOS model parameters

Parameter symbol	Parameter name	Default value	Unit
N_t N_{LUMO} (electron) N_{HOMO} (hole)	N_LUMO N_HOMO	1×10^{21} 1×10^{21}	cm^{-3}
σ_{DOS} σ_n σ_p	sigmaDOS_e sigmaDOS_h	0.052 0.052	eV
$E_{0cn} = E_0 - E_C$ $E_{0v} = E_V - E_0$	E0_c E0_v	0.1 0.1	eV

Multivalley Band Structure

[Eq. 43](#) and [Eq. 44, p. 194](#) give a dependency of electron and hole concentrations on the quasi-Fermi level for a single-valley representation of the semiconductor band structure. Stress-induced change of the silicon band structure and the nature of low bandgap materials require you to consider several valleys in the conduction and valence bands to compute the correct dependency of the carrier concentration on the quasi-Fermi level.

With the parabolic band assumption and Fermi-Dirac distribution function applied to each valley, the multivalley electron and hole concentrations are represented as follows:

$$n = N_C F_{\text{MV},n} \left(\frac{E_{\text{F},n} - E_C}{kT} \right) = N_C \sum_{i=1}^{N_n} g_n^i F_{1/2} \left(\frac{E_{\text{F},n} - E_C - \Delta E_n^i}{kT} \right) \quad (179)$$

$$p = N_V F_{MV,p} \left(\frac{E_V - E_{F,p}}{kT} \right) = N_V \sum_{i=1}^{N_p} g_p^i F_{1/2} \left(\frac{\Delta E_p^i + E_V - E_{F,p}}{kT} \right) \quad (180)$$

where N_n and N_p are the electron and hole numbers of valleys, g_n^i and g_p^i are the electron and hole DOS valley factors, and ΔE_n^i and ΔE_p^i are the electron and hole valley energy shifts relative to the band edges E_C and E_V , respectively. All other variables in these equations are the same as in [Eq. 43](#) and [Eq. 44](#), p. 194.

Similarly to the Fermi statistics, the inverse functions of $F_{MV,n}$ and $F_{MV,p}$ are used to define the variables γ_n and γ_p that bring an additional drift term with $\nabla(\ln \gamma)$ as it is expressed in [Eq. 47](#) and [Eq. 48](#), p. 194:

$$\gamma_n = \frac{n}{N_C} \exp \left(-F_{MV,n}^{-1} \left(\frac{n}{N_C} \right) \right) \quad (181)$$

$$\gamma_p = \frac{p}{N_V} \exp \left(-F_{MV,p}^{-1} \left(\frac{p}{N_V} \right) \right) \quad (182)$$

To compute the inverse functions $F_{MV,n}^{-1} \left(\frac{n}{N_C} \right)$ and $F_{MV,p}^{-1} \left(\frac{p}{N_V} \right)$, an internal Newton solver is used.

Together with the fast Joyce–Dixon approximation of the Fermi–Dirac integral $F_{1/2}$, the multivalley model of Sentaurus Device provides an option for a numeric integration over the energy to compute the carrier density using Gauss–Laguerre quadratures. This extends applicability of the model to account for band nonparabolicity (see [Nonparabolic Band Structure](#)) and the MOSFET channel quantization effect using the multivalley MLDA model (see [Modified Local-Density Approximation on page 293](#)). To control the accuracy and CPU time of such a numeric integration, it is possible to set a user-defined number of integration points in the Gauss–Laguerre quadratures.

Nonparabolic Band Structure

The Fermi–Dirac integral $F_{1/2}$ assumes a typical parabolic band approximation where an energy dependency of the DOS is approximated with $\sqrt{\epsilon}$. Usually, such a simple approximation is valid only in the vicinity of the band minima. Generally, the isotropic nonparabolicity parameter α is introduced into the dispersion relation as follows: $\epsilon(1 + \alpha\epsilon) = (\hbar\mathbf{k})^2/(2m)$, where $\mathbf{k}$ is the wavevector, and m is the effective mass.

With such a dispersion, the multivalley carrier density is computed similarly to Eq. 179 and Eq. 180, but $F_{1/2}$ is replaced by the following integral:

$$F_{1/2}^\alpha(\eta, T) = \int_0^\infty \frac{(1 + 2kT\alpha\epsilon)\sqrt{\epsilon(1 + kT\alpha\epsilon)}}{1 + e^{\epsilon - \eta}} d\epsilon \quad (183)$$

where η is the carrier quasi Fermi energy defined by Eq. 49 and Eq. 50, p. 195. Compared to $F_{1/2}$, the above integral explicitly depends on the carrier temperature T , which is accounted for in hydrodynamic and thermodynamic problems.

The more general case of nonparabolic bands can be accounted for with the **six-band** $k \cdot p$ band-structure model for holes (see **Strained Hole Effective Mass and DOS on page 697**). The model considers three hole bands (the heavy-hole, light-hole, and split-off bands) and the DOS of band n is given generally by:

$$D_n(\epsilon) = \frac{2}{(2\pi)^3} \oint_{\epsilon_n(\mathbf{k}) = \epsilon} \frac{dS}{|\nabla_{\mathbf{k}} \epsilon_n(\mathbf{k})|} \quad (184)$$

Therefore, accounting for Fermi–Dirac carrier distribution over the energy, the total hole concentration in the valence band can be expressed as follows:

$$p(E_{F,p}, T) = \sum_i \int_0^\infty \frac{D_i(\epsilon)}{1 + \exp\left(\frac{\epsilon + \Delta E_p^i + E_V - E_{F,p}}{kT}\right)} d\epsilon \quad (185)$$

where variable notations correspond to ones in Eq. 180 and integrals over the energy in Eq. 185 are computed numerically using Gauss–Laguerre quadrature.

Using Multivalley Band Structure

Multivalley statistics is activated with the keyword `MultiValley` in the `Physics` section. If the model must be activated only for electrons or holes, the keywords `eMultiValley` and `hMultiValley` can be used, respectively. **The model can be defined regionwise as well.**

For testing and comparison purposes, the numeric Gauss–Laguerre integration (mentioned in **Nonparabolic Band Structure on page 268**) can be activated only for the Fermi–Dirac integral $F_{1/2}$. To set such an integration, the keyword `DensityIntegral` must be used as in the following statement:

```
Physics { (e|h)MultiValley(DensityIntegral) }
```

12: Semiconductor Band Structure

Multivalley Band Structure

Similarly, to activate the carrier density computation that accounts for nonparabolicity as in [Eq. 183](#), the following statement must be used:

```
Physics { (e|h)Multivalley(Nonparabolicity) }
```

To activate the six-band $k \cdot p$ band-structure model for holes described in [Strained Hole Effective Mass and DOS on page 697](#) and [Nonparabolic Band Structure on page 268](#), the following should be specified:

```
Physics { hMultivalley(kpDOS) }
```

To control the number of Gauss–Laguerre integration points (the default is 30, which gives a good compromise between accuracy of the numeric integration and CPU time), the global Math statement must be used:

```
Math { DensityIntegral(30) }
```

NOTE Together with the keyword `Fermi` (see [Fermi Statistics on page 194](#)), [Eq. 179](#) and [Eq. 180](#) are used. Without Fermi statistics, the Fermi–Dirac integrals $F_{1/2}$ in these equations are replaced by exponents that correspond to the Boltzmann statistics for each valley.

NOTE For the numeric Gauss–Laguerre integration option used to compute the carrier density (see [Nonparabolic Band Structure on page 268](#)), the Fermi statistics is on always, that is, this numeric integration option, similar to the keyword `Fermi`, brings Fermi statistics into the device simulation.

NOTE For stress simulations (with the keyword `Piezo` or `hMultivalley(kpDOS)` in the `Physics` section), all multivalley model parameters (N_n , N_p , g_n^i , g_p^i , ΔE_n^i , and ΔE_p^i) are defined by stress models (see [Multivalley Band Structure on page 699](#)), and user control of the parameters is possible only through these stress models.

For nonstress simulations, the multivalley model parameters are defined in the `MultiValley` section of the parameter file. You can define an arbitrary number of valleys; however, by default (for silicon), Sentaurus Device defines three electron valleys and two hole valleys as follows:

```
MultiValley{
  eValley(0.914, 0.196, 0.196, 0, 2, 0.5) #[1,1,1,eV,1,eV^-1]
  eValley(0.196, 0.914, 0.196, 0, 2, 0.5) #[1,1,1,eV,1,eV^-1]
  eValley(0.196, 0.196, 0.914, 0, 2, 0.5) #[1,1,1,eV,1,eV^-1]
  hValley(0.16, 0.16, 0.16, 0, 1, 0)      #[1,1,1,eV,1,eV^-1]
  hValley(0.49, 0.49, 0.49, 0, 1, 0)      #[1,1,1,eV,1,eV^-1]
}
```

A general format for each valley is $(e/h)\text{Valley}(m_{100}, m_{010}, m_{001}, \Delta E, d, \alpha)$, where:

- m_{100} , m_{010} , and m_{001} are the effective DOS masses along the crystal axis.
- ΔE is the valley energy shift with respect to the band edge, which defines ΔE_n^i and ΔE_p^i in Eq. 179 and Eq. 180.
- d is the degeneracy.
- α is the nonparabolicity parameter that is used only with the numeric Gauss–Laguerre integration option considered in [Nonparabolic Band Structure on page 268](#).

The model parameters g_n^i and g_p^i are defined by the effective DOS masses as follows:

$$g_{n,p}^i = \frac{d^i (m_{100}^i m_{010}^i m_{001}^i)^{0.5}}{\sum_{k=1}^N d^k (m_{100}^k m_{010}^k m_{001}^k)^{0.5} \exp\left(\mp \frac{E_{\text{bandedge}} - \Delta E_{n,p}^k}{kT}\right)} \quad (186)$$

where E_{bandedge} is the conduction or valence band-edge energy that accounts for the valley energy shifts ΔE_n^k and ΔE_p^k (minimum for the conduction band and maximum for the valence band); a negative sign in the exponent corresponds to electrons, and a positive sign in the exponent corresponds to holes.

If all valley energy shifts equal zero, [Eq. 186](#) provides the single-valley condition where, as usual, only the effective DOS and band edge define the semiconductor band structure and carrier concentration.

References

- [1] W. Bludau, A. Onton, and W. Heinke, “Temperature dependence of the band gap in silicon,” *Journal of Applied Physics*, vol. 45, no. 4, pp. 1846–1848, 1974.
- [2] H. S. Bennett and C. L. Wilson, “Statistical comparisons of data on band-gap narrowing in heavily doped silicon: Electrical and optical measurements,” *Journal of Applied Physics*, vol. 55, no. 10, pp. 3582–3587, 1984.
- [3] J. W. Slotboom and H. C. de Graaff, “Measurements of Bandgap Narrowing in Si Bipolar Transistors,” *Solid-State Electronics*, vol. 19, no. 10, pp. 857–862, 1976.
- [4] J. W. Slotboom and H. C. de Graaff, “Bandgap Narrowing in Silicon Bipolar Transistors,” *IEEE Transactions on Electron Devices*, vol. ED-24, no. 8, pp. 1123–1125, 1977.
- [5] J. W. Slotboom, “The pn-Product in Silicon,” *Solid-State Electronics*, vol. 20, no. 4, pp. 279–283, 1977.

12: Semiconductor Band Structure

References

- [6] D. B. M. Klaassen, J. W. Slotboom, and H. C. de Graaff, "Unified Apparent Bandgap Narrowing in n- and p-Type Silicon," *Solid-State Electronics*, vol. 35, no. 2, pp. 125–129, 1992.
- [7] J. del Alamo, S. Swirhun, and R. M. Swanson, "Simultaneous Measurement of Hole Lifetime, Hole Mobility and Bandgap Narrowing in Heavily Doped n-Type Silicon," in *IEDM Technical Digest*, Washington, DC, USA, pp. 290–293, December 1985.
- [8] J. del Alamo, S. Swirhun, and R. M. Swanson, "Measuring and Modeling Minority Carrier Transport in Heavily Doped Silicon," *Solid-State Electronics*, vol. 28, no. 1–2, pp. 47–54, 1985.
- [9] S. E. Swirhun, Y.-H. Kwark, and R. M. Swanson, "Measurement of Electron Lifetime, Electron Mobility and Band-Gap Narrowing in Heavily Doped p-Type Silicon," in *IEDM Technical Digest*, Los Angeles, CA, USA, pp. 24–27, December 1986.
- [10] S. E. Swirhun, J. A. del Alamo, and R. M. Swanson, "Measurement of Hole Mobility in Heavily Doped n-Type Silicon," *IEEE Electron Device Letters*, vol. EDL-7, no. 3, pp. 168–171, 1986.
- [11] J. A. del Alamo and R. M. Swanson, "Measurement of Steady-State Minority-Carrier Transport Parameters in Heavily Doped n-Type Silicon," *IEEE Transactions on Electron Devices*, vol. ED-34, no. 7, pp. 1580–1589, 1987.
- [12] S. C. Jain and D. J. Roulston, "A Simple Expression for Band Gap Narrowing (BGN) in Heavily Doped Si, Ge, GaAs and $\text{Ge}_x\text{Si}_{1-x}$ Strained Layers," *Solid-State Electronics*, vol. 34, no. 5, pp. 453–465, 1991.
- [13] A. Schenk, "Finite-temperature full random-phase approximation model of band gap narrowing for silicon device simulation," *Journal of Applied Physics*, vol. 84, no. 7, pp. 3684–3695, 1998.
- [14] M. A. Green, "Intrinsic concentration, effective densities of states, and effective mass in silicon," *Journal of Applied Physics*, vol. 67, no. 6, pp. 2944–2954, 1990.
- [15] J. E. Lang, F. L. Madarasz, and P. M. Hemenger, "Temperature dependent density of states effective mass in nonparabolic p-type silicon," *Journal of Applied Physics*, vol. 54, no. 6, p. 3612, 1983.
- [16] G. Paasch and S. Scheinert, "Charge carrier density of organics with Gaussian density of states: Analytical approximation for the Gauss–Fermi integral," *Journal of Applied Physics*, vol. 107, no. 10, p. 104501, 2010.

CHAPTER 13 Incomplete Ionization

This chapter discusses how incomplete ionization is accounted for in Sentaurus Device.

Overview

In silicon, with the exception of indium, dopants can be considered to be fully ionized at room temperature because the impurity levels are sufficiently shallow. However, when impurity levels are relatively deep compared to the thermal energy kT , incomplete ionization must be considered. This is the case for indium acceptors in silicon and nitrogen donors and aluminum acceptors in silicon carbide. In addition, for simulations at reduced temperatures, incomplete ionization must be considered for all dopants. For these situations, Sentaurus Device has an ionization probability model based on activation energy. The ionization (activation) is computed separately for each species present.

Sentaurus Device supports the most significant dopants used in silicon technologies: the donors As, P, Sb, and N, and the acceptors B and In. In SiC, N is supported as donor and Al as acceptor. For the simulation of other semiconductors such as III–V compounds, the actual donors and acceptors used (Si, Be, and so on) are not supported. However, it is reasonable to replace the real dopant species with an appropriate silicon donor or acceptor and adjust the physical parameters accordingly.

For example, to simulate GaAs, which is doped n-type using Si, P can be substituted as the donor. The donor level and cross-sections of P should then be changed in the parameter file to represent the values for silicon in GaAs. Alternatively, Sentaurus Device supports the generic dopants ‘NDopant’ and ‘PDopant.’

The doping profiles for these generic dopants are read from the doping file under the names NDopantConcentration and PDopantConcentration.

Using Incomplete Ionization

The incomplete ionization model is activated with the keyword `IncompleteIonization` in the `Physics` section:

```
Physics{ IncompleteIonization }
```

The incomplete ionization model for selected species is activated with the additional keyword `Dopants`:

```
Physics{ IncompleteIonization(Dopants = "Species_name1 Species_name2 ...") }
```

For example, the following command line activates the model only for boron:

```
Physics{ IncompleteIonization(Dopants = "BoronActiveConcentration") }
```

The incomplete ionization model can be specified in region or material physics (see [Region-specific and Material-specific Models on page 57](#)). In this case, the model is activated only in these regions or materials.

Incomplete Ionization Model

The concentration of ionized impurity atoms is given by Fermi–Dirac distribution:

$$N_D = \frac{N_{D,0}}{1 + g_D \exp\left(\frac{E_{F,n} - E_D}{kT}\right)} \text{ for } N_{D,0} < N_{D,\text{crit}} \quad (187)$$

$$N_A = \frac{N_{A,0}}{1 + g_A \exp\left(\frac{E_A - E_{F,p}}{kT}\right)} \text{ for } N_{A,0} < N_{A,\text{crit}} \quad (188)$$

where $N_{D,0}$ and $N_{A,0}$ are the substitutional (active) donor and acceptor concentrations, g_D and g_A are the degeneracy factors for the impurity levels, and E_D and E_A are the donor and acceptor ionization (activation) energies.

In the literature [1], incomplete ionization in SiC material has been considered and another general distribution function has been proposed, which can be expressed as:

$$N_D = \frac{N_{D,0}}{1 + G_D(T) \exp\left(\frac{E_{F,n} - E_C}{kT}\right)} \quad (189)$$

$$N_A = \frac{N_{A,0}}{1 + G_A(T) \exp\left(-\frac{E_{F,p} - E_V}{kT}\right)} \quad (190)$$

where $G_D(T)$ and $G_A(T)$ are the ionization factors discussed in [2][3]. These factors can be defined by a PMI (see [1] and [Example: Matsuura Incomplete Ionization Model on page 1120](#)).

By comparing [Eq. 187](#) and [Eq. 188](#) to [Eq. 189](#) and [Eq. 190](#), it can be seen that, in the case of the Fermi distribution function, the ionization factors can be written as:

$$G_D(T) = g_D \cdot \exp\left(\frac{\Delta E_D}{kT}\right), \Delta E_D = E_C - E_D \text{ and } G_A(T) = g_A \cdot \exp\left(\frac{\Delta E_A}{kT}\right), \Delta E_A = E_A - E_V \quad (191)$$

In Sentaurus Device, the basic variables are potential, electron concentration, and hole concentration. Therefore, it is more convenient to rewrite [Eq. 187](#) and [Eq. 188](#) in terms of the carrier concentration instead of the quasi-Fermi levels:

$$N_D = \frac{N_{D,0}}{1 + g_D \frac{n}{n_1}}, \text{ with } n_1 = N_C \exp\left(-\frac{\Delta E_D}{kT}\right) \text{ for } N_{D,0} < N_{D,crit} \quad (192)$$

$$N_A = \frac{N_{A,0}}{1 + g_A \frac{p}{p_1}}, \text{ with } p_1 = N_V \exp\left(-\frac{\Delta E_A}{kT}\right) \text{ for } N_A < N_{A,crit} \quad (193)$$

The expressions for n_1 and p_1 in these two equations are valid for Boltzmann statistics and without quantization. If Fermi–Dirac statistics or a quantization model (see [Chapter 14 on page 279](#)) is used, n_1 and p_1 are multiplied by the coefficients γ_n and γ_p defined in [Eq. 47](#) and [Eq. 48, p. 194](#) (see [Fermi Statistics on page 194](#)). For $N_{D/A} > N_{D/A,crit}$, the dopants are assumed to be completely ionized, in which case, every donor and acceptor species is considered in the Poisson equation. The values of $N_{D,crit}$ and $N_{A,crit}$ can be adjusted in the parameter file of Sentaurus Device.

The donor and acceptor activation energies are effectively reduced by the total doping in the semiconductor. This effect is accounted for in the expressions:

$$\Delta E_D = \Delta E_{D,0} - \alpha_D \cdot N_{tot}^{1/3} \quad (194)$$

$$\Delta E_A = \Delta E_{A,0} - \alpha_A \cdot N_{tot}^{1/3} \quad (195)$$

where $N_{tot} = N_{A,0} + N_{D,0}$ is the total doping concentration.

In transient simulations, the terms:

$$\frac{\partial N_D}{\partial t} = \sigma_D v_{th}^n \left[\frac{n_1}{g_D} N_{D,0} - \left(n + \frac{n_1}{g_D} \right) N_D \right] \quad (196)$$

$$\frac{\partial N_A}{\partial t} = \sigma_A v_{th}^p \left[\frac{p_1}{g_A} N_{A,0} - \left(p + \frac{p_1}{g_A} \right) N_A \right] \quad (197)$$

are included in the continuity equations ($v_{th}^{n,p}$ denote the carrier thermal velocities).

Physical Model Parameters

The values of the dopant level $E_{A/D,0}$, the doping-dependent shift parameter $\alpha_{A/D}$, the impurity degeneracy factor $g_{A/D}$, and the cross section $\sigma_{A/D}$ are accessible in the parameter set `Ionization`.

For each user-defined dopant (see [User-Defined Species on page 50](#)), the `Ionization` parameter set must contain a separate subsection where the parameters $E_{A/D,0}$, $\alpha_{A/D}$, $g_{A/D}$, and $\sigma_{A/D}$ are defined. For example, for the dopant Nitrogen described in [User-Defined Species on page 50](#) for the material SiC, the parameter set is:

```
Material = "SiC" {
  ...
  Ionization {
    ...
    Species("Nitrogen") {
      type = donor
      E_0  = 0.1
      alpha = 0
      g    = 2
      Xsec = 1.0e-15
    }
  }
}
```

The field `type` can be omitted because the dopant type is specified in the `datexcodes.txt` file. It is used here for informative purposes only.

Table 33 Default coefficients for incomplete ionization model for dopants in silicon

Symbol	Parameter name	Default value for species *								Unit
		As	P	Sb	B	In	N	NDopant	PDopant	
$E_{A/D,0}$	$E_{_0}$	0.054	0.045	0.039	0.045	0.16	0.045	0.045	0.045	eV
$\alpha_{A/D}$	$\alpha_{_}$	3.1×10^{-8}								eVcm
$g_{A/D}$	$g_{_}$	2	2	2	4	4	2	2	4	1
$\sigma_{A/D}$	$Xsec_{_}$	1.0×10^{-12}								$cm^2 s^{-1}$
$N_{D,crit}$	NdCrit	1.0×10^{22}								cm^{-3}
$N_{A,crit}$	NaCrit	1.0×10^{22}								cm^{-3}

References

- [1] H. Matsuura, "Influence of Excited States of Deep Acceptors on Hole Concentration in SiC," in *International Conference on Silicon Carbide and Related Materials (ICSCRM)*, Tsukuba, Japan, pp. 679–682, October 2001.
- [2] P. Y. Yu and M. Cardona, *Fundamentals of Semiconductors: Physics and Materials Properties*, Berlin: Springer, 2nd ed., 1999.
- [3] K. F. Brennan, *The Physics of Semiconductors: With applications to optoelectronic devices*, Cambridge: Cambridge University Press, 1999.

13: Incomplete Ionization

References

CHAPTER 14 Quantization Models

This chapter describes the features that Sentaurus Device offers to model quantization effects.

Some features of current MOSFETs (oxide thickness, channel width) have reached quantum-mechanical length scales. Therefore, the wave nature of electrons and holes can no longer be neglected. The most basic quantization effects in MOSFETs are the shift of the threshold voltage and the reduction of the gate capacity. Tunneling, another important quantum effect, is described in [Chapter 24 on page 605](#).

Overview

To include quantization effects in a classical device simulation, Sentaurus Device introduces a potential-like quantity Λ_n in the classical density formula:

$$n = N_C F_{1/2} \left(\frac{E_{F,n} - E_C - \Lambda_n}{kT_n} \right) \quad (198)$$

An analogous quantity Λ_p is introduced for holes.

The most important effects related to the density modification (due to quantization) can be captured by proper models for Λ_n and Λ_p . Other effects (for example, single electron effects) exceed the scope of this approach.

Sentaurus Device implements four quantization models, that is, four different models for Λ_n and Λ_p . They differ in physical sophistication, numeric expense, and robustness:

- The van Dort model (see [van Dort Quantization Model on page 280](#)) is a numerically robust, fast, and proven model. It is only suited to bulk MOSFET simulations. While important terminal characteristics are well described by this model, it does not give the correct density distribution in the channel.
- The 1D Schrödinger equation (see [1D Schrödinger Solver on page 281](#)) is the most physically sophisticated quantization model. It can be used for MOSFET simulation, and quantum well and ultrathin SOI simulation. Simulations with this model tend to be slow and often lead to convergence problems, which restrict its use to situations with small current flow. Therefore, the Schrödinger equation is used mainly for the validation and calibration of other quantization models.

- The density gradient model (see [Density Gradient Quantization Model on page 288](#)) is numerically robust, but significantly slower than the van Dort model. It can be applied to MOSFETs, quantum wells and SOI structures, and gives a reasonable description of terminal characteristics and charge distribution inside a device. Compared to the other quantization models, it can describe 2D and 3D quantization effects.
- The modified local-density approximation (MLDA) model (see [Modified Local-Density Approximation on page 293](#)) is a numerically robust and fast model. It can be used for bulk MOSFET simulations and thin SOI simulations. Although it sometimes fails to calculate the accurate carrier distribution in the saturation regions because of its one-dimensional characteristic, it is suitable for three-dimensional device simulations because of its numeric efficiency.

van Dort Quantization Model

van Dort Model

The van Dort model [1] computes Λ_n of Eq. 198 as a function of $|\hat{n} \cdot \vec{F}|$, the electric field normal to the semiconductor–insulator interface:

$$\Lambda_n = \frac{13}{9} \cdot k_{\text{fit}} \cdot G(\vec{r}) \cdot \left(\frac{\epsilon \epsilon_0}{4kT} \right)^{1/3} \cdot \left| |\hat{n} \cdot \vec{F}| - E_{\text{crit}} \right|^{2/3} \quad (199)$$

and likewise for Λ_p . k_{fit} and E_{crit} are fitting parameters.

The function $G(\vec{r})$ is defined by:

$$G(\vec{r}) = \frac{2 \cdot \exp(-a^2(\vec{r}))}{1 + \exp(-2a^2(\vec{r}))} \quad (200)$$

where $a(\vec{r}) = l(\vec{r})/\lambda_{\text{ref}}$ and $l(\vec{r})$ is a distance from the point $\vec{r}$ to the interface. The parameter λ_{ref} determines the distance to the interface up to which the quantum correction is relevant. The quantum correction is applied to those carriers (electrons or holes) that are drawn towards the interface by the electric field; for the carriers driven away from the interface, the correction is 0.

Using the van Dort Model

To activate the model for electrons or holes, specify the `eQCvanDort` or `hQCvanDort` flag in the **Physics** section:

```
Physics { ... eQCvanDort}
```

[Table 34](#) lists the parameters in the `vanDortQMModel` parameter set. The default value of λ_{ref} is from the literature [\[1\]](#). Other parameters were obtained by fitting the model to experimental data for electrons [\[1\]](#) and holes [\[2\]](#).

Table 34 Default parameters for van Dort quantum correction model

Symbol	Electrons		Holes		Unit
k_{fit}	eFit	2.4×10^{-8}	hFit	1.8×10^{-8}	eVcm
E_{crit}	eEcritQC	10^5	eEcritQC	10^5	V/cm
λ_{ref}	dRef	2.5×10^{-6}	dRef	2.5×10^{-6}	cm

By default, in [Eq. 199](#), $\hat{n}$ is the normal to the closest semiconductor–insulator interface. The interface can be specified explicitly by `EnormalInterface` in the **Math** section (see [Normal to Interface on page 336](#)).

1D Schrödinger Solver

The 1D Schrödinger solver implements the most physically sophisticated quantization model in Sentaurus Device. To use the Schrödinger solver:

1. Construct a special purpose ‘nonlocal’ mesh (see [Nonlocal Mesh for 1D Schrödinger on page 282](#)).
2. Activate the Schrödinger solver on the nonlocal line mesh with appropriate parameters (see [Using 1D Schrödinger on page 283](#)).

Sometimes, especially for heteromaterials, the physical model parameters need to be adapted (see [1D Schrödinger Parameters on page 283](#)).

NOTE The 1D Schrödinger solver is time consuming and often causes converge problems. Furthermore, small-signal analysis (see [Small-Signal AC Analysis on page 127](#)) and noise and fluctuation analysis (see [Chapter 23 on page 581](#)) are not possible when using this solver.

Nonlocal Mesh for 1D Schrödinger

The specification of the nonlocal mesh determines where in the device the 1D Schrödinger equation can be solved. This section summarizes the features that are typically needed to obtain a correct nonlocal mesh for the 1D Schrödinger equation. For more information about constructing nonlocal meshes, see [Nonlocal Meshes on page 166](#).

To generate a nonlocal mesh, specify `NonLocal` in the global `Math` section. Options to `NonLocal` control the construction of the nonlocal mesh. For example:

```
Math {  
  NonLocal "NLM" (  
    RegionInterface="gateoxide/channel"  
    Length=10e-7  
    Permeation=1e-7  
    Direction=(0 1 0) MaxAngle=5  
    -Transparent(Region="gateoxide")  
  )  
}
```

generates a nonlocal mesh named "NLM" at the interface between region `gateoxide` and `channel`. The nonlocal lines extend 10 nm (according to `Length`) to one side of the interface and 1 nm (according to `Permeation`) to the other side. The segments of the nonlocal lines on the two sides are handled differently; see below. In the example, `Direction` and `MaxAngle` restrict the nonlocal mesh to lines that run along the y-axis, with a tolerance of 5°. `Direction` defines a vector that is typically perpendicular to the interface; therefore, the specification above is appropriate for an interface in the xz plane.

For nonlocal meshes constructed for the Schrödinger equation, the `-Transparent` switch is important. In the above example, it suppresses the construction of nonlocal lines for which the section with length `Length` passes through `gateoxide`. Therefore, Sentaurus Device only constructs nonlocal lines that extend 10 nm into `channel` and 1 nm into `gateoxide`, and suppresses the nonlocal lines that extend 1 nm into `channel` and 10 nm into `gateoxide`. Without the `-Transparent` switch, Sentaurus Device would construct both line types and, therefore, would solve the Schrödinger equation not only in the channel, but also in the poly gate (provided that a poly gate exists and `gateoxide` is thinner than 10 nm).

Using 1D Schrödinger

To activate the 1D Schrödinger equation for electrons or holes, specify the `Electron` or `Hole` switch as an option to `Schroedinger` in the global `Physics` section. For example:

```
Physics {  
    Schroedinger "NLM" (Electron)  
}
```

activates the Schrödinger equation for electrons on the nonlocal mesh named "NLM".

Additional options to `Schroedinger` determine numeric accuracy, details of the density computation, and which eigenstates will be computed. [Table 257 on page 1314](#) lists the optional keywords.

`MaxSolutions`, `Error`, and `EnergyInterval` support the option `electron` or `hole`. For example, the specification `MaxSolutions=30` sets the value for both electrons and holes. The specifications `MaxSolutions(electron)=2` and `MaxSolutions(hole)=3` set different values for electrons and holes.

For backward compatibility, you can define `Schroedinger` in an interface-specific or a contact-specific `Physics` section; in this case, omit the nonlocal mesh name. The specification applies to an unnamed nonlocal mesh that has been constructed for the same location (see [Unnamed Meshes on page 172](#)).

1D Schrödinger Parameters

Typically, the Schrödinger equation must be solved repeatedly, with different masses and potential profiles, resulting in a number of distinct ‘ladders’ of eigenenergies. For example, the six-fold degenerate, anisotropic conduction band valleys in silicon require the Schrödinger equation to be solved up to three times with different parameters. Sentaurus Device allows you to specify the number of ladders and the associated parameters explicitly, or it can extract the number of ladders and the parameters from other parameters and the nonlocal line direction automatically.

Parameters for the Schrödinger equation are region specific. The Schrödinger equation must not be solved across regions with fundamentally different band structures. Sentaurus Device displays an error message if band structure compatibility is violated.

Explicit Ladder Specification

Any number of $\text{eLadder}(m_{z,v}, m_{xy,v}, d_v, \Delta E_v)$ (for electrons) or $\text{hLadder}(m_{z,v}, m_{xy,v}, d_v, \Delta E_v)$ (for holes) specifications is possible in the `SchroedingerParameters` parameter set. The v -th specification for a particular carrier type defines the quantization mass $m_{z,v}$, the mass perpendicular to the quantization direction $m_{xy,v}$, the ladder degeneracy d_v , and a nonnegative band edge shift ΔE_v .

The parameter pair `ShiftTemperature` in the `SchroedingerParameters` parameter set is given in kelvin and determines the temperatures T_{ref} for electrons and holes that enter scaling factors applied to $m_{xy,v}$. The scaling factors ensure that the masses for the Schrödinger equation are consistent with the density-of-states masses. For electrons, the scaling factor is:

$$\frac{m_n^{3/2}}{\sum_v d_v m_{xy,v} m_{z,v}^{1/2} \exp(-\Delta E_v / kT_{\text{ref}})} \quad (201)$$

and likewise for holes. T_{ref} defaults to a large value. When $T_{\text{ref}} = 0$, scaling is disabled.

Automatic Extraction of Ladder Parameters

When no explicit ladder specification is present, Sentaurus Device attempts to extract the required parameters automatically. The `formula` parameter pair in the `SchroedingerParameters` parameter set specifies which expressions for the masses to use.

For each carrier, if the formula is 0, Sentaurus Device uses the isotropic density-of-states mass as both the quantization mass and the mass perpendicular to it.

For electrons, if the formula is 1, Sentaurus Device uses the anisotropic (silicon-like) masses specified in the `eDOSMass` parameter set. The quantization mass for the conduction band valley v reads:

$$\frac{1}{m_{z,v}} = \sum_{i=1}^3 \frac{z_i^2}{m_{i,v}} \quad (202)$$

where $m_{i,v}$ are the effective mass components for valley v , and z_i are the coefficients of the unit normal vector pointing in the quantization direction, expressed in the crystal coordinate system. The `LatticeParameters` parameter set determines the relation to the mesh coordinate system (see [Crystal Reference System on page 669](#)).

For holes, ‘warped’ band structure (formula 1), or heavy-hole and light-hole masses (formula 2) are available (see Table 35). For the former, the quantization mass is given by:

$$m_{z,v} = \frac{m_0}{A \pm \sqrt{B + C \cdot (z_1^2 z_2^2 + z_2^2 z_3^2 + z_3^2 z_1^2)}} \quad (203)$$

For the latter, m_l and m_h (given as multiples of m_0) determine directly the masses for the light-hole and heavy-hole bands.

The masses $m_{xy,v}$ are chosen to achieve the same density-of-states mass as used in classical simulations. For electrons:

$$m_{xy,v} = \frac{m_n^{3/2}}{m_{z,v}^{1/2} n_v} \quad (204)$$

where n_v is the number of band valleys (or bands). The expression for holes is analogous.

SchroedingerParameters offers a parameter pair `offset` to model the lifting of the degeneracy of band minima in strained materials. Unless exactly two ladders exist, `offset` must be zero. Positive offsets apply to the electron ladder with lower degeneracy or the heavy-hole band; negative values apply to the electron ladder of higher degeneracy or the light-hole band. The modulus of the value is added to the band edge for the respective ladder, moving it away from mid-gap, while the other ladder remains unaffected. For holes, the shift is taken into account in the scaling of $m_{xy,v}$ as it is in Eq. 201.

Table 35 Parameters for hole effective masses in Schrödinger solver

Formula	Symbol	Parameter name	Default value	Unit
1	A	A	4.22	1
	B	B	0.6084	1
	C	C	23.058	1
2	m_l	m_l	0	1
	m_h	m_h	0	1

Visualizing Schrödinger Solutions

To visualize the results obtained by the Schrödinger equation, Sentaurus Device offers special keywords for the NonLocalPlot section (see [Visualizing Data Defined on Nonlocal Meshes on page 167](#)).

To plot the wavefunctions, specify WaveFunction in the NonLocalPlot section. Sentaurus Device plots wavefunctions in units of $\mu\text{m}^{-1/2}$. To plot the eigenenergies, specify EigenEnergy; Sentaurus Device plots them in units of eV. The names of the curves in the output have two numeric indices. The first index denotes the ladder index and the second index denotes the number of zeros of the wavefunction (see v and j in [Eq. 205](#)).

Without further specification, Sentaurus Device will plot all eigenenergies and wavefunctions it has computed. The Electron and Hole options to WaveFunction and EigenEnergy restrict the output to the wavefunctions and eigenenergies for electrons and holes, respectively.

The Electron and Hole options have a sub-option Number=<int> to restrict the output to a certain number of states of lowest energy. For example:

```
NonLocal(
  (0 0)
){
  WaveFunction(Electron(Number=3) Hole)
}
```

will plot all hole wavefunctions and the three lowest-energy electron wavefunctions for the nonlocal mesh line close to the coordinate (0, 0, 0).

1D Schrödinger Model

Omitting x- and y-coordinates for simplicity, the 1D Schrödinger equation for electrons is:

$$\left(-\frac{\partial}{\partial z} \frac{\hbar^2}{2m_{z,v}(z)} \frac{\partial}{\partial z} + E_C(z) + \Delta E_v\right) \Psi_{j,v}(z) = E_{j,v} \Psi_{j,v}(z) \quad (205)$$

where z is the quantization direction (typically, the direction perpendicular to the silicon–oxide interface in a MOSFET), v labels the ladder, ΔE_v is the (region-dependent) energy offset for the ladder v , $m_{z,v}$ is the effective mass component in the quantization direction, $\Psi_{j,v}$ is the j -th normalized eigenfunction, and $E_{j,v}$ is the j -th eigenenergy.

From the solution of this equation, the density is computed as:

$$n(z) = \frac{kT(z)}{\pi\hbar^2} \sum_{j,v} |\Psi_{j,v}(z)|^2 d_v m_{xy,v}(z) \left[F_0\left(\frac{E_{F,n}(z) - E_{j,v}}{kT(z)}\right) - F_0\left(\frac{E_{F,n}(z) - E_{\max,v}}{kT(z)}\right) \right] + \sum_v n_{cl,v} \quad (206)$$

where $m_{xy,v}$ is the mass component perpendicular to the quantization direction and d_v is the degeneracy for ladder v . For the parameters $m_{z,v}$, $m_{xy,v}$, d_v , and ΔE_v , see [1D Schrödinger Parameters on page 283](#).

If `DensityTail=MaxEnergy`, $E_{\max,v}$ in Eq. 206 is the energy of the highest computed subband for ladder v ; otherwise, it is an estimate of the energy of the lowest not computed subband. $n_{cl,v}$ is the contribution to the classical density for ladder v by energies above $E_{\max,v}$.

At the ends of the nonlocal line, Sentaurus Device optionally smoothly blends from the classical density to the density according to Eq. 206. Equating the resulting density to Eq. 198 determines Λ_n .

The Schrödinger equation is solved over a finite domain $[z_-, z_+]$. At the endpoints of this domain, the boundary condition:

$$\frac{\Psi'_{j,v}}{\Psi_{j,v}} = \mp \frac{\sqrt{2m_{z,v}|E_{j,v} - E_C|}}{\hbar} \quad (207)$$

is applied, where for the upper sign, all position-dependent functions are taken at z_+ and, for the lower sign, at z_- .

1D Schrödinger Application Notes

- Convergence problems: Near the flatband condition, minor changes in the band structure can change the number of bound states between zero and one. By default, Sentaurus Device computes only bound states and, therefore, this change switches from a purely classical solution to a solution with quantization. To avoid the large change in density that this transition can cause, set `EnergyInterval` to a nonzero value (for example, 1). Then, a few unbound states are computed and a hard transition is avoided.
- Always include a part of the ‘barrier’ in the nonlocal line on which the Schrödinger equation is solved. In a MOSFET, besides the channel region, solve the Schrödinger equation in a part of the oxide adjacent to the channel (for example, 1 nm deep). Use the `Permeation` option to `NonLocal` to achieve this (see [Nonlocal Mesh for 1D Schrödinger on page 282](#)). Failure to solve in a part of the oxide adjacent to the channel blinds the Schrödinger solver to the barrier. It does not know the electrons are confined and the quantization effects are not obtained.

Density Gradient Quantization Model

Density Gradient Model

For the density gradient model [3][4], Λ_n in Eq. 198 is given by a partial differential equation:

$$\Lambda_n = -\frac{\gamma\hbar^2}{12m_n}\left\{\nabla^2\ln n + \frac{1}{2}(\nabla\ln n)^2\right\} = -\frac{\gamma\hbar^2}{6m_n}\frac{\nabla^2\sqrt{n}}{\sqrt{n}} \quad (208)$$

where γ is a fit factor.

Introducing the reciprocal thermal energy $\beta = 1/kT_n$, the mass-driving term $\Phi_m = -kT_n\ln(N_C/N_{\text{ref}})$ (with an arbitrary normalization constant N_{ref}) and the potential-like quantity $\bar{\Phi} = E_C + \Phi_m + \Lambda_n$, Eq. 208 can be rewritten and generalized as:

$$\Lambda_n = -\frac{\hbar^2\gamma}{12m_n}\{\nabla \cdot \alpha(\xi\nabla\beta E_{F,n} - \nabla\beta\bar{\Phi} + (\eta-1)q\nabla\beta\phi) + \vartheta(\xi\nabla\beta E_{F,n} - \nabla\beta\bar{\Phi} + (\eta-1)q\nabla\beta\phi) \cdot \alpha(\xi\nabla\beta E_{F,n} - \nabla\beta\bar{\Phi} + (\eta-1)q\nabla\beta\phi)\} + \Lambda_{\text{PMI}} \quad (209)$$

with the default parameters $\xi = \eta = 1$, $\vartheta = 1/2$, and α is a symmetric matrix that defaults to one. In insulators, Sentaurus Device does not compute the Fermi energy; therefore, in insulators, $\xi = \eta = 0$. By default, Sentaurus Device uses Eq. 209, which for Fermi statistics deviates from Eq. 208, even when $\xi = \eta = 1$ and $\vartheta = 1/2$. The density-based expression Eq. 208 is available as an option (see [Using the Density Gradient Model on page 289](#)).

In Eq. 209, Λ_{PMI} is a locally dependent quantity computed from a user-specified PMI model (see [Apparent Band-Edge Shift on page 1061](#)), from the Schenk bandgap narrowing model (see [Schenk Bandgap Narrowing Model on page 255](#)), or from apparent band-edge shifts caused by multistate configurations (see [Apparent Band-Edge Shift on page 435](#)). By default, $\Lambda_{\text{PMI}} = 0$.

At Ohmic contacts, interfaces to metals with Ohmic boundary conditions, resistive contacts, and current contacts, the boundary condition $\Lambda_n = \Lambda_{\text{PMI}}$ is imposed. At Schottky contacts, gate contacts, interfaces to metals with Schottky boundary conditions, and external boundaries, homogeneous Neumann boundary conditions are used:
 $\hat{n} \cdot \alpha(\xi\nabla\beta E_{F,n} - \nabla\beta\bar{\Phi} + (\eta-1)q\nabla\beta\phi) = 0$.

At internal interfaces between regions where the density gradient equation is solved, $\bar{\Phi}$ and $\hat{n} \cdot \alpha(\xi\nabla\beta E_{F,n} - \nabla\beta\bar{\Phi} + (\eta-1)q\nabla\beta\phi)$ must be continuous.

At internal interfaces between a region where the equation is solved (indicated below by 0^-) and a non-metal region where it is not solved (0^+), an inhomogeneous Neumann boundary condition obtained from the analytic solution of the 1D step problem is applied:

$$\hat{n} \cdot \nabla \Lambda_n = \sqrt{\frac{24m_n(0^+)k^3T_n^3}{\hbar^2\gamma(0^+)\vartheta(0^+)^2\bar{\alpha}}} [E_C(0^+) - E_C(0^-) - \Lambda_n] f\left(\frac{2\vartheta(0^+)}{kT_n} [\Lambda_n - E_C(0^+) + E_C(0^-)]\right) \quad (210)$$

where $f(x) = \sqrt{[(\exp x - 1)/x - 1]/x}$ and $\bar{\alpha} = \det[\alpha(0^+)]^{1/3}$.

Optionally, Sentaurus Device offers a modified mobility to improve the modeling of tunneling through semiconductor barriers:

$$\mu = \frac{\mu_{cl} + r\mu_{tunnel}}{1 + r} \quad (211)$$

where μ_{cl} is the usual (classical) mobility as described in [Chapter 15 on page 301](#), μ_{tunnel} is a fit parameter, and $r = \max(0, n/n_{cl} - 1)$. Here, n_{cl} is the ‘classical’ density (see [Eq. 345](#)). In this modification, for $n > n_{cl}$, the additional carriers (density $n - n_{cl}$) are considered as tunneling carriers that are subject to a different mobility μ_{tunnel} than the classical carriers (density n_{cl}).

Using the Density Gradient Model

The density gradient equation for electrons and holes is activated by the `eQuantumPotential` and `hQuantumPotential` switches in the `Physics` section. Use `-eQuantumPotential` and `-hQuantumPotential` to switch off the equations. These switches can also be used in regionwise or materialwise `Physics` sections. In metal regions, the equations are never solved. For a summary of available options, see [Table 238 on page 1302](#).

The `Ignore` switch to `eQuantumPotential` and `hQuantumPotential` instructs Sentaurus Device to compute the quantum potential, but not to use it. `Ignore` is intended for backward compatibility to earlier versions of Sentaurus Device.

The `Resolve` switch enables a numeric approach that handles discontinuous band structures, at moderately refined interfaces that are not heterointerfaces, more accurately than the default approach.

The switch `Density` to `eQuantumPotential` and `hQuantumPotential` activates the density-based formula [Eq. 208](#) instead of the potential-based formula [Eq. 209](#). If this option is used, the parameters ξ and η must both be 1.

14: Quantization Models

Density Gradient Quantization Model

The `LocalModel` option of `eQuantumPotential` and `hQuantumPotential` specifies the name of a PMI model (see [Physical Model Interface on page 979](#)) or the Schenk bandgap narrowing model (see [Schenk Bandgap Narrowing Model on page 255](#)) for Λ_{PMI} in Eq. 209. An additional apparent band-edge shift contribution can be added if the corresponding model of a multistate configuration is switched on (see [Apparent Band-Edge Shift on page 435](#)).

The `BoundaryCondition` option of the `eQuantumPotential` and `hQuantumPotential` specifications in metal interface-specific or electrode-specific `Physics` sections allows you to explicitly specify the boundary condition for the quantum potential, overriding the default boundary condition. Possible values are `Dirichlet` and `Neumann`, to enforce homogeneous Dirichlet and Neumann boundary conditions, respectively.

Apart from activating the equations in the `Physics` section, the equations for the quantum corrections must be solved by using `eQuantumPotential` or `hQuantumPotential`, or both in the `Solve` section. For example:

```
Physics {
  eQuantumPotential
}
Plot {
  eQuantumPotential
}
Solve {
  Coupled { Poisson eQuantumPotential }
  Quasistationary (
    DoZero InitialStep=0.01 MaxStep=0.1 MinStep=1e-5
    Goal { Name="gate" Voltage=2 }
  ){
    Coupled { Poisson Electron eQuantumPotential }
  }
}
```

The quantum corrections can be plotted. To this end, use `eQuantumPotential`, or `hQuantumPotential`, or both in the `Plot` section.

To activate the mobility modification according to Eq. 211, specify the `Tunneling` switch to `Mobility` in the `Physics` section of the command file. Specify μ_{tunnel} for electrons and holes by the `mutunnel` parameter pair in the `ConstantMobility` parameter set.

The parameters γ , ϑ , ξ , and η are available in the `QuantumPotentialParameters` parameter set. The parameters can be specified regionwise and materialwise. They cannot be functions of the mole fraction in heterodevices. In insulators, ξ is always assumed as zero, regardless of user specifications.

The diagonal of α in the coordinate system specified by the `LatticeParameters` parameter set (see [Crystal Reference System on page 669](#)) is determined by the parameter pairs

`alpha[1]`, `alpha[2]`, and `alpha[3]` (for the x -, y -, and z -direction, respectively) in the `QuantumPotentialParameters` parameter set. Unless α is a multiple of the unit matrix, Sentaurus Device solves an anisotropic problem. For example:

```
QuantumPotentialParameters {  
    alpha[1] = 1    1  
    alpha[2] = 0.5  0.5  
}
```

decreases the quantization effect along the y -direction to half, for both electrons and holes.

NOTE The anisotropic density gradient equation supports the `AverageAniso` approximation only (see [Chapter 28 on page 665](#)).

Named Parameter Sets for Density Gradient

The `QuantumPotentialParameters` parameter set can be named. For example, in the parameter file, you can write:

```
QuantumPotentialParameters "myset" { ... }
```

to declare a parameter set with the name `myset`. To use a named parameter set, specify its name with `ParameterSetName` as an option to either `eQuantumPotential` or `hQuantumPotential` in the command file, for example:

```
Physics {  
    eQuantumPotential (ParameterSetName = "myset")  
}
```

By default, the unnamed parameter set is used.

Auto-Orientation for Density Gradient

The `eQuantumPotential` and `hQuantumPotential` models support the auto-orientation framework (see [Auto-Orientation Framework on page 75](#)) that switches between different named parameter sets based on the orientation of the nearest interface. This can be activated by specifying `AutoOrientation` as an argument to either `eQuantumPotential` or `hQuantumPotential` in the command file:

```
Physics {  
    eQuantumPotential (AutoOrientation)  
}
```

Density Gradient Application Notes

- **Fitting parameters:** The parameter γ has been calibrated only for silicon. The quantum correction affects the densities and field distribution in a device. Therefore, parameters for mobility and recombination models that have been calibrated to classical simulations (or simulations with the van Dort model) may require recalibration.
- **Tunneling:** The density gradient model increases the current through the semiconducting potential barriers. However, this effect is not a trustworthy description of tunneling through the barrier. To model tunneling, use one of the dedicated models that Sentaurus Device provides (see [Chapter 24 on page 605](#)). To suppress unwanted tunneling or to fit tunneling currents despite these concerns, consider using the modified mobility model according to [Eq. 211](#).
- **Convergence:** In general and particularly for the density gradient corrections, solving additional equations worsens convergence. Typically, it is advisable to solve the equations for the quantum potentials whenever the Poisson equation is solved (using a Coupled statement). Usually, the best strategy to obtain an initial solution at the beginning of the simulation is to do a coupled solve of the Poisson equation and the quantum potentials, without the current and temperature equations.

To obtain this initial solution, it is sometimes necessary that you set the `LineSearchDamping` optional parameter (see [Damped Newton Iterations on page 161](#)) to a value less than 1; a good value is 0.01.

Often, using initial bias conditions that induce a current flow work well for a classical simulation, but do not work when the density gradient model is active. In such cases, start from equilibrium bias conditions and put an additional voltage ramping at the beginning of the simulation.

If an initial solution is still not possible, consider using Fermi statistics (see [Fermi Statistics on page 194](#)). Check grid refinement and pay special attention to interfaces between insulators and highly doped semiconductor regions. For classical simulations, the refinement perpendicular to such interfaces is often not critical, whereas for quantum mechanical simulations, quantization introduces variations at small length scales, which must be resolved.

- **Speed:** Activate the equations only for regions where the quantum corrections are physically needed. For example, usually, the density gradient equations do not need to be computed in insulators. Using the model selectively, typically, also benefits convergence.
- By default, the DOS mass used in the expressions of the density gradient method ignores the effects of strain (see [Strained Effective Masses and Density-of-States on page 695](#)). This is accomplished by using the expression $m_n = m_{n0} + v \cdot \Delta m_n$ with a default value of $v = 0$, where m_{n0} is the unstrained mass and Δm_n is the change in mass due to strain. The parameter v can be specified in the `QuantumPotentialParameters` parameter set.

Modified Local-Density Approximation

MLDA Model

The modified local-density approximation (MLDA) model is a quantum-mechanical model that calculates the confined carrier distributions that occur near semiconductor–insulator interfaces [5]. It can be applied to both inversion and accumulation, and simultaneously to electrons and holes. It is based on a rigorous extension of the local-density approximation and provides a good compromise between accuracy and run-time.

Following Paasch and Übensee [5], the confined electron density at a distance z from a Si–SiO₂ interface is given, under Fermi statistics, by:

$$n_{\text{MLDA}}(\eta_n) = N_C \left(\frac{2}{\sqrt{\pi}} \right) \int_0^{\infty} d\epsilon \frac{\sqrt{\epsilon}}{1 + \exp[(\epsilon - \eta_n)]} [1 - j_0(2z\sqrt{\epsilon}/\lambda_n)] \quad (212)$$

where η_n is given by Eq. 49, p. 194, j_0 is the 0th-order spherical Bessel function, and $\lambda_n = \sqrt{\hbar^2/2m_{qn}kT_n}$ is the electron thermal wavelength which depends on the quantization mass m_{qn} . The integrand of Eq. 212 is very similar to the classical Fermi integrand, with an additional factor describing confinement for small z . A similar equation is applied to holes.

Sentaurus Device provides two MLDA model options:

- One option is the simple original model that uses only one user-defined parameter λ per carrier type.
- The second option (considered in detail in the next section) accounts for the multivalley/band property of the electron and hole band structures.

For the simple model, in Boltzmann statistics, Eq. 212 simplifies to the following expression for the electron density (the hole density is similar):

$$n_{\text{MLDA}}(\eta_n) = N_C \exp(\eta_n) [1 - \exp(-(z/\lambda_n)^2)] \quad (213)$$

Interface Orientation and Stress Dependencies

The MLDA model allows you to consider a dependency of the quantization effect on interface orientation and stress. Mainly, such dependencies come from mass anisotropy and stress related valley/band energy change.

Mass anisotropy has been considered in the literature [6] where ellipsoidal type of bands, which correspond to three electron Δ_2 -valleys in silicon, were used. The main result of the

14: Quantization Models

Modified Local-Density Approximation

article is that Eq. 212 is still valid for each of three electron Δ_2 -valleys with the effective DOS equal to $N_C/3$. Another result is that the quantization mass of each valley m_{qn}^i should be computed as a rotation of the inverse mass tensor from the crystal system (where the tensor is diagonal) to another one where one axis is perpendicular to the interface and the mass component, which corresponds to the axis, should be taken as m_{qn}^i . Considering three electron Δ_2 -valleys, Eq. 212 could be rewritten as follows:

$$n_{\text{MLDA}}(\eta_n, z) = \sum_{i=1}^3 \int_0^{\infty} \frac{D_n^i(\epsilon, z)}{1 + \exp(\epsilon - \eta_n - \Delta\eta^i)} d\epsilon \quad (214)$$

$$D_n^i(\epsilon, z) = N_C g_n^i \left(\frac{2}{\sqrt{\pi}} \right) \sqrt{\epsilon} \left[1 - j_0 \left(2z \sqrt{\frac{2m_{qn}^i k T_n \epsilon}{\hbar^2}} \right) \right]$$

where:

- $D_n^i(\epsilon, z)$ is the electron DOS of valley i with a dependency on the normalized energy ϵ and on the distance to the interface z .
- g_n^i is defined by Eq. 186, p. 271; however, in the case of stress, $N_C g_n^i$ is defined by Eq. 792, p. 696.
- $\Delta\eta^i$ represents the stress-induced valley energy change (see Deformation of Band Structure on page 691).

Similar to Eq. 214, the same multivalley MLDA quantization model could be applied to holes with hole bands defined in the MultiValley section of the parameter file described in Multivalley Band Structure on page 267, but such spherical or ellipsoidal hole bands do not produce a correct dependency of the hole quantization on the interface orientation. Therefore, for holes, the six-band $k \cdot p$ band structure is used. The bulk six-band $k \cdot p$ model is described in [13][14] of Chapter 30 on page 687. An extension of the MLDA model is formulated for arbitrary bands and applied to the six-band $k \cdot p$ band structure [7].

The DOS of one hole band is written generally as follows:

$$D^i(\epsilon, z) = \frac{2}{(2\pi)^3} \oint \frac{dS}{|\nabla_k \epsilon^i(\vec{k})|} \left[1 - \exp \left(i 2z \gamma_{kp} \sum_j k_j \frac{w_{j3}(\vec{k})}{w_{33}(\vec{k})} \right) \right] \quad (215)$$

where:

- $\epsilon^i(\vec{k})$ is a hole band dispersion of the bulk six-band $k \cdot p$ model (the model considers three bands: heavy holes, light holes, and the spin-orbit split-off band holes).
- The surface integral is in k -space over isoenergy surface S defined by $\epsilon^i(\vec{k}) = \epsilon$.

- w_{ij} are components of the reciprocal mass tensor computed locally on the isoenergy surface.
- γ_{kp} is a fitting parameter that accounts for the nonparabolicity of hole bands in the MLDA model.

The hole density near the interface is computed similarly to the electron one in [Eq. 214](#), but with the DOS from [Eq. 215](#). The interface orientation and stress dependencies of the hole quantization are defined naturally by the physical property of the six-band $k \cdot p$ model.

This MLDA interface orientation-dependent quantization option is implemented using the multivalley carrier density model described in [Multivalley Band Structure on page 267](#). It uses all of the valley and mass parameter definitions of the multivalley model in the `MultiValley` section of the parameter file. In addition, it applies a numeric integration of [Eq. 212](#) based on Gauss–Laguerre quadratures as described in [Nonparabolic Band Structure on page 268](#). By default, the model automatically finds the closest interface and accounts for its orientation by recomputing the quantization mass for each valley and channel mesh point. The interface orientation is found using a vector normal to the interface and an orientation of the simulation coordinate system in reference to the crystal system defined in the `LatticeParameters` section of the parameter file (see [Crystal Reference System on page 669](#)). For fitting purposes, you have an option to define a specific quantization mass for each valley using the parameter file (see [Using MLDA](#)).

NOTE The multivalley orientation-dependent MLDA model always uses Fermi–Dirac statistics ([Eq. 212](#), [Eq. 214](#)).

Using MLDA

To activate the multivalley orientation-dependent model, the keyword `MLDA` must be used in the `MultiValley` statement of the `Physics` section:

```
Physics { MultiValley(MLDA) }
```

To activate the model only for electrons or holes, use `eMultiValley(MLDA)` or `hMultiValley(MLDA)`, respectively.

You can modify parameters of the model in the `MultiValley` section of the parameter file (see [Using Multivalley Band Structure on page 269](#)). By default, masses specified in the `(e|h)Valley` sections (m_{100} , m_{010} , and m_{001}) form the inverse mass tensor in the crystal system, and it is used to compute the quantization mass m_q by the previously described tensor rotation. For users who want to define a constant quantization mass per valley, ignoring automatic interface orientation dependency, there is an option to define it as the last value in the `Valley` statement:

```
MultiValley { (e|h)Valley(  $m_{100}$ ,  $m_{010}$ ,  $m_{001}$ ,  $\Delta E$ ,  $d$ ,  $\alpha$ ,  $m_q$  ) }
```

14: Quantization Models

Modified Local-Density Approximation

To activate the multivalley MLDA model based on the six-band $k \cdot p$ band structure for holes (see [Eq. 215](#)), an additional keyword should be used:

```
Physics { hMultiValley(MLDA kpDOS) }
```

In this case, all six-band $k \cdot p$ model parameters are defined in the `LatticeParameters` section of the parameter file (see [Using Strained Effective Masses and DOS on page 698](#)). The fitting parameter γ_{kp} in [Eq. 215](#) should be specified as a value of the parameter `hkpDOSfactor` in the `MLDAQMModel` section of the parameter file as follows:

```
MLDAQMModel{  
    hkpDOSfactor = 0.4  
    ekpDOSfactor = 1.0  
}
```

The parameter `ekpDOSfactor` can be used as a fitting parameter for the electron multivalley MLDA model (as a factor to z in [Eq. 214](#)) if `eMultiValley(MLDA kpDOS)` is specified in the `Physics` section of the command file.

By default, the MLDA model looks for all semiconductor–insulator interfaces and calculates the normal distance z in [Eq. 212–Eq. 215](#) to the nearest interface. The MLDA model also uses the `EnormalInterface` option and the `GeometricDistances` option (see [Normal to Interface on page 336](#)). To improve numerics if there is a coarse mesh in highly doped regions, there is an option to compute an averaged distance from a vertex to the interface based on element volumes and interface distances of elements, which contain the vertex (surrounding elements). The option is implemented only for the multivalley MLDA and has an adjusting parameter that can be specified as follows:

```
Math{  
    MVMLDAcontrols(AveDistanceFactor = 0.05)  
}
```

The default value of the parameter is 0.05 but, if it is equal to zero, the normal distance z is not modified by such averaging. If the parameter is unit, the distance z will be computed as an averaged distance using the normal distances of all vertices of the surrounded semiconductor elements with element–vertex volume weights.

To control a distance from the interface where the multivalley MLDA models will be applied, use the following statement:

```
Math {  
    MVMLDAcontrols(MaxIntDistance = 1e-6)  
}
```

The default value of `MaxIntDistance` is 10^{-6} cm.

To control the multivalley MLDA models by the doping concentration (for example, to exclude partially source/drain regions), the following command can be used:

```
Math {
    MVMLDAcontrols(MaxDoping4Majority = 1e19)
}
```

where `MaxDoping4Majority` defines the maximum doping concentration where the multivalley MLDA models are applied for majority carriers. For example, if `MaxDoping4Majority` is equal to zero, it excludes source/drain regions completely and the models are applied only for minority carriers. The default value of `MaxDoping4Majority` is 10^{22} cm^{-3} , that is, there are no limitations related to the doping concentration.

If the statement `hMultivalley(MLDA kpDOS)` is activated, the model performs an initial computation of energy-dependent data based on the bulk six-band $k \cdot p$ dispersion model. This computation may be time consuming, especially for 3D simulations and with the `hSubband` model active (see [Stress-dependent Mobility Models on page 700](#)). To separate this initial computation from bias ramping, there is an option to save and load the energy-dependent data file, for example:

```
Math { MVMLDAcontrols(Save = "file_name") }
Math { MVMLDAcontrols(Load = "file_name") }
```

In addition, to exclude source/drain or other regions, the keyword `MLDAbox` can be used to define the range of the interface from which the MLDA distance function is calculated:

```
Math{
    EnormalInterface(
        regionInterface=["SemiRegion/InsulRegion"]
        materialInterface=["OxideAsSemiconductor/Silicon"]
    )
    * single MLDAbox
    MLDAbox( MinX=-0.1, MaxX=0.1, MinY=-0.01, MaxY=0.01 )
    * multiple MLDAbox
    MLDAbox( { MinX=-0.1, MaxX=0.1, MinY=-0.01, MaxY=0.01 }
        { MinX=-0.1, MaxX=0.1, MinY= 0.05, MaxY=0.06 } )
}
```

The unit of the parameters `MinX`, `MaxX`, `MinY`, and `MaxY` in the `MLDAbox` is micrometer.

To activate the simple MLDA model (with one parameter λ per carrier type), use the keyword `MLDA` in the `Physics` section. To activate the model for electrons or holes only, use `eMLDA` or `hMLDA`. The model also can be specified in the regionwise or materialwise `Physics` section, and can be switched off by using `-MLDA`, `-eMLDA`, or `-hMLDA`.

By default, the thermal wavelength is computed from the thermal wavelengths at 300 K as $\lambda = \lambda_{300} \sqrt{300 \text{ K} / T_n}$; alternatively, $\lambda = \lambda_{300}$ can be used.

To activate or deactivate the temperature dependence of the thermal wavelength, the LambdaTemp switch can be specified as an option to MLDA:

```
Physics {
  MLDA (
    * eMLDA | hMLDA for one carrier
    -LambdaTemp * switch off temperature dependency
  )
}
```

[Table 36](#) lists the coefficients for this model. The default values of λ_{300} were determined by comparison with the Schrödinger equation solver at 23.5 Å and 25.0 Å for electrons and holes, respectively. These parameters are accessible in the MLDA parameter set.

Table 36 Default coefficients for MLDA model

Symbol	Electrons		Holes		Unit
λ_{300}	eLambda	23.5×10^{-8}	hLambda	25×10^{-8}	cm

MLDA Application Notes

- For each carrier type, only one of the two MLDA options (either multivalley (e|h)MultiValley(MLDA) or only (e|h)MLDA) can be used in the Physics section.
- The multivalley MLDA model can be used to compute stress-related mobility change (see [Stress-dependent Mobility Models on page 700](#)) and, therefore, it is required to be switched on for these mobility models. However, for some cases, other quantum models may be needed in the carrier density computation. For such cases, the multivalley MLDA model can be activated for all models except the density as follows:

(e|h)MultiValley(MLDA -Density)

Such activation (exclusion of the multivalley option from the density computation) is global for the whole device even if it appears in one region of the device only.

- Theoretically, the MLDA model should give zero carrier density at the semiconductor–insulator interface. However, since the zero carrier density results in various numeric problems, the actual values of carrier density at the interface calculated by the program are very small but greater than zero. This is achieved by assuming a very thin transition layer (one-hundredth of an ångström) between the insulator and the semiconductor.

References

- [1] M. J. van Dort, P. H. Woerlee, and A. J. Walker, "A Simple Model for Quantisation Effects in Heavily-Doped Silicon MOSFETs at Inversion Conditions," *Solid-State Electronics*, vol. 37, no. 3, pp. 411–414, 1994.
- [2] S. A. Hareland et al., "A Simple Model for Quantum Mechanical Effects in Hole Inversion Layers in Silicon PMOS Devices," *IEEE Transactions on Electron Devices*, vol. 44, no. 7, pp. 1172–1173, 1997.
- [3] M. G. Ancona and H. F. Tiersten, "Macroscopic physics of the silicon inversion layer," *Physical Review B*, vol. 35, no. 15, pp. 7959–7965, 1987.
- [4] M. G. Ancona and G. J. Iafrate, "Quantum correction to the equation of state of an electron gas in a semiconductor," *Physical Review B*, vol. 39, no. 13, pp. 9536–9540, 1989.
- [5] G. Paasch and H. Übensee, "A Modified Local Density Approximation: Electron Density in Inversion Layers," *Physica Status Solidi (b)*, vol. 113, no. 1, pp. 165–178, 1982.
- [6] G. Paasch and H. Übensee, "Carrier Density near the Semiconductor–Insulator Interface: Local Density Approximation for Non-Isotropic Effective Mass," *Physica Status Solidi (b)*, vol. 118, no. 1, pp. 255–266, 1983.
- [7] O. Penzin et al., "Extended Quantum Correction Model Applied to Six-Band $k \cdot p$ Valence Bands Near Silicon/Oxide Interfaces," *IEEE Transactions on Electron Devices*, vol. 58, no. 6, pp. 1614–1619, 2011.

14: Quantization Models

References

CHAPTER 15 Mobility Models

This chapter presents information about the different mobility models implemented in Sentaurus Device.

Sentaurus Device uses a modular approach for the description of the carrier mobilities. In the simplest case, the mobility is a function of the lattice temperature. This so-called constant mobility model described in [Mobility due to Phonon Scattering on page 302](#) should only be used for undoped materials. For doped materials, the carriers scatter with the impurities. This leads to a degradation of the mobility. [Doping-dependent Mobility Degradation on page 302](#) introduces the models that describe this effect.

Models that describe the effects of carrier–carrier scattering are presented in [Carrier–Carrier Scattering on page 309](#). The Philips unified mobility model, described in [Philips Unified Mobility Model on page 310](#), is a well-calibrated model that accounts for both impurity and carrier–carrier scattering.

Models that describe the mobility degradation at interfaces, for example, the silicon–oxide interface in the channel region of a MOSFET, are introduced in [Mobility Degradation at Interfaces on page 315](#). These models account for the scattering with surface phonons and surface roughness.

Finally, the models that describe mobility degradation in high electric fields are discussed in [High-Field Saturation on page 342](#). A flexible model for hydrodynamic simulations is described in [Energy-dependent Mobility on page 658](#).

How Mobility Models Combine

The mobility models are selected in the Physics section as options to `Mobility`, `eMobility`, or `hMobility`:

```
Physics{ Mobility( <arguments> ) ... }
```

Specifications with `eMobility` apply to electrons; specifications with `hMobility` apply to holes; and specifications with `Mobility` apply to both carrier types.

15: Mobility Models

Mobility due to Phonon Scattering

If more than one mobility model is activated for a carrier type, the different mobility contributions ($\mu_1, \mu_2, \dots$) for bulk, surface mobility, and thin layers are combined by Matthiessen's rule:

$$\frac{1}{\mu} = \frac{1}{\mu_1} + \frac{1}{\mu_2} + \dots \quad (216)$$

If the high-field saturation model is activated, the final mobility is computed in two steps. First, the low field mobility μ_{low} is determined according to [Eq. 216](#). Second, the final mobility is computed from a (model-dependent) formula as a function of a driving force F_{hfs} :

$$\mu = f(\mu_{\text{low}}, F_{\text{hfs}}) \quad (217)$$

Mobility due to Phonon Scattering

The constant mobility model [\[1\]](#) is active by default. It accounts only for phonon scattering and, therefore, it is dependent only on the lattice temperature:

$$\mu_{\text{const}} = \mu_L \left(\frac{T}{300\text{K}} \right)^{-\zeta} \quad (218)$$

where μ_L is the mobility due to bulk phonon scattering. The default values of μ_L and the exponent ζ are listed in [Table 37](#). The constant mobility model parameters are accessible in the ConstantMobility parameter set.

Table 37 Constant mobility model: Default coefficients for silicon

Symbol	Parameter name	Electrons	Holes	Unit
μ_L	mumax	1417	470.5	cm^2/Vs
ζ	exponent	2.5	2.2	1

In some special cases, it may be necessary to deactivate the default constant mobility. This can be accomplished by specifying the -ConstantMobility option to Mobility:

```
Physics { Mobility( -ConstantMobility ... ) ... }
```

Doping-dependent Mobility Degradation

In doped semiconductors, scattering of the carriers by charged impurity ions leads to degradation of the carrier mobility. Sentaurus Device supports three built-in models, one multistate configuration-dependent model, and two PMIs for doping-dependent mobility.

Using Doping-dependent Mobility

The models for the mobility degradation due to impurity scattering are activated by specifying the `DopingDependence` flag to `Mobility`:

```
Physics{ Mobility( DopingDependence ... ) ... }
```

Different models are available and are selected by options to `DopingDependence`:

```
Physics{ Mobility( DopingDependence( [ Masetti | Arora | UniBo ] ) ...) ... }
```

If `DopingDependence` is specified without options, Sentaurus Device uses a material-dependent default. For example, in silicon, the default is the Masetti model; for GaAs, the default is the Arora model. The default model (Masetti or Arora) for each material can be specified by the variable `formula`, which is accessible in the `DopingDependence` parameter set:

```
DopingDependence: {
    formula= 1 , 1 # [1]
    ... }
```

If `formula` is set to 1, the Masetti model is the default. To use the Arora model as the default, set `formula` to 2.

The `DopingDependence` parameter set also determines the other parameters for the Masetti and Arora models. The parameters for the University of Bologna model are in the `UniBoDopingDependence` parameter set.

Sentaurus Device allows more than one doping-dependent mobility model to be used in a simulation. For example, it may be useful to include additional degradation components that are created as PMI models into the bulk mobility calculation. When more than one model is specified as an option to `DopingDependence`, they are combined using Matthiessen's rule. For example, the specification:

```
Physics { Mobility( DopingDependence( Masetti pmi_model1 pmi_model2 ) ...) ... }
```

calculates the total bulk mobility using:

$$\frac{1}{\mu_b} = \frac{1}{\mu_{\text{Masetti}}} + \frac{1}{\mu_{\text{pmi_model1}}} + \frac{1}{\mu_{\text{pmi_model2}}} \quad (219)$$

Masetti Model

The default model used by Sentaurus Device to simulate doping-dependent mobility in silicon was proposed by Masetti *et al.* [2]:

$$\mu_{\text{dop}} = \mu_{\text{min1}} \exp\left(-\frac{P_c}{N_{A,0} + N_{D,0}}\right) + \frac{\mu_{\text{const}} - \mu_{\text{min2}}}{1 + ((N_{A,0} + N_{D,0})/C_r)^\alpha} - \frac{\mu_1}{1 + (C_s/(N_{A,0} + N_{D,0}))^\beta} \quad (220)$$

The reference mobilities μ_{min1} , μ_{min2} , and μ_1 , the reference doping concentrations P_c , C_r , and C_s , and the exponents α and β are accessible in the parameter set DopingDependence.

The corresponding values for silicon are given in Table 38.

Table 38 Masetti model: Default coefficients

Symbol	Parameter name	Electrons	Holes	Unit
μ_{min1}	mumin1	52.2	44.9	cm ² /Vs
μ_{min2}	mumin2	52.2	0	cm ² /Vs
μ_1	mu1	43.4	29.0	cm ² /Vs
P_c	Pc	0	9.23×10^{16}	cm ⁻³
C_r	Cr	9.68×10^{16}	2.23×10^{17}	cm ⁻³
C_s	Cs	3.43×10^{20}	6.10×10^{20}	cm ⁻³
α	alpha	0.680	0.719	1
β	beta	2.0	2.0	1

The low-doping reference mobility μ_{const} is determined by the constant mobility model (see [Mobility due to Phonon Scattering on page 302](#)).

Arora Model

The Arora model [3] reads:

$$\mu_{\text{dop}} = \mu_{\text{min}} + \frac{\mu_d}{1 + ((N_{A,0} + N_{D,0})/N_0)^{A^*}} \quad (221)$$

with:

$$\mu_{\text{min}} = A_{\text{min}} \cdot \left(\frac{T}{300\text{K}}\right)^{\alpha_m}, \mu_d = A_d \cdot \left(\frac{T}{300\text{K}}\right)^{\alpha_d} \quad (222)$$

and:

$$N_0 = A_N \cdot \left(\frac{T}{300\text{K}}\right)^{\alpha_N}, A^* = A_a \cdot \left(\frac{T}{300\text{K}}\right)^{\alpha_a} \quad (223)$$

The parameters are accessible in the DopingDependence parameter set.

Table 39 Arora model: Default coefficients for silicon

Symbol	Parameter name	Electrons	Holes	Unit
$A_{\min}$	Ar_mumin	88	54.3	cm ² /Vs
α_m	Ar_alm	-0.57	-0.57	1
A_d	Ar_mud	1252	407	cm ² /Vs
α_d	Ar_ald	-2.33	-2.23	1
A_N	Ar_N0	1.25×10 ¹⁷	2.35×10 ¹⁷	cm ⁻³
α_N	Ar_alN	2.4	2.4	1
A_a	Ar_a	0.88	0.88	1
α_a	Ar_ala	-0.146	-0.146	1

University of Bologna Bulk Mobility Model

The University of Bologna bulk mobility model was developed for an extended temperature range between 25°C and 973°C. It should be used together with the University of Bologna inversion layer mobility model (see [University of Bologna Surface Mobility Model on page 330](#)). The model [4][5] is based on the Masetti approach with two major extensions. First, attractive and repulsive scattering are separately accounted for, therefore, leading to a function of both donor and acceptor concentrations. This automatically accounts for different mobility values for majority and minority carriers, and ensures continuity at the junctions as long as the impurity concentrations are continuous functions. Second, a suitable temperature dependence for most model parameters is introduced to predict correctly the temperature dependence of carrier mobility in a wider range of temperatures, with respect to other models. The temperature dependence of lattice mobility is reworked, with respect to the default temperature.

The model for lattice mobility is:

$$\mu_L(T) = \mu_{\max} \left(\frac{T}{300\text{K}}\right)^{-\gamma + c \left(\frac{T}{300\text{K}}\right)} \quad (224)$$

15: Mobility Models

Doping-dependent Mobility Degradation

where $\mu_{\max}$ denotes the lattice mobility at room temperature, and c gives a correction to the lattice mobility at higher temperatures. The maximum mobility $\mu_{\max}$ and the exponents γ and c are accessible in the parameter file.

The model for bulk mobility reads:

$$\mu_{\text{dop}}(T) = \mu_0(T) + \frac{\mu_L(T) - \mu_0(T)}{1 + \left(\frac{N_{D,0}}{C_{r1}(T)}\right)^\alpha + \left(\frac{N_{A,0}}{C_{r2}(T)}\right)^\beta} - \frac{\mu_1(N_{D,0}, N_{A,0}, T)}{1 + \left(\frac{N_{D,0}}{C_{s1}(T)} + \frac{N_{A,0}}{C_{s2}(T)}\right)^{-2}} \quad (225)$$

In turn, μ_0 and μ_1 are expressed as weighted averages of the corresponding limiting values for pure acceptor-doping and pure donor-doping densities:

$$\mu_0(T) = \frac{\mu_{0d}N_{D,0} + \mu_{0a}N_{A,0}}{N_{A,0} + N_{D,0}} \quad (226)$$

$$\mu_1(T) = \frac{\mu_{1d}N_{D,0} + \mu_{1a}N_{A,0}}{N_{A,0} + N_{D,0}} \quad (227)$$

The reference mobilities μ_{0d} , μ_{0a} , μ_{1d} , and μ_{1a} , and the reference doping concentrations C_{r1} , C_{r2} , C_{s1} , and C_{s2} are accessible in the parameter file.

[Table 40](#) lists the corresponding values of silicon for arsenic, phosphorus, and boron; $T_n = T/300$ K.

Table 40 Parameters of University of Bologna bulk mobility model

Silicon	Parameter name	Electrons (As)	Electrons (P)	Holes (B)	Unit
$\mu_{\max}$	mumax	1441	1441	470.5	cm^2/Vs
c	Exponent2	-0.11	-0.11	0	1
γ	Exponent	2.45	2.45	2.16	1
μ_{0d}	mumin1	$55.0T_n^{-\gamma_{0d}}$	$62.2T_n^{-\gamma_{0d}}$	$90.0T_n^{-\gamma_{0d}}$	cm^2/Vs
γ_{0d}	mumin1_exp	0.6	0.7	1.3	1
μ_{0a}	mumin2	$132.0T_n^{-\gamma_{0a}}$	$132.0T_n^{-\gamma_{0a}}$	$44.0T_n^{-\gamma_{0a}}$	cm^2/Vs
γ_{0a}	mumin2_exp	1.3	1.3	0.7	1
μ_{1d}	mu1	$42.4T_n^{-\gamma_{1d}}$	$48.6T_n^{-\gamma_{1d}}$	$28.2T_n^{-\gamma_{1d}}$	cm^2/Vs
γ_{1d}	mu1_exp	0.5	0.7	2.0	1
μ_{1a}	mu2	$73.5T_n^{-\gamma_{1a}}$	$73.5T_n^{-\gamma_{1a}}$	$28.2T_n^{-\gamma_{1a}}$	cm^2/Vs
γ_{1a}	mu2_exp	1.25	1.25	0.8	1
C_{r1}	Cr	$8.9 \times 10^{16} T_n^{\gamma_{r1}}$	$8.5 \times 10^{16} T_n^{\gamma_{r1}}$	$1.3 \times 10^{18} T_n^{\gamma_{r1}}$	cm^{-3}

Table 40 Parameters of University of Bologna bulk mobility model

Silicon	Parameter name	Electrons (As)	Electrons (P)	Holes (B)	Unit
γ_{r1}	Cr_exp	3.65	3.65	2.2	1
C_{r2}	Cr2	$1.22 \times 10^{17} T_n^{\gamma_{r2}}$	$1.22 \times 10^{17} T_n^{\gamma_{r2}}$	$2.45 \times 10^{17} T_n^{\gamma_{r2}}$	cm^{-3}
γ_{r2}	Cr2_exp	2.65	2.65	3.1	1
C_{s1}	Cs	$2.9 \times 10^{20} T_n^{\gamma_{s1}}$	$4.0 \times 10^{20} T_n^{\gamma_{s1}}$	$1.1 \times 10^{18} T_n^{\gamma_{s1}}$	cm^{-3}
γ_{s1}	Cs_exp	0	0	6.2	1
C_{s2}	Cs2	7.0×10^{20}	7.0×10^{20}	6.1×10^{20}	cm^{-3}
α	alpha	0.68	0.68	0.77	1
β	beta	0.72	0.72	0.719	1

The default parameters of Sentaurus Device for electrons are those for arsenic. The bulk mobility model was calibrated with experiments [5] in the temperature range from 300 K to 700 K. It is suitable for isothermal simulations at large temperatures or nonisothermal simulations.

The pmi_msc_mobility Model

The mobility model `pmi_msc_mobility` depends on the lattice temperature and the occupation probabilities of the states of a multistate configuration (MSC). The model averages the mobilities of all MSC states according to:

$$\mu = \sum_i \mu_i(T) s_i \quad (228)$$

where the sum is taken over all MSC states, μ_i are the state mobilities, and s_i are the state occupation probabilities. Each state mobility can be temperature dependent according to:

$$\mu_i = \begin{cases} \mu_{i, \text{ref}} & \text{if } T < T_{\text{ref}} \\ \frac{1}{T_g - T_{\text{ref}}} [(T_g - T)\mu_{i, \text{ref}} + (T - T_{\text{ref}})\mu_{i, g}] & \text{if } T_{\text{ref}} \leq T < T_g \\ \mu_{i, g} & \text{if } T_g \leq T \end{cases} \quad (229)$$

where:

- The index i refers to the state.
- T_{ref} and T_g are the reference and glass temperatures (terminology is borrowed from and the model is used for phase transition dynamics).
- $\mu_{i, \text{ref}}$ and $\mu_{i, g}$ are the (state-specific) mobilities for the corresponding temperatures.

15: Mobility Models

Doping-dependent Mobility Degradation

The model is activated (here, for MSC "m0" and model string "e") by:

```
PMIModel ( Name="pmi_msc_mobility" MSConfig="m0" String="e" )
```

Note that the given String is used to determine the names of the parameters in the parameter file.

Table 41 Global and model-string parameters of pmi_msc_mobility

Name	Symbol	Default	Unit	Range	Description
plot	–	0	–	{0,1}	Plot parameter to screen
Tref	T_{ref}	300.	K	$\geq 0.$	Reference temperature
Tg	T_{g}	-1.	K	$\geq T_{\text{ref}}$ or equal -1.	Glass temperature
mu_ref	–	1.	cm/Vs	$> 0.$	Value at reference temperature
mu_g	–	1.	cm/Vs	$> 0.$	Value at glass temperature

Table 42 State parameters of pmi_msc_mobility

Name	Symbol	Default	Unit	Range	Description
mu_ref	$\mu_{i, \text{ref}}$	–	cm^2/Vs	$> 0.$	Value at reference temperature
mu_g	$\mu_{i, \text{g}}$	–	cm^2/Vs	$> 0.$	Value at glass temperature

The model uses a three-level hierarchy of sets of parameters to define the parameters for each state, namely, the global, the model-string, and the state parameters. The global parameters of the model are given in [Table 41](#). They serve as defaults for the model-string parameters, which are the global parameters with the prefix:

```
<model_string>_
```

where <model_string> is the String parameter given in the command file. The model-string parameters, in turn, serve as defaults for the state parameters (see [Table 42](#)), which are prefixed by:

```
<model_string>_<state_name>_
```

where <state_name> is the name of the MSC state.

PMIs for Bulk Mobility

The PMIs for bulk mobility are described in [Doping-dependent Mobility on page 1023](#) and [Multistate Configuration-dependent Bulk Mobility on page 1029](#).

Carrier–Carrier Scattering

Two models are supported for the description of carrier–carrier scattering. One model is based on Choo [14] and Fletcher [15] and uses the Conwell–Weisskopf screening theory. As an alternative to the Conwell–Weisskopf model, Sentaurus Device supports the Brooks–Herring model. The carrier–carrier contribution to the overall mobility degradation is captured in the mobility term μ_{eh} . This is combined with the mobility contributions from other degradation models (μ_{other}) according to Matthiessen’s rule:

$$\frac{1}{\mu} = \frac{1}{\mu_{other}} + \frac{1}{\mu_{eh}} \quad (230)$$

Using Carrier–Carrier Scattering

The carrier–carrier scattering models are activated by specifying the `CarrierCarrierScattering` option to `Mobility`. Either of the two different models are selected by an additional flag:

```
Physics{ Mobility(
    CarrierCarrierScattering( [ ConwellWeisskopf | BrooksHerring ] )
    ... ) ... }
```

The Conwell–Weisskopf model is the default in Sentaurus Device for carrier–carrier scattering and is activated when `CarrierCarrierScattering` is specified without an option.

Conwell–Weisskopf Model

$$\mu_{eh} = \frac{D(T/300\text{ K})^{3/2}}{\sqrt{np}} \left[\ln \left(1 + F \left(\frac{T}{300\text{ K}} \right)^2 (pn)^{-1/3} \right) \right]^{-1} \quad (231)$$

The parameters D and F are accessible in the parameter file. The default values appropriate for silicon are given in [Table 43 on page 310](#).

Brooks–Herring Model

$$\mu_{eh} = \frac{c_1 (T/300\text{ K})^{3/2}}{\sqrt{np}} \frac{1}{\phi(\eta_0)} \quad (232)$$

where $\phi(\eta_0) = \ln(1 + \eta_0) - \eta_0/(1 + \eta_0)$ and:

$$\eta_0(T) = \frac{c_2}{N_C F_{-1/2}(n/N_C) + N_V F_{-1/2}(p/N_V)} \left(\frac{T}{300\text{ K}} \right)^2 \quad (233)$$

Table 44 lists the silicon default values for c_1 and c_2 .

Physical Model Parameters

Parameters for the carrier–carrier scattering models are accessible in the parameter set CarrierCarrierScattering.

Table 43 Conwell–Weisskopf model: Default parameters

Symbol	Parameter name	Value	Unit
D	D	1.04×10^{21}	$\text{cm}^{-1} \text{V}^{-1} \text{s}^{-1}$
F	F	7.452×10^{13}	cm^{-2}

Table 44 Brooks-Herring model: Default parameters

Symbol	Parameter name	Value	Unit
c_1	c1	1.56×10^{21}	$\text{cm}^{-1} \text{V}^{-1} \text{s}^{-1}$
c_2	c2	7.63×10^{19}	cm^{-3}

Philips Unified Mobility Model

The Philips unified mobility model, proposed by Klaassen [16], unifies the description of majority and minority carrier bulk mobilities. In addition to describing the temperature dependence of the mobility, the model takes into account electron–hole scattering, screening of ionized impurities by charge carriers, and clustering of impurities.

The Philips unified mobility model is well calibrated. Though it was initially used primarily for bipolar devices, it is widely used for MOS devices.

Using Philips Model

The Philips unified mobility model is activated by specifying the `PhuMob` option to `Mobility`:

```
Physics{ Mobility( PhuMob ... ) ... }
```

The model can also be activated with one or two additional options:

```
Physics{ Mobility( PhuMob( [ Arsenic | Phosphorus ]  
    [ Klaassen | Meyer ] ) ...) ... }
```

The option `Arsenic` or `Phosphorus` specifies which parameters (see [Table 46 on page 314](#)) are used. These parameters reflect the different electron mobility degradation that is observed in the presence of these donor species.

The option `Klaassen` or `Meyer` specifies which parameters (see [Table 45 on page 314](#)) are used for the evaluation of $G(P_i)$. Sentaurus Device defaults to the `Klaassen` parameters.

NOTE The Philips unified mobility model describes mobility degradation due to both impurity scattering and carrier–carrier scattering mechanisms. Therefore, the keyword `PhuMob` must not be combined with the keyword `DopingDependence` or `CarrierCarrierScattering`. If a combination of these keywords is specified, Sentaurus Device uses only the Philips unified mobility model.

Using an Alternative Philips Model

Sentaurus Device provides an extended PMI reimplementation of the Philips unified mobility model (see [Doping-dependent Mobility on page 1023](#)). To use it, specify `PhuMob2` as the option for `DopingDependence`:

```
Physics{Mobility(DopingDependence(PhuMob2)...)...}
```

The reimplementation uses the `PhuMob2` parameter set. The new parameter, `FACT_G`, has a default value of 1. It modifies the function $G(P_i)$ shown in [Eq. 246](#) by a factor $G(P_i) = \text{FACT_G}$ (right-hand side in [Eq. 246](#)). Otherwise, the reimplementation fully agrees with the model described below.

Philips Model Description

According to the Philips unified mobility model, there are two contributions to carrier mobilities. The first, $\mu_{i,L}$, represents phonon (lattice) scattering and the second, $\mu_{i,DAeh}$,

15: Mobility Models

Philips Unified Mobility Model

accounts for all other bulk scattering mechanisms (due to free carriers, and ionized donors and acceptors). These partial mobilities are combined to give the bulk mobility $\mu_{i,b}$ for each carrier according to Matthiessen's rule:

$$\frac{1}{\mu_{i,b}} = \frac{1}{\mu_{i,L}} + \frac{1}{\mu_{i,DAeh}} \quad (234)$$

In Eq. 234 and all of the following model equations, the index i takes the value 'e' for electrons and 'h' for holes. The first contribution due to lattice scattering takes the form:

$$\mu_{i,L} = \mu_{i,max} \left(\frac{T}{300\text{ K}} \right)^{-\theta_i} \quad (235)$$

The second contribution has the form:

$$\mu_{i,DAeh} = \mu_{i,N} \left(\frac{N_{i,sc}}{N_{i,sc,eff}} \right) \left(\frac{N_{i,ref}}{N_{i,sc}} \right)^{\alpha_i} + \mu_{i,c} \left(\frac{n+p}{N_{i,sc,eff}} \right) \quad (236)$$

with:

$$\mu_{i,N} = \frac{\mu_{i,max}^2}{\mu_{i,max} - \mu_{i,min}} \left(\frac{T}{300\text{ K}} \right)^{3\alpha_i - 1.5} \quad (237)$$

$$\mu_{i,c} = \frac{\mu_{i,max} \mu_{i,min}}{\mu_{i,max} - \mu_{i,min}} \left(\frac{300\text{ K}}{T} \right)^{0.5} \quad (238)$$

for the electrons:

$$N_{e,sc} = N_D^* + N_A^* + p \quad (239)$$

$$N_{e,sc,eff} = N_D^* + G(P_e)N_A^* + f_e \frac{p}{F(P_e)} \quad (240)$$

and for holes:

$$N_{h,sc} = N_A^* + N_D^* + n \quad (241)$$

$$N_{h,sc,eff} = N_A^* + G(P_h)N_D^* + f_h \frac{n}{F(P_h)} \quad (242)$$

The effects of clustering of donors (N_D^*) and acceptors (N_A^*) at ultrahigh concentrations are described by ‘clustering’ functions Z_D and Z_A , which are defined as:

$$N_D^* = N_{D,0} Z_D = N_{D,0} \left[1 + \frac{N_{D,0}^2}{c_D N_{D,0}^2 + N_{D,ref}^2} \right] \quad (243)$$

$$N_A^* = N_{A,0} Z_A = N_{A,0} \left[1 + \frac{N_{A,0}^2}{c_A N_{A,0}^2 + N_{A,ref}^2} \right] \quad (244)$$

The analytic functions $G(P_i)$ and $F(P_i)$ in Eq. 240 and Eq. 242 describe minority impurity and electron–hole scattering. They are given by:

$$F(P_i) = \frac{0.7643 P_i^{0.6478} + 2.2999 + 6.5502(m_i^*/m_j^*)}{P_i^{0.6478} + 2.3670 - 0.8552(m_i^*/m_j^*)} \quad (245)$$

and:

$$G(P_i) = 1 - a_g \left[b_g + P_i \left(\frac{m_0}{m_i^*} \frac{T}{300 \text{ K}} \right)^{\alpha_g} \right]^{-\beta_g} + c_g \left[P_i \left(\frac{m_i^*}{m_0} \frac{300 \text{ K}}{T} \right)^{\alpha'_g} \right]^{-\gamma_g} \quad (246)$$

where m_i^* denotes a fit parameter for carrier i (which is related to the effective carrier mass) and m_j^* denotes a fit parameter for the other carrier.

Screening Parameter

The screening parameter P_i is given by a weighted harmonic mean of the Brooks–Herring approach and Conwell–Weisskopf approach:

$$P_i = \left[\frac{f_{CW}}{3.97 \times 10^{13} \text{ cm}^{-2} N_{i,sc}^{-2/3}} + f_{BH} \frac{(n+p)}{1.36 \times 10^{20} \text{ cm}^{-3} m_i^*} \right]^{-1} \left(\frac{T}{300 \text{ K}} \right)^2 \quad (247)$$

If the Klaassen parameter set is selected, the evaluation of $G(P_i)$ depends on the value of the screening parameter P_i . For values of $P_i < P_{i,min}$, $G(P_{i,min})$ is used instead of $G(P_i)$, where $P_{i,min}$ is the value at which $G(P_i)$ reaches its minimum.

If the Meyer parameter set is selected, Eq. 246 is used independently of the value of P_i .

Philips Model Parameters

Sentaurus Device supports two different built-in values for each of the parameters a_g , b_g , c_g , α_g , α'_g , β_g , and γ_g in Eq. 246. They are selected by the Klaassen or Meyer option (see [Using Philips Model on page 311](#)). The corresponding values are given in Table 45. Sentaurus Device defaults to the Klaassen parameters.

Table 45 Philips unified mobility model: Klaassen versus Meyer parameters

Symbol	Klaassen	Meyer	Unit
a_g	0.89233	4.41804	1
b_g	0.41372	39.9014	1
c_g	0.005978	0.52896	1
α_g	0.28227	0.0001	1
α'_g	0.72169	1.595187	1
β_g	0.19778	0.38297	1
γ_g	1.80618	0.25948	1

Other parameters for the Philips unified mobility model are accessible in the parameter set PhuMob.

Table 46 and Table 47 on page 315 list the silicon defaults for other parameters. Sentaurus Device supports different parameters for electron mobility, which are optimized for situations where the dominant donor species in the silicon is either arsenic or phosphorus. The arsenic parameters are used by default.

Table 46 Philips unified mobility model: Electron and hole parameters (silicon)

Symbol	Parameter name	Electrons (arsenic)	Electrons (phosphorus)	Holes (boron)	Unit
$\mu_{\max}$	mumax_*	1417	1414	470.5	cm^2/Vs
$\mu_{\min}$	mumin_*	52.2	68.5	44.9	cm^2/Vs
θ	theta_*	2.285	2.285	2.247	1
$N_{\{e,h\},\text{ref}}$	n_ref_*	9.68×10^{16}	9.2×10^{16}	2.23×10^{17}	cm^{-3}
α	alpha_*	0.68	0.711	0.719	1

The original Philips unified mobility model uses four fit parameters: the weight factors f_{CW} and f_{BH} , and the ‘effective masses’ m^*_e and m^*_h . The optimal parameter set, determined by accurate fitting to experimental data [16] is shown in Table 47. The Philips unified mobility

model can be slightly modified by setting parameters $f_e = 0$ and $f_h = 0$ as required for the Lucent mobility model (see [Lucent Model on page 346](#)).

Table 47 Philips unified mobility model: Other fitting parameters (silicon)

Symbol	Parameter name	Value	Unit
m_e^*/m_0	me_over_m0	1	1
m_h^*/m_0	mh_over_m0	1.258	1
f_{CW}	f_CW	2.459	1
f_{BH}	f_BH	3.828	1
f_e	f_e	1.0	1
f_h	f_h	1.0	1

Mobility Degradation at Interfaces

In the channel region of a MOSFET, the high transverse electric field forces carriers to interact strongly with the semiconductor–insulator interface. Carriers are subjected to scattering by acoustic surface phonons and surface roughness. The models in this section describe mobility degradation caused by these effects.

Using Mobility Degradation at Interfaces

To activate mobility degradation at interfaces, select a method to compute the transverse field $F_{\perp}$ (see [Computing Transverse Field on page 335](#)).

To select the calculation of field perpendicular to the semiconductor–insulator interface, specify the `Enormal` option to `Mobility`:

```
Physics{ Mobility( Enormal ...) ...}
```

Alternatively, to select calculation of $F_{\perp}$ perpendicular to current flow, specify:

```
Physics{ Mobility( ToCurrentEnormal ...) ...}
```

To select a mobility degradation model, specify an option to `Enormal` or `ToCurrentEnormal`. Valid options are `Lombardi`, `Lombardi_highk`, `IAlMob`, `UniBo`, or a PMI model provided by you. In addition, one or more mobility degradation components can be specified. These include `NegInterfaceCharge`, `PosInterfaceCharge`, and `Coulomb2D`. For example:

```
Physics{ Mobility( Enormal(UniBo) ...) ...}
```

15: Mobility Models

Mobility Degradation at Interfaces

selects the University of Bologna model (see [University of Bologna Surface Mobility Model on page 330](#)). The default model is Lombardi (see [Enhanced Lombardi Model on page 316](#)).

NOTE The mobility degradation models discussed in this section are very sensitive to mesh spacing. It is recommended that the vertical mesh spacing be reduced to 0.1nm in the silicon at the oxide interface underneath the gate. For the extensions of the Lombardi model (see [Eq. 252](#)), even smaller spacing of 0.05 nm is appropriate. This fine spacing is only required in the two uppermost rows of mesh and can be increased progressively moving away from the interface.

Sentaurus Device allows more than one interface degradation mobility model to be used in a simulation. For example, it may be useful to include additional degradation components that are created as PMI models into the Enormal mobility calculation. When more than one model is specified as an option to Enormal, they are combined using Matthiessen's rule. For example, the specification:

```
Physics { Mobility( Enormal( Lombardi pmi_model1 pmi_model2) ...) ... }
```

calculates the total Enormal mobility using:

$$\frac{1}{\mu_{\text{Enormal}}} = \frac{1}{\mu_{\text{Lombardi}}} + \frac{1}{\mu_{\text{pmi_model1}}} + \frac{1}{\mu_{\text{pmi_model2}}} \quad (248)$$

Enhanced Lombardi Model

The surface contribution due to acoustic phonon scattering has the form:

$$\mu_{\text{ac}} = \frac{B}{F_{\perp}} + \frac{C((N_{\text{A},0} + N_{\text{D},0} + N_2)/N_0)^{\lambda}}{F_{\perp}^{1/3} (T/300 \text{ K})^k} \quad (249)$$

and the contribution attributed to surface roughness scattering is given by:

$$\mu_{\text{sr}} = \left(\frac{(F_{\perp}/F_{\text{ref}})^{A^*}}{\delta} + \frac{F_{\perp}^3}{\eta} \right)^{-1} \quad (250)$$

These surface contributions to the mobility (μ_{ac} and μ_{sr}) are then combined with the bulk mobility μ_{b} according to Matthiessen's rule (see [Mobility due to Phonon Scattering on page 302](#) and [Doping-dependent Mobility Degradation on page 302](#)):

$$\frac{1}{\mu} = \frac{1}{\mu_{\text{b}}} + \frac{D}{\mu_{\text{ac}}} + \frac{D}{\mu_{\text{sr}}} \quad (251)$$

The reference field $F_{\text{ref}} = 1 \text{ V/cm}$ ensures a unitless numerator in Eq. 250. $F_{\perp}$ is the transverse electric field normal to the semiconductor–insulator interface, see [Computing Transverse Field on page 335](#). $D = \exp(-x/l_{\text{crit}})$ (where x is the distance from the interface and l_{crit} a fit parameter) is a damping that switches off the inversion layer terms far away from the interface. All other parameters are accessible in the parameter file.

In the Lombardi model [1], the exponent A^* in Eq. 250 is equal to 2. According to another study [6], an improved fit to measured data is achieved if A^* is given by:

$$A^* = A + \frac{(\alpha_{\perp,n}n + \alpha_{\perp,p}p)N_{\text{ref}}^v}{(N_{A,0} + N_{D,0} + N_1)^v} \quad (252)$$

where n and p denote the electron and hole concentrations, respectively. For electron mobility, $\alpha_{\perp,n} = \alpha_{\perp}$ and $\alpha_{\perp,p} = \alpha_{\perp}a_{\text{other}}$; for hole mobility, $\alpha_{\perp,n} = \alpha_{\perp}a_{\text{other}}$ and $\alpha_{\perp,p} = \alpha_{\perp}$.

The reference doping concentration $N_{\text{ref}} = 1 \text{ cm}^{-3}$ cancels the unit of the term raised to the power v in the denominator of Eq. 252. The Lombardi model parameters are accessible in the parameter set EnormalDependence.

The respective default parameters that are appropriate for silicon are given in Table 48. The parameters B , C , N_0 , and λ were fitted at SGS Thomson and are not contained in the literature [1].

Table 48 Lombardi model: Default coefficients for silicon

Symbol	Parameter name	Electrons	Holes	Unit
B	B	4.75×10^7	9.925×10^6	cm/s
C	C	5.80×10^2	2.947×10^3	$\text{cm}^{5/3}\text{V}^{-2/3}\text{s}^{-1}$
N_0	N0	1	1	cm^{-3}
N_2	N2	1	1	cm^{-3}
λ	lambda	0.1250	0.0317	1
k	k	1	1	1
δ	delta	5.82×10^{14}	2.0546×10^{14}	cm^2/Vs
A	A	2	2	1
$\alpha_{\perp}$	alpha	0	0	cm^3
N_1	N1	1	1	cm^{-3}
v	nu	1	1	1
η	eta	5.82×10^{30}	2.0546×10^{30}	$\text{V}^2\text{cm}^{-1}\text{s}^{-1}$

15: Mobility Models

Mobility Degradation at Interfaces

Table 48 Lombardi model: Default coefficients for silicon

Symbol	Parameter name	Electrons	Holes	Unit
a_{other}	aother	0	0	1
l_{crit}	l_crit	1×10^{-6}	1×10^{-6}	cm

The modifications of the Lombardi model suggested in [6] can be activated by setting $\alpha_{\perp}$ to a nonzero value.

Table 49 lists a consistent set of parameters for the modified model.

Table 49 Lombardi model: Lucent coefficients for silicon

Symbol	Parameter name	Electrons	Holes	Unit
B	B	3.61×10^7	1.51×10^7	cm/s
C	C	1.70×10^4	4.18×10^3	$\text{cm}^{5/3}\text{V}^{-2/3}\text{s}^{-1}$
N_0	N0	1	1	cm^{-3}
N_2	N2	1	1	cm^{-3}
λ	lambda	0.0233	0.0119	1
k	k	1.7	0.9	1
δ	delta	3.58×10^{18}	4.10×10^{15}	cm^2/Vs
A	A	2.58	2.18	1
$\alpha_{\perp}$	alpha	6.85×10^{-21}	7.82×10^{-21}	cm^3
N_1	N1	1	1	cm^{-3}
ν	nu	0.0767	0.123	1
η	eta	1×10^{50}	1×10^{50}	$\text{V}^2\text{cm}^{-1}\text{s}^{-1}$
a_{other}	aother	1	1	1
l_{crit}	l_crit	1	1	cm

Named Parameter Sets for Lombardi Model

The `EnormalDependence` parameter set can be named. For example, in the parameter file, you can write:

```
EnormalDependence "myset" { ... }
```

to declare a parameter set with the name `myset`. To use a named parameter set, specify its name with `ParameterSetName` as an option to `Lombardi` in the command file, for example:

```
Mobility (
    Enormal( Lombardi( ParameterSetName = "myset" ...) ...)
)
```

By default, the unnamed parameter set is used.

Orientation-dependent Lombardi Parameters

To facilitate simulations that account for surface orientation dependency of mobility, three named parameter sets for the `Lombardi` model are available in the parameter file:

```
EnormalDependence "100" {...}
EnormalDependence "110" {...}
EnormalDependence "111" {...}
```

These parameter sets represent `Lombardi` model parameters for surface orientations of {100}, {110}, and {111}. They can be selected for use in a simulation by specifying the `ParameterSetName` option for `Lombardi` (see [Named Parameter Sets for Lombardi Model on page 319](#)).

NOTE The default parameters for the "100", "110", and "111" named parameter sets are currently the same as those for the unnamed `EnormalDependence` parameter set.

Alternatively, `AutoOrientation` can be specified as an option for `Lombardi` to automatically switch between these parameter sets based on the orientation of the nearest interface (see [Auto-Orientation Framework on page 75](#)):

```
Physics { Mobility( Enormal( Lombardi(AutoOrientation) ) ...) ...}
```

Enhanced Lombardi Model with High-k Degradation

High-k gate dielectrics are being considered as an alternative to SiO_2 to reduce unacceptable leakage currents as transistor dimensions become smaller. One obstacle when using high-k gate dielectrics is that a degraded carrier mobility is often observed for such devices. Although the

15: Mobility Models

Mobility Degradation at Interfaces

causes of high-k mobility degradation are not completely understood, two possible contributors are remote Coulomb scattering (RCS) and remote phonon scattering (RPS).

Sentaurus Device supports a version of the Lombardi model (see [Enhanced Lombardi Model on page 316](#)) that includes empirical degradation terms accounting for RCS and RPS. These are combined with the Lombardi model using Matthiessen's rule:

$$\frac{1}{\mu} = \alpha_{\text{lom}} \cdot \frac{1}{\mu_{\text{Lombardi}}} + \alpha_{\text{rcs}} \cdot \frac{D_{\text{rcs}} D_{\text{rcs_highk}}}{\Delta\mu_{\text{rcs}}} + \alpha_{\text{rps}} \cdot \frac{D_{\text{rps}} D_{\text{rps_highk}}}{\Delta\mu_{\text{rps}}} \quad (253)$$

The $\Delta\mu_{\text{rcs}}$ term is taken from [7] and accounts for the mobility degradation observed with HfSiON MISFETs. This term is mostly attributed to RCS.

$$\Delta\mu_{\text{rcs}} = \mu_{\text{rcs}} \left(\frac{N_{\text{A,D}}}{3 \times 10^{16} \text{ cm}^{-3}} \right)^{\gamma_1} \left(\frac{T}{300 \text{ K}} \right)^{\gamma_2} (g_{\text{screening}})^{\gamma_3 + \gamma_4 \cdot \ln \left(\frac{N_{\text{A,D}}}{3 \times 10^{16} \text{ cm}^{-3}} \right)} / f(F_{\perp}) \quad (254)$$

In Eq. 254, $g_{\text{screening}}$ is a screening factor that has one of two possible forms depending on the value of the ScreeningType parameter:

$$g_{\text{screening}} = \begin{cases} \frac{N_s}{10^{11} \text{ cm}^{-2}}, & \text{ScreeningType}=0 \text{ (default)} \\ s + \frac{c}{c_{\text{trans}} \left(\frac{N_{\text{A,D}}}{3 \times 10^{16} \text{ cm}^{-3}} \right)^{\gamma_5}}, & \text{ScreeningType}=1 \end{cases} \quad (255)$$

In the first form (ScreeningType=0), N_s is the inversion carrier density [cm^{-2}] and is approximated with a local function of doping, temperature, and normal electric field. In the second form (ScreeningType=1), c is the carrier concentration (n for electron mobility and p for hole mobility), and s , c_{trans} , and γ_5 are parameters. Default model parameters are adjusted automatically based on ScreeningType, so that both forms result in similar degradation behavior.

In Eq. 254, $f(F_{\perp})$ is a function that confines the degradation to areas of the structure where $F_{\perp}$ is large enough to initiate inversion:

$$f(F_{\perp}) = 1 - \exp(-\xi F_{\perp} / N_{\text{depl}}) \quad (256)$$

In the above expression, N_{depl} is the depletion charge density [cm^{-2}] and is approximated with a local function of doping and temperature.

The $\Delta\mu_{\text{rps}}$ term is extracted from figures in [8] and accounts for a portion of the mobility degradation observed with HfO₂-gated MOSFETs. This term is mostly attributed to RPS:

$$\Delta\mu_{\text{rps}} = \mu_{\text{rps}} \left(\frac{F_{\perp}}{10^6 \text{ V/cm}} \right)^{\gamma_6} \left(\frac{T}{300 \text{ K}} \right)^{\gamma_7} \quad (257)$$

The α_{rcs} and α_{rps} parameters in Eq. 253 can be used to emphasize or de-emphasize the degradation components. A value of 1 for these parameters includes the full degradation term. A value of zero disables the degradation term. Specifying $\alpha_{\text{lom}} = 0$ (the default is 1) can be used to disable the Lombardi portion of this model. This makes it possible to combine the RCS and RPS degradation components with mobility models other than the built-in Lombardi model if required.

The degradation terms include two different types of damping factor. In Eq. 253, D_{rcs} and D_{rps} are damping factors that reduce the degradation components based on the distance measured from the semiconductor–insulator interface. The $D_{\text{rcs_highk}}$ and $D_{\text{rps_highk}}$ factors reduce the degradation components based on the distance measured from any high-k insulator found in the structure:

$$D_{\text{rcs}} = \exp(-(dist + d_{\text{crit,rcs}})/l_{\text{crit,rcs}}) \quad (258)$$

$$D_{\text{rps}} = \exp(-(dist + d_{\text{crit,rps}})/l_{\text{crit,rps}}) \quad (259)$$

$$D_{\text{rcs_highk}} = \exp(-dist_{\text{highk}}/l_{\text{crit,rcs_highk}}) \quad (260)$$

$$D_{\text{rps_highk}} = \exp(-dist_{\text{highk}}/l_{\text{crit,rps_highk}}) \quad (261)$$

In these expressions, $dist$ is the distance from the semiconductor–insulator interface and $dist_{\text{highk}}$ is the distance from any high-k insulator found in the structure. If no high-k insulator is found in the structure, the damping factors $D_{\text{rcs_highk}}$ and $D_{\text{rps_highk}}$ are ignored.

The enhanced Lombardi model with high-k degradation can be selected by specifying the parameter `Lombardi_highk` in the command file, for example:

```
Physics { Mobility( Enormal(Lombardi_highk) ...) ... }
```

The `ScreeningType` parameter can be specified either as an argument to `Lombardi_highk` in the command file or in the `Lombardi_highk` parameter set in the parameter file.

15: Mobility Models

Mobility Degradation at Interfaces

Model parameters associated with the model are accessible in the Lombardi_highk parameter set. Their values for silicon are shown in [Table 50](#) to [Table 52 on page 323](#).

Table 50 Lombardi parameters for Lombardi_highk model: Default coefficients for silicon

Symbol	Electron parameter name	Electron value	Hole parameter name	Hole value	Unit
α_{lom}	alpha_lom_e	1.0	alpha_lom_h	1.0	1
B	B_e	4.75×10^7	B_h	9.925×10^6	cm/s
C	C_e	5.80×10^2	C_h	2.947×10^3	$\text{cm}^{5/3}\text{V}^{-2/3}\text{s}^{-1}$
λ	lambda_e	0.1250	lambda_h	0.0317	1
k	k_e	1	k_h	1	1
δ	delta_e	5.82×10^{14}	delta_h	2.0546×10^{14}	cm^2/Vs
A	A_e	2	A_h	2	1
$\alpha_{\perp}$	alpha_e	0	alpha_h	0	cm^3
ν	nu_e	1	nu_h	1	1
η	eta_e	5.82×10^{30}	eta_h	2.0546×10^{30}	$\text{V}^2\text{cm}^{-1}\text{s}^{-1}$
l_{crit}	l_crit_e	1×10^{-6}	l_crit_h	1×10^{-6}	cm

Table 51 RCS parameters for Lombardi_highk model: Default coefficients for silicon

Symbol	Electron parameter name	Electron value	Hole parameter name	Hole value	Unit
α_{rcs}	alpha_rcs_e	1.0	alpha_rcs_h	1.0	1
μ_{rcs} ScreeningType=0 ScreeningType=1	mu_rcs_e	240.0 149.0	mu_rcs_h	240.0 149.0	cm^2/Vs
γ_1 ScreeningType=0 ScreeningType=1	N1_exp_e	-0.30 -0.23187	N1_exp_h	-0.30 -0.23187	1
γ_2	T1_exp_e	2.1	T1_exp_h	2.1	1
γ_3	Ns1_exp_e	0.40	Ns1_exp_h	0.40	1
γ_4 ScreeningType=0 ScreeningType=1	Ns2_exp_e	0.035 0.05	Ns2_exp_h	0.035 0.05	1
γ_5	N2_exp_e	1.0	N2_exp_h	1.0	1
s	s_e	0.1	s_h	0.1	1

Table 51 RCS parameters for Lombardi_highk model: Default coefficients for silicon

Symbol	Electron parameter name	Electron value	Hole parameter name	Hole value	Unit
c_{trans}	c_trans_e	3.0×10^{16}	c_trans_h	3.0×10^{16}	cm^{-3}
$d_{crit,rCS}$	d_crit_rcs_e	0.0	d_crit_rcs_h	0.0	cm
$l_{crit,rCS}$	l_crit_rcs_e	1×10^{-6}	l_crit_rcs_h	1×10^{-6}	cm
l_{crit,rCS_highk}	l_crit_rcs_highk_e	1×10^6	l_crit_rcs_highk_h	1×10^6	cm
ξ	xi_e	1.3042×10^7	xi_h	1.3042×10^7	$\text{V}^{-1}\text{cm}^{-1}$

Table 52 RPS parameters for Lombardi_highk model: Default coefficients for silicon

Symbol	Electron parameter name	Electron value	Hole parameter name	Hole value	Unit
α_{rps}	alpha_rps_e	0.0	alpha_rps_h	0.0	1
μ_{rps}	mu_rps_e	496.7	mu_rps_h	496.7	cm^2/Vs
γ_6	En_exp_e	-0.68	En_exp_h	-0.68	1
γ_7	T2_exp_e	-0.34	T2_exp_h	-0.34	1
$d_{crit,rps}$	d_crit_rps_e	0.0	d_crit_rps_h	0.0	cm
$l_{crit,rps}$	l_crit_rps_e	1×10^{-6}	l_crit_rps_h	1×10^{-6}	cm
l_{crit,rps_highk}	l_crit_rps_highk_e	1×10^6	l_crit_rps_highk_h	1×10^6	cm

NOTE The Lombardi_highk model has its own set of Lombardi parameters (see [Table 50 on page 322](#)). The default values of these parameters are the same as the standard Lombardi model (see [Table 48 on page 317](#)). In addition, the default hole parameter values for the RCS and RPS components of the model are the same as the default electron parameter values. Some calibration of these parameters is necessary to obtain accurate results for a specific technology.

Inversion and Accumulation Layer Mobility Model

The inversion and accumulation layer mobility model [9] is similar to the Lucent (Darwish) model [6] but contains additional terms that account for ‘two-dimensional’ Coulomb impurity scattering in the inversion and accumulation regions of a MOSFET. Although this model is based on modified versions of the [Philips Unified Mobility Model on page 310](#) and the [Enhanced Lombardi Model on page 316](#), it is completely self-contained and all parameters associated with this model are specified independently of those models. A complete description is given here.

15: Mobility Models

Mobility Degradation at Interfaces

The expressions that follow apply to both electron mobility and hole mobility when the following substitutions are made:

$$\begin{aligned} \text{Electron Mobility: } c = n, c_{\text{other}} = p, N_{\text{inv}} = N_A, N_{\text{acc}} = N_D, P = P_e, m^* = m_e^*, m_{\text{other}}^* = m_h^* \\ \text{Hole Mobility: } c = p, c_{\text{other}} = n, N_{\text{inv}} = N_D, N_{\text{acc}} = N_A, P = P_h, m^* = m_h^*, m_{\text{other}}^* = m_e^* \end{aligned} \quad (262)$$

The doping-dependent and transverse field-dependent portion of the inversion and accumulation layer mobility model has contributions from phonon scattering, surface roughness scattering, and Coulomb impurity scattering:

$$\frac{1}{\mu} = \frac{1}{\mu_{\text{ph}}} + \frac{D}{\mu_{\text{sr}}} + \frac{1}{\mu_C} \quad (263)$$

The phonon-scattering portion of Eq. 263 has 2D and 3D parts. The manner in which these parts are combined depends on the value of the IALMob command file parameter PhononCombination:

$$\mu_{\text{ph}} = \begin{cases} \min\left\{\frac{\mu_{\text{ph},2\text{D}}}{D}, \mu_{\text{ph},3\text{D}}\right\}, & \text{PhononCombination}=0 \\ \left[\frac{D}{\mu_{\text{ph},2\text{D}}} + \frac{1}{\mu_{\text{ph},3\text{D}}}\right]^{-1}, & \text{PhononCombination}=1 \text{ (default)} \\ \left[\frac{D}{\mu_{\text{ph},2\text{D}}} + \frac{f(F_{\perp}, T)}{\mu_{\text{ph},3\text{D}}}\right]^{-1}, & \text{PhononCombination}=2 \end{cases} \quad (264)$$

where:

$$\mu_{\text{ph},2\text{D}} = \frac{B}{F_{\perp}} + \frac{C((N_A + N_D)/1 \text{ cm}^{-3})^{\lambda}}{F_{\perp}^{1/3} (T/300 \text{ K})^k} \quad (265)$$

$$\mu_{\text{ph},3\text{D}} = \mu_{\text{max}} \left(\frac{T}{300 \text{ K}} \right)^{-\theta} \quad (266)$$

In the previous expressions, $D = \exp(-x/l_{\text{crit}})$ (where x is the distance from the interface) and $f(F_{\perp}, T)$ is a field-dependent Fermi function given by:

$$f(F_{\perp}, T) = \frac{1}{1 + \exp\left((0.3042 \text{ cm}^{2/3} \text{ K/V}^{2/3}) \frac{F_{\perp}^{2/3}}{T} - 4\right)} \quad (267)$$

The surface roughness scattering is given by:

$$\mu_{sr} = \frac{((N_A + N_D)/1 \text{ cm}^{-3})^{\lambda_{sr}}}{\left(\frac{(F_{\perp}/1 \text{ V/cm})^{A^*}}{\delta} + \frac{F_{\perp}^3}{\eta} \right)} \quad (268)$$

where:

$$A^* = A + \frac{\alpha_{\perp}(n+p)}{(1 + (N_A + N_D)/1 \text{ cm}^{-3})^v} \quad (269)$$

The Coulomb impurity scattering term has 2D and 3D parts that are combined using the same field-dependent Fermi function that is given in [Eq. 267](#):

$$\mu_C = f(F_{\perp}, T)\mu_{C,3D} + (1 - f(F_{\perp}, T))\mu_{C,2D} \quad (270)$$

The 2D Coulomb scattering has both inversion and accumulation layer contributions:

$$\frac{1}{\mu_{C,2D}} = \frac{1}{\mu_{C,2D,inv}} + \frac{1}{\mu_{C,2D,acc}} \quad (271)$$

where:

$$\mu_{C,2D,inv} = \max \left\{ \frac{D_1 \cdot (c/10^{18} \text{ cm}^{-3})^{v_0}}{(N_{inv}/10^{18} \text{ cm}^{-3})^{v_1}}, \frac{D_2}{(N_{inv}/10^{18} \text{ cm}^{-3})^{v_2}} \right\} \quad (272)$$

$$\mu_{C,2D,acc} = \max \left\{ \frac{D_1 \cdot (c/10^{18} \text{ cm}^{-3})^{v_0}}{(N_{acc}/10^{18} \text{ cm}^{-3})^{v_1}}, \frac{D_2}{(N_{acc}/10^{18} \text{ cm}^{-3})^{v_2}} \right\} G(P) \quad (273)$$

The 3D Coulomb scattering part of [Eq. 270](#) is taken from the Philips unified mobility model (PhuMob) and is given by:

$$\mu_{C,3D} = \mu_N \left(\frac{N_{sc}}{N_{sc,eff}} \right) \left(\frac{N_{ref}}{N_{sc}} \right)^{\alpha} + \mu_c \left(\frac{c + c_{other}}{N_{sc,eff}} \right) \quad (274)$$

where:

$$\mu_N = \frac{\mu_{max}^2}{\mu_{max} - \mu_{min}} \left(\frac{T}{300K} \right)^{3\alpha - 1.5} \quad (275)$$

15: Mobility Models

Mobility Degradation at Interfaces

$$\mu_c = \frac{\mu_{\max}\mu_{\min}}{\mu_{\max} - \mu_{\min}} \left(\frac{300\text{K}}{T} \right)^{1/2} \quad (276)$$

$$N_{\text{sc}} = N_{\text{D}}^* + N_{\text{A}}^* + c_{\text{other}} \quad (277)$$

$$N_{\text{sc,eff}} = N_{\text{acc}}^* + G(P)N_{\text{inv}}^* + \frac{c_{\text{other}}}{F(P)} \quad (278)$$

In the previous expressions, quantities marked with an asterisk (*) indicate that the clustering formulas from the Philips unified mobility model are invoked:

$$N_{\text{D}}^* = N_{\text{D},0} \left[1 + \frac{N_{\text{D},0}^2}{c_{\text{D}}N_{\text{D},0}^2 + N_{\text{D,ref}}^2} \right] \quad (279)$$

$$N_{\text{A}}^* = N_{\text{A},0} \left[1 + \frac{N_{\text{A},0}^2}{c_{\text{A}}N_{\text{A},0}^2 + N_{\text{A,ref}}^2} \right] \quad (280)$$

As an option, the clustering formulas can be used for all occurrences of N_{A} and N_{D} in the model by specifying the IALMob command file parameter `ClusteringEverywhere=1`.

The functions $F(P)$ and $G(P)$ describe electron-hole and minority impurity scattering, respectively, and P is a screening parameter:

$$F(P) = \frac{0.7643P^{0.6478} + 2.2999 + 6.5502(m^*/m_{\text{other}}^*)}{P^{0.6478} + 2.3670 - 0.8552(m^*/m_{\text{other}}^*)} \quad (281)$$

$$G(P) = 1 - \frac{0.89233}{\left[0.41372 + P \left(\frac{m_0}{m^*} \frac{T}{300\text{K}} \right)^{0.28227} \right]^{0.19778}} + \frac{0.005978}{\left[P \left(\frac{m^*}{m_0} \frac{300\text{K}}{T} \right)^{0.72169} \right]^{1.80618}} \quad (282)$$

$$P = \left[\frac{2.459}{3.97 \times 10^{13} \text{ cm}^{-2} N_{\text{sc}}^{-2/3}} + \frac{3.828(c + c_{\text{other}})(m_0/m^*)}{1.36 \times 10^{20} \text{ cm}^{-3}} \right]^{-1} \left(\frac{T}{300\text{K}} \right)^2 \quad (283)$$

By default, the full PhuMob model described by the previous expressions is used with IALMob (`FullPhuMob=1`). However, the ‘other’ carrier in these expressions can be ignored ($c_{\text{other}} = 0$) by specifying `FullPhuMob=0` in the command file. This specification also ignores the standard PhuMob modification of the $G(P)$ function, which invokes $G = G(P_{\text{min}})$ for $P < P_{\text{min}}$, where P_{min} is the value where $G(P)$ reaches its minimum.

Parameters associated with the model are accessible in the IALMob parameter set. Their values for silicon are shown in [Table 53](#) to [Table 55](#).

Table 53 IALMob parameters for clustering: Default coefficients for silicon

Symbol	Donor parameter name	Donor value	Acceptor parameter name	Acceptor value	Unit
$N_{\{D,A\},ref}$	nref_D	4×10^{20}	nref_A	7.2×10^{20}	cm^{-3}
$c_{\{D,A\}}$	cref_D	0.21	cref_A	0.5	1

Table 54 IALMob parameters for phonon and surface roughness scattering: Default coefficients for silicon

Symbol	Electron parameter name	Electron value	Hole parameter name	Hole value	Unit
B	B_e	9.0×10^5	B_h	9.0×10^5	cm/s
C	C_e	4400.0	C_h	4400.0	$\text{cm}^{5/3}\text{V}^{-2/3}\text{s}^{-1}$
λ	lambda_e	0.057	lambda_h	0.057	1
k	k_e	1	k_h	1	1
δ	delta_e	3.97×10^{13}	delta_h	3.97×10^{13}	cm^2/Vs
η	eta_e	1.0×10^{50}	eta_h	1.0×10^{50}	$\text{V}^2\text{cm}^{-1}\text{s}^{-1}$
λ_{sr}	lambda_sr_e	0.057	lambda_sr_h	0.057	1
A	A_e	2	A_h	2	1
$\alpha_{\perp}$	alpha_sr_e	0	alpha_sr_h	0	cm^3
ν	nu_e	0	nu_h	0	1
l_{crit}	l_crit_e	10^3	l_crit_h	10^3	cm

Table 55 IALMob parameters for lattice and Coulomb scattering: Default coefficients for silicon

Symbol	Electron parameter name	Electron value	Hole parameter name	Hole value	Unit
μ_{max}	mumax_e	1417.0	mumax_h	470.5	cm^2/Vs
μ_{min}	mumin_e	52.2	mumin_h	44.9	cm^2/Vs
θ	theta_e	2.285	theta_h	2.247	1
N_{ref}	n_ref_e	9.68×10^{16}	n_ref_h	2.23×10^{17}	cm^{-3}
α	alpha_e	0.68	alpha_h	0.719	1

15: Mobility Models

Mobility Degradation at Interfaces

Table 55 IALMob parameters for lattice and Coulomb scattering: Default coefficients for silicon

Symbol	Electron parameter name	Electron value	Hole parameter name	Hole value	Unit
m^*/m_0 FullPhuMob=1 FullPhuMob=0	me_over_m0	1.0 0.26	mh_over_m0	1.258 0.26	1
D_1	D1_e	135.0	D1_h	135.0	cm ² /Vs
D_2	D2_e	40.0	D2_h	40.0	cm ² /Vs
v_0	nu0_e	1.5	nu0_h	1.5	1
v_1	nu1_e	2.0	nu1_h	2.0	1
v_2	nu2_e	0.5	nu2_h	0.5	1

Using Inversion and Accumulation Layer Mobility Model

The doping-dependent and transverse field-dependent portion of the inversion and accumulation layer mobility model is selected by specifying the IALMob keyword as an argument to Enormal in the command file. No mobility DopingDependence should be specified.

To prevent the automatic inclusion of ConstantMobility in this case, specify the -ConstantMobility option for Mobility. The complete model is obtained by combining IALMob with the Hänsch model [10] for high-field saturation (see [Extended Canali Model on page 343](#)). For example:

```
Physics {
  Mobility (
    -ConstantMobility
    Enormal(IALMob)
    HighFieldSaturation(EparallelToInterface)
  )
}
```

In addition, parameter file specifications are required to use this model as described in [9]:

- To select the Hänsch model, specify the parameter $\alpha = 1$ in the HighFieldDependence section. In addition, the β exponent in the Hänsch model is equal to 2, which can be specified by setting $\beta_0 = 2$ and $\beta_{\text{exp}} = 0$.

The required parameter file specifications are summarized here:

```
HighFieldDependence {
    alpha    = 1.0, 1.0    # [1]
    beta0    = 2.0, 2.0    # [1]
    betaexp  = 0.0, 0.0
}
```

Mole Fraction–dependent IALMob Parameters

Any of the parameters in [Table 53 on page 327](#) to [Table 55 on page 327](#) can be assigned x-mole-dependent values. Mole fraction intervals are assigned by specifying values for the Xmax parameter in an IALMob parameter set. The specified values represent the endpoints for x-mole-fraction ranges between which mole fraction–dependent parameters vary linearly. Each parameter that is mole fraction dependent should be specified with values corresponding to the x-mole values given by Xmax.

For example, this specification:

```
IALMob {
    Xmax      = ( 0.0, 0.2, 0.4, 0.7)
    mumax_e  = (1417.0, 1600.0, 1900.0, 3000.0)
    mumax_h  = 470.5
}
```

defines mole fraction–dependent values for the parameter mumax_e at x-mole values of 0.0, 0.2, 0.4, and 0.7. Between these x-mole values, mumax_e varies linearly. Since the parameter mumax_h is given as a single value in this example, it is not mole fraction dependent.

Named Parameter Sets for IALMob

The IALMob parameter set can be named. For example, in the parameter file, you can write:

```
IALMob "myset" { ... }
```

to declare a parameter set with the name myset. To use a named parameter set, specify its name with ParameterSetName as an option to IALMob in the command file, for example:

```
Mobility (
    Enormal( IALMob( ParameterSetName = "myset" ... ) ... )
)
```

By default, the unnamed parameter set is used.

Auto-Orientation for IALMob

The IALMob model supports the auto-orientation framework (see [Auto-Orientation Framework on page 75](#)) that switches between different named parameter sets based on the orientation of the nearest interface. This can be activated by specifying the IALMob command file parameter `AutoOrientation=1`:

```
Mobility (
    Enormal( IALMob( AutoOrientation=1 ...) ...)
)
```

University of Bologna Surface Mobility Model

The University of Bologna surface mobility model was developed for an extended temperature range between 25°C and 648°C. It should be used together with the University of Bologna bulk mobility model (see [University of Bologna Bulk Mobility Model on page 305](#)). The inversion layer mobility in MOSFETs is degraded by Coulomb scattering at low normal fields and by surface phonons and surface roughness scattering at large normal fields.

In the University of Bologna model [4], all these effects are combined by using Matthiessen's rule:

$$\frac{1}{\mu} = \frac{1}{\mu_{\text{bsc}}} + \frac{D}{\mu_{\text{ac}}} + \frac{D}{\mu_{\text{sr}}} \quad (284)$$

where $1/\mu_{\text{bsc}}$ is the contribution of Coulomb scattering, and $1/\mu_{\text{ac}}$, $1/\mu_{\text{sr}}$ are those of surface phonons and surface roughness scattering, respectively. $D = \exp(-x/l_{\text{crit}})$ (where x is the distance from the interface and l_{crit} a fit parameter) is a damping that switches off the inversion layer terms far away from the interface.

The term μ_{bsc} is associated with substrate impurity and carrier concentration. It is decomposed in an unscreened part (due to the impurities) and a screened part (due to local excess carrier concentration):

$$\mu_{\text{bsc}}^{-1} = \mu_{\text{b}}^{-1} [D(1 + f_{\text{sc}}^{\tau})^{-1/\tau} + (1 - D)] \quad (285)$$

where μ_{b} is given by the bulk mobility model, and τ is a fit parameter. The screening function is given by:

$$f_{\text{sc}} = \left(\frac{N_1}{N_{\text{A},0} + N_{\text{D},0}} \right)^{\eta} \frac{N_{\text{min}}}{N_{\text{A},0} + N_{\text{D},0}} \quad (286)$$

where N_{min} is the minority carrier concentration.

If surface mobility is plotted against the effective normal field, mobility data converges toward a universal curve. Deviations from this curve appear at the onset of weak inversion, and the threshold field changes with the impurity concentration at the semiconductor surface [11]. The term μ_{bsc} models these deviations, in that, it is the roll-off in the effective mobility characteristics. As the effective field increases, the mobilities become independent of the channel doping and approach the universal curve.

The main scattering mechanisms are, in this case, surface phonons and surface roughness scattering, which are expressed by:

$$\mu_{\text{ac}} = C(T) \left(\frac{N_{\text{A},0} + N_{\text{D},0}}{N_2} \right)^a \frac{1}{F_{\perp}^{\delta}} \quad (287)$$

$$\mu_{\text{sr}} = B(T) \left(\frac{N_{\text{A},0} + N_{\text{D},0} + N_3}{N_4} \right)^b \frac{1}{F_{\perp}^{\lambda}} \quad (288)$$

$F_{\perp}$ is the electric field normal to semiconductor–insulator interface, see [Computing Transverse Field on page 335](#).

The parameters for the model are accessible in the parameter set UniBoEnormalDependence. [Table 56](#) lists the silicon default parameters. The reported parameters are detailed in the literature [12]. The model was calibrated with experiments [11][12] in the temperature range from 300 K to 700 K.

Table 56 Parameters of University of Bologna surface mobility model ($T_n = T/300\text{ K}$)

Symbol	Parameter name	Electrons	Holes	Unit
N_1	N1	2.34×10^{16}	2.02×10^{16}	cm^{-3}
N_2	N2	4.0×10^{15}	7.8×10^{15}	cm^{-3}
N_3	N3	1.0×10^{17}	2.0×10^{15}	cm^{-3}
N_4	N4	2.4×10^{18}	6.6×10^{17}	cm^{-3}
B	B	$5.8 \times 10^{18} T_n^{\gamma_B}$	$7.82 \times 10^{15} T_n^{\gamma_B}$	cm^2/Vs
γ_B	B_exp	0	1.4	1
C	C	$1.86 \times 10^4 T_n^{-\gamma_C}$	$5.726 \times 10^3 T_n^{-\gamma_C}$	cm^2/Vs
γ_C	C_exp	2.1	1.3	1
τ	tau	1	3	1
η	eta	0.3	0.5	1
a	ac_exp	0.026	-0.02	1

15: Mobility Models

Mobility Degradation at Interfaces

Table 56 Parameters of University of Bologna surface mobility model ($T_n = T/300K$)

Symbol	Parameter name	Electrons	Holes	Unit
b	sr_exp	0.11	0.08	1
l_{crit}	l_crit	1×10^{-6}	1×10^{-6}	cm
δ	delta	0.29	0.3	1
λ	lambda	2.64	2.24	1

Mobility Degradation Components due to Coulomb Scattering

Three mobility degradation components due to Coulomb scattering are available in Sentaurus Device that can be combined with other interface mobility degradation models:

- **NegInterfaceCharge:** Accounts for mobility degradation due to a negative interface charge (from charged traps and fixed charge).
- **PosInterfaceCharge:** Accounts for mobility degradation due to a positive interface charge (from charged traps and fixed charge).
- **Coulomb2D:** Accounts for mobility degradation due to ionized impurities near the interface.

These degradation components can be specified separately or in combination with each other. If specified, they will be combined using Matthiessen's rule:

$$\frac{1}{\mu} = \frac{1}{\mu_{other}} + \frac{1}{\mu_{nic}} + \frac{1}{\mu_{pic}} + \frac{1}{\mu_{C2D}} \quad (289)$$

where:

- μ_{other} represents the other DopingDependence and Enormal models specified for the simulation.
- μ_{nic} represents the NegInterfaceCharge mobility degradation component.
- μ_{pic} represents the PosInterfaceCharge mobility degradation component.
- μ_{C2D} represents the Coulomb2D mobility degradation component.

The general form of the mobility degradation components due to Coulomb scattering is given by:

$$\mu_{C_{n,p}} = \frac{\mu_1 \left(\frac{T}{300K} \right)^k \left\{ 1 + \left[c / \left(c_{trans} \left(\frac{N_{A,D}}{10^{18} \text{ cm}^{-3}} \right)^{\gamma_1} \left(\frac{N_C}{N_0} \right)^{\eta_1} \right) \right]^v \right\}}{\left(\frac{N_{A,D}}{10^{18} \text{ cm}^{-3}} \right)^{\gamma_2} \left(\frac{N_C}{N_0} \right)^{\eta_2} \cdot D \cdot f(F_{\perp})} \quad (290)$$

where:

- $N_C = \begin{cases} \text{negative interface charge density, for the NegInterfaceCharge component} \\ \text{positive interface charge density, for the PosInterfaceCharge component} \\ N_{A,D} \text{ (for electrons, holes),} & \text{for the Coulomb2D component} \end{cases}$
- $N_0 = \begin{cases} 10^{11} \text{ cm}^{-2}, \text{ for the Neg/PosInterfaceCharge components} \\ 10^{18} \text{ cm}^{-3}, \text{ for the Coulomb2D component} \end{cases}$
- $c = n, p \text{ (for electrons, holes)}$

and:

$$f(F_{\perp}) = 1 - \exp[-(F_{\perp}/E_0)^{\gamma}] \quad (291)$$

$$D = \exp(-x/l_{crit}) \quad (292)$$

In [Eq. 292](#), x is the distance from the interface.

Parameters associated with the components are accessible in the parameter sets NegInterfaceChargeMobility, PosInterfaceChargeMobility, and Coulomb2DMobility. [Table 57](#) lists their default values.

Table 57 Parameters for mobility degradation components due to Coulomb scattering

Symbol	Parameter name	μ_{nic} Electron	μ_{nic} Hole	μ_{pic} Electron	μ_{pic} Hole	μ_{C2D} Electron	μ_{C2D} Hole	Unit
μ_1	mu1	40.0	40.0	40.0	40.0	40.0	40.0	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
k	T_exp	1.0	1.0	1.0	1.0	1.0	1.0	1
c_{trans}	c_trans	10^{18}	10^{18}	10^{18}	10^{18}	10^{18}	10^{18}	cm^{-3}
v	c_exp	1.5	1.5	1.5	1.5	1.5	1.5	1
η_1	Nc_exp1	1.0	1.0	1.0	1.0	1.0	1.0	1

15: Mobility Models

Mobility Degradation at Interfaces

Table 57 Parameters for mobility degradation components due to Coulomb scattering

Symbol	Parameter name	μ_{nic} Electron	μ_{nic} Hole	μ_{pic} Electron	μ_{pic} Hole	μ_{C2D} Electron	μ_{C2D} Hole	Unit
η_2	Nc_exp2	0.5	0.5	0.5	0.5	0.5	0.5	1
γ_1	N_exp1	0.0	0.0	0.0	0.0	0.0	0.0	1
γ_2	N_exp2	0.0	0.0	0.0	0.0	0.0	0.0	1
l_{crit}	l_crit	10^{-6}	10^{-6}	10^{-6}	10^{-6}	10^{-6}	10^{-6}	cm
E_0	E0	2.0×10^5	2.0×10^5	2.0×10^5	2.0×10^5	2.0×10^5	2.0×10^5	V/cm
γ	En_exp	2.0	2.0	2.0	2.0	2.0	2.0	1

Using Mobility Degradation Components

The Coulomb2D model is a local model that uses local values of N_A and N_D . To use this model, specify Coulomb2D in addition to the standard Enormal model, for example:

```
Physics {
  Mobility (
    PhuMob HighFieldSaturation
    Enormal (Lombardi Coulomb2D)
  )
}
```

The NegInterfaceCharge and PosInterfaceCharge models are nonlocal models because N_C used in these models is a charge density at a location that may be different from the point where mobility is being calculated. If Math{GeometricDistances} is specified, N_C is the charge density at the point on the interface that is the closest distance to the point where mobility is being calculated. If there is not a vertex at this interface point, N_C is interpolated from the charge density at the surrounding vertices.

If Math{GeometricDistances} is not specified, N_C is the charge density at the vertex on the interface that is the closest distance to the point where mobility is being calculated.

To use these models, specify NegInterfaceCharge, or PosInterfaceCharge, or both in addition to a standard Enormal model. For convenience, both models can be specified with the single keyword InterfaceCharge, for example:

```
Physics {
  Mobility (
    PhuMob HighFieldSaturation
    Enormal (Lombardi InterfaceCharge)
  )
}
```

NOTE When using mobility degradation components, a standard `Enormal` model (such as `Lombardi`) must be specified explicitly in addition to the mobility degradation component. Sentaurus Device will not include the `Lombardi` model by default (this only occurs when `Enormal` is specified with no arguments).

By default, the interface used with the `NegInterfaceCharge` and `PosInterfaceCharge` models is any semiconductor–insulator interface. However, these models also can be used with one or more surfaces that are defined in the global `Math` section. Each surface has a name and is the union of an arbitrary number of interfaces.

For example:

```
Math {
  Surface "S1" (
    MaterialInterface="HfO2/Oxide"
    RegionInterface="region_1/region_2v"
  )
}
```

specifies a surface named `S1` that consists of all HfO_2 –oxide interfaces, as well as the `region_1/region_2` interface. Then, the surfaces can be specified as arguments to the interface charge models using the keyword `SurfaceName`:

```
Physics {
  Mobility (
    PhuMob HighFieldSaturation
    Enormal (Lombardi
      InterfaceCharge(SurfaceName="S1")
      InterfaceCharge(SurfaceName="S2")
      ...
    )
  )
}
```

Computing Transverse Field

Sentaurus Device supports two different methods for computing the normal electric field $F_{\perp}$:

- Using the normal to the interface.
- Using the normal to the current.

Furthermore, for vertices at the interface, an optional correction F_{corr} for the field value is available.

Normal to Interface

Assume that mobility degradation occurs at an interface Γ . By default, for a given point $\vec{r}$, Sentaurus Device locates the nearest vertex $\vec{r}_i$ on the interface Γ , approximates the distance of $\vec{r}$ to the interface by the distance to $\vec{r}_i$, and determines the direction $\hat{n}$ normal to the interface at vertex $\vec{r}_i$. When the `GeometricDistances` option is specified in the `Math` section, the distance to the interface is the true geometric distance, and $\hat{n}$ is the gradient of the distance to the interface. From $\hat{n}$, the normal electric field is:

$$F_{\perp}(\vec{r}) = \left| \vec{F}(\vec{r}) \cdot \hat{n} + F_{\text{corr}} \right| \quad (293)$$

To activate the Lombardi model with this method of computing $F_{\perp}$, specify the `Enormal` flag to `Mobility`. The keyword `ToInterfaceEnormal` is synonymous with `Enormal`.

By default, the interface Γ is a semiconductor–insulator interface. Sometimes, it is important to change this default interface definition, for example, when the insulator (oxide) is considered a wide-bandgap semiconductor. In this case, Sentaurus Device allows this interface to be specified with `EnormalInterface` in the `Math` section.

In the following example, Sentaurus Device takes as Γ the union of interfaces between materials, `OxideAsSemiconductor` and `Silicon`, and regions, `regionK1` and `regionL1`:

```
Math {
    ...
    EnormalInterface(regioninterface=["regionK1/regionL1"],
        materialinterface=["OxideAsSemiconductor/Silicon"])
    ...
}
```

Normal to Current Flow

Using this method, $F_{\perp}(\vec{r})$ is defined as the component of the electric field normal to the electron ($c = n$) or hole ($c = p$) currents $J_c(\vec{r})$:

$$F_{c,\perp}(\vec{r}) = F(\vec{r}) \sqrt{1 - \left(\frac{\vec{F}(\vec{r}) \cdot \vec{J}_c(\vec{r})}{F(\vec{r}) J_c(\vec{r})} \right)^2} + F_{\text{corr}} \quad (294)$$

where $F_{n,\perp}$ is used for the evaluation of electron mobility and $F_{p,\perp}$ is used for the evaluation of hole mobility. Through corrections of the current, $F_{n,\perp}$ and $F_{p,\perp}$ also are affected by the keyword `ParallelToInterfaceInBoundaryLayer` (see [Field Correction Close to Interfaces on page 349](#)).

NOTE For very low current levels, the computation of the electric field component normal to the currents may be numerically problematic and lead to convergence problems. It is recommended to use the option `Enormal`. Besides possible numeric problems, both approaches give the same or very similar results.

Field Correction on Interface

For vertices on the interface, due to discretization, the normal electric field in inversion is underestimated systematically due to screening by the charge at the same vertex. Therefore, Sentaurus Device supports a correction of the field:

$$F_{\text{corr}}(\vec{r}) = \frac{\alpha l \rho(\vec{r})}{\epsilon} \quad (295)$$

where α is dimensionless and is specified with `NormalFieldCorrection` in the `Math` section (reasonable values range from 0 to 1; the default is zero), ρ is the space charge, ϵ is the dielectric constant in the semiconductor, and l is an estimate for the depth of the box of the vertex in the normal direction.

Thin-Layer Mobility Model

The thin-layer mobility model applies to devices with silicon layers that are only a few nanometers thick. In such devices, geometric quantization leads to a mobility that cannot be expressed with a normal field-dependent interface model such as those described in [Mobility Degradation at Interfaces on page 315](#), but it depends explicitly on the layer thickness. The thin-layer mobility model described here is based on the Lombardi model (see [Enhanced Lombardi Model on page 316](#)) and the model described in the literature [13].

NOTE It is recommended to use this model together with the [Philips Unified Mobility Model on page 310](#). The Philips unified mobility model must be activated separately from the thin-layer mobility model.

The thin-layer mobility μ_{tl} consists of contributions of bulk phonon-scattering, surface-roughness scattering, thickness fluctuation scattering, and surface phonon scattering:

$$\frac{1}{\mu_{\text{tl}}} = \frac{D}{\mu_{\text{bp}}} + \frac{D}{\mu_{\text{sr}}} + \frac{D}{\mu_{\text{tf}}} + \frac{D}{\mu_{\text{sp}}} \quad (296)$$

where:

- $D = \exp(-x/l_{\text{crit}})$ (see [Enhanced Lombardi Model on page 316](#)).
- μ_{sr} is given by [Eq. 250, p. 316](#).

15: Mobility Models

Thin-Layer Mobility Model

- $\mu_{\text{tf}} = \mu_{\text{tf0}}(t_{\text{b}}/1\text{ nm})^6[1 + F_{\perp}/F_{\text{tf0}}]$.
- $\mu_{\text{sp}} = \mu_{\text{sp0}} \exp(t_{\text{b}}/t_{\text{sp0}})$.
- μ_{bp} is given by:

$$\mu_{\text{bp}} = P_1\mu_{\text{ac},1} + (1 - P_1)\mu_{\text{ac},2} \quad (297)$$

$$P_1 = p_1 + \frac{1 - p_1}{1 + p_2 \exp(-p_3 \Delta E/kT)} \quad (298)$$

$$\Delta E = \frac{\hbar^2 \pi^2}{2t_{\text{b}}^2} \left(\frac{1}{m_{z2}} - \frac{1}{m_{z1}} \right) \quad (299)$$

$$\mu_{\text{ac},v} = \frac{\mu_{\text{ac0},v}}{[1 + (W_{\text{Tv}}/W_{\text{Fv}})^{\beta}]^{1/\beta}} \left(\frac{W_{\text{Tv}}}{1\text{ nm}} \right) \quad (300)$$

$$W_{\text{Tv}} = \frac{2}{3}t_{\text{b}} + W_{\text{T0v}} \left(\frac{t_{\text{b}}}{1\text{ nm}} \right)^4 \left(\frac{F_{\perp}}{1\text{ MV/cm}} \right) \quad (301)$$

$$\frac{W_{\text{Fv}}}{1\text{ nm}} = \zeta^{v-1} \frac{\mu_{\text{ac}}}{P_{\text{bulk}}\mu_{\text{ac0},1} + \zeta(1 - P_{\text{bulk}})\mu_{\text{ac0},2}} \quad (302)$$

where:

- μ_{ac} is given by [Eq. 249](#).
- $P_{\text{bulk}} = p_1 + (1 - p_1)/(1 + p_2)$.
- The thickness t_{b} is the larger of the local layer thickness and t_{min} .
- t_{min} , μ_{tf0} , F_{tf0} , μ_{sp0} , t_{sp0} , $\mu_{\text{ac0},v}$, p_1 , p_2 , p_3 , m_{z1} , m_{z2} , β , W_{T0v} , and ζ are model parameters.

Using the Thin-Layer Mobility Model

To activate the thin-layer mobility model, specify `ThinLayer` as an option to `eMobility`, `hMobility`, or `Mobility`. `ThinLayer` has an optional argument `Thickness` to explicitly specify the layer thickness (in micrometers). If this argument is present, it must agree for electron and hole mobility. If the argument is not present, Sentaurus Device extracts the thickness automatically. For the extraction, it assumes that all regions where `ThinLayer` is specified (regardless of carrier type) belong to the thin layer.

NOTE The external boundary is not considered as a boundary of the thin layer. However, each interface between a region that specifies `ThinLayer` and a region that does not is considered as a boundary of the thin layer, even if these two regions are both semiconductor regions.

For example:

```
Physics(Region="A") { eMobility(ThinLayer) }
Physics(Region="B") { hMobility(ThinLayer(Thickness=0.005)) }
```

activates the thin-layer mobility model for electrons in region A and for holes in region B. The thin layer geometrically consists of regions A and B. Its thickness is extracted in region A and is assumed to be 5 nm in region B.

You can verify the proper thickness extraction using the plot variable `LayerThickness`. Note that the layer thickness is defined in such a way that it becomes zero in concave corners of the layer. In addition, due to interpolation, `LayerThickness` can deviate by approximately half of the local mesh spacing from the thickness actually used for mobility computation. For more information on thickness extraction and the options to control it, see [Thickness Extraction on page 340](#).

The parameters to compute D , μ_{ac} , and μ_{sr} are the same as used for the Lombardi model (see [Table 48 on page 317](#) and [Table 49 on page 318](#)). Synopsys considers the parameters in [Table 49](#) to be more suitable for the thin-layer mobility model; however, the defaults are still given by [Table 48](#). The other parameters are specified as electron-hole pairs in the `ThinLayerMobility` parameter set. [Table 58](#) summarizes the parameters.

Table 58 Parameters for thin-layer mobility model

Symbol	Parameter name	Electron	Hole	Unit
t_{min}	tmin	0.002	0.002	μm
μ_{tf0}	mutf0	0.15	0.28	$cm^2V^{-1}s^{-1}$
F_{tf0}	ftf0	6250	10^{100}	Vcm^{-1}
μ_{sp0}	musp0	1.145×10^{-8}	1.6×10^{-10}	$cm^2V^{-1}s^{-1}$
t_{sp0}	tsp0	10^{-4}	10^{-4}	μm
$\mu_{ac0,1}$	muac01	315	30.2	$cm^2V^{-1}s^{-1}$
$\mu_{ac0,2}$	muac02	6.4	69	$cm^2V^{-1}s^{-1}$
p_1	p1	0.55	0	1
p_2	p2	400	0.66	1
p_3	p3	1.44	1	1
m_{z1}	mz1	0.916	0.29	m_0

Table 58 Parameters for thin-layer mobility model

Symbol	Parameter name	Electron	Hole	Unit
m_{z2}	mz2	0.19	0.25	m_0
β	beta	4	4	1
W_{T01}	wt01	3×10^{-6}	0	μm
W_{T02}	wt02	3.5×10^{-7}	0	μm
ζ	zeta	2.88	1.05	1

Thickness Extraction

The definition of *thickness* of a layer in a general geometry is not self-evident. This section describes how Sentaurus Device defines thickness.

To determine the thickness of a layer at a given point P , Sentaurus Device first finds the closest point C_1 on the surface of the layer. Then, a sphere is constructed that touches the surface in C_1 and has its midpoint within the layer, on the line passing through C_1 and P .

The diameter of this sphere then is increased, until it touches the surface at a second point, C_2 , and the surface normal at this point encloses an angle with the vector pointing from C_1 to P of at least β . By default, $\beta = 0$, so any second touch point C_2 is accepted, and the construction ends up with a sphere that is completely within the layer.

The thickness is a linear combination between the diameter of the final sphere, d , and the distance c_{12} between C_1 and C_2 (the chord length):

$$t = \alpha c_{12} + (1 - \alpha)d \quad (303)$$

The parameter α can be set with the keyword `ChordWeight`, which is an option of `ThinLayer`. The default value is zero.

The value for the angle β can be changed with the keyword `MinAngle`, an option to `ThinLayer`. `MinAngle` expects two values: The first value sets β , and the second value sets an angle γ that must be larger than β .

If the angle of the surface normal at C_2 and the vector pointing from C_1 to P is between β and γ , the value of the diameter d is multiplied by a factor that is very large if the angle is close to β and that approaches one when the angle approaches γ . The purpose of the transition region between β and γ is to smooth the transition between touch points that fulfill the angle criterion and points that do not.

For example:

```
ThinLayer(  
  MinAngle=(89,90)  
  ChordWeight=0.5  
)
```

sets α to 0.5, and β and γ to 89° and 90° , respectively.

The default value for β is zero, which causes the extracted layer thickness to go to zero in concave corners of the layer, even when the corners have a wide opening angle. This can be unwanted, and choosing a larger value for β can resolve this issue. However, there can be situations when a larger β can make the thickness extraction ambiguous, for example, when the choice of C_1 is not unique.

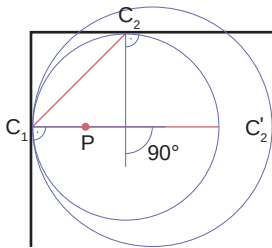


Figure 29 Thin-layer thickness extraction

To illustrate the thickness extraction, consider the example in [Figure 29](#). The thick black line is the boundary of the thin layer, and the red dot P indicates the position for which the thickness is extracted. First, the closest boundary point C_1 is found, which is to the left of P . Then, with the default $\beta = 0$, the largest circle touching at C_1 is inscribed into the layer. This is the smaller circle, with a second touch point C_2 . From this circle, the layer thickness is obtained by a linear combination of the chord length (diagonal red line) and the diameter (horizontal red line).

The angle between the normal vectors C_1 and C_2 is 90° . Therefore, if a value of β is greater than 90° but smaller than 180° is set, the smaller circle is not considered. Instead, the larger circle (with a second touch point C_2') determines the layer thickness; for this circle, the angle between the normal vectors is 180° .

High-Field Saturation

In high electric fields, the carrier drift velocity is no longer proportional to the electric field, instead, the velocity saturates to a finite speed v_{sat} . Sentaurus Device supports different models for the description of this effect:

- The Canali model, the transferred electron model, and two PMIs are available for all transport models.
- The basic model and the Meinerzhagen–Engl model both require hydrodynamic simulations.
- Another flexible model for hydrodynamic simulation is described in [Energy-dependent Mobility on page 658](#).

Using High-Field Saturation

The high-field saturation models comprise three submodels: the actual mobility model, the velocity saturation model, and the driving force model. With some restrictions, these models can be freely combined.

The actual mobility model is selected by flags to `eHighFieldSaturation` or `hHighFieldSaturation`. The default is the Canali model (see [Extended Canali Model on page 343](#)).

The flag `TransferredElectronEffect` selects the transferred electron model (see [Transferred Electron Model on page 344](#)).

The flags `CarrierTempDriveBasic` and `CarrierTempDriveME` activate the ‘basic’ and the Meinerzhagen–Engl model, respectively (see [Basic Model on page 345](#) and [Meinerzhagen–Engl Model on page 345](#)). These two models require hydrodynamic simulations.

For the Canali model, the transferred electron model, and the `PMI_HighFieldMobility` PMI, the driving force model is selected by a flag to `eHighFieldSaturation` or `hHighFieldSaturation`. Available flags are `GradQuasiFermi` (the default), `Eparallel`, `EparallelToInterface`, and `CarrierTempDrive` (see [Driving Force Models on page 348](#)). The latter is only available in hydrodynamic simulations. The `PMI_HighFieldMobility2` PMI supports `EparallelToInterface`, `Eparallel`, and `ElectricField` for the electric-field driving force. For the ‘basic’ model and the Meinerzhagen–Engl model, the driving force is part of the actual mobility model and cannot be chosen independently.

For all except the ‘basic’ model and the PMIs, a velocity saturation model can be selected in the HighFieldDependence parameter set (see [Velocity Saturation Models on page 347](#)).

Named Parameter Sets for High-Field Saturation

The HighFieldDependence and HydroHighFieldDependence parameter sets can be named. For example, in the parameter file, you can write:

```
HighFieldDependence "myset" { ... }
```

to declare a parameter set with the name myset. To use a named parameter set, specify its name with ParameterSetName as an option to HighFieldSaturation, eHighFieldSaturation, or hHighFieldSaturation. For example:

```
eMobility (
    HighFieldSaturation( ParameterSetName = "myset" ...) ...
)
```

By default, the unnamed parameter set is used.

Extended Canali Model

The Canali model [17] originates from the Caughey–Thomas formula [18], but has temperature-dependent parameters, which were fitted up to 430 K by Canali *et al.* [17]:

$$\mu(F) = \frac{(\alpha + 1)\mu_{\text{low}}}{\alpha + \left[1 + \left(\frac{(\alpha + 1)\mu_{\text{low}}F_{\text{hfs}}}{v_{\text{sat}}} \right)^{\beta} \right]^{1/\beta}} \quad (304)$$

where μ_{low} denotes the low-field mobility. Its definition depends on which of the previously described mobility models have been activated (see [Mobility due to Phonon Scattering on page 302](#) to [Philips Unified Mobility Model on page 310](#)). The exponent β is temperature dependent according to:

$$\beta = \beta_0 \left(\frac{T}{300\text{K}} \right)^{\beta_{\text{exp}}} \quad (305)$$

Details about the saturation velocity v_{sat} and driving field F_{hfs} are discussed in [Velocity Saturation Models on page 347](#) and [Driving Force Models on page 348](#). All other parameters are accessible in the parameter set HighFieldDependence.

The silicon default values are listed in [Table 59 on page 344](#).

A modified version of the Canali model is the Hänsch model [10]. It is activated by setting the parameter $\alpha = 1$. The Hänsch model is part of the Lucent mobility model (see [Lucent Model on page 346](#)). When using the hydrodynamic driving force [Eq. 316](#), α must be zero.

For the hydrodynamic driving force, [Eq. 316](#) can be substituted into [Eq. 304](#). Solving for μ yields the hydrodynamic Canali model:

$$\mu = \frac{\mu_{\text{low}}}{\left[\sqrt{1 + \gamma^2 \max(w_c - w_0, 0)^\beta} + \gamma \max(w_c - w_0, 0)^{\beta/2} \right]^{2/\beta}} \quad (306)$$

where γ is given by:

$$\gamma = \frac{1}{2} \left(\frac{\mu_{\text{low}}}{q \tau_{e,c} v_{\text{sat}}^2} \right)^{\beta/2} \quad (307)$$

In this form, the model has a discontinuous derivative at $w_0 = w_c$, which can lead to numeric problems. Therefore, Sentaurus Device applies a smoothing algorithm in the carrier temperature region $T < T_c < (1 + K_{dT})T$ to create a smooth transition between the low-field mobility μ_{low} and the mobility given in [Eq. 306](#). K_{dT} defaults to 0.2 and can be accessed in the parameter set HighFieldDependence.

Table 59 Canali model parameters (default values for silicon)

Symbol	Parameter name	Electrons	Holes	Unit
β_0	beta0	1.109	1.213	1
β_{exp}	betaexp	0.66	0.17	1
α	alpha	0	0	1

Transferred Electron Model

For GaAs and other materials with a similar band structure, a negative differential mobility can be observed for high driving fields. This effect is caused by a transfer of electrons into a energetically higher side valley with a much larger effective mass. Sentaurus Device includes a transferred electron model for the description of this effect, as given by [19]:

$$\mu = \frac{\mu_{\text{low}} + \left(\frac{v_{\text{sat}}}{F_{\text{hfs}}} \right) \left(\frac{F_{\text{hfs}}}{E_0} \right)^4}{1 + \left(\frac{F_{\text{hfs}}}{E_0} \right)^4} \quad (308)$$

Details of the saturation velocity v_{sat} and the driving field F_{hfs} are discussed in [Velocity Saturation Models on page 347](#) and [Driving Force Models on page 348](#). The reference field strength E_0 can be set in the parameter set HighFieldDependence.

The HighFieldDependence parameter set also includes a variable K_{smooth} , which is equal to 1 by default. If $K_{\text{smooth}} > 1$, a smoothing algorithm is applied to the formula for mobility in the driving force interval $F_{\text{vmax}} < F < K_{\text{smooth}} F_{\text{vmax}}$, where F_{vmax} is the field strength at which the velocity is at its maximum, $v_{\text{max}} = \mu F_{\text{vmax}}$. In this interval, [Eq. 308](#) is replaced by a polynomial that produces the same values and derivatives at the points F_{vmax} and $K_{\text{smooth}} F_{\text{vmax}}$. It is sometimes numerically advantageous to set $K_{\text{smooth}} \approx 20$.

Table 60 Transferred electron model: Default parameters

Symbol	Parameter	Electrons	Holes	Unit
E_0	E0_TrEf	4000	4000	Vcm^{-1}
K_{smooth}	Ksmooth_TrEf	1	1	1

Basic Model

According to this very simple model, the mobility decays inversely with the carrier temperature:

$$\mu = \mu_{\text{low}} \left(\frac{300 \text{ K}}{T_c} \right) \quad (309)$$

where μ_{low} is the low-field mobility and T_c is the carrier temperature.

Meinerzhagen–Engl Model

According to the Meinerzhagen–Engl model [\[20\]](#), the high field mobility degradation is given by:

$$\mu = \frac{\mu_{\text{low}}}{\left[1 + \left(\mu_{\text{low}} \frac{3k(T_c - T)}{2q \tau_{e,c} v_{\text{sat}}^2} \right)^\beta \right]^{1/\beta}} \quad (310)$$

where $\tau_{e,c}$ is the energy relaxation time. The coefficients of the saturation velocity v_{sat} (see [Eq. 311](#)) and the exponent β (see [Eq. 305](#)) are accessible in the parameter file:

```
HighFieldDependence {
    vsat0   = <value for electrons> <value for holes>
    vsatexp = <value for electrons> <value for holes>
```

```

}

HydroHighFieldDependence {
    beta0    = <value for electrons> <value for holes>
    betaexp  = <value for electrons> <value for holes>
}

```

The silicon default values are given in [Table 61](#) and [Table 62 on page 347](#).

Table 61 Meinerzhagen–Engl model: Default parameters

Silicon	Electrons	Holes	Unit
β_0	0.6	0.6	1
β_{exp}	0.01	0.01	1

Physical Model Interface

Sentaurus Device offers two physical model interfaces (PMIs) for high-field mobility saturation.

The first PMI is activated by specifying the name of the model as an option of `HighFieldSaturation`, `eHighFieldSaturation`, or `hHighFieldSaturation`. For more details about this model, see [High-Field Saturation Model on page 1041](#).

The second PMI allows you to implement models that depend on two driving forces. It is specified by `PMIModel` as the option to `HighFieldSaturation`, `eHighFieldSaturation`, or `hHighFieldSaturation`. For details, see [High-Field Saturation with Two Driving Forces on page 1050](#).

Lucent Model

The Lucent model has been developed by Darwish *et al.* [6]. Sentaurus Device implements this model as a combination of:

- An extended Philips unified mobility model (see [Philips Unified Mobility Model on page 310](#)) with the parameters $f_e = 0$ and $f_h = 0$ (see [Table 47 on page 315](#)). The Lucent model described in [6] also does not include clustering. To disable clustering in the Philips unified mobility model, set the parameters $N_{\text{ref,A}}$ and $N_{\text{ref,D}}$ to very large numbers.
- The enhanced Lombardi model (see [Enhanced Lombardi Model on page 316](#)) with the parameters from [Table 49 on page 318](#).

- The Hänsch model (see [Extended Canali Model on page 343](#)) with the parameter $\alpha = 1$ (see [Table 59 on page 344](#)). In addition, the β exponent in the Hänsch model is equal to 2, which can be accomplished by setting $\beta_0 = 2$ and $\beta_{\text{exp}} = 0$.

Velocity Saturation Models

Sentaurus Device supports two velocity saturation models. Model 1 is part of the Canali model and is given by:

$$v_{\text{sat}} = v_{\text{sat},0} \left(\frac{300 \text{ K}}{T} \right)^{v_{\text{sat},\text{exp}}} \quad (311)$$

This model is recommended for silicon.

Model 2 is recommended for GaAs. Here, v_{sat} is given by:

$$v_{\text{sat}} = \begin{cases} A_{\text{vsat}} - B_{\text{vsat}} \left(\frac{T}{300 \text{ K}} \right) & v_{\text{vsat}} > v_{\text{sat},\text{min}} \\ v_{\text{sat},\text{min}} & \text{otherwise} \end{cases} \quad (312)$$

The parameters of both models are accessible in the HighFieldDependence parameter set.

Selecting Velocity Saturation Models

The variable Vsat_formula in the HighFieldDependence parameter set selects the velocity saturation model. If Vsat_formula is set to 1, [Eq. 311](#) is used. If Vsat_formula is set to 2, [Eq. 312](#) is selected. The default value of Vsat_formula depends on the semiconductor material, for example, for silicon the default is 1; for GaAs, it is 2.

Table 62 Velocity saturation parameters

Symbol	Parameter	Electrons	Holes	Unit
$v_{\text{sat},0}$	vsat0	1.07×10^7	8.37×10^6	cm/s
$v_{\text{sat},\text{exp}}$	vsatexp	0.87	0.52	1
A_{vsat}	A_vsatsat	1.07×10^7	8.37×10^6	cm/s
B_{vsat}	B_vsatsat	3.6×10^6	3.6×10^6	cm/s
$v_{\text{sat},\text{min}}$	vsat_min	5.0×10^5	5.0×10^5	cm/s

Driving Force Models

Sentaurus Device supports five different models for the driving force F_{hfs} , the first two of which also are affected by the `ParallelToInterfaceInBoundaryLayer` keyword (see [Field Correction Close to Interfaces on page 349](#)).

For the first model (flag `Eparallel`), the driving field for electrons is the electric field parallel to the electron current:

$$F_{\text{hfs},n} = \vec{F} \cdot \frac{\vec{J}_n}{J_n} \quad (313)$$

For small currents, $J_n < c_{\text{min}}\mu_n/\mu_0$, the parallel electric field $F_{\text{hfs},n}$ is set to zero. Here, $\mu_0 = (q/300k)\text{cm}^2\text{K}^{-1}\text{s}^{-1}$ and c_{min} is specified (in Acm^{-2}) by `CDensityMin` in the `Math` section (default is $3\text{e-}8$).

For the second model (flag `GradQuasiFermi`), the driving field for electrons is:

$$F_{\text{hfs},n} = |\nabla\Phi_n| \quad (314)$$

The driving fields for holes are analogous.

NOTE Usually, [Eq. 313](#) and [Eq. 314](#) give the same or very similar results. However, numerically, one model may prove to be more stable. For example, in regions with small current, the evaluation of the parallel electric field can be numerically problematic.

For numeric reasons, Sentaurus Device actually implements generalizations of [Eq. 313](#) and [Eq. 314](#), where $F_{\text{hfs},n} = nF \cdot J_n/J_n(n + n_0)$ and $F_{\text{hfs},n} = n|\nabla\Phi_n|/(n + n_0)$, respectively. Here, n_0 is a numeric damping parameter. The values for electrons and holes default to zero, and are set (in cm^{-3}) with the `eDrForceRefDens` and `hDrForceRefDens` parameters in the `Math` section. Using positive values for n_0 can improve convergence for problems where strong generation–recombination occurs in regions with small density.

The third model (keyword `EparallelToInterface`) computes the driving force as the electric field parallel to the closest semiconductor–insulator interface:

$$F_{\text{hfs}} = |(I - \hat{n}\hat{n}^T)\vec{F}| \quad (315)$$

The vector $\hat{n}$ is a unit vector pointing to the closest semiconductor–insulator interface. It is determined in the same way as for the mobility degradation at interfaces. To select explicitly a semiconductor–insulator interface, use the `EnormalInterface` specification in the `Math` section (see [Normal to Interface on page 336](#)).

The driving force of `EparallelToInterface` is the same for electrons and holes. It is numerically stable because it does not depend on the direction of the current. However, this model will only give valid results if the current flows predominantly parallel to the interface (such as in the channel of MOSFET devices).

The fourth model (keyword `CarrierTempDrive`) requires hydrodynamic simulation. The driving field for electrons is:

$$F_{\text{hfs},n} = \sqrt{\frac{\max(w_n - w_0, 0)}{\tau_{e,n} q \mu_n}} \quad (316)$$

where $w_n = 3kT_n/2$ is the average electron thermal energy, $w_0 = 3kT/2$ is the equilibrium thermal energy, and $\tau_{e,n}$ is the energy relaxation time. The driving fields for holes are analogous.

The fifth model (keyword `ElectricField`) uses the electric field as an approximation for F_{hfs} .

The default model is `GradQuasiFermi` and, only for hydrodynamic simulations, it is `CarrierTempDrive`. See [Table 225 on page 1296](#) for a summary of keywords.

Field Correction Close to Interfaces

`ParallelToInterfaceInBoundaryLayer` in the `Math` section controls the computation of driving forces for mobility and avalanche models along interfaces. With this switch, the avalanche and mobility computations in boundary elements along interfaces use only the component parallel to the interface of the following vectors:

- Current vector
- Gradient of the quasi-Fermi potential

In this context, an interface is either a semiconductor–insulator region interface or an external boundary interface of the device.

This switch can be useful to avoid nonphysical breakdowns along an interface with a coarse mesh. It may be specified regionwise, in which case it only applies to boundary elements in a given region.

The switch `ParallelToInterfaceInBoundaryLayer` supports two “layer” options:

```
Math {
    ParallelToInterfaceInBoundaryLayer (PartialLayer)
    ParallelToInterfaceInBoundaryLayer (FullLayer)
}
```

If `PartialLayer` is specified, parallel fields are only used in elements that are connected to the interface by an edge (in 2D) or a face (in 3D). This is the default. The option `FullLayer` uses parallel fields in all elements that touch the interface by either a face, an edge, or a vertex.

The switch `ParallelToInterfaceInBoundaryLayer` is enabled by default. It can be disabled by specifying `-ParallelToInterfaceInBoundaryLayer`. Additional options are available for `ParallelToInterfaceInBoundaryLayer` to disable it only on portions of the interface. Specifying `-ExternalBoundary` disables it on all external boundaries; whereas, `-ExternalXPlane`, `-ExternalYPlane`, and `-ExternalZPlane` disables it only on external boundaries perpendicular to the x-axis, y-axis, and z-axis, respectively. Specifying `-Interface` disables it on semiconductor–insulator region interfaces.

Non-Einstein Diffusivity

By default, in the drift-diffusion equation (see [Drift-Diffusion Model on page 198](#)), Sentaurus Device assumes that the Einstein relation holds, and the diffusivity is related to the mobility by $D_n = kT\mu_n$ and $D_p = kT\mu_p$. For short-channel devices with steep doping gradients, this relation is no longer valid. Therefore, Sentaurus Device allows you to compute the diffusivities independently from the mobilities.

To be able to reuse existing mobility models, Sentaurus Device expresses the diffusivities in terms of *diffusivity mobilities*, $D_n = kT\mu_{n,diff}$ and $D_p = kT\mu_{p,diff}$, and you can specify models and parameters for $\mu_{n,diff}$ and $\mu_{p,diff}$, which are independent from μ_n and μ_p .

To compute the diffusivity mobilities, specify the keyword `Diffusivity`, `eDiffusivity`, or `hDiffusivity` as an option to `eMobility`, `hMobility`, or `Mobility`. The options for `eDiffusivity` and `hDiffusivity` are the same as for `HighFieldSaturation`. The low-field mobility that enters the diffusivity mobility is the same as for the high-field mobility.

The simplest way to obtain diffusivity mobilities different from the mobilities is to use named parameters sets to achieve a different parameterization (see [Named Parameter Sets for High-Field Saturation on page 343](#)). It is also possible to use entirely different models for mobilities and diffusivity mobilities.

The diffusivity mobilities can be plotted. The names of the datasets are `eDiffusivityMobility` and `hDiffusivityMobility`.

The current implementation of non-Einstein diffusivities has several restrictions. In particular, the following models use a current density that assumes the Einstein relation still holds:

- The models to compute the driving fields for mobility ([Driving Force Models on page 348](#)) and avalanche generation (see [Driving Force on page 387](#)).
- The J-model for trap cross sections (see [J-Model Cross Sections on page 415](#)).

- The current densities that appear in [Eq. 74](#) and [Eq. 75](#), p. 211.

Transport in magnetic fields is not supported. Support for anisotropy is restricted to the tensor grid method with current-independent anisotropy. The hydrodynamic model is supported, but it is not recommended for use with non-Einstein diffusivity.

Monte Carlo–computed Mobility for Strained Silicon

Based on Monte Carlo simulations [\[21\]](#), the parameter file `StrainedSilicon.par` has been created, which is shipped with Sentaurus Device (see [Default Parameters on page 73](#)). This file contains in-plane transport parameters at 300 K for silicon under biaxial tensile strain that is present when a thin silicon film is grown on top of a relaxed SiliconGermanium substrate. (*In-plane* refers to charge transport that is parallel to the interface to SiliconGermanium, as is the case in MOSFETs.)

In the Physics section of the command file, the germanium content (XFraction) of the SiliconGermanium substrate at the interface to the StrainedSilicon channel must be specified (this value determines the strain in the top silicon film) according to:

```
MoleFraction (RegionName=["TopLayer"] XFraction = 0.2 Grading = 0.0)
```

where the material of TopLayer is StrainedSilicon.

Band offsets and bulk mobility data are obtained from the model-solid theory of Van de Walle and descriptions of Monte Carlo simulations [\[21\]](#).

A parameterization of the surface mobility model as a function of strain is not yet possible. The present values for the parameters B and C of the surface mobility were extracted in a 1 μm bulk MOSFET at a high effective field (where the silicon control measurements of the references [\[22\]\[23\]](#) were in good agreement with the universal mobility curve of silicon, and where the neglected effect of the SiGe substrate is minimal) of approximately 0.7 MV/cm to reproduce reported experimental data [\[22\]\[23\]](#), for the typical germanium content in the SiliconGermanium substrate of 30%. The values for other germanium contents are given as comments.

Monte Carlo–computed Mobility for Strained SiGe in npn-SiGe HBTs

Based on Monte Carlo simulations [\[24\]](#), the parameter file `SiGeHBT.par` has been created, which is shipped with Sentaurus Device (see [Default Parameters on page 73](#)). This file contains transport parameters at 300 K for silicon germanium under biaxial compressive strain present

15: Mobility Models

Incomplete Ionization–dependent Mobility Models

when a thin SiGe film is grown on top of a relaxed silicon substrate, which occurs in the base of npn-SiGe heterojunction bipolar transistors (HBTs).

The electron parameters refer to the out-of-plane direction (that is, perpendicular to the SiGe–silicon interface) and the hole parameters to the in-plane direction (that is, parallel to the SiGe–silicon interface). The profile of the germanium content can either originate from a process simulation or be specified in the command file of Sentaurus Device (see [Abrupt and Graded Heterojunctions on page 48](#)).

The transport parameters have been obtained from the full-band Monte Carlo simulations [24].

The band gap in relaxed SiGe alloys has been extracted from the measurements in [25] and the values in strained SiGe calculated according to the model solid theory of C. G. Van de Walle.

Consistent limiting values for silicon (and polysilicon) are also provided.

Incomplete Ionization–dependent Mobility Models

Sentaurus Device supports incomplete ionization–dependent mobility models. This dependence is activated by specifying the keyword `IncompleteIonization` in `Mobility` sections. The incomplete ionization model (see [Chapter 13 on page 273](#)) must be activated also. The `Physics` sections for this case can be as follows:

```
Physics {  
    IncompleteIonization  
    Mobility( Enormal IncompleteIonization )  
}
```

In this case, for all equations that contain $N_{A,0}$, $N_{D,0}$, N_{tot} , Sentaurus Device will use N_A , N_D , N_i .

The following mobility models depend on incomplete ionization:

- Masetti model, see [Eq. 220](#)
- Arora model, see [Eq. 221](#)
- University of Bologna bulk model, see [Eq. 225–Eq. 227](#)
- Lombardi model, see [Eq. 249](#) and [Eq. 252](#)
- University of Bologna inversion layer model, see [Eq. 286–Eq. 288](#)
- Philips unified model, see [Eq. 243](#) and [Eq. 244](#)

Poole–Frenkel Mobility (Organic Material Mobility)

Most organic semiconductors have mobilities dependent on the electric field. Sentaurus Device supports mobilities having a square-root dependence on the electric field, which is a typical mobility dependence for organic semiconductors. The mobility as a function of the electric field is given by:

$$\mu = \mu_0 \exp\left(-\frac{E_0}{kT}\right) \exp\left(\sqrt{F}\left(\frac{\beta}{T} - \gamma\right)\right) \quad (317)$$

where μ_0 is the low-field mobility, β and γ are fitting parameters, E_0 is the effective activation energy, and F is the driving force (electric field).

The parameters E_0 , β , and γ can be adjusted in the PFMob section of the parameter file:

```
Material = "pentacene"
...
PFMob {
    beta_e = 1.1
    beta_h = 1.1
    E0_e = 0.0
    E0_h = 0.0
    gamma_e = 0.1
    gamma_h = 0.2
}
...
}
```

with their default parameters: $\beta_e = \beta_h = 0.1$, $E0_e = E0_h = 0$ (in eV), and $\gamma_e = \gamma_h = 0.0$.

Since the derivative of mobility with respect to the driving force is infinite at zero fields, the evaluation of this derivative is performed by replacing the square root of the driving force with an equivalent approximation having a finite derivative for zero field. The approximation of the square root with the regular square root can be adjusted by specifying the parameter `delta_sqrtReg` in the PFMob section of the parameter file. Typical values are in the range 0.1–0.0001; its default value is 0.1.

The model can be activated for both electrons and holes by specifying the keyword PFMob as a model for HighFieldSaturation in the Mobility section and the driving force as a parameter:

```
Physics(Region="pentacene") {
    ...
    Mobility (
        HighFieldSaturation(PFMob Eparallel)
```

```

    )
    ...
}

```

The model can also be selected for electrons or holes separately:

```

Physics{ Mobility( [eHighFieldSaturation(PFMob Eparallel)]
                  [hHighFieldSaturation(PFMob Eparallel)]) ...}

```

Mobility Averaging

Sentaurus Device computes separate values of the mobility for each vertex of each semiconductor element of the mesh. To guarantee current conservation, these mobilities are then averaged to obtain either one value for each semiconductor element of the mesh or one value for each edge of each semiconductor element.

Element averaging is used by default and requires less memory than element–edge averaging. Element–edge averaging results in a smaller discretization error than element averaging, in particular, when the enhanced Lombardi model (see [Enhanced Lombardi Model on page 316](#)) with $\alpha_{\perp} \neq 0$ is used. The time to compute the mobility is nearly identical for both approaches.

To select the mobility averaging approach, specify `eMobilityAveraging` and `hMobilityAveraging` in the Math section. The value is `Element` for element averaging, and `ElementEdge` for element–edge averaging.

Mobility Doping File

The `Grid` parameter in the `File` section of a command file can be used to read a TDR file that contains the device geometry, mesh, and doping.

By default, the doping read from this file is used in the mobility calculations previously described.

As an alternative, Sentaurus Device allows the donor and acceptor concentrations *for mobility calculations only* to be read from a separate TDR file. This is accomplished using the `MobilityDoping` parameter:

```

File {
    Grid          = "mosfet.tdr"
    MobilityDoping = "mosfet_mobility.tdr"
}

```


Notes:

- The geometry and mesh in the `MobilityDoping` file must match the `Grid` file.
- If a `MobilityDoping` file is specified, it disables the mobility dependency on `IncompleteIonization` if this mobility option is specified.
- The donor and acceptor concentrations read from a `MobilityDoping` file can be specified in the `Plot` section of the command file with `MobilityDonorConcentration` and `MobilityAcceptorConcentration`, respectively.
- For PMI models, the `MobilityDoping` file concentrations can be read using:

```
double Nd = ReadDoping("MobilityDonorConcentration")
double Na = ReadDoping("MobilityAcceptorConcentration")
```

Effective Mobility

It is often useful to know the effective channel mobility seen by carriers as a function of effective channel electric field. For this purpose, Sentaurus Device provides a `CurrentPlot` PMI model that calculates the following quantities for an n-channel device (expressions for a p-channel device are analogous):

$$n_{\text{sheet}} = \int_0^{y_1} n(y) dy \quad (318)$$

$$E_{\text{eff},n} = \frac{1}{n_{\text{sheet}}} \int_0^{y_1} E_{\perp}(y) n(y) dy + \left(\eta_e - \frac{1}{2} \right) \frac{q}{\epsilon_s} \int_0^{y_1} n(y) dy \quad (319)$$

$$E_{\text{eff(charge),n}} = \frac{q}{\epsilon_s} (N_{\text{depl}} + \eta_e N_s) = \frac{q}{\epsilon_s} \left(\int_0^{y_1} (N_A(y) - N_D(y) - p(y)) dy + \eta_e \int_0^{y_1} n(y) dy \right) \quad (320)$$

$$\mu_{\text{eff},n} = \frac{1}{n_{\text{sheet}}} \int_0^{y_1} \mu_n(y) n(y) dy \quad (321)$$

In the above expressions, y is the normal distance from the interface, and the integrals are evaluated from the interface to a distance y_1 where the contributions to the integrals are negligible.

To include these quantities in the current plot file, specify the following in the `CurrentPlot` section of the command file:

```
CurrentPlot {
  PMIModel (
    Name="EffectiveMobility"
    Start=(x0, y0, z0) [Direction = (dx, dy, dz)] [Depth = 0.5]
    [SnapToNode=<1 or 0>] [ChannelType = <-1, 0, or 1>]
    [eta_e = 0.5] [eta_h = 0.333333] [epsilon = 11.7]
  )
}
```

In the above specification, `Start` is the location along the interface where the integrations begin (if `SnapToNode=1` (the default), Sentaurus Device snaps the location to the nearest interface node). By default, the integration direction is normal to the interface but, if necessary, it can be specified using the vector parameter `Direction`. `Depth` is the integration distance, which defaults to 0.5 μm . In addition, by default (with `ChannelType=0`), the quantities shown in [Eq. 318](#) through [Eq. 321](#) are calculated for both electrons and holes, even though the electron (hole) quantities are only meaningful for n-channel (p-channel) devices. Specify `ChannelType=-1` to calculate the electron quantities only, or `ChannelType=1` to calculate the hole quantities only.

References

- [1] C. Lombardi *et al.*, "A Physically Based Mobility Model for Numerical Simulation of Nonplanar Devices," *IEEE Transactions on Computer-Aided Design*, vol. 7, no. 11, pp. 1164–1171, 1988.
- [2] G. Masetti, M. Severi, and S. Solmi, "Modeling of Carrier Mobility Against Carrier Concentration in Arsenic-, Phosphorus-, and Boron-Doped Silicon," *IEEE Transactions on Electron Devices*, vol. ED-30, no. 7, pp. 764–769, 1983.
- [3] N. D. Arora, J. R. Hauser, and D. J. Roulston, "Electron and Hole Mobilities in Silicon as a Function of Concentration and Temperature," *IEEE Transactions on Electron Devices*, vol. ED-29, no. 2, pp. 292–295, 1982.
- [4] S. Reggiani *et al.*, "A Unified Analytical Model for Bulk and Surface Mobility in Si n- and p-Channel MOSFET's," in *Proceedings of the 29th European Solid-State Device Research Conference (ESSDERC)*, Leuven, Belgium, pp. 240–243, September 1999.
- [5] S. Reggiani *et al.*, "Electron and Hole Mobility in Silicon at Large Operating Temperatures—Part I: Bulk Mobility," *IEEE Transactions on Electron Devices*, vol. 49, no. 3, pp. 490–499, 2002.
- [6] M. N. Darwish *et al.*, "An Improved Electron and Hole Mobility Model for General Purpose Device Simulation," *IEEE Transactions on Electron Devices*, vol. 44, no. 9, pp. 1529–1538, 1997.

- [7] H. Tanimoto *et al.*, “Modeling of Electron Mobility Degradation for HfSiON MISFETs,” in *International Conference on Simulation of Semiconductor Processes and Devices (SISPAD)*, Monterey, CA, USA, September 2006.
- [8] W. J. Zhu and T. P. Ma, “Temperature Dependence of Channel Mobility in HfO₂-Gated NMOSFETs,” *IEEE Electron Device Letters*, vol. 25, no. 2, pp. 89–91, 2004.
- [9] S. A. Mujtaba, *Advanced Mobility Models for Design and Simulation of Deep Submicrometer MOSFETs*, Ph.D. thesis, Stanford University, Stanford, CA, USA, December 1995.
- [10] W. Hänsch and M. Miura-Mattausch, “The hot-electron problem in small semiconductor devices,” *Journal of Applied Physics*, vol. 60, no. 2, pp. 650–656, 1986.
- [11] S. Takagi *et al.*, “On the Universality of Inversion Layer Mobility in Si MOSFET’s: Part I—Effects of Substrate Impurity Concentration,” *IEEE Transactions on Electron Devices*, vol. 41, no. 12, pp. 2357–2362, 1994.
- [12] G. Baccarani, *A Unified mobility model for Numerical Simulation, Parasitics Report*, DEIS-University of Bologna, Bologna, Italy, 1999.
- [13] S. Reggiani *et al.*, “Low-Field Electron Mobility Model for Ultrathin-Body SOI and Double-Gate MOSFETs With Extremely Small Silicon Thicknesses,” *IEEE Transactions on Electron Devices*, vol. 54, no. 9, pp. 2204–2212, 2007.
- [14] S. C. Choo, “Theory of a Forward-Biased Diffused-Junction P-L-N Rectifier—Part I: Exact Numerical Solutions,” *IEEE Transactions on Electron Devices*, vol. ED-19, no. 8, pp. 954–966, 1972.
- [15] N. H. Fletcher, “The High Current Limit for Semiconductor Junction Devices,” *Proceedings of the IRE*, vol. 45, no. 6, pp. 862–872, 1957.
- [16] D. B. M. Klaassen, “A Unified Mobility Model for Device Simulation—I. Model Equations and Concentration Dependence,” *Solid-State Electronics*, vol. 35, no. 7, pp. 953–959, 1992.
- [17] C. Canali *et al.*, “Electron and Hole Drift Velocity Measurements in Silicon and Their Empirical Relation to Electric Field and Temperature,” *IEEE Transactions on Electron Devices*, vol. ED-22, no. 11, pp. 1045–1047, 1975.
- [18] D. M. Caughey and R. E. Thomas, “Carrier Mobilities in Silicon Empirically Related to Doping and Field,” *Proceedings of the IEEE*, vol. 55, no. 12, pp. 2192–2193, 1967.
- [19] J. J. Barnes, R. J. Lomax, and G. I. Haddad, “Finite-Element Simulation of GaAs MESFET’s with Lateral Doping Profiles and Submicron Gates,” *IEEE Transactions on Electron Devices*, vol. ED-23, no. 9, pp. 1042–1048, 1976.
- [20] B. Meinerzhagen and W. L. Engl, “The Influence of the Thermal Equilibrium Approximation on the Accuracy of Classical Two-Dimensional Numerical Modeling of Silicon Submicrometer MOS Transistors,” *IEEE Transactions on Electron Devices*, vol. 35, no. 5, pp. 689–697, 1988.

15: Mobility Models

References

- [21] F. M. Bufler and W. Fichtner, "Hole and electron transport in strained Si: Orthorhombic versus biaxial tensile strain," *Applied Physics Letters*, vol. 81, no. 1, pp. 82–84, 2002.
- [22] M. T. Currie *et al.*, "Carrier mobilities and process stability of strained Si n- and p-MOSFETs on SiGe virtual substrates," *Journal of Vacuum Science & Technology B*, vol. 19, no. 6, pp. 2268–2279, 2001.
- [23] C. W. Leitz *et al.*, "Hole mobility enhancements and alloy scattering-limited mobility in tensile strained Si/SiGe surface channel metal–oxide–semiconductor field-effect transistors," *Journal of Applied Physics*, vol. 92, no. 7, pp. 3745–3751, 2002.
- [24] F. M. Bufler, *Full-Band Monte Carlo Simulation of Electrons and Holes in Strained Si and SiGe*, München: Herbert Utz Verlag, 1998.
- [25] R. Braunstein, A. R. Moore, and F. Herman, "Intrinsic Optical Absorption in Germanium-Silicon Alloys," *Physical Review*, vol. 109, no. 3, pp. 695–710, 1958.

CHAPTER 16 Generation–Recombination

This chapter describes the generation–recombination processes that can be modeled in Sentaurus Device.

Generation–recombination processes are processes that exchange carriers between the conduction band and the valence band. They are very important in device physics, in particular, for bipolar devices. This chapter describes the generation–recombination models available in Sentaurus Device. Most models are local in the sense that their implementation (sometimes in contrast to reality) does not involve spatial transport of charge. For each individual generation or recombination process, the electrons and holes involved appear or vanish at the same location. The only exceptions are one of the trap-assisted tunneling models and one of the band-to-band tunneling models (see [Dynamic Nonlocal Path Trap-assisted Tunneling on page 367](#) and [Dynamic Nonlocal Path Band-to-Band Model on page 395](#)). For other models that couple the conduction and valence bands and account for spatial transport of charge, see [Tunneling and Traps on page 417](#) and [Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612](#).

Shockley–Read–Hall Recombination

Recombination through deep defect levels in the gap is usually labeled Shockley–Read–Hall (SRH) recombination. In Sentaurus Device, the following form is implemented:

$$R_{\text{net}}^{\text{SRH}} = \frac{np - n_{\text{i,eff}}^2}{\tau_p(n + n_1) + \tau_n(p + p_1)} \quad (322)$$

with:

$$n_1 = n_{\text{i,eff}} \exp\left(\frac{E_{\text{trap}}}{kT}\right) \quad (323)$$

and:

$$p_1 = n_{\text{i,eff}} \exp\left(\frac{-E_{\text{trap}}}{kT}\right) \quad (324)$$

where E_{trap} is the difference between the defect level and intrinsic level. The variable E_{trap} is accessible in the parameter file. The silicon default value is $E_{\text{trap}} = 0$.

The lifetimes τ_n and τ_p are modeled as a product of a doping-dependent (see [SRH Doping Dependence on page 361](#)), field-dependent (see [SRH Field Enhancement on page 363](#)), and temperature-dependent (see [SRH Temperature Dependence on page 362](#)) factor:

$$\tau_c = \tau_{\text{dop}} \frac{f(T)}{1 + g_c(F)} \quad (325)$$

where $c = n$ or $c = p$. For an additional density dependency of the lifetimes, see [Trap-assisted Auger Recombination on page 371](#).

For simulations that use Fermi statistics (see [Fermi Statistics on page 194](#)) or quantization (see [Chapter 14 on page 279](#)), [Eq. 322](#) needs to be generalized. The modified equation reads:

$$R_{\text{net}}^{\text{SRH}} = \frac{np - \gamma_n \gamma_p n_{i,\text{eff}}^2}{\tau_p (n + \gamma_n n_1) + \tau_n (p + \gamma_p p_1)} \quad (326)$$

where γ_n and γ_p are given by [Eq. 47](#) and [Eq. 48](#), p. 194.

Using SRH Recombination

The generation–recombination models are selected in the `Physics` section as an argument to the `Recombination` keyword:

```
Physics{ Recombination( <arguments> ) ... }
```

The SRH model is activated by specifying the SRH argument:

```
Physics{ Recombination( SRH ... ) ... }
```

The keyword for plotting the SRH recombination rate is:

```
Plot{ ...
    SRHRecombination
}
```

SRH Doping Dependence

The doping dependence of the SRH lifetimes is modeled in Sentaurus Device with the Scharfetter relation:

$$\tau_{\text{dop}}(N_{A,0} + N_{D,0}) = \tau_{\text{min}} + \frac{\tau_{\text{max}} - \tau_{\text{min}}}{1 + \left(\frac{N_{A,0} + N_{D,0}}{N_{\text{ref}}} \right)^\gamma} \quad (327)$$

Such a dependence arises from experimental data [1] and the theoretical conclusion that the solubility of a fundamental, acceptor-type defect (probably a divacancy (E5) or a vacancy complex) is strongly correlated to the doping density [2][3][4]. Default values are listed in [Table 63 on page 363](#). The Scharfetter relation is used when the argument `DopingDependence` is specified for the SRH recombination. Otherwise, $\tau = \tau_{\text{max}}$ is used.

The evaluation of the SRH lifetimes according to the Scharfetter model is activated by specifying the additional argument `DopingDependence` for the SRH keyword in the Recombination statement:

```
Physics{ Recombination( SRH( DopingDependence ... ) ... ) ... }
```

Lifetime Profiles from Files

Sentaurus Device can use spatial lifetime profiles provided by a file. Such profiles can be precomputed or generated manually by an editor such as Sentaurus Structure Editor (refer to the *Sentaurus Structure Editor User Guide*).

The names of the datasets for the electron and hole lifetimes must be `eLifetime` and `hLifetime`, respectively. They are loaded from a file named by the keyword `LifeTime` in the `File` section:

```
File {
    Grid      = "MyDev_msh.tdr"
    LifeTime  = "MyDev_msh.tdr"
    ...
}
```

For each grid point, the values defined by the lifetime profile are used as τ_{max} in [Eq. 327](#).

The lifetime data can be in a separate file from the doping, but must correspond to the same grid.

SRH Temperature Dependence

To date, there is no consensus on the temperature dependence of the SRH lifetimes. This appears to originate from a different understanding of lifetime. From measurements of the recombination lifetime [5][6]:

$$\tau = \delta n / R \quad (328)$$

in power devices (δn is the excess carrier density under neutral conditions, $\delta n = \delta p$), it was concluded that the lifetime increases with rising temperature. Such a dependence was modeled either by a power law [5][6]:

$$\tau(T) = \tau_0 \left(\frac{T}{300\text{K}} \right)^\alpha \quad (329)$$

or an exponential expression of the form:

$$\tau(T) = \tau_0 e^{C \left(\frac{T}{300\text{K}} - 1 \right)} \quad (330)$$

A calculation using the low-temperature approximation of multiphonon theory [7] gives:

$$\tau_{\text{SRH}}(T) = \tau_{\text{SRH}}(300\text{K}) \cdot f(T) \text{ with } f(T) = \left(\frac{T}{300\text{K}} \right)^{T_\alpha} \quad (331)$$

with $T_\alpha = -3/2$, which is the expected decrease of minority carrier lifetimes with rising temperature. Since the temperature behavior strongly depends on the nature of the recombination centers, there is no universal law $\tau_{\text{SRH}}(T)$.

In Sentaurus Device, the power law model, Eq. 329, can be activated with the keyword TempDependence in the SRH statement:

```
Physics{ Recombination( SRH( TempDependence ... ) ... ) }
```

Additionally, Sentaurus Device supports an exponential model for $f(T)$:

$$f(T) = e^{C \left(\frac{T}{300\text{K}} - 1 \right)} \quad (332)$$

This model is activated with the keyword ExpTempDependence:

```
Physics{ Recombination( SRH( ExpTempDependence ... ) ... ) }
```


SRH Doping- and Temperature-dependent Parameters

All the parameters of the doping- and temperature-dependent SRH recombination models are accessible in the parameter set `Scharfetter`.

Table 63 Default parameters for doping- and temperature-dependent SRH lifetime

Symbol	Parameter name	Electrons	Holes	Unit
$\tau_{\min}$	taumin	0	0	s
$\tau_{\max}$	taumax	1×10^{-5}	3×10^{-6}	s
N_{ref}	Nref	1×10^{16}	1×10^{16}	cm^{-3}
γ	gamma	1	1	1
T_{α}	Talpha	–1.5	–1.5	1
C	Tcoeff	2.55	2.55	1
E_{trap}	Etrap	0	0	eV

SRH Field Enhancement

Field enhancement reduces SRH recombination lifetimes in regions of strong electric fields. It must not be neglected if the electric field exceeds a value of approximately $3 \times 10^5 \text{ V/cm}$ in certain regions of the device. For example, the I–V characteristics of reverse-biased p–n junctions are extremely sensitive to defect-assisted tunneling, which causes electron–hole pair generation before band-to-band tunneling or avalanche generation sets in. Therefore, it is recommended that field-enhancement is included in the simulation of drain reverse leakage and substrate currents in MOS transistors.

Sentaurus Device provides two field-enhancement models: the Schenk trap-assisted tunneling model (see [Schenk Trap-assisted Tunneling \(TAT\) Model on page 364](#)) and the Hurkx trap-assisted tunneling model (see [Hurkx TAT Model on page 366](#)). The Hurkx model is also available for trap capture and emission rates (see [Hurkx Model for Cross Sections on page 415](#)).

Using Field Enhancement

The local field-dependence of the SRH lifetimes is activated by parameters in the `ElectricField` option of SRH:

```
SRH( ...  
    ElectricField (  
        Lifetime = Schenk | Hurkx | Constant  
        DensityCorrection = Local | None  
    )  
)
```

`Lifetime` selects the lifetime model. The default is `Constant`, for field-independent lifetime. `Schenk` selects the Schenk lifetimes (see [Schenk Trap-assisted Tunneling \(TAT\) Model on page 364](#)), and `Hurkx` selects the Hurkx lifetimes (see [Hurkx TAT Model on page 366](#)).

`DensityCorrection` defaults to `None`. A value of `Local` selects the model described in [Schenk TAT Density Correction on page 366](#).

For backward compatibility:

- `SRH (Tunneling)` selects `Lifetime = Schenk` and `DensityCorrection = Local`.
- `SRH(Tunneling(Hurkx))` selects `Lifetime = Hurkx` and `DensityCorrection = None`.

NOTE The inclusion of defect-assisted tunneling may lead to convergence problems. In such cases, it is recommended that the flag `NoSRHperPotential` is specified in the `Math` section. This causes Sentaurus Device to exclude derivatives of $g(F)$ with respect to the potential from the Jacobian matrix.

Schenk Trap-assisted Tunneling (TAT) Model

The field dependence of the recombination rate is taken into account by the field enhancement factors:

$$[1 + g(F)]^{-1} \tag{333}$$

of the SRH lifetimes [\[7\]](#) (see [Eq. 325](#)).

In the case of electrons, $g(F)$ has the form:

$$g_n(F) = \left(1 + \frac{(\hbar\Theta)^{3/2} \sqrt{E_t - E_0}}{E_0 \hbar\omega_0} \right)^{-\frac{1}{2}} \frac{(\hbar\Theta)^{3/4} (E_t - E_0)^{1/4}}{2 \sqrt{E_t E_0}} \left(\frac{\hbar\Theta}{kT} \right)^{\frac{3}{2}} \quad (334)$$

$$\times \exp \left(-\frac{E_t - E_0}{\hbar\omega_0} + \frac{\hbar\omega_0 - kT}{2\hbar\omega_0} + \frac{2E_t + kT}{2\hbar\omega_0} \ln \frac{E_t}{\epsilon_R} - \frac{E_0}{\hbar\omega_0} \ln \frac{E_0}{\epsilon_R} + \frac{E_t - E_0}{kT} - \frac{4}{3} \left(\frac{E_t - E_0}{\hbar\Theta} \right)^{\frac{3}{2}} \right)$$

where E_0 denotes the energy of an optimum horizontal transition path, which depends on field strength and temperature in the following way:

$$E_0 = 2 \sqrt{\epsilon_F [\sqrt{\epsilon_F + E_t + \epsilon_R} - \sqrt{\epsilon_F}] - \epsilon_R}, \epsilon_F = \frac{(2\epsilon_R kT)^2}{3(\hbar\Theta)} \quad (335)$$

In this expression, $\epsilon_R = S\hbar\omega_0$ is the lattice relaxation energy, S is the Huang–Rhys factor, $\hbar\omega_0$ is the effective phonon energy, E_t is the energy level of the recombination center (thermal depth), and $\Theta = (q^2 F^2 / 2\hbar m_{\Theta,n})^{1/3}$ is the electro-optical frequency. The mass $m_{\Theta,n}$ is the electron tunneling mass in the field direction and is given in the parameter file. The expression for holes follows from Eq. 334 by replacing $m_{\Theta,n}$ with $m_{\Theta,p}$ and E_t with $E_{g,\text{eff}} - E_t$.

For electrons, E_t is related to the defect level E_{trap} of Eq. 323 and Eq. 324 by:

$$E_t = \frac{1}{2} E_{g,\text{eff}} + \frac{3}{4} kT \ln \left(\frac{m_n}{m_p} \right) - E_{\text{trap}} - (32R_C \hbar^3 \Theta^3)^{1/4} \quad (336)$$

where the effective Rydberg constant R_C is:

$$R_C = m_C \left(\frac{Z^2}{\epsilon^2} \right) Ry \quad (337)$$

where Ry is the Rydberg energy (13.606 eV), ϵ is the relative dielectric constant, and Z is a fit parameter.

For holes, E_t is given by:

$$E_t = \frac{1}{2} E_{g,\text{eff}} - \frac{3}{4} kT \ln \left(\frac{m_n}{m_p} \right) + E_{\text{trap}} - (32R_V \hbar^3 \Theta^3)^{1/4} \quad (338)$$

Note that E_{trap} is measured from the intrinsic level and not from mid gap. The zero-field lifetime τ_{SRH} is defined by Eq. 327.

Schenk TAT Density Correction

In the original Schenk model [7], the density n in Eq. 322 is replaced by:

$$\tilde{n} = n \exp\left(-\frac{\gamma_n |\nabla E_{F,n}| (E_t - E_0)}{kTF}\right) \quad (339)$$

with E_0 and E_t according to Eq. 335 and Eq. 336. p is replaced by an analogous expression.

The parameters $\gamma_n = n/(n + n_{\text{ref}})$ and $\gamma_p = p/(p + p_{\text{ref}})$ are close to one for significantly large densities and vanish for small densities, switching off the density correction and improving numeric robustness. The reference densities n_{ref} and p_{ref} are specified (in cm^{-3}) by the DenCorRef parameter pair in the TrapAssistedTunneling parameter set. They default to 10^3 cm^{-3} .

Hurkx TAT Model

The following equations apply to electrons and holes. The lifetimes and capture cross sections become functions of the trap-assisted tunneling factor $g(F) = \Gamma_{\text{tat}}$:

$$\tau = \tau_0 / (1 + \Gamma_{\text{tat}}), \quad \sigma = \sigma_0 (1 + \Gamma_{\text{tat}}) \quad (340)$$

where Γ_{tat} is given by:

$$\Gamma_{\text{tat}} = \int_0^{\tilde{E}_n} \exp\left[u - \frac{2}{3} \frac{\sqrt{u^3}}{\tilde{E}}\right] du \quad (341)$$

with the approximate solutions:

$$\Gamma_{\text{tat}} \approx \begin{cases} \sqrt{\pi} \tilde{E} \cdot \exp\left[\frac{1}{3} \tilde{E}^2\right] \left(2 - \text{erfc}\left[\frac{1}{2} \left(\frac{\tilde{E}_n}{\tilde{E}} - \tilde{E}\right)\right]\right), & \tilde{E} \leq \sqrt{\tilde{E}_n} \\ \sqrt{\pi} \tilde{E} \cdot \tilde{E}_n^{1/4} \exp\left[-\tilde{E}_n + \tilde{E} \sqrt{\tilde{E}_n} + \frac{1}{3} \frac{\sqrt{\tilde{E}_n}^3}{\tilde{E}}\right] \text{erfc}[\tilde{E}_n^{1/4} \sqrt{\tilde{E} - \tilde{E}_n}^{3/4} / \sqrt{\tilde{E}}], & \tilde{E} > \sqrt{\tilde{E}_n} \end{cases} \quad (342)$$

where $\tilde{E}$ and $\tilde{E}_n$ are respectively defined as:

$$\tilde{E} = \frac{E}{E_0}, \text{ where } E_0 = \frac{\sqrt{8m_0m_t k^3 T^3}}{q\hbar} \quad (343)$$

$$\tilde{E}_n = \frac{E_n}{kT} = \begin{cases} 0, & kT \ln \frac{n}{n_i} > 0.5E_g \\ \frac{0.5E_g}{kT} - \ln \frac{n}{n_i}, & E_{\text{trap}} \leq kT \ln \frac{n}{n_i} \leq 0.5E_g \\ \frac{0.5E_g}{kT} - \frac{E_{\text{trap}}}{kT}, & E_{\text{trap}} > kT \ln \frac{n}{n_i} \end{cases} \quad (344)$$

where m_t is the carrier tunneling mass and E_{trap} is an energy of trap level that is taken from SRH recombination if the model is applied to the lifetimes (τ) or from trap equations if it is applied to cross sections (σ).

When quantization is active (see [Chapter 14 on page 279](#)), the classical density:

$$n_{\text{cl}} = n \exp\left(\frac{\Lambda}{kT}\right) \quad (345)$$

rather than the true density n enters [Eq. 344](#).

Field-Enhancement Parameters

The parameters for the Schenk model are accessible in the parameter set TrapAssistedTunneling. The default parameters implemented in Sentaurus Device are related to the gold acceptor level: $E_{\text{trap}} = 0 \text{ eV}$, $S = 3.5$, and $\hbar\omega_0 = 0.068 \text{ eV}$. TrapAssistedTunneling provides a parameter MinField (specified in Vcm^{-1}) used for smoothing at small electric fields. A value of zero (the default) disables smoothing.

The Hurkx model has only one parameter, m_t , the carrier tunneling mass, which can be specified in the parameter file for electrons and holes as follows:

```
HurkxTrapAssistedTunneling {
    mt = <value>, <value>
}
```

Dynamic Nonlocal Path Trap-assisted Tunneling

The local Schenk and Hurkx TAT models described in [SRH Field Enhancement on page 363](#) may fail to give accurate results at low temperatures or near abrupt junctions with a strong, nonuniform, electric field profile. Moreover, the local TAT models are not suitable for computing TAT in heterojunctions where the band edge is modulated by the material composition rather than the electric field. In addition to the local Schenk and Hurkx TAT models, Sentaurus Device provides dynamic nonlocal path Schenk and Hurkx TAT models. These models take into account nonlocal TAT processes with the WKB transmission

coefficient based on the exact tunneling barrier. In the present models, electrons and holes are captured in or emitted from the defect level at different locations by the phonon-assisted tunneling process. As a result, the position-dependent electron and hole recombination rates as well as the recombination rate at the defect level are all different. For each location and for each carrier type, the tunneling path is determined dynamically based on the energy band profile rather than predefined by the nonlocal mesh. Therefore, the present models do not require user-specification of the nonlocal mesh.

NOTE The nonlocal path TAT models are not suitable for AC or noise analysis as nonlocal derivative terms in the Jacobian matrix are not taken into account. The lack of the derivative terms can degrade convergence when the high-field saturation mobility model is switched on, or when the series resistance is defined at electrodes, or when there is a floating region.

Recombination Rate

To compute the net electron recombination rate, the model dynamically searches for the tunneling path with the following assumptions:

- The tunneling path is a straight line with its direction equal to the gradient of the conduction band at the starting position.
- The tunneling energy is equal to the conduction band energy at the starting position and is equal to the defect level ($E_T(x) = E_{\text{trap}}(x) + E_i(x)$) at the ending position.
- Electrons can be captured in or emitted from the defect level at any location between the starting position and the ending position of the tunneling path.
- When the tunneling path encounters Neumann boundaries or semiconductor–insulator interfaces, it undergoes specular reflection.

For a given tunneling path of length l that starts at $x = 0$ and ends at $x = l$, electron capture and emission between the conduction band at $x = 0$ and the defect level at any position x for $0 \leq x \leq l$ is possible. As a result, the net electron recombination rate at $x = 0$ from the conduction band due to the TAT process $R_{\text{net}, n}^{\text{TAT}}$ involves integration along the path as follows:

$$R_{\text{net}, n}^{\text{TAT}} = C_n \int_0^l \frac{\Gamma_C(x, E_C(0))}{\tau_n(x)} \frac{T(0) + T(x)}{2\sqrt{T(0)T(x)}} \left[\exp\left[\frac{E_{F, n}(0) - E_C(0)}{kT(0)}\right] f^p(x) - \exp\left[\frac{E_T(x) - E_C(0)}{kT(x)}\right] f^n(x) \right] dx \quad (346)$$

$$C_n = |\nabla E_C(0)| \frac{N_C(0)}{kT(0)} \left(\exp\left[\frac{E_{F, n}(0) - E_C(0)}{kT(0)}\right] + 1 \right)^{-\alpha} \quad (347)$$

$$\Gamma_C(x, \varepsilon) = W_C(x, \varepsilon) \exp\left(-2 \int_0^x \kappa_C(x, \varepsilon) dr\right) \quad (348)$$

where:

- τ_n is the electron lifetime.
- $\alpha = 0$ for Boltzmann statistics and $\alpha = 1$ for Fermi–Dirac statistics.
- f^n is the electron occupation probability at the defect level.
- f^p is the hole occupation probability at the defect level.
- κ_C is the magnitude of the imaginary wavevector obtained from the effective mass approximation or from the Franz two-band dispersion relation (Eq. 646, p. 618).
- $W_C(x, \varepsilon) = 1$ for the nonlocal Hurkx TAT model. For the nonlocal Schenk TAT model, $W_C(x, \varepsilon)$ is modeled as:

$$W_C(x, \varepsilon) = \frac{\hbar[E_C(x) - E_C(0)] \exp(0.5) W[\varepsilon - E_T(x)]}{4x[E_C(x) - E_T(x)] \sqrt{\pi m_C kT(x)} W[E_C(x) + kT(x)/2 - E_T(x)]} \quad (349)$$

$$W(\varepsilon) = \frac{1}{\sqrt{\chi}} \exp\left(\frac{\varepsilon}{2kT} + \chi\right) \left(\frac{z}{l + \chi}\right)^l \quad (350)$$

where:

- $\chi = \sqrt{l^2 + z^2}$.
- $l = \varepsilon / \hbar \omega_0$.
- $z = S / \sinh(\hbar \omega_0 / 2kT)$.
- $\hbar \omega_0$ is the phonon energy.
- S is the Huang–Rhys constant.

The net hole recombination rate can be obtained in a similar way. The quasistatic electron and hole occupation probabilities at the defect level can be determined by balancing the net electron capture rate and the net hole capture rate.

Using Dynamic Nonlocal Path TAT Model

The nonlocal path TAT model is switched on when the model is selected in the SRH option as follows:

```
Recombination( ...
  SRH( ...
    NonlocalPath (
      Lifetime = Schenk | Hurkx      # Hurkx model is used by default
      Fermi | -Fermi                # use alpha=1 (alpha=0 by default)
      TwoBand | -TwoBand            # use Franz two band dispersion (off by default)
    )
  )
)
```

16: Generation–Recombination

Shockley–Read–Hall Recombination

`Lifetime=Hurkx` (default) selects the nonlocal Hurkx model, while `Lifetime=Schenk` selects the nonlocal Schenk model. The `Fermi` option sets α equal to 1 ($\alpha = 0$ by default). The `TwoBand` option activates the Franz two-band dispersion (Eq. 646, p. 618) instead of the effective mass approximation for the computation of the imaginary wavevector.

The nonlocal path TAT model introduces no additional model parameters. The same minority carrier lifetime for the SRH model is used in the nonlocal path TAT model. The doping- and temperature-dependent lifetime as well as the lifetime loaded from file can be specified together with the nonlocal path TAT model.

NOTE The SRH field-enhancement model should not be used together with the nonlocal path TAT model to avoid double-counting of the TAT process.

The nonlocal Hurkx model uses the tunneling mass specified in the `HurkxTrapAssistedTunneling` parameter set. The tunneling mass, the phonon energy, and the Huang–Rhys coupling constant for the nonlocal Schenk model are specified in the `TrapAssistedTunneling` parameter set.

The maximum length of the tunneling path is determined by the parameter `MaxTunnelLength` in the `Band2BandTunneling` parameter set.

You can consider electron TAT from Schottky contacts or Schottky metal–semiconductor interfaces using the nonlocal path TAT model. To consider the electron capture and emission from the Schottky contact, you must specify the parameter `TATNonlocalPathNC`, which represents the room temperature effective density-of-states ($N_C(300\text{K})$) for the Schottky contact in the electrode-specific `Physics` section, for example:

```
Physics (electrode = "source") { ...
    TATNonlocalPathNC = 2.0e19
}
```

Similarly, to consider the TAT from the Schottky metal–semiconductor interface, you must specify the room temperature effective density-of-states in the corresponding interface-specific `Physics` section, for example:

```
Physics (MaterialInterface = "Gold/Silicon") { ...
    TATNonlocalPathNC = 2.0e19
}
```

Sentaurus Device assumes that the effective density-of-states at T follows:

$$N_C(T) = N_C(300\text{K}) \left(\frac{T}{300\text{K}} \right)^{\frac{3}{2}} \quad (351)$$

The electron and hole recombination rates as well as the recombination rate at the defect level can be plotted as:

```
Plot { ...
    eSRHRecombination    # electron recombination rate
    hSRHRecombination    # hole recombination rate
    tSRHRecombination    # recombination rate at the defect level
}
```

Trap-assisted Auger Recombination

Trap-assisted Auger (TAA) recombination is a modification to the SRH recombination (see [Shockley–Read–Hall Recombination on page 359](#)) and coupled defect level (see [Coupled Defect Level \(CDL\) Recombination on page 373](#)) models. When TAA is active, Sentaurus Device uses the lifetimes [4]:

$$\frac{\tau_p}{1 + \tau_p/\tau_p^{\text{TAA}}} \quad (352)$$

$$\frac{\tau_n}{1 + \tau_n/\tau_n^{\text{TAT}}} \quad (353)$$

in place of the lifetimes τ_p and τ_n in [Eq. 322](#) and [Eq. 360](#).

The TAA lifetimes in [Eq. 352](#) depend on the carrier densities:

$$\frac{1}{\tau_n^{\text{TAA}}} \approx C_p^{\text{TAA}}(n + p) \quad (354)$$

$$\frac{1}{\tau_p^{\text{TAA}}} \approx C_n^{\text{TAA}}(n + p) \quad (355)$$

A reasonable order of magnitude for the TAA coefficients C_n^{TAA} and C_p^{TAA} is $1 \times 10^{-12} \text{ cm}^3 \text{ s}^{-1}$ to $1 \times 10^{-11} \text{ cm}^3 \text{ s}^{-1}$; the default values are $C_n^{\text{TAA}} = C_p^{\text{TAA}} = 1 \times 10^{-12} \text{ cm}^3 \text{ s}^{-1}$.

TAA recombination is activated by using the keyword `TrapAssistedAuger` in the `Recombination` statement in the `Physics` section (see [Table 183 on page 1266](#)):

```
Physics {
    Recombination(TrapAssistedAuger ...)
    ...
}
```

The trap-assisted Auger parameters C_n^{TAA} and C_p^{TAA} are accessible in the parameter set TrapAssistedAuger.

Surface SRH Recombination

The surface SRH recombination model can be activated at semiconductor–semiconductor and semiconductor–insulator interfaces (see [Interface-specific Models on page 59](#)).

At interfaces, an additional formula is used that is structurally equivalent to the bulk expression of the SRH generation–recombination:

$$R_{\text{surf, net}}^{\text{SRH}} = \frac{np - n_{i,\text{eff}}^2}{(n + n_1)/s_p + (p + p_1)/s_n} \quad (356)$$

with:

$$n_1 = n_{i,\text{eff}} \exp\left(\frac{E_{\text{trap}}}{kT}\right) \text{ and } p_1 = n_{i,\text{eff}} \exp\left(\frac{-E_{\text{trap}}}{kT}\right) \quad (357)$$

For Fermi statistics and quantization, the equations are modified in the same manner as for bulk SRH recombination (see [Eq. 326](#)).

The recombination velocities of otherwise identically prepared surfaces depend, in general, on the concentration of dopants at the surface [\[8\]\[9\]\[10\]](#). Particularly, in cases where the doping concentration varies along an interface, it is desirable to include such a doping dependence. Sentaurus Device models doping dependence of surface recombination velocities according to:

$$s = s_0 \left[1 + s_{\text{ref}} \left(\frac{N_i}{N_{\text{ref}}} \right)^\gamma \right] \quad (358)$$

The results of Cuevas [\[10\]](#) indicate that for phosphorus-diffused silicon, $\gamma = 1$; while the results of King and Swanson [\[9\]](#) seem to imply that no significant doping dependence exists for the recombination velocities of boron-diffused silicon surfaces.

To activate the model, specify the option SurfaceSRH to the Recombination keyword in the Physics section for the respective interface. To plot the surface recombination, specify SurfaceRecombination in the Plot section. The parameters E_{trap} , s_{ref} , N_{ref} , and γ are

accessible in the parameter set SurfaceRecombination. The corresponding values for silicon are given in [Table 64](#).

Table 64 Surface SRH parameters

Symbol	Parameter name	Electrons	Holes	Unit
s_0	S0	1×10^3	1×10^3	cm/s
s_{ref}	Sref	1×10^{-3}		1
N_{ref}	Nref	1×10^{16}		cm^{-3}
γ	gamma	1		1
E_{trap}	Etrap	0		eV

The doping dependence of the recombination velocity can be suppressed by setting s_{ref} to zero.

NOTE The SurfaceSRH keyword also is supported for metal–semiconductor interfaces, but the model it activates there is different from the one described in this section (see [Electric Boundary Conditions for Metals on page 240](#)).

Coupled Defect Level (CDL) Recombination

The steady state recombination rate for two coupled defect levels generalizes the familiar single-level SRH formula. An important feature of the model is a possibly increased field effect that may lead to large excess currents. The model is discussed in the literature [\[11\]](#).

Using CDL

The CDL recombination can be switched on using the keyword CDL in the Physics section:

```
Physics{ Recombination( CDL ...) ...}
```

The contributions R_1 and R_2 in [Eq. 361](#) can be plotted by using the keywords CDL1 and CDL2 in the Plot section. For the net rate and coupling term, $R - R_1 - R_2$, the keywords CDL and CDL3 must be specified.

CDL Model

The notation of the model parameters is illustrated in [Figure 30](#).

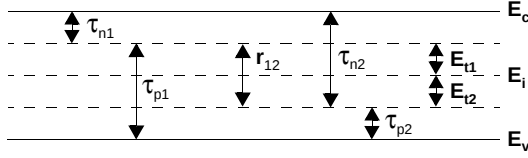


Figure 30 Notation for CDL recombination including all capture and emission processes

The CDL recombination rate is given by:

$$R = R_1 + R_2 + \left(\sqrt{R_{12}^2 - S_{12}} - R_{12} \right) \times \frac{\tau_{n1}\tau_{p2}(n + n_2)(p + p_1) - \tau_{n2}\tau_{p1}(n + n_1)(p + p_2)}{r_1 r_2} \quad (359)$$

with:

$$r_j = \tau_{nj}(p + p_j) + \tau_{pj}(n + n_j) \quad (360)$$

$$R_j = \frac{np - n_{i,eff}^2}{r_j}, \quad j = 1, 2 \quad (361)$$

$$R_{12} = \frac{r_1 r_2}{2r_{12}\tau_{n1}\tau_{n2}\tau_{p1}\tau_{p2}(1 - \varepsilon)} + \frac{\tau_{n1}(p + p_1) + \tau_{p2}(n + n_2)}{2\tau_{n1}\tau_{p2}(1 - \varepsilon)} + \frac{\varepsilon[\tau_{n2}(p + p_2) + \tau_{p1}(n + n_1)]}{2\tau_{n2}\tau_{p1}(1 - \varepsilon)} \quad (362)$$

$$S_{12} = \frac{1}{\tau_{n1}\tau_{p2}(1 - \varepsilon)} \left(1 - \frac{\tau_{n1}\tau_{p2}}{\tau_{n2}\tau_{p1}} \varepsilon \right) (np - n_{i,eff}^2) \quad (363)$$

$$\varepsilon = \exp\left(-\frac{|E_{t2} - E_{t1}|}{kT}\right) \quad (364)$$

where τ_{ni} and τ_{pi} denote the electron and hole lifetimes of the defect level i . The coupling parameter between the defect levels is called r_{12} (keyword `TrapTrapRate` in the parameter file). The carrier lifetimes are calculated analogously to the carrier lifetimes in the SRH model (see [Shockley–Read–Hall Recombination on page 359](#)). The number of parameters is doubled compared to the SRH model, and they are changeable in the parameter set CDL.

The quantities n_2 and p_2 are the corresponding quantities of n_1 and p_1 for the second defect level. They are defined analogously to [Eq. 323, p. 359](#) and [Eq. 324, p. 359](#).

For Fermi statistics and quantization, the equations are modified in the same manner as for SRH recombination (see [Eq. 326](#), p. 360).

Radiative Recombination

Using Radiative Recombination

The radiative recombination model is activated in the `Physics` section by the keyword `Radiative`:

```
Physics {  
    Recombination (Radiative)  
}
```

It can also be switched on or off by using the notation `+Radiative` or `-Radiative`:

```
Physics (Region = "gate") {  
    Recombination (+Radiative)  
}  
  
Physics (Material = "AlGaAs") {  
    Recombination (-Radiative)  
}
```

The value of the radiative recombination rate is plotted as follows:

```
Plot {  
    RadiativeRecombination  
}
```

The value of the parameter C can be changed in the parameter file:

```
RadiativeRecombination {  
    C = 2.5e-10  
}
```

Radiative Model

The radiative (direct) recombination model expresses the recombination rate as:

$$R_{\text{net}} = C \cdot (np - n_{i,\text{eff}}^2) \quad (365)$$

16: Generation-Recombination

Auger Recombination

By default, Sentaurus Device selects $C = 2 \times 10^{-10} \text{ cm}^3 \text{ s}^{-1}$ for GaAs and $C = 0 \text{ cm}^3 \text{ s}^{-1}$ for other materials. For Fermi statistics and quantization, the equations are modified in the same manner than for SRH recombination (see [Eq. 326, p. 360](#)).

Auger Recombination

The rate of band-to-band Auger recombination R_{net}^A is given by:

$$R_{\text{net}}^A = (C_n n + C_p p)(np - n_{i,\text{eff}}^2) \quad (366)$$

with temperature-dependent Auger coefficients [\[12\]\[13\]\[14\]](#):

$$C_n(T) = \left(A_{A,n} + B_{A,n} \left(\frac{T}{T_0} \right) + C_{A,n} \left(\frac{T}{T_0} \right)^2 \right) \left[1 + H_n \exp \left(-\frac{n}{N_{0,n}} \right) \right] \quad (367)$$

$$C_p(T) = \left(A_{A,p} + B_{A,p} \left(\frac{T}{T_0} \right) + C_{A,p} \left(\frac{T}{T_0} \right)^2 \right) \left[1 + H_p \exp \left(-\frac{p}{N_{0,p}} \right) \right] \quad (368)$$

where $T_0 = 300 \text{ K}$. There is experimental evidence for a decrease of the Auger coefficients at high injection levels [\[14\]](#). This effect is explained as resulting from exciton decay: at lower carrier densities, excitons, which are loosely bound electron-hole pairs, increase the probability for Auger recombination. Excitons decay at high carrier densities, resulting in a decrease of recombination. This effect is modeled by the terms $1 + H \exp(-n/N_0)$ in [Eq. 367](#) and [Eq. 368](#).

Auger recombination is typically important at high carrier densities. Therefore, this injection dependence will only be seen in devices where extrinsic recombination effects are extremely low, such as high-efficiency silicon solar cells. The injection dependence of the Auger coefficient can be deactivated by setting H to zero in the parameter file.

For Fermi statistics and quantization, the equations are modified in the same manner as for SRH recombination (see [Eq. 326, p. 360](#)). Default values for silicon are listed in [Table 65](#).

Table 65 Default coefficients of Auger recombination model

Symbol	$A_A [\text{cm}^6 \text{ s}^{-1}]$	$B_A [\text{cm}^6 \text{ s}^{-1}]$	$C_A [\text{cm}^6 \text{ s}^{-1}]$	$H [1]$	$N_0 [\text{cm}^{-3}]$
Parameter name	A	B	C	H	N0
Electrons	6.7×10^{-32}	2.45×10^{-31}	-2.2×10^{-32}	3.46667	1×10^{18}
Holes	7.2×10^{-32}	4.5×10^{-33}	2.63×10^{-32}	8.25688	1×10^{18}

Auger recombination is activated with the argument `Auger` in the `Recombination` statement:

```
Physics{ Recombination( Auger ...) ...}
```

By default, Sentaurus Device uses [Eq. 366](#) only if R_{net}^A is positive and replaces the value by zero if R_{net}^A is negative. To use [Eq. 366](#) for negative values (that is, to allow for Auger generation of electron–hole pairs), use the `WithGeneration` option to the `Auger` keyword:

```
Physics { Recombination( Auger(WithGeneration) ...) ...}
```

The Auger parameters are accessible in the parameter set `Auger`.

Avalanche Generation

Electron–hole pair production due to avalanche generation (impact ionization) requires a certain threshold field strength and the possibility of acceleration, that is, wide space charge regions. If the width of a space charge region is greater than the mean free path between two ionizing impacts, charge multiplication occurs, which can cause electrical breakdown. The reciprocal of the mean free path is called the ionization coefficient α . With these coefficients for electrons and holes, the generation rate can be expressed as:

$$G_{ii} = \alpha_n n v_n + \alpha_p p v_p \quad (369)$$

Sentaurus Device implements six models of the threshold behavior of the ionization coefficients: van Overstraeten–de Man, Okuto–Crowell, Lackner, University of Bologna, the new University of Bologna, and Hatakeyama.

Sentaurus Device allows you to select the appropriate driving force for the simulation, that is, the method used to compute the accelerating field.

Using Avalanche Generation

Avalanche generation is switched on by using the keyword `Avalanche` in the `Recombination` statement in the `Physics` section. The models are selected by using the keywords `vanOverstraeten`, `Okuto`, `Lackner`, `UniBo`, `UniBo2`, and `Hatakeyama`. The default model is `vanOverstraeten`. For example:

```
Physics{
  Recombination(eAvalanche(CarrierTempDrive) hAvalanche(Okuto)...
}
```

16: Generation–Recombination

Avalanche Generation

selects a driving force derived from electron temperature for electron impact ionization process and the default driving force based on GradQuasiFermi with the Okuto–Crowell model for holes.

To plot the avalanche generation rate, specify `AvalancheGeneration` in the `Plot` section. To plot either of the two terms on the right-hand side of [Eq. 369](#) separately, specify `eAvalanche` or `hAvalanche`. To plot α_n or α_p , specify `eAlphaAvalanche` or `hAlphaAvalanche`, respectively.

van Overstraeten – de Man Model

This model is based on the Chynoweth law [\[15\]](#):

$$\alpha(F_{\text{ava}}) = \gamma a \exp\left(-\frac{\gamma b}{F_{\text{ava}}}\right) \quad (370)$$

with:

$$\gamma = \frac{\tanh\left(\frac{\hbar\omega_{\text{op}}}{2kT_0}\right)}{\tanh\left(\frac{\hbar\omega_{\text{op}}}{2kT}\right)} \quad (371)$$

The factor γ with the optical phonon energy $\hbar\omega_{\text{op}}$ expresses the temperature dependence of the phonon gas against which carriers are accelerated. The coefficients a , b , and $\hbar\omega_{\text{op}}$, as measured by van Overstraeten and de Man [\[16\]](#), are applicable over the range of fields $1.75 \times 10^5 \text{ Vcm}^{-1}$ to $6 \times 10^5 \text{ Vcm}^{-1}$ and are listed in [Table 66 on page 379](#).

Two sets of coefficients a and b are used for high and low ranges of electric field. The values $a(\text{low})$, $b(\text{low})$ apply in the low field range $1.75 \times 10^5 \text{ Vcm}^{-1}$ to E_0 and the values $a(\text{high})$, $b(\text{high})$ apply in the high field range E_0 to $6 \times 10^5 \text{ Vcm}^{-1}$. For electrons, the impact ionization coefficients are by default the same in both field ranges.

You can adjust the coefficient values in the Sentaurus Device parameter set `vanOverstraetendeMan`.

Table 66 Coefficients for van Overstraeten–de Man model (Eq. 370)

Symbol	Parameter name	Electrons	Holes	Valid range of electric field	Unit
a	a(low)	7.03×10^5	1.582×10^6	$1.75 \times 10^5 \text{ Vcm}^{-1}$ to E_0	cm^{-1}
	a(high)	7.03×10^5	6.71×10^5	E_0 to $6 \times 10^5 \text{ Vcm}^{-1}$	
b	b(low)	1.231×10^6	2.036×10^6	$1.75 \times 10^5 \text{ Vcm}^{-1}$ to E_0	V/cm
	b(high)	1.231×10^6	1.693×10^6	E_0 to $6 \times 10^5 \text{ Vcm}^{-1}$	
E_0	E0	4×10^5	4×10^5		V/cm
$\hbar\omega_{\text{op}}$	hbarOmega	0.063	0.063		eV

Okuto–Crowell Model

Okuto and Crowell [17] suggested the empirical model:

$$\alpha(F_{\text{ava}}) = a \cdot \left(1 + c(T - T_0)\right) F_{\text{ava}}^\gamma \exp\left[-\left(\frac{b[1 + d(T - T_0)]}{F_{\text{ava}}}\right)^\delta\right] \quad (372)$$

where $T_0 = 300 \text{ K}$ and the user-adjustable coefficients are listed in Table 67 with their default values for silicon. These values are applicable to the range of electric field 10^5 Vcm^{-1} to 10^6 Vcm^{-1} . You can adjust the parameters in the parameter set Okuto.

Table 67 Coefficients for Okuto–Crowell model (Eq. 372)

Symbol	Parameter name	Electrons	Holes	Unit
a	a	0.426	0.243	V^{-1}
b	b	4.81×10^5	6.53×10^5	V/cm
c	c	3.05×10^{-4}	5.35×10^{-4}	K^{-1}
d	d	6.86×10^{-4}	5.67×10^{-4}	K^{-1}
γ	gamma	1	1	1
δ	delta	2	2	1

Lackner Model

Lackner [18] derived a pseudo-local ionization rate in the form of a modification to the Chynoweth law, assuming stationary conditions. The temperature-dependent factor γ was introduced to the original model:

$$\alpha_v(F_{\text{ava}}) = \frac{\gamma a_v}{Z} \exp\left(-\frac{\gamma b_v}{F_{\text{ava}}}\right) \text{ where } v = n, p \quad (373)$$

with:

$$Z = 1 + \frac{\gamma b_n}{F_{\text{ava}}} \exp\left(-\frac{\gamma b_n}{F_{\text{ava}}}\right) + \frac{\gamma b_p}{F_{\text{ava}}} \exp\left(-\frac{\gamma b_p}{F_{\text{ava}}}\right) \quad (374)$$

and:

$$\gamma = \frac{\tanh\left(\frac{\hbar\omega_{\text{op}}}{2kT_0}\right)}{\tanh\left(\frac{\hbar\omega_{\text{op}}}{2kT}\right)} \quad (375)$$

The default values of the coefficients a , b , and $\hbar\omega_{\text{op}}$ are applicable in silicon for the range of the electric field from 10^5 Vcm^{-1} to 10^6 Vcm^{-1} .

You can adjust the coefficients in the parameter set Lackner.

Table 68 Coefficients for Lackner model (Eq. 373)

Symbol	Parameter name	Electrons	Holes	Unit
a	a	1.316×10^6	1.818×10^6	cm^{-1}
b	b	1.474×10^6	2.036×10^6	V/cm
$\hbar\omega_{\text{op}}$	hbarOmega	0.063	0.063	eV

University of Bologna Impact Ionization Model

The University of Bologna impact ionization model was developed for an extended temperature range between 25°C and 400°C (see also [New University of Bologna Impact Ionization Model on page 382](#) for an updated version of this model). It is based on impact ionization data generated by the Boltzmann solver HARM [19]. It covers a wide range of electric fields (50 kVcm^{-1} to 600 kVcm^{-1}) and temperatures (300 K to 700 K). It is calibrated against impact ionization measurements [20][21] in the whole temperature range.

The model reads:

$$\alpha(F_{\text{ava}}, T) = \frac{F_{\text{ava}}}{a(T) + b(T) \exp\left[\frac{d(T)}{F_{\text{ava}} + c(T)}\right]} \quad (376)$$

The temperature dependence of the model parameters, determined by fitting experimental data, reads (for electrons):

$$a(T) = a_0 + a_1 t^{a_2} \quad b(T) = b_0 \quad c(T) = c_0 + c_1 t + c_2 t^2 \quad d(T) = d_0 + d_1 t + d_2 t^2 \quad (377)$$

and for holes:

$$a(T) = a_0 + a_1 t \quad b(T) = b_0 \exp[b_1 t] \quad c(T) = c_0 t^{c_1} \quad d(T) = d_0 + d_1 t + d_2 t^2 \quad (378)$$

where $t = T/1\text{K}$.

[Table 69](#) lists the model parameters. The model parameters are accessible in the parameter set UniBo.

Table 69 Coefficients for University of Bologna impact ionization model

Symbol	Parameter name	Electrons	Holes	Unit
a_0	ha0	4.3383	2.376	V
a_1	ha1	-2.42×10^{-12}	1.033×10^{-2}	V
a_2	ha2	4.1233	0	1
b_0	hb0	0.235	0.17714	V
b_1	hb1	0	-2.178×10^{-3}	1
c_0	hc0	1.6831×10^4	9.47×10^{-3}	Vcm^{-1}
c_1	hc1	4.3796	2.4924	Vcm^{-1} , 1
c_2	hc2	0.13005	0	Vcm^{-1} , 1
d_0	hd0	1.2337×10^6	1.4043×10^6	Vcm^{-1}
d_1	hd1	1.2039×10^3	2.9744×10^3	Vcm^{-1}
d_2	hd2	0.56703	1.4829	Vcm^{-1}

16: Generation-Recombination

Avalanche Generation

NOTE When the University of Bologna impact ionization model is selected for both carriers, that is, `Recombination(Avalanche(UniBo))`, Sentaurus Device also enables Auger generation (see [Auger Recombination on page 376](#)). Specify Auger (-WithGeneration) to disable this generation term if necessary.

New University of Bologna Impact Ionization Model

The impact ionization model described in [University of Bologna Impact Ionization Model on page 380](#) was developed further in [\[22\]\[23\]\[24\]](#) to cover an extended temperature range between 25°C and 500°C (773 K). It is based on impact ionization data generated by the Boltzmann solver HARM [\[19\]](#) and is calibrated against specially designed impact ionization measurements [\[20\]\[21\]](#). It covers a wide range of electric fields. The model reads:

$$\alpha(F_{\text{ava}}, T) = \frac{F_{\text{ava}}}{a(T) + b(T) \exp\left[\frac{d(T)}{F_{\text{ava}} + c(T)}\right]} \quad (379)$$

where the coefficients a , b , c , and d are polynomials of T :

$$a(T) = \sum_{k=0}^3 a_k \left(\frac{T}{1\text{K}}\right)^k \quad (380)$$

$$b(T) = \sum_{k=0}^{10} b_k \left(\frac{T}{1\text{K}}\right)^k \quad (381)$$

$$c(T) = \sum_{k=0}^3 c_k \left(\frac{T}{1\text{K}}\right)^k \quad (382)$$

$$d(T) = \sum_{k=0}^3 d_k \left(\frac{T}{1\text{K}}\right)^k \quad (383)$$

Table 70 lists the model parameters.

Table 70 Coefficients for UniBo2 impact ionization model

Symbol	Electrons		Holes		Unit
	Parameter Name	Value	Parameter Name	Value	
a_0	a0_e	4.65403	a0_h	2.26018	V
a_1	a1_e	-8.76031×10^{-3}	a1_h	0.0134001	V
a_2	a2_e	1.34037×10^{-5}	a2_h	-5.87724×10^{-6}	V
a_3	a3_e	-2.75108×10^{-9}	a3_h	-1.14021×10^{-9}	V
b_0	b0_e	-0.128302	b0_h	0.058547	V
b_1	b1_e	4.45552×10^{-3}	b1_h	-1.95755×10^{-4}	V
b_2	b2_e	-1.0866×10^{-5}	b2_h	2.44357×10^{-7}	V
b_3	b3_e	9.23119×10^{-9}	b3_h	-1.33202×10^{-10}	V
b_4	b4_e	-1.82482×10^{-12}	b4_h	2.68082×10^{-14}	V
b_5	b5_e	-4.82689×10^{-15}	b5_h	0	V
b_6	b6_e	1.09402×10^{-17}	b6_h	0	V
b_7	b7_e	-1.24961×10^{-20}	b7_h	0	V
b_8	b8_e	7.55584×10^{-24}	b8_h	0	V
b_9	b9_e	-2.28615×10^{-27}	b9_h	0	V
b_{10}	b10_e	2.73344×10^{-31}	b10_h	0	V
c_0	c0_e	7.76221×10^3	c0_h	1.95399×10^4	Vcm^{-1}
c_1	c1_e	25.18888	c1_h	-104.441	Vcm^{-1}
c_2	c2_e	-1.37417×10^{-3}	c2_h	0.498768	Vcm^{-1}
c_3	c3_e	1.59525×10^{-4}	c3_h	0	Vcm^{-1}
d_0	d0_e	7.10481×10^5	d0_h	2.07712×10^6	Vcm^{-1}
d_1	d1_e	3.98594×10^3	d1_h	993.153	Vcm^{-1}
d_2	d2_e	-7.19956	d2_h	7.77769	Vcm^{-1}
d_3	d3_e	6.96431×10^{-3}	d3_h	0	Vcm^{-1}

16: Generation-Recombination

Avalanche Generation

The model parameters can be specified in the Sentaurus Device parameter file:

```
UniBo2 {  
    a0_e = 4.65403  
    a0_h = 2.26018  
    ...  
}
```

NOTE The name of the UniBo2 model is case sensitive. Both in the command file and parameter file of Sentaurus Device, the exact spelling UniBo2 must be used.

Hatakeyama Avalanche Model

The Hatakeyama avalanche model has been proposed [29] to describe the anisotropic behavior in 4H-SiC power devices. The impact ionization coefficient α is obtained according to the Chynoweth law [15]:

$$\alpha = \gamma a e^{-\frac{\gamma b}{F}} \quad (384)$$

with:

$$\gamma = \frac{\tanh\left(\frac{\hbar\omega_{op}}{2kT_0}\right)}{\tanh\left(\frac{\hbar\omega_{op}}{2kT}\right)} \quad (385)$$

The coefficients a and b are computed dependent on the direction of the driving force $\vec{F}$. For this purpose, the driving force F is decomposed into a component F_{0001} parallel to the anisotropic axis 0001 and a remaining component $F_{11\bar{2}0}$ perpendicular to the anisotropic axis.

The norm $F = \|\vec{F}\|_2$ satisfies the equation:

$$F^2 = F_{0001}^2 + F_{11\bar{2}0}^2 \quad (386)$$

Based on the two projections F_{0001} and $F_{11\bar{2}0}$, the coefficients a and b then are computed as follows:

$$B = \frac{F}{\sqrt{\left(\frac{F_{11\bar{2}0}}{b_{11\bar{2}0}}\right)^2 + \left(\frac{F_{0001}}{b_{0001}}\right)^2}} \quad (387)$$

$$a = a_{11\bar{2}0} \left(\frac{BF_{11\bar{2}0}}{b_{11\bar{2}0}F} \right)^2 a_{0001} \left(\frac{BF_{0001}}{b_{0001}F} \right)^2 \quad (388)$$

$$A = \log \frac{a_{0001}}{a_{11\bar{2}0}} \quad (389)$$

$$b = B \sqrt{1 - \theta A^2 \left(\frac{BF_{11\bar{2}0}F_{0001}}{Fb_{11\bar{2}0}b_{0001}} \right)^2} \quad (390)$$

With the default $\theta = 1$, the coefficient b may become undefined for large values of the driving force F (the argument of the square root in Eq. 390 becomes negative). This can only happen if:

$$F > \frac{2\min(b_{0001}, b_{11\bar{2}0})}{\left| \log \frac{a_{0001}}{a_{11\bar{2}0}} \right|} \quad (391)$$

By setting $\theta = 0$, you have $b = B$, and the coefficients a and b only depend on the direction of the driving force F , but not its magnitude.

Table 71 shows the default parameters for 4H-SiC.

Table 71 4H-SiC coefficients for Hatakeyama model

Symbol	Parameter name	Electrons	Holes	Unit
a_{0001}	a_0001	1.76×10^8	3.41×10^8	cm^{-1}
$a_{11\bar{2}0}$	a_1120	2.10×10^7	2.96×10^7	cm^{-1}
b_{0001}	b_0001	3.30×10^7	2.50×10^7	V/cm
$b_{11\bar{2}0}$	b_1120	1.70×10^7	1.60×10^7	V/cm
$\hbar\omega_{\text{op}}$	hbarOmega	0.19	0.19	eV
θ	theta	1	1	1

Driving Force

In contrast to other avalanche models in Sentaurus Device (see [Driving Force on page 387](#)), which only use the magnitude $F_{\rightarrow}$ of the driving force, the Hatakeyama avalanche model requires a vectorial driving force $\vec{F}$ to compute the coefficients a and b . Depending on the driving force model, the following expressions are used:

- **ElectricField**: The driving force $\vec{F}$ is defined as the straight electric field $\vec{E}$.
- **Eparallel**: The driving force $\vec{F}$ is given by (where $\vec{j}$ is the electron or hole current density):

$$\vec{F} = \frac{\vec{j} \cdot \vec{j}^T \vec{E}}{\|\vec{j}\|} \quad (392)$$

- **GradQuasiFermi**: The driving force $\vec{F}$ is given by (where Φ is the electron or hole quasi-Fermi potential):

$$\vec{F} = \nabla \Phi \quad (393)$$

- **CarrierTempDrive**: The driving force $\vec{F}$ is given by (where E^{eff} is the effective field obtained from the electron or hole carrier temperature):

$$\vec{F} = \frac{\vec{j} \cdot \vec{j}^T E^{\text{eff}}}{\|\vec{j}\|} \quad (394)$$

See also [Avalanche Generation with Hydrodynamic Transport on page 387](#).

Anisotropic Coordinate System

In a 2D simulation, Sentaurus Device assumes that the y-axis in the crystal system is the anisotropic axis 0001, and the x-axis in the crystal system is the isotropic axis 11 $\bar{2}$ 0. In a 3D simulation, the z-axis in the crystal system is the anisotropic axis, and the x-axis and y-axis span the isotropic plane.

The simulation coordinate system relative to the crystal system is defined by the X and Y vectors in the `LatticeParameters` section of the parameter file (see [Crystal Reference System on page 669](#) and [Coordinate Systems on page 738](#)).

Usage

The Hatakeyama avalanche model uses a special-purpose interpolation formula to compute the coefficients a and b based on the direction of the driving force. This formula differs from the standard approach as described in [Anisotropic Avalanche Generation on page 675](#).

NOTE The Hatakeyama avalanche model must not be used together with an Aniso(Avalanche) specification in the Physics section.

Driving Force

In Sentaurus Device, the driving force F_{ava} for impact ionization can be computed as the component of the electrostatic field in the direction of the current, $F_{\text{ava}} = F \cdot \hat{J}_{n,p}$, (keyword Eparallel), or the value of the gradient of the quasi-Fermi level, $F_{\text{ava}} = |\nabla \Phi_{n,p}|$, (keyword GradQuasiFermi). For these two possibilities, F_{ava} is affected by the keyword ParallelToInterfaceInBoundaryLayer (see [Field Correction Close to Interfaces on page 349](#)). For hydrodynamic simulations, F_{ava} can be computed from the carrier temperature (keyword CarrierTempDrive, see [Avalanche Generation with Hydrodynamic Transport on page 387](#)). The default model is GradQuasiFermi and, only for hydrodynamic simulations, it is CarrierTempDrive. See [Table 184 on page 1267](#) for a summary of keywords.

The option ElectricField is used to perform breakdown simulations using the ‘ionization integral’ method (see [Approximate Breakdown Analysis: Poisson Equation Approach on page 389](#)).

Avalanche Generation with Hydrodynamic Transport

If the hydrodynamic transport model is used, the default driving force F_{ava} equals an effective field E^{eff} obtained from the carrier temperature. The usual conversion of local carrier temperatures to effective fields E^{eff} is described by the algebraic equations:

$$n\mu_n(E_n^{\text{eff}})^2 = n\frac{3kT_n - T}{2q\lambda_n\tau_{en}} \quad (395)$$

$$p\mu_p(E_p^{\text{eff}})^2 = p\frac{3kT_p - T}{2q\lambda_p\tau_{ep}} \quad (396)$$

which are obtained from the energy conservation equation under time-independent, homogeneous conditions. [Eq. 395](#) and [Eq. 396](#) have been simplified in Sentaurus Device by using the assumption $\mu_n E_n^{\text{eff}} = v_{\text{sat},n}$ and $\mu_p E_p^{\text{eff}} = v_{\text{sat},p}$. This assumption is true for high values of the electric field. However, for low field, the impact ionization rate is negligibly small.

The parameters λ_n and λ_p are fitting coefficients (default value 1) and their values can be changed in the parameter set AvalancheFactors, where they are represented as n_l_f and p_l_f, respectively.

16: Generation–Recombination

Avalanche Generation

The conventional conversion formulas [Eq. 395](#) and [Eq. 396](#) can be activated by specifying parameters $\Upsilon_n = 0$, $\Upsilon_p = 0$ in the same `AvalancheFactors` section.

The simplified conversion formulas discussed above predict a linear dependence of effective electric field on temperature for high values of carrier temperature. For silicon, however, Monte Carlo simulations do not confirm this behavior. To obtain a better agreement with Monte Carlo data, additional heat sinks must be taken into account by the inclusion of an additional term in the equations for E^{eff} .

Such heat sinks arise from nonelastic processes, such as the impact ionization itself. Sentaurus Device supports the following model to account for these heat sinks:

$$nv_{\text{sat},n}E_n^{\text{eff}} = n\frac{3kT_n - T}{2q\lambda_n\tau_{en}} + \frac{\Upsilon_n}{q}(E_g + \delta_n kT_n)\alpha_n nv_{\text{sat},n} \quad (397)$$

A similar equation E_p^{eff} is used to determine E_p^{eff} . To activate this model, set the parameters Υ_n and Υ_p to 1. This is the default for silicon, where the generalized conversion formula [Eq. 397](#) gives good agreement with Monte Carlo data for $\delta_n = \delta_p = 3/2$. For all other materials, the default of the parameters Υ_n and Υ_p is 0.

Table 72 Hydrodynamic avalanche model: Default parameters

Symbol	Parameter name	Default value
λ_n	n_l_f	1
λ_p	p_l_f	1
Υ_n	n_gamma	1
Υ_p	p_gamma	1
δ_n	n_delta	1.5
δ_p	p_delta	1.5

NOTE This procedure ensures that the same results are obtained as with the conventional local field–dependent models in the bulk case. Conversely, the temperature-dependent impact ionization model usually gives much more accurate predictions for the substrate current in short-channel MOS transistors.

Approximate Breakdown Analysis: Poisson Equation Approach

Junction breakdown due to avalanche generation is simulated by inspecting the ionization integrals:

$$I_n = \int_0^W \alpha_n(x) e^{-\int_x^W (\alpha_n(x') - \alpha_p(x')) dx'} dx \quad (398)$$

$$I_p = \int_0^W \alpha_p(x) e^{-\int_0^x (\alpha_p(x') - \alpha_n(x')) dx'} dx \quad (399)$$

where α_n , α_p are the ionization coefficients for electrons and holes, respectively, and W is the width of the depletion zone. The integrations are performed along field lines through the depletion zone. Avalanche breakdown occurs if an ionization integral equals one. [Eq. 398](#) describes electron injection (electron primary current) and [Eq. 399](#) describes hole injection. Since these breakdown criteria do not depend on current densities, a breakdown analysis can be performed by computing only the Poisson equation and ionization integrals under the assumption of constant quasi-Fermi levels in the depletion region.

Using Breakdown Analysis

To enable breakdown analysis, Sentaurus Device provides the driving force `ElectricField`, which can be computed even for constant quasi-Fermi levels. This driving force is less physical than the others available in Sentaurus Device. Therefore, Synopsys discourages using it for any purpose other than approximate breakdown analysis.

`ComputeIonizationIntegrals` in the `Math` section switches on the computation of the ionization integrals for ionization paths crossing the local field maxima in the semiconductor. By default, Sentaurus Device reports only the path with the largest I_{mean} . With the addition of the keyword `WriteAll`, information about the ionization integrals for all computed paths is written to the log file.

The `Math` keyword `BreakAtIonIntegral` is used to terminate the quasistationary simulation when the largest ionization integral is greater than one.

16: Generation-Recombination

Approximate Breakdown Analysis: Poisson Equation Approach

The complete syntax of this keyword is `BreakAtIonIntegral(<number> <value>)` where a quasistationary simulation finishes if the **number** ionization integral is greater than **value**, and the ionization integrals are ordered with respect to decreasing value:

```
Math { BreakAtIonIntegral }
```

Three optional keywords in the `Plot` section specify the values of the corresponding ionization integrals that are stored along the breakdown paths:

```
Plot { eIonIntegral | hIonIntegral | MeanIonIntegral }
```

These ion integrals can be visualized by using Tecplot SV. A typical command file of Sentaurs Device is:

```
Electrode {
  { name="anode" Voltage=0 }
  { name="cathode" Voltage=600 }
}
File {
  grid      = "@grid@"
  doping    = "@doping@"
  current   = "@plot@"
  output    = "@log@"
  plot      = "@data@"
}
Physics {
  Mobility (DopingDependence HighFieldSaturation)
  Recombination(SRH Auger Avalanche(ElectricField))
}
Solve {
  Quasistationary(
    InitialStep=0.02 MaxStep=0.01 MinStep=0.01
    Goal {name=cathode voltage=1000}
  )
  { poisson }
}
Math {
  Iterations=100
  BreakAtIonIntegral
  ComputeIonizationIntegrals(WriteAll)
}
Plot {
  eIonIntegral hIonIntegral MeanIonIntegral
  eDensity hDensity
  ElectricField/Vector
  eAlphaAvalanche hAlphaAvalanche
}
```

Band-to-Band Tunneling Models

Sentaurus Device provides several band-to-band tunneling models:

- Schenk model (see [Schenk Model on page 392](#)).
- Hurkx model (see [Hurkx Band-to-Band Model on page 394](#)).
- A family of simple models (see [Simple Band-to-Band Models on page 393](#)).
- Nonlocal path model (see [Dynamic Nonlocal Path Band-to-Band Model on page 395](#)).

The Schenk, Hurkx, and simple models use a common approach to suppress artificial band-to-band tunneling near insulator interfaces and for rapidly varying fields (see [Tunneling Near Interfaces and Equilibrium Regions on page 395](#)).

Using Band-to-Band Tunneling

Band-to-band tunneling is controlled by the Band2Band option of Recombination:

```
Recombination( ...
  Band2Band (
    Model = Schenk | Hurkx | E1 | E1_5 | E2 | NonlocalPath1 | NonlocalPath2 |
    NonlocalPath3
    DensityCorrection = Local | None
    InterfaceReflection | -InterfaceReflection
    FranzDispersion | -FranzDispersion
  )
)
```

Model determines the model:

- Schenk selects the Schenk model.
- Hurkx selects the Hurkx model.
- E1, E1_5, and E2 select one of the simple models.
- NonlocalPath1, NonlocalPath2, and NonlocalPath3 select the nonlocal path model with the number of tunneling processes equal to 1, 2, and 3, respectively.

DensityCorrection is used by the Schenk, Hurkx, and simple models, and its default value is None. A value of Local activates a local-density correction (see [Schenk Density Correction on page 393](#)).

InterfaceReflection is used by the nonlocal path models, and this option is switched on by default. This option allows a tunneling path reflected at semiconductor–insulator interfaces.

When it is switched off, band-to-band tunneling is neglected when the tunneling path encounters semiconductor-insulator interfaces.

FranzDispersion is used by the nonlocal path models, and this option is switched off by default. When it is switched on, the Franz dispersion relation (Eq. 646, p. 618) instead of the Kane dispersion relation (Eq. 408, p. 397) for the imaginary wavevector is used in the direct tunneling process. The indirect tunneling process is not affected by this option.

For backward compatibility:

- Band2Band alone (without parameters) selects the Schenk model with local-density correction.
- Band2Band(Hurkx) selects the Hurkx model without density correction.
- Band2Band(E1), Band2Band(E1_5), and Band2Band(E2) select one of the simple models.

The parameters for all band-to-band models are available in the Band2BandTunneling parameter set. The parameters specific to individual models are described in the respective sections. The parameter MinField (specified in Vcm^{-1}) is used by the Schenk, Hurkx, and simple models for smoothing at small electric fields. A value of zero (the default) disables smoothing.

Schenk Model

Phonon-assisted band-to-band tunneling cannot be neglected in steep p-n junctions (with a doping level of $1 \times 10^{19} \text{cm}^{-3}$ or more on both sides) or in high normal electric fields of MOS structures. It must be switched on if the field, in some regions of the device, exceeds (approximately) $8 \times 10^5 \text{V/cm}$. In this case, defect-assisted tunneling (see [SRH Field Enhancement on page 363](#)) must also be switched on.

Band-to-band tunneling is modeled using the expression [25]:

$$R_{\text{net}}^{\text{bb}} = AF^{7/2} \frac{\tilde{n}\tilde{p} - n_{i,\text{eff}}^2}{(\tilde{n} + n_{i,\text{eff}})(\tilde{p} + n_{i,\text{eff}})} \left[\frac{(F_C^{\mp})^{-3/2} \exp\left(-\frac{F_C^{\mp}}{F}\right)}{\exp\left(\frac{\hbar\omega}{kT}\right) - 1} + \frac{(F_C^{\pm})^{-3/2} \exp\left(-\frac{F_C^{\pm}}{F}\right)}{1 - \exp\left(-\frac{\hbar\omega}{kT}\right)} \right] \quad (400)$$

where $\tilde{n}$ and $\tilde{p}$ equal n and p for DensityCorrection=None, and are given by Eq. 402 for DensityCorrection=Local. The critical field strengths read:

$$F_C^{\pm} = B(E_{g,\text{eff}} \pm \hbar\omega)^{3/2} \quad (401)$$

The upper sign in Eq. 400 refers to tunneling generation ($np < n_{i,\text{eff}}^2$) and the lower sign refers to recombination ($np > n_{i,\text{eff}}^2$). The quantity $\hbar\omega$ denotes the energy of the transverse acoustic phonon.

For Fermi statistics and quantization, Eq. 400 is modified in the same way as for SRH recombination (see Eq. 326, p. 360).

The parameters [25] are given in Table 73 and can be accessed in the parameter set Band2BandTunneling. The defaults were obtained assuming the field direction to be $\langle 111 \rangle$.

Table 73 Coefficients for band-to-band tunneling (Schenk model)

Symbol	Parameter name	Default value	Unit
A	A	8.977×10^{20}	$\text{cm}^{-1}\text{s}^{-1}\text{V}^{-2}$
B	B	2.14667×10^7	$\text{Vcm}^{-1}\text{eV}^{-3/2}$
$\hbar\omega$	hbarOmega	18.6	meV

Schenk Density Correction

The modified electron density reads:

$$\tilde{n} = n \left(\frac{n_{i,\text{eff}}}{N_C} \right)^{\frac{\gamma_n |\nabla E_{F,n}|}{F}} \quad (402)$$

and there is a similar relation for $\tilde{p}$. The parameters $\gamma_n = n/(n + n_{\text{ref}})$ and $\gamma_p = p/(p + p_{\text{ref}})$ work as discussed in Schenk TAT Density Correction on page 366. The reference densities n_{ref} and p_{ref} are specified (in cm^{-3}) by the DenCorRef parameter pair in the Band2BandTunneling parameter set. An additional parameter MinGradQF (specified in eVcm^{-1}) is available for smoothing at small values of the gradient for the Fermi potential.

Simple Band-to-Band Models

Sentaurus Device provides a family of simple band-to-band models. Compared to advanced models, the most striking weakness of the simple models is that they predict a nonzero generation rate even in equilibrium. A general expression for these models can be written for the generation term [26] as:

$$G^{\text{b2b}} = AF^P \exp\left(-\frac{B}{F}\right) \quad (403)$$

Depending on value of Model, P takes the values 1, 1.5, or 2.

[Table 74](#) lists the coefficients of models and their defaults. The coefficients A and B can be changed in the parameter set `Band2BandTunneling`.

Table 74 Coefficients for band-to-band tunneling (simple models)

Model	P	A	B
E1	1	$1.1 \times 10^{27} \text{ cm}^{-2} \text{ s}^{-1} \text{ V}^{-1}$	$21.3 \times 10^6 \text{ Vcm}^{-1}$
E1_5	1.5	$1.9 \times 10^{24} \text{ cm}^{-1.5} \text{ s}^{-1} \text{ V}^{-1.5}$	$21.9 \times 10^6 \text{ Vcm}^{-1}$
E2	2	$3.4 \times 10^{21} \text{ cm}^{-1} \text{ s}^{-1} \text{ V}^{-2}$	$22.6 \times 10^6 \text{ Vcm}^{-1}$

Hurkx Band-to-Band Model

Similar to the other band-to-band tunneling models, in the Hurkx model [\[27\]](#), the tunneling carriers are modeled by an additional generation-recombination process. Its contribution is expressed as:

$$R_{\text{net}}^{\text{bb}} = A \cdot D \cdot \left(\frac{F}{1 \text{ V/cm}} \right)^P \exp \left(- \frac{BE_g(T)^{3/2}}{E_g(300\text{K})^{3/2} F} \right) \quad (404)$$

where:

$$D = \frac{np - n_{i,\text{eff}}^2}{(n + n_{i,\text{eff}})(p + n_{i,\text{eff}})} (1 - |\alpha|) + \alpha \quad (405)$$

Here, specifying $\alpha = 0$ gives the original Hurkx model, whereas $\alpha = -1$ gives only generation ($D = -1$), and $\alpha = 1$ gives only recombination ($D = 1$). Therefore, if $D < 0$, it is a net carrier generation model. If $D > 0$, it is a recombination model. For Fermi statistics and quantization, [Eq. 405](#) is modified in the same way as for SRH recombination (see [Eq. 326](#)).

For `DensityCorrection=Local`, n and p in [Eq. 405](#) are replaced by $\tilde{n}$ and $\tilde{p}$ (see [Schenk Density Correction on page 393](#)).

The coefficients A (in $\text{cm}^{-3}\text{s}^{-1}$), B (in V/cm), P , and α can be specified in the `Band2BandTunneling` parameter set. By default, Sentaurus Device uses $\alpha = 0$ and the parameters from the E2 model (see [Table 74 on page 394](#)). Different values for the generation (A_{gen} , B_{gen} , P_{gen}) and recombination (A_{rec} , B_{rec} , P_{rec}) of carriers are supported. For example, to change the parameters to those used in [\[27\]](#), use:

```
Band2BandTunneling {
  Agen = 4e14 # [1/(cm3s)]
  Bgen = 1.9e7 # [V/cm]
  Pgen = 2.5 # [1]
  Arec = 4e14 # [1/(cm3s)]
```



```
Brec = 1.9e7 # [V/cm]
Prec = 2.5   # [1]
alpha = 0    # [1]
}
```

Tunneling Near Interfaces and Equilibrium Regions

Physically, band-to-band tunneling occurs over a certain tunneling distance. If the material properties or the electric field change significantly over this distance, [Eq. 400](#), [Eq. 403](#), and [Eq. 404](#) become inaccurate. In particular, near insulator interfaces, band-to-band tunneling vanishes, as no states to tunnel to are available in the insulator.

In some parts of the device (near equilibrium regions), it is possible that the electric field is large but changes rapidly, so that the actual tunneling distance (the distance over which the electrostatic potential change amounts to the band gap) is bigger and, therefore, tunneling is much smaller than expected from the local field alone.

To account for these effects, two additional control parameters are introduced in the `Band2BandTunneling` parameter set:

```
dDist = <value> # [cm]
dPot  = <value> # [V]
```

By default, both these parameters equal zero. Sentaurus Device disables band-to-band tunneling within a distance `dDist` from insulator interfaces. If `dPot` is nonzero (reasonable values for `dPot` are of the order of the band gap), Sentaurus Device disables band-to-band tunneling at each point where the change of the electrostatic potential in field direction within a distance of $dPot/F$ is smaller than $dPot/2$.

Dynamic Nonlocal Path Band-to-Band Model

Sufficient band-bending caused by electric fields or heterostructures can make electrons in the valence band valley (Γ -valley in the k -space), at a certain location, reach the conduction band valley (Γ -, X -, or L -valley in the k -space) at different locations using direct or phonon-assisted band-to-band tunneling process.

The present model implements the nonlocal generation of electrons and holes caused by direct and phonon-assisted band-to-band tunneling processes [\[28\]](#). In direct semiconductors such as GaAs and InAs, the direct tunneling process is usually dominant. On the other hand, the phonon-assisted tunneling process is dominant in indirect semiconductors such as Si and Ge. If the energy differences between the conduction band valleys are small, it is possible that both the direct and the phonon-assisted tunneling processes are important.

The generation rate is obtained from the nonlocal path integration, and electrons and holes are generated nonlocally at the ends of the tunneling path. As a result, the position-dependent electron and hole generation rates are different in the present model. **The model can be applied to heterostructure devices with abrupt and graded heterojunctions.**

The main difference between the present model and the band-to-band tunneling model based on the nonlocal mesh (see [Band-to-Band Contributions to Nonlocal Tunneling Current on page 621](#)) is that the tunneling path is determined dynamically based on the energy band profile rather than predefined by the nonlocal mesh. Therefore, the present model does not require user-specification of the nonlocal mesh.

The model dynamically searches for the tunneling path with the following assumptions:

- The tunneling path is a straight line with its direction opposite to the gradient of the valence band at the starting position.
- The tunneling energy is equal to the valence band energy at the starting position and is equal to the conduction band energy plus band offset at the ending position.
- **When the tunneling path encounters Neumann boundaries or semiconductor-insulator interfaces, it undergoes specular reflection.**

NOTE The present model is not suitable for AC or noise analysis as nonlocal derivative terms in the Jacobian matrix are not taken into account. The lack of the derivative terms can degrade convergence when the high-field saturation mobility model is switched on or the series resistance is defined at electrodes.

Band-to-Band Generation Rate

For a given tunneling path of length l that starts at $x = 0$ and ends at $x = l$, holes are generated at $x = 0$ and electrons are generated at $x = l$. The net hole recombination rate at $x = 0$ due to the direct band-to-band tunneling process R_{net}^d can be written as:

$$R_{\text{net}}^d = |\nabla E_V(0)| C_d \exp\left(-2 \int_0^l \kappa dx\right) \left[\left(\exp\left[\frac{\epsilon - E_{F,n}(l)}{kT(l)}\right] + 1 \right)^{-1} - \left(\exp\left[\frac{\epsilon - E_{F,p}(0)}{kT(0)}\right] + 1 \right)^{-1} \right] \quad (406)$$

$$C_d = \frac{g\pi}{36h} \left(\int_0^l \frac{dx}{\kappa} \right)^{-1} \left[1 - \exp\left(-k_m^2 \int_0^l \frac{dx}{\kappa} \right) \right] \quad (407)$$

where:

- h is Planck's constant.
- g is the degeneracy factor.
- $\varepsilon = E_V(0) = E_C(l) + \Delta_C(l)$ is the tunneling energy.
- Δ_C is the conduction band offset. Δ_C can be positive if the considered tunneling process involves the conduction band valley whose energy minimum is greater than the conduction band energy E_C .
- κ is the magnitude of the imaginary wavevector obtained from the Kane two-band dispersion relation [28]:

$$\kappa = \frac{1}{\hbar} \sqrt{m_r E_{g,\text{tun}} (1 - \alpha^2)} \quad (408)$$

$$\alpha = -\frac{m_0}{2m_r} + 2 \sqrt{\frac{m_0}{2m_r} \left(\frac{\varepsilon - E_V}{E_{g,\text{tun}}} - \frac{1}{2} \right) + \frac{m_0^2}{16m_r^2} + \frac{1}{4}} \quad (409)$$

$$\frac{1}{m_r} = \frac{1}{m_V} + \frac{1}{m_C} \quad (410)$$

$E_{g,\text{tun}} = E_{g,\text{eff}} + \Delta_C$ is the effective band gap including the band offset, and k_m is the maximum transverse momentum determined by the maximum valence-band energy ε_{max} and the minimum conduction-band energy ε_{min} :

$$k_m^2 = \min(k_{\text{vm}}^2, k_{\text{cm}}^2) \quad (411)$$

$$k_{\text{vm}}^2 = \frac{2m_V(\varepsilon_{\text{max}} - \varepsilon)}{\hbar^2} \quad (412)$$

$$k_{\text{cm}}^2 = \frac{2m_C(\varepsilon - \varepsilon_{\text{min}})}{\hbar^2} \quad (413)$$

In the Kane two-band dispersion relation, the effective mass in the conduction band m_C and the valence band m_V are related [28]:

$$\frac{1}{m_C} = \frac{1}{2m_r} + \frac{1}{m_0} \quad (414)$$

$$\frac{1}{m_V} = \frac{1}{2m_r} - \frac{1}{m_0} \quad (415)$$

The net hole recombination rate due to the phonon-assisted band-to-band tunneling process R_{net}^p can be written as:

$$R_{\text{net}}^p = |\nabla E_V(0)| C_p \exp \left(-2 \int_0^{x_0} \kappa_V dx - 2 \int_{x_0}^l \kappa_C dx \right) \left[\left(\exp \left[\frac{\varepsilon - E_{F,n}(l)}{kT(l)} \right] + 1 \right)^{-1} - \left(\exp \left[\frac{\varepsilon - E_{F,p}(0)}{kT(0)} \right] + 1 \right)^{-1} \right] \quad (416)$$

$$C_p = \int_0^l \frac{g(1 + 2N_{\text{op}}) D_{\text{op}}^2}{2^6 \pi^2 \rho \varepsilon_{\text{op}} E_{g,\text{tun}}} \sqrt{\frac{m_V m_C}{\hbar l \sqrt{2m_t E_{g,\text{tun}}}}} dx \left(\int_0^{x_0} \frac{dx}{\kappa_V} \right)^{-1} \left(\int_{x_0}^l \frac{dx}{\kappa_C} \right)^{-1} \left[1 - \exp \left(-k_{\text{vm}}^2 \int_0^{x_0} \frac{dx}{\kappa_V} \right) \right] \left[1 - \exp \left(-k_{\text{cm}}^2 \int_{x_0}^l \frac{dx}{\kappa_C} \right) \right] \quad (417)$$

where D_{op} , ε_{op} , and $N_{\text{op}} = [\exp(\varepsilon_{\text{op}}/kT) - 1]^{-1}$ are the deformation potential, energy, and number of optical phonons, respectively, ρ is the mass density, and κ_V and κ_C are the magnitude of the imaginary wavevectors from the Keldysh dispersion relation:

$$\kappa_V = \frac{1}{\hbar} \sqrt{2m_V |\varepsilon - E_V|} \Theta(\varepsilon - E_V) \quad (418)$$

$$\kappa_C = \frac{1}{\hbar} \sqrt{2m_C |E_C + \Delta_C - \varepsilon|} \Theta(E_C + \Delta_C - \varepsilon) \quad (419)$$

and x_0 is the location where $\kappa_V = \kappa_C$.

As Eq. 406, p. 396 and Eq. 416 are the extension of the results in [28] to arbitrary band profiles, these expressions are reduced to the well-known Kane and Keldysh models in the uniform electric-field limit [28]:

$$R_{\text{net}} = A \left(\frac{F}{F_0} \right)^P \exp \left(-\frac{B}{F} \right) \quad (420)$$

where $F_0 = 1\text{V/cm}$, $P = 2$ for the direct tunneling process, and $P = 2.5$ for the phonon-assisted tunneling process.

At $T = 300\text{ K}$ without the bandgap narrowing effect, the prefactor A and the exponential factor B for the direct tunneling process can be expressed by [28]:

$$A = \frac{g \pi m_r^{1/2} (qF_0)^2}{9 \hbar^2 [E_g(300\text{K}) + \Delta_C]^{1/2}} \quad (421)$$

$$B = \frac{\pi^2 m_r^{1/2} [E_g(300\text{K}) + \Delta_C]^{3/2}}{q \hbar} \quad (422)$$

For the phonon-assisted tunneling process, A and B can be expressed by [28]:

$$A = \frac{g(m_v m_c)^{3/2} (1 + 2N_{op}) D_{op}^2 (qF_0)^{5/2}}{2^{21/4} h^{5/2} m_r^{5/4} \rho \epsilon_{op} [E_g(300K) + \Delta_C]^{7/4}} \quad (423)$$

$$B = \frac{2^{7/2} \pi m_r^{1/2} [E_g(300K) + \Delta_C]^{3/2}}{3qh} \quad (424)$$

Using Nonlocal Path Band-to-Band Model

The parameters for the nonlocal path band-to-band model are available in the parameter set `Band2BandTunneling`. To make the parameter set of the model consistent with the existing band-to-band tunneling models, the prefactor A and the exponential factor B are chosen as the input parameters. In addition, the conduction band offset Δ_C , phonon energy ϵ_{op} , and the effective mass ratio m_v/m_c can be specified.

Specifying $\epsilon_{op} = 0$ selects the direct tunneling process, and parameters A , B , and Δ_C determine g and m_r from Eq. 421, p. 398 and Eq. 422, p. 398.

When $\epsilon_{op} = 0$ and the option `FranzDispersion` is switched on in the `Band2Band` option of the command file, the magnitude of the imaginary wavevector is obtained from the Franz two-band dispersion relation (Eq. 646, p. 618) instead of the Kane two-band dispersion relation (Eq. 408, p. 397). In this case, g and m_r are still obtained from Eq. 421 and Eq. 422. Then, Eq. 410, p. 397 and m_v/m_c determine m_v and m_c .

Specifying $\epsilon_{op} > 0$ selects the phonon-assisted tunneling process, and parameters A , B , Δ_C , ϵ_{op} , and m_v/m_c determine gD_{op}^2/ρ , m_v , and m_c from Eq. 423 and Eq. 424. When $m_v/m_c = 0$, m_v and m_c are determined from Eq. 414, p. 397 and Eq. 415, p. 397.

Up to three different tunneling paths can be specified in the parameter file, and they are activated by setting `Model=NonlocalPath1 | NonlocalPath2 | NonlocalPath3` in the `Band2Band` option of the command file. `NonlocalPath1` selects the first tunneling path, `NonlocalPath2` selects the first and second tunneling paths, and `NonlocalPath3` selects all three tunneling paths.

NOTE Each tunneling path must have a consistent tunneling process (either direct or phonon-assisted tunneling process) in different regions. For example, specifying `Ppath1=0` (direct tunneling) in silicon regions and specifying `Ppath1` greater than zero (phonon-assisted tunneling) in polysilicon regions will induce an error message.

16: Generation-Recombination

Band-to-Band Tunneling Models

MaxTunnelLength in the parameter file specifies the maximum length of the tunneling path. If the length reaches MaxTunnelLength before a valid tunneling path is found, the band-to-band tunneling is neglected.

In the parameter file, eQuantumPotentialFactor and hQuantumPotentialFactor in the Band2BandTunneling parameter set specify the prefactors for the electron and hole quantum potentials that can be added to the effective band gap for tunneling. By default, they are zero such that the quantum potentials are neglected in the computation of the generation rate. When they are nonzero, the quantum potentials multiplied by the corresponding prefactors are added to the conduction and valence band edges when the transmission coefficient is computed.

Table 75 lists the coefficients of models and their defaults. These parameters can be mole fraction-dependent. The default parameters A and B are obtained from [27].

Table 75 Default parameters for nonlocal path band-to-band tunneling model

Symbol	Path index	Parameter name	Default value	Unit
A	1	Apath1	4×10^{14}	$\text{cm}^{-3}\text{s}^{-1}$
B	1	Bpath1	1.9×10^7	Vcm^{-1}
Δ_C	1	Dpath1	0	eV
ϵ_{op}	1	Ppath1	0.037	eV
m_V/m_C	1	Rpath1	0	1
A	2	Apath2	0	$\text{cm}^{-3}\text{s}^{-1}$
B	2	Bpath2	0	Vcm^{-1}
Δ_C	2	Dpath2	0	eV
ϵ_{op}	2	Ppath2	0	eV
m_V/m_C	2	Rpath2	0	1
A	3	Apath3	0	$\text{cm}^{-3}\text{s}^{-1}$
B	3	Bpath3	0	Vcm^{-1}
Δ_C	3	Dpath3	0	eV
ϵ_{op}	3	Ppath3	0	eV
m_V/m_C	3	Rpath3	0	1

Visualizing Nonlocal Band-to-Band Generation Rate

To plot the electron and hole generation rates, specify eBand2BandGeneration and hBand2BandGeneration in the Plot section, respectively.

NOTE Band2BandGeneration is equal to hBand2BandGeneration.

Bimolecular Recombination

The bimolecular recombination model describes the interaction of electron–hole pairs and singlet excitons (see [Singlet Exciton Equation on page 235](#)).

Physical Model

The rate of electron–hole pair and singlet exciton recombination follows the Langevin form, that is, it is proportional to the carrier mobility. The bimolecular recombination rate is given by:

$$R_{\text{bimolec}} = \gamma \cdot \frac{q}{\epsilon_0 \epsilon_r} \cdot (\mu_n + \mu_p) \left(np - n_{i,\text{eff}}^2 \frac{n_{\text{se}}}{n_{\text{se}}^{\text{eq}}} \right) \quad (425)$$

where:

- γ is a prefactor for the singlet exciton.
- q is the elementary charge.
- ϵ_0 and ϵ_r denote the free space and relative permittivities, respectively.
- Electron and hole mobilities are given by μ_n and μ_p , accordingly.
- n , p , and $n_{i,\text{eff}}$ describe the electron, hole, and effective intrinsic density, respectively.
- n_{se} is the singlet exciton density.
- $n_{\text{se}}^{\text{eq}}$ denotes the singlet-exciton equilibrium density.

Using Bimolecular Recombination

The bimolecular recombination model is activated by using the keyword **Bimolecular** as an argument of the **Recombination** statement in the **SingletExciton** section (see [Table 201 on page 1277](#)). It is switched off by default and can be activated regionwise (see [Singlet Exciton Equation on page 235](#)). An example is:

```
Physics (Region="EML-ETL") {
  SingletExciton (
    Recombination ( Bimolecular )
  )
}
```

[Table 76](#) lists the parameter of the bimolecular recombination model, which is accessible in the SingletExciton section of the parameter file.

Table 76 Default parameter for bimolecular recombination

Symbol	Parameter name	Default value	Unit
γ	gamma	0.25	1

Exciton Dissociation Model

The exciton dissociation model describes the dissociation of singlet excitons into electron-hole pairs at semiconductor-semiconductor and semiconductor-insulator interfaces.

Physical Model

The rate of singlet exciton interface dissociation is modeled as:

$$R_{se,diss}^{surf} = v_{se,0} \sigma_{se-N_{diss}} N_{se,diss}^{surf} (n_{se} - n_{se}^{eq}) \quad (426)$$

where:

$$v_{diss}^{surf} = v_{se,0} \sigma_{se-N_{diss}} N_{se,diss}^{surf} \quad (427)$$

is the singlet exciton recombination velocity in cm/s, $\sigma_{se-N_{diss}}$ is the capture cross-section of exciton dissociation centers with the surface density $N_{se,diss}^{surf}$, $v_{se,0}$ is the singlet exciton thermal velocity, and n_{se} and n_{se}^{eq} are the exciton and equilibrium exciton densities, respectively.

In the dissociation process, a singlet exciton generates an electron-hole pair. The rate in [Eq. 426](#) is a recombination rate for the singlet exciton equation and a generation rate for the electron and hole continuity equations.

Using Exciton Dissociation

The exciton dissociation model is activated using the keyword `Dissociation` as an argument of the `Recombination` statement in the `SingletExciton` section (see [Table 201 on page 1277](#)). It is switched off by default and can be activated regionwise (see [Singlet Exciton Equation on page 235](#)).

The following example activates exciton dissociation at the EML/ETL region interface:

```
Physics (RegionInterface="EML/ETL") {
  SingletExciton (Recombination ( Dissociation ))
}
```

Table 77 lists the parameters of the exciton dissociation model, which is accessible in the SingletExciton section of the parameter file.

Table 77 Default parameters for exciton dissociation model

Symbol	Parameter name	Default value	Unit
$\sigma_{se} - N_{diss}$	ex_dxsection	1×10^{-8}	cm^2
$N_{se,diss}^{surf}$	N_diss	1×10^{10}	cm^{-2}

References

- [1] D. J. Roulston, N. D. Arora, and S. G. Chamberlain, “Modeling and Measurement of Minority-Carrier Lifetime versus Doping in Diffused Layers of n+-p Silicon Diodes,” *IEEE Transactions on Electron Devices*, vol. ED-29, no. 2, pp. 284–291, 1982.
- [2] J. G. Fossum, “Computer-Aided Numerical Analysis of Silicon Solar Cells,” *Solid-State Electronics*, vol. 19, no. 4, pp. 269–277, 1976.
- [3] J. G. Fossum and D. S. Lee, “A Physical Model for the Dependence of Carrier Lifetime on Doping Density in Nondegenerate Silicon,” *Solid-State Electronics*, vol. 25, no. 8, pp. 741–747, 1982.
- [4] J. G. Fossum *et al.*, “Carrier Recombination and Lifetime in Highly Doped Silicon,” *Solid-State Electronics*, vol. 26, no. 6, pp. 569–576, 1983.
- [5] M. S. Tyagi and R. Van Overstraeten, “Minority Carrier Recombination in Heavily-Doped Silicon,” *Solid-State Electronics*, vol. 26, no. 6, pp. 577–597, 1983.
- [6] H. Goebel and K. Hoffmann, “Full Dynamic Power Diode Model Including Temperature Behavior for Use in Circuit Simulators,” in *Proceedings of the 4th International Symposium on Power Semiconductor Devices & ICs (ISPSD)*, Tokyo, Japan, pp. 130–135, May 1992.
- [7] A. Schenk, “A Model for the Field and Temperature Dependence of Shockley–Read–Hall Lifetimes in Silicon,” *Solid-State Electronics*, vol. 35, no. 11, pp. 1585–1596, 1992.
- [8] R. R. King, R. A. Sinton, and R. M. Swanson, “Studies of Diffused Phosphorus Emitters: Saturation Current, Surface Recombination Velocity, and Quantum Efficiency,” *IEEE Transactions on Electron Devices*, vol. 37, no. 2, pp. 365–371, 1990.

16: Generation-Recombination

References

- [9] R. R. King and R. M. Swanson, "Studies of Diffused Boron Emitters: Saturation Current, Bandgap Narrowing, and Surface Recombination Velocity," *IEEE Transactions on Electron Devices*, vol. 38, no. 6, pp. 1399–1409, 1991.
- [10] A. Cuevas *et al.*, "Surface Recombination Velocity and Energy Bandgap Narrowing of Highly Doped n-Type Silicon," in *13th European Photovoltaic Solar Energy Conference*, Nice, France, pp. 337–342, October 1995.
- [11] A. Schenk and U. Krumbein, "Coupled defect-level recombination: Theory and application to anomalous diode characteristics," *Journal of Applied Physics*, vol. 78, no. 5, pp. 3185–3192, 1995.
- [12] L. Hultdt, N. G. Nilsson, and K. G. Svantesson, "The temperature dependence of band-to-band Auger recombination in silicon," *Applied Physics Letters*, vol. 35, no. 10, pp. 776–777, 1979.
- [13] W. Lochmann and A. Haug, "Phonon-Assisted Auger Recombination in Si with Direct Calculation of the Overlap Integrals," *Solid State Communications*, vol. 35, no. 7, pp. 553–556, 1980.
- [14] R. Häcker and A. Hangleiter, "Intrinsic upper limits of the carrier lifetime in silicon," *Journal of Applied Physics*, vol. 75, no. 11, pp. 7570–7572, 1994.
- [15] A. G. Chynoweth, "Ionization Rates for Electrons and Holes in Silicon," *Physical Review*, vol. 109, no. 5, pp. 1537–1540, 1958.
- [16] R. van Overstraeten and H. de Man, "Measurement of the Ionization Rates in Diffused Silicon p-n Junctions," *Solid-State Electronics*, vol. 13, no. 1, pp. 583–608, 1970.
- [17] Y. Okuto and C. R. Crowell, "Threshold Energy Effect on Avalanche Breakdown Voltage in Semiconductor Junctions," *Solid-State Electronics*, vol. 18, no. 2, pp. 161–168, 1975.
- [18] T. Lackner, "Avalanche Multiplication in Semiconductors: A Modification of Chynoweth's Law," *Solid-State Electronics*, vol. 34, no. 1, pp. 33–42, 1991.
- [19] M. C. Vecchi and M. Rudan, "Modeling Electron and Hole Transport with Full-Band Structure Effects by Means of the Spherical-Harmonics Expansion of the BTE," *IEEE Transactions on Electron Devices*, vol. 45, no. 1, pp. 230–238, 1998.
- [20] S. Reggiani *et al.*, "Electron and Hole Mobility in Silicon at Large Operating Temperatures—Part I: Bulk Mobility," *IEEE Transactions on Electron Devices*, vol. 49, no. 3, pp. 490–499, 2002.
- [21] M. Valdinoci *et al.*, "Impact-ionization in silicon at large operating temperature," in *International Conference on Simulation of Semiconductor Processes and Devices (SISPAD)*, Kyoto, Japan, pp. 27–30, September 1999.
- [22] E. Gnani *et al.*, "Extraction method for the impact-ionization multiplication factor in silicon at large operating temperatures," in *Proceedings of the 32nd European Solid-State Device Research Conference (ESSDERC)*, Florence, Italy, pp. 227–230, September 2002.

- [23] S. Reggiani *et al.*, “Investigation about the High-Temperature Impact-Ionization Coefficient in Silicon,” in *Proceedings of the 34th European Solid-State Device Research Conference (ESSDERC)*, Leuven, Belgium, pp. 245–248, September 2004.
- [24] S. Reggiani *et al.*, “Experimental extraction of the electron impact-ionization coefficient at large operating temperatures,” in *IEDM Technical Digest*, San Francisco, CA, USA, pp. 407–410, December 2004.
- [25] A. Schenk, “Rigorous Theory and Simplified Model of the Band-to-Band Tunneling in Silicon,” *Solid-State Electronics*, vol. 36, no. 1, pp. 19–34, 1993.
- [26] J. J. Liou, “Modeling the Tunnelling Current in Reverse-Biased p/n Junctions,” *Solid-State Electronics*, vol. 33, no. 7, pp. 971–972, 1990.
- [27] G. A. M. Hurkx, D. B. M. Klaassen, and M. P. G. Knuvers, “A New Recombination Model for Device Simulation Including Tunneling,” *IEEE Transactions on Electron Devices*, vol. 39, no. 2, pp. 331–338, 1992.
- [28] E. O. Kane, “Theory of Tunneling,” *Journal of Applied Physics*, vol. 32, no. 1, pp. 83–91, 1961.
- [29] T. Hatakeyama *et al.*, “Physical Modeling and Scaling Properties of 4H-SiC Power Devices,” in *International Conference on Simulation of Semiconductor Processes and Devices (SISPAD)*, Tokyo, Japan, pp. 171–174, September 2005.

16: Generation-Recombination

References

CHAPTER 17 Traps and Fixed Charges

This chapter presents information on how traps are handled by Sentaurus Device.

Traps are important in device physics. For example, they provide doping, enhance recombination, and increase leakage through insulators. Several models (for example, Shockley–Read–Hall recombination, described in [Shockley–Read–Hall Recombination on page 359](#)) depend on traps implicitly, but do not actually model them. This chapter describes models that take the occupation and the space charge stored on traps explicitly into account. It also describes the specification of fixed charges.

Sentaurus Device provides several trap types (electron and hole traps, fixed charges), five types of energetic distribution (level, uniform, exponential, Gaussian, and table), and various models for capture and emission rates, including the V-model (optionally, with field-dependent cross sections), the J-model, and nonlocal tunneling. Traps are available for both bulk and interfaces.

Basic Syntax for Traps

The specification of trap distributions and trapping models appears in the `Physics` section. In contrast to other models, most model parameters are specified here as well. Parameter specifications in the parameter file serve as defaults for those in the command file.

Traps can be specified for interfaces or bulk regions. Specifications take the form:

```
Physics (Region="gobbledygook"){  
    Traps(  
        ( <trap specification> )  
        ( <trap specification> )  
        ...  
    )  
}
```

and likewise for `Material`, `RegionInterface`, and `MaterialInterface`. Above, each `<trap specification>` describes one particular trap distribution, and the models and parameters applied to it. When only a single trap specification is present, the inner pair of parentheses can be omitted. [Table 261 on page 1316](#) summarizes the options that can appear in the trap specification.

NOTE Wherever a contact exists at a specified region interface, Sentaurus Device does not recognize the interface traps within the bounds of the contact because the contact itself constitutes a region and effectively overwrites the interface between the two ‘material’ regions. This is true even if the contact is not declared in the `Electrode` statement.

Trap Types

The keywords `FixedCharge`, `Acceptor`, `Donor`, `eNeutral`, and `hNeutral` select the type of trap distribution:

- `FixedCharge` traps are always completely occupied.
- `Acceptor` and `eNeutral` traps are uncharged when unoccupied and they carry the charge of one electron when fully occupied.
- `Donor` and `hNeutral` traps are uncharged when unoccupied and they carry the charge of one hole when fully occupied.

Energetic and Spatial Distribution of Traps

The keywords `Level`, `Uniform`, `Exponential`, `Gaussian`, and `Table` determine the energetic distribution of traps. They select a single-energy level, a uniform distribution, an exponential distribution, a Gaussian distribution, and a user-defined table distribution, respectively:

$$\begin{aligned}
 & N_0 && \text{for } E = E_0 && \text{for Level} \\
 & N_0 && \text{for } E_0 - 0.5E_S < E < E_0 + 0.5E_S && \text{for Uniform} \\
 & N_0 \exp\left(-\left|\frac{E - E_0}{E_S}\right|\right) && && \text{for Exponential} \\
 & N_0 \exp\left(-\frac{(E - E_0)^2}{2E_S^2}\right) && && \text{for Gaussian} \\
 & \begin{pmatrix} N_1 & \text{for } E = E_1 \\ \dots & \dots \\ N_m & \text{for } E = E_m \end{pmatrix} && && \text{for Table}
 \end{aligned} \tag{428}$$

N_0 is set with the `Conc` keyword. For a `Level` distribution, `Conc` is given in cm^{-3} (for regionwise or materialwise specifications) or cm^{-2} (for interface-wise specifications). For the other energetic distributions, `Conc` is given in $\text{eV}^{-1}\text{cm}^{-3}$ or $\text{eV}^{-1}\text{cm}^{-2}$. For `FixedCharge`, the sign of `Conc` denotes the sign of the fixed charges. For the other trap types, `Conc` must not be negative.

For a Table distribution, individual levels are given as a pair of energies (in eV) and the corresponding concentrations (in $\text{eV}^{-1}\text{cm}^{-3}$ or $\text{eV}^{-1}\text{cm}^{-2}$). Depending on the presence of the keywords `fromCondBand`, `fromMidBandGap`, or `fromValBand`, the energy levels in the table are relative to the conduction band, intrinsic energy, or valence band, respectively. When the Table distribution contains only one energy–concentration pair, it degenerates into a Level distribution.

E_0 and E_S are given in eV by `EnergyMid` and `EnergySig`. The energy of the center of the trap distribution, E_{trap}^0 , is obtained from E_0 depending on the presence of one of the keywords `fromCondBand`, `fromMidBandGap`, or `fromValBand`:

$$E_{\text{trap}}^0 = \begin{cases} E_C - E_0 - E_{\text{shift}} & \text{fromCondBand} \\ [E_C + E_V + kT \ln(N_V/N_C)]/2 + E_0 + E_{\text{shift}} & \text{fromMidBandGap} \\ E_V + E_0 + E_{\text{shift}} & \text{fromValBand} \end{cases} \quad (429)$$

Internally, Sentaurus Device approximates trap-energy distributions by discrete energy levels. The number of levels defaults to 13 and is set by `TrapDLN` in the `Math` section.

In [Eq. 429](#), E_{shift} is zero (the default) or is computed by a PMI specified with `EnergyShift=<model name>` or `EnergyShift=(<model name>, <int>)`. The PMI depends on the electric field and the lattice temperature. By default, these quantities are taken at the location of the trap; alternatively, with `ReferencePoint=<vector>`, you can specify a coordinate in the device from where these quantities are to be taken. For more details, see [Trap Energy Shift on page 1112](#).

For traps located at interfaces, `Region` or `Material` allows you to specify the region or material on one side of the interface; the energy specification then refers to the band structure on that side. For heterointerfaces, the charge and recombination rate due to the traps will be fully accounted for on that side. Without this side specification, the energy parameters refer to the lower bandgap material; the charge and recombination rates due to traps are evenly distributed on both sides.

By default, trap concentrations are uniform over the domain for which they are specified. With `SFactor = "<dataset name>"`, the given dataset determines the spatial distribution. If `Conc` is zero or is omitted, the dataset determines the density directly. Otherwise, it is scaled by the maximum of its absolute value and multiplied by N_0 . Datasets available for `SFactor` are `DeepLevels`, `xMoleFraction`, and `yMoleFraction` (read from the doping file), and `eTrappedCharge` and `hTrappedCharge` (read from the file specified by `DevFields` in the `File` section), as well as the PMI user fields (read from the file specified by `PMIUserFields` in the `File` section).

Alternatively, with `SFactor = "<pmi_model_name>"`, the spatial distribution is computed using the PMI. See [Space Factor on page 1105](#) for more details. During a transient simulation,

this PMI gives the possibility to have a time-dependent trap concentration. In this case, spatial distribution can be computed based on the solution from the previous time step available through the PMI.

SpatialShape selects a multiplicative modifier function for the trap concentration. If SpatialShape is Uniform (the default), the multiplier for point (x, y, z) is:

$$\Theta(\sigma_x - |x - x_0|)\Theta(\sigma_y - |y - y_0|)\Theta(\sigma_z - |z - z_0|) \quad (430)$$

If SpatialShape is Gaussian, the multiplier is:

$$\exp\left(-\frac{(x - x_0)^2}{2\sigma_x^2} - \frac{(y - y_0)^2}{2\sigma_y^2} - \frac{(z - z_0)^2}{2\sigma_z^2}\right) \quad (431)$$

In Eq. 430 and Eq. 431, (x_0, y_0, z_0) and $(\sigma_x, \sigma_y, \sigma_z)$ are given (in μm) by the SpaceMid and SpaceSig options, respectively. By default, the components of SpaceSig are huge, so that the multiplier becomes one.

Specifying Single Traps

The keyword SingleTrap has been provided to simplify the specification of parameters to mimic the behavior of a single carrier trap or single fixed-charge trap. Including SingleTrap in the trap specification results in the following behavior:

- The coordinates specified with SpaceMid= (x_0, y_0, z_0) snap to the nearest node in the material, region, or interface that is specified as part of the Physics command. If a global Physics specification is used, the coordinates snap to the nearest node in the device.
- SpatialShape=Uniform is used automatically for the trap. You do not need to specify this parameter.
- SpaceSig=(1e-6, 1e-6, 1e-6) is used automatically for the trap. This is intended to confine the trap to a single node. You do not need to specify this parameter.
- The trap concentration is computed automatically such that a filled trap corresponds to one electronic charge. If Conc is specified by users, it is ignored.
- SingleTrap is only allowed with a Level energetic distribution for carrier traps (eNeutral, hNeutral, Acceptor, or Donor) or with FixedCharge traps.
- When SingleTrap is specified with FixedCharge, the sign of Conc (if specified) is used for the sign of the fixed charge. If Conc is not specified or if Conc=0 is specified, the fixed charge is positive. To obtain a negative fixed charge with SingleTrap, specify any negative value for Conc.
- For 2D simulations, AreaFactor specified in the Physics section is used for the z-direction width when calculating the SingleTrap concentration.

For example, the following command file fragment places two single-electron traps at the silicon–oxide interface:

```
Physics (MaterialInterface="Silicon/Oxide") {  
  Traps (  
    (SingleTrap eNeutral Level EnergyMid=0 fromMidBandGap  
      SpaceMid=(0.0,0.0,0.1))  
    (SingleTrap eNeutral Level EnergyMid=0 fromMidBandGap  
      SpaceMid=(0.2,0.0,0.3))  
  )  
}
```

Trap Randomization

The spatial distribution of traps can be randomized by including the keyword `Randomize` in the trap specification. Specifying `Randomize` results in the following behavior:

- If `SingleTrap` is specified, the location of the single trap is randomized in the material, region, or interface that is specified as part of the `Physics` command. If a global `Physics` specification is used, the location of the single trap is randomized in the whole device. If `SpaceMid` is specified, it is ignored.
- If `SingleTrap` is not specified, the concentration of traps at each node is randomized. This is accomplished by first determining the average number of traps at a node based on the spatial distribution created from the trap specification. This is used as the expectation value for a Poisson-distribution random number generator to obtain a new random number of traps at the node. Finally, this is converted back to a trap concentration.
- `Randomize` is only allowed with a `Level` energetic distribution for carrier traps (`eNeutral`, `hNeutral`, `Acceptor`, or `Donor`) or with `FixedCharge` traps.
- If `Randomize` is specified, the randomization is different every time the command file is executed. However, this behavior can be altered by specifying `Randomize` with an integer argument:
 - `Randomize < 0`: No randomization occurs.
 - `Randomize = 0`: Same as `Randomize` without an argument. The randomization is different for every execution of the command file.
 - `Randomize > 0`: The specified number is used as the seed for the random number generator. This allows a particular randomization to be repeated for repeated executions of the same command file on the same platform.

Example: Randomize location of a single electron trap at the silicon–oxide interface:

```
Physics (MaterialInterface="Silicon/Oxide") {
  Traps (
    (SingleTrap Randomize eNeutral Level EnergyMid=0 fromMidBandGap)
  )
}
```

Example: Randomize a Gaussian spatial distribution of traps in silicon:

```
Physics (Material="Silicon") {
  Traps (
    (eNeutral Conc=1e16 Level EnergyMid=0 fromMidBandGap
    Randomize SpatialShape=Gaussian
    SpaceMid=(0.0,0.0,0.05) SpaceSig=(0.05,0.05,0.05))
  )
}
```

Trap Models and Parameters

Trap Occupation Dynamics

The electron occupation f^n of an `eNeutral` or `Acceptor` trap is a number between 0 and 1, and changes due to the capture and emission of electrons:

$$\frac{\partial f^n}{\partial t} = \sum_i r_i^n \quad (432)$$

$$r_i^n = (1 - f^n)c_i^n - f^n e_i^n \quad (433)$$

c_i^n denotes an electron capture rate for an empty trap and e_i^n denotes an electron emission rate for a full trap, respectively. The sum in [Eq. 432](#) is over all capture and emission processes. For example, the capture of an electron from the conduction band is a process distinct from the capture of an electron from the valence band.

For the stationary state, the time derivative in [Eq. 432](#) vanishes. The occupation becomes:

$$f^n = \frac{\sum_i c_i^n}{\sum_i (c_i^n + e_i^n)} \quad (434)$$

and the net electron capture rate due to process k becomes:

$$r_k^n = \frac{c_k^n \sum e_i^n - e_k^n \sum c_i^n}{\sum (c_i^n + e_i^n)} \quad (435)$$

Analogous equations hold for hNeutral and Donor traps (use $c_i^p = e_i^n$ and $e_i^p = c_i^n$ to derive them).

In particular, in the stationary state, for traps with concentration N_0 , the V-model (see [Local Trap Capture and Emission on page 414](#)) and [Eq. 435](#) lead to the Shockley–Read–Hall recombination rate:

$$R_{\text{net}} = \frac{N_0 v_{\text{th}}^n v_{\text{th}}^p \sigma_n \sigma_p (np - n_{\text{eff}}^2)}{v_{\text{th}}^n \sigma_n (n + n_1/g_n) + v_{\text{th}}^p \sigma_p (p + p_1/g_p)} \quad (436)$$

Each capture and emission process couples the trap to a reservoir of carriers. If only a single process would be effective, in the stationary state, the trap would be in equilibrium with the reservoir for this process. This consideration (the ‘principle of detailed balance’) relates capture and emission rates:

$$e_i = \frac{c_i}{g} \exp\left(\frac{E_{\text{trap}} - E_F^i}{kT_i}\right) \quad (437)$$

Here, E_{trap} is the energy of the trap, E_F^i is the Fermi energy of the reservoir, T_i is the temperature of the reservoir, and g is the degeneracy factor. Sentaurus Device supports distinct degeneracy factors g_n and g_p for coupling to the conduction and valence bands. They default to 1 and are set with `eGfactor` and `hGfactor` in the command file, and with the `G` parameter pair in the Traps parameter set.

Capture and emission rates are either local (see [Local Trap Capture and Emission on page 414](#)) or nonlocal (see [Tunneling and Traps on page 417](#)). Sentaurus Device does not solve the current continuity equations in insulators and, therefore, supports local rates only for traps in semiconductors or at semiconductor interfaces. Therefore, traps in insulators that are not connected to a semiconductor region by a nonlocal model do not have any capture and emission processes, and their occupation is undefined (except for FixedCharge ‘traps’). Sentaurus Device ignores them.

Local Trap Capture and Emission

The electron capture rate from the conduction band at the same location as the trap is:

$$c_C^n = \sigma_n \left[(1 - g_n^J) v_{th}^n n + g_n^J \frac{J_n}{q} \right] \quad (438)$$

and similarly, the hole capture rate from the valence band is $c_V^p = \sigma_p [(1 - g_p^J) v_{th}^p p + g_p^J J_p / q]$.

g_n^J and g_p^J are set with `eJfactor` and `hJfactor` in the command file, and with the `Jcoef` parameter pair in the Traps parameter set. g_n^J and g_p^J can take any value between 0 and 1. When zero (the default), the V-model is obtained, and when 1, the J-model (popular for modeling radiation problems) is obtained.

The electron emission rate to the conduction band is $e_C^n = v_{th}^n \sigma_n \gamma_n n_1 / g_n + e_{const}^n$ and the hole emission to the valence band is $e_V^p = v_{th}^p \sigma_p \gamma_p p_1 / g_p + e_{const}^p$. For $g_n^J = e_{const}^n = 0$ and $g_p^J = e_{const}^p = 0$, these rates obey the principle of detailed balance.

Above, $n_1 = N_C \exp[(E_{trap} - E_C)/kT]$ and $p_1 = N_V \exp[(E_V - E_{trap})/kT]$. For Fermi statistics or with quantization (see [Chapter 14 on page 279](#)), γ_n and γ_p are given by [Eq. 47](#) and [Eq. 48, p. 194](#) (see [Fermi Statistics on page 194](#)), whereas otherwise, $\gamma_n = \gamma_p = 1$.

v_{th}^n and v_{th}^p are the thermal velocities. Sentaurus Device offers two options, selected by the `VthFormula` parameter pair in the Traps parameter set (default is 1):

$$v_{th}^{n,p} = \begin{cases} v_o^{n,p} \sqrt{\frac{T}{300K}} & \text{VthFormula=1} \\ \sqrt{\frac{3kT}{m_{n,p}(300K)}} & \text{VthFormula=2} \end{cases} \quad (439)$$

$v_o^{n,p}$ are given by the `Vth` parameter pair in the Traps section of the parameter file.

Sentaurus Device offers several options for the cross sections σ_n and σ_p . They are selected by keywords in the command file or by the value of the `XsecFormula` parameter pair in the Traps parameter set. In any case, trap cross sections are derived from the constant cross sections σ_n^0 and σ_p^0 , which are set by the `exsection` and `hxsection` keywords in the command file, and by the `XSEC` parameter pair in the Traps parameter set.

By default, the cross sections are constant: $\sigma_n = \sigma_n^0$ and $\sigma_p = \sigma_p^0$.

e_{const}^n and e_{const}^p represent the constant emission rate terms for carrier emission from the trap level to the conduction and valence band. They can be set by the `eConstEmissionRate` and `hConstEmissionRate` keywords in the command file, and by the `ConstEmissionRate` parameter pair in the `Traps` parameter set (default $e_{\text{const}}^n = 0 \text{ s}^{-1}$, $e_{\text{const}}^p = 0 \text{ s}^{-1}$).

J-Model Cross Sections

This model is activated by the `ElectricField` keyword in the command file or by a value of 2 in the `XsecFormula` parameter pair in the `Traps` parameter set. It is intended for use with the J-model and reads:

$$\sigma_{n,p} = \sigma_{n,p}^0 \left(1 + a_1 \left| \frac{F}{1 \text{ Vm}^{-1}} \right|^{p_1} + a_2 \left| \frac{F}{1 \text{ Vm}^{-1}} \right|^{p_2} \right)^{p_0} \quad (440)$$

where a_1 , a_2 , p_0 , p_1 , and p_2 are adjustable parameters available as parameter pairs `a1`, `a2`, `p0`, `p1`, and `p2` in the `Traps` parameter set.

Hurkx Model for Cross Sections

The Hurkx model is selected by the `Tunneling(Hurkx)` keyword in the command file or by a value of 3 in the `XsecFormula` parameter pair in the `Traps` parameter set. The cross sections are obtained from σ_n^0 and σ_p^0 using [Eq. 340](#), [p. 366](#).

Poole–Frenkel Model for Cross Sections

The Poole–Frenkel model [\[1\]](#) is frequently used for the interpretation of transport effects in dielectrics and amorphous films. The model predicts an enhanced emission probability Γ_{pf} for charged trap centers where the potential barrier is reduced because of the high external electric field.

The model is selected by the `PooleFrenkel` keyword in the command file or by a value of 4 in the `XsecFormula` parameter pair in the `Traps` parameter set. In the Poole–Frenkel model:

$$\begin{aligned} \sigma_{n,p}^{\text{enh}} &= \sigma_{n,p}^0 (1 + \Gamma_{\text{pf}}) \\ \Gamma_{\text{pf}} &= \frac{1}{\alpha^2} [1 + (\alpha - 1) \exp(\alpha)] - \frac{1}{2} \\ \alpha &= \frac{1}{kT} \sqrt{\frac{q^3 F}{\pi \epsilon_{\text{pf}}}} \end{aligned} \quad (441)$$

For Donor and hNeutral traps, $\sigma_n = \sigma_n^{\text{enh}}$ and $\sigma_p = \sigma_p^0$. For Acceptor and eNeutral traps, $\sigma_p = \sigma_p^{\text{enh}}$ and $\sigma_n = \sigma_n^0$. ϵ_{pf} is an adjustable parameter and is set by the epsPF parameter pair in the PooleFrenkel parameter set.

Local Capture and Emission Rates Based on Makram-Ebeid–Lannoo Phonon-assisted Tunnel Ionization Model

The Makram-Ebeid–Lannoo model [2] is a phonon-assisted tunnel emission model for carriers trapped on deep semiconductor levels, with its main application in two-band charge transport in silicon nitride [3].

In this model, the deep trap acts as an oscillator or core embedded in the nitride lattice, attracting electrons (electron trap) or holes (hole trap). The deep trap is defined by the phonon energy W_{ph} , the thermal energy W_T and the optical energy W_{opt} . The trap ionization rate is give by:

$$P = \sum_{n=-\infty}^{\infty} \exp\left[\frac{nW_{\text{ph}}}{2kT} - S \coth \frac{W_{\text{ph}}}{2kT}\right] I_n\left(\frac{S}{\sinh(W_{\text{ph}}/(2kT))}\right) P_i(W_T + nW_{\text{ph}}) \quad (442)$$

$$P_i(W) = \frac{eF}{2\sqrt{2m}W} \exp\left(-\frac{4\sqrt{2m}}{3\hbar eF} W^{3/2}\right) \quad (443)$$

where I_n is the modified Bessel function of the order n , $S = (W_{\text{opt}} - W_T)/W_{\text{ph}}$, and $P_i(W)$ is the tunnel escape rate through the triangle barrier of height W .

Sentaurus Device implements the Makram-Ebeid–Lannoo model for Level traps, eNeutral and hNeutral. The electron emission rate to the conduction band e_C^n and the hole emission to the valence band e_V^p are the trap ionization rates described by Eq. 442 with the corresponding W_{ph} , W_T and W_{opt} defining the associated deep trap.

The electron capture rate from the conduction band c_C^n and the hole capture rate from the valence band c_V^p are computed based on user selection. They can be obtained from the emission rates by the principle of detailed balance or using the constant capture cross-sections: $c_C^n = \sigma_n^0 \left[(1 - g_n^J) v_{\text{th}}^n n + g_n^J \frac{J_n}{q} \right]$, $c_V^p = \sigma_p^0 \left[(1 - g_p^J) v_{\text{th}}^p p + g_p^J \frac{J_p}{q} \right]$.

The model is selected by the keyword Makram-Ebeid in the command file of the trap parameter set. When the Makram-Ebeid keyword with no options is used, the trap couples to both the conduction and valence bands through Makram-Ebeid–Lannoo trap emission rates. When Makram-Ebeid is used with the option electron in parentheses, an eNeutral trap couples to the conduction band through the Makram-Ebeid–Lannoo emission rate and to the valence band using the default emission rate. Similarly, Makram-Ebeid with hole in parentheses couples an hNeutral trap to the valence band through the Makram-Ebeid–Lannoo emission rate and to the conduction band using the default emission rate.

The electron capture rate from the conduction band to an eNeutral trap (c_C^n) and the hole capture rate from the valence band to an hNeutral trap (c_V^p) are, by default, computed from the corresponding emission rates using the detailed balance principle. By using the keyword `simpleCapt` in parentheses as an option for `Makram-Ebeid`, capture rates are computed as $c_C^n = \sigma_n \left[(1 - g_n^J) v_{th}^n n + g_n^J \frac{n}{q} \right]$ for exchange with the conduction band and $c_V^p = \sigma_p \left[(1 - g_p^J) v_{th}^p p + g_p^J \frac{p}{q} \right]$ for exchange with the valence band, where g_n^J and g_p^J are set with `eJfactor` and `hJfactor` in the command file and with the `Jcoef` parameter pair in the Traps parameter set. g_n^J and g_p^J can take any value between 0 and 1. When zero or not specified (the default), the V-model is obtained with the simplified rates $c_C^n = \sigma_n^0 v_{th}^n n$ and $c_V^p = \sigma_p^0 v_{th}^p p$, respectively. When 1, the J-model is obtained.

The deep trap parameters W_{ph} , W_T , and W_{opt} can be adjusted in the `Makram-Ebeid` section of the parameter file. Their default values are $W_{ph} = 0.06 \text{ eV}$, $W_T = 1.4 \text{ eV}$, and $W_{opt} = 2.8 \text{ eV}$. In addition, the Makram-Ebeid–Lannoo tunneling masses m can be adjusted in the parameter file. The default value is 0.5 for both electrons and holes.

Local Capture and Emission Rates from PMI

As an alternative to models described above, the local capture and emission rates c_C^n , c_V^p , e_C^n , and e_V^p can be computed directly using a physical model interface (PMI). Using this PMI only makes sense for `Level` traps. For more details, see [Trap Capture and Emission Rates on page 1108](#).

Tunneling and Traps

Traps can be coupled to nearby interfaces and contacts by tunneling. Sentaurus Device models nonlocal tunneling to traps as the sum of an inelastic, phonon-assisted process and an elastic process [4][5]. To use the nonlocal tunneling model for tunneling to traps:

- Generate nonlocal meshes that connect the trap locations with the interfaces as required (see [Defining Nonlocal Meshes on page 612](#)).
- Specify `eBarrierTunneling` (for coupling to the conduction band) and `hBarrierTunneling` (for coupling to the valence band) in the trap specification in the command file. Provide the names of the nonlocal meshes the trap must be coupled to using the `NonLocal` option of `eBarrierTunneling` and `hBarrierTunneling`.
- Adjust `TrapVolume`, `HuangRhys`, and `PhononEnergy` (see below), as well as the tunneling masses and the interface-specific prefactors (see [Nonlocal Tunneling Parameters on page 615](#)).

The electron capture rate for the phonon-assisted transition from the conduction band is:

$$c_{C, \text{phonon}}^n = \frac{\sqrt{m_t m_0^3 k^3 T_n^3} g_C}{\hbar^3 \sqrt{\chi}} V_T S \omega \left[\frac{\alpha (S - l)^2}{S} + 1 - \alpha \right] \exp \left[-S(2f_B + 1) + \frac{\Delta E}{2kT} + \chi \right] \times \left(\frac{z}{l + \chi} \right)^l F_{1/2} \left(\frac{E_{F,n} - E_C(0)}{kT_n} \right) \frac{|\Psi(z_o)|^2}{|\Psi(0)|^2} \quad (444)$$

where:

- V_T is the interaction volume of the trap.
- S is the Huang–Rhys factor.
- $\hbar\omega$ is the energy of the phonons involved in the transition.
- α is a dimensionless parameter.

These parameters are set by `TrapVolume` (in μm^3), `HuangRhys` (dimensionless), `PhononEnergy` (in eV), and `alpha` (dimensionless) in the command file, and by parameters of the same name in the `Traps` parameter set. V_T , S , and $\hbar\omega$ default to 0, and α defaults to 1. `TrapVolume` must be positive when tunneling is activated.

In Eq. 444, l is the number of the phonons emitted in the transition, $f_B = [\exp(\hbar\omega/kT) - 1]^{-1}$ is the Bose–Einstein occupation of the phonon state, $z = 2S\sqrt{f_B(f_B + 1)}$ and $\chi = \sqrt{l^2 + z^2}$. The dissipated energy is $\Delta E = E_C + 3kT_n/2 - E_{\text{trap}}$. The Fermi energy and the electron temperature are obtained at the interface or contact, while the lattice temperature is obtained at the site z_0 of the trap. m_t is the effective tunneling mass and g_C is the prefactor for the Richardson constant at the interface or contact. See [Nonlocal Tunneling Parameters on page 615](#) for more details regarding these parameters.

The electron capture rate for the elastic transition from the conduction band is:

$$c_{C, \text{elastic}}^n = \frac{\sqrt{8m_t m_0^3/2} g_C}{\hbar^4 \pi} V_T [E_C(z_o) - E_{\text{trap}}]^2 \Theta[E_{\text{trap}} - E_C(0)] \sqrt{E_{\text{trap}} - E_C(0)} f \left(\frac{E_{F,n} - E_{\text{trap}}}{kT_n} \right) \frac{|\Psi(z_o)|^2}{|\Psi(0)|^2} \quad (445)$$

The emission rates are obtained from the capture rates by the principle of detailed balance (see Eq. 437). The hole terms are analogous to the electrons terms. However, `hBarrierTunneling` for contacts and metals is unphysical and, therefore, ignored.

The ratio of wavefunction is obtained from Γ_{CC} described in [WKB Tunneling Probability on page 618](#) as:

$$\frac{|\Psi(z_o)|^2}{|\Psi(0)|^2} = \frac{v(0)}{v(z_o)} \Gamma_{CC} \quad (446)$$

where v denotes the (possibly imaginary) velocities. To avoid singularities, for the inelastic transition with the WKB tunneling model, the velocity $v(z_o)$ at the trap site is replaced by the

thermal velocity. For the inelastic process, Γ_{CC} is computed at the tunneling energy $E_C + kT_n/2$ and, for elastic process, at energy E_{trap} .

Trap Numeric Parameters

When used with Fermi statistics, the trap model sometimes leads to convergence problems, especially at the beginning of a simulation when Sentaurus Device tries to find an initial solution. This problem can often be solved by changing the numeric damping of the trap charge in the nonlinear Poisson equation.

To this end, set the `Damping` option to the `Traps` keyword in the global `Math` section to a nonnegative number, for example:

```
Math {
    Traps(Damping=100)
}
```

Larger values of `Damping` increase damping of the trap charge; a value of 0 disables damping. The default value is 10. Depending on the particular example, increasing damping can improve or degrade the convergence behavior. There are no guidelines regarding the optimal value of this parameter.

At nonheterointerface vertices, bulk traps are considered only from the region with the lowest band gap. In cases where the bulk traps from other regions are important, use `RegionWiseAssembly` in `Traps` of the global `Math` section, which properly considers bulk traps from all adjacent regions.

Visualizing Traps

To plot the concentration of electrons trapped in `eNeutral` and `Acceptor` traps and of negative fixed charges, specify `eTrappedCharge` in the `Plot` section. Similarly, for the concentration of holes trapped in `hNeutral` or `Donor` traps and positive fixed charges, specify `hTrappedCharge`. These datasets include the contribution of interface charges as well. To that end, Sentaurus Device converts the interface densities to volume densities and, therefore, their contribution depends on the mesh spacing. To plot the interface charges separately as interface densities, use `eInterfaceTrappedCharge` and `hInterfaceTrappedCharge`.

For backward compatibility, for traps specified in insulators, at vertices on interfaces to semiconductors, the charge density in the insulator parts associated with the vertices will be reassigned to the semiconductor parts. This involves the rescaling of the densities with the ratio of the volumes of the two parts and, typically, this leads to a distortion of the data for visualization. However, the distortion has no effect on the actual solution.

17: Traps and Fixed Charges

Visualizing Traps

To plot the recombination rates for the conduction band and valence band due to trapping and de-trapping, specify `eGapStatesRecombination` and `hGapStatesRecombination`, respectively.

In addition, Sentaurus Device allows you to plot trapped carrier density and occupancy probability versus energy at positions specified in the command file.

The plot file is a `.plt` file and its name must be defined in the `File` section by the `TrappedCarPlotFile` keyword:

```
File {  
    ...  
    TrappedCarPlotFile = "itraps_trappedcar"  
}
```

The plotting is activated by including the `TrappedCarDistrPlot` section (similar to the `CurrentPlot` section) in the command file:

```
TrappedCarDistrPlot {  
    MaterialInterface="Silicon/Oxide" {(0.5 0)}  
    ...  
}
```

The positions defining where the trapped carrier density, occupancy probability, and trap density versus energies are to be plotted are specified materialwise or regionwise in the `TrappedCarDistrPlot` section, grouped on regions, materials, region interfaces, and material interfaces.

A set of coordinates of positions in parentheses follows the region or region interface:

```
TrappedCarDistrPlot {  
    MaterialInterface="Silicon/Oxide" {(0.5 0)}  
    RegionInterface="Region_2/Region_3" {(0.1 0.001)}  
    Region="Region_1" {(0.3 0) (0 0) (-0.1 0.2)}  
    Material="Silicon" {(0.27 0) (0 0.01)}  
    ...  
}
```

For each position defined by its coordinates, Sentaurus Device searches for the closest vertex inside the corresponding region or on the corresponding region interface. This is the actual position where the plotting is performed. The difference between user coordinates and actual coordinates is displayed in the log file for each valid position in the `TrappedCarDistrPlot` section in the following format:

[Position(User)	ClosestVertex	PositionClosestVertex]
[(2.4000, -0.1000)	718	(2.3130, -0.0500)]
[(2.5000, -0.1000)	743	(2.3500, -0.0500)]

In addition, a simplified syntax for a global position inside the device is available:

```
TrappedCarDistrPlot {
  (0.5 0)
  ...
}
```

In this case, the closest vertex is used in distribution plotting.

Based on trap types and positions, a unique entry is created in the generated graph. The naming is `TrapTypeCounter(position)`, where `Counter` is used when multiple traps of the same type are used. For each of these entries, the `Energy`, `TrappedChargeDistribution`, `DistributionFunction`, and `TrapDensity` fields are available for plotting. In addition, for each entry, `eQuasiFermi` and `hQuasiFermi` are available for plotting. This allows you to monitor the evolution of `DistributionFunction` relative to quasi-Fermi levels.

Explicit Trap Occupation

For the investigation of, for example, time-delay effects, it may be advantageous to start a transient simulation from an initial state with either totally empty or totally filled trap states. However, depending on the position of the equilibrium Fermi level, it may be impossible to reach such an initial state from steady-state or quasistationary simulations (for example, it may be a metastable state with a very long lifetime).

For this purpose, two different mechanisms are available to initialize trap occupancies in the `Solve` section.

The first mechanism is invoked by `TrapFilling` in the `Set` and `Unset` statements of the `Solve` section. [Table 164 on page 1238](#) summarizes the syntax of these statements, and [Trap Examples on page 422](#) presents an example.

The second mechanism, invoked by `Traps` in a `Set` statement within a `Solve` section, allows you to set trap occupancies for individual named traps to spatial- and solution-independent values, and to freeze and unfreeze the trap occupancies of all traps. To set a trap you need to define an identifying `Name` in its definition in `Traps`, which enables you to reference this trap in the `Set` command. The `Frozen` option allows you to freeze the trap occupancies for subsequent `Solve` statements and to unfreeze them again. Freezing traps implies that the traps are decoupled from both the conduction band and valence band, that is, their recombination terms are set to zero. Observe that `Frozen` applies to all defined traps. Traps not explicitly initialized in the `Set` are frozen at their actual values. The options of `Traps` in `Set` are summarized in [Table 164 on page 1238](#).

17: Traps and Fixed Charges

Trap Examples

```
Device "MOS" {
  Physics { ...
    Traps ( ( Name="t1" eNeutral ... ) ( Name="t2" hNeutral ... ) ... )
  }
}
System { ...
  MOS "mos1" ( ... ) { ... }
}
Solve { ...
  Set ( Traps ( "mos1"."t1" = 1. Frozen ) )
  ...
  Set ( Traps ( -Frozen ) )
  ...
}
```

Here in the example, some traps are named. In the first `Set`, the trap "t1" of device "mos1" is explicitly set to one. Due to the `Frozen` option, trap "t1" is frozen at the specified value and trap "t2" is frozen at its actual value. The second `Set` releases all traps again.

NOTE Freezing and unfreezing of traps using the `Solve-Set-Traps` command implies freezing and unfreezing of multistate configurations, respectively (see [Manipulating MSCs During Solve on page 437](#)).

Trap Examples

The following example of a trap statement illustrates one donor trap level at the intrinsic energy with a concentration of $1 \times 10^{15} \text{ cm}^{-3}$ and capture cross sections of $1 \times 10^{-14} \text{ cm}^2$:

```
Traps(Donor Level EnergyMid=0 fromMidBandGap
      Conc=1e15 eXsection=1e-14 hXsection=1e-14)
```

This example shows trap specifications appropriate for a polysilicon TFT, with four exponential distributions:

```
Traps( (eNeutral Exponential fromCondBand Conc=1e21 EnergyMid=0
        EnergySig=0.035 eXsection=1e-10 hXsection=1e-12)
        (eNeutral Exponential fromCondBand Conc=5e18 EnergyMid=0
        EnergySig=0.1 eXsection=1e-10 hXsection=1e-12)
        (hNeutral Exponential fromValBand Conc=1e21 EnergyMid=0
        EnergySig=0.035 eXsection=1e-12 hXsection=1e-10)
        (hNeutral Exponential fromValBand Conc=5e18 EnergyMid=0
        EnergySig=0.2 eXsection=1e-12 hXsection=1e-10) )
```

The following Solve statement fills traps consistent with a high electron and a low hole concentration; performs a Quasistationary, keeping that trap filling; and, finally, simulates the transient evolution of the trap occupation:

```
Solve{
  Set(TrapFilling=n)
  Quasistationary{...}
  Unset(TrapFilling)
  Transient{...}
}
```

References

- [1] L. Colalongo *et al.*, “Numerical Analysis of Poly-TFTs Under Off Conditions,” *Solid-State Electronics*, vol. 41, no. 4, pp. 627–633, 1997.
- [2] S. Makram-Ebeid and M. Lannoo, “Quantum model for phonon-assisted tunnel ionization of deep levels in a semiconductor,” *Physical Review B*, vol. 25, no. 10, pp. 6406–6424, 1982.
- [3] K. A. Nasyrov *et al.*, “Two-bands charge transport in silicon nitride due to phonon-assisted trap ionization,” *Journal of Applied Physics*, vol. 96, no. 8, pp. 4293–4296, 2004.
- [4] A. Palma *et al.*, “Quantum two-dimensional calculation of time constants of random telegraph signals in metal-oxide–semiconductor structures,” *Physical Review B*, vol. 56, no. 15, pp. 9565–9574, 1997.
- [5] F. Jiménez-Molinos *et al.*, “Direct and trap-assisted elastic tunneling through ultrathin gate oxides,” *Journal of Applied Physics*, vol. 91, no. 8, pp. 5116–5124, 2002.

17: Traps and Fixed Charges

References

CHAPTER 18 Phase and State Transitions

This chapter presents a framework for the simulation of local phase or state transitions.

State transitions appear in device physics in various forms. Typical examples are the charge traps as described in [Chapter 17 on page 407](#). In phase-change memory (PCM) devices, different phases (for example, crystalline and amorphous) of chalcogenides are used to store information and can be modeled with the framework described here. Furthermore, in the hydrogen transport degradation model (see [MSC–Hydrogen Transport Degradation Model on page 449](#)), diffusing mobile hydrogen species may be trapped in localized states.

In this chapter, a general modeling framework called *multistate configuration* (MSC) is presented to describe transitions between phases or states. The framework allows the specification of an arbitrary number of states that interact locally by an arbitrary number of transitions. The states can be charged and carry hydrogen atoms. Transitions between two states may interact with the conduction and valence band, or with hydrogen diffusion equations to preserve charge and the number of hydrogen atoms. The structure of transitions is limited to a linear local dependency between the state occupation rates. However, arbitrary nonlinear local dependency on the solution variables of the transport equations are allowed using PMI models. The state occupation rates are solved self-consistently with the transport model both for stationary and dynamic characteristics.

Multistate Configurations and Their Dynamic

A multistate configuration (MSC) is defined by the number of states N and the state occupation probabilities $s_1, \dots, s_N$ satisfying the condition:

$$\sum_i s_i = 1 \quad (447)$$

For two states i and j , an arbitrary number of transitions (described by capture and emission rates) is allowed. The dynamic equation is then given by:

$$\dot{s}_i = \sum_{j \neq i} \sum_{t \in T_{ij}} c_{ij} s_j - e_{ij} s_i \quad (448)$$

18: Phase and State Transitions

Multistate Configurations and Their Dynamic

where T_{ij} is the set of transitions between i and j . For such a transition $t \in T_{ij}$ with capture and emission rates c and e , respectively, you have $c = c_{ij} = e_{ji}$ and $e = e_{ij} = c_{ji}$ if i and j denote the reference state and interacting state, respectively. The problem can be written in the compact form:

$$\dot{s} = Ts \quad (449)$$

where T is the total transition matrix, composed of the individual transition matrices.

Each state can carry a number K^Q of (positive) charges and a number K^H of hydrogen atoms (both numbers can be positive or negative, and are zero by default). Transitions between two states must satisfy conservation laws for both quantities. Required particles can be taken from several reservoirs. Reservoir particles are characterized by the corresponding numbers K_r^Q and K_r^H , and transitions specify the number P_r of involved reservoir particles. The conservation laws then read:

$$K_i - K_j = \sum_r P_r K_r \quad (450)$$

where K_i and K_j are the characteristic numbers for the reference and interacting states of the transition, respectively. The sum is taken over all reservoirs involved in the transition.

[Table 78](#) lists the available particle reservoirs and their characteristics. The reservoirs of hydrogen atoms, molecules, and ions represent the corresponding mobile species in the hydrogen transport degradation model. Their use is illustrated in [Reactions with Multistate Configurations on page 453](#).

Table 78 MSC particle reservoirs and their characteristics

Description	Identifying string	Particle	Charge K^Q	Hydrogen K^H	Equation
Conduction band	CB	electron	-1	0	Electron
Valence band	VB	hole	1	0	Hole
Hydrogen atoms	HydrogenAtom	H	0	1	HydrogenAtom
Hydrogen molecules	HydrogenMolecule	H_2	0	2	HydrogenMolecule
Hydrogen ions	HydrogenIon	H^+	1	1	HydrogenIon

The resulting space charge and recombination terms with the corresponding equations are taken into account automatically.

Specifying Multistate Configurations

A multistate configuration is specified by an `MSConfig` section placed into an `MSConfigs` section of a (region or material or global) `Physics` section. It is described by its states (at least two) and transitions (each state must be involved in at least one transition) using the keywords `State` and `Transition`. An arbitrary number of `MSConfig` sections is allowed, for example:

```
Physics ( Region = "si" ) {
  MSConfigs (
    MSConfig ( Name = "ca"
      State ( Name = "c" ) State ( Name = "a" )
      Transition ( Name = "t1"
        To="c" From="a" CEModel("pmi_ce_msc" 0))
      )
    ...
  )
}
```

This example specifies a multistate configuration with two states and one transition between them. See [Table 248 on page 1309](#) for the parameters supported by `MSConfig`.

`State` requires a `Name` as an identifier. The state can carry both a number of positive charges by specifying `Charge`, and a number of hydrogen atoms by using `Hydrogen`.

`Transition` requires several parameters. The keyword `CEModel` specifies a transition model including its optional model index (see [Transition Models on page 428](#)). The reference and interacting states of the transition are selected by the keywords `To` and `From`, respectively. A `Name` specification is mandatory as well.

A transition between differently charged states (correspondingly for hydrogen atoms) requires additional charged particles to preserve the total charge. With `Reservoirs`, you can specify a list of reservoirs (`CB` and `VB` for the conduction band and valence band, respectively), which provides the necessary number of particles specified by `Particles` as an argument to the reservoir. The conduction band serves as an electron reservoir, and the valence band is a hole reservoir.

An `eNeutral` level trap of the form:

```
(Name="eN" eNeutral Conc=1.e+13 CBRate=("pmi_ce0" 0) VBRate=("pmi_ce0" 1))
```

can be specified equivalently as an `MSC` in the following way:

```
MSConfig ( Name="eN" Conc=1.e+13
  State ( Name="s0" Charge=0 ) State ( Name="s1" Charge=-1 )
  Transition ( Name="tCB" CEModel("pmi_ce0" 0)
    To="s1" From="s0" Reservoirs("CB"(Particles=+1)))
```

```

Transition ( Name="tVB" CEModel("pmi_ce0" 1)
            To="s0" From="s1" Reservoirs("VB"(Particles=+1)))
)

```

For all multistate configurations, the dynamic is always solved implicitly, that is, no extra equation needs to be specified as an argument to the `Coupled` or `Transient` solve statements.

NOTE Only quasistationary and transient simulations support multistate configurations. Small-signal analysis (see [Small-Signal AC Analysis on page 127](#)), harmonic balance analysis (see [Harmonic Balance on page 130](#)), and noise analysis (see [Chapter 23 on page 581](#)) do not support multistate configurations.

NOTE The computation of state occupation is influenced by the `TrapFilling` option of the `Set` and `Unset` commands in the `Solve` section (see [Table 147 on page 1224](#)). It is frozen if the trap-filling option `Frozen` is set; otherwise, the occupation rates are treated as free. The frozen status is released again by the `Unset(TrapFilling)` command.

The transition dynamic may become numerically unstable, that is, it cannot be solved with sufficient accuracy if, for example, the forward and backward rates of all transitions at one operation point differ by several orders of magnitude. Using `-Elimination` in `MSConfig` applies a different solving algorithm, often improving the numeric robustness in such cases.

The state occupation probabilities can be plotted in groups or individually by specifying `MSConfig` (see [Table 268 on page 1322](#)) in the `Plot` section.

Transition Models

The MSC framework allows an arbitrary number of transitions between two states of an MSC. Each transition model can be either the model `pmi_ce_msc`, or a trap capture and emission PMI (see [Trap Capture and Emission Rates on page 1108](#)).

The `pmi_ce_msc` Model

The `pmi_ce_msc` transition model supports the following features:

- Arbitrary number of states and transitions
- Charged states
- Several transition rate models (nucleation, growth, electron and hole exchange)

- Equilibrium computation (necessary for detailed balance processes)
- Energy and particle exchange

States

The states are described by a base energy E_i , a degeneracy factor g_i , the number of negative elementary charges K_i (an arbitrary integer), and the energy E_i^- of one electron in the state. The inner energy of the state is then:

$$H_i = E_i + K_i E_i^- \quad (451)$$

The electron energy is the sum of the valence band energy and the user-specified value $E_{i, \text{user}}^-$.

Equilibrium

The equilibrium occupation probabilities s_i^* are needed to guarantee the detailed balance principle for all transitions. The equilibrium is determined by the state parameters, the temperature, and the Fermi levels of the involved particle reservoirs. You have:

$$Z_i = g_i \exp(-\beta(H_i - K_i E_F)) \quad (452)$$

$$Z = \sum_i Z_i \quad (453)$$

$$s_i^* = Z_i / Z \quad (454)$$

Here, E_F is the quasi-Fermi energy and β is the thermodynamic beta. The quasi-Fermi energy is approximated inside the PMI. Let the (approximated) intrinsic density n_I and the intrinsic Fermi energy E_I be:

$$n_I = \sqrt{N_C N_V} \exp(-\beta E_g) \quad (455)$$

$$E_I = \frac{1}{2} \left[(E_C + E_V) + kT \ln \left(\frac{N_V}{N_C} \right) \right] \quad (456)$$

Then, the carrier quasi-Fermi energies are approximated from the carrier densities by:

$$E_{F,n} = E_I + kT \ln \left(\frac{n}{n_I} \right) \quad (457)$$

$$E_{F,p} = E_I - kT \ln \left(\frac{p}{n_I} \right) \quad (458)$$

and equilibrium Fermi energy then as:

$$E_F = \frac{1}{2}(E_{F,n} + E_{F,p}) \quad (459)$$

Transitions

Several transition models, listed in [Table 79](#), are available and selected by the Formula parameter in the parameter file.

Table 79 The pmi_ce_msc transition models

Description	Formula
Arrhenius law	0
Nucleation according to Peng	1
Growth according to Peng	2
Single trap	7
MSC trap	8

In this section, i denotes the ‘to’ state, while j specifies the ‘from’ state of the transition. For most of the models, only the forward reaction rate (capture) is given, while the backward reaction rate (emission) is computed by:

$$\frac{e}{c} = \frac{s_j^*}{s_i^*} \quad (460)$$

if not stated otherwise.

Arrhenius Law (Formula=0)

The forward reaction rate (capture) is given by:

$$c = r_0 \exp(-\beta E_{\text{act}}) \quad (461)$$

where r_0 is the maximal transition frequency and E_{act} is the activation energy.

Nucleation According to Peng (Formula=1)

A nucleation model, in analogy to the model in [\[1\]](#), [\[2\]](#), and [\[3\]](#), is given by:

$$c_N = r_0 \exp(-\beta(E_{\text{act}} + \Delta G^*(T))) \quad (462)$$

where:

$$\Delta G^*(T) = \frac{16\pi}{3} \frac{\gamma_{SL}^3}{\Delta G(T)^2} \quad (463)$$

$$\Delta G(T) = \begin{cases} \Delta H_2 \left[1 - \frac{T}{T_g} \left(1 - \frac{\Delta H_1 T_m - T_g}{\Delta H_2 T_m} \right) \right] & \text{if } T < T_g \\ \Delta H_1 \frac{T_m - T}{T_m} & \text{if } T_g < T < T_m \\ 0 & \text{if } T_m < T \end{cases} \quad (464)$$

Growth According to Peng (Formula=2)

Growing crystalline phases is often described by a growth velocity, for example, in [1]. Some of the growing model has been adopted in the following local model, which reads as:

$$c_G = r_0 [1 - \exp(-\beta \Delta G(T) v_{MM})] \exp(-\beta E_{act}) \quad (465)$$

if $T < T_m$; otherwise, it is zero.

Single-Trap Transition (Formula=7)

This model depends on the carrier density d_c . It resembles some standard trap models and is intended to be used in two-state MSCs only. If the number of electrons in the reference state is greater than in the interacting state, that is, $K_i > K_j$, then the capture process depends on the electron density and uses $d_c = n$. If $K_i < K_j$, then $d_c = p$; otherwise, you use $d_c = 0$. The capture rate is then given by:

$$c = \sigma v_{th} d_c \quad (466)$$

where σ is the cross section and v_{th} is the thermal velocity. The emission rate for electron capturing is computed as:

$$e = c \exp(\beta(E_T - E_{F,n})) \quad (467)$$

where E_T is the trap energy. For hole-capturing, the emission rate reads as:

$$e = c \exp(\beta(E_{F,p} - E_T)) \quad (468)$$

MSC Trap with Emulated Detailed Balance (Formula=8)

The actual model generalizes the single-trapping model (Formula=7) to general MSC states and transitions. The capture rate is computed as in the single trapping case above. The emission rate, however, is computed as follows. Let N_n and N_p be the number of electrons and holes, respectively, which are destroyed during the process. Then, you have:

$$N_n - N_p = K_i - K_j \quad (469)$$

and you require:

$$\frac{e}{c} = g_{ji} \exp(-N_n \beta_n (E_{F,n} - \overline{E_C}) + N_p \beta_p (E_{F,p} - \overline{E_V}) + \beta_l (-N_n \overline{E_C} + N_p \overline{E_V} - H_{ji})) \quad (470)$$

where you used $g_{ji} = g_j/g_i$ and $H_{ji} = H_j - H_i$. The average conduction band and valence band energies are:

$$\overline{E_C} = E_C + \frac{3}{2}kT_n \quad \text{and} \quad \overline{E_V} = E_V - \frac{3}{2}kT_p \quad (471)$$

In thermal equilibrium, the detailed balance principle is satisfied.

Model Parameters

The model requires parameters for all states and transitions to allow a consistent computation of the thermal equilibrium of the MSC as a whole. Therefore, the parameters are grouped into global, state, and transition parameters.

The parameters `nb_states` and `nb_transitions` determine the number of states and transitions, respectively, and must be both consistent with the command file specification. The parameter `sindex<int>` specifies for the <int>-th state a parameter index, which determines a prefix for the state parameters. For example, given `sindex0=5`, the parameters prefixed with `s5_` are read for the 0-th state. A similar parameter index selection is available for transitions using the parameters `tindex<int>` (for example, `tindex2=7` reads the parameters prefixed by `t7_`).

For transitions, the specified parameter index must coincide with the model index of the transition in the command file. [Table 80 on page 433](#) lists the available global parameters.

Table 80 Global parameters

Name	Symbol	Default	Unit	Range	Description
plot	–	0	–	{0, 1}	Plots some properties to screen
nb_states	–	0	–	≥ 0 , integer	Number of states
nb_transitions	–	0	–	≥ 0 , integer	Number of transitions
sindex<int>	–	–	–	≥ 0 , integer	Parameter index of <int>-th state
tindex<int>	–	–	–		Parameter index of <int>-th transition

Table 81 lists the global material and default transition parameters.

Table 81 Global material and default transition parameters of pmi_ce_msc

Name	Symbol	Default	Unit	Range	Description
E _g	E_g	0.5	eV	> 0 .	Band gap
N _C	N_C	1×10^{19}	cm ⁻³	> 0 .	Electron density-of-states
N _V	N_V	1×10^{19}	cm ⁻³	> 0 .	Hole density-of-states
T _m	T_m	1×10^{100}	K	> 0 .	Melting temperature
T _g	T_g	1×10^{100}	K	> 0 .	Glass temperature
DeltaH1	ΔH_1	0.	J/cm ³	> 0 .	Heat of solid-to-liquid
DeltaH2	ΔH_2	0.	J/cm ³	> 0 .	Heat of amorphous-to-crystalline
GammaSL	γ_{SL}	0.	J/cm ²	> 0 .	Interfacial free-energy density
MM_volume	v_{MM}	0.	cm ³	> 0 .	Monomer volume
Reference_E_T	–	0	–	{0, 1, 2}	Reference energy for E_T

Table 82 Reference_E_T interpretation

Value	Symbol	Description
0	$E_T = (E_C + E_V)/2 + E_{T, \text{user}}$	Midgap
1	$E_T = E_C - E_{T, \text{user}}$	Conduction band
2	$E_T = E_V + E_{T, \text{user}}$	Valence band

[Table 83](#) lists the state parameters. The parameter E specifies a constant base energy.

Table 83 State parameters

Name	Symbol	Default	Unit	Range	Description
E	E_i	0.	eV	real	Base energy
g	g_i	1.	1	>0.	Degeneracy
charge	$-K_i$	0	1	integer	Number of positive elementary charges
E_charge	$E_{i, \text{user}}^-$	0.	eV	real	Particle energy with respect to valence band
E_nb_Tpairs	–	0	–	>=0	Number of interpolation points for base energy
E_Tp<int>_X	–	–	K		Temperature of <int>-th interpolation point
E_Tp<int>_Y	–	–	eV		Energy of <int>-th interpolation point

Alternatively, the base energy can be described as a piecewise linear (pwl) function of the temperature: With E_nb_Tpairs, you specify the number of interpolation points. For the <int>-th interpolation point ($0 \leq \text{<int>} < \text{E_nb_Tpairs}$), you must specify with E_Tp<int>_X the temperature and with E_Tp<int>_Y, the corresponding energy. The pwl specification is used if E_nb_Tpairs is greater than zero.

Some transition parameters are inherited from the global specification (see [Table 81 on page 433](#)). They can be overwritten for specific transitions by using the appropriate parameter prefix. [Table 84](#) lists additional transition parameters.

Table 84 Transition parameters of pmi_ce_msc

Name	Symbol	Default	Unit	Range	Description
CB_nb_particles	N_n	0	1	integer	Particle number of reservoir CB
E_T	$E_{T, \text{user}}$	0.	eV	real	Trap energy
Eact	E_{act}	0.	eV	real	Activation energy
formula	–	–	–	Table 79 on page 430	Model selection
r0	r_0	1.	Hz	>0.	Maximal frequency
s0	–	–	–		Parameter index of ‘to’ state
s1	–	–	–		Parameter index of ‘from’ state
sigma	σ	1×10^{-15}	cm^2	>0	Cross section
VB_nb_particles	N_p	0	1	integer	Particle number of reservoir VB
vth	v_{th}	1×10^7	cm/s	>0	Thermal velocity

Interaction of Multistate Configurations with Transport

The MSCs have a direct impact on the transport through their charge density and their recombination rates with selected reservoirs. Furthermore, several physical models can be made explicitly dependent on the occupation probabilities of a specific MSC by using the PMI.

Apparent Band-Edge Shift

The keywords `eBandEdgeShift`, `hBandEdgeShift`, or `BandEdgeShift` in an `MSConfig` section switch on band-edge shift models for the conduction band, the valence band, or both, respectively. The model itself is either the MSC-dependent `pmi_msc_abes` model (see [The pmi_msc_abes Model on page 436](#)) or a user-defined `PMI_MSC_ApparentBandEdgeShift` model as described in [Multistate Configuration-dependent Apparent Band-Edge Shift on page 1064](#).

The apparent band-edge shifts become visible only if the density gradient transport model is used. To avoid the implicit use of the density gradient quantum correction model, the (electron and hole) gamma parameters in the `QuantumPotentialParameters` parameter set must be set to zero in the parameter file.

A typical simulation is:

```
Physics {
  MSConfigs (
    MSConfig ( ...
      eBandEdgeShift ( "pmi_abes" 0 )
      hBandEdgeShift ( "pmi_abes" 1 )
    )
  )
  eQuantumPotential
  hQuantumPotential
}
Solve {
  Coupled { Poisson Electron Hole Temperature eQuantumPotential
            hQuantumPotential }
  Transient ( ... ) {
    Coupled { Poisson Electron Hole Temperature eQuantumPotential
            hQuantumPotential }
  }
}
```

Here, the user PMI model `pmi_abes` has been used for both the conduction and valence bands.

The pmi_msc_abes Model

The pmi_msc_abes model depends on the lattice temperature T and, if an MSC is specified, on the state occupation probabilities s_i , and it reads as:

$$\Lambda = \sum_i \Lambda_i(T) s_i \quad (472)$$

where the sum is taken over all MSC states, and $\Lambda_i(T)$ is the apparent band-edge shift (ABES) of state i .

The model is activated by using (here, for electrons, a MSC named "m0", and a model index 5) as described above:

```
eBandEdgeShift ( "pmi_msc_abes" 5 )
```

The model uses a two-level or three-level hierarchy (depending on use_mi_pars) to determine the state parameters, namely, the global, the model index, and the state parameters. [Table 85](#) lists the global parameters.

Table 85 Global, model-string, and state parameters of pmi_msc_abes

Name	Symbol	Default	Unit	Range	Description
plot	—	0	—	{0,1}	Plot parameter to screen
use_mi_pars	—	0	—	{0,1}	Use model index for parameter set
lambda	Λ	0.	eV	real	Constant value
lambda_nb_Tpairs	—	0	—	≥ 0	Number of interpolation points
lambda_Tp<int>_X	—	—	K	$> 0.$	Temperature at <int>-th interpolation point
lambda_Tp<int>_Y	—	—	eV	real	Value at <int>-th interpolation point

The names of the model index parameters are prefixed with:

```
mi<model_index>_
```

and the state parameters have one of the prefixes:

```
mi<model_index>_<state_name>_
<state_name>_
```

depending on the value of use_mi_pars. The lambda_ parameters enable either a constant or pwl apparent band-edge shift for each state, which is fully analogous to the specification described in [The pmi_msc_heatcapacity Model on page 746](#).

Thermal Conductivity, Heat Capacity, and Mobility

The following models allow the dependency on MSC occupation probabilities:

- [The pmi_msc_thermalconductivity Model on page 748](#)
- [The pmi_msc_heatcapacity Model on page 746](#)
- [The pmi_msc_mobility Model on page 307](#)

Descriptions of PMIs for MSC-dependent thermal conductivity, heat capacity, and mobility can be found in [Multistate Configuration-dependent Thermal Conductivity on page 1088](#), [Multistate Configuration-dependent Heat Capacity on page 1095](#), and [Multistate Configuration-dependent Bulk Mobility on page 1029](#), respectively.

Manipulating MSCs During Solve

The MSC dynamic is, in general, solved implicitly. However, sometimes it may be useful to manipulate the computations. For example, you may want to initialize MSCs with nonsteady-state solutions or to freeze the dynamic of MSCs because time constants are very large compared to electronic and thermal effects. Furthermore, it may be of interest to disable specific MSC transitions for certain applications (for example, in phase-change memory applications to freeze phases but to allow electronic transitions). The available mechanisms are described here.

Explicit State Occupations

You can set explicitly the state occupations of MSCs to a spatial-independent and solution-independent value, and freeze and unfreeze the dynamic of the MSCs during Solve by using `MSConfigs` in a `Set` command (see [Table 156 on page 1234](#) for a summary of options).

To set the state occupations of an MSC, you use the `MSConfig` command within `MSConfigs`, identify the MSC by using its name (and, for mixed-mode simulations, the device name using `Device`), and specify the occupancies for the involved states using `State`.

The state occupations are then set collectively and normalized implicitly, while unspecified states default to zero occupation. The `Frozen` option of `MSConfigs` applies to all existing MSCs and freezes the state occupations at the set or actual values. To unfreeze the dynamics of the MSCs again, the `-Frozen` option is used.

18: Phase and State Transitions

Manipulating MSCs During Solve

The following example illustrates the syntax for single-device simulations:

```
Physics {
  MSConfigs ( ...
    MSConfig ( Name="m1" State ( Name="s0" ) State ( Name="s1" ) ... )
  )
}
Solve { ...
  Set (
    MSConfigs (
      MSConfig ( Name="m1"
        State (Name="s0" Value=0.1) State (Name="s1" Value=0.4) )
        Frozen                                # freeze the MSC dynamic
      )
    )
  ...
  Set ( MSConfigs ( -Frozen ) )              # unfreeze the MSC dynamic
}
```

Here, the occupancies of states s0 and s1 of the MSC m1 are set to 0.2 and 0.8, respectively, due to the implicit normalization; all other states are unoccupied.

NOTE Freezing and unfreezing MSCs implies freezing and unfreezing traps, respectively (see the `Solve-Set-Traps` command in [Explicit Trap Occupation on page 421](#)).

Manipulating Transition Dynamics

You can manipulate the transition dynamic by accessing prefactors for individual transition forward (capture) and backward (emission) reaction rates. This means that the modified rate $\tilde{c} = \kappa c$ (κ is the prefactor, and c is the true reaction rate) is used in the dynamic equation. By setting selected transition prefactors to zero, you can decouple states into independent groups. Especially, if you decouple a certain state from all others, the state is frozen, that is, it retains its occupation in the subsequent analysis. With:

```
Set( (Device="d1" MSConfig="m7" Transition="t3" CPreFactor=0. EPreFactor=0.) )
```

both reaction rates of the specified transition are set to zero (`PreFactor` can be used for common settings of both reaction rates). Omitting one of the specifying string keywords `Device`, `MSConfig`, and `Transition` applies the setting to all corresponding objects in the `Device-MSConfig-Transition` hierarchy.

NOTE The prefactors affect only subsequent computations, but they do not change the physical parameters of the MSC.

Example: Two-State Phase-Change Memory Model

Phase-change memory (PCM) devices store a bit as the phase (crystalline or amorphous) of a (chalcogenide) material. Reading information takes advantage of the different conductances of the phases. Storing information requires switching the phases. To switch to the amorphous phase, a high current is passed through the material, heating up the device above the melting point. When switching off the current, the molten material cools so rapidly that it cannot crystallize, but remains in a metastable amorphous phase. To switch to the crystalline phase, the material is reheated more gently. The material remains solid, but the temperature is sufficient that the phase can relax to the equilibrium crystalline phase.

The PCM device is modeled by a two-state model representing the crystalline and amorphous phase using an MSC with two uncharged states. The transition is modeled by the Arrhenius law formula (Formula=0) of the `pmi_ce_msc` transition model. The model has four parameters:

- The base energies and the degeneracy factors of the states determine the equilibrium.
- The activation energy E_{act} and r_0 of the transition determine the velocity of the transition.

A short interpretation is given:

Let $s = s_c$ denote the degree of crystallization of the material and $s_a = 1 - s$, the amorphization rate. Assuming an energy difference $\delta E > 0$ between the amorphous and crystalline phases, and that the amorphous phase has $g > 1$ times more microscopic realizations than the crystalline state, in equilibrium $s = 1/[g \exp(-\delta E/kT) + 1]$, that is, the amorphous state is preferred at higher temperatures. The critical temperature where the equilibrium occupation rates s_c and s_a are equal is $T_{\text{crit}} = \delta E/k \ln(g)$.

The dynamic is given by crystallization and amorphization rates $r_c = r_0 \exp(-E_{\text{act}}/kT)$ and $r_a = r_0 g \exp(-(E_{\text{act}} + \delta E)/kT)$, respectively, assuming a simple Arrhenius law for the crystallization rate. Here, r_0 is the maximal crystallization rate, and E_{act} is the activation energy.

References

- [1] C. Peng, L. Cheng, and M. Mansuripur, “Experimental and theoretical investigations of laser-induced crystallization and amorphization in phase-change optical recording media,” *Journal of Applied Physics*, vol. 82, no. 9, pp. 4183–4191, 1997.
- [2] S. Senkader and C. D. Wright, “Models for phase-change of $\text{Ge}_2\text{Sb}_2\text{Te}_5$ in optical and electrical memory devices,” *Journal of Applied Physics*, vol. 95, no. 2, pp. 504–511, 2004.
- [3] D.-H. Kim *et al.*, “Three-dimensional simulation model of switching dynamics in phase change random access memory cells,” *Journal of Applied Physics*, vol. 101, p. 064512, March 2007.

CHAPTER 19 Degradation Model

This chapter discusses the degradation models used in Sentaurus Device.

Overview

A necessary part of predicting CMOS reliability is the simulation of the time dependence of interface trap generation. To cover as wide a range as possible, this simulation should accurately reflect the physics of the interface trap formation process. Sentaurus Device provides three degradation models:

- Trap degradation model
- Multistate configuration (MSC)–hydrogen transport degradation model
- Two-stage negative bias temperature instability (NBTI) degradation model

Trap Degradation Model

Disorder-induced variations among the Si-H activation energies at the passivated Si–SiO₂ interface have been shown [1] to be a plausible source of the sublinear time dependence of this trap generation process. Diffusion of hydrogen from the passivated interface was used to explain some time dependencies [2]. In addition, this could be due to a Si-H density–dependent activation energy [3], which may be due to the effects of Si-H breaking on the electrical and chemical potential of hydrogens at the interface. Furthermore, the field dependence of the activation energy due to the Poole–Frenkel effect can be considered, so that all these factors lead to enhanced trap formation kinetics.

Trap Formation Kinetics

The main assumption about trap formation is that initially dangling silicon bonds at the Si–SiO₂ interface were passivated by hydrogen (H) or deuterium (D) [4], and degradation is a depassivation process where hot carrier interactions with Si-H/D bonds or other mechanisms are responsible for this. The equations of the model are solved self-consistently with all transport equations.

Power Law and Kinetic Equation

The experimental data for the kinetics of interface trap formation [5] shows that the time dependence of trap generation can be described by a simple power law: $N_{it} - N_{it}^0 = N_{hb}^0 / (1 + (vt)^\alpha)$, where N_{it} is the concentration of interface traps, and N_{hb}^0 and N_{it}^0 are the initial concentrations of Si-H bonds (or the concentration of hydrogen on Si bonds) and interface traps, respectively.

Assuming $N = N_{hb}^0 + N_{it}^0$ total Si bonds at the interface, the remaining number of Si-H bonds at the interface after stress is $N_{hb} = N - N_{it}$ and follows the power law:

$$N_{hb} = \frac{N_{hb}^0}{1 + (vt)^\alpha} \quad (473)$$

Based on experimental observations, the power α is stress dependent and varies between 0 and 1.

From first-order kinetics [1], it is expected that the Si-H concentration during stress obeys:

$$\frac{dN_{hb}}{dt} = -vN_{hb} \quad (474)$$

where v is a reaction constant that can be described by $v \propto v_A \exp(-\epsilon_A/kT)$ in the Arrhenius approximation, ϵ_A is the Si-H activation energy, and T is the Si-H temperature. The exponential kinetics given by this equation ($N_{hb} = N_{hb}^0 \exp(-vt)$) do not fully describe the experimental data because a constant activation energy will behave like the power law in Eq. 473, but with power $\alpha \approx 1$.

Si-H Density-dependent Activation Energy

This section describes an activation energy parameterization to capture the sublinear power law for the time dependence of interface trap generation. There is evidence that the hydrogen atoms, when removed from the silicon, remain negatively charged [6]. If this correct, the hydrogen can be expected to remain in the vicinity of the interface and will affect the breaking of additional silicon-hydrogen bonds by changing the electrical potential.

The concentration of released hydrogen is equal to $N - N_{hb}$, so the activation energy dependence (assuming the activation energy changes logarithmically with the breaking of Si-H) can be expressed as:

$$\epsilon_A = \epsilon_A^0 + (1 + \beta)kT \ln \left(\frac{N - N_{hb}}{N - N_{hb}^0} \right) \quad (475)$$

where the last term represents the Si-H density-dependent change with a prefactor $1 + \beta$. Note that $(N - N_{hb}) / (N - N_{hb}^0)$ is the fraction of traps generated to the total initial traps, and this gives the form of the chemical potential of Si-H bonds with the prefactor $1 + \beta$.

The numeric solution of the kinetic equation with the varying activation energy above clearly shows that such a Si-H density-dependent activation energy gives a power law, and the power α is a function of the prefactor $1 + \beta$. From the available experimental data of interface trap generation, it was noted that for negative gate biases $\alpha > 0.5$, but for positive ones $\alpha < 0.5$. It is interesting that the solution of the above kinetic equation gives $\alpha \approx 0.5$ in the equilibrium case where a unity prefactor is used. In nonequilibrium, a polarity-dependent modification of the prefactor (field stretched and pressed Si-H bonds) is possible.

Diffusion of Hydrogen in Oxide

Another model that can interpret negative bias temperature instability (NBTI) phenomena and different experimental slopes in degradation kinetics is the R-D model [7]. This model considers hydrogen in oxide, which diffuses from the silicon-oxide interface, but the part of the hydrogen that remains at the interface controls the degradation kinetics.

The diffusion of hydrogen in oxide can be expressed as follows:

$$\begin{aligned} D \frac{dN_H}{dx} &= \frac{dN_{hb}}{dt} & x &= 0 \\ \frac{dN_H}{dt} &= D \frac{d^2 N_H}{dx^2} & 0 < x < x_p \\ D \frac{dN_H}{dx} &= -k_p(N_H - N_H^0) & x &= x_p \end{aligned} \quad (476)$$

where N_H is a concentration of hydrogen in oxide, $D = D_0 \exp(-\epsilon_H/kT)$ is its diffusion coefficient, $x = 0$ is a coordinate of the silicon-oxide interface, $x = x_p$ is the coordinate of the oxide-polysilicon gate interface (which is equal to the oxide thickness), k_p is the surface recombination velocity at the oxide-polysilicon gate interface, and N_H^0 is an equilibrium (initial) concentration of hydrogen in the oxide.

Syntax and Parameterized Equations

The general kinetic equation (left column of Eq. 477) with an added passivation term can be activated by the keyword `Degradation`. The power law (right column of Eq. 477) is activated by the keyword `Degradation(PowerLaw)`:

Kinetic	Power Law	
$\frac{dN_{hb}}{dt} = -vN_{hb} + \gamma(N - N_{hb})$	$N_{hb} = \frac{N_{hb}^0}{1 + (vt)^\alpha}$	
$\gamma = \gamma_0[N_H/N_H^0 + \Omega(N_{hb}^0 - N_{hb})]$	$\alpha = 0.5 + \beta$	(477)
$\gamma_0 = \frac{N_{hb}^0}{N - N_{hb}^0} v_0$		

where γ_0 is the passivation constant, which is computed automatically by default, to provide the equilibrium, but you can specify it directly in the command file. Ω is the passivation volume (by default, it is equal to zero), and it represents a simple model of the effect that depassivated hydrogen can be trapped by dangling silicon bonds again. Depassivated hydrogen increases the average hydrogen concentration N_H near traps, which is computed from the R-D model (see [Diffusion of Hydrogen in Oxide on page 443](#)). If the R-D model is not activated, then $N_H/N_H^0 = 1$.

The degradation model can be activated for any trap level or distribution (see [Chapter 17 on page 407](#)). Its keywords are described in [Table 261 on page 1316](#) (see the options referred to in [Chapter 19 on page 441](#)) and can be used with any other trap options listed in the same table. The model applied to the trap distribution $N(\epsilon)$ controls the total trap concentration $\int_{E_V}^{E_C} N(\epsilon) d\epsilon$.

The parameterized system of equations for the reaction constant v based on the trap formation model (see [Trap Formation Kinetics on page 441](#)) and [3] can be expressed as:

$$\begin{aligned}
 v &= v_0 \exp\left(\frac{\epsilon_A^0}{kT_0} - \frac{\epsilon_A^0 + \Delta\epsilon_A}{\epsilon_T}\right) k_{FN} k_{HC} k_{SHE} \\
 \epsilon_T &= kT + \delta_{//} |F_{//}|^{p_{//}} \\
 k_{HC} &= 1 + \delta_{HC} |I_{HC}|^{p_{HC}} \\
 k_{FN} &= 1 + \delta_{Tun} |I_{Tun}|^{p_{Tun}} \\
 \Delta\epsilon_A &= -\delta_{\perp} |F_{\perp}|^{p_{\perp}} + (1 + \beta) \epsilon_T \ln \frac{N - N_{hb}}{N - N_{hb}^0} \\
 \beta &= \beta_0 + \beta_{\perp} F_{\perp} + \beta_{//} F_{//}
 \end{aligned} \tag{478}$$

where v_0 (given in the command file) is the reaction (depassivation) constant at the passivation equilibrium (for the passivation temperature T_0 and for no changes in the activation energy $\Delta\epsilon_A = 0$). $F_\perp, F_\parallel$ are perpendicular and parallel components of the electric field F to the interface where traps are located. The perpendicular electric field $F_\perp$ has a positive sign if the space charge at the interface is positive. I_{HC}, I_{Tun} are the local densities of hot carrier and tunneling (Fowler–Nordheim, direct, and direct nonlocal tunneling) currents at the interface where traps are located. See [Chapter 24 on page 605](#) and [Chapter 25 on page 625](#) for more information about tunneling and hot-carrier injection models, respectively.

ϵ_T is the energy of hydrogen on Si-H bonds and is equal to kT plus some possible gain from hot carriers represented as the additional term that is dependent on the parallel component of the electric field $\delta_\parallel |F_\parallel|^{\rho_\parallel}$. $\Delta\epsilon_A$ is a change of the activation energy because of stretched Si-H bonds [8] by the electric field (first term) and due to a change of the chemical potential (second term) [3]. Effectively, the influence of the chemical potential also can be different in the presence of the electric field, and the coefficient β represents this. Coefficients $\delta_\perp, \rho_\perp, \delta_\parallel, \rho_\parallel, \delta_{HC}, \rho_{HC}, \delta_{Tun}, \rho_{Tun}$, and $\beta_0, \beta_\perp, \beta_\parallel$ are field and current enhancement parameters of the model. [Table 261 on page 1316](#) lists the options available for the degradation model.

When the electron energy distribution is available by specifying the keyword `eSHEDistribution` in the Physics section (see [Using Spherical Harmonics Expansion Method on page 638](#)), you can include the following additional spherical harmonics expansion (SHE) distribution enhancement factor:

$$k_{SHE} = 1 + \delta_{SHE} \frac{qg_v}{2} \int_{\epsilon_{th}}^{\infty} \left(\min \left[\exp \left(\frac{\epsilon - \epsilon_a + \delta_\perp |F_\perp / F_0|^{\rho_\perp}}{kT} \right), 1 \right] g(\epsilon) f(\epsilon) v(\epsilon) \right) d\epsilon \quad (479)$$

where:

- δ_{SHE} is a prefactor.
- ϵ_{th} is a threshold energy.
- ϵ_a is an activation energy.
- $\delta_\perp$ is a normal field–induced activation energy–lowering factor.
- $\rho_\perp$ is a normal field–induced activation energy–lowering exponent.
- $F_0 = 1 \text{ V/cm}$.
- g_v is the valley degeneracy.
- g is the density-of-states.

19: Degradation Model

Trap Degradation Model

- f is the electron energy distribution.
- v is the magnitude of the electron velocity.

This enhancement factor is multiplied by the reaction coefficient. The SHE distribution enhancement factor can be specified by the keyword `eSHEDistribution` in the `Traps` statement. Similarly, the keyword `hSHEDistribution` specifies the enhancement factor of the hole energy distribution.

The following example specifies the SHE distribution enhancement factor with $\delta_{\text{SHE}} = 100 \text{ cm}^2/\text{A}$, $\epsilon_{\text{th}} = 2 \text{ eV}$, $\epsilon_{\text{a}} = 2.1 \text{ eV}$, $\delta_{\perp} = 10^{-7} \text{ eV}$, and $\rho_{\perp} = 1$:

```
Physics(MaterialInterface="Silicon/Oxide") {  
    Traps(...  
        Degradation  
        eSHEDistribution=(1.0e2 2 2.1 1.0e-7 1) # (del_SHE E_th E_a del_p rho_p)  
        ...  
    )  
}
```

Device Lifetime and Simulation

Based on [Syntax and Parameterized Equations on page 444](#), the `Physics` section of a command file that describes the parameters of the degradation model can be:

```
Physics(MaterialInterface="Silicon/Oxide"){  
    Traps(Conc=1e8 EnergyMid=0 Acceptor #FixedCharge  
        Degradation #(PowerLaw)  
            ActEnergy=2 BondConc=1e12  
            DePasCoeff=8e-10  
            FieldEnhan=(0 1 1.95e-3 0.33)  
            CurrentEnhan=(0 1 6e+5 1)  
            PowerEnhan=(0 0 -1e-7)  
    )  
    GateCurrent(GateName="gate" Lucky(CarrierTemperatureDrive) Fowler)  
}
```

For this input, the initially specified trap concentration is 10^8 cm^{-2} and, in the process of degradation, it can be increased up to 10^{12} cm^{-2} . The activation energy of hydrogen on Si-H bonds is 2 eV and the depassivation constant at the equilibrium is equal to $8 \times 10^{-10} \text{ s}^{-1}$. The degradation simulation can be separated into two parts:

- Simulation of extremely stressed devices with existing experimental data and fitting to the data by modification of the field and current-dependent parameters.
- Simulation of normal-operating devices to predict device reliability (lifetime).

For the first part of the degradation simulation, the typical Solve section can be:

```
Solve {
  NewCurrentPrefix="tmp"
  coupled (iterations=100) { Poisson }
  coupled { poisson electron hole }
  Quasistationary( InitialStep=0.1 MaxStep=0.2 MinStep=0.0001
    increment=1.5 Goal{name="gate" voltage=-10} )
    { coupled { poisson electron hole } }
  NewCurrentPrefix=""
  coupled { poisson electron hole }
  transient( InitialTime=0 Finaltime = 100000
    increment=2 InitialStep=0.1 MaxStep=100000 ){
    coupled{ poisson electron hole }
  }
}
```

The first Quasistationary ramps the device to stress conditions (in this particular case, to high negative gate voltage), the second transient simulates the degradation kinetics (up to 10^5 s, which is a typical time for stress experimental data).

NOTE The hot carrier currents are postprocessed values and, therefore, InitialStep should not be large.

To monitor the trap formation kinetics in transient, you can use Plot and CurrentPlot statements to output TotalInterfaceTrapConcentration, TotalTrapConcentration, and OneOverDegradationTime, for example:

```
CurrentPlot{
  eDensity(359) Potential(359)
  eInterfaceTrappedCharge(359) hInterfaceTrappedCharge(359)
  OneOverDegradationTime(359) TotalInterfaceTrapConcentration(359)
}
```

where a vertex number is specified to have a plot of the fields at some location on the interface. As a result, the behavior of these values versus time can be seen in the plot file of Sentaurus Device.

The prediction of the device lifetime can be performed in two different ways:

- Direct simulation of a normal-operating device in transient for a long time (for example, 30 years).
- Extrapolation of the degradation of a stressed device by computation of the ratio between depassivation constants for stressed and unstressed conditions.

19: Degradation Model

Trap Degradation Model

The important value here is the critical trap concentration N_{crit} , which defines an edge between a properly working and improperly working device. Using N_{crit} , the device lifetime τ_D is defined as follows (according to the two different prediction ways):

1. In transient, direct computation of time $t = \tau_D$ gives the trap concentration equal to N_{crit} .
2. In Quasistationary, if the previously finished transient statement computes the device lifetime τ_D^{stress} and the depassivation constant v^{stress} at stress conditions, then $\tau_D = (v^{stress}/v)\tau_D^{stress}$.

So, the plotted value of OneOverDegradationTime is equal to $1/\tau_D$ for one trap level and the sum $\sum 1/\tau_D^i$ if several trap levels are defined for the degradation. It is computed for each vertex where the degradation model is applied and can be considered as the lifetime of local device area.

For the second approach to device lifetime computation, the following Solve statement can be used:

```
Solve {
  NewCurrentPrefix="tmp"
  coupled (iterations=100) { Poisson }
  coupled { poisson electron hole }

  Quasistationary( InitialStep=0.1 MaxStep=0.2 Minstep=0.0001
    increment=1.5 Goal{name="gate" voltage=-10} )
    { coupled { poisson electron hole } }

  NewCurrentPrefix=""
  coupled { poisson electron hole }
  transient( InitialTime=0 Finaltime = 100000
    increment=2 InitialStep=0.1 MaxStep=100000 ){
    coupled{ poisson electron hole }
  }

  set(Trapfilling=-Degradation)

  coupled { poisson electron hole }
  Quasistationary( InitialStep=0.1 MaxStep=0.2 Minstep=0.0001 increment=1.5
    Goal{name="gate" voltage=1.5} )
    { coupled { poisson electron hole } }
  Quasistationary( InitialStep=0.1 MaxStep=0.2 Minstep=0.0001 increment=1.5
    Goal{name="drain" voltage=3} )
    { coupled { poisson electron hole } }
}
```

The statement `set(Trapfilling=-Degradation)` returns the trap concentrations to their unstressed values (it is not necessary to include, but it may be interesting to check the

influence). The first `Quasistationary` statement after the `set` command returns the normal-operating voltage on the gate and, in the second, you can plot the dependence of $1/\tau_D$ on applied drain voltage. The last dependence could be useful to predict an upper limit of operating voltages where the device will work for a specified time.

MSC–Hydrogen Transport Degradation Model

Bias and temperature stresses generate interface fixed charges and trapped charges near the oxide interface. These immobile charges affect the threshold voltage (see [Chapter 7 on page 191](#)) and the carrier mobility (see [Mobility Degradation Components due to Coulomb Scattering on page 332](#)). To explain these degradation phenomena, various physical models have been proposed [\[9\]\[10\]\[11\]\[12\]](#). Although the details of these models are different, they commonly involve charge-trapping, silicon-hydrogen bond depassivation, and hydrogen transport.

To model charge-trapping, interactions between localized trapping centers and mobile carriers must be considered. Contrary to the standard traps in [Chapter 17 on page 407](#), the trapping centers used in the degradation model usually involve hydrogens. Therefore, interactions between the localized trapping centers and mobile hydrogens should be considered. In addition, the trapping centers may have more than two internal states depending on the structural relaxation and the presence of charges and hydrogens [\[12\]](#).

The multistate configurations (MSCs) introduced in [Chapter 18 on page 425](#) can be used to represent these complex trapping centers because the MSCs can handle an arbitrary number of states and their transitions involving the capture and emission of charges and hydrogens.

Finally, hydrogen transport must be considered if the degradation model involves mobile hydrogens. Hydrogen atoms, hydrogen molecules, and hydrogen ions can contribute to the hydrogen transport, and there can be chemical reactions between them [\[9\]\[10\]\[11\]](#).

Sentaurus Device provides the following capabilities to model oxide degradation:

- Transport equations for hydrogen atoms, hydrogen molecules, and hydrogen ions.
- An arbitrary number of interface and bulk reactions among mobile elements (hydrogen atoms, hydrogen molecules, hydrogen ions, electrons, and holes).
- An arbitrary number of interface and bulk reactions among mobile elements and localized states by using the multistate configurations (see [Chapter 18 on page 425](#)).

Hydrogen Transport

Transport equations for hydrogen atoms (X_1), hydrogen molecules (X_2), and hydrogen ions (X_3) can be written as:

$$\frac{\partial}{\partial t}[X_i] + \nabla \cdot \left[D_i \exp\left(-\frac{E_{di}}{kT}\right) \left(\frac{qK_i^Q}{kT} F[X_i] - \nabla[X_i] - \alpha_{td}[X_i] \nabla \ln T \right) \right] + R_{\text{net}} + r_i([X_i] - [X_i]_0) = 0 \quad (480)$$

where:

- D_i is the diffusion coefficient.
- E_{di} is the diffusion activation energy.
- α_{td} is the prefactor of the thermal diffusion term.
- K_i^Q is the number of charges for element X_i .
- R_{net} is the net recombination rate due to chemical reactions.
- r_i is the explicit recombination rate.
- $[X_i]_0$ is the reference density.

At electrodes, $[X_i] = [X_i]_0$ is assumed as a boundary condition.

By default, the initial densities of hydrogen atoms, molecules, and ions are all zero. You can load the initial density of each hydrogen element from the `PMIUserField` by using the `Set` command in the `Solve` section, for example:

```
Solve {
  ...
  Set ( HydrogenAtom = "PMIUserField0" )
  Set ( HydrogenMolecule = "PMIUserField1" )
  Set ( HydrogenIon = "PMIUserField3" )
  ...
}
```

Reactions Between Mobile Elements

In the model, you can specify an arbitrary number of bulk and interface chemical reactions between hydrogen atoms (X_1), hydrogen molecules (X_2), hydrogen ions (X_3), electrons (X_4), and holes (X_5). Each reaction is defined by the following reaction equation:

$$\sum_{i=1}^5 \alpha_i X_i \leftrightarrow \sum_{i=1}^5 \beta_i X_i \quad (481)$$

where the nonnegative integers α_i and β_i are the particle numbers of element X_i to be removed and created by the forward reaction.

These coefficients must satisfy the charge and hydrogen conservation laws:

$$\sum_{i=1}^5 (\alpha_i - \beta_i) K_i^Q = 0 \quad (482)$$

$$\sum_{i=1}^5 (\alpha_i - \beta_i) K_i^H = 0 \quad (483)$$

where K_i^Q and K_i^H are the number of charges and the number of hydrogen atoms for element X_i (for K_i^Q and K_i^H of each mobile element, see [Table 78 on page 426](#)).

The forward and reverse reaction rates R_f and R_r are modeled as:

$$R_f = k_f \exp\left(\delta_f F - \frac{E_f}{kT}\right) \prod_{i=1}^5 \left(\frac{[X_i]}{1/\text{cm}^3}\right)^{\alpha_i} \quad (484)$$

$$R_r = k_r \exp\left(\delta_r F - \frac{E_r}{kT}\right) \prod_{i=1}^5 \left(\frac{[X_i]}{1/\text{cm}^3}\right)^{\beta_i} \quad (485)$$

where:

- F is the magnitude of the electric field [V/cm].
- k_f and k_r are the forward and reverse reaction coefficients, respectively ($[\text{cm}^{-3}\text{s}^{-1}]$ for bulk reactions and $[\text{cm}^{-2}\text{s}^{-1}]$ for interface reactions).
- δ_f and δ_r are the forward and reverse reaction field coefficients, respectively $[\text{cm V}^{-1}]$.
- E_f and E_r are the forward and reverse reaction activation energies, respectively [eV].

NOTE For a reaction specified at a semiconductor–insulator interface, the insulator electric field is used instead of the semiconductor electric field.

A reaction can be specified in the Physics section as an argument `HydrogenReaction` to the `HydrogenDiffusion` keyword. For example, consider the dimerization of two hydrogen atoms into a hydrogen molecule $2\text{H} \leftrightarrow \text{H}_2$ in the oxide region with $k_f = 10^{-3} \text{ cm}^{-3}\text{s}^{-1}$ and $k_r = 10^2 \text{ cm}^{-3}\text{s}^{-1}$.

19: Degradation Model

MSC–Hydrogen Transport Degradation Model

The corresponding command file can be written as:

```
Physics ( Material = "Oxide" ) {
  HydrogenDiffusion(
    HydrogenReaction( # 2H <-> H_2
      LHSCoef ( # alpha_i
        HydrogenAtom = 2
        HydrogenMolecule = 0
        HydrogenIon = 0
        Electron = 0
        Hole = 0
      )
      RHSCoef ( # beta_i
        HydrogenAtom = 0
        HydrogenMolecule = 1
        HydrogenIon = 0
        Electron = 0
        Hole = 0
      )
      ForwardReactionCoef = 1.0e-3 # k_f
      ReverseReactionCoef = 1.0e2 # k_r
      ForwardReactionEnergy = 0 # E_f
      ReverseReactionEnergy = 0 # E_r
      ForwardReactionFieldCoef = 0 # delta_f
      ReverseReactionFieldCoef = 0 # delta_r
    )
    ...
  )
}
```

The default values of all the coefficients are zero. [Table 243 on page 1304](#) lists the options available for the reaction specification.

For heterointerfaces, the keyword `Region` or `Material` allows you to specify the region or material where the reaction process enters as a generation–recombination rate. In addition, the keyword `FieldFromRegion` or `FieldFromMaterial` allows you to specify the region or material where the electric field is obtained. For example:

```
Physics ( Material = "Silicon/OxideAsSemiconductor" ) {
  HydrogenDiffusion(
    HydrogenReaction(...
      Material = "Silicon"
      FieldFromMaterial = "OxideAsSemiconductor"
    )
  )
}
```

Reactions with Multistate Configurations

A multistate configuration (MSC) can be used to model reactions between the mobile hydrogen elements and localized hydrogen states such as silicon-hydrogen bonds at the silicon–oxide interface.

For example, consider a hydrogen depassivation model based on the hole capture process:



where Si-H, p , Si^+ , and H represent the silicon-hydrogen bond, hole, silicon dangling bond, and hydrogen atom, respectively. This reaction can be specified by a two-state MSC defined at the silicon–oxide interface:

```
Physics ( MaterialInterface = "Silicon/Oxide" ) {
  MSConfigs (
    MSConfig ( Name = "SiHBond" Conc=5.0e12
      State ( Name = "s1" Hydrogen=1 Charge=0 ) # Si-H bond
      State ( Name = "s2" Hydrogen=0 Charge=1 ) # Si^+ dangling bond
      Transition ( Name = "t21" # Si-H + p <-> H + Si^+
        To="s2" From="s1" CEModel("CEModel_Depassivation" 1)
        Reservoirs("VB"(Particles=+1) "HydrogenAtom"(Particles=-1) )
        FieldFromInsulator
      )
    )
  )
  ...
}
```

The capture and emission rates are obtained from:

$$c_{21} = c_{\text{PMI}}(n, p, T, T_n, T_p, F) \prod_{i=1}^3 \left(\frac{[X_i]}{1/\text{cm}^3} \right)^{\alpha_i} \quad (487)$$

$$e_{21} = e_{\text{PMI}}(n, p, T, T_n, T_p, F) \prod_{i=1}^3 \left(\frac{[X_i]}{1/\text{cm}^3} \right)^{\beta_i} \quad (488)$$

where c_{PMI} and e_{PMI} are the capture and emission rates specified by the trap capture and emission PMI model, respectively. For more information about multistate configurations, see [Chapter 18 on page 425](#).

19: Degradation Model

MSC–Hydrogen Transport Degradation Model

NOTE Contrary to the conventional trap capture and emission PMI model, the insulator electric field can be used in the PMI model at the semiconductor–insulator interface instead of the semiconductor electric field when the keyword `FieldFromInsulator` is set in the transition.

The CEModel_Depassivation Model

The built-in capture and emission model `CEModel_Depassivation` models the hydrogen depassivation process. In the model, the hydrogen depassivation (electron capture) rate induced by hot-electron distribution is written as:

$$c_{\text{PMI}} = 2g_v\sigma \int_{\epsilon_{\min}}^{\epsilon_{\max}} v(\epsilon)g(\epsilon)f(\epsilon) \exp\left[\gamma\left(\frac{F}{1 \text{ V/cm}}\right)^\rho - \frac{\Theta(W_f - q\alpha F - \chi\epsilon)(W_f - q\alpha F - \chi\epsilon)}{kT}\right] d\epsilon \quad (489)$$

where:

- $\epsilon_{\min}$ and $\epsilon_{\max}$ are the minimum and maximum kinetic energies of the integration.
- σ is the capture cross-section.
- g_v is the valley degeneracy.
- $v(\epsilon)$ is the magnitude of the group velocity.
- $g(\epsilon)$ is the density-of-states.
- $f(\epsilon)$ is the distribution function.
- γ is the field enhancement prefactor.
- ρ is the field enhancement exponent.
- W_f is the activation energy for the depassivation process.
- α is the field-induced barrier-lowering factor.
- χ is the kinetic energy–induced barrier-lowering factor.

Similarly, the hydrogen depassivation rate induced by cold electrons can be written as:

$$c_{\text{PMI}} = \sigma v_{\text{th}} n \exp\left[\gamma\left(\frac{F}{1 \text{ V/cm}}\right)^\rho - \frac{\Theta(W_f - q\alpha F)(W_f - q\alpha F)}{kT}\right] \quad (490)$$

where v_{th} is the thermal velocity.

As the hydrogen passivation (electron emission) rate is not related to hot carriers, it is modeled simply as:

$$e = e_{\text{PMI}} \times \left(\frac{[\text{H}]}{1/\text{cm}} \right)^3 \quad (491)$$

$$e_{\text{PMI}} = e_0 \exp\left(-\frac{W_r}{kT}\right) \quad (492)$$

where e_0 is the passivation rate prefactor, and W_r is the activation energy for the passivation process.

The capture-emission model `CEModel_Depassivation` implements [Eq. 489](#) and [Eq. 490](#) for electrons and holes. The additional index in the `CEModel_Depassivation` model selects the equation and the carrier type as follows:

```
MSConfig( ...
  Transition( ... CEModel("CEModel_Depassivation" 0)) # Eq. 490 for electrons
  Transition( ... CEModel("CEModel_Depassivation" 1)) # Eq. 490 for holes
  Transition( ... CEModel("CEModel_Depassivation" 2)) # Eq. 489 for electrons
  Transition( ... CEModel("CEModel_Depassivation" 3)) # Eq. 489 for holes
)
```

To use the `CEModel_Depassivation` model with the index 2 or 3 (hot electron–induced or hot hole–induced degradation), you must specify `eSHEDistribution` or `hSHEDistribution` in the `Physics` section to compute the electron or hole distribution function (see [Using Spherical Harmonics Expansion Method on page 638](#)).

The model coefficients and their defaults are given in [Table 86](#). The model coefficients can be changed in the `CEModel_Depassivation` section of the parameter file.

Table 86 Default coefficients for `CEModel_Depassivation`

Symbol	Parameter name (Electrons)	Default value (Electrons)	Parameter name (Holes)	Default value (Holes)	Unit
σ	Xsec_e	1.0×10^{-24}	Xsec_h	1.0×10^{-24}	cm^2
v_{th}	Vth_e	2.0×10^7	Vth_h	2.0×10^7	cm/s
γ	gamma_e	2.7×10^{-7}	gamma_h	2.7×10^{-7}	1
ρ	rho_e	1.0	rho_h	1.0	1
α	alpha_e	9.0×10^{-9}	alpha_h	9.0×10^{-9}	cm
χ	chi_e	1.0	chi_h	1.0	1
W_f	wf_e	0.5	wf_h	0.5	eV

19: Degradation Model

MSC–Hydrogen Transport Degradation Model

Table 86 Default coefficients for CEModel_Depassivation

Symbol	Parameter name (Electrons)	Default value (Electrons)	Parameter name (Holes)	Default value (Holes)	Unit
e_0	e0_e	3.0×10^{-9}	e0_h	3.0×10^{-9}	1/s
W_r	wr_e	0	wr_h	0	eV
$d\epsilon$	dE_e	0.01	dE_h	0.01	eV
$\epsilon_{\min}$	Emin_e	0	Emin_h	0	eV
$\epsilon_{\max}$	Emax_e	5	Emax_h	5	eV

Using MSC–Hydrogen Transport Degradation Model

You can select the regions and interfaces where the hydrogen transport equations are to be solved by specifying the keyword `HydrogenDiffusion` in the corresponding `Physics` section of the command file. For example:

```
Physics ( Material = "Oxide" ) {  
    HydrogenDiffusion  
}
```

The keyword `HydrogenDiffusion` can be specified with the `HydrogenReaction` arguments (see [Reactions Between Mobile Elements on page 450](#)).

Specifying `HydrogenDiffusion` in the interface-specific `Physics` section gives a surface recombination term determined by r_i and $[X_i]_0$ at the corresponding interface.

To activate the transport of hydrogen atoms, hydrogen molecules, and hydrogen ions, the keywords `HydrogenAtom`, `HydrogenMolecule`, and `HydrogenIon` must be specified inside the `Coupled` command of the `Solve` section. For example, transient drift-diffusion simulation with transport of hydrogen atoms and hydrogen molecules can be specified by:

```
Solve {  
    Transient(...) {  
        Coupled { Poisson Electron Hole HydrogenAtom HydrogenMolecule }  
    }  
}
```

The parameters D_i , E_{di} , $[X_i]_0$, and r_i can be specified in the region-specific and interface-specific `HydrogenDiffusion` parameter set in the parameter file. For example:

```
Material = "Oxide" {  
    HydrogenDiffusion {  
        HydrogenAtom {
```

```

        d0 = 1.0e-13 # [cm^2*s^-1]
        Ed = 0      # [eV]
        atd = 1     # [1]
        n0 = 0      # [cm^-3]
        krec = 0     # [cm^-3*s^-1]
    }
    HydrogenMolecule {
        d0 = 1.0e-14 # [cm^2*s^-1]
        Ed = 0      # [eV]
        atd = 1     # [1]
        n0 = 0      # [cm^-3]
        krec = 0     # [cm^-3*s^-1]
    }
    HydrogenIon {
        ...
    }
}

MaterialInterface = "Oxide/PolySi" {
    HydrogenDiffusion {
        HydrogenAtom {
            n0 = 1      # [cm^-3]
            krec = 1     # [cm^-2*s^-1]
        }
    }
}

```

Here d_0 , E_d , α_{td} , n_0 , and k_{rec} correspond to D_i , E_{di} , α_{td} , $[X_i]_0$, and r_i . The default values of D_i , E_{di} , $[X_i]_0$, and r_i are zero; while $\alpha_{td} = 1$ by default.

The keywords for plotting the densities of hydrogen atoms, hydrogen molecules, and hydrogen ions are:

```

Plot { ...
    HydrogenAtom HydrogenMolecule HydrogenIon
}

```

Two-Stage NBTI Degradation Model

Negative bias temperature instability (NBTI) refers to the generation of positive oxide charges and interface traps in MOS structures under negative gate bias at elevated temperature, which affects the threshold voltage and on-currents of PMOSFETs [9].

19: Degradation Model

Two-Stage NBTI Degradation Model

Sentaurus Device provides a two-stage NBTI degradation model [12][13], which assumes that the NBTI degradation proceeds using a two-stage process:

- The first stage includes the creation of E' centers (dangling bonds in amorphous oxides) from their neutral oxygen vacancy precursors, the charging and discharging of E' centers, and the total annealing of E' centers to neutral oxygen vacancy precursors.
- The second stage considers the creation of poorly recoverable P_b centers (dangling bonds at silicon–oxide interfaces).

In the two-stage NBTI degradation model, the involved energy levels and activation energies are distributed widely and are treated as random variables. A random sampling technique is used to obtain the average change of the interface charge.

NOTE As the two-stage NBTI model is based on the semiclassical carrier density at the semiconductor–insulator interface, it is not recommended to use the model with quantum-correction models.

Formulation

The two-stage NBTI degradation model considers a special trap having four internal states:

- s_1 : Oxygen vacancy as a precursor state.
- s_2 : Positive E' center.
- s_3 : Neutral E' center.
- s_4 : Fixed positive charge with a P_b center.

Each NBTI trap is characterized by seven independent random variables:

- E_1 : Trap level of the precursor [eV] (by default, $-1.14 \leq E_1 < -0.31$).
- E_2 : Trap level of the E' center [eV] (by default, $0.01 \leq E_2 < 0.3$).
- E_4 : Trap level of the P_b center [eV] (by default, $0.01 \leq E_4 < 0.5$).
- E_A : Barrier energy of a transition from s_3 to s_1 [eV] (by default, $0.01 \leq E_A < 1.15$).
- E_B : Barrier energy of a transition from s_1 to s_2 [eV] (by default, $0.01 \leq E_B < 1.15$).
- E_D : Barrier energy of a transition between s_2 to s_4 [eV] (by default, $\langle E_D \rangle = 1.46, (\langle E_D^2 \rangle - \langle E_D \rangle^2)^{1/2} = 0.44$).
- r : Uniform number between 0 and 1. When $r < C$, a transition between s_2 to s_4 is allowed (by default, $C = 0.12$).

E_1 , E_2 , E_4 , E_A , and E_B follow the uniform distribution between their minimum and maximum values. E_D follows the Fermi-derivative distribution characterized by its average and standard deviation. The trap levels E_1 , E_2 , and E_4 are defined relative to the valence band energy.

The state occupation probability s_i satisfies the normalization condition:

$$\sum_{i=1}^4 s_i = 1 \quad (493)$$

and the following rate equations:

$$\dot{s}_1 = -s_1 k_{12} + s_3 k_{31} \quad (494)$$

$$\dot{s}_2 = s_1 k_{12} - s_2 (k_{23} + k_{24}) + s_3 k_{32} + s_4 k_{42} \quad (495)$$

$$\dot{s}_3 = s_2 k_{23} - s_3 (k_{32} + k_{31}) \quad (496)$$

$$\dot{s}_4 = s_2 k_{24} - s_4 k_{42} \quad (497)$$

with the transition rates:

$$k_{12} = e_C^{n,1} + c_V^{p,1} \quad (498)$$

$$k_{23} = c_C^{n,2} + e_V^{p,2} \quad (499)$$

$$k_{32} = e_C^{n,2} + c_V^{p,2} \quad (500)$$

$$k_{31} = v_1 \exp(-E_A/kT) \quad (501)$$

$$k_{24} = v_2 \exp[-(E_D - \gamma F)/kT] \Theta(C - r) \quad (502)$$

$$k_{42} = v_2 \exp[-(E_D + \Delta E_D + \gamma F)/kT] \Theta(C - r) \quad (503)$$

where:

- v_1 and v_2 are the attempt frequencies (by default, $v_1 = 1.0 \times 10^{13}$ and $v_2 = 5.11 \times 10^{15} \text{ s}^{-1}$).
- γ is the prefactor for the field-dependent barrier energy (by default, $\gamma = 7.4 \times 10^{-8} \text{ cm} \cdot \text{eV/V}$).
- ΔE_D is the additional barrier energy for k_{42} (by default, $\Delta E_D = 0 \text{ eV}$).

In addition, it is assumed that the hole occupation probability f_{it}^p of the P_b center in the state 4 is determined by:

$$f_{it}^p = (e_C^{n,4} + c_V^{p,4})(1 - f_{it}^p) - (c_C^{n,4} + e_V^{p,4})f_{it}^p \quad (504)$$

19: Degradation Model

Two-Stage NBTI Degradation Model

The electron emission rate and the hole capture rate for the state 1 are given by:

$$e_C^{n,1} = \sigma_n v_{th}^n N_C \exp[-H(E_C - E_1)/kT + \Theta(F_{c,n})(|F|/F_{c,n})^{\rho_n} - E_B/kT] \quad (505)$$

$$c_V^{p,1} = \sigma_p v_{th}^p p \exp[-H(E_V - E_1)/kT + \Theta(F_{c,p})(|F|/F_{c,p})^{\rho_p} - E_B/kT] \quad (506)$$

where:

- σ_n and σ_p are the electron and hole capture cross-sections (by default, $\sigma_n = 1.08 \times 10^{-15}$ and $\sigma_p = 1.24 \times 10^{-14} \text{ cm}^2$).
- v_{th}^n and v_{th}^p are the electron and hole thermal velocity (by default, $v_{th}^n = 1.5 \times 10^7$ and $v_{th}^p = 1.2 \times 10^7 \text{ cm/s}$).
- F is the insulator electric field.
- $F_{c,n}$ and $F_{c,p}$ are the critical electric field (by default, $F_{c,n} = -1$ and $F_{c,p} = 2.83 \times 10^6 \text{ V/cm}$).
- ρ_n and ρ_p are the exponents of the field-dependent term (by default, $\rho_n = 2$ and $\rho_p = 2$).
- $H(x) = x\Theta(x)$.

When the energy-dependent hole distribution is available (hSHEDistribution is activated in the semiconductor region), you can use the following expression for $c_V^{p,1}$ instead of Eq. 506:

$$c_V^{p,1} = 2g_v \sigma_p \int_0^\infty v(\epsilon) g(\epsilon) f(\epsilon) \exp[-H(E_V - E_1 - \epsilon)/kT + \Theta(F_{c,p})(|F|/F_{c,p})^{\rho_p} - E_B/kT] d\epsilon \quad (507)$$

The electron capture rate, the electron emission rate, the hole capture rate, and the hole emission rate for state 2 and state 4 are given by:

$$c_C^{n,i} = \sigma_n v_{th}^n n \exp[-H(E_i - E_C)/kT] \quad (508)$$

$$e_C^{n,i} = \sigma_n v_{th}^n N_C \exp[-H(E_C - E_i)/kT] \quad (509)$$

$$c_V^{p,i} = \sigma_p v_{th}^p p \exp[-H(E_V - E_i)/kT] \quad (510)$$

$$e_V^{p,i} = \sigma_p v_{th}^p N_V \exp[-H(E_i - E_V)/kT] \quad (511)$$

where $i = 2$ or 4 .

Using Two-Stage NBTI Model

You can activate the two-stage NBTI model by specifying the NBTI command in the interface-specific Physics section of the command file. For example:

```
Physics ( MaterialInterface = "Silicon/Oxide" ) {
  NBTI (
    Conc = 5.0e12                      # N_0 [/cm^2]
    NumberOfSamples = 1000             # N_sample [1]
    hSHEDistribution | -hSHEDistribution # (off by default)
  )
}
```

where Conc represents the density of the precursor N_0 , and NumberOfSamples represents the number of random samples N_{sample} . Sentaurus Device generates N_{sample} random configurations for each interface vertex. Then, the interface charge density is obtained from the ensemble average as follows:

$$Q = Q_{\text{ox}} + Q_{\text{it}} = qN_0 \langle s_2 + s_4 \rangle + qN_0 \langle s_4^p \rangle_{\text{it}} \quad (512)$$

where:

$$\langle x \rangle = \frac{1}{N_{\text{sample}}} \sum_{j=1}^{N_{\text{sample}}} x^j \quad (513)$$

During transient simulation, state occupation probabilities will change following the kinetic equations, which will change the interface charge density. It is assumed that $s_1 = 1$ at the beginning. In addition, you can restart NBTI degradation simulations by loading the saved data.

When the hSHEDistribution keyword is specified in the NBTI command (off by default), $c_v^{p,1}$ is computed from [Eq. 507](#) instead of [Eq. 506](#).

NOTE You need to specify hSHEDistribution in the Physics section to compute the hole distribution function (see [Using Spherical Harmonics Expansion Method on page 638](#)).

You can plot the density of charge and the density of each state as follows:

```
Plot {
  InterfaceNBTICharge    # [1/cm^2]
  InterfaceNBTIState1    # [1/cm^2]
  InterfaceNBTIState2    # [1/cm^2]
  InterfaceNBTIState3    # [1/cm^2]
  InterfaceNBTIState4    # [1/cm^2]
}
```

The model parameters are defined in the interface-specific NBTI parameter set.

References

- [1] A. Plonka, *Time-Dependent Reactivity of Species in Condensed Media*, Lecture Notes in Chemistry, vol. 40, Berlin: Springer, 1986.
- [2] C. Hu *et al.*, “Hot-Electron-Induced MOSFET Degradation—Model, Monitor, and Improvement,” *IEEE Journal of Solid-State Circuits*, vol. SC-20, no. 1, pp. 295–305, 1985.
- [3] O. Penzin *et al.*, “MOSFET Degradation Kinetics and Its Simulation,” *IEEE Transactions on Electron Devices*, vol. 50, no. 6, pp. 1445–1450, 2003.
- [4] K. Hess *et al.*, “Theory of channel hot-carrier degradation in MOSFETs,” *Physica B*, vol. 272, no. 1–4, pp. 527–531, 1999.
- [5] Z. Chen *et al.*, “On the Mechanism for Interface Trap Generation in MOS Transistors Due to Channel Hot Carrier Stressing,” *IEEE Electron Device Letters*, vol. 21, no. 1, pp. 24–26, 2000.
- [6] B. Tuttle and C. G. Van de Walle, “Structure, energetics, and vibrational properties of Si-H bond dissociation in silicon,” *Physical Review B*, vol. 59, no. 20, pp. 12884–12889, 1999.
- [7] M. A. Alam and S. Mahapatra, “A comprehensive model of PMOS NBTI degradation,” *Microelectronics Reliability*, vol. 45, no. 1, pp. 71–81, 2005.
- [8] J. W. McPherson, R. B. Khamankar, and A. Shanware, “Complementary model for intrinsic time-dependent dielectric breakdown in SiO₂ dielectrics,” *Journal of Applied Physics*, vol. 88, no. 9, pp. 5351–5359, 2000.
- [9] J. H. Stathis and S. Zafar, “The negative bias temperature instability in MOS devices: A review,” *Microelectronics Reliability*, vol. 46, no. 2-4, pp. 270–286, 2006.
- [10] A. E. Islam *et al.*, “Recent Issues in Negative-Bias Temperature Instability: Initial Degradation, Field Dependence of Interface Trap Generation, Hole Trapping Effects, and Relaxation,” *IEEE Transactions on Electron Devices*, vol. 54, no. 9, pp. 2143–2154, 2007.

- [11] T. Grassler, W. Gös, and B. Kaczer, “Dispersive Transport and Negative Bias Temperature Instability: Boundary Conditions, Initial Conditions, and Transport Models,” *IEEE Transactions on Device and Materials Reliability*, vol. 8, no. 1, pp. 79–97, 2008.
- [12] T. Grassler *et al.*, “A Two-Stage Model for Negative Bias Temperature Instability,” in *IEEE International Reliability Physics Symposium (IRPS)*, Montréal, Québec, Canada, pp. 33–44, April 2009.
- [13] W. Goes *et al.*, “A Model for Switching Traps in Amorphous Oxides,” in *International Conference on Simulation of Semiconductor Processes and Devices (SISPAD)*, San Diego, CA, USA, pp. 159–162, September 2009.

19: Degradation Model

References

CHAPTER 20 Organic Devices

This chapter describes the organic models available in Sentaurus Device.

The electrical conduction process in organic materials is different from crystal lattice semiconductors. However, similar concepts in semiconductor transport theory can be used in treating the conduction process in organic materials.

Introduction to Organic Device Simulation

An organic material or semiconductor is formed from molecule chains, and the primary transport of carriers (electrons and holes) is through a *hopping* process. The lowest unoccupied molecular orbital (LUMO) and highest occupied molecular orbital (HOMO) energy levels in organic materials are analogous to the conduction and valence bands, respectively. In addition, excitons need to be considered since these bound electron–hole pairs contribute to the distribution of electron and hole populations. Traps are also central to organic transport, and these need to be taken into account appropriately.

Therefore, a combination of semiconductor models and organic physics models is required to model reasonably the physical transport processes of organic devices within the framework of Sentaurus Device. The models needed in a typical organic device simulation are:

- The Poole–Frenkel mobility model is used to model the hopping transport of the carriers (electrons and holes). This model is dependent on electric field and temperature. Note that it is common for electrons to have two orders of magnitude higher mobility than holes (see [Poole–Frenkel Mobility \(Organic Material Mobility\) on page 353](#)).
- Organic–organic heterointerface physics requires special treatment for the ballistic transport of carriers and bulk excitons across heterointerfaces with energy barriers (see [Gaussian Transport Across Organic Heterointerfaces on page 653](#)).
- Gaussian density-of-states (DOS) approximates the effective DOS for electrons and holes in disordered organic materials and semiconductors (see [Gaussian Density-of-States for Organic Semiconductors on page 264](#)).
- The traps model must be initialized with the appropriate capture cross sections and densities (see [Chapter 17 on page 407](#)).
- The Langevin bimolecular recombination model is used to model the recombination process of carriers and the generation process of singlet excitons (see [Bimolecular Recombination on page 401](#)).

- A singlet exciton equation is introduced to model the diffusion process, the generation from bimolecular recombination, the loss from decay, and the optical emissions of singlet excitons. Note that only Frenkel excitons (electron–hole pairs existing on the same molecule) participate in the optical process in organic materials (see [Singlet Exciton Equation on page 235](#)).

The organic device simulator is based on the work of Kozłowski [1], and other useful papers are available [2]–[10].

Many acronyms are used for the description of organic device layers and transport mechanisms. [Table 87](#) lists commonly used acronyms for organic transport.

Table 87 Commonly used acronyms for organic transport

Acronym	Definition
EBL	Electron blocking layer
EML	Emission layer
ETL	Electron transport layer
HOMO	Highest occupied molecular orbital
HTL	Hole transport layer
LUMO	Lowest unoccupied molecular orbital
SCL	Space charge limited
TCL	Trapped charge limited
TSL	Thermally stimulated luminescence

References

- [1] F. Kozłowski, *Numerical simulation and optimisation of organic light emitting diodes and photovoltaic cells*, Ph.D. thesis, Technische Universität Dresden, Germany, 2005.
- [2] S.-H. Chang *et al.*, “Numerical simulation of optical and electronic properties for multilayer organic light-emitting diodes and its application in engineering education,” in *Proceedings of SPIE, Light-Emitting Diodes: Research, Manufacturing, and Applications X*, vol. 6134, pp. 26-1–26-10, 2006.
- [3] P. E. Burrows *et al.*, “Relationship between electroluminescence and current transport in organic heterojunction light-emitting devices,” *Journal of Applied Physics*, vol. 79, no. 10, pp. 7991–8006, 1996.

- [4] M. Hoffmann and Z. G. Soos, "Optical absorption spectra of the Holstein molecular crystal for weak and intermediate electronic coupling," *Physical Review B*, vol. 66, no. 2, p. 024305, 2002.
- [5] J. Staudigel *et al.*, "A quantitative numerical model of multilayer vapor-deposited organic light emitting diodes," *Journal of Applied Physics*, vol. 86, no. 7, pp. 3895–3910, 1999.
- [6] E. Tutiš *et al.*, "Numerical model for organic light-emitting diodes," *Journal of Applied Physics*, vol. 89, no. 1, pp. 430–439, 2001.
- [7] B. Ruhstaller *et al.*, "Simulating Electronic and Optical Processes in Multilayer Organic Light-Emitting Devices," *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 9, no. 3, pp. 723–731, 2003.
- [8] B. Ruhstaller *et al.*, "Transient and steady-state behavior of space charges in multilayer organic light-emitting diodes," *Journal of Applied Physics*, vol. 89, no. 8, pp. 4575–4586, 2001.
- [9] A. B. Walker, A. Kambili, and S. J. Martin, "Electrical transport modelling in organic electroluminescent devices," *Journal of Physics: Condensed Matter*, vol. 14, no. 42, pp. 9825–9876, 2002.
- [10] S. Odermatt, N. Ketter, and B. Witzigmann, "Luminescence and absorption analysis of undoped organic materials," *Applied Physics Letters*, vol. 90, p. 221107, May 2007.

20: Organic Devices

References

CHAPTER 21 Optical Generation

This chapter describes various methods that are used to compute the optical generation rate when an optical wave penetrates into the device, is absorbed, and produces electron–hole pairs.

The methods include simple optical beam absorption, the raytracing method, the transfer matrix method, the finite-difference time-domain (including an option for hardware acceleration), the beam propagation method, and loading external profiles from file. Different types of refractive index and absorption models, parameter ramping, and optical AC analysis are also described.

Overview

A unified interface for optical generation computation is available to provide a consistent simulation setup irrespective of the underlying optical solver methods. This allows for a gradual refinement of results and a balance of accuracy versus computation time in the course of a simulation, while only the solver-specific parameters have to change. Several approaches for computing the optical generation exist that are independent of the chosen optical solver:

- Compute the optical generation resulting from a monochromatic optical source.
- Compute the optical generation resulting from an illumination spectrum.
- Set a constant value for the optical generation rate either per region or globally.
- Read an optical generation profile from file.
- Compute the optical generation as a sum of the above contributions.

For each of the contributions listed, a separate scaling factor can be specified. For transient simulations, it is possible to apply a time-dependent scaling factor that can be used, for example, to model the response to a light pulse or any other time-dependent light signal. This feature can also be used to improve convergence if the optical generation rate is very high. The unified interface also allows you to save the computed optical generation rate to a file for reuse in other simulations.

The optical generation resulting from a monochromatic optical source or an illumination spectrum is determined as the product of the absorbed photon density, which is computed by the optical solver, and the quantum yield. Several models for the quantum yield are available ranging from a constant scaling factor to a spatially varying quantum yield based on the band gap of the underlying material and the excitation wavelength of the optical source.

21: Optical Generation

Specifying the Type of Optical Generation Computation

In the following sections, the different approaches for computing the optical generation and their corresponding parameters are described.

Specifying the Type of Optical Generation Computation

In the `OpticalGeneration` section of the command file, at least one of the following methods to compute the optical generation must be specified; otherwise, the optical generation is not computed and is set to zero everywhere:

- `ComputeFromMonochromaticSource` activates optical generation computation with a single wavelength that is either specified in the `Excitation` section or as a ramping variable.
- `ComputeFromSpectrum` allows the sum of optical generation to be computed from an input spectrum of wavelengths.
- `ReadFromFile` imports the optical generation profile.
- `SetConstant` enables you to set a background constant optical generation in the specified region or material.

NOTE If the keyword `Scaling` is used in the above optical generation methods, the scaling applies only to quasistationary simulations. To scale transient simulations, see [Specifying Time Dependency for Transient Simulations on page 478](#).

If several methods are specified, the total optical generation rate is given by the sum of the contributions computed with each method. The general syntax is:

```
Physics {  
    ...  
    Optics (  
        OpticalGeneration (  
            ...  
            ComputeFromMonochromaticSource (...)  
            ComputeFromSpectrum (...)  
            ReadFromFile (...)  
            SetConstant ( ... Value = <float> )  
        )  
        Excitation (...)  
        OpticalSolver (...)  
        ComplexRefractiveIndex (...)  
    )  
}
```

Each method can have its own options such as a scaling factor or a particular time-dependency specification used in transient simulations. An example setup where the optical generation read

from a file is scaled by a factor of 1.1 and a Gaussian time dependency is assumed for the monochromatic source is:

```
OpticalGeneration (
  ...
  ComputeFromMonochromaticSource (
    ...
    TimeDependence (
      WaveTime = (<t1>, <t2>)
      WaveTSigma = <float>
    )
  )
  ReadFromFile (
    ...
    Scaling = 1.1
  )
)
```

Note that both `Scaling` and `TimeDependence` can also be specified directly in the `OpticalGeneration` section if the same parameters will apply to all methods. For an overview of all available options, see [Table 196 on page 1274](#).

By default, the optical generation rate is calculated for every semiconductor region. However, you can suppress the computation of the optical generation rate for a specific region or material by specifying the keyword `-OpticalGeneration` in the corresponding region or material `Physics` section:

```
Physics ( region="coating" ) { Optics ( -OpticalGeneration ) }
Physics ( material="InP" ) { Optics ( -OpticalGeneration ) }
```

To visualize the optical intensity and generation, the keywords `OpticalIntensity` and `OpticalGeneration` must be added to the `Plot` section:

```
Plot {
  ...
  OpticalIntensity
  OpticalGeneration
}
```

Specifying `OpticalGeneration` in the `Plot` section plots not only the total optical generation that enters the electrical equations, but also the contributions from the different methods of optical generation computation. See [Appendix F on page 1185](#) for the corresponding data names.

Optical Generation from Monochromatic Source

Specifying `ComputeFromMonochromaticSource (...)` in the `OpticalGeneration` section activates the computation of the optical generation assuming a monochromatic light source. Details of the light source such as angle of incidence, wavelength, and intensity, and the optical solver used to model it must be set in the `Excitation` section and `OpticalSolver` section, respectively (see [Specifying the Optical Solver on page 480](#) and [Setting the Excitation Parameters on page 486](#)).

Together with the possibility of ramping parameters (see [Controlling Computation of Optical Problem in Solve Section on page 493](#)), for example, the wavelength of the incident light, this model can be used to simulate the optical generation rate as a function of wavelength.

Illumination Spectrum

The optical generation resulting from a spectral illumination source, which is sometimes also known as *white light generation*, can be modeled in Sentaurus Device by superimposing the spectrally resolved generation rates. To this end, `ComputeFromSpectrum (...)` must be listed in the `OpticalGeneration` section. The illumination spectrum is then read from a text file whose name must be specified in the `File` section:

```
File {
    ...
    IlluminationSpectrum = "illumination_spectrum.txt"
}
Physics {
    ...
    Optics (
        ...
        OpticalGeneration (
            ...
            ComputeFromSpectrum ( ... )
        )
    )
}
```

In its simplest form, the illumination spectrum file has a two-column format. The first column contains the wavelength in μm and the second column contains the intensity in W/cm^2 . The characters `#` and `*` mark the beginning of a comment.

As a default, the integrated generation rate resulting from the illumination spectrum is only computed once, that is, the first time the optical problem is solved and remains constant

thereafter. However, it is possible to force its recomputation on every occasion by specifying the keyword `RefreshEveryTime` in the `ComputeFromSpectrum` section.

Multidimensional Illumination Spectra

Often, illumination spectra depend on additional parameters, which may be related to an experimental setup that users want to model. For example, the intensity of the incident light may depend on not only the wavelength but also the angle of incidence. To account for such simulation setups, Sentauros Device supports multidimensional illumination spectra. The format of a corresponding illumination spectrum file is:

```
# some comment
* another comment # and so on
Optics/Excitation/Wavelength [um] theta [deg] phi [deg] intensity [W*cm^-2]
0.62 0 30 0.1
0.86 0 30 0.2
1.1 0 30 0.3
```

The header contains optional comment lines and a line defining the parameters assigned to each column. A parameter name or path is followed by its corresponding unit; if no unit is specified, the default unit of 1 is assumed. Listed parameters can refer either directly to existing parameters of the `Optics` section in Sentauros Device or to user-defined parameters. The latter comes into play when using illumination spectra in combination with loading absorbed photon density or optical generation profiles from file. See [Loading Solution of Optical Problem from File on page 553](#) for more details.

NOTE The parameter `Intensity` is mandatory in every illumination spectrum file. However, the order of columns is arbitrary. Parameter names are case insensitive.

For multidimensional illumination spectrum files, the active columns must be selected in the command file. All other columns are ignored in the simulation. The required syntax in the `OpticalGeneration` section is:

```
ComputeFromSpectrum (
    ...
    Select (
        Parameter = ("Optics/Excitation/Wavelength" "Theta")
    )
)
```

The parameter `Intensity` is selected by default and does not need to be specified.

Loading and Saving Optical Generation from and to File

Sometimes, solving the optical problem may require long computation times, in which case, it can be useful to save the solution to a file and to load the optical generation or absorbed photon density profile in later simulations whose optical properties remain constant. In the following example, optical generation is used as a synonym for absorbed photon density. The command file syntax for saving the optical generation rate to a file is:

```
File {  
    OpticalGenerationOutput = <filename>  
}
```

For loading the optical generation rate from a file, the syntax is:

```
File {  
    OpticalGenerationInput = <filename>  
}  
  
Physics {  
    Optics (  
        OpticalGeneration (  
            ReadFromFile ( DatasetName = AbsorbedPhotonDensity | OpticalGeneration  
                          GridInterpolation = Linear | Conservative )  
        )  
    )  
}
```

If the input file to be loaded contains both an optical generation and an absorbed photon density profile, the keyword `DatasetName` controls which one to use. By default, the absorbed photon density is used. The optical generation profile to be loaded also may be defined on a mixed-element grid that is different from the one used in the device simulation, or on a tensor grid resulting from an EMW simulation. In that case, the profile is interpolated automatically onto the simulation grid upon loading. By default, vertex-based linear interpolation is used.

As an alternative for mixed-element to mixed-element interpolation, conservative interpolation can be selected using `GridInterpolation=Conservative` in the `ReadFromFile` section. The interpolation algorithm always interpolates the values even if the material of the source and destination grid are different at a certain vertex. However, optical generation is always set to zero in non-semiconductor regions irrespective of the source profile.

Further options of the `ReadFromFile` section can be found in [Table 196 on page 1274](#). Similar but more powerful functionality is provided by the feature `FromFile` (see [Loading Solution of Optical Problem from File on page 553](#)).

Constant Optical Generation

Assigning a constant optical generation rate to a certain region or material is achieved by specifying a value for a particular region or material as shown here:

```
Physics (Region = <name of region>) {  
    ...  
    Optics (  
        OpticalGeneration (  
            SetConstant (  
                Value = <float>  
            )  
        )  
    )  
}  
  
Physics (Material = <name of material>) {  
    ...  
    Optics (  
        OpticalGeneration (  
            SetConstant (  
                Value = <float>  
            )  
        )  
    )  
}
```

If all semiconductor regions are supposed to have the same optical generation rate, its value is best specified in the global `Physics` section as follows:

```
Physics {  
    ...  
    Optics (  
        OpticalGeneration (  
            SetConstant (  
                Value = <float>  
            )  
        )  
    )  
}
```

Quantum Yield Models

The quantum yield model describes how many of the absorbed photons are converted to generated electron–hole pairs. The simplest model, `QuantumYield(Unity)`, assumes that all absorbed photons result in generated charge carriers irrespective of the band gap or other

21: Optical Generation

Specifying the Type of Optical Generation Computation

properties of the underlying material. This corresponds to a global quantum yield factor of one, which is the default value, except for non-semiconductor regions where it is always set to zero even if photons are actually absorbed.

A more realistic model, `QuantumYield(Stepfunction(...))`, takes the band gap into account. If the excitation energy is greater than or equal to the bandgap energy E_g , the quantum yield is set to one; otherwise, it is set to zero. The bandgap energy used can be set directly by specifying a value for `Energy` in eV or, alternatively, a corresponding `Wavelength` in micrometers. Another option is `Bandgap`, which uses the temperature-dependent bandgap energy as given in [Bandgap and Electron-Affinity Models on page 250](#) (Eq. 140, p. 250). To include bandgap narrowing effects in the form of [Eq. 145, p. 251](#), the keyword `EffectiveBandgap` must be specified.

In the presence of free carrier absorption (FCA), which is activated in Sentaurus Device by specifying `CarrierDep(Imag)` in the `ComplexRefractiveIndex` section of the command file, the spatially varying quantum yield factor is reduced according to the ratio of free carrier absorption, α_{FCA} , to total absorption, α_{tot} :

$$\eta_G = \eta_{G_0} \left(1 - \frac{\alpha_{FCA}}{\alpha_{tot}} \right) \quad (514)$$

where η_{G_0} represents a step function, which is 1 if the photon energy is sufficiently large to allow for interband optical absorption, or 0 otherwise. α_{FCA} and α_{tot} are determined by the `ComplexRefractiveIndex` specification in the parameter file given the following relation between the absorption coefficient and the extinction coefficient:

$$\alpha = \frac{4\pi k}{\lambda} \quad (515)$$

The change of the extinction coefficient due to free carrier absorption is represented by Δk_{carr} (see [Carrier Dependency on page 497](#)).

The command file syntax for specifying quantum yield models is:

```
Physics {
  Optics (
    OpticalGeneration (
      ...
      QuantumYield = <float>
      QuantumYield ( Unity )
      QuantumYield (
        StepFunction ( Wavelength = <float> #[um]
                      # OR
                      Energy = <float>      #[eV]
        )
        StepFunction ( Bandgap | EffectiveBandgap )
      )
    )
  )
}
```

```
}
)
)
```

For more sophisticated quantum yield models, Sentaurus Device offers a PMI interface (see [Optical Quantum Yield on page 1098](#)). All quantum yield models also can be specified in a region or material `Physics` section. The resulting quantum yield can be plotted by specifying `QuantumYield` in the `Plot` section.

Optical Absorption Heat

The absorbed photon energy E_{ph} is distributed among different processes by quantum yield factors. The following two channels are accounted for:

- Thermalization to the band gap (interband absorption): When a photon is absorbed across the band gap in a semiconductor, it is absorbed to create an electron–hole pair. The excess energy (photon energy minus the band gap) of the new electron–hole pair is assumed to thermalize, resulting eventually in lattice heating.
- Complete thermalization (intraband absorption): In the case of the photon energy being smaller than the band gap, the photon can be absorbed to increase the energy of a carrier. The excess energy relaxes eventually, contributing to lattice heating.

In both processes, it is assumed that the eventual lattice heating occurs in the locality of photon absorption. The corresponding energy equation reads as:

$$E_{ph} = \eta_{T_{Eg}}(E_{ph} - E_g) + \eta_G E_g + \eta_{T_0} E_{ph} \quad (516)$$

where the energy contributions of the first term and the third term are dissipated into the lattice through thermalization. The energy of the second term is consumed for the generation of an electron–hole pair.

In general, $\eta_{T_{Eg}} = \eta_G$ since it is assumed that every photon generates a charge carrier, while the residual energy $E_{ph} - E_g$ is dissipated into the lattice. Therefore, ignoring free carrier absorption, if the chosen quantum yield model evaluates to 1, $\eta_{T_{Eg}} = \eta_G = 1$ and $\eta_{T_0} = 0$. If the quantum yield model evaluates to 0, $\eta_{T_{Eg}} = \eta_G = 0$ and $\eta_{T_0} = 1$.

Distinguishing between $\eta_{T_{Eg}}$ and η_G allows for the control of multiple-generation processes as well. For example, in the UV spectrum, it is possible that $E_{ph} > 2E_g$, and so more than one charge carrier can be generated per absorbed photon. To model multiple-generation processes, the `OpticalQuantumYield` PMI (see [Optical Quantum Yield on page 1098](#)) must be used where all quantum yield factors can be specified independently for each vertex.

Supporting several optical absorption processes affects the value of η_G as can be seen from [Eq. 516](#). It no longer depends on the local effective bandgap energy only. Factoring in free carrier absorption means that photons absorbed through this process do not contribute to the

21: Optical Generation

Specifying the Type of Optical Generation Computation

optical generation rate, which is used in the drift-diffusion equations. In general, the quantum yield factors are independent as long as the energy equation is fulfilled locally.

The quantum yield factor η_G is given by [Eq. 514](#), and the quantum yield factor attributed to complete thermalization is computed as:

$$\eta_{T_0} = 1 - \eta_G \quad (517)$$

The quantum yield factor $\eta_{T_{Eg}}$ is set to η_G unless it is specified explicitly using the `OpticalQuantumYield` PMI.

NOTE If η_G is set to a constant value in the command file, η_{T_0} is still given by [Eq. 517](#) to ensure energy balance.

Specifying `OpticalAbsorptionHeat` and `ThermalizationYield` in the `Plot` section of the command file results in the following quantities being written to the plot file for each vertex:

- `OpticalAbsorptionHeat`: $(\eta_{T_{Eg}}(E_{ph} - E_{Eg}) + \eta_{T_0}E_{ph}N_{ph})N_{ph}$
- `OpticalAbsorptionHeat(Bandgap)`: $\eta_{T_{Eg}}(E_{ph} - E_{Eg})N_{ph}$
- `OpticalAbsorptionHeat(Vacuum)`: $\eta_{T_0}E_{ph}N_{ph}$
- `ThermalizationYield(Bandgap)`: $\eta_{T_{Eg}}$
- `ThermalizationYield(Vacuum)`: η_{T_0}

where N_{ph} represents the number of absorbed photons.

Specifying Time Dependency for Transient Simulations

To model the electrical response of a light pulse, incident on a device, a description of the light signal over time can be specified either globally or separately for each type of optical generation computation. For the former, `TimeDependence` (...) is listed directly in the `OpticalGeneration` section, while for the latter, it is given as an argument of the chosen type of optical generation computation. `TimeDependence` can only be specified in the global `Physics` section and not for a particular region or material. The given time dependency essentially scales the optical generation rate, resulting from a stationary solution of the optical problem as a function of time. The time dependency is only taken into account inside a `Transient` statement. If the optical generation needs to be scaled inside a `Quasistationary`, the keyword `Scaling` in the `OpticalGeneration` section can be used. However, this scaling factor does not apply inside a `Transient` statement. Instead, `Scaling` can be set directly in the respective `TimeDependence` section.

Three types of time dependency are available:

- Linear time dependency
- Gaussian time dependency
- Arbitrary time dependency read from a file

For the linear and Gaussian time dependencies, a time interval `WaveTime = (<t1>, <t2>)` can be specified. Before `<t1>`, the optical generation rate undergoes a linear or Gaussian increase from zero. After `<t2>`, the optical generation rate experiences a linear or Gaussian decrease.

The linear time function is expressed as:

$$F(t) = \begin{cases} \max(0, m(t-t_1) + 1) & , t < t_1 \\ 1 & , t_1 \leq t \leq t_2 \\ \max(0, m(t_2-t) + 1) & , t > t_2 \end{cases} \quad (518)$$

where m is given by `WaveTSlope`.

The Gaussian time function is expressed as:

$$F(t) = \begin{cases} \exp\left(-\left(\frac{t_1-t}{\sigma}\right)^2\right) & , t < t_1 \\ 1 & , t_1 \leq t \leq t_2 \\ \exp\left(-\left(\frac{t-t_2}{\sigma}\right)^2\right) & , t > t_2 \end{cases} \quad (519)$$

where σ is defined by `WaveTSigma`.

If no time interval is specified, $t_1 = t_2 = 0$ is assumed. The linear time dependency is selected by specifying the parameter `WaveTSlope`; whereas, the Gaussian time dependency is chosen by specifying the parameter `WaveTSigma`.

An arbitrary time dependency can be applied by defining a time function whose interpolation points are given in a file with a whitespace-separated, two-column format. The first column contains the time points in seconds and the second column contains the corresponding function values. Linear interpolation is used to obtain function values between the interpolation points. This type of time dependency can be activated using the following syntax:

```
File {
    OptGenTransientScaling = <filename> # file containing interpolation points
}
```

21: Optical Generation

Solving the Optical Problem

```
Physics {  
    Optics (  
        OpticalGeneration (  
            TimeDependence (FromFile)  
        )  
    )  
}
```

Solving the Optical Problem

ComputeFromMonochromaticSource and ComputeFromSpectrum require the solution of the optical problem for a given excitation to obtain the optical generation rate. Several optical solvers are available and the choice for a specific method is determined usually by the optimum combination of accuracy of results and computation time.

Besides selecting a certain optical solver and specifying its particular parameters, it is necessary to define the excitation parameters, to choose an appropriate refractive index model, and to control when a solution is computed in the Solve section. These steps are explained in the following sections.

Specifying the Optical Solver

The following optical solvers are supported:

- Transfer matrix method (TMM)
- Finite-difference time-domain (FDTD) method including the hardware-accelerated option
- Raytracing (RT)
- Beam propagation method (BPM)
- Loading solution of optical problem from file
- Optical beam absorption method

Transfer Matrix Method

The TMM solver is selected using the following syntax:

```
Physics {  
    ...  
    Optics (  
        ...  
        OpticalSolver (  
            TMM (  
                <TMM options>  
            )  
        )  
    )  
}
```

The TMM-specific options, such as the parameters for the extraction of the layer stack, are described in [Using Transfer Matrix Method on page 543](#).

Finite-Difference Time-Domain Method

In contrast to the TMM solver, the FDTD-specific parameters, such as boundary conditions and extractors, are defined in a separate command file that is set in the `File` section with the keyword `OpticalSolverInput`. To provide some basic control of the FDTD solver from within Sentaurus Device, excitation parameters such as `Wave length`, `Theta`, and `Phi`, as well as the polarization `Psi`, are overwritten by their counterparts in the `Excitation` section of the Sentaurus Device command file if specified.

NOTE The `Psi` keyword in EMW corresponds to the `PolarizationAngle` keyword in Sentaurus Device.

NOTE The FDTD solver in Sentaurus Device supports both the 2D and 3D excitation specification of EMW as outlined in [Sentaurus Device Electromagnetic Wave Solver User Guide, Specifying Direction and Polarization on page 52](#).

The available FDTD solvers are:

- Default kernel (keyword `EMWplusOption`).
- Acceleware hardware acceleration kernel (keyword `AccelewareOption`) (separate from TCAD Sentaurus tools; go to www.acceleware.com for information).

21: Optical Generation

Solving the Optical Problem

The syntax for activating the various FDTD solvers is similar and requires the input of the file name of the command file of the specific FDTD solver, and a keyword option to choose the specific solver. The general syntax is:

```
File {
    OpticalSolverInput = <command file for emw, 'emw -acceleware'>
}

Physics {
    Optics (
        OpticalSolver (
            FDTD ( EMWplusOption )      # or AccelewareOption
        )
    )
}
```

The FDTD algorithm is based on a tensor mesh, which can be generated independently before the device simulation. Another option is to build the tensor mesh during the simulation before the call to EMW. The main advantage of the latter option is that the tensor grid can be adjusted to a possibly changing excitation wavelength in a Quasistationary statement. Since the accuracy and stability of the FDTD method crucially depend on the discretization with respect to the wavelength, this feature becomes important when a large range of the light spectrum is scanned (see [Illumination Spectrum on page 472](#) and [Parameter Ramping on page 494](#)).

To activate this feature, `GenerateMesh ( . . . )` must be specified in the FDTD section and a common base name (that is, file name excluding suffix) for the boundary file and the Sentauros Mesh command file must be defined in the `File` section using the keyword `MeshInput`. Sentauros Mesh is then called with the specified base name prepended by the basename of the `Plot` file given in the `File` section of the Sentauros Device command file. The resulting tensor grid file is detected automatically and replaces the grid file in the user-provided EMW command file.

By default, a tensor mesh is generated before the first call to EMW and remains in use during further calls to the solver. However, if the excitation wavelength varies and the initially built tensor mesh no longer fulfills the requirements, two options exist that control the update of the mesh.

Using the keyword `ForEachWavelength` in the `GenerateMesh` section triggers the computation of a new tensor mesh whenever the wavelength changes compared to the previous solution of the FDTD solver. Internally, the wavelength specified in the user-provided Sentauros Mesh command file is replaced with the current value.

Since the slope of the complex refractive index as a function of wavelength can vary from almost zero to high values, depending on the wavelength interval, it is possible to limit the mesh generation according to a list of strictly monotonically increasing wavelengths. If the

current wavelength enters a new interval, the mesh is updated and remains in use until the wavelength moves beyond the interval boundaries. The syntax for such a use case is:

```
FDTD (
    ...
    GenerateMesh (
        Wavelength = ( 0.35 0.55 0.7 0.8 ) # wavelength in [μm]
    )
)
```

For the command file syntax and options of the FDTD solver, refer to the *Sentaurus Device Electromagnetic Wave Solver User Guide*.

Raytracing

More details about the raytracer can be found in [Raytracing on page 510](#). The raytracer requires the use of the complex refractive index model, and various excitation variables can be ramped. These rampable excitation variables are Intensity, Wavelength, Theta, Phi, and PolarizationAngle or Polarization. The required syntax is:

```
Physics {...
    Optics (
        ComplexRefractiveIndex(...)
        OpticalGeneration(...)
        OpticalSolver(
            RayTracing(...)
        )
        Excitation(...)
    )
}
```

where the RayTracing section contains:

```
RayTracing(
    # Setting keywords of raytracer
    # -----
    CompactMemoryOption
    MonteCarlo
    OmitReflectedRays
    OmitWeakerRays
    RedistributeStoppedRays

    PolarizationVector = vector
    PolarizationVector = Random
    DepthLimit = integer
    MinIntensity = float           # fraction, relative value
    Print(Skip(integer))
    ProgressMarkers = integer
```

21: Optical Generation

Solving the Optical Problem

```
RetraceCRChange = float          # fractional change to retrace rays
VirtualRegions { "string" "string" ... }
VirtualRegions {}

# Defining starting rays:
# -----
RayDistribution("windowname")( ... )
RectangularWindow (
    RectangleV1 = vector          # vertex 1 of rectangular window [um]
    RectangleV2 = vector          # vertex 2 of rectangular window [um]
    RectangleV3 = vector          # vertex 3 needed only for 3D [um]
    # number of rays = LengthDiscretization * WidthDiscretization
    LengthDiscretization = integer # longer side
    WidthDiscretization = integer  # shorter side needed only for 3D
    RayDirection = vector
)

UserWindow (
    NumberOfRays = integer        # number of rays in file
    RaysFromFile = "filename.txt" # position(um) direction area(cm^2)
)
)
```

Some comments about the RT syntax:

- The keyword `RetraceCRChange` specifies the fractional change of the complex refractive index (either the real or imaginary part) from its previous state that will force a total recomputation of raytracing.
- Starting rays for raytracing can be set using `RectangularWindow` or `UserWindow`. Details can be found in [Window of Starting Rays on page 517](#). Only one of the windows can be chosen but not both.
- Starting rays also can be set using the illumination window (see [Illumination Window on page 487](#)) and the `RayDistribution` section (see [Distribution Window of Rays on page 518](#)).

Beam Propagation Method

The BPM solver is selected using the following syntax:

```
Physics {
    Optics (
        OpticalSolver (
            BPM (
                <BPM options>
            )
        )
    )
}
```

```
)  
}
```

The BPM-specific options, such as the specific excitation type or the discretization parameters, are described in [Using Beam Propagation Method on page 561](#).

Loading Solution of Optical Problem from File

The solution of the optical problem, which can be either an absorbed photon density profile or an optical generation profile, also can be loaded from a file. In contrast to using `OpticalGeneration ( ReadFromFile ( ... ) )`, the optical solver `FromFile` offers more flexibility for simulation setups that involve an illumination spectrum or parameter ramping.

The required syntax is:

```
File {  
    OpticalSolverInput = "<file name or file name pattern>"  
}  
  
Physics {  
    Optics (  
        OpticalGeneration (  
            ComputeFromMonochromaticSource ()  
        )  
        OpticalSolver (  
            FromFile (  
                <FromFile options>  
            )  
        )  
    )  
}
```

The use of the optical solver `FromFile` is described in [Loading Solution of Optical Problem from File on page 553](#).

Optical Beam Absorption Method

The following syntax is used to select the optical beam absorption method as the optical solver:

```
Physics {  
    Optics (  
        OpticalSolver (  
            OptBeam (  
                <OptBeam options>  
            )  
        )  
    )  
}
```

```
} )
```

The OptBeam-specific options, such as the parameters for the extraction of the layer stack, are described in [Using Optical Beam Absorption Method on page 558](#).

Setting the Excitation Parameters

The Excitation section is common to all optical solvers and mainly specifies a plane wave excitation. It contains the following keywords:

- Intensity [W/cm²]
- Wavelength [μm]
- Theta [deg]
- Phi [deg]
- PolarizationAngle [deg] or Polarization

In combination with the optical solvers TMM, OptBeam, and FromFile, an additional Window section is required to specify the parameters of the illumination window, which is described in [Illumination Window on page 487](#).

NOTE For the optical solver FromFile, the requirement of a Window section only applies when one-dimensional profiles are loaded.

The propagation direction is defined by the angles with the positive z-axis and x-axis, respectively. In two dimensions, the propagation direction is well-defined by specifying Theta only, where Theta corresponds to the angle between the propagation direction and the positive y-axis. However, in 3D, the propagation direction is defined by Theta and Phi; Theta is the angle from the positive z-axis, and Phi is the counterclockwise angle from the positive x-axis.

For vectorial methods, such as the FDTD solver, the polarization is set using the keyword PolarizationAngle, which represents the angle between the H-field and the $\mathbf{z} \times \hat{\mathbf{k}}$ axis, where $\hat{\mathbf{k}}$ is the unit wavevector (see [Sentaurus Device Electromagnetic Wave Solver User Guide, Plane Waves on page 49](#)). For the TMM and RT solvers, the conventions for polarization are as follows:

- In 2D, the keyword Polarization is a real number in the interval [0,1], where Polarization=0 refers to TM, and Polarization=1 refers to TE excitations, respectively. If the keyword PolarizationAngle is specified, it takes precedence over the keyword Polarization. A simple relationship between PolarizationAngle and Polarization can be established: $\text{Polarization} = \sin^2(\text{PolarizationAngle})$.
- In 3D, the polarization vector is defined by rotating the $\mathbf{z} \times \hat{\mathbf{k}}$ vector counterclockwise about the wave direction by an angle defined by PolarizationAngle.

NOTE For the RT solver, specifying the `PolarizationVector` explicitly overrides the keywords `Polarization` and `PolarizationAngle`.

Illumination Window

The concept of an illumination window to confine the light that is incident on the surface of a device structure plays an important role in various simulation setups for photo-diode devices, such as photodetectors, solar cells, and image sensors. Sentaurus Device supports a flexible user interface for the specification of one or more illumination windows, which also allows you to move these windows during a simulation by ramping the corresponding parameters. This common interface is currently available for the TMM, `FromFile` (only for loading 1D profiles), and `OptBeam` solvers, while different solver-specific implementations exist for the RT (see [Using the Raytracer on page 514](#)) and BPM (see [Using Beam Propagation Method on page 561](#)) solvers.

The illumination window is described using a local coordinate system specified by the global location of its origin and the x- and y-directions given as vectors or in terms of rotation angles. Within this local coordinate system, several window shapes can be defined using relative quantities such as width and height, together with an origin anchor for a line in 2D or a rectangular window in 3D. Alternatively, or for shapes that do not support relative quantities, absolute coordinates are used to characterize the shape of the window. For example, a polygon would be defined by specifying a series of vertices in the local 2D coordinate system.

The following window shapes are supported:

- Line (in 2D only)
- Rectangle (in 3D only)
- Polygon (in 3D only)
- Circle (in 3D only)

You can define more than one illumination window. The specification of an optional name tag allows you to refer to the respective window when ramping one of its parameters. It is also helpful for identifying results for a specific window in the current file. If two windows overlap, the solutions of the corresponding windows will be added in the intersection.

The global location of the origin of the local coordinate system is specified with the keyword `Origin`, whose default is $(0, 0, 0)$. To specify its orientation, the directions of the coordinate axes can be defined using the vectors `XDirection` and `YDirection`. In 2D, only `XDirection` needs to be specified. Alternatively, you can set three rotation angles `theta` (angle with z-axis), `phi` (angle between x-axis and projection of window normal $\mathbf{n}$ on xy plane), and `psi` (angle between local x-axis and the vector $\hat{\mathbf{z}} \wedge \hat{\mathbf{n}}$), which define the vector `RotationAngles`.

21: Optical Generation

Solving the Optical Problem

If the location of a window in the local coordinate system is not specified by its corresponding vertex coordinates, its placement is determined by the bounding box of the window shape and a cardinal direction such as North, South, NorthWest, and so on, or Center defining which point of the bounding box coincides with the local origin. The cardinal direction is specified using the keyword `OriginAnchor`.

The illumination window is specified in the `Excitation` section as follows:

```
Physics {
  Optics (
    Excitation (
      Window (
        <Window options>
      )
    )
  )
}
```

The following examples introduce the supported window shapes and show different ways of specifying them. Depending on the simulation setup, one specification may be more convenient than others, for example, if the window needs to be moved by using the parameter ramping feature.

Line

Line normal to y-axis at $y=-10$, $xmin=-5$, $xmax=5$:

```
Window (
  Origin = (0,-10)
  OriginAnchor = Center # Default
  Line ( Dx = 10 )
)
```

Line normal to y-axis at $y=-10$, $xmin=5$, $xmax=500$:

```
Window (
  Origin = (5,-10)
  OriginAnchor = West
  Line ( Dx = 545 )
)
```

or alternatively:

```
Window (
  Origin = (0,-10)
  XDirection = (1, 0, 0) # Default
  Line (
    X1 = 5
  )
)
```

```

        X2 = 500
    )
)

```

Two lines normal to y-axis at y=-10, with xmin1=5, xmax1=10, and xmin2=15, xmax2=20:

```

Window ("W1") (
    Origin = (5, -10)
    OriginAnchor = West
    Line ( Dx = 5 )
)

Window ("W2") (
    Origin = (15, -10)
    OriginAnchor = West
    Line ( Dx = 5 )
)

```

Rectangle

Rectangle normal to z-axis at z=10, with width=10 and height=5, centered at x=0 and y=0:

```

Window (
    Origin = (0, 0, -10)
    OriginAnchor = Center # Default
    Rectangle (
        Dx = 10
        Dy = 5
    )
)

```

or alternatively:

```

Window (
    Origin = (0, 0, -10)
    Rectangle (
        Corner1 = (-5, -2.5)
        Corner2 = (5, 2.5)
    )
)

```

Polygon

Triangle in xz plane at y=4, and corners at (0, 0, 0), (1, 0, 0), and (0, 0, 1):

```

Window (
    Origin = (0, 4, 0)
    XDirection = (1, 0, 0) # Default
    YDirection = (0, 0, 1)
)

```

21: Optical Generation

Solving the Optical Problem

```
Polygon( (0, 0), (1, 0), (0, 1) )
)
```

NOTE More complex polygon specifications also are supported by specifying several polygon loops, which may or may not intersect. In this case, the vertices of each loop must be enclosed in extra parentheses within the Polygon section.

Circle

Circle in xy plane with center at (5, 4, -10) and radius 3:

```
Window (
  Origin = (5, 4, -10)
  XDirection = (1, 0, 0) # Default
  YDirection = (0, 1, 0) # Default
  Circle ( Radius = 3 )
)
```

Moving Windows Using Parameter Ramping

The following syntax demonstrates how illumination windows can be moved and resized during a simulation using the parameter ramping framework:

```
Physics {
  Optics (
    Excitation (
      Window("L1") (
        Origin = (-0.5, -0.1, 0)
        XDirection = (1,0,0) # Default
        Line(
          x1 = -0.5
          x2 = 0.5
        )
      )
    )
    Window("L2") (
      Origin = (0.5, -0.1, 0)
      OriginAnchor = Center # Default
      XDirection = (1,0,0) # Default
      Line( Dx = 1 )
    )
  )
}

Solve{
  Optics
  Quasistationary (
```



```
InitialStep=0.2 MaxStep=0.2 MinStep=0.2
Goal { ModelParameter="Optics/Excitation/Window(L2)/Line/Dx" value=0.2 }
) { Optics }

Quasistationary (
  InitialStep=0.2 MaxStep=0.2 MinStep=0.2
  Goal { ModelParameter="Optics/Excitation/Window(L2)/Origin[0]"
    value=0.9 }
  Goal { ModelParameter="Optics/Excitation/Window(L2)/Origin[1]"
    value=0.5 }
  Goal { ModelParameter="Optics/OpticalSolver/TMM/
    LayerStackExtraction(L2)/Position[0]" value=0.9 }
) { Optics }
}
```

For more details on ramping parameters, including the ramping of vector parameters, see [Parameter Ramping on page 494](#).

Choosing Refractive Index Model

The complex refractive index model as described in [Complex Refractive Index Model on page 496](#) is currently available for the TMM solver, the RT solver, the optical beam absorption solver, and the FDTD solver. It must be specified in the `Optics` section when using the unified interface for optical generation computation.

Extracting the Layer Stack

The optical solvers TMM (see [Transfer Matrix Method on page 538](#)) and `OptBeam` (see [Optical Beam Absorption Method on page 557](#)) require the extraction of a layer stack from the actual device structure as they are based on 1D algorithms whose solution is extended to the dimension of the simulation grid. Both the extension of the solution and the extraction of the layer stack are bound to the illumination window (see [Illumination Window on page 487](#)), which determines the domain of the device structure being represented by the 1D optical problem.

The layer stack extraction algorithm works by extracting all grid elements along a line normal to the corresponding illumination window starting from a user-specified position within the window. The direction of extraction, that is, positive or negative, is derived from the propagation direction of the incident light as specified in the `Excitation` section.

By default, all elements belonging to the same region will form a single layer, which is part of the entire stack. This assumes that the material properties do not vary within the region. If this is not the case or not a good approximation, you can create a separate layer for each extracted

21: Optical Generation

Solving the Optical Problem

element by selecting the option `ElementWise` instead of `RegionWise` for the keyword `Mode` in the `LayerStackExtraction` section. As this can lead to a large number of layers whose difference in material properties may be insignificant for the solution of the optical solver, yet impact its performance, a threshold value can be specified to control the creation of the layer stack from the extracted grid elements.

All elements whose relative difference in the refractive index compared with the previous layer is lower than the threshold are merged into one layer. The material properties of this merged layer are obtained by averaging the corresponding element properties. The thickness of each element computed by the distance between the intersection points of the extraction line with the corresponding element is summed to yield the total thickness of the representative layer of the stack.

Three options are available for defining the starting point of the extraction line:

- Exact position in the global coordinate system using the keyword `Position`.
- Position within the bounding box of the window designated by a cardinal direction.
- Position within the bounding box of the window specified as a point in the local coordinate system of the illumination window.

The last two options require the specification of the keyword `WindowPosition`. Its argument can be either one of `North`, `South`, `NorthWest`, and so on, and `Center` or a 2D point, for example, `WindowPosition = (1.5, 2)`.

The following is an example for a `LayerStackExtraction` section used by the TMM solver:

```
Physics {
  Optics (
    OpticalSolver (
      TMM (
        LayerStackExtraction (
          WindowName = "L1"
          Position = (-0.5, -0.1, 0)
          Mode = ElementWise # Default RegionWise
        )
      )
    )
  )
}
```

The keyword `WindowName` is required if more than one illumination window exists. Its argument specifies to which illumination window the `LayerStackExtraction` refers.

Controlling Computation of Optical Problem in Solve Section

Specifying the keyword `Optics` in the `Solve` section triggers the solution of the optical problem. For example, the following is sufficient to compute the optical generation and the optical intensity:

```
Physics {
  Optics (
    OpticalGeneration ( ... )
  )
}

Solve { Optics }
```

If only the keyword `Optics` is specified in the `Solve` section, either directly or within a `Quasistationary` or `Transient` statement, and no other equations are solved, the simulation is classified as an optics stand-alone simulation. In general, such simulations do not require the specification of contacts in the grid file. In optics stand-alone simulations, it is possible to switch on the creation of double points at region interfaces independent of the type of interface. In other words, a heterojunction interface is treated in the same way as a semiconductor–insulator interface. This feature is enabled by specifying the keyword `Discontinuity` in the `Physics` section.

The `Solve` section of a typical transient device simulation reads as follows:

```
Solve {
  Optics
  Poisson
  Coupled { Poisson Electron Hole }

  Transient (
    InitialTime = 0
    FinalTime = 3e-6
  ) { Coupled { Poisson Electron Hole } }
}
```

In this example, the optical generation rate is computed at the beginning when the optical problem is solved and is used afterwards in the electronic equations. To recompute the optical generation at a later point, the `Optics` statement can be listed wherever a `Coupled` statement is allowed.

The optical generation depends on the solution of the optical problem and, therefore, has an implicit dependency on all quantities that serve as input to the optical equation. Internally, an automatic update scheme ensures that, whenever the optical generation rate is read, all its

underlying variables are up-to-date. Otherwise, it will be recomputed. This mechanism can be switched off by specifying `-AutomaticUpdate` in the `OpticalGeneration` section. By default, `AutomaticUpdate` is switched on.

Parameter Ramping

Sentaurus Device can be used to ramp the values of physical parameters (see [Ramping Physical Parameter Values on page 109](#)). For example, it is possible to sweep the wavelength of the incident light to extract the spectral dependency of a certain output characteristic conveniently. Ramping model parameters of the unified interface for optical generation computation works the same way except that instead of using the expression `Parameter = "<parameter name>"`, it uses the keyword `ModelParameter = "<parameter name or path>"`.

The value of `ModelParameter` can be either the parameter name itself such as `"wavelength"` if it is unambiguous or the parameter name including its full path:

```
Solve {
  Quasistationary (
    Goal {
      [ Device = <device> ]
      [ Material = <material> | MaterialInterface = <interface> |
        Region = <region> | RegionInterface = <interface> ]
      ModelParameter = "Optics/Excitation/Wavelength" Value = <float>
    }
  ){ Optics }
}
```

Specifying the device and location (material, material interface, region, or region interface where applicable) is optional. However, `ModelParameter` and its `Value` must always be specified.

Similarly, the corresponding `CurrentPlot` section reads as follows:

```
CurrentPlot {
  [ Device = <device> ]
  [ Material = <material> | MaterialInterface = <interface> |
    Region = <region> | RegionInterface = <interface> ]
  ModelParameter = "Optics/Excitation/Wavelength"
}
```

Parameters whose value type is a vector of floating-point numbers support ramping of the individual vector components.

For example, to ramp the x- and y-components of the illumination window origin, the following syntax is required:

```
Quasistationary (
  InitialStep=0.2 MaxStep=0.2 MinStep=0.2
  Plot { Range = ( 0., 1 ) Intervals = 5 }
  Goal { ModelParameter="Optics/Excitation/Window(L2)/Origin[0]" value=0.9 }
  Goal { ModelParameter="Optics/Excitation/Window(L2)/Origin[1]" value=0.5 }
) {
  Optics
}
```

NOTE When ramping parameters that reside in sections that can occur multiple times, such as the Window section, an identifying section name or tag in parentheses must follow the section name in the parameter path as shown in the example above.

Table 88 lists the parameters that can be ramped using the unified interface for optical generation computation.

Table 88 Parameters that can be ramped

Parameter path	Parameter name
Optics/OpticalGeneration	Scaling
Optics/OpticalGeneration/ComputeFromMonochromaticSource	Scaling
Optics/OpticalGeneration/ReadFromFile	Scaling
Optics/OpticalGeneration/ComputeFromSpectrum	Scaling
Optics/OpticalGeneration/SetConstant	Value
Optics/Excitation	Wavelength, Intensity, Theta, Phi, PolarizationAngle, Polarization
Optics/Excitation/Window	All parameters that are not of type string or identifier.
Optics/OpticalSolver/<SolverName>	Except for raytracing, all parameters that are not of type string or identifier.

A list of parameter names that can be ramped in the command file is printed when the following command is executed:

```
sdevice --parameter-names <command file>
```

Complex Refractive Index Model

The complex refractive index model in Sentaurus Device allows you to define the refractive index and the extinction coefficient depending on mole fraction, wavelength, temperature, carrier density, and local material gain. In addition, it provides a flexible interface that can be used to add new complex refractive index models as a function of almost any internally available variable.

This complex refractive index model is designed primarily for use in laser and LED simulations, the unified interface for optical generation computation, the raytracer, the TMM, and the FDTD optical solvers. It is also supported by other tools besides Sentaurus Device, such as Sentaurus Device Electromagnetic Wave Solver (EMW) and Sentaurus Mesh, allowing for a consistent source of optical material parameters across the entire tool flow. A common Sentaurus Device parameter file can be shared and only the syntax for activating the different models may slightly change from tool to tool to comply with the respective command file syntax.

Physical Model

The complex refractive index $\tilde{n}$ can be written as:

$$\tilde{n} = n + i \cdot k \quad (520)$$

with:

$$n = n_0 + \Delta n_\lambda + \Delta n_T + \Delta n_{\text{carr}} + \Delta n_{\text{gain}} \quad (521)$$

$$k = k_0 + \Delta k_\lambda + \Delta k_{\text{carr}} \quad (522)$$

The real part n is composed of the base refractive index n_0 , and the correction terms Δn_λ , Δn_T , Δn_{carr} , and Δn_{gain} . The correction terms include the dependency on wavelength, temperature, carrier density, and gain.

The imaginary part k is composed of the base extinction coefficient k_0 , and the correction terms Δk_λ and Δk_{carr} . The correction terms include the dependency on wavelength and carrier density. The absorption coefficient α is computed from k and wavelength λ according to:

$$\alpha = \frac{4\pi}{\lambda} \cdot k \quad (523)$$

Wavelength Dependency

The complex refractive index model offers three ways to take wavelength dependency into account:

- An analytic formula considers a linear and square dependency on the wavelength λ :

$$\begin{aligned}\Delta n_{\lambda} &= C_{n,\lambda} \cdot \lambda + D_{n,\lambda} \cdot \lambda^2 \\ \Delta k_{\lambda} &= C_{k,\lambda} \cdot \lambda + D_{k,\lambda} \cdot \lambda^2\end{aligned}\tag{524}$$

The parameters $C_{n,\lambda}$, $D_{n,\lambda}$, $C_{k,\lambda}$, and $D_{k,\lambda}$ are adjusted in the `ComplexRefractiveIndex` section of the parameter file (see [Table 91 on page 500](#)).

- Tabulated values can be read from the parameter file. The table contains three columns specifying wavelength λ , refractive index n' , and the extinction coefficient k' . Thereby, n' and k' represent $n_0 + \Delta n_{\lambda}$ and $k_0 + \Delta k_{\lambda}$, respectively. The character * is used to insert a comment. The row is terminated by a semicolon. For wavelengths not listed in the table, the refractive index and the extinction coefficient are obtained using linear interpolation or spline interpolation. At least two rows are required for linear interpolation. To form a cubic spline from the table entries, at least four rows are needed.
- Tabulated values can be read from an external file. The file holds a table that is structured as described in the previous point. The name of the file is specified in the `ComplexRefractiveIndex` section of the parameter file.

Temperature Dependency

The temperature dependency of the real part of the complex refractive index follows the relation according to:

$$\Delta n_T = n_0 \cdot C_{n,T} \cdot (T - T_{\text{par}})\tag{525}$$

The parameters $C_{n,T}$ and T_{par} can be adjusted in the `ComplexRefractiveIndex` section of the parameter file (see [Table 94 on page 501](#)).

Carrier Dependency

The change in the real part of the complex refractive index due to free carrier absorption is modeled according to [\[1\]](#):

$$\Delta n_{\text{carr}} = -C_{n,\text{carr}} \cdot \frac{q^2 \lambda^2}{8\pi^2 c^2 \epsilon_0 n_0} \cdot \left(\frac{n}{m_n} + \frac{p}{m_p} \right)\tag{526}$$

21: Optical Generation

Complex Refractive Index Model

where:

- $C_{n, \text{carr}}$ is a fitting parameter.
- q is the elementary charge.
- λ is the wavelength.
- c is the speed of light in free space.
- ϵ_0 is the permittivity.

Furthermore, n and p are the electron and hole densities, and m_n and m_p are the effective masses of electron and hole.

To account for the change of the extinction coefficient due to free carrier absorption, a model is available that assumes linear dependency on carrier concentration and power-law dependency on wavelength:

$$\Delta k_{\text{carr}} = \frac{10^{-4}}{4\pi} \cdot (\lambda^{\Gamma_{k, \text{carr}, n}} C_{k, \text{carr}, n} \cdot n + \lambda^{\Gamma_{k, \text{carr}, p}} C_{k, \text{carr}, p} \cdot p) \quad (527)$$

where:

- $C_{k, \text{carr}, n}$, $C_{k, \text{carr}, p}$, $\Gamma_{k, \text{carr}, n}$, $\Gamma_{k, \text{carr}, p}$ are fitting parameters.
- λ is the wavelength in μm .
- n and p are the electron and hole densities in units of cm^{-3} .

For linear dependency on wavelength, $C_{k, \text{carr}, n}$ and $C_{k, \text{carr}, p}$ are given in units of cm^{-2} .

An alternative formulation found in the literature [2][3] reads as follows:

$$\Delta \alpha_{\text{FCA}} = A n \lambda^B + C p \lambda^D \quad (528)$$

where the free carrier absorption coefficient $\Delta \alpha_{\text{FCA}}$ is given in units of cm^{-1} , the wavelength is given in nanometers, and the carrier concentrations are given in cm^{-3} . The fitting parameters A , B , C , and D in Eq. 528 are related to the corresponding fitting parameters in Eq. 527 by the following expressions:

$$\Gamma_{k, \text{carr}, n} = B + 1 \quad (529)$$

$$C_{k, \text{carr}, n} = 10^{3B} A \quad (530)$$

$$\Gamma_{k, \text{carr}, p} = C + 1 \quad (531)$$

$$C_{k, \text{carr}, p} = 10^{3C} D \quad (532)$$

The parameters $C_{n, \text{carr}}$, $C_{k, \text{carr}, n}$, $C_{k, \text{carr}, p}$, $\Gamma_{k, \text{carr}, n}$, and $\Gamma_{k, \text{carr}, p}$ are adjusted in the ComplexRefractiveIndex section of the parameter file (see [Table 95 on page 501](#)).

Gain Dependency

The complex refractive index model offers two formulas to take into account the gain dependency:

- The linear model is given by:

$$\Delta n_{\text{gain}} = C_{n, \text{gain}} \cdot \left(\frac{n+p}{2N_{\text{par}}} - 1 \right) \quad (533)$$

- The logarithmic model reads:

$$\Delta n_{\text{gain}} = C_{n, \text{gain}} \cdot \ln \left(\frac{n+p}{2N_{\text{par}}} \right) \quad (534)$$

The parameters $C_{n, \text{gain}}$ and N_{par} are adjusted in the ComplexRefractiveIndex section of the parameter file (see [Table 96 on page 502](#)).

Using Complex Refractive Index

The complex refractive index model is activated by using the ComplexRefractiveIndex statement in the Optics section of the Physics section. When using the unified interface for optical generation computation, ComplexRefractiveIndex can be specified globally, per region or per material. Otherwise, support is only available for global specification. [Table 233 on page 1300](#) lists the available keywords, which allow you to select the dependencies that change the complex refractive index, for example:

```
Physics {
  Optics (
    ComplexRefractiveIndex (
      WavelengthDep (real imag)
      TemperatureDep (real)
      CarrierDep (real imag)
      GainDep (real(log))
    )
  )
}
```

All parameters valid for the ComplexRefractiveIndex section of the parameter file are summarized in [Table 89 on page 500](#) to [Table 96 on page 502](#). They can be specified for mole-dependent materials using the standard Sentaurus Device technique with linear interpolation on specified mole intervals.

21: Optical Generation

Complex Refractive Index Model

Interpolation for mole-dependent tables is more complex due to the 2D nature of the problem (interpolation with respect to mole fraction and wavelength) and the sharp discontinuities near the absorption edges. Therefore, if `Formula = 1–3`, the complex refractive index at a given wavelength and mole fraction is computed by interpolating its value with respect to the wavelength for the lower and upper limits of the mole-fraction interval and subsequently with respect to the mole fraction. Optionally, the latter interpolation can be linear or piecewise constant. By default, mole-fraction interpolation for `NumericalTable` is switched off, and Sentauros Device will exit with an error if the complex refractive index is requested at a mole fraction for which no `NumericalTable` is defined.

[Table 89](#) lists the base parameters.

Table 89 Base parameters

Symbol	Parameter name	Unit
n_0	n0	1
k_0	k0	1

The keyword `Formula` is used in the parameter file to select one of the three models that are available to describe the wavelength dependency.

Table 90 Model selection for wavelength dependency

Keyword	Value	Description
Formula	=0	Use analytic formula (default).
	=1	Read tabulated values from parameter file.
	=2	Read tabulated values from external file.
	=3	Use the tabulated values from the <code>TableODB</code> .

[Table 91](#) lists the parameters that are used to describe wavelength dependency.

Table 91 Parameters for wavelength dependency

Symbol	Parameter name	Unit
$C_{n,\lambda}$	Cn_lambda	μm^{-1}
$D_{n,\lambda}$	Dn_lambda	μm^{-2}
$C_{k,\lambda}$	Ck_lambda	μm^{-1}
$D_{k,\lambda}$	Dk_lambda	μm^{-2}

Table 92 lists the different interpolation methods available for NumericalTable. In Table 92, logarithmic interpolation refers to taking the natural logarithm of the coordinates, performing linear interpolation, and exponentiating the results. If a value must be interpolated in an interval where one or both of the coordinates at the interval boundaries is smaller than or equal to zero, linear interpolation is used instead.

Table 92 Specifying interpolation method for NumericalTable

Keyword	Value	Description
TableInterpolation	=Linear	Use linear interpolation.
	=Logarithmic	Use logarithmic interpolation.
	=PositiveSpline	Use cubic spline interpolation. If interpolated values are negative, set them to zero (default).
	=Spline	Use cubic spline interpolation.

Table 93 lists the different interpolation methods available for interpolation of mole fraction–dependent NumericalTable.

Table 93 Specifying interpolation method for mole fraction–dependent NumericalTable

Keyword	Value	Description
MolefractionTableInterpolation	=Linear	
	=Off	Default
	=PiecewiseConstant	

Table 94 lists the parameters that are used to describe temperature dependency.

Table 94 Parameters for temperature dependency

Symbol	Parameter name	Unit
$C_{n,T}$	Cn_temp	K ⁻¹
T_{par}	Tpar	K

Table 95 lists the parameters that are used to describe carrier dependency.

Table 95 Parameters for carrier dependency

Symbol	Parameter name	Unit	Description
$C_{n,\text{carr}}$	Cn_carr	1	
$C_{k,\text{carr},n}, C_{k,\text{carr},p}$	Ck_carr	cm ²	Values for electrons and holes are separated by a comma.

21: Optical Generation

Complex Refractive Index Model

Table 95 Parameters for carrier dependency

Symbol	Parameter name	Unit	Description
$\Gamma_{k, \text{carr}, n}, \Gamma_{k, \text{carr}, p}$	Gamma_k_carr	1	Values for electrons and holes are separated by a comma.

Table 96 lists the parameters that are used to describe gain dependency.

Table 96 Parameters for gain dependence

Symbol	Parameter name	Unit
$C_{n, \text{gain}}$	Cn_gain	1
N_{par}	Npar	cm ⁻³

An example of the ComplexRefractiveIndex section in the parameter file is:

```
ComplexRefractiveIndex {
    * Complex refractive index model: n_complex = n + i*k (unitless)
    * Base refractive index and extinction coefficient
    n_0 = 3.45      # [1]
    k_0 = 0.00      # [1]

    * Wavelength dependence
    * Example for analytical formula:
    Formula = 0
    Cn_lambda = 0.0000e+00      # [um^-1]
    Dn_lambda = 0.0000e+00      # [um^-2]
    Ck_lambda = 0.0000e+00      # [um^-1]
    Dk_lambda = 0.0000e+00      # [um^-2]
    * Example for reading values from parameter file:
    Formula = 1
    TableInterpolation = Spline
    NumericalTable
    ( * wavelength [um]   n [1]   k [1]
      0.30    5.003    4.130;
      0.31    5.010    3.552;
      0.32    5.023    3.259;
      0.33    5.053    3.009;
    )
    * Example for reading values from external file:
    Formula = 2
    NumericalTable = "GaAs.txt"

    * Example for reading values from TableODB
    Formula = 3

    * Temperature dependence (real):
    Cn_temp = 2.0000e-04      # [K^-1]
```

```

Tpar = 3.0000e+02      # [K]

* Carrier dependence (real)
Cn_carr = 1              # [1]
* Carrier dependence (imag)
Ck_carr = 0.0000e+00 , 0.0000e+00  # [cm^2]

* Gain dependence (real)
Cn_gain = 0.0000e+00    # [cm^3]
Npar = 1.0000e+18      # [cm^-3]
}

```

The following `ComplexRefractiveIndex` section demonstrates the use of mole fraction-dependent `NumericalTables`:

```

ComplexRefractiveIndex {
  Formula = 1
  TableInterpolation = Linear
  MolefractionTableInterpolation = Linear
  Xmax(0)=0.0
  NumericalTable(0)
  ( * wavelength [um]   n [1]   k [1]
    0.30    5.003    4.130;
    0.31    5.010    3.552;
    0.32    5.023    3.259;
    0.33    5.053    3.009;
  )
  Xmax(1)=0.25
  NumericalTable(1)
  ( * wavelength [um]   n [1]   k [1]
    0.20    5.003    4.130;
    0.29    4.010    2.552;
    0.34    4.023    1.259;
    0.49    4.053    2.009;
  )
  Xmax(2)=0.6
  NumericalTable(2)
  ( * wavelength [um]   n [1]   k [1]
    0.25    3.123    0.130;
    0.27    3.410    0.552;
    0.29    3.523    0.259;
    0.43    3.753    0.009;
  )
}

```

The complex refractive index is plotted when the keyword `ComplexRefractiveIndex` is defined in the `Plot` section.

21: Optical Generation

Complex Refractive Index Model

If `TableODB` is present but the `ComplexRefractiveIndex` section is absent in the parameter file, then `Formula=3` is used to set the `ComplexRefractiveIndex` numeric table with the values from `TableODB`.

The complex refractive index model is available for the following optical solvers:

- Finite-element solver (scalar and vectorial) (see [Finite-Element Formulation on page 854](#)).
- Effective index and transfer matrix method for VCSEL (see [Effective Index Method for VCSELs on page 869](#) and [Transfer Matrix Method for VCSELs on page 867](#)).
- Transfer matrix method for optical generation computation (see [Transfer Matrix Method on page 538](#)).
- Raytracing (see [Raytracing on page 510](#)).

Complex Refractive Index Model Interface

The complex refractive index model interface (CRIMI) allows the addition of new complex refractive index models as a function of almost any internally available variable. These models must be implemented as C++ functions, and Sentauros Device loads the functions at run-time using the dynamic loader. No access to the Sentauros Device source code is necessary. The concept is similar to that of the [Physical Model Interface on page 979](#); however, it is not limited to Sentauros Device.

The generated shared object containing the model implementation can be used together with Sentauros Device Electromagnetic Wave Solver (see [Sentauros Device Electromagnetic Wave Solver User Guide, Complex Refractive Index Model Interface on page 42](#)) and Sentauros Mesh (see [Mesh Generation Tools User Guide, Computing Cell Size Automatically \(EMW Applications\) on page 77](#)).

Three main steps are required for integrating user-defined models:

- First, a C++ class implementing the complex refractive index model must be written.
- Second, a shared object must be created that can be loaded at run-time.
- Third, the model must be activated in the command file.

These steps are described in the following sections in more detail.

C++ Application Programming Interface (API)

For each complex refractive index model, two C++ subroutines must be implemented: one to compute the refractive index and another to compute the extinction coefficient. More specifically, a C++ class must be implemented that is derived from a base class declared in the header file `CRIModels.h`. In addition, a so-called virtual constructor function, which allocates

an instance of the derived class, and a virtual destructor function, which de-allocates it, must be provided.

The public interface of the base class from which a model-specific class must be derived is declared in the header file `CRIModels.h` as:

```
class CRI_Model : public CRI_Model_Interface {
public:
    CRI_Model(const CRI_Environment& env);
    virtual ~CRI_Model();
    //definition of complex refractive index
    struct ComplexRefractiveIndexConstituentsReal {
        double n0;
        double dn_lambda;
        double dn_temp;
        double dn_carr;
        double dn_gain;
    };

    struct ComplexRefractiveIndexConstituentsImag {
        double k0;
        double dk_lambda;
        double dk_carr;
    };

    //methods to be implemented by user

    //definition of complex refractive index in terms of its constituents
    virtual void Compute_n(ComplexRefractiveIndexConstituentsReal& data);
    virtual void Compute_k(ComplexRefractiveIndexConstituentsImag& data);

    //direct definition of complex refractive index
    virtual void Compute_n(double& n) = 0;
    virtual void Compute_k(double& k) = 0;
};
```

The CRIMI provides a *basic interface* where only the actual value of the refractive index and the extinction coefficient can be overridden as well as an *advanced interface*. In the advanced interface, the total values are computed automatically from its constituents, and it has the advantage that the user-specified constituents can be visualized instead of its default values. If a CRIMI is used in a thermal simulation where free carrier absorption plays a role, using the advanced interface is mandatory. For more details, see [Quantum Yield Models on page 475](#).

The arguments of the Compute functions indicate that they are passed as references to the respective functions. They carry the values corresponding to the model specification in the `ComplexRefractiveIndex` section in the command file. When implementing a user-defined

21: Optical Generation

Complex Refractive Index Model

complex refractive index model, the values for n and k (in the case of the basic interface) or the members of `struct ComplexRefractiveIndexConstituentsReal` and `struct ComplexRefractiveIndexConstituentsImag` must be overwritten in the function body. Otherwise, the original values remain unchanged, which can be useful if, for example, only a new model for either the real or the imaginary part of the complex refractive index must be implemented or if only a specific constituent needs to be modified.

Often, a complex refractive index model is based on several material-specific parameters. For each region or material, these parameters can be defined in special sections of the parameter file that carry the model name as given in the command file. The parameters are best initialized in the constructor as it is called once for every region. For this purpose, the member function called `InitParameter` is used, which has two arguments. The first argument is the name of the parameter as listed in the parameter file, and the second argument is its default value. The latter is used if no value is assigned to the parameter for a particular region or material. The supported value types for user-defined parameters are integer, floating-point number, and string.

The following example shows how to initialize a parameter called `Gamma` in the constructor:

```
Constant_CRI_Model::Constant_CRI_Model(const CRI_Environment& env) :  
CRI_Model(env) {  
  
    Gamma = InitParameter("Gamma", 1.5);  
  
}
```

where `Gamma` was declared as a data member of type `double` in the class `Constant_CRI_Model`.

In addition to the implementation of the two class member functions `Compute_n` and `Compute_k`, the virtual constructor function and destructor function must be defined as:

```
extern "C" {  
  
    opto_n_cri::CRI_Model*  
    new_CRI_Model(const opto_n_cri::CRI_Environment& env)  
    {  
        return new opto_n_cri::Constant_CRI_Model(env);  
    }  
  
    void  
    delete_CRI_Model(opto_n_cri::CRI_Model* cri_model)  
    {  
        delete cri_model;  
    }  
  
}
```


In the above sample functions, it is assumed that the name of the user-provided derived class is `Constant_CRI_Model`.

Run-Time Support

The base class of `CRI_Model` is derived from another class called `CRI_Model_Interface`, which adds several functions that extend the possibilities for defining new complex refractive index models. Among them are functions that query basic properties of the model such as its name, for which region and material it is called and which models have been specified in the command file. Another group of functions allows you to read the values of the most common variables on which the complex refractive index depends, namely, wavelength, carrier densities, and temperature as well as the default values for the various contributions to the complex refractive index. For most models, this set of functions should be sufficient.

For advanced models, it may be necessary to have access to additional internal variables to model the dependencies correctly. To this end, three interface functions are available that allow you to query, to register, and to read the internal variables available. A specific variable can only be read in the `Compute` functions if it has been registered in the constructor. The internal name of the corresponding variable can be obtained from the function `GetAvailableVariables`, which returns a vector of strings containing the names of all supported variables.

The remaining functions offer support for mole fraction-dependent models and for direct access of the `NumericalTables` defined in the parameter file.

The signatures of the run-time support functions discussed here can be found in the header file `CRIModels.h` and are briefly described below. Corresponding type declarations are contained in the header file `ExportedTypes.h` in the same installation directory. These header files are located in `$STROOT/tcad/$STRELEASE/lib/opto/include/`.

General utility functions:

- `std::string Name() const`: Returns the name of the `CRIModel` as specified in the command file.
- `std::string ReadRegionName() const`: Returns the name of the region to which the vertex belongs.
- `std::string ReadMaterialName() const`: Returns the name of the material to which the vertex belongs.
- `bool WithModel(ComplexRefractiveIndexModelType model) const`: Returns true if `model` is activated for the region to which the vertex belongs; otherwise, false. For each `model`, a corresponding specification in the `ComplexRefractiveIndex` section of the command file exists (see [Using Complex Refractive Index on page 499](#)).

21: Optical Generation

Complex Refractive Index Model

Functions for reading the values of the most common variables on which the complex refractive index depends:

- `double ReadWavelength() const`: Returns wavelength in [μm].
- `double ReadTemperature() const`: Returns temperature in [K].
- `double Read_eDensity() const`: Returns electron density in [cm^{-3}].
- `double Read_hDensity() const`: Returns hole density in [cm^{-3}].
- `double ReadQW_eDensity() const`: Returns quantum-well electron density corresponding to the bound states of the well in [cm^{-3}].
- `double ReadQW_hDensity() const`: Returns quantum-well hole density corresponding to the bound states of the well in [cm^{-3}].

Functions for accessing additional internal variables to model the dependencies correctly:

- `const std::vector<std::string>& GetAvailableVariables() const`: Returns a vector of strings containing all internal variable names.
- `void RegisterVariableToRead(std::string variable_name)`: Use variable name string from previous call to `GetAvailableVariables()` to register a variable for reading in the function body of the Compute functions. This is performed typically in the constructor.
- `double ReadVariableValue(std::string variable_name) const`: Returns current value of variable selected by specifying variable name string from previous call to `GetAvailableVariables()` as argument.

Functions for reading the real and imaginary parts of the complex refractive index as well as its corresponding constituents computed according to the command file and parameter file specification:

- `double Read_n() const`: Returns the real part of the complex refractive index.
- `double Read_k() const`: Returns the imaginary part of the complex refractive index.
- `double Read_n0() const`: Returns the base refractive index given in [Eq. 521, p. 496](#).
- `double Read_k0() const`: Returns the base extinction coefficient given in [Eq. 522, p. 496](#).
- `double Read_d_n_lambda() const`: Returns the change in refractive index due to its wavelength dependency.
- `double Read_d_k_lambda() const`: Returns the change in extinction coefficient due to its wavelength dependency.
- `double Read_d_n_temp() const`: Returns the change in refractive index due to its temperature dependency.
- `double Read_d_n_carr() const`: Returns the change in refractive index due to its carrier dependency.

- `double Read_d_k_carr() const`: Returns the change in extinction coefficient due to its carrier dependency.
- `double Read_d_n_gain() const`: Returns the change in extinction coefficient due to its gain dependency.

Functions for retrieving information about mole fraction-dependent models and for direct access of the `NumericalTables` defined in the parameter file:

- `int ReadNumberOfMoleFractionIntervals() const`: Returns the number of mole-fraction intervals defined in the `ComplexRefractiveIndex` section of the parameter file.
- `double ReadxMoleFractionUpperLimit(int interval) const`: Returns upper limit of x-mole fraction interval specified as an argument.
- `double ReadNumericalTableValue_n(int interval = 0) const`: Returns the refractive index for the current wavelength and the mole-fraction interval specified as an argument.
- `double ReadNumericalTableValue_k(int interval = 0) const`: Returns the extinction coefficient for the current wavelength and the mole-fraction interval specified as an argument.
- `NumericalTable<double>* ReadNumericalTable(int interval = 0) const`: Returns the numeric table for the mole-fraction interval specified as an argument. This function may be useful if a custom table interpolation algorithm must be implemented.

Shared Object Code

Sentaurus Device assumes that the shared object code corresponding to a CRI model can be found in the file `modelname.so.arch`. The base name of this file must be identical to the name of the CRI model. The extension `.arch` depends on the hardware architecture.

The script `cmi`, which is also a part of the CMI (see [Compact Models User Guide, Chapter 4 on page 119](#)), can be used to produce the shared object files (see [Compact Models User Guide, Run-Time Support on page 135](#)).

Command File of Sentaurus Device

To load CRI models into Sentaurus Device, the `PMIPath` search path must be defined in the `File` section of the command file. The value of `PMIPath` consists of a sequence of directories, for example:

```
File {  
    PMIPath = ". /home/joe/lib /home/mary/sdevice/lib"  
}
```

21: Optical Generation

Raytracing

For each CRI model, which appears in the `ComplexRefractiveIndex` section, the given directories are searched for a corresponding shared object file `modelName.so.arch`.

A CRI model can be activated in the `ComplexRefractiveIndex` section of the command file by specifying the name of the model as shown in the following example:

```
Physics {
  Optics (
    ComplexRefractiveIndex (
      CRIModel (Name = "modelName")
    )
  )
}
```

A CRI model name can only consist of alphanumeric characters and underscores (_). The first character must be either a letter or an underscore. All the CRI models can be specified regionwise or materialwise:

```
Physics (region = "Region.1") {
  ...
}

Physics (material = "SiO2") {
  ...
}
```

when using the unified interface for optical generation computation in Sentaurus Device; otherwise, only global specification is supported.

Raytracing

Sentaurus Device supports the simulation of photogeneration by raytracing in 2D and 3D for arbitrarily shaped structures. The calculation of refraction, transmission, and reflection follows geometric optics, and special boundary conditions can be defined. A dual-grid setup can be used to speed up simulations that include raytracing.

Raytracer

In Sentaurus Device, the raytracer has been implemented based on linear polarization. It is optimized for speed and needs to be used in conjunction with the complex refractive index model (see [Complex Refractive Index Model on page 496](#)). Each region/material must have a complex refractive index section defined in the parameter file. If the refractive index is zero, it is set to a default value of 1.0.

The raytracer uses a recursive algorithm: It starts with a source ray and builds a binary tree that tracks the transmission and reflection of the ray. A reflection/transmission process occurs at interfaces with refractive index differences. This is best illustrated in [Figure 31](#).

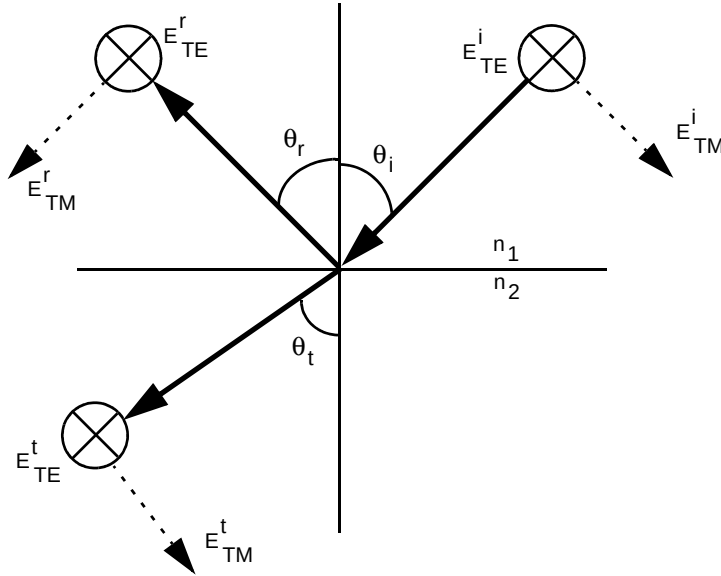


Figure 31 Incident ray splits into reflected and transmitted rays at an interface: the TE component of the polarization vector maintains the same direction, whereas the TM component changes direction

An incident ray impinges on the interface of two different refractive index (n_1 and n_2) regions, resulting in a reflected ray and a transmitted ray. The incident, reflected, and transmitted rays are denoted by the subscripts i , r , and t , respectively. Likewise, the incident, reflected, and transmitted angles are denoted by θ_i , θ_r , and θ_t , respectively. These angles can be derived from the concept of interface tangential phase-matching (commonly called Snell's law) using:

$$n_1 \sin \theta_i = n_2 \sin \theta_t \quad (535)$$

To define these angles, a plane of incidence must be clearly defined. It is apparent that the plane of incidence is the plane that contains both the normal to the interface and the vector of the ray. When the plane of incidence is defined, the concept of TE and TM polarization can then be established.

A ray can be considered a plane wave traveling in a particular direction with its polarization vector perpendicular to the direction of propagation. The length of the polarization vector represents the amplitude, and the square of its length denotes the intensity. The TE polarization (s-wave) applies to the ray polarization vector component that is perpendicular to the plane of incidence. On the other hand, the TM polarization (p-wave) applies to the ray polarization vector component that is parallel to the plane of incidence. In [Figure 31 on page 511](#), the TE and TM components of the ray polarization vector are denoted by E_{TE} and E_{TM} , respectively.

21: Optical Generation

Raytracing

The TE and TM components of the ray polarization vector experience different reflection and transmission coefficients. These coefficients are:

Amplitude reflection coefficients:

$$r_{\text{TE}} = \frac{k_{1z} - k_{2z}}{k_{1z} + k_{2z}} \quad (536)$$

$$r_{\text{TM}} = \frac{\epsilon_2 k_{1z} - \epsilon_1 k_{2z}}{\epsilon_2 k_{1z} + \epsilon_1 k_{2z}} \quad (537)$$

Amplitude transmission coefficients:

$$t_{\text{TE}} = \frac{2k_{1z}}{k_{1z} + k_{2z}} \quad (538)$$

$$t_{\text{TM}} = \frac{2\epsilon_2 k_{1z}}{\epsilon_2 k_{1z} + \epsilon_1 k_{2z}} \quad (539)$$

Power reflection coefficients:

$$R_{\text{TE}} = |r_{\text{TE}}|^2 \quad (540)$$

$$R_{\text{TM}} = |r_{\text{TM}}|^2 \quad (541)$$

Power transmission coefficients:

$$T_{\text{TE}} = \frac{k_{2z}}{k_{1z}} |t_{\text{TE}}|^2 \quad (542)$$

$$T_{\text{TM}} = \frac{\epsilon_1 k_{2z}}{\epsilon_2 k_{1z}} |t_{\text{TM}}|^2 \quad (543)$$

where:

$$k_0 = 2\pi/\lambda_0 \quad (544)$$

$$k_{1z} = n_1 k_0 \cos \theta_i \quad (545)$$

$$k_{2z} = n_2 k_0 \cos \theta_t \quad (546)$$

$$\epsilon_1 = n_1^2 \quad (547)$$

$$\epsilon_2 = n_2^2 \quad (548)$$

where λ_0 is the free space wavelength, and k_0 is the free space wave number. Note that for amplitude coefficients, $1 + r = t$. For power coefficients, $R + T = 1$. These relations can be verified easily by substituting the above definitions of the reflection and transmission coefficients of the respective TE and TM polarizations. For normal incidence when $\theta_i = \theta_t = 0$, $r_{TE} = -r_{TM}$, and $R_{TE} = R_{TM}$.

If the refractive index is complex, the reflection and transmission coefficients are also complex. In such cases, only the absolute value is taken into account.

The raytracer automatically computes the plane of incidence at each interface, decomposes the polarization vector into TE and TM components, and applies the respective reflection and transmission coefficients to these TE and TM components.

Ray Photon Absorption and Optical Generation

When there is an imaginary component (extinction coefficient), κ , to the complex refractive index, absorption of photons occurs. To convert the absorption coefficient to the necessary units, the following formula is used for power/intensity absorption:

$$\alpha(\lambda)[\text{cm}^{-1}] = \frac{4\pi\kappa}{\lambda} \quad (549)$$

In the complex refractive index model, the refractive index is defined element-wise. In each element, the intensity of the ray is reduced by an exponential factor defined by $\exp(-\alpha L)$ where L is the length of the ray in the element. Therefore, the photon absorption rate in each element is:

$$G^{\text{opt}}(x, y, z, t) = I(x, y, z)[1 - e^{-\alpha L}] \quad (550)$$

$I(x, y, z)$ is the rate intensity (units of s^{-1}) of the ray in the element. After all of the photon absorptions in the elements have been computed, the values are interpolated onto the neighboring vertices and are divided by its sustaining volume to obtain the final units of $\text{s}^{-1} \cdot \text{cm}^{-3}$. The absorption of photons occurs in all materials (including nonsemiconductor) with a positive extinction coefficient for raytracing. Depending on the quantum yield (see [Quantum Yield Models on page 475](#)), a fraction of this value is added to the carrier continuity equation as a generation rate so that correct accounting of particles is maintained.

Using the Raytracer

The raytracer must be invoked within the unified interface for optical generation computation (see [Raytracing on page 483](#)), and can have the following options:

- Raytracing used in simple optical generation.
- Monte Carlo raytracing.
- Multithreading for raytracing.
- Compact memory model for raytracing.
- Rectangular and user-defined windows of starting rays.
- Different boundary conditions for raytracing.
- Visualizing raytracing results.

Optical generation by raytracing is activated by the `RayTracing` statement in the `Physics` section. An example of optical generation by raytracing is:

```
Plot {
  RayTrees
}
Physics {
  Optics (
    ComplexRefractiveIndex(
      WavelengthDep(real imag)           # switch on wavelength dependency
      CarrierDep( real imag )
      GainDep( real )                    # or real(log)
      TemperatureDep( real )
      CRImodel ( Name = "crimodelname" )
    )
    OpticalGeneration(...)
    OpticalSolver (
      RayTracing(
        PolarizationVector=(0, 0, 1)
        MinIntensity = 1e-3
        DepthLimit = 100
        Print(Skip(integer))
        RetraceCRChange = float          # fractional change for retrace
        RectangularWindow(...)
      )
    )
  )
}
```

The complex refractive index model must be used in conjunction with the raytracer. Various dependencies of the complex refractive index can be included, and they are controlled by parameters in the `ComplexRefractiveIndex` section of the parameter file (see [Complex](#)

[Refractive Index Model on page 496](#)). The CRI model can be defined regionwise or materialwise. The keyword `RetraceCRIchange` specifies the fractional change of the complex refractive index (either the real or imaginary part) from its previous state that will force a total recomputation of raytracing. To force retracing at every ramping point, you can set `RetraceCRIchange` to a negative number.

The starting rays are defined by a rectangular window or a user-defined window (see [Window of Starting Rays on page 517](#)).

`Print` creates a `.grd` file with information about the paths that the rays take. The file name is obtained from the `Plot` file name by appending `_ray.grd`. If no `Plot` file name is specified, the name defaults to `Raytrace_ray.grd`.

Terminating Raytracing

Raytracing offers two termination conditions:

- Raytracing is terminated if the ray intensity becomes less than n times (`MinIntensity` specifies n) the original intensity.
- `DepthLimit` specifies the maximum number of material boundaries that the ray can pass through.

Monte Carlo Raytracing

In instances where rays are randomly scattered, for example on rough surfaces, a Monte Carlo-type raytracing is required, since you need to look at the aggregate solution of the raytracing process.

The concept of Monte Carlo raytracing follows that of the Monte Carlo method for carrier transport simulation. Suppose a ray impinges an interface. In the deterministic framework, the ray will split into a reflected part and a transmitted part at this interface. In the Monte Carlo framework, you track only one ray path and take the reflectivity as a probability constraint to decide if the ray is to be reflected or transmitted. As more rays impinge this material interface, the aggregate number of reflected rays will recover information about the reflectivity, and this is the crux of the Monte Carlo method. In a likewise manner, rough surface scattering gives an angular probability and, using the same strategy, the ensemble average of rays can model the physics of rough surface scattering.

As an example, the algorithm for the Monte Carlo raytracing at a regular material interface is:

- Compute the reflection and transmission coefficients, R and T , of the ray at the material interface.

21: Optical Generation

Raytracing

- Generate a random number, r .
- If $r \leq R$, then choose to propagate the reflected ray only.
- If $r > R$, then choose to propagate the transmitted ray only.

In the case of special rays, such as those from the raytrace PMI, care must be taken to choose only a single propagating ray based on a new set of probabilistic rules.

The Monte Carlo raytracing has been implemented in both general raytracing and LED raytracing. The syntax is:

```
Physics {  
    Optics (  
        OpticalSolver (  
            RayTracing (  
                MonteCarlo  
            )  
        )  
    )  
}
```

NOTE Since only one ray path is chosen at each scattering event in the Monte Carlo method, the rays in the raytree will appear to have the same intensity values after scattering.

Multithreading for Raytracer

Each raytree traced from the list of starting rays is mutually exclusive, so that the raytracer is an excellent candidate for parallelization.

To activate the multithreading capability of the raytracer, include the following syntax in the Math section of the command file:

```
Math {  
    Number_Of_Threads = 2    # or maximum  
    StackSize = 20000000    # increase to, for example, 20 MB  
}
```

NOTE Raytracing is a recursive process, so it can easily obliterate the default (1 MB) stack space if the level of the raytree becomes increasingly deep. This may lead to unexplained segmentation faults or abortion of the program. To resolve this issue, increase the stack space using the keyword `StackSize` in the Math section as shown in the above syntax. The other solution is to use the compact memory model as described next.

Compact Memory Model for Raytracer

The compact memory model for the raytracer does not store the raytree as it is being created, so it reduces the use of significant memory. Only essential information is tracked and stored, and this alleviates the amount of memory required. A reduced structure is used, and clever ways of extracting information in this reduced structure have been implemented without upsetting the multithreading feature of raytracing.

NOTE If this reduced memory option is activated, plotting or printing of raytrees is not possible since the raytrees are not stored.

The command file syntax is:

```
Physics {
  RayTrace (
    CompactMemoryOption
  )
}
```

Window of Starting Rays

Three mutually exclusive options for defining a set of starting rays are available: a rectangular window, a user-defined set of rays, or a distribution window. These can be defined within the Physics-Optics-OpticalSolver-Raytracing statement syntax:

```
RectangularWindow (
  RectangleV1 = vector          # vertex 1 of rectangular window [um]
  RectangleV2 = vector          # vertex 2 of rectangular window [um]
  RectangleV3 = vector          # vertex 3 needed only for 3D [um]
  # number of rays = LengthDiscretization * WidthDiscretization
  LengthDiscretization = integer # longer side
  WidthDiscretization = integer  # shorter side needed only for 3D
  RayDirection = vector
)

UserWindow (
  NumberOfRays = integer        # number of rays in file
  RaysFromFile = "filename.txt" # position(um) direction area(cm^2)
)

RayDistribution("windowname1")( ... )
```

Rectangular Window of Rays

In the RectangularWindow definition of starting rays, you select the discretization of the window, and the position of each starting ray will be centered in each discretized cell of the window. The starting position remains as defined even if the ray direction is changed. If RayDirection is omitted, the direction is computed from Theta and Phi of the Excitation section. In this way, you also can ramp the direction of the input rays.

User-Defined Window of Rays

In the UserWindow section, you can enter your own set of starting rays using a text file. This means that you can choose the positions, directions, and area of each starting ray, thereby achieving greater flexibility. The power (W) of each ray then is computed by multiplying the input area (cm^2) by the input wave power (W/cm^2). Remarks are allowed but they must begin with a hash sign (#). A sample of this file is:

```
filename.txt:
# Position(x,y,z)[um] Direction(x,y,z)[um] Area[cm^2]
0.0 0.0 0.0 0.0 0.0 1.0 1.0e-5
0.0 0.05 0.0 0.0 0.0 1.0 1.0e-5
0.0 0.05 0.05 0.0 0.0 1.0 1.0e-5
...
```

Distribution Window of Rays

When the illumination window (see [Illumination Window on page 487](#)) is defined for raytracing, a corresponding RayDistribution section must be created to define how the excitation parameters can be translated in a raytracing context:

```
Physics {
  Optics (
    Excitation(
      Window ("windowname1")(
        Origin = (xx,yy)
        OriginAnchor = Center | North | South | East | West | NorthEast |
          SouthEast | NorthWest | SouthWest
        RotationAngles = (phi, theta, psi)
        xDirection = (x,y,z)
        yDirection = (x,y,z)
        # You can define one of the following excitation shapes.
        # Rectangle, Circle, and Polygon are for three dimensions.
        Rectangle( ... )
        Line( ... )
        Circle( ... )
        Polygon( ... )
      )
    )
  )
}
```

```

        Window ("windowname2") (...)
        Window ("windowname3") (...)
    )
    OpticalSolver(
        Raytracing(
            RayDistribution("windowname1")(
                Mode = Equidistant | MonteCarlo | AutoPopulate
                NumberOfRays = integer      # for Mode=MonteCarlo | AutoPopulate
                Dx = float                    # for Mode=Equidistant
                Dy = float                    # for Mode=Equidistant
            )
            RayDistribution("windowname2")(...)
            RayDistribution("windowname3")(...)
        )
    )
}

```

Multiple excitation windows and RayDistribution windows can be created. The only restriction is that every RayDistribution section must have a matching Excitation section. Matching is through the user-specified window name. If no window name for the RayDistribution section is specified, the parameters in that window are applied to all excitation windows, except those with matching window names.

The shape of the excitation is defined in the Excitation(...Window(...)) section, and the shape can be a Line for two dimensions, and a Rectangle, Circle, or Polygon for three dimensions.

There are three different ways in which you can create a set of starting rays on the excitation shape:

- **Mode=Equidistant:** The starting positions of the rays are arranged in a regular grid with intervals defined by Dx and Dy. Then, the grid is superimposed onto the excitation shape to extract an appropriate set of starting rays.
- **Mode=MonteCarlo:** Random positions are generated with the excitation shape to form the set of starting rays.
- **Mode=AutoPopulate:** A uniform grid of variable grid size is superimposed onto the excitation shape. The grid size is varied until the maximum possible number of grid points that is less than NumberOfRays can be fitted into the shape, and the set of starting ray positions is extracted from the fitted grid vertices.

Boundary Condition for Raytracing

Special and spatially arbitrary boundary conditions can be specified in raytracing. There are two ways to define a boundary condition (BC) for raytracing by:

- Using special contacts.

In the contact-based definition, the boundaries are drawn as contacts (see [Specifying Electrical Boundary Conditions on page 99](#)) and are labeled accordingly using Sentaurus Structure Editor. These contacts can be constructed specifically for setting a raytrace boundary condition, or they can coincide with an electrode or a thermode. To define clearly special raytrace boundary conditions, each line or surface contact defined must have a distinct name. This means that two parallel contacts must have different contact names, and intersecting contacts must also have different contact names. The contact is discretized into edges (2D) or faces (3D) after meshing.

- Using Physics MaterialInterface or Physics RegionInterface.

In the Physics MaterialInterface– or Physics RegionInterface–based definition, the boundaries are defined at the interface between the material or region. As with typical Sentaurus Device operation, RegionInterface takes precedence over MaterialInterface. After meshing, the interface is discretized into edges (2D) or faces (3D).

A mixture of contact-based and physics interface definitions of BCs is allowed. However, the contact-based definition takes precedence over the physics interface–based definition if there is an overlap.

Each edge or face defined as a special contact can only associate itself to one type of boundary condition. The following boundary conditions are listed in the order of preference, that is, if two boundary conditions are specified for the same edge or face, the higher ranked one is chosen:

1. Fresnel BC.
2. Constant reflectivity/transmittivity BC.
3. Raytrace PMI BC.
4. Multilayer antireflective coating BC.
5. Photon-recycling BC.

Fresnel Boundary Condition

The physics interface–based BC can quickly set many interfaces to a particular type of BC, especially if the material interface contains many region interfaces. What is missing is the disabling of a particular region interface from the general BC definition. Therefore, the Fresnel

BC is introduced as a complement set to unset a particular region interface-based or contact-based BC. This can greatly simplify the command syntax input and enhance ease of use.

Furthermore, the Fresnel BC also can be used to *sense* the photon flux flowing through a particular interface when users activate the `PlotInterfaceFlux` feature (see [Plotting Interface Flux on page 531](#)).

The syntax for activating the Fresnel BC is:

<pre>RayTraceBC { { Name = "contact1" Fresnel } }</pre>		<pre>Physics(RegionInterface="reg1/reg2") { RaytraceBC (Fresnel) }</pre>
---	--	--

Constant Reflectivity and Transmittivity Boundary Condition

In the command file, constant reflectivity and transmittivity boundaries must be specified by the following syntax:

<pre>RayTraceBC { { Name = "ref_contact1" Reflectivity = float } { Name = "ref_contact2" Reflectivity = float Transmittivity = float } ... }</pre>		<pre>Physics (RegionInterface="regionname1/regionname2") { RayTraceBC (Reflectivity = float Transmittivity = float) }</pre>
--	--	---

These are power reflection, R , and transmission, T , coefficients. It is not necessary for $R + T = 1$. If R is specified only, $T = 1 - R$. If T is specified only, $R = 1 - T$.

For total absorbing or radiative boundary conditions, set `Reflectivity = 0` and `Transmittivity = 0` (or `Transmittivity = 1`). Defining `Reflectivity = 1` ensures that rays are totally reflected at that boundary. In the 2D case, reflection occurs at the edge of the element. In the 3D case, reflection occurs at the face of the element. This versatile boundary condition feature enables you to use symmetry to reduce the simulation domain.

21: Optical Generation

Raytracing

Examples of the boundary condition whereby Reflectivity has been set to 1.0 are shown in [Figure 32](#) and [Figure 33](#). In the 2D case, a star boundary is drawn within a device; in the 3D case, a rectangular boundary is drawn within the device.

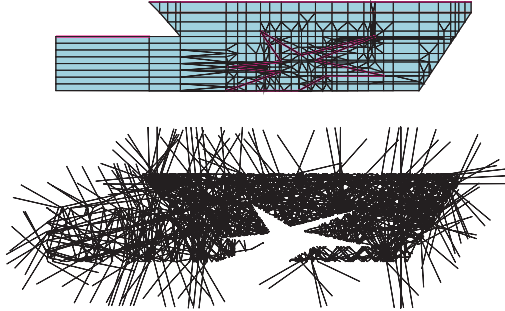


Figure 32 Applying the reflecting boundary condition in a 2D LED simulation; the boundary is drawn as a star inside the device

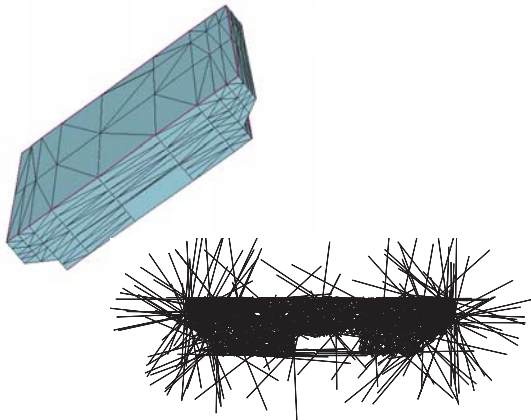


Figure 33 Applying the reflecting boundary condition in a 3D LED simulation; the boundary is drawn as a hollow rectangular waveguide inside the device

Raytrace PMI Boundary Condition

A special raytrace PMI BC can be defined. You can obtain useful information about the ray with this raytrace PMI and can modify some parameters of the ray. For details about this PMI and how it can be incorporated into the simulation, see [Special Contact PMI for Raytracing on page 1139](#).

As with the standard PMI, you can create a PMI section in the parameter file where you can initialize the parameters of the PMI. Note that the parameter must be specified either regionwise or material-wise in accordance to the standard PMI framework.

To activate the PMI, specify the location of the PMI BC and include the following syntax in the RayTraceBC section of the command file:

```
RayTraceBC {...
  { Name = "pmi_contact"
    PMIModel = "pmi_modelname"
  }
}
Physics (MaterialInterface="materialname1/materialname2") {
  RayTraceBC (
    pmiModel = "modelname"
  )
}
```

NOTE Care must be taken to ensure that your PMI code is thread safe since the raytracing algorithm is multithreaded. Use only local variables and avoid global variables in your PMI code (see [Parallelization on page 1009](#)).

Thin-Layer-Stack Boundary Condition

A thin-layer-stack boundary condition can be used to model interference effects in raytracing. The modeling of antireflection coatings used in solar cells is a typical example of the use of such boundary condition. The coatings are specified as special contacts and are treated as a boundary condition for the raytracer. The angle at which the ray is incident on the coating is passed as input to the TMM solver (the theory is described in [Transfer Matrix Method on page 538](#)), which returns the reflectance, transmittance, and absorbance for both parallel and perpendicular polarizations to the raytracer. The angle of refraction is calculated by the raytracer according to Snell's law, a direct implication of phase matching. This boundary condition is available for both 2D and 3D simulations.

The following command file excerpt, along with its demonstration in [Figure 34 on page 524](#), shows the use of this boundary condition:

```
RayTraceBC {
  { Name="rayContact1" reflectivity=1.0 }
  { Name="rayContact2"
    ReferenceMaterial = "Gas"
    LayerStructure {
      70e-3 "Nitride"; # um
      6e-3 "Oxide"    # um
    }
  }
}
Physics (MaterialInterface="Gas/Silicon") {
  RayTraceBC (
    TMM (
      ReferenceMaterial = "Gas"
      LayerStructure {
        70e-3 "Nitride"; # um
        6e-3 "Oxide";   # um
      }
    )
  )
}
```

The first line in the RayTraceBC section above shows the definition of a constant reflectivity as a boundary condition (see [Fresnel Boundary Condition on page 520](#)). This option is usually chosen for the boundary of the simulation domain.

21: Optical Generation

Raytracing

The other section defines a multilayer structure, for which the corresponding contact (rayContact2) in the grid file can be seen as a placeholder. Alternatively, you can use the Physics interface method of specifying such a special BC (as shown in the above syntax to the right). For each layer, the corresponding thickness (μm) and material name must be specified. For the calculation of transmittance and reflectance, the transfer matrix method reads the complex refractive index from the respective parameter file. An additional parameter file or a new set of parameters must be added by you in either of the following cases: (a) a layer contains a material that does not exist in the grid file or (b) the material properties of a layer differ from the properties of a region in the grid file with the same material.

To fix the orientation of the multilayer structure with respect to the specified contact in the grid file, a ReferenceMaterial or a ReferenceRegion that is adjacent to the contact must be specified. The topmost entry of the LayerStructure table corresponds to the layer that is adjacent to the ReferenceMaterial or ReferenceRegion.

NOTE It is only possible to specify a ReferenceMaterial or ReferenceRegion if the material names or region names, respectively, on either side of the contact do not coincide.

The multilayer structure is allowed to have an arbitrary number of layers.

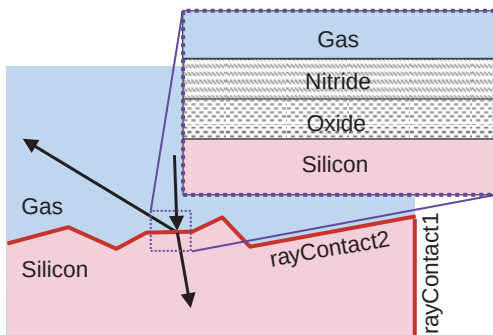


Figure 34 Illustration of thin-layer-stack boundary condition for simulation of an antireflection coated solar cell

TMM Optical Generation in Raytracer

In modern thin-film solar-cell design, the multilayer thin film can be made of materials that can generate carriers by absorbing photons. To cater to such a phenomenon, the TMM contact in the raytracer has been modified to collect optical generations as rays traverse the TMM contact. The collected optical generations can then be distributed into specific regions of the electrical grid.

In addition, to model the correct optical geometry, the thin-film layers must be drawn into the device structure. However, the raytracer treats the physics of thin film using a TMM contact, and these thin layers should effectively be ignored during the raytracing process. As such, you

need to use `VirtualRegions` (see [Virtual Regions in Raytracer on page 528](#)) to ignore these thin layers in the raytracing process. The required syntax is:

```
RayTraceBC {...
  { Name="TMMcontact"
    ReferenceMaterial = "Gas"
    LayerStructure {...}
    MapOptGenToRegions {"thinlayer1"
                        "thinlayer2" ...}
    QuantumEfficiency = float
  }
}

Physics (RegionInterface="regionname1/
regionname2") {
  RayTraceBC (
    TMM (
      ReferenceMaterial = "Gas"
      LayerStructure {...}
      MapOptGenToRegions {"thinlayer1"
                        "thinlayer2" ...}
      QuantumEfficiency = float
    )
  )
}

Physics {...
  Optics(...
    OpticalSolver(
      RayTracing (...
        VirtualRegions{"toppml" "nitride" "oxide" ...}
      )
    )
  )
}
```

A few comments about the syntax:

- `QuantumEfficiency` denotes the fraction of absorbed photons in the TMM BC that will be converted to optical generation. The keyword `QuantumYield` also can be specified in the unified raytracing interface. Therefore, the overall quantum yield of the TMM BC is a product of `QuantumEfficiency` and `QuantumYield`.
- The region names in the `MapOptGenToRegion` section refer to regions in the electrical device grid. The `MapOptGenToRegions{...}` list and `VirtualRegions{...}` list are independent, and there is no need for a one-to-one correspondence. For example, you can have `VirtualRegions{r1,r2,r3}` and `MapOptGenToRegions{r2,r4}` whereby the entire optical generation from a TMM BC will be mapped onto regions `r2` and `r4` according to their volume ratio. The order of the region list is not relevant.
- For the optical generation mapping, a constant optical generation profile is assumed. The optical generation from a TMM BC is lumped into a constant value and distributed to different `MapOptGenToRegions` regions according to their volume ratios.
- The region names in the `VirtualRegions` section refer to regions in the optical device grid.

Diffuse Surface Boundary Condition

Rough surfaces can be modeled by diffuse surface boundary conditions. The available diffuse BC models are:

- Phong scattering model with distribution:

$$I/I_0 = \cos^{\omega}(\alpha + \gamma) \quad (551)$$

where ω is the order of the Phong model, α is the scattered angle with respect to the scattering surface, and γ is an offset angle for the lobe of the distribution.

- Lambert scattering model with distribution:

$$I/I_0 = \cos(\alpha + \gamma) \quad (552)$$

NOTE The Lambert scattering model is a special case of the Phong model.

- Gaussian scattering model with distribution:

$$I/I_0 = \exp\left(-\frac{(\alpha + \gamma)^2}{2\sigma^2}\right) \quad (553)$$

where σ^2 is the variance of the distribution.

- Random scattering model with uniform distribution.

Within Sentaurus Device, a random number $x \in [0, 1]$ is generated, and a mapping function is used to compute the scattering angle α . The mapping function is computed as follows:

The random number $x \in [0, 1]$ has a uniform distribution such that:

$$\int_0^1 f(x) dx = 1 \quad (554)$$

The scattering angle $\alpha \in [0, \pi/2]$ has a distribution function, $p(\alpha)$, which can be the Phong, Lambert, or Gaussian model.

3D Case

The mapping requirement is:

$$p(\alpha) 2\pi R \sin \alpha d\alpha = f(x) dx \quad (555)$$

and the normalization requirement is:

$$\int_0^{\pi/2} 2\pi R p(\alpha) \sin \alpha d\alpha = 1 \quad (556)$$

2D Case

The mapping requirement is:

$$p(\alpha)Rd\alpha = f(x)dx \quad (557)$$

and the normalization requirement is:

$$\int_0^{\pi/2} p(\alpha)Rd\alpha = 1 \quad (558)$$

For the different scattering models, R must be derived based on the normalization conditions for 2D and 3D, respectively.

To find the inverse mapping of the random variable x to α , the integration of the probability density functions to α and x is performed, respectively:

$$\int_0^{\alpha} 2\pi R p(\alpha) \sin \alpha d\alpha = x \text{ for 3D} \quad (559)$$

and:

$$\int_0^{\alpha} p(\alpha) R d\alpha = x \text{ for 2D} \quad (560)$$

Then, the objective would be to express α as a function of x . As an example, the inverse mapping function for the 3D Phong model is:

$$\alpha = \cos^{-1}[\omega + \sqrt[3]{1-\omega}] \quad (561)$$

The inverse mapping functions of the other scattering models can be derived in a similar manner.

The diffuse surface BC has been implemented as an installed PMI and can be invoked by including the following syntax in the Sentaurus Device command file:

<pre>RayTraceBC { { Name = "rough_contact" PMImodel = pmi_rtDiffuseBC (...) } }</pre>	}	<pre>Physics(RegionInterface="region1/region2") { RayTraceBC (PMImodel = pmi_rtDiffuseBC (...)) }</pre>
---	---	---

where ... stands for the following scattering model parameters:

roughsurfacemodel = 0	# 0=Phong, 1=Lambert, 2=Random, 3=Gaussian
phong_w = 1	# w-parameter of Phong model
gaussian_sigma = 0.1	# sigma parameter of Gaussian model

21: Optical Generation

Raytracing

```
surfacereflectivity = 1.0      # reflectivity of rough surface
surfacetranmittivity = 0.0    # transmittivity of rough surface
set_randomseed = 123         # 0 to 10000, or -1:do not set
debug = 0                    # 1=print debug message, 0=do not print,
                             # 99999=interactive debug
```

Some comments about the parameter settings:

- You can set the specific reflectivity and transmittivity of the diffusive surface. However, if you still want to use the original reflectivity and transmittivity computed from the adjoining regions, set `SurfaceReflectivity=-1` and `SurfaceTransmittivity=-1`.
- Setting a random seed allows for reproducibility of the simulation results.
- A useful debug option is included to ensure that the correct distribution is obtained.

NOTE Instead of specifying the PMI parameters in the command file, you also can define them in the parameter file. To do so, in the command file, in the `RayTraceBC` section, set `PMIModel = "pmi_rtDiffuseBC"`. In the material parameter file, add a section `pmi_rtDiffuseBC { ... }`, where ... corresponds to the scattering model parameters as previously described.

Virtual Regions in Raytracer

This feature is a prelude to the TMM optical generation in the raytracer feature. Virtual regions can be defined in the raytracer such that rays ignore the presence of these regions during the raytracing process. In other words, when rays enter or leave a virtual region, no reflection or refraction occurs, and the ray is transmitted without change. This allows for additional flexibility in dual-grid simulations where some regions are important for electrical transport but insignificant for optics. The syntax is:

```
Physics {...
  Optics(...
    OpticalSolver(
      RayTracing (...
        VirtualRegions{"nitride" "oxide" "layer555" ...}
      )
    )
  )
}
```

Additional Options for Raytracing

Several options enable better control of raytracing in Sentaurus Device:

- Omitting reflected rays when performing raytracing.
- Omitting weaker rays when performing raytracing.
- The number of ray paths can be reduced by including the `Skip(<integer>)` keyword in the `Print` statement. For example, `Print(Skip(5))` will only print every other fifth ray path.
- The status of building the raytree can be checked by using the keyword `ProgressMarkers`, which shows the incremental percentage of the completion of the raytree-building process.

The syntax for these options is:

```
Physics { ...
  Optics(...
    OpticalSolver(
      RayTracing (
        OmitReflectedRays
        OmitWeakerRays
        Print(Skip(<integer>))
        ProgressMarkers = <integer> # between 1 and 100
      )
    )
  )
}
```

Redistributing Power of Stopped Rays

When the raytracing terminates at a designated `DepthLimit` or `MinIntensity` value, there is still leftover power in those terminated rays. The sum of the powers contained in all these stopped rays can be redistributed into the raytree. The total power of the rays is:

$$P_{\text{Total}} = P_{\text{abs}} + P_{\text{escape}} + P_{\text{stopped}} \quad (562)$$

where P_{abs} is the absorbed power, P_{escape} is the power of the escaped rays (out of the device), and P_{stopped} is the power of the rays terminated by the `DepthLimit` or `MinIntensity` condition. Rearranging the equation, the following expression is obtained:

$$P_{\text{Total}} = \frac{1}{\left(1 - \frac{P_{\text{stopped}}}{P_{\text{Total}}}\right)} (P_{\text{abs}} + P_{\text{escape}}) \quad (563)$$

Therefore, by multiplying a redistribution factor to P_{abs} and P_{escape} , the leftover power in the stopped rays can be accounted for. The syntax is:

```
Physics {...  
  Optics(...  
    OpticalSolver(  
      RayTracing (...  
        RedistributeStoppedRays  
      )  
    )  
  )  
}
```

In addition, this feature applies to LED raytracing.

NOTE This feature does not work with Monte Carlo raytracing due to the fundamental assumption of the Monte Carlo method.

Visualizing Raytracing

The `Print` option in the `Raytrace` section creates a `.grd` file with information about the paths that the rays take (see [Using the Raytracer on page 514](#)).

Alternatively, the keyword `RayTrees` can be set in the `Plot` section to visualize raytracing using the TDR format. Thereby, an additional geometry is added to the plot file representing the ray paths. The following datasets are available for each ray element:

- **Depth:** Number of material boundaries that the ray has passed through.
- **Intensity:** Ray intensity.
- **Transmitted (Boolean):** True, as long as the ray is transmitted.

NOTE `RayTrees` cannot be plotted with the compact memory option.

Reporting Various Powers in Raytracing

Various powers are reported in the log file after a raytracing event, and the format is as follows:

Summary of RayTrace Total Photons and Powers:						
	Input	Escaped	StoppedMinInt	StoppedDepth	AbsorbedBulk	AbsorbedBC
Photons [#s]:	4.531E+11	1.333E+11	4.142E+07	0.000E+00	6.236E+10	2.574E+11
Powers [W]:	3.000E-07	8.826E-08	2.743E-11	0.000E+00	4.129E-08	1.704E-07

A brief summary of these powers is:

- Input power is computed by multiplying the input wave power (units of W/cm^2) by the rectangular or circular window area where the starting rays have been defined.
- Escaped power refers to the sum of powers of the rays that are ejected from the device and are unable to re-enter the device.
- StoppedMinInt power sums the powers of the rays terminated by the MinIntensity condition.
- StoppedDepth power sums the powers of the rays terminated by the DepthLimit criteria.
- AbsorbedBulk power refers to power absorbed in bulk regions.
- AbsorbedBC power refers to power absorbed in the TMM contacts. This column is shown only if TMM contacts have been defined.

In the plot file, the raytrace photon rates and powers are listed under RaytracePhoton and RaytracePower, respectively. In addition, the ratio of the various powers and the input power are plotted under RaytraceFraction.

Plotting Interface Flux

The photon flux flowing through any interface can be tracked with the PlotInterfaceFlux feature. The tracking is made possible by recording the reflected, transmitted, and absorbed flux flowing through each interface or contact BC. Since photon flux is directional, there is a need to distinguish between the flux flowing from Region 1 to Region 2, or from Region 2 to Region 1, as shown in Figure 35.

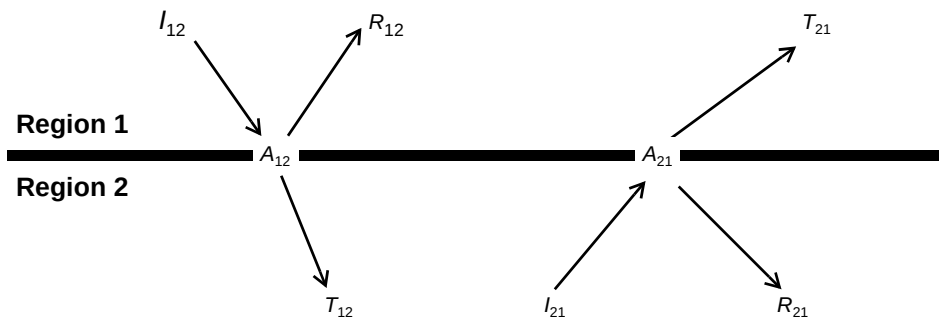


Figure 35 Distinguishing directional photon flux

Consider the photon fluxes (carried by rays) flowing from Region 1 to Region 2. For photon flux conservation:

$$I_{12} = R_{12} + T_{12} + A_{12} \quad (564)$$

21: Optical Generation

Raytracing

where:

- I_{12} is the incident flux.
- R_{12} is the reflected flux.
- T_{12} is the transmitted flux.
- A_{12} is the flux absorbed at the interface.

The reverse is true for flux flowing from Region 2 to Region 1. At each interface, the summation of R_{12} , T_{12} , A_{12} , R_{21} , T_{21} and A_{21} is stored in the raytracing process.

With this information, you can compute various reflection and transmission coefficients at the interface that has been declared as a raytrace BC. For example, assume that light is illuminated from Region 1 to Region 2. Within Region 2, there can be multiple reflections, and some rays may escape back into Region 1 through T_{21} .

Therefore, the power reflection coefficient for the case in [Figure 35 on page 531](#) is:

$$\tilde{R}_{12} = \frac{\sum R_{12} + \sum T_{21}}{\sum R_{12} + \sum T_{12} + \sum A_{12}} \quad (565)$$

The following syntax activates the feature for tracking and plotting interface fluxes:

```
Physics {...
  Optics (...
    OpticalSolver (...
      RayTracing (
        PlotInterfaceFlux
      )
    )
  )
}
```

This feature is limited to the unified raytracing interface. In the plot file, only the fluxes of the interfaces or contacts that have been declared as a raytrace BC are output. In addition, activating the `PlotInterfaceFlux` feature also enables the output of the absorbed photon density in every layer of the TMM BC. The output variables in the plot file are listed as follows:

```
RaytraceInterfaceFlux          # directional
R(region1/region3)
T(region1/region3)
A(region1/region3)
R(region3/region1)
T(region3/region1)
A(region3/region1)
```

```

RaytraceContactFlux                                # directional
  R(contactname1(region1/region3))                 # the region is only a sample
  T(contactname1(region1/region3))
  A(contactname1(region1/region3))
  R(contactname1(region3/region1))
  T(contactname1(region3/region1))
  A(contactname1(region3/region1))

RaytraceInterfaceTMMLayerFlux                      # not directional
  A(region1/region3).layer1
  A(region1/region3).layer2
  A(region1/region3).layer3

RaytraceContactTMMLayerFlux                       # not directional
  A(contactname1(region1/region3)).layer1
  A(contactname1(region1/region3)).layer2
  A(contactname1(region1/region3)).layer3

```

A contact-based BC can possibly intersect many region interfaces. In the plot file output, only a representative region interface is used to indicate the directional flow of photon flux for the entire contact BC.

When the `PlotInterfaceFlux` feature is activated, the `OpticalIntensity` computation will change such that the intensity within each region is scaled to the net photon flux gained in that region. Therefore, the `OpticalIntensity` values can be negative in some regions. Integration of the `OpticalIntensity` within the region recovers the net photon flux flowing into that region.

Far Field and Sensors for Raytracing

To collect information on the rays that exit a device, the concept of far field and sensors is introduced in the unified raytracer interface.

The far field is collected on a virtual circle for two dimensions and a virtual sphere for three dimensions. Ray collection does not destroy the rays. On the virtual far-field surface, the far-field intensity is computed based on a unit measure of radius, whereby the collected ray powers at each interval are divided by the radian angular arc (2D) or area (3D):

$$\text{In 2D, far-field intensity} = \frac{\sum \text{raypowers}}{d\phi}.$$

$$\text{In 3D, far-field intensity} = \frac{\sum \text{raypowers}}{\{\cos((\theta_1)) - \cos(\theta_2)\} d\phi}.$$

21: Optical Generation

Raytracing

On the other hand, a sensor sums the total power of all rays impinging the sensor. Two types of sensor can be defined:

- A line segment sensor in two dimensions or a flat rectangular sensor in three dimensions.
- An angular sensor that collects the power of the rays that radiate within the angular span of the sensor. In two dimensions, a range of ϕ defines the sensor; whereas in three dimensions, ranges of θ and ϕ are needed. These are defined according to the regular Cartesian coordinate system.

The `SensorSweep` option is also available for 3D simulations. It allows you to visualize the variation of ray power collected on a latitude or longitude ring band.

The syntax for activating the far field and sensors in the unified interface for optical generation computation is:

```
Physics {
  Optics (
    OpticalSolver (
      Raytracing(
        Farfield(
          Origin = auto | <vector>      # default is auto
          Discretization = <integer>    # default is 360 for 2D, 36 for 3D
          ObservationRadius = <float>   # default is 1e6 um (1 meter)
          Sensor(
            Name = "sensorname1"
            Rectangle (                  # 3D only
              Corner1 = <vector>
              Corner2 = <vector>
              Corner3 = <vector>
              UseNormalFlux
              AxisAligned                 # 3D only
            )
            Line (                       # 2D only
              Corner1 = <vector>
              Corner2 = <vector>
              UseNormalFlux
            )
            Angular (
              Theta = (<float> <float>)
              Phi = (<float> <float>)
            )
          )
          Sensor(
            Name = "sensorname2"
            Rectangle ( ... )           # 3D only
            Line ( ... )                # 2D only
            Angular ( ... )
          )
        )
      )
    )
  )
}
```

```

        SensorSweep(                                # 3D only
            Name = "sensorname3"
            Ndivisions = <integer>
            VaryPhi
            Theta = (<float> <float>)                # Use with VaryPhi
        )
        SensorSweep(                                # 3D only
            Name = "sensorname4"
            Ndivisions = <integer>
            VaryTheta
            Phi = (<float> <float>)                  # Use with VaryTheta
        )
    )
)
}

```

Some comments about the syntax:

- Multiple far-field Sensor and SensorSweep sections can be defined. However, each Sensor and SensorSweep section must have a unique name for identification.
- When Line or Rectangle sensors are used, the option UseNormalFlux allows you to sum the normal-projected power from the ray that is impinging the sensor at an angle.
- In the case of a Rectangle sensor in three dimensions, if you specify AxisAligned, only opposing corners of the rectangle are required. Sentaurus Device automatically computes the other two corners of the rectangle.
- If an empty Farfield() section is specified, the far field is plotted with the default values of Discretization, Origin, and ObservationRadius.
- The far field takes values of intensity type. Therefore, to recover the total number of photons, you must integrate over the angle (in radian).
- The 3D far field is projected onto a staggered grid (see [Staggered 3D Grid LED Radiation Pattern on page 839](#)).
- The far field is plotted at every instance where the Plot statement in the Solve section is activated. The file names of the far field and sensor sweep are derived from the plot file name. For example, this syntax:

```

File {...
    Plot = "n99_des"
}
Solve {...
    Plot ( Range=(0,1) Intervals=5 )
}

```

21: Optical Generation

Raytracing

produces the following far-field files:

```
n99_000000_des_farfield.tdr
n99_000001_des_farfield.tdr
...
n99_000000_des_sensorsweep.plt
n99_000001_des_sensorsweep.plt
...
```

- In the SensorSweep plot file, all the user-defined SensorSweep sections are defined in the following fields of the plot file:

```
sensorname3
  Phi
  Photons_DelTheta
sensorname4
  Theta
  Photons_DelPhi
```

- The values of the sensor are added to the general plot file of the simulation, and the field entries are:

```
RaytraceSensor
  sensorname1
  sensorname2
```

Dual-Grid Setup for Raytracing

Raytracing in Sentaurus Device is based on tracing the rays in each cell of the simulation mesh. For simulations in which the optical material properties do not vary on a short length scale, this may lead to the unnecessary deterioration of simulation performance. In such cases, a dual-grid setup can be used, which allows the use of a coarse mesh for raytracing, while the electronic equations are solved on a finer mesh (see [Figure 36](#)).

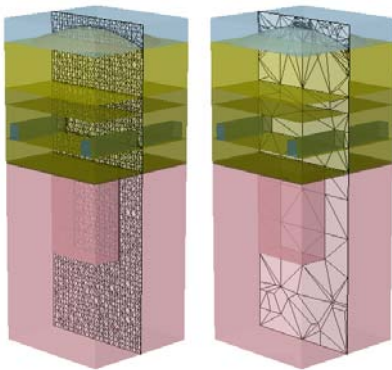


Figure 36 Comparison of two grids used in a 3D dual-grid CMOS image sensor simulation; the figures show cuts through the device center; (*left*) the electronic equations are solved on a fine grid and (*right*) a coarse grid is used for raytracing

The basic syntax for setting up a raytracing dual-grid simulation is:

```
File {
    Output = "output"
    Current = "current"
}
Plot {
    OpticalIntensity
    OpticalGeneration
}

# Specify grid and material parameter file names for raytracing:
OpticalDevice OptGrid {
    File {
        Grid = "raytrace.tdr"
        Doping = "raytrace.tdr"
        Current = "opto"
        Plot = "opto"
        Parameter = "CIS.par"
    }
    Physics { HeteroInterface }
}

# Specify grid, material parameters, and physical models for electrical
# simulation:
Device CIS {
    Electrode {
        { Name="sub" Voltage = 0.0 }
        { Name="pd" Voltage = 0.0 }
    }
    File {
        Grid = "in_elec_pof.tdr"
        Doping = "in_elec_pof.tdr"
        Parameter = "CIS.par"
        Current = "current"
        Plot = "plot"
    }
    Physics {
        AreaFactor = 1
        Optics(...
            ComplexRefractiveIndex(...
                WavelengthDep(real imag)
            )
            OpticalGeneration(...)
            OpticalSolver(
                RayTrace (
                    PolarizationVector=(1,0,0)
                    DepthLimit = 6
                    MinIntensity = 1e-3
                )
            )
        )
    }
}
```

```

        RectangularWindow (...)
    )
)
Excitation(...)
}

# Specify system connectivity:
System {
    OptGrid opt ()
    CIS d1 (pd=vdd sub=0) { Physics{ OptSolver = "opt" } }
    Vsource_pset drive(vdd 0){ dc = 0.0 }
}

Solve { Poisson }

```

The dual-grid simulation setup also allows the unified raytracing interface to be used (see [Raytracing on page 483](#)).

Transfer Matrix Method

Sentaurus Device can calculate the propagation of plane waves through layered media by using a transfer matrix approach.

Physical Model

In the underlying model of the optical carrier generation rate, monochromatic plane waves with arbitrary angles of incidence and polarization states penetrating a number of planar, parallel layers are assumed. Each layer must be homogeneous, isotropic, and optically linear. In this case, the amplitudes of forward and backward running waves $A_j^\pm$ and $B_j^\pm$ in each layer in [Figure 37 on page 539](#) are calculated with help of transfer matrices.

These matrices are functions of the complex wave impedances Z_j given by $Z_j = n_j \cdot \cos\Theta_j$ in the case of E polarization (TE) and by $Z_j = n_j / (\cos\Theta_j)$ in the case of H polarization (TM). Here, n_j denotes the complex index of refraction and Θ_j is the complex counterpart of the angle of refraction ($n_0 \cdot \sin\Theta_0 = n_j \cdot \sin\Theta_j$).

The transfer matrix of the interface between layers j and $j + 1$ is defined by:

$$T_{j,j+1} = \frac{1}{2Z_j} \cdot \begin{bmatrix} Z_j + Z_{j+1} & Z_j - Z_{j+1} \\ Z_j - Z_{j+1} & Z_j + Z_{j+1} \end{bmatrix} \quad (566)$$

The propagation of the plane waves through layer j can be described by the transfer matrix:

$$T_j(d_j) = \begin{bmatrix} \exp\left(2\pi i n_j \cos \Theta_j \frac{d_j}{\lambda}\right) & 0 \\ 0 & \exp\left(-2\pi i n_j \cos \frac{\Theta_j d_j}{\lambda}\right) \end{bmatrix} \quad (567)$$

with the thickness d_j of layer j and the wavelength λ of the incident light.

The transfer matrices connect the amplitudes of [Figure 37](#) as follows:

$$\begin{pmatrix} B_j^+ \\ A_j^+ \end{pmatrix} = T_{j,j+1} \cdot \begin{pmatrix} A_{j+1}^- \\ B_{j+1}^- \end{pmatrix} \quad (568)$$

$$\begin{pmatrix} A_j^- \\ B_j^- \end{pmatrix} = T_j(d_j) \cdot \begin{pmatrix} B_j^+ \\ A_j^+ \end{pmatrix}$$

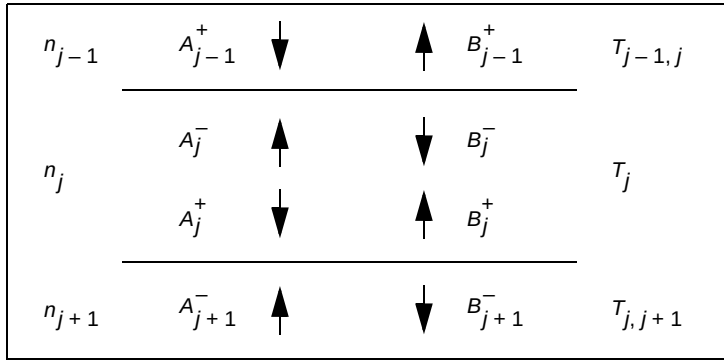


Figure 37 Wave amplitudes in a layered medium and transfer matrices connecting them

It is assumed that there is no backward-running wave behind the layered medium, and the intensity of the incident radiation is known. Therefore, the amplitudes A_j^+ and B_j^+ at each interface can be calculated with appropriate products of transfer matrices.

For both cases of polarization, the intensity in layer j at a distance d from the upper interface ($j, j + 1$) is given by:

$$I_{T(\text{TE}, \text{TM})}(d) = \frac{\Re(Z_j)}{\Re(Z_0)} \cdot \left\| T_j(d) \cdot \begin{pmatrix} A_j^- \\ B_j^- \end{pmatrix} \right\|^2 \quad (569)$$

with the proper wave impedances. If δ is the angle between the vector of the electric field and the plane of incidence, the intensities have to be added according to:

$$I(d) = I_{\text{TM}}(d) + I_{\text{TE}}(d) \quad (570)$$

where $I_{\text{TM}} = (1 - a)I(d)$ and $I_{\text{TE}} = aI(d)$ with $a = \cos^2 \delta$.

One of the layers must be the electrical active silicon layer where the optical charge carrier generation rate G^{opt} is calculated. The rate is proportional to the photon flux $\Phi(d) = I(d)/\hbar\omega$.

In the visible and ultraviolet region, the photon energy $\hbar\omega$ is greater than the band gap of silicon. In this region, the absorption of photons by excitation of electrons from the valence to the conduction band is the dominant absorption process for nondegenerate semiconductors. Far from the absorption threshold, the absorption is considered to be independent of the free carrier densities and doping. Therefore, the silicon layer is considered to be a homogeneous region.

The absorption coefficient α is the relative rate of decrease in light intensity along its path of propagation due to absorption. This decrease must be distinguished from variations caused by the superposition of waves. Therefore, the rate of generated electron-hole pairs is:

$$G_0^{\text{opt}} = \alpha \eta \frac{I(d)}{\hbar\omega} \quad (571)$$

where the absorption coefficient α is given by the imaginary part of $4\pi Z_{\text{Si}}/\lambda$. The quantum yield η is defined as the number of carrier pairs generated by one photon. More details about the available quantum yield models and their specification in the `OpticalGeneration` section of the command file are given in [Quantum Yield Models on page 475](#).

Rough Surface Scattering

To model the effect of light trapping due to scattering of incident light at rough interfaces of a planar multilayer structure, the standard TMM approach describing the propagation of coherent light is extended. Part of the coherent light incident at a rough interface is scattered and spreads incoherently throughout the structure. Scalar scattering theory [4][5] allows to approximate the amount of scattered light at a rough interface. The so-called haze parameter defines the ratio of diffused (scattered) light to total (diffused + specular) light. The directional

dependency of the scattering process is modeled by angular distribution functions (ADFs). In this scenario, a rough interface is characterized by its haze function (haze parameter as a function of the wavelength of incident light) for reflection and transmission as well as the ADF for direct coherent light incident at the interface and the ADF for scattered light incident at the interface.

The implementation of rough surface scattering within the existing TMM framework follows the semi-coherent optical model outlined in [6]. It is based on the following assumptions:

- The haze factor determines the fraction of direct light, transferred to scattered light.
- Coherent and diffused light incident on a rough interface are scattered differently using different ADFs.
- Interfaces are nonabsorptive, meaning photon flux is conserved at interfaces.
- For scattered light incident at a rough interface, the mean value of TE and TM polarized light for reflectance and transmittance is used.
- For the scattered light, the total reflectance and transmittance for a rough interface are assumed to be equal to the ones for the corresponding flat interface.
- The phase change of the electric field traveling across a rough interface is the same as in the case of the corresponding flat interface.
- Scattered light incident at a rough interface is further split, according to its haze factor, into a specular part, which corresponds to light incident at a flat interface, and a diffused part leading to further spreading of the incident scattered light.

Given these assumptions, the specular and diffused components of light at each interface can be expressed in terms of the corresponding haze functions and ADFs. The description of coherent light incident at a rough interface differs from the approach taken in [6] in that a generalized interface matrix has been derived. It accounts for the reduced reflection and transmission according to the respective haze functions and allows to keep the common TMM computation approach for the electric field and intensity. The reduced interface matrix reads as follows:

$$\tilde{T}_{j,j+1} = \frac{1}{h_{j+1,j}^t} \begin{bmatrix} 1 & (h_{j+1,j}^r r_{j,j+1}) \\ (h_{j,j+1}^r r_{j,j+1}) & (h_{j,j+1}^t h_{j+1,j}^t + (h_{j+1,j}^r h_{j,j+1}^r - h_{j,j+1}^t h_{j+1,j}^t) r_{j,j+1}^2) \end{bmatrix} \quad (572)$$

where $r_{j,j+1}$ and $t_{j,j+1}$ denote the Fresnel reflection and transmission coefficients for normal incidence given by:

$$r_{j,j+1} = \frac{n_j - n_{j+1}}{n_j + n_{j+1}} \quad (573)$$

$$t_{j,j+1} = \frac{2n_j}{n_j + n_{j+1}} \quad (574)$$

and $h_{j,j+1}^r$ and $h_{j,j+1}^t$ are reduction factors determined by the haze functions for reflection and transmission:

$$h_{j,j+1}^r = \sqrt{1 - H_{j,j+1}^r} \quad (575)$$

$$h_{j,j+1}^t = \sqrt{1 - H_{j,j+1}^t} \quad (576)$$

The haze functions for reflection and transmission derived from the scalar scattering theory are:

$$H_{j,j+1}^r(\lambda, \varphi_j) = 1 - \exp\left[-\left(\frac{4\pi\sigma_{\text{rms}}c_r(\lambda, \sigma_{\text{rms}})n_j\cos\varphi_j}{\lambda}\right)^{a^r}\right] \quad (577)$$

$$H_{j,j+1}^t(\lambda, \varphi_j) = 1 - \exp\left[-\left(\frac{4\pi\sigma_{\text{rms}}c_t(\lambda, \sigma_{\text{rms}})|n_j\cos\varphi_j - n_{j+1}\cos\varphi_{j+1}|}{\lambda}\right)^{a^t}\right] \quad (578)$$

where σ_{rms} stands for the root-mean-square roughness of the interface and λ is the wavelength of the incident light. The exponent $a^{r/t}$ and the correction function $c_{r/t}$ are fitting parameters to better match experimental data.

NOTE The haze functions defined in [Eq. 577](#) and [Eq. 578](#) are given in their general form, which also covers light incident at the angle φ_j from the surface normal. This expression is needed in the description of scattered light incident at a rough interface.

For the propagation of the angular intensity components of scattered light through each layer, the effective path length is used to stay within the one-dimensional formalism:

$$I(\lambda, \varphi_j, d) = I(\lambda, \varphi_j) \exp\left(-\frac{\alpha(\lambda)d}{\cos\varphi_j}\right) \quad (579)$$

The scattering solver first propagates all forward-going components through the layer structure and completes a round trip by propagating all backward-going components. In each round trip, part of the light may be absorbed within the structure as well as transmitted through the front or back side, depending on the material properties at the given wavelength. This leads to a reduction of the light intensity components at the various interfaces with increasing number of iterations. When the ratio of the largest remaining intensity component in the system to the sum of all initial intensity components falls below the value of `Tolerance`, the solver terminates. As a consequence, the residual intensity of each component at all interfaces is not included in the extracted results.

The overall simulation flow consists of the following steps:

- Solving the coherent propagation problem following the default TMM solver algorithm, but using the generalized interface matrices for rough interfaces. In this step, the starting light intensity for scattered light is calculated at each interface as well.
- The scattered light is propagated through the structure following an iterative approach similar to raytracing as outlined in [6]. However, other than raytracing, instead of single rays, ray bundles with discrete angular distribution are propagated from layer to layer. The number of iterations needed to solve the scattering problem to the required accuracy mainly depends on how much light is absorbed within the structure and how much light is coupled out at the front and back ends during a single round trip.
- The scattered and coherent absorbed photon density profiles are summed and passed to the quantum yield model.

NOTE Support for rough surface scattering is limited to normal incident light.

For more information on how to apply the rough surface scattering model and its configuration options, see [Using Scattering Solver on page 545](#).

Using Transfer Matrix Method

The transfer matrix method is only supported by the unified interface for optical generation computation (see [Overview on page 469](#)). The `OpticalGeneration` section activates the computation of the optical generation along with optional parameters such as the quantum yield. The `OpticalSolver` section determines the underlying optical solver together with solver-specific parameters as shown in the following example:

```
Physics {...
  Optics (...
    OpticalGeneration (...)
    ComplexRefractiveIndex (...)
    Excitation (...
      Window ("L1") (...)
    )
    OpticalSolver (
      TMM (
        PropagationDirection = Perpendicular # Default Refractive
        NodesPerWavelength = 20
        LayerStackExtraction (...
          WindowName = "L1"
          Medium (...)
        )
      )
    )
  )
}
```

21: Optical Generation

Transfer Matrix Method

```
        Excitation (...
            Window ("L1") (...)
        )
    )
}
```

The properties of the incident light such as Wavelength, Intensity, Polarization, and angle of incidence Theta are specified in the Excitation section (see [Setting the Excitation Parameters on page 486](#)) along with one or several illumination windows (see [Illumination Window on page 487](#)), which confine the incident light to a certain part of the device structure.

An illumination window determines how the 1D solution of the transfer matrix method is interpolated to the higher dimensional device grid. The minimum coordinate of the 1D profile is pinned to the illumination window, and the interpolation of the profile in propagation direction can be performed either perpendicular to it or according to Snell's law, which is the default. The corresponding keyword in the TMM section is PropagationDirection, which can take either Perpendicular or Refractive as its argument.

The LayerStackExtraction section determines how a 1D complex refractive index profile is extracted automatically from the grid file (see [Extracting the Layer Stack on page 491](#)). Based on this profile, the polarization-dependent optical intensity is calculated and interpolated onto the grid according to the referenced illumination window and the value of PropagationDirection. The optical generation profile then is calculated from the window-specific intensities. For overlapping windows, the intensity are summed in the region of intersection.

By default, the surrounding media at the top and bottom of the extracted layer stack are assumed to have the material properties of vacuum. However, the default can be overwritten by specifying a separate Medium section in the LayerStackExtraction section for the top and bottom of the layer stack if necessary. In the Medium section, a Location must be specified and a Material, or, alternatively, the values of RefractiveIndex and ExtinctionCoefficient:

```
LayerStackExtraction(
    Medium (
        Location = top
        Material = "Silicon"
    )
    Medium (
        Location = bottom
        RefractiveIndex = 1.4
        ExtinctionCoefficient = 1e-3
    )
)
```

If the optical generation profile is the sole quantity of interest, it is sufficient to specify the keyword `Optics` exclusively in the `Solve` section.

The TMM solver offers two options for computing the optical intensity from the complex field amplitudes of the forward-propagating and backward-propagating waves. Specifying `IntensityPattern=StandingWave` computes the optical intensity I as follows:

$$I = \text{Re}(A + B)^2 + \text{Im}(A + B)^2 \quad (580)$$

whereas using the syntax:

```
TMM ( ...
    IntensityPattern = Envelope
)
```

computes the optical intensity I using the following formula to prevent oscillations on the wavelength scale that may not be possible to resolve on the mixed-element simulation grid:

$$I = \text{Re}(A)^2 + \text{Im}(A)^2 + \text{Re}(B)^2 + \text{Im}(B)^2 \quad (581)$$

where A and B are the complex field amplitudes of the forward-propagating and backward-propagating waves, respectively. `IntensityPattern` can be specified globally, per region, or per material.

The keywords of TMM are summarized in [Table 208 on page 1280](#).

Using Scattering Solver

To model the effect of scattering at rough interfaces, the scattering solver (see [Rough Surface Scattering on page 540](#)) must be configured in the command file, and the parameters characterizing each rough interface must be specified in the corresponding parameter file section.

Command File Specification

The scattering solver is activated by specifying a `Scattering` section within the TMM section. By default, all interfaces are treated as rough. However, whether part of the light incident on a specific interface is actually scattered depends on the corresponding haze functions for reflection and transmission defined in the parameter file. A haze function evaluating to zero is equivalent to the interface being treated as flat.

21: Optical Generation

Transfer Matrix Method

You can flag explicitly a specific region or material interface as not being rough as in the following example:

```
Physics {
  Optics (
    OpticalSolver (
      TMM (
        Scattering ( ... )
        RoughInterface    #Default
      )
    )
  )
}

Physics (RegionInterface="<region1>/<region2>") {
  Optics (
    OpticalSolver (
      TMM (
        -RoughInterface
      )
    )
  )
}
```

Sometimes, it may be more practical to set all interfaces as flat by specifying `-RoughInterface` in the global `Physics` section and only to label specific interfaces as rough in a corresponding region or material interface `Physics` section.

The angular discretization of the interval $[-\pi/2, \pi/2]$ used for the ray bundles in the scattering solver can be set with the keyword `AngularDiscretization` in the `Scattering` section. The keywords `Tolerance` and `MaxNumberOfIterations` determine the termination condition of the iterative scattering solver (see [Rough Surface Scattering on page 540](#)):

```
Physics {
  Optics (
    OpticalSolver (
      TMM (
        Scattering (
          AngularDiscretization = 90
          MaxNumberOfIterations = 150
          Tolerance = 1e-3
        )
      )
    )
  )
}
```


For weakly absorbing structures, such as semiconductor layer stacks illuminated with light in the infrared part of the spectrum, many iterations may be necessary until the given `Tolerance` is reached. To detect such cases and to limit the total simulation time, it is advisable to adjust the value of `MaxNumberOfIterations`.

Parameter File Specification

All parameters characterizing a rough interface are defined in the `OpticalSurfaceRoughness` section of the respective region or material interface section in the parameter file. The parameters can be classified into two groups: One relates to the specification of the haze functions and one contains the selection of the various angular distribution functions (ADFs).

For specifying the haze function for reflection (`HazeFunction_R`) and transmission (`HazeFunction_T`), two options are available:

- **Constant:** For each of the two haze functions H_R and H_T , a constant value can be set using the keywords `H_R` and `H_T`. This option may be useful if a simulation is performed at a single wavelength and the surface roughness is not varied as the haze functions usually depend on both.
- **Analytic:** The analytic expressions given by [Eq. 577, p. 542](#) and [Eq. 578, p. 542](#) are evaluated, where the surface roughness σ_{rms} , the exponents a_R and a_T , and the correction functions c_R and c_T are supplied by the user. In the simplest approximation, c_R and c_T are assumed to be constants over the whole wavelength range. However, in general, the correction functions are used to fit the analytic haze functions against experimental data. For that purpose, tabulated values are supported by setting `CorrectionFunction_R` or `CorrectionFunction_T` equal to `Table`.

Besides the haze functions, the ADFs are used to characterize the behavior of light incident at rough interfaces. The model supports the specification of different ADFs for direct incident light at a rough interface as well as for scattered light incident at a rough interface. Some ADFs depend on an additional parameter apart from the angle of incidence, whose value must be supplied by the user.

The detailed syntax including all options for the definition of the haze functions and ADFs are given in the following tables (where applicable, the default value is given at the beginning of the description).

Table 97 Specification of haze functions

Symbol	Keyword	Value	Description
$H^r(\lambda, \varphi)$	<code>HazeFunction_R</code>	=Constant Analytic	Constant Type of haze function for reflection.
$H^t(\lambda, \varphi)$	<code>HazeFunction_T</code>	=Constant Analytic	Constant Type of haze function for transmission.

Table 97 Specification of haze functions

Symbol	Keyword	Value	Description
H_R	H_R	=<float>	0 Constant value for haze function for reflection.
H_T	H_T	=<float>	0 Constant value for haze function for transmission.
σ_{rms}	Sigma	=<float>	0 Optical surface roughness in [nm] used in analytic haze functions for reflection and transmission.
a_R	a_R	=<float>	2 Exponent of analytic haze function for reflection.
a_T	a_T	=<float>	3 Exponent of analytic haze function for transmission.
	CorrectionFunction_R	=Constant Table	Constant Type of correction function of analytic haze function for reflection.
	CorrectionFunction_T	=Constant Table	Constant Type of correction function of analytic haze function for transmission.
c_R	c_R	=<float>	1 Constant value in interval [0 1] for correction function of analytic haze function for reflection.
c_T	c_T	=<float>	1 Constant value in interval [0 1] for correction function of analytic haze function for transmission.
$c_R(\lambda)$	c_R	(<table>)	Tabulated values for correction function of analytic haze function for reflection. First column contains wavelength in [μm] and second column contains value of correction function in interval [0 1].
$c_T(\lambda)$	c_T	(<table>)	Tabulated values for correction function of analytic haze function for transmission. First column contains wavelength in [μm] and second column contains value of correction function in interval [0 1].
	TableInterpolation_c_R	=Linear Spline	Linear Type of interpolation used for tabulated values of correction function for analytic haze function for reflection.
	TableInterpolation_c_T	=Linear Spline	Linear Type of interpolation used for tabulated values of correction function for analytic haze function for transmission.

Table 98 Specification of angular distribution functions (not normalized)

Symbol	Keyword	Value	Description
$f_1(\varphi)$	ADFDirect_R	=Constant Triangle Gauss Lorentz Cosine Ellipse	Cosine Type of angular distribution function for direct incident light reflected at a rough interface.
C	CDirect_R	=<float>	2 Fitting parameter used in angular distribution function (where applicable) for direct incident light reflected at a rough interface.
$f_1(\varphi)$	ADFDirect_T	=Constant Triangle Gauss Lorentz Cosine Ellipse	Cosine Type of angular distribution function for direct incident light transmitted through a rough interface.
C	CDirect_T	=<float>	2 Fitting parameter used in angular distribution function (where applicable) for direct incident light transmitted through a rough interface.
$f_2(\varphi)$	ADFScatter_R	=Constant Triangle Gauss Lorentz Cosine Ellipse	Constant Type of angular distribution function for scattered light reflected at a rough interface.
C	CScatter_R	=<float>	1 Fitting parameter used in angular distribution function (where applicable) for scattered light reflected at a rough interface.
$f_2(\varphi)$	ADFScatter_T	=Constant Triangle Gauss Lorentz Cosine Ellipse	Constant Type of angular distribution function for scattered light transmitted through a rough interface.
C	CScatter_T	=<float>	1 Fitting parameter used in angular distribution function (where applicable) for scattered light transmitted through a rough interface.

Table 99 Definition of different ADF types used in [Table 98](#)

Type	Formula
Constant	$f(\varphi) = 1$
Triangle	$f(\varphi) = 1 - 2 \varphi /\pi$
Gauss	$f(\varphi) = e^{-\varphi^2/(2C)}$
Lorentz	$f(\varphi) = e^{1/(\varphi^2 + C^2)}$
Cosine	$f(\varphi) = \cos^C(\varphi)$

21: Optical Generation

Transfer Matrix Method

Table 99 Definition of different ADF types used in Table 98

Type	Formula
Ellipse	$f(\varphi) = \frac{\cos(\varphi)}{\cos^2(\varphi) + (2C)^{-2} \sin^2(\varphi)}$

The following figures illustrate the various ADFs specified in Table 98 on page 549.

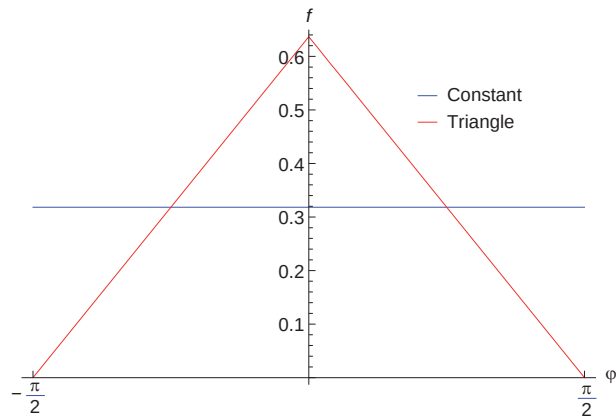


Figure 38 ADF of type Constant and Triangle

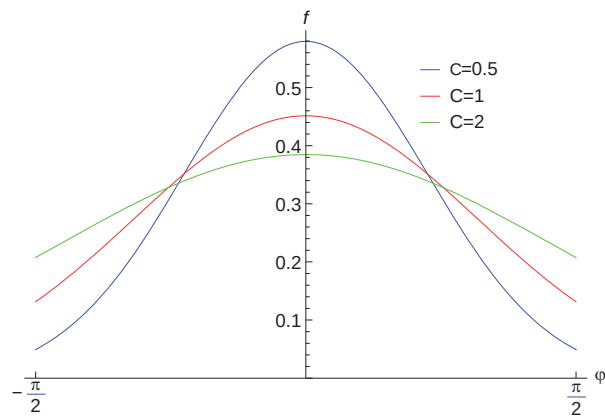


Figure 39 ADF of type Gauss

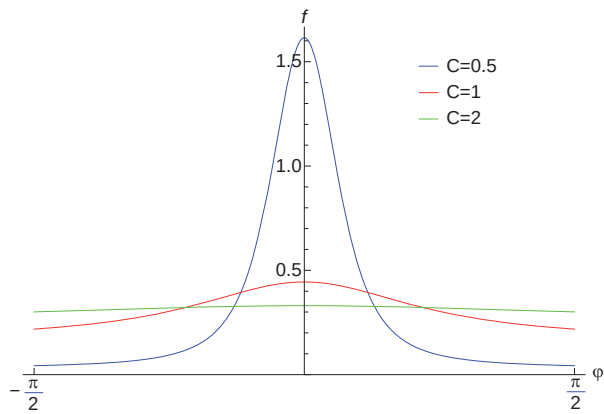


Figure 40 ADF of type Lorentz

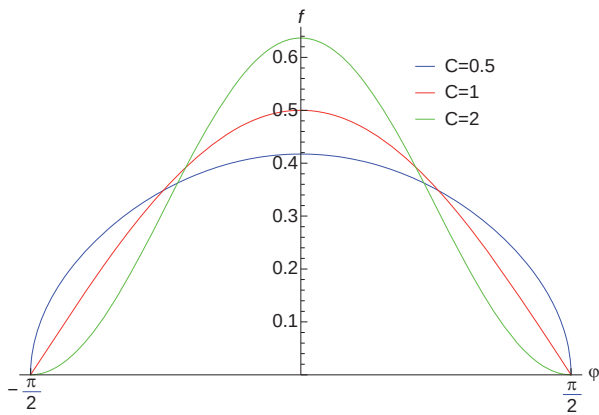


Figure 41 ADF of type Cosine

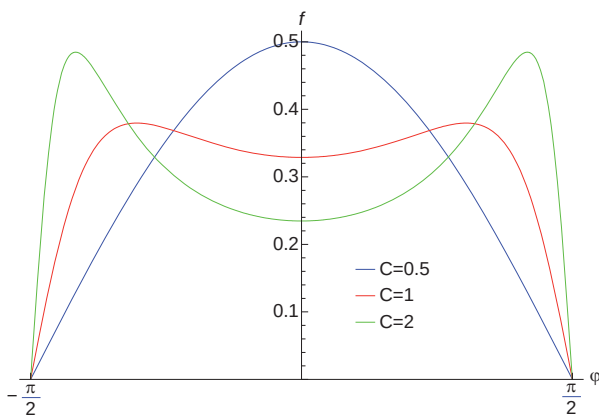


Figure 42 ADF of type Ellipse

21: Optical Generation

Transfer Matrix Method

The following `OpticalSurfaceRoughness` section is an example for the specification of analytic haze functions and the default angular distribution functions:

```
OpticalSurfaceRoughness {
    HazeFunction_R=Analytic
    HazeFunction_T=Analytic
    Sigma=20    #[nm]
    a_R=2
    a_T=3
    CorrectionFunction_R=Table
    TableInterpolation_c_R=Spline
    c_R(
        0.3 0.4
        0.4 0.65
        0.8 0.9
        1.3 0.95
    )
    CorrectionFunction_T=Constant
    c_T=0.7

    ADFDirect_R=Cosine
    CDirect_R=2
    ADFDirect_T=Cosine
    CDirect_T=2
    ADFScatter_R=Constant
    ADFScatter_T=Constant
}
```

Plot Quantities

When specifying `OpticalIntensity` and `AbsorbedPhotonDensity` in the `Plot` section of the command file, the following additional datasets are written to the plot file:

- `OpticalIntensitySpecular`: Specular part of optical intensity resulting from propagation of coherent light.
- `OpticalIntensityDiffuse`: Diffuse part of optical intensity resulting from propagation of incoherent light.
- `AbsorbedPhotonDensitySpecular`: Specular part of absorbed photon density resulting from propagation of coherent light.
- `AbsorbedPhotonDensityDiffuse`: Diffuse part of absorbed photon density resulting from propagation of incoherent light.

Loading Solution of Optical Problem from File

In solar-cell and CIS simulations, it is often necessary to load several optical generation profiles as a function of one or more parameters, for example, excitation wavelength or angle of incidence, in Sentaurus Device. These profiles then are used to compute the response to each of them separately or to compute the response to the spectrum as a whole. A typical example is the computation of white-light generation using several generation profiles previously computed by Sentaurus Device Electromagnetic Wave Solver (EMW) or different wavelengths of the visible spectrum.

The parameters corresponding to a specific absorbed photon density or optical generation profile are written to the respective output file as special TDR tags as well. This ensures that a profile can be identified later when loaded into a Sentaurus Device simulation. For example, if the optical problem is solved externally, it is not possible to determine the quantum yield, which is necessary to compute the optical generation based on the electronic properties of the underlying device structure. However, since the corresponding excitation wavelength is stored along with the optical solution, the quantum yield computation can be performed later in Sentaurus Device after loading the profile to compute the optical generation.

The following requirements exist for loading absorbed photon density or optical generation profiles from file using the optical solver `FromFile`:

- Profiles must be saved in a TDR file.
- Only TDR files created with Version E-2010.12 or later of Sentaurus Device and EMW are supported.
- One-dimensional profiles that are to be imported into higher-dimensional grids must comply with the PLX file format described in [Importing 1D Profiles into Higher-dimensional Grids on page 555](#).

If the grid on which the profile is saved is not the same as that used for the device simulation (different mixed-element grid or arbitrary tensor grid arising from EMW simulation), the profile is interpolated automatically onto the simulation grid upon loading. By default, vertex-based linear interpolation is used. As an alternative for mixed-element to mixed-element interpolation, conservative interpolation can be selected using `GridInterpolation=Conservative`. The interpolation algorithm always interpolates the values even if the material of the source and destination grid are different at a certain vertex. However, optical generation is always set to zero in non-semiconductor regions irrespective of the source profile.

The basic command file syntax for loading profiles from file is:

```
File {  
    OpticalSolverInput = "<filename or filename pattern of profiles>"  
}
```

21: Optical Generation

Loading Solution of Optical Problem from File

```
Physics {
  Optics (
    OpticalGeneration (
      ComputeFromMonochromaticSource ()          # or
      ComputeFromSpectrum ()
    )
    OpticalSolver (
      FromFile (
        DatasetName = AbsorbedPhotonDensity      # Default
        SpectralInterpolation = Linear           # Default Off
        IdentifyingParameter = ("Wavelength" "Theta")
        GridInterpolation = Conservative        # Default Linear
      )
    )
  )
}
```

NOTE The optical solver `FromFile` requires the specification of `ComputeFromMonochromaticSource` or `ComputeFromSpectrum` in the `OpticalGeneration` section. Otherwise, the computation of the optical generation based on the loaded profile is not activated.

The keyword `OpticalSolverInput` takes as its argument either the name of a single TDR or PLX file, or a UNIX-style file name pattern to select several such files whose names match the specified pattern. In both cases, a single file may contain more than one absorbed photon density or optical generation profile. In TDR files, the different profiles are saved in separate TDR states; whereas, in PLX files, they are simply listed sequentially, including their respective headers (see [Importing 1D Profiles into Higher-dimensional Grids on page 555](#)).

Each profile is characterized by its set of parameters, whose values must be unique among the profiles selected in the `File` section. As profiles may be characterized by different numbers of parameters, you must specify the ones to be considered for checking uniqueness by using the keyword `IdentifyingParameter`. Supported arguments are a simple parameter name such as `Wavelength` or, if ambiguous, a full parameter path such as `Optics/Excitation/Wavelength`. It is also possible to introduce new parameters, that is, parameters that do not exist in Sentauros Device, by assigning them to a profile and referring to them in the list of `IdentifyingParameter`. Added parameters will appear under the path `Optics/OpticalSolver/FromFile/UserDefinedParameters/<Name of added parameter>`. User-defined parameters also are supported in the `IlluminationSpectrum` file in the `File` section (see [Illumination Spectrum on page 472](#)).

NOTE Only profiles that carry intensity as a parameter tag are supported for loading from file. This is necessary so that the intensity can be scaled accordingly upon loading. Other profiles will be ignored with a warning message.

When using the feature `ComputeFromSpectrum` (see [Illumination Spectrum on page 472](#)) or when ramping parameters, it is possible that, for a requested set of parameter values, no profile can be found that matches it. By default, the program will exit with an appropriate error message; however, you have two further options to deal with such situations. Using the keyword `SpectralInterpolation`, either `PiecewiseConstant` or `Linear` interpolation can be selected. Interpolation is based on the leading identifying parameter as opposed to complex multidimensional interpolation.

As mentioned earlier, two profile quantities are supported, absorbed photon density and optical generation, which can be selected using the keyword `DatasetName`. The choice determines whether a quantum yield model (see [Quantum Yield Models on page 475](#)) will be applied. If optical generation is specified, quantum yield is assumed to be 1. Otherwise, the corresponding quantum yield model specified in the `OpticalGeneration` section is used to compute the optical generation profile from the loaded absorbed photon density.

Importing 1D Profiles into Higher-dimensional Grids

Importing 1D profiles into higher-dimensional grids is supported for files in PLX format. The following example documents the syntax of PLX files for two 1D profiles characterized by three parameters in a single file:

```
# some comments
"<optional string for tagging a profile>"
Wavelength = 0.5 [um] Theta = 0 [deg] Intensity = 0.1 [W*cm^-2]
0.0 1e19
0.8 1e20
1.4 1e17
2.0 5e18

# some comments
"<optional string for tagging a profile>"
Wavelength = 0.6 [um] Theta = 30[deg] Intensity = 0.1 [W*cm^-2]
0.0 1e19
0.8 1e20
1.4 1e17
2.0 5e18

...
```

Each profile can be preceded by an optional comment starting with the hash sign (#), an optional tag given in double quotation marks, and a list of parameters with their corresponding value and unit as shown in the example above. If a tag is supplied, it will be used as a curve name when visualizing the file in *Inspect*, which also supports PLX files containing several profiles.

21: Optical Generation

Loading Solution of Optical Problem from File

After the 1D profiles have been loaded into Sentaurus Device, they must be interpolated onto the 2D or 3D grid. Similar to the TMM solver (see [Using Transfer Matrix Method on page 543](#)), the concept of an illumination window is applied for this purpose and it requires the specification of at least one Window section in the Excitation section. See [Illumination Window on page 487](#) for further details.

NOTE For the interpolation of the 1D profile onto the 2D or 3D grid, the first data point of a profile is always pinned to the illumination window plane, independent of its actual coordinate value.

Ramping Profile Index

The optical solver `FromFile` supports a keyword `ProfileIndex`, which can be used to address a certain profile directly by its index instead of its identifying parameters. The index is derived from all valid profiles sorted according to the value of the leading identifying parameter specified in the command file. This feature can be used to ramp through all available profiles in a `Quasistationary` statement without having to know the exact parameter values for each profile. The spectral interpolation options described in [Loading Solution of Optical Problem from File on page 553](#) above also are supported for ramping the `ProfileIndex`. If an index value is requested in a `Quasistationary` statement that exceeds the number of valid profiles, the `Quasistationary` is finished and control is given to the following statement in the `Solve` section.

The following example shows the ramping of the `ProfileIndex`:

```
File {
    OpticalSolverInput = "absorbed_photon_density_input_*file.tdr"
}

Physics {
    OpticalGeneration (
        ComputeFromMonochromaticSource ()
    )
    OpticalSolver (
        FromFile (
            ProfileIndex = 0
            DatasetName = AbsorbedPhotonDensity
            SpectralInterpolation = Off
            IdentifyingParameter = ("Optics/Excitation/Wavelength" "Theta" "Phi")
        )
    )
}

Solve {
    Quasistationary (
        InitialStep=0.01 MaxStep=0.01 MinStep=0.01
    )
}
```

```

Plot { Range = ( 0., 1 ) Intervals = 5 }
Goal { ModelParameter="ProfileIndex" value=100 }
) { Optics }
}

```

Note that the counting of profiles starts with index zero. For more information about ramping parameters, see [Parameter Ramping on page 494](#).

Optical Beam Absorption Method

The optical beam absorption method in Sentaurus Device computes optical generation by simple photon absorption using Beer's law. Thereby, multiple optical beams can be defined to represent the incident light.

Physical Model

Figure 43 illustrates one optical beam and its optical intensity distribution in a 3D device.

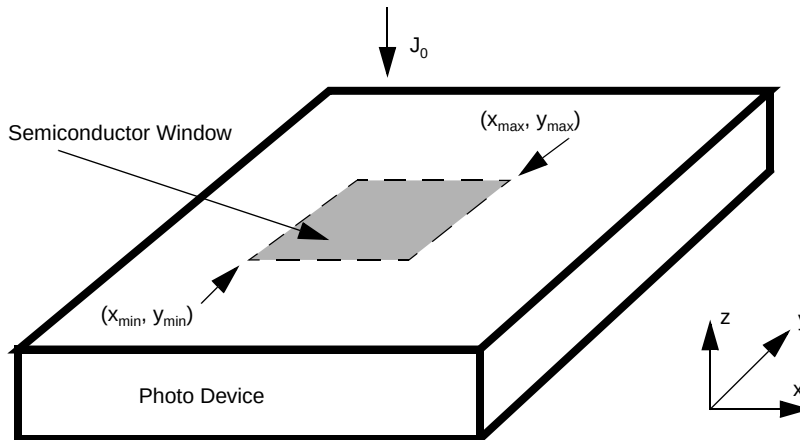


Figure 43 Intensity distribution of an optical beam modeled by a rectangular illumination window

J_0 denotes the optical beam intensity (number of photons that cross an area of 1 cm^2 per 1 s) incident on the semiconductor device. $(x_{\min}, y_{\min})$ and $(x_{\max}, y_{\max})$ define the rectangular illumination window. The space shape F_{xy} of the incident beam intensity is defined by the illumination window, where F_{xy} is equal to 1 inside of it and zero otherwise.

21: Optical Generation

Optical Beam Absorption Method

The following equations describe useful relations for the photogeneration problem:

$$\begin{aligned} E_{\text{ph}} &= \frac{hc}{\lambda} \\ J_0 &= \frac{P_0}{E_{\text{ph}}} \end{aligned} \quad (582)$$

where:

- P_0 is the incident wave power per area [W/cm^2].
- λ is the wavelength [cm].
- h is Planck's constant [J s].
- c is the speed of light in a vacuum [cm/s].
- E_{ph} is the photon energy that is approximately equal to $\frac{1.24}{\lambda[\mu\text{m}]}$ in eV.

The optical beam absorption model computes the optical generation rate along the z-axis taking into account that the absorption coefficient varies along the propagation direction of the beam according to:

$$G^{\text{opt}}(z, t) = J_0 F_t(t) F_{xy} \cdot \alpha(\lambda, z) \cdot \exp\left(-\left|\int_{z_0}^z \alpha(\lambda, z') dz'\right|\right) \quad (583)$$

where:

- t is the time.
- $F_t(t)$ is the beam time behavior function. For a Gaussian pulse, it is equal to 1 for t in $[t_{\min}, t_{\max}]$ and shows a Gaussian distribution decay outside the interval with the standard deviation σ_t .
- z_0 is the coordinate of the semiconductor surface.
- $\alpha(\lambda, z)$ is the nonuniform absorption coefficient along the z-axis.

As described in the following section, the optical beam absorption method supports a wide range of window shapes (see [Illumination Window on page 487](#)) as well as arbitrary beam time behavior functions (see [Specifying Time Dependency for Transient Simulations on page 478](#)). It is also not limited to beams propagating along the z-axis. The example above has only been used for demonstration purposes.

Using Optical Beam Absorption Method

The optical beam absorption method is activated by the optical solver OptBeam. The profile of the absorption coefficient used in [Eq. 583](#) is determined by the LayerStackExtraction section inside the OptBeam section; whereas, the shape of the beam is specified by the illumination window. The wavelength and intensity of the incident light are defined in the

Excitation section. The required syntax of the optical beam absorption method is summarized here:

```
Physics {  
    ...  
    Optics (  
        ...  
        Excitation (  
            Wavelength = 0.5  
            Intensity = 0.1  
            Window("L1") (  
                <window options>  
            )  
        )  
        OpticalSolver (  
            OptBeam (  
                LayerStackExtraction (  
                    WindowName = "L1"  
                    <layer stack extraction options>  
                )  
            )  
        )  
    )  
}
```

Further details about the Window and LayerStackExtraction sections can be found in [Illumination Window on page 487](#) and [Extracting the Layer Stack on page 491](#), respectively.

NOTE You can simulate several beams at the same time by specifying the corresponding number of Window and LayerStackExtraction sections. However, all beams are assigned the same wavelength and intensity, which is specified in the common Excitation section.

Beam Propagation Method

In Sentaurus Device, the beam propagation method (BPM) can be applied to find the light propagation and penetration into devices such as photodetectors. Despite being an approximate method, its efficiency and relative accuracy make it attractive for bridging the gap between the raytracer and the FDTD solver (EMW) featured in Sentaurus Device. The BPM solver is available for both 2D and 3D device geometries, where its computational efficiency compared with a full-wave approach becomes particularly apparent in three dimensions.

NOTE The use of BPM for 3D CMOS image sensors is discouraged because the BPM cannot resolve the polarization transformation caused by the 3D lens.

Physical Model

The BPM implemented in Sentaurus Device is based on the fast Fourier transform (FFT) and is a variant of the FFT BPM, which was developed by Feit and Fleck [7].

The solution of the scalar Helmholtz equation:

$$\left[\nabla_t^2 + \frac{\partial^2}{\partial z^2} + k_0^2 n^2(x, y, z) \right] \phi(x, y, z) = 0 \quad (584)$$

at $z + \Delta z$ with $\nabla_t^2 = \partial^2/\partial x^2 + \partial^2/\partial y^2$ and $n(x, y, z)$ being the complex refractive index can be written as:

$$\frac{\partial}{\partial z} \phi(x, y, z + \Delta z) = \mp i \sqrt{k_0^2 n^2 + \nabla_t^2} \phi(x, y, z) = \pm i \wp \phi(x, y, z) \quad (585)$$

In the paraxial approximation, the operator $\wp$ reduces to:

$$\wp \equiv \sqrt{k_0^2 n_0^2 + \nabla_t^2} + k_0 \delta n \quad (586)$$

where n_0 is taken as a constant reference refractive index in every transverse plane. By expressing the field ϕ at z as a spatial Fourier decomposition of plane waves, the solution to Eq. 584 for forward-propagating waves reads:

$$\phi(x, y, z + \Delta z) = \frac{1}{(2\pi)^2} \exp(ik_0 \delta n \Delta z) \int_{-\infty}^{\infty} d\vec{k}_t \exp(i\vec{k}_t \cdot \vec{r}) \exp(i\sqrt{k_0^2 n_0^2 - \vec{k}_t^2} \Delta z) \tilde{\phi}(\vec{k}_t, z) \quad (587)$$

where $\tilde{\phi}$ denotes the transverse spatial Fourier transform. As can be seen from Eq. 587, each Fourier component experiences a phase shift, which represents the propagation in a medium characterized by the reference refractive index n_0 . The phase-shifted Fourier wave is then inverse-transformed and given an additional phase shift to account for the refractive index inhomogeneity at each $(x, y, z + \Delta z)$ position. In the numeric implementation of Eq. 587, an FFT algorithm is used to compute the forward and inverse Fourier transform.

Bidirectional BPM

The bidirectional algorithm is based on two operators as described by Kaczmariski and Lagasse [8]. The first operator defines a unidirectional propagation, which is outlined in the previous section. The second operator accounts for the reflections at interfaces of the refractive index along the propagation direction. In a first pass, the reflections at all interfaces are computed. The following pass in the opposite direction adds these contributions to the propagating field and calculates the reflections of the forward-propagating field. By iterating this procedure until convergence is reached, a self-consistent algorithm is established.

Boundary Conditions

Due to the finite size of the computational domain, appropriate boundary conditions must be chosen, which minimize any numeric errors in the propagation of the optical field related to boundary effects. In the standard FFT BPM, any waves propagating through a boundary will reappear as a new perturbation at the opposite side of the computation window, effectively representing periodic boundary conditions. In situations where the optical field vanishes at the domain boundaries, this effect can be neglected. In other cases, an absorbing boundary condition is needed.

Perfectly matched layers (PMLs) boundary conditions have the advantage of absorbing the optical field when it reaches the domain boundaries. The BPM implemented in Sentaurus Device supports the PML boundary condition, which has been developed for continuum spectra.

The formulation is based on the introduction of a stretching operator:

$$\nabla_s \equiv \frac{1}{S_x} \hat{x} \frac{\partial}{\partial x} + \frac{1}{S_y} \hat{y} \frac{\partial}{\partial y} + \frac{1}{S_z} \hat{z} \frac{\partial}{\partial z} \quad (588)$$

from which an equivalent set of the Maxwell equations can be derived. In [Eq. 588](#), S_x , S_y , and S_z are called stretching parameters, which are complex numbers defined in different PML regions. In non-PML regions, these stretching parameters are equal to one. The modified propagation equation then reads:

$$\phi(x, y, z + \Delta z) = \frac{1}{(2\pi)^2} \exp(i S_z k_0 \delta n \Delta z) \iint_{-\infty}^{\infty} d\vec{k}_t \exp(i \vec{k}_t \cdot \vec{r}) \exp(i S_z) \exp(i \sqrt{k_0^2 n_0^2 - \hat{k}_t^2} \Delta z) \tilde{\phi}(\vec{k}_t, z) \quad (589)$$

where $\hat{k}_t^2 = (k_x/S_x)^2 + (k_y/S_y)^2$ is the square of the stretched transverse wave number. The stretching parameters can be fine-tuned to minimize spurious reflections.

Using Beam Propagation Method

General

The following code excerpt describes the general setup for using the scalar BPM solver to compute the optical generation. The computation of the optical generation is activated by an `OpticalGeneration` section, in which the `QuantumYield` model, as defined in [Eq. 571](#), [p. 540](#) and [Eq. 572](#), [p. 541](#), can be specified. Solver-specific parameters are listed in the `BPM` section. The excitation parameters are split into the general `Excitation` section for parameters common to all optical solvers and the `BPM Excitation` section. In the latter section, you can select either a `PlaneWave` or `Gaussian` excitation.

21: Optical Generation

Beam Propagation Method

The `GridNodes` parameter determines the number of discretization points for each spatial dimension. In the next line, the reference refractive index is specified that is used in the operator expansion in [Eq. 586, p. 560](#). Parameters related to the excitation field and the boundary conditions for each spatial dimension are grouped in subsections as explained below:

```
Physics {  
    ...  
    Optics (  
        OpticalGeneration (  
            QuantumYield (  
                ...  
            )  
        )  
        OpticalSolver (  
            BPM (  
                GridNodes = (256,256,1600)  
                ReferenceRefractiveIndex = 2.2  
                Excitation (  
                    ...  
                )  
                Boundary (  
                    ...  
                )  
            )  
        )  
    )  
    Excitation (  
        Wavelength = 0.5  
        Intensity = 0.1  
        Theta = 0.0  
    )  
}
```

As the reference refractive index can greatly influence the accuracy of the solution due its use in the expansion of the propagation operator, several options are available for its specification. The simplest option is to specify a globally constant value as shown above. For multilayer structures with large refractive index contrasts, better results can be achieved by using either the average or the maximum value of the refractive index in each propagation plane. In some cases, a reference refractive index that is computed as the field-weighted average of the refractive index in each propagation plane may be the best option. In addition to these options, a global offset can be specified to further optimize the results. For details about selecting the various options, see [Table 186 on page 1269](#).

Bidirectional BPM

The bidirectional BPM solver is activated by specifying a `Bidirectional` section in the `BPM` section:

```
Optics (
  OpticalSolver (
    BPM (
      ...
      Bidirectional (
        Iterations = 3
        Error = 1e-3
      )
    )
  )
)
```

In the `Bidirectional` section, two break criteria can be set to control the total number of forward and backward passes that are performed in the beam propagation method. If the number of iterations exceeds the value of `Iterations` or if the relative error with respect to the previous iteration is smaller than the value of `Error`, it is assumed that the solution has been found. Note that a single iteration can be either a forward or backward pass. From a performance point of view, it is important to mention that consecutive iterations only require a fraction of CPU time compared with the initial pass.

For plotting the optical field after every iteration, the keyword `OpticalField` must be listed in the `Plot` section of the command file. If only the final optical intensity is of interest, it is sufficient to specify the keyword `OpticalIntensity` in the `Plot` section.

Excitation

In the beam propagation method, a given input field, which is referred to as excitation in the remainder of this section, is propagated through the device structure. Two types of excitation are supported: a `Gaussian` and a (truncated) `PlaneWave`. Both are characterized by a propagation direction, wavelength, and power as shown below. In two dimensions, the propagation direction is determined by the angle between the positive y-axis and the propagation vector. To specify the propagation direction in three dimensions, two angles, `Theta` and `Phi`, must be given. Their definition is illustrated in [Figure 44 on page 564](#).

21: Optical Generation

Beam Propagation Method

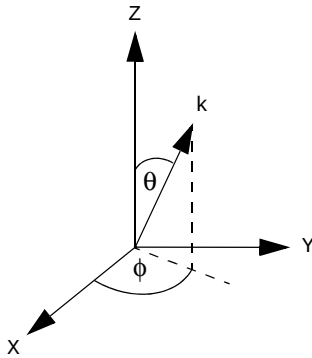


Figure 44 Definition of angles for specification of propagation direction in three dimensions

The incident light power in the plane wave is specified as the **Intensity** in $[\text{W}/\text{cm}^2]$. For a Gaussian excitation, the given value refers to its maximum. All of these parameters are listed in the global **Excitation** section.

The excitation parameters common to all types of excitation are:

```
Physics {
  ...
  Optics (
    Excitation (
      Theta = 10.0      # [degree]
      Phi = 60.0        # [degree], only in 3D
      Wavelength = 0.3  # [um]
      Intensity = 0.1   # [W/cm^2]
      ...
    )
    ...
  )
}
```

A Gaussian excitation along the propagation direction defined by k is determined by its half-width **SigmaGauss** $[\mu\text{m}]$ and its center position **CenterGauss** $[\mu\text{m}]$ as shown below for a 2D and 3D device geometry.

Gaussian excitation (2D):

```
Physics {
  ...
  Optics(
    OpticalSolver (
      BPM (
        Excitation (
          Type = "Gaussian"
        )
      )
    )
  )
}
```

```

        SigmaGauss = (2.0)    # [um]
        CenterGauss = (0.0)   # [um]
    )
)
...
}

```

Gaussian excitation (3D):

```

Physics {
    ...
    Optics(
        OpticalSolver (
            BPM (
                Excitation (
                    Type = Gaussian
                    SigmaGauss = (2.0,4.0)    # [um]
                    CenterGauss = (0.0,1.0)   # [um]
                )
            )
        )
    )
}

```

For a plane wave excitation, it is possible to specify the truncation positions for each coordinate axis as well as a parameter that determines the fall-off. For numeric reasons, a discrete truncation must be avoided. Instead, a Fermi function–like fall-off is available:

$$F_{xy} = \left[1 + \exp\left(\frac{|\tilde{x}| - x_0}{s_x}\right) \right]^{-1} \cdot \left[1 + \exp\left(\frac{|\tilde{y}| - y_0}{s_y}\right) \right]^{-1} \quad (590)$$

$$\phi(x, y, z=0) = F_{xy} \cdot \phi_0$$

which can be tuned by adjusting the `TruncationSlope` and `TruncationPosition` parameters in the `Excitation` section. In [Eq. 590](#), ϕ_0 is the amplitude of the incident light, $s_{x,y}$ denotes the `TruncationSlope`, $\tilde{x}, \tilde{y}$ is the position with respect to the symmetry point, and x_0, y_0 is the half width. Both $\tilde{x}, \tilde{y}$ and x_0, y_0 are deduced from the `TruncationPosition` parameters. In two dimensions, the second factor on the right-hand side in [Eq. 590](#) is omitted.

Truncated plane wave excitation (2D):

```

Physics {
    Optics (
        OpticalSolver (
            BPM (
                Excitation (

```

21: Optical Generation

Beam Propagation Method

```
        Type = "PlaneWave"
        TruncationPositionX = (-1.0,1.0)
        TruncationSlope = (0.3)
    )
)
)
...
)
}
```

Truncated plane wave excitation (3D):

```
Physics {
    Optics (
        OpticalSolver (
            BPM (
                Excitation (
                    Type = "PlaneWave"
                    TruncationPositionX = (-1.0,1.0)
                    TruncationPositionY = (-2.0,3.0)
                    TruncationSlope = (0.3,0.3)
                )
            )
        )
    )
}
```

Boundary

For every spatial dimension, a separate boundary condition and corresponding parameters can be defined. Periodic boundary conditions inherent to the FFT BPM solver are the default in the transverse dimensions. The specification of PML boundary conditions in the x-direction and y-direction is shown below. With the keyword `Order`, the spatial variation of the complex stretching parameter can be specified. In this example, a quadratic increase from the minimum value to the maximum value within the given number of `GridNodes` on either side has been chosen.

Optionally, several `VacuumGridNodes` can be inserted between the physical simulation domain and the PML boundary:

```
Physics {
    Optics(
        OpticalSolver (
            BPM (
                Boundary (
                    Type = "PML"
                    Side = "X"
                )
            )
        )
    )
}
```

```

        Order = 2
        StretchingParameterReal = (1.0,1,0)
        StretchingParameterImag = (1.0,5.0)
        GridNodes = (5,5)
        VacuumGridNodes = (4,4)
    )
    Boundary (
        Type = "PML"
        Side = "y"
        Order = 2
        StretchingParameterReal = (1.0,1,0)
        StretchingParameterImag = (1.0,4.0)
        GridNodes = (8,8)
    )
)
...
}

```

Ramping Input Parameters

Several input parameters of the BPM solver such as the wavelength of the incident light can be ramped to obtain device characteristics as a function of the specified parameter. The general concept of ramping the values of physical parameters is described in [Ramping Physical Parameter Values on page 109](#). More detailed information about ramping Optics parameters can be found in [Parameter Ramping on page 494](#).

Visualizing Results on Native Tensor Grid

For an accurate analysis of the BPM results, the complex refractive index, the complex optical field, and the optical intensity can be plotted on the native tensor grid. In general, the results from the BPM solver are interpolated from a tensor grid to a mixed-element grid on which the device simulation is performed.

For physical and numeric reasons, the mesh resolution of the optical grid needs to be much finer than that of the electrical grid in most regions of the device. This can lead to the introduction of interpolation errors, for example, by undersampling the respective dataset on the electrical grid. To obtain an estimate of the interpolation error, it can be beneficial to visualize the BPM results on its native tensor grid. As typical tensor grids in three dimensions tend to be very large, it is also possible to extract a limited domain for more efficient visualization. In two dimensions, a rectangular subregion or an axis-aligned straight line can be plotted; whereas in three dimensions, a subvolume, a plane perpendicular to the coordinate axes, and a straight line can be visualized directly on the tensor grid.

21: Optical Generation

Optical AC Analysis

Tensor plots can be activated by specifying one or more `TensorPlot` sections and an optional base name for the tensor-plot file name in the `File` section.

The following syntax extracts a plane perpendicular to the z-axis at position $Z = 1.3 \mu\text{m}$, which extends from $-2 \mu\text{m}$ to $3 \mu\text{m}$, and from $-1 \mu\text{m}$ to $5 \mu\text{m}$ in the x- and y-direction, respectively:

```
File (
    ...
    TensorPlot = "tensor_plot"
)
...
TensorPlot (
    Name = "plane_z_const"
    Zconst = 1.3
    Xmin = -2
    Xmax = 3
    Ymin = -1
    Ymax = 5
){
    ComplexRefractiveIndex
    OpticalField
    OpticalIntensity
}
```

To extract a box in three dimensions that extends over the whole device horizontally but is bound in the vertical direction, the `TensorPlot` section looks like:

```
TensorPlot (
    Name = "bounded_box"
    Zmin = 3
    Zmax = 8
){
    ...
}
```

Optical AC Analysis

An optical AC analysis calculates the quantum efficiency as a function of the frequency of the optical signal intensity. The method is based on the AC analysis technique and provides real and imaginary parts of the quantum efficiency versus the frequency.

During an optical AC analysis, a small perturbation of the incident wave power δP_0 is applied. Therefore, the photogeneration rate is perturbed as $G^{\text{opt}} + \delta G^{\text{opt}} e^{i\omega t}$, where $\omega = 2\pi f$ (f is the frequency) and δG^{opt} is an amplitude of a local perturbation. The resulting small-signal

device current perturbation δI_{dev} is the sum of real and imaginary parts, and the expressions for the quantum efficiency are:

$$\begin{aligned}\eta &= \frac{\text{Re}[\delta I_{\text{dev}}]/q}{\delta P^{\text{tot}}\lambda/hc} \\ C_{\text{opt}} &= \frac{1}{\omega} \cdot \frac{\text{Im}[\delta I_{\text{dev}}]/q}{\delta P^{\text{tot}}\lambda/hc} \\ \delta P^{\text{tot}} &= \int_S \delta P_0 ds\end{aligned}\tag{591}$$

where the quantity $\delta P^{\text{tot}}\lambda/hc$ gives a perturbation of the total number of photons and $\text{Re}[\delta I_{\text{dev}}]/q$ is a perturbation of the total number of electrons at an electrode. As a result, for each electrode, Sentaurus Device places two values into the AC output file, photo_a and photo_c, that correspond to η and C_{opt} , respectively. To start the optical AC analysis, add the keyword `Optical` in the `ACCoupled` statement, for example:

```
ACCoupled ( StartFrequency=1.e4 EndFrequency=1.e9
  NumberOfPoints=31 Decade Node(a c) Optical )
{ poisson electron hole }
```

NOTE If an element is excluded (`Exclude` statement) in optical AC (this is usually the case for voltage sources in regular AC simulation), it means that this element is not present in the simulated circuit and, correspondingly, it provides zero AC current for all branches that are connected to the element. Therefore, do *not* exclude voltage sources.

References

- [1] B. R. Bennett, R. A. Soref, and J. A. Del Alamo, "Carrier-Induced Change in Refractive Index of InP, GaAs, and InGaAsP," *IEEE Journal of Quantum Electronics*, vol. 26, no. 1, pp. 113–122, 1990.
- [2] D. A. Clugston and P. A. Basore, "Modelling Free-carrier Absorption in Solar Cells," *Progress in Photovoltaics: Research and Applications*, vol. 5, no. 4, pp. 229–236, 1997.
- [3] D. K. Schroder, R. N. Thomas, and J. C. Swartz, "Free Carrier Absorption in Silicon," *IEEE Journal of Solid-State Circuits*, vol. SC-13, no. 1, pp. 180–187, 1978.
- [4] H. E. Bennett and J. O. Porteus, "Relation Between Surface Roughness and Specular Reflectance at Normal Incidence," *Journal of the Optical Society of America*, vol. 51, no. 2, pp. 123–129, 1961.
- [5] P. Beckmann and A. Spizzichino, *The Scattering of Electromagnetic Waves from Rough Surfaces*, Norwood, Massachusetts: Artech House, 1987.

21: Optical Generation

References

- [6] J. Krc, F. Smole, and M. Topic, “Analysis of Light Scattering in Amorphous Si:H Solar Cells by a One-Dimensional Semi-coherent Optical Model,” *Progress in Photovoltaics: Research and Applications*, vol. 11, no. 1, pp. 15–26, 2003.
- [7] M. D. Feit and J. A. Fleck, Jr., “Light propagation in graded-index optical fibers,” *Applied Optics*, vol. 17, no. 24, pp. 3990–3998, 1978.
- [8] P. Kaczmariski and P. E. Lagasse, “Bidirectional Beam Propagation Method,” *Electronics Letters*, vol. 24, no. 11, pp. 675–676, 1988.

This chapter presents the radiation models used in Sentaurus Device.

When high-energy particles penetrate a semiconductor device, they deposit their energy by the generation of electron–hole pairs. These charges can perturb the normal operation of the device. This chapter describes the models for carrier generation by gamma radiation, alpha particles, and heavy ions.

Generation by Gamma Radiation

Using Gamma Radiation Model

The radiation model is activated by specifying the keyword `Radiation( . . . )` (with optional parameters) in the `Physics` section:

```
Radiation{
  Dose = <float> | DoseRate = <float>
  DoseTime = (<float>,<float>)
  DoseTSigma = <float>
}
```

where `DoseRate` (in rad/s) represents D in [Eq. 592](#). The optional `DoseTime` (in s) allows you to specify the time period during which exposure to the constant `DoseRate` occurs. `DoseTSigma` (in s) can be combined with `DoseTime` to specify the standard deviation of a Gaussian rise and fall of the radiation exposure. As an alternative to `DoseRate`, `Dose` (in rad) can be specified to represent the total radiation exposure over the `DoseTime` interval. In this case, `DoseTime` must be specified.

To plot the generation rate due to gamma radiation, specify `RadiationGeneration` in the `Plot` section.

Yield Function

Generation of electron–hole pairs due to radiation is an electric field–dependent process [1] and is modeled as follows:

$$G_r = g_0 D \cdot Y(F)$$

$$Y(F) = \left(\frac{F + E_0}{F + E_1} \right)^m \quad (592)$$

where D is the dose rate, g_0 is the generation rate of electron–hole pairs, and E_0 , E_1 , and m are constants.

All these constants can be specified in parameter file of Sentaurus Device as follows:

```
Radiation {
  g = 7.6000e+12 # [1/(rad*cm^3)]
  E0 = 0.1 # [V/cm]
  E1 = 1.3500e+06 # [V/cm]
  m = 0.9 # [1]
}
```

Alpha Particles

Using Alpha Particle Model

Specify the AlphaParticle in the Physics section:

```
Physics { ...
  AlphaParticle (<optional keywords>)
}
```

[Table 228 on page 1297](#) lists the keyword options for alpha particles.

Table 100 Coefficients for carrier generation by alpha particles

Symbol	Parameter name	Default value	Unit
s	s	2×10^{-12}	s
w_t	wt	1×10^{-5}	cm
c_2	c2	1.4	1
a	alpha	90	cm^{-1}

Table 100 Coefficients for carrier generation by alpha particles

Symbol	Parameter name	Default value	Unit
α_2	alpha2	5.5×10^{-4}	cm
α_3	alpha3	2×10^{-4}	cm
E_p	Ep	3.6	eV
a_0	a0	-1.033×10^{-4}	cm
a_1	a1	2.7×10^{-10}	cm/eV
a_2	a2	4.33×10^{-17}	cm/eV ²

To plot the instant generation rate $G(t)$ due to alpha particles, specify AlphaGeneration in the Plot section; to plot $\int_{-\infty}^{\infty} dtG$, specify AlphaCharge.

The generation by alpha particles cannot be used except in transient simulations. The amount of electron-hole pairs generated before the initial time of the transient is added to the carrier densities at the beginning of the simulation.

An option to improve the spatial integration of the charge generation is presented in [Improved Alpha Particle/Heavy Ion Generation Rate Integration on page 579](#).

Alpha Particle Model

The generation rate caused by an alpha particle with energy E is computed by [2]:

$$G(u, v, w, t) = \frac{a}{\sqrt{2\pi} \cdot s} \exp \left[-\frac{1}{2} \left(\frac{t-t_m}{s} \right)^2 - \frac{1}{2} \left(\frac{v^2 + w^2}{w_t^2} \right) \right] \left[c_1 e^{\alpha u} + c_2 \exp \left(-\frac{1}{2} \left(\frac{u-\alpha_1}{\alpha_2} \right)^2 \right) \right] \quad (593)$$

if $u < \alpha_1 + \alpha_3$, and by:

$$G(u, v, w, t) = 0 \quad (594)$$

if $u \geq \alpha_1 + \alpha_3$. In this case, u is the coordinate along the particle path and v and w are coordinates orthogonal to u . The direction and place of incidence are defined in the Physics section of the command file with the keywords Direction and Location (or StartPoint, see [Note on page 577](#)), respectively. Parameter t_m is the time of the generation peak defined by the keyword Time. A Gaussian time dependence can also be used to simulate the typical generation due to pulsed laser or electron beams.

The maximum of the Bragg peak, α_1 , is fitted to data [3] by a polynomial function:

$$\alpha_1 = a_0 + a_1 E + a_2 E^2 \quad (595)$$

The parameter c_1 is given by:

$$c_1 = \exp[\alpha(\alpha_1[10\text{MeV}] - \alpha_1[E])] \quad (596)$$

The scaling factor a is determined from:

$$\int_0^\infty \int_{-\infty}^\infty \int_{-\infty}^\infty \int_{-\infty}^\infty G(u, v, w, t) dt dw dv du = \frac{E}{E_p} \quad (597)$$

where E_p is the average energy needed to create an electron–hole pair. The remaining parameters are listed in [Table 100 on page 572](#). They are available in the parameter set `AlphaParticle` and are valid for alpha particles with energies between 1 MeV and 10 MeV.

Heavy Ions

When a heavy ion penetrates a device structure, it loses energy and creates a trail of electron–hole pairs. These additional electrons and holes may cause a large enough current to switch the logic state of a device, for example, a memory cell. Important factors are:

- The energy and type of the ion.
- The angle of penetration of the ion.
- The relation between the lost energy or linear energy transfer (LET) and the number of pairs created.

Using Heavy Ion Model

The simulation of an SEU caused by a heavy ion impact is activated by using the keyword `HeavyIon` in an appropriate `Physics` section:

```
# Default heavy ion
Physics {
    HeavyIon (<keyword_options>) }

# User-defined heavy ion.
Physics {
    HeavyIon (<IonName>) (<keyword_options>) }
# User-defined heavy ions do not have default parameters. Therefore,
# in the parameter file, a specification of the following form must appear:
# HeavyIon(<IonName>){ ... } (see Table 102 on page 577)
```

[Table 229 on page 1298](#) describes the options for `HeavyIon`. The generation rate by the heavy ion is generally used in transient simulations. The number of electron–hole pairs generated before the initial time of the transient is added to the carrier densities at the beginning of the

simulation. The total charge density and the instant generation rate are plotted using HeavyIonCharge and HeavyIonGeneration in the Plot section, respectively.

If the value of Wt_hi is 0, then uniform generation is selected. If the value of LET_f is 0, the keyword LET_f can be ignored.

An option to improve the spatial integration of the charge generation is presented in [Improved Alpha Particle/Heavy Ion Generation Rate Integration on page 579](#).

Heavy Ion Model

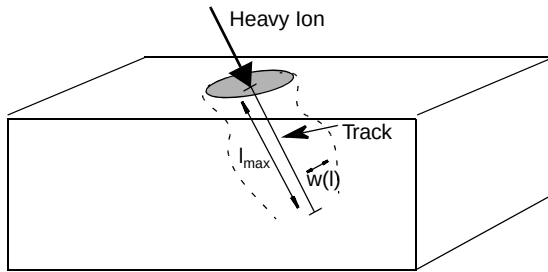


Figure 45 A heavy ion penetrating a semiconductor; its track is defined by a length and the transverse spatial influence is assumed to be symmetric about the track axis

A simple model for the heavy ion impinging process is shown in [Figure 45](#). The generation rate caused by the heavy ion is computed by:

$$G(l, w, t) = G_{\text{LET}}(l)R(w, l)T(t) \quad (598)$$

if $l < l_{\max}$ ($l_{\max}$ is the length of the track), and by:

$$G(l, w, t) = 0 \quad (599)$$

if $l \geq l_{\max}$. $R(w)$ and $T(t)$ are functions describing the spatial and temporal variations of the generation rate. $G_{\text{LET}}(l)$ is the linear energy transfer generation density and its unit is pairs/cm³.

$T(t)$ is defined as a Gaussian function:

$$T(t) = \frac{2 \cdot \exp\left(-\left(\frac{t - t_0}{\sqrt{2} \cdot s_{\text{hi}}}\right)^2\right)}{\sqrt{2} \cdot s_{\text{hi}} \sqrt{\pi} \left(1 + \operatorname{erf}\left(\frac{t_0}{\sqrt{2} \cdot s_{\text{hi}}}\right)\right)} \quad (600)$$

22: Radiation Models

Heavy Ions

where t_0 is the moment of the heavy ion penetration (see the keyword `Time` in [Table 229 on page 1298](#)), and s_{hi} is the characteristic value of the Gaussian (see s_{hi} in [Table 102 on page 577](#)).

The spatial distribution, $R(w, l)$, can be defined as an exponential function (default):

$$R(w, l) = \exp\left(-\frac{w}{w_t(l)}\right) \quad (601)$$

or a Gaussian function:

$$R(w, l) = \exp\left(-\left(\frac{w}{w_t(l)}\right)^2\right) \quad (602)$$

where w is a radius defined as the perpendicular distance from the track. The characteristic distance w_t is defined as `Wt_hi` in the `HeavyIon` statement and can be a function of the length l (see [Table 229 on page 1298](#)).

In addition, the spatial distribution $R(w, l)$ can be defined as a PMI function (see [Spatial Distribution Function on page 1146](#)).

The linear energy transfer (LET) generation density, $G_{LET}(l)$, is given by:

$$G_{LET}(l) = a_1 + a_2 l + a_3 e^{a_4 l} + k' \left[c_1 (c_2 + c_3 l)^{c_4} + LET_f(l) \right] \quad (603)$$

where $LET_f(l)$ (defined likewise by the keyword `LET_f`) is a function of the length l . Example 2 in [Examples: Heavy Ions on page 577](#) shows how you can use an array of values to specify the length dependence of $LET_f(l)$. A linear interpolation is used for values between the array entries of `LET_f`.

There are two options for the units of `LET_f`: pairs/cm^3 (default) or $\text{pC}/\mu\text{m}$ (activated by the keyword `PicoCoulomb`). Depending on the units of `LET_f` chosen, k' takes on different values in order to make the above equation dimensionally consistent. The appropriate values of k' for different device dimensions are summarized in [Table 101](#). Great care must also be

Table 101 Setting correct k' to make LET generation density equation dimensionally consistent

Condition	Two-dimensional device	Three-dimensional device
<code>LET_f</code> has units of pairs/cm^3 for $R(w, l)$ is exponential or Gaussian	$k' = k$	$k' = k$
<code>LET_f</code> has units of $\text{pC}/\mu\text{m}$ and $R(w, l)$ is exponential	$k' = \frac{k}{2w_t d}$ $d = 1\mu\text{m}$	$k' = \frac{k}{2\pi w_t^2}$

Table 101 Setting correct k' to make LET generation density equation dimensionally consistent

Condition	Two-dimensional device	Three-dimensional device
LET_f has units of pC/ μm and $R(w, l)$ is Gaussian	$k' = \frac{k}{\sqrt{\pi} w_t d}$ $d = 1\mu\text{m}$	$k' = \frac{k}{\pi w_t^2}$

Great care must also be exercised to choose the correct units for w_{t_hi} and Length. The default unit of w_{t_hi} and Length is centimeter. If the keyword PicoCoulomb is specified, the unit becomes μm . Examples illustrating the correct unit use are shown in [Examples: Heavy Ions](#). The other coefficients used in [Eq. 603](#) are listed in [Table 102](#) with their default values, and they can be adjusted in the parameter file of Sentaurus Device.

Table 102 Coefficients for carrier generation by heavy ion (Heavylon parameter set)

	s_{hi}	a_1	a_2	a_3	a_4	k	c_1	c_2	c_3	c_4
Keyword	s_hi	a_1	a_2	a_3	a_4	k_hi	c_1	c_2	c_3	c_4
Default value	2e-12	0	0	0	0	1	0	1	0	1
Default unit	s	pairs/cm ³	pairs/cm ³ /cm	pairs/cm ³	cm ⁻¹	1	pairs/cm ³	1	cm ⁻¹	1
Unit if PicoCoulomb is chosen	s	pairs/cm ³	pairs/cm ³ / μm	pairs/cm ³	μm^{-1}	1	pC/ μm	1	μm^{-1}	1

NOTE The keyword Location defines a bidirectional track for which Sentaurus Device computes the generation rate in both directions from the place of incidence along the Direction vector. The keyword StartPoint defines a one-directional track. In this case, Sentaurus Device computes the generation rate only in the positive direction from the place of incidence.

Examples: Heavy Ions

Example 1

The track has a constant LET_f value of 0.2 pC/ μm across the track. The track length is 1 μm ($l_{\text{max}} = 1\mu\text{m}$) and the heavy ion crosses the device at the time 0.1 ps. The unit of LET_f is pC/ μm and the spatial distribution is Gaussian. Since PicoCoulomb was chosen, the values of Length and w_{t_hi} are expressed in terms of μm .

22: Radiation Models

Heavy Ions

The keyword `HeavyIonChargeDensity` in the `Plot` statement plots the charge density generated by the ion:

```
Physics { Recombination ( SRH(DopingDependence) )
  Mobility (DopingDependence Enormal HighFieldSaturation)
  HeavyIon (
    Direction=(0,1)
    Location=(1.5,0)
    Time=1.0e-13
    Length=1
    Wt_hi=3
    LET_f=0.2
    Gaussian
    PicoCoulomb )
  }
Plot { eDensity hDensity ElectricField HeavyIonChargeDensity
}
```

Example 2

The `LET_f` and radius (`Wt_hi`) values are functions of the position along the track (in this case, $l_{\max} = 1.7 \times 10^{-4} \text{ cm}$). Values in between the array entries are linearly interpolated. The unit of `LET_f` is pairs/cm^3 (because the keyword `PicoCoulomb` is not used), and the unit for `Length` and `Wt_hi` is centimeter. For each value of length, there is a corresponding value of `LET_f` and a value for the radius. The spatial distribution in the perpendicular direction from the track is exponential:

```
Physics { Recombination ( SRH(DopingDependence) )
  HeavyIon (
    Direction=(0,1)
    Location=(1.5,0)
    Time=1.0e-13
    Length = [1e-4 1.5e-4 1.6e-4 1.7e-4]
    LET_f = [1e6 2e6 3e6 4e6]
    Wt_hi = [0.3e-4 0.2e-4 0.25e-4 0.1e-4]
    Exponential )
  }
```

Example 3

This example illustrates multiple ion strikes in the SEU model.

```
Physics { Recombination ( SRH(DopingDependence) )
  HeavyIon (
    Direction=(0,1)
    Location=(0,0)
    Time=1.0e-13
```



```

Length = [1e-4 1.5e-4 1.6e-4 1.7e-4]
LET_f = [1e6 2e6 3e6 4e6]
Wt_hi = [0.3e-4 0.2e-4 0.25e-4 0.1e-4] )

HeavyIon ("Ion1")(
# User-defined heavy ions do not have default parameters. Therefore,
# in the parameter file, a specification of the following form must appear:
# HeavyIon("Ion1"){ ... } (see Table 102 on page 577)
  Direction=(0,1)
  Location=(1,0)
  Time=1.0e-13
  Length = [1e-4 1.5e-4 1.6e-4 1.7e-4]
  LET_f = [1e6 2e6 3e6 4e6]
  Wt_hi = [0.3e-4 0.2e-4 0.25e-4 0.1e-4] )
}

```

Improved Alpha Particle/Heavy Ion Generation Rate Integration

Accurate integration of alpha particle or heavy ion generation rates is very important for predictive modeling of SEU phenomena. By default, in Sentaurus Device, the integration of the generation rate over the control volume associated with each vertex in the mesh is performed under the assumption that the generation rate is constant inside the vertex control volume and equal to the generation rate value at the vertex. As alpha particle or heavy ion generation rates can change very rapidly in space, the approximation error with such an approach may lead to large errors on a coarse mesh (in particular, the method does not guarantee charge conservation).

To eliminate this source of numeric error, an improved spatial integration can be performed. The procedure used in Sentaurus Device for optical generation (see [Chapter 21 on page 469](#)) extends to the alpha particle/heavy ion case. Each control volume is covered by a set of small rectangular boxes and the generation rate is integrated numerically inside these boxes. To activate this procedure, the keyword `RecBoxIntegr` must be specified in the `Math` section. The same keyword is used as for optical generation, with the same set of default parameters. By changing the default parameters, you can also control the accuracy of the integration (see [Transfer Matrix Method on page 538](#)).

References

- [1] J.-L. Leray, “Total Dose Effects: Modeling for Present and Future,” *IEEE Nuclear and Space Radiation Effects Conference (NSREC) Short Course*, 1999.
- [2] A. Erlebach, *Modellierung und Simulation strahlensensitiver Halbleiterbauelemente*, Aachen: Shaker, 1999.
- [3] L. C. Northcliffe and R. F. Schilling, “Range and Stopping - Power Tables for Heavy Ions,” *Nuclear Data Tables*, vol. A7, no. 1–2, New York: Academic Press, pp. 233–463, 1969.

CHAPTER 23 Noise, Fluctuations, and Sensitivity

This chapter discusses noise analysis, fluctuation analysis, and sensitivity analysis in Sentaurus Device.

This chapter explains the Sentaurus Device features to model noise, statistical fluctuations of doping, trap concentration, workfunction, and geometry, and sensitivity to variations of model parameters, doping, and geometry. All these topics deal with the response of the device characteristics to small, device-internal variations. For noise, these variations occur randomly over time within a single device; for statistical fluctuations, the variations are random device-to-device variations; and for sensitivity, the variations are user supplied. Despite the different nature of the variations, they all can be handled by the same linearization approach called the *impedance field method* and, therefore, Sentaurus Device treats them all in a common framework.

[Using the Impedance Field Method](#) explains how to perform noise, fluctuation, and sensitivity analysis. [Noise Sources on page 585](#) discusses the noise and random fluctuation models that Sentaurus Device offers. [Statistical Impedance Field Method on page 592](#) describes a modeling approach for random dopant fluctuations, trap concentration fluctuations, and workfunction variations based on statistical sampling. [Deterministic Variations on page 595](#) discusses user-supplied variations of doping, geometry, and model parameters.

[Impedance Field Method on page 598](#) discusses the background of the method [1], and [Noise Output Data on page 601](#) summarizes the data available for visualization.

Using the Impedance Field Method

Sentaurus Device treats noise analysis, fluctuation analysis, and sensitivity analysis by the impedance field method, and as extensions of small-signal analysis (see [Small-Signal AC Analysis on page 127](#)).

NOTE Currently, the impedance field method is not supported with the HeteroInterface option (see [Abrupt and Graded Heterojunctions on page 48](#)), the Thermionic option (see [Thermionic Emission Current on page 651](#)), or dipole layers ([Dipole Layer on page 192](#)).

For noise and random fluctuations, Sentaurus Device computes the variances and correlation coefficients for the voltages at selected circuit nodes, assuming the net current to these nodes is fixed. As the computation is performed in frequency space, the computed quantities are

23: Noise, Fluctuations, and Sensitivity

Using the Impedance Field Method

called the noise voltage spectral densities. Sentaurus Device also computes the variances and correlation coefficients of the currents through the nodes, assuming fixed voltages; these quantities are the noise current spectral densities.

For the statistical impedance field method (sIFM) and for deterministic variations, Sentaurus Device computes responses of node voltages assuming fixed currents and responses of node currents assuming fixed voltages.

To use noise and random fluctuation analysis, specify the physical models for the microscopic origin of the deviations (called the local noise sources, LNS) as options to the keyword **Noise** in the **Physics** section of the command file of Sentaurus Device:

```
Physics {  
    ...  
    Noise ( <Noisemodels> )  
}
```

where **<Noisemodels>** is a list that specifies any number of noise models listed in [Table 250 on page 1310](#).

For the sIFM, specify **RandomizedVariation** in the global **Math** section and in a device-specific global **Physics** section:

```
Math {  
    RandomizedVariation <string> ( <specification> )  
    ...  
}  
Device <string> {  
    Physics {  
        RandomizedVariation <string> ( <specification> )  
    }  
}
```

where **<specification>** specifies the details for the randomization (see [Statistical Impedance Field Method on page 592](#)).

For deterministic variations, specify the variations as options to **DeterministicVariation** in the global **Physics** section:

```
Physics {  
    DeterministicVariation( <variations> )  
    ...  
}
```

where **<variations>** is a list that specifies any number of deterministic variations (see [Deterministic Variations on page 595](#) and [Table 234 on page 1300](#)).

In any case, use the `ObservationNode` option to the `ACCoupled` statement to specify the device nodes for which the noise spectral densities or deviations are required, for example:

```
ACCoupled (
  StartFrequency = 1.e8 EndFrequency = 1.e11
  NumberOfPoints = 7 Decade
  Node (n_source n_drain n_gate)
  Exclude (v_drain v_gate)
  ObservationNode (n_drain n_gate)
  ACExtract = "mos"
  NoisePlot = "mos"
){
  poisson electron hole contact circuit
}
}
```

The keyword `ObservationNode` enables noise and fluctuation analysis (in this case, for the nodes `n_drain` and `n_gate`). `NoisePlot` specifies a file name prefix for device-specific plots (see [Noise Output Data on page 601](#)). For more information on the `ACCoupled` statement, see [Small-Signal AC Analysis on page 127](#).

NOTE The observation nodes must be a subset of the nodes specified in `Node(...)`.

The results of the analysis are the noise voltage spectral densities, the noise current spectral densities, and voltage and current deviations. They appear in the `ACExtract` file (see [Table 103](#)) and, in the case of the sIFM, in a separate `.csv` file. The units are V^2s for the voltage spectral densities and A^2s for the current noise spectral densities.

Table 103 Noise and fluctuation data written into `ACExtract` file

Name	Description
<code>S_V</code>	Autocorrelation noise voltage spectral density (NVSD)
<code>S_I</code>	Autocorrelation noise current spectral density (NISD)
<code>S_V_ee</code> <code>S_V_hh</code>	Electron NVSD Hole NVSD
<code>S_V_eeDiff</code> <code>S_V_hhDiff</code>	Electron NVSD due to diffusion LNS Hole NVSD due to diffusion LNS
<code>S_V_eeMonoGR</code> <code>S_V_hhMonoGR</code>	Electron NVSD due to monopolar GR LNS Hole NVSD due to monopolar GR LNS
<code>S_V_eeFlickerGR</code> <code>S_V_hhFlickerGR</code>	Electron NVSD due to flicker GR LNS Hole NVSD due to flicker GR LNS
<code>S_V_Doping</code> <code>S_I_Doping</code>	NVSD and NISD due to random dopant fluctuations

23: Noise, Fluctuations, and Sensitivity

Using the Impedance Field Method

Table 103 Noise and fluctuation data written into ACExtract file

Name	Description
S_V_Geometric S_I_Geometric	NVSD and NISD due to geometric fluctuations
S_V_TrapConcentration S_I_TrapConcentration	NVSD and NISD due to trap concentration fluctuations
S_V_Trap S_I_Trap	NVSD and NISD due to trapping noise
S_V_Workfunction S_I_Workfunction	NVSD and NISD due to workfunction fluctuations
ReS_VXV ImS_VXV	Real/imaginary parts of the cross correlation noise voltage spectral density (NVXVSD)
ReS_IXI ImS_IXI	Real/imaginary parts of the cross correlation noise current spectral density
ReS_VXV_ee ImS_VXV_ee ReS_VXV_hh ImS_VXV_hh	Real/imaginary parts of the electron/hole NVXVSD
ReS_VXV_eeDiff ImS_VXV_eeDiff ReS_VXV_hhDiff ImS_VXV_hhDiff	Real/imaginary parts of the electron/hole NVXVSD due to diffusion LNS
ReS_VXV_eeMonoGR ImS_VXV_eeMonoGR ReS_VXV_hhMonoGR ImS_VXV_hhMonoGR	Real/imaginary parts of the electron/hole NVXVSD due to monopolar GR LNS
ReS_VXV_eeFlickerGR ImS_VXV_eeFlickerGR ReS_VXV_hhFlickerGR ImS_VXV_hhFlickerGR	Real/imaginary parts of the electron/hole NVXVSD due to flicker GR LNS
ReS_VXV_Doping ReS_IXI_Doping	Real part of the NVXVSD and NIXISD due to random dopant fluctuations
ReS_VXV_Geometric ReS_IXI_Geometric	Real part of the NVXVSD and NIXISD due to geometric fluctuations
ReS_VXV_TrapConcentration ReS_IXI_TrapConcentration	Real part of the NVXVSD and NIXISD due to trap concentration fluctuations
ReS_VXV_Trap ImS_VXV_Trap ReS_IXI_Trap ImS_IXI_Trap	Real/imaginary parts of the NVXVSD and NIXISD due to trapping noise

Table 103 Noise and fluctuation data written into ACExtract file

Name	Description
ReS_VXV_Workfunction ReS_IXI_Workfunction	Real parts of the NVXVSD and NIXISD due to workfunction fluctuations
dV<name>	Voltage deviation due to deterministic variation <name>
dI<name>	Current deviation due to deterministic variation <name>

Noise Sources

Diffusion Noise

The diffusion noise source (keyword `DiffusionNoise`) available in Sentaurus Device reads:

$$K_{n,n}^{\text{Diff}}(\vec{r}_1, \vec{r}_2, \omega) = 4qn(\vec{r}_1)kT_n(\vec{r}_1)\mu_n(\vec{r}_1)\delta(\vec{r}_1 - \vec{r}_2) \quad (604)$$

A corresponding expression is used for holes. $K_{n,n}^{\text{Diff}}$ is a diagonal tensor.

T_n is either the lattice or carrier temperature, depending on the specification in the command file:

```
DiffusionNoise ( <temp_option> )
```

where <temp_option> is `LatticeTemperature`, `eTemperature`, `hTemperature`, or `e_h_Temperature`. The default is `LatticeTemperature`.

For example, if the following command is specified:

```
Physics {
  Noise ( DiffusionNoise ( eTemperature ) )
}
```

Sentaurus Device uses the electron temperature for the electron noise source and the lattice temperature for the hole noise source. The keyword `e_h_Temperature` forces the corresponding carrier temperature to be used for the diffusion noise source for each carrier type.

Equivalent Monopolar Generation–Recombination Noise

An equivalent monopolar generation–recombination (GR) noise source model (keyword `MonopolarGRNoise`) for a two-level, GR process can be expressed as a tensor [2]:

$$K_{n,n}^{\text{GR}}(\vec{r}_1, \vec{r}_2, \omega) = \frac{J_n(\vec{r}_1)J_n(\vec{r}_2)}{n} \cdot \frac{4\alpha\tau_{\text{eq}}}{1 + \omega^2\tau_{\text{eq}}^2} \delta(\vec{r}_1 - \vec{r}_2) \quad (605)$$

where α is a fitting parameter and τ_{eq} an equivalent GR lifetime. The parameters τ_{eq} and α can be modified in the parameter file of Sentaurus Device. A similar expression is used for holes.

Bulk Flicker Noise

The flicker generation–recombination (GR) noise model (keyword `FlickerGRNoise`) for electrons (similar for holes) is:

$$K_{n,n}^{\text{fGR}}(\vec{r}_1, \vec{r}_2, \omega) = \frac{J_n(\vec{r}_1)J_n(\vec{r}_2)}{n(\vec{r}_1)} \frac{2\alpha_H}{\pi\nu\ln(\tau_1/\tau_0)} [\text{atan}(\omega\tau_1) - \text{atan}(\omega\tau_0)] \delta(\vec{r}_1 - \vec{r}_2) \quad (606)$$

where α_H is a fit parameter; $\omega = 2\pi\nu$, the angular frequency; and the time constants fulfill $\tau_0 < \tau_1$. The parameters α_H , τ_0 , and τ_1 for electrons and holes are accessible in the parameter file. With increasing frequency, the noise source changes from constant to $1/\nu$ behavior close to the frequency $\nu_1 = 1/\tau_1$ and, ultimately, to $1/\nu^2$ behavior at $\nu_0 = 1/\tau_0$.

Trapping Noise

The `Traps` option to `Noise` activates a trapping noise model that follows the microscopic model in [3]. The trapping noise sources are determined fully by the trap model and, therefore, do not require additional adjustable parameters. For example:

```
Physics { Noise(Traps) }
```

activates trapping noise for all traps in the device. It is not possible to activate trapping noise for selected traps only. All trap models apart from tunneling to electrodes are supported. For more details on traps, see [Chapter 17 on page 407](#).

Consider a trap level k with concentration $N_{t,k}$ and trap electron occupation f_k . Trapping noise is expressed by adding a Langevin source $s_{i,k}$ to the net electron capture rate for each process i :

$$R_{i,k} = N_{t,k}(1 - f_k)c_{i,k} - N_{t,k}f_k e_{i,k} + s_{i,k} \quad (607)$$

where $c_{i,k}$ is the electron capture rate for a trap occupied by zero electrons, and $e_{i,k}$ is the electron emission rate for a trap occupied by one electron (see also [Eq. 433, p. 412](#)). Denoting the expectation value with $\langle \rangle$, the trapping noise source takes the form:

$$K_{ij,k}^{\text{trap}}(\omega) = 2q^2 \int dt \langle s_{i,k}(0) s_{j,k}(t) \rangle \exp(-i\omega t) = 2q^2 \delta_{ij} N_{t,k} [(1 - f_k)c_{i,k} + f_k e_{i,k}] \quad (608)$$

That is, different capture or emission processes are independent, and the noise source for a given process is twice the total trapping rate (the sum of the capture and emission rates). The model also assumes that different trap distributions are independent, so that their noise contributions add.

Random Dopant Fluctuations

The noise sources for random dopant fluctuations are activated by the `Doping` keyword. The noise source for acceptor fluctuations reads:

$$K_A^{\text{RDF}}(\vec{r}_1, \vec{r}_2, \omega) = N_A(\vec{r}_1) \frac{\Theta(0.5\text{Hz} - |\nu|)}{1\text{Hz}} \delta(\vec{r}_1 - \vec{r}_2) \quad (609)$$

Here, $\nu = \omega/2\pi$ is the frequency. An analogous expression holds for the noise source for donors, K_D^{RDF} . Physically, the noise sources are static. However, to avoid a δ -function in frequency space, Sentaurus Device spreads the spectral density of the noise source over a 1 Hz frequency interval. [Eq. 609](#) is based on the assumption that individual dopant atoms are completely uncorrelated.

Acceptors and donors are considered to be independent. Their contributions to the noise spectral densities add.

By default, Sentaurus Device neglects the impact of the random dopant fluctuations on mobility and bandgap narrowing. To take their impact into account, specify the `Mobility` and `BandGapNarrowing` options to the `Doping` keyword. For example:

```
Physics { Noise( Doping(Mobility) ) }
```

activates the random dopant fluctuation noise source, taking into account the impact of the fluctuations on mobility.

Random Geometric Fluctuations

The geometric fluctuation model accounts for random displacements of insulator–insulator and semiconductor–insulator interfaces. The noise source reads:

$$K^{\text{geo}}(\vec{r}_1, \vec{r}_2, \omega) = \frac{\Theta(0.5\text{Hz} - |\nu|)}{1\text{Hz}} \langle \delta s(\vec{r}_1) \delta s(\vec{r}_2) \rangle \quad (610)$$

where $\nu = \omega/2\pi$ is the frequency, and the correlation of the displacements δs is given by:

$$\langle \delta s(\vec{r}_1) \delta s(\vec{r}_2) \rangle = \hat{n}(\vec{r}_1) \cdot \hat{n}(\vec{r}_2) [a_{\text{iso}}(\vec{r}_1) + |\vec{a}(\vec{r}_1) \cdot \hat{n}(\vec{r}_1)|] [a_{\text{iso}}(\vec{r}_2) + |\vec{a}(\vec{r}_2) \cdot \hat{n}(\vec{r}_2)|] \exp\left[-\frac{(\vec{r}_1 - \vec{r}_2)^2}{\lambda^2}\right] \quad (611)$$

where:

- $\vec{r}_1$ and $\vec{r}_2$ are positions on the interface.
- $\hat{n}(\vec{r})$ is the local interface normal in point $\vec{r}$.
- a_{iso} (the isotropic correlation amplitude), $\vec{a}$ (the vectorial correlation amplitude), and λ (the correlation length) are adjustable parameters.

The correlation has the following noteworthy properties:

- Displacements of interface positions occur only in the interface normal direction.
- The displacement amplitude depends on the interface direction when $\vec{a}$ is nonzero.
- Displacements are spatially correlated; the correlations decay with increasing distance.
- Correlations depend on the relative normal direction. Perpendicular interfaces are uncorrelated; opposing interfaces are negatively correlated (so the actual shifts are positively correlated).

The impact of displacements is accounted for in the variation of the dielectric constant in the Poisson equation, in the variation of the space charge in the Poisson equation, and in the variation of the band-edge profile in the continuity equations. The impact through other quantities, such as tunneling rates, is currently neglected.

To use geometric fluctuations, in the global `Math` section for a device, specify one or more surfaces. Each surface has a name and is the union of an arbitrary number of interfaces and electrodes. For example:

```
Device "MOS" {
  Math {
    Surface "S2" (
      RegionInterface="channel/gateoxide"
      MaterialInterface="PolySi/Oxide"
    )
    ...
  }
}
```

```
}
```

specifies a surface S2 that consist of all poly–oxide interfaces in the device, as well as the channel/gateoxide region interface. The order in which regions or materials in the specification of interfaces are named determines the sense of interface displacements: The positive direction points from the region or material named second into the region or material named first.

NOTE Surfaces are used for purposes other than random geometric fluctuations. Therefore, they can contain electrodes and all interface types, even though random geometric fluctuations support only certain interface types.

The surfaces are then used to activate the noise sources. For example:

```
Physics {
  Noise(
    GeometricFluctuations "S1"
    GeometricFluctuations "S2"
  ) ...
} ...
} ...
```

activates geometric fluctuations for surfaces S1 and S2. Points on the same surface are correlated according to [Eq. 611](#). Points on different surfaces are uncorrelated. The contributions of different surfaces to the noise spectral densities add.

The parameters a_{iso} and $\vec{a}$ are specified as the parameters Amplitude_Iso and Amplitude in a surface-specific or an interface-specific GeometricFluctuations parameter set. Interface-specific values take precedence over surface-specific values. λ is specified as the parameter lambda in a surface-specific GeometricFluctuations parameter set. All three parameters are given in micrometers, and default to zero. a_{iso} and the components of $\vec{a}$ must be nonnegative; λ must be positive. For example:

```
RegionInterface "poly/gateoxide" {
  GeometricFluctuations {
    Amplitude = (0, 1e-4, 0)
    Amplitude_Iso = 2e-3
  } ...
}
GeometricFluctuations "S1" {
  lambda = 5e-3
  Amplitude = (9e-4, 3e-4, 0)
}
```

NOTE For 3D structures, the mesh spacing on the fluctuating interfaces must be small compared to the correlation length λ to be able to resolve the Gaussian in [Eq. 611](#). For 2D, the mesh does not need to resolve the Gaussian.

Random Trap Concentration Fluctuations

Trap concentration variations are activated by the `TrapConcentration` option to `Noise`. For each trap level, Sentaurus Device uses a noise source of the form:

$$K_{ii}^{\text{trapconc}}(\vec{r}_1, \vec{r}_2, \omega) = N_i(\vec{r}_1) \frac{\Theta(0.5\text{Hz} - |\nu|)}{1\text{Hz}} \delta(\vec{r}_1 - \vec{r}_2) \quad (612)$$

Here, N_i is the concentration for the trap level i .

All trap levels are assumed to be mutually independent. Their contributions to the noise spectral densities add.

The trap concentration fluctuation model accounts for the impact of the trap concentration on the space charge in the Poisson equation and on the trap-related generation–recombination rates in the continuity equations.

Random Workfunction Fluctuations

The random workfunction fluctuations describe the impact of the fluctuation of the workfunction at contacts on insulators and at metal–insulator interfaces. The noise source reads:

$$K^{\text{workfunction}}(\vec{r}_1, \vec{r}_2, \nu) = \frac{\Theta(0.5\text{Hz} - |\nu|)}{1\text{Hz}} a(\vec{r}_1) a(\vec{r}_2) \exp\left[-\frac{(\vec{r}_1 - \vec{r}_2)^2}{\lambda^2}\right] \quad (613)$$

where:

- $\nu = \omega/2\pi$ is the frequency.
- a is the workfunction standard deviation.
- λ is the correlation length.

NOTE The spatial correlation in [Eq. 613](#) differs from the one created by the randomization procedure used for the sIFM-based approach (see [Workfunction Variations on page 594](#)).

Workfunction fluctuations are activated by the `WorkfunctionFluctuations` option to `Noise`. This option must be followed by a string that denotes the name of a surface that consists exclusively of contacts on insulators and metal–insulator interfaces. For example:

```
Device "MOS" {
  Math {
    Surface "S3" (
      Electrode = "gate"
      MaterialInterface = "Nitride/Aluminum"
    )
  }
  Physics {
    Noise(WorkfunctionFluctuations "S3") ...
  } ...
} ...
```

The definition of surfaces is described in [Random Geometric Fluctuations on page 588](#). Any number of `WorkfunctionFluctuations` (with different names) can be specified. They are considered to be uncorrelated, and their contributions to the noise spectral densities add.

The parameter a is specified (in eV) as `Amplitude` in a surface-specific or an interface-specific `WorkfunctionFluctuations` parameter set. Interface-specific values take precedence over surface-specific values.

The correlation length λ is specified (in μm) as `lambda` in a surface-specific `WorkfunctionFluctuations` parameter set and it must be positive. For example:

```
MaterialInterface "Nitride/Aluminum" {
  WorkfunctionFluctuations {
    Amplitude = 0.4
  } ...
}
WorkfunctionFluctuations "S3" {
  lambda = 0.01
  Amplitude = 0.4
}
```

Noise from SPICE Circuit Elements

To take into account the noise generated by SPICE circuit elements (see [Compact Models User Guide, Chapter 1 on page 1](#)), specify the `CircuitNoise` option to `ACCoupled`. The form of the noise source for a particular circuit element is defined by the respective compact model.

Due to a restriction of the SPICE noise models, SPICE circuit elements contribute only to the autocorrelation noise. For cross-correlation noise, Sentaurus Device considers SPICE circuit

elements as noiseless. Non-SPICE compact circuit elements do not implement noise at all and, therefore, Sentaurus Device always treats them as noiseless.

Statistical Impedance Field Method

The statistical impedance field method (sIFM) can be applied to doping concentration, trap concentration, and the workfunction at contacts on insulators and metal–insulator interfaces. It creates a large number of randomized variations of the quantities under investigation (dopant concentrations, trap concentrations, or workfunction) and computes the modification of the device characteristics in linear response.

The sIFM is specified in the global Math section:

```
Math {
  RandomizedVariation <string> (
    NumberOfSamples = <int>
    Randomize = <int>
  )
  ...
}
```

Any number of RandomizedVariation specifications can be present, each of which specifies the sample size and the random seed for one set of variations. The sets are distinguished by the optional string after the RandomizedVariation keyword.

NumberOfSamples determines the number of variations in a set. Randomize determines the seed for the random number generator. For a value of zero (the default), the seed value is selected automatically and differently in each simulation run. A negative value disables the sIFM, and a positive value is directly taken as the seed for the random number generator. The last possibility allows to reproduce results for different simulation runs of the same device, on the same platform, and the same release of Sentaurus Device.

The quantities to be randomized are specified in a device-global Physics section:

```
Device <string> {
  Physics {
    RandomizedVariation <string> (<specifications>)
  }
}
```

The string after RandomizedVariation in the Physics section is matched with the one after RandomizedVariation in the global Math section, and <specifications> is described along with the individual variations in the following sections.

For one ACCoupled statement, for each RandomizedVariation specification, the sIFM creates two files per ObservationNode: one for voltage and one for current variation at the node. The file names are derived from the base name given by ACExtract, the string after the RandomizedVariation keyword, the name of the type of variation (current or voltage), the node name, and the extension .csv. The file contains comma separated values (CSV), a very simple format that can be imported by many applications (for example, spreadsheet programs).

The first line contains the column headings, using double-quoted strings. Each subsequent line corresponds to one bias point and one frequency.

The first couple of columns contain the data specified in the ACPlot statement. They serve to connect each line to a bias point. The rest of the columns within each line correspond to the current or voltage shift for one particular randomized profile. For all lines, the same column always corresponds to the same profile. For frequencies larger than 0.5 Hz, the shifts are zero.

Doping Variations

Doping variations are active by default for the sIFM. They are disabled using the -Doping keyword:

```
Physics {
    RandomizedVariation <string> (-Doping)
}
```

Acceptor and donor concentrations are randomized independently, assuming dopants are spatially uncorrelated and using a Poisson distribution function. This means that for a vertex i of the mesh with a box volume V_i and an average doping concentration N_i , the probability to find exactly k dopants in the box for vertex i is:

$$P_i(k) = \frac{(N_i V_i)^k}{k!} \exp(-N_i V_i) \quad (614)$$

The volume V_i is a 3D volume. If the simulated structure is 2D, the AreaFactor is taken into account to compute V_i . [Eq. 614](#) is consistent with the assumptions that are used to obtain [Eq. 609](#), [p. 587](#).

The sIFM accounts for the impact of dopant concentration on space charge, mobility, and bandgap narrowing.

When the keyword RandomizedDoping appears in a device-specific Plot section, all randomized acceptor and donor profiles are written to the Plot file. The individual profiles are identified by a number in their name. This number is the same one that appears in the first line of the .csv files.

NOTE If the number of random samples is large, plotting randomized doping profiles can occupy a large amount of memory.

Trap Concentration Variations

For the sIFM, variations of the random trap concentration are activated with the keyword `TrapConcentration`:

```
Physics {  
    RandomizedVariation <string> (TrapConcentration)  
}
```

All trap levels are randomized independently, assuming traps are spatially uncorrelated. The same expression as for doping concentration, [Eq. 614](#), [p. 593](#), is used.

The sIFM accounts for the impact of the trap concentration on the space charge in the Poisson equation and on the trap-related generation–recombination rates in the continuity equations.

Workfunction Variations

For the sIFM, workfunction variations are activated and controlled using the `Workfunction` keyword:

```
Physics {  
    RandomizedVariation <string> (  
        Workfunction(  
            Surface = <string>  
            AverageGrainSize = <float>          * in micrometers  
            GrainProbability = (<float>...)  
            GrainWorkfunction = (<float>...)    * in eV  
        )  
    )  
}
```

`Surface` specifies the name of a surface that consists of contacts on insulators and metal–insulator interfaces only. The specification of surfaces is described in [Random Geometric Fluctuations on page 588](#). The workfunction is randomized for interfaces and contacts that are part of the given `Surface`.

`AverageGrainSize` specifies the average size of a metal grain (in μm). `GrainWorkfunction` is a list of workfunction values (in eV) that can occur for a metal grain. `GrainProbability` is a list that gives the probabilities with which these workfunctions

occur. The randomization procedure is the same as described in [Metal Workfunction Randomization on page 243](#).

As the sIFM models the effect of deviations, the values of `GrainWorkfunction` are adjusted to have an average that agrees with the nominal workfunction specified for the electrode or metal. Therefore, adding the same value to all `GrainWorkfunction` values will not affect the results.

The sIFM accounts for the impact of the workfunction on the boundary condition for the electrostatic potential at metal–insulator interfaces and at contacts on insulators.

Deterministic Variations

For deterministic variations, you specify the variations directly, by specifying the actual deviation in doping, geometry, or model parameters. Sentaurus Device computes the effect of the variations on the observation node voltages and currents in a linear response. As for random fluctuations, deterministic variations are assumed to be different from zero only for analysis frequencies up to 0.5 Hz.

Compared to random fluctuations, deterministic variations give you more control over the variation and are easier to understand, because no statistical interpretation is required and no second-order moments appear. In the case of geometric variations, the computational effort is smaller as well.

Deterministic Doping Variations

Deterministic doping variations are specified as an option to `DeterministicVariation` in the Physics section:

```
Physics {
  DeterministicVariation(
    DopingVariation <name> (
      SFactor = <string>
      Amplitude = <vector>
      Amplitude_Iso = <float>
      Conc = <float>
      Factor = <float>
      Type = Doping | Acceptor | Donor
      SpatialShape = Gaussian | Uniform
      SpaceMid = <vector>
      SpaceSig = <vector>
    ) ...
  ) ...
}
```

```
}
```

Here, <name> is a string that names the variation and will be used to identify output to the ACExtract file. If it coincides with the name of a geometric or a parameter variation, the contributions of the variations are added.

SFactor specifies a scalar dataset. Allowed datasets are doping concentrations and PMIUserFields. If you do not specify any dataset, a constant density of 1 cm^{-3} is assumed.

By specifying Conc, the concentration is normalized and then multiplied by the concentration specified with Conc. If the SFactor dataset is not a PMIUserField, it is recommended to use this feature, because then you are not concerned about the units of the dataset. Conc is given in cm^{-3} . If Conc is not specified or is zero, the values from the dataset will be used unaltered.

By default, the resulting concentration X determines the fluctuation δN directly, $\delta N = X$. If you use Amplitude_Iso or Amplitude to specify nonzero isotropic or vectorial amplitudes a_{iso} and a , the fluctuation is computed from as $\delta N = \vec{a} \cdot \nabla X + a_{\text{iso}} |\nabla X|$. The vectorial amplitude models the small displacement of a profile, and the isotropic amplitude models a shift of a doping front (perpendicular to the equi-doping lines). For this to work properly, the mesh must accurately resolve the variations of X . Both amplitudes are specified in μm .

Optionally, a dimensionless value Factor can be specified, which is multiplied by the resulting δN . The factor defaults to one. Its purpose is to easily specify variations that are a certain percentage of a given doping data field.

Using the keywords SpatialShape, SpaceMid, and SpaceSig, you can multiply δN by either a Gaussian function or a window function. These keywords work in the same way as for traps (see [Energetic and Spatial Distribution of Traps on page 408](#)). The shape functions must adequately be resolved by the mesh.

Type selects whether the variation applies to the acceptor concentration ($\delta N_A = \delta N$), the donor concentration ($\delta N_D = \delta N$), or doping as a whole. In the latter case, a negative variation increases the acceptor concentration; a positive variation increases donor concentration.

For a summary of options to DopingVariation, see [Table 235 on page 1301](#).

Deterministic Geometric Variations

Deterministic geometric variations are specified as an option to DeterministicVariation in the Physics section:

```
Physics {
    DeterministicVariation(
        GeometricVariation <name> (
```

```

        Surface = <string>
        Amplitude = <vector>
        Amplitude_Iso = <float>
        SpatialShape = Gaussian
        SpaceMid = <vector>
        SpaceSig = <vector>
    ) ...
) ...
}

```

Here, <name> is a string that names the variation and will be used to identify output to the ACExtract file. If it coincides with the name of a doping or a parameter variation, the contributions of the variations are added.

Surface identifies the displaced interfaces. Its specification is described in [Random Geometric Fluctuations on page 588](#). The amount of displacement along the positive normal in a point $\vec{r}$ on the surface is determined by Amplitude_Iso (a_{iso}) and Amplitude (a), both specified in μm , as $\delta s(\vec{r}) = a_{\text{iso}}(\vec{r}) + \vec{a}(\vec{r}) \cdot \hat{n}(\vec{r})$.

Using the keywords SpatialShape, SpaceMid, and SpaceSig, you can multiply δs by a Gaussian function. These keywords work in the same way as for traps (see [Energetic and Spatial Distribution of Traps on page 408](#)). The Gaussian must be resolved adequately by the mesh.

For a summary of options of GeometricVariation, see [Table 241 on page 1304](#).

Parameter Variations

Sentaurus Device allows to compute the linear response to the variation of any parameter that can be ramped. The parameter variations are specified as an option to DeterministicVariation in the Physics section:

```

Physics {
    DeterministicVariation (
        ParameterVariation <name> (
            (
                Material = <string>
                Region = <string>
                MaterialInterface = <string>
                RegionInterface = <string>
                Model = <string>
                Parameter = <string>
                Value      = <float>
                Factor     = <float>
                Summand    = <float>
            )
        )
    )
}

```

```
    )...  
  )...  
 )...  
}
```

Here, <name> is a string that names the variation and is used to identify output to the ACExtract file. If it coincides with the name of a doping or a geometry variation, the contributions of the variations are added.

The option of `ParameterVariation` is a list of an arbitrary number of individual parameter specifications, each of which is enclosed by a pair of parentheses. All these variations are performed together to compute their cumulative impact.

`Material`, `Region`, `MaterialInterface`, or `RegionInterface` specify the location of the parameter that is varied. At most, one of these keywords must be specified. `Model` specifies the name of the model to which the varied parameter belongs, and `Parameter` is the name of the parameter within this model. All these keywords specify a parameter in the same way as for parameter ramping (see [Ramping Physical Parameter Values on page 109](#)).

To specify the correct location for a parameter can be complicated. [Combining Parameter Specifications on page 68](#) explains how the specifications in the parameter file determine the location from which the parameters used in the computation are taken.

Each varied parameter has an original value γ (the default value, or the value that was set in the parameter file). The modified value γ' can be given by one of two ways:

- Directly by specification of the modified value using `Value= $\gamma'$`
- By using the keywords `Factor` and `Summand`: $\gamma' = \text{Factor} \cdot \gamma + \text{Summand}$

Sentaurus Device assembles the right-hand side twice: once with the original parameters, and once with the modified parameters. Then, the difference in the right-hand sides is used to compute the variation of output characteristics in the linear response. However, as the right-hand sides are not linear in the parameters, the method is not perfectly linear, which becomes visible if the parameter variation $\gamma' - \gamma$ is not small. On the other hand, if the parameter variation becomes too small, numeric noise can obscure the results.

Impedance Field Method

The impedance field method splits noise and fluctuation analysis into two tasks. The first task is to provide models for the noise sources, that is, for the local microscopic fluctuations inside the devices (see [Noise Sources on page 585](#)). The selection of the appropriate models depends on the problem. You have to pick the models according to the kind of noise you are interested in. The second task is to determine the impact of the local fluctuations on the terminal

characteristics. To solve this task, the response of the contact voltage to local fluctuation is assumed to be linear. For each contact, Green's functions are computed that describe this linear relationship. In contrast to the first task, the second task is purely numeric, as the Green's functions are completely specified by the transport model.

A Green's function describes the response $G_{\xi}^V(x, x', \omega)$ of the potential at location x due to a perturbation at location x' with angular frequency ω in the right-hand side of the partial differential equation for solution variable ξ . ξ can be ϕ (see Eq. 37, p. 191), n or p (see Eq. 53, p. 197), T_n (see Eq. 71, p. 211), T_p (see Eq. 72, p. 211) or T .

The implemented models result in the following expression for the noise voltage spectral density:

$$S_V(x, x', \omega) = \int \underline{G}_n^V(x, x'', \omega) K_{n,n}^{\text{Diff}}(x'', \omega) \underline{G}_n^{V*}(x', x'', \omega) dx'' \quad (615)$$

$$+ \int \underline{G}_n^V(x, x'', \omega) K_{n,n}^{\text{GR}}(x'', \omega) \underline{G}_n^{V*}(x', x'', \omega) dx'' \quad (616)$$

$$+ \int \underline{G}_n^V(x, x'', \omega) K_{n,n}^{\text{fGR}}(x'', \omega) \underline{G}_n^{V*}(x', x'', \omega) dx'' \quad (617)$$

$$+ \int \underline{G}_p^V(x, x'', \omega) K_{p,p}^{\text{Diff}}(x'', \omega) \underline{G}_p^{V*}(x', x'', \omega) dx'' \quad (618)$$

$$+ \int \underline{G}_p^V(x, x'', \omega) K_{p,p}^{\text{GR}}(x'', \omega) \underline{G}_p^{V*}(x', x'', \omega) dx'' \quad (619)$$

$$+ \int \underline{G}_p^V(x, x'', \omega) K_{p,p}^{\text{fGR}}(x'', \omega) \underline{G}_p^{V*}(x', x'', \omega) dx'' \quad (620)$$

$$+ \int G_A^V(x, x'', \omega) K_A^{\text{RDF}}(x'', \omega) G_A^{V*}(x', x'', \omega) dx'' \quad (621)$$

$$+ \int G_D^V(x, x'', \omega) K_D^{\text{RDF}}(x'', \omega) G_D^{V*}(x', x'', \omega) dx'' \quad (622)$$

$$+ \sum_s \int \int_s G_{\text{geo}}^V(x, r_1, \omega) K_{\text{geo}}(r_1, r_2, \omega) G_{\text{geo}}^{V*}(x', r_2, \omega) dr_1 dr_2 \quad (623)$$

$$+ \int \sum_{ik} G_{\text{trap}, i, k}^V(x, x'', \omega) K_{ii, k}^{\text{trap}}(x'', \omega) G_{\text{trap}, i, k}^{V*}(x', x'', \omega) dx'' \quad (624)$$

$$+ \int \sum_{ik} G_{\text{trapconc}, i}^V(x, x'', \omega) K_{ii}^{\text{trapconc}}(x'', \omega) G_{\text{trapconc}, i}^{V*}(x', x'', \omega) dx'' \quad (625)$$

$$+ \sum_s \iint_s G_{\text{workfunction}}^V(x, r_1, \omega) K_{\text{workfunction}}(r_1, r_2, \omega) G_{\text{workfunction}}^{V*}(x', r_2, \omega) dr_1 dr_2 \quad (626)$$

where the K are the noise sources (see [Noise Sources on page 585](#)). The spatial coordinates x and x' each correspond to a contact. The dx'' integrations run over the device volume; the r_1 and r_2 integrations run over surface s .

For drift-diffusion simulations, $G_n^V = \nabla G_n^V$. For hydrodynamic simulations, G_n^V is replaced by an effective Green's function, $G_n^V = \nabla G_n^V + G_{T_n}^V \nabla E_C - 5r_n kT_n \nabla G_{T_n}^V / 2$ (see [Hydrodynamic Model for Current Densities on page 200](#) and [Hydrodynamic Model for Temperatures on page 211](#)). Similar relations hold for G_p^V .

The Green's function G_A^V for random dopant fluctuations reads:

$$G_A^V = G_n^V \frac{\partial \nabla \cdot \vec{J}_n}{\partial N_A} + G_p^V \frac{\partial \nabla \cdot \vec{J}_p}{\partial N_A} + G_{T_n}^V \frac{\partial (\nabla \cdot \vec{S}_n - \vec{J}_n \cdot \nabla E_C)}{\partial N_A} + G_{T_p}^V \frac{\partial (\nabla \cdot \vec{S}_p - \vec{J}_p \cdot \nabla E_V)}{\partial N_A} - G_\phi^V \quad (627)$$

An analogous expression holds for G_D^V . The expression above is valid for hydrodynamic simulations. The expression for drift-diffusion simulations is similar. The derivatives with respect to N_A describe the impact of dopant fluctuations on mobility and bandgap narrowing. Sentaurus Device takes them into account only if explicitly requested by you (see [Random Dopant Fluctuations on page 587](#)).

The Green's function G_{geo}^V accounts for the impact of geometric fluctuations through the dielectric constant in the Poisson equation, the space charge in the Poisson equation, and the band-edge profiles in the continuity equations.

The Green's function $G_{\text{trap}, i, k}^V$ for trap level k and the capture or emission process i is not given here in detail. Interested readers are referred to the literature [3], where full expressions for an equivalent approach are presented.

The Green's function $G_{\text{trapconc}, i}^V$ for trap level i accounts for the fluctuations of the impact trap concentration through the trap space charge in the Poisson equation and through the trap-related generation–recombination rates in the continuity equations.

The Green's function $G_{\text{workfunction}}^V$ accounts for the impact of workfunction fluctuations through the boundary condition for the electrostatic potential.

The equations for the noise current spectral densities are obtained from those for the noise voltage spectral densities by replacing the Green's functions G_ξ^V by the Green's functions G_ξ^I that denote the responses of node currents to fluctuations inside the device.

Voltage variations due to deterministic variations for a certain name are computed as:

$$\delta V(x, \omega) = \int G_{\text{geo}}^V(x, r, \omega) \delta s(r) dr + \int G_D^V(x, x'', \omega) \delta N_D dx'' + \int G_A^V(x, x'', \omega) \delta N_A dx'' \quad (628)$$

Current variations are handled analogously.

Noise Output Data

Several variables can be plotted during noise analysis. For each device, a **NoisePlot** section can be specified similar to the **Plot** section, where the data to be plotted is listed. Besides the standard data, additional noise-specific data or groups of data can be specified, as listed in [Table 104](#). In the tables, the abbreviations LNS (local noise source) and LNVSD (local noise voltage spectral density) are used.

Autocorrelation data refers to [Eq. 615](#) to [Eq. 626](#), when x and x' are identical. Data selected in the **NoisePlot** section is plotted for each device and observation node at a given frequency into a separate file. File names with the following format are used:

`<noise-plot>_<device-name>_<ob-node>_<number>_acgf_des.tdr`

where `<noise-plot>` is the prefix specified by the **NoisePlot** option to **ACCoupled**.

In the case of $x \neq x'$ in [Eq. 615](#) to [Eq. 626](#), node cross-correlation spectra are computed and integrands become complex. Data specified in the **NoisePlot** section is plotted for each device and each pair of observation nodes at a given frequency into a separate file.

The file names have the format:

`<noise-plot>_<device-name>_<ob-node-1>_<ob-node-2>_<number>_acgf_des.tdr`

Table 104 Device noise data

Keyword	Description	Reference equation
eeDiffusionLNS	Electron diffusion LNS	Eq. 604
hhDiffusionLNS	Hole diffusion LNS	
eeMonopolarGRLNS	Trace of electron monopolar GR LNS	Eq. 605
hhMonopolarGRLNS	Trace of hole monopolar GR LNS	
eeFlickerGRLNS	Trace of electron flicker GR LNS	Eq. 606
hhFlickerGRLNS	Trace of hole flicker GR LNS	
ReLNVXVSD ImLNVXVSD	Real/imaginary parts of LNVSD	Sum of integrands in Eq. 615–Eq. 624

23: Noise, Fluctuations, and Sensitivity

Noise Output Data

Table 104 Device noise data

Keyword	Description	Reference equation
ReLNISD ImLNISD	Real/imaginary parts of local noise current spectral density	Sum of integrands in Eq. 615–Eq. 624 with G_{ξ}^V replaced by G_{ξ}^I
ReeeLNVXVSD ImeeLNVXVSD	Real/imaginary parts of LNVSD for electrons	Sum of integrands in Eq. 615–Eq. 617
RehhLNVXVSD ImhhLNVXVSD	Real/imaginary parts of LNVSD for holes	Sum of integrands in Eq. 618–Eq. 620
ReeeDiffusionLNVXVSD ImeeDiffusionLNVXVSD	Real/imaginary parts of diffusion LNVSD for electrons	Integrand of Eq. 615
RehhDiffusionLNVXVSD ImhhDiffusionLNVXVSD	Real/imaginary parts of diffusion LNVSD for holes	Integrand of Eq. 618
ReeeMonopolarGRLNVXVSD ImeeMonopolarGRLNVXVSD	Real/imaginary parts of electron monopolar LNVSD	Integrand of Eq. 616
RehhMonopolarGRLNVXVSD ImhhMonopolarGRLNVXVSD	Real/imaginary parts of hole monopolar LNVSD	Integrand of Eq. 619
ReeeFlickerGRLNVXVSD ImeeFlickerGRLNVXVSD	Real/imaginary parts of electron flicker GR LNVSD	Integrand of Eq. 617
RehhFlickerGRLNVXVSD ImhhFlickerGRLNVXVSD	Real/imaginary parts of hole flicker GR LNVSD	Integrand of Eq. 620
ReTrapLNISD ImTrapLNISD	Real/imaginary parts of local trapping noise current spectral density	Integrand of Eq. 624 with G_{ξ}^V replaced by G_{ξ}^I
ReTrapLNVSD ImTrapLNVSD	Real/imaginary parts of local trapping noise voltage spectral density	Integrand of Eq. 624
PoECReACGreenFunction PoECImACGreenFunction	Real/imaginary parts of G_n^V	
PoHCreACGreenFunction PoHCImACGreenFunction	Real/imaginary parts of G_p^V	
PoETReACGreenFunction PoETImACGreenFunction	Real/imaginary parts of $G_{T_n}^V$	
PoHTReACGreenFunction PoHTImACGreenFunction	Real/imaginary parts of $G_{T_p}^V$	
PoLTrACGreenFunction PoLTImACGreenFunction	Real/imaginary parts of G_T^V	
PoPotReACGreenFunction PoPotImACGreenFunction	Real/imaginary parts of G_{ϕ}^V	
PoGeoGreenFunction CurGeoGreenFunction	Real part of G_{geo}^V and G_{geo}^I	

Table 104 Device noise data

Keyword	Description	Reference equation
CurECReACGreenFunction CurECImACGreenFunction CurHCreACGreenFunction CurHCImACGreenFunction CurETReACGreenFunction CurETImACGreenFunction CurHTReACGreenFunction CurHTImACGreenFunction CurLTReACGreenFunction CurLTImACGreenFunction CurPotReACGreenFunction CurPotImACGreenFunction	Real/imaginary parts of G_n^I , G_p^I , $G_{T_n}^I$, $G_{T_p}^I$, G_T^I , and G_ϕ^I	
GradPoECReACGreenFunction GradPoECImACGreenFunction GradPoHCreACGreenFunction GradPoHCImACGreenFunction GradPoETReACGreenFunction GradPoETImACGreenFunction GradPoHTReACGreenFunction GradPoHTImACGreenFunction	Real/imaginary parts of $\nabla \cdot G_n^V$, $\nabla \cdot G_p^V$, $\nabla \cdot G_{T_n}^V$, $\nabla \cdot G_{T_p}^V$	
Grad2PoECACGreenFunction Grad2PoHCACGreenFunction	$ G_n^V ^2$ and $ G_p^V ^2$	
AllLNS	All used LNS	
AllLNVXSD	All used LNVSD	
GreenFunctions	Green's functions and their gradients	

References

- [1] F. Bonani *et al.*, “An Efficient Approach to Noise Analysis Through Multidimensional Physics-Based Models,” *IEEE Transactions on Electron Devices*, vol. 45, no. 1, pp. 261–269, 1998.
- [2] J.-P. Nougier, “Fluctuations and Noise of Hot Carriers in Semiconductor Materials and Devices,” *IEEE Transactions on Electron Devices*, vol. 41, no. 11, pp. 2034–2049, 1994.
- [3] F. Bonani and G. Ghione, “Generation–recombination noise modelling in semiconductor devices through population or approximate equivalent current density fluctuations,” *Solid-State Electronics*, vol. 43, no 2, pp. 285–295, 1999.

23: Noise, Fluctuations, and Sensitivity

References

CHAPTER 24 Tunneling

This chapter presents the tunneling models available in Sentaurus Device.

In current microelectronic devices, tunneling has become a very important physical effect. In some devices, tunneling leads to undesired leakage currents (for gates in small MOSFETs). For other devices such as EEPROMs, tunneling is essential for the operation of the device.

The tunneling models discussed in this chapter refer to elastic charge-transport processes at interfaces or contacts. Tunneling also plays a role in some generation–recombination models (see [Chapter 16 on page 359](#)). These models do not deal with spatial transport of charge and, therefore, are not discussed here. In addition to tunneling, hot-carrier injection can also contribute to carrier transport across barriers. To model hot-carrier injection, see [Chapter 25 on page 625](#). Tunneling to traps is discussed in [Tunneling and Traps on page 417](#).

Tunneling Model Overview

Sentaurus Device offers three tunneling models. The most versatile tunneling model is the nonlocal tunneling model (see [Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612](#)). This model:

- Handles arbitrary barrier shapes.
- Includes carrier heating terms.
- Allows you to describe tunneling between the valence band and conduction band.
- Offers several different approximations for the tunneling probability.

Use this model to describe tunneling at Schottky contacts, tunneling in heterostructures, and gate leakage through thin, stacked insulators.

The second most powerful model is the direct tunneling model (see [Direct Tunneling on page 608](#)). This model:

- Assumes a trapezoidal barrier (this restricts the range of application to tunneling through insulators).
- Neglects heating of the tunneling carriers.
- Optionally, accounts for image charge effects (at the cost of reduced numeric robustness).

24: Tunneling

Fowler–Nordheim Tunneling

Use this model to describe leakage through thin gate insulators, provided those are of uniform or of uniformly graded composition.

The simplest tunneling model is the Fowler–Nordheim model (see [Fowler–Nordheim Tunneling](#)). Fowler–Nordheim tunneling is a special case of tunneling also covered by the nonlocal and direct tunneling models, where tunneling is to the conduction band of the oxide. The model is simple and efficient, and has proven useful to describe erase operations in EEPROMs, which is the application for which this model is recommended.

If the simulated device contains a floating gate, gate currents are used to update the charge on the floating gate after each time step in transient simulations. If EEPROM cells are simulated in 2D, it is generally necessary to include an additional coupling capacitance between the control and floating gates to account for the additional influence of the third dimension on the capacitance between these electrodes (see [Floating Metal Contacts on page 225](#)). The additional floating gate capacitance can be specified as FGcap in the `Electrode` statement (see [Physical Models and the Hierarchy of Their Specification on page 57](#)).

Fowler–Nordheim Tunneling

Using Fowler–Nordheim

To switch on the Fowler–Nordheim tunneling model, specify `Fowler` as an option to `GateCurrent` in an appropriate interface `Physics` section, as follows:

```
Physics(MaterialInterface="Silicon/Oxide"){ GateCurrent(Fowler) }
```

or:

```
Physics(MaterialInterface="Silicon/Oxide"){ GateCurrent(Fowler(EVB)) }
```

The second specification activates the tunneling of electrons from and to the valence band as discussed in [Fowler–Nordheim Model on page 607](#).

If `GateCurrent` is specified as above, Sentaurus Device computes gate currents between all silicon–oxide interfaces and the electrodes. The computation can be restricted to selected interfaces using region-interface `Physics` sections.

For simple one-gate devices, the tunneling current can be monitored on a contact specified by the keyword `GateName` in the `GateCurrent` statement.

The Fowler–Nordheim tunneling model can be used with any of the hot-carrier injection models (see [Chapter 25 on page 625](#)), for example:

```
GateCurrent( Fowler Lucky )
```

switches on the Fowler–Nordheim tunneling model and the classical lucky electron injection model simultaneously.

Fowler–Nordheim Model

The Fowler–Nordheim model reads:

$$j_{\text{FN}} = AF_{\text{ins}}^2 \exp\left(-\frac{B}{F_{\text{ins}}}\right) \quad (629)$$

where j_{FN} is the tunnel current density, F_{ins} is the insulator electric field at the interface, and A and B are physical constants. The electric field at the interface shown by Tecplot SV is an interpolation of the fields in both regions, but Sentaurus Device uses the insulator field internally.

Due to the large energy difference between oxide and silicon conduction bands, tunneling electrons create electron–hole pairs in the erase operation when they enter the semiconductor. This additional carrier generation is included by an energy-independent multiplication factor $\gamma > 1$:

$$j_n = \gamma \cdot j_{\text{FN}} \quad (630)$$

$$j_p = (\gamma - 1) \cdot j_{\text{FN}} \quad (631)$$

If the electrons flow in an opposite direction, by default, $\gamma = 1$. The formulas above reflect the default behavior of Sentaurus Device, but sometimes the Fowler–Nordheim equation is used to emulate other tunneling effects, for example, the tunneling of electrons from the valence band into the gate. Such capability is activated by the additional keyword EVB in the command file. Sentaurus Device will continue to function even if $\gamma < 1$. For any electron tunneling direction, particularly a floating body SOI, this tunneling current is important because it strongly defines floating body potential. Different coefficients are needed for the write and erase operations because, in the first case, the electrons are emitted from monocrystalline silicon and, in the latter case, they are emitted from the polysilicon contact into the oxide.

The tunneling current is implemented as a current boundary condition at the interface where the current is produced. For transient simulations, if the `Interface` keyword is also present in the `GateCurrent` section, carrier tunneling with explicitly evaluated boundary conditions for continuity equations is activated (similar to carrier injection with explicitly evaluated

boundary conditions for continuity equations in [Carrier Injection with Explicitly Evaluated Boundary Conditions for Continuity Equations on page 647](#)).

Fowler–Nordheim Parameters

The parameters of the Fowler–Nordheim model can be modified in the `FowlerModel` parameter set. Sentaurus Device uses different parameters (denoted as `erase` and `write`) depending on the direction of the electric field between the contact and semiconductor in the oxide layer. For example, if the field points from the contact to the semiconductor (that is, electrons flow into the contact), the ‘write’ parameter set is used.

[Table 105](#) lists the parameters and their default values.

Table 105 Coefficients for Fowler–Nordheim tunneling (defaults for silicon–oxide interface)

Symbol	Parameter name	Default value	Unit	Remarks
A (erase)	Ae	1.87×10^{-7}	A/V^2	A for the erase cycle
B (erase)	Be	1.88×10^8	V/cm	B for the erase cycle
A (write)	Aw	1.23×10^{-6}	A/V^2	A for the write cycle
B (write)	Bw	2.37×10^8	V/cm	B for the write cycle
γ	Gm	1	1	

Direct Tunneling

Direct tunneling is the main gate leakage mechanism for oxides thinner than 3 nm. It turns into Fowler–Nordheim tunneling at oxide fields higher than approximately 6 MV/cm, independent of the oxide thickness. This section describes a fully quantum-mechanical tunneling model that is restricted to trapezoidal tunneling barriers and covers both direct tunneling and the Fowler–Nordheim regime. Optionally, the model considers the reduction of the tunneling barrier due to image forces.

Using Direct Tunneling

The direct tunneling model is specified as an option of the `GateCurrent` statement (see [Using Fowler–Nordheim on page 606](#)) in an appropriate interface `Physics` section:

```
Physics(MaterialInterface="Silicon/Oxide") {  
    GateCurrent( DirectTunneling )  
}
```

The keyword `GateName` and the compatibility with the hot-carrier injection models (see [Chapter 25 on page 625](#)) are as for the Fowler–Nordheim model, see [Fowler–Nordheim Tunneling on page 606](#).

To switch on the image force effect, the parameters $E0$, $E1$, and $E2$ must be specified (in eV) in the parameter file. If these values are all equal (the default), the image force is neglected. Recommended values for both electrons and holes are $E0=0$, $E1=0.3$, and $E2=0.7$. Including the image force effect degrades convergence. For most purposes, the effect can be approximated by reducing the overall barrier height. For more information on model parameters, see [Direct Tunneling Parameters on page 611](#).

To plot the direct tunneling current, the keywords `eDirectTunneling` or `hDirectTunneling` must be included in the `Plot` section:

```
Plot {eDirectTunneling hDirectTunneling}
```

Direct Tunneling Model

This section summarizes the model described in the literature [1]. It presents the formulas for electrons only; the hole expressions are analogous. The electron tunneling current density is:

$$j_n = \frac{qm_C k}{2\pi^2 \hbar^3} \int_0^\infty dE \Upsilon(E) \left\{ T(0) \ln \left(\exp \left[\frac{E_{F,n}(0) - E_C(0) - E}{kT(0)} \right] + 1 \right) - T(d) \ln \left(\exp \left[\frac{E_{F,n}(d) - E_C(0) - E}{kT(d)} \right] + 1 \right) \right\} \quad (632)$$

where d is the effective thickness of the barrier, m_C is a mass prefactor, the argument 0 denotes one (the ‘substrate’) side of the barrier and d denotes the other (‘gate’) side, and E is the energy of the elastic tunnel process (relative to $E_C(0)$).

$\Upsilon(E) = 2/(1 + g(E))$ is the transmission coefficient for a trapezoidal potential barrier, with:

$$g(E) = \frac{\pi^2}{2} \left\{ \sqrt{\frac{E_T}{E}} \sqrt{\frac{m_{Si}}{m_G}} (Bi'_d Ai_0 - Ai'_d Bi_0)^2 + \sqrt{\frac{E}{E_T}} \sqrt{\frac{m_G}{m_{Si}}} (Bi_d Ai'_0 - Ai_d Bi'_0)^2 + \frac{\sqrt{m_G m_{Si}}}{m_{ox}} \frac{\hbar \Theta_{ox}}{\sqrt{E E_T}} (Bi'_d Ai'_0 - Ai'_d Bi'_0)^2 + \frac{m_{ox}}{\sqrt{m_G m_{Si}}} \frac{\sqrt{E E_T}}{\hbar \Theta_{ox}} (Bi_d Ai_0 - Ai_d Bi_0) \right\} \quad (633)$$

and:

$$Ai_0 = Ai \left(\frac{E_B(E) - E}{\hbar \Theta_{ox}} \right), \quad Ai_d = Ai \left(\frac{E_B(E) - qF_{ox}d - E}{\hbar \Theta_{ox}} \right) \quad (634)$$

and so on, where $\hbar\Theta_{\text{ox}} = (q^2\hbar^2F_{\text{ox}}^2/2m_{\text{ox}})^{1/3}$ and $E_B(E)$ denotes the (substrate-side) barrier height for electrons of energy E . E_T is the tunneling energy with respect to the conduction band edge on the gate side, $E_T = E - E_C(d) - E_C(0)$.

For compatibility with an earlier implementation of the model, E_T is truncated to the value specified by the parameter `E_F_M` if it exceeds this value. $F_{\text{ox}} = V_{\text{ox}}/d$ is the electric field in the oxide (assumed to be uniform within the oxide, and including a band edge-related term when different barrier heights are specified at the two insulator interfaces). The quantities m_G , m_{ox} , and m_{Si} represent the electron masses in the three materials, respectively. A_i and B_i are Airy functions, and A_i' and B_i' are their derivatives.

Image Force Effect

Ultrathin oxide barriers are affected by the image force effect. If the latter is neglected, $E_B(E)$ is the bare barrier height E_B , which is an input parameter. The image force effect is included in the model by taking $E_B(E)$ as an energy-dependent pseudobarrier:

$$E_B(E) = E_B(E_0) + \frac{E_B(E_2) - E_B(E_0)}{(E_2 - E_0)(E_1 - E_2)}(E - E_0)(E_1 - E) - \frac{E_B(E_1) - E_B(E_0)}{(E_1 - E_0)(E_1 - E_2)}(E - E_0)(E_2 - E) \quad (635)$$

where E_0 , E_1 , and E_2 are chosen in the lower energy range of the barrier potential (between 0 eV and 1.5 eV in practical cases). If these values are chosen to be equal, the image force effect is automatically switched off. Otherwise, for each bias point:

$$S_{\text{tra}}(E) = S_{\text{im}}(E) \quad (636)$$

is solved for $E_j (j = 0, 1, 2)$, which results in three pseudobarrier heights $E_B(E_j)$ used in Eq. 635. In Eq. 636, $S_{\text{tra}}(E)$ is the action of the trapezoidal pseudobarrier:

$$S_{\text{tra}}(E) = \frac{2}{3} \left| \left[\frac{E_B(E) - qF_{\text{ox}}d - E}{\hbar\Theta_{\text{ox}}} \right]^3 \Theta[E_B(E) - qF_{\text{ox}}d - E] - \left[\frac{E_B(E) - E}{\hbar\Theta_{\text{ox}}} \right]^3 \right| \quad (637)$$

and $S_{\text{im}}(E)$ is the action of the respective image potential barrier:

$$S_{\text{im}}(E) = \sqrt{\frac{2m_{\text{ox}}}{\hbar^2}} \int_{x_l(E)}^{x_r(E)} d\xi \sqrt{E_B - qF_{\text{ox}}\xi + E_{\text{im}}(\xi) - E} \quad (638)$$

$x_{l,r}(E)$ denotes the classical turning points for a given carrier energy that follow from:

$$E_T - qF_{\text{ox}}x_{l,r} + E_{\text{im}}(x_{l,r}) = E \quad (639)$$

For the thickness of the pseudobarrier, $d = x_i(0) - x_l(0)$ is used, that is, the distance between the two turning points at the energy of the semiconductor conduction band edge of the interface. Therefore, d is smaller than the user-given oxide thickness, when the image force effect is considered. The image potential itself is given by:

$$E_{\text{im}}(x) = \frac{q^2}{16\pi\epsilon_{\text{ox}}} \sum_{n=0}^{10} (k_1 k_2)^n \left[\frac{k_1}{nd+x} + \frac{k_2}{d(n+1)-x} + \frac{2k_1 k_2}{d(n+1)} \right] \quad (640)$$

with:

$$k_1 = \frac{\epsilon_{\text{ox}} - \epsilon_{\text{Si}}}{\epsilon_{\text{ox}} + \epsilon_{\text{Si}}}, \quad k_2 = \frac{\epsilon_{\text{ox}} - \epsilon_{\text{G}}}{\epsilon_{\text{ox}} + \epsilon_{\text{G}}} = -1 \quad (641)$$

In [Eq. 641](#), ϵ_{ox} , ϵ_{Si} , and ϵ_{G} denote the dielectric constants for the oxide, substrate, and gate material, respectively.

Direct Tunneling Parameters

The parameters of the direct tunneling model are modified in the `DirectTunneling` parameter set. The appropriate default parameters for an oxide barrier on silicon are in [Table 106](#). The parameters are specified according to the interface.

Table 106 Coefficients for direct tunneling (defaults for oxide barrier on silicon)

Symbol	Parameter name	Electrons	Holes	Unit	Description
ϵ_{ox}	eps_ins	2.13	2.13	1	Optical dielectric constant
$E_{\text{F}}(d)$	E_F_M	11.7	11.7	eV	Fermi energy for gate contact
m_{G}	m_M	1	1	m_0	Effective mass in gate contact
m_{ox}	m_ins	0.50	0.77	m_0	Effective mass in insulator
m_{Si}	m_s	0.19	0.16	m_0	Effective mass in substrate
m_{C}	m_dos	0.32	0	m_0	Semiconductor DOS effective mass
E_{B}	E_barrier	3.15	4.73	eV	Semiconductor/insulator barrier (no image force)
E_0, E_1, E_2	E0, E1, E2	0	0	eV	Energy nodes 0, 1, and 2 for pseudobarrier calculation

If tunneling occurs between two semiconductors, the coefficients for metal (m_{M} and $E_{\text{F_M}}$) are not used. The semiconductor parameters for the respective interface are used. It is not valid to have contradicting parameter sets for two interfaces of an insulator between which tunneling

24: Tunneling

Nonlocal Tunneling at Interfaces, Contacts, and Junctions

occurs. In particular, `eps_ins`, `m_ins`, `E0`, `E1`, and `E2` must agree in the parameter specifications for both interfaces.

Nonlocal Tunneling at Interfaces, Contacts, and Junctions

The tunneling current depends on the band edge profile along the entire path between the points connected by tunneling. This makes tunneling a nonlocal process. In general, the band edge profile has a complicated shape, and Sentaurus Device must compute it by solving the transport equations and the Poisson equation. The model described here takes this dependence fully into account.

To use the nonlocal tunneling model:

1. Construct a special purpose ‘nonlocal’ mesh (see [Defining Nonlocal Meshes](#)).
2. Specify the physical details of the tunneling model (see [Specifying Nonlocal Tunneling Model on page 614](#)).
3. Adjust the physical and numeric parameters (see [Nonlocal Tunneling Parameters on page 615](#)).

Defining Nonlocal Meshes

It is necessary to specify a special purpose ‘nonlocal’ mesh for each part of the device for which you want to use the nonlocal tunneling model. Nonlocal meshes consist of ‘nonlocal’ lines that represent the tunneling paths for the carriers.

To control the construction of the nonlocal mesh, use the options of the keyword `NonLocal` in the global `Math` section. Nonlocal meshes can be constructed using two different forms:

- In the simpler form, `NonLocal` specifies barrier regions. For example:

```
Math { Nonlocal "NLM" ( Barrier(Region="oxtop" Region="oxbottom") ) }
```

constructs a nonlocal mesh for a tunneling barrier that consists of the regions `oxtop` and `oxbottom`. See [Specification Using Barrier on page 169](#) for details.

- In the more general form, `NonLocal` specifies an interface or a contact where the mesh is constructed, and a distance `Length` (given in centimeters). Sentaurus Device then connects semiconductor vertices with a distance from the interface or contact no larger than `Length` to the interface or contact. These connections form the centerpiece of the nonlocal lines. For example, with:

```
Math { Nonlocal "NLM" (Electrode="Gate" Length=5e-7) }
```

Sentaurus Device constructs a nonlocal mesh called "NLM" that connects vertices up to a distance of 5 nm to the Gate electrode. See [Specification Using a Reference Surface on page 169](#) for details.

Sentaurus Device introduces a coordinate along each nonlocal line. The interface or contact is at coordinate zero; the vertex for which the nonlocal line is constructed is at a positive coordinate. Below, the terms **upper** and **lower** refer to the orientation according to these nonlocal line-specific coordinates. This orientation is not related to the orientation of the mesh axes.

To form the nonlocal lines, Sentaurus Device extends the connection at the upper end, to fully cover the box for the vertex that is connected. **Optionally, for nonlocal meshes at interfaces, the connection can be extended at the lower end (beyond the interface) by an amount specified by Permeation (in centimeters).**

Sentaurus Device handles each nonlocal line separately. Therefore, two nonlocal lines connecting two vertices to the same interface do not allow the computing of tunneling between these vertices. **To compute tunneling between vertices on the two sides of an interface, they must be covered by a single nonlocal line. This is where Permeation is needed.**

For nonlocal meshes not used for trap tunneling, *Permeation* also affects the integration range for tunneling. With zero *Permeation*, the lower endpoint for tunneling is always at the interface; for positive *Permeation*, it can be anywhere on the nonlocal line. Therefore, for tunneling at smooth barriers (for example, band-to-band tunneling at a steep p-n junction), the nonlocal mesh used should not be reused for trap tunneling (see [Tunneling and Traps on page 417](#)), and *Permeation* should be positive.

When using the more general form of nonlocal mesh construction, specify a nonlocal mesh for only one side of a tunneling barrier. If on one side of the tunneling barrier there is a contact or a metal–nonmetal interface, specify the mesh there. **In other cases, specify the nonlocal mesh for the interface from which the carriers will tunnel away. If this side can change during the simulation, specify a nonzero Permeation (approximately as much as Length exceeds the barrier thickness).** For tunneling barriers with multiple layers, do not specify a nonlocal mesh for the interfaces internal to the barrier.

For more options to the keyword `NonLocal`, see [Table 179 on page 1257](#). For details about the nonlocal mesh and its construction, see [Constructing Nonlocal Meshes on page 168](#).

NOTE The nonlocality of tunneling can increase dramatically the time that is needed to solve the linear systems in the Newton iteration. To limit the speed degradation, keep *Length* and *Permeation* as small as possible. Consider optimizing the nonlocal mesh using the advanced options of `NonLocal` (see [Constructing Nonlocal Meshes on page 168](#)).

Specifying Nonlocal Tunneling Model

The nonlocal tunneling model is activated and controlled in the global `Physics` section. Each tunneling event connects two points on a nonlocal line. Sentaurus Device distinguishes between tunneling to the conduction band and to the valence band at the lower of the two points.

To switch on these terms, use the options `eBarrierTunneling` and `hBarrierTunneling`. For example:

```
Physics {
    eBarrierTunneling "NLM" hBarrierTunneling "NLM"
}
```

switches on electron and hole tunneling for the nonlocal mesh called "NLM", and:

```
Physics {
    eBarrierTunneling "NLM" (PeltierHeat)
}
```

switches on tunneling to the conduction band only and activates the Peltier heating of the tunneling particles.

By default, all tunneling to the conduction band at the lower point on the line originates from the conduction band at the upper point, $j_C = j_{CC}$ ('electron tunneling'), see [Eq. 651, p. 621](#). Likewise, by default, the tunneling to the valence band at the lower point originates from the valence band at the upper point, $j_V = j_{VV}$ (that is, 'hole tunneling').

In addition, Sentaurus Device supports nonlocal band-to-band tunneling. To include the contributions by tunneling to and from the valence band at the upper point j_C , specify `eBarrierTunneling` with the `Band2Band=Simple` option. Then, $j_C = j_{CC} + j_{CV}$, see [Eq. 651](#) and [Eq. 653, p. 622](#). To include contributions by tunneling to and from the conduction band at the upper point to j_V , specify `hBarrierTunneling` with the `Band2Band=Simple` option. Then, $j_V = j_{VC} + j_{VV}$. With `Band2Band=Full`, the nonlocal band-to-band tunneling model uses [Eq. 406, p. 396](#) instead of [Eq. 652, p. 621](#) to calculate the band-to-band tunneling contribution, which makes the model consistent with the [Dynamic Nonlocal Path Band-to-Band Model on page 395](#) provided that the dynamic nonlocal path band-to-band model uses the direct tunneling model with the Franz dispersion relation. With `Band2Band=UpsideDown` (or only `Band2Band`), an obsolete, physically flawed, band-to-band tunneling model is activated.

For backward compatibility, you can activate the nonlocal tunneling model in an interface-specific or contact-specific `Physics` section; in that case, specify `eBarrierTunneling` and `hBarrierTunneling` as options to `Recombination`, and omit the nonlocal mesh name. The

specification applies to an unnamed nonlocal mesh that has been constructed for the same location (see [Unnamed Meshes on page 172](#)).

Figure 46 illustrates the four contributions to the total tunneling current.

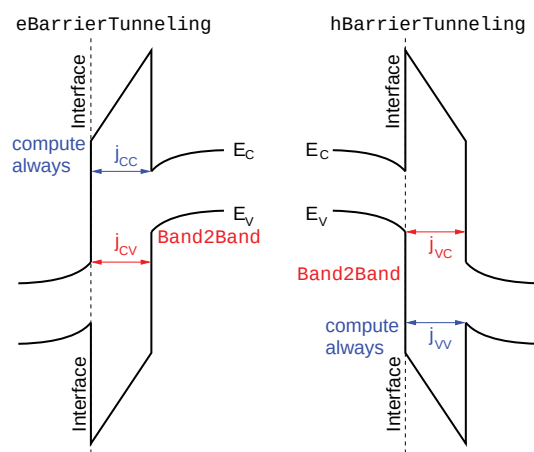


Figure 46 Various nonlocal tunneling current contributions

By default, Sentaurus Device assumes a single-band parabolic band structure for the tunneling particles, uses a WKB-based model for the tunneling probability, and ignores band-to-band tunneling and Peltier heating. Options to `eBarrierTunneling` and `hBarrierTunneling` override the default behavior. For available options, see [Table 187 on page 1270](#). For a detailed discussion of the physics of the nonlocal tunneling model, see [Physics of Nonlocal Tunneling Model on page 618](#).

Nonlocal Tunneling Parameters

The nonlocal tunneling model has several fit parameters. They are specified in the `BarrierTunneling` parameter set.

The parameter pair `g` determines the dimensionless prefactors g_C and g_V (see [Eq. 650](#) and [Eq. 652](#)). For unnamed meshes, specify them in the parameter set that is specific to the contact or interface for which the nonlocal tunneling model is activated. For named meshes, specify them in the global parameter set.

The parameter pair `mt` determines the tunneling masses m_C and m_V (see [Eq. 642](#) and [Eq. 643](#)). They are properties of the materials that form the tunneling barrier. Therefore, specify them (in units of m_0) in region-specific or material-specific parameter sets. For tunneling at contacts, also specify the masses for the contact parameter set when using `Transmission` or `Schroedinger` (see [Eq. 647, p. 619](#) and [Schrödinger Equation-based Tunneling Probability on page 620](#)).

24: Tunneling

Nonlocal Tunneling at Interfaces, Contacts, and Junctions

Sentaurus Device treats effective tunneling masses of value zero as undefined. If an effective tunneling mass for a region is undefined, Sentaurus Device uses the effective tunneling mass for the interface or contact for which tunneling is activated (for unnamed meshes) or the tunneling mass from the global parameter set (for named meshes). By default, all effective tunneling masses are undefined.

`eoffset` and `hoffset` are lists of nonnegative energy shifts (in eV) that are added to the conduction-band and valence-band edges, shifting them outwards, away from the midgap. Specify them in a region-specific or material-specific parameter set. By default, a single shift value of zero is assumed. Sentaurus Device computes the tunneling current as the sum of the currents for each shift. Shifts in different regions are combined according to the position where they appear in the list; if the lists in different regions have different lengths, the last value in the shorter lists are repeated to extend them to the length of the longest list.

The parameter pair `alpha` determines the fit factors α_n and α_p for the quantization corrections (see [Density Gradient Quantization Correction on page 620](#)). The default values are zero, disabling the corrections. To enable them, set the parameters to one (or another positive value) in the interface- or contact-specific parameter set (for unnamed meshes) or in the global parameter set (for named meshes).

For example:

```
BarrierTunneling {
    mt = 0.5, 0.5
    g = 1 , 2
}
Material = "Oxide" {
    BarrierTunneling {
        mt = 0.42 , 1.0
    }
}
```

changes the prefactors g_C and g_V to 1 and 2, respectively. The tunneling masses are set to $0.5m_0$. In oxide, it sets the tunneling masses m_C and m_V to 0.42 and 1, respectively; these values take precedence over the masses specified globally.

`BarrierTunneling` can also be applied specifically to named nonlocal meshes. For example:

```
Material = "Oxide" {
    BarrierTunneling "NLM" {
        mt = 0.42 , 1.0
    }
}
```

changes the oxide tunneling mass for the nonlocal mesh NLM only. Specifications for named nonlocal meshes take precedence over the general specifications.

Specify numeric parameters for the model in the `Math` section specific to the interface or contact to which the nonlocal tunneling model is applied. The parameter `EnergyResolution(NonLocal)` (given in eV) is a lower limit for the energy step that Sentaurus Device uses to perform integrations over band-edge energies.

The parameter `Digits(Nonlocal)` determines the relative accuracy (the number of valid decimal digits) to which Sentaurus Device computes the tunneling currents. The default value for `Digits(Nonlocal)` is 2, whereas for `EnergyResolution(NonLocal)`, it is 0.005.

For example:

```
Math {
  NonLocal "NLM" (
    ...
    Digits=3
    EnergyResolution=0.001
  )
}
```

increases the energy resolution for tunneling on the nonlocal mesh "NLM" to 1 meV and the relative accuracy of the tunneling current computation to three digits.

Visualizing Nonlocal Tunneling

To visualize nonlocal tunneling, specify the keyword `eBarrierTunneling` or `hBarrierTunneling` in the `Plot` section (see [Device Plots on page 144](#)). The quantities plotted are in units of $\text{cm}^{-3}\text{s}^{-1}$ and represent the rate at which electrons and holes are generated or disappear due to tunneling. To plot the Peltier heat generated in the conduction band and valence band due to nonlocal tunneling, specify `eNLLTunnelingPeltierHeat` and `hNLLTunnelingPeltierHeat`. The Peltier heat is plotted in units of Wcm^{-3} and is available regardless of whether it is accounted for in the temperature equation.

The rates are plotted vertexwise and are averaged over the semiconductor volume controlled by a vertex. Therefore, they depend on the mesh spacing. This dependence can become particularly strong at interfaces where the band edges change abruptly.

Physics of Nonlocal Tunneling Model

The nonlocal tunneling model in Sentaurus Device is based on the approach presented in the literature [2], but provides significant enhancements over the model presented there.

WKB Tunneling Probability

The computation of the tunneling probabilities $\Gamma_{C,v}$ (for carriers tunneling to the v -th shifted conduction band at the interface or contact) and $\Gamma_{V,v}$ (for tunneling to the v -th shifted valence band) is, by default, based on the WKB approximation. The WKB approximation uses the local (imaginary) wave numbers of particles at position r and with energy ε :

$$\kappa_{C,v}(r, \varepsilon) = \sqrt{2m_C(r)[E_{C,v}(r) - \varepsilon]} \Theta[E_{C,v}(r) - \varepsilon] / \hbar \quad (642)$$

$$\kappa_{V,v}(r, \varepsilon) = \sqrt{2m_V(r)[\varepsilon - E_{V,v}(r)]} \Theta[\varepsilon - E_{V,v}(r)] / \hbar \quad (643)$$

Here, m_C is the conduction-band tunneling mass and m_V is the valence-band tunneling mass. Both tunneling masses are adjustable parameters (see [Nonlocal Tunneling Parameters on page 615](#)). $E_{C,v}$ and $E_{V,v}$ are the conduction and valence bands energies shifted by the v -th value in `eoffset` and `hoffset` (see [Nonlocal Tunneling Parameters on page 615](#)).

Using the local wave numbers and the interface transmission coefficients $T_{CC,v}$ and $T_{VV,v}$, the tunneling probability between positions l and $u > l$ for a particle with energy ε can be written as:

$$\Gamma_{CC,v}(u, l, \varepsilon) = T_{CC,v}(l, \varepsilon) \exp\left(-2 \int_l^u \kappa_{C,v}(r, \varepsilon) dr\right) T_{CC,v}(u, \varepsilon) \quad (644)$$

and:

$$\Gamma_{VV,v}(u, l, \varepsilon) = T_{VV,v}(l, \varepsilon) \exp\left(-2 \int_l^u \kappa_{V,v}(r, \varepsilon) dr\right) T_{VV,v}(u, \varepsilon) \quad (645)$$

If the option `TwoBand` is specified to `eBarrierTunneling` (see [Table 187 on page 1270](#)), Sentaurus Device replaces $\kappa_{C,v}$ in [Eq. 644](#) with the two-band dispersion relation:

$$\kappa_v = \frac{\kappa_{C,v} \kappa_{V,v}}{\sqrt{\kappa_{C,v}^2 + \kappa_{V,v}^2}} \quad (646)$$

If the option `TwoBand` is specified to `hBarrierTunneling`, Sentaurus Device replaces $\kappa_{v,v}$ in Eq. 645 with the two-band relation Eq. 646. Near the conduction and the valence band edge, the two-band dispersion relation approaches the single band dispersion relations Eq. 642 and Eq. 643, and provides a smooth interpolation in between. Figure 47 illustrates this.

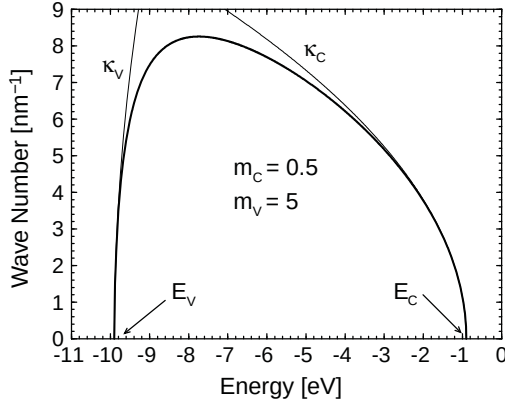


Figure 47 Comparison of two-band and single-band dispersion relations

The two-band dispersion relation does not distinguish between electrons and holes. In particular, for the two-band dispersion relation, $\Gamma_{CC,v} = \Gamma_{VV,v}$.

The two-band dispersion relation is most useful when band-to-band tunneling is active (keyword `Band2Band`, see Table 187 on page 1270). However, the two-band dispersion relation can be used independently from band-to-band tunneling.

By default, the interface transmission coefficients $T_{CC,v}$ and $T_{VV,v}$ in Eq. 644 and Eq. 645 equal one. If the `Transmission` option is specified to `eBarrierTunneling` [3]:

$$T_{CC,v}(x, \epsilon) = \frac{v_{-,v}(x, \epsilon) \sqrt{v_{-,v}(x, \epsilon)^2 + 16v_{+,v}(x, \epsilon)^2}}{v_{+,v}(x, \epsilon)^2 + v_{-,v}(x, \epsilon)^2} \quad (647)$$

Here, $v_{-,v}(x, \epsilon)$ denotes the velocity of a particle with energy ϵ on the side of the interface or contact at position x where the particle moves freely in the conduction band, and $v_{+,v}(x, \epsilon)$ denotes the imaginary velocity on the side of the tunneling barrier (where the particle is in the gap). If the particle is free or in the barrier on both sides, $T_{CC,v}(x, \epsilon) = 1$. Velocities are the derivatives of the particle energy with respect to the wave number, $v_v = |\partial\epsilon/\partial\hbar\kappa_v|$. With the `TwoBand` option, $v_{+,v} = |\partial\epsilon/\partial\hbar\kappa_v|$ [4] (see Eq. 646). If the `Transmission` option is specified to `hBarrierTunneling`, analogous expressions hold for $T_{VV,v}$.

By default, the band-to-band tunneling probabilities $\Gamma_{CV,v}$ and $\Gamma_{VC,v}$ are also given by the expressions Eq. 644 and Eq. 645, respectively. When the `TwoBand` and `Transmission` options are specified for `eBarrierTunneling` to compute $\Gamma_{CV,v}$, $v_{-,v}$ in the expression for $T_{CC,v}(r, \epsilon)$ (see Eq. 647) is the velocity of the particle in the valence band and is computed

24: Tunneling

Nonlocal Tunneling at Interfaces, Contacts, and Junctions

from valence-band parameters. The converse holds for $\Gamma_{VC,v}$ when those options are specified for `hBarrierTunneling`.

For metals and contacts, the band-edge energy to compute the interface transmission coefficients is obtained from the `FermiEnergy` parameter of the `BandGap` section for the metal or contact. The masses used to compute the velocities are the tunneling effective masses for the metal or contact.

Schrödinger Equation–based Tunneling Probability

In addition to the WKB-based models discussed in [WKB Tunneling Probability on page 618](#), Sentaurus Device can compute tunneling probabilities based on the Schrödinger equation. For the Schrödinger equation–based model, Sentaurus Device computes the tunneling probability $\Gamma_{CC,v}(E)$ (or $\Gamma_{VV,v}(E)$) by solving the 1D Schrödinger equation:

$$\left(-\frac{d}{dr} \frac{1}{m(r)} \frac{d}{dr} + E_{C,v}(r)\right) \Psi(r) = E(r) \Psi(r) \quad (648)$$

between the outermost classical turning points that belong to the tunneling energy E . For $m(r)$, Sentaurus Device uses the tunneling mass `mt` (see [Nonlocal Tunneling Parameters on page 615](#)). $E_{C,v}$ is the conduction-band energy shifted by the v -th value in `eoffset` (see [Nonlocal Tunneling Parameters on page 615](#)).

For boundary conditions, Sentaurus Device assumes incident and reflected plane waves outside the barrier on one side, and an evanescent plane wave on the other side. The energy for the plane waves is the greater of kT_n and $E - E_{C,v}$, where T_n and $E_{C,v}$ are taken at the point immediately outside the barrier on the respective side. The masses outside the barrier are the tunneling masses `mt` that are valid there.

Density Gradient Quantization Correction

In combination with the density gradient model (see [Density Gradient Quantization Model on page 288](#)), the nonlocal tunneling model offers an optional correction for quantization effects. The corrections are multipliers to $T_{CC,v}(r, \epsilon)$ and read:

$$\exp\left(\alpha_n \frac{\Lambda_{fb,n}(r) - \Lambda_n(r)}{kT_n(r)}\right) \quad (649)$$

where α_n is an adjustable parameter (see [Nonlocal Tunneling Parameters on page 615](#)), and $\Lambda_{fb,n}$ is the solution of the 1D density gradient equations for flatband (that is, $\phi = \text{const}$) conditions. A multiplier analogous to that in [Eq. 649](#) exists for $T_{VV,v}$.

Nonlocal Tunneling Current

For a point at u , the expression for the net conduction band electron recombination rate due to tunneling to and from the v -th shifted conduction band at point $l < u$ with energy ε is:

$$R_{CC,v}(u, l, \varepsilon) - G_{CC,v}(u, l, \varepsilon) = \frac{A_{CC}}{qk} \vartheta\left[\varepsilon - E_{C,v}(u), -\frac{dE_{C,v}}{du}(u)\right] \vartheta\left[\varepsilon - E_{C,v}(l), \frac{dE_{C,v}}{dl}(l)\right] \Gamma_{CC,v}(u, l, \varepsilon) \times \quad (650)$$

$$\left[T_n(u) \ln\left(1 + \exp\left[\frac{E_{F,n}(u) - \varepsilon}{kT_n(u)}\right]\right) - T_n(l) \ln\left(1 + \exp\left[\frac{E_{F,n}(l) - \varepsilon}{kT_n(l)}\right]\right) \right]$$

where $\vartheta(x, y) = \delta(x)|y|\Theta(y)$, $A_{CC} = g_C A_0$ is the effective Richardson constant (with A_0 the Richardson constant for free electrons), and g_C is a fit parameter (see [Nonlocal Tunneling Parameters on page 615](#)). $R_{CC,v}$ relates to the first term and $G_{CC,v}$ to the second term in the second line of [Eq. 650](#). $\Gamma_{CC,v}$ is the tunneling probability (see [Eq. 644](#)). For nonlocal lines with zero Permeation, the second ϑ -function is replaced with $\Theta[\varepsilon - E_{C,v}(l)]\delta(l - 0^-)$ (with 0^- infinitesimally smaller than 0). For contacts, metals, or in the presence of the option BandGap, the second ϑ -function is replaced with $\delta(l - 0^-)$.

The current density of electrons that tunnel from the conduction band at points above l to the conduction band at point l is the integral over the recombination rate ([Eq. 650](#)):

$$j_{CC}(l) = -q \sum_v \int_{l-\infty}^{\infty} \int_{-\infty}^{\infty} [R_{CC,v}(u, l, \varepsilon) - G_{CC,v}(u, l, \varepsilon)] d\varepsilon du \quad (651)$$

The options for hBarrierTunneling and the expressions for the valence band to valence band tunneling current density j_{VV} are analogous.

Band-to-Band Contributions to Nonlocal Tunneling Current

When you specify the Band2Band option to eBarrierTunneling (see [Table 187 on page 1270](#)), the total current that flows to the conduction band at point l also contains electrons that originate from the valence band at position $u > l$. For Band2Band=Simple, the recombination rate of the valence-band electrons with energy ε at point u (in other words, the generation rate of holes at u) due to tunneling to or from the v -th shifted conduction band at l is:

$$R_{CV,v}(u, l, \varepsilon) - G_{CV,v}(u, l, \varepsilon) = \frac{A_{CV}}{2qk} \vartheta\left[\varepsilon - E_{V,v}(u), \frac{dE_{V,v}}{du}(u)\right] \vartheta\left[\varepsilon - E_{C,v}(l), \frac{dE_{C,v}}{dl}(l)\right] \Gamma_{CV,v}(u, l, \varepsilon) \times \quad (652)$$

$$[T_p(u) + T_n(l)] \left[\left(1 + \exp\left[\frac{\varepsilon - E_{F,p}(u)}{kT_p(u)}\right]\right)^{-1} - \left(1 + \exp\left[\frac{\varepsilon - E_{F,n}(l)}{kT_n(l)}\right]\right)^{-1} \right]$$

where $A_{CV} = \sqrt{g_C g_V} A_0$. The prefactor g_V is a fit parameter, see [Nonlocal Tunneling Parameters on page 615](#). $\Gamma_{CV,v}$ is the band-to-band tunneling probability discussed above. The

24: Tunneling

Nonlocal Tunneling at Interfaces, Contacts, and Junctions

modifications for zero Permeation, contacts, metals, and the BandGap option are as for [Eq. 650](#).

The current density of electrons that tunnel from the valence band at points above l to the conduction band at point l is the integral over the recombination rate ([Eq. 652](#)):

$$j_{CV}(l) = -q \sum_v \int_{l-\infty}^{\infty} \int_{-\infty}^{\infty} [R_{CV,v}(r, l, \epsilon) - G_{CV,v}(r, l, \epsilon)] d\epsilon dr \quad (653)$$

For band-to-band tunneling processes, the energy of the tunneling particles often lies deep in the gap of the barrier. The single-band dispersion relations that Sentaurus Device uses by default (see [Eq. 642](#) and [Eq. 643](#)) are based on the band structure near the band edges, and may not be useful in this regime. The two-band dispersion relation according [Eq. 646](#) is a better choice. To use the two-band dispersion relation, specify the option `TwoBand` to `eBarrierTunneling` (see [Table 187 on page 1270](#)).

The options for `hBarrierTunneling` and the expressions for the conduction band to valence band tunneling current density j_{VC} are analogous.

Carrier Heating

In hydrodynamic simulations, carrier transport leads to energy transport and, therefore, to heating or cooling of electrons and holes. The energy transport has a convective and a Peltier part. By default, Sentaurus Device ignores the Peltier part. To include the Peltier terms for the tunneling particles, specify the option `PeltierHeat` to `eBarrierTunneling` or `hBarrierTunneling` (see [Table 187 on page 1270](#)).

The convective part of the heat generation in the conduction and valence bands at position u due to tunneling of carriers with energy ϵ to and from the v -th shifted conduction band at $l < u$ is approximated as:

$$H_{\text{conv},CC,v}(u, l, \epsilon) = \frac{\delta}{2} [G_{CC,v}(u) k T_n(l) - R_{CC,v}(u) k T_n(u)] \quad (654)$$

and:

$$H_{\text{conv},CV,v}(u, l, \epsilon) = \frac{\delta}{2} [R_{CV,v}(u) k T_p(u) - G_{CV,v}(u) k T_n(l)] \quad (655)$$

By default, Sentaurus Device neglects Peltier heating and uses $\delta = 3$, which corresponds to the three degrees of freedom of the carriers. If Peltier heating is included in a simulation, the convective contribution due to one degree of freedom is already contained in the Peltier heating term. Therefore, in this case, Sentaurus Device uses $\delta = 2$. The convective parts of the heat generation in the conduction band at l , due to tunneling of carriers of energy ϵ from the

conduction band and the valence band at $u > l$, are $-H_{\text{conv,CC},v}(u, l, \varepsilon)$ and $-H_{\text{conv,CV},v}(u, l, \varepsilon)$. The expressions for the convective part of the heat generation in the valence band are analogous.

If the computation of Peltier heating is activated (see [Table 187 on page 1270](#)), Sentaurus Device computes additional heating terms. The Peltier part of the heat generation in the electron system at point u due to tunneling to and from the v -th shifted conduction band at point $l < u$ is:

$$H_{\text{Pelt,CC},v}(u, l, \varepsilon) = [E_C(u^+) - \varepsilon][R_{\text{CC},v}(u, l, \varepsilon) - G_{\text{CC},v}(u, l, \varepsilon)] \quad (656)$$

where u^+ is infinitesimally larger than u . [Eq. 656](#) vanishes everywhere except at abrupt jumps of the band edge. The Peltier part of the heat generation in the electron system at point l due to tunneling from the conduction band at $u > l$ obeys an equation similar to [Eq. 656](#), with $E_C(u^+) - \varepsilon$ replaced by $\varepsilon - E_C(l)$.

When the Band2Band option is used (see [Table 187 on page 1270](#)), Sentaurus Device also takes into account the band-to-band terms of the Peltier part of the heat generation at point u :

$$H_{\text{Pelt,CV},v}(u, l, \varepsilon) = [E_V(u^+) - \varepsilon][R_{\text{CV},v}(u, l, \varepsilon) - G_{\text{CV},v}(u, l, \varepsilon)] \quad (657)$$

and, similarly, for the Peltier heat generation at point l .

The expressions for $H_{\text{Pelt,VV},v}$ and $H_{\text{Pelt,VC},v}$ are analogous to those for conduction band tunneling.

References

- [1] A. Schenk and G. Heiser, "Modeling and simulation of tunneling through ultra-thin gate dielectrics," *Journal of Applied Physics*, vol. 81, no. 12, pp. 7900–7908, 1997.
- [2] M. Jeong *et al.*, "Comparison of Raised and Schottky Source/Drain MOSFETs Using a Novel Tunneling Contact Model," in *IEDM Technical Digest*, San Francisco, CA, USA, pp. 733–736, December 1998.
- [3] F. Li *et al.*, "Compact Model of MOSFET Electron Tunneling Current Through Ultra-thin SiO₂ and High-k Gate Stacks," in *Device Research Conference*, Salt Lake City, UT, USA, pp. 47–48, June 2003.
- [4] L. F. Register, E. Rosenbaum, and K. Yang, "Analytic model for direct tunneling current in polycrystalline silicon-gate metal–oxide–semiconductor devices," *Applied Physics Letters*, vol. 74, no. 3, pp. 457–459, 1999.

24: Tunneling

References

CHAPTER 25 Hot-Carrier Injection Models

This chapter discusses the hot-carrier injection models used in Sentaurus Device.

Hot-carrier injection is a mechanism for gate leakage. The effect is especially important for write operations in EEPROMs. Sentaurus Device provides different built-in hot-carrier injection models and a PMI for user-defined models:

- Classical lucky electron injection (Maxwellian energy distribution)
- Fiegna's hot-carrier injection (non-Maxwellian energy distribution)
- SHE distribution hot-carrier injection (non-Maxwellian energy distribution calculated from the spherical harmonics expansion (SHE) of the Boltzmann transport equation)
- Hot-carrier injection PMI (see [Hot-Carrier Injection on page 1125](#))

Overview

To activate the hot-carrier injection models for electrons (or holes), use the `eLucky` (`hLucky`), `eFiegna` (`hFiegna`), `eSHEDistribution` (`hSHEDistribution`), or `PMIModel_name(electron)` (`PMIModel_name(hole)`) options to the `GateCurrent` statement in an interface-specific Physics section.

To activate the models for both carrier types, use the `Lucky`, `Fiegna`, `SHEDistribution`, or `PMIModel_name()` options. The hot-carrier injection models can be combined with all tunneling models (see [Chapter 24 on page 605](#)).

NOTE The SHE distribution hot-carrier injection model calculates the tunneling component together with the thermionic emission term. Therefore, combining it with other tunneling models can result in double-counting of the tunneling component.

The meaning of a specification in the global Physics section is the same as for the Fowler–Nordheim model (see [Fowler–Nordheim Tunneling on page 606](#)).

Destination of Injected Current

The destination of the injection current depends on the user selection and the material properties of the hot interface (interface source for hot carriers).

When the hot interface is a semiconductor–insulator interface, the injection current is sent nonlocally across the insulator region to an associated closest vertex. For each vertex of a hot interface, Sentaurus Device searches for an associated closest vertex located on a contact or semiconductor–insulator interface. The contacts or semiconductor–insulator interfaces used in the searching algorithm must be connected through adjacent insulator regions to the hot interface. Then, each vertex of the hot interface is associated with the closest vertex on a valid contact or semiconductor–insulator interface. Each vertex of the hot interface has either an associated contact vertex or an associated semiconductor–insulator interface vertex (see Figure 48).

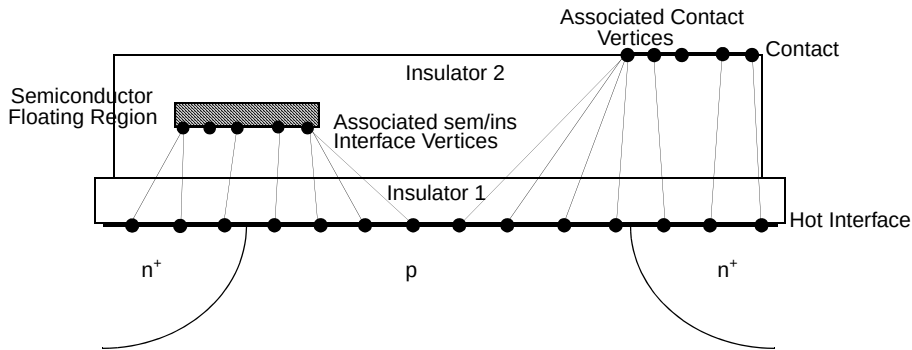


Figure 48 Mapping of hot interface vertices to associated contact or semiconductor–insulator interface vertices

When the hot interface is an interface between a semiconductor and a wide-bandgap semiconductor, you can select the destination. By default, the wide-bandgap semiconductor is treated as an insulator region, and the injection current is sent nonlocally across the region to the associated closest vertex as described for semiconductor–insulator interfaces. By using Thermionic(HCI) in the Physics section of the hot interface and specifying the injection region destination (the wide-bandgap semiconductor region) using the InjectionRegion option in the GateCurrent section, the points on the hot interface are made double-points and the injection current is injected locally in the same location where it was produced. The injected current on the wide-bandgap semiconductor side of the hot interface becomes the current boundary condition for the continuity equations solved in the region.

In the case where hot carriers are injected into semiconductor floating regions during transient simulations, the way the charge is added to the floating region is determined by the existence of a charge boundary condition (a region with a charge contact) associated with the region. If the charge boundary condition has been specified for a semiconductor floating region, the charge is added as a total charge update, using the integral boundary condition as defined by

Eq. 116, p. 227. If the charge boundary condition is not specified for the semiconductor floating region, the charge is added as an interface boundary condition for continuity equations (see [Carrier Injection with Explicitly Evaluated Boundary Conditions for Continuity Equations on page 647](#)). The total hot-carrier injection current added to semiconductor floating regions where the charge boundary condition is specified will be displayed on the electrode associated with the region (floating contact).

Floating semiconductor regions with a charge boundary condition inside a wide-bandgap semiconductor region are made possible by generalizing the concept of a semiconductor floating well.

Sentaurus Device defines a *semiconductor floating well* as a continuous zone of the same doping semiconductor regions in contact with each other and marked in the command file by a charge contact connected to one of regions in the zone. The concept is generalized by treating the inner semiconductor region (embedded floating region) as a separate well. Geometrically, the charge contact is defined on the interface between the inner and outer semiconductor regions. You must indicate in the **Electrode** section of the charge contact using either the **Region** or **Material** keyword which region is to be used as the floating region with a charge boundary condition.

Equilibrium boundary conditions are imposed on the surface of the special floating region previously described. The interface between the inner and outer semiconductor regions is treated as a heterointerface. You must specify the keyword **Heterointerface** in the **Physics** section of the interface between the inner and outer regions. If the inner region is the floating region, the boundary condition for continuity equations at the interface between the two regions, on the outer-region side, will be equilibrium for carrier concentrations. The charge update for the inner floating region is computed as the total current flowing through the floating region boundary (integral of current density over the floating region surface) multiplied by the time step.

Equilibrium carrier concentrations in a point on the double-point interface, on the outer-region side, are computed based on the carrier concentration on the inner-region side of the interface adjusted by $(N_{C,V}^{WB}/N_{C,V}^{FG}) \exp(-\delta E_{C,V}/kT)$ where $\delta E_{C,V}$ is the jump in the conduction band for electrons or in the valence band for holes, and $N_{C,V}^{WB}/N_{C,V}^{FG}$ is the ratio between density-of-states in the two regions for electrons and holes, respectively. For example, to have a **PolySi** nanocrystal floating region with a charge boundary condition embedded in an outer **OxideAsSemiconductor** region, with the charge contact "fg1" geometrically somewhere on the interface between the two regions, you must specify:

```
Electrode {
    ...
    {Name "fg1" Charge= -1e-18 Material="PolySi"}
    ...
}
```

```

Physics(MaterialInterface="OxideAsSemiconductor/PolySi") {
    Heterointerface                                # required by GeneralizedFG
}

```

Metal floating gates inside a wide-bandgap semiconductor region are allowed as well. In this case, equilibrium boundary conditions are imposed for the carrier densities at the interface between the semiconductor and the metal floating gate (on metal floating contact). Electrostatic potential at the metal floating contact is determined by the charge on the metal floating gate according to [Eq. 114, p. 225](#). The equilibrium quasi-Fermi potential on the metal floating contact is given by $\Phi = \Phi_n = \Phi_p = \phi + \phi_{MS}$, where ϕ_{MS} is the workfunction difference between the metal and the semiconductor. Carrier densities at the metal floating contact are determined then by ϕ_{MS} . The charge update for the metal floating gate is computed as the total current flowing through the metal floating region contact multiplied by the time step.

To activate this special case of the metal floating gate, you specify the `Meta l` keyword in the `Electrode` section of the metal floating contact and give the value of the workfunction (which determines ϕ_{MS}):

```

Electrode {
    ...
    {Name "fg1" Charge=0 Metal Workfunction=4.1}
    ...
}

```

The workfunction can be used as a calibration parameter.

For simple one-gate devices where the sole purpose is to evaluate the hot-carrier current injected across the oxide layer into a semiconductor region, the keyword `GateName` must be specified in the `GateCurrent` statement. In this case, the hot-carrier current is displayed on the electrode specified by `GateName` for both quasistationary and transient simulations.

Injection Barrier and Image Potential

All hot-carrier injection models are implemented as a postprocessing computation after each Sentaurus Device simulation point. The lucky electron model and Fiegna model specify some properties of semiconductor–insulator interfaces. The most important parameter is the height of the Si–SiO₂ barrier (E_B).

The height is a function of the insulator field F_{ins} and, at any point along the interface, it can be written as:

$$E_B = \begin{cases} E_{B0} - \alpha q |F_{\text{ins}}|^{\frac{1}{2}} - \beta q |F_{\text{ins}}|^{\frac{2}{3}} - P_{\perp} V_{\text{sem}} & F_{\text{ins}} < 0 \\ E_{B0} - \alpha q |F_{\text{ins}}|^{\frac{1}{2}} - \beta q |F_{\text{ins}}|^{\frac{2}{3}} - P_{\perp} V_{\text{sem}} + V_{\text{ins}} & F_{\text{ins}} > 0 \end{cases} \quad (658)$$

where E_{B0} is the zero field barrier height at the semiconductor–insulator interface. The second term in the equation represents barrier lowering due to the image potential. The third term of the barrier lowering is due to the tunneling processes. For the Si–SiO₂ interface, $\alpha = 2.59 \times 10^{-4} \text{ V}^{1/2} \text{ cm}^{1/2}$. There is a large deviation in the literature for the value of β , so it can be considered a fitting parameter. The fourth term $P_{\perp} V_{\text{sem}}$ appears only in the lucky electron model where $P_{\perp}$ is a model parameter ($P_{\perp} = 1$ by default), and V_{sem} represents the kinetic energy gain when carriers travel a distance y to the interface without losing any energy. In the Fiegna model, V_{sem} is zero.

The insulator field F_{ins} is defined as:

$$F_{\text{ins}} = \vec{F}_{\text{ins}} \cdot \hat{n} \left[1 - P_{\text{total}} + P_{\text{total}} \frac{|\vec{F}_{\text{ins}}|}{|\vec{F}_{\text{ins}} \cdot \hat{n}|} \right] \quad (659)$$

where $\hat{n}$ is a normal vector along the direction of hot-carrier injection, and P_{total} is a model parameter ($P_{\text{total}} = 0$ by default).

In addition, all hot-carrier models contain a probability P_{ins} of scattering in the image-force potential well:

$$P_{\text{ins}} = \begin{cases} \exp\left(-\frac{x_0}{\lambda_{\text{ins}}}\right) & F_{\text{ins}} < 0 \\ \exp\left(-\frac{t_{\text{ins}} - x_0}{\lambda_{\text{ins}}}\right) P_{\text{repel}} & F_{\text{ins}} > 0 \end{cases} \quad (660)$$

where λ_{ins} is the scattering mean free path in the insulator, t_{ins} is the distance between the vertex at the hot interface and the closest associated vertex, P_{repel} is a model parameter ($P_{\text{repel}} = 1$ by default), and the distance x_0 is given as:

$$x_0 = \min\left(\sqrt{\frac{q}{16\pi\epsilon_{\text{ins}}|F_{\text{ins}}|}}, \frac{t_{\text{ins}}}{2}\right) \quad (661)$$

In the above expression, $\epsilon_{\text{ins}} = \epsilon_{\text{ins}}(\epsilon_{\text{sem}} + \epsilon_{\text{ins}})/(\epsilon_{\text{sem}} - \epsilon_{\text{ins}})$ is the effective dielectric constant of the insulator where ϵ_{sem} and ϵ_{ins} are the high-frequency dielectric constant of the semiconductor and insulator, respectively. In the lucky electron model and Fiegna model, ϵ_{ins}

is directly accessible from the parameter file. In the SHE distribution model, ϵ_{sem} and ϵ_{ins} are separately set from the parameter file.

Effective Field

The lucky electron model and Fiegna model have an effective electric field F_{eff} as a parameter. In Sentaurus Device, there are three possibilities to calculate the effective field:

- With the electric field parallel to the carrier flow (switched on by the keyword `Eparallel`, which is default for hot carrier currents). See [Driving Force on page 387](#).
- With recomputation of the carrier temperature of the hydrodynamic simulation (switched on by the keyword `CarrierTempDrive`). See [Avalanche Generation with Hydrodynamic Transport on page 387](#).
- With a simplified approach (compared to the second method): The drift-diffusion model is used for the device simulation, and carrier temperature is estimated as the solution of the simplified and linearized energy balance equation. As this is a postprocessing calculation, the keyword `CarrierTempPost` activates this option.

These keywords are parameters of the model keywords. For example, the lucky electron model looks like `eLucky(CarrierTempDrive)`. However, you must remember that if the model includes the keyword `CarrierTempDrive`, `Hydro` and a carrier temperature calculation must be specified in the `Physics` section.

Classical Lucky Electron Injection

The classical total lucky electron current from an interface to a gate contact can be written as [1]:

$$I_g = \iint J_n(x, y) P_s P_{\text{ins}} \left(\int_{E_B}^{\infty} P_{\epsilon} P_r d\epsilon \right) dx dy \quad (662)$$

where P_s is the probability that the electron will travel a distance y to the interface without losing any energy, $P_{\epsilon} d\epsilon$ is the probability that the electron has energy between ϵ and $\epsilon + d\epsilon$, P_{ins} is the probability of scattering in the image force potential well (Eq. 660), and P_r is the probability that the electron will be redirected. These probabilities are given by the expressions:

$$P_r(\epsilon) = \frac{1}{2\lambda_r} \left(1 - \sqrt{\frac{E_B}{\epsilon}} \right) \quad (663)$$

$$P_s(y) = \exp\left(-\frac{y}{\lambda}\right) \quad (664)$$

$$P_\varepsilon(\varepsilon) = \frac{1}{\lambda F_{\text{eff}}} \exp\left(-\frac{\varepsilon}{\lambda F_{\text{eff}}}\right) \quad (665)$$

where λ is the scattering mean free path in the semiconductor, λ_r is redirection mean free path, F_{eff} is the effective electric field, see [Effective Field on page 630](#). E_B is the height of the semiconductor–insulator barrier. The model coefficients and their defaults are given in [Table 107](#).

They can be changed in the parameter file in the section:

```
LuckyModel { ... }
```

Table 107 Default coefficients for lucky electron model

Symbol	Parameter name (Electrons)	Default value (Electrons)	Parameter name (Holes)	Default value (Holes)	Unit
λ	eLsem	8.9×10^{-7}	hLsem	1.0×10^{-7}	cm
λ_{ins}	eLins	3.2×10^{-7}	hLins	3.2×10^{-7}	cm
λ_r	eLsemR	6.2×10^{-6}	hLsemR	6.2×10^{-6}	cm
E_{B0}	eBar0	3.1	hBar0	4.7	eV
α	eBL12	2.6×10^{-4}	hBL12	2.6×10^{-4}	$(\text{V} \cdot \text{cm})^{1/2}$
β	eBL23	3.0×10^{-5}	hBL23	3.0×10^{-5}	$(\text{V} \cdot \text{cm}^2)^{1/3}$
ε_{ins}	eps_ins	3.1	eps_ins	3.1	1
$P_\perp$	Pvertical	1	Pvertical	1	1
P_{repel}	Prepel	1	Prepel	1	1
P_{total}	Ptotal	0	Ptotal	0	1

Fiegna Hot-Carrier Injection

The total hot-carrier injection current according to the Fiegna model [\[2\]](#) can be written as:

$$I_g = q \int P_{\text{ins}} \left(\int_{E_{B0}}^{\infty} v_\perp(\varepsilon) f(\varepsilon) g(\varepsilon) d\varepsilon \right) ds \quad (666)$$

25: Hot-Carrier Injection Models

Fiegna Hot-Carrier Injection

where ϵ is the electron energy, E_{B0} is the height of the semiconductor–insulator barrier, $v_{\perp}$ is the velocity normal to the interface, $f(\epsilon)$ is the electron energy distribution, $g(\epsilon)$ is the density-of-states of the electrons, P_{ins} is the probability of scattering in the image force potential well as described by [Eq. 660](#), and $\int ds$ is an integral along the semiconductor–insulator interface.

The following expression for the electron energy distribution was proposed for a parabolic and an isotropic band structure, and equilibrium between lattice and electrons:

$$f(\epsilon) = A \exp\left(-\chi \frac{\epsilon^3}{F_{\text{eff}}^{1.5}}\right) \quad (667)$$

Therefore, the gate current can be rewritten as:

$$I_g = q \frac{A}{3\chi} \int P_{\text{ins}} n \frac{F_{\text{eff}}^{3/2}}{\sqrt{E_B}} e^{-\frac{\chi E_B^3}{F_{\text{eff}}^{3/2}}} ds \quad (668)$$

where F_{eff} is an effective field (see [Effective Field on page 630](#)).

The coefficients and their defaults are given in [Table 108](#).

Table 108 Coefficients for Fiegna model

Symbol	Parameter name (Electrons)	Default value (Electrons)	Parameter name (Holes)	Default value (Holes)	Unit
A	eA	4.87×10^4	hA	4.87×10^4	cm/s/eV ^{2.5}
χ	eChi	1.3×10^8	hChi	1.3×10^8	(V/cm/eV) ^{1.5}
λ_{ins}	eLins	3.2×10^{-7}	hLins	3.2×10^{-7}	cm
E_{B0}	eBar0	3.1	hBar0	4.7	eV
α	eBL12	2.6×10^{-4}	hBL12	2.6×10^{-4}	(V · cm) ^{1/2}
β	eBL23	1.5×10^{-5}	hBL23	1.5×10^{-5}	(V · cm ²) ^{1/3}
ϵ_{ins}	eps_ins	3.1	eps_ins	3.1	1
P_{repel}	Prepel	1	Prepel	1	1
P_{total}	Ptotal	0	Ptotal	0	1

The above coefficients can be changed in the parameter file in the FiegnaModel section.

SHE Distribution Hot-Carrier Injection

To obtain the hot-carrier injection current, accurate knowledge of the nonequilibrium electron-energy distribution is required. The spherical harmonics expansion (SHE) distribution hot-carrier injection model calculates the hot-carrier injection current using the nonequilibrium energy distribution f obtained from the lowest-order SHE of the semiclassical Boltzmann transport equation (BTE) (see [Spherical Harmonics Expansion Method on page 634](#)).

The total hot-carrier injection current is obtained from [3]:

$$I_g = \frac{2qAg_v}{4} \int P_{\text{ins}} \left[\int_0^\infty g(\epsilon) v(\epsilon) f(\epsilon) \left(\int_0^1 \Gamma \left[\epsilon - \frac{\hbar^3 g(\epsilon) v(\epsilon) x}{8\pi m_{\text{ins}}} \right] dx \right) d\epsilon \right] ds \quad (669)$$

where:

- A is a dimensionless prefactor ($A = 1$ by default).
- g_v is the valley degeneracy factor.
- P_{ins} is the probability of electrons moving from the interface to the barrier peak without scattering (see [Eq. 660, p. 629](#)).
- g is the density-of-states per valley and per spin.
- v is the magnitude of the electron velocity.
- Γ is the transmission coefficient obtained from the WKB approximation including the image-potential barrier-lowering.
- m_{ins} is the insulator effective mass.
- $\int ds$ is an integral along the semiconductor–insulator interface.

The transmission coefficient can be written as:

$$\Gamma(\epsilon_\perp) = \exp \left(-\frac{2}{\hbar} \int_0^{t_{\text{ins}}} \sqrt{2m_{\text{ins}} [E_B(r) - \epsilon_\perp]} \Theta [E_B(r) - \epsilon_\perp] dr \right) \quad (670)$$

$$E_B(r) = E_{B0} + qF_{\text{ins}}r + E_{\text{im}}(r) \quad (671)$$

$$E_{\text{im}}(r) = -\frac{q^2}{16\pi\epsilon_{\text{ins}}} \sum_{n=0}^{\infty} \left(\frac{\epsilon_{\text{sem}} - \epsilon_{\text{ins}}}{\epsilon_{\text{sem}} + \epsilon_{\text{ins}}} \right)^{2n+1} \left[\frac{1}{nt_{\text{ins}} + r} + \frac{1}{(n+1)t_{\text{ins}} - r} - \frac{2}{(n+1)t_{\text{ins}}} \left(\frac{\epsilon_{\text{sem}} - \epsilon_{\text{ins}}}{\epsilon_{\text{sem}} + \epsilon_{\text{ins}}} \right) \right] \quad (672)$$

where E_{B0} is the barrier height.

To use the SHE distribution hot-carrier injection model, you must obtain the distribution function by solving the SHE method (see [Spherical Harmonics Expansion Method on page 634](#)).

Eq. 672 represents the image-potential barrier-lowering. You can switch off the image-potential barrier-lowering using:

```
Physics(MaterialInterface="Silicon/Oxide") { ...
    GateCurrent( eSHEDistribution( -ImagePotential ) )
}
```

Spherical Harmonics Expansion Method

The spherical harmonics expansion (SHE) method computes the microscopic carrier energy distribution function $f(\vec{r}, \epsilon)$ by solving the lowest-order SHE of the Boltzmann transport equation (BTE) [4]:

$$-\nabla \cdot \left[\frac{v^2(\vec{r}, \epsilon_t)}{3} \tau(\vec{r}, \epsilon_t) g(\vec{r}, \epsilon_t) \nabla f(\vec{r}, \epsilon_t) \right] = g(\vec{r}, \epsilon_t) s(\vec{r}, \epsilon_t) \quad (673)$$

where:

- f is the occupation probability of electrons (f can be larger than one as nondegenerate statistics is assumed).
- ϵ_t is the total energy including the conduction band energy E_C and the kinetic energy ϵ .
- v is the magnitude of the electron velocity.
- $1/\tau$ is the total scattering rate.
- g is the density-of-states per valley and per spin.
- s is the net in-scattering rate due to inelastic scattering and generation–recombination processes.

In Eq. 673, $g(\epsilon)$ and $v(\epsilon)$ can be obtained from the energy–wavevector dispersion relation, $\epsilon_b(\vec{k})$, of semiconductors:

$$g_v g(\epsilon) = g_v \sum_{b=1}^{N_b} g_b(\epsilon) = \sum_{b=1}^{N_b} \int_{\epsilon} \frac{1}{4\pi^2 h v_b(\vec{k})} d\vec{s}_k \quad (674)$$

$$g_v g(\epsilon) v^2(\epsilon) = g_v \sum_{b=1}^{N_b} g_b(\epsilon) v_b^2(\epsilon) = \sum_{b=1}^{N_b} \int_{\epsilon} \frac{v_b(\vec{k})}{4\pi^2 h} d\vec{s}_k \quad (675)$$

where:

- $\epsilon = \epsilon_t - E_c(\vec{r})$ is the kinetic energy.
- g_v is the valley degeneracy.
- b is the band index.
- N_b is the number of bands.
- $v_b(\vec{k})$ is the magnitude of wavevector-dependent group velocity.
- h is Planck's constant.
- The integration is over the equienergy surface of the first Brillouin zone.

In addition, the energy-dependent square wavevector is defined as:

$$g_v k^2(\epsilon) = g_v \sum_{b=1}^{N_b} k_b^2(\epsilon) = \sum_{b=1}^{N_b} \int \frac{1}{4\pi} ds_{\vec{k}} \quad (676)$$

These energy-dependent, band structure-related quantities can be obtained either from the precalculated band-structure file or from the single band, analytic, nonparabolic, band-structure model. For more information on the band-structure file, see [Using Spherical Harmonics Expansion Method on page 638](#).

The analytic nonparabolic band-structure model gives [5]:

$$\frac{v^2(\epsilon)}{3} = \frac{2\epsilon(1 + \alpha\epsilon)}{3m_c(1 + 2\alpha\epsilon)^2} \quad (677)$$

$$g(\epsilon) = \frac{2\pi(2m_n)^{3/2}}{h^3} [\epsilon(1 + \alpha\epsilon)]^{1/2} (1 + 2\alpha\epsilon) \quad (678)$$

$$k^2(\epsilon) = \pi h g(\epsilon) v(\epsilon) \quad (679)$$

where α is the nonparabolicity factor, m_c is the conductivity effective mass, and m_n is the density-of-states effective mass.

The total scattering rate and the net in-scattering rate can be written as:

$$\frac{1}{\tau(\epsilon)} = \frac{1}{\tau_c(\epsilon)} + \frac{1}{\tau_{ii}(\epsilon)} + \frac{1}{\tau_{ac}(\epsilon)} + \frac{1}{\tau_{ope}(\epsilon)} + \frac{1}{\tau_{opa}(\epsilon)} \quad (680)$$

$$s(\epsilon) = \frac{f_{loc}(\epsilon) - f(\epsilon)}{\tau_{ii}(\epsilon)} + \frac{f(\epsilon - \epsilon_{op}) \exp\left(-\frac{\epsilon_{op}}{kT}\right) - f(\epsilon)}{\tau_{ope}(\epsilon)} + \frac{f(\epsilon + \epsilon_{op}) \exp\left(\frac{\epsilon_{op}}{kT}\right) - f(\epsilon)}{\tau_{opa}(\epsilon)} - \frac{R_{net} f_{loc}(\epsilon)}{n} \quad (681)$$

where:

- $1/\tau_c$ is the Coulomb scattering rate.
- $1/\tau_{ii}$ is the impact ionization scattering rate.
- $1/\tau_{ac}$ is the acoustic phonon scattering rate.
- $1/\tau_{ope}$ is the scattering rate due to optical phonon emissions.
- $1/\tau_{opa}$ is the scattering rate due to optical phonon absorptions.
- ϵ_{op} is the optical phonon energy.
- R_{net} is the net recombination rate.
- n is the electron density.
- $f_{loc} = \exp[(E_{F,n} - E_C - \epsilon)/kT]$ is the local equilibrium distribution function.

The Coulomb scattering rate can be written as [6]:

$$\frac{1}{\tau_c(\epsilon)} = \frac{q^4 \pi^2 g(\epsilon) N_{i,eff}}{2h\epsilon_{sem}^2 [k^2(\epsilon) + k_0^2]^2} \left[\ln(1+b) - \frac{b}{1+b} \right] \quad (682)$$

$$b(\epsilon) = \frac{4[k^2(\epsilon) + k_0^2]kT\epsilon_{sem}}{q^2(n+p)} \quad (683)$$

$$N_{i,eff} = \begin{cases} (N_D + N_A)\zeta_{major}(N_{major}) & N_{major} > N_{minor} \\ (N_D + N_A)\zeta_{minor}(N_{minor}) & N_{major} < N_{minor} \end{cases} \quad (684)$$

where:

- ϵ_{sem} is the dielectric constant of a semiconductor material.
- k_0^2 is an adjustable parameter that controls the energy dependency of the impurity scattering.
- N_{major} is N_D for electrons and N_A for holes.
- N_{minor} is N_A for electrons and N_D for holes.
- ζ_{major} and ζ_{minor} are tabulated fitting functions introduced to match the experimental low-field mobility curve as a function of majority and minority doping concentrations, respectively.

Two different expressions for the impact ionization scattering rate are available. The first expression can be written as [6]:

$$\frac{1}{\tau_{ii}(\epsilon)} = \begin{cases} \left(\frac{\epsilon - \epsilon_{ii,1}}{1 \text{ eV}}\right)^{v_{ii,1}} s_{ii,1} & \epsilon_{ii,1} < \epsilon < \epsilon_{ii,3} \\ \left(\frac{\epsilon - \epsilon_{ii,2}}{1 \text{ eV}}\right)^{v_{ii,2}} s_{ii,2} & \epsilon > \epsilon_{ii,3} \end{cases} \quad (685)$$

where:

- $s_{ii,1}$ and $s_{ii,2}$ are the impact ionization coefficients.
- $v_{ii,1}$ and $v_{ii,2}$ are the exponents.
- $\epsilon_{ii,1}$, $\epsilon_{ii,2}$, and $\epsilon_{ii,3}$ are the reference energies.

The second expression can be written as [7]:

$$\frac{1}{\tau_{ii}(\epsilon)} = \sum_{j=1}^3 \left(\frac{\epsilon - \epsilon_{ii,j}}{1 \text{ eV}}\right)^{v_{ii,j}} \Theta(\epsilon - \epsilon_{ii,j}) s_{ii,j} \quad (686)$$

Specifying `ii_formula=1` in the `SHEDistribution` parameter set activates the first expression; while `ii_formula=2` activates the second expression. The impact ionization model parameters for electrons and holes are obtained from [6] and [8], respectively.

The acoustic-phonon and optical-phonon scattering rates can be written as [5][6]:

$$\frac{g(\epsilon)}{\tau_{ac}(\epsilon)} = \sum_{i=1}^{N_b} \sum_{j=1}^{N_b} \frac{4\pi^2 k T D_{ac,ij}^2}{h \rho c_L^2} g_i(\epsilon) g_j(\epsilon) \quad (687)$$

$$\frac{g(\epsilon)}{\tau_{ope}(\epsilon)} = \sum_{i=1}^{N_b} \sum_{j=1}^{N_b} \frac{h D_{op,ij}^2}{2 \rho \epsilon_{op}} (N_{op} + 1) g_i(\epsilon) g_j(\epsilon - \epsilon_{op}) \quad (688)$$

$$\frac{g(\epsilon)}{\tau_{opa}(\epsilon)} = \sum_{i=1}^{N_b} \sum_{j=1}^{N_b} \frac{h D_{op,ij}^2}{2 \rho \epsilon_{op}} N_{op} g_i(\epsilon) g_j(\epsilon + \epsilon_{op}) \quad (689)$$

where:

- i and j are the band indices.
- $D_{ac,ij}$ and $D_{op,ij}$ are the deformation potentials for acoustic and g-type optical phonons, respectively ($D_{ac,ij} = D_{ac,ji}$ and $D_{op,ij} = D_{op,ji}$).

- ρ is the mass density.
- c_L is the sound velocity.
- $N_{op} = [\exp(\epsilon_{op}/kT) - 1]^{-1}$ is the phonon number.

If $D_{ac,ij} = D_{ac}$ and $D_{op,ij} = D_{op}$ regardless of the band indices, [Eq. 687](#), [Eq. 688](#), and [Eq. 689](#) can be simplified to:

$$\frac{1}{\tau_{ac}(\epsilon)} = \frac{4\pi^2 k T D_{ac}^2}{h \rho c_L^2} g(\epsilon) \quad (690)$$

$$\frac{1}{\tau_{ope}(\epsilon)} = \frac{h D_{op}^2}{2 \rho \epsilon_{op}} (N_{op} + 1) g(\epsilon - \epsilon_{op}) \quad (691)$$

$$\frac{1}{\tau_{opa}(\epsilon)} = \frac{h D_{op}^2}{2 \rho \epsilon_{op}} N_{op} g(\epsilon + \epsilon_{op}) \quad (692)$$

Dirichlet boundary condition as $f(\epsilon) = \exp[(E_{F,n} - E_C - \epsilon)/kT]$ is assumed for electrodes.

Boundary conditions for abrupt heterointerfaces are similar to the corresponding boundary conditions for the thermionic emission model. Assume that at the heterointerface between materials 1 and 2, the conduction band edge jump is positive ($\Delta E_C = E_{C,2} - E_{C,1} > 0$). If $J_{n,2}(\epsilon)$ and $J_{n,1}(\epsilon)$ are the energy-dependent electron current density per spin and per valley entering material 2 and leaving material 1, the interface condition can be written as:

$$J_{n,2}(\epsilon) = J_{n,1}(\epsilon) \quad (693)$$

$$J_{n,2}(\epsilon) = \frac{q}{4} g_2(\epsilon) v_2(\epsilon) [f_2(\epsilon) - f_1(\epsilon)] \quad (694)$$

where $g_i(\epsilon)$, $v_i(\epsilon)$, and $f_i(\epsilon)$ are the density-of-states, the group velocity, and the occupation probability of material i , respectively.

All other boundaries are treated with reflective boundary conditions.

Using Spherical Harmonics Expansion Method

The electron-energy distribution function is calculated from [Eq. 673](#), [p. 634](#) in the semiconductor regions specified by the global, region-specific, or material-specific **Physics** section:

```
Physics { eSHEDistribution( <arguments> ) ... }
```

By default, $g(\epsilon)$, $v(\epsilon)$, and $k^2(\epsilon)$ are obtained from the analytic band model. These band structure-related quantities also can be obtained from the default electron-band file `eSHEBandSilicon.dat` in the directory `$STR00T/tcad/$STRELEASE/lib/sdevice/MaterialDB` by specifying the argument `FullBand`:

```
Physics { eSHEDistribution( FullBand ... ) ... }
```

The default electron-band file `eSHEBandSilicon.dat` contains the band-structure quantities obtained from the nonlocal empirical pseudopotential method for relaxed silicon.

Similarly, there is a default hole-band file `hSHEBandSilicon.dat` in the same directory.

NOTE For silicon regions, it is recommended to use the `FullBand` option as the default model parameters are calibrated based on the full band structure. In addition, there is no performance penalty when using the `FullBand` option.

You also can specify your own band file as:

```
Physics { eSHEDistribution( FullBand = "filename" ... ) ... }
```

The band file is a plain text file composed of $1 + 3N_b$ columns of data where N_b is the number of bands ($1 \leq N_b \leq 4$). The first column represents the kinetic energy ϵ [eV]. The subsequent columns represent $g_v g_b(\epsilon)$ [$\text{cm}^{-3} \text{eV}^{-1}$], $g_v g_b(\epsilon) v_b^2(\epsilon)$ [$\text{cm}^{-1} \text{s}^{-2} \text{eV}^{-1}$], and $g_v k_b^2(\epsilon)$ [cm^{-2}] for band b ($1 \leq b \leq N_b$). The kinetic energy should start from zero, and the energy spacing between the neighbour rows should be uniform. Refer to `eSHEBandSilicon.dat` for more information.

Sentaurus Device provides a simplified SHE model based on the relaxation time approximation (RTA) for the hot-carrier injection current computation. Although the RTA is a poor approximation, the RTA can remove the energy coupling and reduce simulation time. You can activate the RTA mode as follows:

```
Physics { eSHEDistribution( RTA ... ) ... }
```

When the RTA mode is selected, the relaxation time is defined as:

$$\frac{1}{\tau_{\text{RTA}}(\epsilon)} = \frac{v(\epsilon)}{\lambda_{\text{sem}}} + \frac{1}{\tau_0} \quad (695)$$

where λ_{sem} and τ_0 are adjustable parameters representing a mean free path and relaxation time. In the RTA mode, [Eq. 681](#) is replaced by:

$$s(\epsilon) = \frac{f_{\text{loc}}(\epsilon) - f(\epsilon)}{\tau_{\text{RTA}}(\epsilon)} \quad (696)$$

NOTE In the RTA mode, you must specify `SHEIterations=1` together with `SHESOR` in the global `Math` section. The RTA mode should not be used for self-consistent computations as the carrier flux is not conserved.

In the SHE method, the low-field mobility is determined by the microscopic scattering rate:

$$\mu_{\text{low}} = \frac{q \int_0^{\infty} \tau(\epsilon) g(\epsilon) v^2(\epsilon) \exp\left(-\frac{\epsilon}{kT}\right) d\epsilon}{3kT \int_0^{\infty} g(\epsilon) \exp\left(-\frac{\epsilon}{kT}\right) d\epsilon} \quad (697)$$

The mobility obtained from [Eq. 697](#) and that from the macroscopic mobility model specified in the `Physics` section generally differ. For example, [Eq. 697](#) overestimates the low-field mobility in the inversion layer of MOSFETs as the scattering rate in [Eq. 680, p. 635](#) does not account for the mobility degradation at interfaces. To resolve this inconsistency, the Coulomb scattering rate is adjusted locally to match the low-field mobility obtained from the mobility model. This option is activated by default. To switch off this option, specify:

```
Physics { eSHEDistribution( -AdjustImpurityScattering ... ) ... }
```

Similarly, for the hole-energy distribution function, specify `hSHEDistribution` in the `Physics` section.

[Eq. 673, p. 634](#) is a coupled energy-dependent conservation equation with diffusion and source terms. The number of unknown variables in the SHE method is much larger than that in the drift-diffusion model or hydrodynamic model because of the additional total energy coordinate.

By default, the blockwise successive over-relaxation (SOR) method is used to solve the equation iteratively where the SOR iteration is performed over different total energies. The linear solver for the block system, the number of SOR iterations, and the SOR parameter can be accessed by the keywords `SHEMethod`, `SHEIterations`, and `SHESORParameter` in the global `Math` section. The default values are:

```
Math {
    SHEMethod=super
    SHEIterations=20
    SHESORParameter=1.1
}
```

Instead of using the blockwise SOR method, you also can solve [Eq. 673](#) for different energies simultaneously by switching off the keyword `SHESOR` in the global `Math` section. Although any matrix solver can be used, the iterative linear solver ILS is the only practical option to solve [Eq. 673](#) because of the large matrix size.

The ILS default parameters in `set=3` can be used for the SHE method. For example:

```
Math {
  -SHESOR
  SHEMethod=ILS(set = 3)
  ...
}
```

The ILS default parameters in `set=3` are defined as:

```
set (3) {
  iterative (gmres(100), tolrel=1e-8, tolunprec=1e-4, tolabs=0, maxit=200);
  preconditioning (ilut(0.00011, -1));
  ordering ( symmetric=rcm, nonsymmetric=mpsilst );
  options ( compact=yes, verbose=0, refinebasis=0, refinescaling=none,
            refineresidual=0 );
};
```

The global `Math` section provides some parameters related to the energy grid specification. The minimum and the maximum of the total energy coordinate are defined as:

$$\epsilon_{t,\min} = \min[E_C(\vec{r})] \quad (698)$$

$$\epsilon_{t,\max} = \max[E_C(\vec{r})] + \epsilon_{\text{margin}} \quad (699)$$

where ϵ_{margin} is an energy margin ($\epsilon_{\text{margin}} = 1\text{eV}$ by default). The energy grid spacing is defined by the fraction of the phonon energy $\Delta\epsilon_t = \epsilon_{\text{op}}/N_{\text{refine}}$ where N_{refine} is a positive integer ($N_{\text{refine}} = 1$ by default).

The parameters ϵ_{margin} and N_{refine} can be set in the global `Math` section:

```
Math {
  SHETopMargin = 1.0 # e_margin [eV]
  SHERefinement = 1  # N_refine
  ...
}
```

While the total energy grid is used for the computation, a uniform kinetic energy grid is used for plotting. The maximum kinetic energy to be plotted can be specified by the keyword `SHECutoff` (5 eV by default) in the global `Math` section:

```
Math { SHECutoff = 5.0 ... }
```

By default, the carrier energy distribution is updated in the postprocessing computation after each Sentaurus Device simulation point. You can suppress or activate the postprocessing computation using the `Set` statement of the `Solve` section. For example:

```
Solve { ...
  Set ( eSHEDistribution (Frozen) ) # freeze the distribution function
  ...
  Set ( eSHEDistribution (-Frozen) ) # unfreeze the distribution function
  ...
}
```

As long as the distribution function is frozen, the distribution is unchanged during simulation.

Instead of the postprocessing computation, you also can obtain the self-consistent DC solution by using the `Plugin` statement. For example:

```
Solve { ...
  Plugin (iterations=100) { Poisson eSHEDistribution hole }
  ...
}
```

In the self-consistent mode, the carrier density and the terminal current are obtained directly from the SHE method. You also can include the quantum correction in the SHE method. For example:

```
Solve { ...
  Plugin { Coupled {Poisson eQuantumPotential} eSHEDistribution hole }
  ...
}
```

NOTE In the self-consistent mode, you must specify the keyword `DirectCurrent` in the global `Math` section. In addition, you may need to increase `SHERefinement` to improve the resolution of the energy grid. The self-consistent mode does not support transient, AC, and noise analysis. In general, the lowest-order SHE method may not be sufficiently accurate to simulate nanoscale transistors as the contribution of higher-order terms increases with decreasing device length [9]. The convergence rate of the `Plugin` method can be very slow when large biases are applied.

For backward compatibility, the following pairs of keywords are recognized as synonyms in the command file:

- `SHEDistribution` and `TailDistribution`
- `eSHEDistribution` and `TaileDistribution`
- `hSHEDistribution` and `TailhDistribution`
- `SHEIterations` and `TailDistributionIterations`

- SHEMethod and TailDistributionMethod
- SHESOR and TailDistributionSOR
- SHESORParameter and TailDistributionSORParameter

In the PMI, you can read the distribution function, density-of-states, and group velocity obtained from the SHE method using the following read functions:

- ReadeSHEDistribution: Returns $f(\epsilon)$ for electrons.
- ReadeSHETotalDOS: Returns $2g_v g(\epsilon)$ for electrons.
- ReadeSHETotalGSV: Returns $2g_v g(\epsilon)v^2(\epsilon)$ for electrons.
- ReadhSHEDistribution: Returns $f(\epsilon)$ for holes.
- ReadhSHETotalDOS: Returns $2g_v g(\epsilon)$ for holes.
- ReadhSHETotalGSV: Returns $2g_v g(\epsilon)v^2(\epsilon)$ for holes.

For more information, see [Chapter 41 on page 979](#).

The parameters for the SHE method are available in the SHEDistribution parameter set. [Table 109](#) lists the coefficients of models and their default values.

NOTE The optical phonon energy ϵ_{op} is closely related to the energy grid spacing. Therefore, the same optical phonon energy must be used in the simulation domain.

Table 109 Default parameters for SHE distribution model

Symbol	Parameter name	Electrons	Holes	Unit
ρ	rho	2.329		g/cm^3
ϵ_{sem}	epsilon	11.7		ϵ_0
ϵ_{ins}	eps_ins	2.15		ϵ_0
m_c	m_s	0.26	0.26	m_0
m_n	m_dos	0.328	0.689	m_0
m_{ins}	m_ins	0.5	0.77	m_0
α	alpha	0.5	0.669	eV^{-1}
g_v	g	6	1	1
A	A	1	1	1
E_{B0}	E_barrier	3.1	4.73	eV
λ_{ins}	Lins	2.0×10^{-7}	2.0×10^{-7}	cm
λ_{sem}	Lsem	5.0×10^{-6}	1.0×10^{-6}	cm

Table 109 Default parameters for SHE distribution model

Symbol	Parameter name	Electrons	Holes	Unit
τ_0	tau0	1.0×10^{-12}	1.0×10^{-12}	s
D_{ac}/c_L	Dac_cl	1.027×10^{-5}	6.29×10^{-6}	eVs/cm
D_{op}	Dop	1.25×10^9	8.7×10^8	eV/cm
ϵ_{op}	HbarOmega	0.06	0.0633	eV
k_0^2	swv0	0.0	0.0	cm ⁻²
	ii_formula1	1	1	
$s_{ii,1}$	ii_rate1	1.49×10^{11}	0.0	1/s
$s_{ii,2}$	ii_rate2	1.13×10^{12}	1.14×10^{12}	1/s
$s_{ii,3}$	ii_rate3	0.0	0.0	1/s
$\epsilon_{ii,1}$	ii_energy1	1.128	1.128	eV
$\epsilon_{ii,2}$	ii_energy2	1.572	1.49	eV
$\epsilon_{ii,3}$	ii_energy3	1.75	1.49	eV
$v_{ii,1}$	ii_exponent1	3.0	0.0	1
$v_{ii,2}$	ii_exponent2	2.0	3.4	1
$v_{ii,3}$	ii_exponent3	0.0	0.0	1

Table 110 lists the coefficients and the default values of the tabulated doping-dependent fitting parameters ζ_{major} and ζ_{minor} for electrons and holes.

NOTE By default, the fitting parameters ζ_{major} and ζ_{minor} are neglected as the impurity scattering rate is adjusted automatically according to the low-field mobility. You must switch off AdjustImpurityScattering to use these parameters.

Table 110 Default parameters for unitless doping-dependent functions ζ_{major} and ζ_{minor}

Doping	Parameter name (electrons)	ζ_{major} (electrons)	ζ_{minor} (electrons)	Parameter name (holes)	ζ_{major} (holes)	ζ_{minor} (holes)
$10^{15.00}/\text{cm}^3$	efit(0)	1.20698	2.63089	hfit(0)	2.36872	3.84998
$10^{15.25}/\text{cm}^3$	efit(1)	1.26585	2.61522	hfit(1)	2.47647	3.82989
$10^{15.50}/\text{cm}^3$	efit(2)	1.35031	2.62123	hfit(2)	2.65631	3.87730
$10^{15.75}/\text{cm}^3$	efit(3)	1.45972	2.64751	hfit(3)	2.91784	3.98847
$10^{16.00}/\text{cm}^3$	efit(4)	1.59727	2.68504	hfit(4)	3.28127	4.16424

Table 110 Default parameters for unitless doping-dependent functions ζ_{major} and ζ_{minor}

Doping	Parameter name (electrons)	ζ_{major} (electrons)	ζ_{minor} (electrons)	Parameter name (holes)	ζ_{major} (holes)	ζ_{minor} (holes)
$10^{16.25}/\text{cm}^3$	efit(5)	1.76810	2.73218	hfit(5)	3.77842	4.40187
$10^{16.50}/\text{cm}^3$	efit(6)	1.97625	2.77580	hfit(6)	4.44356	4.68485
$10^{16.75}/\text{cm}^3$	efit(7)	2.22278	2.80091	hfit(7)	5.29810	4.97515
$10^{17.00}/\text{cm}^3$	efit(8)	2.50474	2.79066	hfit(8)	6.33175	5.21189
$10^{17.25}/\text{cm}^3$	efit(9)	2.81348	2.72938	hfit(9)	7.48564	5.32107
$10^{17.50}/\text{cm}^3$	efit(10)	3.13088	2.60729	hfit(10)	8.64257	5.23752
$10^{17.75}/\text{cm}^3$	efit(11)	3.42620	2.42644	hfit(11)	9.62681	4.93200
$10^{18.00}/\text{cm}^3$	efit(12)	3.66329	2.20490	hfit(12)	10.2280	4.42987
$10^{18.25}/\text{cm}^3$	efit(13)	3.82090	1.97450	hfit(13)	10.2758	3.80695
$10^{18.50}/\text{cm}^3$	efit(14)	3.91451	1.77291	hfit(14)	9.74236	3.16136
$10^{18.75}/\text{cm}^3$	efit(15)	4.00744	1.63637	hfit(15)	8.78324	2.57856
$10^{19.00}/\text{cm}^3$	efit(16)	4.21180	1.59940	hfit(16)	7.66672	2.11166
$10^{19.25}/\text{cm}^3$	efit(17)	4.69302	1.70363	hfit(17)	6.65698	1.78292
$10^{19.50}/\text{cm}^3$	efit(18)	5.69842	2.01596	hfit(18)	5.94642	1.59808
$10^{19.75}/\text{cm}^3$	efit(19)	7.63117	2.65859	hfit(19)	5.66599	1.56334
$10^{20.00}/\text{cm}^3$	efit(20)	11.1923	3.85825	hfit(20)	5.94556	1.70207

Visualizing Spherical Harmonics Expansion Method

For plotting purposes, the SHE method provides several macroscopic variables that can be obtained from the energy distribution function:

$$n_{\text{SHE}} = 2g_v \int_0^{\infty} g(\epsilon) f(\epsilon) d\epsilon \quad (700)$$

$$T_{n, \text{SHE}} = \frac{2g_v}{n_{\text{SHE}}} \int_0^{\infty} \frac{2\epsilon}{3k} g(\epsilon) f(\epsilon) d\epsilon \quad (701)$$

$$G_{n, \text{SHE}}^{\text{ii}} = 2g_v \int_0^{\infty} \frac{1}{\tau_{\text{ii}}(\epsilon)} g(\epsilon) f(\epsilon) d\epsilon \quad (702)$$

$$\vec{J}_{n, \text{SHE}} = \frac{2qg_v}{3} \int_0^{\infty} g(\epsilon) v^2(\epsilon) \nabla f(\epsilon) d\epsilon \quad (703)$$

$$\vec{v}_{n, \text{SHE}} = \frac{\vec{J}_{n, \text{SHE}}}{qn_{\text{SHE}}} \quad (704)$$

where n_{SHE} , $T_{n, \text{SHE}}$, $G_{n, \text{SHE}}^{\text{ii}}$, $\vec{J}_{n, \text{SHE}}$, and $\vec{v}_{n, \text{SHE}}$ are the electron density, the average energy, the avalanche generation rate, the current density, and the average velocity, respectively.

The corresponding keywords for plotting these macroscopic variables are:

```
Plot{
    eSHEDensity           # electron density [/cm^3]
    eSHEEnergy            # average electron energy [K]
    eSHEAvalancheGeneration # electron avalanche generation rate [/cm^3s]
    eSHECurrentDensity/Vector # electron current density [A/cm^2]
    eSHEVelocity/Vector   # electron average velocity [cm/s]
}
```

The corresponding keywords for holes are hSHEDensity, hSHEEnergy, hSHEAvalancheGeneration, hSHECurrentDensity, and hSHEVelocity.

To plot the position-dependent electron-energy distribution function f for each kinetic energy grid, specify eSHEDistribution/SpecialVector in the Plot section:

```
Plot{
    ...
    eSHEDistribution/SpecialVector
}
```

Similarly, for the hole-energy distribution function, specify hSHEDistribution/SpecialVector. The column i of the special vector represents the distribution function at $\epsilon = (i + 1)\Delta\epsilon$. For example, eSHEDistribution_C2 represents f at $\epsilon = 3\Delta\epsilon$.

In addition, Sentaurus Device allows you to plot the electron-energy distribution function versus kinetic energy at positions specified in the command file.

The plot file is a .plt file, and its name must be defined in the File section by the keyword eSHEDistribution:

```
File {
    ...
    eSHEDistribution = "edist"
}
```

Plotting is activated by including the `eSHEDistributionPlot` section (similar to the `CurrentPlot` section) in the command file with a set of coordinates of positions:

```
eSHEDistributionPlot {
  (-0.02 0) (0 0) (0.02 0)
  ...
}
```

For each position defined by its coordinates, Sentaurus Device determines the enclosing element and interpolates the distribution function using the data at the element vertices. Similarly, for the hole-energy distribution function, define `hSHEDistribution` in the `File` section and include the `hSHEDistributionPlot` section in the command file.

Carrier Injection with Explicitly Evaluated Boundary Conditions for Continuity Equations

Hot-carrier current can be added as an interface boundary condition for continuity equations in adjacent semiconductor regions. A typical structure (see [Figure 49](#)) consists of sequences of semiconductor–insulator–semiconductor regions. Hot-carrier current produced at one semiconductor–insulator interface from the sequence is added to the second semiconductor–insulator interface, using the closest vertex algorithm described in [Destination of Injected Current on page 626](#). This feature is available only for transient simulations and is especially useful for writing and erasing memory cells.

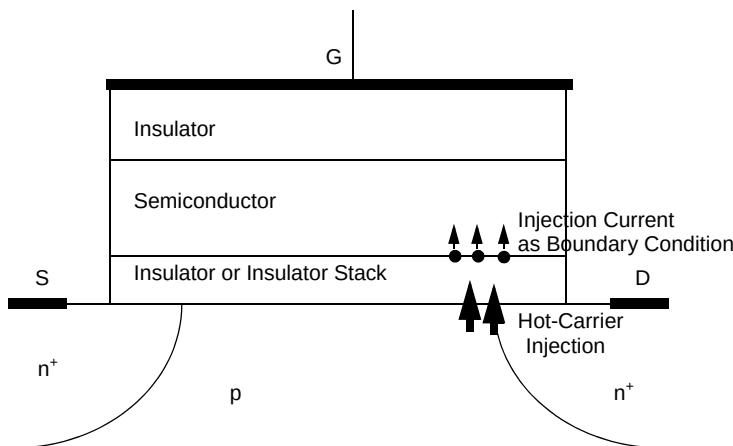


Figure 49 Injection of hot-carrier current in a MOSFET structure

At each time step after the solution is computed, the hot-carrier injection (HCI) currents are post-evaluated using the solution. For the next time step, the HCI currents are added using the current boundary condition for continuity equations in semiconductor regions, where carriers are injected, and then the whole carrier transport task is solved self-consistently.

The carriers leave the semiconductor region where they are produced and enter nonlocally into the semiconductor region where they are injected. To conserve current, injection current is subtracted from the former semiconductor region and added to the latter.

By using this method, the solution is obtained self-consistently (with one time-step delay) even if there is no carrier transport through the insulator region.

The feature is activated automatically during a transient simulation when any of the hot-carrier injection models is activated in the `GateCurrent` section and the floating semiconductor region where the hot carriers are to be injected does not have a charge boundary condition specified. Specifying `GateName` in the `GateCurrent` section disables the feature.

In addition, hot-carrier injection and the currents of the Fowler–Nordheim tunneling model can be computed at semiconductor–oxide–semiconductor interfaces. This is an extension of the searching algorithm described in [Destination of Injected Current on page 626](#). In this case, there is a semiconductor–semiconductor interface instead of semiconductor–insulator interface. To avoid ambiguity, one of the interface regions must be selected as an insulator using the keyword `InjectionRegion`:

```
Physics(RegionInterface="Region_sem12/Region_sem2") {  
    GateCurrent(Fowler eLucky InjectionRegion="Region_sem2")  
}
```

References

- [1] K. Hasnat *et al.*, “A Pseudo-Lucky Electron Model for Simulation of Electron Gate Current in Submicron NMOSFET’s,” *IEEE Transactions on Electron Devices*, vol. 43, no. 8, pp. 1264–1273, 1996.
- [2] C. Fiegna *et al.*, “Simple and Efficient Modeling of EPROM Writing,” *IEEE Transactions on Electron Devices*, vol. 38, no. 3, pp. 603–610, 1991.
- [3] S. Jin *et al.*, “Gate Current Calculations Using Spherical Harmonic Expansion of Boltzmann Equation,” in *International Conference on Simulation of Semiconductor Processes and Devices (SISPAD)*, San Diego, CA, USA, pp. 202–205, September 2009.
- [4] A. Gnudi *et al.*, “Two-dimensional MOSFET Simulation by Means of a Multidimensional Spherical Harmonics Expansion of the Boltzmann Transport Equation,” *Solid-State Electronics*, vol. 36, no. 4, pp. 575–581, 1993.
- [5] C. Jacoboni and L. Reggiani, “The Monte Carlo method for the solution of charge transport in semiconductors with applications to covalent materials,” *Reviews of Modern Physics*, vol. 55, no. 3, pp. 645–705, 1983.
- [6] C. Jungemann and B. Meinerzhagen, *Hierarchical Device Simulation: The Monte-Carlo Perspective*, Vienna: Springer, 2003.

- [7] E. Cartier *et al.*, “Impact ionization in silicon,” *Applied Physics Letters*, vol. 62, no. 25, pp. 3339–3341, 1993.
- [8] T. Kunikiyo *et al.*, “A model of impact ionization due to the primary hole in silicon for a full band Monte Carlo simulation,” *Journal of Applied Physics*, vol. 79, no. 10, pp. 7718–7725, 1996.
- [9] S.-M. Hong and C. Jungemann, “A fully coupled scheme for a Boltzmann-Poisson equation solver based on a spherical harmonics expansion,” *Journal of Computational Electronics*, vol. 8, no. 3-4, pp. 225–241, 2009.

25: Hot-Carrier Injection Models

References

This chapter describes carrier transport boundary conditions for heterointerfaces.

Thermionic Emission Current

Conventional transport equations cease to be valid at a heterojunction interface, and currents and energy fluxes at the abrupt interface between two materials are better defined by the interface condition at the heterojunction. In defining thermionic current and thermionic energy flux, Sentaurus Device follows the literature [1].

Using Thermionic Emission Current

To activate the thermionic current model for electrons at a region-interface (material-interface) heterojunction, the keyword `eThermionic` must be specified in the appropriate region-interface (material-interface) `Physics` section, for example:

```
Physics(MaterialInterface="GaAs/AlGaAs") {  
    eThermionic  
}
```

Similarly, to activate thermionic current for holes, the keyword `hThermionic` must be specified. The keyword `Thermionic` activates the thermionic emission model for both electrons and holes. If any of these keywords is specified in the `Physics` section for a region `Region.0`, where `Region.0` is a semiconductor, the appropriate model will be applied to each `Region.0`-semiconductor interface.

For small particle and energy fluxes across the interface, the condition of continuous quasi-Fermi level and carrier temperature is sometimes used. This option is activated by the keyword `Heterointerface` in the appropriate `Physics` section. In realistic transistors, such an approach may lead to unsatisfactory results [2].

You can change the coefficients of the thermionic emission model in the `ThermionicEmission` parameter set in an interface-specific section in the parameter file:

```
RegionInterface = "regionA/regionB" {  
    ThermionicEmission {  
        A = 2, 2    # [1]  
        B = 4, 4    # [1]  
    }  
}
```

```

    C = 1, 1  # [1]
  }
}

```

Thermionic Emission Model

Assume that at the heterointerface between materials 1 and 2, the conduction edge jump is positive, that is $\Delta E_C > 0$, where $\Delta E_C = E_{C,2} - E_{C,1}$ (that is, $\chi_1 > \chi_2$). If $J_{n,2}$ and $S_{n,2}$ are the electron current density and electron energy flux density entering material 2, and $J_{n,1}$ and $S_{n,1}$ are the electron current density and electron energy flux density leaving material 1, the interface condition can be written as:

$$J_{n,2} = J_{n,1} \quad (705)$$

$$J_{n,2} = a_n q \left[v_{n,2} n_2 - \frac{m_{n,2}}{m_{n,1}} v_{n,1} n_1 \exp\left(-\frac{\Delta E_C}{kT_{n,1}}\right) \right] \quad (706)$$

$$S_{n,2} = S_{n,1} + \frac{c_n}{q} J_{n,2} \Delta E_C \quad (707)$$

$$S_{n,2} = -b_n \left[v_{n,2} n_2 kT_{n,2} - \frac{m_{n,2}}{m_{n,1}} v_{n,1} n_1 kT_{n,1} \exp\left(-\frac{\Delta E_C}{kT_{n,1}}\right) \right] \quad (708)$$

where the ‘emission velocities’ are defined as:

$$v_{n,i} = \sqrt{\frac{kT_{n,i}}{2\pi m_{n,i}}} \quad (709)$$

and by default, the coefficients in the above equations are $a_n = 2$, $b_n = 4$, and $c_n = 1$, which corresponds to the literature [1]. Similar equations for the hole thermionic current and hole thermionic energy flux are presented below:

$$J_{p,2} = J_{p,1} \quad (710)$$

$$J_{p,2} = -a_p q \left[v_{p,2} p_2 - \frac{m_{p,2}}{m_{p,1}} v_{p,1} p_1 \exp\left(\frac{\Delta E_V}{kT_{p,1}}\right) \right] \quad (711)$$

$$S_{p,2} = S_{p,1} + \frac{c_p}{q} J_{p,2} \Delta E_V \quad (712)$$

$$S_{p,2} = -b_p \left[v_{p,2} p_2 kT_{p,2} - \frac{m_{p,2}}{m_{p,1}} v_{p,1} p_1 kT_{p,1} \exp\left(\frac{\Delta E_V}{kT_{p,1}}\right) \right] \quad (713)$$

$$v_{p,i} = \sqrt{\frac{kT_{p,i}}{2\pi m_{p,i}}} \quad (714)$$

An equivalent set of equations are used if Fermi carrier statistics is selected.

Gaussian Transport Across Organic Heterointerfaces

A thermionic-like current boundary condition has been introduced to correctly account for carrier transport across organic heterointerfaces. An organic heterointerface is defined in this context as an heterointerface with the Gaussian density-of-states (DOS) model (see [Gaussian Density-of-States for Organic Semiconductors on page 264](#)) activated in both regions that are adjacent to the heterointerface.

Using Gaussian Transport at Organic Heterointerfaces

The model is activated by switching to the Gaussian DOS model in both regions of the heterointerface that are meant to be organic:

```
Physics(Region="OrganicRegion_1") {
    EffectiveMass(GaussianDOS)
}

Physics(Region="OrganicRegion_2") {
    EffectiveMass(GaussianDOS)
}
```

and then specifying the keyword `Organic_Gaussian` as an option for the Thermionic model in the Physics section of the organic heterointerface:

```
Physics(RegionInterface="OrganicRegion_1/OrganicRegion_2") {
    Thermionic(Organic_Gaussian)
}
```

This syntax also automatically switches on the double points at the organic heterointerface.

The parameters $v_{n,org}$ and $v_{p,org}$ in [Eq. 716](#) and [Eq. 718, p. 654](#) can be adjusted in the ThermionicEmission section of the parameter file (their default values are 1×10^6 cm/s):

```
RegionInterface="OrganicRegion_1/OrganicRegion_2" {
    ThermionicEmission {
        vel_org = 1e7, 1e7 # [cm/s]
    }
}
```

Gaussian Transport at Organic Heterointerface Model

Assuming a positive conduction edge jump and a negative valence edge jump from material 1 to material 2, the boundary conditions at the organic heterointerface are given by:

$$J_{n,2} = J_{n,1} \quad (715)$$

$$J_{n,2} = v_{n,\text{org}} q \left(n_2 - n_1 \exp\left(-\frac{\Delta E_C}{kT_{n,1}}\right) \right) \quad (716)$$

$$J_{p,2} = J_{p,1} \quad (717)$$

$$J_{p,2} = v_{p,\text{org}} q \left(p_2 - p_1 \exp\left(-\frac{\Delta E_V}{kT_{p,1}}\right) \right) \quad (718)$$

where:

- $J_{n,2}$ and $J_{n,1}$ are the electron current densities entering material 2 and leaving material 1, respectively.
- $J_{p,2}$ and $J_{p,1}$ are the hole current densities leaving material 2 and entering material 1, respectively.
- $\Delta E_C = d_{C,2} - d_{C,1}$ where $d_{C,2}$ and $d_{C,1}$ are the distances of the Gaussian distribution peaks to the conduction band edges.
- $\Delta E_V = d_{V,2} - d_{V,1}$ where $d_{V,2}$ and $d_{V,1}$ are the distances of the Gaussian distribution peaks to the valence band edges.

References

- [1] D. Schroeder, *Modelling of Interface Carrier Transport for Device Simulation*, Wien: Springer, 1994.
- [2] K. Horio and H. Yanai, "Numerical Modeling of Heterojunctions Including the Thermionic Emission Mechanism at the Heterojunction Interface," *IEEE Transactions on Electron Devices*, vol. 37, no. 4, pp. 1093–1098, 1990.

CHAPTER 27 Energy-dependent Parameters

This chapter describes extensions for the temperature-dependent models.

Overview

Sentaurus Device provides the possibility to specify some parameters as a ratio of two irrational polynomials. The general form of such ratio is written as:

$$G(w, s) = f \frac{((\sum a_i w^{p_i}) + d_n s)^{g_n}}{((\sum a_j w^{p_j}) + d_d s)^{g_d}} \quad (719)$$

where subscripts n and d corresponds to numerator and denominator, respectively; f is a factor, w is a primary variable, and s is an additional variable. It is possible to use [Eq. 719](#) with different coefficients for different intervals k defined by the segment $[w_{k-1}^{\max}, w_k^{\max}]$. By default, it is assumed that only one interval $k = 0$ with the boundaries $[0, \infty]$ exists, and function G is constant, that is, $a_0 = 0$, $a_i = 0$, $p = d = 0$, $g = 1$. Factor f is defined accordingly for each model.

A simplified syntax is introduced to define the piecewise linear function G . The boundaries of the intervals and the value of factor must be specified, which means the value of G is at the right side of the interval. All other coefficients should not be specified to use this possibility. As there are some peculiarities in parameter specification and model activation, the specific models for which the approximation by [Eq. 719](#) is supported are described here separately.

Energy-dependent Energy Relaxation Time

For the specification of the energy relaxation time, the following modification of [Eq. 719](#) is used:

$$\tau(w) = \tau_w^0 \frac{((\sum a_i w^{p_i})^{g_n})}{((\sum a_j w^{p_j})^{g_d})} \quad (720)$$

27: Energy-dependent Parameters

Energy-dependent Energy Relaxation Time

where $w = 1.5kT_n/q$ for electrons and $w = kT_p/q$ for holes. The factor f in Eq. 719 is defined by τ_w^0 , which can be specified in the parameter file by the values `tau_w_ele` and `tau_w_hol`.

To activate the specification of the energy-dependent energy relaxation time, the parameter `Formula(tau_w_ele)` (or `Formula(tau_w_hol)` for holes) must be set to 2.

The following example shows the energy relaxation time section of the parameter file and provides a short description of the syntax:

```
EnergyRelaxationTime
{ * Energy relaxation times in picoseconds
  tau_w_ele = 0.3 # [ps]
  tau_w_hol = 0.25 # [ps]
  * Below is the example of energy relaxation time approximation
  * by the ratio of two irrational polynomials.
  * If Wmax(interval-1) < Wc < Wmax(interval), then:
  * tau_w = (tau_w)*(Numerator^Gn)/(Denominator^Gd),
  * where (Numerator or Denominator)=SIGMA[A(i)(Wc^P(i))],
  * Wc=1.5(k*Tcar)/q (in eV).
  * By default: Wmin(0)=Wmax(-1)=0; Wmax(0)=infinity.
  * The option can be activated by specifying appropriate Formula equals 2
  *   Formula(tau_w_ele) = 2
  *   Formula(tau_w_hol) = 2
  *   Wmax(interval)_ele =
  *   tau_w_ele(interval) =
  *   Numerator(interval)_ele{
  *     A(0) =
  *     P(0) =
  *     A(1) =
  *     P(1) =
  *     D =
  *     G =
  *   }
  *   Denominator(interval)_ele{
  *     A(0) =
  *     P(0) =
  *     D =
  *     G =
  *   }
  *   Wmax(interval)_hol =
  *   tau_w_hol(interval) =
  tau_w_ele = 0.3 # [ps]
  tau_w_hol = 0.25 # [ps]

  Formula(tau_w_ele) = 2
  Numerator(0)_ele{
    A(0) = 0.048200
```

```

P(0) = 0.00
A(1) = 1.00
P(1) = 3.500
A(2) = 0.0500
P(2) = 2.500
A(3) = 0.0018100
P(3) = 1.00
}
Denominator(0)_ele{
  A(0) = 0.048200
  P(0) = 0.00
  A(1) = 1.00
  P(1) = 3.500
  A(2) = 0.100
  P(2) = 2.500
}

```

The following example shows a simplified syntax for piecewise linear specification of energy relaxation time:

```

EnergyRelaxationTime:
{ * Energy relaxation times in picoseconds
  Formula(tau_w_ele) = 2
  tau_w_ele = 0.3      # [ps]

  Wmax(0)_ele = 0.5    # [eV]
  tau_w_ele(1) = 0.46  # [ps]

  Wmax(1)_ele = 1.0    # [eV]
  tau_w_ele(2) = 0.4   # [ps]

  Wmax(2)_ele = 2.0    # [eV]
  tau_w_ele(3) = 0.2   # [ps]

  tau_w_hol = 0.25     # [ps]
}

```

Spline Interpolation

Sentaurus Device also allows spline approximation of energy relaxation time over energy. In this case, the parameter `Formula(tau_w_ele)` for electron energy relaxation time (and similarly, parameter `Formula(tau_w_hol)` for hole energy relaxation time) must be equal to 3.

27: Energy-dependent Parameters

Energy-dependent Mobility

Inside the braces following the keyword `Spline(tau_w_ele)` (or `Spline(tau_w_hol)`), an energy [eV] and tau [ps] value pair must be specified in each line. For the values outside of the specified intervals, energy relaxation time is treated as a constant and equal to the closest boundary value.

The following example shows a spline approximation specification for energy-dependent energy relaxation time for electrons:

```
EnergyRelaxationTime {
  Formula(tau_w_ele) = 3
  Spline(tau_w_ele) {
    0.    0.3  # [eV] [ps]
    0.5   0.46 # [eV] [ps]
    1.    0.4  # [eV] [ps]
    2.    0.2  # [eV] [ps]
  }
}
```

NOTE Energy relaxation times can be either energy-dependent or mole fraction-dependent (see [Abrupt and Graded Heterojunctions on page 48](#)), but not both.

Energy-dependent Mobility

In addition to the existing energy-dependent mobility models (such as Caughey–Thomas, where the effective field is computed inside Sentaurus Device as a function of the carrier temperature), a more complex, user-supplied mobility model can be defined. For such specification of energy-dependent mobility, a modification to [Eq. 719](#) is used:

$$\mu(\bar{w}, N_{\text{tot}}) = \mu_{\text{low}} \frac{((\sum a_i \bar{w}^{p_i}) + d_n N_{\text{tot}})^{g_n}}{((\sum a_j \bar{w}^{p_j}) + d_d N_{\text{tot}})^{g_d}} \quad (721)$$

where $\bar{w} = T_n/T$ for electrons or $\bar{w} = T_p/T$ for holes, and μ_{low} is the low field mobility.

To activate the model, `CarrierTemperatureDrivePolynomial`, the driving force keyword, must be specified as a parameter of the high-field saturation mobility model. Parameters of the polynomials must be defined in the `HydroHighFieldMobility` parameter set.

This example shows the output of the HydroHighFieldMobility section and the specification of coefficients:

```
HydroHighFieldDependence:
{ * Parameter specifications for the high field degradation in
  * some hydrodynamic models.
  * B) Approximation by the ratio of two irrational polynomials
  * (driving force 'CarrierTempDrivePolynomial'):
  * If Wmax(interval-1) < w < Wmax(interval), then:
  *  $\mu_{hf} = \mu * factor * (Numerator^{Gn}) / (Denominator^{Gd})$ ,
  * where (Numerator or Denominator)={SIGMA[A(i)(w^P(i))]+D*Ni},
  *  $w = T_c / T_l$ ;  $Ni (cm^{-3})$  is total doping.
  * By default: Wmin(0)=Wmax(-1)=0; Wmax(0)=infinity.

  * Wmax(interval)_ele =
  * F(interval)_ele =
  * Numerator(interval)_ele{
  *   A(0) =
  *   P(0) =
  *   A(1) =
  *   P(1) =
  *   D =
  *   G =
  * }
  * Denominator(interval)_ele{
  *   A(0) =
  *   P(0) =
  *   D =
  *   G =
  * }
  * F(interval)_hol =
  * Wmax(interval)_hol =
  * Denominator(0)_ele
{
  A(0) = 0.3
  P(0) = 0.0
  A(1) = 1.0
  P(1) = 2.
  A(2) = 0.001
  P(2) = 2.500
  D = 3.00e-16
  G = 0.2500
}
}
```

Spline Interpolation

Instead of the rational polynomial in [Eq. 721](#), a spline interpolation can be used as well. In this case, the option `CarrierTempDriveSpline` must be used for the high-field mobility model in the command file:

```
Physics {  
  Mobility (  
    HighFieldSaturation (CarrierTempDriveSpline)  
  )  
}
```

The energy-dependent mobility is computed as

$$\mu(\bar{w}) = \mu_{\text{low}} \cdot \text{spline}(\bar{w}) \quad (722)$$

where the function $\text{spline}(\bar{w})$ is defined by a sequence of value pairs in the parameter file:

```
HydroHighFieldDependence {  
  Spline (electron) {  
    0  1  
    1  1  
    2  2.5  
    4  4  
    10 5  
  }  
  
  Spline (hole) {  
    0  1  
    1  1  
    2  0.75  
    4  0.5  
    10 0.2  
  }  
}
```

The given data points are interpolated by a cubic spline. Zero derivatives are imposed as boundary conditions at the end points. The spline function remains constant beyond the end points.

Energy-dependent Peltier Coefficient

Sentaurus Device allows for the following modification of the expression of the energy flux equation:

$$\vec{S}_n = -\frac{5r_n}{2} \left(\frac{kT_n}{q} \vec{J}_n + f_n^{\text{hf}} \hat{\kappa}_n \frac{\partial(w_n \Pi_n)}{\partial w_n} \nabla T_n \right) \quad (723)$$

The standard expression corresponds to $\Pi_n = 1$. If $\Pi_n = 1 + P(\bar{w})$, then:

$$\frac{\partial(w_n \Pi_n)}{\partial w_n} = 1 + \bar{w} \frac{\partial P(\bar{w})}{\partial \bar{w}} \quad (724)$$

Sentaurus Device allows you to specify the function Q :

$$Q(\bar{w}) = \bar{w} \frac{\partial P(\bar{w})}{\partial \bar{w}} \quad (725)$$

For the specification of Q , the following modification of [Eq. 719](#) is used:

$$Q(\bar{w}) = f \frac{((\sum a_i \bar{w}^{p_i}))^{g_n}}{((\sum a_j \bar{w}^{p_j}))^{g_d}} \quad (726)$$

Coefficients must be specified in the HeatFlux parameter set, and the dependence can be activated by specifying a nonzero factor f .

For $\Pi_n = 1 + 1/(\sqrt{1 + \bar{w}^2})$, the result is $Q = 1/(\bar{w}^2 + 1)^{1.5}$. This is an example of the parameter file section for such a function Q_n :

```
HeatFlux
{ * Heat flux factor (0 <= hf <= 1)
  hf_n = 1    # [1]
  hf_p = 1    # [1]
  * Coefficients can be defined also as:
  *   hf_new = hf*(1.+Delta(w))
  * where Delta(w) is the ratio of two irrational polynomials.
  * If Wmax(interval-1) < Wc < Wmax(interval), then:
  * Delta(w) = factor*(Numerator^Gn)/(Denominator^Gd),
  * where (Numerator or Denominator)=SIGMA[A(i)(w^P(i))], w=Tc/Tl
  * By default: Wmin(0)=Wmax(-1)=0; Wmax(0)=infinity.
  * Option can be activated by specifying nonzero 'factor'.
  *   Wmax(interval)_ele =
  *   F(interval)_ele = 1
```

27: Energy-dependent Parameters

Energy-dependent Peltier Coefficient

```
*      Numerator(interval)_ele{
*      A(0)  =
*      P(0)  =
*      A(1)  =
*      P(1)  =
*      G      =
*      }
*      Denominator(interval)_ele{
*      A(0)  =
*      P(0)  =
*      G      =
*      }
*      Wmax(interval)_hol =
*      F(interval)_hol = 1
*      f(0)_ele = 1
*      Denominator(0)_ele{
*      A(0)  = 1.
*      P(0)  = 0.
*      A(1)  = 1.
*      P(1)  = 2.
*      G      = 1.5
*      }
```

Spline Interpolation

Instead of the rational polynomial in [Eq. 726](#), a spline interpolation can be used as well. The function $Q(\bar{w})$ is defined in the parameter file by a sequence of value pairs:

```
HeatFlux {
  hf_n = 1
  hf_p = 1

  Spline (electron) {
    0  1
    1  1
    2  2.5
    4  4
    10 5
  }

  Spline (hole) {
    0  1
    1  1
    2  0.75
    4  0.5
    10 0.2
  }
}
```

}

The given data points are interpolated by a cubic spline. Zero derivatives are imposed as boundary conditions at the end points. The spline function remains constant beyond the end points.

27: Energy-dependent Parameters

Energy-dependent Peltier Coefficient

CHAPTER 28 Anisotropic Properties

This chapter discusses the anisotropic properties of semiconductor devices.

Overview

In general, all equations for semiconductor devices can be written in the following form:

$$-\nabla \cdot \vec{J} = R \quad (727)$$

Here vector $\vec{J} = \mu A \vec{g}$ where μ is a scalar function, tensor A is the 3×3 (2×2 , in the 2D case) symmetric matrix, and vector $\vec{g}$ is a first-order differential expression. In the isotropic case, tensor A is the unit matrix. There are two reasons why tensor A may be anisotropic: anisotropic properties of semiconductors (such as SiC) and mechanical stress (see [Chapter 30 on page 687](#)). Sentaurus Device has three approximations for solving an anisotropic task.

The first and default approximation uses local (mesh vertex) linear transformation, which transforms an anisotropic task to an isotropic case (as a result, the transformed mesh can be non-Delaunay). After this, the algorithm uses `AverageBoxMethod` to compute control volumes and coefficients (see [Chapter 40 on page 949](#)). The keyword `AverageAniso` in the `Math` section activates this algorithm. It is active by default and can be switched off by specifying `-AverageAniso`. Due to the requirements of `AverageBoxMethod`, the `AverageAniso` option requires that either the input mesh consists of triangles only (tetrahedra in 3D) or the mesh is tensorial, with the main axes of this tensor mesh and the main axes of the anisotropy aligned to the simulation coordinate system.

The second approximation (for stress tasks only) is simple and correct only for tensor grids or near-tensor grids. The anisotropic effects are modeled using a tensor-grid approximation, where off-diagonal elements of tensor A are ignored. The diagonal elements of the tensor A are used as multiplication factors for the projections of the vector $\vec{g}$ on mesh edges. The keyword `TensorGridAniso` in the `Math` section activates this algorithm.

The third approximation (for anisotropic tasks only) corrects the isotropic discretization of the semiconductor equations and it is correct if the off-diagonal elements of the tensor A are small (in comparison to diagonal values). The keyword `-AverageAniso` in the `Math` section activates this algorithm.

NOTE If the mesh is a tensor grid and tensor A has no off-diagonal elements, all three approximations are correct and the solutions are the same.

NOTE In general, anisotropic models are not compatible with cylindrical coordinates. To avoid unexpected results, do not use the `Cylindrical` keyword in the `Math` section (see [Reading a Structure on page 47](#)).

Coordinate Systems

Sentaurus Device uses two coordinate systems: the simulation system (mesh geometry) and the crystal system. Anisotropic material parameters are defined in the crystal system.

The x-axis and y-axis of the simulation system are defined in the parameter file:

```
LatticeParameters {  
    X = (1, 0, 0)  
    Y = (0, 0, -1)  
}
```

The z-axis is computed as the outer vector product of the x-axis and y-axis. The simulation system is defined relative to the crystal system. If the keyword `CrystalAxis` is present, the crystal system is defined relative to the simulation system.

In the above example, the x-axis of the simulation system coincides with the x-axis of the crystal system. The y-axis of the simulation system runs along the negative z-axis of the crystal system. This is a common definition for 2D simulations of `PiezoelectricPolarization` (see [Chapter 30 on page 687](#)).

Instead of `LatticeParameters`, the keywords `Piezo` and `PiezoParameters` are recognized as well. By default, Sentaurus Device uses $X=(1,0,0)$, $Y=(0,1,0)$, and $Z=(0,0,1)$.

Cylindrical Symmetry

Sentaurus Device supports only anisotropy with cylindrical symmetry. This means matrix A can be written as:

$$A = Q \begin{bmatrix} e \\ e \\ e_a \end{bmatrix} Q^T \text{ or } A = Q_{2:2} \begin{bmatrix} e \\ e_a \end{bmatrix} Q_{2:2}^T \quad (728)$$

where e, e_a are eigenvalues of A , and Q is the 3×3 orthogonal matrix from eigenvectors of A :

$$Q = \begin{bmatrix} \vec{A}_x & \vec{A}_y & \vec{A}_z \end{bmatrix} \quad (729)$$

and the quantity $Q_{2:2}$ denotes the leading 2×2 submatrix of Q .

Anisotropic Direction

The vector $\vec{A}_z$ defines anisotropic direction. This direction may be defined in the Aniso section of the command file in crystal system coordinates; the default value is the z-axis (the y-axis in the 2D case):

```
Physics {
  Aniso( direction = (0, 0, 1) )
}
```

The symbolic definition, such as `direction = zAxis` (`xAxis` or `yAxis`) is acceptable. This vector defines the anisotropic direction for all Aniso models.

The next examples are equivalent.

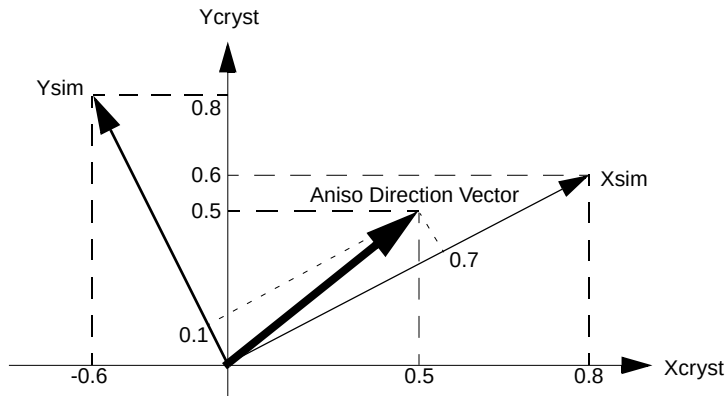
Example 1

```
Parameter file:
LatticeParameters {
  X = ( 0.8, 0.6, 0)
  Y = (-0.6, 0.8, 0)
}

Input command file:
Physics {
  Aniso(Poisson direction=(0.5, 0.5))
}
```

28: Anisotropic Properties

Overview

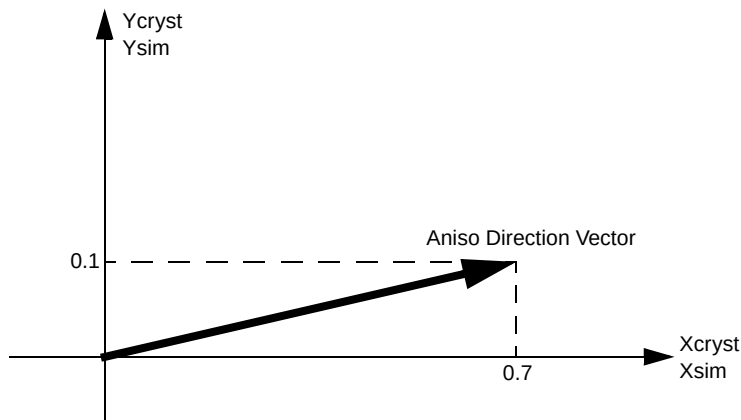


Example 2

Parameter file: default LatticeParameters, that is, $X = (1, 0, 0)$ and $Y = (0, 1, 0)$

Input command file:

```
Physics {
  Aniso(Poisson direction=(0.7, 0.1))
}
```



Orthogonal Matrix from Eigenvectors Q

If the anisotropic direction has the default value (z-axis in 3D or y-axis in 2D), then the matrix is:

$$Q = R^T, R = \begin{bmatrix} \frac{\vec{X}}{\|\vec{X}\|} & \frac{\vec{Y}}{\|\vec{Y}\|} & \frac{\vec{Z}}{\|\vec{Z}\|} \end{bmatrix} \quad (730)$$

where $\vec{x}, \vec{y}, \vec{z}$ are defined in the `LatticeParameters` section.

If the anisotropic direction is a vector $\vec{A}_d = (x, y, z)$, then the matrix is:

$$Q = \begin{bmatrix} \vec{A}_x & \vec{A}_y & \vec{A}_z \end{bmatrix}, \vec{A}_z = R^T \cdot \vec{A}_d / \|\vec{A}_d\| \quad (731)$$

The vectors $\vec{A}_x, \vec{A}_y$ are computed by Sentaurus Device to ensure that the matrix Q is orthogonal.

Anisotropic Mobility

In some semiconductors, such as silicon carbide, the electrons and holes may exhibit different mobilities along different crystallographic axes.

Crystal Reference System

The crystal reference system of the semiconductor can be specified in the parameter file as follows:

```
LatticeParameters {
  X = (1, 0, 0)
  Y = (0, 1, 0)
}
```

Instead of `LatticeParameters`, the keywords `Piezo` and `PiezoParameters` are recognized as well. By default, Sentaurus Device uses $X=(1,0,0)$, $Y=(0,1,0)$, and $Z=(0,0,1)$.

Anisotropy Factor

In a 3D simulation, Sentaurus Device assumes that the electrons or holes exhibit a mobility μ along the x-axis and y-axis, and an anisotropic mobility μ_{aniso} along the z-axis. In a 2D simulation, the regular mobility μ is observed along the x-axis, and μ_{aniso} is observed along the y-axis. The anisotropy factor r is defined as the ratio:

$$r = \frac{\mu}{\mu_{\text{aniso}}} \quad (732)$$

Current Densities

In the isotropic case, the current densities can be expressed by:

$$\vec{J}_n = \mu_n \vec{g}_n \quad (733)$$

$$\vec{J}_p = \mu_p \vec{g}_p \quad (734)$$

where $\vec{g}_n$ and $\vec{g}_p$ are the *currents without mobilities*.

In the drift-diffusion model, you have:

$$\vec{g}_n = -nq\nabla\Phi_n \quad (735)$$

$$\vec{g}_p = -pq\nabla\Phi_p \quad (736)$$

as can be seen from [Eq. 56](#) and [Eq. 57, p. 199](#). For the thermodynamic model, [Eq. 58](#) and [Eq. 59, p. 199](#) imply that:

$$\vec{g}_n = -nq(\nabla\Phi_n + P_n\nabla T) \quad (737)$$

$$\vec{g}_p = -pq(\nabla\Phi_p + P_p\nabla T) \quad (738)$$

In the hydrodynamic case, you have:

$$\vec{g}_n = n\nabla E_C + kT_n\nabla n + f_n^{\text{td}}kn\nabla T_n - 1.5nkT_n\nabla \ln m_n \quad (739)$$

$$\vec{g}_p = p\nabla E_V - kT_p\nabla p - f_p^{\text{td}}kp\nabla T_p - 1.5pkT_p\nabla \ln m_p \quad (740)$$

according to [Eq. 60](#) and [Eq. 61, p. 200](#).

For anisotropic mobilities, [Eq. 733](#) and [Eq. 734](#) need to be rewritten as:

$$\vec{J}_n = \mu_n A_{r_n} \vec{g}_n \quad (741)$$

$$\vec{J}_p = \mu_p A_{r_p} \vec{g}_p \quad (742)$$

where r_n and r_p are the anisotropy factors for electrons and holes, respectively. If the crystal reference system coincides with the coordinate system of Sentaurus Device, the matrices A_r are given by:

$$A_r = \begin{bmatrix} 1 & & \\ & 1 & \\ & & 1/r \end{bmatrix} \text{ or } A_r = \begin{bmatrix} 1 & & \\ & 1/r & \\ & & 1 \end{bmatrix} \quad (743)$$

depending on the dimension of the problem.

In general, however, A_r needs to be written as:

$$A_r = Q \begin{bmatrix} 1 & & \\ & 1 & \\ & & 1/r \end{bmatrix} Q^T \text{ or } A_r = Q_{2:2} \begin{bmatrix} 1 & & \\ & 1/r & \\ & & 1 \end{bmatrix} Q_{2:2}^T \quad (744)$$

where Q is defined in [Eq. 728, p. 666](#) – [Eq. 731, p. 669](#).

Driving Forces

In the isotropic case, the electric field parallel to the electron or hole current is given by (see [Eq. 313](#)):

$$F_c = \frac{\vec{F} \cdot \vec{J}_c}{J_c} = \frac{\vec{F} \cdot \vec{g}_c}{g_c} \quad (745)$$

For anisotropic mobilities:

$$F_c = \frac{\vec{F} \cdot \vec{A} \vec{g}_c}{|\vec{A} \vec{g}_c|} \quad (746)$$

Similarly, the electric field perpendicular to the current, as given in [Eq. 294, p. 336](#), needs to be rewritten as:

$$F_{c,\perp} = \sqrt{F^2 - \frac{(\vec{F} \cdot \vec{A} \vec{g}_c)^2}{|\vec{A} \vec{g}_c|^2}} \quad (747)$$

[Eq. 314, p. 348](#) shows how the gradient of the Fermi potential Φ_c may be used as the driving force in high-field saturation models. Instead, for anisotropic mobilities, Sentaurus Device uses:

$$F_c = A \nabla \Phi_c \quad (748)$$

In the isotropic hydrodynamic Canali model, the driving force $\vec{F}_c$ satisfies:

$$\vec{F}_c \cdot \mu \vec{F}_c = \frac{w_c - w_0}{\tau_{ec} q} \quad (749)$$

as can be seen from [Eq. 316](#), p. 349. To derive the appropriate expression in the anisotropic case, it is assumed that F_c operates parallel to the current, that is:

$$\vec{F}_c = F_c \hat{e}_c \quad (750)$$

where:

$$\hat{e}_c = \frac{A \vec{g}_c}{|A \vec{g}_c|} \quad (751)$$

is the direction of the electron or hole current. Instead of [Eq. 749](#), you now have:

$$\mu F_c^2 (A \hat{e}_c) \cdot \hat{e}_c = \frac{w_c - w_0}{\tau_{ec} q} \quad (752)$$

or:

$$F_c = \sqrt{\frac{w_c - w_0}{\tau_{ec} q \mu A \hat{e}_c \cdot \hat{e}_c}} \quad (753)$$

Total Anisotropic Mobility

This is the simplest mode in Sentaurus Device. Only a total anisotropy factor r_e or r_h is specified in the command file:

```
Physics {
  Aniso(
    eMobilityFactor (Total) = r_e
    hMobilityFactor (Total) = r_h
  )
}
```

Sentaurus Device computes the mobility μ for electrons or holes along the main crystallographic axis as usual. The mobility μ_{aniso} is then given by:

$$\mu_{\text{aniso}} = \frac{\mu}{r} \quad (754)$$

NOTE In this mode, Sentaurus Device does not update the driving forces as discussed in [Driving Forces on page 671](#).

Total Direction-dependent Anisotropic Mobility

This mode is activated by specifying the total direction-dependent anisotropy factor r_e or r_h in the command file of Sentaurus Device:

```
Physics {
  Aniso(
    eMobilityFactor (TotalDD) = r_e
    hMobilityFactor (TotalDD) = r_h
  )
}
```

First, Sentaurus Device updates the driving forces as discussed in [Driving Forces on page 671](#). Afterwards, the electron or hole mobility μ along the main crystallographic axis is computed as usual, and the mobility μ_{aniso} is given by:

$$\mu_{\text{aniso}} = \frac{\mu}{r} \quad (755)$$

Self-Consistent Anisotropic Mobility

This is the most accurate, but also the most expensive, mode in Sentaurus Device. The electron and hole mobility models specified in the Physics section are evaluated separately for the major and minor crystallographic axes, but with different parameters for each axis. This option is activated in the Physics section for electron or hole mobilities as follows:

```
Physics {
  Aniso(
    eMobility
    hMobility
  )
}
```

To simplify matters, it is possible to specify:

```
Physics {
  Aniso(
    Mobility
  )
}
```

to activate self-consistent, anisotropic, mobility calculations for both electrons and holes.

28: Anisotropic Properties

Anisotropic Mobility

[Table 111](#) lists the mobility models that offer an anisotropic version.

Table 111 Anisotropic mobility models

Isotropic model	Anisotropic model
ConstantMobility	ConstantMobility_aniso
DopingDependence	DopingDependence_aniso
EnormalDependence	EnormalDependence_aniso
HighFieldDependence	HighFieldDependence_aniso
UniBoDopingDependence	UniBoDopingDependence_aniso
UniBoEnormalDependence	UniBoEnormalDependence_aniso
UniBoHighFieldDependence	UniBoHighFieldDependence_aniso
HydroHighFieldDependence	HydroHighFieldDependence_aniso

The PMI also supports anisotropic mobility calculations. The constructors of the classes `PMI_DopingDepMobility`, `PMI_EnormalMobility`, and `PMI_HighFieldMobility` contain an additional flag to distinguish between the isotropic and anisotropic case (see [Chapter 41 on page 979](#)).

For example, you can specify these parameters for the constant mobility model in the parameter file of Sentaurus Device:

```
ConstantMobility {  
    mumax = 1.4170e+03, 4.7050e+02  
    Exponent = 2.5, 2.2  
}
```

The following parameters would then compute a reduced constant mobility along the anisotropic axis:

```
ConstantMobility_aniso {  
    mumax = 1.0e+03, 4.0e+02  
    Exponent = 2.5, 2.2  
}
```

In each vertex, Sentaurus Device introduces the anisotropy factors r_e and r_h as two additional unknowns. For a given value of r , the driving forces F_c and $F_{c,\perp}$ are computed as discussed in [Driving Forces on page 671](#), and the mobilities along the isotropic and anisotropic axes are obtained.

The equation for the unknown factor r is then given by:

$$r = \frac{\mu(r)}{\mu_{\text{aniso}}(r)} \quad (756)$$

This nonlinear equation is solved in each vertex for both electron and hole mobilities.

Plot Section

[Table 112](#) lists the plot variables that may be useful for visualizing anisotropic mobility calculations.

Table 112 Plot variables for anisotropic mobility

Plot variable	Description
eMobility	Electron mobility along main axis
hMobility	Hole mobility along main axis
eMobilityAniso	Electron mobility along anisotropic axis
hMobilityAniso	Hole mobility along anisotropic axis
eMobilityAnisoFactor	Anisotropic factor for electrons
hMobilityAnisoFactor	Anisotropic factor for holes

Anisotropic Avalanche Generation

Sentaurus Device computes the avalanche generation according to [Eq. 369](#), [p. 377](#). In the isotropic case, the terms nv_n and pv_p can also be written, respectively, as:

$$nv_n = \mu_n |\vec{g}_n| \quad (757)$$

and:

$$pv_p = \mu_p |\vec{g}_p| \quad (758)$$

If anisotropic mobilities are switched on, [Eq. 757](#) and [Eq. 758](#) are replaced by:

$$nv_n = \mu_n |A_{r_n} \vec{g}_n| \quad (759)$$

and:

$$p\nu_p = \mu_p |A_{r_p} \vec{g}_p| \quad (760)$$

NOTE [Eq. 759](#) and [Eq. 760](#) only apply to total direction-dependent and self-consistent mobility calculations. If the total anisotropic option (see [Total Anisotropic Mobility on page 672](#)) is selected, [Eq. 757](#) and [Eq. 758](#) are used.

Anisotropic avalanche calculations can be activated in the Physics section, independently for electrons and holes:

```
Physics {
  Aniso(
    eAvalanche
    hAvalanche
  )
}
```

The keyword `Avalanche` activates calculations of anisotropic avalanche for both electrons and holes:

```
Physics {
  Aniso(
    Avalanche
  )
}
```

In the anisotropic mode, different avalanche parameters can be specified along the isotropic and anisotropic axes. [Table 113](#) shows the avalanche models that are supported.

Table 113 Anisotropic avalanche models

Isotropic model	Anisotropic model
vanOverstraetendeMan	vanOverstraetendeMan_aniso
Okuto	Okuto_aniso

Sentaurus Device uses interpolation to compute avalanche parameters for an arbitrary direction of the current. Let $\hat{e}_c$ be the direction of the electron or hole current as defined in [Eq. 751](#). In the crystal reference system, the current is given by:

$$\hat{e}'_c = Q^T \hat{e}_c = \begin{bmatrix} e'_x \\ e'_y \\ e'_z \end{bmatrix} \quad (761)$$

Sentaurus Device interpolates an avalanche parameter p depending on the direction of the current $\hat{e}_c$ according to:

$$p(\hat{e}_c) = (e_x'^2 + e_y'^2) \cdot p_{\text{isotropic}} + e_z'^2 \cdot p_{\text{anisotropic}} \quad (762)$$

in a 3D simulation, and, in a 2D simulation:

$$p(\hat{e}_c) = e_x'^2 \cdot p_{\text{isotropic}} + e_y'^2 \cdot p_{\text{anisotropic}} \quad (763)$$

The PMI also supports the calculation of anisotropic avalanche generation. The current without mobility Ag_c is passed as an input parameter, and it can be used by the PMI code to determine the model parameters depending on the direction of the current (see [Avalanche Generation Model on page 1015](#)).

NOTE The Hatakeyama avalanche model is another anisotropic avalanche model (see [Hatakeyama Avalanche Model on page 384](#)). However, it must not be used with an Aniso(Avalanche) specification.

Anisotropic Electrical Permittivity

The electrical permittivity ϵ in [Eq. 37, p. 191](#) can have different values along different crystallographic axes. If the crystallographic axes coincide with the coordinate system of Sentaurus Device, the scalar ϵ is replaced by the matrix:

$$E = \begin{bmatrix} \epsilon & & \\ & \epsilon & \\ & & \epsilon_{\text{aniso}} \end{bmatrix} \text{ or } E = \begin{bmatrix} \epsilon & & \\ & \epsilon_{\text{aniso}} & \end{bmatrix} \quad (764)$$

depending on the dimension of the problem. For general crystallographic axes, the matrix E is given by:

$$E = Q \begin{bmatrix} \epsilon & & \\ & \epsilon & \\ & & \epsilon_{\text{aniso}} \end{bmatrix} Q^T \text{ or } E = Q_{2:2} \begin{bmatrix} \epsilon & & \\ & \epsilon_{\text{aniso}} & \end{bmatrix} Q_{2:2}^T \quad (765)$$

where Q is defined in [Eq. 728, p. 666](#) – [Eq. 731, p. 669](#).

Anisotropic electrical permittivity is switched on using the keyword `Poisson` in the `Physics` section of the command file (regionwise or materialwise specification is supported):

```
Physics (Material = "SiC"){
  Aniso (Poisson)
}
```

The model parameters for ϵ and ϵ_{aniso} can be specified in the parameter file. [Table 114](#) lists the names of the corresponding models.

Table 114 Anisotropic electrical permittivity models

Isotropic model	Anisotropic model
Epsilon	Epsilon_aniso

Different parameters can be specified for each region or each material. The following statement in the command file of Sentaurus Device can be used to plot the electrical permittivities:

```
Plot {
    DielectricConstant
    "DielectricConstantAniso"
}
```

Anisotropic Thermal Conductivity

The thermal conductivity κ in [Eq. 68, p. 208](#) can have different values along different crystallographic axes. If the crystallographic axes coincide with the coordinate system of Sentaurus Device, the scalar κ is replaced by the matrix:

$$K = \begin{bmatrix} \kappa & & \\ & \kappa & \\ & & \kappa_{\text{aniso}} \end{bmatrix} \text{ or } K = \begin{bmatrix} \kappa & & \\ & \kappa_{\text{aniso}} & \\ & & \end{bmatrix} \quad (766)$$

depending on the dimension of the problem.

For general crystallographic axes, the matrix K is given by:

$$K = Q \begin{bmatrix} \kappa & & \\ & \kappa & \\ & & \kappa_{\text{aniso}} \end{bmatrix} Q^T \text{ or } K = Q_{2:2} \begin{bmatrix} \kappa & & \\ & \kappa_{\text{aniso}} & \\ & & \end{bmatrix} Q_{2:2}^T \quad (767)$$

where Q is defined in [Eq. 728, p. 666](#) – [Eq. 731, p. 669](#).

Anisotropic thermal conductivity is switched on using the keyword `Temperature` in the Physics section of the command file (regionwise or materialwise specification is supported):

```
Physics(Material = "SiC"){
    Aniso (Temperature)
}
```

The model parameters for κ and κ_{aniso} can be specified in the parameter file. [Table 115](#) lists the names of the corresponding models.

Table 115 Anisotropic thermal conductivity models

Isotropic model	Anisotropic model
Kappa	Kappa_aniso

Different parameters can be specified for each region or each material. The PMI can also be used to compute anisotropic thermal conductivities. The constructor of the class `PMI_ThermalConductivity` has an additional parameter to distinguish between the isotropic and anisotropic directions (see [Thermal Conductivity on page 1084](#)).

The following statement in the command file can be used to plot the thermal conductivities:

```
Plot {
    "ThermalConductivity"
    "ThermalConductivityAniso"
}
```

Anisotropic Density Gradient Model

The density gradient model provides support for anisotropic quantization. For more details, see [Density Gradient Quantization Model on page 288](#).

28: Anisotropic Properties

Anisotropic Density Gradient Model

CHAPTER 29 Ferroelectric Materials

This chapter explains how ferroelectric materials are treated in a simulation using Sentaurus Device.

In ferroelectric materials, the polarization $\vec{P}$ depends nonlinearly on the electric field $\vec{F}$. The polarization at a given time depends on the electric field at that time and the electric field at previous times. The history dependence leads to the well-known phenomenon of hysteresis, which is used in nonvolatile memory technology.

Using Ferroelectrics

Sentaurus Device implements a model for ferroelectrics that features minor loop nesting and memory wipeout. [Figure 50](#) demonstrates these properties and [Ferroelectrics Model on page 683](#) discusses them further.

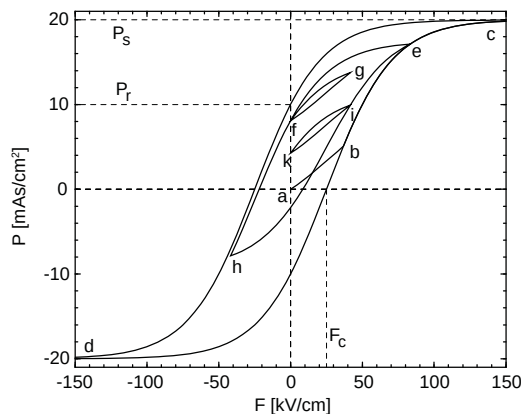


Figure 50 Example polarization curve

To activate the model, specify the keyword `Polarization` in the `Physics` section of the command file. Use the optional parameter `Memory` to prescribe the maximum allowed nesting depth of minor loops. The smallest allowed value for `Memory` is 2; the default value is 10. If minor loop nesting becomes too deep, the nesting property of the minor loops can be lost. However, the polarization curve remains continuous. For example:

```
Physics (region = "Region.17") {  
    Polarization (Memory=20)  
}
```

29: Ferroelectric Materials

Using Ferroelectrics

switches on the ferroelectric model in region `Region.17` and sets the size of the memory to 20 turning points for each element and each mesh axis.

To obtain a plot of the polarization field, specify `Polarization/Vector` in the `Plot` section of the command file.

Sentaurus Device characterizes the static properties of a ferroelectric material by three parameters: the remanent polarization P_r , the saturation polarization P_s , and the coercive field F_c . The hysteresis curve in [Figure 50 on page 681](#) illustrates these quantities. Furthermore, Sentaurus Device parameterizes the transient response of the ferroelectric material by the relaxation times τ_E and τ_P , and by a nonlinear coupling constant k_n (see [Ferroelectrics Model on page 683](#)).

Specify the values for these parameters in the `Polarization` parameter set, for example:

```
Polarization
{ * Remanent polarization P_r, saturation polarization P_s,
  * and coercive field F_c for x,y,z direction (crystal axes)
    P_r = (1.0000e-05, 1.0000e-05, 1.0000e-05) #[C/cm^2]
    P_s = (2.0000e-05, 2.0000e-05, 2.0000e-05) #[C/cm^2]
    F_c = (2.5000e+04, 2.5000e+04, 2.5000e+04) #[V/cm]
  * Relaxation time for the auxiliary field tau_E, relaxation
  * time for the polarization tau_P, nonlinear coupling kn.
    tau_E = (0.0000e+00, 0.0000e+00, 0.0000e+00) #[s]
    tau_P = (0.0000e+00, 0.0000e+00, 0.0000e+00) #[s]
    kn     = (0.0000e+00, 0.0000e+00, 0.0000e+00) #[cm*s/V]
}
```

The parameters in this example are the defaults for the material `InsulatorX`. For all other materials, all default values are zero. Each of the three numbers given for any of the parameters corresponds to the value for the respective coordinate axis of the mesh. If a P_s component is zero, the ferroelectric model is disabled along the corresponding direction. If a P_s component is nonzero, the respective P_r and F_c components must also be nonzero. Furthermore, the P_r component must be smaller than the P_s component. By default, the relaxation times are zero, which means that polarization follows the applied electric field instantaneously.

In devices with ferroelectric and semiconductor regions, it is sometimes difficult to obtain an initial solution of the Poisson equation. In many cases, the `LineSearchDamping` option can solve these problems. To use this option, start the simulation like:

```
coupled (LineSearchDamping=0.01) { Poisson }
```

See [Damped Newton Iterations on page 161](#) for details about this parameter.

Ferroelectrics Model

The vector quantity $\vec{P}$ is split into its components along the main axes of the mesh coordinate system. This results in one to three scalar problems. Sentaurus Device handles each problem separately using the model from [1] with extensions for transient behavior [2].

First, Sentaurus Device computes an auxiliary field F_{aux} from the electric field F :

$$\frac{d}{dt}F_{\text{aux}}(t) = \frac{F(t) - F_{\text{aux}}(t)}{\tau_E} \quad (768)$$

Here, τ_E is a material-specific time constant. For $\tau_E = 0$ or for quasistationary simulations, $F_{\text{aux}} = F$.

From the auxiliary field, Sentaurus Device computes the auxiliary polarization P_{aux} . The auxiliary polarization P_{aux} is an algebraic function of the auxiliary field F_{aux} :

$$P_{\text{aux}} = c \cdot P_s \cdot \tanh(w \cdot (F_{\text{aux}} \pm F_c)) + P_{\text{off}} \quad (769)$$

where P_s is the saturation polarization, F_c is the coercive field, and:

$$w = \frac{1}{2F_c} \ln \frac{P_s + P_r}{P_s - P_r} \quad (770)$$

where P_r is the remanent polarization. In Eq. 769, the plus sign applies to the decreasing auxiliary field and the minus sign applies to the increasing auxiliary field. The different signs reflect the hysteretic behavior of the material. P_{off} and c in Eq. 769 result from the polarization history of the material, see below.

Finally, from the auxiliary polarization and auxiliary field, Sentaurus Device computes the actual polarization P :

$$\frac{d}{dt}P(t) = \frac{P_{\text{aux}}[F_{\text{aux}}(t)] - P(t)}{\tau_p} \left(1 + k_n \left| \frac{d}{dt}F_{\text{aux}}(t) \right| \right) \quad (771)$$

Here, τ_p and k_n are material-specific constants. For $\tau_p = 0$ or for quasistationary simulations, $P = P_{\text{aux}}$.

Upper and lower turning points are points in the $P_{\text{aux}} - F_{\text{aux}}$ diagram where the sweep direction of the auxiliary field F_{aux} changes from increasing to decreasing, and from decreasing to increasing, respectively. At each bias point, the most recent upper and lower turning points, (F_u, P_u) and (F_l, P_l) , must both be on each of the two curves defined by Eq. 769; this requirement determines P_{off} and c .

Sentaurus Device ‘memorizes’ turning points as they are encountered during a simulation. The memory always contains (∞, P_s) as the oldest and $(-\infty, -P_s)$ as the second oldest turning point. By using [Eq. 769](#), these two points define a pair of curves with $c = 1$ and $P_{\text{off}} = 0$; together, the two curves form the saturation loop. All other pairs of turning points result in $c < 1$ and define a pair of curves forming minor loops.

When the auxiliary field leaves the interval defined by F_l and F_u of the two newest turning points, these two turning points are removed from the memory; this reflects the memory wipeout observed in experiments. The pair of turning points that are newest in the memory, after this removal, determines the further $P_{\text{aux}}(F_{\text{aux}})$ relationship.

As the older of the dropped turning points was originally reached by walking on the curve defined by the turning points that now (after dropping) again determine $P_{\text{aux}}(F_{\text{aux}})$, the polarization curve remains continuous. For example, see points e, f, and i in [Figure 50 on page 681](#). Furthermore, the minor loop defined by the two dropped turning points is nested inside the minor loop defined by the present turning points. The nesting of minor loops is also a feature known from experiments on ferroelectrics. [Figure 50](#) illustrates this by the loops f-g and i-k, both of which are nested in loop e-h, which in turn is nested in loop c-d.

In small-signal (AC) analysis (see [Small-Signal AC Analysis on page 127](#)), a very small periodic signal is added to the DC bias. As a result, the (auxiliary) polarization at each point of the ferroelectric material changes along a very small minor loop nested in the main loop that stems from the DC variation of the bias voltage. The average slope of this minor loop is always smaller than the slope of the main loop at the point where the loops touch. Consequently, even at very low frequencies, the AC response of the system is different from what would be obtained by taking the derivative of the DC curves.

As an example of how the turning point memory works, see [Figure 50](#). The points on the polarization curve are reached in the sequence a, b, c, d, e, f, g, f, h, i, k, i, e, c. For simplicity, a quasistationary process is assumed and, therefore, $F_{\text{aux}} = F$ and $P_{\text{aux}} = P$. Starting the simulation at point a, the newest point in memory is b (Sentaurus Device initializes it in this way). The second newest point is a negative saturation point (l), and the oldest point is a positive saturation point (u). This memory state is denoted by [blu]. For this state, a is on the decreasing field curve. The curve segment (that is, the coefficients c and P_{off}) from a to b is determined by the points l and b.

Ramping up from a makes a a turning point, so the memory becomes [ablu]. When crossing b and proceeding to c, a and b are dropped from the memory. Therefore, the memory becomes [lu]. These two points determine the curve from b to c. Turning at c, c is added to the memory, giving [clu]. From c to d, use c and l; at d, the memory becomes [dclu]; at point e, [edclu]; at f, [fedclu]; at g, [gfedclu]. Passing through f, the two newest points, f and g, are dropped and the memory is [edclu]; at h, [hedclu]; at i, [ihedclu]; at k, [kihedclu]. At i again, i and k are dropped, giving [hedclu]; at e, e and h are dropped, giving [dclu].

At the beginning of a simulation, the memory contains one turning point chosen such that the point $E_{\text{aux}} = 0$, $P_{\text{aux}} = 0$ is on the minor loop so defined (for example, point b in [Figure 50 on page 681](#)). The nature of the model is such that it is not possible to have a state of the system that is completely symmetric. In particular, even for a symmetric device and at the very beginning of the simulation, $P(E) \neq -P(-E)$. This asymmetry is most prominent for the virgin curves (for example, a-b in [Figure 50](#)) of the ferroelectric, which are different for different signs of voltage ramping.

References

- [1] B. Jiang *et al.*, “Computationally Efficient Ferroelectric Capacitor Model for Circuit Simulation,” in *Symposium on VLSI Technology*, Kyoto, Japan, pp. 141–142, June 1997.
- [2] K. Dragosits, *Modeling and Simulation of Ferroelectric Devices*, Ph.D. thesis, Technische Universität Wien, Vienna, Austria, December 2000.

29: Ferroelectric Materials

References

CHAPTER 30 Modeling Mechanical Stress Effect

This chapter presents an overview of the importance of stress in device simulation.

Stress engineering is a key point to ensuring the high performance of CMOS devices. Mechanical stress can affect workfunction, band gap, effective mass, carrier mobility, and leakage currents. The stress is generated by many technological processes due to different process temperatures and material properties. In addition, it can be added (such as silicon layers onto SiGe bulk) to improve device performance.

Overview

Mechanical distortion of semiconductor microstructures results in a change in the band structure and carrier mobility. These effects are well known, and appropriate computations of the change in the strain-induced band structure are based on the deformation potential theory [1]. The implementation of the deformation potential model in Sentaurus Device is based on data and approaches presented in the literature [1][2][3][4]. Other approaches [5][6] implemented in Sentaurus Device focus more on the description of piezoresistive effects.

Stress and Strain in Semiconductors

Generally, the stress tensor $\bar{\sigma}$ is a symmetric 3×3 matrix. Therefore, it only has six independent components, and it is convenient to express it in a contracted six-component vector notation:

$$\bar{\sigma} = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} & \sigma_{xz} \\ \sigma_{xy} & \sigma_{yy} & \sigma_{yz} \\ \sigma_{xz} & \sigma_{yz} & \sigma_{zz} \end{bmatrix} \rightarrow \begin{bmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{yz} \\ \sigma_{xz} \\ \sigma_{xy} \end{bmatrix} = \begin{bmatrix} \sigma_{11} \\ \sigma_{22} \\ \sigma_{33} \\ \sigma_{23} \\ \sigma_{13} \\ \sigma_{12} \end{bmatrix} = \begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \sigma_3 \\ \sigma_4 \\ \sigma_5 \\ \sigma_6 \end{bmatrix} \quad (772)$$

where pairs of indices are contracted using:

$$11 \rightarrow 1, 22 \rightarrow 2, 33 \rightarrow 3, 23 \rightarrow 4, 13 \rightarrow 5, 12 \rightarrow 6 \quad (773)$$

30: Modeling Mechanical Stress Effect

Stress and Strain in Semiconductors

The contracted tensor notation simplifies tensor expressions. For example, to compute the strain tensor $\bar{\epsilon}$ (which is needed for the deformation potential model), the generalized Hooke's law for anisotropic materials is applied:

$$\epsilon_{ij} = \sum_{k=1}^3 \sum_{l=1}^3 S_{ijkl} \sigma_{kl} \quad (774)$$

where S_{ijkl} is a component of the elastic compliance tensor $\bar{S}$. The elastic compliance tensor is symmetric, which allows Eq. 774 to be written in a simplified contracted form:

$$\epsilon_i = \sum_{j=1}^6 S_{ij} \sigma_j \quad (775)$$

In addition to index contraction (Eq. 773), the following contraction rules for ϵ_{ij} and S_{ijkl} were used to obtain this result [7]:

$$\begin{aligned} \epsilon_p &= \epsilon_{ij} \quad , \text{ if } p < 4 \\ &= 2\epsilon_{ij} \quad , \text{ if } p > 3 \\ S_{pq} &= S_{ijkl} \quad , \text{ if } p < 4 \text{ and } q < 4 \\ &= 4S_{ijkl} \quad , \text{ if } p > 3 \text{ and } q > 3 \\ &= 2S_{ijkl} \quad , \text{ otherwise} \end{aligned} \quad (776)$$

Note that ϵ_4 , ϵ_5 , and ϵ_6 are often called *engineering shear-strain components* and are related to the double-subscripted shear components that are used in the model equations by:

$$\begin{aligned} \epsilon_4 &= 2\epsilon_{23} = 2\epsilon_{32} \\ \epsilon_5 &= 2\epsilon_{13} = 2\epsilon_{31} \\ \epsilon_6 &= 2\epsilon_{12} = 2\epsilon_{21} \end{aligned} \quad (777)$$

In crystals with cubic symmetry such as silicon, the number of independent coefficients of the elastic compliance tensor (as well as with other material property tensors) reduces to three by rotating the coordinate system parallel to the high-symmetric axes of the crystal [8]. This gives the following (contracted) compliance tensor $\bar{S}$:

$$\bar{S} = \begin{bmatrix} S_{11} & S_{12} & S_{12} & 0 & 0 & 0 \\ S_{12} & S_{11} & S_{12} & 0 & 0 & 0 \\ S_{12} & S_{12} & S_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & S_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & S_{44} & 0 \\ 0 & 0 & 0 & 0 & 0 & S_{44} \end{bmatrix} \quad (778)$$

where the coefficients S_{11} , S_{12} , and S_{44} correspond to parallel, perpendicular, and shear components, respectively.

In Sentaurus Device, the stress tensor can be defined in the stress coordinate system $(\vec{e}_1, \vec{e}_2, \vec{e}_3)$. To transfer this tensor to another coordinate system (for example, the crystal system $(\vec{e}'_1, \vec{e}'_2, \vec{e}'_3)$, which is a common operation), the following transformation rule between two coordinate systems is applied:

$$\sigma'_{ij} = \sum_{k=1}^3 \sum_{l=1}^3 a_{ik} a_{jl} \sigma_{kl} \quad (779)$$

where $\vec{a}$ is the rotation matrix:

$$a_{ik} = \frac{\vec{e}'_i \cdot \vec{e}_k}{|\vec{e}'_i| |\vec{e}_k|} \quad (780)$$

Using Stress and Strain

Stress-dependent models are selected in the `Physics{Piezo( )}` section of the command file. Components of the stress tensor (if stress is constant over the device or region) also are specified here, as well as the components $\vec{e}_1$ `OriKddX` and $\vec{e}_2$ `OriKddY` of the coordinate system where the stress is defined (see [Table 116](#)):

```
Physics {
  Piezo (
    Stress = (XX, YY, ZZ, YZ, XZ, XY)
    OriKddX = (1,0,0)
    OriKddY = (0,1,0)
    Model (... )
  )
}
```

Table 116 General keywords for Piezo

Parameter	Description
Stress=(XX, YY, ZZ, YZ, XZ, XY)	Specifies uniform stress [Pa] if the <code>Piezo</code> file is not given in the <code>File</code> section.
OriKddX = (1,0,0)	Defines Miller indices of the stress system relative to the simulation system.
OriKddY = (0,1,0)	Defines Miller indices of the stress system relative to the simulation system.
Model(<options>)	Selects stress-dependent models in <options> (see sections from Deformation of Band Structure on page 691 to Stress-dependent Mobility Models on page 700).

NOTE The stress system is always defined relative to the simulation coordinate system of Sentaurus Device (in the `Piezo` section of the command file). The simulation coordinate system is defined relative to the crystal orientation system by default but, in the parameter file (see below), it is possible to define the crystal system relative to the simulation system. By default, all three coordinate systems coincide.

There are two ways to define position-dependent stress values. In one option, a field of stress values [Pa] (as obtained by mechanical structure analysis) is read by specifying the `Piezo` entry in the `File` section:

```
File { ...  
    Piezo = <piezofile>  
}
```

NOTE Sentaurus Device recognizes stress either as a single symmetric second-order tensor of dimension 3 (`Stress`), or as six scalar values (`StressXX`, `StressXY`, `StressXZ`, `StressYY`, `StressYZ`, and `StressZZ`).

Otherwise, a physical model interface can be used for stress specification. Optional parameters, which depend on the selected stress model, are described in the following sections.

NOTE Stress values in all these stress specifications should be in Pa ($1 \text{ Pa} = 10 \text{ dyn/cm}^2$) and tensile stress should be positive according to convention.

The coordinate system of a Sentaurus Device simulation relative to the crystal orientation system can be defined by the `X` and `Y` vectors in the `LatticeParameters` section of the parameter file. The default is `X=(1 0 0)`, `Y=(0 1 0)`. In addition, there is an option to represent the crystal system relative to the Sentaurus Device simulation system. In this case, the keyword `CrystalAxis` must be in the `LatticeParameters` section, and the `X` and `Y` vectors will represent the $\langle 100 \rangle$ and $\langle 010 \rangle$ axes of the crystal system in the Sentaurus Device simulation system.

The elastic compliance coefficients S_{ij} [$10^{-12} \text{ cm}^2/\text{dyn}$] can be specified in the field `S[i][j]` in the `LatticeParameters` section of the parameter file. If the cubic crystal system is selected (by specifying `CrystalSystem=0`), it is sufficient to specify S_{11} , S_{12} , and S_{44} . For a hexagonal crystal system (`CrystalSystem=1`), S_{33} and S_{13} must also be specified. Otherwise, all unspecified coefficients are set to 0.

The following section of the parameter file shows the defaults for silicon:

```
LatticeParameters {
* Crystal system and elasticity.
X = (1, 0, 0)          #[1]
Y = (0, 1, 0)          #[1]
S[1][1] = 0.77         # [1e-12 cm^2/dyn]
S[1][2] = -0.21        # [1e-12 cm^2/dyn]
S[4][4] = 1.25         # [1e-12 cm^2/dyn]
CrystalSystem = 0      # [1]
}
```

Deformation of Band Structure

In deformation potential theory [2][9], the strains are considered to be relatively small. The change in energy of each carrier subvalley, caused by the deformation of the lattice, is a linear function of the strain. By default (for silicon), Sentaurus Device considers three bands for electrons (which are applied to three two-fold bands in the conduction band) and two bands for holes (which are applied to heavy-hole and light-hole bands in the valence band). The number of carrier bands can be changed in the parameter file (see [Using Deformation Potential Model on page 693](#)).

Bir and Pikus [9] proposed a model for the strain-induced change in the energy of carrier bands in silicon where they ignore the shear strain for electrons and suggest nonlinear dependence for holes:

$$\begin{aligned}\Delta E_{C,i} &= \Xi_d(\epsilon'_{11} + \epsilon'_{22} + \epsilon'_{33}) + \Xi_u\epsilon'_{ii} \\ \Delta E_{V,i} &= -a(\epsilon'_{11} + \epsilon'_{22} + \epsilon'_{33}) \pm \delta E\end{aligned}\tag{781}$$

$$\delta E = \sqrt{\frac{b^2}{2}((\epsilon'_{11} - \epsilon'_{22})^2 + (\epsilon'_{22} - \epsilon'_{33})^2 + (\epsilon'_{11} - \epsilon'_{33})^2) + d^2(\epsilon'^2_{12} + \epsilon'^2_{13} + \epsilon'^2_{23})}$$

where Ξ_d, Ξ_u, a, b, d are other deformation potentials that correspond to the model, i corresponds to the carrier band number, and ϵ'_{ij} are the components of the strain tensor in the crystal coordinate system (see [Stress and Strain in Semiconductors on page 687](#) for a description of tensor transformations). The sign $\pm$ separates heavy-hole and light-hole bands of silicon. Both expressions in Eq. 781 have the common part $(\epsilon'_{11} + \epsilon'_{22} + \epsilon'_{33})$ and, therefore, these expressions were combined in a general one that gives flexibility to its definition in the parameter file (see [Using Deformation Potential Model on page 693](#)):

$$\Delta E_{B,i} = \xi_{i1}^{B2} \epsilon'_{11} + \xi_{i2}^{B2} \epsilon'_{22} + \xi_{i3}^{B2} \epsilon'_{33} + \quad (782)$$

$$\xi_{i4}^{B2} \sqrt{\frac{(\xi_{i5}^{B2})^2}{2} ((\epsilon'_{11} - \epsilon'_{22})^2 + (\epsilon'_{22} - \epsilon'_{33})^2 + (\epsilon'_{11} - \epsilon'_{33})^2) + (\xi_{i6}^{B2})^2 (\epsilon'_{12}^2 + \epsilon'_{13}^2 + \epsilon'_{23}^2)}$$

where ξ_{ij}^{B2} are deformation potentials that correspond to the Bir and Pikus model, and ξ_{i4}^{B2} is a unitless constant that defines mainly a sign.

Sentaurus Device uses [Eq. 782](#) for all electron and hole bands by default, but there are other options that allow you to select more sophisticated models separately for electrons and holes. Using a degenerate $k \cdot p$ theory at the zone boundary X-point, the authors of [\[10\]](#) derived an additional shear term for the electron bands in [Eq. 781](#):

$$\Delta E_{C,i} = \Xi_d(\epsilon'_{11} + \epsilon'_{22} + \epsilon'_{33}) + \Xi_u \epsilon'_{ii} + \begin{cases} -\frac{\Delta}{4} \eta_i^2 & , |\eta_i| \leq 1 \\ -(2|\eta_i| - 1) \frac{\Delta}{4} & , |\eta_i| > 1 \end{cases} \quad (783)$$

where:

- $\eta_i = \frac{4\Xi_u \epsilon'_{jk}}{\Delta}$ is a dimensionless off-diagonal strain with $j \neq k \neq i$.
- ϵ'_{jk} is a shear strain component.
- Δ is the band separation between the two lowest conduction bands.
- Ξ_u is the deformation potential responsible for the band-splitting of the two lowest conduction bands [\[10\]](#): $(E_{\Delta_1} - E_{\Delta_2})|_{X[001]} = 4\Xi_u \epsilon'_{jk}$.

The strain-induced shifts of valence bands can be computed using 6x6 $k \cdot p$ theory for the heavy-hole, light-hole, and split-off bands as described in [\[11\]](#). The specification of the deformation potentials for both of these models is described in [Using Deformation Potential Model on page 693](#).

Using the stress tensor $\bar{\sigma}$ from the command file, Sentaurus Device recomputes it from the stress coordinate system to the tensor $\bar{\sigma}'$ in the crystal system by [Eq. 779](#). The strain tensor $\bar{\epsilon}$ is a result of applying Hooke's law [Eq. 775](#) to the stress $\bar{\sigma}'$. Using [Eq. 782](#), [Eq. 783](#), or solving the cubic equation from [\[11\]](#), the energy band change can be computed for each conduction and valence carrier bands.

By default, Sentaurus Device does not modify the effective masses, but instead it computes strain-induced conduction and valence band-edge shifts, ΔE_C and ΔE_V , using an averaged value of the individual band-edge shifts:

$$\begin{aligned}\frac{\Delta E_C}{kT_{300}} &= -\ln \left[\frac{1}{n_C} \sum_{i=1}^{n_C} \exp \left(\frac{-\Delta E_{C,i}}{kT_{300}} \right) \right] \\ \frac{\Delta E_V}{kT_{300}} &= \ln \left[\frac{1}{n_V} \sum_{i=1}^{n_V} \exp \left(\frac{\Delta E_{V,i}}{kT_{300}} \right) \right]\end{aligned}\tag{784}$$

where n_C and n_V are the number of subvalleys considered in the conduction and valence bands, respectively, and $T_{300} = 300$ K.

Alternatively, a more accurate representation of the band gap can be obtained by using the minimum and maximum of the individual conduction and valence band shifts, respectively, as follows:

$$\begin{aligned}\Delta E_C &= \min(\Delta E_{C,i}) \\ \Delta E_V &= \max(\Delta E_{V,i})\end{aligned}\tag{785}$$

In this case, however, the strain dependency of the effective mass and the density-of-states should be accounted for (see [Strained Effective Masses and Density-of-States on page 695](#)).

The band gap and affinity can be modified:

$$\begin{aligned}E_g &= E_{g0} + \Delta E_C - \Delta E_V \\ \chi &= \chi_0 - \Delta E_C\end{aligned}\tag{786}$$

where the index '0' corresponds to the affinity and bandgap values before stress deformation.

Using Deformation Potential Model

To activate the deformation potential model, the name of the model must be specified in the `Piezo` section of the command file, for example:

```
Physics (Region = "StrainedSilicon") {
    Piezo(
        Model(DeformationPotential)
    )
}
```

To apply Eq. 785 to the strain-induced conduction and valence band-edge shift, you must specify an additional option for the model: `DeformationPotential(minimum)`. To activate $k \cdot p$ models (Eq. 783 for electrons, or solving the cubic equation from [11] for holes), use `DeformationPotential(ekp hkp)`.

All parameters of the model can be modified in the `LatticeParameters` section of the parameter file. The number of conduction and valence band subvalleys n_C and n_V are defined by `NC` and `NV` in the parameter file; By default, $n_C = 3$, $n_V = 2$. Appropriate deformation potential constants ξ_{ij}^C , ξ_{ij}^V , ξ_{ij}^{C2} , and ξ_{ij}^{V2} [eV] are defined in the fields `DC[i]`, `DV[i]`, `DC2[i]`, and `DV2[i]` of the parameter file, respectively.

As an example, the following section represents silicon `LatticeParameters` defined for the Bir and Pikus model in the parameter file:

```
LatticeParameters {
* Deformation potentials.
  NC = 3          # [1]
  NV = 2          # [1]
  DC2(1) = 0.9, -8.6, -8.6, 0, 0, 0
  DC2(2) = -8.6, 0.9, -8.6, 0, 0, 0
  DC2(3) = -8.6, -8.6, 0.9, 0, 0, 0
  DV2(1) = -2.1, -2.1, -2.1, -1, 0.5, 4
  DV2(2) = -2.1, -2.1, -2.1, 1, 0.5, 4
}
```

To modify the deformation potentials of $k \cdot p$ models, the following parameters in the section `LatticeParameters` should be used:

```
LatticeParameters {
* Deformation potentials of k.p model for electron bands
  xis = 7      # [eV]
  dbs = 0.53   # [eV]
  xiu = 9.16   # [eV]
  xid = 0.77   # [eV]
* Deformation potentials of k.p model for hole bands
  adp = 2.1    # [eV]
  bdp = -2.3300e+00 # [eV]
  ddp = -4.7500e+00 # [eV]
  dso = 0.044  # [eV]
}
```

The parameters `xis`, `dbs`, `xiu`, and `xid` correspond to the conduction band deformation potentials Ξ_u , Δ , Ξ_u , Ξ_d in Eq. 783, p. 692. The other parameters are the valence-band deformation potentials described in [11]:

- `adp` is the hydrostatic deformation potential.
- `bdp` is the shear deformation potential.

- ddp is the deformation potential.
- dso is the spin-orbit splitting energy.

Strained Effective Masses and Density-of-States

Sentaurus Device provides options for computing strain-dependent effective mass for both electrons and holes and, consequently, the strain-dependent conduction band and valence band effective density-of-states (DOS).

Strained Electron Effective Mass and DOS

The conduction band in silicon is approximated by three pairs of equivalent Δ_2 valleys. Without stress, the DOS of each valley is:

$$N_{C,i} = \frac{N_C}{3}, \quad i = 1, 3 \quad (787)$$

where N_C can be defined by two effective mass components m_l and m_t (see [Eq. 163, p. 262](#)).

As described in [Deformation of Band Structure on page 691](#), an applied stress induces a relative shift of the energy $\Delta E_{C,i}$ that is different for each Δ_2 valley. In addition, in the presence of shear stress, there is a large effective mass change for electrons. An analytic derivation for this mass change can be found in [\[12\]](#) and is based on a two-band $k \cdot p$ theory. To simplify the final expressions, the same dimensionless off-diagonal strain introduced in [Eq. 783](#) is used here:

$$\eta_i = \frac{4\Xi_u \varepsilon'_{jk}}{\Delta}, \quad j \neq k \neq i \quad (788)$$

Note that the ε'_{jk} strain affects only the Δ_2 valley along the i -axis, where $i = 1, 2$, or 3 represents the $[100]$, $[010]$, or $[001]$ axis, respectively.

When the $k \cdot p$ model [\[12\]](#) is evaluated at the band minima of the first conduction band, two different branches for the transverse effective mass are obtained where $m_{t1,i}$ is the mass across the stress direction, and $m_{t2,i}$ is the mass along the stress direction:

$$m_{t1,i}/m_t = \begin{cases} \left(1 - \frac{\eta_i}{M}\right)^{-1}, & |\eta_i| \leq 1 \\ \left(1 - \frac{\text{sign}(\eta_i)}{M}\right)^{-1}, & |\eta_i| > 1 \end{cases} \quad (789)$$

$$m_{t2,i}/m_t = \begin{cases} \left(1 + \frac{\eta_i}{M}\right)^{-1} & , |\eta_i| \leq 1 \\ \left(1 + \frac{\text{sign}(\eta_i)}{M}\right)^{-1} & , |\eta_i| > 1 \end{cases} \quad (790)$$

$$m_{l,i}/m_l = \begin{cases} (1 - \eta_i^2)^{-1} & , |\eta_i| < 1 \\ \left(1 - \frac{1}{|\eta_i|}\right)^{-1} & , |\eta_i| > 1 \end{cases} \quad (791)$$

In the above equations, M is a $k \cdot p$ model parameter that has been adjusted to provide a good fit with the empirical pseudopotential method (EPM) results.

The stress-induced change of the effective DOS for each valley can then be written as:

$$N_{C,i} = \sqrt{\left(\frac{m_{t1,i}}{m_t}\right)\left(\frac{m_{t2,i}}{m_t}\right)\left(\frac{m_{l,i}}{m_l}\right)} \cdot \left(\frac{N_C}{3}\right) \quad (792)$$

Accounting for the change of the stress-induced valley energy $\Delta E_{C,i}$ and the carrier redistribution between valleys, the strain-dependent conduction-band effective DOS can be derived for Boltzmann statistics:

$$N'_C = \gamma \cdot N_C \quad (793)$$

where:

$$\gamma = \frac{1}{N_C} \cdot \sum_{i=1}^3 N_{C,i} \cdot \exp\left(\frac{\Delta E_{C,\min} - \Delta E_{C,i}}{kT_n}\right) \quad (794)$$

and:

$$\Delta E_{C,\min} = \min(\Delta E_{C,i}) \quad (795)$$

This is incorporated into Sentaurus Device as a strain-dependent electron effective mass using:

$$m'_n = \gamma^{2/3} \cdot m_n \quad (796)$$

and:

$$N'_C = \left(\frac{m'_n}{m_n}\right)^{3/2} N_C \quad (797)$$

Strained Hole Effective Mass and DOS

To compute the hole effective DOS mass for arbitrary strain in silicon, the band structure provided by the six-band $k \cdot p$ method is explicitly integrated assuming Boltzmann statistics [13][14]. In this approximation, the total hole effective DOS mass is given by:

$$m'_p(T) = [m_{cc,1}^{3/2} e^{-(E_3 - E_1)/(kT)} + m_{cc,2}^{3/2} e^{-(E_3 - E_2)/(kT)} + m_{cc,3}^{3/2}]^{2/3} \quad (798)$$

where E_1 , E_2 , and E_3 are the ordered band edges for each of the three valence valleys, and $m_{cc,1}$, $m_{cc,2}$, and $m_{cc,3}$ are the carrier-concentration masses for each of the valleys given by:

$$m_{cc}^{3/2}(T) = \frac{2}{\sqrt{\pi}} (kT)^{-3/2} \int_0^{\infty} dE m_{DOS}^{3/2}(E) \sqrt{E} e^{-E/(kT)} \quad (799)$$

The energy-dependent DOS mass of each valley, m_{DOS} , is given by:

$$m_{DOS}^{3/2}(E) = \frac{\sqrt{2}\pi^2 \hbar^3}{\sqrt{E}} \frac{1}{(2\pi)^3} \int_0^{2\pi} d\phi \int_0^{\pi} d\theta \sin(\theta) \left[\left[\frac{\partial k}{\partial E} \right] k^2 \right]_{\phi, \theta, E} \quad (800)$$

The band structure–related integrand is computed from the inverse six-band $k \cdot p$ method in polar k -space coordinates. The integrals in Eq. 799 and Eq. 800 are evaluated using optimized quadrature rules.

The six-band $k \cdot p$ method is controlled by seven parameters:

- Three Luttinger–Kohn parameters (γ_1 , γ_2 , γ_3) that determine the band dispersion.
- The spin-orbit split-off energy (Δ_{so}).
- Three deformation potentials (a, b, d) that determine the strain response.

The Luttinger–Kohn parameters and Δ_{so} have been set to reproduce the DOS mass for relaxed silicon, m_p , as given by Eq. 166, p. 263 as a function of temperature. The default deformation potentials have been taken from the literature [15].

The strain-dependent valence-band effective DOS is then calculated from:

$$N'_V = \left(\frac{m'_p}{m_p} \right)^{3/2} N_V \quad (801)$$

Using Strained Effective Masses and DOS

The strain-dependent effective mass and DOS calculations can be selected by specifying `DOS(eMass)`, `DOS(hMass)`, or `DOS(eMass hMass)` as an argument to `Piezo(Model())` in the Physics section of the command file. For example:

```
Physics {  
    Piezo (  
        Model (  
            DOS (eMass hMass)  
        )  
    )  
}
```

Currently, these models have been calibrated only for strained silicon. Parameters affecting the `DOS(eMass)` model include the deformation potentials of the $k \cdot p$ model for electron bands (`xis`, `dbs`, `xiu`, `xid`) and the Sverdlov $k \cdot p$ parameter (`Mkp`). Parameters affecting the `DOS(hMass)` model include the deformation potentials of the $k \cdot p$ model for hole bands (`adp`, `bdp`, `ddp`, `dso`) and the Luttinger parameters (`gamma_1`, `gamma_2`, `gamma_3`). These parameters can be specified in the `LatticeParameters` section of the parameter file.

The `DOS(hMass)` model involves stress-dependent and lattice temperature-dependent numeric integrations that can be very CPU intensive. For simulations at a constant lattice temperature, you should only observe this CPU penalty once, usually at the beginning of the simulation. For nonisothermal thermal simulations, these integrations would usually have to be repeated for every lattice temperature change during the solution, resulting in a very prohibitive simulation.

As an alternative, Sentaurus Device provides an option for modeling the lattice temperature dependency of the strain-affected hole mass with analytic expressions that are fit to the full numeric integrations for each stress in the device. A CPU penalty will still be observed at the beginning of the simulation while fitting parameters are determined, but the remainder of the simulation should proceed as usual since the analytic expression evaluations are very fast.

By default, the analytic lattice temperature fit is used with thermal simulations and numeric integration is used for isothermal simulations. These defaults can be overridden by using the `AnalyticLTfit` or `NumericalIntegration` options for `DOS(hMass)`:

```
DOS(eMass hMass(NumericalIntegration))  
DOS(eMass hMass>AnalyticLTfit))
```


Multivalley Band Structure

For a general description of the multivalley model and its implementation, see [Multivalley Band Structure on page 267](#).

For stress simulations, the multivalley model uses analytic bands computed by two-band $k \cdot p$ [12] and 6×6 $k \cdot p$ [11] models for electrons and holes, respectively. These models consider three separate valleys in both the conduction and valence bands. The valley energy shifts related to the models are described in [Deformation of Band Structure on page 691](#).

For electrons, the $k \cdot p$ model gives an analytic expression (Eq. 783, p. 692 for $\Delta E_{c,i}$), which defines the valley energy shifts. However, for holes, it is based on a numeric solution of the 6×6 $k \cdot p$ cubic equation.

Computation of the effective DOS factors of Eq. 179, p. 267 and Eq. 180, p. 268 is based on the same $k \cdot p$ models and uses a stress-induced change of the effective DOS masses for each valley described in [Strained Effective Masses and Density-of-States on page 695](#). Referring to Eq. 792 and Eq. 798, and using their variables, the effective DOS factors of each valley can be expressed as follows:

$$g_{n,i} = \frac{1}{3} \sqrt{\left(\frac{m_{t1,i}}{m_t}\right) \left(\frac{m_{t2,i}}{m_t}\right) \left(\frac{m_{l,i}}{m_l}\right)} \quad g_{p,i} = \left(\frac{m_{cc,i}}{m_p}\right)^{3/2} \quad (802)$$

Using Multivalley Band Structure

For stress simulations (with the Piezo statement in the Physics section), the model ignores most of the valley parameters except the nonparabolicity α (see [Nonparabolic Band Structure on page 268](#)) and the optional quantization mass m_q (see [Using MLDA on page 295](#)) in the parameter file, which are described also in [Using Multivalley Band Structure on page 269](#).

To transfer the user-defined parameters α and m_q to stressed $k \cdot p$ bands, there is a simple scheme:

- For electrons, the first eValley statement assigns these parameters to the (100) valley, the second one to the (010) valley, and the third one to the (001) valley.
- For holes, the first hValley statement assigns it to the light-hole band, and the second one assigns it to the heavy-hole band.

NOTE Using the multivalley MLDA model, the quantization mass m_q is optional in the eValley and hValley statements. For stress problems, by default, the quantization mass for electrons is computed as described in [Inversion Layer on page 704](#). However, for holes, as suggested by Reggiani [16], $m_{q,th} = 0.245$ and $m_{q,hh} = 0.43$.

The model is activated with the keyword `MultiValley` in the `Physics` section. If the model must be activated only for electrons or holes, the keywords `eMultiValley` and `hMultiValley` can be used. The model allows you to simplify the `Piezo` section where the `DeformationPotential` and `DOS` statements should not be used.

NOTE Although the multivalley model can work together with any `DeformationPotential` model (it recomputes the $k \cdot p$ valley energy shifts with reference to the band edges defined by the deformation potential model), Sentaurus Device stops with an error message if the `DOS` statement is used (see [Strained Effective Masses and Density-of-States on page 695](#)).

Stress-dependent Mobility Models

The presence of mechanical stress in device structures results in anisotropic carrier mobility that must be described by a mobility tensor. The electron and hole current densities under such conditions are given by:

$$\vec{J}_\alpha = \left(\frac{\bar{\mu}_\alpha}{\mu_{\alpha 0}} \right) \vec{J}_{\alpha 0}, \quad \alpha = n, p \quad (803)$$

where:

- $\bar{\mu}_\alpha$ is the stress-dependent mobility tensor.
- $\mu_{\alpha 0}$ denotes the isotropic mobility without stress.
- $J_{\alpha 0}$ is the carrier current density without stress.

The following sections describe options available in Sentaurus Device for including the effects of stress on carrier mobility.

Stress-induced Electron Mobility Model

To calculate stress-induced electron mobility, Sentaurus Device considers several approaches [\[6\]\[12\]\[17\]\[18\]](#) for the mobility approximation, and it computes the mobility ratio (3×3 tensor), which corrects the relaxed current density in [Eq. 803](#). The simplest approach [\[6\]](#) focuses on the modeling of the mobility changes due to the carrier redistribution between bands in silicon. As a known example, the electron mobility is enhanced in a strained-silicon layer grown on top of a thick, relaxed SiGe layer. Due to the lattice mismatch (which can be controlled by the Ge mole fraction), the thin silicon layer appears to be ‘stretched’ (under biaxial tension).

The origin of the electron mobility enhancement can be explained [6] by considering the six-fold degeneracy in the conduction band. The biaxial tensile strain lowers two perpendicular valleys (Δ_2) with respect to the four-fold in-plane valleys (Δ_4). Therefore, electrons are redistributed between valleys and Δ_2 is occupied more heavily. It is known that the perpendicular effective mass is much lower than the longitudinal one. Therefore, this carrier redistribution and reduced intervalley scattering enhance the electron mobility.

The model consistently accounts for a change of band energy as described in [Deformation of Band Structure on page 691](#), that is, a modification of the deformation potentials in [Eq. 782, p. 692](#) will affect the strain-silicon mobility model. In the crystal coordinate system, the model gives only the diagonal elements of the electron mobility matrix.

The following expressions have been suggested for the electron mobility:

$$\mu_{n,ii} = \mu_{n0} \left[1 + \frac{1 - m_{nl}/m_{nt}}{1 + 2(m_{nl}/m_{nt})} \left(\frac{F_{1/2} \left(\frac{F_n - E_C - \Delta E_{C,i}}{kT} \right)}{F_{1/2} \left(\frac{F_n - E_C - \Delta E_C}{kT} \right)} - 1 \right) \right] \quad (804)$$

where:

- μ_{n0} is electron mobility without the strain.
- m_{nl} and m_{nt} are the electron longitudinal and transverse masses in the subvalley, respectively.
- $\Delta E_{C,i}$ and ΔE_C are computed by [Eq. 782](#) and [Eq. 784](#) or [Eq. 785](#), respectively.
- The index i corresponds to a direction (for example, $\mu_{n,11}$ is the electron mobility in the direction of the x-axis of the crystal system and, therefore, $\Delta E_{C,1}$ should correspond to the two-fold subvalley along the x-axis).
- F_n is quasi-Fermi level of electrons.

NOTE For the carrier quasi-Fermi levels, as for [Eq. 804](#), there are two options to be computed: (a) assuming the charge neutrality between carrier and doping, and it gives only the doping dependence of the model and (b) using the local carrier concentration (see [Using Stress-induced Electron Mobility Model on page 706](#)).

In the case of Boltzmann statistics, [Eq. 804](#) is simplified [6] and the dependence on the carrier (doping) concentration disappears:

$$\mu_{n,ii} = \mu_{n0} \left[1 + \frac{1 - m_{nl}/m_{nt}}{1 + 2(m_{nl}/m_{nt})} \left(\exp \left(\frac{\Delta E_C - \Delta E_{C,i}}{kT} \right) - 1 \right) \right] \quad (805)$$

Derivation of [Eq. 804](#) is based on consideration of the change in the stress-induced band energy in bulk silicon. However, in MOSFETs, there is an additional influence of a quantization that appears in the carrier channel at the silicon–oxide interface. For example, there is a reduction

of the stress effect with applied gate voltage. Partially, this is due to the Fermi-level dependency on the carrier concentration, as well as to the quantization in the channel gives a different carrier redistribution between electron bands (see [Inversion Layer on page 704](#)).

Intervalley Scattering

Another effect of the stress-induced mobility change is intervalley scattering. In the literature [\[17\]](#), the total relaxation time in valley i is expressed as follows:

$$\frac{1}{\tau_i} = \frac{1}{\tau^g} + \frac{1}{\tau_i^f} + \frac{1}{\tau^l} \quad (806)$$

where:

- τ^g denotes the momentum relaxation time due to acoustic intravalley scattering and intervalley scattering between equivalent valleys (g-type scattering).
- τ^l is the impurity scattering relaxation time.
- τ_i^f is the relaxation time for intervalley scattering between nonequivalent valleys (f-type scattering).

The intervalley scattering rate for electrons to scatter from initial valley i to final valley j can be defined as [\[17\]](#):

$$\begin{aligned} S(\epsilon, \Delta_{ij}) &= C \left[(\epsilon - \Delta_{ij}^{\text{emi}})^{1/2} + \exp\left(\frac{\hbar\omega_{\text{opt}}}{kT}\right) (\epsilon - \Delta_{ij}^{\text{abs}})^{1/2} \right] \\ \Delta_{ij}^{\text{emi}} &= \Delta E_{C,j} - \Delta E_{C,i} - \hbar\omega_{\text{opt}} \\ \Delta_{ij}^{\text{abs}} &= \Delta E_{C,j} - \Delta E_{C,i} + \hbar\omega_{\text{opt}} \end{aligned} \quad (807)$$

where $\hbar\omega_{\text{opt}}$ is the phonon energy and C is a constant. The relaxation time for this scattering event can be defined as follows:

$$\frac{1}{\tau_{(i \rightarrow j)}} = \int_0^\infty S(\epsilon, \Delta_{ij}) f(\epsilon, F_n) d\epsilon \quad (808)$$

where $f(\epsilon, F_n)$ is the electron distribution function.

Using the Fermi–Dirac distribution function and considering that electrons from the valley i can be scattered into two valleys j and l ($1/\tau_i^f = 1/\tau_i^f(i \rightarrow j) + 1/\tau_i^f(i \rightarrow l)$), a ratio between unstrained and strained relaxation times for this intervalley scattering in valley i can be written as:

$$h_i = \frac{\tau_i^{f0}}{\tau_i^f} = \frac{F_{1/2}\left(\frac{\eta - \Delta_{ij}^{\text{emi}}}{kT}\right) + F_{1/2}\left(\frac{\eta - \Delta_{il}^{\text{emi}}}{kT}\right) + \exp\left(\frac{\hbar\omega_{\text{opt}}}{kT}\right) \left[F_{1/2}\left(\frac{\eta - \Delta_{ij}^{\text{abs}}}{kT}\right) + F_{1/2}\left(\frac{\eta - \Delta_{il}^{\text{abs}}}{kT}\right) \right]}{2 \left[F_{1/2}\left(\frac{\eta + \hbar\omega_{\text{opt}}}{kT}\right) + \exp\left(\frac{\hbar\omega_{\text{opt}}}{kT}\right) F_{1/2}\left(\frac{\eta - \hbar\omega_{\text{opt}}}{kT}\right) \right]} \quad (809)$$

where $\eta = F_n - E_C$.

NOTE The authors in [17] used the Boltzmann distribution function to express h_i . As a result, Eq. 20 of [17] does not have any doping dependence, but Eq. 809 does have it through the doping dependence of the Fermi level F_n .

Considering undoped/doped and unstrained/strained cases, the paper [17] derives an expression for total mobility change in valley i , which is based on a doping-dependent mobility model. With simplified doping dependence, a ratio between strained and unstrained total relaxation times for the valley i can be expressed as follows:

$$\frac{\tau_i}{\tau_0} = \frac{1}{1 + (h_i - 1) \frac{1 - \beta^{-1}}{1 + \left(\frac{N_{\text{tot}}}{N_{\text{ref}}}\right)^\alpha}} \quad (810)$$

where N_{tot} is the sum of donor and acceptor impurities, and $\beta = 1 + \tau^g/\tau^{f0}$ is a fitting parameter [17] as both others N_{ref} and α .

The final modification of the mobility along valley i , which includes the stress-induced carrier redistribution and change in the intervalley scattering, is:

$$\mu_{n,ii} = \frac{3\mu_{n0}}{1 + 2(m_{nl}/m_{nt})} \cdot \frac{\frac{\tau_i}{\tau_0} F_{1/2}\left(\frac{\eta - \Delta E_{C,i}}{kT}\right) + \frac{\tau_j m_{nl}}{\tau_0 m_{nt}} F_{1/2}\left(\frac{\eta - \Delta E_{C,j}}{kT}\right) + \frac{\tau_l m_{nl}}{\tau_0 m_{nt}} F_{1/2}\left(\frac{\eta - \Delta E_{C,l}}{kT}\right)}{F_{1/2}\left(\frac{\eta - \Delta E_{C,i}}{kT}\right) + F_{1/2}\left(\frac{\eta - \Delta E_{C,j}}{kT}\right) + F_{1/2}\left(\frac{\eta - \Delta E_{C,l}}{kT}\right)} \quad (811)$$

NOTE The model (Eq. 809 and Eq. 810) was developed originally for the bulk case. However, in MOSFET channels, to obtain an agreement with experimental data, the model and parameter β , responsible for the unstressed ratio between g-type and f-type scatterings, must be modified (see Inversion Layer on page 704).

Effective Mass

It is known that the effective mass of electrons does not change significantly if the stress is applied along the crystal axis, and it allows you to write the stress-induced mobility change in the form of [Eq. 804](#) and [Eq. 811](#). However, an analysis based on the empirical nonlocal pseudopotential theory [\[18\]](#) shows that, for the stresses applied along $\langle 110 \rangle$, the effective mass changes strongly and it affects the electron mobility. The literature [\[18\]](#) suggests a simple empirical polynomial approximation for the dependency of the effective mass on stress applied along $\langle 110 \rangle$. Later, the two-band $k \cdot p$ theory [\[12\]](#) was developed for electrons with the main analytic results for the effective masses described in [Strained Electron Effective Mass and DOS on page 695](#).

To formulate the stress-induced change of the mobility generally (accounting for arbitrary channel and interface orientations), the inverse conductivity mass tensor for the valley i is represented in the following very general form:

$$(\bar{m}_{C,i})^{-1} = \frac{1}{\hbar^2} \frac{\partial^2 \varepsilon_i}{\partial k_j \partial k_l} \quad (812)$$

where:

- ε_i is the energy defined by the two-band $k \cdot p$ theory.
- k_j, k_l are the wavevectors.
- The inverse mass tensor $(\bar{m}_{C,i})^{-1}$ is computed in the valley energy minima, which is a good approximation for electron conductivity masses in silicon.

To account for such stress-induced mass change, [Eq. 804](#) and [Eq. 811](#) are generalized as stated in the next section, [Inversion Layer](#).

Inversion Layer

Generally, using only the valley data, the total mobility tensor could be written in the following general form, which generalizes [Eq. 804](#), [Eq. 805](#), and [Eq. 811](#):

$$\bar{\mu}_n = q\tau_0 \sum_i \frac{n_i \tau_i}{n \tau_0} (\bar{m}_{C,i})^{-1} \quad (813)$$

where:

- n_i/n is a local valley occupation.
- τ_i/τ_0 is the ratio of stressed and unstressed relaxation times as it is in [Eq. 810](#).
- $(\bar{m}_{C,i})^{-1}$ is the inverse conductivity mass tensor [Eq. 812](#).

To use [Eq. 813](#) in Sentaurus Device stress modeling, the mobility tensor $\bar{\mu}_n$ should be divided by unstressed mobility μ_{n0} because the stress effect is accounted for as a multiplication factor to TCAD mobility (for example, the Lombardi model) used in the device simulation. Therefore, the unstressed mobility μ_{n0} also is computed by [Eq. 813](#), but with zero stress.

In the multivalley representation, [Eq. 813](#) is general enough to describe the mobility for both bulk and MOSFET inversion layer cases, but a difference between these cases is in the n_i/n and τ_i/τ_0 terms. For inversion layers, the quantization effect must be accounted for in each valley separately.

At this point, only the multivalley MLDA model gives such a possibility (see [MLDA Model on page 293](#)). Therefore, for the inversion layer mobility, the local valley occupation is computed by the multivalley MLDA model where the quantization mass m_q in the perpendicular direction to the interface is computed using stressed $k \cdot p$ bands.

NOTE The multivalley MLDA model accounts for arbitrary interface orientation automatically (see [Interface Orientation and Stress Dependencies on page 293](#)), which complements the general inverse conductivity mass tensor $(\bar{m}_{C,i})^{-1}$ in [Eq. 812](#), and it gives users an automatic option that accounts for both channel and interface orientations simultaneously.

For the inversion layer mobility model, there are two changes in the bulk scattering ratio τ_i/τ_0 in [Eq. 809](#). First, the DOS used in [Eq. 807](#) is a parabolic bulk DOS. The MLDA model ([Eq. 212](#)) gives an MLDA DOS that becomes the bulk DOS at a distance from the interface where the quantum effect is small. You have an option to activate the MLDA DOS in [Eq. 807](#) and have a numeric integration in [Eq. 808](#) with Gauss–Laguerre quadratures as described in [Nonparabolic Band Structure on page 268](#). Second, based on comparisons to various stress- and orientation-dependent experimental data, the new user-defined parameter for β in the inversion layer is introduced (see [Using Stress-induced Electron Mobility Model on page 706](#)).

For the bulk case, the unstressed mobility μ_{n0} computed by [Eq. 813](#) is always isotropic. However, for the MOSFET on a (110) silicon substrate, it is not. In this case, [Eq. 813](#) without stress gives anisotropic unstressed electron mobility, and a ratio between the mobilities of the $\langle 100 \rangle$ and $\langle 110 \rangle$ channel orientations is equal to approximately 1.1. This ratio is in reasonable agreement to experimentally observed data where the ratio changes from 1.2 to 1.1 for a high-gate electric field.

NOTE By default, the unstressed mobility μ_{n0} (in [Eq. 803](#)) is always computed for a $\langle 110 \rangle$ channel orientation and for a substrate orientation that corresponds to the auto-orientation Lombardi option (see [Orientation-dependent Lombardi Parameters on page 319](#)). Such a default requires you to have calibrated the Lombardi model parameters for the $\langle 110 \rangle$ channel (critical for MOSFETs on (110) substrate where the unstressed mobility is anisotropic).

Using Stress-induced Electron Mobility Model

The stress-induced mobility model can be activated regionwise or materialwise with the following keyword in the `Mobility` statement of the `Piezo` model:

```
Physics {  
    Piezo( Model(Mobility(eSubband)) )  
}
```

The keyword `eSubband` assumes Boltzmann statistics and, in this case, [Eq. 805](#) is used. To activate doping dependency and [Eq. 804](#), the keywords `eSubband(Doping)` should be used, but if you need to influence the Fermi level on carrier redistribution between bands (for example, in the MOSFET channel), then use `eSubband(Fermi)`.

The model parameters (see [Eq. 804](#)) can be specified in the parameter file in the `StressMobility` section as follows:

```
StressMobility {  
    me_lt = 4.81      # [1]  
    elec_100 = 1      # [1]  
    elec_010 = 2      # [1]  
    elec_001 = 3      # [1]  
}
```

where `me_lt` is the ratio m_{nl}/m_{nt} used in [Eq. 804](#), [Eq. 805](#), and [Eq. 811](#). The numbers `elec_100`, `elec_010`, and `elec_001` correspond to indices of deformation potentials in the `LatticeParameters` section (see [Using Deformation Potential Model on page 693](#)) for two-fold electron subvalleys in the direction `[100]`, `[010]`, and `[001]`, respectively.

NOTE The parameters `elec_100`, `elec_010`, and `elec_001` are not used for the advanced two-band $k \cdot p$ model `DeformationPotential(eKP)`.

To activate the intervalley scattering model (see [Intervalley Scattering on page 702](#)) with Boltzmann statistics (no doping and carrier concentration dependency), use the keywords `eSubband(Scattering)`. However, to activate doping dependency in both scattering and carrier redistribution ([Eq. 811](#)), use `eSubband(Doping Scattering)`.

The parameters of the intervalley scattering model can be specified in the same `StressMobility` section of the parameter file:

```
StressMobility {  
    Ephonon = 0.06     # [eV]  
    beta = 1.22        # [1]  
    Nref = 3e19         # [cm^-3]  
    alpha = 0.65       # [1]  
}
```


where Ephoton is $\hbar\omega_{\text{opt}}$ in Eq. 807, and beta, Nref, and alpha are parameters of Eq. 808.

To activate the model for a stress-induced change of the electron effective mass (see [Effective Mass on page 704](#)), use the keywords `Mobility(eSubband(EffectiveMass))`.

The unstressed longitudinal and perpendicular effective masses m_l and m_t for this model (to be used in two-band $k \cdot p$ theory; see [Strained Electron Effective Mass and DOS on page 695](#)) are defined as follows:

```
StressMobility {
    me_l0 = 0.914      # [1]
    me_t0 = 0.196      # [1]
}
```

To activate the inversion layer model (see [Inversion Layer on page 704](#)), the statement `eMultiValley(MLDA)` must be specified in the `Physics` section. It activates the multivalley MLDA quantization model (see [MLDA Model on page 293](#)), which is applied to both the density and the inversion layer stress-induced mobility model. To use another quantization model in the density computation, specify the `-Density` option in the `eMultiValley` statement (see [MLDA Application Notes on page 298](#)). To ensure that electron stress-related models are consistent and physically correct, the `Physics` section should contain the following:

```
Physics{
    eMultiValley(MLDA)
    Piezo( Model( Mobility( eSubband(Fermi Scattering EffectiveMass) ) ) )
}
```

For the Dhar scattering model, the MLDA DOS can be used with the statement `Scattering(MLDA)`, but note that, due to the numeric integration in Eq. 808, it will increase simulation time. In the vicinity of a MOSFET channel, the scattering model will use the user-defined parameter `beta_mlda` to define the model parameter β in Eq. 810:

```
StressMobility {
    beta_mlda = 1.5      # [1]
}
```

NOTE The inversion layer model is designed to work with arbitrary channel and interface orientations. Generally, this orientation is simply defined by a specification of only the X and Y simulation coordinate axes in reference to the crystal coordinate system in the `LatticeParameters` section (see [Using Deformation Potential Model on page 693](#)).

NOTE If you want to use the $\langle 100 \rangle / (110)$ Lombardi model parameters (not the default $\langle 110 \rangle$ channel orientation for (110) substrate), the keyword `-RelChDir110` must be used inside the `eSubband` statement.

Stress-induced Hole Mobility Model

Similar to the electron stress-induced mobility model (Eq. 813), the total hole mobility tensor can be written in the following general form, which can be applied to both bulk and inversion layer cases [19]:

$$\bar{\mu}_p = q\tau_0 \sum_i \frac{p_i}{p} \frac{\tau}{\tau_0} (\bar{m}_{C,i})^{-1} \quad (814)$$

where:

- p_i/p is a local hole-band occupation.
- τ/τ_0 is a ratio of the stressed and unstressed valence-band relaxation times.
- $(\bar{m}_{C,i})^{-1}$ is the inverse conductivity hole mass tensor.

The model accounts for the six-band $\mathbf{k} \cdot \mathbf{p}$ hole band structure [13] in all of the above three terms of Eq. 814. In the case of the inversion layer, the model computes the six-band $\mathbf{k} \cdot \mathbf{p}$ MLDA DOS of each band as described in MLDA Model on page 293 and, correspondingly, it affects all terms of Eq. 814. Finally, the model computes the mobility ratio (3×3 tensor), which corrects the relaxed current density in Eq. 803.

Effective Mass

The inverse mass tensor $(\bar{m}_{C,i})^{-1}$ of the band i in Eq. 814 is based on an averaging of the reciprocal mass tensor in $\mathbf{k}$ -space and energy space. If the reciprocal mass tensor at any $\mathbf{k}$ -vector is expressed as follows:

$$w_{jl}^i(\vec{k}) = \frac{1}{\hbar^2} \frac{\partial^2 \varepsilon_i(\vec{k})}{\partial k_j \partial k_l} \quad (815)$$

then the averaging in $\mathbf{k}$ -space is performed in accordance to the DOS computation (Eq. 215, p. 294), which for the inversion layer can be formulated as:

$$w_{jl}^i(\varepsilon, z) = \frac{\frac{2}{(2\pi)^3} \oint_{\varepsilon(\vec{k})=\varepsilon} \frac{w_{jl}^i(\vec{k})}{|\nabla_{\vec{k}} \varepsilon(\vec{k})|} \left[1 - \exp \left(i 2z \gamma_{kp} \sum_j k_j \frac{w_{j3}(\vec{k})}{w_{33}(\vec{k})} \right) \right] dS}{D^i(\varepsilon, z)} \quad (816)$$

Finally, the inverse mass tensor of the band i is computed as an averaged value over the energy space with the carrier distribution function $f(\epsilon, F_p)$ accounted:

$$(\bar{m}_{C,i})^{-1} = \frac{\int_0^\infty D^i(\epsilon, z) w_{jl}^i(\epsilon, z) f(\epsilon, F_p) d\epsilon}{p_i} \quad (817)$$

where:

- p_i is a hole-band concentration ($p_i(F_p, z) = \int_0^\infty D^i(\epsilon, z) f(\epsilon, F_p) d\epsilon$).
- $(\bar{m}_{C,i})^{-1}$ is a symmetric 3×3 tensor, which depends on the quasi-Fermi energy F_p and the distance from interface z (for the inversion layer case).

Similar to the computation of the hole-band concentration p_i , the integral over the energy in Eq. 817 is computed using Gauss–Laguerre quadratures and, for that, the energy-dependent reciprocal mass tensor (Eq. 816) is computed on a predefined energy mesh (same as for the DOS $D^i(\epsilon, z)$).

Scattering

The scattering model defines the ratio of the stressed and unstressed valence-band momentum relaxation times τ/τ_0 . The model considers four scattering mechanisms (which are affected by the stress) for holes in each band assisted by: acoustic phonon, optical phonon emission, optical phonon absorption, and simplified impurity scattering. The phonon-assisted relaxation times can be expressed as follows for the band i in the case of the inversion layer:

$$\begin{aligned} \frac{\sum D^j(\epsilon, z)}{\tau_{ac}^i(\epsilon, z)} &= \frac{2\pi kT}{\hbar \rho} \left(\frac{D_{ac}}{c_l} \right)^2 \sum_j \sum_k D_{bulk}^j(\epsilon) D_{bulk}^k(\epsilon - \Delta\epsilon_0^{k,i}) \\ \frac{\sum D^j(\epsilon, z)}{\tau_{ope}^i(\epsilon, z)} &= \frac{\pi \hbar D_{op}^2}{\rho \epsilon_{op}} (N_{op} + 1) \sum_j \sum_k D_{bulk}^j(\epsilon) D_{bulk}^k(\epsilon - \epsilon_{op} - \Delta\epsilon_0^{k,i}) \\ \frac{\sum D^j(\epsilon, z)}{\tau_{opa}^i(\epsilon, z)} &= \frac{\pi \hbar D_{op}^2}{\rho \epsilon_{op}} N_{op} \sum_j \sum_k D_{bulk}^j(\epsilon) D_{bulk}^k(\epsilon + \epsilon_{op} - \Delta\epsilon_0^{k,i}) \end{aligned} \quad (818)$$

where:

- $D_{bulk}^i(\epsilon) = \frac{2}{(2\pi)^3} \oint_{\epsilon^i(\vec{k}) = \epsilon} \frac{dS}{|\nabla_{\vec{k}} \epsilon^i(\vec{k})|}$ is the hole bulk DOS of the band i .
- $\Delta\epsilon_0^{k,i}$ is a band minimum energy difference between bands i and k .
- $\frac{D_{ac}}{c_l}$ is a ratio of the acoustic-phonon deformation potential by the sound velocity.

- ρ is the mass density.
- ϵ_{op} is the optical-phonon energy.
- D_{op} is the optical-phonon deformation potential.
- $N_{op} = [\exp(\epsilon_{op}/kT) - 1]^{-1}$ is the phonon number.

NOTE For the bulk case, $D^i(\epsilon, z) = D_{bulk}^i(\epsilon)$ and, therefore, Eq. 818 can be simplified to regular bulk scattering rate equations. Eq. 818 accounts for both interband and intraband phonon scattering with the same deformation potentials, but you have an option to control the interband scattering factor.

Based on Eq. 818, the total phonon-limited hole-scattering rate in the band i is written as:

$$\frac{1}{\tau_{ph}^i(\epsilon, z)} = \frac{1}{\tau_{ac}^i(\epsilon, z)} + \frac{1}{\tau_{ope}^i(\epsilon, z)} + \frac{1}{\tau_{opa}^i(\epsilon, z)} \quad (819)$$

Therefore, the macroscopic phonon-limited momentum relaxation time of the full valence band can be expressed similarly for electrons (Eq. 809):

$$\frac{1}{\tau^{ph}} = \frac{\sum_i \int_0^\infty D^i(\epsilon, z) \frac{1}{\tau_{ph}^i(\epsilon, z)} f(\epsilon, F_p) d\epsilon}{p} \quad (820)$$

where τ^{ph} depends on the quasi-Fermi energy F_p and the distance from interface z (for the inversion layer case). The impurity scattering is introduced also similarly to how it is performed for electrons in Eq. 810, and the ratio of the stressed and unstressed valence-band relaxation times in Eq. 814 is expressed as follows:

$$\frac{\tau}{\tau_0} = \frac{1}{1 + \left(\frac{\tau_0^{ph}}{\tau^{ph}} - 1 \right) \frac{1 - \beta^{-1}}{1 + \left(\frac{N_{tot}}{N_{ref}} \right)^\alpha}} \quad (821)$$

where:

- τ_0^{ph} is the phonon-limited momentum relaxation time in the valence band for zero stress computed using Eq. 819 and Eq. 820.
- N_{tot} is the doping concentration.
- N_{ref}, α are fitting parameters of the impurity scattering model.
- β is a fitting parameter that can be represented as $\beta = 1 + \tau^{indep}/\tau_0^{ph}$, where τ^{indep} is a relaxation time of other scattering mechanisms that are not accounted for in Eq. 818 and that are independent of the stress. Therefore, for example, if the stress-independent

scattering rate (not accounted for in Eq. 818) is much higher than $1/\tau_0^{\text{ph}}$, then $\beta = 1$ and therefore $\tau/\tau_0 = 1$. However, if this rate is negligible, then $\beta^{-1} = 0$.

NOTE The default value of the parameter β is different for the bulk and inversion layer simulations. Some fitting of this parameter was performed to obtain a reasonable agreement of the model (Eq. 814) to multiple stress and orientation data.

Using Stress-induced Hole Mobility Model

The stress-induced hole mobility model can be activated only with `hMultivalley(kpDOS)` because the model uses the six-band $k \cdot p$ valence band structure. The model works in two modes: bulk and inversion layer (in the presence of `hMultivalley(MLDA kpDOS)` and inside semiconductor–insulator interface vicinity). With `hMultivalley(MLDA)`, the multivalley MLDA quantization model (see [MLDA Model on page 293](#)) is applied to both the density and the inversion layer stress-induced mobility model. To use another quantization model in the density computation, specify the `-Density` option in the `hMultivalley` statement (see [MLDA Application Notes on page 298](#)).

The model can be used regionwise or materialwise and, typically, for PMOSFET stress simulations, it can be activated with the following keyword in the `Mobility` statement of the `Piezo` model:

```
Physics {
  hMultivalley( MLDA kpDOS )
  Piezo( Model(Mobility(hSubband(Fermi EffectiveMass Scattering(MLDA)))) )
}
```

The keyword `Fermi` defines that the Fermi–Dirac distribution function is used in Eq. 817 and Eq. 820 with the carrier self-consistent quasi-Fermi energy. To activate only doping-dependent quasi-Fermi energy, the keywords `hSubband(Doping)` should be used. If neither of these keywords (`Fermi` and `Doping`) is used, the Boltzmann distribution function is used and this illuminates the quasi-Fermi energy dependency in the mobility model (Eq. 814).

The keyword `EffectiveMass` means that the inverse mass tensor (Eq. 817) is computed. If this keyword is not present, the unit diagonal (and stress-independent) tensor is used as $(\bar{m}_{C,i})^{-1}$ in Eq. 814.

The statement `Scattering(MLDA)` means that the scattering model (Eq. 818–Eq. 821) is used. If the keyword `Scattering` is not present, then τ/τ_0 is unit and stress independent in Eq. 814. The keyword `MLDA` in the `Scattering` statement defines that MLDA DOS is used in the scattering rate (exactly as it is in Eq. 818), but using only the keyword `Scattering` (without `MLDA`) gives the bulk scattering rates ($D^j(\epsilon, z) = D_{\text{bulk}}^j(\epsilon)$ in Eq. 818).

The scattering model parameters (see [Eq. 818](#) and [Eq. 821](#)) can be specified in the parameter file in the StressMobility section as follows:

```
StressMobility {
  Ephnon_h = 0.0612          # [eV]
  Dop_h = 7.47e8              # [eV/cm]
  Dac_cl_h = 7.5e-6          # [eVs/cm]
  beta_h = 1e10               # [1]
  beta_mlda_h = (6.5,1.2,2.5) # [1]
  Nref_h = 3e19               # [cm^-3]
  alpha_h = 0.85              # [1]
}
```

where Ephnon_h corresponds to ϵ_{op} used in [Eq. 818](#), Dop_h is D_{op} , Dac_cl_h is D_{ac}/c_l , beta_h is β in [Eq. 821](#) for the bulk case, and beta_mlda_h defines β for the inversion layer case for three interface orientations (100), (110), (111). These β values are a result of the calibration of the stress effect in mobility produced by the model in comparison to measurement and other simulation data. The parameters Nref_h and alpha_h correspond to the impurity scattering parameters N_{ref} , α in [Eq. 821](#).

All conductivity mass-related parameters (used to compute $(\bar{m}_{C,i})^{-1}$ in [Eq. 817](#)) are defined by the six-band $k \cdot p$ model and must be specified in the LatticeParameter section (see [Using Strained Effective Masses and DOS on page 698](#)).

NOTE By default, the unstressed mobility μ_{p0} (in [Eq. 803](#)) is always computed for a <110> channel orientation and for substrate orientation, which corresponds to the auto-orientation Lombardi option (see [Orientation-dependent Lombardi Parameters on page 319](#)). Such a default requires you to have calibrated the Lombardi model parameters for the <110> channel (critical for MOSFETs on (110) substrate where the unstressed mobility is anisotropic). However, if you want to use <100>/(110) Lombardi model parameters, the keyword -RelChDir110 must be used inside the hSubband statement.

NOTE To compute the unstressed mobility μ_{p0} for (100) substrate orientation (and to use only (100) Lombardi model parameters), the auto-orientation Lombardi option must not be activated and the keyword -AutoOrientation must be used inside the hSubband statement.

Intel Stress-induced Hole Mobility Model

Intel [20] suggested a mobility model for strained PMOS devices based on the occupancy of different parts of the topmost valence band. As shown in Figure 51, under zero stress, the topmost valence band has a fourfold symmetry with arms along $\langle 110 \rangle$ and $\langle \bar{1}10 \rangle$. Each ellipsoid is characterized by a transverse mass m_t and a longitudinal mass m_l .

As compressive uniaxial stress along $\langle 110 \rangle$ is applied, one of the arms shrinks and the band becomes a single ellipsoidal band. In Figure 51, this modification of the band structure is modeled as the splitting and change in curvature of two ellipsoidal bands. With applied stress, the maximum energy associated with each of these bands splits with carriers preferentially occupying the upper valley. In addition, the transverse and longitudinal effective masses of each of these two valleys change with stress.

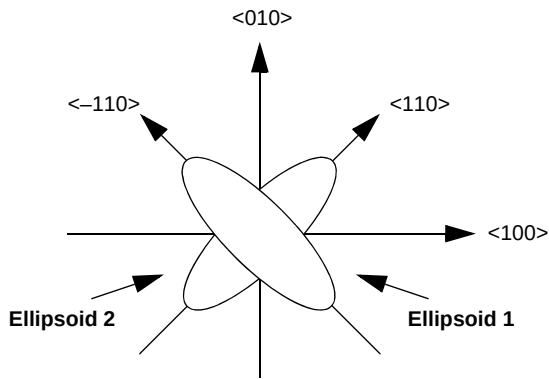


Figure 51 Sketch of Intel two-ellipsoid model; one ellipsoid is aligned along $\langle 110 \rangle$ and the other along $\langle \bar{1}10 \rangle$

Under zero stress, the occupancies of the two ellipsoids are assumed equal and the mobility is isotropic with a value:

$$\mu_0 = q \langle \tau \rangle \left[\frac{0.5}{m_{t0}} + \frac{0.5}{m_{l0}} \right] \quad (822)$$

where $\langle \tau \rangle$ is the scattering time and the index '0' in effective masses corresponds to the unstressed case.

Accounting for stress-induced reoccupation of these ellipsoids f_1 and f_2 , and assuming that the scattering time $\langle \tau \rangle$ is the same for both valleys and is independent of stress, the relative change in mobility is given by a diagonal tensor in the $\langle 110 \rangle$, $\langle \bar{1}10 \rangle$ coordinate system as [20]:

$$\begin{bmatrix} 1 + \frac{\Delta\mu_{110}}{\mu_0} \\ 1 + \frac{\Delta\mu_{\bar{1}10}}{\mu_0} \end{bmatrix} = \frac{2m_{10}m_{t0}}{m_{10} + m_{t0}} \begin{bmatrix} \frac{f_1}{m_{t1}} + \frac{f_2}{m_{12}} & 0 \\ 0 & \frac{f_1}{m_{11}} + \frac{f_2}{m_{t2}} \end{bmatrix} \quad (823)$$

Denoting the energy split between the two values as Δ , the occupancies for the two valleys are given by:

$$\begin{aligned} f_1 &= \frac{1}{1 + \exp(-\Delta/kT)} \\ f_2 &= 1 - f_1 \end{aligned} \quad (824)$$

where T is the lattice temperature.

Stress Dependencies

Intel decomposes the planar stress tensor in the crystallographic coordinate system as:

$$S = \begin{bmatrix} b + a & s \\ s & b - a \end{bmatrix} \quad (825)$$

where b is the biaxial component, a is the anti-symmetric component, and s is the shear component. The basic model parameters ($1/m_t$ and Δ) are expanded in powers of these components. The longitudinal mass m_l is assumed to be stress independent, that is, $m_{l1} = m_{l2} = m_{l0}$.

The symmetry of the top valence band enforces some consistency relations on the basic parameters and its power-law expansions. For example, the mobility enhancement along $\langle 110 \rangle$, when uniaxial stress along $\langle 110 \rangle$ is applied, must be equal to the enhancement along $\langle \bar{1}10 \rangle$ when uniaxial stress along $\langle \bar{1}10 \rangle$ is applied. In addition, when biaxial stress is applied, the mobility enhancements along $\langle 110 \rangle$ and $\langle \bar{1}10 \rangle$ must be the same.

Enforcing these symmetries, the following expressions can be written:

$$\begin{aligned} \Delta &= d_1 s \\ \frac{1}{m_{t1}} &= \frac{1}{m_{t0}} (1 + s_{t1}s + s_{t2}s^2 + b_{t1}b + b_{t2}b^2) \\ \frac{1}{m_{t2}} &= \frac{1}{m_{t0}} (1 - s_{t1}s + s_{t2}s^2 + b_{t1}b + b_{t2}b^2) \end{aligned} \quad (826)$$

where the fitting parameters $d_1, s_{t1}, s_{t2}, b_{t1}, b_{t2}$ can be specified in the `StressMobility` section of the parameter file. The default values of these parameters are based on fitting the model to various PMOSFET stress data described in the literature [21].

Generalization of Model

The original Intel model considered only 2D planes. Therefore, it was extended and generalized in several issues: the three-dimensional case, doping dependence, and carrier redistribution between more than two valleys.

To generalize the model for the three-dimensional case, three $\langle 100 \rangle$ planes where the model is applied separately are considered. It is assumed that the valence band is a sum of six ellipsoids and this is consistent with the model suggested in the literature [22]. Sentaurus Device transforms the stress tensor into the crystal system and then, considering these three $\{100\}$ planes, it selects only the corresponding components of the stress tensor. For example, for the $[100]$, $[010]$ plane, Sentaurus Device takes only the s_1, s_2 , and s_6 stress components and recomputes them into a, b , and s of Eq. 825. Additionally, a modification of the effective mass perpendicular to the plane (parallel to the $[001]$ direction) was introduced to account better for $\langle 001 \rangle$ simulation cases: $1/m_{t\langle 001 \rangle} = (1 + b_{tt}b)/m_{t0}$, where the parameter b_{tt} can be modified in the parameter file.

With this modification, the diagonal tensor of the stress-induced mobility change in the $\langle 110 \rangle$, $\langle \bar{1}10 \rangle$, $\langle 001 \rangle$ coordinate system can be written as:

$$\begin{bmatrix} \Delta\mu_{110} \\ \Delta\mu_{\bar{1}10} \\ \Delta\mu_{001} \end{bmatrix} = \mu_0 \begin{bmatrix} \left(\frac{f_1}{m_{t1}} + \frac{f_2}{m_{t2}}\right) \left(\frac{0.5}{m_{t0}} + \frac{0.5}{m_{t0}}\right) - 1 & 0 & 0 \\ 0 & \left(\frac{f_1}{m_{t1}} + \frac{f_2}{m_{t2}}\right) \left(\frac{0.5}{m_{t0}} + \frac{0.5}{m_{t0}}\right) - 1 & 0 \\ 0 & 0 & m_{t0}/m_{t\langle 001 \rangle} - 1 \end{bmatrix} \quad (827)$$

Next, the relative change of the mobility is computed independently for each plane and it is summed in the crystal system. Finally, the mobility change tensor is transformed from the crystal system to the simulation one.

The carrier occupation of the two valleys (see Eq. 824) is derived assuming Boltzmann statistics and the same density-of-states in both these valleys. To obtain a doping dependence, it is necessary to consider Fermi–Dirac statistics and, in this case, the following expression is obtained:

$$f_1 = \frac{1}{1 + F_{1/2}\left(\frac{E_V - F_p - \Delta}{kT}\right) / F_{1/2}\left(\frac{E_V - F_p}{kT}\right)} \quad (828)$$

$$f_2 = 1 - f_1$$

where F_p is the hole quasi-Fermi level that is computed either assuming charge neutrality between carrier and doping, which gives only the doping dependency of the model or using carrier local concentration.

As previously discussed, the generalized model considers six ellipsoids and, therefore, the carrier reoccupation between all these valleys must be accounted for. This is not a simple problem because the stress-induced energy shift of each valley must be accounted for.

As an experimental option, Sentaurus Device gives the following simplified expressions:

$$f_1 = \frac{F_{1/2}\left(\frac{E_V - F_p}{kT}\right)}{(N_e - 1)F_{1/2}\left(\frac{E_V - F_p}{kT}\right) + F_{1/2}\left(\frac{E_V - F_p - \Delta}{kT}\right)} \quad (829)$$

$$f_2 = \frac{F_{1/2}\left(\frac{E_V - F_p - \Delta}{kT}\right)}{(N_e - 1)F_{1/2}\left(\frac{E_V - F_p}{kT}\right) + F_{1/2}\left(\frac{E_V - F_p - \Delta}{kT}\right)}$$

where N_e is equal to the number of ellipsoids that you want to consider. The value of N_e can be specified in the parameter file. The default value is 2, which transforms [Eq. 829](#) into [Eq. 828](#).

NOTE For this multi-ellipsoid option and $N_e > 2$, the sum $f_1 + f_2 < 1$ even without the stress. Therefore, [Eq. 827](#) is slightly modified also to account for this different initial occupation.

Using Intel Mobility Model

To select the Intel stress-induced hole mobility model, you must include the following keyword in the `Mobility` statement of the `Piezo` model:

```
Physics {
  Piezo( Model(Mobility(hSixBand)) )
}
```

The keyword `hSixBand` assumes Boltzmann statistics and, in this case, [Eq. 824](#) will be activated. To have doping dependence (see [Eq. 828](#)), the keyword `hSixBand(Doping)` should be specified, but to have carrier concentration dependency (Fermi statistics), use the keywords `hSixBand(Fermi)`. The model parameters (see [Eq. 826](#)) can be specified in the `StressMobility` section of the parameter file as follows:

```
StressMobility {
  mh_l0 = 0.48      # [1]
  mh_t0 = 0.15      # [1]
  ne = 2            # [1]
```

```

d1 = -6.0000e-11 # [eV/Pa]
st1 = -9.4426e-10 # [1/Pa]
st2 = 4.3066e-19 # [1/Pa^2]
bt1 = -1.0086e-10 # [1/Pa]
bt2 = 6.5886e-21 # [1/Pa^2]
btt = 1.2000e-10 # [1/Pa]
}

```

PMOS transistors are usually oriented in the direction to have maximum stress effect. To define this direction for the x-axis of a Sentaurus Device simulation, the following parameter set can be specified for the 2D case:

```

LatticeParameters {
  X = (1, 0, 1) #[1]
  Y = (0, 1, 0) #[1]
}

```

The hSixBand carrier occupancies f_1 and f_2 are calculated in the crystallographic coordinate system for each band. To plot these values, use the following keywords in the Plot section of the command file:

- f1BandOccupancy001 = Occupancy of the $\langle \bar{1}10 \rangle$ ellipsoid in the (001) plane
- f2BandOccupancy001 = Occupancy of the $\langle 110 \rangle$ ellipsoid in the (001) plane
- f1BandOccupancy010 = Occupancy of the $\langle \bar{1}10 \rangle$ ellipsoid in the (010) plane
- f2BandOccupancy010 = Occupancy of the $\langle 110 \rangle$ ellipsoid in the (010) plane
- f1BandOccupancy100 = Occupancy of the $\langle \bar{1}10 \rangle$ ellipsoid in the (100) plane
- f2BandOccupancy100 = Occupancy of the $\langle 110 \rangle$ ellipsoid in the (100) plane

Piezoresistance Mobility Model

This approach [5][7][23] focuses on the modeling of the piezoresistive effect. The model is based on an expansion of the mobility enhancement tensor in terms of stress. Sentaurus Device provides options for computing either a first-order or a second-order piezoresistance mobility model. The first-order model accounts for a linear dependency on stress; whereas, the second-order model accounts for both linear and quadratic dependencies on stress.

The electron or hole mobility enhancement tensor, expanded up to second order in stress, is given by:

$$\frac{\mu_{ij}}{\mu_0} = \delta_{ij} + \sum_{k=1}^3 \sum_{l=1}^3 \Pi_{ijkl} \sigma_{kl} + \sum_{k=1}^3 \sum_{l=1}^3 \sum_{m=1}^3 \sum_{n=1}^3 \Pi_{ijklmn} \sigma_{kl} \sigma_{mn} \quad (830)$$

where:

- μ_{ij} is a component of the electron or hole stress-dependent mobility tensor.
- μ_0 denotes the isotropic mobility without stress.
- δ_{ij} is the Kronecker delta function.
- σ_{kl} is a component of the stress tensor.
- Π_{ijkl} is a component of the first-order electron or hole piezoconductance tensor.
- Π_{ijklmn} is a component of the second-order electron or hole piezoconductance tensor.

The piezoconductance tensors are symmetric, which allows [Eq. 830](#) to be written in a contracted form using index contraction ([Eq. 773](#)) and conventional contraction rules (see [\[7\]](#), for example):

$$\frac{\mu_i}{\mu_0} = (\delta_{i1} + \delta_{i2} + \delta_{i3}) + \sum_{j=1}^6 \Pi_{ij} \sigma_j + \sum_{j=1}^6 \sum_{k=1}^6 \Pi_{ijk} \sigma_j \sigma_k \quad (831)$$

The components of the piezoconductance tensors are related to the components of the more familiar piezoresistance tensors through the following relations [\[7\]](#):

$$\begin{aligned} \pi_{ij} &= -\Pi_{ij} & \leftrightarrow & \Pi_{ij} = -\pi_{ij} \\ \pi_{ijk} &= -\Pi_{ijk} + \Pi_{ij}\Pi_{ik} & \leftrightarrow & \Pi_{ijk} = -\pi_{ijk} + \pi_{ij}\pi_{ik} \end{aligned} \quad (832)$$

where π_{ij} and π_{ijk} are components of the first-order and second-order piezoresistance tensors, respectively. When the first-order model is used (the default), only the first summation in [Eq. 831](#) is included in the calculation. When the second-order model is used, the full expression given by [Eq. 831](#) is used.

In crystals with cubic symmetry such as silicon, the number of independent coefficients of the first-order piezoresistance tensor reduces to three by rotating the coordinate system parallel to the high-symmetric axes of the crystal [\[8\]](#) resulting in the following 6×6 tensor:

$$[\pi_{ij}] = \begin{bmatrix} \pi_{11} & \pi_{12} & \pi_{12} & 0 & 0 & 0 \\ \pi_{12} & \pi_{11} & \pi_{12} & 0 & 0 & 0 \\ \pi_{12} & \pi_{12} & \pi_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & \pi_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & \pi_{44} & 0 \\ 0 & 0 & 0 & 0 & 0 & \pi_{44} \end{bmatrix} \quad (833)$$

The second-order piezoresistance tensor has only nine independent components and can be expressed with the following $6 \times 6 \times 6$ tensor:

$$\begin{aligned}
 [\pi_{1jk}] &= \begin{bmatrix} \pi_{111} & \pi_{112} & \pi_{112} & 0 & 0 & 0 \\ \pi_{112} & \pi_{122} & \pi_{123} & 0 & 0 & 0 \\ \pi_{112} & \pi_{123} & \pi_{122} & 0 & 0 & 0 \\ 0 & 0 & 0 & \pi_{144} & 0 & 0 \\ 0 & 0 & 0 & 0 & \pi_{166} & 0 \\ 0 & 0 & 0 & 0 & 0 & \pi_{166} \end{bmatrix} & [\pi_{2jk}] &= \begin{bmatrix} \pi_{122} & \pi_{112} & \pi_{123} & 0 & 0 & 0 \\ \pi_{112} & \pi_{111} & \pi_{112} & 0 & 0 & 0 \\ \pi_{123} & \pi_{112} & \pi_{122} & 0 & 0 & 0 \\ 0 & 0 & 0 & \pi_{166} & 0 & 0 \\ 0 & 0 & 0 & 0 & \pi_{144} & 0 \\ 0 & 0 & 0 & 0 & 0 & \pi_{166} \end{bmatrix} & [\pi_{3jk}] &= \begin{bmatrix} \pi_{122} & \pi_{123} & \pi_{112} & 0 & 0 & 0 \\ \pi_{123} & \pi_{122} & \pi_{112} & 0 & 0 & 0 \\ \pi_{112} & \pi_{112} & \pi_{111} & 0 & 0 & 0 \\ 0 & 0 & 0 & \pi_{166} & 0 & 0 \\ 0 & 0 & 0 & 0 & \pi_{166} & 0 \\ 0 & 0 & 0 & 0 & 0 & \pi_{144} \end{bmatrix} \\
 [\pi_{4jk}] &= \begin{bmatrix} 0 & 0 & 0 & \pi_{441} & 0 & 0 \\ 0 & 0 & 0 & \pi_{661} & 0 & 0 \\ 0 & 0 & 0 & \pi_{661} & 0 & 0 \\ \pi_{441} & \pi_{661} & \pi_{661} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \pi_{456} \\ 0 & 0 & 0 & 0 & \pi_{456} & 0 \end{bmatrix} & [\pi_{5jk}] &= \begin{bmatrix} 0 & 0 & 0 & 0 & \pi_{661} & 0 \\ 0 & 0 & 0 & 0 & \pi_{441} & 0 \\ 0 & 0 & 0 & 0 & \pi_{661} & 0 \\ 0 & 0 & 0 & 0 & 0 & \pi_{456} \\ \pi_{661} & \pi_{441} & \pi_{661} & 0 & 0 & 0 \\ 0 & 0 & 0 & \pi_{456} & 0 & 0 \end{bmatrix} & [\pi_{6jk}] &= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & \pi_{661} \\ 0 & 0 & 0 & 0 & 0 & \pi_{661} \\ 0 & 0 & 0 & 0 & 0 & \pi_{441} \\ 0 & 0 & 0 & 0 & \pi_{456} & 0 \\ 0 & 0 & 0 & \pi_{456} & 0 & 0 \\ \pi_{661} & \pi_{661} & \pi_{441} & 0 & 0 & 0 \end{bmatrix}
 \end{aligned} \tag{834}$$

Since the coordinate system of the simulation is not necessarily parallel to the high-symmetric axis of the crystal, the orientations of the x-axis and y-axis of the crystal system can be specified in the command file (see [Using Stress and Strain on page 689](#)).

Doping and Temperature Dependency

A simple model for the doping and temperature variation of the first-order and second-order piezoconductance coefficients is given by [7][24]:

$$\begin{aligned}
 \Pi_{ij}(N, T) &= P_1(N, T) \Pi_{ij}(0, 300\text{K}) \\
 \Pi_{ijk}(N, T) &= P_2(N, T) \Pi_{ijk}(0, 300\text{K})
 \end{aligned} \tag{835}$$

where $\Pi_{ij}(0, 300\text{K})$ and $\Pi_{ijk}(0, 300\text{K})$ are piezoconductance coefficients for low-doped silicon at 300 K, and $P_1(N, T)$ and $P_2(N, T)$ are doping factors given by:

$$\begin{aligned}
 P_1(N, T) &= \left(\frac{300\text{K}}{T} \right) \frac{F_{-1}(\eta)}{F_0(\eta)} \\
 P_2(N, T) &= \left(\frac{300\text{K}}{T} \right)^2 \frac{F_{-2}(\eta)}{F_0(\eta)}
 \end{aligned} \tag{836}$$

In [Eq. 836](#), $F_r(\eta)$ is a Fermi–Dirac integral of order “ r ” and η is the Fermi energy measured from the band edge in units of kT ($\eta_n = (E_{F,n} - E_C)/(kT)$ and $\eta_p = (E_V - E_{F,p})/(kT)$).

Doping dependency is introduced by assuming charge neutrality in the calculation of the Fermi energy.

NOTE For minority carriers, the doping factors $P_1(N, T)$ and $P_2(N, T)$ are equal to 1. In addition, Sentaurus Device allows the calculation to be suppressed for majority carriers when the doping is below a user-specified doping threshold (see [Stress Mobility Model for Minority Carriers on page 729](#)).

The temperature and doping variation of the piezoresistance coefficients is obtained by combining [Eq. 832](#) and [Eq. 835](#). An exception to this is when the first-order model is used. In this case, the piezoresistance coefficients are split between a constant part (associated with changes in the effective masses) and a part that varies with temperature and doping (associated with anisotropic scattering):

$$\pi_{ij}(N, T) = \pi_{ij, \text{var}} P_1(N, T) + \pi_{ij, \text{con}} \quad (837)$$

The default values of the piezoresistance coefficients for low-doped silicon at 300 K are listed in [Table 117 on page 721](#) and [Table 118 on page 721](#). They can be changed in the parameter file as described in [Using Piezoresistance Mobility Model on page 720](#).

Isotropic Mobility Factors

Sentaurus Device provides an option that allows one of the diagonal components of the mobility enhancement tensor in the simulation coordinate system (that is, [Eq. 831](#) after transformation from the crystallographic coordinate system to the simulation coordinate system) to be used as an isotropic factor that is applied to mobility. This option provides a robust approximate solution when current in the device is predominantly in a direction parallel to one of the coordinate axes.

When using this option, the channel (or current) direction must be specified (the x-direction is the default), which determines the isotropic factor that is applied to mobility.

Using Piezoresistance Mobility Model

The command file syntax for the piezoresistance mobility models, including options, is:

```
Physics {
  Piezo (
    Model (
      Mobility (
        [ Tensor (
          [FirstOrder | SecondOrder] [Kanda]
          [ParameterSetName="<psname>" | AutoOrientation]
        )
      )
    )
  )
}
```

```

    |
    | Factor (
    |   [FirstOrder | SecondOrder] [Kanda] [ChannelDirection=<n>]
    |   [ParameterSetName="<psname>" | AutoOrientation]
    |   [SFactor=<dataset-or-pmi_model>]
    | )
  ]
)
)
}

```

The keyword **Tensor** selects the anisotropic tensor model for electron and hole mobility, and the keyword **Factor** selects the isotropic factor model for electron and hole mobility. Alternatively, the keywords **eTensor**, **hTensor**, **eFactor**, or **hFactor** can be used to select these models for only one carrier.

The keyword **FirstOrder** or **SecondOrder** selects the first-order or second-order model, respectively. The default is **FirstOrder**. When the **FirstOrder** model is used, Sentaurus Device uses the piezoresistance coefficients shown in [Table 117](#). These can be changed from their default values in the **Piezoresistance** section of the parameter file.

Table 117 Piezoresistance coefficients for FirstOrder model: Defaults for silicon

Symbol	Parameter name	Electrons	Holes	Unit
$\pi_{11, \text{var}}$	p11var	-1.026×10^{-9}	1.5×10^{-11}	1/Pa
$\pi_{12, \text{var}}$	p12var	5.34×10^{-10}	1.5×10^{-11}	1/Pa
$\pi_{44, \text{var}}$	p44var	-1.36×10^{-10}	1.1×10^{-9}	1/Pa
$\pi_{11, \text{con}}$	p11con	0.0	5.1×10^{-11}	1/Pa
$\pi_{12, \text{con}}$	p12con	0.0	-2.6×10^{-11}	1/Pa
$\pi_{44, \text{con}}$	p44con	0.0	2.8×10^{-10}	1/Pa

When the **SecondOrder** model is used, Sentaurus Device uses the piezoresistance coefficients shown in [Table 118](#). These also can be changed from their default values in the **Piezoresistance** section of the parameter file. Note that the **SecondOrder** model uses a different set of first-order piezoresistance coefficients than the **FirstOrder** model. This allows the **FirstOrder** and **SecondOrder** models to be calibrated separately.

Table 118 Piezoresistance coefficients for SecondOrder model: Defaults for silicon

Symbol	Parameter name	Electrons	Holes	Unit
π_{11}	p11	-1.1×10^{-9}	0.0	1/Pa
π_{12}	p12	4.5×10^{-10}	2.0×10^{-11}	1/Pa

Table 118 Piezoresistance coefficients for SecondOrder model: Defaults for silicon

Symbol	Parameter name	Electrons	Holes	Unit
π_{44}	p44	2.5×10^{-10}	1.19×10^{-9}	1/Pa
π_{111}	p111	6.6×10^{-19}	-4.5×10^{-19}	1/Pa ²
π_{112}	p112	-5.5×10^{-20}	2.8×10^{-19}	1/Pa ²
π_{122}	p122	-2.2×10^{-20}	-2.5×10^{-19}	1/Pa ²
π_{123}	p123	8.8×10^{-19}	2.0×10^{-20}	1/Pa ²
π_{144}	p144	1.0×10^{-20}	-3.3×10^{-19}	1/Pa ²
π_{166}	p166	-6.9×10^{-19}	6.6×10^{-19}	1/Pa ²
π_{661}	p661	6.0×10^{-21}	-3.1×10^{-19}	1/Pa ²
π_{456}	p456	2.0×10^{-20}	-3.0×10^{-19}	1/Pa ²
π_{441}	p441	2.0×10^{-20}	0.0	1/Pa ²

The keyword Kanda activates the calculation of the doping factors $P_1(N, T)$ and $P_2(N, T)$ (see [Eq. 836](#)). If Kanda is not specified, these factors are equal to 1.

Named Parameter Sets for Piezoresistance

The Piezoresistance parameter set can be named. For example, in the parameter file, you can write the following to declare a parameter set with the name myset:

```
Piezoresistance "myset" { ... }
```

To use a named parameter set, specify its name with ParameterSetName as an option to either Tensor or Factor as shown in the command file syntax (see [Using Piezoresistance Mobility Model on page 720](#)).

By default, the unnamed parameter set is used.

Auto-Orientation for Piezoresistance

The piezoresistance models support the auto-orientation framework (see [Auto-Orientation Framework on page 75](#)) that switches between different named parameter sets based on the orientation of the nearest interface. This can be activated by specifying AutoOrientation as an argument to either Tensor or Factor in the command file.

Additional Factor Model Options

When the Factor model is used, the channel (or current) direction can be specified using `ChannelDirection = 1, 2, or 3` corresponding to the x-, y-, or z-direction. The default is `ChannelDirection = 1`.

As an alternative to using the built-in calculations for the Factor model, the `SFactor` parameter can be used to specify a dataset name (which includes the PMI user fields `PMIUserField0` through `PMIUserField99`) from which the vertex Factor values will be taken, or a space factor PMI model can be specified that calculates the vertex Factor values directly (see [Space Factor on page 1105](#)).

Enormal- and MoleFraction-dependent Piezo Coefficients

Measured data shows that the piezoresistive coefficients can have dependencies with respect to the normal electric field $E_{\perp}$ or the mole fraction x or both. Sentaurus Device has a calibration option to specify these dependencies of the piezoresistive coefficients.

This model is based on the piezoresistive prefactors $P_{ij} = P_{ij}(E_{\perp}, x)$. The new coefficients are calculated from:

$$\pi_{ij, \text{new}} = P_{ij}(E_{\perp}, x) \cdot \pi_{ij} \quad (838)$$

Sentaurus Device allows a piecewise linear or spline approximation (the third degree) of the prefactors over the normal electric field and a piecewise linear or piecewise cubic approximation over the mole fraction (see [Ternary Semiconductor Composition on page 63](#)).

Using Piezoresistive Prefactors Model

To activate this model, add the keyword `Enormal` to the subsection `{e,h}Tensor`, for example:

```
Physics {
  ...
  Piezo(
    Model(Mobility(eTensor(Kanda Enormal)))
  )
}
```

The implementation of this model is based on the PMI (see [Piezoresistive Coefficients on page 1130](#)). The keyword `Enormal` means that Sentaurus Device uses the PMI predefined model `PmiEnormalPiezoResist`. You can create your own PMI models and, in this case, the keyword `Enormal` must be replaced by the name of the PMI model (for example, `eTensor("my_pmi_model")`) and the parameter file must contain a corresponding section.

30: Modeling Mechanical Stress Effect

Stress-dependent Mobility Models

NOTE Piezoresistive prefactors cannot be used with the isotropic Factor model. In addition, these prefactors are only available when using the FirstOrder piezoresistance mobility model.

By default, the values of all prefactors are equal to 1. These values can be changed in the section PmiEnormalPiezoResist of the parameter file. The following examples show how to use this section.

Example 1

This example shows the section of the parameter file for purely Enormal-dependent prefactors:

```
Material = "Silicon" {
* Example 1: Only Enormal dependence of piezoresistive coefficients
PmiEnormPiezoResist
{
    eEnormalFormula = 2 # cubic spline
*                   = 0 # no Enormal dependence (default)
*                   = 1 # piecewise linear approximation
*                   = 2 # cubic spline approximation

    eNumberOfEnormalNodes = 3 # number of nodes for Spline(Enormal)

    # _k is _"index of node"
    eEnormal_1 = 1.0e+5 # [V/cm]
    eP11_1 = 1.0 # [1]
    eP12_1 = 1.0 # [1]
    eP44_1 = 1.0 # [1]

    eEnormal_2 = 4.0e+5 # [V/cm]
    eP11_2 = 0.75 # [1]
    eP12_2 = 0.75 # [1]
    eP44_2 = 0.75 # [1]

    eEnormal_3 = 7.0e+5 # [V/cm]
    eP11_3 = 0.5 # [1]
    eP12_3 = 0.5 # [1]
    eP44_3 = 0.5 # [1]

    hEnormalFormula = 1 # piecewise linear approximation
    hNumberOfEnormalNodes = 4 # number of nodes

    # _k is _"index of the node"
    hEnormal_1 = 1.0e+5 # [V/cm]
    hP11_1 = 1.0 # [1]
    hP12_1 = 1.0 # [1]
    hP44_1 = 1.0 # [1]
```

```

hEnormal_2 = 1.9e+5 # [V/cm]
  hP11_2 = 0.969625 # [1]
  hP12_2 = 0.969625 # [1]
  hP44_2 = 0.969625 # [1]

hEnormal_3 = 6.1e+5 # [V/cm]
  hP11_3 = 0.530375 # [1]
  hP12_3 = 0.530375 # [1]
  hP44_3 = 0.530375 # [1]

hEnormal_4 = 7.0e+5 # [V/cm]
  hP11_4 = 0.5 # [1]
  hP12_4 = 0.5 # [1]
  hP44_4 = 0.5 # [1]
}
}

```

Figure 52 shows the dependency of these piezo prefactors with respect to E_{normal} .

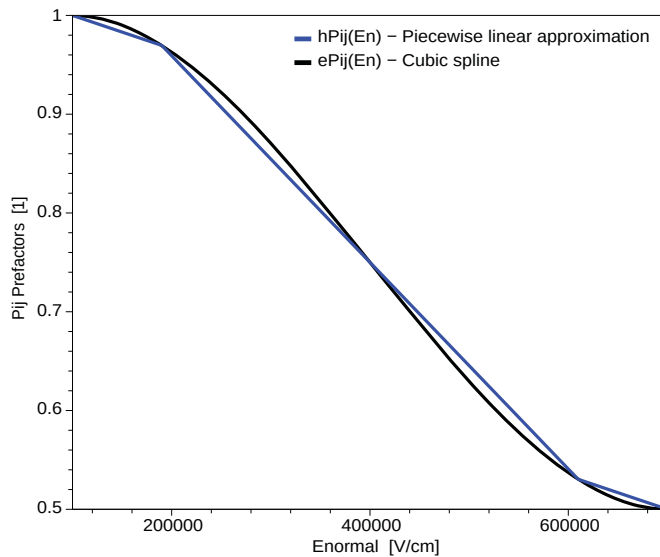


Figure 52 Example of cubic spline and piecewise linear approximation of piezo prefactors

Example 2

This example shows the section of the parameter file for purely MoleFraction-dependent prefactors:

```

Material = "SiliconGermanium" {
  * Mole dependent material: SiliconGermanium (x=0) = Silicon
  * Mole dependent material: SiliconGermanium (x=1) = Germanium
  * Example 2: Only MoleFraction dependence.
}

```

30: Modeling Mechanical Stress Effect

Stress-dependent Mobility Models

```

PmiEnormPiezoResist
{
    eMoleFormula = 1          # piecewise linear approximation
    eMoleFractionIntervals = 1 # number of MoleFraction intervals
                                # i.e. x0=0, x1=1

    # _i is "index of mole fraction value"
    eXmax_0 = 0
    eP11_0 = 1 # [1]
    eP12_0 = 1 # [1]
    eP44_0 = 1 # [1]

    eXmax_1 = 1
    eP11_1 = 0.5 # [1]
    eP12_1 = 0.5 # [1]
    eP44_1 = 0.5 # [1]

    hMoleFormula = 2          # piecewise cubic approximation
    hMoleFractionIntervals = 2
        # see Ternary Semiconductor Composition on page 63
        #  $P = P[i-1] + A[i]*dx + B[i]*dx^2 + C[i]*dx^3$ 
        # where:
        #  $A[i] = (P[i]-P[i-1])/dx[i] - B[i]*dx[i] - C[i]*dx[i]$ 
        #  $dx[i] = xMax[i] - xMax[i-1]$ 
        #  $dx = x - xMax[i-1]$ 
        #  $i = 1, \dots, nbMoleIntervals$ 

    # _i is "index of mole fraction value"
    hXmax_0 = 0
    hP11_0 = 1 # [1]
    hP12_0 = 1 # [1]
    hP44_0 = 1 # [1]

    hXmax_1 = 0.5
    hP11_1 = 2 # [1]
    hP12_1 = 2 # [1]
    hP44_1 = 2 # [1]

    # B and C coefficients
    hB11_1 = 4.
    hC11_1 = 0.
    hB12_1 = 4.
    hC12_1 = 0.
    hB44_1 = 4.
    hC44_1 = 0.

    hXmax_2 = 1
    hP11_2 = 3 # [1]

```

```

hP12_2 = 3 # [1]
hP44_2 = 3 # [1]

# B and C coefficients
hB11_2 = -4.
hC11_2 = 0.
hB12_2 = -4.
hC12_2 = 0.
hB44_2 = -4.
hC44_2 = 0.
}
}

```

Figure 53 shows the dependency of the hP_{ij} prefactors of the previous example with respect to MoleFraction.

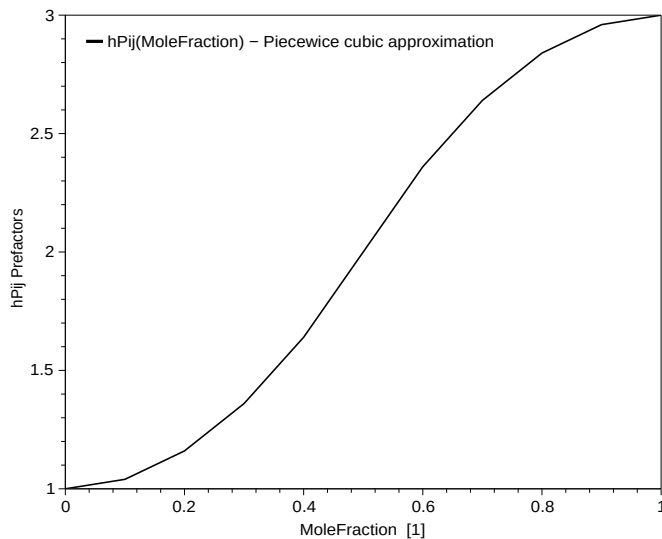


Figure 53 Example of piecewise cubic approximation

Example 3

This example shows the section of the parameter file for Enormal- and MoleFraction-dependent prefactors:

```

Material = "SiliconGermanium" {
* Mole dependent material: SiliconGermanium (x=0) = Silicon
* Mole dependent material: SiliconGermanium (x=1) = Germanium

* Example 3: Enormal and MoleFraction dependent piezoresistance prefactors
* (general case)
PmiEnormPiezoResist
{

```

30: Modeling Mechanical Stress Effect

Stress-dependent Mobility Models

```
eMoleFormula = 2          # piecewise cubic approximation
eMoleFractionIntervals = 1 # i.e. x0=0, x1=1

# begin description for mole fraction x0=0
eXmax_0 = 0
eEnormalFormula_0 = 2      # cubic spline
eNumberOfEnormalNodes_0 = 3 # number of nodes for Spline(Enormal)

    # _i_k is _"index of mole fraction value"_"index of node for Spline"
eEnormal_0_1 = 1.0e+5 # [V/cm]
    eP11_0_1 = 1.0 # [1]
    eP12_0_1 = 1.0 # [1]
    eP44_0_1 = 1.0 # [1]

eEnormal_0_2 = 4.0e+5 # [V/cm]
    eP11_0_2 = 0.75 # [1]
    eP12_0_2 = 0.75 # [1]
    eP44_0_2 = 0.75 # [1]

eEnormal_0_3 = 7.0e+5 # [V/cm]
    eP11_0_3 = 0.5 # [1]
    eP12_0_3 = 0.5 # [1]
    eP44_0_3 = 0.5 # [1]
# end description for mole fraction x0=0

# begin description for mole fraction x1=1
eXmax_1 = 1
eEnormalFormula_1 = 1      # piecewise linear approximation
eNumberOfEnormalNodes_1 = 2 # number of nodes

    # _i_k is _"index of mole fraction value"_"index of node for Spline"
eEnormal_1_1 = 1.0e+5 # [V/cm]
    eP11_1_1 = 1.0 # [1]
    eP12_1_1 = 1.0 # [1]
    eP44_1_1 = 1.0 # [1]

eEnormal_1_2 = 7.0e+5 # [V/cm]
    eP11_1_2 = 0.5 # [1]
    eP12_1_2 = 0.5 # [1]
    eP44_1_2 = 0.5 # [1]

# B and C coefficients
hB11_1 = 3.
hC11_1 = -2.
hB12_1 = 3.
hC12_1 = -2.
hB44_1 = 3.
```

```
hC44_1 = -2.

# end description for mole fraction x1=1

# hPij prefactors are equal to default values (i.e. 1)
}
}
```

Figure 54 shows the difference between I_d - V_g curves for a MOSFET. The parameter file section `PmiEnormPiezoResist` is the same as in [Example 1 on page 724](#). The Physics section is:

```
Physics {
  Fermi
  Mobility( Phumob HighFieldsat Enormal )
  EffectiveIntrinsicDensity( OldSlotboom )
  Recombination( SRH(DopingDependence) )
  Piezo(
    Model( Mobility(eTensor(Kanda Enormal)) DeformationPotential )
    Stress = (1.3e8, 0, 0, 0, 0, 0)
  )
}
```

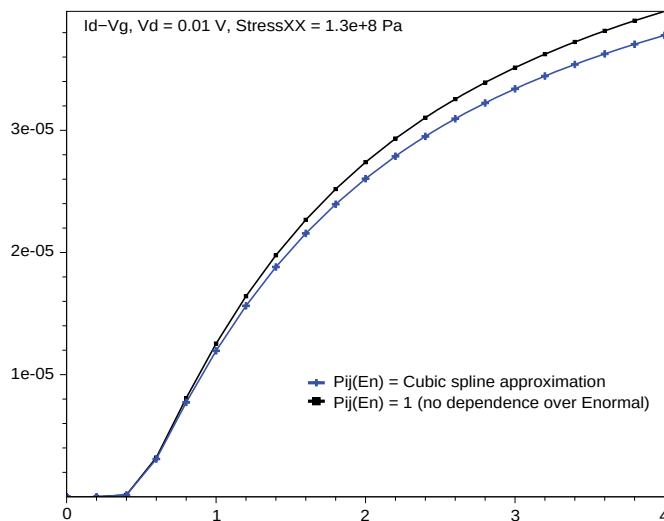


Figure 54 I_d - V_g curve, with E_{normal} -dependent piezo prefactors, shifts downwards

Stress Mobility Model for Minority Carriers

Measured data shows that the stress dependency of minority carrier mobility (like the mobility of carriers in the channel of a MOSFET) is different from the stress dependency of majority carrier mobility. For example, the stress effect for minority carriers may have a dependency on electric field and, perhaps, a different (or smaller) doping dependency.

As a calibration option, Sentaurus Device provides an additional factor β for the stress effect applied to minority carrier mobility:

$$\bar{\mu} = \mu_0 \left(\bar{1} + \beta \frac{\Delta \bar{\mu}_{\text{stress}}}{\mu_0} \right) \quad (839)$$

where $\Delta \bar{\mu}_{\text{stress}}$ is the stress-induced change of the mobility tensor for any stress model described in this chapter. The minority carrier factor β can be specified (the default value is equal to 1) in the `Piezo` section of the command file using the keywords `eMinorityFactor` and `hMinorityFactor` for electrons and holes, respectively, for example:

```
Physics {
  Piezo( Model(Mobility(eMinorityFactor = 0.5 hMinorityFactor = 0.5) ) )
}
```

In some cases, it is useful to switch off the doping dependency of the stress models and also to have the minority carrier factor β applied to portions of the device where the carrier is actually a majority carrier (for example, in low-doped parts of MOSFET source/drain regions). This can be achieved by specifying the parameter `DopingThreshold= $N_{\text{th}}$` as an argument to `eMinorityFactor` or `hMinorityFactor`, for example:

```
Physics {
  Piezo( Model(Mobility(eMinorityFactor(DopingThreshold=1e18) = 0.5
                                hMinorityFactor(DopingThreshold=-1e18) = 0.5) ) )
}
```

When `DopingThreshold= $N_{\text{th}}$` is specified, the behavior of the program depends on the relation of N_{th} to the local net doping, $N_D - N_A$:

$$\begin{aligned} \mu_n: N_D - N_A < N_{\text{th}} &\rightarrow \text{eMinorityFactor} = \beta \text{ is applied, stress model doping dependency is switched off} \\ \mu_p: N_D - N_A > N_{\text{th}} &\rightarrow \text{hMinorityFactor} = \beta \text{ is applied, stress model doping dependency is switched off} \end{aligned} \quad (840)$$

NOTE `DopingThreshold=0` corresponds to the regular definition of minority and majority carriers. To apply the factor β to a portion of the majority carrier mobility, specify $N_{\text{th}} > 0$ for electrons and $N_{\text{th}} < 0$ for holes.

NOTE When the `DopingThreshold` condition is met (and the doping dependency of the stress models is switched off), the `Tensor(Kanda)` model will behave like `Tensor()`, the `eSubBand(Doping)` model will behave like `eSubBand()`, and the `hSixBand(Doping)` model will behave like `hSixBand()`.

NOTE Specifying `eMinorityFactor=0.5` is not the same as specifying `eMinorityFactor(ThresholdDoping=0)=0.5` because, in the latter case, the stress model doping dependency is switched off for carriers where this factor is applied.

Dependency of Saturation Velocity on Stress

Eq. 839 can be rewritten in the following form (see [Current Densities on page 670](#)):

$$\bar{\mu} = \mu_0 Q \begin{bmatrix} 1+t_1 & 0 & 0 \\ 0 & 1+t_2 & 0 \\ 0 & 0 & 1+t_3 \end{bmatrix} Q^T \quad (841)$$

where:

- t_i are eigenvalues of the tensor $\beta \frac{\Delta \bar{\mu}_{\text{stress}}}{\mu_0}$.
- Q is the orthogonal matrix from eigenvectors (main directions) of this tensor.

In high electric fields, the mobility along the main directions is proportional to the saturation velocity: $\mu_i \sim \frac{v_{\text{sat}}}{E} (1+t_i)$. Therefore, the dependence of the saturation velocity on stress is similar to the mobility (with the factor $1+t_i$). However, measured data shows that saturation velocity can have a different stress effect or no stress effect. Sentaurus Device has a calibration option to modify the dependency of saturation velocity on stress in the following form for the main directions:

$$v_{\text{sat}, i} = v_{\text{sat}, 0} \cdot \frac{(1 + \alpha \cdot t_i)}{(1 + t_i)} \quad (842)$$

where $v_{\text{sat}, 0}$ is the stress-independent saturation velocity and α is a user-defined scalar factor. This factor can be specified using the keyword `SaturationFactor` in the `Piezo` section of the command file. For example:

```
Physics {
  Piezo( Model( Mobility(SaturationFactor = 0.5) ) )
}
```

This parameter can be specified separately for electrons and holes, for example:

```
Physics {
  Piezo( Model( Mobility(eSaturationFactor = 0.5 hSaturationFactor = 0) ) )
}
```

The default value for α is 1. This is equivalent to applying the stress enhancement factor to total mobility with $v_{\text{sat},i} = v_{\text{sat},0}$. Using $\alpha = 0$ with a Caughey–Thomas-type mobility (see Eq. 304, p. 343) is equivalent to applying the stress enhancement factor only to the low-field mobility, μ_{low} , with $v_{\text{sat},i} = v_{\text{sat},0}$.

Figure 55 shows how the I_d – V_d curves depend on the parameter SaturationFactor.

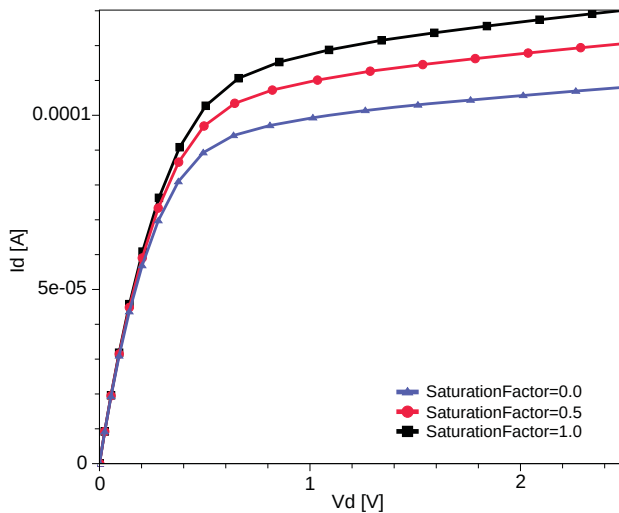


Figure 55 Dependency of I_d – V_d curves on SaturationFactor, with $V_g = 1.5$ V, and Stress = (0.6e9, 0, 0, 0, 0, 0)

Mobility Enhancement Limits

With high stress values, the stress-dependent mobility models in Sentaurus Device can sometimes cause the mobility to become negative or, in some cases, to become unrealistically large. This is particularly true for the piezoresistance mobility models. To prevent this, minimum and maximum stress enhancement factors for both electron and hole mobility can be specified in the Piezoresistance section of the parameter file. The following example shows the default values for MinStressFactor and MaxStressFactor:

```
Piezoresistance {
  MinStressFactor = 1e-5 , 1e-5 # [1]
  MaxStressFactor = 10   , 10   # [1]
}
```

NOTE Although these parameters are specified in the Piezoresistance section of the parameter file, these limits are applied to all stress-dependent mobility models.

Plotting Mobility Enhancement Factors

The mobility multiplication tensor ($\bar{\mu}/\mu_0$) can be plotted on the mesh nodes. This tensor is a symmetric 3×3 matrix and, therefore, has six independent values. To plot these values on the mesh nodes for electron and hole mobilities, use the keywords:

- eMobilityStressFactorXX, eMobilityStressFactorYY, eMobilityStressFactorZZ
- eMobilityStressFactorYZ, eMobilityStressFactorXZ, eMobilityStressFactorXY
- hMobilityStressFactorXX, hMobilityStressFactorYY, hMobilityStressFactorZZ
- hMobilityStressFactorYZ, hMobilityStressFactorXZ, hMobilityStressFactorXY

Numeric Approximations for Tensor Mobility

Tensor Grid Option

Due to an applied mechanical stress, the mobility can become a tensor. The numeric approximation of the transport equations with tensor mobility is complicated (see [Chapter 28 on page 665](#)). However, if the mesh is a tensor one, the approximation is simpler. For this option, off-diagonal mobility elements are not used and, therefore, there are no mixed derivatives in the approximation. Such an approximation gives an M-matrix property for the Jacobian and it permits stable stress simulations. Very often, critical regions are simple and the mesh constructed in such regions can be close to a tensor one.

NOTE The off-diagonal elements of the mobility tensor appear only if there is a shear stress or the simulation coordinate system of Sentaurus Device is different from the crystal system.

To activate this simple tensor-grid approximation, the keyword `TensorGridAniso` in the Math section must be specified. By default, Sentaurus Device uses the `AverageAniso` approximation (see [Chapter 28 on page 665](#)) which is based on a local transformation of an anisotropic task (stress-induced mobility tensor) to an isotropic one.

NOTE Both approximation options do not guarantee a correct solution for arbitrary mesh and stress. Experiments with both these options can give an estimation of ignored terms.

Stress Tensor Applied to Low-Field Mobility

In all the previous models, the stress tensor factor was a linear factor applied to high-field mobility. Sentaurus Device allows also to apply the following diagonal mobility tensor factor to the low-field mobility:

$$\mu_{\text{high}} = \begin{bmatrix} \mu_{\text{high}}(\mu_{\text{low}} \cdot s_{xx}, \dots) & 0 & 0 \\ 0 & \mu_{\text{high}}(\mu_{\text{low}} \cdot s_{yy}, \dots) & 0 \\ 0 & 0 & \mu_{\text{high}}(\mu_{\text{low}} \cdot s_{zz}, \dots) \end{bmatrix} \quad (843)$$

where:

- s_{ii} are the diagonal elements of the stress tensor factor $\left(1 + \beta \frac{\Delta \bar{\mu}_{\text{stress}}}{\mu_0}\right)$ in [Eq. 839, p. 730](#). In this model, off-diagonal elements are not used.
- $\mu_{\text{high}}(\dots)$ is a scalar function that computes the high-field mobility (see [High-Field Saturation on page 342](#)).

This model can be specified in the Math section of the command file as follows:

```
Math { StressMobilityDependence = TensorFactor }
```

In this case, the dependency of saturation velocity on stress is given by:

$$v_{\text{sat}, i} = v_{\text{sat}, 0} \cdot (1 + \alpha \cdot t_i) \quad (844)$$

which results in the same dependency on SaturationFactor described in [Dependency of Saturation Velocity on Stress on page 731](#) when a Caughey–Thomas-type mobility model is used.

The high-field mobility tensor (in [Eq. 843](#)) and the corresponding factors $\mu_{\text{high}}(\mu_{\text{low}} \cdot s_{ii}, \dots) / \mu_{\text{high}}(\mu_{\text{low}}, \dots)$ can be plotted on the mesh nodes. To plot these values for electron and hole mobilities, use the following keywords in the Plot section:

- eTensorMobilityXX, eTensorMobilityYY, eTensorMobilityZZ
- hTensorMobilityXX, hTensorMobilityYY, hTensorMobilityZZ
- eTensorMobilityFactorXX, eTensorMobilityFactorYY, eTensorMobilityFactorZZ
- hTensorMobilityFactorXX, hTensorMobilityFactorYY, hTensorMobilityFactorZZ

Piezoelectric Polarization

Sentaurus Device provides two models (strain and stress) to compute polarization effects in GaN devices. They can be activated in the Physics section of the command file as follows:

```
Physics {
  Piezoelectric_Polarization (strain)
  Piezoelectric_Polarization (stress)
}
```

The piezoelectric polarization vector and the piezoelectric charge can be plotted by:

```
Plot {
  PE_Polarization/vector
  PE_Charge
}
```

Sentaurus Device also offers a corresponding PMI model (see [Piezoelectric Polarization on page 1114](#)).

Strain Model

This model is based on the work by Ambacher *et al.* [25][26]. It captures the first-order effect of polarization vectors in AlGaIn/GaN HFETs: The interface charge induced is due to the discontinuity in the vertical component of the polarization vector at material interfaces.

The polarization vector is computed as follows:

$$\begin{bmatrix} P_x \\ P_y \\ P_z \end{bmatrix} = \begin{bmatrix} P_x^{\text{sp}} \\ P_y^{\text{sp}} \\ P_z^{\text{sp}} + P_{\text{strain}} \end{bmatrix} \quad (845)$$

where:

- $P_{\text{strain}} = 2d_{31} \cdot \text{strain} \cdot (c_{11} + c_{12} - 2c_{13}^2/c_{33})$ for Formula=1.
- $P_{\text{strain}} = 2\text{strain} \cdot (e_{31} - e_{33}c_{13}/c_{33})$ for Formula=2.
- P^{sp} denotes the spontaneous polarization vector [C/cm^2].
- d_{31} is a piezoelectric coefficient [cm/V].

- c_{ij} are stiffness constants [Pa].
- e_{31}, e_{33} are strain–charge piezoelectric coefficients [C/cm^2]. The value of strain is computed as:

$$\text{strain} = (1 - \text{relax}) \cdot (a_0 - a)/a \quad (846)$$

where a_0 represents the strained lattice constant [$\text{\AA}$], a is the unstrained lattice constant [$\text{\AA}$], and ‘relax’ denotes a relaxation parameter [1].

The quantities P^{sp} , d_{ij} , c_{ij} , and e_{ij} are defined in the crystal system. The polarization vector is first computed in crystal coordinates and is converted to simulation coordinates afterwards.

Stress Model

Although, in most practical situations, there are only in-plane stress components due to lattice mismatch, the vertical and shear stress components give rise to in-plane piezopolarization components, which lead to volume charge densities and, therefore, potential variations. The stress model computes the full polarization vector in tensor form without simplifying assumptions:

$$\begin{bmatrix} P_x \\ P_y \\ P_z \end{bmatrix} = \begin{bmatrix} P_x^{\text{sp}} \\ P_y^{\text{sp}} \\ P_z^{\text{sp}} \end{bmatrix} + \begin{bmatrix} d_{11} & d_{12} & d_{13} & d_{14} & d_{15} & d_{16} \\ d_{21} & d_{22} & d_{23} & d_{24} & d_{25} & d_{26} \\ d_{31} & d_{32} & d_{33} & d_{34} & d_{35} & d_{36} \end{bmatrix} \begin{bmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{yz} \\ \sigma_{xz} \\ \sigma_{xy} \end{bmatrix} \quad (847)$$

where P^{sp} denotes the spontaneous polarization vector [C/cm^2], d_{ij} are the piezoelectric coefficients [cm/V], and:

$$\begin{bmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{yz} \\ \sigma_{xz} \\ \sigma_{xy} \end{bmatrix} \quad (848)$$

is the stress tensor [Pa].

The quantities P^{sp} and d_{ij} are defined in the crystal system. The stress tensor σ is defined in the stress system. It is first converted from stress coordinates to crystal coordinates. Then, the

polarization vector is computed in crystal coordinates. Afterwards, the polarization vector is converted to simulation coordinates.

Poisson Equation

Based on the polarization vector, the piezoelectric charge is computed according to:

$$q_{PE} = -\text{activation} \nabla P \quad (849)$$

where `activation` is a nonnegative real calibration parameter (default is 1). This value can be defined in the `Physics` section:

```
Physics (MaterialInterface="AlGaN/GaN"){
  Piezoelectric_Polarization (strain activation=0.5)
}
```

The q_{PE} value is added to the right-hand side of the Poisson equation:

$$\nabla \epsilon \cdot \nabla \phi = -q(p - n + N_D - N_A + q_{PE}) \quad (850)$$

Only the first d components of the polarization vector are used to compute the polarization charge, where d denotes the dimension of the problem.

Parameter File

The following parameters can be specified in the parameter file. All of them are mole fraction dependent, except `a0` and `relax`:

```
Piezoelectric_Polarization {
  # piezoelectric coefficients [cm/V]
  d11 = ...
  d12 = ...
  ...
  d36 = ...

  # spontaneous polarization [C/cm^2]
  psp_x = ...
  psp_y = ...
  psp_z = ...

  Formula = ...

  # stiffness constants [Pa]
  c11 = ...
```

30: Modeling Mechanical Stress Effect

Piezoelectric Polarization

```
c12 = ...
c13 = ...
c33 = ...

# piezoelectric coefficients [C/cm^2]
e31 = ...
e33 = ...

# strain parameters [Å]
a0 = ...
a = ...
relax = ... # [1]
}
```

Coordinate Systems

The x-axis and y-axis of the simulation system are defined in the parameter file:

```
LatticeParameters {
  X = (1, 0, 0)
  Y = (0, 0, -1)
}
```

The z-axis is computed as the outer vector product of the x-axis and y-axis. The simulation system is defined relative to the crystal system. If the keyword `CrystalAxis` is present, the crystal system is defined relative to the simulation system (see [Using Stress and Strain on page 689](#)).

In the above example, the x-axis of the simulation system coincides with the x-axis of the crystal system. The y-axis of the simulation system runs along the negative z-axis of the crystal system. This is a common definition for 2D simulations.

The x-axis and y-axis of the stress system are defined in the Physics section:

```
Physics {
  Piezo (
    OriKddX = (-0.96 0.28 0)
    OriKddY = (0.28 0.96 0)
  )
}
```

The z-axis is computed as the outer vector product of the x-axis and y-axis. The stress system is defined relative to the simulation system (see [Using Stress and Strain on page 689](#)).

Converse Piezoelectric Field

The values of the converse piezoelectric field are very important for applications. The dataset `ConversePiezoelectricField` can be added to the `Plot` variables (see [Table 140 on page 1186](#)). This dataset is a dimensionless tensor and is computed using the following tensor relationship:

$$\begin{bmatrix} \text{ConversePiezoelectricFieldXX} \\ \text{ConversePiezoelectricFieldYY} \\ \text{ConversePiezoelectricFieldZZ} \\ \text{ConversePiezoelectricFieldYZ} \\ \text{ConversePiezoelectricFieldXZ} \\ \text{ConversePiezoelectricFieldXY} \end{bmatrix} = \begin{bmatrix} d_{11} & d_{12} & d_{13} & d_{14} & d_{15} & d_{16} \\ d_{21} & d_{22} & d_{23} & d_{24} & d_{25} & d_{26} \\ d_{31} & d_{32} & d_{33} & d_{34} & d_{35} & d_{36} \end{bmatrix}^t \begin{bmatrix} E_x \\ E_y \\ E_z \end{bmatrix} \quad (851)$$

where d_{ij} are piezoelectric coefficients and E_k are electric-field components.

Gate-dependent Polarization in GaN Devices

In this model, the polarization vector P has an additional term along the z-axis in the crystal system, which is dependent on the E_z component of the electric field [\[27\]](#):

$$P_z^{\text{new}} = P_z + (e_{33}^2/c_{33}) \cdot E_z \quad (852)$$

This problem can be converted into an equivalent problem with an anisotropic permittivity tensor (z-component increased by e_{33}^2/c_{33}):

$$\nabla \cdot \left(- \begin{bmatrix} \kappa_a E_x \\ \kappa_a E_y \\ \kappa_c E_z \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ \frac{e_{33}^2}{c_{33}} E_z \end{bmatrix} \right) = \nabla \cdot \left(\begin{bmatrix} \kappa_a \\ \kappa_a \\ \kappa_c + \frac{e_{33}^2}{c_{33}} \end{bmatrix} \nabla \phi \right) = -\rho + \nabla \cdot \mathbf{P} \quad (853)$$

where $\mathbf{P}$ is equal to [Eq. 845, p. 735](#) (P_{strain} is Formula 2).

The gate-dependent polarization model can be activated in the `Physics` section as follows:

```
Physics {
  Piezoelectric_Polarization (strain(GateDependent))
}
```

Two-dimensional Simulations

For a 2D simulation, you must ensure that the anisotropic direction (defined in the crystal coordinate system) lies within the xy plane of the simulation system. Depending on the values of `LatticeParameters` in the parameter file (see [Coordinate Systems on page 738](#)), the anisotropic direction must be specified in the `Physics` section of the command file.

The default values of `LatticeParameters` are given by:

```
LatticeParameters {  
    X = (1, 0, 0)  
    Y = (0, 1, 0)  
}
```

Similarly, the default anisotropic direction in a 2D simulation is the y-axis in the crystal system, which coincides with the default y-axis in the simulation system.

However, the following values of `LatticeParameters` are selected frequently for 2D simulations:

```
LatticeParameters {  
    X = (1, 0, 0)  
    Y = (0, 0, -1)  
}
```

In this case, the y-axis in the simulation system runs along the negative z-axis in the crystal system and, therefore, it is necessary to declare the z-axis in the crystal system as the anisotropic direction:

```
Physics {  
    Aniso(direction = zAxis)  
}
```

References

- [1] J. Bardeen and W. Shockley, "Deformation Potentials and Mobilities in Non-Planar Crystals," *Physical Review*, vol. 80, no. 1, pp. 72–80, 1950.
- [2] I. Goroff and L. Kleinman, "Deformation Potentials in Silicon. III. Effects of a General Strain on Conduction and Valence Levels," *Physical Review*, vol. 132, no. 3, pp. 1080–1084, 1963.
- [3] J. J. Wortman, J. R. Hauser, and R. M. Burger, "Effect of Mechanical Stress on p-n Junction Device Characteristics," *Journal of Applied Physics*, vol. 35, no. 7, pp. 2122–2131, 1964.

- [4] P. Smeys, *Geometry and Stress Effects in Scaled Integrated Circuit Isolation Technologies*, Ph.D. thesis, Stanford University, Stanford, CA, USA, August 1996.
- [5] M. Lades *et al.*, “Analysis of Piezoresistive Effects in Silicon Structures Using Multidimensional Process and Device Simulation,” in *Simulation of Semiconductor Devices and Processes (SISDEP)*, vol. 6, Erlangen, Germany, pp. 22–25, September 1995.
- [6] J. L. Egle and D. Chidambarrao, “Strain Effects on Device Characteristics: Implementation in Drift-Diffusion Simulators,” *Solid-State Electronics*, vol. 36, no. 12, pp. 1653–1664, 1993.
- [7] K. Matsuda *et al.*, “Nonlinear piezoresistance effects in silicon,” *Journal of Applied Physics*, vol. 73, no. 4, pp. 1838–1847, 1993.
- [8] J. F. Nye, *Physical Properties of Crystals*, Oxford: Clarendon Press, 1985.
- [9] G. L. Bir and G. E. Pikus, *Symmetry and Strain-Induced Effects in Semiconductors*, New York: John Wiley & Sons, 1974.
- [10] E. Ungersboeck *et al.*, “The Effect of General Strain on the Band Structure and Electron Mobility of Silicon,” *IEEE Transactions on Electron Devices*, vol. 54, no. 9, pp. 2183–2190, 2007.
- [11] T. Manku and A. Nathan, “Valence energy-band structure for strained group-IV semiconductors,” *Journal of Applied Physics*, vol. 73, no. 3, pp. 1205–1213, 1993.
- [12] V. Sverdlov *et al.*, “Effects of Shear Strain on the Conduction Band in Silicon: An Efficient Two-Band $k \cdot p$ Theory,” in *Proceedings of the 37th European Solid-State Device Research Conference (ESSDERC)*, Munich, Germany, pp. 386–389, September 2007.
- [13] F. L. Madarasz, J. E. Lang, and P. M. Hemeger, “Effective masses for nonparabolic bands in p -type silicon,” *Journal of Applied Physics*, vol. 52, no. 7, pp. 4646–4648, 1981.
- [14] C.Y.-P. Chao and S. L. Chuang, “Spin-orbit-coupling effects on the valence-band structure of strained semiconductor quantum wells,” *Physical Review B*, vol. 46, no. 7, pp. 4110–4122, 1992.
- [15] M. V. Fischetti and S. E. Laux, “Band structure, deformation potentials, and carrier mobility in strained Si, Ge, and SiGe alloys,” *Journal of Applied Physics*, vol. 80, no. 4, pp. 2234–2252, 1996.
- [16] S. Reggiani, *Report on the Low-Field Carrier Mobility Model in MOSFETs with biaxial/uniaxial stress conditions*, Internal Report, Advanced Research Center on Electronic Systems (ARCES), University of Bologna, Bologna, Italy, 2009.
- [17] S. Dhar *et al.*, “Electron Mobility Model for Strained-Si Devices,” *IEEE Transactions on Electron Devices*, vol. 52, no. 4, pp. 527–533, 2005.

30: Modeling Mechanical Stress Effect

References

- [18] E. Ungersboeck *et al.*, “Physical Modeling of Electron Mobility Enhancement for Arbitrarily Strained Silicon,” in *11th International Workshop on Computational Electronics (IWCE)*, Vienna, Austria, pp. 141–142, May 2006.
- [19] O. Penzin, L. Smith, and F. O. Heinz, “Low Field Mobility Model for MOSFET Stress and Surface/Channel Orientation Effects,” as discussed at the *42nd IEEE Semiconductor Interface Specialists Conference (SISC)*, Arlington, VA, USA, December 2011.
- [20] B. Obradovic *et al.*, “A Physically-Based Analytic Model for Stress-Induced Hole Mobility Enhancement,” in *10th International Workshop on Computational Electronics (IWCE)*, West Lafayette, IN, USA, pp. 26–27, October 2004.
- [21] L. Smith *et al.*, “Exploring the Limits of Stress-Enhanced Hole Mobility,” *IEEE Electron Device Letters*, vol. 26, no. 9, pp. 652–654, 2005.
- [22] J. R. Watling, A. Asenov, and J. R. Barker, “Efficient Hole Transport Model in Warped Bands for Use in the Simulation of Si/SiGe MOSFETs,” in *International Workshop on Computational Electronics (IWCE)*, Osaka, Japan, pp. 96–99, October 1998.
- [23] Z. Wang, *Modélisation de la piézorésistivité du Silicium: Application à la simulation de dispositifs M.O.S.*, Ph.D. thesis, Université des Sciences et Technologies de Lille, Lille, France, 1994.
- [24] Y. Kanda, “A Graphical Representation of the Piezoresistance Coefficients in Silicon,” *IEEE Transactions on Electron Devices*, vol. ED-29, no. 1, pp. 64–70, 1982.
- [25] O. Ambacher *et al.*, “Two-dimensional electron gases induced by spontaneous and piezoelectric polarization charges in N- and Ga-face AlGaIn/GaN heterostructures,” *Journal of Applied Physics*, vol. 85, no. 6, pp. 3222–3233, 1999.
- [26] O. Ambacher *et al.*, “Two dimensional electron gases induced by spontaneous and piezoelectric polarization in undoped and doped AlGaIn/GaN heterostructures,” *Journal of Applied Physics*, vol. 87, no. 1, pp. 334–344, 2000.
- [27] A. Ashok *et al.*, “Importance of the Gate-Dependent Polarization Charge on the Operation of GaN HEMTs,” *IEEE Transactions on Electron Devices*, vol. 56, no. 5, pp. 998–1006, 2009.

CHAPTER 31 Galvanic Transport Model

This chapter describes the model for carrier transport in magnetic fields.

Model Description

For analysis of magnetic field effects in semiconductor devices, the transport equations governing the flow of electrons and holes in the interior of the device must be set up and solved.

To this end, the commonly used drift-diffusion-based model of the carrier current densities $\vec{J}_n$ and J_p must be augmented by magnetic field-dependent terms that account for the action of the transport equations governing the flow of electrons and holes in the interior of the device, the *Lorentz force* on the motion of the carriers [1][2][3]:

$$\vec{J}_\alpha = \mu_\alpha \vec{g}_\alpha + \mu_\alpha \frac{1}{1 + (\mu_\alpha^* B)^2} [\mu_\alpha^* \vec{B} \times \vec{g}_\alpha + \mu_\alpha^* \vec{B} \times (\mu_\alpha^* \vec{B} \times \vec{g}_\alpha)] \quad \text{with } \alpha = n, p \quad (854)$$

where:

- $\vec{g}_\alpha$ is current vector without mobility (see [Current Densities on page 670](#)).
- μ_α^* is the Hall mobility.
- $\vec{B}$ is the magnetic induction vector, and B is the magnitude of this vector.

The perpendicular (transverse) components of Hall and drift mobility are related by $\mu_n^* = r_n \mu_n$ and $\mu_p^* = r_p \mu_p$, where r_n and r_p denote the Hall scattering factors. In the case of bulk silicon, typical values are $r_n = 1.1$ and $r_p = -0.7$.

Using Galvanic Transport Model

In the Physics section of the command file, specify the magnetic field vector using the keyword `MagneticField = (<x>, <y>, <z>)`. In the following example, a field of 0.1 Tesla is applied parallel to the z-axis:

```
Physics {...  
    MagneticField = (0.0, 0.0, 0.1)  
}
```

This parameter can be ramped (see [Ramping Physical Parameter Values on page 109](#)).

Discretization Scheme for Continuity Equations

Sentaurus Device uses a modified discretization scheme for continuity equations in a constant magnetic field. This scheme contains an additive term to the effective electric field in the Scharfetter–Gummel approximation, that is, in this approximation, the argument to the Bernoulli function has an additional magnetic term. This discretization scheme has good convergence and mesh stability (the new approximation does not demand a special grid).

NOTE The galvanic transport model cannot be combined with the following models: hydrodynamic, impact ionization, aniso, piezo, and quantum modeling.

NOTE The value of the magnetic field is a critical parameter for convergence. For doping-dependent mobility, the convergence is reliable up to $B = 10\text{ T}$. For more general mobility, the convergence has a limit of the magnetic field up to $B = 1\text{ T}$.

References

- [1] W. Allegretto, A. Nathan, and H. Baltes, “Numerical Analysis of Magnetic-Field-Sensitive Bipolar Devices,” *IEEE Transactions on Computer-Aided Design*, vol. 10, no. 4, pp. 501–511, 1991.
- [2] C. Riccobene *et al.*, “Operating Principle of Dual Collector Magnetotransistors Studied by Two-Dimensional Simulation,” *IEEE Transactions on Electron Devices*, vol. 41, no. 7, pp. 1136–1148, 1994.
- [3] C. Riccobene *et al.*, “First Three-Dimensional Numerical Analysis of Magnetic Vector Probe,” in *IEDM Technical Digest*, San Francisco, USA, pp. 727–730, December 1994.

CHAPTER 32 Thermal Properties

This chapter describes the models that are important for lattice heating and the thermodynamic model: heat capacity, thermal conductivity, and thermoelectric power.

Heat Capacity

[Table 119](#) lists the values of the heat capacity used in the simulator.

Table 119 Values of heat capacity c for various materials

Material	c [J/K cm ³]	Reference
Silicon	1.63	[1]
Ceramic	2.78	[2]
SiO ₂	1.67	[2]
Poly Si	1.63	$\approx$ Si

By default, or when you specify `HeatCapacity(TempDep)` in the Physics section, the temperature dependency of the lattice heat capacity is modeled by the empirical function:

$$c_L = cv + cv_b T + cv_c T^2 + cv_d T^3 \quad (855)$$

The equation coefficients can be specified in the parameter file by using the syntax:

```
LatticeHeatCapacity{
  cv = 1.63          # [J/(K cm^3)]
  cv_b = 0.0000e+00 # [J/(K^2 cm^3)]
  cv_c = 0.0000e+00 # [J/(K^3 cm^3)]
  cv_d = 0.0000e+00 # [J/(K^3 cm^3)]
}
```

All these coefficients can be mole fraction–dependent for mole-dependent materials.

To use a PMI model to compute heat capacity, specify the name of the model as a string as an option to `HeatCapacity` (see [Heat Capacity on page 1092](#)). To use a multistate configuration–dependent PMI for heat capacity, specify the model and its parameters as arguments to `PMIModel`, which, in turn, is an argument to `HeatCapacity` (see [Multistate Configuration–dependent Heat Capacity on page 1095](#)).

To use a constant lattice heat capacity without touching the parameter file, specify `HeatCapacity(Constant)`.

The pmi_msc_heatcapacity Model

The model may depend on state occupation probabilities of a multistate configuration (MSC). If no explicit dependency on an MSC is given, it enables a piecewise linear (pwl) dependency on the lattice temperature, that is, it reads:

$$c_V = c_V(T) \quad (856)$$

If the model depends on an MSC, for each MSC state, the heat capacity can be pwl temperature-dependent. The overall heat capacity is then averaged according to:

$$c_V = \sum_i c_{V,i}(T) s_i \quad (857)$$

where the sum is taken over all MSC states, and s_i are the state occupation probabilities.

The model is activated in the Physics section by:

```
HeatCapacity ( PMIModel ( Name="pmi_msc_heatcapacity" MSCConfig="msc0" ) )
```

Table 120 Parameters of pmi_msc_heatcapacity

Name	Symbol	Default	Unit	Range	Description
plot	–	0	–	{0,1}	Plot parameter to screen
cv	c_V	0.	J/Kcm ³	real	Constant value
cv_nb_Tpairs	–	0	–	≥ 0	Number of interpolation points
cv_Tp<int>_X	–	–	K	real	Temperature at <int>-th interpolation point
cv_Tp<int>_Y	–	–	J/Kcm ³	real	Value at <int>-th interpolation point

Table 120 lists the parameters of the model. With `cv`, you specify a constant heat capacity, which is used as a global (no MSC dependency) or default state heat capacity. However, if you specify `cv_nb_Tpairs` greater than zero, the global and default state heat capacities are piecewise linear. In that case, you must specify the interpolation points as pairs of temperature and heat capacity values by `cv_Tp<int>_X` and `cv_Tp<int>_Y`, respectively. <int> ranges from zero to one less than `cv_nb_Tpairs`. The global `cv_` parameters can be prefixed with:

<state_name>_

for the named MSC states to overwrite the default state behavior.

Thermal Conductivity

Sentaurus Device uses the following temperature-dependent thermal conductivity κ in silicon [3]:

$$\kappa(T) = \frac{1}{a + bT + cT^2} \quad (858)$$

where $a = 0.03 \text{ cmKW}^{-1}$, $b = 1.56 \times 10^{-3} \text{ cmW}^{-1}$, and $c = 1.65 \times 10^{-6} \text{ cmW}^{-1} \text{ K}^{-1}$. The range of validity is from 200 K to well above 600 K. Values of the thermal conductivity for some materials are given in Table 121.

Table 121 Values of thermal conductivity κ of silicon versus temperature

Material	κ [W/(cm K)]	Reference
Silicon	Eq. 858	[3]
Ceramic	0.167	[4]
SiO ₂	0.014	[1]
Poly Si	1.5	$\approx$ Si

As additional options to the standard specification of the thermal conductivity model, there are two different expressions to define either thermal resistivity $\chi = 1/\kappa$ or thermal conductivity for any material. This is performed by using Formula in the parameter file or special keywords in the command file.

For Formula=0 (thermal resistivity specification), Sentaurus Device uses:

$$\chi = 1/\kappa + 1/\kappa_b T + 1/\kappa_c T^2 \quad (859)$$

For Formula=1 (thermal conductivity specification), it is:

$$\kappa = \kappa + \kappa_b T + \kappa_c T^2 \quad (860)$$

32: Thermal Properties

Thermal Conductivity

Using the following syntax in the parameter file, the expressions can be switched coefficients specified:

```
kappa{
  Formula = 0
  1/kappa = 0.03          # [K cm/W]
  1/kappa_b = 1.5600e-03  # [cm/W]
  1/kappa_c = 1.6500e-06  # [cm/(W K)]
  kappa = 1.5             # [W/(K cm)]
  kappa_b = 0.0000e+00    # [W/(K^2 cm)]
  kappa_c = 0.0000e+00    # [W/(K^3 cm)]
}
```

The **Physics** section of the command file provides more flexibility to switch these expressions by using the keywords:

```
ThermalConductivity(
  TempDep Conductivity # Formula = 1
  Constant Conductivity # Formula = 1 without temperature dependence
  TempDep Resistivity # Formula = 0
  Constant Resistivity # Formula = 0 without temperature dependence
)
```

By default, Sentaurus Device uses **Formula** specified in the parameter file. All these coefficients in the parameter file can be mole dependent for mole-dependent materials.

Furthermore, a simple PMI and a multistate configuration–dependent PMI are available to compute thermal conductivity. See [Thermal Conductivity on page 1084](#) and [Multistate Configuration–dependent Thermal Conductivity on page 1088](#) for details.

The pmi_msc_thermalconductivity Model

This model depends on the lattice temperature and the state occupation probabilities of an MSC. If it does not depend on an MSC, it enables a pwl temperature dependency and reads as:

$$\kappa = \kappa(T) \quad (861)$$

If an explicit MSC dependency is given, the global thermal conductivity is computed as:

$$\kappa = \sum_i \kappa_i(T) s_i \quad (862)$$

where the sum is taken over all MSC states, κ_i are the state thermal conductivities, and s_i are the state occupation probabilities.

The model is enabled in the Physics section by:

```
ThermalConductivity (
  PMIModel ( Name="pmi_msc_thermalconductivity" MSConfig="m0" )
)
```

Table 122 Parameters of pmi_msc_thermalconductivity

Name	Symbol	Default	Unit	Range	Description
plot	—	0	—	{0,1}	Plot parameter to screen
kappa	κ	0.	W/Kcm	real	Constant value
kappa_nb_Tpairs	—	0	—	≥ 0	Number of interpolation points
kappa_Tp<int>_X	—	—	K	real	Temperature at <int>-th interpolation point
kappa_Tp<int>_Y	—	—	W/Kcm	real	Value at <int>-th interpolation point

Table 122 lists the parameters of the model. Refer to [The pmi_msc_heatcapacity Model on page 746](#) for an explanation of the individual parameters and how the parameters can be overwritten for individual MSC states. Only the name of the parameters (use kappa) and the unit changes.

Thermoelectric Power (TEP)

Sentaurus Device supports the following choices for computing the thermoelectric powers P_n and P_p in semiconductors:

- Use a tabulated set of experimental values for silicon as a function of temperature and carrier concentration.
- Use analytic formulas with two adjustable parameters κ and s .
- As a user-defined function of the carrier density and the lattice temperature (thermoelectric power PMI).

In metals, the thermoelectric power P is only allowed as a user-defined function of the electric field vector and the lattice temperature (as a PMI model).

Physical Model

By default, Sentaurus Device computes the thermoelectric powers P_n and P_p using a table of experimental values of TEPs for silicon published by Geballe and Hull [5] as functions of temperature and carrier concentration. Sentaurus Device extrapolates $P_n T$ and $P_p T$ linearly

32: Thermal Properties

Thermoelectric Power (TEP)

for temperatures between 360 K and 500 K, thereby preserving the $1/T$ dependency of data presented at higher temperatures by Fulkerson *et al.* [6], which holds up to near the intrinsic temperature.

P_n and P_p are shown in Figure 56 as a function of temperature and carrier concentration as used in Sentaurus Device.

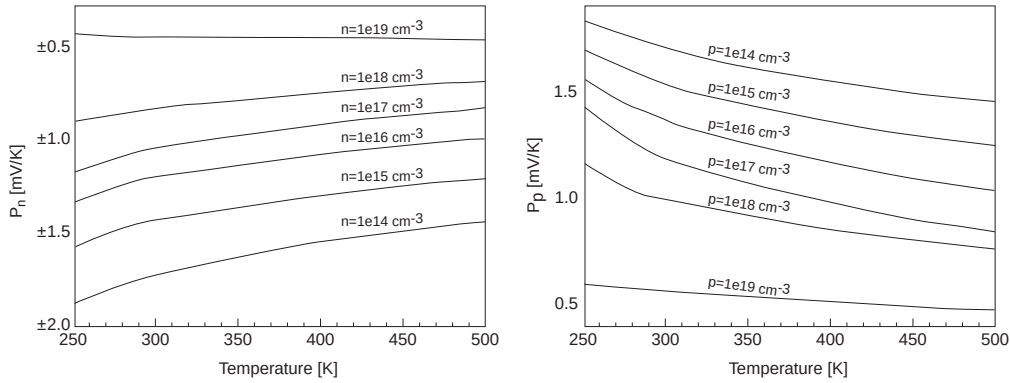


Figure 56 TEPs P_n (left) and P_p (right) as a function of temperature and carrier concentration

As an alternative, analytic formulas as described in [7][8] can be used to compute thermoelectric powers in nongenerate semiconductors:

$$P_n = -\kappa_n \frac{k}{q} \left[\left(\frac{5}{2} - s_n \right) + \ln \left(\frac{N_C}{n} \right) \right] \quad (863)$$

$$P_p = \kappa_p \frac{k}{q} \left[\left(\frac{5}{2} - s_p \right) + \ln \left(\frac{N_V}{p} \right) \right] \quad (864)$$

where you can adjust the parameters κ and s in the parameter file. The parameter default values are listed in Table 121 on page 747.

In the most general case, the TEPs in semiconductors can be computed using the thermoelectric power PMI (see Thermoelectric Power on page 1158) as functions of the lattice temperature and the carrier density. Both the standard and simplified PMI (with automatic derivatives) are supported.

In metals, thermoelectric power is defined as a PMI depending on the gradient of the Fermi potential $\nabla \Phi_M$ and the lattice temperature T (see Metal Thermoelectric Power on page 1163). It is used in connection with thermodynamic transport in metals (Seebeck effect).

Using Thermoelectric Power

The thermoelectric powers in semiconductors are computed automatically when the Temperature equation is solved and the keyword `Thermodynamic` is specified in the global `Physics` section. By default, tabulated silicon data is used.

To enable an analytic formula (Eq. 863, p. 750, Eq. 864, p. 750) or thermoelectric power PMI models either regionwise, or materialwise, or globally, the keyword `TEPower` with `Analytic` or the PMI name as an option must be specified in the corresponding `Physics` section. For example:

```
Physics(Region="region1") {
    TEPower(Analytic)           # analytic formula TEP model in "region1"
}

Physics(Region="region2") {
    TEPower(pmi_tepower)       # PMI in "region2"
}
```

activates an analytic formula in "region1", the thermoelectric power PMI `pmi_tepower` in "region2", and tabulated data interpolation in all other semiconductor regions.

For backward compatibility, the keyword `AnalyticTEP` in the `Physics` section is still available to activate an analytic formula TEP globally. It is equivalent to specifying `TEPower(Analytic)` in the global `Physics` section.

The coefficients for the thermoelectric powers defined by the analytic formula are available in the `TEPower` parameter set (see [Table 121 on page 747](#)).

The thermoelectric power in metals is activated selectively when the Temperature equation is solved and the keyword `Thermodynamic` is specified in the global `Physics` section. To active it regionwise, materialwise, or globally, the keyword `MetalTEPower` with the metal thermoelectric power PMI name as an option must be specified in the corresponding `Physics` sections. For example:

```
Physics(Material="Copper") {
    MetalTEPower(pmi_tepower)
}
```

activates the metal TEP computation and thermodynamic transport in the copper part of the device.

Heating at Contacts, Metal–Semiconductor and Conductive Insulator–Semiconductor Interfaces

The Peltier heat at a contact, or a metal–semiconductor interface, or a conductive insulator–semiconductor interface is modeled as:

$$Q = J_n(\alpha_n \Delta E_n + (1 - \alpha_n) \Delta \varepsilon_n) \quad (865)$$

$$Q = J_p(\alpha_p \Delta E_p + (1 - \alpha_p) \Delta \varepsilon_p) \quad (866)$$

where:

- Q is the heat density at the interface or contact (when $Q > 0$, there is heating; when $Q < 0$, cooling).
- J_n and J_p are the electron and hole current densities normal to the interface or contact.
- ΔE_n and ΔE_p are the energy differences for electrons and holes across the interface or at the contact.
- α_n , α_p , $\Delta \varepsilon_n$, and $\Delta \varepsilon_p$ are fitting parameters with $0 \leq \alpha_n, \alpha_p \leq 1$.

For the thermodynamic model, Sentaurus Device computes the energy differences for electrons and holes as:

$$\Delta E_n = \Phi_M - \beta_n(\Phi_n + \gamma_n TP_n) + (1 - \beta_n)E_C/q \quad (867)$$

$$\Delta E_p = \Phi_M - \beta_p(\Phi_p + \gamma_p TP_p) + (1 - \beta_p)E_V/q \quad (868)$$

where β_n , β_p , γ_n , and γ_p are fitting parameters.

For the default lattice temperature model and the hydrodynamic model, Sentaurus Device computes the energy differences for electrons and holes as:

$$\Delta E_n = \Phi_M + E_C/q \quad (869)$$

$$\Delta E_p = \Phi_M + E_V/q \quad (870)$$

To activate Peltier heat, the keyword `MSPeltierHeat` must be specified inside the corresponding interface or electrode `Physics` section:

```
Physics(MaterialInterface="Silicon/Metal") {
    MSPeltierHeat
    ...
}
```

or:

```
Physics(Electrode="cathode") {
  MSPeltierHeat
  ...
}
```

The fitting parameters α , β , γ , and $\Delta\epsilon$ can be specified in the MSPeltierHeat parameter set of the region interface or electrode for which the Peltier heat is computed:

```
MaterialInterface = "Silicon/Metal" {
  MSPeltierHeat
  {
    alpha = 1.0 , 1.0 # [1]
    beta  = 1.0 , 1.0
    gamma = 1.0 , 1.0
    deltaE = 0.0 , 0.0 # [eV]
  }
}
```

Their default values are $\alpha_n = \alpha_p = \beta_n = \beta_p = \gamma_n = \gamma_p = 1$ and $\Delta\epsilon_n = \Delta\epsilon_p = 0$ eV.

References

- [1] S. M. Sze, *Physics of Semiconductor Devices*, New York: John Wiley & Sons, 2nd ed., 1981.
- [2] D. J. Dean, *Thermal Design of Electronic Circuit Boards and Packages*, Ayr, Scotland: Electrochemical Publications Limited, 1985.
- [3] C. J. Glassbrenner and G. A. Slack, "Thermal Conductivity of Silicon and Germanium from 3°K to the Melting Point," *Physical Review*, vol. 134, no. 4A, pp. A1058–A1069, 1964.
- [4] S. S. Furkay, "Thermal Characterization of Plastic and Ceramic Surface-Mount Components," *IEEE Transactions on Components, Hybrids, and Manufacturing Technology*, vol. 11, no. 4, pp. 521–527, 1988.
- [5] T. H. Geballe and G. W. Hull, "Seebeck Effect in Silicon," *Physical Review*, vol. 98, no. 4, pp. 940–947, 1955.
- [6] W. Fulkerson *et al.*, "Thermal Conductivity, Electrical Resistivity, and Seebeck Coefficient of Silicon from 100 to 1300°K," *Physical Review*, vol. 167, no. 3, pp. 765–782, 1968.

32: Thermal Properties

References

- [7] R. A. Smith, *Semiconductors*, Cambridge: Cambridge University Press, 2nd ed., 1978.
- [8] C. Herring, “The Role of Low-Frequency Phonons in Thermoelectricity and Thermal Conduction,” in *Semiconductors and Phosphors: Proceedings of the International Colloquium*, Garmisch-Partenkirchen, Germany, pp. 184–235, August 1956.

Part III Physics of Lasers and Light-Emitting Diodes

This part of the *Sentaurus Device User Guide* contains the following chapters:

[Chapter 33 Introduction to Lasers and Light-Emitting Diodes on page 757](#)

[Chapter 34 Lasers on page 777](#)

[Chapter 35 Light-Emitting Diodes on page 811](#)

[Chapter 36 Optical Mode Solver for Lasers on page 853](#)

[Chapter 37 Modeling Quantum Wells on page 895](#)

[Chapter 38 Additional Features of Laser or LED Simulations on page 919](#)

CHAPTER 33 Introduction to Lasers and Light-Emitting Diodes

This chapter is an introduction to the simulation of lasers and light-emitting diodes in Sentaurus Device.

Sentaurus Device includes optional models for the comprehensive simulation of semiconductor lasers and light-emitting diodes (LEDs). Both edge-emitting lasers and vertical-cavity surface-emitting lasers (VCSELs) are supported. Drift-diffusion or hydrodynamic transport equations for the carriers, the Schrödinger equation for quantum well gain, modal optical rate equations, and the Helmholtz equation are solved self-consistently in the quasistationary and transient modes.

Spontaneous and stimulated optical recombinations are calculated in the active and bulk regions according to Fermi's golden rule. They are added as carrier recombination mechanisms in the continuity equations and as modal gain and spontaneous emission in the photon rate equations. Different line-width broadening models are available for gain broadening.

Both bulk and quantum well (QW) lasers can be simulated. In the case of QW lasers, the optical polarization dependence of the optical matrix element is automatically taken into account. The QW subbands are calculated as the solution of the single-band Schrödinger equation in the effective mass approximation, assuming a box-shaped potential. Strain effects can also be taken into account. The distribution of carriers in the well is determined according to the quantum mechanical wavefunctions and QW density-of-states. Alternatively, you can create your own stimulated and spontaneous gain model through a physical model interface (PMI).

The next section is a short introduction on how to set up a laser simulation with single and dual grids, and which type of output can be extracted from the simulation. The theoretical foundations of the laser simulator with detailed equations are discussed, followed by a description of other useful features of the laser and LED simulator. Finally, there is a discussion on how to simulate different types of laser and LED.

Overview

A laser or an LED simulation is different from a silicon device simulation in three aspects:

- The Helmholtz equation must be solved to determine the optics of the device.
- A photon rate equation must be included to couple the electronics to the optics.

- The carrier-scattering processes at the quantum well must be treated differently because it is no longer in the drift-diffusion regime. In this case, a new syntax must be introduced to activate the laser simulation within the Sentaurus Device framework. The laser-specific extension of the command file are examined in the next sections.

The procedure for performing a laser simulation is similar to that for a silicon device simulation. [Figure 57](#) illustrates the procedure.

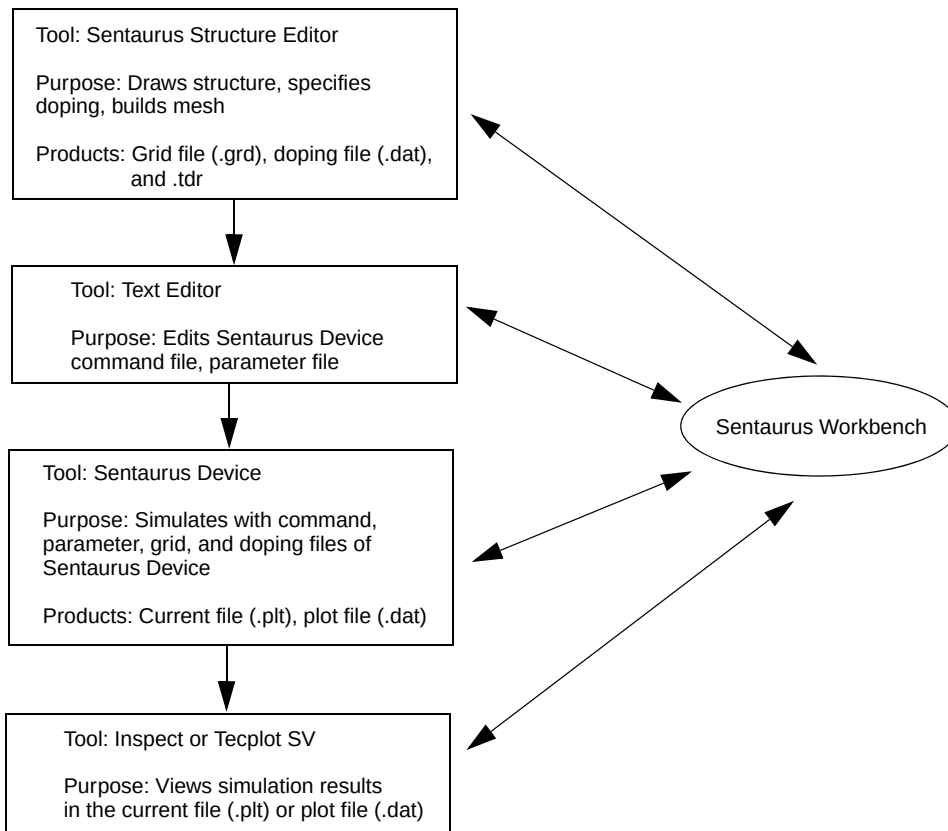


Figure 57 Flow of tasks in a single laser simulation run

First, the structure and doping profile are drawn in Sentaurus Structure Editor (refer to the *Sentaurus Structure Editor User Guide*). Second, Sentaurus Structure Editor generates the mesh and the doping information on the mesh, and saves them in the grid file (with extension .grd) and doping file (with extension .dat), respectively. Instead of drawing the structure laboriously, you have the option to write a script to construct the structure, and build the mesh and doping automatically. In this way, it is possible to parameterize any device dimension, material composition, and doping profile easily. This feature is used by Sentaurus Workbench to control batch jobs and automatically optimize user-specified performance benchmarks through parameter variation.

Next, you can use any editor to edit the command file and parameter file. The parameter file contains a complete list of default material parameter models such as band gap, thermal conductivities, recombination parameters, traps parameters, effective masses, and mobilities. You can edit any of these material parameters. The command file contains instructions to run the simulation. If you run a 1D, 2D, or 3D simulation, only the grid and doping files change. The command file and parameter file are unchanged.

After the simulation is completed, Sentaurus Device saves the final results in the current file (with extension .plt) and the plot file (with extension .dat). These results can be viewed and plotted in Inspect and Tecplot SV, respectively. If you specify some other feature such as far-field computation, additional files are produced that can be viewed in Inspect or Tecplot SV, depending on the file type.

All the abovementioned steps can be controlled from Sentaurus Workbench, which can also schedule jobs over the network and extract specified results of the simulation. In this manner, automation can be achieved for device optimization. Refer to the *Sentaurus Workbench User Guide* for more information.

Command File Syntax

The command file permits full control of how a simulation will run. This includes choosing the type of numeric solver, the physics to be included, and the equations to couple. This fundamental framework provides a platform with unlimited expansion possibilities for new features, new models, and new devices. The fastest way to understand the command file syntax is to look at a generic example that you can copy and modify for your specific needs. Three generic examples are presented for typical laser simulations.

NOTE The character '#' in the command file means that any text proceeding from '#' is treated as a remark or as a preprocessor directive.

Simulating Single-Grid Edge-Emitting Laser

This generic example uses the same grid for both the optical and electrical problems. The optical problem is defined by the optics solver and the electrical problem is defined by the semiconductor transport equations.

```
File {
  Grid      = "mesh_mdr.tdr"
  Parameters = "des_las.par"

  Current = "current"
  Output  = "output"
  Plot    = "plot"

  SaveOptField = "laserfield" # save optics
  FarField     = "farfield"   # farfield computation
  Gain         = "gain"       # save gain curves
}

Electrode {
  {Name="p_Contact" voltage=0.8 AreaFactor = 200}
  {Name="n_Contact" voltage=0.0 AreaFactor = 200}
}

Plot {
  # ----- Extra variables for laser simulation -----
  MatGain
  LaserIntensity
  OpticalIntensityMode0
  OpticalIntensityMode1
  OpticalPolarizationAngleMode0
  OpticalPolarizationAngleMode1
}

Physics {
  AreaFactor=2 # for symmetric simulation
  Laser (
    Optics (
      FEScalar (
        EquationType = Waveguide
        Symmetry = Symmetric           # or NonSymmetric or Periodic
        LasingWavelength = 800         # [nm]
        TargetEffectiveIndex = (3.4 3.34) # initial guess
        Polarization = (TE TM)         # for scalar solver only
        Boundary = ("Type2" "Type1")   # choose boundary conditions
        ModeNumber = 2                 # up to 10 modes
      )
    )
  )
}
```

```

)

TransverseModes          # for edge emitter simulation
CavityLength = 900        # [micrometer]
lFacetReflectivity = 0.9  # Left facet power reflectivity
rFacetReflectivity = 0.4  # Right facet power reflectivity
OpticalLoss = 10.0        # [1/cm]
WaveguideLoss             # activate feedback of loss from Optics

Broadening (Type=Lorentzian Gamma=0.010)  # Gamma in [eV]

# ----- Specify QW physics -----
qwTransport
qwExtension = AutoDetect  # auto read of QW widths

Strain                # include QW Strain effects
StimScaling = 1.0
SponScaling = 1.0
RefractiveIndex (TemperatureDep CarrierDep)
)

# ----- Specify transport physics -----
Thermionic            # thermionic emission over interfaces
HeteroInterface       # discontinuous band gap & quasi-Fermi level
Mobility ( DopingDependence )
Recombination ( SRH Auger )
EffectiveIntrinsicDensity ( NoBandGapNarrowing )
Fermi
}

Physics (region="pbulk") { MoleFraction(xfraction=0.28) }
Physics (region="nbulk") { MoleFraction(xfraction=0.28) }
Physics (region="psch") { MoleFraction(xfraction=0.09) }
Physics (region="nsch") { MoleFraction(xfraction=0.09) }

Physics (region="qw1") {
    MoleFraction( xfraction = 0.8 Grading=0.00 )
    Active                      # keyword to indicate QW region
}

Math {
    Digits = 5
    Extrapolate
    Method = pardiso
    Iterations = 30
    Notdamped = 50
    RelErrControl
}

```

33: Introduction to Lasers and Light-Emitting Diodes

Command File Syntax

```
GainPlot      { Range=(1.22,1.32) Intervals=150 }
FarFieldPlot { Range=(40,60)      Intervals=40 Scalar1D Scalar2D Vector2D }

Solve {
  Poisson
  Coupled { Poisson Electron Hole }
  Coupled { Poisson Electron Hole PhotonRate }

  Quasistationary (
    InitialStep = 0.001
    MaxStep     = 0.05
    Minstep     = 1e-5

    Plot          {Range=(0,1) Intervals=1}
    SaveOptField {Range=(0,1) Intervals=1}
    PlotFarField {Range=(0,1) Intervals=1}
    PlotGain      {Range=(0,1) Intervals=1}

    Goal { name="p_Contact" Voltage=1.8 }

  ) {
    Plugin (BreakOnFailure) {
      Coupled { Poisson Electron Hole PhotonRate }
      Optics
      Wavelength
    }
  }
}
```

To elaborate on some remarks in this example of code:

- The general skeleton for the command file contains the **Electrode**, **File**, **Plot**, **Math**, **Physics**, and **Solve** sections. Within each section, additional subsections can be defined and, therefore, provide an object-oriented approach for defining the various parameters of the device and simulation.
- In the **File** section, the keyword of a specific option must be included to activate that option. For example, Sentaurus Device will only save the optical field if **SaveOptField** is included.
- In the **Electrode** section, **AreaFactor** gives the scaling factor to account for the device width in the longitudinal direction (1 μm by default).
- In the **Plot** section, a more complete list of the standard variables of Sentaurus Device can be found in [Device Plots on page 144](#). The laser simulation-specific plot variables are listed in the following sections.
- In the **Physics-Laser** section, **Laser** and **LED** simulations share these common options. The main difference between the different types of laser simulation is the choice of optical

solver. In this example, the FEScalar (finite-element scalar) solver is used. Other optical solver choices include FEVectorial, RayTrace, TMM1D, and EffectiveIndex. The different types of optical solver are described in [Chapter 36 on page 853](#).

- In the Physics-Laser section, you can select different gain-broadening functions. The gain-broadening functions, nonlinear gain saturation model, and quantum well parameters are described in [Chapter 37 on page 895](#).
- In the Physics section, there are options to switch on the thermodynamic or hydrodynamic systems. To solve for the temperatures, it is necessary to define the thermode and couple the relevant equation in the command file. (This is not shown in this example.) Details of how to do this are in [Performing Temperature Simulation on page 923](#).
- In the Math section, there is a selection of numeric engines including pardiso, ils, super. You are encouraged to use different engines to find out which is the fastest and most accurate for your applications (see [Linear Solvers on page 165](#)).
- The parameters to control the resolution of the gain plot and the far field are specified in the GainPlot and FarFieldPlot sections, respectively. These sections are at the same level as the Physics, Math, and Solve sections.
- In the Solve-Quasistationary section, the step size is expressed as a number between 0 and 1, which is mapped to the actual voltage ramping goal. In this case, the device was initially at 0.8 V and the goal was set to 1.8 V. Therefore, the unitless step of [0,1] maps to [0.8,1.8] V in the voltage ramp.
- In the Solve-quasistationary section, the keyword Coupled ensures that the listed equations are solved using Newton iterations. To enforce self-consistency of other systems (in this case, the Wavelength and Optics) in a Gummel-type iteration scheme, the Plugin feature was used. If it is not expected that the wavelength or optics will change as a function of the bias, they can be commented out.

Simulating Dual-Grid Edge-Emitting Laser

In some cases, you may require different mesh sizes for the optical and electrical problems. As a general rule, the discretization for the optical mesh should be at least 20 points per wavelength for an accurate solution, but such fine discretization may not be necessary for the electrical problem.

In this case, the dual-grid capability is invoked. This capability is derived from the circuit mixed-mode feature of Sentaurus Device where the coupling of a few devices can be simulated in a self-consistent manner.

33: Introduction to Lasers and Light-Emitting Diodes

Command File Syntax

The syntax of how this is performed is:

```
File {
    Output = "dual_log"
}

Math {
    # ----- Differences in a dual grid -----
    NoAutomaticCircuitContact
    DirectCurrent
    Method = blocked
    Submethod = pardiso
    # ----- The rest are the same as the single grid -----
    Digits = 5
    Extrapolate
    Iterations = 30
    Notdamped = 50
    RelErrControl
    # Cylindrical          # for VCSEL
}

* ----- Define the optical device ----- *
OpticalDevice optgrid {
    File {
        Grid = "optmesh_mdr.tdr"
        Parameters = "des_las.par"
    }

    Plot {
        LaserIntensity
        OpticalIntensityMode0
        OpticalIntensityMode1
    }

    Physics (region="pbulk") { MoleFraction(xfraction=0.28) }
    Physics (region="nbulk") { MoleFraction(xfraction=0.28) }
    Physics (region="psch") { MoleFraction(xfraction=0.09) }
    Physics (region="nsch") { MoleFraction(xfraction=0.09) }

    Physics (region="qw1") {
        MoleFraction( xfraction = 0.8 Grading=0.0 )
        Active
    }
}

* ----- Define the electrical device ----- *
Device electricaldev {
    Electrode {
        {Name="p_Contact" voltage=0.8 AreaFactor = 200}
    }
}
```

```

    {Name="n_Contact" voltage=0.0 AreaFactor = 200}
}

File {
    Grid      = "elecmesh_mdr.tdr"
    Parameters = "des_las.par"

    Current    = "elec_current"
    Plot        = "elec_plot"
    SaveOptField = "laserfield"
    FarField    = "farfield"
    Gain        = "gain"
    # VCSELNearField = "vcselfield"    # for VCSEL
}

Plot {
    # ----- Extra variables for laser simulation -----
    MatGain
}

Physics {
    AreaFactor=2    # for symmetric simulation
    Laser (
        Optics (
            FEVectorial (
                EquationType = Waveguide          # or Cavity for VCSEL
                Symmetry = Symmetric              # for EEL only
                LasingWavelength = 800
                TargetEffectiveIndex = (3.4 3.32)  # for EEL only
                Boundary = ("Type2" "Type1")      # for EEL only
                ModeNumber = 2
                # TargetWavelength = (850.4 851.7) # for VCSEL
                # AzimuthalExpansion = (0 1)      # for VCSELs
                # Coordinates = Cylindrical        # Cylindrical for VCSEL
            )
        )
        # VCSEL()          # for VCSEL
        TransverseModes    # for EEL
        CavityLength = 900  # [micrometer], for EEL
        lFacetReflectivity = 0.9 # Left facet power reflectivity, for EEL
        rFacetReflectivity = 0.4 # Right facet power reflectivity, for EEL
        OpticalLoss = 10.0    # [1/cm]
        WaveguideLoss        # activate feedback of loss from Optics
        Broadening (Type=Lorentzian Gamma=0.10) # [eV]

        # ----- Specify QW physics -----
        qwTransport
        qwExtension = AutoDetect    # auto read QW widths
    )
}

```

33: Introduction to Lasers and Light-Emitting Diodes

Command File Syntax

```
        Strain                # include QW Strain effects
        StimScaling = 1.0
        SponScaling = 1.0
        RefractiveIndex (TemperatureDep CarrierDep)
    )

    # ----- Specify transport physics -----
    Thermionic                # thermionic emission over interfaces
    HeteroInterface           # discontinuous band gap & quasi-Fermi level
    Mobility ( DopingDependence )
    Recombination ( SRH Auger )
    EffectiveIntrinsicDensity ( NoBandGapNarrowing )
    Fermi
}

Physics (region="pbulk") { MoleFraction(xfraction=0.28) }
Physics (region="nbulk") { MoleFraction(xfraction=0.28) }
Physics (region="psch") { MoleFraction(xfraction=0.09) }
Physics (region="nsch") { MoleFraction(xfraction=0.09) }

Physics (region="qw1") {
    MoleFraction( xfraction=0.8 Grading=0.0 )
    Active                # keyword to indicate QW region
}

GainPlot      { Range=(1.22,1.32) Intervals=150 }
FarFieldPlot { Range=(40,60)      Intervals=40 Scalar1D Scalar2D Vector2D }
# VCSELNearFieldPlot { Position=0 Radius=10.0 NumberOfPoints=50 } # VCSEL
}

System {
    # --- Define d2 of type optgrid ---
    optgrid d2 ()
    # --- Define d1 of type diode, and coupled to d2 ---
    diode d1 (p_Contact=vdd n_Contact=gnd) {Physics{OptSolver="d2"}}
    # --- Set the initial bias voltage to circuit contacts ---
    Vsource_pset drive(vdd gnd){ dc=0.8 }
    Set ( gnd = 0.0 )
}

Solve {
    Poisson
    Coupled { Poisson Electron Hole }
    Coupled { Poisson Electron Hole PhotonRate Contact Circuit }

    Quasistationary (
        InitialStep = 0.001
    )
}
```

```

MaxStep = 0.05
Minstep = 1e-5

Plot          {Range=(0,1) Intervals=1}
SaveOptField {Range=(0,1) Intervals=1}
PlotFarField {Range=(0,1) Intervals=1}
PlotGain      {Range=(0,1) Intervals=5}
# VCSELNearField {Range=(0,1) Intervals=1} # for VCSEL

Goal { Parameter=drive.dc Value=1.6 }
) {
  Plugin (BreakOnFailure) {
    Coupled { Poisson Electron Hole PhotonRate ontact Circuit }
    Optics
    Wavelength
  }
}
}

```

This example uses the same device structure as in [Simulating Single-Grid Edge-Emitting Laser on page 760](#), so that you can compare the single-grid and dual-grid simulations more clearly. Some important differences and similarities are:

- The optical and electrical grids may have different material-region physics, but the structural shape should be the same. This is to ensure that the interpolating of parameters and variables between the two grids is accurate. In this example, the electrical and optical devices have the same material regions, but the meshing resolutions are different, leading to two different grids.
- The control of the laser physics is encompassed in the definition of the electrical grid or device. The keyword to define the electrical grid or device is `Device`; `OpticalDevice` defines the optical grid.
- In the `System` section, `d2` is defined as a device instance of type `optgrid` and `d1` is defined as a device instance of type `diode`. The optical device instance `d2` is then coupled as a circuit element to the electrical device instance `d1` by specifying `OptSolver="d2"`. In addition, `p_Contact` and `n_Contact` of the electrical device are given the new names of `vdd` and `gnd`, respectively, in this coupled circuit (see [System Section on page 87](#) for more information about the `System` command).
- In the `Solve` section, the keywords `Contact` and `Circuit` in the coupled statement ensure that self-consistency between the optical and electrical devices in the circuit is imposed.

Simulating Vertical-Cavity Surface-Emitting Laser

A vertical-cavity surface-emitting laser (VCSEL) simulation is only performed if the keyword VCSEL is included in the Physics-Laser section of the command file:

```
# ----- Sentaurus Device command file for dual grid VCSEL simulation -----
File {
    Output = "dual_log"
}

# ----- Control of the numerical method in the mixed-mode circuit -----
Math {
    NoAutomaticCircuitContact
    DirectCurrent
    Method = blocked
    Submethod = pardiso
    Digits = 5
    Extrapolate
    ErrRef(electron) = 1.e3
    ErrRef(hole) = 1.e3
    Iterations = 30
    Notdamped = 50
    RelErrControl
}

# ===== Define the Optical grid =====
# ----- Use keyword OpticalDevice -----
OpticalDevice optgrid {

    File {
        # ----- Read in the optical mesh -----
        Grid = "optmesh_mdr.tdr"
        Parameters = "des_las.par"
    }
    Plot {
        LaserIntensity
        OpticalIntensityMode0
    }
    # ----- Material region physics for the optical problem -----
    ... # all the layers including the DBR layers

    # ----- Quantum wells -----
    Physics (region="qw1") {
        MoleFraction(xfraction = 0.8 Grading=0.00)
        Active
    }
    Physics (region="barr") { MoleFraction(xfraction=0.09) }
```

```

Physics (region="qw2") {
    MoleFraction(xfraction = 0.8 Grading=0.00)
    Active
}

}
# ===== End of Optical grid definition =====

# ===== Define the electrical grid and solver info =====
# ----- Use keyword Device-----
Device electricaldev {

    Electrode {
        { Name="p_Contact" voltage=0.8 AreaFactor=1 }
        { Name="n_Contact" voltage=0.0 AreaFactor=1 }
    }
    File {
        # ----- Read in electrical grid mesh -----
        Grid = "elecmesh_mdr.tdr"
        Parameters = "des_las.par"
        Current = "elec_current"
        Plot = "elec_plot"
        SaveOptField = "laserfield"
        Gain = "gain"
        VCSELNearField = "nf"
        FarField = "far"
    }
    Plot {
        # ----- Can include a long list -----
        LaserIntensity
        OpticalIntensityMode0
    }
    Physics {
        AreaFactor = 1
        # ----- Laser definition -----
        Laser(
            Optics(
                FEVectorial(EquationType = Cavity
                    Coordinates = Cylindrical
                    TargetWavelength = (781.2 776.5) # initial guesses [nm]
                    TargetLifetime = (2.4 1.2) # initial guesses [ps]
                    AzimuthalExpansion = (1 0) # cylindrical harmonic order
                    ModeNumber = 2
                )
            )
        )
        # ----- Signify this is a VCSEL simulation -----
        VCSEL()
        OpticalLoss = 10.0 # [1/cm]
    }
}

```

33: Introduction to Lasers and Light-Emitting Diodes

Command File Syntax

```
# ----- Choose Gain broadening -----
Broadening (Type=Lorentzian Gamma=0.10)      # [eV]
# ----- Specify QW parameters -----
qwTransport
qwExtension = AutoDetect      # auto read QW widths
# ----- QW Strain effects -----
Strain
SplitOff = 0.34                # [eV]
# ----- can scale stim and spon gain independently -----
StimScaling = 1.0
SponScaling = 1.0
# ----- Specify dependency of refractive index ----
RefractiveIndex(TemperatureDep CarrierDep)
)
# ----- Specify transport physics -----
Thermionic
HeteroInterface
Mobility ( DopingDependence )
Recombination ( SRH Auger )
EffectiveIntrinsicDensity ( NoBandGapNarrowing )
Fermi
}
# ----- Material region physics for electrical problem -----
# --- Simplify the DBR layers by an equivalent bulk region ---
Physics (region="pDBR_equivalent") { MoleFraction(xfraction=0.35) }
Physics (region="nDBR_equivalent") { MoleFraction(xfraction=0.35) }
Physics (region="psch") { MoleFraction(xfraction=0.09) }
Physics (region="nsch") { MoleFraction(xfraction=0.09) }
# ----- Quantum wells -----
Physics (region="qw1") {
    MoleFraction(xfraction = 0.8 Grading=0.00)
    Active
}
Physics (region="barr") { MoleFraction(xfraction=0.09) }
Physics (region="qw2") {
    MoleFraction(xfraction = 0.8 Grading=0.00)
    Active
}

# ----- Control the numerical method in the electrical problem -----
Math {
    Digits = 7
    # ----- Need to include this to specify cylindrical symmetry -----
    Cylindrical
}
# ----- Specify plot parameters -----
GainPlot { range=(1.22,1.32) intervals=150 }
FarFieldPlot { range=(80,80) intervals=50 Scalar1D Vector2D }
```



```

VCSELNearFieldPlot { Position=0 Radius=10.0 NumberOfPoints=50 }}

# ===== End of electrical grid definition =====

# ===== Define the circuit mixed-mode system =====
System {
  # ----- Define opt1 of type optgrid -----
  optgrid opt1 ()
  # ----- Define d1 of type electricaldev, and coupled to opt1 -----
  electricaldev d1 (p_Contact=vdd n_Contact=gnd) {Physics{OptSolver="opt1"}}
  # ----- Set the initial bias voltage to circuit contacts -----
  Vsource_pset drive(vdd gnd){ dc=0.8 }
  Set ( gnd=0.0 )
}

# ===== Solver part =====
Solve {
  Poisson
  coupled { Hole Electron Poisson Contact Circuit }
  coupled { Hole Electron Poisson Contact Circuit PhotonRate }

  quasistationary (
    InitialStep = 0.001
    MaxStep = 0.01
    Minstep = 1e-7
    # ----- Plot and save various quantities at specified intervals of bias -
    -----
    Plot { range=(0,1) intervals=5 }
    PlotGain { range=(0,1) intervals=5 }
    SaveOptField { range=(0,1) intervals=3 }
    VCSELNearField { range=(0,1) intervals=3 }
    PlotFarField { range=(0,1) intervals=2 }
  # ----- Specify final voltage -----
  Goal { Parameter=drive.dc Value=1.6 })
  {
    Plugin (BreakOnFailure) {
      Coupled { Electron Hole Poisson Contact Circuit PhotonRate }
      Optics
      Wavelength
    }
  }
}

```

Compare this VCSEL example with the edge-emitting laser example in [Simulating Dual-Grid Edge-Emitting Laser on page 763](#), to highlight the syntaxes required for a VCSEL simulation. Some comments about the above example are:

- The vectorial optical solver (FEVectorial) has been used. To use the scalar solvers, you need only replace the FEVectorial section with the EffectiveIndex or TMM1D section.
- Dual-grid mixed-mode simulation is required for VCSELs regardless of whether the vectorial or scalar optical solvers are chosen.
- In the Math section for the electrical device, the keyword Cylindrical must be included so that the mesh volumes and areas in the finite box integration method are computed with cylindrical symmetry in mind.

Investigating Simulation Results

If the option `Current = "current"` is specified in the `File` section of the command file, Sentaurus Device produces the current file at the end of the simulation. The current file (`current_des.plt`) contains laser-specific result variables as a function of bias. These result variables can be plotted using `Inspect`.

Scripts to automatically extract the threshold current, the slope efficiency, and so on are available. You can refer to the TCAD library or contact the Synopsys Technical Support Center to obtain these scripts.

If the option `Plot = "plot"` is specified in the `File` section of the command file, Sentaurus Device creates the plot file at the end of the simulation. The plot file (`plot_des.tdr`) contains the plot variables that you have specified in the `Plot` statement, and these plot variables are given in TDR format on the vertices of the mesh. The plot variable names are usually meaningful to make it easier to identify what they represent.

Apart from the list of standard plot variables from a Sentaurus Device simulation (see [Appendix F on page 1185](#)), the `Laser` option introduces an additional set of plot variables.

By inputting the grid file and plot file into Tecplot SV, you can visualize the results.

In addition, there is a list of options that you can switch on to obtain specific output files. These options are discussed in the next sections.

Some options were included in the example in [Simulating Single-Grid Edge-Emitting Laser on page 760](#), so that you understand how to activate these options. A summary of these options is:

- Gain curves – Gain versus energy at different biases. If the full photon recycling feature is activated in an LED simulation, the modified spontaneous emission spectrum is also plotted.

- Band structure plots – Results of $k \cdot p$ band structure and wavefunction calculations in quantum well regions
- Far-field plots – Far-field patterns at different biases
- Optical field vector and intensity – Optical intensities at different biases
- VCSEL near field – Transverse VCSEL near field at different biases

Current File and Plot Variables for Laser Simulation

Table 123 and Table 124 on page 774 list the current file output and the plot variables for laser simulation.

Table 123 Current file for laser simulation

Dataset group	Dataset	Unit	Description
Time		1 s	For quasistationary. For transient.
n_Contact p_Contact	OuterVoltage	V	
	InnerVoltage	V	
	eCurrent	A	
	hCurrent	A	
	TotalCurrent	A	
	Charge	C	
TotalPower		W	Total optical output power of all modes.
TotalPhotons		s ⁻¹	Rate of total photon production in the active region of all modes.

33: Introduction to Lasers and Light-Emitting Diodes

Investigating Simulation Results

Table 123 Current file for laser simulation

Dataset group	Dataset	Unit	Description
Mode0-9	TotalPower	W	Total optical output power of the mode.
	PowerLeft	W	Optical output power at the left (for edge-emitting laser only).
	PowerRight	W	Optical output power at the right (for edge-emitting laser only).
	OpticalGain	$\text{cm}^{-1} \text{s}^{-1}$	Model gain at lasing wavelength: Edge-emitting laser VCSEL
	OpticalLoss	$\text{cm}^{-1} \text{s}^{-1}$	Model optical loss at lasing wavelength: Edge-emitting laser VCSEL
	TotalOutcouplingLoss	$\text{cm}^{-1} \text{s}^{-1}$	Model optical outcoupling loss (edge-emitting laser). Decay rate through top mirror (VCSEL).
	OutcouplingLossLeft	cm^{-1}	For edge-emitting laser only.
	OutcouplingLossRight	cm^{-1}	For edge-emitting laser only.
	SpontEmission	$\text{cm}^{-1} \text{s}^{-1}$	Model spontaneous emission: Edge-emitting laser VCSEL
	Wavelength	nm	Lasing wavelength.
	RefIndexChange	1	
	EffectiveIndex	1	
	OpticalConfinementFactor	1	

Table 124 Plot variables for laser simulation

Plot variable	Dataset name	Unit	Description
LaserIntensity	OpticalIntensity	Wm^{-3}	Total laser intensity for multimode lasing.
DielectricConstant		1	Dielectric profile.
RefractiveIndex		1	Refractive index profile.
MatGain	OpticalMaterialGain	m^{-1}	Local material gain.
OpticalIntensityMode0..9		Wm^{-3}	Optical intensity of mode0 to mode9.

Table 124 Plot variables for laser simulation

Plot variable	Dataset name	Unit	Description
OpticalPolarizationAngleMode0..9		rad	Polarization angle of the optical field vector (0 for TE, $\pi/2$ for TM).
eDifferentialGain hDifferentialGain		cm^2	$\partial r^{\text{st}}/\partial n, \partial r^{\text{st}}/\partial p$ fundamental mode only.
StimulatedRecombination SpontaneousRecombination		$\text{cm}^{-3}\text{s}^{-1}$	Sum of stimulated and spontaneous emission in all modes.

33: Introduction to Lasers and Light-Emitting Diodes

Investigating Simulation Results

This chapter describes the physics and the models used in laser simulations.

For laser simulations, Fabry–Perot cavity and VCSEL cavity treatments are discussed. Small-signal photon intensity and phase modulation, and relative intensity and phase noise are also described.

Overview

Simulating a laser diode is one of the most complex problems in device simulation. The fundamental set of equations used in a laser simulation is:

- Poisson equation
- Carrier continuity equations
- Lattice temperature equation and hydrodynamic equations
- Quantum well scattering equations (for QW carrier capture)
- Quantum well gain calculations (Schrödinger equation)
- Photon rate equation
- Helmholtz equation

The first three equations are semiconductor transport equations for the drift-diffusion regime (see [Chapter 8 on page 197](#)). Other than the semiconductor transport equations, which solve the electrical drift-diffusion problem for holes and electrons, the carrier capture in the quantum well (QW) and the gain calculations are specific to the laser problem, and they link the electronics and optics. The QW carrier capture and gain calculations are discussed in [Chapter 37 on page 895](#). The solution of the Helmholtz equation provides the optical modes and other optical quantities, and this is presented in [Chapter 36 on page 853](#). Finally, the ‘moderator’ between the electronics and optics is the photon rate equation, which solves for the total number of photons. In the next section, the relationship between all these equations is shown and how to achieve self-consistency between all these equations is explained.

Coupling Between Optics and Electronics

The coupling between the different equations in the laser simulation is illustrated in [Figure 58](#).

The complexity of the laser problem is apparent from this relational chart. The key quantities exchanged between different equations are placed alongside the directional flows between the equation blocks. The optical problem is separated from the electrical problem. For the optical problem, an eigenvalue equation is solved, and the eigenvector (mode intensity) and eigenvalue (effective index or mode frequency) are inserted into the electrical equations and the photon rate equation. As a result, the coupling between the optics and electronics becomes nonlocal, making it difficult to solve these problems simultaneously by the Newton method. Therefore, a Gummel iteration method (instead of a coupled Newton iteration) is required to couple the electrical and optical problems self-consistently.

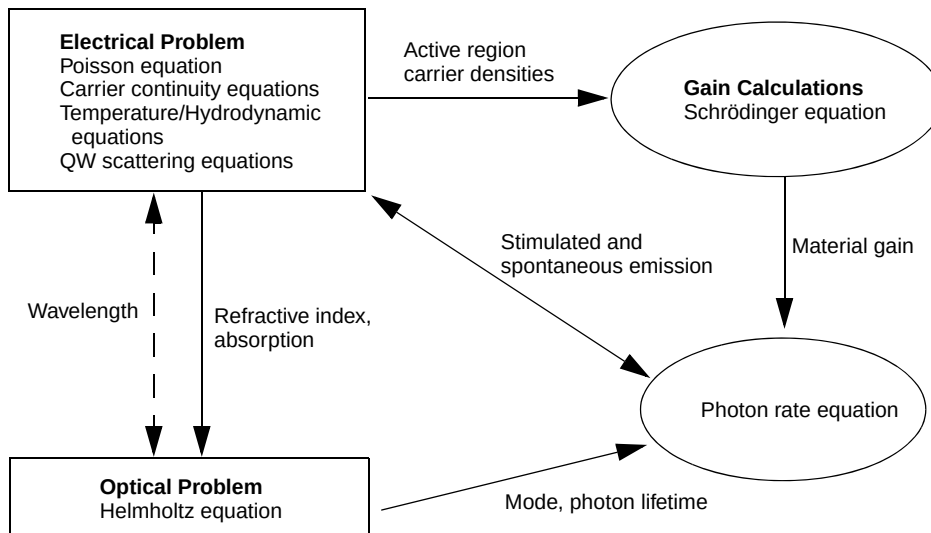


Figure 58 Coupling between the semiconductor transport and optics equations

The solution of the electrical problem provides the refractive index and absorption to the optical solver, which solves for the modes. In the case of the Fabry–Perot edge-emitting laser, the wavelength is computed from the peak of the gain curve and fed into the optical problem. In VCSELs and distributed Bragg reflector (DBR) lasers, the wavelength is computed as part of the optical resonance problem and is an input to the electrical problem instead.

The gain calculations involve the solution of the Schrödinger equation for the subband energy levels and wavefunctions. These quantities are used with the active-region carrier statistics to compute the optical matrix elements for stimulated and spontaneous emission. The formulation of these emission processes is based on Fermi’s golden rule. For more details about the gain calculations, see [Chapter 37 on page 895](#).

The optical recombination processes increase the photon population, thereby reducing the carrier population. Therefore, they are included in the photon rate equation that is used to calculate the number of photons present in the laser device and also in the carrier continuity equations to ensure the conservation of particles (see [Photon Rate and Photon Phase Equations on page 780](#)).

Algorithm for Coupling Electrical and Optical Problems

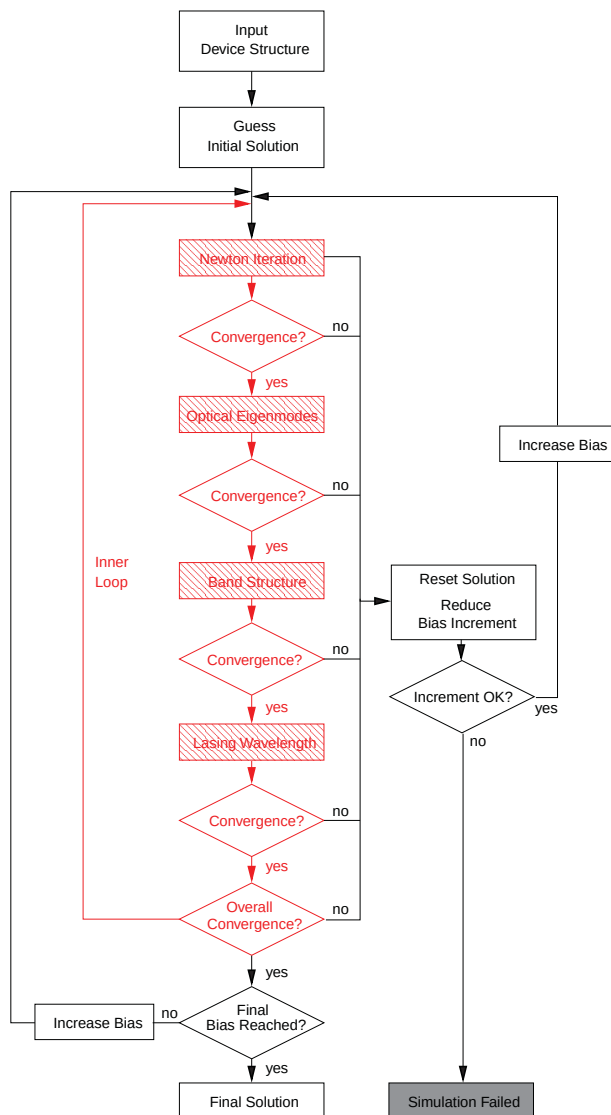


Figure 59 Algorithm flowchart for the self-consistent solution of laser equations

34: Lasers

Photon Rate and Photon Phase Equations

The Newton iteration is performed in Sentaurus Device using the `Coupled` keyword, while the Gummel iteration is activated using the keyword `Plugin`. [Figure 59 on page 779](#) shows the algorithm flowchart for the laser simulation. In addition, refer to the `Solve` section in [Simulating Single-Grid Edge-Emitting Laser on page 760](#) for details about the syntax.

The statement `Coupled {Electron Hole Poisson QwScatter QwhScatter PhotonRate}` activates the Newton iterations for the electron and hole continuity equations, the Poisson equation, the QW carrier capture equations, and the photon rate equation. This is indicated by the Newton iteration block in [Figure 59](#).

After the convergence of the Newton iterations, the updated refractive index profile (if it changes with temperature and carrier densities) is passed to the optical solver to solve for the optical eigenmodes. At this instance, the band structures are also computed. In the case of VCSELs and DBR lasers, the resonant (or lasing) wavelength is also computed by the optical solver. The keyword `Plugin` ensures that the electrical and optical solutions are iterated self-consistently, that is, a Gummel-type iteration. Through this self-consistency, all the physics should be satisfied. Sentaurus Device automatically handles the flow of the result variables between the electrical and optical solvers. There are also additional checkpoints to restrict the solution of each part from diverging during the Gummel iteration, as shown in [Figure 59](#).

When convergence is attained for the Gummel iteration (`Plugin`), the solution set for this bias is used as the initial guess for the next bias. In this way, the continuous wave operation of the laser diodes can be simulated.

Photon Rate and Photon Phase Equations

The origin of the photon rate equation and the photon phase equation can be traced to the famous work of Henry [1] on the theory of spontaneous emission noise in open resonators. The cavity of an edge-emitting Fabry–Perot laser is considered an open electromagnetic resonator, and the modes are driven by the radiative recombination processes.

From the Maxwell equations, the wave equation for the electric field is derived as:

$$\nabla \times \nabla \times \mathbf{E} = -\mu_0 \left(\sigma \frac{\partial \mathbf{E}}{\partial t} + \epsilon_0 \epsilon_r \frac{d^2 \mathbf{E}}{dt^2} + \frac{d^2 \mathbf{P}}{dt^2} \right) \quad (871)$$

where $\mathbf{E}$ is the electric field, σ is the conductivity, and ϵ_r is the relative permittivity. The source term $\mathbf{P}$ can be identified as the polarization due to the spontaneous recombination of electron–hole pairs. For the electric field, the following separation ansatz is made:

$$\mathbf{E}(x, y, z, t) = \Psi(x, y; t) \sqrt{S(t)} e^{i\varphi(t)} e^{i(\omega_0 t - \kappa_0 z)} + c.c. \quad (872)$$

where cc denotes the complex conjugate. The optical field, Ψ , is complex; while the photon density, $S(t)$, and phase factor, Φ , are real quantities. Ψ is normalized according to:

$$\iint |\Psi|^2 dx dy = 1 \quad (873)$$

Inserting Eq. 872 into Eq. 871 results in three equations:

(i) The Helmholtz equation for $\Psi(x, y)$, which is discussed in [Chapter 37 on page 895](#):

$$\left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right) \Psi + k_0^2 (n^2(x, y) - \epsilon_{eff}) \Psi = 0 \quad (874)$$

(ii) The equation for the photon phase $\Phi(t)$:

$$\frac{\partial \Phi}{\partial t} - [G(\omega) - L] \frac{\left(\frac{c}{n}\right) \alpha_v}{2} = 0 \quad (875)$$

where $G(\omega)$ is the modal gain, L is the optical loss, and the line-width enhancement factor α_v must be entered by you (using the keywords `PhotonPhaseAlpha` and `LinewidthEnhancementFactor`).

(iii) The photon rate equation for the photon number $S(t)$:

$$\frac{\partial S}{\partial t} - (G(\omega) - L)S = \frac{c}{\epsilon_r} \beta T^{sp}(\omega) \quad (876)$$

The photon rate equation contains the modal spontaneous emission, $T^{sp}(\omega)$, and the spontaneous emission factor β that can be defined by you (using the keyword `SponEmiss`) and that has a default value of 1.

The parameters are defined as:

$$G(\omega) = \iint r^{st}(x, y, E_\omega) |\Psi(x, y)|^2 dx dy \quad (877)$$

$$T^{sp}(\omega) = \iint r^{sp}(x, y, E_\omega) |\Psi(x, y)|^2 dx dy \quad (878)$$

$$L = \left(\frac{c}{n}\right) \left\{ \iint \alpha(x, y) |\Psi(x, y)|^2 dx dy + L_{bg} + L_{cavity} + L_{waveguide} \right\} \quad (879)$$

34: Lasers

Photon Rate and Photon Phase Equations

The stimulated emission coefficient $r^{\text{st}}(x, y, E_\omega)$ and the spontaneous emission coefficient $r^{\text{sp}}(x, y, E_\omega)$ are computed locally at each active vertex from Fermi's golden rule, and their values are taken at the lasing energy E_ω . These radiative emission coefficients are discussed in [Chapter 37 on page 895](#). The local optical intensity $|\Psi(x, y)|^2$ is solved from the Helmholtz equation. In the simulation, the spatial integrations are performed in the active regions of the laser only.

The total optical loss L contains the loss due to local free carrier absorption $\alpha(x, y)$, the cavity loss L_{cavity} , the background optical loss L_{bg} , and the waveguide loss $L_{\text{waveguide}}$. The free carrier loss is described in detail in [Free Carrier Loss on page 792](#). You specify the background loss. The waveguide loss (imaginary part of propagation constant) is fed back from the optical solver to the photon rate equation, and this includes the losses due to leakage waves from the waveguide.

The cavity loss for a Fabry–Perot cavity mainly contains the mirror loss and is given by:

$$L_{\text{cavity}} = \frac{1}{2l} \ln\left(\frac{1}{r_0 r_1}\right) \cdot \frac{c}{n} \quad (880)$$

where l is the cavity length, r_0 and r_1 are the facet power reflectivities, c is the speed of light in a vacuum, and n is modal effective refractive index.

This formula is used mainly in the simulation of edge-emitting lasers. For VCSELs, the cavity loss is more complicated due to scattering and diffraction effects, and can only be accurately computed by a vectorial cavity solution. In this case, the net loss in the VCSEL photon rate equation, $(L - G(\omega))$, must be solved from the optics equation when gain-guiding effects are included. This is discussed more fully in [Cavity Optical Modes in VCSELs on page 801](#).

Each optical mode corresponds to a distinct photon rate equation. For example, if you specify up to ten modes in a laser simulation, Sentaurus Device automatically solves up to ten photon rate equations. The stimulated and spontaneous emissions from each photon rate equation are then summed to give a total radiative emission rate. This total rate is then added to the carrier continuity equation as a radiative recombination to ensure the conservation of electron–hole pair recombination and photon emission.

At lasing threshold, the modal gain of the laser saturates to the value of the optical losses, which results in a singularity in the steady-state photon rate:

$$S = \left(\frac{\bar{\epsilon}_r}{c}\right) \cdot \frac{\beta T^{\text{sp}}(\omega)}{L - G(\omega)} \quad (881)$$

A small fluctuation of the QW carrier densities and, therefore, gain during Newton iterations leads to large changes in the photon rate. In cases where the gain exceeds the loss, the photon rate suddenly becomes negative, which is nonphysical and pushes the entire Newton iteration towards divergence. To solve this problem, a slack variable, λ_{slack} , is introduced to constrain $(L - G(\omega))$ to be always positive:

$$\lambda_{\text{slack}}^2 = L - G(\omega) \quad (882)$$

so that the photon rate will always be positive. This important numeric technique to laser simulation was first proposed by Smith *et al.* [2] and, in Sentaurus Device, it is called the photon stabilization equation. When the photon rate equation is activated, Sentaurus Device automatically activates the photon stabilization equation as well.

The syntax for specifying the various parameters in the photon rate and photon phase equations can be found in [Table 152 on page 1229](#) and [Table 212 on page 1284](#).

Laser Small-Signal AC Analysis and Modulation Response

Small-signal AC analysis for the laser intensity and photon phase is analogous to the small-signal AC analysis described in [Optical AC Analysis on page 130](#).

Intensity and Photon Phase Modulation Response

The dynamic laser device properties under current modulation can be calculated from a converged static solution of the system of [Eq. 37, p. 191](#), [Eq. 53, p. 197](#), [Eq. 68, p. 208](#), [Eq. 875](#), and [Eq. 888](#) using a small-signal expansion. The intensity modulation response can then be described as the change in the modal photon number S_v of a mode v caused by a change in the injection current I :

$$\partial S_v(\omega) = \iiint G_{\partial I}^{S_v}(r, \omega) \partial I(r, \omega) dV \quad (883)$$

where $G_{\partial I}^{S_v}$ is the Green's function of the photon number S_v due to a current excitation. Analogously, the modulation response of the photon phase is given by:

$$\partial \Phi_v(\omega) = \iiint G_{\partial I}^{\Phi_v}(r, \omega) \partial I(r, \omega) dV \quad (884)$$

The Green's functions are obtained by inverting the Fourier-transformed Jacobian matrix of the converged electro-optothermal system of [Eq. 37](#), [Eq. 53](#), [Eq. 68](#), [Eq. 875](#), and [Eq. 888](#).

Syntax for Small-Signal AC Extraction

The syntax for small-signal AC analysis for lasers is analogous to the syntax for other devices as described in [Small-Signal AC Analysis on page 127](#). First, the device must be ramped up to its operating point, before the AC analysis can be performed:

```
Solve { ...
    # Ramp up to operating point:
    Quasistationary (
        InitialStep = 0.01
        MaxStep = 0.05
        MinStep = 1e-5
        Goal { Parameter=i_drive.dc value = -5e-3 } )
    { Coupled{ Poisson Hole Electron Contact Circuit PhotonRate }
    }

    # Perform chirp analysis:
    ACCoupled (
        StartFrequency = 1e8 EndFrequency = 1e12
        ACExtract = "ac_chirp"
        NumberOfPoints = 100
        Decade
        Node(nanode)
        Exclude(i_drive)
        PhotonicAC)
    { Poisson Hole Electron Contact Circuit PhotonRate PhotonPhase }
}
```

For a detailed description of the arguments of the ACCoupled statement, see [Small-Signal AC Analysis on page 127](#) and [Table 148 on page 1225](#).

NOTE A small-signal analysis can also be performed for the calculation of the relative intensity noise and the frequency noise of a laser (see [Modeling Relative Intensity Noise and Frequency Noise on page 785](#)).

Modeling Relative Intensity Noise and Frequency Noise

Sentaurus Device contains a spatially distributed noise model for the simulation of relative intensity noise (RIN) and frequency noise (FN) in semiconductor lasers. The model is based on spatially distributed Langevin forces and correlation functions that are formulated in the frequency domain assuming small-signal noise sources.

Relative Intensity Noise and Frequency Noise

To model noise in lasers, the continuity equations (Eq. 53, p. 197) and the photon phase and photon rate equations (Eq. 875 and Eq. 876) are extended by appropriate local Langevin noise forces $F_n(t)$, $F_p(t)$, $F_\Phi(t)$, and $F_S(t)$ for electrons, holes, photon phase, and photon rate, respectively:

$$\nabla \cdot \vec{J}_n = qR + q\frac{\partial n}{\partial t} + F_n(t) \quad -\nabla \cdot \vec{J}_p = qR + q\frac{\partial p}{\partial t} + F_p(t) \quad (885)$$

$$\frac{\partial \Phi}{\partial t} - (G(\omega) - L)\frac{\left(\frac{c}{n}\right)\alpha_v}{2} = F_\Phi(t) \quad (886)$$

$$\frac{\partial S}{\partial t} - (G(\omega) - L)S = \frac{c}{\epsilon_r}\beta T^{\text{sp}}(\omega) + F_S(t) \quad (887)$$

Relative intensity noise in a laser is defined as a function of frequency f according to:

$$\frac{\text{RIN}(f)}{\Delta f} = \text{Re} \left(\frac{\sum_{\mu, \nu} S_{S_\nu S_\mu}(f)}{\sum_{\nu} S_\nu^2} \right) \quad (888)$$

where $S_{S_\nu S_\mu}$ denotes the correlation function between the photon numbers S_ν and S_μ of modes ν and μ .

The line width of mode ν is defined as:

$$\Delta f = \lim_{f \rightarrow 0} S_{S_\nu S_\nu}(f) \quad (889)$$

where $S_{S_\nu S_\nu}$ is the autocorrelation function of the lasing frequency of mode ν , which can be interpreted as frequency noise.

34: Lasers

Modeling Relative Intensity Noise and Frequency Noise

This is related to the photon phase autocorrelation function $S_{\Phi_v \Phi_v}$ by:

$$S_{S_v S_v}(f) = f^2 S_{\Phi_v \Phi_v}(f) \quad (890)$$

Therefore, RIN and line width can be calculated through the correlation functions of the photon number and photon phase by either explicit calculations in the time domain or assuming small-signal noise sources. As the time domain approach is computationally very expensive, Sentaurus Device uses the small-signal approach.

Small-Signal Noise Modeling

For the computation of RIN, the correlation functions of the modal photon numbers $S_{S_v S_\mu}$ must be evaluated:

$$S_{S_v S_\mu}(\omega) = \sum_{\gamma} \sum_{\delta} \iiint \hat{G}_{\gamma}^{S_v}(r_1, \omega) K_{\gamma, \delta}(r_1, \omega) \hat{G}_{\delta}^{S_v*}(r_1, \omega) dr_1 \quad (891)$$

where the summations are over the other system variables (potential, densities, photon number, and photon phase). $\hat{G}_{\gamma}^{S_v}(r_1; \omega)$ is the Green's function of the photon number S_v caused by an excitation with quantity γ at a coordinate r_1 with an angular frequency ω and * denotes the complex conjugate.

$K_{\gamma, \delta}(r_1, \omega)$ denotes the noise source correlation between the quantities γ and δ at r_1 . The Green's function can be obtained from the Fourier-transformed Jacobian matrix of the system of Eq. 37, p. 191, Eq. 68, p. 208, Eq. 885, Eq. 886, and Eq. 887. According to Eq. 891, frequency noise for mode v can be calculated using:

$$S_{\Phi_v \Phi_\mu}(\omega) = \sum_{\gamma} \sum_{\delta} \iiint \hat{G}_{\gamma}^{\Phi_v}(r_1, \omega) K_{\gamma, \delta}(r_1, \omega) \hat{G}_{\delta}^{\Phi_v*}(r_1, \omega) dr_1 \quad (892)$$

The local noise sources $K_{\gamma, \delta}(r_1, \omega)$ are given by:

$$K_{\gamma, \delta} = \int \langle F_{\gamma}(t) F_{\delta}^*(t - \tau) \rangle \exp(-i\omega\tau) d\tau \quad (893)$$

where the angle brackets denote the expectation value.

Sentaurus Device includes only the autocorrelation functions for the photon numbers and the photon phases. The terms for the $K_{\gamma, \delta}(r_1, \omega)$ are given by:

$$K_{S_v, S_v} = 2T_v^{sp} S_v \quad K_{\Phi_v, \Phi_v} = \frac{T_v^{sp}}{2S_v} \quad (894)$$

Syntax for Relative Intensity Noise and Frequency Noise Model

The noise model described here can be used with the small-signal AC analysis described in [Laser Small-Signal AC Analysis and Modulation Response on page 783](#) to obtain the frequency-resolved relative intensity noise (RIN) and frequency noise (FN) of a laser device.

The syntax to activate the noise sources described in [Eq. 894](#) is:

```
Physics {
    Noise(PhotonNumberNoise PhotonPhaseNoise)
}
```

where PhotonNumberNoise and PhotonPhaseNoise denote K_{S_v, S_v} and K_{Φ_v, Φ_v} , respectively.

The syntax for ACCoupled differs to the syntax described in [Syntax for Small-Signal AC Extraction on page 784](#) only in the missing Node keyword, as there are no terminals for which the analysis is performed.

The syntax for RIN is:

```
ACCoupled(
    StartFrequency = 1e8 EndFrequency = 1e12
    ACExtract = "int_noise"
    NumberOfPoints = 100
    Decade
    Exclude(i_drive)
    PhotonicIntensityNoise)
{ Poisson Hole Electron Contact Circuit PhotonRate PhotonPhase }
```

For frequency noise, the syntax is:

```
ACCoupled(
    StartFrequency = 1e8 EndFrequency = 1e12
    ACExtract = "int_noise"
    NumberOfPoints = 100
    Decade
    Exclude(i_drive)
    PhotonicPhaseNoise)
{ Poisson Hole Electron Contact Circuit PhotonRate PhotonPhase }
```

For more information about the syntax of the ACCoupled statement, see [Small-Signal AC Analysis on page 127](#) and [Table 148 on page 1225](#).

Plotting the Laser Power Spectrum

The power spectrum of a semiconductor laser can be calculated to a good approximation according to:

$$P_{\text{out}}(\hbar\omega) = \sum_v \hbar\omega_v \cdot L_{v, \text{out}} \cdot S_v \cdot \frac{(\Delta\omega_v/2)^2}{(\omega - \omega_v)^2 + (\Delta\omega_v/2)^2} \quad (895)$$

where ω_v is the lasing frequency, $\hbar\omega_v$ is the lasing energy of mode v , $L_{v, \text{out}}$ is the power out-coupling factor of the mode, and S_v is the number of photons in the mode. $\Delta\omega_v$ is the FWHM Lorentzian broadening of mode v , which is given by the expression:

$$\Delta\omega_v = \frac{(1 + \alpha_H)^2}{2S_v} \cdot R_v^{\text{sp}} \quad (896)$$

with the line width enhancement factor α_H and the total spontaneous emission rate R_v^{sp} into mode v . As power spectrum plots are usually used to investigate detuning effects, the plotting of such spectra is coupled to the plotting of modal gain spectra. The plotting of laser spectra is activated by adding the keyword `PlotLasingSpectrum` to the `GainPlot` section of the command file. The file names of power spectra plots are derived from the base name for gain plots:

```
File {...
  Gain = "gn"
}
GainPlot{ Range=(1.22,1.32)  # energy range [eV]
          Intervals=120      # discretization of energy range
          PlotLasingSpectrum } # also plot power spectrum
...
Solve {...
  Quasistationary (...
    # ----- Specify plot gain parameters -----
    PlotGain {Range=(0,1) Intervals=3}
    ...
    {...}
  }
}
```

This syntax example results in the modal gain and spontaneous emission being plotted into the files `gn_gain_000000_des.plt`, ..., `gn_gain_000003_des.plt`, as well as the laser power spectra being plotted into `gn_powspec_000000_des.plt`, ..., `gn_powspec_000003_des.plt`. For details about the syntax of this example, see [Modal Gain as Function of Energy/Wavelength on page 920](#).

Refractive Index, Dispersion, and Optical Loss

The refractive index (dielectric constant) is a function of temperature, carrier density, and wavelength. The temperature dependence is assumed to be linear and the change in refractive index due to carriers is attributed to the free carrier plasma effect [3].

You can specify the refractive index dependence on temperature and carrier density with the keywords `TemperatureDep` and `CarrierDep` in the `Physics-Laser` section of the command file:

```
Physics {...
  Laser (...
    RefractiveIndex(TemperatureDep CarrierDep)
  )
}
```

You can select either or both type of dependence. If none is chosen (that is, there are no keywords), Sentaurus Device assumes that the refractive index is a constant value.

NOTE The refractive index and its associated temperature parameters for each material region can be changed by you in the parameter file.

Temperature Dependence of Refractive Index

The temperature dependence of the refractive index follows the relation:

$$n(T) = n \cdot (1 + \alpha(T - T_{\text{par}})) \quad (897)$$

and the coefficients can be changed in the parameter file:

```
RefractiveIndex
{ * Optical Refractive Index
  * refractiveindex() = refractiveindex * (1 + alpha * (T-Tpar))
    Tpar = 3.0000e+02          # [K]
    refractiveindex = 3.60e+00
    alpha = 0.0000e-04         # [1/K]
}
```

Carrier-Density Dependence of Refractive Index

The change in refractive index due to free carriers [3] is:

$$\Delta n = -\text{CarrDepCoeff} \times \frac{e^2 \lambda^2}{8\pi^2 c^2 \epsilon_0 n_g} \left(\frac{n}{m_e} + \frac{p}{m_h} \right) \quad (898)$$

where CarrDepCoeff is introduced as a tuning parameter and has a default value of 1. λ is the lasing wavelength and n_g is the refractive index. n_g can also change if it is specified as TemperatureDep.

In the case of QW carriers, the heavy-hole mass is modified to become [3]:

$$m_h = \frac{m_{hh}^{1.5} + m_{lh}^{1.5}}{\sqrt{m_{hh}} + \sqrt{m_{lh}}} \quad (899)$$

The activating syntax in the command file is:

```
Physics {...
  Laser (...
    Optics (...)
      RefractiveIndex(CarrierDep)          # use lasing wavelength of mode0
#     RefractiveIndex(CarrierDep(LasingWavelength=777.77))
                                           # fixed wavelength [nm]
    )
  }
```

There is an option to fix a LasingWavelength if required. Otherwise, Sentaurus Device automatically uses the lasing wavelength that it computes in the simulation. The tuning parameter CarrDepCoeff is defined for each region and can be input in the parameter file:

```
RefractiveIndex
{ * Optical Refractive Index
  refractiveindex = 3.893 # [1]
  * Mole fraction dependent model.
  * If just above parameters are specified, then its values will be
  * used for any mole fraction instead of an interpolation below.
  * The linear interpolation is used on interval [0,1].
  refractiveindex(1) = 3.51 # [1]

  * Tune the region-wise carrier dependence (plasma effect)
  *
  * del_n = - CarrDepCoeff * ( e^2 * lambda^2 * ( n / m_e + p / m_h ) ) /
  * ( 8 * pi^2 * c^2 * epsilon0 * n_refr )
  * Default of CarrDepCoeff is 1.
```

```
CarrDepCoeff = 1.0      # [1]
CarrDepCoeff(0) = 0.5
CarrDepCoeff(1) = 1.0
}
```

If CarrDepCoeff is not entered in the parameter file, it is assumed to be the default value of 1 during the simulation. CarrDepCoeff is a mole fraction–dependent parameter, so it works the same way as the other mole fraction–dependent parameters.

Wavelength Dependence and Absorption of Refractive Index

Wavelength-dependent optical absorption can also be included by using the TableODB section in the parameter file:

```
Physics {...
  Laser (...
    Optics (...
      FEVectorial (...
        Absorption(ODB)
      )
    )
  )
}
```

This feature is also available for the other optical solvers such as FEScalar, TMM1D, and EffectiveIndex.

In this case, you must create a corresponding ODB table for each material region in the parameter file, for example:

```
TableODB
{ * WAVELEN(um) n k
  0.5904 3.940 0.240
  0.6199 3.878 0.211
  0.6526 3.826 0.179
  0.6888 3.785 0.151
  0.7293 3.742 0.112
  0.7749 3.700 0.091
  0.8266 3.666 0.080
  0.8856 3.614 0.0017
  0.9184 3.569 0.0
  1.0332 3.492 0.0
  1.1271 3.455 0.0
  1.2399 3.423 0.0
  1.3776 3.397 0.0
}
```

34: Lasers

Free Carrier Loss

```
1.5498 3.374 0.0
1.7712 3.354 0.0
2.0664 3.338 0.0
2.4797 3.324 0.0
}
```

Free Carrier Loss

The free carrier loss is caused by plasma-induced effects and contributes to the total optical loss as shown in [Eq. 879](#). The free carrier loss can be modeled by:

$$L_{\text{carr}} = \iint (\alpha_n n + \alpha_p p) |\Psi|^2 dV \quad (900)$$

where the loss is linearly dependent on the electron and hole carrier densities. This model is activated by the keyword `FreeCarr` in the `Physics-Laser` section of the command file:

```
Plot {...
    eFreeCarrierAbsorption
    hFreeCarrierAbsorption
}
...
Physics {...
    Laser (...
        Optics (...
            FEVectorial (...
                )
            )
        FreeCarr
    )
}
```

In this example, `eFreeCarrierAbsorption` and `hFreeCarrierAbsorption` in the `Plot` section enable the electron and hole contributions to the free carrier loss to be plotted. The coefficients for the free carrier loss, α_n and α_p , for each material region are specified in the parameter file:

```
FreeCarrierAbsorption
{
    * Coefficients for free carrier absorption:
    * alpha_n for electrons,
    * alpha_p for holes

    * FCA = (alpha_n * n + alpha_p * p) * Light Intensity
    * Mole fraction dependent model.
    * If only constant parameters are specified, those values will be
    * used for any mole fraction instead of the interpolation below.
```

```
* Linear interpolation is used on the interval [0,1].
  fcaalpha_n(0) = 4.0000e-18 # [cm^2]
  fcaalpha_n(1) = 5.0000e-18 # [cm^2]
  fcaalpha_p(0) = 8.0000e-18 # [cm^2]
  fcaalpha_p(1) = 9.0000e-18 # [cm^2]
}
```

Edge-Emitting Lasers

The 2D edge-emitting laser structure is essentially a waveguide problem (see [Waveguide Optical Modes and Fabry–Perot Cavity](#)) in the transverse plane, with the cavity resonance occurring in the longitudinal direction. The main type of cavity resonance in the longitudinal direction used is the Fabry–Perot type where the resonant wavelength is chosen as the location of the gain peak. There is also an option to activate a simple distributed feedback (DFB) laser simulation. In addition, multiple transverse modes or multiple longitudinal modes can be simulated.

Two choices of optical mode solver are available for edge-emitting lasers: FEScalar and FEVectorial. These solvers have been described in [Syntax of FE Scalar and FE Vectorial Optical Solvers on page 855](#).

Waveguide Optical Modes and Fabry–Perot Cavity

In an edge-emitting laser with a Fabry–Perot type cavity having high reflecting facets, the optical mode is invariant in the longitudinal direction. In this case, the optical propagation in the longitudinal z -direction can always be described by forward and backward traveling waves with characteristic $\exp(\pm i\beta z)$, where β is the longitudinal propagation constant.

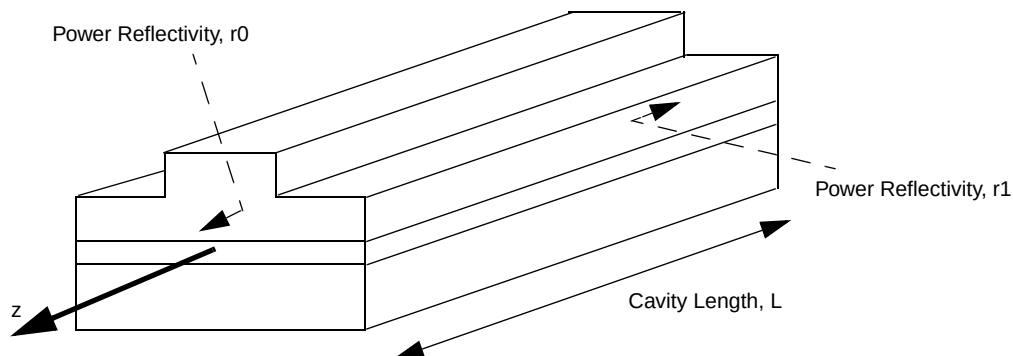


Figure 60 Waveguide problem in Fabry–Perot cavity assuming longitudinal invariance in z -direction

Therefore, the vector wave equation reduces to the Helmholtz equation:

$$(\nabla_{xy} \times \nabla_{xy} \times - \nabla_{xy}(\epsilon^{-1} \nabla_{xy} \cdot \epsilon) + (\omega^2 \mu_0 \epsilon(x, y) - \beta^2)) \cdot \mathbf{E}_t(x, y) = \mathbf{0} \quad (901)$$

where $\mathbf{E}_t(x, y)$ is the transverse vectorial electric field, ω is the angular frequency, ϵ is the permittivity, β is the complex propagation constant, and ∇_{xy} is the transverse gradient operator. The optical eigenmodes are the waveguide modes, which are characterized by the mode profile and propagation constant, β .

In a weakly guiding waveguide where the term caused by the refractive index inhomogeneity, $\nabla_{xy}(\epsilon^{-1} \nabla_{xy} \cdot \epsilon) \mathbf{E}_t(x, y)$, is small, the vector Helmholtz equation reduces to its scalar form:

$$\left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right) \Psi + k_0^2 (n^2(x, y) - \epsilon_{eff}) \Psi = 0 \quad (902)$$

where the scalar wavefunction $\Psi(x, y)$ describes either the TE-polarized or TM-polarized optical field component, ϵ_{eff} is the effective dielectric constant, $n(x, y)$ is the refractive index profile, and k_0 is the wavenumber of the mode. Sentaurus Device solves the vectorial and scalar Helmholtz equations by the finite-element method [4].

In the waveguide problem, the input parameters are:

- Wavelength (encompassed in $\lambda = (2\pi c / \omega) = 2\pi / k_0$)
- Refractive index profile $n(x, y)$
- Boundary conditions

The output is:

- Optical field profile
- Effective index (real part of propagation constant, $\text{Re}(\epsilon_{eff}) = n_{eff} \cdot \frac{2\pi}{\lambda}$)
- Net optical loss (imaginary part of propagation constant)

The optical intensity is the square of the optical field, and it is normalized for use in the photon rate equation to compute the modal gain, spontaneous emission, and absorption loss (see Eq. 877–Eq. 879, p. 781).

NOTE The imaginary part of the propagation constant is only included in the photon rate equation if the keyword `WaveguideLoss` is specified in the `Laser` statement.

Lasing Wavelength in Fabry–Perot Cavity

In a Fabry–Perot edge-emitting laser cavity, the wavelength is computed in the electronic solution and then passed to the Helmholtz equation solver. The physics is briefly outlined here. A Fabry–Perot cavity has eigenfrequencies:

$$\omega_p = \frac{\pi c}{\sqrt{\epsilon_r} \cdot L} p \quad (903)$$

where p is an integer containing the number of wavelengths that fit into the longitudinal cavity and L is the cavity length. In a Fabry–Perot cavity (typically a few millimeters), p is usually a very large number and the longitudinal mode spacing is very narrow (a few nanometers). The modal gain has a bandwidth of the order of 10 nm. Therefore, the lasing wavelength is computed as the cavity mode that has maximum modal gain $G(\omega)$ by varying p . Changing the wavelength changes the radiative recombination in the continuity equations. The default setting updates the lasing wavelength after each Newton iteration. This is a good approximation as long as the coupling between the electronic properties and the choice of wavelength is weak.

Alternatively, a self-consistent coupling is possible by including the keyword `WaveLength` in the `Plugin` statement:

```
Solve {...
  quasistationary (...
    Goal {name="p_Contact" voltage=1.6})
  {
    Plugin(BreakOnFailure){
      Coupled {Electron Hole Poisson PhotonRate}
      WaveLength
    }
  }
}
```

Output Power from a Fabry–Perot Laser

The output power at the facet 0 of the Fabry–Perot cavity is obtained by integrating the power flow (Poynting vector) in the z -direction over the facet surface [1]. This gives:

$$P_0 = S \frac{\hbar \omega c}{2 \sqrt{\epsilon_r} L} \ln \left(\frac{1}{r_0 r_1} \right) \frac{\sqrt{r_1} (1 - r_0)}{(\sqrt{r_0} + \sqrt{r_1}) (1 - \sqrt{r_0 r_1})} \quad (904)$$

The power output from the two facets is given in the current file at the end of the simulation. The keywords for specifying the cavity length and facet power reflectivities are in [Table 212 on page 1284](#).

Symmetry Considerations

Imposing symmetry of the device in laser simulations requires the treatment of symmetry in both the optical and electrical parts of the problem. In particular, symmetry changes the boundary conditions for the optical solvers and must be handled carefully. See [Chapter 36 on page 853](#) for a detailed discussion about symmetry in the optics.

Conversely, symmetry in the electrical part of the problem is simpler. With the finite box method used in the simulation, the Neumann boundary condition is implicitly imposed at all boundaries that are not defined as the contacts. Therefore, there is no need for you to set the boundary specifically for symmetric structures.

However, an additional area factor of 2 must be specified in the Physics section:

```
Physics {...  
    AreaFactor = 2  
    Laser (...  
        Optics(  
            FEVectorial(...  
                Symmetry = Symmetric  
            )  
        )  
    )  
}
```

This adjusts the total electrode area and the optical output power, both of which double in the symmetric simulation mode.

Multiple Transverse Modes

A maximum of ten transverse modes are allowed. Different transverse modes have different spatial distribution of the optical density. Therefore, each mode experiences different modal gains, giving rise to different stimulated gain spectra. Assuming a Fabry–Perot cavity, the wavelength of each mode is given by the location of the peaks of the corresponding gain spectrum. In a DFB simulation, the wavelength is set and the modal gain of each mode is taken as the value of its respective gain spectrum at this wavelength.

Depending on the current flow distribution, the carriers available to contribute to the material gain of each mode are determined by carrier transport. The different distributions of spatial

optical intensity of different modes mean that the carriers are depleted (by stimulation recombination) at different locations, and this is commonly called the spatial hole-burning effect. You can view this effect by plotting the carrier density distribution in the active region of the device.

Traverse mode calculation is activated by specifying the keyword `TransverseMode`, which is the default and can be omitted.

Multiple Longitudinal Modes

The longitudinal mode simulation is only possible with Fabry–Perot-type cavities. If the keyword `LongitudinalModes` is set in the `Physics-Laser` section of the command file, a multilongitudinal mode calculation is performed. Sentaurus Device automatically selects an odd number of modes, which ensures one central (or main) mode and an equal number of modes, smaller and greater than the main mode frequency. The frequency of the main mode is determined as in the single-mode case by calculating the Fabry–Perot mode with maximum mode gain.

The spacing of the frequencies of the side modes is calculated according to the resonant frequencies in the Fabry–Perot cavity, which is in turn defined by the cavity length (keyword `CavityLength` in the `Physics-Laser` section). There is only one transverse mode associated with the multiple longitudinal modes.

For 2D simulations, this means that all the longitudinal modes share the same modal gain spectrum and, hence, the same carrier population for stimulated recombination. The multiple longitudinal-mode simulation is important for analyzing the suppression ratio of the side mode of the laser.

NOTE The longitudinal mode option cannot be used with the DFB option.

Sentaurus Device offers two choices of multimode simulation: transverse and longitudinal modes. Such a simulation can be performed by setting the keyword `Modenumber=<int>` to greater than one, and specifying one of the keywords `TransverseModes` (which is the default and can be omitted) or `LongitudinalModes` in the `Physics-Laser` section of the command file:

```
Physics {...
  Laser (...
    Optics (
      FEVectorial (...)
    )
    # --- Choose transverse or longitudinal modes, but NOT both ---
    TransverseModes
  #   LongitudinalModes
```

```
        ModeNumber = 5          # can choose up to 10 modes
        ...
    )
}
```

For each longitudinal or transverse mode, one photon rate equation is solved.

NOTE You can select either multiple transverse modes or multiple longitudinal modes simulation, but not both simultaneously.

Specifying Fixed Optical Confinement Factor

The Helmholtz equation is not solved if the keyword `optconfin=<float>` is specified:

```
Physics {...
  Laser (...
    Optics (...
      optconfin = 0.9
    )
  )
}
```

In this case, the spatial optical intensity Ψ is assumed to be constant over the cavity, that is, the local mode gain and spontaneous emission at each active vertex are multiplied by the constant `optconfin=<float>` factor.

Simple Distributed Feedback Model

A simple distributed feedback (DFB) laser simulation can be performed by setting the keyword `DFB` in the Physics-Laser section of the command file:

```
Physics {...
  Laser (...
    Optics (
      FEVectorial (...)
    )
    # --- Specify DFB simulation
    TransverseModes
    DFB (DFBPeriod=0.11)      # [micrometers]
    CavityLength = 200       # [micrometers]
    lFacetReflectivity = 0.3
  )
}
```

```

    rFacetReflectivity = 0.3
    GroupRefract = 3.4      # fix effective index
    ...
)
}

```

The argument `DFBPeriod=<float>` specifies the period of the DFB grating, Δ , in units of μm . As a result, the lasing wavelength is assumed to occur at the Bragg wavelength, $\lambda_B = 2n_{\text{eff}}\Delta$, where n_{eff} is the effective refractive index. The effective index is solved from the Helmholtz equation using either the scalar (`FEScalar`) or vectorial (`FEVectorial`) optical solvers and can change if the refractive index profile changes in the simulation. The wave propagation in the longitudinal DFB grating, however, is not simulated. You can also set a fixed constant effective index with the keyword `GroupRefract=<float>`.

Bulk Active-Region Edge-Emitting Lasers

Bulk active-region lasers can also be simulated. Similar to the case of the quantum well laser, you must specify the active material region by the keyword `Active`. In the `Physics-Laser` section of the command file, all the quantum well-related keywords should be removed: `QWTransport`, `QWExtension`, `QWScatModel`, `QWScatTime`, `QWhScatTime`, `Strain`, and `SplitOff`. In this case, Sentaurus Device treats the active region as a bulk region, and the stimulated gain is computed as a bulk material gain. The carriers are assumed to scatter into the bulk active region by thermionic emission.

Leaky Waveguide Lasers

In many laser designs, the refractive index of the substrate or top layers is near the value of the guiding layer and is higher than that of the cladding layers. In such a case, any waves penetrating into the substrate or top layer will radiate outwards contributing to additional losses of the mode. Such leakage can also be used to discriminate higher order modes (which have higher losses) from the fundamental mode so that single-mode high-power laser diodes can be designed.

The `FEVectorial` optical solver coupled with PML (see [Perfectly Matched Layers on page 865](#)) makes Sentaurus Device an ideal tool to simulate such leaky waveguide lasers. In fact, Sentaurus Device has been successfully used to optimize leaky waveguide lasers for single-mode high-power operations [5].

Device Physics and Parameter Tuning

There are many quantities that you may want to optimize for the best laser diode performance or may want to tune in order to match experimental results. A selected list of these quantities and ways to tune them are:

Optical mode shape

The shape is controlled by the geometry and refractive index of the device. Sentaurus Workbench provides an automatic parameterization option for you to optimize geometric feature sizes, layer thickness, refractive index profile, and so on for the required mode shape.

Discretization of optical grid

The optical solvers use the finite-element method and a general rule is to use 20 points per wavelength to generate the mesh. This should provide an accurate solution for the optical mode.

Lasing threshold current

The threshold current is a function of the dark recombinations (Auger and SRH), optical losses, gain spectrum, and radiative recombination. The SRH lifetimes and Auger coefficients can be changed in the parameter file. Additional background optical loss can be input by `OpticalLoss=<float>` in the Physics-Laser section of the command file. The stimulated and spontaneous gain spectrum can be scaled by the keywords `StimScaling=<float>` and `SponScaling=<float>` in the Physics-Laser section of the command file.

Slope efficiency of L–I curve

This is primarily determined by the injection efficiency and the photon lifetime, which can be changed by the optical losses. However, changing the optical losses also affects the threshold current.

Temperature profile

The temperature distribution is sensitive to the boundary conditions specified at the thermodes. You can change the `SurfaceResistance` in the Thermode section of the command file to tune the temperature profile (see [Performing Temperature Simulation on page 923](#)).

Thermal rollover

The rollover is caused by self-heating effects of the laser diode. Possible major causes are Auger recombination and increased quantum well current leakage.

Vertical-Cavity Surface-Emitting Lasers

Vertical-cavity surface-emitting lasers (VCSELs) are difficult devices to simulate due to their many layers (usually more than 100 layers). A full 3D simulation of a VCSEL is not feasible because it requires an excessive amount of computing resources.

To reduce the computational load, cylindrical symmetry is assumed so that the VCSEL problem is reduced to a quasi-2.5-dimensional problem. Each mesh on the (ρ, z) plane represents a solid ring rotated around the z -axis. As a result of such symmetry, the optical field can be expanded in cylindrical harmonics, and this further helps to reduce the size of the problem (see [Cavity Optical Modes in VCSELs](#)).

Three different types of optical solver are available to solve for the cavity modes of a VCSEL. There is one vectorial solver (`FEVectorial`) and two scalar solvers (`EffectiveIndex` and `TMM1D`). Only the vectorial solver provides accurate computation of the scattering and diffraction losses in a VCSEL of any type of geometry. This is important in computing the photon lifetime, which is a critical parameter in determining the slope efficiency of the light power output and the threshold current.

Nevertheless, the scalar solvers are extremely fast and most useful in the initial design phase of the VCSEL structure. In particular, the effective index method (keyword `EffectiveIndex`) can compute fairly accurate resonant wavelengths for strongly index-guided VCSELs, including oxide-confined VCSELs.

All VCSEL simulations in Sentaurus Device must be performed on two different grids: one for the electrical problem and one for the optical problem.

Cavity Optical Modes in VCSELs

The cavity eigenproblem deals with resonance in a cavity and is a difficult problem to handle for an arbitrary geometry and inhomogeneous cavity. Cavity resonance is defined as the condition whereby a wave can sustain itself in harmony inside the cavity. This means that even after undergoing multiple internal reflections inside the cavity, the optical waves at every point in the cavity are in phase with each other. If the cavity is leaky, some waves leak and the resonance decreases in amplitude and eventually diminishes. However, in a laser cavity, when stimulated emission of photons into the resonant mode is greater than the leakage, the resonance is sustained.

The cavity eigenproblem is then a statement of how to find these resonant modes (resonant wavelength) and the required gain within the active region that will balance the leakage (optical loss) to sustain resonance, that is, laser action.

34: Lasers

Vertical-Cavity Surface-Emitting Lasers

Therefore, the cavity eigenproblem is different from the waveguide problem. In summary, the input is:

- Refractive index profile
- Boundary conditions

The required output for the cavity eigenproblem is:

- Resonant wavelength
- Resonant optical field profile
- Net optical loss

The detailed treatment of the cavity vectorial eigensolver for VCSELs in Sentaurus Device has been published [6] and only a summary of key ideas is presented here. The treatment essentially follows the original idea of Henry [1], which has been extended to a nonadiabatic form [7].

First, there is the time-dependent vector wave equation:

$$\nabla \times (\nabla \times \mathbf{E}(\mathbf{r}, t)) + \frac{1}{c^2} \frac{\partial^2}{\partial t^2} [\epsilon_r(\mathbf{r}, t, \tau) \otimes \mathbf{E}(\mathbf{r}, t - \tau)] = -\mu_0 \frac{\partial^2}{\partial t^2} \mathbf{K}(\mathbf{r}, t) \quad (905)$$

where $\mathbf{K}(\mathbf{r}, t)$ is a source term caused by spontaneous emission that contributes to the electric field density. The term in the brackets is a time convolution of the dielectric function with the electric field.

The electric field is expanded (spectrally decomposed) into a discrete set of orthogonal modes:

$$\mathbf{E}(\mathbf{r}, t) = \sum_v a_v(t) e^{\int_0^t \omega'_v(\tau) d\tau} \Psi_v(\mathbf{r}, \omega) + cc \quad (906)$$

where $\Psi_v(\mathbf{r}, \omega)$ are vectorial modes, $a_v(t)$ are complex values, and cc are the complex conjugate terms. ω'_v is the frequency of the mode, a real valued function.

By substituting Eq. 906 into Eq. 905 and taking only the first-order terms of the time derivatives, a set of inhomogeneous equations is obtained:

$$\sum_v e^{\int_0^t \omega'_v(\tau) d\tau} \cdot \left[\left[\nabla \times (\nabla \times \Psi_v) - \frac{\omega_v'^2}{c^2} \epsilon_r \Psi_v \right] \cdot a_v(t) + \tilde{\epsilon}_r \Psi_v \cdot \dot{a}_v(t) \right] = -\mu_0 \frac{\partial^2}{\partial t^2} \mathbf{K}(\mathbf{r}, t) \quad (907)$$

where:

$$\tilde{\epsilon}_r = \frac{2i\omega'_v}{c^2} \left(\frac{\omega'_v}{2} \frac{\partial}{\partial \omega'_v} \epsilon_r + \epsilon_r \right) \quad (908)$$

In this case, the frequency dispersion of the dielectric function has been taken into account in [Eq. 908](#). To solve the above set of inhomogeneous equations, the auxiliary homogeneous equation is introduced:

$$\left[\nabla \times (\nabla \times) - \frac{\omega_v'^2}{c^2} \epsilon_r \right] \cdot \Psi_v = \omega_v'' \tilde{\epsilon}_r \Psi_v \quad (909)$$

where ω_v'' is defined to be the eigenvalue and the orthogonality relation is:

$$\int_{\text{volume}} \Psi_u \cdot \tilde{\epsilon}_r \Psi_v d\mathbf{r} = \delta_{uv} \quad (910)$$

At resonance, ω_v'' must be purely real [\[6\]](#) and this forms the basis for finding the solution to the cavity eigenproblem. The frequency ω_v' is varied and [Eq. 909](#) is solved repeatedly for ω_v'' . As ω_v' approaches the resonance value, the imaginary part of ω_v'' approaches zero in an approximately linear manner. This gives you an advantage in accelerating the solution-hunting. Therefore, it is important that a ‘close enough’ target eigenvalue is provided by you to benefit from this advantage.

Next, to derive the photon rate equation for each cavity mode v , [Eq. 909](#) is substituted into [Eq. 907](#) and the orthogonality relation, [Eq. 910](#), is applied. After some manipulation, the cavity photon rate equation evaluates to the familiar form:

$$\frac{d}{dt} S_v = -2\omega_v'' S_v + R_v^{\text{sp}} \quad (911)$$

where R_v^{sp} is the spontaneous emission rate. It transpires that the eigenvalue of [Eq. 909](#), $2\omega_v''$, is the net loss rate.

The material gain and absorption enter the photon rate equation indirectly through the dielectric function of the active region:

$$\epsilon_{r, \text{active}}(\mathbf{r}, \omega_v') = \left(n(\mathbf{r}, \omega_v') - \frac{c^2}{4\omega_v'^2} r^{\text{st}}(\mathbf{r}, h\omega_v')^2 \right) + i \frac{n(\mathbf{r}, \omega_v')c}{\omega_v'} r^{\text{st}}(\mathbf{r}, h\omega_v') \quad (912)$$

$r^{\text{st}}(\mathbf{r}, h\omega_v')$ is the stimulated emission coefficient and is discussed in [Radiative Recombination and Gain Coefficients on page 898](#). This approach ensures that the material gain and absorption are coupled rigorously between the electronics and optics. Therefore, the total cavity loss (or gain) is computed accurately. With the capability to model rapid changes in material gain, this cavity solver allows you to handle gain-guided VCSELs as well.

The photon rate equation is coupled to the Poisson, carrier continuity, and temperature (and hydrodynamic) equations using the Newton method, so the derivatives of $2\omega_v''$ with respect to a host of quantities for the Jacobian matrix entries are required.

$2\omega_v''$ can be identified as the relative change of the average electromagnetic energy stored in mode v :

$$2\omega_v'' = \frac{1}{\int_{\text{volume}} \langle W_v \rangle d\mathbf{r}} \cdot \int_{\text{volume}} \left\langle \frac{\partial W_v}{\partial t} \right\rangle d\mathbf{r} \quad (913)$$

where W_v is the energy density of the electromagnetic field of mode v derived from the Poynting vector. Assuming that the optical mode only changes slowly, the derivatives of $2\omega_v''$ can be computed from the derivatives of the dielectric function, $\epsilon_r(\mathbf{r}, t, \tau)$.

VCSEL Output Power

When the vectorial optical solver is used for VCSEL simulation, you must ensure that perfectly matched layers (PMLs) are used for the simulation structure. The PMLs act as wave absorbers to prevent reflections and artificially simulate the radiative boundary condition (see [Perfectly Matched Layers on page 865](#)). The output power emitted from the top surface is the time averaged dissipation of the optical power in the top PML.

The emitted power for mode v is, therefore, computed by the integral:

$$P_{\text{TopEmit}, v} = \int_{\text{vol} - \text{TopPML}} \left\langle \frac{\partial W_v}{\partial t} \right\rangle d\mathbf{r} \quad (914)$$

and takes into account the material absorption of the top PML. For purposes of calibration, it is possible to apply a scaling factor to the emitted power.

This factor can be specified in the **Physics-Laser-Optics** section of the command file as **PowerCouplingFactor=<float>**.

The optical model described above, however, does not take into account the reduction of the optical output power caused by perturbations of the optical cavity modes by free carrier absorption and other distributed internal optical losses. This is because this data is not fed into the optical solver that solves the optical eigenvalue problem, thereby determining the optical eigenmodes. To account for the reduced output power, a phenomenological correction can be used. In this case, the effective output power can be expressed as:

$$P_{\text{Total}, v} = P_{\text{TopEmit}, v} - x_{\text{FCA}} L_{\text{carr}} - x_{\text{bg}} L_{\text{bg}} \quad (915)$$

where L_{carr} is the free carrier absorption rate given by [Eq. 900, p. 792](#) and L_{bg} is the background loss of the cavity that can be specified with the command file keyword **OpticalLoss** in the **Physics-Laser** section.

The parameters x_{FCA} and x_{bg} can be specified in the Physics-Laser section:

```
Physics {...
  Laser (...
    ReducedOutcoupling(FreeCarrierAbsorption = <float> #  $x_{\text{FCA}}$ 
                      OpticalLoss = <float>)           #  $x_{\text{bg}}$ 
  )
}
```

Cylindrical Symmetry

Discretizing the VCSEL structure in 3D results in a prohibitively huge mesh size. Therefore, cylindrical geometry is assumed to reduce the size of the problem. By considering cylindrical symmetry in a body-of-revolution (BOR) about the symmetry axis, the cavity modes of the VCSEL can be further decomposed into cylindrical harmonics:

$$\Psi_v(\rho, \phi, z) = \sum_m \Psi_v^{(m)}(\rho, z) \cdot e^{im\phi} \quad (916)$$

It is well known that the cylindrical harmonics are orthogonal and, hence, the vectorial resonant optical field $\Psi_v^{(m)}(\rho, z)$ is solved for each cylindrical order, m , in 2D space. In the command file, the cylindrical harmonic order m is input using the keyword `AzimuthalExpansion`. (The syntax is discussed further in [FE Scalar Solver on page 856](#).)

Using experience from optical fiber modeling, the fundamental mode of an optical fiber is HE11, followed by its immediate higher order modes, TE01, TM01, and so on. This means that the fundamental mode has cylindrical harmonic order of $m = 1$, and the next two higher order modes have orders $m = 0$. You are encouraged to use different cylindrical harmonic orders to verify which one contains the fundamental resonant mode of the VCSEL cavity.

VCSEL structures are generally assumed to be cylindrically symmetric, and cylindrical symmetry in Sentaurus Device is managed in a slightly different way. The 2D plane whereby the device is drawn is treated as the (ρ, z) plane in cylindrical symmetry.

In addition to the specification of `Coordinates=Cylindrical` in the Physics-Laser-Optics-FEVectorial section, the keyword `Cylindrical` must also be added to the Math section:

```
Physics {...
  AreaFactor = 1
  Laser (...
    Optics(
      FEVectorial(...
        Coordinates = Cylindrical
      )
    )
  )
}
```

34: Lasers

Vertical-Cavity Surface-Emitting Lasers

```
    )
    VCSEL()    # specify this is a VCSEL simulation
  )
}
...
Math {...
  Cylindrical
}
```

This is to ensure that the area and volume computations associated with each vertex is based on cylindrical symmetry.

NOTE In the cylindrical symmetry case, it is not necessary to specify an AreaFactor of 2.

Different Grid and Structure for Electrical and Optical Problems

A typical VCSEL structure contains over 100 layers. Sentaurus Device can manage such a large electrical problem, but the resultant Jacobian matrix may be too large for standard computers to handle. The carrier transport behavior in the distributed Bragg reflectors (DBRs) is not interesting with regard to the laser physics, and the DBRs will probably contribute only to an additional series resistance. Therefore, the size of the electrical problem can be simplified and reduced by replacing the DBR layers with an equivalent homogeneous bulk material with similar total resistance and thermal conduction properties. However, the actual layered structure must be used for the optical simulation because the optical resonance depends on the exact geometry of the VCSEL cavity.

Therefore, there will be two different structures and grids for the electrical and optical problems. To handle this, the dual-grid mixed-mode capability of Sentaurus Device is used. The syntax for such a dual-grid simulation is described in [Simulating Vertical-Cavity Surface-Emitting Laser on page 768](#).

For the vectorial optical (FEVectorial) solver, the optical mesh is discretized according to the requirements of the finite-element method. Adaptive meshing can be used to refine the mesh at sharp corners where scattering and diffraction effects are strong. For the scalar optical solvers (EffectiveIndex and TMM1D), the meshing of the optical grid can be relaxed: you will draw the VCSEL structure and apply coarse meshing to generate the optical grid.

Aligning Resonant Wavelength Within Gain Spectrum

Besides being a cavity problem, a VCSEL is different from an edge-emitting laser mainly in the size of the gain volume. The gain volume in a VCSEL is many times smaller than that of an edge-emitting laser. Therefore, ensuring a VCSEL lases is an intricate design task of minimizing the source of optical and electrical losses and maximizing the gain. One critical issue in VCSEL design is to align the resonant wavelength such that it is within the gain spectrum during lasing.

Self-heating of the VCSEL cavity causes red-shifts in both the gain spectrum (bandgap reduction as shown in [Figure 61](#)) and the resonant wavelength (thermal lensing effect). However, the shift in the gain spectrum is many times that of the resonant wavelength. Therefore, it is important to adjust the resonant wavelength such that it is near the peak of the gain spectrum at the required operating bias.

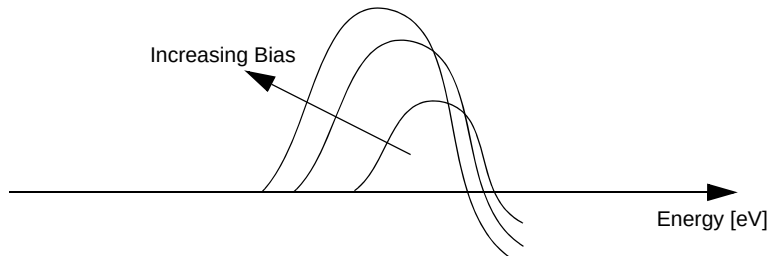


Figure 61 As bias increases, the gain spectrum red-shifts (bandgap reduction) due to self-heating effects

[Figure 62](#) shows two starting positions of the resonant mode. As the bias increases, the gain spectrum shifts left, and the resonant mode of the model in [Figure 62 \(right\)](#) runs the risk of not obtaining enough gain required for lasing.

Therefore, it becomes a design issue of geometry and quantum well material gain to ensure that the resonant wavelength and gain spectrum (before lasing) resembles the model in [Figure 62 \(left\)](#).

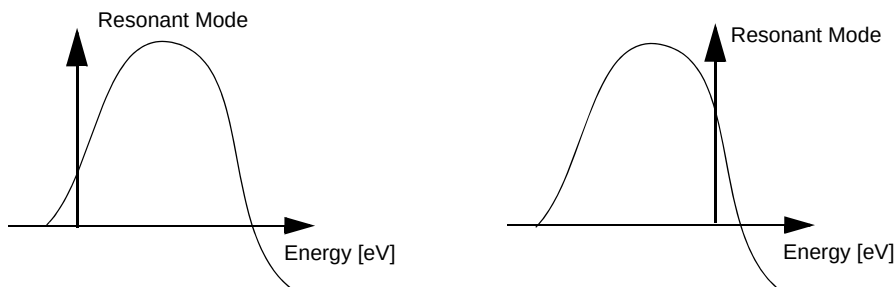


Figure 62 Resonant wavelength chosen to with different starting positions from the gain spectrum at very low bias (not lasing yet)

Approximate Methods for VCSEL Cavity Problem

Apart from the rigorous vectorial treatment of VCSEL cavity optics, Sentaurus Device also contains two approximate methods to compute the resonant modes in VCSEL cavities:

- Transfer matrix method for multilayers with a Gaussian transverse mode profile
- Effective index method

Both methods are scalar in nature and fast, and have low memory requirements. In particular, the transfer matrix method is suitable for users who want to look at how the transverse mode shape affects the different lasing characteristics of a VCSEL. The effective index method is best suited for index-guided VCSELs and it has been shown to compute accurate resonant wavelengths for many index-guided VCSEL structures including oxide-confined ones. When these approximate methods are used, Sentaurus Device uses the same photon rate equation as for edge-emitting lasers.

For further details about these solvers, see [Transfer Matrix Method for VCSELs on page 867](#) and [Effective Index Method for VCSELs on page 869](#).

Device Physics and Parameter Tuning

For a VCSEL, many quantities such as threshold current, slope efficiency, and stimulated and spontaneous gain can be similarly tuned as in the case of an edge-emitting laser. Other quantities that are of interest in a VCSEL simulation are:

Resonant wavelength selection

The resonant wavelength can be changed by changing the thickness or refractive indices of the DBR layers or the lambda cavity of the VCSEL. You can use the optics stand-alone option and the `EffectiveIndex` scalar solver to accomplish this optical design problem efficiently.

Aligning wavelength with gain spectrum

It is easier to tune the resonant wavelength rather than the gain spectrum. You are advised to run the simulation once to look at the gain spectrum shifts, then change the resonant wavelength as described in the previous paragraph.

Gain spectrum shifts

The band gap is reduced as temperature increases (due to self-heating). Higher carrier densities also result in bandgap renormalization caused by manybody effects. However, the bandgap renormalization shift is very small compared to the temperature bandgap shift.

Scattering and diffraction losses

The optical losses give the photon lifetime which is a critical parameter in the threshold current and slope efficiency. Only the vectorial optical (FEVectorial) solver can compute the optical losses accurately.

If you plan to use the scalar optical solvers (EffectiveIndex or TMM1D), it is advisable to run the vectorial optical solver once to obtain the accurate optical losses, then append an appropriate background loss (using keyword `OpticalLoss` in `Physics-Laser` section) or diffraction loss (using keyword `DiffractionLoss` in `EffectiveIndex` section) to the scalar optical solvers to enhance the accuracy of the scalar simulation.

References

- [1] C. H. Henry, “Theory of Spontaneous Emission Noise in Open Resonators and its Application to Lasers and Optical Amplifiers,” *Journal of Lightwave Technology*, vol. LT-4, no. 3, pp. 288–297, 1986.
- [2] R. K. Smith *et al.*, “Numerical Methods for Semiconductor Laser Simulations,” in *Fifth International Workshop on Computational Electronics (IWCE)*, Notre Dame, IN, USA, May 1997.
- [3] K. Rajkanan, R. Singh, and J. Shewchun, “Absorption Coefficient of Silicon for Solar Cell Calculations,” *Solid-State Electronics*, vol. 22, no. 9, pp. 793–795, 1979.
- [4] M. Koshiha, *Optical Waveguide Theory by the Finite Element Method*, Tokyo: KTK Scientific Publishers, 1992.
- [5] A. Witzig *et al.*, “Optimization of a Leaky-Waveguide Laser Using DESSIS,” in *3rd International Conference on Numerical Simulation of Semiconductor Optoelectronic Devices (NUSOD-03)*, Tokyo, Japan, October 2003.
- [6] M. Streiff *et al.*, “A Comprehensive VCSEL Device Simulator,” *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 9, no. 3, pp. 879–891, 2003.
- [7] G. A. Baraff and R. K. Smith, “Nonadiabatic semiconductor laser rate equations for the large-signal, rapid-modulation regime,” *Physical Review A*, vol. 61, no. 4, p. 043808, 2000.

34: Lasers

References

CHAPTER 35 Light-Emitting Diodes

This chapter describes the physics and the models used in light-emitting diode simulations.

NOTE LED simulations present unique challenges that require problem-specific model and numerics setups. Contact TCAD Support for advice if you are interested in simulating LEDs (see [Contacting Your Local TCAD Support Team Directly on page xl](#)).

Modeling Light-Emitting Diodes

From an electronic perspective, light-emitting diodes (LEDs) are similar to lasers operating below the lasing threshold. Consequently, the electronic model contains similar electrothermal parts and quantum-well physics as in the case of a laser simulation.

Therefore, the theory presented for quantum-well modeling in [Chapter 37 on page 895](#) is applicable to LED simulations as well. The key difference between an LED and a laser is a resonant cavity design for lasers that enhances the coherent stimulated emission at a single frequency (for each mode). An LED emits a continuous spectrum of wavelengths based on spontaneous emission of photons in the active region. However, an alternative design for LEDs with a resonant cavity – the resonant cavity LED (RCLED) – uses the resonance characteristics to cause an amplified spontaneous emission in a narrower spectrum to allow for superbright emissions.

The simulation of LEDs presents many challenges. The large dimension of typical LED structures, in the range of a few hundred micrometers, prohibits the use of standard time-domain electromagnetic methods such as finite difference and finite element. These methods require at least 10 points per wavelength and typical emissions are of the order of $1\text{ }\mu\text{m}$. A quick estimate gives a necessary mesh size in the order of 10 million mesh points for a 2D geometry. Alternatively, the use of the raytracing method approximates the optical intensity inside the device as well as the amount of light that can be extracted from the device. In many cases, a 2D simulation is not sufficient and a 3D simulation is required to give an accurate account of the physical effects associated with the geometric design of the LED.

Innovative designs such as inverted pyramid structures, chamfering of various corners, surface roughening, and drilling holes are performed in an attempt to extract the maximum amount of light from the device. The device editor Sentaurus Structure Editor is well equipped to create complex 3D devices and provides great versatility in exploring different realistic LED designs.

Photon recycling is important because most of the light rays are trapped within the device by total internal reflection. There are two types of photon recycling: for nonactive regions and for the active region. The nonactive-region photon recycling involves absorption of photons in the nonactive regions to produce optically generated electron-hole pairs, and these subsequently join the drift-diffusion processes of the general carrier population. The active-region photon recycling is more complicated, and interested users are referred to [1] for its basic theory.

Coupling Electronics and Optics in LED Simulations

Similar to a laser simulation, an LED simulation solves the Poisson equation, carrier continuity equations, temperature equation, and Schrödinger equation self-consistently. Figure 63 illustrates the coupling of the various equation systems in an LED simulation.

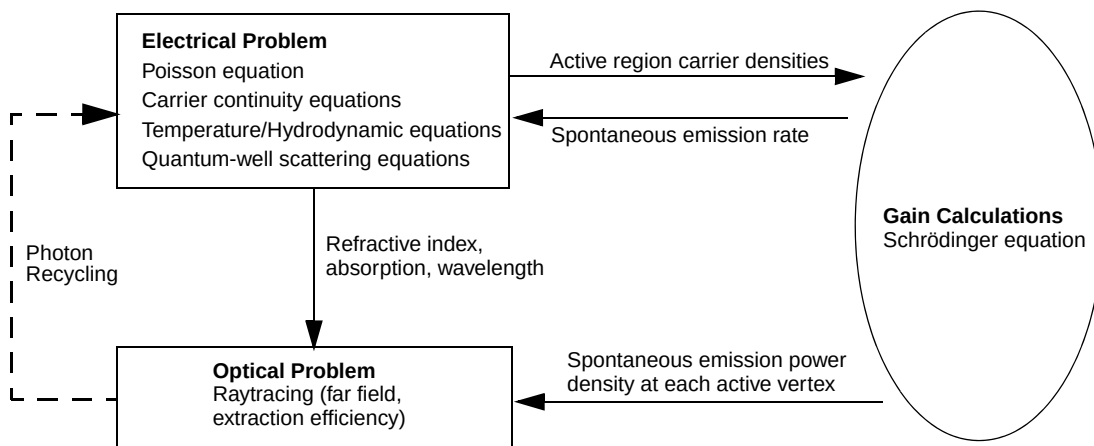


Figure 63 Flowchart of the coupling between the electronics and optics for an LED simulation

Single-Grid Versus Dual-Grid LED Simulation

Both single-grid and dual-grid LED simulations are possible. However, in the case of a single-grid simulation, raytracing takes a longer time for the following reason: Raytracing builds a binary tree for each starting ray. Each branch of the tree corresponds to a ray at a mesh cell boundary. If the materials in two adjoining cells are different, the ray splits into refracted and reflected rays, creating two new branches. If the materials are the same in adjoining cells, the propagated ray creates a new branch. A fine mesh increases the depth of the branching significantly. Each new branch of the binary tree is created dynamically and, if dynamic memory allocation of the machine is not sufficiently fast, the tree creation of the raytracing becomes a bottleneck in the simulation.

To overcome this problem, the grids for the electrical problem and the raytracing problem are separated. The optical grid for raytracing is meshed coarsely. The binary tree created will be smaller and raytracing is more efficient. Such a coarse mesh enables you to compute the extraction efficiency and output radiation pattern. However, the optical intensity within the device cannot be resolved well with a coarse mesh.

Electrical Transport in LEDs

Besides the drift-diffusion transport of typical semiconductor devices, the LED requires additional physical models to compute various optical effects. These physical models are described in the following sections.

Spontaneous Emission Rate and Power

In the active region, the spontaneous emission of photons depletes the carrier population. At each active vertex, the spontaneous emission (or recombination of carriers) rate (units of $\#s^{-1}m^{-3}$) is an integral of the spontaneous emission:

$$R_{\text{active}}^{\text{sp}}(x, y, z) = \int_0^{\infty} r^{\text{sp}}(E) \rho^{\text{opt}}(E) dE \quad (917)$$

where the optical mode density is:

$$\rho^{\text{opt}}(E) = \frac{n_g^2 E^2}{\pi^2 \hbar^3 c^2} \quad (918)$$

and $r^{\text{sp}}(E)$ is defined in [Eq. 978, p. 898](#). The total spontaneous emission power density at each active vertex (units of $Js^{-1}m^{-3}$) is:

$$\Delta P^{\text{sp}}(x, y, z) = \int_0^{\infty} r^{\text{sp}}(E) \rho^{\text{opt}}(E) \cdot (\hbar\omega) dE \quad (919)$$

This equation is similar to [Eq. 917](#), except that an additional energy term, $E = \hbar\omega$, is included in the integrand to account for the energy spectrum of the spontaneous emission.

In LED simulations, the spontaneous emission spectrum, $r^{\text{sp}}(E)$, broadens significantly with increasing injected carriers. The integrals in [Eq. 917](#) and [Eq. 919](#) are based on a Riemann sum and, as with numeric integration, truncates at an internally set energy value. You can change the truncation value and the Riemann integration interval with the following syntax in the command file:

```
Physics {...
  LED (...
    Optics (...)
    SponScaling = 1.0
    SponIntegration(<energyspan>,<numpoints>)
  )
}
```

where <energyspan> is a floating-point number (in eV) and is measured from the edge of the energy bandgap, and <numpoints> is an integer denoting the number of discretized intervals to use within this energy span.

The total spontaneous emission power is the volume integral of the power density over all the active vertices:

$$P_{\text{total}}^{\text{sp}} = \int_{(\text{active} - \text{region})} \Delta P^{\text{sp}}(x, y, z) dV \quad (920)$$

This is the total spontaneous emission power that is computed and output in an LED simulation.

Since you are dealing with a spectrum for the spontaneous emission, it is evident from [Eq. 917](#) and [Eq. 919](#) that the integral sum of the photon rate and the integral sum of the photon power are not simply related by a constant photon energy, that is:

$$\Delta P^{\text{sp}}(x, y, z) \neq (\hbar\omega_0) R_{\text{active}}^{\text{sp}}(x, y, z) \quad (921)$$

Spontaneous Emission Power Spectrum

The LED spontaneous emission power spectrum can be plotted by activating the GainPlot section (see [Modal Gain as Function of Energy/Wavelength on page 920](#)). The syntax consists of defining the number of discretized points for the spectrum and the span of the spectrum to plot:

```
File {...
  ModeGain = "ngainplot_des"
}

GainPlot {
  Range = (<float>,<float>) # specific range in eV
```

```

Range = Auto           # automatically determines range
Intervals = <integer>  # number of discretized points
}

Solve {...
  PlotGain( Range=(0,1) Intervals=5 )
}

```

In the gain file, the quantity SponEmissionPowerPereV (W/eV) is plotted. Integrating this quantity over the energy (eV) span recovers the total LED power.

Current File and Plot Variables for LED Simulation

When an LED simulation is run, specific LED result variables are output to the plot file. [Table 125](#) and [Table 126 on page 816](#) list the current file output and the plot variables valid for LED simulation.

Table 125 Current file for LED simulation

Dataset group	Dataset	Unit	Description
Time		1 s	For quasistationary. For transient.
n_Contact p_Contact	OuterVoltage	V	
	InnerVoltage	V	
	eCurrent	A	
	hCurrent	A	
	TotalCurrent	A	
	Charge	C	
LedWavelength		nm	Average wavelength of LED simulation.
Photon_ExtEfficiency		1	Photon extraction efficiency.
Photon_Spontaneous		s ⁻¹	Spontaneous emission photon rate.
Photon_Exited		s ⁻¹	Rate of photon escaping from device.
Photon_Trapped		s ⁻¹	Rate of trapped photons in device.
Photon_NetPhotonRecycle		s ⁻¹	Net photon-recycling photon rate.
Photon_NonActiveAbsorb		s ⁻¹	Rate of photon absorption in nonactive regions.
Power_ExtEfficiency		1	Power extraction efficiency.

Table 125 Current file for LED simulation

Dataset group	Dataset	Unit	Description
Power_Spontaneous		W	Spontaneous emission power.
Power_ASE		W	Amplified spontaneous emission power.
Power_ReEmit		W	Re-emission power.
Power_Absorption		W	Absorption power.
Power_SpecConvertGain		W	Net power gain of spectral conversion.
Power_NetPhotonRecycle		W	Net photon-recycling power.
Power_Exited		W	Power escaping from device.
Power_Trapped		W	Power trapped inside device.
Power_NonActiveAbsorb		W	Power absorbed in nonactive regions.
Power_Total		W	Total internal optical power of LED.

Table 126 Plot variables for LED simulation

Plot variable	Dataset name	Unit	Description
DielectricConstant		1	Dielectric profile.
MatGain	OpticalMaterialGain	m^{-1}	Local material gain.
LED_TraceSource			Influence of each active vertex on the total extracted light.
SpontaneousRecombination		$\text{cm}^{-3}\text{s}^{-1}$	Sum of spontaneous emission.
RayTraceIntensity		Wcm^{-3}	Optical intensity from raytracing.
RayTrees			Raytree structure. Not available for the compact memory option.

LedWavelength is computed automatically in the simulation. It is taken as the wavelength where the peak of the spontaneous spectrum occurs. Different options for computing the wavelength are available (see [LED Wavelength on page 817](#)). For clarity, photon rate and power output results are separated into two groups:

- The photon rate group has units of s^{-1} .
- The power group has units of W.

Photon and power quantities need to be computed separately if a spectrum is involved. Suppose that the spontaneous emission coefficient is r^{sp} (units of $\text{eV}^{-1}\text{cm}^{-3}\text{s}^{-1}$). The photon rate is

$\int r^{sp'} dE$ (Eq. 917, p. 813 with $r^{sp'} = r^{sp}(E) \times p^{opt}(E)$), while the power is $\int r^{sp'} \cdot E dE$ (Eq. 919, p. 813). The extraction coefficient is the ratio of the exited and internal quantities, so the photon rate (Photon_ExtEfficiency) and power (Power_ExtEfficiency) extraction efficiencies are different.

The only case when Photon_ExtEfficiency=Power_ExtEfficiency is for a single wavelength simulation that does not involve a spectrum.

The total outcoupled (exited) power is then (Power_ExtEfficiency x (Power_Total-Power_NonActiveAbsorb)), where the total internal optical power is:

$$\text{Power_Total} = \text{Power_Spontaneous} + \text{Power_NetPhotonRecycle} + \text{Power_SpecConvertGain} \quad (922)$$

The photon rate extraction efficiency has been computed using:

$$\text{Photon_ExtEfficiency} = \text{Photon_Exited} / (\text{Photon_Spontaneous} + \text{Photon_NetPhotonRecycle} - \text{Photon_NonActiveAbsorb}) \quad (923)$$

For power conservation in a nonactive photon-recycling case, you have:

$$\begin{aligned} \text{Power_Total} &= \text{Power_Spontaneous} \\ &= \text{Power_Exited} + \text{Power_Trapped} + \text{Power_NonActiveAbsorb} \end{aligned} \quad (924)$$

Power_Trapped refers to the power of the photons that are trapped (by total internal reflection or possibly nonconverging raytracing) indefinitely in the raytracing simulation. In a realistic scenario, the trapped photons decay in the device by some mechanism.

NOTE Users are responsible for introducing losses within the device (perhaps by introducing nonzero extinction coefficients in appropriate regions) to ensure proper treatment of the trapped photons.

NOTE The Power_NetPhotonRecycle, Power_SpecConvertGain, and Photon_NetPhotonRecycle quantities pertain only to the active-region photon-recycling model. These quantities are zero if the active-region photon-recycling model is not activated.

LED Wavelength

There are different ways of computing or inputting the LED wavelength:

- **AutoPeak:** A robust algorithm based on the multisection method is used to search for the peak of the spontaneous emission rate spectrum ($r^{sp'}$ versus E-curve). Then, the energy of the peak $r^{sp'}$ is translated into the LED wavelength.

- **AutoPeakPower:** The same algorithm as for AutoPeak is used on the spontaneous emission power spectrum ($r^{sp}E$ versus E-curve).
- **Effective:** An effective wavelength is computed such that:

$$\text{LED power} = \text{LED photon rate} \times \text{photon_energy} \quad (925)$$

where photon_energy is a direct inverse function of the effective wavelength.

- User inputs a fixed LED wavelength.

The keywords for activating the various LED wavelength options are:

```
Physics {...  
  LED (...  
    Optics (...  
      RayTrace (...  
        Wavelength = <float> | AutoPeak | Effective | AutoPeakPower  
      )  
    )  
  )  
}
```

NOTE If no Wavelength keyword is specified, the old peak wavelength search algorithm is used.

Optical Absorption Heat

Photon absorption heat in semiconductor materials can be simplified into two processes:

- *Interband absorption:* When a photon is absorbed across the forbidden band gap in a semiconductor, it is absorbed to create an electron–hole pair. The excess energy (photon energy minus the band gap) of the new electron–hole pair is assumed to thermalize, resulting in eventual lattice heating.
- *Intraband absorption:* A photon can be absorbed to increase the energy of a carrier. The excess energy relaxes eventually, contributing to lattice heating.

In both processes, it is assumed that the eventual lattice heating (in a quasistationary simulation) occurs in the locality of photon absorption.

In LEDs, the extraction efficiency ranges between 40% and 60% commonly. This means a significant portion of the spontaneous light power is trapped within the device. The trapped light is ultimately absorbed and becomes the source of photon absorption heat.

New keywords in the Physics section have been added. As far as possible, the syntax has been kept similar to that of the QuantumYield model (see [Quantum Yield Models on page 475](#)):

```
Plot {...
  OpticalAbsorptionHeat      # units of [W/cm^3]
}

Physics {...
  LED(...)
  OpticalAbsorptionHeat (
    StepFunction (
      Wavelength = float      # um
      Energy = float          # eV
      Bandgap          # auto-checking of band gap
      EffectiveBandgap    # auto-checking of band gap
    )
    ScalingFactor = float      # default is 1.0
  )
}
```

Some comments about the different StepFunction choices:

- When the keyword **Wavelength** or **Energy** is used, the wavelength or energy of the photons is checked against this value. If the photon wavelength or energy is smaller or larger than this cutoff value, convert the optical generation totally into optical absorption heat, and set optical generation at the vertex to 0.
- The keywords **Bandgap** and **EffectiveBandgap** refer to the cutoff energy of the step function at E_g and $(E_g + 2 \cdot (3/2)kT)$, respectively. If the photon energy is greater than the cutoff energy (interband absorption), deduct the band gap to obtain the excess energy but do not deduct the optical generation. If the photon energy is less than the cutoff energy (intraband absorption), set the excess energy to the photon energy. In both cases, the excess energy will be converted to a heat source term for the lattice temperature equation.
- A **ScalingFactor** is introduced to allow for flexible fine-tuning.
- The **OpticalAbsorptionHeat** statement can be specified globally in the **Physics** section or the materialwise or regionwise **Physics** section.

QW Physics

The LED simulator shares the same QW physics as the laser simulator. However, due to the size of the LED structure, it is recommended that you start with representing QWs as bulk active regions and then contact TCAD Support for assistance in the more advanced settings (see [Contacting Your Local TCAD Support Team Directly on page xl](#)).

NOTE QW physics treatment is very complicated and depends strongly on the material system. The models used in one material system may not be applicable to others, and convergence cannot be guaranteed for all material systems.

Localized QW Model

In GaN-based QW systems, the polarization charge sheets at the interfaces of the QW/barrier induce a large field within the QW. These fields skew the energy bands and cause mismatches in the alignment of the electron and hole wavefunctions.

A simple localized QW model has been introduced that takes into account field effects in a fully coupled methodology. The activation syntax is included in the `Physics` section of each active QW:

```
Physics (region="QW1") {...
  Active(Type=QuantumWell)
  QWLocal (
    NumberOfElectronSubbands = 5
    NumberOfLightHoleSubbands = 2
    NumberOfHeavyHoleSubbands = 4
    # -ElectricFieldDep
    WidthExtraction (
      #indicate side regions or materials of QW if sides
      #do not coincide with domain boundaries
      SideRegion = ("reg1", ..., "regn")
      SideMaterial = ("mat1", ..., "matn")
      MinAngle = (<float>, <float>)
      ChordWeight = <float>
    )
  )
}
```

By default, the electric field dependency is activated in the localized QW model. However, it can be deactivated by using the keyword `-ElectricFieldDep`. The maximum number of subbands for the electrons, heavy holes, and light holes must be specified to enable Sentaurus Device to limit the scope of the computation. The actual number of subbands used is computed as the simulation progresses. A `WidthExtraction` section is included to enable you to specify how the QW thickness can be extracted. This thickness is important to define the QW width so as to compute the solution to the Schrödinger equation. `MinAngle` and `ChordWeight` are parameters used in a special method to compute the thickness of the QW layer that is not aligned to one of the major axes (see [Thickness Extraction on page 340](#)).

If the `QWLocal` section is defined in the global `Physics` section, the parameters of the maximum number of bands in each band are applied to all the specified `Active(Type=QuantumWell)` regions.

Table 127 lists the different localized QW results that can be plotted.

Table 127 Variables available for plotting the localized QW model

Variable	Description
QW_eEigenEnergy	Eigenenergies of the electron bound states [eV].
QW_ElectricFieldProjection	Electric field in the QW [V/cm].
QW_eNumberOfBoundStates	Actual number of QW bound states for electrons.
QW_hhEigenEnergy	Eigenenergies of the heavy-hole bound states [eV].
QW_hhNumberOfBoundStates	Actual number of QW bound states for heavy holes.
QW_lhEigenEnergy	Eigenenergies of the light-hole bound states [eV].
QW_lhNumberOfBoundStates	Actual number of QW bound states for light holes.
QW_OverlapIntegral	Overlap integrals between electron and hole wavefunctions.
QW_QuantizationDirection	Quantization direction of the QW.
QW_Width	Extracted width of the QW [μm].

You also can include any of the variables in Table 127 in the CurrentPlot statement, for example:

```
CurrentPlot { ...
    QW_eEigenEnergy (
        Minimum (Region = "QW1")
        Maximum (Region = "QW1")
        Average (Region = "QW1")
    )
}
```

NOTE As this is a new model, if you are interested in using this model, contact TCAD Support for assistance in evaluating whether this model is suitable for use in your device (see [Contacting Your Local TCAD Support Team Directly on page xl](#)).

Accelerating Gain Calculations and LED Simulations

The computation bottleneck in LED simulations with gain tables is the calculation of the spontaneous emission rate at every Newton step. The spontaneous emission rate (unit is s^{-1}) is an energy integral of the spontaneous emission coefficient (unit is $\text{s}^{-1}\text{eV}^{-1}\text{cm}^{-3}$), and the integration is performed as a Riemann sum. To accelerate this calculation, a Gaussian quadrature integration can be used instead.

All gain calculations (with or without the gain tables) can be accelerated by specifying the following syntax in the Math section of the command file:

```
Math {...  
    BroadeningIntegration ( GaussianQuadrature ( Order = 10 ) )  
    SponEmissionIntegration ( GaussianQuadrature ( Order = 5 ) )  
}
```

The Gaussian quadrature numeric integration then is used in all parts of the gain calculations including broadening effects.

The order can be from 1 to 20. This order refers to the order of the Legendre polynomial that is used to fit the spontaneous emission spectrum in the energy range of the integration. The Gaussian quadrature integration is exact for any spectrum that can be expressed as a polynomial.

NOTE Convergence issues can be experienced in some situations. In such cases, switch off the Gaussian quadrature integration for SponEmissionIntegration and BroadeningIntegration in the Math section.

Discussion of LED Physics

Many physical effects manifest in an LED structure. Current spreading is important to ensure that the current is channeled to supply the spontaneous emission sources at strategic locations that will provide the optimal extraction efficiency.

Changes to the geometric shape of the LED are made to extract more light from the structure. In most cases, the major part of the light produced is trapped within the structure through total internal reflection. As a result, the nonactive-region and active-region photon-recycling effect becomes relevant. This important physics has been incorporated into different physical models within Sentaurus Device. The nonactive photon-recycling (absorption of photons in nonactive regions) is switched on automatically by default. In most cases, the nonactive photon-recycling model is sufficient to capture the major physical effects because the volume of the active region is insignificant compared to the nonactive regions. Nonetheless, the active-region photon-recycling model can become important if there is a very high stimulated gain, which is usually not the case for GaN LEDs.

Important aspects of LED design are simulated easily by Sentaurus Device. These include current spreading flow, geometric design, and extraction efficiency.

LED Raytracing

Raytracing is used to compute the intensity of light inside an LED, as well as the rays that escape from the LED cavity to give the signature radiation pattern for the LED output. The basic theory of raytracing is presented in [Raytracing on page 510](#).

Arbitrary boundary conditions can be defined. A detailed description of how to set up the boundary conditions for raytracing is discussed in [Boundary Condition for Raytracing on page 520](#). This is particularly useful in 3D simulations where you can define reflecting planes to use symmetry for reducing the size of the simulation model.

In addition, you can use reflecting planes to take into account external components such as reflectors, an example of which is shown in [Figure 64](#).

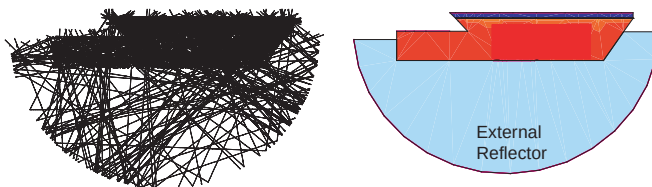


Figure 64 Using the reflecting boundary condition to define a reflector for LED raytracing

The raytracer needs to include the use of the `ComplexRefractiveIndex` model and to define the polarization vector. The syntax for this is:

```
Physics {...
  ComplexRefractiveIndex (...
    WavelengthDep ( real imag )
    CarrierDep( real imag )
    GainDep( real )           # or real(log)
    TemperatureDep( real )
    CRImodel ( Name = "crimodelname" )
  )
  LED (...
    Optics (
      RayTrace(
        PolarizationVector = Random    # or (x y z) vector
        RetraceCRChange = float       # fractional change to retrace rays
        ...
      )
    )
  )
}
```

When `PolarizationVector=Random` is chosen, random vectors that are perpendicular to the starting ray directions are generated and assigned to be the polarization vector of each

starting ray. The direction of each starting ray is described in [Isotropic Starting Rays from Spontaneous Emission Sources](#) and [Anisotropic Starting Rays from Spontaneous Emission Sources](#) on page 825.

The keyword `RetraceCRIchange` specifies the fractional change of the complex refractive index (either the real or imaginary part) from its previous state that will force a total recomputation of raytracing.

Compact Memory Raytracing

A compact memory model has been built for LED raytracing. In the compact memory model, the raytrees are no longer saved, and necessary quantities are computed as required and extracted to compact storages of optical generation and optical intensity. As a result, the memory use and footprint are significantly reduced, thereby enabling the raytracing simulation of large LED structures. The syntax for activation is:

```
Physics { ...  
    LED ( ...  
        RayTrace( ...  
            CompactMemoryOption  
        )  
    )  
}
```

NOTE The full active photon-recycling model does not work with the compact memory model.

Isotropic Starting Rays from Spontaneous Emission Sources

The source of radiation from an LED is mainly from spontaneous emissions in the active region (this is further discussed in [Spontaneous Emission Rate and Power](#) on page 813). The spontaneous emission in the active region of the LED is assumed to be an isotropic source of radiation and can be conveniently represented by uniform rays emitting from each active vertex, as shown in [Figure 65](#) on page 825.

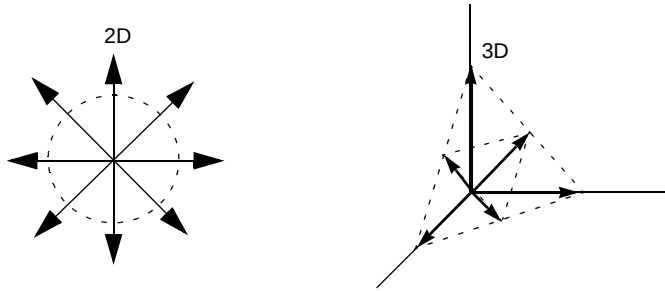


Figure 65 Uniform rays radiating isotropically from an active vertex source in 2D (*left*) and 3D (*right*) space: only one-eighth of spherical space is shown for the 3D case

Isotropy requires that the surface area associated with each ray must be the same. The isotropy of the rays in 2D space is apparent. In 3D space, achieving isotropy is not as simple as dividing the angles uniformly. The elemental surface area of a sphere is $r^2 \sin \theta (d\theta)(d\phi)$, so uniformly angular-distributed rays are weighted by $\sin \theta$ and, therefore, do not signify isotropy.

To approximate this problem in 3D, a geodesic dome approximation is used (the geodesic dome is not strictly isotropic). Rays are directed at the vertices of the geodesic dome. The algorithm starts by constructing an octahedron and, then, recursively splits each triangular face of the octahedron into four smaller triangles.

The first stage of this splitting process is shown in [Figure 65 \(right\)](#), where rays are directed at the vertices of each triangle. The minimum number of rays is six, that is, one is directed along each positive and negative direction of the axes. If the first stage of recursive splitting is applied, a few more rays are constructed as shown in [Figure 65](#), and the number of starting rays becomes 18. The second stage of recursive splitting gives 68 rays and so on. Therefore, you are constrained to selecting a fixed set of starting rays in the 3D case. Alternatively, you can input your own set of isotropic starting rays (see [Reading Starting Rays from File on page 827](#)).

Anisotropic Starting Rays from Spontaneous Emission Sources

In some LED designs, the geometry governs the polarization of the optical field in the device. The spontaneous gain is dependent on the direction of this polarization. Consequently, this leads to an anisotropic spontaneous-emission pattern at the source.

The anisotropic emission pattern is proposed to be described by the following parametric equations:

$$E_x = d1 \cdot \sin(\phi) + d4 \cdot \cos(\phi) \quad (926)$$

35: Light-Emitting Diodes

LED Raytracing

$$E_y = d2 \cdot \sin(\phi) + d5 \cdot \cos(\phi) \quad (927)$$

$$E_z = d3 \cdot \sin(\theta) + d6 \cdot \cos(\theta) \quad (928)$$

where the intensity is given by:

$$I = E_x^2 + E_y^2 + E_z^2 \quad (929)$$

The bases of sine and cosine are chosen based on the fact that the optical matrix element has such a functional form when polarization is considered (see [Polarization-dependent Optical Matrix Element on page 910](#)). By changing the values of d1 to d6, different emission shapes can be orientated in different directions, and this feature allows you to modify the anisotropy of the spontaneous emission.

The syntax required to activate the anisotropic spontaneous emission feature is:

```
Physics {...
  LED (...
    Optics(...
      RayTrace(...
        EmissionType(
          #Isotropic          # default
          Anisotropic(
            Sine(d1 d2 d3)
            Cosine(d4 d5 d6)
          )
        )
      )
    )
  )
}
```

Randomizing Starting Rays

Spontaneous emission is a random process. To take into account the random nature of this process and still ensure that the emission of the starting rays from each active vertex source is isotropic, a randomized shift of the entire isotropic ray emission is introduced.

This is best illustrated in [Figure 66 on page 827](#) where only four starting rays are used for clarity. For each active vertex, a random angle is generated to determine the random shift of the distribution of the isotropic starting rays. The same concept is also used for the 3D case, and this gives a simple randomization strategy for using raytracing to model the spontaneous emissions.

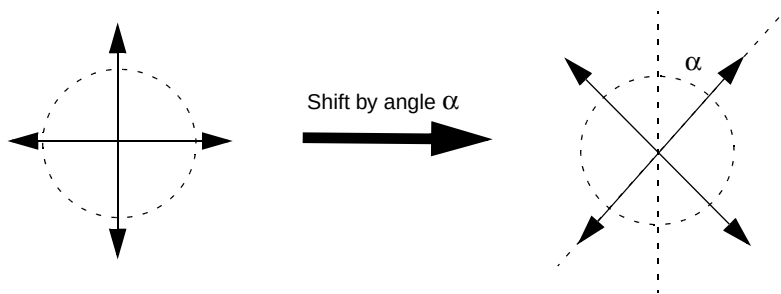


Figure 66 Shifting the distribution of entire isotropic starting rays by an angle α

Pseudorandom Starting Rays

Randomization of the starting rays is activated by the keyword `RaysRandomOffset` in the following syntax:

```
Physics {...
  LED (...
    RayTrace (...
      RaysRandomOffset
#      RaysRandomOffset (RandomSeed = 123)    # seeding the generator
    )
  )
}
```

You also can fix the seed of the random number generator, and this will compute a pseudorandom set of starting rays, so that repeated runs of the same simulation will reproduce exactly the same raytracing results.

NOTE Without the keyword `RaysRandomOffset`, the default is a fixed angular shift that is determined by the active vertex number.

Reading Starting Rays from File

The geodesic ray distribution is not truly isotropic. As a result, some percentages of error are incurred when isotropy is really needed. To circumvent this issue, a new feature to allow you to read in a set of source isotropic starting rays has been implemented. The syntax is:

```
Physics {...
  LED (...
    Optics (...
      RayTrace (...
        RaysPerVertex = 1000
        SourceRaysFromFile("sourcerays.txt")
      )
    )
  )
}
```

```
    )  
  )  
}  
}
```

It is important to note that `RaysPerVertex` specifies the number of direction vectors to read from the file. The file specified with `SourceRaysFromFile` contains 3D direction vectors for each starting ray; an example of which is `sourcerays.txt`:

```
-0.239117 0.788883 0.566115  
0.776959 0.548552 -0.308911  
-0.607096 -0.042603 0.793485  
-0.158313 -0.276546 -0.947871  
0.347036 -0.805488 0.480370  
...
```

There are several methods for generating 3D isotropic rays, for example, the constrained centroid Voronoï tessellation (CCVT). On the other hand, you can also use this option to import experimentally measured ray distribution profiles.

Moving Starting Rays on Boundaries

Starting rays from active vertices on boundaries create the problem that half of those rays are propagated directly out of the device. To alleviate this problem, a new feature is implemented to allow you to shift the starting position of such rays inwards of the device. Typical values used are from 1 to 5 nm. The syntax is:

```
Physics {...  
  LED (...  
    Optics (...  
      RayTrace (...  
        MoveBoundaryStartRays(float) # [nm]  
      )  
    )  
  )  
}
```

Clustering Active Vertices

In 3D LED simulations, the number of active vertices can grow significantly when the electrical mesh is refined. This directly implies that the number of starting rays for raytracing increases significantly because that is a direct function of the number of active vertices.

Consequently, the resultant raytree is an exponential function of the number of starting rays, and this results in a very large raytracing problem, which can derail the simulation time and, in part, the memory usage (since the compact memory model declares storage arrays of the size of the number of active vertices).

The solution is to group the active vertices into clusters, with each cluster serving as a distributed source of starting rays for raytracing.

Three possible strategies of clustering the active vertices have been implemented.

Plane Area Cluster

Users select the total number of clusters to be generated. Then, this number is translated into equal area zones (also taking into account the aspect ratio) of the automatically detected QW plane. The active vertices are subsequently grouped into each of these zones such that each zone forms a cluster. The algorithm for plane area clustering is described as follows:

Assume you input the required cluster size, N_c . The aim is to fit as many *squarish* elements (of size $d \times d$) into the QW plane area (size XY) as possible, as shown in [Figure 67](#).

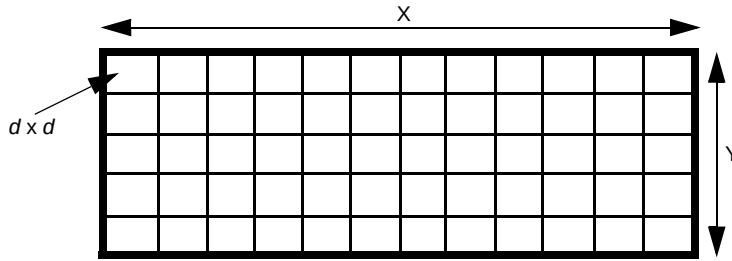


Figure 67 Fitting as many squarish elements of size $d \times d$ into QW plane

The constraints of the problem are:

$$N_c = \frac{XY}{d^2} \quad (930)$$

$$X = N_x d, N_x \text{ is an integer} \quad (931)$$

$$Y = N_y d, N_y \text{ is an integer} \quad (932)$$

Solving gives:

$$N_x = (\text{INTEGER}) \sqrt{(X/Y)N_c} \quad (933)$$

$$N_y = (\text{INTEGER})(Y/X)N_x \quad (934)$$

Finally, the adjusted cluster size becomes:

$$N_c' = N_x \times N_y \quad (935)$$

If no active vertices fall within a plane area segment, that segment is not added to the list of clusters. Therefore, you may see a smaller number of clusters than N_c' .

Nodal Clustering

A recursive algorithm is used to calculate how to group the active vertices for each cluster, such that each cluster receives, more or less, the same number of active vertices. The algorithm alternates the x- and y-coordinate partitioning of the list of active vertices, in an attempt to group a cluster of nearest neighboring active vertices. Unfortunately, this may not result in an even spatial distribution of clusters, which is a disadvantage of this method.

Optical Grid Element Clustering

The number of clusters cannot be set by users as it is defined by the number of optical grid elements. Active regions must be defined in both the electrical and optical grids. An effective bulk region in the optical grid is used to describe the active QW layers and can be meshed according to user requirements. Then, the electrical active vertices are grouped inside each of the optical grid active elements such that each optical-grid active element forms a cluster. In this way, you can control the distribution of the clusters for greater modeling flexibility.

NOTE In all the above clustering methodologies, the center of each cluster is determined by the average of the active vertices that it encloses.

Using the Clustering Feature

The following keywords in the LED framework activate the clustering feature:

```
Physics {...
  LED ( ...
    Raytrace (...
      ClusterActive(
        ClusterQuantity = Nodes | PlaneArea | OpticalGridElement
        NumberOfClusters = <integer>      # for Nodes | PlaneArea
      )
    )
  )
}
```

Debugging Raytracing

Rays are assumed, by default, to irradiate isotropically from each active vertex. In the case of quantum wells, an artifact of this assumption may cause unrealistic spikes in the radiation pattern. Consider those source rays that are directed within the plane of the quantum wells. These rays will mostly transmit out of the device at the ending vertical edges of the quantum wells. Realistically, the rays would have a much higher probability of being absorbed and re-emitted into another direction than traversing the entire plane of the quantum well.

To circumvent this problem, use the anisotropic emission as described in [Anisotropic Starting Rays from Spontaneous Emission Sources on page 825](#). Alternatively, one can try to exclude the source rays emitting within a certain angular range from the horizontal plane, that is, the plane of the quantum well. The power from these excluded rays will be distributed equally to the rest of the rays that are not within this angular range. This results in an approximate anisotropic emission shape at each active vertex. The syntax for this exclusion is:

```
Physics { ...
  LED ( ...
    Optics ( ...
      RayTrace ( ...
        ExcludeHorizontalSource(<float>)  # in degrees
      )
    )
  )
}
```

To add more flexibility to LED raytracing, debugging features are implemented. These include:

- Setting a fixed observation center for the LED radiation calculations.
- Fixing a constant wavelength for raytracing.
- Allowing you to print and track the LED radiation rays (in a certain angular zone) back to its source active vertex.

These features are described in this syntax:

```
Physics { ...
  LED ( ...
    Optics ( ...
      RayTrace ( ...
        ObservationCenter = (<float> <float>)
                                # in micrometers, 3 entries for 3D
        Wavelength = 888        # [nm] set fixed wavelength
        DebugLEDRadiation(<filename> <StartAngle> <EndAngle>
                           <MinIntensity>)
      )
    )
  )
}
```

```
)  
}
```

Print Options in Raytracing

The original `Print` feature of raytracing causes all ray paths to be printed. With multiple reflections or refractions and a large number of starting rays, the resultant image can become a black smudge. To reduce the number of ray paths that are printed, a `Skip` option is implemented within the `Print` feature. In addition, you can trace the ray paths originating from only a single active vertex. These features are described in this syntax:

```
Physics {  
  LED (  
    Optics (  
      RayTrace (  
        Print(Skip(<int>))          # skip printing every <int> ray paths  
        Print(ActiveVertex(<int>)) # print only rays from active vertex  
                                   # <int>  
        PrintSourceVertices(<filename>)  
        ProgressMarkers = <int>    # 1-100% intervals (integer)  
      )  
    )  
  )  
}
```

The option `PrintSourceVertices(<filename>)` outputs the list of active vertices, their global index numbering, and coordinates into the file specified by `<filename>`. If the number of source rays is large or the optical mesh is fine, raytracing takes some time to be completed. In this case, you can set the incremental completion meter by the keyword `ProgressMarkers = <int>`.

The `Print` option only outputs rays as lines with no other information. To obtain a raytree that contains intensity and other information, you can plot the raytree using the keyword `RayTrees` in the `Plot` section of the command file:

```
Plot {...  
  RayTrees  
}
```

The resultant raytree is plotted in TDR format and can be visualized by Tecplot SV, where each branch of the raytree can be accessed individually.

Interfacing LED Starting Rays to LightTools®

To facilitate the rapid design of LED structures with its luminaire, the starting rays from active region vertices can be output directly to a ray file formatted for use in LightTools. [Figure 68](#) shows a probable design flow.

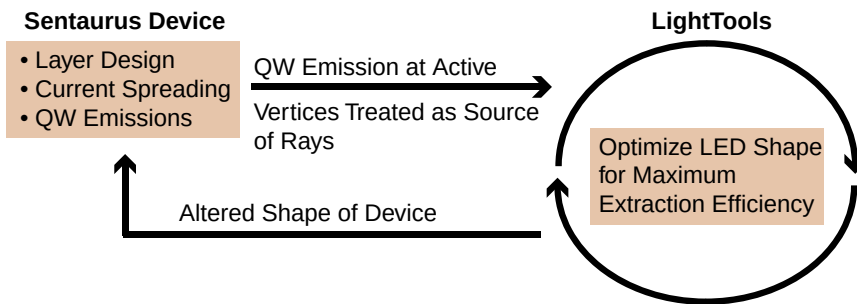


Figure 68 LED design flow to optimize extraction efficiency by LightTools and coupled to Sentaurus Device LED device simulation

The set of rays is derived from the LED spontaneous emission power spectrum at each vertex of the device. This means that there is wavelength variability such that each starting ray carries a starting position, a direction, an intensity value, and a wavelength.

This feature is an extension of the `Disable` keyword, so that the internal Sentaurus Device raytracing engine will not be activated. The syntax is:

```

File {...
    Plot = "n99_des.tdr"
}

Physics {...
    LED (
        RayTrace(
            Disable(
                OutputLightToolsRays (
                    WavelengthDiscretization = <integer>    # spectrum discretization
                    RaysPerCluster = <integer>              # rays emitting from each
                                                            # active cluster
                    IsotropyType = InBuilt | Random | UserRays # default is InBuilt
                    SaveType = Ascii | Binary                # choose ASCII or binary format
                )
            )
        )
        ClusterActive()
        RaysPerVertex = <integer>                          # used with SourceRaysFromFile()
        SourceRaysFromFile(string)
    )
}
  
```

35: Light-Emitting Diodes

LED Raytracing

```
Solve {...  
  Plot( Range=(0,1) Intervals=5 )  
}
```

This new feature can be used in conjunction with the `ClusterActive` section, so that the active vertices can be grouped into clusters to reduce the final number of rays. If the `ClusterActive` section is not present, every active vertex will be used as emission centers for the starting rays. It is also possible to import an isotropic distribution of point source rays from a file to be used for random-rotated distribution at different vertices. The span of the spectrum at each active cluster is computed automatically and divided into the numbers as specified by `WavelengthDiscretization`. The total number of starting rays is:

$$\text{WavelengthDiscretization} * \text{RaysPerCluster} * \text{NumberOfActiveClusters}$$

Two types of ray file format for LightTools can be chosen: ASCII or binary. The base name for the LightTools ray files is derived from the plot file name and is appended with either `_lighttools.txt` for the ASCII format or `_lighttools.ray` for the binary format.

For example, according to the above syntax, the following are the corresponding file names for the LightTools ray files:

```
n99_000000_des_lighttools.txt  
n99_000001_des_lighttools.txt  
...  
n99_000000_des_lighttools.ray  
n99_000001_des_lighttools.ray  
...
```

These files can be read directly by LightTools as volume sources of rays (refer to the LightTools manual for more information).

Example: n99_000000_des_lighttools.txt

This example is an ASCII-formatted LightTools ray file (version 2.0 format):

```
# Synopsys Sentaurus Device to LightTools  
# Ray Data Export File  
LT_RDF_VERSION: 2.0  
DATANAME: SentaurusDeviceRayData  
LT_DATATYPE: radiant_power  
LT_RADIANT_FLUX: 1.946806e-04  
LT_FAR_FIELD_DATA: NO  
LT_COLOR_INFO: wavelength  
LT_LENGTH_UNITS: micrometers  
LT_DATA_ORIGIN: 0 0 0  
LT_STARTOFDATA  
0.000000e+00 4.290000e-01 0.000000e+00 -0.163343 -0.986569 0.000000
```



```
5.451892e-05 470.395656
0.000000e+00 4.290000e-01 0.000000e+00 -0.163343 -0.986569 0.000000
1.817405e-04 459.664919
....
LT_ENDOFDATA
```

The first three columns denote the starting position (in micrometers) of the ray, the next three columns show the direction vector, and this is followed by the fractional power (as a fraction of LT_RADIANT_FLUX), and ends with the wavelength (in nanometers).

LED Radiation Pattern

Raytracing does not contain phase information, so it is not possible to compute the far-field pattern for an LED structure. Instead, the outgoing rays from the LED raytracing are used to produce the radiation pattern.

In 2D space, this is equivalent to moving a detector in a circle around the LED as shown in [Figure 69](#).

In 3D space, the detector is moved around on a sphere. The circle and sphere have centers that correspond to the center of the device. Sentaurus Device automatically determines the center of the device to be the midpoint of the device on each axis. Nonetheless, there is an option to allow you to set the center of observation (see [Debugging Raytracing on page 831](#) and [Table 221 on page 1290](#)).

To examine the optical intensity inside the LED, use RayTraceIntensity in the Plot statement.

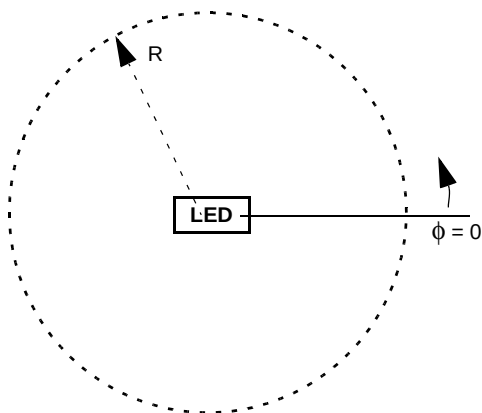


Figure 69 Measuring the radiation pattern in a circular path around LED at observation radius, R

35: Light-Emitting Diodes

LED Radiation Pattern

The syntax required to activate and plot the LED radiation pattern is located in the `File`, `Physics-LED-Optics-RayTrace`, and `Solve-quasistationary` sections of the command file:

```
File {...
  # ----- Activate LED radiation pattern and save -----
  LEDRadiation = "rad"
}
...
Physics {...
  LED (...
    Optics (...
      RayTrace(...
        LEDRadiationPara(1000.0,180) # (radius_micrometers, Npoints)
        ObservationCenter = (2.0 3.5 5.5) # set center
      )
    )
  )
}
...
Solve {...

  # ----- Specify quasistationary -----
  quasistationary (...

    PlotLEDRadiation { range=(0,1) intervals=3 }

    Goal {name="p_Contact" voltage=1.8}
    {...}
  }
}
```

The LED radiation plot syntax works in the same way as the `GainPlot` (see [Plotting Gain and Power Spectra on page 919](#)) and the `OptFarField` plot (see [Far Field on page 878](#)) in the `Quasistationary` statement.

An explanation of this example is:

- The base file name, "rad", of the LED radiation pattern files is specified by `LEDRadiation` in the `File` section. The keyword `LEDRadiation` also activates the LED radiation plot.
- The parameters for the LED radiation plot are specified by the keyword `LEDRadiationPara` in the `Physics-Optics-RayTrace` section. You must specify the observation radius (in micrometers) and the discretization of the observation circle (2D) or sphere (3D).

- You can set the center of observation by including the keyword `ObservationCenter`. If this is not set, the center is automatically computed as the middle point of the span of the device on each axis.
- The LED radiation pattern can only be computed and plotted within the `Quasistationary` statement. The keyword `PlotLEDRadiation` controls the number of LED radiation plots to produce.
- The argument `range=(0,1)` in the `PlotLEDRadiation` keyword is mapped to the initial and final bias conditions. In this example, the initial and final (goal) `p_Contact` voltages are 0 V and 1.8 V, respectively. The number of `intervals=3`, which gives a total of four ($= 3+1$) LED radiation plots at 0 V, 0.6 V, 1.2 V, and 1.8 V. In general, specifying `intervals=n` produces $(n+1)$ plots.
- If the LED structure is symmetric, the LED radiation is only computed on a semicircle.

The following sections briefly describe the files that are produced in the LED radiation plot for the 2D and 3D cases.

Two-dimensional LED Radiation Pattern and Output Files

Activating the LED radiation plot for a 2D LED simulation produces two different files (using the base name "rad"):

<code>rad_000000_LEDRad.plt</code>	The normalized radiation pattern versus observation angle, which can be viewed in Inspect.
<code>rad_000000_LEDRad_Polar0.tdr</code>	The normalized radiation pattern projected onto a grid file and can be viewed in Tecplot SV. The polar plot of the LED radiation pattern is then shown.

A sample output of the radiation plot of a 2D nonsymmetric LED structure is shown in [Figure 70 on page 838](#). The lower-left image corresponds to the file `rad_000000_LEDRad.plt` plotted by Inspect, and the right image is the product of the file `rad_000000_LEDRad_Polar.tdr` plotted by Tecplot SV.

35: Light-Emitting Diodes

LED Radiation Pattern

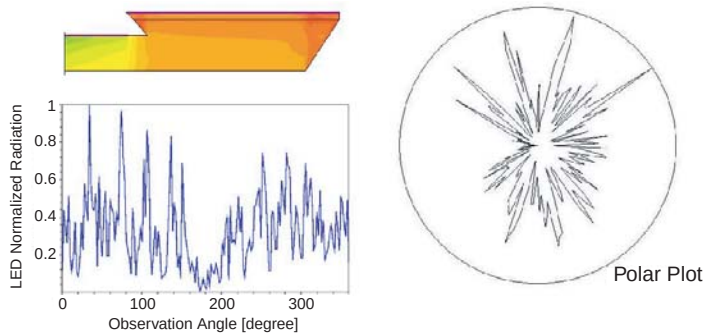


Figure 70 (Upper left) LED internal optical intensity, (lower left) normalized radiation intensity versus observation angle, and (right) polar radiation plot computed by Sentaurus Device in 2D LED simulation

Three-dimensional LED Radiation Pattern and Output Files

There are two output files for the radiation pattern in the case of a 3D LED simulation:

`rad_000000_des.tdr`

Data file containing the normalized radiation pattern. Use Tecplot SV for this file, and the spherical plot of the 3D LED radiation pattern is then shown. A sample of the radiation pattern of a 3D LED simulation is shown in [Figure 71 on page 839](#).

`rad_000000_des_3Dslices.plt`

The 3D radiation pattern is extracted into polar plots on three planes that can be visualized using Inspect:

- XYplane: $\theta = \pi/2$; plot far field for $\phi = 0$ to 2π .
- XZplane: $\phi = 0, \pi$; plot far field for $\theta = 0$ to 2π .
- YZplane: $\phi = \pi/2, 3\pi/2$; plot far field for $\theta = 0$ to 2π .

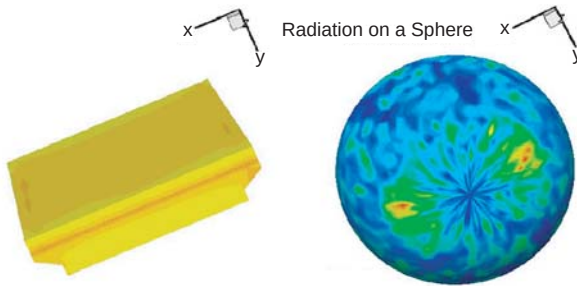


Figure 71 (Left) LED internal optical intensity and (right) normalized radiation intensity projected on a sphere of 3D LED simulation

Staggered 3D Grid LED Radiation Pattern

In discretizing a 3D far-field collection sphere with regular longitudinal and latitude lines, the size of each surficial element critically depends on its location on the sphere. At the polar regions, the elements are very small compared to those along the equator. As the discretization increases, the disparity between the polar and equator surficial elements increases at the same rate.

The sampling rate for rays is constrained by the smallest collecting element, which, in this case, is the much smaller surficial element at the poles. In most cases, the polar regions are undersampled and the equator regions are oversampled due to the disparity in surficial element areas between the two regions. Undersampling at the poles also could lead to anomalous spikes in the far field since the far-field intensity is computed by the total ray power impinging that element and is divided by the elemental area. To adequately sample the polar regions, a significant number of rays must be used, resulting in a very large raytracing problem that may not be necessary.

To circumvent the abovementioned problems, the spherical collection space must be composed of more uniformly distributed elemental areas. The baseline requirement is that each elemental area on the surface of the collection sphere must be approximately the same size. The following simple approach is used:

1. Divide the sphere into concentric rings in the latitude planes. Each ring has a thickness of:

$$d_{\theta} = \int R d\theta = R \Delta\theta \quad (936)$$

2. For each concentric ring, further subdivide the ring into elements with a width that is approximately $d\theta$.

Mathematically, assume that the sphere is divided into N_θ rings, so that the thickness of each ring is:

$$d_\theta = R \left(\frac{\pi}{N_\theta} \right) \quad (937)$$

In each concentric ring (i) bound between θ_1 and θ_2 , choose the order of θ such that:

$$\sin(\theta_2) > \sin(\theta_1) \quad (938)$$

The circumference at θ_2 is $C_{\theta_2} = R \sin(\theta_2) \times 2\pi$. The constraint needed here is such that:

$$\frac{C_{\theta_2}}{N_\phi(i)} = d_\theta \quad (939)$$

where $N_\phi(i)$ is the number of divisions for the concentric ring (i) and is an integer. Solving gives:

$$N_\phi(i) = (\text{INTEGER})[2\sin(\theta_2) \times N_\theta] \quad (940)$$

At the poles of the sphere ($\theta = 0$ and $\theta = \pi$), the concentric rings collapse into a cap. The following constraint is imposed for these polar cap regions (essentially trying to match triangular area elements with squarish area elements):

$$0.5 \times \left(\frac{C_{\theta_2}}{N_\phi(i)} \right) \times d_\theta = d_\theta^2 \quad (941)$$

Simplifying gives the number of divisions for the polar cap rings:

$$N_\phi(i) = (\text{INTEGER})[\sin(\theta_2) \times N_\theta] \quad (942)$$

The total number of elemental areas is $\sum N_\phi(i)$.

[Table 128](#) lists the total number of elements obtained from this simple approach.

Table 128 Total number of elements and area information as a function of N_θ

N_θ	Total number of surface elements	Smallest area (unit radius)	Largest area (unit radius)	Area skew = ratio of largest/smallest area
5	36	0.314159	0.399994	1.273
10	146	0.074372	0.097081	1.305
20	554	0.017705	0.024573	1.388
50	3296	0.002857	0.003945	1.381
100	12976	0.000715	0.000987	1.380

Table 128 Total number of elements and area information as a function of N_θ

N_θ	Total number of surface elements	Smallest area (unit radius)	Largest area (unit radius)	Area skew = ratio of largest/smallest area
150	29013	0.000318	0.000439	1.381
200	51426	0.000179	0.000247	1.380

As N_θ increases, it is clear from [Table 128](#) that the area skew (ratio of largest to smallest elemental area) stabilizes to a value of approximately 1.38.

To activate the new staggered 3D far-field grid, use the syntax:

```
Physics {
  LED (
    RayTrace(
      Staggered3DFarfieldGrid
    )
  )
}
```

If the keyword `Staggered3DFarfieldGrid` is not specified, the collection sphere reverts to the old (θ, ϕ) scheme to maintain backward compatibility.

Spectrum-dependent LED Radiation Pattern

Unlike a laser beam with single-frequency emissions, rays emitting from an LED carry a spectrum of frequencies (or energies). Sentaurus Device monitors the spectrum of each ray as it undergoes the process of raytracing in and out of the device. The resultant spectrum of the LED radiation pattern can then be plotted.

To activate this feature, include the keyword `LEDspectrum` in the command file:

```
Physics {...
  LED (...
    Optics (...
      RayTrace(...
        LEDspectrum(<startenergy> <endenergy> <numpoints>)
      )
    )
  )
}
```

This feature must be used in conjunction with the `LEDRadiation` feature so that the file names of the LED radiation plots and the observation angles can be specified. Other notable aspects of the syntax are:

- `<startenergy>` and `<endenergy>` give the energy range of the spectrum to be monitored. These parameters are floating-point entries with units of eV.
- `<numpoints>` is an integer determining the number of discretized points in the specified energy range.

Tracing Source of Output Rays

Optimizing the extraction efficiency is critical in an LED design. Other than modifying the shape of the device, you can use the fact that the rays originating from certain zones in the active region have a higher escape or extraction rate. By designing the shape of the contact to channel higher currents into these zones, the extraction efficiency can effectively be increased.

To facilitate the identification of such zones, a feature that correlates the output rays to their source active vertices is implemented. The intensity of each output ray is added to an `LED_TraceSource` variable at each active vertex. At the end of the simulation, a profile of `LED_TraceSource` is obtained, which provides an indication of the best localized regions to which more current can be channeled.

The activation of this feature requires keywords to be inserted into two different statements of the command file:

```
Plot { ...
    LED_TraceSource
}

Physics { ...
    LED ( ...
        Optics(
            RayTrace ( ...
                TraceSource()
            )
        )
    )
}
```

NOTE A far-field observation radius must be set for the `TraceSource` feature.

Nonactive Region Absorption (Photon Recycling)

The default setting for LED simulations includes the contribution of absorbed photons in nonactive regions to the continuity equation as a generation rate. This is the basic concept of the nonactive-region photon recycling. In most cases, LEDs are designed with larger bandgap material in nonactive regions to eradicate intraband absorption.

As a result, it may not be necessary to include very small values of nonactive absorption in some situations. To allow for such flexibility, nonactive absorption can be switched off as an option. The syntax is:

```
Physics { ...
  LED ( ...
    Optics ( ...
      RayTrace ( ...
        NonActiveAbsorptionOff
        OptGenScaling = <float>
        BackgroundOptGen(<float>)    # [# /cm^3]
      )
    )
  )
}
```

If the keyword `NonActiveAbsorptionOff` is used, the plot variable `OpticalGeneration` still show values, but these values are not included in the continuity equation. A scaling factor, `OptGenScaling`, can be defined to scale the final value of optical generation. You also can set a background optical generation with the keyword `BackgroundOptGen`.

Interface to LightTools

LightTools is a robust raytracer that accepts a list of source rays as input, and it can optimize the packaging design for LEDs.

An interface has been built in Sentaurus Device to output rays from an LED device simulation that can be input into LightTools as source rays. This feature requires the LED radiation plot to be activated simultaneously so that the rays can be output at various quasistationary and transient states. The syntax for activating this feature is:

```
Physics { ...
  LED ( ...
    Optics ( ...
      RayTrace ( ...
        LEDRadiationPara(...)
        PrintORArays(
```

35: Light-Emitting Diodes

Interface to LightTools

```
        FileName = "string"
        SplitSpectrumRays = integer
        RefractiveIndexDelta = float
        NumberOfSpectralPoints = integer
        RangeOfSpectrum = (float float)
        VectorUnits = Micron      # or Millimeter
    )
)
)
}
```

Each output ray is allowed to be split into several rays indicated by the parameter `SplitSpectrumRays`. These split rays will be given the weights of the spectrum sampled at different wavelengths. If a wavelength-dependent refractive index has been specified, the direction of each split ray will be perturbed by a small change in angle, given by the following formula:

$$d\theta = -\left(\frac{dn}{n}\right) \cdot \tan\theta \quad (943)$$

This formula is derived by taking small changes in Snell's law: $n \cdot \sin\theta = \text{constant}$. If the refractive index is not wavelength dependent, the wavelength changes will be a fraction of the parameter `RefractiveIndexDelta`.

NOTE In Sentaurus Device, raytracing and wavelength dependency of the refractive index can only be specified by using the complex refractive index model.

The other parameters of `NumberOfSpectralPoints` and `RangeOfSpectrum` define the output spectrum to be printed.

Two specific files are produced: one with the extension `_rays.txt` containing the ray information, and one with the extension `_spectrum.txt` containing the spectrum information.

The ray information consists of the position vector (x,y,z), the direction cosine (x,y,z), and the relative intensity. The spectrum information is indicated by the wavelength and relative intensity. Both files contain identifying headers that are required by LightTools. Sample formats of the two files are presented.

If the compact memory LED raytracing is activated, only a limited set of output rays can be reconstructed because the raytrees are not stored. Nonetheless, the output rays should be able to reproduce the far-field radiation pattern.

Example: orainteface_rays.txt

```
# Synopsys Sentaurus Device to LightTools
# Ray Data Export File
DATANAME: SentaurusDeviceRayData
LT_FAR_FIELD_DATA: NO
LT_DATA_ORIGIN:          0          0          0
LT_RADIANT_FLUX:         1.000
LT_STARTOFDATA
-35.3165701  -0.8510182  11.5477636  0.7179369  0.5564080  -0.4183022
6.2207040e-08
-35.3165701  -0.8510182  11.5477636  0.9423966  -0.0113507  -0.3343049
4.6799284e-08
...
LT_ENDOFDATA
```

Example: orainteface_spectrum.txt

```
# Synopsys Sentaurus Device to LightTools
# Aggregate Ray Spectrum Data Export File
DFAT 1.0
DATANAME: SentaurusDeviceSpectrum
CONTINUOUS
RADIOMETRIC
8.3369229e+02 0.0952224
8.4831847e+02 0.1650206
8.6346702e+02 0.2883805
8.7916642e+02 0.4638275
8.9544728e+02 0.6425363
9.1234251e+02 0.8041440
9.2988755e+02 1.0000000
9.4812064e+02 0.8919842
9.6708306e+02 0.5533345
9.8681945e+02 0.0430633
...
```

Device Physics and Tuning Parameters

Unlike a laser diode, an LED does not have a threshold current. Therefore, the carriers in the active region are not limited to any threshold value. This means that the spontaneous gain spectrum continues to grow as the bias current increases. The limiting factor for growth is when the QW active region is completely filled and leakage current increases significantly, or dark recombination processes start to dominate with increasing bias and temperature.

There are four main design concerns for an LED:

- Designing the basic layers to optimize the internal quantum efficiency. Polarization charge sheets are needed for AlGaIn–GaIn–InGaIn interfaces, and junction tunneling models might be needed for thin electron-blocking layers. For mature QW technology such as InGaAsP systems, there can be predictive value in advanced $k \cdot p$ gain calculations for optical gain. However, for difficult-to-grow materials such as InGaIn/AlGaIn QWs, the uncertainties of growth, mole fraction grading, interface clarity, and so on, make predictive modeling of optical gain almost impossible.
- Extraction efficiency. This is mainly a problem of the geometric shape of the LED structure and the complex refractive index profile of the structure. Temperature, wavelength, and carrier distribution can alter the complex refractive index profile, so the extraction efficiency changes with increasing current injection. Many LED structures have tapered sidewalls to help couple more light out of the device. The slope of the taper can be set as a parameter using Sentaurus Workbench, and the automatic parameter variation feature can be used to optimize the extraction efficiency of the LED geometry.
- Current spreading. It is desirable to spread the current uniformly across the entire active region so that total spontaneous emissions can be increased. In Sentaurus Device, there is an option to switch off raytracing in an LED simulation. Switching off raytracing only forgoes the extraction efficiency and radiation pattern computation; the total spontaneous emission power is still calculated. This can assist you in the faster optimization of an LED device for uniform current spreading.
- Thermal management, hot spots, how much heat is produced and how to cool the device. Temperature distribution also affects the mobility and complex refractive index profile and, therefore, impacts the current spreading profile and optical extraction efficiency.

Example of 3D GaN LED Simulation

An LED simulation is best run with a dual-grid approach. The electrical grid must be dense in vicinities where carrier transport details are important. On the other hand, a coarse grid is needed for raytracing. The following is a skeletal sample of command file syntax for a dual-grid 3D GaN LED simulation. Only highlights of the syntax that are important to LED simulations have been included.

The typical command file syntax is:

```
#####  
## Global declarations ##  
#####  
File {...}  
  
Math {  
    Digits = 5  
    NoAutomaticCircuitContact
```

```

DirectCurrent
Method = blocked
# ILS should be chosen for big 3D simulations
# For 2D, use Pardiso
Submethod=ILS(set= 5)
  ILSrc= "
    set (5) {
      iterative(gmres(100), tolrel=1e-11, tolunprec=1e-4, tolabs=0,
        maxit=200);
      preconditioning(ilut(1e-9,-1), right);
      ordering(symmetric=nd, nonsymmetric=mpsilst);
      options(compact=yes, linscale=0, fit=5, refinebasis=1,
        refineresidual=30, verbose=5);
    }; "
Derivatives
Notdamped=20
Iterations=15
RelErrControl
ErReff(electron)=1e7
ErReff(hole)=1e7
ElementEdgeCurrent
ExtendedPrecision
DualGridInterpolation ( Method=Simple )
NumberOfThreads = maximum
Extrapolate
}

#####
## Define the optical solver part ##
#####
OpticalDevice optDevice {
  File {
    Grid = "optic_msh.tdr"
    Parameters = "optparafile.par"
  }
  RaytraceBC {...}
  Physics {
    ComplexRefractiveIndex (
      WavelengthDep (real imag)
      TemperatureDep(real)
    )
  }
  Physics(Region="EffectiveQW") { Active }
}

#####
## Define the electronic solver part ##
#####
Device elDevice {

```

35: Light-Emitting Diodes

Device Physics and Tuning Parameters

```
Electrode {...}

Thermode {...
  { Name="T_contact" Temperature=300.0 SurfaceResistance=0.05 }
}

File {
  Grid = "elec_msh.tdr"
  Parameters = "elecparafile.par"
  Current = "elsolver"
  Plot = "elsolver"
  LEDRadiation = "farfield"
  Gain = "gainfilename"
}

# ----- Choose special LED related plot variables to output -----
Plot {...
  RayTraceIntensity
  OpticalGeneration
  LED_TraceSource
  OpticalAbsorptionHeat
# RayTrees          # not for compact memory option
}

# ----- Specify gain plot parameters -----
GainPlot { ... }

Physics {
  Mobility ()
  EffectiveIntrinsicDensity (NoBandGapNarrowing)
  AreaFactor = 1
  IncompleteIonization
  Thermionic
  Fermi
  RecGenHeat

  OpticalAbsorptionHeat(
    Scaling = 1.0
    StepFunction( EffectiveBandgap )
  )

  # Complex refractive index model needed for raytracer
  ComplexRefractiveIndex (
    WavelengthDep (real imag)
    TemperatureDep (real)
  )

  LED (
```

```

SponScaling = 1    * scale matrix element with this factor
Optics (
    RayTrace(
#        Disable      # this is for purely electrical investigation
        CompactMemoryOption    # use compact memory model
        Coordinates = Cartesian
        Staggered3DFarfieldGrid

        # ----- Set ray starting and terminating conditions -----
        PolarizationVector = Random
        RaysPerVertex = 20
        RaysRandomOffset (RandomSeed = 123)
        Depthlimit = 100
        MinIntensity = 1e-5

        # ----- Set output options -----
        TraceSource()
            LEDRadiationPara(10000,60)    # (<radius-microns>, Npoints)

        # Choose LED wavelength option
        # Fixed wavelength by entering a <float>, units in [nm]
        Wavelength=AutoPeak    # or Effective or AutoPeakPower or <float>
    )
)
# ----- Quantum well options -----
    QWTransport
    QWExtension = autodetect
    Strain
    Broadening = 0.04
    Lorentzian
)

# Turn on tunneling for electron blocking layer if necessary
# eBarrierTunneling "rline1" ()

}

# ----- Define recombination models for non-active regions -----
Physics (material = "AlGaN") { Recombination (SRH(TempDep) Radiative) }
Physics (material = "GaN") { Recombination (SRH(TempDep) Radiative) }

# ----- Set QWs as active regions ----
Physics (region="QW1") { Recombination(-Radiative SRH(TempDep)) Active }
...
Physics (region="QW6") { Recombination(-Radiative SRH(TempDep)) Active }

# ---- Include polarization sheet charges between interfaces ----
Physics (RegionInterface="Window/Cap") {

```

35: Light-Emitting Diodes

Device Physics and Tuning Parameters

```
    Traps(FixedCharge Conc=-2.666746e+12)
  }
  Physics (RegionInterface="Barrier6/Buffer") {
    Traps(FixedCharge Conc=-1e+12)
  }
  Physics (RegionInterface="Barrier0/Window") {
    Traps(FixedCharge Conc=3.8e+12)
  }
  Physics(RegionInterface="Barrier0/QW1") {Traps((FixedCharge Conc=1e12))}
  Physics(RegionInterface="QW1/Barrier1") {Traps((FixedCharge Conc=-1e12))}
  ...

  Math {
    # ---- Define tunneling nonlocal line for electron blocking layer ----
    NonLocal "rline1" (
      Barrier(Region = "Window")
    )
  }
}

#####
## Define mixed-mode system section for dual grid simulation ##
#####
System {
  elDevice d1 ( anode=vdd cathode=gnd ) { Physics { OptSolver="opt" } }
  Vsource_pset drive(vdd gnd) { dc = 2.72 }
  Set ( gnd = 0.0 )
  optDevice opt ()
}

#####
## Define solving sequence ##
#####
Solve {
  Coupled (Iterations = 40) { Poisson }
  Coupled (Iterations = 40) { Poisson Electron Hole }
  Coupled { Poisson Electron Hole Contact Circuit }
  Coupled { Poisson Electron Hole Contact Circuit Temperature }
  Quasistationary (
    InitialStep = 0.05
    MaxStep = 0.05
    Minstep = 5e-3
    Plot { range=(0,1) intervals=5 }
    PlotGain { range=(0,1) intervals=5 }
    PlotLEDRadiation { range=(0,1) intervals=5 }
    Goal { Parameter=drive.dc Value=3.8 }
  ) {
    # Full self-consistent electrical+thermal+optics simulation
    Plugin(breakonfailure) {
```



```

        Coupled(Iterations = 20) {Electron Hole Poisson Contact Circuit
                                Temperature}
    Optics
}
}
}

```

Some comments about the command file syntax:

- Multithreading for the raytracer can be activated by specifying `NumberOfThreads` in the global `Math` section. A value of maximum has been chosen in this case so that Sentaurus Device will use the maximum number of threads available on the machine.
- The solver must be ILS for 3D simulations, due to the large matrix that needs to be solved. The parameters for ILS must be tuned for optimal convergence.
- `ExtendedPrecision` is recommended for use in GaN device simulations.
- Various special raytrace boundary conditions can be chosen and are set in the `RayTraceBC` section. For details about these special boundary contacts, see [Boundary Condition for Raytracing on page 520](#).
- Polarization charges at AlGaIn–GaIn–InGaIn interfaces are added using fixed trap charges.
- The wavelength used in raytracing is computed automatically to be the wavelength at the peak of the spontaneous emission spectrum. If a fixed wavelength is required, you have the option of setting it by using the `WaveLength` keyword.
- The QW regions must be labelled as `Active` in both the optical and electrical device declarations so that proper internal mapping can be performed. You also must include the keyword `Recombination(-Radiative)` in the QW active regions because the spontaneous emission computation in these regions uses the special LED model and, therefore, the default radiative recombination model should be switched off.
- The `ComplexRefractiveIndex` model must be used in conjunction with the raytracer. In addition, a `PolarizationVector` must be defined with the raytracer.
- When `CompactMemoryOption` is chosen, the raytrees are not saved internally, so plotting the `RayTrees` or using `Print(Skip(<integer>))` is disabled.
- Activating the nonlocal tunneling model requires the keyword `eBarrierTunneling` in the `Physics` section and the `NonLocal` line definition in the `Math` section (see [Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612](#) for more details).
- The `Plugin` feature is used to create the full self-consistent simulation framework between the electrical, thermal, and optics. To disengage full self-consistency, remark off the `Plugin` and `Optics` statements, in which case, the `Optics` will only be solved once at each bias step.
- The dual-grid electrical and optical feature has been used. If you select a single-grid simulation, you only need to copy the `Physics-LED` section of this example and insert it into the `Physics` section of a typical single-grid command file.

NOTE Raytracing can be disabled in an LED simulation by using the keyword `Disable` in the `RayTrace` statement if you do not require the computation of the extraction efficiency and radiation pattern.

[Table 221 on page 1290](#) lists all the arguments for the LED RayTrace option.

References

- [1] W.-C. Ng and M. Pfeiffer, “Generalized Photon Recycling Theory for 2D and 3D LED Simulation in Sentaurus Device,” in *6th International Conference on Numerical Simulation of Optoelectronic Devices (NUSOD-06)*, Singapore, pp. 127–128, September 2006.
- [2] W.-C. Ng and G. Létay, “A Generalized 2D and 3D White LED Device Simulator Integrating Photon Recycling and Luminescent Spectral Conversion Effects,” in *Proceedings of SPIE, Light-Emitting Diodes: Research, Manufacturing, and Applications XI*, vol. 6486, February 2007.

CHAPTER 36 Optical Mode Solver for Lasers

This chapter describes various methods and boundary conditions used to compute the optical modes of lasers.

These methods include the finite-element method, the transfer matrix method, and the effective index method. Far-field derivation from the near-field pattern and methods for automatic mode searching are also described.

Overview

In a laser simulation, the optics problem is challenging. The refractive index distribution in a laser structure changes with temperature and carrier density (plasma effect). In Sentaurus Device, the refractive index is taken from a corresponding parameter file and wavelength dispersion of the refractive indices can also be taken into consideration (see [Refractive Index, Dispersion, and Optical Loss on page 789](#)). To handle such inhomogeneous refractive index geometries, the finite-element method was selected as the core method to discretize and solve the optics problem. In addition, there are other approximate methods such as the effective index method, transfer matrix method, and raytracing. These additional methods allow you to run the simulation faster, but with some loss of accuracy. They are described in detail in the following sections.

The resulting quantities of interest from the optics solution are the optical mode profiles, optical loss, and resonant wavelength. In a Fabry–Perot cavity for an edge-emitting laser, the wavelength is determined by the peak of the material gain from the electrical simulation. In other cases such as distributed feedback (DFB) lasers, distributed Bragg reflector (DBR) lasers, and VCSELs, the wavelength is determined by the shape and design of the cavity structure.

The optics problem can be iterated self-consistently with the electrical problem by Gummel iteration. Referring to [Simulating Single-Grid Edge-Emitting Laser on page 760](#), the self-consistent coupling is activated by using the keyword `Optics` in the `Plugin` statement:

```
Solve {...
  quasistationary (...
    Goal {name="p_Contact" voltage=1.6})
  {
    Plugin(BreakOnFailure){
      Coupled { Electron Hole Poisson QWeScatter QWhScatter PhotonRate }
```

```

    Optics
    Wavelength
  }
}

```

If the coupling between the optics and electronics is weak, for example, in an isothermal simulation and at low-current injection conditions, it is not necessary to iterate the optics problem with the electrical problem since the optical eigenmode is not expected to change greatly. In such a case, remove the `Optics` keyword from the `Plugin` statement, and the optical eigenmode is computed only once at the beginning of the simulation.

There are two types of optical problem in laser simulations: the waveguide problem for edge-emitting lasers and the resonant cavity problem, for example, in VCSELs. In the waveguide problem, essentially, the Helmholtz equation is solved, while the cavity problem solves the wave equation directly. These two types of problem are discussed in the next sections.

Finite-Element Formulation

The finite-element method is a standard variational approach for solving the electromagnetic wave equations [1]. The Ritz procedure is used to evaluate the functional:

$$F(\Psi) = \frac{1}{2} \langle \mathfrak{S}\Psi, \Psi \rangle - \langle \Psi, f \rangle - \langle f, \Psi \rangle \quad (944)$$

where $\mathfrak{S}$ is the linear operator and Ψ is a trial function. The governing differential equation is $\mathfrak{S}\Phi = f$. The wave equations in the waveguide and cavity problems are both of the form of the governing differential equation and, therefore, are well suited to the finite-element method.

For example, use the scalar Helmholtz equation for the waveguide problem to trace the formulation of the finite-element method. The Ritz procedure is applied to the scalar Helmholtz equation and gives the variational functional:

$$F(\Psi) = \frac{1}{2} \iint ((\partial_x \Psi)^2 + (\partial_y \Psi)^2 - k_0^2 \epsilon_r \Psi^2) d\Omega \quad (945)$$

The edge-emitting laser structure is discretized into finite elements, and the trial function Ψ is constructed by assuming weighted, linear basis functions on each edge of the finite element. These basis functions are commonly referred to as shape functions. In this case, the variables are the coefficients (weights) of the linear basis function on each edge. By the method of variation, the minimization of the functional $F(\Psi)$ gives the optimal solution, Ψ , for the finite-element discretization of the scalar Helmholtz equation. Therefore, the resolution of the discretization is important to determine the accuracy of the solution. As a general rule, 20 points per wavelength will provide an accurate solution for most structures.

To find the minimization of the functional $F(\Psi)$, take the first derivative of $F(\Psi)$ with respect to the weights of the linear basis function, $\{\Phi\}$, and set the derivative to zero. This yields a set of linear equations that can be cast into an algebraic, generalized, eigenvalue problem:

$$[A]\{\Phi\} = \epsilon_{\text{eff}}[B]\{\Phi\} \quad (946)$$

with:

$$[A] = \sum_{\text{allElements}} \left(k_0^2 \epsilon_{r,e} \int_{\Omega^e} \{L\} \cdot \{L\}^T d\Omega^e - \int_{\Omega^e} \{\nabla_t L\} \cdot \{\nabla_t L\}^T d\Omega^e \right) \quad (947)$$

$$[B] = \sum_{\text{allElements}} \int_{\Omega^e} \{L\} \cdot \{L\}^T d\Omega^e \quad (948)$$

where $\{L\}$ are the linear shape functions. The relative dielectric constant ϵ_r remains constant across the element e , and the integration occurs over the area of the element, Ω^e .

The sparse matrices $[A]$ and $[B]$ are derived from the assembly process. They are complex symmetric but nonhermitian. As the system is nonhermitian, it causes the eigenvalues ϵ_{eff} to be complex. $\{\Phi\}$ is the column vector of the unknown weights. After the weights, $\{\Phi\}$, have been solved, they are substituted back into the linear shape functions to recover the electric fields on the nodes of the grid.

The Jacobi–Davidson QZ algorithm is applied [2][3] to solve the sparse matrix generalized eigenvalue problem. This is an iterative solver, so an initial guess for the eigenvalue is required. The better the initial guess, the faster the solution will be found. In addition, the numeric solver has also been parallelized.

Syntax of FE Scalar and FE Vectorial Optical Solvers

The finite-element (FE) scalar solver caters only to the waveguide problem, while the FE vectorial solver handles both waveguide and cavity problems. The choice of FE scalar or FE vectorial solvers is determined in the `Optics` statement. The default is the FE scalar solver for waveguide modes.

FE Scalar Solver

The FEScalar solver for the waveguide problem is activated in the Physics-Laser-Optics statement in the command file:

```
Physics {...
  Laser (...
    Optics(
      FEScalar(EquationType = Waveguide
        Symmetry = Symmetric
        LasingWavelength = 800      # [nm]
        TargetEffectiveIndex = 3.4  # initial guess
        TargetLoss = 10.0           # initial guess [1/cm]
        Polarization = TE
        Boundary = "Type1"
        ModeNumber = 1
      )
    )
  )
}
```

This example shows how to activate the FE scalar solver to solve for one waveguide mode. As discussed in [Finite-Element Formulation on page 854](#), an iterative method is used to solve for the modes. Therefore, specifying a good TargetEffectiveIndex is important to speed up the computation of the solution. For multimodes, increase ModeNumber and specify multiple entries for TargetEffectiveIndex, TargetLoss, and so on (see [Specifying Multiple Entries for Parameters in FEScalar and FEVectorial on page 859](#)). Only Cartesian coordinates and waveguide modes are associated with the FE scalar solver. For scalar cavity solvers for VCSELs, refer to the transfer matrix method and effective index method.

In Sentaurus Device, there is an option to run only the optical solver without activating the laser simulation. This is called the optics stand-alone option (see [Optics Stand-alone Option on page 887](#)). In this case, you must specify the keyword Boundary in the FEScalar statement if Symmetric or Periodic is chosen. In a laser simulation, the default values of Boundary listed in [Table 132 on page 862](#) are used unless you select the boundary types explicitly. A detailed discussion of the Boundary keyword is in [Boundary Conditions and Symmetry for Optical Solvers on page 861](#).

[Table 216 on page 1287](#) describes all of the possible arguments that can be used inside the FEScalar statement.

There are three types of symmetry entry:

- `Symmetric` is symmetry about the y-axis at $x = 0$.
- `Nonsymmetric` is nonsymmetry.
- `Periodic` means the left and right boundaries of the Neumann type.

The default boundary conditions for the different symmetry types are listed in [Table 132 on page 862](#). The `LasingWavelength` entered is solely for the initial computation of the optical mode and the way the lasing wavelength is computed is discussed in [Waveguide Optical Modes and Fabry–Perot Cavity on page 793](#). `TargetEffectiveIndex` is the initial guess for the eigenvalue of the waveguide problem and is a necessary input to ensure that the required mode is computed.

FE Vectorial Solver

The FE vectorial solver handles both waveguide and cavity-type problems. The syntax of both problems is described separately.

FE Vectorial Solver for Waveguides

The syntax for using the vectorial FE solver for waveguides is:

```
Physics {...
  Laser (...
    Optics(
      FEVectorial(EquationType = Waveguide
        Symmetry = Symmetric
        Coordinates = Cartesian
        LasingWavelength = 800      # [nm]
        TargetEffectiveIndex = 3.4  # initial guess
        TargetLoss = 10.0          # initial guess [1/cm]
        Boundary = "Type1"
        ModeNumber = 1
      )
    )
  )
}
```

The argument list for `FEVectorial` statement for the waveguide problem is similar to that of the `FEScalar` statement.

36: Optical Mode Solver for Lasers

Syntax of FE Scalar and FE Vectorial Optical Solvers

FE Vectorial Solver for VCSEL Cavity

The syntax for using the FE vectorial solver for a VCSEL cavity is:

```
Physics {...
  Laser (...
    Optics(
      FEVectorial(EquationType = Cavity
        Coordinates = Cylindrical
        TargetWavelength = 777    # initial guess [nm]
        TargetLifeTime = 1.4      # initial guess [ps]
        AzimuthalExpansion = 1    # cylindrical harmonic order
        ModeNumber = 1
      )
    )
    VCSEL()    # specify this is a VCSEL simulation
  )
}
```

The syntax for the VCSEL cavity problem is very different from that of the waveguide problem. The keyword **VCSEL** must be included in the **Laser** statement to indicate that this is a VCSEL simulation. An initial guess for the resonant wavelength must be specified by **TargetWavelength**.

Cylindrical symmetry has been specified by **Coordinates=Cylindrical**, so the modes contain the angular dependence of $e^{im\phi}$. The cylindrical harmonic order, m , can be changed by the keyword **AzimuthalExpansion**.

NOTE Unless stated specifically, the argument should apply to both cavity and waveguide problems.

[Table 217 on page 1288](#) describes the arguments that can be used with the **FEVectorial** keyword. [Table 129](#) and [Table 130 on page 859](#) group the keywords for the waveguide and cavity problems, respectively.

Table 129 Dependencies of keywords for EquationType=Waveguide in FEVectorial optical solver

Keyword	Dependencies				
EquationType	Waveguide				
Coordinates	Not valid				
Symmetry	Symmetric		NonSymmetric	Periodic	
Boundary	Type1	Type2	Not valid	Type1	Type2
TargetWavelength	Not valid				

Table 129 Dependencies of keywords for EquationType=Waveguide in FEVectorial optical solver

Keyword	Dependencies
TargetLifetime	Not valid
LongitudinalWavevector	Not valid
AzimuthalExpansion	Not valid
LasingWavelength	<float>
TargetEffectiveIndex	<float>
TargetLoss	<float>

Table 130 Dependencies of keywords for EquationType=Cavity in FEVectorial optical solver

Keyword	Dependencies		
EquationType	Cavity		
Coordinates	Cartesian		Cylindrical
Symmetry	Symmetric		Not valid
Boundary	Type1	Type2	
TargetWavelength	<float>		
TargetLifetime	<float>		
LongitudinalWavevector	<float>		Not valid
AzimuthalExpansion	Not valid		<int>
LasingWavelength	Not valid		
TargetEffectiveIndex	Not valid		
TargetLoss	Not valid		

Specifying Multiple Entries for Parameters in FEScalar and FEVectorial

When multimodes are needed, ModeNumber is used to specify the number of modes required. In this case, you may want to specify different initial guesses for the effective index (for waveguide modes), resonant wavelength (for cavity modes), and so on. This is accomplished by extending the syntax for these initial guess parameters. The following is an example for waveguide modes and one for VCSEL cavity modes.

36: Optical Mode Solver for Lasers

Syntax of FE Scalar and FE Vectorial Optical Solvers

Multiple Waveguide Modes

The syntax extension of `FEScalar` for multiple waveguide modes is shown here. `FEVectorial` for multiple waveguide modes has the same syntax extension. The boundary type is specified explicitly in this example. If the `Boundary` keyword is omitted, the default values in [Table 132 on page 862](#) are taken:

```
Physics {...
  Laser (...
    Optics(
      FEScalar(EquationType = Waveguide
        Symmetry = Symmetric
        LasingWavelength = 780
        TargetEffectiveIndex = (3.54 3.47 3.5 3.33 3.4)
        TargetLoss = (5.0 6.0 8.0 7.0 10.0)
        Polarization = (TE TE TM TE TM)
        ModeNumber = 5
        Boundary = ("Type1" "Type2" "Type1" "Type1" "Type2")
      )
    )
  )
}
```

Multiple Cavity Modes

The syntax extension of `FEVectorial` for multiple cavity modes is:

```
Physics {...
  Laser (...
    Optics(
      FEVectorial(EquationType = Cavity
        Coordinates = Cylindrical
        TargetWavelength = (777 762 760 753 751)
        TargetLifeTime = (1.5 1.3 1.22 1.16 1.1)
        AzimuthalExpansion = (1 0 0 2 2)
        ModeNumber = 5
      )
    )
    VCSEL()
  )
}
```

Boundary Conditions and Symmetry for Optical Solvers

There are three main types of boundary condition for the optical problem:

- Dirichlet boundary condition
- Neumann boundary condition
- Radiative boundary condition

These boundary conditions can be controlled to some extent with the `Symmetry` keyword in the `FEScalar` and `FEVectorial` statements. In the optical solvers of Sentaurus Device, the external boundaries of the optical simulation space are assumed (by default) to satisfy the Dirichlet boundary condition, that is, the optical fields on this outer boundary are set to zero. By specifying different types of symmetry, the Neumann boundary condition can be imposed on different external boundaries.

The boundary conditions for raytracing are treated differently and are discussed in [Boundary Condition for Raytracing on page 520](#).

[Table 131](#) summarizes the symmetry types (appearing in the `FEScalar` and `FEVectorial` statements) and lists the associated boundary conditions.

Table 131 Symmetry types in `FEScalar` and `FEVectorial` statements and their associated optical boundary conditions

Symmetry type	Boundary condition
NonSymmetric	Dirichlet boundary condition on all external boundaries, that is, the optical field is set to zero at the boundaries.
Periodic	Neumann or Dirichlet boundary conditions on all vertical boundaries, and Dirichlet boundary condition on all horizontal boundaries.
Symmetric	Neumann or Dirichlet boundary condition for even or odd on the symmetry axis, and Dirichlet boundary condition elsewhere. The symmetry axis is defined as the y-axis at $x = 0$.

If `Symmetry=Symmetric` and the optics stand-alone option is used, the argument keyword `Boundary` must be specified. In laser simulations, `Boundary` is chosen automatically unless explicitly specified. The argument `Boundary` dictates the boundary condition at the symmetry y-axis and is specific only for Cartesian coordinates because odd and even modes require different types of boundary condition at the symmetry y-axis. A summary of the default boundary conditions used in different cases is presented in [Table 132 on page 862](#). The following examples explain the different cases.

36: Optical Mode Solver for Lasers

Boundary Conditions and Symmetry for Optical Solvers

Table 132 Default boundary conditions for the optical solvers

Case	Periodic	Symmetric	NonSymmetric
FEScalar	Boundary = "Type1" for first n/2 modes and Boundary = "Type2" for the rest of the modes.		Boundary not used.
FEVectorial, Waveguide	Boundary = "Type2" for first n/2 modes and Boundary = "Type1" for the rest of the modes.		
FEVectorial, Cavity	Boundary chosen automatically.		

NOTE The boundary condition for every mode can be explicitly specified by the keyword `Boundary = ("Type2" "Type1" ...)`.

Symmetric FEScalar Waveguide Mode in Cartesian Coordinates

In this example, the FEScalar optical mode solver is applied to a symmetric edge-emitting laser to obtain the even and odd modes. The top row of Figure 72 shows the even modes. They are calculated when the keyword `Boundary = "Type1"` is set. The bottom row shows the odd modes. They are computed when the keyword `Boundary = "Type2"` is set. The fundamental mode is obtained if the keyword `Boundary = "Type1"` is set while the first-order mode is calculated for `Boundary = "Type2"`.

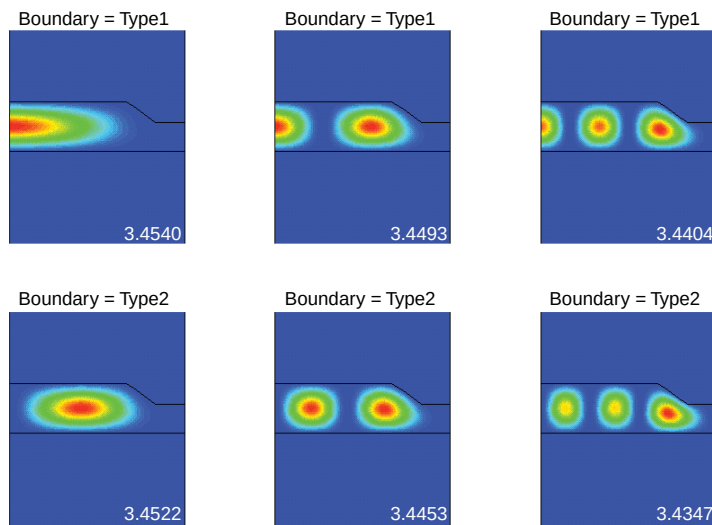


Figure 72 Scalar mode patterns of a symmetric edge-emitter structure

Symmetric FEVectorial Waveguide Modes in Cartesian Coordinates

In this example, the FEVectorial optical mode solver is applied to a symmetric edge-emitting laser to find the horizontally and vertically polarized modes. The top row of [Figure 73](#) shows the calculated modes when the keyword `Boundary = "Type1"` is set. The bottom row shows the modes that are calculated when the argument `Boundary = "Type2"` is set. The horizontally polarized fundamental mode is obtained for `Boundary = "Type2"`.

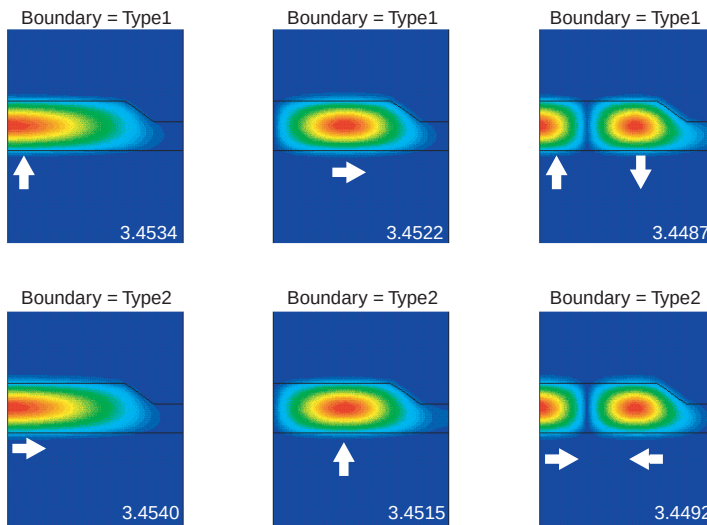


Figure 73 Vectorial mode patterns of a symmetric edge-emitter structure

Symmetric FEVectorial VCSEL Cavity Modes in Cartesian Coordinates

In addition to cylindrical symmetry, the FEVectorial optical mode solver can also be applied to a symmetric VCSEL in Cartesian coordinates to find the modes that are polarized in-plane or perpendicular to the plane.

The top row of [Figure 74 on page 864](#) shows the modes when the keyword `Boundary = "Type1"` is set. The bottom row shows the modes that are computed when the keyword `Boundary = "Type2"` is set.

The in-plane polarized fundamental mode is obtained for `Boundary = "Type2"`, while the perpendicularly polarized fundamental mode is computed for `Boundary = "Type1"`.

36: Optical Mode Solver for Lasers

Boundary Conditions and Symmetry for Optical Solvers

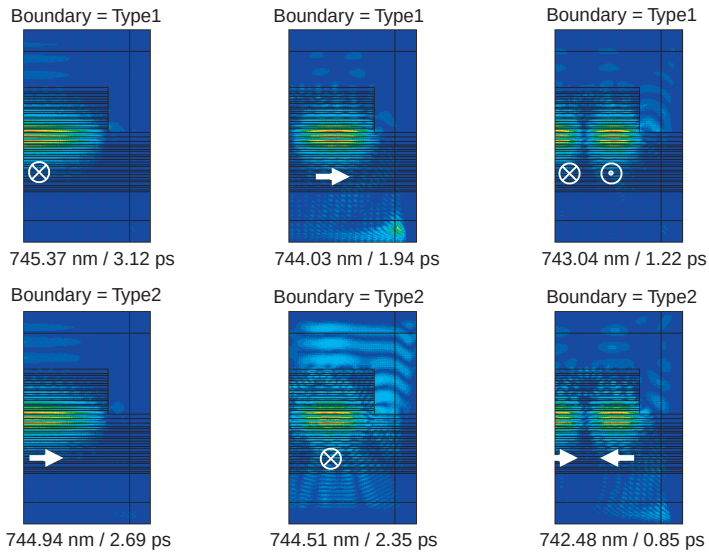


Figure 74 Vectorial mode patterns of a VCSEL in Cartesian coordinates

Symmetric FEVectorial VCSEL Cavity Modes in Cylindrical Coordinates

The argument `AzimuthalExpansion` in the `FEVectorial` optical mode solver is used to calculate different types of mode in a cylindrical VCSEL. The left column of [Figure 75 on page 865](#) shows the computed modes for the argument `AzimuthalExpansion=0`. The middle column shows the modes that are calculated when `AzimuthalExpansion=1` is set. The right column shows the mode pattern for `AzimuthalExpansion=2`. For example, the fundamental mode `HE11` is obtained by setting the argument `AzimuthalExpansion=1`.

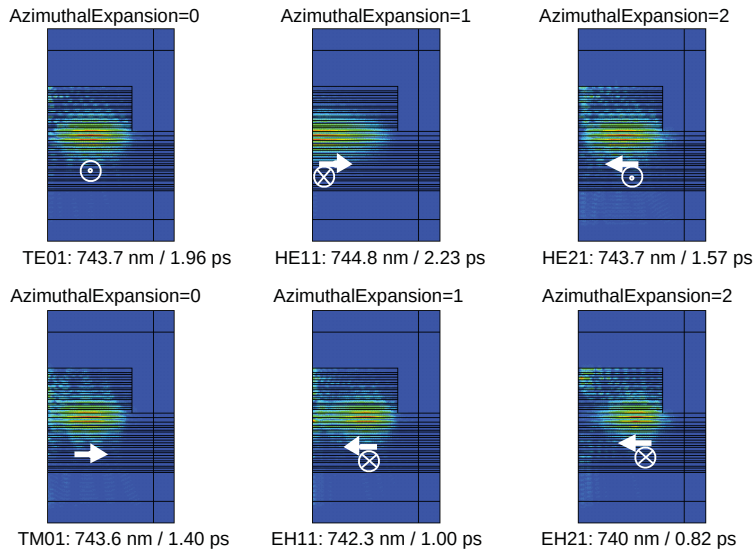


Figure 75 Vectorial mode patterns of a VCSEL in cylindrical coordinates

Perfectly Matched Layers

Apart from Dirichlet and Neumann boundary conditions, Sentaurus Device can simulate the radiative boundary condition artificially using the concept of a perfectly matched layer (PML).

The PML in Sentaurus Device is implemented by using a tensorial permeability quantity [4], which can be interpreted as a uniaxial anisotropic medium. This can be proven to be the same as coordinate stretching for the curl, divergence, and gradient operators [5]. (Further information is available from the literature: the original PML work [6], PML interpreted as a coordinate-stretching concept [5], and the generalization of the PML concept to various coordinate systems and general anisotropic and dispersive media [7][8].) A mathematical presentation of the PML is beyond the scope of this manual. Therefore, only the physical concept behind the method is discussed.

NOTE Perfectly matched layers should only be added when using the vectorial (FEVectorial) optical solver.

The Dirichlet boundary condition is assumed at the outer boundaries of the simulation space. In many cases, the optical field may radiative outwards, for example, waves from the lasing region leak into the substrate because the substrate has about the same refractive index as the guiding layers. In this case, the solution obtained using the Dirichlet boundary condition will include reflections of these radiative waves from the boundaries, which are nonphysical. To solve this problem, the boundaries of the structure can be coated with PMLs to reduce and

eliminate unwanted reflections from the Dirichlet boundary condition in a truncated simulation space. This enables the artificial simulation of the radiative boundary condition.

The structure is terminated by an inner boundary Γ_i (see Figure 76), and the PML is placed between the inner boundary Γ_i and outer boundary Γ_o . A gradually increasing loss is introduced in the PML from the inner boundary Γ_i towards the outer boundary Γ_o . Therefore, upon first impact of the wave into the PML at the inner boundary Γ_i , negligible reflection occurs. As the wave propagates deeper into the PML, it is absorbed more and more. Any reflected waves within the PML also suffer from absorption. Ultimately, it appears that the propagating wave in the PML is totally absorbed and, therefore, this is how the PML simulates the radiative boundary condition. The loss profile in the PML has been set automatically in Sentaurus Device.

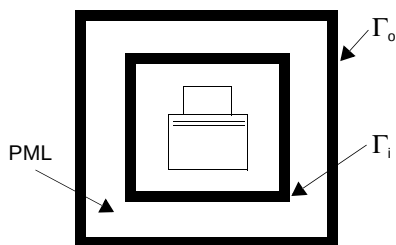


Figure 76 PML coating the structure

PMLs are indicated by special keywords appended to the beginning of the region names when the optical device is drawn. The region names of PMLs at the top, bottom, left, and right of the structure start with TPML, BPML, LPML, and RPML, respectively, as shown in Figure 77.

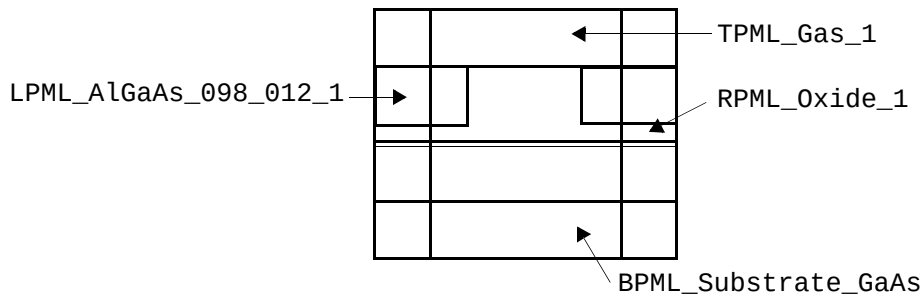


Figure 77 Naming of PML regions

This naming convention is required to impose the correct boundary condition in the optical solver. These special PML regions are declared in the same way as other regions in the command file of Sentaurus Device. A Tcl-based script can be used to add automatically PML regions to the outer boundaries of the structure. This script can be obtained from the Synopsys Technical Support Center.

Transfer Matrix Method for VCSELs

The 1D transfer matrix method (TMM) is a simplified scalar solver for the VCSEL cavity problem. It computes the spatial optical intensity and the corresponding characteristics of the fundamental mode in a cylindrically symmetric VCSEL structure. The scalar wave equation in the axial direction is:

$$\left(\frac{\partial^2}{\partial z^2} + k_z^2 \right) \phi_z = 0 \quad (949)$$

where ϕ_z denotes the wave component in the axial direction. The TMM is applied at the symmetry axis in the vertical direction (see Figure 78).

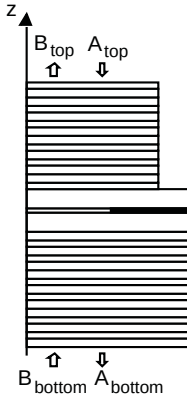


Figure 78 Transfer matrix method applied to a symmetric VCSEL structure

The solution to the axial wave equation in each layer i can be expressed as the sum of a forward-propagating and backward-propagating wave:

$$\phi_z = A(z_i)e^{(ik_{zi}z)} + B(z_i)e^{(-ik_{zi}z)} \quad (950)$$

The TMM relates the two waves at the interfaces i and $i + 1$ by:

$$\begin{bmatrix} A(z_i) \\ B(z_i) \end{bmatrix} = \begin{bmatrix} T_{11} & T_{12} \\ T_{21} & T_{22} \end{bmatrix}_i \cdot \begin{bmatrix} A(z_{i+1}) \\ B(z_{i+1}) \end{bmatrix} \quad (951)$$

The transfer matrix for an index step $n_1 \rightarrow n_2$ is:

$$T_{1 \rightarrow 2} = \frac{1}{t_{12}} \begin{bmatrix} 1 & r_{12} \\ r_{12} & 1 \end{bmatrix} \quad (952)$$

36: Optical Mode Solver for Lasers

Transfer Matrix Method for VCSELs

with $r_{12} = -r_{21} = \frac{n_1 - n_2}{n_1 + n_2}$ and $t_{12} = t_{21} = \frac{2n_1}{n_1 + n_2}$.

For a homogeneous medium of length d , the transfer matrix becomes:

$$T_d = \begin{bmatrix} e^{ikd} & 0 \\ 0 & e^{-ikd} \end{bmatrix} \quad (953)$$

With these basic transfer matrix blocks, a final transfer matrix can be set up that relates the forward-propagating and backward-propagating waves at the top ($A_{\text{top}}, B_{\text{top}}$) of the device to the wave at the bottom ($A_{\text{bottom}}, B_{\text{bottom}}$) of the device:

$$\begin{bmatrix} B_{\text{top}} \\ A_{\text{top}} \end{bmatrix} = \begin{bmatrix} T_{11} & T_{12} \\ T_{21} & T_{22} \end{bmatrix}_0 \cdot \dots \cdot \begin{bmatrix} T_{11} & T_{12} \\ T_{21} & T_{22} \end{bmatrix}_{2n+1} \cdot \begin{bmatrix} B_{\text{bottom}} \\ A_{\text{bottom}} \end{bmatrix} = \begin{bmatrix} T_{11, \text{tot}} & T_{12, \text{tot}} \\ T_{21, \text{tot}} & T_{22, \text{tot}} \end{bmatrix} \cdot \begin{bmatrix} B_{\text{bottom}} \\ A_{\text{bottom}} \end{bmatrix} \quad (954)$$

where n denotes the number of layers of the structure. Resonance is achieved only if the outward-propagating waves are established at the top and bottom of the device ($A_{\text{top}} = 0$ and $B_{\text{bottom}} = 0$). This condition is found by varying the frequency ω and the gain in the quantum wells. The corresponding characteristics of this instance are the resonant wavelength and quantum well gain. At sustained resonance, the threshold gain in the quantum wells must balance the radiation losses from the device.

Since the 1D TMM only provides the change of field in the axial direction, a Gaussian variation is assumed for the optical intensity in the transverse direction:

$$\text{Gauss}(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}} \quad (955)$$

where x is the distance in the transverse direction from the symmetry axis and σ is the width of the Gaussian shape.

The activation syntax for the TMM for VCSEL cavity problems in the command file is:

```
Physics {...
  Laser (...
    Optics(
      TMM1D(SigmaGauss=4.0    # width of Gaussian, [micrometer]
      TargetWavelength=800    # initial guess [nm]
    )
  )
  VCSEL(                      # specify this is a VCSEL simulation
  )
}
```

The solution of the resonant wavelength is solved iteratively, so you must specify an initial guess in `TargetWavelength`. The default and only symmetry type supported by TMM1D is `Symmetric`; only the fundamental mode is computed. The rest of the entries in the `Physics` and `Laser` statements are similar to that in [Simulating Single-Grid Edge-Emitting Laser on page 760](#).

[Table 223 on page 1293](#) lists the arguments of TMM1D.

Effective Index Method for VCSELs

The effective index method (EIM) is a fast scalar solver and well suited to computing fairly accurate resonant wavelength and optical intensity for index-guided VCSELs. However, the EIM cannot compute the scattering losses accurately for very small aperture (less than 2 μm radius) VCSELs. Nevertheless, the EIM is an option in Sentaurus Device to cater to the rapid design of index-guided VCSELs, including oxide-confined VCSELs.

[Figure 79](#) shows a typical VCSEL geometry suitable for the EIM. The structure is divided into two regions: the core and the cladding. Within each core and cladding region, multiple homogeneous layers of semiconductor are allowed.

NOTE In Sentaurus Device, the EIM is only applicable to the VCSEL cavity problem, *not* the waveguide problem.

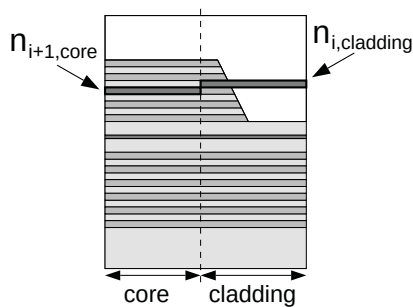


Figure 79 VCSEL partitioned into the core and cladding regions for the EIM

Formulation of Effective Index Method

The EIM solves the scalar Helmholtz equation:

$$\nabla^2 \Psi + k^2 \Psi = 0 \quad (956)$$

by assuming that the wavefunction Ψ is separable, that is:

$$\Psi = \phi_t(x, y) \cdot \phi_z(z) \quad (957)$$

where the subscripts t and z refer to the transverse and z components. Substituting [Eq. 957](#) into [Eq. 956](#) gives two independent equations:

The axial wave equation is:

$$\left[\frac{\partial^2}{\partial z^2} + k_z^2 \right] \cdot \phi_z = 0 \quad (958)$$

The transverse wave equation is:

$$\left[\nabla_t^2 + \left[n(r) \cdot \frac{2\pi}{\lambda_0} \right]^2 - \beta^2 \right] \cdot \phi_t = 0 \quad (959)$$

The transverse and axial wave equations are coupled using the dispersion relation:

$$k^2 = k_t^2 + k_z^2 = n^2 \cdot k_0^2 \quad (960)$$

where the free space wavenumber is $k_0 = \frac{2\pi}{\lambda_0}$.

The solution strategy of the EIM for VCSELs is best illustrated by the flowchart in [Figure 80 on page 871](#).

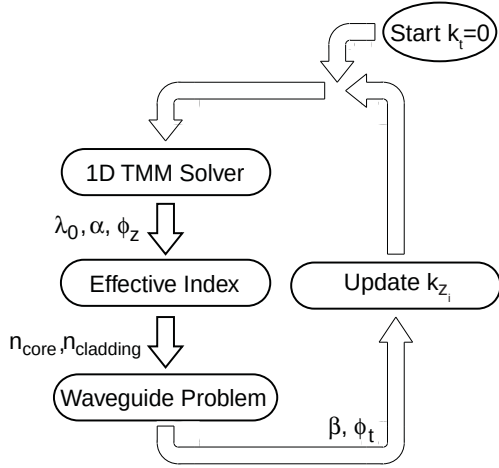


Figure 80 Solving the axial and transverse wave equations self-consistently in the EIM

First, the axial wave equation is solved by the transfer matrix method. The required output is the resonant wavelength λ_0 , the net cavity loss α , and the z-resonant field $\phi_z(z)$.

Next, $\phi_z(z)$ is used to compute the effective indices in the core (n_{core}) and the cladding (n_{cladding}) regions by the following averaging relation:

$$n_{\text{core,cladding}}^2 = \frac{\sum_i \int_{z_i}^{z_{i+1}} n_{i,\text{core/cladding}}^2 \cdot |\phi_z|^2 dz}{\int |\phi_z|^2 dz} \quad (961)$$

The refractive index of each layer i is weighted by the z-resonant optical intensity, and the effective index can be viewed as an average of these weighted refractive indices.

n_{core} and n_{cladding} are subsequently used in the transverse wave equation to formulate an optical fiber or waveguide-type problem.

Solving the transverse wave equation yields the propagation constant β , which is then used to update k_{z_i} of the axial wave equation using the relations:

$$\left(n_{\text{core}} \cdot \frac{2\pi}{\lambda_0}\right)^2 = k_t^2 + \beta^2 \quad (962)$$

and:

$$k_{z_i}^2 = \left(n_i \cdot \frac{2\pi}{\lambda_0}\right)^2 - k_t^2 \quad (963)$$

This is performed so that the concept of phase-matching is enforced at the tangential boundaries of all the layers. The entire process is iterated until convergence is achieved.

Transverse Mode Pattern of VCSELs

The transverse wave equation (Eq. 959) in the EIM can be solved in two different coordinate systems: Cartesian and cylindrical coordinates. The core and cladding effective indices form a symmetric three-layer waveguide problem. In Cartesian coordinates, this is equivalent to the classic step-index planar waveguide problem. In cylindrical coordinates, this becomes an optical fiber problem. Both problems can be solved by a semianalytic approach, and the range of effective index in this waveguide problem is:

$$n_{\text{cladding}} < n_{\text{eff}} < n_{\text{core}} \quad (964)$$

where the propagation constant $\beta = n_{\text{eff}} \cdot k_0$. This ensures that the transverse mode is a guided mode.

The step-index planar waveguide in Cartesian coordinates is an approximation to solving the transverse mode profile in square aperture VCSELs. It is assumed that the square aperture VCSEL is symmetric to a plane, as shown in Figure 81. The core and cladding indices form the step-index layers of the waveguide, and the TE modal field component, $\phi_t(x, y) = \phi_y(x)$, varies as $\sin(x)$ or $\cos(x)$ in the core region and as $\exp(-|x|)$ in the cladding region.

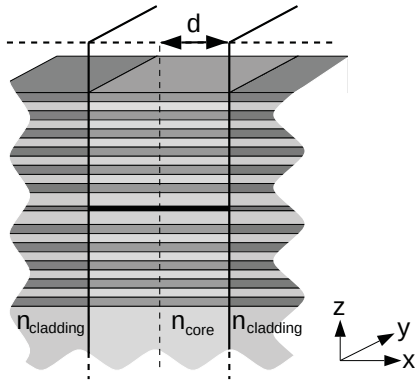


Figure 81 Transverse mode in square aperture VCSEL is treated as a step-index planar waveguide problem

In a circular aperture VCSEL (see [Figure 82](#)), the optical fiber problem is solved. The modal field components, $\phi_t(r, \varphi)$, vary as $J_{m-1}(r)$ (Bessel function) in the core region and as $K_{m-1}(r)$ (modified Bessel function) in the cladding region. The parameter m denotes the cylindrical harmonic order, $e^{im\varphi}$. The modes in an optical fiber are commonly classified as HE_{mn} , EH_{mn} , TE_{0n} , and TM_{0n} , and this convention is followed in the command file syntax. Generally, the fundamental mode is HE_{11} , followed by the higher order modes TE_{01} , TM_{01} , and so on.

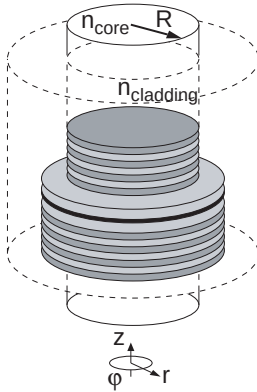


Figure 82 Transverse mode in circular aperture VCSEL is treated as an optical fiber problem

Syntax for Effective Index Method

The EIM solver can be activated by specifying the keyword `EffectiveIndex` in the `Physics-Laser-Optics` part of the command file.

Cylindrical Modes

```
Physics {...
  Laser (...
    Optics (
      EffectiveIndex (
        TargetWavelength = (780.0 750.0 775.0 770.0) # [nm]
        CoreWidth = 4.0 # [micrometer]
        ModeType = (HEmn EHmn TE0n TM0n) # choose mode type
        mModeIndex = (1 1 0 0) # m in HEmn, EHmn
        nModeIndex = (1 2 2 1) # n in HEmn, EHmn, TE0n, TM0n
        Coordinates = Cylindrical
        Absorption(ODB)
        DiffractionLoss = (5.0 8.0 6.0 7.0) # [1/cm]
        ModeNumber = 4 # solve for 4 modes
      )
    )
  )
}
```

```

    VCSEL()
  )
}

```

Cartesian Modes

```

Physics {...
  Laser (...
    Optics (
      EffectiveIndex (
        TargetWavelength = (780.0 750.0 775.0 770.0) # [nm]
        CoreWidth = 4.0 # [micrometer]
        ModeType = (TE0n TE0n TE0n TE0n) # only TE0n allowed for Cartesian
        nModeIndex = (1 2 3 4) # n in TE0n
        Coordinates = Cartesian
        Absorption(ODB)
        DiffractionLoss = (5.0 8.0 6.0 7.0) # [1/cm]
        ModeNumber = 4 # solve for 4 modes
      )
    )
    VCSEL()
  )
}

```

This example shows the activation of the EIM for multiple cylindrical and Cartesian modes. The more notable aspects of the syntax are:

- The **CoreWidth** is the radius of the circular aperture in **Cylindrical** coordinates or the half-length of the square aperture in **Cartesian** coordinates.
- The keywords **HE_mn**, **EH_mn**, **TE₀n**, and **TM₀n** refer to the common optical fiber modes in the cylindrical modes. For the Cartesian modes, only **TE₀n** modes are handled.
- The keywords **ModeType**, **mModeIndex**, and **nModeIndex** are used to specify the required modes.
- The keyword **VCSEL** must be included in the **Laser** statement to specify that this is a VCSEL simulation.
- The number of modes to solve is four in this example, and the parameters corresponding to each mode are specified as shown.

[Table 215 on page 1286](#) lists the full range of keywords for the EIM solver.

A sample output from the EIM solver is shown in [Figure 83 on page 875](#). The peak of the z-resonant mode aligns with the active quantum well region. Two vectorial transverse modes, HE₁₁ and HE₂₁, computed from the optical fiber problem are also shown.

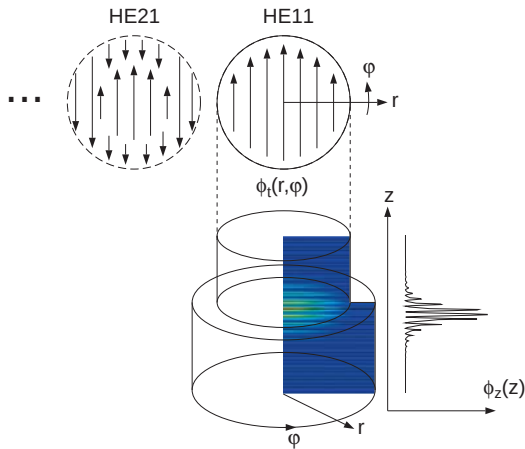


Figure 83 Transverse and longitudinal resonant modes of cylindrically symmetric VCSEL computed by EIM

Saving and Loading Optical Modes

The calculation of the optical modes of a laser can be time-consuming especially for large geometries. To save simulation time, transverse optical mode patterns can be saved and loaded from data files. However, when the optical modes are loaded from a file, the optical modes are assumed to be constant throughout the simulation. As a result, self-consistent iteration between the optics and electronics is not possible in this case.

Saving Optical Modes on Optical or Electrical Mesh

The optical field can be saved on the grid of the optical device (referred to as the optical mesh) in a dual-grid simulation by using the keyword `SaveOptField`. To understand more about dual-grid simulation, see [Simulating Dual-Grid Edge-Emitting Laser on page 763](#). If the optical mesh is not found, the keyword `SaveOptField` saves the optical field to the only grid that is available – the electrical mesh as it is in [Simulating Single-Grid Edge-Emitting Laser on page 760](#). The activation keyword is `SaveOptField` in the `File` and `Solve-quasistationary` sections of the command file:

```
File {...
  SaveOptField = "laserfield"
}
...
Solve {...
  quasistationary (
    SaveOptField { range=(0,1) intervals=3 }
```

36: Optical Mode Solver for Lasers

Saving and Loading Optical Modes

```
Goal({name="p_Contact" voltage=1.8})
{...}
}
```

Refer to [Simulating Single-Grid Edge-Emitting Laser on page 760](#) to see where this syntax is inserted in the command file. The `SaveOptField` has the same format as the `OptFarField` (see [Far Field on page 878](#)) and `GainPlot` (see [Plotting Gain and Power Spectra on page 919](#)). In this way, you can track the evolution of the optical field as the bias increases. With regard to the syntax and its output:

- In the File section, saving the optical field is activated by the keyword `SaveOptField`. The value assigned to it, "laserfield" in this case, becomes the base name for the output files of the optical fields.
- In the Solve-quasistationary section, the argument `range=(0,1)` in the `SaveOptField` keyword is mapped to the initial and final bias conditions. In this example, the initial and final (goal) `p_Contact` voltages are 0 V and 1.8 V, respectively. The number of `intervals=3`, which gives a total of four ($= 3+1$) optical field saved at 0 V, 0.6 V, 1.2 V, and 1.8 V. In general, specifying `intervals=n` will produce $(n+1)$ sets of optical field files.
- The output files in this example are:
 - laserfield_000000_mode0_des.dat
 - laserfield_000001_mode0_des.dat
 - laserfield_000002_mode0_des.dat
 - laserfield_000003_mode0_des.dat

Using a vectorial optical solver, each file contains four datasets: the optical field intensity, real and imaginary parts of the vectorial optical fields, and polarization. For the scalar optical solver `FEScalar`, only the optical field intensity and polarization are saved.

- For an LED simulation, only the intensity file is produced because the LED raytracing contains no vectorial or phase information.

Loading Optical Modes

Sentaurus Device can read optical modes that have been computed separately. The activating keywords are `OptField<n>` in the File section and `OpticsFromFile` in the Physics section. `OptField<n>` specifies the file from which optical intensity and polarization are loaded. Optical mode properties are specified in the `OpticsFromFile` section. Its keywords are summarized in [Table 219 on page 1289](#).

The following example shows the syntaxes for loading optical fields for an edge-emitting laser and a VCSEL.

Loading Optical Fields for Edge-emitting Laser

TargetEffectiveIndex and TargetLoss specify the real and imaginary part of the complex propagation constant of the mode. LasingWavelength defines the lasing wavelength of the laser.

```
File {...
  OptField0 = "field_mode0_des.tdr"
  OptField1 = "field_mode1_des.tdr"
}
...
Physics {...
  Laser (...
    Optics (...
      OpticsFromFile (
        TargetEffectiveIndex = (3.45 3.44) # [1]
        TargetLoss = (5.3510e-7 1.0400e-6) # [1]
        LasingWavelength = 850.0           # [nm]
        ModeNumber = 2
      )
    )
  )
}
```

Loading Optical Fields for VCSEL

TargetWavelength and TargetLoss specify the lasing wavelength and total decay rate. OutcouplingLoss defines the decay rate through the top mirror.

```
File {...
  OptField0 = "field_mode0_des.dat"
}
...
Physics {...
  Laser (...
    Optics (...
      OpticsFromFile (
        TargetWavelength = 750.19 # [nm]
        TargetLoss = 0.0821 # [1/ps]
        OutcouplingLoss = 0.0679 # [1/ps]
        ModeNumber = 1
      )
    )
  )
  VCSEL()
  OpticalConfinementFactor = 0.0338
}
}
```

With regard to the syntax:

- The number of optical field files specified must match the number of modes specified by ModeNumber.
- The number of target values must match the number of modes specified by ModeNumber.
- Up to ten optical modes can be selectively load.

NOTE When optical modes are read from files, self-consistent simulation between the optics and electronics is not possible.

The optical mode that is read into Sentaurus Device must strictly be on the same mesh as the optics would be computed. If the optical mode has been saved on another mesh (for example, in an optics stand-alone simulation with a different mesh), it must be interpolated to the corresponding mesh. Contact the TCAD Support Team for further information on this topic.

Far Field

The far field is important to determine the beam divergence of the laser diode emission. From antenna theory, the far field is defined for observation distance:

$$r \geq \frac{2D^2}{\lambda} \quad (965)$$

where D is the largest dimension of the near-field shape and λ is the free space wavelength. For example, given a near-field shape of dimension $2 \times 2 \mu\text{m}$ and a wavelength of $1 \mu\text{m}$, the far field occurs at the observation distance ($r \geq 8$) μm .

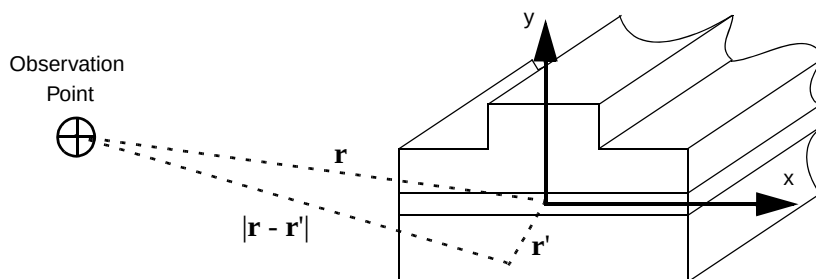


Figure 84 Origin is conveniently placed at center of laser end facet so that $z'=0$; distance between the observation point and a source point on laser end facet is $|r - r'|$

Figure 84 is referred to for the coordinate used in the following derivation of the vectorial far field. The optical electric field at the observation distance $\mathbf{r}$ [9] is:

$$\mathbf{E}(\mathbf{r}) = -\int \left[\nabla \times \vec{\mathbf{G}}(\mathbf{r}, \mathbf{r}') \bullet \mathbf{M}(\mathbf{r}') \right] d\mathbf{r}' \quad (966)$$

where $\vec{\mathbf{G}}$ is the dyadic Green's operator and $\mathbf{M}$ is an equivalent magnetic source representing the near field, $\mathbf{E}_{\text{near}}(\mathbf{r}')$, of the laser mode:

$$\mathbf{M}(\mathbf{r}') = -2\hat{\mathbf{z}} \times \mathbf{E}_{\text{near}}(\mathbf{r}') \quad (967)$$

In the far field, it is assumed that the radiation becomes plane waves, and the curl of the dyadic Green's operator can be simplified to:

$$\nabla \times \vec{\mathbf{G}}(\mathbf{r}, \mathbf{r}') = ik\hat{\mathbf{r}} \times \vec{\mathbf{G}}(\mathbf{r}, \mathbf{r}') \quad (968)$$

where the unit vector:

$$\hat{\mathbf{r}} = \hat{\mathbf{x}} \sin \theta \cos \phi + \hat{\mathbf{y}} \sin \theta \sin \phi + \hat{\mathbf{z}} \cos \theta \quad (969)$$

In addition, the following approximations are assumed for far-field calculations:

- $|\mathbf{r} - \mathbf{r}'| \approx r$ for amplitude
- $|\mathbf{r} - \mathbf{r}'| \approx r - \hat{\mathbf{r}} \bullet \mathbf{r}'$ for phase

With these assumptions, formulas for the vectorial and scalar far-field calculations can be derived.

In the vectorial case, simplifying the dyadic Green's operator with the scalar Green function and setting $z' = 0$, the vectorial far field can be derived as:

$$\begin{aligned} \mathbf{E}(r, \theta, \phi) = & -\frac{ike^{ikr}}{4\pi r} \left\{ \hat{\mathbf{x}} 2 \cos \theta \iint E_{\text{near}(x)}(x', y') e^{-ik\hat{\mathbf{r}} \bullet \hat{\mathbf{r}}'} dx' dy' \right. \\ & + \hat{\mathbf{y}} 2 \cos \theta \iint E_{\text{near}(y)}(x', y') e^{-ik\hat{\mathbf{r}} \bullet \hat{\mathbf{r}}'} dx' dy' \\ & \left. - \hat{\mathbf{z}} 2 \sin \theta \iint [E_{\text{near}(x)}(x', y') \cos \phi + E_{\text{near}(y)}(x', y') \sin \phi] e^{-ik\hat{\mathbf{r}} \bullet \hat{\mathbf{r}}'} dx' dy' \right\} \end{aligned} \quad (970)$$

where $E_{\text{near}(x)}$ and $E_{\text{near}(y)}$ are the x- and y-components of the near field, respectively.

Using the transformations:

$$\sin \Theta_x = \sin \theta \cdot \cos \phi \quad (971)$$

$$\sin \Theta_y = \sin \theta \cdot \sin \phi \quad (972)$$

you can derive from [Eq. 970](#) the commonly used expression for the constant radius, relative far-field intensity:

$$I_{\text{far}}(\Theta_x, \Theta_y) = (1 - (\sin^2 \Theta_x + \sin^2 \Theta_y)) \left| \iint \Phi(x, y) e^{ik_0(x \sin \Theta_x + y \sin \Theta_y)} dx dy \right|^2 \quad (973)$$

In [Eq. 973](#), it is clear that the scalar near field, Φ , can represent either $E_{\text{near}(x)}$ (TE mode) or $E_{\text{near}(y)}$ (TM mode) in [Eq. 970](#).

NOTE If the square root of the optical intensity is used to compute the far field, the phase information will be lost and the far field will be incorrect. Therefore, it is important to ensure that full scalar or vectorial near-fields are used in the far-field calculations.

Far-Field Observation Angle

The mapped far-field observation angles, (Θ_x, Θ_y) , refer to the angles measured from the z-axis along the x-axis and y-axis, respectively. The laser end facet (location of the near field) is assumed to be at the origin, facing the direction of positive z (see [Figure 85 on page 881](#)).

The far-field observation distance is fixed at a constant radius from the origin where the device is, that is, the observation space is a hemisphere. As a result, constraints must be imposed on (Θ_x, Θ_y) because the aim is to map a rectangular $(\Theta_x, \Theta_y) [-\pi/2, \pi/2]$ space onto a hemisphere. The best way to visualize this constraint is to think of placing a square piece of fishnet over a hemisphere. The corner regions of the net will exceed the boundary of the hemisphere and, hence, do not contribute to the description of the hemisphere. These are the invalid zones of the observation space.

NOTE In the far-field calculation, the origin is aligned with the peak intensity of the fundamental mode of the edge-emitting laser, and the observation angles are defined with respect to this origin.

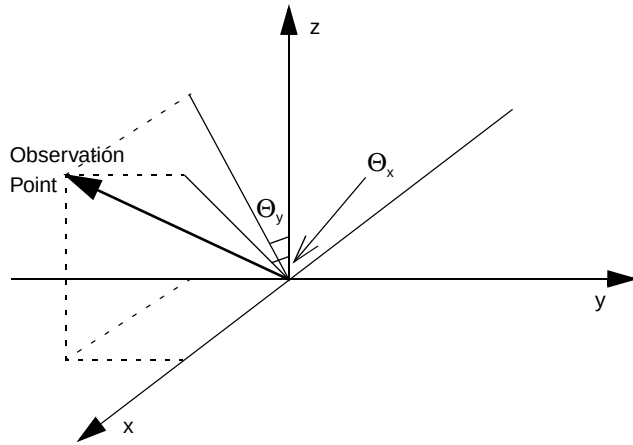


Figure 85 Defining the observation angles for far field

Syntax for Far Field

The optical far field is computed and plotted if the keyword `OptFarField` is specified in the File section of the command file:

```
File {...
  # ----- Activate farfield computation and save -----
  OptFarField = "far"
}
FarFieldPlot{range=(40,60) intervals=40 Scalar1D Scalar2D Vector2D}
...
Solve {...

  # --- Farfield plot parameters must be specified inside quasistationary ---
  quasistationary (...)

    PlotFarField{range=(0,1) intervals=3}

    Goal {name="p_Contact" voltage=1.8})
    {...}
}
```

See [Simulating Single-Grid Edge-Emitting Laser on page 760](#) for a clearer picture of the placement of the far-field syntax inside the command file. The far field syntax works in the same way as the `GainPlot` (see [Plotting Gain and Power Spectra on page 919](#)) and `Plot` options in the `Quasistationary` statement.

The more notable features of the syntax are:

- The base file name "far" of the far-field files is specified by `OptFarField` in the `File` statement. The keyword `OptFarField` also activates the far-field plot.
- The far field can only be computed and plotted within the `Quasistationary` statement. The keyword `PlotFarField` and the `FarFieldPlot` statement control the number and type of far field plots to produce.
- The argument `range=(0,1)` in the `PlotFarField` keyword is mapped to the initial and final bias conditions. In this example, the initial and final (goal) `p_Contact` voltages are 0 V and 1.8 V, respectively. The number of `intervals=3`, which gives a total of four ($= 3+1$) far-field plots at 0 V, 0.6 V, 1.2 V, and 1.8 V. In general, specifying `intervals=n` will produce $(n+1)$ plots.
- The argument `range=(40,60)` in the `FarFieldPlot` statement is the range for the observation angles. In this example, $\Theta_x = [-20^\circ, 20^\circ]$ and $\Theta_y = [-30^\circ, 30^\circ]$ were chosen. The number of discretized points in each range of the observation angles is given by `intervals=40`.
- You can select any one or a combination of the three types of far-field plot: `Scalar1D`, `Scalar2D`, and `Vector2D`. These keywords correspond to the scalar one-dimensional far field, scalar two-dimensional far field, and vectorial two-dimensional far field, respectively. The different types of far-field plot will generate different types of output files.

[Table 146 on page 1221](#), [Table 158 on page 1234](#), and [Table 266 on page 1322](#) give the activating syntaxes for the far field.

Far-Field Output Files

Activating different types of far-field plot produces different output files. Referring to the example syntax listed in [Syntax for Far Field on page 881](#), where the far-field base name has been specified by `OptFarField="far"`, output examples where different far-field types are selected are shown.

Scalar1D Far Field

The scalar 1D far field is activated by the keyword `Scalar1D`. Two lines per mode are produced for the 1D far-field plot. One is the far-field intensity measured along Θ_x with $\Theta_y = 0$; the other, along Θ_y with $\Theta_x = 0$. These far field results are output in the files `far_ff_000000_des.plt`, `far_ff_000001_des.plt`, and so on at the requested bias intervals specified in the argument of `PlotFarField`.

These files can be viewed in Inspect and an example of the plot is shown in [Figure 86](#).

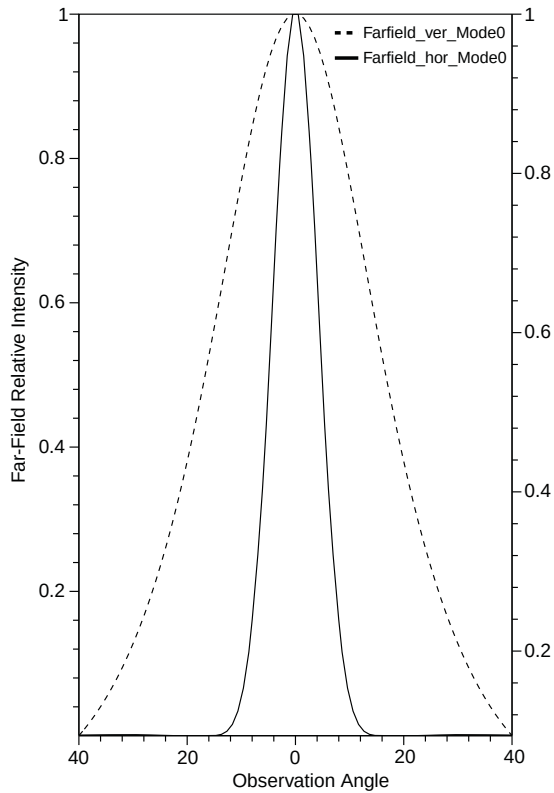


Figure 86 Scalar 1D far-field patterns showing vertical (*dashed line*) and horizontal (*solid line*) variations

Scalar2D and Vector2D Far Fields

The 2D scalar and vectorial far fields are activated by the keywords `Scalar2D` and `Vector2D`, respectively. An observation angle grid, `far_ff_des.grd`, is created if either keyword is detected. The vectorial far field can only be activated if the vectorial optical solver has been chosen or a vectorial mode is input from a file.

The data file for the scalar 2D far field contains the normalized far-field pattern of each mode, and the file names are `far_ff_des_000000_Scalar.dat`, and so on. Each file contains as many far-field patterns as the number of modes in the simulation. If a vectorial optical solver has been chosen, Sentaurus Device automatically selects the strongest field component (either E_x or E_y) for its scalar far-field computation.

36: Optical Mode Solver for Lasers

Far Field

The data file for the vectorial 2D far field contains the normalized far-field pattern for the x-, y-, and z-components and the total far field for each mode. These data files are named as `far_ff_des_000000_Vector.dat`, and so on. The 2D scalar and vectorial far fields can be viewed in Tecplot SV like any other .grd and .dat files. Examples of these far-field patterns are shown in [Figure 87](#) and [Figure 88](#).

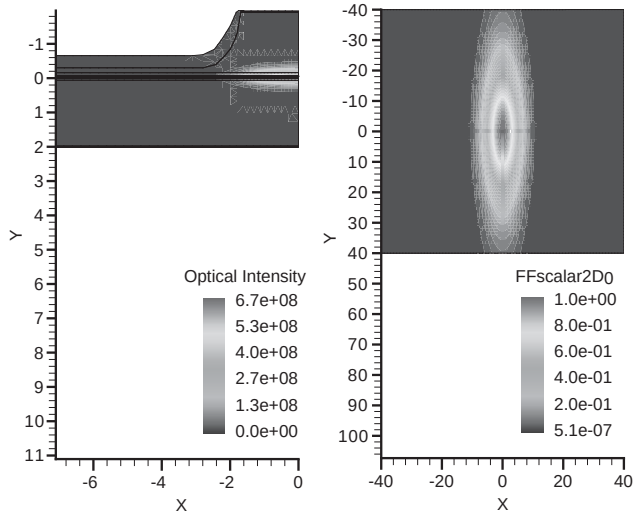


Figure 87 (Left) Near field and (right) scalar 2D far field of edge-emitting laser

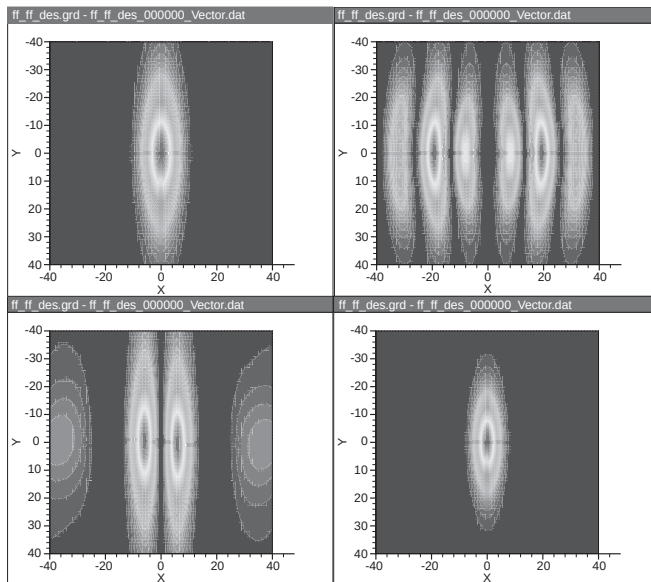


Figure 88 The (upper left) x-component, (upper right) y-component, (lower left) z-component, and (lower right) the total absolute value of vectorial 2D far fields of edge-emitting laser; x-component is a few orders of magnitude stronger than y- and z-components, so the absolute of the vectorial far field resembles that of x-component

Far Field from Loaded Optical Field File

Far fields can be computed from the optical mode loaded into Sentaurus Device (see [Saving and Loading Optical Modes on page 875](#)). To compute the far field correctly, the full scalar or vectorial optical near field with the complete phase information must be loaded. The optical intensity does not contain phase information and will give an incorrect far-field pattern.

VCSEL Near Field and Far Field

While the near field of an edge emitter is apparent, the location of the near field of a VCSEL is not automatically detected in Sentaurus Device. Different VCSELs have different geometric designs and the entire top surface may not be the emitting surface. Therefore, in the case of VCSELs, you must specify the location of the near field that will be used to compute the far field.

The activating syntaxes for the VCSEL near field are in the File, VCSELNearFieldPlot, and Solve-quasistationary sections of the command file:

```
File {...
    VCSELNearField = "vcseInf"
}
...
Physics {...
    Laser (...
        Optics (...
            FEVectorial(...)
        )
        VCSEL ( )
    )
}
VCSELNearFieldPlot(Position=0.0 Radius=10.0 NumberOfPoints=100)
# (<z> <radius> Npoints) [micrometers]
...
Solve {...
    # -- VCSEL near field parameters must be specified inside quasistationary --
    quasistationary (...
        VCSELNearField { range=(0,1) intervals=3 }
        Goal({name="p_contact" voltage=1.8})
        {...}
    )
}
```

An analysis of the syntax is:

- In the File section, the VCSEL near field is activated by the keyword `VCSELNearField`, and the value assigned to it, "vcseInf" in this case, is the base name for the output files of the near field.
- In the `VCSELNearFieldPlot` statement, the location and discretization mesh of the near field is defined by the keywords `Position=<z>`, `Radius=<radius>`, and `NumberOfPoints=<n>`. In VCSELs where cylindrical geometry has been assumed, `Radius` and `Position` (in micrometers) define the location of the near field, that is, the near field is taken from (0,<z>) to (<radius>,<z>). `n` defines the number of points on each side of the square mesh where the near field will be plotted.
- In the Solve-quasistationary section, the argument `range=(0,1)` in the `VCSELNearField` keyword is mapped to the initial and final bias conditions. In this example, the initial and final (goal) `p_Contact` voltages are 0 V and 1.8 V, respectively. The number of `intervals=3`, which gives a total of four (= 3+1) near field plots at 0 V, 0.6 V, 1.2 V, and 1.8 V. In general, specifying `intervals=n` will produce (n+1) plots.
- The square mesh grid file for the near field will be named `vcseInf.grd`. At each requested bias output, a file named `vcseInf_xxx_des.dat` will be created. For each mode, it contains the datasets `Abs(modeX_real_OpticalField)`, `Abs(modeX_imag_OpticalField)`, and `modeX_OpticalIntensity`. The near fields can be viewed in Tecplot SV.
- Cylindrical symmetry has been assumed so that the optical fields decompose into cylindrical harmonics, $e^{im\phi}$, where m is the cylindrical harmonic order defined by either the keyword `AzimuthalExpansion` for FEVectorial VCSEL simulation or the keyword `mModeIndex` for EffectiveIndex VCSEL simulation. The VCSEL near field can be divided into the even part and the odd part. Therefore, the field components are multiplied by either $\cos(m\phi)$ or $\sin(m\phi)$. Sentaurus Device plots the even part of the VCSEL near field.

When the VCSEL near field is computed, the activation of the far-field calculations is similar to that of other laser simulations.

NOTE In the case of VCSELs, the far field is not computed unless the near field is specified.

Optics Stand-alone Option

In many cases, the initial phase of design of a laser diode involves the optimization of the geometry of the laser structure to achieve specific optical mode properties, for example, mode shape. Sentaurus Device provides an option to run the optical solver without invoking the entire laser simulation.

An example of a stand-alone optical simulation of an edge-emitting laser, waveguiding structure is:

```
# ----- Optics Stand-alone command file -----

# ----- Specify only a dummy electrode -----
Electrode {
    { Name="dummy" voltage=0.0 }
}

# ----- Tell Sentaurus Device where to read/save the parameters/results -----
File {
    Grid = "optmesh_mdr.tdr"
    Parameter = "des_las.par"
    Plot = "opt_plot"
    Output = "log"
    # ----- Compute and save the farfield -----
    OptFarField = "farfield"
    # ----- Save the optical field -----
    SaveOptField = "optmode"
}

Plot {
    RefractiveIndex
}

# ----- Specify the material region properties, as usual -----
Physics (region="pbulk") { MoleFraction(xfraction=0.28) }
Physics (region="nbulk") { MoleFraction(xfraction=0.28) }
Physics (region="psch") { MoleFraction(xfraction=0.09) }
Physics (region="nsch") { MoleFraction(xfraction=0.09) }
Physics (region="barr") { MoleFraction(xfraction=0.09) }
# ----- Quantum Wells -----
Physics (region="qw1") {
    MoleFraction(xfraction = 0.8 Grading=0.00)
    Active          # keyword to specify active region
}
Physics (region="qw2") {
    MoleFraction(xfraction = 0.8 Grading=0.00)
    Active
```

36: Optical Mode Solver for Lasers

Optics Stand-alone Option

```
}

# ----- Major difference from the laser command file -----
Physics {
  Optics (
    FEVectorial (
      EquationType = Waveguide      # or Cavity
      Symmetry = Nonsymmetric      # or Symmetric or Periodic
      LasingWaveLength = 656       # [nm]
      TargetEffectiveIndex = 3.45
      Boundary = "Type2"
      ModeNumber = 1
    )
  )
  HeteroInterface
}

FarFieldPlot{ Range=(30,40) Intervals=60 }

Solve { Optics }
```

Compare this code to [Simulating Single-Grid Edge-Emitting Laser on page 760](#). The most significant difference in the optics stand-alone simulation is that the entire Laser section has been removed. Other notable changes are:

- In the `Electrode` statement, a dummy electrode must be specified in order to conform to the input format of Sentaurus Device. It has no other purpose in the optical simulation.
- In the `File` section, you can choose to plot the far field with the keyword `OptFarField`. The far-field parameters are set in the `FarFieldPlot` section.
- In the `File` section, you must specify the base name for the optical-field files with the keyword `SaveOptField`. These files can be read into the laser simulation (see [Saving and Loading Optical Modes on page 875](#)).
- The optical-active region is specified in the material region `Physics` statement by the keyword `Active`. This is important so that the optical confinement factor can be computed within Sentaurus Device.
- In the `Optics` section, the optical mode solver type (`FEScalar`, `FEVectorial`, `TMM1D`, or `EffectiveIndex`) is specified. The arguments for these mode solver types are discussed in:
 - [Syntax of FE Scalar and FE Vectorial Optical Solvers on page 855](#)
 - [Transfer Matrix Method for VCSELs on page 867](#)
 - [Effective Index Method for VCSELs on page 869](#)

- Optics stand-alone simulation does not support multimode simulation, that is, ModeNumber=1.
- In the Solve section, the stand-alone calculation of the optical mode solver is activated by the sole keyword Optics.

Automatic Optical Mode Searching

The cavity and waveguide mode solvers in Sentaurus Device require you to choose an initial guess so that a sparse matrix solver can be used to find the correct optical mode. It is not always easy to estimate a good initial guess. To circumvent this problem, a model has been developed to help identify the probability of physical modes through a mode density calculation. The concept is simple: A Hertzian dipole is placed in the cavity or waveguide, and the deterministic equations:

$$\nabla \times (\nabla \times \bar{E}) - \frac{\omega^2}{c^2} \epsilon_r \bar{E} = -i\omega\mu \cdot j_{src}(r, \omega) \quad (974)$$

are solved for the cylindrically symmetric cavity resonance problem and:

$$\nabla \times (\nabla \times \bar{E}) - \frac{\omega^2}{c^2} \epsilon_r(x, y) \cdot \bar{E}(x, y) + \gamma^2 \cdot \bar{E}(x, y) = -i\omega\mu \cdot j_{src}(r, \omega) \quad (975)$$

are solved for the waveguide problem.

The source dipole $j_{src}(r, \omega)$ is placed strategically in the cavity (see [Figure 89](#)) or waveguide (see [Figure 90 on page 890](#)).

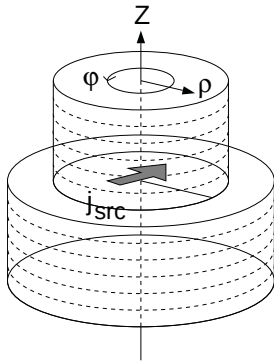


Figure 89 Dipole source in a cylindrically symmetric cavity problem can be radially or azimuthally defined

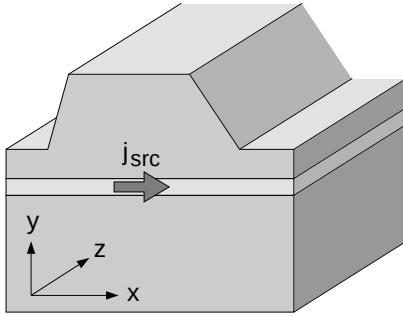


Figure 90 Dipole source in a waveguide problem can be orientated in x-direction or y-direction to excite TE and TM modes

The source dipole will excite a plethora of modes containing guided, resonating, and leaky modes. By sweeping through the wavelengths for the cavity problem or the effective indices for the waveguide problem, the change of energy in the cavity or waveguide at each frequency or effective index can be computed. This energy change is most sensitive near a resonating or guided mode and, therefore, provides the key to identifying the physical resonating or guided modes. The total energy contained in the system is given by:

$$\int \int_{\text{surf}} (\bar{S} \cdot d\bar{A}) = -\frac{1}{2} \iiint_V \{ \sigma \bar{E} \cdot \bar{E}^* + i\omega(\mu \bar{H} \cdot \bar{H}^* + \epsilon \bar{E} \cdot \bar{E}^*) \} dV \quad (976)$$

The real part gives the physically dissipated energy of the system and the imaginary part gives the energy stored in the system. In any system containing resonances, it is well known that the energy dissipation at the resonance point reduces sharply to a minimum. Therefore, by tracking the changes in dissipated energy, it is possible to estimate the resonant wavelength or effective index of the required mode.

There are remote possibilities that the dipole source is placed at the null of the required mode. In this case, the mode will not be excited and will not show up in the plot of the change in dissipation energy. You can place the dipole source at alternative positions to ensure that the required mode is excited.

Syntax for Automatic Mode Searching

There are some differences in the syntax for the automatic mode search for the cavity and waveguide problems. [Table 214 on page 1285](#) gives a summary of the keywords for `DeterministicProblem()`.

Searching for Cavity Resonances

Cylindrical symmetry is assumed for the cavity problem. This is applicable to the case of cylindrically symmetric VCSEL structures. The syntax for setting up the deterministic search for cavity resonances is:

```
File {
  ...
  SaveOptSpectrum = "spectrum"
}
...
Physic {
  Laser (
    Optics (
      DeterministicProblem (
        EquationType = Cavity
        Coordinates = Cylindrical
        AzimuthalExpansion = 1
        SourcePosition (0.25 0.5)      # [um]
        SourcePolarization = rhoDirection
        Sweep (740.0 760.0 80)        # [nm]
      )
    )
    VCSEL()
  )
  ...
}
....
Solve { Optics }
```

The SourcePolarization of the dipole can be chosen to be in different directions. The range of wavelengths to search is given in units of nanometers. An example of the output plot is shown in [Figure 91 on page 892](#). As expected, the resonances occur at locations with sharp energy changes.

36: Optical Mode Solver for Lasers

Automatic Optical Mode Searching

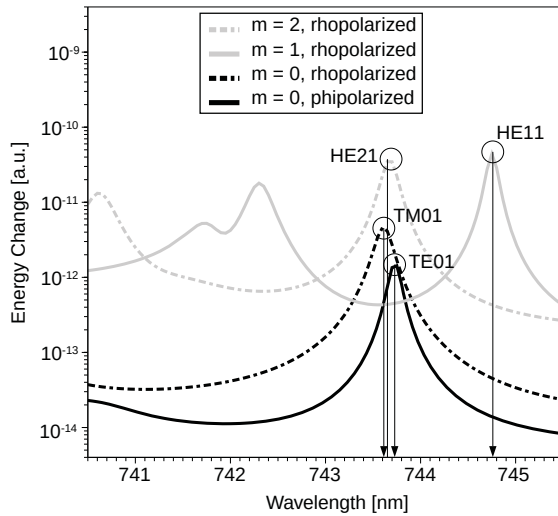


Figure 91 Energy changes plotted as a function of wavelength for the cavity problem; resonances of the different modes are clearly identified

Searching for Waveguide Modes

Searching for waveguide modes requires sweeping through the effective indices, and the syntax for this is:

```
File {
  SaveOptSpectrum = "spectrum"
}
...
Physics {
  Laser (
    Optics (
      DeterministicProblem (
        EquationType = Waveguide
        Symmetry = Symmetric
        Boundary = Type1
        Coordinates = Cartesian
        LasingWavelength = 530 # [nm]
        SourcePosition (1.0 1.533)
        SourcePolarization = xDirection
        Sweep (3.391 3.401 100)
      )
    )
  )
}
...
Solve { Optics }
```

An example of the output is shown in [Figure 92](#).

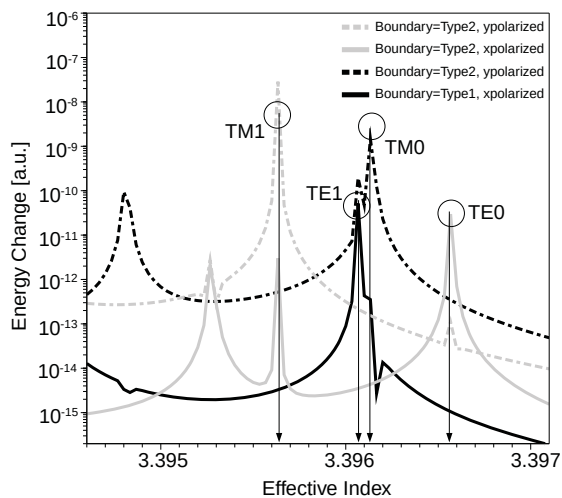


Figure 92 Energy changes plotted as a function of the effective index; guided modes of the waveguide are clearly identified

References

- [1] J. Jin, *The Finite Element Method in Electromagnetics*, New York: John Wiley & Sons, 2nd ed., 2002.
- [2] G. L. G. Sleijpen *et al.*, "Jacobi-Davidson Type Methods for Generalized Eigenproblems and Polynomial Eigenproblems," *BIT*, vol. 36, no. 3, pp. 595–633, 1996.
- [3] D. R. Fokkema, G. L. G. Sleijpen, and H. A. van der Vorst, "Jacobi–Davidson Style QR and QZ Algorithms for the Reduction of Matrix Pencils," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 94–125, 1998.
- [4] M. Streiff *et al.*, "A Comprehensive VCSEL Device Simulator," *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 9, no. 3, pp. 879–891, 2003.
- [5] W. C. Chew and W. H. Weedon, "A 3D Perfectly Matched Medium from Modified Maxwell's Equations with Stretched Coordinates," *Microwave and Optical Technology Letters*, vol. 7, no. 13, pp. 599–604, 1994.
- [6] J.-P. Berenger, "A Perfectly Matched Layer for the Absorption of Electromagnetic Waves," *Journal of Computational Physics*, vol. 114, no. 2, pp. 185–200, 1994.
- [7] F. L. Teixeira and W. C. Chew, "Systematic Derivation of Anisotropic PML Absorbing Media in Cylindrical and Spherical Coordinates," *IEEE Microwave and Guided Wave Letters*, vol. 7, no. 11, pp. 371–373, 1997.

36: Optical Mode Solver for Lasers

References

- [8] F. L. Teixeira and W. C. Chew, "A General Approach to Extend Berenger's Absorbing Boundary Condition to Anisotropic and Dispersive Media," *IEEE Transactions on Antennas and Propagation*, vol. 46, no. 9, pp. 1386–1387, 1998.
- [9] S. L. Chuang, *Physics Of Optoelectronic Devices*, New York: John Wiley & Sons, 1995.

CHAPTER 37 Modeling Quantum Wells

This chapter presents the physics of carrier scattering (capture) into quantum wells, meshing restrictions for the quantum wells, and methods of gain calculations for the quantum wells and bulk regions.

These gain methods include the simple rectangular well model, automatic loading of precomputed gain tables, and gain physical model interface (PMI). Various gain-broadening mechanisms, strain effects, and nonlinear gain saturation effects are discussed. An advanced piezoelectric field screening model for GaN-type quantum wells is also presented.

Overview

The drift-diffusion transport phenomena contained in the continuity and hydrodynamic equations are not suitable for modeling the transport across the quantum well (QW) because the feature size of the quantum well is much smaller than the inelastic mean free path of the carrier.

In this section, the focus is on three aspects of modeling the quantum well:

- Carrier capture into the quantum well
- Radiative recombination processes important in a quantum well
- Gain calculations

The QW carrier capture process is treated with a ballistic approach. A few types of recombination processes are important in the quantum well:

- Auger and Shockley–Read–Hall (SRH) recombinations deplete the QW carriers, and they form the dark current.
- Radiative recombination contains the stimulated and spontaneous recombination processes, which are important processes in lasers and LEDs.

These recombinations must be added to the carrier continuity equations to ensure the conservation of particles.

The gain calculation is based on Fermi's golden rule and describes quantitatively the radiative emissions in the form of the stimulated and spontaneous emission coefficients. These coefficients contain the optical matrix element $|M_{ij}|^2$, which describes the probability of the radiative recombination processes. In the quantum well, computing the optical matrix element requires knowledge of the QW subbands and QW wavefunctions.

Sentaurus Device offers two options for computing the gain spectrum:

- A simple finite well model with analytic solutions. In addition, strain effects and polarization dependence of the optical matrix element are handled separately.
- User-specified gain routines in C++ language can be coupled self-consistently with Sentaurus Device using the physical model interface (PMI).

Carrier Capture in Quantum Wells

A ballistic transport approach is used to handle the carrier capture or escape into or out of a quantum well. It follows the treatment of Grupen and Hess [1]. This approach is illustrated in Figure 93.

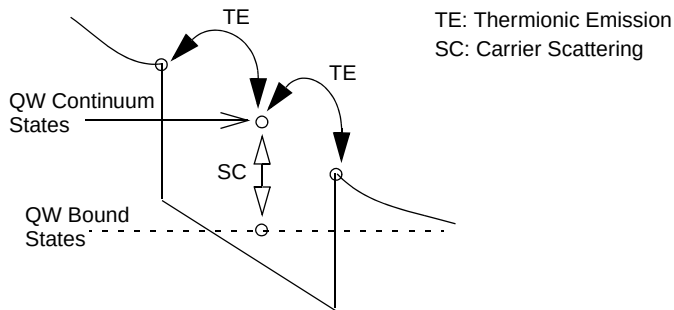


Figure 93 Discretization of quantum well to handle the physics of quantum well transport

The transport perpendicular to the QW plane is treated as follows. The QW is treated as a point source of recombination, which contains separate continuum and bound states. In this case, the continuum and bound states are assumed to have different quasi-Fermi levels that lead to separate continuity equations for the continuum and bound states. Transport of carriers from the regions outside the QW to the QW continuum states is by thermionic emission. The transition from the QW continuum to the bound states is computed by a scattering rate that includes carrier-carrier scattering. The bound states are solved from the Schrödinger equation.

Within the quantum well in the direction parallel to the QW plane, drift-diffusion transport is assumed to be valid.

Special Meshing Requirements for Quantum Wells

As a result of the transport in Figure 93, the spatial discretization of the mesh at the quantum well must be treated specially. The discretization is based on a ‘three-point’ model in the direction perpendicular to the QW plane. Only one vertex is placed in the quantum well and

one vertex is at each quantum well–bulk interface (see [Figure 94](#)). In the quantum well, only rectangular elements are allowed. This must be ensured when building the mesh.

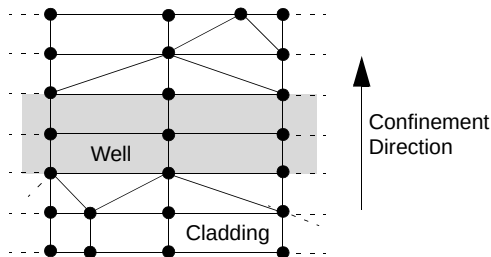


Figure 94 Discretization of quantum well

NOTE The discretization constraint for the quantum well can be chosen by creating a refinement region at the quantum well in the mesh generator.

Thermionic Emission

Thermionic emission is used at the quantum well interface to handle the large momentum changes caused by the band-edge discontinuity. If thermionic emission is used only, without the QW scattering model, all the carriers are assumed to be totally captured in the quantum well.

The default setting uses thermionic emission at the quantum well interfaces. This is activated by default if the keyword `QWTransport` is used in the Physics-Laser section of the command file:

```
Physics {...
  Laser (...
    Optics (...)
    # ----- Specify QW model and physics -----
    QWTransport
    QWExtension = AutoDetect    # QW widths auto detection
  )
}
```

The QW region must be identified by the keyword `Active` (see [Simulating Single-Grid Edge-Emitting Laser on page 760](#)) and must be discretized according to the constraints of the special ‘three-point’ QW model. The keyword `QWTransport` informs Sentaurus Device to use the ‘three-point’ QW model to treat the Active quantum well region.

Radiative Recombination and Gain Coefficients

After the carriers are captured in the active region, they experience either dark recombination processes (such as Auger and SRH) or radiative recombination processes (such as stimulated and spontaneous emissions), or escape from the active region. This section describes how stimulated and spontaneous emissions are computed in Sentaurus Device.

Stimulated and Spontaneous Emission Coefficients

In the active region of the laser, radiative recombination is treated locally at each active vertex. The stimulated and spontaneous emissions are computed using Fermi's golden rule.

At each active vertex of the quantum wells, the local stimulated emission coefficient is:

$$r^{\text{st}}(\hbar\omega) = \sum_{i,j} \int dE C_0 k_{\text{st}} |M_{i,j}|^2 D(E) \times (f_i^C(E) + f_j^V(E) - 1) L(E) \quad (977)$$

and the local spontaneous emission coefficient is:

$$r^{\text{sp}}(\hbar\omega) = \sum_{i,j} \int dE C_0 k_{\text{sp}} |M_{i,j}|^2 D(E) f_i^C(E) f_j^V(E) L(E) \quad (978)$$

where:

$$C_0 = \frac{\pi e^2}{n_g c \epsilon_0 m_0^2 \omega} \quad (979)$$

$$|M_{i,j}|^2 = P_{ij} |O_{i,j}|^2 \left(\frac{m_0}{m_e} - 1 \right) \frac{m_0 E_g (E_g + \Delta)}{12 \left(E_g + \frac{2}{3} \Delta \right)} \quad (980)$$

$$O_{i,j} = \int_{-\infty}^{\infty} dx \zeta_i(x) \zeta_j^*(x) \quad (981)$$

$$f_{(i, \Phi_n, E)}^C = \left(1 + \exp \left(\frac{E_C + E_i + q\Phi_n + \frac{m_r}{m_e} E}{k_B T} \right) \right)^{-1} \quad (982)$$

$$f_{(j, \Phi_p, E)}^V = \left(1 + \exp \left(\frac{E_V - E_j + q\Phi_p - \frac{m_r}{m_h} E}{k_B T} \right) \right)^{-1} \quad (983)$$

$$D_{(E)}^r = \frac{m_r}{\pi \hbar^2 L_x} \quad (984)$$

$$m_r = \left(\frac{1}{m_e} + \frac{1}{m_h} \right)^{-1} \quad (985)$$

$L(E)$ is the gain-broadening function. The electron, light-hole, and heavy-hole subbands are denoted by the indices i and j . $f_{i(E)}^C$ and $f_{j(E)}^V$ are the local Fermi–Dirac distributions for the conduction and valence bands, $D_{(E)}^r$ is the reduced density-of-states, $|O_{i,j}|^2$ is the overlap integral of the quantum mechanical wavefunctions, and P_{ij} is the polarization-dependent factor of the momentum (optical) matrix element $|M_{i,j}|^2$. The spin-orbit split-off energy is Δ and E_g is the bandgap energy. The anisotropic polarization-dependent factor P_{ij} in the optical matrix element is discussed in [Polarization-dependent Optical Matrix Element on page 910](#). These emission coefficients determine the rate of production of photons when given the number of available quantum well carriers at the active vertex.

k_{st} and k_{sp} are scaling factors for the optical matrix element $|M_{i,j}|^2$ of the stimulated and spontaneous emissions, respectively. They have been introduced to allow you to tune the stimulated and spontaneous gain curves. Consequently, these parameters can change the threshold current.

The activating keywords are `StimScaling` and `SponScaling` in the Physics-Laser section of the command file:

```
Physics { ...
  Laser ( ...
    Optics ( ...
      # ---- Scale stimulated & spontaneous gain ----
      StimScaling = 1.0 # default value is 1.0
      SponScaling = 1.0 # default value is 1.0
    )
  }
}
```

As an alternative to the full spontaneous emission model given by [Eq. 978](#), a simpler model is available in Sentaurus Device that is described by:

$$r^{sp}(\hbar\omega) = C \cdot n \cdot p \cdot \left(\frac{T}{T_{par}} \right)^\alpha \quad (986)$$

37: Modeling Quantum Wells

Radiative Recombination and Gain Coefficients

where C is the coefficient of radiative recombination, n is the electron density, p is the hole density, T is the temperature, T_{par} is the reference temperature, and α is a temperature exponent. The model parameters C , T_{par} , and α must be entered in the RadiativeRecombination section of the parameter file as `C` (with unit [cm^3s^{-1}]), `Tpar` (with unit [K]), and `alpha` (with unit [1]), respectively.

The differential gains for electrons and holes are given by the derivatives of the coefficient of stimulated emission $r^{\text{st}}(\hbar\omega)$ (see Eq. 977) with regard to the respective carrier density. They can be plotted by including the keywords `eDifferentialGain` and `hDifferentialGain` in the `Plot` section of the command file.

Active Bulk Material Gain

The stimulated and spontaneous emission coefficients discussed are derived for the quantum well. However, these coefficients can apply to bulk materials with slight modifications. In bulk active materials, it is assumed that the optical matrix element is isotropic. The sum over the subbands is reduced to one electron, and one heavy-hole and one light-hole level, because there is no quantum-mechanical confinement in bulk material. In addition, the subband energies are set to $E_i = 0$, and the following coefficients are modified:

$$O_{i,j} = 1 \quad (987)$$

$$D^r(E) = \frac{1}{2\pi^2} \left(\frac{2m_r}{\hbar^2} \right)^{3/2} E^{1/2} \quad (988)$$

$$P_{i,j} = 1 \quad (989)$$

All other expressions remain the same.

Stimulated Recombination Rate

The radiative emissions contribute to the production of photons but they also deplete the carrier population in the active region. At each active vertex, the stimulated recombination rate of the carriers must be equal to the sum of the photon production rate of every lasing mode so that conservation of particles is ensured. The stimulated recombination rate for each active vertex is:

$$R^{\text{st}}(x, y) = \sum_i r^{\text{st}}(\hbar\omega_i) S_i |\Psi_i(x, y)|^2 \quad (990)$$

where the sum is taken over all lasing modes. The stimulated emission coefficient is computed locally at this active vertex and its value is taken at the lasing energy, $\hbar\omega_i$, of mode i . S_i is the photon rate of mode i , solved from the corresponding photon rate equation of mode i , and $|\Psi_i(x, y)|^2$ is the local optical field intensity of mode i at this active vertex. This stimulated recombination rate is entered in the continuity equations to account for the correct depletion of carriers by stimulated emissions.

The total stimulated recombination rates on active vertices can be plotted by including the keyword `StimulatedRecombination` in the `Plot` section of the command file.

Spontaneous Recombination Rate

Spontaneous emissions also deplete the carrier population in the active region and must be taken into consideration. The spontaneous recombination rate at each active vertex is:

$$R_{\text{tot}}^{\text{sp}}(x, y, z) = \int_0^{\infty} r^{\text{sp}}(E) \rho^{\text{opt}}(E) dE \quad (991)$$

where the optical mode density is:

$$\rho^{\text{opt}}(E) = \frac{n_g^2 E^2}{\pi^2 \hbar^3 c^2} \quad (992)$$

The spontaneous recombination rate is an integral over energy space, and this rate is entered into the carrier continuity equations.

The total spontaneous recombination rates on active vertices can be plotted by including the keyword `SpontaneousRecombination` in the `Plot` section of the command file.

Fitting Stimulated and Spontaneous Emission Spectra

The stimulated and spontaneous emission spectra can be fine-tuned by scaling and shifting. The spectra can be scaled in magnitude by the keywords `StimScaling=<float>` and `SponScaling=<float>` in the `Physics-Laser` or `Physics-LED` section. It is also possible to shift the effective emission wavelength (and, therefore, the spectra) by an energy amount specified as `GainShift=<float>`.

Gain-broadening Models

Three different line-shape broadening models are available: Lorentzian, Landsberg, and hyperbolic-cosine. These line-shape functions, $L(E)$, are embedded in the radiative emission coefficients in [Eq. 977](#) and [Eq. 978](#) to account for broadening of the gain spectrum.

Lorentzian Broadening

Lorentzian broadening assumes that the probability of finding an electron or a hole in a given state decays exponentially in time [\[2\]](#). The line-shape function is:

$$L(E) = \frac{\Gamma/(2\pi)}{(E_g - \hbar\omega + E)^2 + (\Gamma/2)^2} \quad (993)$$

Landsberg Broadening

The Landsberg model gives a narrower, asymmetric line-shape broadening, and its line-shape function is:

$$L(E) = \frac{(\Gamma(E))/(2\pi)}{(E_g - \hbar\omega + E)^2 + (\Gamma(E)/2)^2} \quad (994)$$

where:

$$\Gamma(E) = \Gamma \sum_{k=0}^3 a_k \left(\frac{E}{q\Psi_p - q\Psi_n} \right)^k \quad (995)$$

and $q\Psi_p - q\Psi_n$ is the quasi-Fermi level separation. The coefficients a_k are:

$$\begin{aligned} a_0 &= 1 \\ a_1 &= -2.229 \\ a_2 &= 1.458 \\ a_3 &= -0.229 \end{aligned} \quad (996)$$

Hyperbolic-Cosine Broadening

The hyperbolic-cosine function has a broader tail on the low-energy side compared to Lorentzian broadening, and the line-shape function is:

$$L(E) = \frac{1}{4\Gamma} \cdot \frac{1}{\cosh^2\left(\frac{E}{2\Gamma}\right)} \quad (997)$$

Syntax to Activate Broadening

You can select only one line-shape function for gain broadening. This is activated by the keyword **Broadening** in the Physics-Laser section of the command file:

```
Physics {...
  Laser (...
    Optics (...
      # --- Lineshape broadening functions, choose one only ----
      Broadening (Type=Lorentzian Gamma=0.01)
#    Broadening (Type=Landsberg Gamma=0.01)      # Gamma in [eV]
#    Broadening (Type=CosHyper Gamma=0.01)
    )
  }
```

Gamma is the line width Γ , which must be defined in units of eV. If no **Broadening** keyword is detected, Sentaurus Device assumes the gain is unbroadened and does not perform the energy integral in [Eq. 977](#) and [Eq. 978](#).

Nonlinear Gain Saturation Effects

Nonlinear gain saturation is caused by the interaction of increasing light intensity with the optical matrix element, and this effect is important in the study of modulation response. An infinite order perturbation approach is used in a density matrix formulation to derive the nonlinear gain effects [\[3\]](#). Using this approach, the stimulated emission coefficient is derived to be [\[4\]](#):

$$r^{\text{st}}(\hbar\omega) = \sum_{i,j} \int dE C_o k_{\text{st}} |M_{i,j}|^2 D(E) \times (f_i^C(E) + f_j^V(E) - 1) L(E) \quad (998)$$

where:

$$L(E) = \frac{\Gamma/(2\pi)}{(E - \hbar\omega)^2 + (\Gamma/2)^2 + \varepsilon S} \quad (999)$$

$$\varepsilon = \frac{2(\tau_c + \tau_v)|M_{i,j}|^2(\hbar\omega)\hbar^2}{\tau_{in}m_0^2\varepsilon_0n_g^2E} \cdot |\Psi(x,y)|^2 \quad (1000)$$

τ_c and τ_v are the intraband scattering times of the electrons and holes, respectively. Γ is the line width, which is defined by $\Gamma = 2(\hbar/\tau_{in})$. τ_{in} is the polarization relaxation time and can be defined as:

$$\frac{1}{\tau_{in}} = \frac{1}{2}\left(\frac{1}{\tau_c} + \frac{1}{\tau_v}\right) + \frac{1}{\tau_{sp}} \quad (1001)$$

where τ_{sp} is the electron spontaneous emission lifetime and $|\Psi(x,y)|^2$ is the normalized local optical intensity. The stimulated emission coefficient with nonlinear gain effects in [Eq. 998](#) is of the same form as the original stimulated emission coefficient in [Eq. 977](#). The variables in the two coefficients are the same, except for the gain-broadening function, $L(E)$.

The nonlinear gain saturation effect can be included in the form of a broadening function. The simple gain saturation model commonly cited is $g = g_0/(1 + \varepsilon S)$ (see below). This simple form gives a homogeneous broadening and can be approximated from [Eq. 998](#) by assuming $(\hbar/\tau_{in}) \gg (E - \hbar\omega)$. In this model, $|M_{i,j}|^2$ has no spectral dependence by definition.

Since the nonlinear gain saturation effect exists in the form of a broadening function, it can be activated by the keyword **Broadening** in the Physics-Laser section of the command file:

```
Physics {...
  Laser (...
    Optics (...)
    # ----- Gain saturation model -----
    Broadening(Type = LorentzianSat
      Gamma = 4.39e-4          # [eV]
      eIntrabandRelTime = 1e-13 # [s]
      hIntrabandRelTime = 1e-13 # [s]
      PolarizationRelTime = 3e-12 # [s]
    )
  )
}
```

You must set the electron and hole intraband scattering times with `eIntrabandRelTime` and `hIntrabandRelTime`, respectively. The `PolarizationRelTime` can be computed from [Eq. 1001](#).

NOTE If the nonlinear gain saturation is activated, you cannot select other gain-broadening functions.

As an alternative to the calculation of gain saturation, Sentaurus Device offers a simple local gain saturation model of the form $g = g_0/(1 + \epsilon S)$ where you must specify the gain saturation coefficient ϵ . In contrast to the model described above, this model is not implemented as a broadening model, but as a correction to the coefficient of stimulated emission (see [Eq. 998](#)).

The local epsilon model can be activated in the Physics section:

```
Physics {...
  Laser (...
    Optics (...
      # ----- Simple gain saturation model -----
      GainSaturation(
        Type = LocalEpsilonModel
        EpsilonSat = 1e-16
      )
    )
  )
}
```

Simple Quantum-Well Subband Model

This section describes the solution of the Schrödinger equation for a simple finite quantum-well model. This is the default model in Sentaurus Device. This simple quantum-well (QW) subband model is combined with separate QW strain (see [Strain Effects on page 908](#)) and polarization dependence of the optical matrix element (see [Polarization-dependent Optical Matrix Element on page 910](#)) to model most quantum well systems.

In a quantum well, the carriers are confined in one direction. Of interest are the subband energies and wavefunctions of the bound states, which can be solved from the Schrödinger equation. In this simple QW subband model, it is assumed that the bands for the electron, heavy hole, and light hole are decoupled, and the subbands are solved independently by a 1D Schrödinger equation.

The time-independent 1D Schrödinger equation in the effective mass approximation is:

$$\left(-\frac{\hbar^2}{2} \frac{\partial}{\partial x} \frac{1}{m(x)} \frac{\partial}{\partial x} + V(x) - E^i \right) \zeta^i(x) = 0 \quad (1002)$$

where $\zeta^i(x)$ is the i -th quantum mechanical wavefunction, E^i is the i -th energy eigenvalue, and $V(x)$ is the finite well shape potential.

With the following ansatz for the even wavefunctions:

$$\zeta(x) = C_1 \begin{cases} \cos\left(\frac{\kappa l}{2}\right) e^{-\alpha(|x| - l/2)} & , |x| > l/2 \\ \cos(\kappa x) & , |x| \leq l/2 \end{cases} \quad (1003)$$

and the odd wavefunctions:

$$\zeta(x) = C_2 \begin{cases} \pm \sin\left(\frac{\kappa l}{2}\right) e^{\mp(x \mp l/2)} & , |x| > l/2 \\ \sin(\kappa x) & , |x| \leq l/2 \end{cases} \quad (1004)$$

Eq. 1002 becomes [2]:

$$\alpha \frac{l}{2} + \frac{m_b}{m_w} \kappa \frac{l}{2} \cot\left(\kappa \frac{l}{2}\right) = 0 \quad (1005)$$

$$\alpha \frac{l}{2} - \frac{m_b}{m_w} \kappa \frac{l}{2} \tan\left(\kappa \frac{l}{2}\right) = 0 \quad (1006)$$

with:

$$\kappa = \frac{\sqrt{2m_w E}}{\hbar} \quad (1007)$$

$$\alpha = \frac{\sqrt{2m_b(\Delta E_c - E)}}{\hbar} \quad (1008)$$

The first transcendental equation gives the even eigenvalues, and the second one gives the odd eigenvalues. The wavefunctions are immediately obtained with Eq. 1003 and Eq. 1004 after the subband energy E has been computed. Having obtained the wavefunctions and subband energies, the carrier densities of the 1D-confined system are also computed by:

$$n(x) = N_e^{2D} \sum_i |\zeta_i(x)|^2 F_0(\eta_n - E_i) \quad (1009)$$

$$p(x) = N_{hh}^{2D} \sum_j |\zeta_j(x)|^2 F_0(\eta_p - E_{hh}^j) + N_{lh}^{2D} \sum_m |\zeta_m(x)|^2 F_0(\eta_p - E_{lh}^m) \quad (1010)$$

where $F_0(x)$ is the Fermi integral of the order 0, and η_n , η_p denote the chemical potentials. The indices hh and lh denote the heavy and light holes, respectively.

The effective densities of states are:

$$N_e^{2D} = \frac{k_B T m_e}{\hbar^2 \pi L_x} \quad (1011)$$

$$N_{lh/hh}^{2D} = \frac{k_B T m_{lh/hh}}{\hbar^2 \pi L_x} \quad (1012)$$

where L_x is the thickness of the quantum well. The thickness of each quantum well is automatically detected in Sentaurus Device by scanning the material regions for the keyword Active.

The effective masses of the carriers in the quantum well can be changed inside the parameter file:

```
eDOSMass
{
  * For effective mass specification Formula1 (me approximation):
  * or Formula2 (Nc300) can be used :
    Formula = 2      # [1]
  * Formula2:
  * me/m0 = (Nc300/2.540e19)^2/3
  * Nc(T) = Nc300 * (T/300)^3/2
    Nc300      = 8.7200e+16    # [cm-3]
  * Mole fraction dependent model.
  * If just above parameters are specified, then its values will be
  * used for any mole fraction instead of an interpolation below.
  * The linear interpolation is used on interval [0,1].
    Nc300(1)      = 6.4200e+17    # [cm-3]
}
...
SchroedingerParameters:
{
  * For the hole masses for Schroedinger equation you can
  * use different formulas.
  * formula=1 (for materials with Si-like hole band structure)
  * m(k)/m0=1/(A+sqrt(B+C*((xy)^2+(yz)^2+(zx)^2)))
  * where k=(x,y,z) is unit normal vector in reciprocal
  * space. '+' for light hole band, '-' for heavy hole band
  * formula=2: Heavy hole mass mh and light hole mass ml are
  * specified explicitly.
  * Formula 2 parameters:
    Formula = 2      # [1]
    ml      = 0.027 # [1]
    mh      = 0.08  # [1]
  * Mole fraction dependent model.
  * If just above parameters are specified, then its values will be
  * used for any mole fraction instead of an interpolation below.
```

```
* The linear interpolation is used on interval [0,1].
    ml(1) = 0.094 # [1]
    mh(1) = 0.08 # [1]
}
```

Syntax for Simple Quantum-Well Model

This simple QW subband model is the default model when the QWTransport model is activated. [Table 211 on page 1282](#) provides the keywords that are associated with this simple QW model.

Strain Effects

In semiconductor laser design, it is well known that strain of the quantum well modifies the laser characteristics. Due to the deformation potentials in the crystal at the well–bulk interface and valence band mixing effects, band structure modifications occur mainly for the valence bands. They have an impact on the optical recombination and transport properties.

In the simple QW subband model discussed in the previous section, a simpler approach to the QW strain effects is adopted. The simple QW subband model does not include nonparabolicities of the band structure, arising from valence band mixing and strain, in a rigorous manner. However, by carefully selecting the effective masses in the well, a good approximation of the strained band structure can be obtained [5]. The effective masses can be changed in the parameter file as previously shown.

Basically, strain has two impacts on the band structure. Due to the deformation potentials, the effective band offsets of the conduction and valence bands are modified. This is included in the simple QW subband model by:

$$\delta E_C = 2a_c \left(1 - \frac{C_{12}}{C_{11}}\right) \varepsilon \quad (1013)$$

$$\delta E_V^{HH} = 2a_v \left(1 - \frac{C_{12}}{C_{11}}\right) \varepsilon + b \left(1 + 2 \frac{C_{12}}{C_{11}}\right) \varepsilon \quad (1014)$$

$$\delta E_V^{LH} = 2a_v \left(1 - \frac{C_{12}}{C_{11}}\right) \varepsilon - b \left(1 + 2 \frac{C_{12}}{C_{11}}\right) \varepsilon + \frac{\Delta}{2} - \left(-\frac{1}{2} \sqrt{\Delta^2 + 9\delta E_{sh}^2} - 2\delta E_{sh} \Delta\right) \quad (1015)$$

where a_n and a_c are the hydrostatic deformation potential of the conduction and valence bands, respectively. The shear deformation potential is denoted with b and Δ is the spin-orbit split-off energy. The elastic stiffness constants are C_{11} and C_{12} , and ε is the relative lattice

constant difference in the active region. The hydrostatic component of the strain shifts the conduction band offset by δE_C and shifts the valence band offset by $\delta E_V^0 = 2a_v(1 - C_{12}/C_{11})\epsilon$.

The shear component of the strain decouples the light hole and heavy hole bands at the Γ point, and shifts the valence bands by an amount of $\delta E_{sh} = b(1 + 2C_{12}/C_{11})\epsilon$ in opposite directions.

Syntax for Quantum-Well Strain

The strain shift can be activated by the keyword `Strain` in the `Physics - Laser` section of the command file:

```
Physics {...
  Laser (...
    Optics (...)
    # --- QW physics ---
    QWTransport
    QWExtension = AutoDetect
    # --- QW strain ---
    Strain
  )
}
```

The parameters a_n , a_c , and b can be entered as `a_nu`, `a_c`, and `b_shear`, respectively, in the `QWStrain` section of the parameter file:

```
QWStrain
{
  * Deformation Potentials (a_nu, a_c, b, C_12, C_11
  * and strainConstant eps :
  * Formula:
  * eps = (a_bulk - a_active)/a_active
  * dE_c = ...
  * dE_lh = ...
  * dE_hh = ...
  eps = -1.0000e-02      # [1]
  * a_nu = 1.27          # [1]
  * a_c = -5.0400e+00    # [1]
  * b_shear = -1.7000e+00 # [1]
  * C_11 = 10.11         # [1]
  * C_12 = 5.61          # [1]
}
```

The elastic stiffness constants C_{11} and C_{12} can be specified by `C_11` and `C_12`. Due to valence band mixing and strain, the valence bands can become nonparabolic. However, within a small

37: Modeling Quantum Wells

Polarization-dependent Optical Matrix Element

range from the band edge, parabolicity can still be assumed. In the simulation, you can modify the effective heavy hole and light hole masses in the parameter file for the subband calculation to account for this effect.

The spin-orbit split-off energy can be specified in the `BandstructureParameters` section of the parameter file:

```
BandstructureParameters{  
    ...  
    so = 0.34    # [eV]  
    ...  
}
```

Polarization-dependent Optical Matrix Element

The polarization dependence of the optical matrix element P_{ij} in [Eq. 980, p. 898](#) can be treated in an elegant way for the simple QW subband model. Define the optical polarization angle, γ , as the angle of the vectorial optical field between itself and the quantum well (QW) plane, as shown in [Figure 95](#).

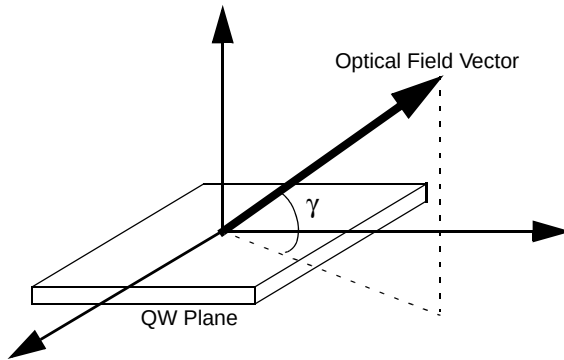


Figure 95 Taking the optical polarization with respect to QW plane so that anisotropic polarization-dependent factor in optical matrix element can be defined

If the vector lies completely in the QW plane as in the case of TE modes, the angle is 0. If it is strictly perpendicular as in TM modes, it is $\pi/2$. The unit is radian.

These polarization angles at each vertex can be visualized by including the keywords `OpticalPolarizationAngleMode0` to `OpticalPolarizationAngleMode9` in the `Plot` section of the command file.

NOTE In bulk material, the optical matrix element is isotropic and the polarization-dependent factor can be taken as $P_{ij} = 1$.

The polarization-dependent factor P_{ij} is a measure of the influence of the polarization of the optical field with the plane wavefunctions of the optical transition. Suppose the optical field polarization is defined by the unit vector:

$$\hat{e} = \alpha \hat{a}_{TE} + \beta \hat{a}_{TM} \quad (1016)$$

where $\alpha^2 + \beta^2 = 1$. It is obvious that $\alpha = \cos\gamma$ and $\beta = \sin\gamma$ from [Figure 95](#).

The resultant polarization-dependent factor P_{ij} for a quantum well can be derived as:

$$\begin{aligned} P_{c, hh} &= \frac{1}{M_b^2} \left| \hat{e} \cdot \langle iSi' | \bar{p} | \frac{3}{2}, -\frac{3}{2} \rangle' \right|^2 \\ &= 3 \left\{ \frac{1}{4} \alpha^2 \cos^2 \Psi + \frac{1}{4} \alpha^2 + \frac{1}{2} \beta^2 \sin^2 \Psi \right\} \end{aligned} \quad (1017)$$

for C-HH transitions, and:

$$\begin{aligned} P_{c, lh} &= \frac{1}{M_b^2} \left\{ \left| \hat{e} \cdot \langle iSi' | \bar{p} | \frac{3}{2}, -\frac{1}{2} \rangle' \right|^2 + \left| \hat{e} \cdot \langle iSi' | \bar{p} | \frac{3}{2}, \frac{1}{2} \rangle' \right|^2 \right\} \\ &= 3 \left\{ \frac{1}{3} \alpha^2 \sin^2 \Psi + \frac{1}{12} \alpha^2 \cos^2 \Psi + \frac{1}{12} \alpha^2 + \frac{2}{3} \beta^2 \cos^2 \Psi + \frac{1}{6} \beta^2 \sin^2 \Psi \right\} \end{aligned} \quad (1018)$$

for C-LH transitions.

The $\mathbf{k}$ -wavevector angle is defined as:

$$\cos^2 \Psi = \frac{E_g + E_i + E_j}{E}, E > E_g + E_i + E_j \quad (1019)$$

The cross-coupling terms between the TE and TM contributions evaluate to zero upon integration over the polar angle in the QW plane. This is equivalent to computing different TE-stimulated and TM-stimulated emission coefficients and, therefore, different modal gains for the TE and TM polarizations, which are subsequently summed to give the overall modal gain of the mode at each active vertex, that is:

$$G_{\text{modal}}(x, y) = r_{TE}^{st} |\bar{E}_{TE}(x, y)|^2 + r_{TM}^{st} |\bar{E}_{TM}(x, y)|^2 \quad (1020)$$

where $|\bar{E}_{TE}(x, y)|^2$ and $|\bar{E}_{TM}(x, y)|^2$ are the local normalized intensities of the vectorial optical field projected onto the TE and TM planes, respectively. In this way, automatic polarization-dependent gain calculations for each separate vertex in the active region are achieved if the vectorial optical solver (FEVectorial) is chosen.

37: Modeling Quantum Wells

Polarization-dependent Optical Matrix Element

In the case of scalar optical solvers (FEScalar), you can select the type of polarization (TE or TM) in the Physics-Laser-Optics-FEScalar section of the command file:

```
Physics {...
  Laser (...
    Optics(
      FEScalar(
        EquationType = Waveguide
        Symmetry = Symmetric
        LasingWavelength = 800          # [nm]
        TargetEffectiveIndex = (3.4 3.34) # initial guess
        TargetLoss = (10.0 12.0)         # initial guess [1/cm]
        # --- Specify the optical polarization ---
        Polarization = (TE TM)
        ModeNumber = 2
      )
    )
  )
}
```

The TE and TM polarizations give a global optical polarization angle of 0 and $\pi/2$, respectively. The default is TE polarization. For purely TE or TM polarizations, [Eq. 1017](#) and [Eq. 1018](#) reduce to the values in [Table 133](#).

Table 133 Polarization-dependent factor for purely TE and TM polarizations

	TE polarization	TM polarization
C-HH	$\frac{3}{4}(1 + \cos^2\Psi)$	$\frac{3}{2}\sin^2\Psi$
C-LH	$\frac{5}{4} - \frac{3}{4}\cos^2\Psi$	$\frac{1}{2}(1 + 3\cos^2\Psi)$

A self-consistent simulation of four vectorial modes is performed for an edge-emitting laser to show the polarization-dependent effects on the gain. Figure 96 shows the modal gain plotted (using Inspect) as a function of transition energy for the four different modes.

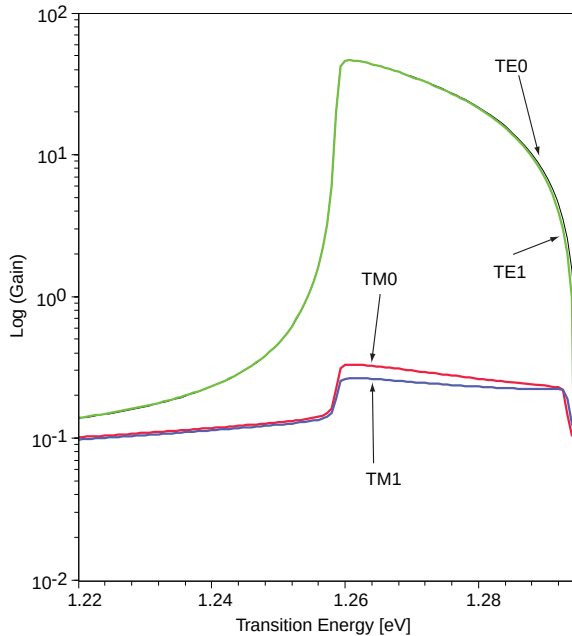


Figure 96 Modal gain curves for two TE and two TM modes

The four modes contain two quasi-TE (TE0 and TE1) and two quasi-TM (TM0 and TM2) modes. The TE gains are much greater than the TM gains because the InGaAs quantum wells used in this example strongly favor TE polarization. Therefore, Sentaurus Device can simulate multimode lasing with mixed optical polarization.

Loading Tabulated Stimulated and Spontaneous Emission

To speed up simulations, nevertheless, including full electronic bandstructure information and the most accurate manybody gain calculations, it is possible to load precomputed tables of stimulated and spontaneous emission data into laser and LED simulations. The tables can be computed by an external user-defined routine and saved in .hdf format. For further information, contact the TCAD Support Team.

When the tables have been computed, the loader can be activated by using the `SponEmissionCoeff` and `StimEmissionCoeff` keywords in the Physics-Laser section of the command file and by specifying the file names of the tables to be loaded. Different tables can be loaded for different regions or materials.

37: Modeling Quantum Wells

Importing Gain and Spontaneous Emission Data with PMI

The command file syntax is:

```
Physics{ ...
  Laser( ...
    StimEmissionCoeff(Tabulated)
    SponEmissionCoeff(Tabulated)
  )
  ...
}
File{
  StimEmissionTable = "somename.hdf" # global specification
  SponEmissionTable = "somename.hdf"
}
```

or alternatively:

```
File(Material = "GaAs") {
  StimEmissionTable = "somename.hdf" # materialwise specification
  SponEmissionTable = "somename.hdf"
}
```

or:

```
File(Region = "someregion") {
  StimEmissionTable = "somename.hdf" # regionwise specification
  SponEmissionTable = "somename.hdf"
}
```

Regionwise specifications override materialwise specifications, which, in turn, override a global specification.

Importing Gain and Spontaneous Emission Data with PMI

Sentaurus Device can import external stimulated and spontaneous emission data through the physical model interface (PMI). The gain PMI concept is illustrated in [Figure 97 on page 915](#).

Sentaurus Device calls the user-written gain calculations through the PMI with the variables: electron density n , hole density p , electron temperature eT , hole temperature hT , and transition energy E . The user-written gain calculation then returns the gain g and the derivatives of gain with respect to n , p , eT , and hT to Sentaurus Device. The derivatives are required to ensure proper convergence of the Newton iterations. In this way, the user-import gain is made self-consistent within the laser simulation.

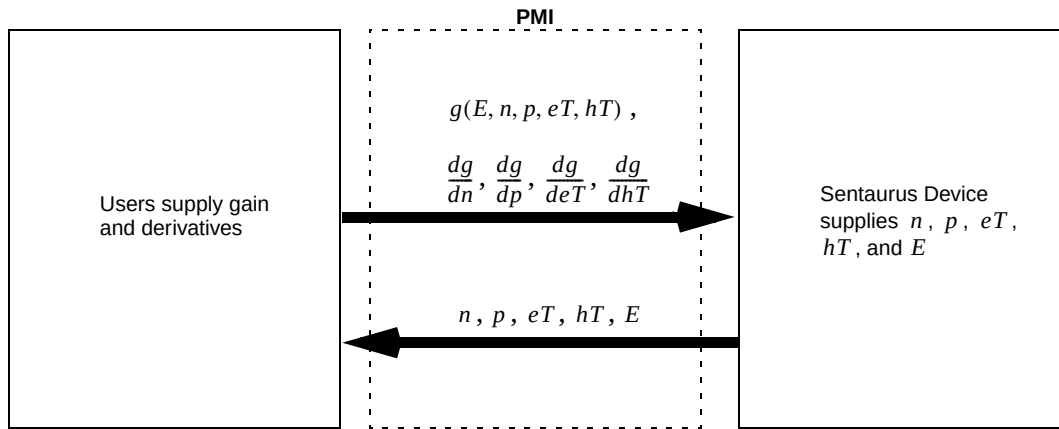


Figure 97 Concept of the gain PMI

Implementing Gain PMI

The PMI uses the object-orientation capability of the C++ language (see [Chapter 41 on page 979](#)). A brief outline is given here of the gain PMI.

In the Sentaurus Device header file `PMIModels.h`, the following base class is defined for gain:

```
class PMI_StimEmissionCoeff : public PMI_Vertex_Interface {
public:
    PMI_StimEmissionCoeff (const PMI_Environment& env);
    virtual ~PMI_StimEmissionCoeff ();

    virtual void Compute_rstim
    (double E,
     double n,
     double p,
     double et,
     double ht,
     double& rstim) = 0;

    virtual void Compute_drstimdn
    (double E,
     double n,
     double p,
     double et,
     double ht,
     double& drstimdn) = 0;

    virtual void Compute_drstimdp
    (double E,
```

37: Modeling Quantum Wells

Importing Gain and Spontaneous Emission Data with PMI

```
double n,  
double p,  
double et,  
double ht,  
double& drstimdp) = 0;  
  
virtual void Compute_drstimdet  
(double E,  
double n,  
double p,  
double et,  
double ht,  
double& drstimdet) = 0;  
  
virtual void Compute_drstimdht  
(double E,  
double n,  
double p,  
double et,  
double ht,  
double& drstimdht) = 0;  
};
```

To implement the PMI model for gain, you must declare a derived class in the user-written header file:

```
#include "PMIModels.h"  
  
class StimEmissionCoeff : public PMI_StimEmissionCoeff {  
// User-defined variables for his/her own routines  
private:  
    double a, b, c, d;  
  
public:  
    // Need a constructor and destructor for this class  
    StimEmissionCoeff (const PMI_Environment& env);  
    ~StimEmissionCoeff ();  
  
    // --- User needs to write the following routines in the .C file ---  
    // The value of the function is return as the last pointer argument  
  
    // stimulated emission coeff value  
    void Compute_rstim (double E,  
                        double n,  
                        double p,  
                        double et,  
                        double ht,  
                        double& rstim);
```

```

// derivative wrt n
void Compute_drstimdn (double E,
                      double n,
                      double p,
                      double et,
                      double ht,
                      double& drstimdn);

// derivative wrt p
void Compute_drstimdp (double E,
                      double n,
                      double p,
                      double et,
                      double ht,
                      double& drstimdp);

// derivative wrt et
void Compute_drstimdet (double E,
                      double n,
                      double p,
                      double et,
                      double ht,
                      double& drstimdet);

// derivative wrt ht
void Compute_drstimdht (double E,
                      double n,
                      double p,
                      double et,
                      double ht,
                      double& drstimdht);

};

```

Next, you must write the functions `Compute_rstim`, `Compute_drstimdn`, `Compute_drstimdp`, `Compute_drstimdet`, and `Compute_drstimdht` to return the values of the stimulated emission coefficient and its derivatives to Sentaurus Device using this gain PMI. If you have, for example, a table of gain values, you must implement the above functions to interpolate the values of the gain and derivatives from the table.

The spontaneous emission coefficient can also be imported using the PMI. The implementation is exactly the same as the stimulated emission coefficient, and you only need to replace `StimEmissionCoeff` with `SponEmissionCoeff` in the above code example.

References

- [1] M. Grupen and K. Hess, "Simulation of Carrier Transport and Nonlinearities in Quantum-Well Laser Diodes," *IEEE Journal of Quantum Electronics*, vol. 34, no. 1, pp. 120–140, 1998.
- [2] L. A. Coldren and S. W. Corzine, *Diode Lasers and Photonic Integrated Circuits*, New York: John Wiley & Sons, 1995.
- [3] M. Ahmed and M. Yamada, "An infinite order perturbation approach to gain calculation in injection semiconductor lasers," *Journal of Applied Physics*, vol. 84, no. 6, pp. 3004–3015, 1998.
- [4] B. Witzigmann, A. Witzig, and W. Fichtner, "Nonlinear Gain Saturation for 2-Dimensional Laser Simulation," in *Lasers and Electro-Optics Society (LEOS) 12th Annual Meeting*, vol. 2, San Francisco, CA, USA, pp. 659–660, November 1999.
- [5] Z.-M. Li *et al.*, "Incorporation of Strain Into a Two-Dimensional Model of Quantum-Well Semiconductor Lasers," *IEEE Journal of Quantum Electronics*, vol. 29, no. 2, pp. 346–354, 1993.
- [6] R. Eppenga, M. F. H. Schuurmans, and S. Colak, "New k.p theory for GaAs/Ga_{1-x}Al_xAs-type quantum wells," *Physical Review B*, vol. 36, no. 3, pp. 1554–1564, 1987.
- [7] S. L. Chuang and C. S. Chang, "A band-structure model of strained quantum-well wurtzite semiconductors," *Semiconductor Science and Technology*, vol. 12, no. 3, pp. 252–263, 1997.
- [8] A. Bykhovski, B. Gelmont, and M. Shur, "The influence of the strain-induced electric field on the charge distribution in GaN-AlN-GaN structure," *Journal of Applied Physics*, vol. 74, no. 11, pp. 6734–6739, 1993.
- [9] F. Bernardini, V. Fiorentini, and D. Vanderbilt, "Spontaneous polarization and piezoelectric constants of III-V nitrides," *Physical Review B*, vol. 56, no. 16, pp. R10024–R10027, 1997.
- [10] M. Pfeiffer, *Industrial-Strength Simulation of Quantum-Well Semiconductor Lasers*, Ph.D. thesis, ETH, Zurich, Switzerland, 2004.
- [11] W. W. Chow and S. W. Koch, *Semiconductor-Laser Fundamentals: Physics of the Gain Materials*, Berlin: Springer, 1999.
- [12] M. Luisier and S. Odermatt, *Simulation of Semiconductor Lasers, Part 1: Many-Body Effects in Semiconductor Lasers*, Diploma thesis, ETH, Zurich, Switzerland, 2003.
- [13] H. Haug and S. W. Koch, *Quantum Theory of the Optical and Electronic Properties of Semiconductors*, Singapore: World Scientific, 3rd ed., 1994.

CHAPTER 38 Additional Features of Laser or LED Simulations

This chapter describes additional features that are available to aid the simulation of both lasers and light-emitting diodes.

NOTE LED simulations present unique challenges that require problem-specific model and numerics setups. Contact TCAD Support for advice if you are interested in simulating LEDs (see [Contacting Your Local TCAD Support Team Directly on page xl](#)).

Plotting Gain and Power Spectra

Modal or material gain is an important parameter in laser operations. It affects the threshold current, modulation response, external efficiency, lasing wavelength, and so on. In the simulation, the modal or material gain can be monitored in three ways: as a function of bias, spatial distribution, and a function of energy. Refer to [Simulating Single-Grid Edge-Emitting Laser on page 760](#) while proceeding through these gain-plotting options.

Modal Gain as Function of Bias

The modal gain for the lasing frequency is automatically output in the `Current` file in a laser simulation if the current file has been specified:

```
File {...  
    Current = "multiqw_curr"  
}
```

At the end of the simulation, the file `multiqw_curr_des.plt` is produced. With `Inspect`, you can plot `OpticalGain` as a function of bias current, voltage, and so on. This modal gain [cm^{-1}] is the total gain of the device at the lasing frequency. In multimode simulations, a modal gain curve corresponds to each mode, and the modal gain is taken at the respective lasing frequencies of the modes.

Material Gain in Active Region

The peak value of the local material gain at each vertex of the active region can be extracted by including the keyword `MatGain` in the `Plot` section:

```
File {...
  Grid = "mesh_mdr.tdr"
  Plot = "multiqw_plot"
}
...
Plot {...
  MatGain
}
...
```

At the end of the simulation, the file `multiqw_plot_des.dat` will be produced. This file can be used in Tecplot SV in conjunction with the grid file, `mesh_mdr.grd`, to view the peak material gain at each spatial location of the active region.

Modal Gain as Function of Energy/Wavelength

One important way to look at the gain is to plot the modal gain as a function of the energy. This is possible by including gain-plotting keywords in the `File` and `Solve-quasistationary` sections of the command file:

```
File {...
  Gain = "gn"
}
GainPlot{ Range=(1.22,1.32) Intervals=120 }
                                     # energy range [eV], discretization
...
Solve {...
  Quasistationary (...
    # ----- Specify plot gain parameters -----
    PlotGain {Range=(0,1) Intervals=3}
    Goal {Name="p_Contact" Voltage=1.8})
    {...}
  }
}
```

The activation syntax is similar to that of `OptFarField` (see [Far Field on page 878](#)) and `SaveOptField` (see [Saving and Loading Optical Modes on page 875](#)). The highlights of the syntax are:

- In the `File` statement, modal gain-plotting is activated by the keyword `Gain` and the value assigned to it, "gn" in this case, becomes the base name for the output files of the modal gain as a function of energy.
- In the `Solve-quasistationary` section, the argument `range=(0,1)` in the `PlotGain` keyword is mapped to the initial and final bias conditions. In this example, the initial and final (goal) `p_Contact` voltages are 0 V and 1.8 V, respectively. The number of `intervals=3`, which gives a total of four ($= 3+1$) modal gain plots at 0 V, 0.6 V, 1.2 V, and 1.8 V. In general, specifying `intervals=n` will produce $(n+1)$ plots.
- The `GainPlot` statement allows you to choose the energy range of the gain curve to plot. In this example, the range has been chosen as 1.22–1.32 eV, and 120 discretization points are to be used.
- The output files in this example are `gn_gain_0000000_des.plt`, ..., `gn_gain_0000003_des.plt`. These files contain the modal gains as a function of energy or wavelength, and can be viewed with `Inspect`.

Transient Simulation

Transient simulation is important in the tracking of the time evolution of carrier dynamics and photon output fluctuations in a laser diode. Sentaurus Device has an option to perform the transient simulation of edge-emitting lasers and VCSELs. A small step bias is applied at the input and, through Newton iterations, the time-varying continuity equations and photon rate equations are solved self-consistently with the Poisson equation, QW scattering equations, and Schrödinger equation.

Upon the application of the step bias, the time steps are increased in small intervals to trace the dynamics of the carriers and photons in the transient simulation. At each time step, the full set of variables everywhere in the device can be saved and plotted. In this way, you can calculate which feature of the device is hampering the faster modulation of the device.

Transient simulations performed at different biases yield different dynamics, so you must perform a quasistationary simulation first to reach the required bias current or voltage level, after which the transient simulation is activated. The syntax for transient simulation of general devices is described in [Transient Command on page 119](#). Here, the emphasis is on explaining the syntax that is related to [Simulating Single-Grid Edge-Emitting Laser on page 760](#).

Syntax for Laser Transient Simulation

The transient simulation can be activated by the keyword `Transient` in the `Solve` section of the command file:

```
Solve {
  # ----- Get initial guesses, coupled means Newton's iteration -----
  Poisson
  coupled {Hole Electron Poisson}
  coupled {Hole Electron Poisson PhotonRate}

  # ----- Ramping the voltage to 1.8 V -----
  quasistationary (
    # ----- Specify ratio step size of voltage ramp -----
    InitialStep = 0.001
    MaxStep = 0.05
    Minstep = 1e-5

    # ----- Specify the final voltage ramp goal -----
    Goal {name="p_Contact" voltage=1.8})
    {
      # ----- Gummel iterations for self-consistency of Optics -----
      Plugin(BreakOnFailure){
        # --- Newton iterations for coupled equations -----
        Coupled { Electron Hole Poisson PhotonRate }
        Wavelength
        Optics
      }
    }
  }

  # ----- At 1.8V, perform transient simulation -----
  transient (
    # ----- Specify the starting and ending time -----
    InitialTime = 0.0          # [s]
    FinalTime = 2.0e-9        # [s]

    # ----- Control the time step size -----
    InitialStep = 1.0e-3
    MinStep = 1.0e-3
    MaxStep = 1.0e-1

    # ----- Save the plot variables at specified intervals -----
    Plot { range=(0,1) intervals=4 }
  )
  {
    Plugin(BreakOnFailure){
      Coupled { Electron Hole Poisson PhotonRate }
    }
  }
}
```



```
#      Wavelength
#      Optics
    }
  }
}
```

The above syntax has been extracted from [Simulating Single-Grid Edge-Emitting Laser on page 760](#):

- A quasistationary simulation is performed first to ramp up the operating voltage to a bias of 1.8 V before the transient simulation is activated.
- In the transient section, you must specify the start and end time. The response of the carriers and photons subjected to a small step bias input will generally settle to a steady state in the time frame of a few thousand picoseconds. This settling time depends on bias conditions and the type of laser diode. You are encouraged to use different values of `FinalTime` so that the important parts of the transient response are observed.
- In the transient section, you must specify the time-step size as well in the same format as is used to specify the bias step size in the quasistationary simulation. In this case, the time-step size is the fraction indicated in `InitialStep`, `MinStep`, and `MaxStep` multiplied by $(\text{FinalTime} - \text{InitialTime})$, which gives 2, 2, and 200 ps, respectively. The scattering time for the carriers is in the order of 1×10^{-13} s, so a time step shorter than this time will not be meaningful. Of course, if the time step is too big, it cannot resolve the finer features of the transient response.
- In the transient section, the `Plot` statement enables you to save the variables that have been defined in the `Plot` section at required intervals of the transient simulation. In this example, `intervals=4` produces five plot files at the time intervals of 0, 400, 800, 1200, 1600, and 2000 ps.
- The transient response computed is saved in the current file. The major results of laser output are saved as a function of time in this current file. You can then perform a FFT of the time response to obtain the modulation response of the laser diode.

Performing Temperature Simulation

There are a few different types of temperatures, but Sentaurus Device only treats two types:

- Lattice temperature, which describes the vibrational states of the crystal lattice
- Carrier temperatures, which determine the distribution of the excited carriers

The lattice temperature is solved by the lattice temperature model described in [Chapter 9 on page 205](#). The carrier temperatures are solved by the hydrodynamic equations discussed in [Hydrodynamic Model for Temperatures on page 211](#).

Lattice Temperature Simulation

To solve the lattice temperature model, the following three steps must be taken:

1. A thermode must be defined in the grid file and command file.
2. In the Physics section, the keyword `Thermodynamic` is optional. If it is included, a lattice temperature gradient term is appended to the carrier current flux density.
3. In the Solve section, the keyword `Temperature` must be added to the Coupled statement.

These steps are embedded in the following syntax:

```
# --- Need to specify where to impose Dirichlet boundary condition for
# temperature solution ---
Thermode {
    {Name="top_thermode" AreaFactor=200 Temperature=300 SurfaceResistance =
    0.14}
    {Name="bot_thermode" AreaFactor=200 Temperature=300 SurfaceResistance =
    0.09}
}

Plot {
    # ----- Temperature variables -----
    Temperature
}
...
Physics {...
    Laser (...
        Optics (...)
    )
    # ----- Turn on thermodynamic simulation -----
    Thermodynamic
    RecGenHeat
}
...
Solve {
    # ----- Get initial guesses, coupled means Newton's iteration -----
    Poisson
    coupled {Hole Electron Poisson}
    coupled {Hole Electron Poisson PhotonRate}
    coupled {Hole Electron Poisson PhotonRate Temperature}

    # ----- Ramping the voltage -----
    quasistationary (
        # ----- Specify ratio step size of voltage ramp -----
        InitialStep = 0.001
```

```

MaxStep = 0.05
Minstep = 1e-5

# ----- Specify the final voltage ramp goal -----
Goal {name="p_Contact" voltage=1.8})
{
    # ----- Gummel iterations for self-consistency of Optics -----
    Plugin(BreakOnFailure){
        # --- Newton iterations for coupled equations -----
        Coupled {Electron Hole Poisson PhotonRate Temperature}
        Wavelength
        Optics
    }
}
}

```

Refer to [Simulating Single-Grid Edge-Emitting Laser on page 760](#) for the rest of the laser simulation syntaxes, which have been omitted here. Some comments about the above syntax are:

- A thermode is a boundary where the Dirichlet boundary condition is set for the lattice temperature equation. At all other boundaries without a thermode, Neumann boundary condition is assumed. It can be set in the same way as the contact electrodes are set in Sentaurus Structure Editor, and this is shown in [Figure 98](#). The SurfaceResistance [$\text{cm}^2\text{K/W}$] can be used to tune the lattice temperature profile in the device.
- In the Physics section, the keyword Thermodynamic contributes an additional term (gradient of the lattice temperature) to the electron and hole flux densities. The keyword RecGenHeat adds heat produced or absorbed by generation–recombination to the lattice temperature equation.
- In the Solve section, the keyword Temperature in the Coupled statement adds the lattice temperature equation to the Newton system to be solved iteratively.

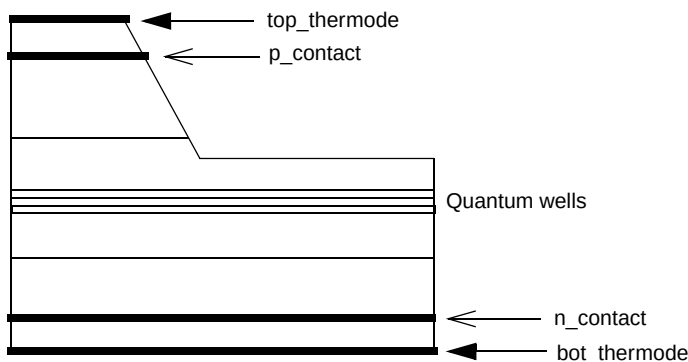


Figure 98 Setting thermodes in the same way as setting contacts; labeled top_thermode and bot_thermode, respectively

Carrier Temperature Simulation

The carrier temperatures can be solved by the hydrodynamic model, and the activation syntaxes are located in similar locations as the lattice temperature model:

```
Plot {
  # ----- Temperature variables -----
  eTemperature
  hTemperature
}
...
Physics {...
  Laser (...
    Optics (...)
  )
  # ----- Specify ambient device temperature -----
  Temperature = 298
  # ----- Turn on hydrodynamic simulation -----
  Hydrodynamic
  RecGenHeat
}
...
Solve {
  # ----- Get initial guesses, coupled means Newton's iteration -----
  Poisson
  coupled {Hole Electron Poisson}
  coupled {Hole Electron Poisson PhotonRate}
  coupled {Hole Electron Poisson PhotonRate eTemperature hTemperature}

  # ----- Ramping the voltage -----
  quasistationary (
    # ----- Specify ratio step size of voltage ramp -----
    InitialStep = 0.001
    MaxStep = 0.05
    Minstep = 1e-5

    # ----- Specify the final voltage ramp goal -----
    Goal {name="p_Contact" voltage=1.8})
  {
```

```
# ----- Gummel iterations for self-consistency of Optics -----
Plugin(BreakOnFailure){
  # --- Newton iterations for coupled equations -----
  Coupled {Electron Hole Poisson PhotonRate eTemperature
           hTemperature}
  Wavelength
  Optics
}
}
}
```

Some comments about the above syntax are:

- You do not need to specify any thermode if the hydrodynamic model is solved without the lattice temperature equation. However, it is possible to solve both the lattice temperature and hydrodynamic models together. You must include the keywords for both models.
- Note that the carrier temperatures are solved everywhere in the device except the quantum wells. The physics of transport into the quantum wells cannot be handled by the hydrodynamic model.
- In the `Plot` section, you can choose to save the electron and hole temperatures everywhere in the device with the keywords `eTemperature` and `hTemperature`.
- In the `Physics` section, if the lattice temperature is not solved, you can specify an ambient device lattice temperature with the keyword `Temperature=<float>`. The hydrodynamic model is activated by the keyword `Hydrodynamic` in this section. The keyword `RecGenHeat` adds the heat produced or absorbed as a result of carrier generation–recombination to the hydrodynamic equations.
- In the `Solve` section, the hydrodynamic model requires two keywords, `eTemperature` and `hTemperature`, to be included in the `Coupled` statement.

Switching from Voltage to Current Ramping

Before the lasing threshold, the voltage varies almost linearly with the spontaneous output power. At and beyond the lasing threshold, a small increase in voltage will lead to an exponential-order increase in the laser output power. Therefore, it is desirable to switch between voltage and current bias ramping in a laser simulation. This is achieved using the following syntax in the `Solve` section of the command file:

```
Solve {
  # --- Solve for initial guesses of the coupled system ---
  Poisson
  Coupled {Poisson Hole Electron}
  Coupled {Poisson Hole Electron PhotonRate}

  # --- Ramp using voltage bias to near lasing threshold ---
```

38: Additional Features of Laser or LED Simulations

Switching from Voltage to Current Ramping

```
Quasistationary (
  InitialStep = 0.01
  MaxStep = 0.1
  Minstep = 1e-9
  Goal {name="p_contact" voltage=1.3})
{
  Plugin(BreakOnFailure){
    Coupled { Poisson Hole Electron PhotonRate }
    Optics
    Wavelength
  }
}

# --- Change the p_contact from voltage to current type ---
Set ("p_contact" mode current)

# --- Then ramp using current bias ---
Quasistationary (
  InitialStep = 0.01
  MaxStep = 0.05
  Minstep = 1e-8
  Goal {name="p_contact" current=2.5e-4})
{
  Plugin(BreakOnFailure){
    Coupled { Poisson Hole Electron PhotonRate }
    Optics
    Wavelength
  }
}
}
```

The threshold voltage of a laser diode can be estimated approximately by considering the laser diode as a p-i-n diode with the quantum well as a strong recombination center. In order for diffusion current to flow into the quantum well, the intrinsic Fermi levels of the bulk region near the p or n contact must approximately align with that of the quantum well. This means a voltage that is approximately the difference in these Fermi levels must be applied for the device to start the current flow towards the quantum wells to fill them.

Of course, the lasing will not start until the quantum wells are filled to a level such that population inversion is achieved and the threshold losses are overcome. Resistance of the semiconductor layers will also contribute to the voltage. Nevertheless, this consideration will give a rough estimate of the threshold voltage for current flow in the laser diode.

Robust Wavelength Search Algorithm

In cases where the default wavelength search algorithm fails to converge, the keyword `NewWavelengthSearch(InitialWavelength=<float>)` can be activated in the Math statement. This switches on a more robust algorithm that, in most cases, resolves the convergence problem. The specification of an initial wavelength for the new search algorithm is optional. If specified, it must be given in units of nanometers and its value must be larger than the expected actual transition wavelength.

Reducing Number of Result Files

Sentaurus Device can be used to reduce the number of output files for laser simulations. Plots are represented by states that are collected in one output file. This feature is supported for VCSEL near field, laser far field, and gain and spectrum plots using the TDR file format. It is activated by adding the option `(collected)` to the keyword that specifies the output file.

An example of the syntax is:

```
File {  
    OptFarField (collected) = "farfield"  
}  
...  
Solve {  
    ...  
    Quasistationary (  
        PlotFarField {Range=(0, 1) Intervals=3}  
    ) {  
        ...  
    }  
}
```

The output file in this example is called `farfield_ff_coll_des.plt`. It contains four states and can be visualized by Tecplot SV.

Scripts

There is a list of scripts that are written in Tcl and can be used to extract various useful parameters from laser and LED simulations, as well as to control different tools in the Synopsys software suite. The laser-related or LED-related scripts available from the Synopsys Technical Support Center are:

- Automatic extraction of the threshold current
- Calculation of the slope efficiency of the L–I curve
- Automatic addition of DBR layers in the VCSEL structure
- Automatic addition of PML to the laser structure
- Generation of multiple–quantum well (MQW) edge-emitting lasers
- Generation of MQW VCSEL structures
- Ternary-material and quaternary-material parameter calculations
- Extraction of far-field angle from far-field patterns

Part IV Mesh and Numeric Methods

This part of the *Sentaurus Device User Guide* contains the following chapters:

[Chapter 39 Automatic Grid Generation and Adaptation Module AGM on page 933](#)

[Chapter 40 Numeric Methods on page 949](#)

CHAPTER 39 Automatic Grid Generation and Adaptation Module AGM

This chapter describes the automatic generation and adaptation module for two-dimensional quadtree-based simulation grids of physical devices in stationary simulations.

The approach is based on a local anisotropic grid adaptation technique for the stationary drift-diffusion model [1][2][3] and has been extended formally to thermodynamic and hydrodynamic simulations.

Overview

NOTE In its current status, AGM is not designed to improve the speed of simulations but rather to support users in generating simulation grids in a semi-automatic fashion. In fact, using AGM slows down the simulation time considerably as the control of the grid sizes is difficult, and the recomputation of solutions on adaptively generated grids is, in the presence of strong nonlinearities, a time-consuming task. Therefore, it is not recommended to use AGM throughout large simulation projects in a fully automatic adaptation mode. The integration of AGM in Sentaurus Device is incomplete as incompatibilities occur with certain features of Sentaurus Device.

The accuracy of approximate solutions computed by many simulation tools depends strongly on the simulation grid used in the discretization of the underlying problem. The major aim of grid adaptation is to obtain numeric solutions with a controlled accuracy tolerance using a minimal amount of computer resources using *a posteriori* error indicators to construct appropriate simulation grids. The main building blocks of a local grid adaptation module for stationary problems are the adaptation criteria (local error indicators that somehow determine the quality of grid elements), the adaptation scheme (determining whether and how the grid will be modified on the basis of the adaptation criteria), and the recomputation procedure of the approximate solution on adaptively generated grids. In the framework of finite-element methods for linear, scalar, and elliptic boundary-value problems, grid adaptation has reached a mature status [4].

The semiconductor device problem consists of a nonlinearly coupled system of partial differential equations, and the true solution of the problem exhibits layer behavior and

singularities, posing additional difficulties for grid adaptation modules. Several adaptation criteria have been proposed in the literature [1]. For the overall robustness of adaptive simulations, the recomputation of the solution is a very serious (and, sometimes, very time-consuming) problem.

The adaptation procedure used in Sentaurus Device is based on the approach developed in [1], [2], and [3]. It uses the idea of equidistributing local dissipation rate errors and aims at accurate computations of the terminal currents of the device. A quadtree mesh structure is used to enable anisotropic grid adaptation on boundary Delaunay meshes required by the discretization used in Sentaurus Device. The recomputation procedure relies on local and global characterizations of dominating nonlinearities and includes relaxation techniques based on the solution of local boundary value problems and a global homotopy technique for large avalanche generation.

The intention of the AGM module in its current status is to support the generation of a simulation grid and to provide some flexibility for users to influence the adaptation process. This allows users to find, in a semi-automatic process, a compromise of accuracy requirements and mesh sizes. The module supports:

- Two-dimensional device structures given by Sentaurus Mesh command and boundary files
- Default quadtree refinement approach of Sentaurus Mesh
- Grid adaptation for stationary problems (adaptive Coupled and Quasistationary)

Coarsening during adaptation is not supported.

General Adaptation Procedure

The general grid adaptation flow is outlined in the following example:

```
compute solution on actual grid
coupled adaptation loop {
    // adaptation decision
    for all adaptive devices {
        check if adaptation is required
    }
    check if coupled adaptation is required

    // adaptation strategy
    for all adaptive devices {
        generate new grid
        initialize data on new grid and perform local smoothing
    }
}
```

```
// recompute solution
solve fully coupled system on new grids
}
```

The core ingredients of the flow are the adaptation criteria, which decide whether and how the grid is modified, and the meshing engine, which performs the changes of the grid. Sentaurus Device makes use of the Sentaurus Mesh meshing engine.

Adaptation Criteria

The adaptation criteria determine whether and how the grid will be modified. In [1], adaptation criteria for the nonlinearly coupled system of equations for the drift-diffusion model have been proposed, aiming at accurate computations of the terminal currents of the device. They use the close relationship between the system dissipation rate and the terminal currents, and estimate the error of the dissipation rate. This is performed by either solving related local Dirichlet problems or using the residual error estimation technique. Both techniques are well known in the framework of finite-element discretizations for scalar and elliptic boundary-value problems [4].

In practice, these criteria are often computationally too expensive, lead to large grid sizes, and are hard to control by users. Sentaurus Device supports two types of refinement criterion. The first type supports refinement based on values of a scalar field. It is a heuristic refinement, does not provide error estimation, is easily controlled by users, and is useful if physically relevant fields are considered. The second type adopts the residual error estimator for the dissipation rate of [1].

Refinement on Local-Field Variation

These criteria control the variation of a scalar field on grid elements. For a user-specified field represented on vertices of the grid, elements are refined if differences of the vertex values exceed a user-supplied value. Using the doping concentration as a field, such criteria are typically used in mesh generation processes. In the adaptation module, you can use any scalar field defined on vertices known by the simulator, for example, `ConductionBandEnergy` or `DissipationRateDensity`.

Refinement on Residual Error Estimation

The residual adaptation criteria estimate the *error of the quantity F* per grid element T . In analogy to standard methods, they measure jumps of the density of interest across inter-element boundaries. The error for a 2D element T is given as:

$$\eta_T(F) = \frac{|T|}{|N_e(T)|} \sum_{T' \in N_e(T)} |\omega_F^h(T') - \omega_F^h(T)| \quad (1021)$$

where:

- $\omega_F^h(T)$ denotes the approximate element functional density (on element T and T' , respectively).
- $N_e(T)$ is the set of (semiconductor) elements sharing an edge with T .
- $|N_e(T)|$ is the number of elements.
- $|T|$ is the volume of T .

So far, the residual refinement criterion is only available for the AGM dissipation rate `AGMDissipationRate`, which is defined as:

$$D_{AGM} = \hat{w}_n \int_{\Omega} \mu_n n |\vec{J}_n|^2 dx + \hat{w}_p \int_{\Omega} \mu_p p |\vec{J}_p|^2 dx + \hat{w}_r k T \int_{\Omega} R_{abs} \left| \ln \left(\frac{np}{n_{i,eff} p_{i,eff}} \right) \right| dx \quad (1022)$$

where $R_{abs} = \sum \hat{w}_{R_i} |R_i|$ is the weighted absolute sum of individual generation–recombination processes with their individual weights $\hat{w}_{R_i}$. You can modify all weights $\hat{w}_i$.

Solution Recomputation

For the recomputation of the solution, data is interpolated onto the new simulation grid, and iterative smoothing techniques are applied to improve the robustness of the procedure.

Device-Level Data Smoothing

In the first step, the electrostatic potential is adjusted to interpolated data by applying the so-called electrostatic potential correction (EPC), that is, a mixed linear and nonlinear Poisson equation is solved using a Newton algorithm. To achieve almost self-consistent solutions for the coupled equations of the device, local Dirichlet problems are solved approximately, resulting in the so-called nonlinear node block Jacobi iteration (NBJI). The node block iterations are performed only on a subset of all grid vertices.

For remarkable avalanche generation, a homotopy technique (or continuation technique), here called *avalanche homotopy*, is applied to improve the robustness of the recomputation

procedure, that is, the true avalanche generation is decoupled globally from the equations and is integrated stepwise into the solution process. As the avalanche generation F_{ava} is an additive term, the equations to be solved $F(x) = F_b(x) + F_{\text{ava}}(x) = 0$ can be split into a basic part F_b and the avalanche generation term F_{ava} , where x represents the unknown solution variables. With the fixed avalanche generation F_{ava} interpolated from the old grid, the avalanche homotopy now reads:

$$H_{\text{ava}}(x, t) = F_b(x) + (1-t)\widehat{F_{\text{ava}}} + tF_{\text{ava}}(x) \quad (1023)$$

where $t \in [0;1]$ is the homotopy parameter ramped from 0 to 1. For $t = 0$, the homotopy is reduced to a simplified problem, while for $t = 1$ the fully coupled system is solved. Therefore, the avalanche homotopy is similar to a quasistationary simulation where the avalanche generation is ramped (instead of specified parameters).

System-Level Data Smoothing

On the system level, a self-consistent solution for the original fully coupled system of equations is computed by the Newton algorithm.

Specifying Grid Adaptations

If you want to perform simulations using the grid adaptation capability, you must specify which device instances should be adaptive by adding a `GridAdaptation` section into the device instance description. In addition, adaptive devices require a boundary description given by `Boundary` in the corresponding `File` section.

Having the device instance adaptive, you must indicate which solve statements should invoke grid adaptation; otherwise, no adaptation occurs.

The following illustrates the basic components for a single-device simulation:

```
File { ...
    Boundary = "meshing_msh.bnd"      * input boundary description
    Grid      = "../DIR-test/grid_des" * reinterpreted as OUTPUT
}
GridAdaptation ( ... ) { ... }      * device adaptation parameters
Solve { ...
    Coupled ( ... GridAdaptation ( ... ) ) { ... }
    Quasistationary ( ... GridAdaptation ( ... ) ) { ... }
}
```

The relevant keywords are:

- **Boundary in File:** Specify the Sentaurus Mesh boundary file. Implicitly, the corresponding Sentaurus Mesh command file is read to obtain the definition of the doping profiles.
- **Grid in File:** In contrast to nonadaptive simulations, where **Grid** specifies the input device structure, here **Grid** is used to output files. As soon as you plot a device structure, the simulator creates additionally a file that contains the grid and all doping species, which can be used as fixed input device structures. These files are numbered automatically.
- **GridAdaptation in Device section or global section:** Specify the device-specific adaptation parameters as described in [Adaptive Device Instances on page 938](#).
- **GridAdaptation in Coupled/Quasistationary:** Make the solve statement adaptive as described in [Adaptive Solve Statements on page 943](#).

NOTE The grid adaptation module described here is not compatible with the adaptation module of former releases.

Adaptive Device Instances

A device instance is adaptive if the keyword **GridAdaptation** is specified as a section of the instance description. Several parameters can be passed to the instance by specifying parameter or body entries for the keyword, that is:

```
GridAdaptation ( <agm-device-par-list> ) { <agm-device-body-list> }
```

For adaptive devices, a structure description in the form of both a Sentaurus Mesh command file and boundary file is necessary.

Device Structure

The AGM module requires a description of the device structure using a Sentaurus Mesh boundary file and command file. From the boundary file, the geometric structure is taken; while in the command file, doping profiles are defined. The boundary file is specified in the **File** section of the device by:

```
File { ... Boundary = "meshing_msh.bnd" }
```

and the corresponding command file is read. All refinement statements in the Sentaurus Mesh command file are ignored by AGM. All refinement criteria, also the refinement on doping concentration, must be supplied as criteria in the **GridAdaptation** section of the device in the Sentaurus Device command file.

Device Adaptation Parameters

There are parameters that affect the grid generation process or the device-specific smoothing process.

Parameters Affecting Grid Generation

The following device parameters are supported:

- **MaxLoops**: Determines the maximum number of adaptations of this instance within one coupled adaptation.
- **Weights**: Modifies the AGM dissipation rate (see [Eq. 1022, p. 936](#)) of the instance used in the adaptation criteria. The keywords `eCurrent`, `hCurrent`, and `Recombination` refer to the weights $\hat{w}_n$, $\hat{w}_p$, and $\hat{w}_r$, respectively; while `Avalanche`, for example, refers to the corresponding weight of the avalanche generation in R_{abs} . By default, all weights are 1.

Example

```
GridAdaptation ( ...
    MaxLoops = 5
    Weights ( eCurrent=1.e-2 hCurrent=1.e-1 Recombination=1.e-3 Avalanche=1. )
) { ... }
```

Parameters Affecting Smoothing

The following parameters influence device-specific data smoothing:

- **Poisson**: Uses the electrostatic potential correction (EPC) procedure.
- **Smooth**: Uses the node block Jacobi iteration (NBJI) procedure.
- **AvaHomotopy**: Uses the avalanche homotopy procedure. You can change to some extent its behavior, as it takes the following parameters:
 - **Iterations**: Sets the number of iterations for the Newton algorithms used during its application (internally, this number is multiplied by 5).
 - **LinearParametrization**: Uses a linear parameterization of the homotopy parameter.
 - **Extrapolate**: Allows extrapolation during avalanche homotopy.
 - **Off**: Switches off avalanche homotopy.

Example

```
GridAdaptation ( ...  
    * parameters affecting recomputation  
    Poisson      * use electrostatic potential correction (EPC)  
    Smooth       * use node block Jacobi iteration (NBJI)  
    AvaHomotopy ( -Off Iterations=5 LinearParametrization Extrapolate )  
 ) { ... }
```

Adaptation Criteria

The adaptation criteria are listed in the body of `Criteria` of the device-specific `GridAdaptation` section (see [Command File Example on page 945](#)). The two adaptation criteria types are `Element` and `Residual`.

Parameters Common to All Refinement Criteria

The following parameters are common for all refinement criteria (see also [Table 172 on page 1254](#)). Observe that some of the parameters can be ramped in quasistationary simulations (see [Rampable Adaptation Parameters on page 944](#)):

- **Name:** A user name for identification and referencing.
- **MaxElementSize:** The maximal-allowed edge length in axis directions.
- **MinElementSize:** The minimal-allowed edge length in axis directions.
- **MeshDomain:** This allows you to reference to a spatial domain where the criterion will be applied. The domains are defined in the `Math` section as described below.

Example

```
GridAdaptation ( ... ) {  
    Criteria { ...  
        Element ( Name="c1" ... )  
        Residual ( Name="c2" ... )  
    }  
}
```

Element Criterion

A given vertex-based scalar quantity must fulfill the condition:

$$|a_1 - a_2| < \varepsilon_A \quad (1024)$$

where a_1 and a_2 are the quantity values at the vertices of each element edge.

If the logarithmic scale is selected, the following condition must be fulfilled:

$$|\text{sign}(a_1) \ln(\max(|a_1|/\epsilon_A, 1)) - \text{sign}(a_2) \ln(\max(|a_2|/\epsilon_A, 1))| < \ln(\epsilon_R) \quad (1025)$$

This condition simplifies for values larger than ϵ_A to $|\ln(a_1) - \ln(a_2)| < \ln(\epsilon_R)$ and accounts for smaller values and even negative values in a suitable fashion.

Parameters

In addition to the general criterion parameters, the following options are supported:

- **DataName:** Selects the physical vertex-based quantity.
- **AbsError:** The parameter ϵ_A in both the linear and logarithmic condition above.
- **Logarithmic:** Uses the logarithmic condition.
- **RelError:** The parameter ϵ_R in the logarithmic condition above.

Example

```

Element (
  Name = "c1"                * user name for criterion
  DataName = "DopingConcentration" * considered quantity
  Logarithmic                * select logarithmic scale
  AbsError = 1.e14           * ignore values below given value
  RelError=10.               * error bound (10. means each decade)
  MaxElementSize = ( 0.1 0.2 ) * maximal edge length in axis direction
  MinElementSize = ( 1.e-2 1.e-3 ) * minimal edge length in axis direction
  MeshDomain = "md1"        * "md1" defined in Math
)

```

Residual Criterion

This criterion estimates an error for the integral of the AGM dissipation rate density over the specified mesh domain. The mesh is not refined if the condition:

$$\eta < \epsilon_R(D + \epsilon_A) \quad (1026)$$

is fulfilled, where η is the sum of the element error estimators of the AGM dissipation rate in the definition domain.

Parameters

Besides the general criterion parameters, the following parameters are supported:

- **AbsError:** The parameter ϵ_A in the condition above.
- **RelError:** The parameter ϵ_R in the condition above.

Example

```

Residual (
  Name = "res1"
  AbsError = 1.e-5      * global lower bound (units of dissipation rate)
  RelError = 1.e-1      * global relative error
  MaxElementSize = (0.1 0.2)
  MinElementSize = (1.e-2 1.e-3)
  MeshDomain = "semi"   * restrict to mesh domain
)

```

Mesh Domains

Mesh domains describe a geometric location of the simulation domain and are specified in the Math section of the instance. They are given as the union or intersection of a list of some basic mesh domain descriptions or other mesh domains, and are used to describe the definition domain of the refinement criteria.

Parameters

The available parameters are (see also [Table 177 on page 1256](#)):

- **Type:** Specifies which operation is applied to the list of spatial domains. You can select either union or intersection by specifying Cup or Cap, respectively. Building the union is the default operation.
- **Region:** The spatial domain covered by the region.
- **Box:** Defines an axis-aligned box by specifying minimum and maximum coordinates.
- **MeshDomain:** Another mesh domain defined before.

Example

```

Math { ...
  * mesh domain "semi" is the union of specified regions
  MeshDomain "semi" ( Region="bulk" Region="R2" )
  * mesh domain "channel" is intersection of the mesh domain "semi"
  * and the specified box
  MeshDomain "channel" (
    Type=Cap MeshDomain="semi" Box((-5 1.e-5) (5 1.e-3)) )
}

```

Adaptive Solve Statements

Grid adaptation is only performed for explicitly adaptive solve statements using the keyword `GridAdaptation`. Only the highest level `Coupled` and `Quasistationary` solve statements can be adaptive.

General Adaptive Solve Statements

The following parameter entries are interpreted by all adaptive solve statements:

- `MaxLoops`: Sets maximal number of adaptation iterations per adaptive coupled system (default is 100000).
- `Plot`: Enables device plots for all intermediate device grids.
- `CurrentPlot`: Enables plotting to current file after each coupled adaptation.

Adaptive Coupled Solve Statements

A `Coupled` solve statement can be adaptive if it is the highest level solve statement, and it can look like:

```
Coupled ( ... GridAdaptation ( MaxLoops = 5 ) )  
  { Poisson Electron Hole }
```

Adaptive Quasistationary Solve Statements

In the current implementation, a `Quasistationary` can be adaptive only if (a) it is the highest level solve statement, and (b) its system consists of a `Coupled` solve statement, that is, `Plugin` statements are not yet supported.

In adaptive quasistationary simulations, it may be useful to restrict the adaptation to certain ranges of the ramped parameter. This can be achieved by specifying parameter ranges or iteration numbers in a similar fashion as in `Plot` in solve statements using the `Time`, `IterationStep`, and `Iterations` keywords, for example.

Parameters

Besides the parameters for all adaptive solve statements, you can provide:

- `Iterations`: Adapts at specific iterations of the ramping process.
- `IterationStep`: Adapts only after a specified number of iterations.

- Time: Adapts if the ramping parameter falls into given ranges.

Example

```
Quasistationary ( ...  
  GridAdaptation ( ...  
    IterationStep = 10  
    Iterations = ( 2 ; 7 )  
    Time = ( Range = ( 0.2 0.4 ) ; range = (0. 1.)  
            intervals = 5 ; 0.1 ; 0.99 )  
  )  
) { ... }
```

The fixed times specified by Time do not force adaptation at these values (which can be achieved by other methods, for example, adding plot statements for the required parameter values).

Performing Adaptive Simulations

Rampable Adaptation Parameters

Some of the criteria parameters can be ramped in quasistationary Goal statements. So far, this set includes the values given by MaxElementSize, MinElementSize, AbsError, and RelError. You have to refer to these parameters by their full qualified name.

Example

```
GridAdaptation ( ... ) {  
  Criteria { ...  
    Element ( Name = "c1" MaxElementSize = ( 0.1 0.2 ) ... )  
  }  
}  
Solve { ...  
  Quasistationary ( ...  
    Goal ( ModelParameter="AGM/Criterion(c1)/AbsError" Value=1.e12 )  
    Goal ( ModelParameter="AGM/Criterion(c1)/MaxElementSize[0]"  
          Value=1.e-2 )  
  ) { ... }  
}
```

Command File Example

The following example excerpt of a command file illustrates the adaptation specification:

```
File { ...
    * device structure input
    * (and implicit corresponding mesh command file)
    Boundary = "meshing_msh.bnd"      * input boundary file
    Grid      = "./DIR-n2/n2_grid_des" * used as OUTPUT base name
}
Math { ...
    MeshDomain "semi" ( Region="R1" Region="R2" )
}
GridAdaptation (      * device-specific adaptation parameters
    MaxCLoops = 5      * max number of adaptations per adaptive Coupled

    * recomputation parameters
    Poisson          * perform electrostatic potential correction
    Smooth           * perform node block Jacobi iteration
    AvaHomotopy ( Iterations=5 )
) {
    Criteria { ...
        * refine on doping concentration
        Element ( Name="c1" DataName="DopingConcentration"
            AbsError=1.e12 RelError=10. Logarithmic
            MaxElementSize=(0.1 0.1) MinElementSize=(1.e-2 1.e-2)
            MeshDomain="semi" )
    }
}
Solve {
    Coupled { Poisson Electron Hole } * compute solution on initial grid

    * switch on some adaptation criteria
    Quasistationary ( ... InitialStep=1. DoZero
        Goal { ModelParameter="AGM/Criterion(dr1)/AbsError" Value=1.e5 }
    ) { Coupled { Poisson Electron Hole } }

    * ramp goals of interest with adaptation
    Quasistationary ( ...
        GridAdaptation * adaptive quasistationary
    ) { Coupled { Poisson Electron Hole } }
}
Plot { ...
    AGMDissipationRate AGMDissipationRateDensity
    GradAGMDissipationRateDensity/Vector
    AGMDissipationRateAbsJump * error estimator of residual criterion
}
```

Limitations and Recommendations

Limitations

The grid adaptation approach implemented in Sentaurus Device has been developed for the drift-diffusion model and 2D quadtree-based simulation grids. For convenience, the approach has been formally extended to support other transport models and features available in Sentaurus Device, that is, AGM is formally compatible with the drift-diffusion, thermodynamic, and hydrodynamic transport models.

Nevertheless, it must be noted that AGM has been applied, so far, only to the drift-diffusion model and silicon devices. Total incompatibility is to be expected with the Schrödinger equation solver, heterostructures, interface conditions, and laser equations. These incompatibilities are only partially checked after command file parsing.

Recommendations

Initial Grid Construction

The very first initial grid constructed consists essentially only of boundary vertices. To construct a realistic initial grid, some artificial quasistationary may be necessary, which ramps the criteria parameters such that these become effective.

Accuracy of Terminal Currents as Adaptation Goal

The choice of appropriate adaptation criteria depends on the adaptation goal. In most adaptive simulation procedures, the local discretization errors are used as adaptation criteria. This approach is not practicable for device simulation as the criteria lead to overwhelmingly large grid sizes. The residual adaptation criterion for the functional `AGMDissipationRate` aims for accurate computations of the device terminal currents, a minimal requirement for all simulation cases, allowing in principle some unresolved solution layers, which do not contribute to the terminal current computation.

AGM Simulation Times

Each coupled adaptation iteration requires remarkable simulation time. In contrast to linear or easy-to-solve problems, for device simulation, most time is consumed in the recomputation

procedure of the solution due to the extreme nonlinearities of the problem and not in the pure adaptation of the grid.

The most time-consuming parts in many AGM simulations are (in order of importance):

1. Avalanche homotopy if the computation for $t = 1$ fails to converge (can be very expensive).
2. NBJI smoothing step (for large grid sizes).
3. EPC smoothing step.
4. Grid generation.

Large Grid Sizes

The low convergence order of the discretization causes relatively large grid sizes even for low accuracy requirements. Especially in the vicinity of solution layers and singularities, the point density is difficult to control.

To avoid such problems, increase `MinElementSize` for the responsible criterion: Elements that reach the allowed minimal edge length are no longer refined, and their local error does not contribute to the global error used in the adaptation decision. Therefore, realistic minimal element sizes stop refinement in singularities and layers.

Convergence Problems After Adaptation

It has been observed that, for very coarse simulation grids, the recomputation procedure shows convergence problems. Such problems can be solved by using slightly refined initial grids or by refining the coarsest possible grid (reduce `MaxElementSize` in refinement criteria).

AGM and Extrapolation

After adaptation within a quasistationary, extrapolation is not possible and the parameter step size may decrease. Extrapolation is supported as soon as two consecutive solutions are computed on the same mesh.

References

- [1] B. Schmithüsen, *Grid Adaptation for the Stationary Two-Dimensional Drift-Diffusion Model in Semiconductor Device Simulation*, Series in Microelectronics, vol. 126, Konstanz, Germany: Hartung-Gorre, 2002.
- [2] B. Schmithüsen, K. Gärtner, and W. Fichtner, *A Grid Adaptation Procedure for the Stationary 2D Drift-Diffusion Model Based on Local Dissipation Rate Error Estimation: Part I - Background*, Technical Report 2001/02, Integrated Systems Laboratory, ETH, Zurich, Switzerland, December 2001.
- [3] B. Schmithüsen, K. Gärtner, and W. Fichtner, *A Grid Adaptation Procedure for the Stationary 2D Drift-Diffusion Model Based on Local Dissipation Rate Error Estimation: Part II - Examples*, Technical Report 2001/03, Integrated Systems Laboratory, ETH, Zurich, Switzerland, December 2001.
- [4] R. Verfürth, *A Review of A Posteriori Error Estimation and Adaptive Mesh-Refinement Techniques*, Chichester: Wiley Teubner, 1996.

This chapter presents some of the numeric methods used in Sentaurus Device.

Discretization

The well-known ‘box discretization’ [1][2][3] is applied to discretize the partial differential equations (PDEs). This method integrates the PDEs over a test volume such as that shown in [Figure 99](#), which applies the Gaussian theorem, and discretizes the resulting terms to a first-order approximation.

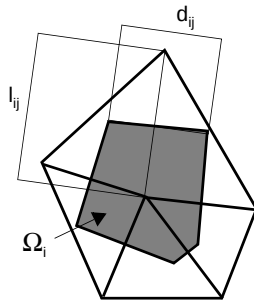


Figure 99 Single box for a triangular mesh in 2D

In general, box discretization discretizes each PDE of the form:

$$\nabla \cdot \vec{J} + R = 0 \quad (1027)$$

into:

$$\sum_{j \neq i} \kappa_{ij} \cdot j_{ij} + \mu(\Omega_i) \cdot r_i = 0 \quad (1028)$$

with values listed in [Table 134](#).

Table 134 Dimension

Dimension	κ_{ij}	$\mu(\Omega_i)$
1D	$1/l_{ij}$	Box length
2D	d_{ij}/l_{ij}	Box area
3D	D_{ij}/l_{ij}	Box volume

In this case, the physical parameters j_{ij} and r_i have the values listed in [Table 135](#), where $B(x) = x/(e^x - 1)$ is the Bernoulli function.

Table 135 Equations

Equation	j_{ij}	r_i
Poisson	$\varepsilon(u_i - u_j)$	$-p_i$
Electron continuity	$\mu^n(n_i B(u_i - u_j) - n_j B(u_j - u_i))$	$R_i - G_i + \frac{d}{dt}n_i$
Hole continuity	$\mu^p(p_j B(u_j - u_i) - p_i B(u_i - u_j))$	$R_i - G_i + \frac{d}{dt}p_i$
Temperature	$\kappa(T_i - T_j)$	$H_i - \frac{d}{dt}T_i c_i$

One special feature of Sentaurus Device is that the actual assembly of the nonlinear equations is performed elementwise, that is:

$$\sum_{e \in \text{elements}(i)} \left\{ \left(\sum_{j \in \text{vertices}(e)} \kappa_{ij}^e \cdot j_{ij}^e \right) + \mu^e(\Omega_i) \cdot r_i^e \right\} = 0 \quad (1029)$$

This expression is equivalent to [Eq. 1028](#), but has the advantage that some parameters (such as ε , μ_n , μ_p) can be handled elementwise, which is useful for numeric stability and physical exactness.

Box Method Coefficients

Basic Definitions

A mesh is a Delaunay mesh if the interior of the circumsphere (circumcircle for 2D) of each element contains no mesh vertices.

Let T be a mesh element. The center circumsphere (circle for 2D) around the element T is called the element Voronoï center V_T . Let f be the face of the element T . The center circumcircle around the face f is called the face Voronoï center V_f .

Let v be a vertex of the mesh and let ev^n ($1 \leq n \leq N$) be the set of edges connected to vertex v . Let P_{ev}^n be the mid-perpendicular plane for the edge ev^n . The plane P_{ev}^n splits 3D space into two half-spaces. Let S_{ev}^n be the half-space that contains the vertex v . The intersection of all half-spaces S_{ev}^n is called the Voronoï box B_v of vertex v . The Voronoï box B_v is the convex

polyhedron and any face of B_v is called a Voronoï face. In addition, T_v^m ($1 \leq m \leq M$) is the set of elements per vertex v and T_{ev}^k ($1 \leq k \leq K$) is the set of elements per edge ev .

For a Delaunay mesh, there are two propositions:

1. The vertices of a Voronoï box B_v are element Voronoï centers, that is, B_v is a polyhedron with the vertices $(V_{T_v^1}, V_{T_v^2}, \dots, V_{T_v^M})$.
2. The Voronoï face associated with the edge ev is the convex polygon with the vertices $(V_{T_{ev}^1}, V_{T_{ev}^2}, \dots, V_{T_{ev}^K})$ and this polygon lies in the mid-perpendicular plane P_{ev} (see [Figure 100](#)).

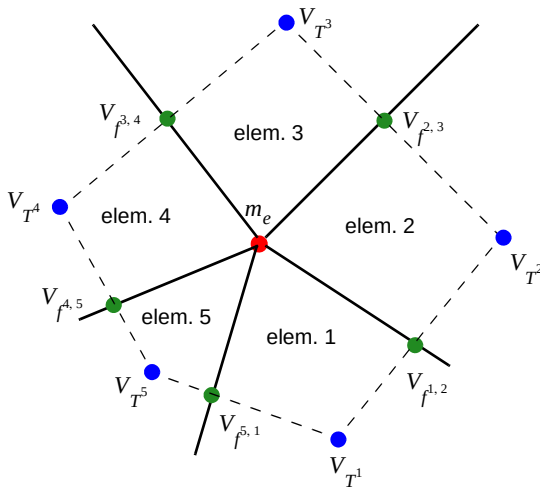


Figure 100 Voronoï face of 3D Delaunay elements: view of mid-perpendicular plane at edge e with element Voronoï centers V_{T^i} and the face Voronoï center $V_{f^{i,i+1}}$ between elements T^i and T^{i+1}

For a non-Delaunay mesh, the Voronoï box is a convex polyhedron, but there are vertices that are not Voronoï element centers (see [Figure 101](#), vertex R).

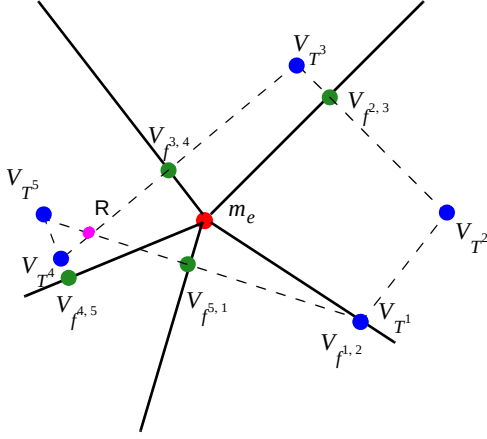


Figure 101 Voronoï face of 3D non-Delaunay elements: non-Delaunay mesh; Voronoï face is polygon $(V_{T^1}, V_{T^2}, V_{T^3}, R, V_{T^1})$

Variation of Box Method Algorithms

The various box methods differ in the way that the 2D volume of the Voronoï face is split into the elements T_{ev}^k associated with edge ev . All box methods give the same result if the mesh has no obtuse elements (an element is obtuse if its element Voronoï center is outside of the element).

Sentaurus Device implements of the following box method (BM) algorithms: element intersection BM, element-face intersection BM, and quadrilateral BM. [Figure 102 on page 953](#) shows a situation where these methods produce different results. For all methods, the Voronoï faces for elements T^1, T^2, T^3 are the same and are equal to the area of the polygons $(m_e, V_{f^{i-1,i}}, V_{T^i}, V_{f^{i,i+1}}, m_e)$. The elements T^4, T^5 have the following Voronoï faces depending on the BM algorithms:

- Element intersection BM algorithms
 - Element T^4 : area of polygon $(m_e, V_{f^{3,4}}, V_{T^4}, V_{T^5}, p, m_e)$
 - Element T^5 : area of polygon $(m_e, p, V_{f^{5,1}}, m_e)$
- Element-face intersection BM algorithms
 - Element T^4 : area of polygon $(m_e, V_{f^{3,4}}, V_{T^4}, V_{T^5}, m_e)$
 - Element T^5 : area of polygon $(m_e, V_{T^5}, V_{f^{5,1}}, m_e)$

- Quadrilateral BM algorithms
 - Element T^4 : area of polygon $(m_e, V_{f^{3,4}}, V_{T^4}, V_{f^{4,5}}, m_e)$
 - Element T^5 : area of polygon $(m_e, V_{T^5}, V_{f^{5,1}}, m_e)$ minus area of polygon $(m_e, V_{T^5}, V_{f^{4,5}}, m_e)$

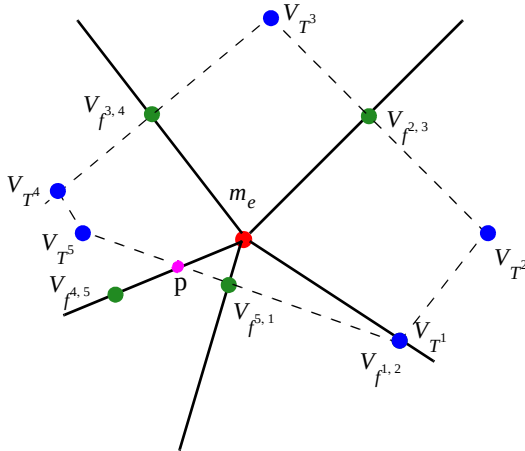


Figure 102 Voronoï face of 3D elements to consider different box method algorithms

Truncated and Non-Delaunay Elements

If an obtuse element has an obtuse face on the material or region boundary, then for this element, an algorithm of truncation can be applied. Figure 103 shows the difference between the original and truncated Voronoï polygons in the 2D case.

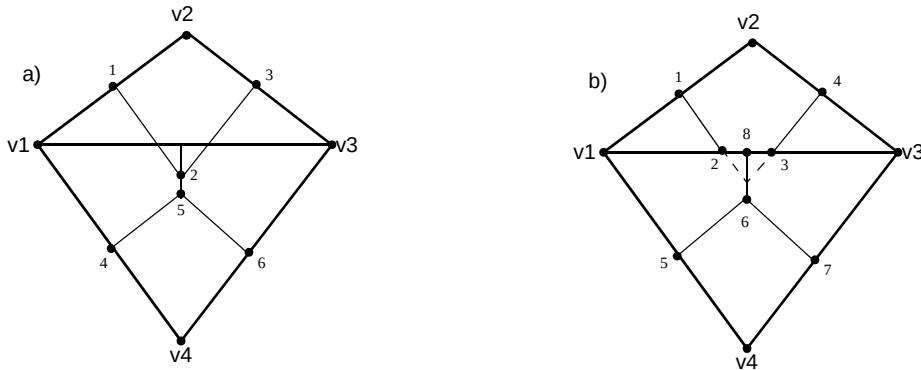


Figure 103 Box method in 2D for truncated element: (a) Voronoï polygons before truncation – $P1(v1,1,2,5,4,v1)$, $P2(v2,1,2,3,v2)$, $P3(v3,3,2,5,6,v3)$, $P4(v4,4,5,6,v4)$; (b) Voronoï polygons after truncation – $P1(v1,1,2,8,6,5,v1)$, $P2(v2,1,2,3,4,v2)$, $P3(v3,4,3,8,6,7,v3)$, $P4(v4,5,6,7,v4)$

For the 3D case, a similar algorithm of truncation is used. If an element is non-Delaunay, it is truncated in any case. In the 2D case, there is a soft truncation for the non-Delaunay element if the neighbor elements have the same material (see [Figure 104](#)).

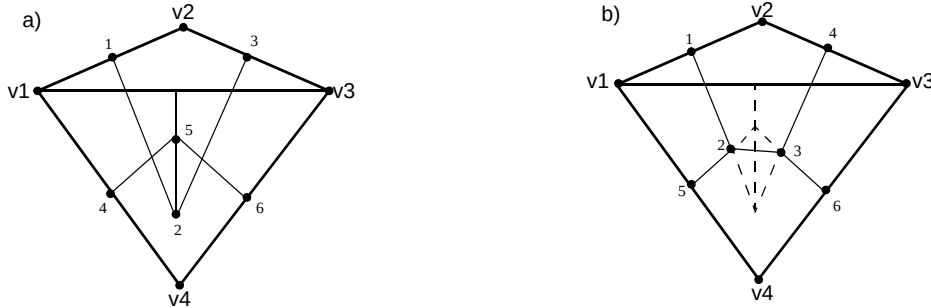


Figure 104 Box method in 2D for non-Delaunay element: (a) Voronoï polygons before truncation – $P1(v1,1,2,5,4,v1)$, $P2(v2,1,2,3,v2)$, $P3(v3,3,2,5,6,v3)$, $P4(v4,4,5,6,v4)$ (the polygons $P1$ and $P3$ are singular; the polygons $P2$ and $P4$ have an intersection); (b) Voronoï polygons after truncation – $P1(v1,1,2,5,v1)$, $P2(v2,1,2,3,4,v2)$, $P3(v3,4,3,6,v3)$, $P4(v4,5,2,3,6,v4)$ (the polygons $P1$ and $P3$ are not singular; the polygons $P2$ and $P4$ have no intersection)

Math Parameters for Box Method Coefficients

The coefficients needed for discretization $\mu^e(\Omega_i)$ and κ_{ij}^e from [Eq. 1029, p. 950](#) (referred to as Measure and Coefficients) can be computed inside Sentaurus Device or read from outside files.

The keyword `-AverageBoxMethod` in the Math section selects the quadrilateral box method (BM) algorithm.

`AverageBoxMethod` and `NaturalBoxMethod` in the Math section select the element intersection BM algorithms. For `AverageBoxMethod`, the algorithm is element oriented (it computes the element Voronoï face for each element and all edges of this element). For `NaturalBoxMethod`, the algorithm is edge oriented (it computes the element Voronoï faces for each edge and all elements around this edge), requires a Delaunay mesh, and can be applied only to 3D structures.

The average box method algorithm has two implementation `AverageBoxMethod` (default) and `CVPL_AverageBoxMethod` (optional). The optional implementation provides different calculations of box coefficients, which may be useful for ‘very non-Delaunay’ meshes.

VoronoiFaceBoxMethod=<scheme> in the Math section selects an element-face intersection BM algorithm, where <scheme> denotes the choice for the truncated correction:

- Truncated for correction for all obtuse elements.
- -Truncated for no correction.
- RegionBoundaryTruncated or MaterialBoundaryTruncated for correction of obtuse elements with an obtuse face (edge for 2D) on boundary of regions or materials.

Note that for non-Delaunay elements, the correction is applied in any case, no matter which scheme is selected.

Table 170 on page 1243 lists all available options for computing Measure and Coefficients.

Only one BM algorithm can be activated. The default options are:

```
Math { ...
    AverageBoxMethod -VoronoiFaceBoxMethod -NaturalBoxMethod
    BoxMethodFromFile -BoxCoefficientsFromFile -BoxMeasureFromFile
    -CVPL_AverageBoxMethod
    ...
}
```

Weighted Box Method Coefficients

The main goal of any space discretization is the generation of a Delaunay mesh. In this case, the box method coefficients are positive and the finite volume scheme [1] (Eq. 1029, p. 950) is monotone. For a non-Delaunay mesh, the AverageBoxMethod coefficients are positive but the order of the approximation of PDEs is less than one. Setaurus Mesh can use special technology – Delaunay–Voronoi weights – for which the *weighted Voronoi diagram* has no overlap control volume for a non-Delaunay mesh. As a result, the finite volume scheme is monotone and the order of the approximation PDEs is equal to one.

Weighted Points

A weighted point $\tilde{p} = (p, P^2)$ is interpreted as a sphere (circle in two dimensions) with a center p and radius P . The weighted distance between $\tilde{p}$ and $\tilde{x} = (x, X)$ is defined as [4][5][6]:

$$\|\tilde{p} - \tilde{x}\| = \sqrt{\|p - x\|^2 - P^2 - X^2} \quad (1030)$$

The weighted points $\tilde{p}$ and $\tilde{x}$ are orthogonal if the weighted distance vanishes: $\|\tilde{p} - \tilde{x}\| = 0$.

In the 3D case, any four weighted points have a common orthogonal sphere called an *orthosphere*. Unless the four centers lie in a common plane, the orthosphere is unique and has a finite radius.

In the 2D case, any three weighted points have a common circle called an *orthocircle*. Unless the three centers lie in a common line, the orthocircle is unique and has a finite radius (see [Figure 105](#)).

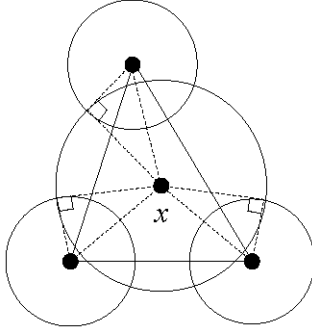


Figure 105 Since the radii of all weighted vertices are positive, their centers lie outside the orthocircle

Weighted Voronoï Diagram

The weighted generalization of the Voronoï diagram is obtained by substituting a weighted vertex for vertices and an orthosphere (orthocircle) for circumspheres (circumcircles).

The weighted bisector plane B_{ij} between $\tilde{p}_i$ and $\tilde{p}_j$ is the locus of points at an equal-weighted distance from $\tilde{p}_i$ and $\tilde{p}_j$. The center of the orthosphere x is the intersection of the bisector planes $x = \bigcap_{i \neq j} B_{ij}$. The weighted middle point m_{ij} between $\tilde{p}_i$ and $\tilde{p}_j$ is the intersection of the segment $[p_i, p_j]$ and the bisector plane B_{ij} . The value m_{ij} is equal to:

$$m_{ij} = \alpha_i p_i + \alpha_j p_j \quad (1031)$$

where:

$$\alpha_i = 0.5 \left(1 - \frac{P_i^2 - P_j^2}{\|p_i - p_j\|^2} \right), \quad \alpha_j = 1 - \alpha_i \quad (1032)$$

Therefore, the weighted bisector plane B_{ij} is orthogonal to the segment $[p_i, p_j]$ and contains the weighted middle point m_{ij} , which is sufficient to compute the weighted coefficients and control volumes. If the radius $P_i \neq P_j$, the middle point $m_{ij} \neq 0.5(p_i + p_j)$ and the weighted Voronoï diagram has no overlap control volume for non-Delaunay meshes (see [Figure 106 on page 957](#)). This is the main property of the weighted Voronoï diagram.

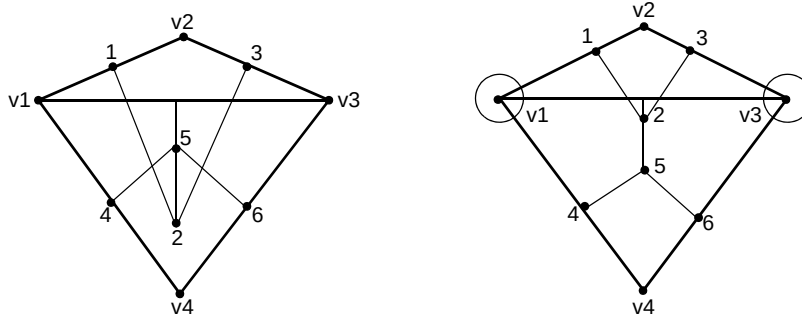


Figure 106 Two-dimensional non-Delaunay mesh: (left) not weighted Voronoï diagram has overlap elements and (right) weighted Voronoï diagram has no overlap elements

Sentaurus Process computes the squared radii (P_i^2 , plot name: DelVorWeight [μm^2]) of the weighted vertices and writes them to a TDR file (see [Sentaurus Process User Guide, Table 66 on page 666](#); option StoreDelaunayWeight). If the keyword WeightedVoronoiBox is specified in the Math section of the command file, Sentaurus Device reads the corresponding arrays from a TDR file and computes the weighted coefficients and measure.

Saving and Restoring Box Method Coefficients

Usually, the coefficients needed for discretization are computed inside Sentaurus Device. For experimental purposes, it may be preferred to use externally provided data. Measure and Coefficients ($\mu^e(\Omega_i)$ and κ_{ij}^e from [Eq. 1029, p. 950](#)) can be stored in, and loaded into and from the debug file. There are two options for element numbering in such files:

- Internal Sentaurus Device numbering with MeasureCoefficientsDebug as the debug file name.
- Mesh numbering (from grid file) with MeasureCoefficients.debug as the debug file name for this option.

If the keyword BoxMeasureFromFile or BoxCoefficientsFromFile is specified in the Math section and there is file MeasureCoefficientsDebug in the simulation directory, Sentaurus Device reads the corresponding arrays from this file.

If the keyword BoxMeasureFromFile(GrdNumbering) or the keyword BoxCoefficientsFromFile(GrdNumbering) is specified and there is the file MeasureCoefficients.debug, Sentaurus Device reads the corresponding arrays from this file. If there are no such debug files but these keywords are specified, Sentaurus Device computes Measure and Coefficients and writes them in the corresponding file.

The format of the MeasureCoefficientsDebug file is as follows. In line k of the Measure section, the control volume for each element-vertex j of element k is stored (that is, value

Measure[k][j]). The numeration of elements and local numeration of vertices inside the element (see [Figure 108 on page 1000](#)) correspond to the internal Sentaurus Device numbering.

The Coefficients section in this file has a similar format. For example, the Measure section in the file can appear as follows:

```
Measure {
    8.719666833501378e-08 4.359833416750702e-08 4.359833416750729e-08
    8.719666833501378e-08 4.359833416750702e-08 4.359833416750729e-08
    ...
}
```

The format of the MeasureCoefficients.debug file is different. There are four section: Info, Elem_type, Measure, and Coefficients. In line k of the Measure section, the control volume for each element-vertex j of element k is stored (that is, value Measure[k][j]). The numeration of elements and local numeration of vertices inside the element correspond to the grid file. The Coefficients section in the debug file has similar format. For example, the file can look like:

```
Info {
    dimension      = 2
    nb_vertices     = 10
    nb_grd_elements = 11
    nb_des_elements = 7
}

Elem_type {
    point    = 0
    line     = 1
    triangle = 2
    rectangle = 3
    tetrahedron = 5
    pyramid   = 6
    prism     = 7
    cuboid    = 8
}

Measure { # unit = [um^2]
#   grd_elem  des_elem  elem_type
    0  0  2  1.828427124999999e+00 9.142135625000004e-01 9.142135625000002e-01
    1  1  2  4.052251462735666e+00 4.052251462735666e+00 8.104502925471332e+00
    ...
    7   -1  1          # contact or interface
    8   -1  1          # contact or interface
    ...
}
```

```

Coefficients { # unit = [1]
# grd_elem des_elem elem_type
  0 0 2 1.093836321204215e+00 1.100111438811216e-16 2.285533906249999e-01
  1 1 2 0.000000000000000e+00 8.379715512271076e-01 8.379715512271076e-01
  ...
  7 -1 1 # contact or interface
  8 -1 1 # contact or interface
  ...
}

```

Log File

A log file contains common data about the mesh and information about non-Delaunay elements per region and per interface, for example:

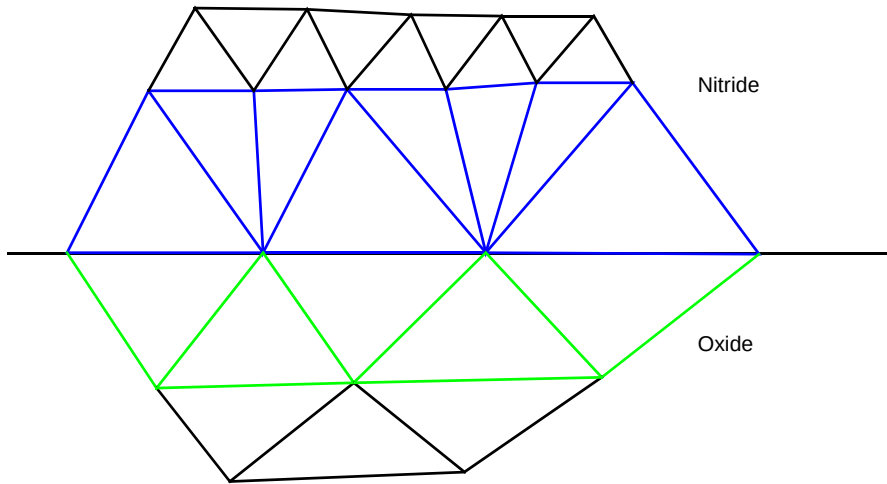
```

/----- Region non Delaunay elements -----
Region      Volume      BoxMethodVolume  DeltaVolume  Elements  non Delaunay
name        [um2]        [um2]          [%]              Elements
-----
Nitride     1.9500000e-04  2.2635574e-04   16.080        53        12 (22.64 %)
. . .
Oxide       6.0618645e-03  8.0705629e-03   33.137        2500      818 (32.72 %)
Silicon     3.5548100e-02  4.9531996e-02   39.338        12656     5057 (39.96 %)
Total       4.6402113e-02  6.4934852e-02   39.939        16550     6383 (38.57 %)
\-----

/----- Interface non Delaunay elements -----
Region1      Volume      BoxMethodVolume  DeltaVolume  Elements  non Delaunay
Region2      [um2]        [um2]          [%]              Elements
-----
Nitride     3.2554709e-06  2.2127731e-05   579.709        21         7 (33.33 %)
Oxide       1.5602383e-05  1.7064897e-05    9.374         15         7 (46.67 %)
. . .
. . .
Oxide       1.2253036e-05  1.6837599e-05   37.416        121         4 ( 3.31 %)
Silicon     4.4053027e-05  4.2893781e-05    2.631        121         1 ( 0.83 %)
. . .
Total       1.6008636e-03  1.6925624e-03    5.728        763        122 (15.99 %)
\-----

```

The *interface elements* are elements that have vertices on the *interface*, for example:



The blue and green elements are nitride and oxide *interface* elements, respectively.

AC Simulation

AC simulation is based on small-signal AC analysis. The response of the device to ‘small’ sinusoidal signals superimposed upon an established DC bias is computed as a function of frequency and DC operating point. Steady-state solution is used to build up a linear algebraic system [7] whose solution provides the real and imaginary parts of the variation of the solution vector $(\phi, n, p, T_n, T_p, T)$ induced by small sinusoidal perturbation at the contacts.

AC Response

The AC response is obtained from the three basic semiconductor equations (see Eq. 37, p. 191 and Eq. 53, p. 197) and from up to three additional energy conservation equations to account for electron, hole, and lattice temperature responses. In the following description of the AC system, the temperatures have been omitted in the solution vector and Jacobian for simplicity, a complete description being formally obtained by adding the temperature responses to the solution vector and the corresponding lines to the system Jacobian.

After discretization, the simplified system of equations can be symbolically represented at the node i of the computation mesh as:

$$F_{\phi i}(\phi, n, p) = 0 \quad (1033)$$

$$F_{ni}(\phi, n, p) = \dot{G}_{ni}(n) \quad (1034)$$

$$F_{pi}(\phi, n, p) = \dot{G}_{pi}(p) \quad (1035)$$

where F and G are nonlinear functions of the vector arguments ϕ, n, p , and the dot denotes time differentiation.

By substituting the vector functions of the form $\xi_{\text{total}} = \xi_{\text{DC}} + \xi e^{i\omega t}$ into Eq. 1033, Eq. 1034, and Eq. 1035 where $\xi = \phi, n, p$, ξ_{DC} is the value of ξ at the DC operating point, and ξ is the corresponding response (or the phasor uniquely identifying the complex perturbation) and then expanding the nonlinear functions F and G in the Taylor's series around the DC operating point and keeping only the first-order terms (the small-signal approximation), the AC system of equations at the node i can be written as:

$$\sum_j \begin{bmatrix} \frac{\partial F_{\phi i}}{\partial \phi_j} & \frac{\partial F_{\phi i}}{\partial n_j} & \frac{\partial F_{\phi i}}{\partial p_j} \\ \frac{\partial F_{ni}}{\partial \phi_j} & \frac{\partial F_{ni}}{\partial n_j} - i\omega \frac{\partial G_{ni}}{\partial n_j} & \frac{\partial F_{ni}}{\partial p_j} \\ \frac{\partial F_{pi}}{\partial \phi_j} & \frac{\partial F_{pi}}{\partial n_j} & \frac{\partial F_{pi}}{\partial p_j} - i\omega \frac{\partial G_{pi}}{\partial p_j} \end{bmatrix}_{DC} \begin{bmatrix} \tilde{\phi}_j \\ \tilde{n}_j \\ \tilde{p}_j \end{bmatrix} = 0 \quad (1036)$$

where the solution vector is scaled with respect to terminal voltages (at the contact where the voltage is applied, $\tilde{\phi}$ is 1). Therefore, the unit of carrier density responses is $\text{cm}^{-3}\text{V}^{-1}$ and the potential response is unitless.

The matrix of Eq. 1036 differs from the Jacobian of the system of equations Eq. 1033, Eq. 1034, and Eq. 1035 only by pure imaginary additive terms involving derivatives of G with respect to carrier densities. The global AC matrix system is obtained by imposing the corresponding AC boundary conditions and performing the summation (assembling the global matrix).

Common AC boundary conditions used in AC simulation are Neumann boundary and oxide-semiconductor jump conditions carried over directly from DC simulation; Dirichlet boundary conditions for carrier densities where n and p at Ohmic contacts are $\tilde{n} = \tilde{p} = 0$; and Dirichlet boundary conditions for AC potential at Ohmic contacts that are used to excite the system.

After assembling the global AC matrix and taking into account the boundary conditions, the AC system becomes:

$$[J + iD]\tilde{X} = B \quad (1037)$$

where J is the Jacobian matrix, D contains the contributions of the G functions to the matrix, B is a real vector dependent on the AC voltage drive, and $\tilde{X}$ is the AC solution vector.

By writing the solution vector as $\tilde{X} = X_R + iX_I$ with X_R and X_I the real and imaginary part of the solution vector respectively, the AC system can be rewritten using only real arithmetic as:

$$\begin{bmatrix} J & -D \\ D & J \end{bmatrix} \begin{bmatrix} X_R \\ X_I \end{bmatrix} = \begin{bmatrix} B \\ 0 \end{bmatrix} \quad (1038)$$

The AC response is actually computed by solving the real system [Eq. 1038](#).

An `ACPlot` statement in the `ACCoupled` command is used to plot AC responses ($\tilde{\phi}$, $\tilde{n}$, $\tilde{p}$, $\tilde{T}_n$, $\tilde{T}_p$, $\tilde{T}$). The responses are plotted in the AC plot file of Sentaurus Device with a separate file for each frequency.

For details of the `ACPlot` statement, see [Table 148 on page 1225](#). For details and examples of small-signal AC analysis, see [Small-Signal AC Analysis on page 127](#).

AC Current Density Responses

When the AC system is solved, the AC current density responses $\tilde{J}_D$, $\tilde{J}_n$, and $\tilde{J}_p$ are computed using:

$$\tilde{J}_D = -i\omega\epsilon\nabla\tilde{\phi} \quad (1039)$$

$$\tilde{J}_n = \left. \frac{\partial J_n}{\partial \phi} \right|_{DC} \tilde{\phi} + \left. \frac{\partial J_n}{\partial n} \right|_{DC} \tilde{n} + \left. \frac{\partial J_n}{\partial p} \right|_{DC} \tilde{p} \quad (1040)$$

$$\tilde{J}_p = \left. \frac{\partial J_p}{\partial \phi} \right|_{DC} \tilde{\phi} + \left. \frac{\partial J_p}{\partial p} \right|_{DC} \tilde{p} + \left. \frac{\partial J_p}{\partial n} \right|_{DC} \tilde{n} \quad (1041)$$

The unit of current density responses is $\text{Acm}^{-2}\text{V}^{-1}$.

The responses of the heat fluxes for the lattice ($\tilde{S}_L$), electrons ($\tilde{S}_n$), and holes ($\tilde{S}_p$) are analogous. Their unit is $\text{Wcm}^{-2}\text{V}^{-1}$.

The `ACPlot` statement in the `System` section is used to plot the AC current density responses. The responses are added to the AC solution response in the AC plot files of Sentaurus Device.

Harmonic Balance Analysis

Harmonic balance (HB) analysis is a frequency domain method to solve periodic and quasi-periodic time-dependent problems for steady-state solutions [8][9]. It is a popular method for RF circuit design applications. While transient discretization schemes allow the simulation of arbitrary time-dependent problems, HB more efficiently models periodic and quasi-periodic problems for systems with time constants that vary by many orders of magnitude. The detailed command file syntax is given [Harmonic Balance on page 130](#).

Harmonic Balance Equation

In general, the dynamic mixed-mode simulation problem takes the form:

$$\frac{d}{dt}q[r, u(t, r)] + f[r, u(t, r), w(t)] = 0 \quad (1042)$$

where f and q are nonlinear functions, w represents explicitly time-dependent devices (in particular, voltage or current sources), and the function u is the vector of all solution variables.

Let $f_1, \dots, f_K$ be a set of different frequencies with $f_k > 0$, then both the sources and the solution are approximated by a truncated Fourier series:

$$u(t) = U_0 + \sum_{1 \leq k \leq K} \{U_k \exp(i\omega_k t) + U_k^* \exp(-i\omega_k t)\} \quad (1043)$$

A formal Fourier transform of [Eq. 1042](#) results in the HB equation for the problem:

$$L(U) = i\Omega Q(U) + F(U) = 0 \quad (1044)$$

where F and Q are the finite Fourier series of f and q , respectively, Ω is the frequency matrix, and U is the vector of all Fourier coefficients of u .

Multitone Harmonic Balance Analysis

The multitone harmonic balance (HB) analysis makes use of the multidimensional Fourier transformation (MDFT). This means that the problem is mapped onto a problem in a multidimensional frequency and multidimensional time domain, hereby exploiting the equivalence of Fourier spectra of quasi-periodic functions with their corresponding multidimensional functions.

Multidimensional Fourier Transformation

The multidimensional Fourier transformation (MDFT) maps multidimensional functions onto a multidimensional spectrum.

Let M be a positive integer, the number of tones, $\hat{f}_1, \dots, \hat{f}_M$, be a finite set of different positive numbers, the base frequencies of tones, and $H_1, \dots, H_M$ nonnegative integer numbers, the maximal number of harmonics for each tone.

Define for each tone m the m -th base period $T_m := 1/\hat{f}_m$, the m -th circular frequency $\omega_m := 2\pi\hat{f}_m$, the m -th (minimum) number of sampling points $S_m := 2H_m + 1$, and the m -th (maximum) sampling interval $\delta_m = T_m/S_m$.

Furthermore, let $\underline{x} = (x_1, \dots, x_M)^T$ denote the M -dimensional vector, and let $D_{\underline{x}} = \text{diag}(x_1, \dots, x_M)$ denote the $M \times M$ -matrix composed of the values $x_1, \dots, x_M$.

The set of multi-indices associated with $\underline{H}$ is given by:

$$K := \{\underline{h} \in \mathbb{Z}^M : -H_m \leq h_m \leq H_m \text{ for all } 1 \leq m \leq M\} \quad (1045)$$

Let u^M be a function on the M -dimensional space $\mathbb{C}^M$ given by:

$$u^M(t_1, \dots, t_M) = \sum_{\underline{h} \in K} U_{\underline{h}}^M \exp(i \underline{h} D_{\underline{\omega}} t) \quad (1046)$$

with given complex numbers $U_{\underline{h}}^M$, then:

$$U_{\underline{h}}^M = \frac{1}{T} \int_0^{T_1} \dots \int_0^{T_M} u^M(t_1, \dots, t_M) \exp(-i \underline{h} D_{\underline{\omega}} t) dt_1 \dots dt_M \quad (1047)$$

with $T := \prod_{1 \leq m \leq M} T_m$. $U^M = (U_{\underline{h}}^M)_{\underline{h} \in K}$ is the multidimensional spectrum of u^M . Sampling the function in all dimensions at the equidistant sampling points $t_{\underline{s}} = D_{\underline{\delta}} \underline{s}$ ($0 \leq \underline{s} < \underline{S}$), the discrete MDFT is written formally as:

$$U^M = \Gamma^M u^M \quad (1048)$$

that is, Γ^M is a linear map from $\mathbb{C}^S$ onto $\mathbb{C}^S$ where $S := \prod_{1 \leq m \leq M} S_m$ is the total number of sampling points.

Quasi-Periodic Functions

The multidimensional function u^M can be projected onto a one-dimensional time space by:

$$u(t) := u^M(t, \dots, t) = \sum_{\underline{h} \in K} U_{\underline{h}} \exp(i \underline{h} \omega t) \quad (1049)$$

Functions satisfying this representation are called quasi-periodic. The set:

$$\Lambda := \{f_{\underline{h}} \in \mathbf{R} : f_{\underline{h}} = \underline{h} \cdot \hat{f} \text{ for all } \underline{h} \in K\} \quad (1050)$$

is the spectrum domain associated with f and K (or H). The projection is invertible if, for two different multi-indices $\underline{h}_1$ and $\underline{h}_2$ in K , the resulting frequencies $f_{\underline{h}_1}$ and $f_{\underline{h}_2}$ are different. Note that the one-dimensional Fourier spectrum of u coincides with the multidimensional spectrum of u^M .

While for the multidimensional function u^M S sample points can be specified to compute the multidimensional spectrum, the one-dimensional sample points for u are not well defined (but are rather virtual in the multidimensional time domain).

Multidimensional Frequency Domain Problem

The multitone HB analysis is essentially a translation of (one-dimensional or multidimensional) time-domain problems in a multidimensional frequency domain. Though originally derived from a time-domain problem, the circuit equations are directly specified in a multidimensional frequency domain. This avoids sampling of (one-dimensional) time-dependent sources, which cannot be performed accurately on a sample set of size S . This is the reason why the compact circuit models must provide the CMI-HB-MDFT function set.

The Fourier transformation of quasi-periodic functions is the composition:

$$\Gamma = \Gamma^M \circ P^{-1} \quad (1051)$$

where Γ^M is the multidimensional Fourier transformation of [Eq. 1048](#) and P^{-1} is the inverse of the projection [Eq. 1049](#).

One-Tone Harmonic Balance Analysis

For one-tone HB analysis, the standard discrete Fourier transformation can be used, which includes that the sampling points are defined explicitly in a (one-dimensional) time domain. Therefore, the problem can be extracted directly from the time-domain formulation of the circuit.

Solving HB Equation

The HB equation (Eq. 1044) is a nonlinear equation in U and is solved by the Newton algorithm. In each Newton step, the linear equation:

$$\frac{\partial L}{\partial U}(U) \cdot \delta U = -L(U) \quad (1052)$$

must be solved.

The Jacobian $\partial L / \partial U$ in Fourier space is computed from the Jacobian in the time domain as follows: For a nonlinear scalar function $g: \mathbf{R} \rightarrow \mathbf{R}$ and a T -periodic scalar signal $u(t)$, the Fourier coefficients $G \in \mathbf{C}^S$ of $g(u(t))$ are approximated:

$$G(U) = \Gamma g(\hat{u}) = \Gamma g(\Gamma^{-1}U) \quad (1053)$$

where Γ and Γ^{-1} are the discrete Fourier transform operator and its inverse, $\hat{u}$ is the vector of the time samples $u(t_i)$, and $g(\hat{u})$ is the vector of values $g(u(t_i))$.

The derivatives of the k -th Fourier component G_k with respect to the j -th Fourier component U_j read:

$$\frac{\partial G_k}{\partial U_j}(U) = \sum_l \Gamma_{kl} \frac{\partial g}{\partial u_j}(\hat{u}_l) = \sum_l \Gamma_{kl} \frac{\partial g}{\partial u}(\hat{u}_l) \Gamma_{lj}^{-1} \quad (1054)$$

The corresponding Jacobian is written in the compact form:

$$\frac{\partial G}{\partial U} = \Gamma \hat{J}_u \Gamma^{-1} \text{ with } \hat{J}_u = \frac{\partial g}{\partial u}(\hat{u}) \quad (1055)$$

For scalar functions g and u , $\hat{J}_u$ is a diagonal matrix; for vector-valued functions g and u , $\hat{J}_u$ is a block-diagonal matrix.

Using the notation above, Eq. 1044 becomes:

$$L(U) = i\Omega \Gamma q(\hat{u}(U)) + \Gamma f(\hat{u}(U)) = 0 \quad (1056)$$

and Eq. 1052 for the Newton step takes the form:

$$(i\Omega \Gamma \hat{J}_q \Gamma^{-1} + \Gamma \hat{J}_f \Gamma^{-1}) \delta U = -L(U) \quad (1057)$$

The Newton algorithm constructs a sequence U^k of the Fourier coefficients of the time-domain solution vector u . The sequence is regarded as converged if both the residual $|L(U^k)|$ and the update error are small.

Solving HB Newton Step Equation

The memory requirements for storing the HB Jacobian matrix typically become very large, as its size is increased by a factor of S^2 compared to the corresponding DC or transient matrix. For a very small number of harmonics and a moderately sized simulation grid, using a direct linear solver may be feasible. However, the use of the GMRES(m) iterative method is recommended for most applications.

Restarted GMRES Method

The HB module makes use of a preconditioned restarted generalized minimum residual GMRES(m) method [10], a Krylow subspace method, which does not need to store the Jacobian in memory, as only matrix-vector products have to be computed.

GMRES(m) requires a suitable preconditioner to achieve convergence. A (left) preconditioner P is a matrix that approximates a given matrix A , but is much easier to invert than A itself. Instead of solving the linear equation $Ax = b$ for given A and b , the (left) preconditioned problem $P^{-1}Ax = P^{-1}b$ is solved. The preconditioner used for HB [11] takes the form:

$$P = i\Omega \begin{bmatrix} \bar{J}_q & 0 \\ & \dots \\ 0 & \bar{J}_q \end{bmatrix} + \begin{bmatrix} \bar{J}_f & 0 \\ & \dots \\ 0 & \bar{J}_f \end{bmatrix} \quad (1058)$$

where the matrix $\bar{J}_f$, and similarly $\bar{J}_q$, is computed as:

$$\bar{J}_f = \frac{1}{S} \sum_{0 \leq s \leq S-1} J_f(t_s) \quad (1059)$$

and J_f denotes the Jacobian of f with respect to u . The preconditioner equals the HB Jacobian in the limit of small signals, where the coupling terms between the frequencies vanish. Therefore, each diagonal block of P corresponds to the AC matrix for the respective harmonic. This preconditioner is well suited to ‘moderately large’ signal applications.

The preconditioner can be computed without an explicit Fourier transform, and its inversion is more economical than for the full Jacobian. The inversion is performed by applying a complex direct solver for each harmonic component separately, thereby requiring the computational costs of solving $(S + 1)/2$ complex-valued linear systems. The computational complexity is of the order $O(S)$ for inverting the preconditioner, and $O(S \ln(S))$ for one complete iteration step of the iterative solver, while the number of iterations necessary to achieve convergence is unknown (but bounded).

Direct Solver Method

For the direct solver, the complex-valued $S \times S$ linear system (Eq. 1057) is transformed to a $S \times S$ real-valued problem, which is possible as only real-valued functions are involved. The resulting linear system is solved by the direct solver PARDISO. The direct solver requires the entire matrix stored in memory. Therefore, the memory capacity is easily exceeded for increasing S . Additionally, the computational complexity is of the order $O(S^3)$.

Transient Simulation

Transient equations used in semiconductor device models and circuit analysis can be formally written as a set of ordinary differential equations:

$$\frac{d}{dt}q(z(t)) + f(t, z(t)) = 0 \quad (1060)$$

which can be mapped to the DC and transient parts of the PDEs.

Sentaurus Device uses implicit discretization of transient equations (see Eq. 1060) and supports two discretization schemes: simple backward Euler (BE) and composite trapezoidal rule/backward differentiation formula (TRBDF), which is the default.

Backward Euler Method

Backward Euler is a very stable method, but it has only a first-order of approximation over time-step h_n . The discretization can be written as:

$$q(t_n + h_n) + h_n f(t_n + h_n) = q(t_n) \quad (1061)$$

The local truncation error (LTE) estimation is based on the comparison of the obtained solution $q(t_n + h_n)$ with the linear extrapolation from the previous time-step. The extrapolated solution is written as:

$$q^{\text{extr}} = q(t_n) - \frac{f(t_n) + f(t_n + h_n)}{2} h_n \quad (1062)$$

Then, in every point, the relative error can be estimated as $(q(t_n + h_n) - q^{\text{extr}})/q(t_n + h_n)$.

Using [Eq. 1061](#) and [Eq. 1062](#), and estimating the norm of relative error, Sentaurus Device computes the value:

$$r = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\frac{f(t_n + h_n) - f(t_n)}{\varepsilon_{R,tr} |q_n(t_n + h_n)| + \varepsilon_{A,tr} h_n} \right)^2} \quad (1063)$$

where the sum is taken over all unknowns (that is, all free vertices of all equations), and $\varepsilon_{R,tr}$ and $\varepsilon_{A,tr}$ are the relative and absolute transient errors, respectively.

The next time-step is estimated as:

$$h_{\text{est}} = h_n r^{-1/2} \quad (1064)$$

The value of the estimated time-step is used for h_{n+1} computation (see [Controlling Transient Simulations on page 970](#)).

TRBDF Composite Method

The transient scheme [\[12\]](#) for the approximation of [Eq. 1060](#) is briefly reviewed in this section. From each time point t_n , the next time point $t_n + h_n$ (h_n is the current step size) is not directly reached. Instead, a step in between to $t_n + \gamma h_n$ is made. This improves the accuracy of the method. $\gamma = 2 - \sqrt{2}$ has been shown to be the optimal value. Using this, two nonlinear systems are reached.

For the trapezoidal rule (TR) step:

$$2q(t_n + \gamma h_n) + \gamma h_n f(t_n + \gamma h_n) = 2q(t_n) - \gamma h_n f(t_n) \quad (1065)$$

and for the BDF2 step:

$$(2 - \gamma)q(t_n + h_n) + (1 - \gamma)h_n f(t_n + h_n) = (1/\gamma)(q(t_n + \gamma h_n) - (1 - \gamma)^2 q(t_n)) \quad (1066)$$

The local truncation error (LTE) is estimated after such a double step as:

$$\tau = \left[\frac{f(t_n)}{\gamma} - \frac{f(t_n + \gamma h_n)}{\gamma(1 - \gamma)} + \frac{f(t_n + h_n)}{1 - \gamma} \right] \quad (1067)$$

$$C = \frac{-3\gamma^2 + 4\gamma - 2}{12(2 - \gamma)} \quad (1068)$$

Sentaurus Device then computes the following value from this:

$$r = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\frac{\tau_i}{\epsilon_{R,tr} |q_n(t_n + h_n)| + \epsilon_{A,tr}} \right)^2} \quad (1069)$$

where the sum is taken over all unknowns (that is, all free vertices of all equations), and $\epsilon_{R,tr}$ and $\epsilon_{A,tr}$ are the relative and absolute transient errors, respectively. Since the TRBDF method has a second-order approximation over h_n , the next step can be estimated as:

$$h_{est} = h_n r^{-1/3} \quad (1070)$$

The value of the estimated time-step is used for h_{n+1} computation (see [Controlling Transient Simulations on page 970](#)).

Controlling Transient Simulations

By default, Sentaurus Device uses the TRBDF method. To switch to backward Euler (BE), the statement `Transient=BE` must be specified in the `Math` section.

To evaluate whether a time-step was successful and to provide an estimate for the next step size, the following rules are applied:

- If one of the nonlinear systems cannot be solved, the step is refused and tried again with $h_n = 0.5 \cdot h_n$.
- Otherwise, the inequality $r < 2f_{rej}$ is tested. If it is fulfilled, the transient simulation proceeds with $h_{n+1} = h_{est}$. Otherwise, the step is re-tried with $h_n = 0.9 \cdot h_{est}$.
- The LTE is checked only if the `CheckTransientError` option is selected; otherwise, the selection of the next time-step is based only on convergence of nonlinear iterations.

To activate LTE evaluation and time-step control, `CheckTransientError` must be specified either globally (in the `Math` section) or locally as an option in the `Transient` statement. The keyword `NoCheckTransientError` disables time-step control. The value of the relative error is defined by the parameter `TransientDigits` according to [Eq. 15, p. 121](#).

Absolute error is given by the keyword `TransientError` or recomputed from `TransientErrRef` ($x_{ref,tr}$) using [Eq. 16, p. 121](#) (if `RelErrControl` is switched on). Sentaurus Device provides the default values of $\epsilon_{R,tr}$, $\epsilon_{A,tr}$, and $x_{ref,tr}$. The coefficient f_{rej} is equal to 1 by default. You can define the values of $\epsilon_{R,tr}$, $\epsilon_{A,tr}$, $x_{ref,tr}$, and f_{rej} globally in the `Math` section, or specify them as options in the `Transient` statement. In the latter case, it overwrites the default and `Math` specifications for this command.

Floating Gates

During a transient time step, the charge Q of a floating gate is updated as a function of the injection current i :

$$\Delta Q = \int_{t_1}^{t_2} i(t) dt \quad (1071)$$

ΔQ represents the charge increase for the time step $[t_1, t_2]$.

Because the floating-gate charge is not updated self-consistently during a transient step, but only as a postprocessing operation, the numeric update is given by:

$$\Delta Q = i(t_1) \cdot \Delta t \quad (1072)$$

where $\Delta t = t_2 - t_1$. The error of this numeric approximation is estimated by:

$$\Delta Q_{\text{error}} = \frac{|i(t_2) - i(t_1)|}{2} \Delta t \quad (1073)$$

With the option `CheckTransientError`, the error ΔQ_{error} in the charge update is monitored as well. In the case of relative error control (`RelErrControl`), a transient step is only accepted if the following condition holds:

$$\frac{\Delta Q_{\text{error}}}{10^{-\text{Digits}}(|Q| + \text{ErrRef})} < 1 \quad (1074)$$

The values of `Digits` and `ErrRef` can be specified in the `Math` section:

```
Math {
  TransientDigits = 3
  TransientErrRef (Charge) = 1.602192e-19
}
```

In the case of absolute error control (`-RelErrControl`), a transient step is only accepted if the following condition holds:

$$\frac{\frac{\Delta Q_{\text{error}}}{q}}{10^{-\text{Digits}} \frac{|Q|}{q} + \text{Error}} < 1 \quad (1075)$$

The electron charge $q = 1.602192 \cdot 10^{-19} \text{ C}$ is used as a scaling factor. The values of `Digits` and `Error` can be specified in the `Math` section:

```
Math {
  TransientDigits = 3
  TransientError (Charge) = 1e-3
}
```

Note that the values of `ErrRef` and `Error` are related by the equation:

$$\text{ErrRef} = \frac{\text{Error}}{10^{-\text{Digits}}} q \quad (1076)$$

Nonlinear Solvers

In the next two sections, the `Digits` variable corresponds to the keyword `Digits`, which can be given in the `Math` section (see [Coupled Error Control on page 159](#)), or in parentheses of each `Plugin` or `Coupled` statement.

Fully Coupled Solution

For the solution of nonlinear systems, the scheme developed by Bank and Rose [13] is applied. This scheme tries to solve the nonlinear system $g(z) = 0$ by the Newton method:

$$\vec{g} + \vec{g}' \vec{x} = 0 \quad (1077)$$

$$\vec{z}^j - \vec{z}^{j+1} = \lambda \vec{x} \quad (1078)$$

where λ is selected such that $\|g_{k+1}\| < \|g_k\|$, but is as close as possible to 1. Sentaurus Device handles the error by computing an error function that can be defined by two methods.

The Newton iterations stop if the convergence criteria are fulfilled. One convergence criterion is the norm of the right-hand side, that is, $\|g\|$ in [Eq. 1077](#). Another natural criterion may be the relative error of the variables measured, such as $\left\| \frac{(\lambda x)}{z} \right\|$.

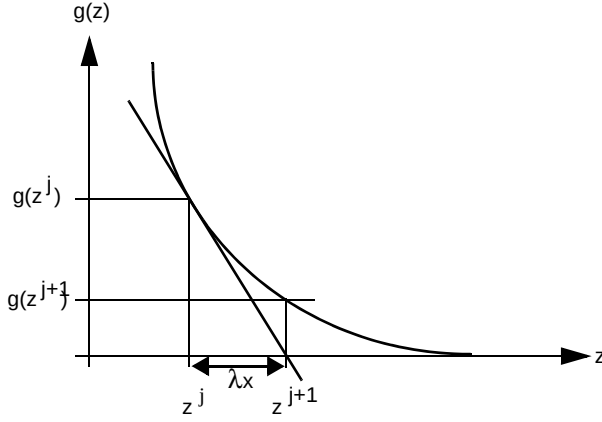


Figure 107 Newton iteration

Conversely, for very small z updates, λx must be measured with respect to some reference value of the variable z_{ref} . The formula used in Sentaurus Device as the second convergence criterion is:

$$\frac{1}{\epsilon_R} \frac{1}{N} \sum_{e,i} \frac{|z(e,i,j) - z(e,i,j-1)|}{|z(e,i,j)| + z_{\text{ref}}(e)} < 1 \quad (1079)$$

where $z(e,i,j)$ is the solution of the equation e (Poisson, electron, hole, and so on) at node i after Newton iteration j . The constant N is given by the total number of nodes multiplied by the total number of equations. The parameter ϵ_R is the relative error criterion.

The value of $\epsilon_R = 10^{-\text{Digits}}$ is set by specifying the following in the Math section:

```
Math{...
  Digits = 5
}
```

where 5 is the default for `Digits`. The reference values $z_{\text{ref}}(e)$ ensure numeric stability even for cases when $z(e,i,j)$ is zero or very small. This error condition ensures that the respective equations are solved to an accuracy of approximately $z_{\text{ref}}(e)\epsilon_R$.

Eq. 1079 can be written in the symbolic form:

$$\frac{1}{\epsilon_R} \left\| \frac{\lambda x}{z^j + z_{\text{ref}}} \right\| < 1 \quad (1080)$$

Eq. 1080 can also be rewritten in the equivalent form:

$$\left\| \frac{\lambda \bar{x}}{\varepsilon_R \bar{z}^j + \varepsilon_A} \right\| < 1 \quad (1081)$$

where $\bar{z}^j = z^j/z^*$ and $\bar{x} = x/z^*$.

z^* is the normalization factor (for example, it is the intrinsic carrier density $n_i = 1.48 \times 10^{10} \text{ cm}^{-3}$ for electron and hole equations, and the thermal voltage $u_{T0} = 25.8 \text{ mV}$ for the Poisson equation).

The absolute error is related to the relative error through:

$$\varepsilon_A = \varepsilon_R \frac{z_{\text{ref}}}{z^*} \quad (1082)$$

Sentaurus Device supports two schemes for controlling the error conditions. The default scheme is based on Eq. 1079. The default values for the parameters z_{ref} are listed in Table 152 on page 1229. They also are accessible in the Math section:

```
Math{...
  ErrRef( Electron ) = 1e10
  ErrRef( Hole )     = 1e10
}
```

The second scheme is activated with the keyword `-RelErrControl` in the Math section and is based on Eq. 1081. The default values for the parameters ε_A are listed in Table 152. They also are accessible in the Math section:

```
Math{...
  -RelErrControl
  Error( Electron ) = 1e-5
  Error( Hole )     = 1e-5
}
```

‘Plugin’ Iterations

This is the traditional scheme, which is also known as ‘Gummel iterations’ in most other device simulators. Consider that there are n sets of nonlinear systems $g_j(z_1 \dots z_n) = 0$. (n can be, for example, 3 and the sets can be the Poisson equation and two continuity equations.) This method starts with values $z_1^{(1)}, \dots, z_n^{(1)}$ and then solves each set $g_j = 0$ separately and consecutively. One loop could be:

$$\begin{aligned} g_1(z_1^{(i)} z_2^{(i)} \dots z_n^{(i)}) &= 0 \Rightarrow z_1^{(i+1)} \\ \dots \\ g_1(z_1^{(i+1)} \dots z_{n-1}^{(i+1)} z_n) &= 0 \Rightarrow z_n^{(i+1)} \end{aligned} \quad (1083)$$

If an update (λx) of the solution between two successive plugin iterations is defined as:

$$(\lambda x) = z_j^{(i+1)} - z_j^{(i)} \quad (1084)$$

[Eq. 1080](#) or [Eq. 1081](#) can be applied for convergence control in plugin iterations.

References

- [1] R. E. Bank, D. J. Rose, and W. Fichtner, “Numerical Methods for Semiconductor Device Simulation,” *IEEE Transactions on Electron Devices*, vol. ED-30, no. 9, pp. 1031–1041, 1983.
- [2] R. S. Varga, *Matrix Iterative Analysis*, Englewood Cliffs, New Jersey: Prentice-Hall, 1962.
- [3] E. M. Buturla *et al.*, “Finite-Element Analysis of Semiconductor Devices: The FIELDAY Program,” *IBM Journal of Research and Development*, vol. 25, no. 4, pp. 218–231, 1981.
- [4] H. Edelsbrunner, “Triangulations and meshes in computational geometry,” *Acta Numerica*, vol. 9, pp. 133–213, March 2000.
- [5] S.-W. Cheng *et al.*, “Sliver Exudation,” *Journal of the ACM*, vol. 47, no. 5, pp. 883–904, 2000.
- [6] H. Edelsbrunner and D. Guoy, “An Experimental Study of Sliver Exudation,” in *Proceedings of the 10th International Meshing Roundtable*, Newport Beach, CA, USA, pp. 307–316, October 2001.

40: Numeric Methods

References

- [7] S. E. Laux, “Application of Sinusoidal Steady-State Analysis to Numerical Device Simulation,” in *New Problems and New Solutions for Device and Process Modelling: An International Short Course held in association with the NASECODE IV Conference*, Dublin, Ireland, pp. 60–71, 1985.
- [8] B. Troyanovsky, Z. Yu, and R. W. Dutton, “Physics-based simulation of nonlinear distortion in semiconductor devices using the harmonic balance method,” *Computer Methods in Applied Mechanics and Engineering*, vol. 181, no. 4, pp. 467–482, 2000.
- [9] P. J. C. Rodrigues, *Computer-Aided Analysis of Nonlinear Microwave Circuits*, Boston: Artech House, 1998.
- [10] Y. Saad, *Iterative Methods for Sparse Linear Systems*, Philadelphia: SIAM, 2nd ed., 2003.
- [11] P. Feldmann, B. Melville, and D. Long, “Efficient Frequency Domain Analysis of Large Nonlinear Analog Circuits,” in *Proceedings of the IEEE Custom Integrated Circuits Conference*, San Diego, CA, USA, pp. 461–464, May 1996.
- [12] R. E. Bank *et al.*, “Transient Simulation of Silicon Devices and Circuits,” *IEEE Transactions on Computer-Aided Design*, vol. CAD-4, no. 4, pp. 436–451, 1985.
- [13] R. E. Bank and D. J. Rose, “Global Approximate Newton Methods,” *Numerische Mathematik*, vol. 37, no. 2, pp. 279–295, 1981.

Part V Physical Model Interface

This part of the *Sentaurus Device User Guide* contains the following chapter:

[Chapter 41 Physical Model Interface on page 979](#)

CHAPTER 41 Physical Model Interface

This chapter discusses the flexible interface that is used to add new physical models to Sentaurus Device.

Overview

The physical model interface (PMI) provides direct access to certain models in the semiconductor transport equations. You can provide new C++ functions to compute these models, and Sentaurus Device loads the functions at run-time using the dynamic loader. No access to the Sentaurus Device source code is necessary. You can modify the following models:

- Generation–recombination rate R_{net} , see [Eq. 53, p. 197](#)
- Avalanche generation, that is, ionization coefficient α in [Eq. 360, p. 374](#)
- Electron and hole mobilities μ_n and μ_p , see [Eq. 56](#) and [Eq. 57, p. 199](#)
- Band gap, see [Chapter 12 on page 249](#)
- Bandgap narrowing E_{bgn} , see [Band Gap and Electron Affinity on page 249](#)
- Complex refractive index, see [Complex Refractive Index Model Interface on page 504](#)
- Electron affinity, see [Band Gap and Electron Affinity on page 249](#)
- Apparent band-edge shift, see [Density Gradient Quantization Model on page 288](#)
- Multistate configuration–dependent apparent band-edge shift, see [Apparent Band-Edge Shift on page 435](#)
- Multistate configuration–dependent thermal conductivity, heat capacity, and mobility, see [Thermal Conductivity, Heat Capacity, and Mobility on page 437](#)
- Effective mass, see [Effective Masses and Effective Density-of-States on page 261](#)
- Energy relaxation times τ , see [Eq. 82, p. 212](#) to [Eq. 84, p. 212](#)
- Lifetimes τ , as used in SRH recombination (see [Eq. 247, p. 313](#)) and CDL recombination (see [Eq. 336, p. 365](#))
- Thermal conductivity κ , see [Eq. 69, p. 209](#) and [Eq. 76, p. 211](#)
- Heat capacity c_L , see [Eq. 69, p. 209](#) and [Eq. 90, p. 213](#)
- Optical quantum yield, see [Quantum Yield Models on page 475](#) and [Optical Quantum Yield on page 1098](#)
- Stress, see [Stress on page 1101](#)
- Space factor, see [Metal Workfunction on page 242](#), [Energetic and Spatial Distribution of Traps on page 408](#), and [Additional Factor Model Options on page 723](#)

41: Physical Model Interface

Overview

- Trap capture and emission rates, see [Local Capture and Emission Rates from PMI on page 417](#)
- Trap energy shift, see [Trap Energy Shift on page 1112](#)
- Piezoelectric polarization, see [Dependency of Saturation Velocity on Stress on page 731](#)
- Incomplete ionization, see [Chapter 13 on page 273](#)
- Hot-carrier injection, see [Chapter 25 on page 625](#)
- Piezoresistive coefficients, see [Piezoresistance Mobility Model on page 717](#)
- Raytracing contact, see [Boundary Condition for Raytracing on page 520](#)
- Spatial distribution function, see [Heavy Ions on page 574](#)
- Metal resistivity, see [Transport in Metals on page 239](#)
- Heat generation rate, see [Thermodynamic Model for Lattice Temperature on page 209](#)
- Thermoelectric power, see [Thermoelectric Power \(TEP\) on page 749](#)
- Metal thermoelectric power, see [Thermoelectric Power \(TEP\) on page 749](#)

A separate interface is provided to add new entries to the current plot file, see [Current Plot File of Sentaurus Device on page 1132](#).

An interface is available that allows postprocessing of data during a transient simulation (see [Postprocess for Transient Simulation on page 1137](#)).

For most models, Sentaurus Device provides two equivalent interfaces:

- The standard interface (see [Standard C++ Interface on page 981](#)) is based on the data type `double`. Separate subroutines must be written to evaluate the model and its derivatives. This interface provides performance comparable to the built-in models in Sentaurus Device.
- The simplified interface (see [Simplified C++ Interface on page 985](#)) is based on the data type `pmi_float`. Only a single subroutine must be implemented to evaluate the model. The derivatives of the model are obtained by automatic differentiation. Furthermore, this interface also supports extended precision floating-point arithmetic (see [Extended Precision on page 185](#)).

The following steps are needed to use a PMI model in a Sentaurus Device simulation:

- A C++ subroutine must be implemented to evaluate the PMI model. In the case of the standard interface, additional C++ subroutines must be written to evaluate the derivatives of the PMI model with respect to all input variables.
- The `cmi` script produces a shared object file that Sentaurus Device loads at run-time (see [Shared Object Code on page 988](#)).

NOTE The version of the C++ compiler used for a PMI model must be identical to the version of the C++ compiler used at Synopsys to compile Sentaurus Device. Use the command `cmi -a` to verify the compiler versions.

- The `PMIPath` variable must be defined in the `File` section of the command file. This defines the search path for the shared object files. A PMI model is activated in the `Physics` section of the command file by specifying its name (see [Command File of Sentaurus Device on page 989](#)).
- Parameters for PMI models can appear in the parameter file (see [Parameter File of Sentaurus Device on page 1007](#)).

These steps are discussed further in the following sections. The source code for the examples is in the directory `$STR00T/tcad/$STRELEASE/lib/sdevice/src`.

Standard C++ Interface

For each PMI model, you must implement a C++ subroutine to evaluate the model. Additional subroutines are necessary to evaluate the derivatives of the model with respect to all the input variables. More specifically, you must implement a C++ class that is derived from a base class declared in the header file `PMIModels.h`. In addition, a so-called virtual constructor function must be provided, which allocates an instance of the derived class.

For example, consider the implementation of Auger recombination as a new PMI model. (The built-in Auger recombination model is discussed in [Auger Recombination on page 376](#).)

In its simplest form, Auger recombination can be written as:

$$R_{\text{net}} = C \cdot (n + p) \cdot (np - n_{i,\text{eff}}^2) \quad (1085)$$

Sentaurus Device needs to evaluate the value of R_{net} and the derivatives:

$$\begin{aligned} \frac{\partial R_{\text{net}}}{\partial n} &= C(np - n_{i,\text{eff}}^2 + (n + p)p) \\ \frac{\partial R_{\text{net}}}{\partial p} &= C(np - n_{i,\text{eff}}^2 + (n + p)n) \\ \frac{\partial R_{\text{net}}}{\partial n_{i,\text{eff}}} &= -2C(n + p)n_{i,\text{eff}} \end{aligned} \quad (1086)$$

41: Physical Model Interface

Standard C++ Interface

In the header file `PMIModels.h`, the following base class is defined for recombination models:

```
class PMI_Recombination : public PMI_Vertex_Interface {

public:
    PMI_Recombination (const PMI_Environment& env);
    virtual ~PMI_Recombination ();

    virtual void Compute_r
        (const double t, const double n, const double p,
         const double nie, const double f, double& r) = 0;

    virtual void Compute_drdt
        (const double t, const double n, const double p,
         const double nie, const double f, double& drdt) = 0;

    virtual void Compute_drdn
        (const double t, const double n, const double p,
         const double nie, const double f, double& drdn) = 0;

    virtual void Compute_drdp
        (const double t, const double n, const double p,
         const double nie, const double f, double& drdp) = 0;

    virtual void Compute_drdnie
        (const double t, const double n, const double p,
         const double nie, const double f, double& drdnie) = 0;

    virtual void Compute_drdf
        (const double t, const double n, const double p,
         const double nie, const double f, double& drdf) = 0;
};
```

To implement a PMI model for Auger recombination, you must declare a derived class:

```
#include "PMIModels.h"

class Auger_Recombination : public PMI_Recombination {

    double C;

public:
    Auger_Recombination (const PMI_Environment& env);
    ~Auger_Recombination ();

    void Compute_r
        (const double t, const double n, const double p,
         const double nie, const double f, double& r);
```

```

void Compute_drdt
    (const double t, const double n, const double p,
     const double nie, const double f, double& drdt);

void Compute_drdn
    (const double t, const double n, const double p,
     const double nie, const double f, double& drdn);

void Compute_drdp
    (const double t, const double n, const double p,
     const double nie, const double f, double& drdp);

void Compute_drdnie
    (const double t, const double n, const double p,
     const double nie, const double f, double& drdnie);

void Compute_drdf
    (const double t, const double n, const double p,
     const double nie, const double f, double& drdf);
};

```

The constructor of the derived class is invoked for each region of the device. In this example, the variable *C* is initialized from the parameter file:

```

Auger_Recombination::
Auger_Recombination (const PMI_Environment& env) :
    PMI_Recombination (env)
{ C = InitParameter ("C", 1e-30);
}

```

If the parameter *C* is not found in the parameter file, a default value of 10^{-30} is used (see [Parameter File of Sentauros Device on page 1007](#)). During a Newton iteration, Sentauros Device evaluates a PMI model for each mesh vertex. The method `Compute_r ( )` computes the recombination rate for a given vertex. According to the parameter list, the recombination rate can depend on the following variables:

t	Lattice temperature
n	Electron density
p	Hole density
nie	Effective intrinsic density
f	Absolute value of electric field

41: Physical Model Interface

Standard C++ Interface

The result of the function is stored in the parameter `r`:

```
void Auger_Recombination::
Compute_r (const double t, const double n, const double p,
           const double nie, const double f, double& r)
{ r = C * (n + p) * (n*p - nie*nie);
  if (r < 0.0) {
    r = 0.0;
  }
}
```

Besides `Compute_r()`, you must implement other methods to compute the partial derivatives of the recombination rate with respect to the input variables `t`, `n`, `p`, `nie`, and `f`. The implementation of `Compute_drdrn()` to compute the value of $\partial R / \partial n$ is:

```
void Auger_Recombination::
Compute_drdrn (const double t, const double n, const double p,
               const double nie, const double f, double& drdn)
{ double r = C * (n + p) * (n*p - nie*nie);
  if (r < 0.0) {
    drdn = 0.0;
  } else {
    drdn = C * ((n*p - nie*nie) + (n + p) * p);
  }
}
```

Finally, you must provide a so-called virtual constructor function, which allocates a variable of the new class:

```
extern "C"
PMI_Recombination* new_PMI_Recombination (const PMI_Environment& env)
{ return new Auger_Recombination (env);
}
```

NOTE This function must have C linkage and exactly the same name as declared in the header file `PMIModels.h`.

Simplified C++ Interface

The simplified interface is based on the data type `pml_float`. This data type behaves similar to a `double`, and it supports all the usual arithmetic operations:

- Assignment:

```
pml_float x = 2;
pml_float y (x);
```

- Unary operators:

```
+x; -y;
```

- Binary operators:

```
x + y; x - y; x * y; x / y;
```

- Comparisons:

```
x == y; x != y; x < y; x <= y; x > y; x >= y;
```

- Mathematical functions:

```
abs(x); acos(x); acosh(x); asin(x); asinh(x); atan(x); atanh(x);
atan2(y,x); cos(x); cosh(x); erf(x); erfc(x); exp(x); expm1(x); hypot(x,y);
isinf(x); isnan(x); ldexp(x,exp); log(x); log1p(x); log10(x); pow(x,y);
pow_int(x,n); sin(x); sinh(x); sqrt(x); tan(x); tanh(x);
```

- Output:

```
std::cout << x;
```

The static function:

```
pml_e_precision pml_float::get_precision ()
```

returns the accuracy of the floating-point arithmetic. The result is expressed as an enumeration type:

```
enum pml_e_precision {
    pml_c_np, // normal precision (double): -ExtendedPrecision
    pml_c_xp, // extended precision (long double): ExtendedPrecision
    pml_c_dd, // double-double: ExtendedPrecision(128)
    pml_c_qd, // quad-double: ExtendedPrecision(256)
    pml_c_mp  // arbitrary precision: ExtendedPrecision(Digits=...)
};
```

41: Physical Model Interface

Simplified C++ Interface

Because the class `pmi_float` supports automatic differentiation, it must store both the value of a variable and the gradient vector of the derivatives with respect to the independent variables. The following methods are available to read the value of a variable, the size of its gradient vector, and the components of the gradient vector:

```
template <class des_t_float> des_t_float get_value ();
size_t size_gradient ();
template <class des_t_float> des_t_float get_gradient (size_t i);
```

Additional methods are available to set the value of a variable or its gradient:

```
template <class des_t_float> void set_value (const des_t_float a);
template <class des_t_float> void set_gradient (size_t i,
                                              const des_t_float a);
```

NOTE These methods for reading and writing the value and the gradient of a variable are not necessary for most models. They may be useful in cases where automatic differentiation yields wrong results.

Compared to the standard interface, the simplified interface only requires the implementation of a single subroutine to evaluate the model. As in [Standard C++ Interface on page 981](#), the following discusses how Auger recombination can be implemented as a PMI model. The header file `PMI.h` defines the following base class for recombination models:

```
class PMI_Recombination_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t;    // lattice temperature
        pmi_float n;    // electron density
        pmi_float p;    // hole density
        pmi_float nie;  // effective intrinsic density
        pmi_float f;    // absolute value of electric field
    };

    class Output {
    public:
        pmi_float r;    // recombination rate
    };

    PMI_Recombination_Base (const PMI_Environment& env);
    virtual ~PMI_Recombination_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```


To implement a user model, you must first declare a derived class:

```
#include "PMI.h"

class Auger_Recombination : public PMI_Recombination_Base {

private:
    double C;

public:
    Auger_Recombination (const PMI_Environment& env);
    ~Auger_Recombination ();

    virtual void compute (const Input& input, Output& output);
};
```

In the constructor, the variable C is initialized from the parameter file:

```
Auger_Recombination::
Auger_Recombination (const PMI_Environment& env) :
    PMI_Recombination_Base (env)

{ C = InitParameter ("C", 1e-30);
}
```

The constructor is called for each region of the device to ensure that regionwise parameters are handled correctly.

Next, the actual Compute function must be implemented. It relies on the auxiliary classes Input and Output to read the input variables, and to store the recombination rate:

```
void Auger_Recombination::
compute (const Input& input, Output& output)

{ output.r = C * (input.n + input.p) *
                (input.n*input.p - input.nie*input.nie);
    if (output.r < 0.0) {
        output.r = 0.0;
    }
}
```

Finally, a virtual constructor must be supplied to allocate instances of the class Auger_Recombination:

```
extern "C"
PMI_Recombination_Base* new_PMI_Recombination_Base
    (const PMI_Environment& env)
```

41: Physical Model Interface

Shared Object Code

```
{ return new Auger_Recombination (env);  
}
```

NOTE This function must have C linkage and exactly the same name as declared in the header file `PMI.h`.

It is possible to implement a model using both the standard interface and the simplified interface within the same file. In this case, Sentaurus Device will select the version based on the floating-point precision:

- The standard interface is selected for normal precision (64 bits). This ensures a performance similar to built-in models.
- The simplified interface is selected for extended precision floating-point arithmetic. No loss of accuracy occurs when the PMI model is invoked.

NOTE The simplified interface is ideally suited for prototyping a new model. No derivatives need to be implemented, which accelerates the development cycle. After a model has been validated, it can be converted easily into the standard interface for performance-critical applications.

As the simplified interface calculates the derivatives for you, it is easy to overlook cases where the expressions themselves are well defined, but their derivatives are not. Assume, for example, that your model computes $\sqrt{n - n_0}$, and you have ensured that $n - n_0$ cannot become negative. This is not enough because, when $n = n_0$, the derivative $1/2\sqrt{n - n_0}$ becomes infinite.

NOTE Ensure that not only the expressions you use, but also their derivatives are always valid.

Shared Object Code

Sentaurus Device assumes that the shared object code corresponding to a PMI model can be found in the file `modelname.so.arch`. The base name of this file must be identical to the name of the PMI model. The extension `.arch` depends on the hardware architecture. The script `cmi`, which is also a part of the CMI, can be used to produce the shared object files (see [Compact Models User Guide, Run-Time Support on page 135](#)).

Command File of Sentaurus Device

To load PMI models into Sentaurus Device, the `PMIPath` search path must be defined in the `File` section of the command file. The value of `PMIPath` consists of a sequence of directories, for example:

```
File {
    PMIPath = ". /home/joe/lib /home/mary/sdevice/lib"
}
```

For each PMI model, which appears in the `Physics` section, the given directories are searched for a corresponding shared object file `modelName.so.arch`.

The PMI in Sentaurus Device provides access to mesh-based scalar fields specified by you. These fields must be defined on the device grid in a separate TDR (extension `.tdr`) data file. Up to 100 datasets (`PMIUserField0`, ..., `PMIUserField99`) can be defined. Sentaurus Device reads the user-defined fields if the corresponding file name is given in the command file:

```
File {
    PMIUserFields = "fields"
}
```

A PMI model can be activated in the `Physics` section of the command file by specifying the name of the PMI model in the appropriate part of the `Physics` section. Examples for different types of PMI models are:

- Generation–recombination models:

```
Physics {
    Recombination (pmi_model_name ...)
}
```

- Avalanche generation:

```
Physics {
    Recombination (Avalanche (pmi_model_name ...))
}
```

- Mobility models:

```
Physics {
    Mobility (
        DopingDependence (pmi_model_name)
        Enormal (pmi_model_name)
        ToCurrentEnormal (pmi_model_name)
        HighFieldSaturation (pmi_model_name driving_force)
    )
}
```

A PMI model name can only consist of alphanumeric characters and underscores (_). The first character must be either a letter or an underscore. A PMI model name can also be quoted as "model_name" to avoid conflicts with Sentaurus Device keywords.

All the PMI models can be specified regionwise or materialwise:

```
Physics (region = "Region.1") {  
    ...  
}  
Physics (material = "AlGaAs") {  
    ...  
}
```

PMI models also recognize parameters in the command file. Usually, command file parameters are listed in parentheses after the model name, for example:

```
Physics {  
    Recombination (pmi_model_name (a = 1  
                                   b = "string"  
                                   c = (1.2 3.4 5.6 7.8)  
                                   d = ("red" "blue" "green")))  
}
```

For certain models, the syntax differs from the above example, and this will be noted in the documentation (see [Appendix G on page 1217](#)).

NOTE PMI model parameters also can be specified in the parameter file (see [Parameter File of Sentaurus Device on page 1007](#)). Parameters in the command file take precedence over parameters in the parameter file.

Certain values of PMI models can be plotted in the **Plot** section of the command file. The following identifiers are recognized:

- Generation–recombination models:

```
Plot {  
    PMIREcombination  
}
```

- User-defined fields:

```
Plot {  
    PMIUserField0 PMIUserField1 ... PMIUserField99  
}
```

- Piezoelectric polarization:

```
Plot {  
    PE_Polarization/vector PE_Charge  
}
```

- Metal conductivity (see [Metal Resistivity on page 1149](#)):

```
Plot {
    MetalConductivity
}
```

The current plot PMI can be used to add entries to the current plot file:

```
CurrentPlot {
    pmi_CurrentPlot
}
```

Vertex-based Run-Time Support

A standard vertex-based PMI is derived from the base class `PMI_Vertex_Interface`; whereas, a simplified vertex-based PMI is derived from the base class `PMI_Vertex_Base`. Both base classes provide the following shared functionality:

```
const char* Name () const;
const char* ReadRegionName () const;
const char* ReadRegionMaterial () const;
const char* ReadDeviceName () const;

const PMIBaseParam* ReadParameter (const char* name
                                   [, const char* modelName]) const;
double InitParameter (const char* name, double defaultvalue
                     [, const char* modelName]) const;
void InitParameter (const char* name, std::vector<double>& value
                   [, const char* modelName]) const;
const char* InitStringParameter (const char* name,
                                const char* defaultvalue
                                [, const char* modelName]) const;
void InitStringParameter (const char* name,
                          std::vector<const char*>& value
                          [, const char* modelName]) const;

int ReadDimension () const;
void ReadReferenceCoordinates (double ref [3][3]) const;

size_t NumberOfMSConfigStates (const std::string& msconfig_name) const;
const std::string& MSConfigStateName (const std::string& msconfig_name,
                                       size_t state_index) const;
```

The method `Name()` returns the name of the PMI model as specified in the command file. Similarly, `ReadRegionName()`, `ReadRegionMaterial()`, and `ReadDeviceName()` return the name of the current region, the material of the current region, and the name of the

device, respectively. The methods `ReadParameter()`, `InitParameter()`, and `InitStringParameter()` read the value of a parameter from the parameter file or command file (see [Parameter File of Sentaurus Device on page 1007](#) and [Command File of Sentaurus Device on page 989](#)). In these methods, `modelName` is an optional argument that allows the value of a parameter to be read from a different PMI model.

`ReadDimension()` returns the dimension of the problem. The method `ReadReferenceCoordinates()` provides access to the reference coordinate system. It will return the identity matrix:

$$\text{ref} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (1087)$$

in the case of the unified coordinate system (UCS). Otherwise, another coordinate system matrix is returned.

Multistate configuration-dependent models can call the two methods `NumberOfMSConfigStates()` and `MSConfigStateName()`. They return the number of states and the name of a state, respectively.

The following interface functions depend on the local vertex where a model is evaluated. They are not available in the constructor of the PMI, but should only be called in the `Compute` function. In the case of the standard interface, these functions are a part of the base class `PMI_Vertex_Interface`; whereas, the simplified interface provides them through the base class `PMI_Vertex_Input_Base`:

```
void ReadCoordinate (double& x, double& y, double& z) const;
double ReadDistanceFromSemiconductorInsulatorInterface() const;
double ReadDistanceFromHighkInsulator() const;
int ReadNearestInterfaceOrientation() const;

double ReadTime () const;
double ReadTransientStepSize () const;
PMI_StepType ReadTransientStepType () const;

double ReadxMoleFraction () const;
double ReadyMoleFraction () const;

double ReadDoping (PMI_DopingSpecies species) const;
double ReadDoping (const char* SpeciesName) const;

int IsUserFieldDefined (PMI_UserFieldIndex index) const;
double ReadUserField (PMI_UserFieldIndex index) const;
void WriteUserField (PMI_UserFieldIndex index, double value) const;

double ReadStress (PMI_StressIndex index) const;
```

```
bool ReadMSCOccupations (const std::string& msc_name, double* values) const;

double ReadSHEDistribution (double energy) const;
double ReadhSHEDistribution (double energy) const;
double ReadSHETotalDOS (double energy) const;
double ReadhSHETotalDOS (double energy) const;
double ReadSHETotalGSV (double energy) const;
double ReadhSHETotalGSV (double energy) const;
```

NOTE The support functions for the simplified interface use the data type `pmi_float`; whereas, the support functions for the standard interface use the data type `double`. However, their functionality is identical, and only the `double` version is documented.

The function `ReadCoordinate()` provides the coordinates of the current vertex [μm].

`ReadDistanceFromSemiconductorInsulatorInterface()` returns the distance of the current vertex to the nearest semiconductor–insulator interface (in μm) if the current vertex is in a semiconductor region; otherwise, the function returns minus that distance.

`ReadDistanceFromHighkInsulator()` returns the distance of the current vertex to the nearest high-k insulator (in μm). If no high-k insulator is found in the structure, the function returns a value < 0 .

`ReadNearestInterfaceOrientation()` returns the auto-orientation framework orientation (as a three-digit integer) that is closest to the actual orientation at the nearest interface vertex (see [Auto-Orientation Framework on page 75](#)).

The functions `ReadTime()` and `ReadTransientStepSize()` return the simulation time and the current step size during a transient simulation [s]. `ReadTransientStepType()` provides access to the actual transient step type. The enumeration type `PMI_StepType` is defined for identification:

```
enum PMI_StepType {
    PMI_UndefinedStepType = 0,
    PMI_TR = 1,
    PMI_BDF = 2,
    PMI_BE = 3
}
```

The methods `ReadxMoleFraction()` and `ReadyMoleFraction()` return the x and y mole fractions, respectively.

The methods `ReadDoping(species)` and `ReadDoping(SpeciesName)` return the doping profiles for the current vertex [cm^{-3}]. The string `SpeciesName` is the same as in the file

datexcodes.txt (see [User-Defined Species on page 50](#)). The enumeration type PMI_DopingSpecies is used to select the doping species, the incomplete ionization doping species, and their derivatives:

```
enum PMI_DopingSpecies {
    // Acceptors
    PMI_BoronActive,      // active Boron concentration
    PMI_BoronChemical,    // chemical Boron concentration
    PMI_IndiumActive,     // active Indium concentration
    PMI_IndiumChemical,   // chemical Indium concentration
    PMI_PDopantActive,    // active PDopant concentration
    PMI_PDopantChemical,  // chemical PDopant concentration
    PMI_Acceptor,         // total acceptor concentration

    // incomplete ionization entries
    PMI_AcceptorMinus,    // total incomplete ionization acceptor concentration
    PMI_AcceptorMinusPer_hDensity,
    PMI_AcceptorMinusPerT,

    // Donors
    PMI_PhosphorusActive, // active Phosphorus concentration
    PMI_PhosphorusChemical, // chemical Phosphorus concentration
    PMI_ArsenicActive,    // active Arsenic concentration
    PMI_ArsenicChemical,  // chemical Arsenic concentration
    PMI_AntimonyActive,   // active Antimony concentration
    PMI_AntimonyChemical, // chemical Antimony concentration
    PMI_NitrogenActive,   // active Nitrogen concentration
    PMI_NitrogenChemical, // chemical Nitrogen concentration
    PMI_NDopantActive,    // active NDopant concentration
    PMI_NDopantChemical,  // chemical NDopant concentration
    PMI_Donor,            // total donor concentration

    // incomplete ionization entries
    PMI_DonorPlus,        // total incomplete ionization donor concentration
    PMI_DonorPlusPer_eDensity,
    PMI_DonorPlusPerT,

    // additional species
    PMI_Carbon,
    PMI_CarbonChemical    // chemical Carbon concentration
};
```

NOTE The species PMI_Acceptor and PMI_Donor are always defined. The remaining entries are only defined if they occur in the simulated device. The incomplete ionization entries are only accessible if the option IncompleteIonization is activated (see [Chapter 13 on page 273](#)).

The method `IsUserFieldDefined()` checks if a user-defined field has been specified. The enumeration type `PMI_UserFieldIndex` selects the desired field:

```
enum PMI_UserFieldIndex {
    PMI_UserField0, PMI_UserField1, ..., PMI_UserField99
};
```

If a specific user-field is defined, the method `ReadUserField()` reads its value for the current vertex. For time-dependent user-fields, this is the value written in the last successful time step. `WriteUserField()` allows the modification of the value of a specific user-field for the current vertex. It writes to an auxiliary field. After a transient time step is finished, this auxiliary field becomes the field accessible with `ReadUserField()`.

The method `ReadStress()` returns the value of one of the following stress components:

```
enum PMI_StressIndex {
    PMI_StressXX, PMI_StressYY, PMI_StressZZ,
    PMI_StressYZ, PMI_StressXZ, PMI_StressXY
};
```

Vertex-based Run-Time Support for Multistate Configuration-dependent Models

The base class `PMI_MSC_Vertex_Interface` provides the same support as `PMI_Vertex_Interface`, plus additional functions needed by models that depend on a multistate configuration (see [Chapter 18 on page 425](#)):

```
class PMI_MSC_Vertex_Interface : public PMI_Vertex_Interface
{
public:
    PMI_MSC_Vertex_Interface(const PMI_Environment&,
        const std::string& msconfig_name,
        int model_index,
        const std::string& model_string);

    const std::string& msconfig_name () const;
    size_t nb_states () const;
    std::string& state ( size_t index ) const;
    int model_index () const;
    const std::string& model_string () const;
    virtual void init_parameter (){};
};
```

NOTE In the case of the simplified interface, the base class `PMI_MSC_Vertex_Base` is used instead. However, it provides the same functionality as the base class `PMI_MSC_Vertex_Interface` and, therefore, it is not documented separately.

The constructor argument `msconfig_name` determines the name of the multistate configuration on which the model depends, and the constructor arguments `model_index` and `model_string` are an integer and a string that you can evaluate in your model. You call the constructor `PMI_MSC_Vertex_Interface` only indirectly using constructors of base classes of multistate configuration-dependent PMIs.

The function `nb_states` returns the number of states in the selected multistate configuration, `state` returns the name of a particular state, and `msconfig_name`, `model_index`, and `model_string` return the arguments of the constructor of the same name.

The function `init_parameter` is always called before the parameters are changed. It allows you to keep model-internal data up-to-date in cases such as ramping of parameters.

Mesh-based Run-Time Support

A standard mesh-based PMI is derived from the base class `PMI_Device_Interface`; whereas, a simplified mesh-based PMI is derived from the base class `PMI_Device_Base`. Both base classes provide the following shared functionality:

```
const char* Name () const;

const PMIBaseParam* ReadParameter (const char* name
                                   [, const char* modelName]) const;
double InitParameter (const char* name, double defaultvalue
                      [, const char* modelName]) const;
void InitParameter (const char* name, std::vector<double>& value
                   [, const char* modelName]) const;
const char* InitStringParameter (const char* name,
                                 const char* defaultvalue
                                 [, const char* modelName]) const;
void InitStringParameter (const char* name,
                          std::vector<const char*>& value
                          [, const char* modelName]) const;

const des_mesh* Mesh () const;
des_data* Data () const;
```

The method `Name()` returns the name of the PMI model as specified in the command file. The methods `ReadParameter()`, `InitParameter()`, and `InitStringParameter()` read the value of a parameter from the parameter file or command file (see [Parameter File of Sentaurus](#)

[Device on page 1007](#) and [Command File of Sentaurus Device on page 989](#)). In these methods, `modelName` is an optional argument that allows the value of a parameter to be read from a different PMI model.

NOTE Parameters for a mesh-based PMI must appear in the global parameter section in the parameter file. Regionwise or materialwise parameters are not supported.

The methods `Mesh()` and `Data()` provide access to the mesh and data of Sentaurus Device (see [Device Mesh on page 998](#) and [Device Data on page 1004](#)).

NOTE In the case of the standard interface, the method `Data()` returns a variable of type `des_data`; whereas, in the case of the simplified interface, it returns a variable of type `device_data`. The type `des_data` is based on the data type `double`, and `sdevice_data` uses the data type `pmi_float`. Otherwise, their functionality is identical.

The following interface functions should only be called in the `Compute` function, but not in the constructor. The standard interface provides them as members of the base class `PMI_Device_Interface`; whereas, the simplified interface provides them as members of the base class `PMI_Device_Input_Base`:

```
double ReadTime () const;  
double ReadTransientStepSize () const;  
PMI_StepType ReadTransientStepType () const;
```

NOTE The support functions for the simplified interface use the data type `pmi_float`; whereas, the support functions for the standard interface use the data type `double`. However, their functionality is identical, and only the `double` version is documented.

The functions `ReadTime()` and `ReadTransientStepSize()` return the simulation time and the current step size during a transient simulation [s]. `ReadTransientStepType()` provides access to the actual transient step type. The enumeration type `PMI_StepType` is defined for identification:

```
enum PMI_StepType {  
    PMI_UndefinedStepType = 0,  
    PMI_TR = 1,  
    PMI_BDF = 2,  
    PMI_BE = 3  
}
```

Device Mesh

A device mesh of Sentaurus Device consists of a number of regions. A region is either a contact region consisting of a list of contact vertices or a bulk region consisting of a list of elements. An element is described by a list of vertices.

Vertex

In the file `PMIModels.h`, the class `des_vertex` is declared as follows:

```
class des_vertex {  
  
public:  
    size_t index () const;  
  
    const double* coord () const;  
  
    bool equal_coord (des_vertex* v) const;  
  
    size_t size_edge () const;  
    des_edge* edge (size_t i) const;  
  
    size_t size_element () const;  
    des_element* element (size_t i) const;  
  
    size_t size_region () const;  
    des_region* region (size_t i) const;  
  
    size_t size_regioninterface () const;  
    des_regioninterface* regioninterface (size_t i) const;  
};
```

The value of `index()` can be used as an index for vertex-based data (see [Device Data on page 1004](#)).

The location of a vertex [μm] is given by its coordinates `coord()`. The function `equal_coord()` should be used to check if two vertices have the same coordinates. For example, Sentaurus Device duplicates vertices along heterointerfaces. Consequently, two vertices with different indices can share the same coordinates.

`size_edge()` returns the number of edges connected to a vertex. The method `edge()` can be used to retrieve the *i*-th edge.

`size_region()` returns the number of regions containing a vertex. The method `region()` can be used to retrieve the *i*-th region.

`size_regioninterface()` reports how many region interfaces a vertex belongs to. The *i*-th region interface is returned by the method `regioninterface()`.

Edge

In the file `PMIModels.h`, the class `des_edge` is declared as follows:

```
class des_edge {  
  
public:  
    size_t index () const;  
  
    des_vertex* start () const;  
    des_vertex* end () const;  
  
    size_t size_element () const;  
    des_element* element (size_t i) const;  
  
    size_t size_region () const;  
    des_region* region (size_t i) const;  
};
```

The value of `index()` can be used as an index for edge-based data (see [Device Data on page 1004](#)).

`start()` and `end()` return the first and second vertex connected to the edge, respectively.

`size_element()` returns the number of elements connected to an edge. The method `element()` can be used to retrieve the *i*-th element.

`size_region()` returns the number of regions containing an edge. The method `region()` can be used to retrieve the *i*-th region.

Element

In the file `PMIModels.h`, the class `des_element` is declared as follows:

```
class des_element {  
  
public:  
    typedef enum { point, line, triangle, rectangle, tetrahedron,  
                  pyramid, prism, cuboid, tetrabrick } des_type;  
  
    size_t index () const;  
  
    des_type type () const;
```

41: Physical Model Interface

Mesh-based Run-Time Support

```

size_t size_vertex () const;
des_vertex* vertex (size_t i) const;

size_t size_edge () const;
des_edge* edge (size_t i) const;

des_bulk* bulk () const;
};

```

Figure 108 shows the numbering of vertices and edges for all element types.

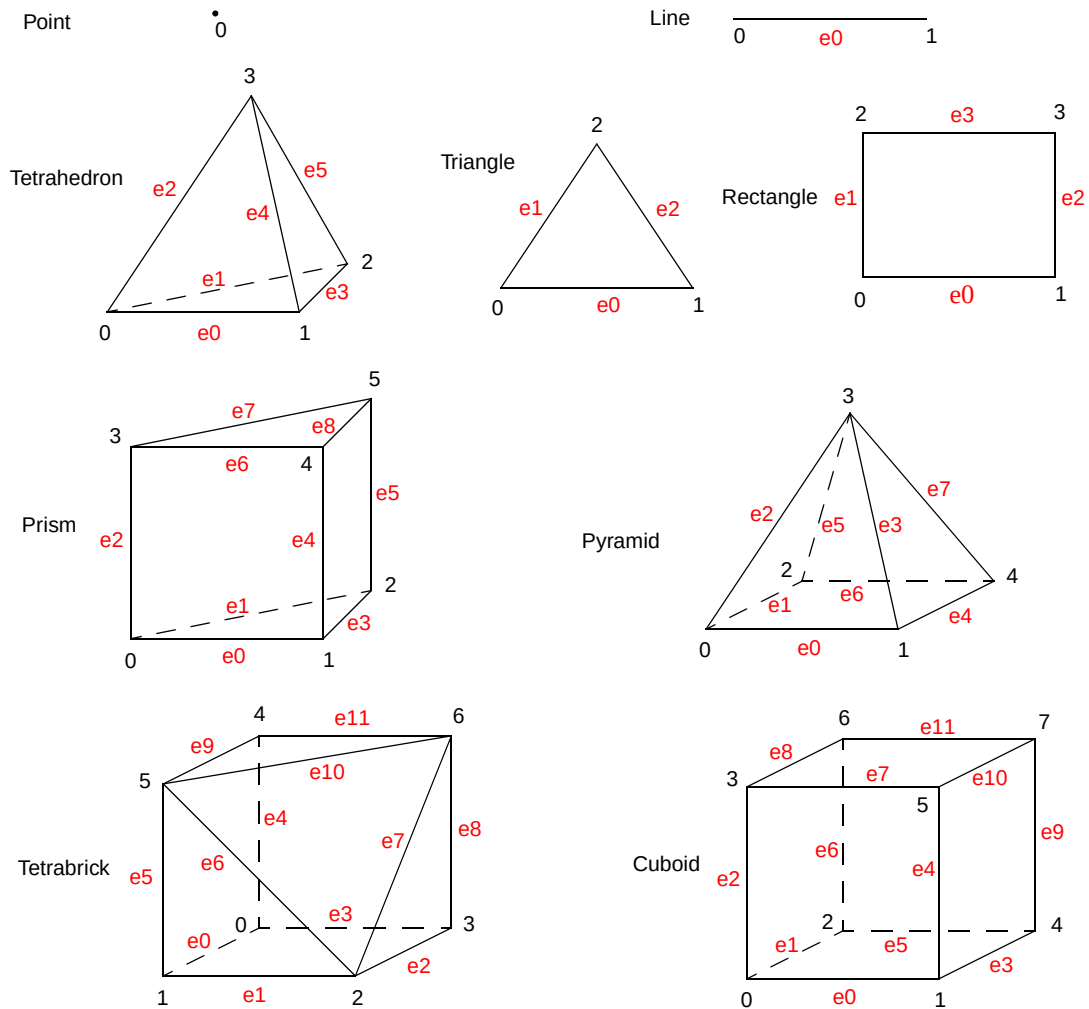


Figure 108 Vertex and edge numbering

The value of `index()` can be used as an index for element-based data (see [Device Data on page 1004](#)).

`type()` returns the type of an element (point, line, triangle, rectangle, tetrahedron, pyramid, prism, cuboid, or tetrabrick). An element is mainly described by its vertices. `size_vertex()` returns the number of vertices in an element, and the method `vertex()` can be used to retrieve the *i*-th vertex. `size_edge()` returns the number of edges of an element. The method `edge()` can be used to retrieve the *i*-th edge. The method `bulk()` returns the bulk region containing the element.

Region

In the file `PMIModels.h`, the base class `des_region` is declared as follows:

```
class des_region {
public:
    typedef enum { bulk, contact } des_type;

    virtual des_type type () const = 0;

    std::string name () const;

    size_t size_vertex () const;
    des_vertex* vertex (size_t i) const;

    size_t size_edge () const;
    des_edge* edge (size_t i) const;
};
```

A mesh of Sentaurus Device consists of two types of region: bulk regions and contacts. The virtual method `type()` returns the type of a region. The name of a region is returned by `name()`. `size_vertex()` returns the number of vertices in a region. The method `vertex()` can be used to retrieve the *i*-th vertex. `size_edge()` returns the number of edges in a region. The method `edge()` can be used to retrieve the *i*-th edge.

The class `des_bulk` is derived from `des_region`:

```
class des_bulk : public des_region {
public:
    des_type type () const;

    std::string material () const;

    size_t size_element () const;
    des_element* element (size_t i) const;
```

```
    size_t size_regioninterface () const;  
    des_regioninterface* regioninterface (size_t i) const;  
};
```

`material()` returns the name of the material in a bulk region. `size_element()` returns the number of elements in a region. The method `element()` can be used to retrieve the *i*-th element. `size_regioninterface()` reports how many region interfaces are connected to this bulk region. The *i*-th region interface can then be retrieved by the method `regioninterface()`.

Similarly, the class `des_contact` is also derived from `des_region`:

```
class des_contact : public des_region {  
  
public:  
    des_type type () const;  
};
```

Region Interface

A region interface separates two bulk regions. It is described by the following class:

```
class des_regioninterface {  
  
public:  
    size_t index () const;  
  
    des_bulk* bulk1 () const;  
    des_bulk* bulk2 () const;  
  
    bool is_heterointerface () const;  
  
    size_t size_vertex () const;  
    des_vertex* vertex (size_t i) const;  
    size_t index (size_t local_vertex_index) const;  
};
```

The value of `index()` is used to access the surface measure array (see [Device Data on page 1004](#)).

The two bulk regions connected to a region interface are returned by `bulk1()` and `bulk2()`.

Use `is_heterointerface()` to determine if double points exist for this interface. For a heterointerface, each vertex belongs to either region 1 or region 2, but not both. For a regular interface, each vertex belongs to both region 1 and region 2.

The number of vertices contained in a region interface is returned by the method `size_vertex()`. The *i*-th vertex can be obtained by invoking `vertex()`. The method `index(size_t local_vertex_index)` is used to obtain the correct index for interface-based data (see [Device Data on page 1004](#)).

Mesh

In the file `PMIModels.h`, the class `des_mesh` is declared as follows:

```
class des_mesh {

public:
    int dim () const;
    void ref_coordinates (double ref [3][3]) const;

    size_t size_vertex () const;
    des_vertex* vertex (size_t i) const;

    size_t size_edge () const;
    des_edge* edge (size_t i) const;

    size_t size_element () const;
    des_element* element (size_t i) const;

    size_t size_region () const;
    des_region* region (size_t i) const;

    size_t size_regioninterface () const;
    des_regioninterface* regioninterface (size_t i) const;
};
```

The dimension of the mesh is given by `dim()`. The possible values are 1, 2, and 3. The method `ref_coordinates()` provides access to the reference coordinate system. It will return the identity matrix:

$$\text{ref} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (1088)$$

in the case of the unified coordinate system. Otherwise, another coordinate system matrix will be returned.

`size_vertex()` returns the number of vertices in the mesh. The method `vertex()` can be used to retrieve the *i*-th vertex. `size_edge()` returns the number of edges in the mesh. The method `edge()` can be used to retrieve the *i*-th edge. `size_element()` returns the number of elements in the mesh. The method `element()` can be used to retrieve the *i*-th element.

`size_region()` returns the number of regions in the mesh. The method `region()` can be used to retrieve the *i*-th region. There are `size_regioninterface()` region interfaces in the mesh, and the *i*-th interface is returned by invoking `regioninterface()`.

Device Data

The class `des_data` provides the following functionality:

```
typedef enum { vertex, edge, element, rivertex } des_location;

const double*const* ReadCoefficient ();
const double*const* ReadMeasure ();
const double*const* ReadSurfaceMeasure ();

const double* ReadScalar (des_location location, std::string name);
const double*const* ReadVector (des_location location, std::string name);
void WriteScalar (des_location location, std::string name,
                  const double* newvalue);

const double*const* ReadGradient (des_location location, std::string name);
const double* ReadFlux (des_location location, std::string name);

size_t NumberOfMSCStates (const std::string& msc_name) const;
bool ReadMSCStateName (const std::string& msc_name, size_t state_index,
                      std::string& state_name) const;
bool ReadMSCOccupations (const std::string& msc_name,
                         const des_region* region,
                         double*const* values) const;

double ReadeSHEDistribution (des_bulk* r, des_vertex* v, double energy) const;
double ReadhSHEDistribution (des_bulk* r, des_vertex* v, double energy) const;
double ReadeSHETotalDOS (des_bulk* r, double energy) const;
double ReadhSHETotalDOS (des_bulk* r, double energy) const;
double ReadeSHETotalGSV (des_bulk* r, double energy) const;
double ReadhSHETotalGSV (des_bulk* r, double energy) const;
```

NOTE In the case of the simplified interface, the class `sdevice_data` provides the same methods, but uses the data type `pmi_float` instead of `double`. Otherwise, the functionality is identical.

The methods `ReadCoefficient()` and `ReadMeasure()` return the box method coefficients κ_{ij} and measure μ_{ij} used in Sentaurus Device (see [Discretization on page 949](#)).

`ReadCoefficient()` returns a two-dimensional array. The two indices are the element index and the local edge number. The units are μm^{-1} in 1D, 1 in 2D, and μm in 3D.

The following code fragment reads the coefficients for all element edges:

```
const des_mesh* mesh = Mesh();
des_data* data = Data();
const double*const* coeff = data->ReadCoefficient();
for (size_t eli = 0; eli < mesh->size_element(); eli++) {
    des_element* el = mesh->element(eli);
    for (size_t ei = 0; ei < el->size_edge(); ei++) {
        des_edge* e = el->edge(ei);
        const double c = coeff[el->index()][ei];
    }
}
```

NOTE The values κ_{ij} returned by `ReadCoefficient()` are element–edge coefficients. The edge coefficients κ_i can be obtained by adding the contributions from all elements connected to an edge i .

`ReadMeasure()` returns a two-dimensional array. The two indices are the element index and the local vertex number. The units are μm in 1D, μm^2 in 2D, and μm^3 in 3D.

The following code fragment reads the measures for all element vertices:

```
const des_mesh* mesh = Mesh();
des_data* data = Data();
const double*const* measure = data->ReadMeasure();
for (size_t eli = 0; eli < mesh->size_element(); eli++) {
    des_element* el = mesh->element(eli);
    for (size_t vi = 0; vi < el->size_vertex(); vi++) {
        des_vertex* v = el->vertex(vi);
        const double m = measure[el->index()][vi];
    }
}
```

NOTE The values μ_{ij} returned by `ReadMeasure()` are element–vertex measures. The node measures μ_i can be obtained by adding the contributions from all elements connected to a vertex i .

The method `ReadSurfaceMeasure()` provides the surface measure associated with region interface vertices (edge length [μm] in 2D and surface area [μm^2] in 3D). The two indices are the region interface index and the local vertex number.

The methods `ReadScalar()`, `ReadVector()`, and `WriteScalar()` provide access to the data of Sentaurus Device. The values can be located on vertices, edges, elements, or region interfaces. See [Table 140 on page 1186](#) and [Table 141 on page 1212](#) for an overview of available scalar and vector data.

`ReadScalar()` returns a read-only one-dimensional array. Use the `index()` method in the classes `des_vertex`, `des_edge`, or `des_element` to access the array elements. The proper addressing of region interface datasets is shown in the code fragment below.

`ReadVector()` returns a read-only two-dimensional array. The first index selects the dimension (0, 1, 2) and the second index is used in the same way as for scalar data.

`WriteScalar()` writes back a one-dimensional array. The organization of the array is the same as for the arrays obtained with `ReadScalar()`. You must ensure that the size of the array is sufficient to hold all required entries.

`ReadGradient()` returns a 2D array that contains, for each vertex, the gradient of a chosen variable. Thereby, the first index selects the partial derivative:

$$[0] = \left(\frac{\partial}{\partial x}\right), [1] = \left(\frac{\partial}{\partial y}\right), [2] = \left(\frac{\partial}{\partial z}\right) \quad (1089)$$

The second index identifies the vertex. The actual implementation of `ReadGradient()` works for vertex-based datasets only.

`ReadFlux()` returns an array that contains, for each vertex, the surface integral of the gradient of a chosen variable taken over the boundary of the box divided by the box volume. The actual implementation of `ReadFlux()` works for vertex-based datasets only.

The following code fragment traverses all region interfaces and reads the surface measure and a dataset for each region interface vertex:

```
const des_mesh* mesh = Mesh();
des_data* data = Data();
const double*const* surface = data->ReadSurfaceMeasure();
const double* hot_elec = data->ReadScalar(des_data::rivertex,
                                         "HotElectronInj");
for (size_t rii = 0; rii < mesh->size_regioninterface(); rii++) {
    des_regioninterface* ri = mesh->regioninterface(rii);
    for (size_t vi = 0; vi < ri->size_vertex(); vi++) {
        des_vertex* v = ri->vertex(vi);
        const double sm = surface[ri->index()][vi];
        const double he = hot_elec[ri->index(vi)];
    }
}
```

Parameter File of Sentaurus Device

For each PMI model, a corresponding section with the same name can appear in the parameter file:

```
PMI_model_name {  
    par1 = value  
    par2 = value  
    ...  
}
```

NOTE Parameter names can only consist of alphanumeric characters and underscores (_). The first character must be either a letter or an underscore.

Parameters can be numbers, or strings, or arrays of numbers or strings:

```
PMI_model_name {  
    a = 1  
    b = "string"  
    c = (1.2 3.4 5.6 7.8)  
    d = ("red" "blue" "green")  
}
```

NOTE Arrays must consist of either numbers or strings only. Mixed arrays containing both numbers and strings are not supported. An empty array, such as `c=( )`, is considered a numeric array of size 0.

The parameters can be specified regionwise and materialwise:

```
Region = "Region.1" {  
    PMI_model_name {  
        ...  
    }  
}  
Material = "AlGaAs" {  
    PMI_model_name {  
        ...  
    }  
}
```

The method `ReadParameter ( )` can be used to obtain the value of a parameter given its name. `ReadParameter ( )` returns a pointer to a variable of type `PMIBaseParam`:

```
const PMIBaseParam* p = ReadParameter ("name of parameter");
```

A NULL pointer indicates that the parameter has not been defined in the parameter file. Otherwise, the method

```
PMIBaseParam::ValueType()
```

returns the value type (`PMIBaseParam::real` or `PMIBaseParam::string`) of the parameter. Similarly, the method

```
PMIBaseParam::DataType()
```

returns the data type (`PMIBaseParam::scalar` or `PMIBaseParam::vector`) of the parameter. In the case of an array parameter, the size of the array can be obtained by

```
PMIBaseParam::size()
```

The size of scalar parameters is always 1.

Depending on the type of a parameter, its value can be accessed through an assignment statement:

```
double a = *p;           // real, scalar
double b = (*p)[index];  // real, vector
const char* c = *p;      // string, scalar
const char* d = (*p)[index]; // string, vector
```

An error occurs if the type of a parameter is incompatible with the left-hand side in the assignment statement.

In the case of a real scalar parameter, the method `InitParameter()` checks if the parameter has been specified in the parameter file:

```
double value = InitParameter ("pi", 3.14159);
```

If the parameter has been specified, the given value is taken. Otherwise, the default value is used.

An alternative version of `InitParameter()` is available for vector-valued parameters:

```
std::vector<double> v;
v.push_back (-1);           // default value
v.push_back (-2);           // default value
InitParameter ("vec", v);
```

If the parameter `vec` is found in the parameter file, the vector `v` is redefined with the new values. Otherwise, the existing default values in `v` are left unchanged.

Variants of these two methods are also available for string parameters:

```
const char* value = InitStringParameter ("scalar string", "default value");
std::vector<const char*> s;
InitStringParameter ("vector string parameter", s);
```

NOTE PMI model parameters also can be specified in the command file (see [Command File of Sentauros Device on page 989](#)). Parameters in the command file take precedence over parameters in the parameter file.

Parallelization

During a parallel simulation, Sentauros Device may invoke a PMI model concurrently from different threads. Therefore the computational functions must be implemented in a thread-safe manner. The following rules guarantee a thread-safe PMI:

- Do not use global variables.
- Class variables are only modified in the constructor and destructor. Class variables may be read in the computational functions, but they must not be modified.
- Within the computational functions, all temporary variables are allocated as either automatic variables or dynamic variables with the help of the `new` and `delete` operators.

Thread-Local Storage

The thread-local storage template class `PMI_TLS` provides a mechanism to store data per thread. This can be useful to optimize the run-time performance of a PMI. Consider an example where a large data structure is allocated and deallocated with each compute call:

```
class Recombination : public PMI_Recombination_Base {
public:
    ...
    void compute (const Input& input, Output& output)
    {
        BigData data;
        data.initialize ();
        // compute output
    }
};
```

41: Physical Model Interface

Parallelization

With the help of the template class `PMI_TLS`, a copy of `BigData` is allocated for each thread on demand:

```
class Recombination : public PMI_Recombination_Base {
private:
    PMI_TLS<BigData> Data;

public:
    ...
    void compute (const Input& input, Output& output)
    {
        bool exists;
        BigData& data = Data.local (exists);
        if (!exists) {
            data.initialize ();
        }
        // compute output
    }
};
```

The function call `Data.local(exists)` creates a new instance of `BigData` for each thread if it does not exist already. The argument `exists` is used to check if data has been freshly allocated. In this case, data is initialized. Otherwise, it was already initialized in a previous `compute()` call.

The template class `PMI_TLS` is declared as follows:

```
template <typename T> class PMI_TLS {
public:
    T& local (bool& exists);
    T& local ();
    size_t size () const;
    T& operator [] (size_t index);
};
```

Both variants of the method `local()` return a reference to a thread-local element. The optional argument `exists` indicates whether the element has been newly allocated (`false`), or if it was already present (`true`).

The function `size()` returns the number of allocated elements. The array access operator `[]` can be used to iterate over the elements of the container.

NOTE The array access operator `[]` should only be used in a sequential section of the code. For example, it may be used in the destructor of the PMI.

Debugging

Print statements represent the simplest approach for debugging a PMI. They can be inserted anywhere in the code to print the values of a variable, for example:

```
void Auger_Recombination::
compute (const Input& input, Output& output)

{ output.r = C * (input.n + input.p) *
      (input.n*input.p - input.nie*input.nie);
  if (output.r < 0.0) {
    output.r = 0.0;
  }
  std::cout << "n   = " << input.n << std::endl;
  std::cout << "p   = " << input.p << std::endl;
  std::cout << "nie = " << input.nie << std::endl;
  std::cout << "r   = " << output.r << std::endl;
}
```

It is also possible to use a debugger to catch errors in a PMI subroutine. The following instructions apply to `gdb`, the GNU debugger. The same approach also can be adjusted to work with other debuggers:

- Compile the PMI source code in debug mode by using the `-g` option:

```
cmi -g pmi_Auger.C
```

- Determine the name of the Sentaurus Device binary. The command:

```
sdevice -@ldd
```

should produce output similar to the following:

```
path to executable: /usr/sentaurus/tcad/G-2012.06/amd64/bin
executable: sdevice-1.4
```

```
/usr/sentaurus/tcad/G-2012.06/amd64/bin/sdevice-1.4:
ELF 64-bit LSB executable, AMD x86-64, version 1 (SYSV),
for GNU/Linux 2.4.0, dynamically linked (uses shared libs), stripped
```

In this example, `/usr/sentaurus/tcad/G-2012.06/amd64/bin/sdevice-1.4` is the name of the Sentaurus Device binary.

- Start the debugger `gdb` on the Sentaurus Device binary:

```
gdb /usr/sentaurus/tcad/G-2012.06/amd64/bin/sdevice-1.4
```

- Verify the setting of the environment variables:

```
show environment
```

In particular, the variables `STROOT` and `STRELEASE` must be defined properly. If necessary, use the following `gdb` command to define these variables:

```
set environment STROOT /usr/sentaurus
set environment STRELEASE G-2012.06
```

- It is not possible to define a break point in the PMI code until the corresponding shared object file has been loaded. Therefore, run your simulation:

```
run pp1_des.cmd
```

and watch for messages regarding the loading of shared object files. All the shared object files for PMIs are loaded at the very beginning, usually within seconds of starting Sentaurus Device. Afterwards, use the shortcut keys `Ctrl+C` to interrupt the simulation.

- You can now define a break point in the PMI source code, for example:

```
break pmi_Auger.C:100
```

This command inserts a break point on line 100 in the file `pmi_Auger.C`.

- After defining all required break points, you can resume the simulation with the command:

```
continue
```

The debugger will now stop whenever the control flow reaches a break point, and all the standard `gdb` commands are available for debugging.

Generation–Recombination Model

The recombination rate R_{net} appears in the electron and hole continuity equations (see [Eq. 53](#), [p. 197](#)).

Dependencies

The recombination rate R_{net} may depend on these variables:

t	Lattice temperature [K]
n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]
n_{ie}	Effective intrinsic density [cm^{-3}]
f	Absolute value of electric field [Vcm^{-1}]

The PMI model must compute the following results:

r Generation–recombination rate [$\text{cm}^{-3}\text{s}^{-1}$]

In the case of the standard interface, the following derivatives must be computed as well:

drdt Derivative of r with respect to t [$\text{cm}^{-3}\text{s}^{-1}\text{K}^{-1}$]

drdn Derivative of r with respect to n [s^{-1}]

drdp Derivative of r with respect to p [s^{-1}]

drdnie Derivative of r with respect to nie [s^{-1}]

drdf Derivative of r with respect to f [$\text{cm}^{-2}\text{s}^{-1}\text{V}^{-1}$]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_Recombination : public PMI_Vertex_Interface {
public:
    PMI_Recombination (const PMI_Environment& env);
    virtual ~PMI_Recombination ();

    virtual void Compute_r
        (const double t, const double n, const double p,
         const double nie, const double f, double& r) = 0;

    virtual void Compute_drdt
        (const double t, const double n, const double p,
         const double nie, const double f, double& drdt) = 0;

    virtual void Compute_drdn
        (const double t, const double n, const double p,
         const double nie, const double f, double& drdn) = 0;

    virtual void Compute_drdp
        (const double t, const double n, const double p,
         const double nie, const double f, double& drdp) = 0;

    virtual void Compute_drdnie
        (const double t, const double n, const double p,
         const double nie, const double f, double& drdnie) = 0;
    virtual void Compute_drdf
```

41: Physical Model Interface

Generation–Recombination Model

```
(const double t, const double n, const double p,  
    const double nie, const double f, double& drdf) = 0;  
};
```

The prototype for the virtual constructor is:

```
typedef PMI_Recombination* new_PMI_Recombination_func  
    (const PMI_Environment& env);  
extern "C" new_PMI_Recombination_func new_PMI_Recombination;
```

By default, Sentaurus Device assumes that a PMI generation–recombination model depends on the electric field. However, you can implement the optional function `PMI_Recombination_ElectricField()` to indicate whether the model depends on the electric field.

If the model does not depend on the electric field (return value of 0), the method `Compute_drdf()` is not called, and the matrix assembly in Sentaurus Device works more efficiently:

```
typedef int PMI_Recombination_ElectricField_func ();  
extern "C"  
PMI_Recombination_ElectricField_func PMI_Recombination_ElectricField;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_Recombination_Base : public PMI_Vertex_Base {  
  
public:  
    class Input : public PMI_Vertex_Input_Base {  
    public:  
        pmi_float t;    // lattice temperature  
        pmi_float n;    // electron density  
        pmi_float p;    // hole density  
        pmi_float nie;  // effective intrinsic density  
        pmi_float f;    // absolute value of electric field  
    };  
  
    class Output {  
    public:  
        pmi_float r;    // recombination rate  
    };  
  
    PMI_Recombination_Base (const PMI_Environment& env);  
    virtual ~PMI_Recombination_Base ();
```

```
virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_Recombination_Base* new_PMI_Recombination_Base_func
(const PMI_Environment& env);
extern "C" new_PMI_Recombination_Base_func new_PMI_Recombination_Base;
```

The optional function `PMI_Recombination_ElectricField()` is recognized as well, as in the case of the standard interface.

Example: Auger Recombination

See [Standard C++ Interface on page 981](#) and [Simplified C++ Interface on page 985](#).

Avalanche Generation Model

The generation rate due to impact ionization can be expressed as:

$$G^{\parallel} = \alpha_n n v_n + \alpha_p p v_p \quad (1090)$$

where α_n and α_p are the ionization coefficients for electrons and holes, respectively (compare with [Eq. 360, p. 374](#)). The PMI in Sentaurus Device allows you to redefine the calculation of α_n and α_p .

Dependencies

The ionization coefficients α_n and α_p may depend on the following variables:

F	Driving force [Vcm^{-1}]
t	Lattice temperature [K]
bg	Band gap [eV]
ct	Carrier temperature [K]
currentwoMob[3]	Current without mobility [cm^{-4}AVs]

41: Physical Model Interface

Avalanche Generation Model

The parameter `ct` represents the electron temperature during the calculation of α_n and the hole temperature during the calculation of α_p .

The parameter `currentWoMob` can be used to compute anisotropic avalanche generation. Only the first d components of the vector `currentWoMob` are defined, where d is equal to the dimension of the problem. It is recommended that only the direction of the vector `currentWoMob` is taken into account, but not its magnitude.

The PMI model must compute the following results:

<code>alpha</code>	Ionization coefficient [cm^{-1}]
--------------------	---

In the case of the standard interface, the following derivatives must be computed as well:

<code>dalphadF</code>	Derivative of <code>alpha</code> with respect to <code>F</code> [V^{-1}]
<code>dalphadt</code>	Derivative of <code>alpha</code> with respect to <code>t</code> [$\text{cm}^{-1}\text{K}^{-1}$]
<code>dalphadbq</code>	Derivative of <code>alpha</code> with respect to <code>bq</code> [$\text{cm}^{-1}\text{eV}^{-1}$]
<code>dalphadct</code>	Derivative of <code>alpha</code> with respect to <code>ct</code> [$\text{cm}^{-1}\text{K}^{-1}$]
<code>dalphadcurrentWoMob[3]</code>	Derivative of <code>alpha</code> with respect to <code>currentWoMob</code> [$\text{cm}^3\text{A}^{-1}\text{V}^{-1}\text{s}^{-1}$]

Only the first d components of the vector `dalphadcurrentWoMob` need to be computed.

Standard C++ Interface

Different driving forces for avalanche generation can be selected in the command file. The enumeration type `PMI_AvalancheDrivingForce`, defined in `PMIModels.h`, is used to reflect your selection:

```
enum PMI_AvalancheDrivingForce {
    PMI_AvalancheElectricField,
    PMI_AvalancheParallelElectricField,
    PMI_AvalancheGradQuasiFermi,
    PMI_AvalancheCarrierTemperatureCanali
};
```

The following base class is declared in the file `PMIModels.h`:

```
class PMI_Avalanche : public PMI_Vertex_Interface {

private:
    const PMI_AvalancheDrivingForce drivingForce;

public:
    PMI_Avalanche (const PMI_Environment& env,
                   const PMI_AvalancheDrivingForce force);
    virtual ~PMI_Avalanche ();

    PMI_AvalancheDrivingForce AvalancheDrivingForce () const
    { return drivingForce; }

    virtual void Compute_alpha
    (const double F, const double t, const double bg,
     const double ct, const double currentWoMob[3], double& alpha) = 0;

    virtual void Compute_dalphadF
    (const double F, const double t, const double bg,
     const double ct, const double currentWoMob[3], double& dalphadF) = 0;

    virtual void Compute_dalphadt
    (const double F, const double t, const double bg,
     const double ct, const double currentWoMob[3], double& dalphadt) = 0;

    virtual void Compute_dalphadbg
    (const double F, const double t, const double bg,
     const double ct, const double currentWoMob[3], double& dalphadbg) = 0;

    virtual void Compute_dalphadct
    (const double F, const double t, const double bg,
     const double ct, const double currentWoMob[3], double& dalphadct) = 0;

    virtual void Compute_dalphadcurrentWoMob
    (const double F, const double t, const double bg,
     const double ct, const double currentWoMob[3],
     double dalphadcurrentWoMob[3]) = 0;
};
```

Two virtual constructors are required for the calculation of the ionization coefficients α_n and α_p :

```
typedef PMI_Avalanche* new_PMI_Avalanche_func
(const PMI_Environment& env, const PMI_AvalancheDrivingForce force);
extern "C" new_PMI_Avalanche_func new_PMI_e_Avalanche;
extern "C" new_PMI_Avalanche_func new_PMI_h_Avalanche;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_Avalanche_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float F;           // driving force
        pmi_float t;           // lattice temperature
        pmi_float bg;          // band gap
        pmi_float ct;          // carrier temperature
        pmi_float currentWoMob[3]; // current density (without mobility)
    };

    class Output {
    public:
        pmi_float alpha;        // ionization coefficient
    };

    PMI_Avalanche_Base (const PMI_Environment& env,
                        const PMI_AvalancheDrivingForce force);
    virtual ~PMI_Avalanche_Base ();

    PMI_AvalancheDrivingForce AvalancheDrivingForce () const;

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_Avalanche_Base* new_PMI_Avalanche_Base_func
    (const PMI_Environment& env, const PMI_AvalancheDrivingForce force);
extern "C" new_PMI_Avalanche_Base_func new_PMI_e_Avalanche_Base;
extern "C" new_PMI_Avalanche_Base_func new_PMI_h_Avalanche_Base;
```


Example: Okuto Model

Okuto and Crowell propose the following expression for the ionization coefficient α :

$$\alpha(F) = a[1 + c(T - T_0)]F \exp\left[-\left(\frac{b[1 + d(T - T_0)]}{F}\right)^2\right] \quad (1091)$$

This built-in model is discussed in [Okuto–Crowell Model on page 379](#) and its implementation as a PMI model is:

```
#include "PMIModels.h"

class Okuto_Avalanche : public PMI_Avalanche {

protected:
    const double T0;
    double a, b, c, d;

public:
    Okuto_Avalanche (const PMI_Environment& env,
                    const PMI_AvalancheDrivingForce force);

    ~Okuto_Avalanche ();

    void Compute_alpha
        (const double F, const double t, const double bg,
         const double ct, const double currentWoMob[3], double& alpha);

    void Compute_dalphadF
        (const double F, const double t, const double bg,
         const double ct, const double currentWoMob[3], double& dalphadF);

    void Compute_dalphadt
        (const double F, const double t, const double bg,
         const double ct, const double currentWoMob[3], double& dalphadt);

    void Compute_dalphadbg
        (const double F, const double t, const double bg,
         const double ct, const double currentWoMob[3], double& dalphadbg);

    void Compute_dalphadct
        (const double F, const double t, const double bg,
         const double ct, const double currentWoMob[3], double& dalphadct);

    void Compute_dalphadcurrentWoMob
        (const double F, const double t, const double bg,
         const double ct, const double currentWoMob[3], double
```

41: Physical Model Interface

Avalanche Generation Model

```
dalphadcurrentWoMob[3]);
};

Okuto_Avalanche::
Okuto_Avalanche (const PMI_Environment& env,
                  const PMI_AvalancheDrivingForce force) :
    PMI_Avalanche (env, force),
    T0 (300.0)
{
}

Okuto_Avalanche::
~Okuto_Avalanche ()
{
}

void Okuto_Avalanche::
Compute_alpha (const double F, const double t, const double bg,
               const double ct, const double currentWoMob[3], double& alpha)
{ const double aa = a * (1.0 + c * (t - T0));
  const double bb = b * (1.0 + d * (t - T0)) / F;
  alpha = aa * F * exp (-bb*bb);
}

void Okuto_Avalanche::
Compute_dalphadF (const double F, const double t, const double bg,
                  const double ct, const double currentWoMob[3], double&
dalphadF)
{ const double aa = a * (1.0 + c * (t - T0));
  const double bb = b * (1.0 + d * (t - T0)) / F;
  const double alpha = aa * F * exp (-bb*bb);
  dalphadF = (alpha / F) * (1.0 + 2.0*bb*bb);
}

void Okuto_Avalanche::
Compute_dalphadt (const double F, const double t, const double bg,
                  const double ct, const double currentWoMob[3], double&
dalphadt)
{ const double aa = a * (1.0 + c * (t - T0));
  const double bb = b * (1.0 + d * (t - T0)) / F;
  const double tmp = F * exp (-bb*bb);
  dalphadt = tmp * (a * c - 2.0 * aa * bb * b * d / F);
}

void Okuto_Avalanche::
Compute_dalphadbg (const double F, const double t, const double bg,
                   const double ct, const double currentWoMob[3], double&
dalphadbg)
```

```

{ dalphadbg = 0.0;
}

void Okuto_Avalanche::
Compute_dalphadct (const double F, const double t, const double bg,
                   const double ct, const double currentWoMob[3], double&
dalphadct)
{ dalphadct = 0.0;
}

void Okuto_Avalanche::
Compute_dalphadcurrentWoMob (const double F, const double t, const double bg,
                             const double ct, const double currentWoMob[3],
                             double dalphadcurrentWoMob[3])
{ const int dim = ReadDimension ();
  for (int k = 0; k < dim; k++) {
    dalphadcurrentWoMob [k] = 0.0;
  }
}

class Okuto_e_Avalanche : public Okuto_Avalanche {

public:
  Okuto_e_Avalanche (const PMI_Environment& env,
                    const PMI_AvalancheDrivingForce force);

  ~Okuto_e_Avalanche () {}
};

Okuto_e_Avalanche::
Okuto_e_Avalanche (const PMI_Environment& env,
                  const PMI_AvalancheDrivingForce force) :
  Okuto_Avalanche (env, force)
{ // default values
  a = InitParameter ("a_e", 0.426);
  b = InitParameter ("b_e", 4.81e5);
  c = InitParameter ("c_e", 3.05e-4);
  d = InitParameter ("d_e", 6.86e-4);
}

class Okuto_h_Avalanche : public Okuto_Avalanche {

public:
  Okuto_h_Avalanche (const PMI_Environment& env,
                    const PMI_AvalancheDrivingForce force);

  ~Okuto_h_Avalanche () {}
};

```

```

Okuto_h_Avalanche::
Okuto_h_Avalanche (const PMI_Environment& env,
                   const PMI_AvalancheDrivingForce force) :
    Okuto_Avalanche (env, force)
{ // default values
  a = InitParameter ("a_h", 0.243);
  b = InitParameter ("b_h", 6.53e+5);
  c = InitParameter ("c_h", 5.35e-4);
  d = InitParameter ("d_h", 5.67e-4);
}

extern "C"
PMI_Avalanche* new_PMI_e_Avalanche
  (const PMI_Environment& env, const PMI_AvalancheDrivingForce force)
{ return new Okuto_e_Avalanche (env, force);
}

extern "C"
PMI_Avalanche* new_PMI_h_Avalanche
  (const PMI_Environment& env, const PMI_AvalancheDrivingForce force)
{ return new Okuto_h_Avalanche (env, force);
}

```

Mobility Models

Sentaurus Device supports three types of PMI mobility models:

- Doping-dependent mobility
- Mobility degradation at interfaces
- High-field saturation

PMI and built-in models can be used simultaneously. See [Chapter 15 on page 301](#) for more information about how the contributions of the different models are combined. PMI mobility models support anisotropic calculations and can be evaluated along different crystallographic axes. The enumeration type:

```

enum PMI_AnisotropyType {
    PMI_Isotropic,
    PMI_Anisotropic
};

```

determines the axis. The default is isotropic mobility. If anisotropic mobilities are activated in the command file, the PMI mobility classes are also instantiated in the anisotropic direction.

Doping-dependent Mobility

A doping-dependent PMI model must account for both the constant mobility and doping-dependent mobility models discussed in [Mobility due to Phonon Scattering on page 302](#) and [Doping-dependent Mobility Degradation on page 302](#).

Dependencies

The constant mobility and doping-dependent mobility μ_{dop} may depend on the following variables:

t	Lattice temperature [K]
n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]

The PMI model must compute the following results:

m	Mobility μ_{dop} [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$]
---	---

In the case of the standard interface, the following derivatives must be computed as well:

dmdn	Derivative of μ_{dop} with respect to n [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]
dmdp	Derivative of μ_{dop} with respect to p [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]
dmdt	Derivative of μ_{dop} with respect to t [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}\text{K}^{-1}$]

In most cases, it is not necessary to compute the derivatives with respect to the dopant concentrations.

However, to model random dopant fluctuations (see [Random Dopant Fluctuations on page 587](#)), the PMI model must override the functions that compute the following values:

dmdNa	Derivative of μ_{dop} with respect to the acceptor concentration [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]
dmdNd	Derivative of μ_{dop} with respect to the donor concentration [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_DopingDepMobility : public PMI_Vertex_Interface {

private:
    const PMI_AnisotropyType anisoType;

public:
    PMI_DopingDepMobility (const PMI_Environment& env,
                          const PMI_AnisotropyType anisotype);
    virtual ~PMI_DopingDepMobility ();

    PMI_AnisotropyType AnisotropyType () const { return anisoType; }

    virtual void Compute_m
        (const double n, const double p,
         const double t, double& m) = 0;

    virtual void Compute_dmdn
        (const double n, const double p,
         const double t, double& dmdn) = 0;

    virtual void Compute_dmdp
        (const double n, const double p,
         const double t, double& dmdp) = 0;

    virtual void Compute_dmdt
        (const double n, const double p,
         const double t, double& dmdt) = 0;

    virtual void Compute_dmdNa
        (const double n, const double p,
         const double t, double& dmdNa);

    virtual void Compute_dmdNd
        (const double n, const double p,
         const double t, double& dmdNd);

};
```

Two virtual constructors are required for electron and hole mobilities:

```
typedef PMI_DopingDepMobility* new_PMI_DopingDepMobility_func
    (const PMI_Environment& env, const PMI_AnisotropyType anisotype);
extern "C" new_PMI_DopingDepMobility_func new_PMI_DopingDep_e_Mobility;
extern "C" new_PMI_DopingDepMobility_func new_PMI_DopingDep_h_Mobility;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_DopingDepMobility_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float n;           // electron density
        pmi_float p;           // hole density
        pmi_float t;           // lattice temperature
        pmi_float acceptor;    // total acceptor concentration
        pmi_float donor;       // total donor concentration
    };

    class Output {
    public:
        pmi_float m;           // doping-dependent mobility
    };

    PMI_DopingDepMobility_Base (const PMI_Environment& env,
                                const PMI_AnisotropyType anisotype);
    virtual ~PMI_DopingDepMobility_Base ();

    PMI_AnisotropyType AnisotropyType () const;

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_DopingDepMobility_Base* new_PMI_DopingDepMobility_Base_func
    (const PMI_Environment& env, const PMI_AnisotropyType anisotype);
extern "C" new_PMI_DopingDepMobility_Base_func
    new_PMI_DopingDep_e_Mobility_Base;
extern "C" new_PMI_DopingDepMobility_Base_func
    new_PMI_DopingDep_h_Mobility_Base;
```

Example: Masetti Model

The built-in Masetti model (see [Masetti Model on page 304](#)) can also be implemented as a PMI model:

```
#include "PMIModels.h"

class Masetti_DopingDepMobility : public PMI_DopingDepMobility {
protected:
    const double T0;
    double mumax, Exponent, mumin1, mumin2, mu1, Pc, Cr, Cs, alpha, beta;

public:
    Masetti_DopingDepMobility (const PMI_Environment& env,
                              const PMI_AnisotropyType anisotype);
    ~Masetti_DopingDepMobility () {}

    void Compute_m
        (const double n, const double p,
         const double t, double& m);

    void Compute_dmdn
        (const double n, const double p,
         const double t, double& dmdn);

    void Compute_dmdp
        (const double n, const double p,
         const double t, double& dmdp);

    void Compute_dmdt
        (const double n, const double p,
         const double t, double& dmdt);
};

Masetti_DopingDepMobility::
Masetti_DopingDepMobility (const PMI_Environment& env,
                           const PMI_AnisotropyType anisotype) :
    PMI_DopingDepMobility (env, anisotype),
    T0 (300.0)
{
}

void Masetti_DopingDepMobility::
Compute_m (const double n, const double p,
           const double t, double& m)
{ const double mu_const = mumax * pow (t/T0, -Exponent);
```



```

const double Ni = Max (ReadDoping (PMI_Donor) +
                      ReadDoping (PMI_Acceptor), 1.0);
m = mumin1 * exp (-Pc / Ni) +
    (mu_const - mumin2) / (1.0 + pow (Ni / Cr, alpha)) -
    mu1 / (1.0 + pow (Cs / Ni, beta));
}

void Masetti_DopingDepMobility::
Compute_dmdn (const double n, const double p,
              const double t, double& dmdn)
{ dmdn = 0.0;
}

void Masetti_DopingDepMobility::
Compute_dmdp (const double n, const double p,
              const double t, double& dmdp)
{ dmdp = 0.0;
}

void Masetti_DopingDepMobility::
Compute_dmdt (const double n, const double p,
              const double t, double& dmdt)
{ const double Ni = Max (ReadDoping (PMI_Donor) +
                      ReadDoping (PMI_Acceptor), 1.0);
  dmdt = mumax * (-Exponent/T0) * pow (t/T0, -Exponent - 1.0) /
        (1.0 + pow (Ni / Cr, alpha));
}

class Masetti_e_DopingDepMobility : public Masetti_DopingDepMobility {
public:
  Masetti_e_DopingDepMobility (const PMI_Environment& env,
                              const PMI_AnisotropyType anisotype);
  ~Masetti_e_DopingDepMobility () {}
};

Masetti_e_DopingDepMobility::
Masetti_e_DopingDepMobility (const PMI_Environment& env,
                              const PMI_AnisotropyType anisotype) :
  Masetti_DopingDepMobility (env, anisotype)

{ // default values
  mumax = InitParameter ("mumax_e", 1417.0);
  Exponent = InitParameter ("Exponent_e", 2.5);
  mumin1 = InitParameter ("mumin1_e", 52.2);
  mumin2 = InitParameter ("mumin2_e", 52.2);
  mu1 = InitParameter ("mu1_e", 43.4);
  Pc = InitParameter ("Pc_e", 0.0);
  Cr = InitParameter ("Cr_e", 9.68e16);
}

```

41: Physical Model Interface

Doping-dependent Mobility

```
Cs = InitParameter ("Cs_e", 3.43e20);
alpha = InitParameter ("alpha_e", 0.680);
beta = InitParameter ("beta_e", 2.0);
}

class Masetti_h_DopingDepMobility : public Masetti_DopingDepMobility {
public:
    Masetti_h_DopingDepMobility (const PMI_Environment& env,
                                const PMI_AnisotropyType anisotype);
    ~Masetti_h_DopingDepMobility () {}
};

Masetti_h_DopingDepMobility::
Masetti_h_DopingDepMobility (const PMI_Environment& env,
                             const PMI_AnisotropyType anisotype) :
    Masetti_DopingDepMobility (env, anisotype)

{ // default values
    mumax = InitParameter ("mumax_h", 470.5);
    Exponent = InitParameter ("Exponent_h", 2.2);
    mumin1 = InitParameter ("mumin1_h", 44.9);
    mumin2 = InitParameter ("mumin2_h", 0.0);
    mu1 = InitParameter ("mu1_h", 29.0);
    Pc = InitParameter ("Pc_h", 9.23e16);
    Cr = InitParameter ("Cr_h", 2.23e17);
    Cs = InitParameter ("Cs_h", 6.10e20);
    alpha = InitParameter ("alpha_h", 0.719);
    beta = InitParameter ("beta_h", 2.0);
}

extern "C"
PMI_DopingDepMobility* new_PMI_DopingDep_e_Mobility
    (const PMI_Environment& env, const PMI_AnisotropyType anisotype)
{ return new Masetti_e_DopingDepMobility (env, anisotype);
}

extern "C"
PMI_DopingDepMobility* new_PMI_DopingDep_h_Mobility
    (const PMI_Environment& env, const PMI_AnisotropyType anisotype)
{ return new Masetti_h_DopingDepMobility (env, anisotype);
}
```

Multistate Configuration–dependent Bulk Mobility

This PMI allows you to implement multistate configuration (MSC)–dependent bulk mobility models.

Command File

To activate a PMI of this type, as an option to `eMobility`, `hMobility`, or `Mobility` in the `Physics` section, specify:

```
DopingDependence(  
  PMIModel (  
    Name = <string>  
    MSConfig = <string>  
    Index = <int>  
    String = <string>  
  )  
)
```

The options of `PMIModel` are described in [Command File on page 1088](#).

Dependencies

The mobility may depend on the variables:

<code>n</code>	Electron density [cm ⁻³]
<code>p</code>	Hole density [cm ⁻³]
<code>T</code>	Lattice temperature [K]
<code>eT</code>	Electron temperature [K]
<code>hT</code>	Hole temperature [K]
<code>s</code>	Multistate configuration occupation probabilities [1]

The model must compute the following quantities:

<code>val</code>	Mobility μ_{dop} [cm ² V ⁻¹ s ⁻¹]
------------------	--

41: Physical Model Interface

Multistate Configuration-dependent Bulk Mobility

In the case of the standard interface, the following derivatives must be computed as well:

dval_dn	Derivative with respect to electron density [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]
dval_dp	Derivative with respect to hole density [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]
dval_dT	Derivative with respect to lattice temperature [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}\text{K}^{-1}$]
dval_deT	Derivative with respect to electron temperature [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}\text{K}^{-1}$]
dval_dhT	Derivative with respect to hole temperature [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}\text{K}^{-1}$]
dval_ds	Derivative with respect to multistate configuration occupation probabilities [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$]

Standard C++ Interface

The PMI offers a base class that presents the following interface:

```
class PMI_MSC_Mobility : public PMI_MSC_Vertex_Interface
{
public:
    PMI_MSC_Mobility (const PMI_Environment& env,
        const std::string& msconfig_name,
        const int model_index,
        const std::string& model_string,
        const PMI_AnisotropyType aniso);
    // otherwise, see Standard C++ Interface on page 1089
};
```

Apart from the name of the base class and the constructor, the explanations in [Standard C++ Interface on page 1089](#) apply here as well.

The following virtual constructor must be implemented:

```
typedef PMI_MSC_Mobility* new_PMI_MSC_Mobility_func
    (const PMI_Environment& env, const std::string& msconfig_name,
    int model_index, const std::string& model_string,
    const PMI_AnisotropyType anisotype);
extern "C" new_PMI_MSC_Mobility_func new_PMI_MSC_Mobility;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_MSC_Mobility_Base : public PMI_MSC_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        const pmi_float& n () const;    // electron density
        const pmi_float& p () const;    // hole density
        const pmi_float& T () const;    // lattice temperature
        const pmi_float& eT () const;   // electron temperature
        const pmi_float& hT () const;   // hole temperature
        const pmi_float& s (size_t ind) const; // phase fraction
    };

    class Output {
    public:
        pmi_float& val ();    // mobility
    };

    PMI_MSC_Mobility_Base (const PMI_Environment& env,
                           const std::string& msconfig_name,
                           const int model_index,
                           const std::string& model_string,
                           const PMI_AnisotropyType anisotype);
    virtual ~PMI_MSC_Mobility_Base ();

    PMI_AnisotropyType AnisotropyType () const;

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_MSC_Mobility_Base* new_PMI_MSC_Mobility_Base_func
(const PMI_Environment& env, const std::string& msconfig_name,
 const int model_index, const std::string& model_string,
 const PMI_AnisotropyType anisotype);
extern "C" new_PMI_MSC_Mobility_Base_func new_PMI_MSC_Mobility_Base;
```

Mobility Degradation at Interfaces

Sentaurus Device uses Mathiessen's rule:

$$\frac{1}{\mu} = \frac{1}{\mu_{\text{dop}}} + \frac{1}{\mu_{\text{enormal}}} \quad (1092)$$

to combine the constant and doping-dependent mobility μ_{dop} , and the surface contribution μ_{enormal} (see [Mobility Degradation at Interfaces on page 315](#)). To express no mobility degradation, for example, in the bulk of a device, it is necessary to set $\mu_{\text{enormal}} = \infty$. To avoid numeric difficulties, the PMI requires the calculation of the inverse mobility $1/\mu_{\text{enormal}}$ instead of μ_{enormal} .

As an additional precaution, Sentaurus Device does not evaluate the PMI model if the normal electric field $F_{\perp}$ is less than EnormMinimum, where EnormMinimum is a parameter that can be specified in the PMI_model_name section of the parameter file. By default, EnormMinimum = 1 V/cm is used.

Dependencies

The mobility degradation at interfaces may depend on the following variables:

dist	Distance to nearest interface [cm]
pot	Electrostatic potential [V]
enorm	Normal electric field [Vcm^{-1}]
t	Lattice temperature [K]
n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]
ct	Carrier temperature [K]

NOTE If Sentaurus Device cannot determine the distance to the nearest interface, the value of $\text{dist} = 10^{10}$ is used.

NOTE The carrier temperature ct represents the electron temperature during the evaluation of the model for electrons, and the hole temperature during the evaluation of the model for holes. The parameter ct is only defined for hydrodynamic simulations. Otherwise, the value of $\text{ct} = 0$ is used.

The PMI model must compute the following results:

`muinv` Inverse of mobility $1/\mu_{\text{enormal}}$ [cm^{-2}Vs]

In the case of the standard interface, the following derivatives must be computed as well:

`dmuinvdpot` Derivative of $1/\mu_{\text{enormal}}$ with respect to `pot` [cm^{-2}s]

`dmuinvdenorm` Derivative of $1/\mu_{\text{enormal}}$ with respect to `enorm` [cm^{-1}s]

`dmuinvdn` Derivative of $1/\mu_{\text{enormal}}$ with respect to `n` [cmVs]

`dmuinvdp` Derivative of $1/\mu_{\text{enormal}}$ with respect to `p` [cmVs]

`dmuinvdt` Derivative of $1/\mu_{\text{enormal}}$ with respect to `t` [$\text{cm}^{-2}\text{VsK}^{-1}$]

`dmuinvdct` Derivative of $1/\mu_{\text{enormal}}$ with respect to `ct` [$\text{cm}^{-2}\text{VsK}^{-1}$]

In most cases, it is not necessary to compute the derivatives with respect to the dopant concentrations. However, to model random dopant fluctuations (see [Random Dopant Fluctuations on page 587](#)), the PMI model must override the functions that compute the following values:

`dmuinvdNa` Derivative of $1/\mu_{\text{enormal}}$ with respect to the acceptor concentration [cmVs]

`dmuinvdNd` Derivative of $1/\mu_{\text{enormal}}$ with respect to the donor concentration [cmVs]

Standard C++ Interface

The enumeration type `PMI_EnormalType` describes the type of the normal electric field $F_{\perp}$:

```
enum PMI_EnormalType {
    PMI_EnormalToCurrent,
    PMI_EnormalToInterface
};
```

The following base class is declared in the file `PMIModels.h`:

```
class PMI_EnormalMobility : public PMI_Vertex_Interface {

public:
    PMI_EnormalMobility (const PMI_Environment& env,
                        const PMI_EnormalType type,
                        const PMI_AnisotropyType anisotype);

    virtual ~PMI_EnormalMobility ();
```

```
PMI_EnormalType EnormalType () const;
PMI_AnisotropyType AnisotropyType () const;

virtual void Compute_muinv
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& muinv) = 0;

virtual void Compute_dmuinvdpot
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdpot) = 0;

virtual void Compute_dmuinvdenorm
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdenorm) = 0;

virtual void Compute_dmuinvdn
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdn) = 0;

virtual void Compute_dmuinvdp
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdp) = 0;

virtual void Compute_dmuinvdt
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdt) = 0;

virtual void Compute_dmuinvdct
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdct) = 0;

virtual void Compute_dmuinvdNa
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdNa);

virtual void Compute_dmuinvdNd
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdNd);
```



```
};
```

Two virtual constructors are required for electron and hole mobilities:

```
typedef PMI_EnormalMobility* new_PMI_EnormalMobility_func
(const PMI_Environment& env, const PMI_EnormalType type,
 const PMI_AnisotropyType anisotype);
extern "C" new_PMI_EnormalMobility_func new_PMI_Enormal_e_Mobility;
extern "C" new_PMI_EnormalMobility_func new_PMI_Enormal_h_Mobility;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_EnormalMobility_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float dist;        // distance to nearest interface
        pmi_float pot;         // electrostatic potential
        pmi_float enorm;       // normal electric field
        pmi_float n;           // electron density
        pmi_float p;           // hole density
        pmi_float t;           // lattice temperature
        pmi_float ct;          // carrier temperature
        pmi_float acceptor;     // total acceptor concentration
        pmi_float donor;       // total donor concentration
    };

    class Output {
    public:
        pmi_float muinv;       // inverse of mobility degradation
    };

    PMI_EnormalMobility_Base (const PMI_Environment& env,
                              const PMI_EnormalType type,
                              const PMI_AnisotropyType anisotype);
    virtual ~PMI_EnormalMobility_Base ();

    PMI_EnormalType EnormalType () const;
    PMI_AnisotropyType AnisotropyType () const;

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_EnormalMobility_Base* new_PMI_EnormalMobility_Base_func
(const PMI_Environment& env, const PMI_EnormalType type,
const PMI_AnisotropyType anisotype);
extern "C" new_PMI_EnormalMobility_Base_func new_PMI_Enormal_e_Mobility_Base;
extern "C" new_PMI_EnormalMobility_Base_func new_PMI_Enormal_h_Mobility_Base;
```

Example: Lombardi Model

This example illustrates the implementation of a slightly simplified Lombardi model (see [Mobility Degradation at Interfaces on page 315](#)) using the PMI. The contribution due to acoustic phonon-scattering has the form:

$$\mu_{ac} = \frac{B}{F_{\perp}} + \frac{CN_f^{\lambda}}{F_{\perp}^{1/3}(T/T_0)} \quad (1093)$$

where $T_0 = 300$ K.

The contribution due to surface roughness scattering is given by:

$$\mu_{sr} = \left(\frac{F_{\perp}^2}{\delta} + \frac{F_{\perp}^3}{\eta} \right)^{-1} \quad (1094)$$

The mobilities μ_{ac} and μ_{sr} are combined according to Mathiessen's rule with an additional damping factor:

$$\frac{1}{\mu_{enormal}} = e^{\frac{l}{l_{crit}}} \cdot \left(\frac{1}{\mu_{ac}} + \frac{1}{\mu_{sr}} \right) \quad (1095)$$

where l is the distance to the nearest semiconductor–insulator interface point:

```
#include "PMIModels.h"

class Lombardi_EnormalMobility : public PMI_EnormalMobility {

protected:
    const double T0;
    double B, C, lambda, delta, eta, l_crit;

public:
    Lombardi_EnormalMobility (const PMI_Environment& env,
                             const PMI_EnormalType type,
                             const PMI_AnisotropyType anisotype);
```

```

~Lombardi_EnormalMobility ();

void Compute_muinv
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& muinv);

void Compute_dmuinvdpot
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdpot);

void Compute_dmuinvdenorm
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdenorm);

void Compute_dmuinvdn
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdn);

void Compute_dmuinvdp
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdp);

void Compute_dmuinvdt
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdt);

void Compute_dmuinvdct
    (const double dist, const double pot,
     const double enorm, const double n, const double p,
     const double t, const double ct, double& dmuinvdct);
};

Lombardi_EnormalMobility::
Lombardi_EnormalMobility (const PMI_Environment& env,
                          const PMI_EnormalType type,
                          const PMI_AnisotropyType anisotype) :
    PMI_EnormalMobility (env, type, anisotype),
    T0 (300.0)
{
}

```

41: Physical Model Interface

Mobility Degradation at Interfaces

```
Lombardi_EnormalMobility::
~Lombardi_EnormalMobility ()
{
}

void Lombardi_EnormalMobility::
Compute_muinv (const double dist, const double pot,
               const double enorm, const double n, const double p,
               const double t, const double ct, double& muinv)
{ const double Ni = ReadDoping (PMI_Donor) + ReadDoping (PMI_Acceptor);
  const double denom_ac_inv =
    B + pow (enorm, 2.0/3.0) * C * pow (Ni, lambda) * T0 / t;
  const double mu_ac_inv = enorm / denom_ac_inv;
  const double mu_sr_inv = enorm * enorm / delta + pow (enorm, 3.0) / eta;
  const double damping = exp (-dist/l_crit);
  muinv = damping * (mu_ac_inv + mu_sr_inv);
}

void Lombardi_EnormalMobility::
Compute_dmuinvdpot (const double dist, const double pot,
                   const double enorm, const double n, const double p,
                   const double t, const double ct, double& dmuinvdpot)
{ dmuinvdpot = 0.0;
}

void Lombardi_EnormalMobility::
Compute_dmuinvdenorm (const double dist, const double pot,
                     const double enorm, const double n, const double p,
                     const double t, const double ct, double& dmuinvdenorm)
{ const double Ni = ReadDoping (PMI_Donor) + ReadDoping (PMI_Acceptor);
  const double denom_ac_inv =
    B + pow (enorm, 2.0/3.0) * C * pow (Ni, lambda) * T0 / t;
  const double dmu_ac_inv_denorm =
    (2.0 * B + denom_ac_inv) / (3.0 * denom_ac_inv * denom_ac_inv);
  const double mu_sr_inv_denorm =
    2.0 * enorm / delta + 3.0 * enorm * enorm / eta;
  const double damping = exp (-dist/l_crit);
  dmuinvdenorm = damping * (dmu_ac_inv_denorm + mu_sr_inv_denorm);
}

void Lombardi_EnormalMobility::
Compute_dmuinvdn (const double dist, const double pot,
                 const double enorm, const double n, const double p,
                 const double t, const double ct, double& dmuinvdn)
{ dmuinvdn = 0.0;
}

void Lombardi_EnormalMobility::
```

```

Compute_dmuinvdp (const double dist, const double pot,
                  const double enorm, const double n, const double p,
                  const double t, const double ct, double& dmuinvdp)
{ dmuinvdp = 0.0;
}

void Lombardi_EnormalMobility::
Compute_dmuinvdt (const double dist, const double pot,
                  const double enorm, const double n, const double p,
                  const double t, const double ct, double& dmuinvdt)
{ const double Ni = ReadDoping (PMI_Donor) + ReadDoping (PMI_Acceptor);
  const double factor = pow (enorm, 2.0/3.0) * C * pow (Ni, lambda) * T0;
  const double denom_ac_inv = B + factor / t;
  const double dmu_ac_inv_dt =
    enorm * factor / (denom_ac_inv * denom_ac_inv * t * t);
  const double damping = exp (-dist/l_crit);
  dmuinvdt = damping * dmu_ac_inv_dt;
}

void Lombardi_EnormalMobility::
Compute_dmuinvdct (const double dist, const double pot,
                  const double enorm, const double n, const double p,
                  const double t, const double ct, double& dmuinvdct)
{ dmuinvdct = 0.0;
}

class Lombardi_e_EnormalMobility : public Lombardi_EnormalMobility {
public:
  Lombardi_e_EnormalMobility (const PMI_Environment& env,
                              const PMI_EnormalType type,
                              const PMI_AnisotropyType anisotype);

  ~Lombardi_e_EnormalMobility () {}
};

Lombardi_e_EnormalMobility::
Lombardi_e_EnormalMobility (const PMI_Environment& env,
                            const PMI_EnormalType type,
                            const PMI_AnisotropyType anisotype) :
  Lombardi_EnormalMobility (env, type, anisotype)
{ // default values
  B = InitParameter ("B_e", 4.750e7);
  C = InitParameter ("C_e", 580.0);
  lambda = InitParameter ("lambda_e", 0.125);
  delta = InitParameter ("delta_e", 5.82e14);
  eta = InitParameter ("eta_e", 5.82e30);
  l_crit = InitParameter ("l_crit_e", 1.0e-6);
}

```

41: Physical Model Interface

Mobility Degradation at Interfaces

```
class Lombardi_h_EnormalMobility : public Lombardi_EnormalMobility {
public:
    Lombardi_h_EnormalMobility (const PMI_Environment& env,
                                const PMI_EnormalType type,
                                const PMI_AnisotropyType anisotype);

    ~Lombardi_h_EnormalMobility () {}
};

Lombardi_h_EnormalMobility::
Lombardi_h_EnormalMobility (const PMI_Environment& env,
                             const PMI_EnormalType type,
                             const PMI_AnisotropyType anisotype) :
    Lombardi_EnormalMobility (env, type, anisotype)
{ // default values
    B = InitParameter ("B_h", 9.925e6);
    C = InitParameter ("C_h", 2947.0);
    lambda = InitParameter ("lambda_h", 0.0317);
    delta = InitParameter ("delta_h", 2.0546e14);
    eta = InitParameter ("eta_h", 2.0546e30);
    l_crit = InitParameter ("l_crit_h", 1.0e-6);
}

extern "C"
PMI_EnormalMobility* new_PMI_Enormal_e_Mobility
    (const PMI_Environment& env, const PMI_EnormalType type,
     const PMI_AnisotropyType anisotype)
{ return new Lombardi_e_EnormalMobility (env, type, anisotype);
}

extern "C"
PMI_EnormalMobility* new_PMI_Enormal_h_Mobility
    (const PMI_Environment& env, const PMI_EnormalType type,
     const PMI_AnisotropyType anisotype)
{ return new Lombardi_h_EnormalMobility (env, type, anisotype);
}
```

High-Field Saturation Model

The high-field saturation model computes the final mobility μ as a function of the low-field mobility μ_{low} and the driving force F_{hfs} (see [High-Field Saturation on page 342](#)).

Dependencies

The mobility μ computed by a high-field mobility model may depend on the following variables:

pot	Electrostatic potential [V]
t	Lattice temperature [K]
n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]
ct	Carrier temperature [K]
mulow	Low-field mobility μ_{low} [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$]
F	Driving force [Vcm^{-1}]

NOTE The carrier temperature `ct` represents the electron temperature during the evaluation of the model for electrons, and the hole temperature during the evaluation of the model for holes. The parameter `ct` is only defined for hydrodynamic simulations. Otherwise, the value of `ct` = 0 is used.

The PMI model must compute the following results:

mu	Mobility μ [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$]
----	--

In the case of the standard interface, the following derivatives must be computed as well:

dmudp	Derivative of μ with respect to <code>pot</code> [$\text{cm}^2\text{V}^{-2}\text{s}^{-1}$]
dmudn	Derivative of μ with respect to <code>n</code> [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]
dmudp	Derivative of μ with respect to <code>p</code> [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]
dmudt	Derivative of μ with respect to <code>t</code> [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}\text{K}^{-1}$]

41: Physical Model Interface

High-Field Saturation Model

<code>dmudct</code>	Derivative of μ with respect to ct [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}\text{K}^{-1}$]
<code>dmudmulow</code>	Derivative of μ with respect to μ_{low} (1)
<code>dmudF</code>	Derivative of μ with respect to F [$\text{cm}^3\text{V}^{-2}\text{s}^{-1}$]

In most cases, it is not necessary to compute the derivatives with respect to the dopant concentrations. However, to model random dopant fluctuations (see [Random Dopant Fluctuations on page 587](#)), the PMI model must override the functions that compute the following values:

<code>dmudNa</code>	Derivative of μ with respect to the acceptor concentration [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]
<code>dmudNd</code>	Derivative of μ with respect to the donor concentration [$\text{cm}^5\text{V}^{-1}\text{s}^{-1}$]

Standard C++ Interface

The enumeration type `PMI_HighFieldDrivingForce` describes the driving force as specified in the command file:

```
enum PMI_HighFieldDrivingForce {
    PMI_HighFieldParallelElectricField,
    PMI_HighFieldParallelToInterfaceElectricField,
    PMI_HighFieldGradQuasiFermi
};
```

The following base class is declared in the file `PMIModels.h`:

```
class PMI_HighFieldMobility : public PMI_Vertex_Interface {

private:
    const PMI_HighFieldDrivingForce drivingForce;
    const PMI_AnisotropyType anisoType;

public:
    PMI_HighFieldMobility (const PMI_Environment& env,
                          const PMI_HighFieldDrivingForce force,
                          const PMI_AnisotropyType anisotype);

    virtual ~PMI_HighFieldMobility ();

    PMI_HighFieldDrivingForce HighFieldDrivingForce () const;
    PMI_AnisotropyType AnisotropyType () const;

    virtual void Compute_mu
        (const double pot, const double n,
```



```
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& mu) = 0;  
  
virtual void Compute_dmudpot  
    (const double pot, const double n,  
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& dmudpot) = 0;  
  
virtual void Compute_dmudn  
    (const double pot, const double n,  
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& dmudn) = 0;  
  
virtual void Compute_dmudp  
    (const double pot, const double n,  
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& dmudp) = 0;  
  
virtual void Compute_dmudt  
    (const double pot, const double n,  
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& dmudt) = 0;  
  
virtual void Compute_dmudct  
    (const double pot, const double n,  
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& dmudct) = 0;  
  
virtual void Compute_dmudmulow  
    (const double pot, const double n,  
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& dmudmulow) = 0;  
  
virtual void Compute_dmudF  
    (const double pot, const double n,  
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& dmudF) = 0;  
  
virtual void Compute_dmudNa  
    (const double pot, const double n,  
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& dmudNa);  
  
virtual void Compute_dmudNd  
    (const double pot, const double n,  
    const double p, const double t, const double ct,  
    const double mulow, const double F, double& dmudNd);  
};
```

41: Physical Model Interface

High-Field Saturation Model

Two virtual constructors are required for electron and hole mobilities:

```
typedef PMI_HighFieldMobility* new_PMI_HighFieldMobility_func
(const PMI_Environment& env, const PMI_HighFieldDrivingForce force,
 const PMI_AnisotropyType anisotype);
extern "C" new_PMI_HighFieldMobility_func new_PMI_HighField_e_Mobility;
extern "C" new_PMI_HighFieldMobility_func new_PMI_HighField_h_Mobility;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_HighFieldMobility_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float pot;           // electrostatic potential
        pmi_float n;             // electron density
        pmi_float p;             // hole density
        pmi_float t;             // lattice temperature
        pmi_float ct;            // carrier temperature
        pmi_float mulow;         // low field mobility
        pmi_float F;             // driving force
        pmi_float acceptor;      // total acceptor concentration
        pmi_float donor;         // total donor concentration
    };

    class Output {
    public:
        pmi_float mu;            // mobility
    };

    PMI_HighFieldMobility_Base (const PMI_Environment& env,
                                const PMI_HighFieldDrivingForce force,
                                const PMI_AnisotropyType anisotype);
    virtual ~PMI_HighFieldMobility_Base ();

    PMI_HighFieldDrivingForce HighFieldDrivingForce () const;
    PMI_AnisotropyType AnisotropyType () const;

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_HighFieldMobility_Base* new_PMI_HighFieldMobility_Base_func
(const PMI_Environment& env, const PMI_HighFieldDrivingForce force,
const PMI_AnisotropyType anisotype);
extern "C" new_PMI_HighFieldMobility_Base_func
new_PMI_HighField_e_Mobility_Base;
extern "C" new_PMI_HighFieldMobility_Base_func
new_PMI_HighField_h_Mobility_Base;
```

Example: Canali Model

This example presents the PMI implementation of the Canali model:

$$\mu = \frac{\mu_{low}}{\left[1 + \left(\frac{\mu_{low} F_{hfs}}{v_{sat}}\right)^{\beta}\right]^{1/\beta}} \quad (1096)$$

where:

$$\beta = \beta_0 \left(\frac{T}{T_0}\right)^{\beta_{exp}} \quad (1097)$$

and:

$$v_{sat} = v_{sat0} \left(\frac{T}{T_0}\right)^{v_{sat,exp}} \quad (1098)$$

The built-in Canali model is discussed in [Extended Canali Model on page 343](#).

```
#include "PMIModels.h"

class Canali_HighFieldMobility : public PMI_HighFieldMobility {

private:
    double beta, vsat, Fabs, val, valb, valb1, valb1b;
    void Compute_internal (const double t, const double mulow,
                           const double F);

protected:
    const double T0;
    double beta0, betaexp, vsat0, vsatexp;

public:
    Canali_HighFieldMobility (const PMI_Environment& env,
                              const PMI_HighFieldDrivingForce force,
```

41: Physical Model Interface

High-Field Saturation Model

```
const PMI_AnisotropyType anisotype);

~Canali_HighFieldMobility ();

void Compute_mu
(const double pot, const double n,
 const double p, const double t, const double ct,
 const double mulow, const double F,
 double& mu);

void Compute_dmudpot
(const double pot, const double n,
 const double p, const double t, const double ct,
 const double mulow, const double F, double& dmudpot);

void Compute_dmudn
(const double pot, const double n,
 const double p, const double t, const double ct,
 const double mulow, const double F, double& dmudn);

void Compute_dmudp
(const double pot, const double n,
 const double p, const double t, const double ct,
 const double mulow, const double F, double& dmudp);

void Compute_dmudt
(const double pot, const double n,
 const double p, const double t, const double ct,
 const double mulow, const double F, double& dmudt);

void Compute_dmudct
(const double pot, const double n,
 const double p, const double t, const double ct,
 const double mulow, const double F, double& dmudct);

void Compute_dmudmulow
(const double pot, const double n,
 const double p, const double t, const double ct,
 const double mulow, const double F, double& dmudmulow);

void Compute_dmudF
(const double pot, const double n,
 const double p, const double t, const double ct,
 const double mulow, const double F, double& dmudF);
};

void Canali_HighFieldMobility::
Compute_internal (const double t, const double mulow, const double F)
```

```

{ beta = beta0 * pow (t/T0, betaexp);
  vsat = vsat0 * pow (t/T0, -vsatexp);
  Fabs = fabs (F);
  val = mulow * Fabs / vsat;
  valb = pow (val, beta);
  valb1 = 1.0 + valb;
  valb11b = pow (valb1, 1.0/beta);
}

Canali_HighFieldMobility::
Canali_HighFieldMobility (const PMI_Environment& env,
                          const PMI_HighFieldDrivingForce force,
                          const PMI_AnisotropyType anisotype) :
  PMI_HighFieldMobility (env, force, anisotype),
  T0 (300.0)
{
}

Canali_HighFieldMobility::
~Canali_HighFieldMobility ()
{
}

void Canali_HighFieldMobility::
Compute_mu (const double pot, const double n,
            const double p, const double t, const double ct,
            const double mulow, const double F, double& mu)
{ Compute_internal (t, mulow, F);
  mu = mulow / valb11b;
}

void Canali_HighFieldMobility::
Compute_dmudpot (const double pot, const double n,
                const double p, const double t, const double ct,
                const double mulow, const double F, double& dmudpot)
{ dmudpot = 0.0;
}

void Canali_HighFieldMobility::
Compute_dmudn (const double pot, const double n,
              const double p, const double t, const double ct,
              const double mulow, const double F, double& dmudn)
{ dmudn = 0.0;
}

void Canali_HighFieldMobility::
Compute_dmudp (const double pot, const double n,

```

41: Physical Model Interface

High-Field Saturation Model

```
        const double p, const double t, const double ct,
        const double mulow, const double F, double& dmudp)
{ dmudp = 0.0;
}

void Canali_HighFieldMobility::
Compute_dmudt (const double pot, const double n,
               const double p, const double t, const double ct,
               const double mulow, const double F, double& dmudt)
{ Compute_internal (t, mulow, F);
  const double mu = mulow / valb11b;
  const double dmudbeta = mu * (log (valb1) / (beta*beta) -
                                valb * log (val) / (beta * valb1));
  const double dmudvsat = (mu * valb) / (valb1 * vsat);
  const double dbetadt = beta * betaexp / t;
  const double dvsatdt = -vsat * vsatexp / t;
  dmudt = dmudbeta * dbetadt + dmudvsat * dvsatdt;
}

void Canali_HighFieldMobility::
Compute_dmudct (const double pot, const double n,
               const double p, const double t, const double ct,
               const double mulow, const double F, double& dmudct)
{ dmudct = 0.0;
}

void Canali_HighFieldMobility::
Compute_dmudmulow (const double pot, const double n,
                  const double p, const double t, const double ct,
                  const double mulow, const double F, double& dmudmulow)
{ Compute_internal (t, mulow, F);
  dmudmulow = 1.0 / (valb1 * valb11b);
}

void Canali_HighFieldMobility::
Compute_dmudF (const double pot, const double n,
               const double p, const double t, const double ct,
               const double mulow, const double F, double& dmudF)
{ Compute_internal (t, mulow, F);
  const double mu = mulow / valb11b;
  const double signF = (F >= 0.0) ? 1.0 : -1.0;
  dmudF = -mu * pow (mulow/vsat, beta) * pow (Fabs, beta-1.0) *
          signF / valb1;
}

class Canali_e_HighFieldMobility : public Canali_HighFieldMobility {
public:
  Canali_e_HighFieldMobility (const PMI_Environment& env,
```

```

        const PMI_HighFieldDrivingForce force,
        const PMI_AnisotropyType anisotype);

~Canali_e_HighFieldMobility () {}
};

Canali_e_HighFieldMobility::
Canali_e_HighFieldMobility (const PMI_Environment& env,
                           const PMI_HighFieldDrivingForce force,
                           const PMI_AnisotropyType anisotype) :
    Canali_HighFieldMobility (env, force, anisotype)
{ // default values
    beta0 = InitParameter ("beta0_e", 1.109);
    betaexp = InitParameter ("betaexp_e", 0.66);
    vsat0 = InitParameter ("vsat0_e", 1.07e7);
    vsatexp = InitParameter ("vsatexp_e", 0.87);
}

class Canali_h_HighFieldMobility : public Canali_HighFieldMobility {
public:
    Canali_h_HighFieldMobility (const PMI_Environment& env,
                               const PMI_HighFieldDrivingForce force,
                               const PMI_AnisotropyType anisotype);

    ~Canali_h_HighFieldMobility () {}
};

Canali_h_HighFieldMobility::
Canali_h_HighFieldMobility (const PMI_Environment& env,
                           const PMI_HighFieldDrivingForce force,
                           const PMI_AnisotropyType anisotype) :
    Canali_HighFieldMobility (env, force, anisotype)
{ // default values
    beta0 = InitParameter ("beta0_h", 1.213);
    betaexp = InitParameter ("betaexp_h", 0.17);
    vsat0 = InitParameter ("vsat0_h", 8.37e6);
    vsatexp = InitParameter ("vsatexp_h", 0.52);
}

extern "C"
PMI_HighFieldMobility* new_PMI_HighField_e_Mobility
    (const PMI_Environment& env, const PMI_HighFieldDrivingForce force,
     const PMI_AnisotropyType anisotype)
{ return new Canali_e_HighFieldMobility (env, force, anisotype);
}

extern "C"
PMI_HighFieldMobility* new_PMI_HighField_h_Mobility

```

41: Physical Model Interface

High-Field Saturation with Two Driving Forces

```
(const PMI_Environment& env, const PMI_HighFieldDrivingForce force,
 const PMI_AnisotropyType anisotype)
{ return new Canali_h_HighFieldMobility (env, force, anisotype);
}
```

High-Field Saturation with Two Driving Forces

This PMI allows you to compute the final mobility μ as a function of the low-field mobility μ_{low} and two driving fields (see [High-Field Saturation on page 342](#)). One field is the gradient of the quasi-Fermi energy; the other is derived from the electric field (see [Driving Force Models on page 348](#)).

Command File

The model is specified using `PMIModel` as an option to `HighFieldSaturation`, `eHighFieldSaturation`, or `hHighFieldSaturation`. The name of the model must be provided with the `Name` parameter of `PMIModel`. Optionally, an index and a string can be specified, which will be passed to and interpreted by the model:

```
eMobility(
  HighFieldSaturation(
    PMIModel (
      Name = <string>
      Index = <int>
      String = <string>)
    EparallelToInterface | Eparallel | ElectricField)
)
```

Dependencies

The high-field mobility may depend on the following variables:

<code>mulow</code>	Low-field mobility μ_{low} [$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$]
<code>n</code>	Electron density [cm^{-3}]
<code>p</code>	Hole density [cm^{-3}]
<code>T</code>	Lattice temperature [K]
<code>cT</code>	Carrier temperature [K]

Epar	Modulus of electric field driving force [V cm^{-1}]
gradQF	Modulus of gradient of the quasi-Fermi energy [eV cm^{-1}]
EprodQF	Scalar product of electric field driving force and gradient of quasi-Fermi energy [eV V cm^{-2}]
Na0	Acceptor concentration [cm^{-3}]
Nd0	Donor concentration [cm^{-3}]

The electric field used to obtain Epar is determined by EparallelToInterface, Eparallel, or ElectricField as for other high-field mobility models (see [Driving Force Models on page 348](#)). With EparallelToInterface, the electric field used to compute EprodQF is the projection of the electric field parallel to the interface; otherwise, the full electric field is used to obtain EprodQF.

The PMI model must compute the following results:

val	High-field mobility μ [$\text{cm}^2 \text{V}^{-1} \text{s}^{-1}$]
-----	---

In the case of the standard interface, the following derivatives must be computed as well:

dval_dmulow	Derivative of μ with respect to mulow [1]
dval_dn	Derivative of μ with respect to n [$\text{cm}^5 \text{V}^{-1} \text{s}^{-1}$]
dval_dp	Derivative of μ with respect to p [$\text{cm}^5 \text{V}^{-1} \text{s}^{-1}$]
dval_dT	Derivative of μ with respect to T [$\text{cm}^2 \text{V}^{-1} \text{s}^{-1} \text{K}^{-1}$]
dval_dcT	Derivative of μ with respect to cT [$\text{cm}^2 \text{V}^{-1} \text{s}^{-1} \text{K}^{-1}$]
dval_dEpar	Derivative of μ with respect to Epar [$\text{cm}^3 \text{V}^{-2} \text{s}^{-1}$]
dval_dgradQF	Derivative of μ with respect to gradQF [$\text{cm}^3 \text{eV}^{-1} \text{V}^{-1} \text{s}^{-1}$]
dval_dEprodQF	Derivative of μ with respect to EprodQF [$\text{cm}^4 \text{eV}^{-1} \text{V}^{-2} \text{s}^{-1}$]
dval_dNa0	Derivative of μ with respect to Na0 [$\text{cm}^5 \text{V}^{-1} \text{s}^{-1}$]
dval_dNd0	Derivative of μ with respect to Nd0 [$\text{cm}^5 \text{V}^{-1} \text{s}^{-1}$]

Standard C++ Interface

The PMI base class is declared in `PMIModels.h` as:

```
class PMI_HighFieldMobility2 : public PMI_Vertex_Interface {
public:
    // the input data coming from the simulator
    class idata {
    public:
        double mulow() const;    // low-field mobility
        double n() const;       // electron density
        double p() const;       // hole density
        double T() const;       // lattice temperature
        double cT() const;      // carrier temperature
        double Epar() const;     // parallel electric field
        double gradQF() const;   // gradient of quasi-Fermi energy
        double EprodQF() const;  // product gradient QF and electric field
        double Na0() const;     // acceptor concentration
        double Nd0() const;     // donor concentration
    };

    // the results computed by the PMI
    class odata {
    public:
        double& val();           // mobility
        double& dval_dmulow();   // derivative wrt. low-field mobility
        double& dval_dn();       // wrt. electron density
        double& dval_dp();       // wrt. hole density
        double& dval_dT();       // wrt. lattice temperature
        double& dval_dcT();      // wrt. carrier temperature
        double& dval_dEpar();    // wrt. parallel electric field
        double& dval_dgradQF();  // wrt. gradient of quasi-Fermi energy
        double& dval_dEprodQF(); // wrt. product gradient QF and field
        double& dval_dNa0();     // wrt. acceptor concentration
        double& dval_dNd0();     // wrt. donor concentration
    };

    // constructor and destructor
    PMI_HighFieldMobility2(const PMI_Environment& env,
                          const int model_index,
                          const std::string& model_string,
                          const PMI_AnisotropyType anisotype);
    virtual ~PMI_HighFieldMobility2();

    PMI_AnisotropyType AnisotropyType () const;
```

```
// compute value and derivatives
virtual void compute(const idata* id, odata* od) = 0;
};
```

The Compute function receives its input from id. It returns the results using od by assignment using the member functions of od. The framework initializes the values of the derivatives to zero, so you only have to compute derivatives for variables the PMI actually uses.

The following virtual constructor must be implemented:

```
typedef PMI_HighFieldMobility2* new_PMI_HighFieldMobility2_func
(const PMI_Environment& env,
 int model_index,
 const std::string& model_string,
 const PMI_AnisotropyType anisotype);
extern "C" new_PMI_HighFieldMobility2_func new_PMI_HighFieldMobility2;
```

Simplified C++ Interface

The following base class is declared in PMI.h:

```
class PMI_HighFieldMobility2_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        Input (const PMI_HighFieldMobility2_Base* highfieldmobility2_base,
              const int vertex);
        pmi_float mulow;      // low-field mobility
        pmi_float n;          // electron density
        pmi_float p;          // hole density
        pmi_float T;          // lattice temperature
        pmi_float cT;         // carrier temperature
        pmi_float Epar;       // parallel electric field
        pmi_float gradQF;     // gradient of quasi-Fermi energy
        pmi_float EprodQF;    // product gradient QF and electric field
        pmi_float Na0;        // acceptor concentration
        pmi_float Nd0;        // donor concentration
    };

    class Output {
    public:
        pmi_float val;        // mobility
    };

    PMI_HighFieldMobility2_Base (const PMI_Environment& env,
                                const int model_index,
```

41: Physical Model Interface

Band Gap

```
                                const std::string& model_string,
                                const PMI_AnisotropyType anisotype);
virtual ~PMI_HighFieldMobility2_Base ();

int model_index () const;
const std::string& model_string () const;
PMI_AnisotropyType AnisotropyType () const;

virtual void compute (const Input& input, Output& output) = 0;
};
```

The following virtual constructor is provided:

```
typedef PMI_HighFieldMobility2_Base* new_PMI_HighFieldMobility2_Base_func
(const PMI_Environment& env, const int model_index,
 const std::string& model_string, const PMI_AnisotropyType anisotype);
extern "C"
new_PMI_HighFieldMobility2_Base_func new_PMI_HighField_Mobility2_Base;
```

Band Gap

Sentaurus Device provides a PMI to compute the energy band gap E_g in a semiconductor. It can be specified in the Physics section of the command file, for example:

```
Physics {
    EffectiveIntrinsicDensity (
        BandGap (pmi_model_name)
    )
}
```

The default bandgap model in Sentaurus Device is selected explicitly by the keyword Default:

```
Physics {
    EffectiveIntrinsicDensity (
        BandGap (Default)
    )
}
```

Dependencies

The band gap E_g may depend on:

t Lattice temperature [K]

The PMI model must compute the following results:

bg Band gap E_g [eV]

In the case of the standard interface, the following derivatives must be computed as well:

dbgdt Derivative of bg with respect to t [eV K^{-1}]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_BandGap : public PMI_Vertex_Interface {
public:
    PMI_BandGap (const PMI_Environment& env);
    virtual ~PMI_BandGap ();

    virtual void Compute_bg
        (const double t, double& bg) = 0;

    virtual void Compute_dbgdt
        (const double t, double& dbgdt) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_BandGap* new_PMI_BandGap_func (const PMI_Environment& env);
extern "C" new_PMI_BandGap_func new_PMI_BandGap;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_BandGap_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t;    // lattice temperature
    };

    class Output {
    public:
        pmi_float bg;    // band gap
    };

    PMI_BandGap_Base (const PMI_Environment& env);
    virtual ~PMI_BandGap_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_BandGap_Base* new_PMI_BandGap_Base_func
    (const PMI_Environment& env);
extern "C" new_PMI_BandGap_Base_func new_PMI_BandGap_Base;
```

Example: Default Bandgap Model

Sentaurus Device uses the following default bandgap model:

$$E_g(t) = E_g(0) - \frac{\alpha t^2}{t + \beta} \quad (1099)$$

$E_g(0)$ denotes the band gap at 0 K.

```
#include "PMIModels.h"

class Default_BandGap : public PMI_BandGap {

private:
    double Eg0, alpha, beta;

public:
```

```

Default_BandGap (const PMI_Environment& env);

~Default_BandGap ();

void Compute_bg (const double t, double& bg);

void Compute_dbgdt (const double t, double& dbgdt);
};

Default_BandGap::
Default_BandGap (const PMI_Environment& env) :
    PMI_BandGap (env)
{
    Eg0 = InitParameter ("Eg0", 1.16964);
    alpha = InitParameter ("alpha", 4.73e-4);
    beta = InitParameter ("beta", 636);
}

Default_BandGap::
~Default_BandGap ()
{
}

void Default_BandGap::
Compute_bg (const double t, double& bg)
{
    bg = Eg0 - alpha * t * t / (t + beta);
}

void Default_BandGap::
Compute_dbgdt (const double t, double& dbgdt)
{
    dbgdt = - alpha * t * (t + 2.0 * beta) / ((t + beta) * (t + beta));
}

extern "C"
PMI_BandGap* new_PMI_BandGap
    (const PMI_Environment& env)
{
    return new Default_BandGap (env);
}

```

Bandgap Narrowing

Sentaurus Device provides a PMI to compute bandgap narrowing (see [Band Gap and Electron Affinity on page 249](#)). A user model is activated with the keyword `EffectiveIntrinsicDensity` in the Physics section of the command file:

```
Physics {  
    EffectiveIntrinsicDensity (pmi_model_name)  
}
```

Dependencies

A PMI bandgap narrowing model has no explicit dependencies. However, it can depend on doping concentrations through the run-time support.

The PMI model must compute:

bgn Bandgap narrowing ΔE_g^0 [eV]

In most cases, it is not necessary to compute the derivatives with respect to the dopant concentrations. However, to model dopant fluctuations (see [Chapter 23 on page 581](#)), in the standard interface the PMI model must override the functions that compute the following values:

dbgndNa Derivative of ΔE_g^0 with respect to the acceptor concentration [cm^3eV]

dbgndNd Derivative of ΔE_g^0 with respect to the donor concentration [cm^3eV]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_BandGapNarrowing : public PMI_Vertex_Interface {  
public:  
    PMI_BandGapNarrowing (const PMI_Environment& env);  
    virtual ~PMI_BandGapNarrowing ();  
    virtual void Compute_bgn (double& bgn) = 0;  
    virtual void Compute_dbgndNa (double& dbgndNa);  
    virtual void Compute_dbgndNd (double& dbgndNd);  
};
```


The following virtual constructor must be implemented:

```
typedef PMI_BandGapNarrowing* new_PMI_BandGapNarrowing_func
(const PMI_Environment& env);
extern "C" new_PMI_BandGapNarrowing_func new_PMI_BandGapNarrowing;
```

Simplified C++ Interface

The following base class is declared in the file PMI.h:

```
class PMI_BandGapNarrowing_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float acceptor; // total acceptor concentration
        pmi_float donor;    // total donor concentration
    };

    class Output {
    public:
        pmi_float bgn;      // bandgap narrowing
    };

    PMI_BandGapNarrowing_Base (const PMI_Environment& env);
    virtual ~PMI_BandGapNarrowing_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_BandGapNarrowing_Base* new_PMI_BandGapNarrowing_Base_func
(const PMI_Environment& env);
extern "C" new_PMI_BandGapNarrowing_Base_func new_PMI_BandGapNarrowing_Base;
```

Example: Default Model

The default bandgap narrowing model in Sentaurus Device (Bennett–Wilson) is given by:

$$\Delta E_g^0 = \begin{cases} E_{\text{ref}} \left[\ln \frac{N_{\text{tot}}}{N_{\text{ref}}} \right]^2, & N_{\text{tot}} > N_{\text{ref}}, \\ 0, & N_{\text{tot}} \leq N_{\text{ref}}. \end{cases} \quad (1100)$$

41: Physical Model Interface

Bandgap Narrowing

See [Band Gap and Electron Affinity on page 249](#).

This model can be implemented as a PMI model as follows:

```
#include "PMIModels.h"

class Bennett_BandGapNarrowing : public PMI_BandGapNarrowing {

private:
    double Ebgn, Nref;

public:
    Bennett_BandGapNarrowing (const PMI_Environment& env);

    ~Bennett_BandGapNarrowing ();

    void Compute_bgn (double& bgn);
};

Bennett_BandGapNarrowing::
Bennett_BandGapNarrowing (const PMI_Environment& env) :
    PMI_BandGapNarrowing (env)
{
    Ebgn = InitParameter ("Ebgn", 6.84e-3);
    Nref = InitParameter ("Nref", 3.162e18);
}

Bennett_BandGapNarrowing::
~Bennett_BandGapNarrowing ()
{
}

void Bennett_BandGapNarrowing::
Compute_bgn (double& bgn)
{
    const double Na = ReadDoping (PMI_Acceptor);
    const double Nd = ReadDoping (PMI_Donor);
    const double Ni = Na + Nd;
    if (Ni > Nref) {
        const double tmp = log (Ni / Nref);
        bgn = Ebgn * tmp * tmp;
    } else {
        bgn = 0.0;
    }
}

extern "C"
PMI_BandGapNarrowing* new_PMI_BandGapNarrowing
    (const PMI_Environment& env)
{
    return new Bennett_BandGapNarrowing (env);
}
```

Apparent Band-Edge Shift

The apparent band-edge shift Λ_{PMI} is a quantity similar to bandgap narrowing. In contrast to bandgap narrowing, the apparent band-edge shift can depend on the solution variables (electron and hole densities, lattice temperature, and electric field). Conversely, the apparent band-edge shift does not take effect in all situations where a real band-edge shift takes effect (this is why the band-edge shift is called ‘apparent’).

Implementationwise, the apparent band-edge shift is an extension of the density gradient model (see [Density Gradient Quantization Model on page 288](#)). For the PMI model, this implies:

- Sentaurus Device applies the apparent band-edge shift Λ_{PMI} everywhere where it applies quantization corrections.
- By default, the apparent band-edge shift that Sentaurus Device computes is not equal to Λ_{PMI} , but contains contributions from quantization. To remove them, set $\gamma = 0$ (see [Density Gradient Quantization Model on page 288](#)).
- Apart from a specification in the Physics section, it is necessary to specify additional equations in the Solve section (see [Using the Density Gradient Model on page 289](#)).

To select a model to compute Λ_{PMI} , specify its name using the LocalModel keyword (see [Table 238 on page 1302](#)). The same models for Λ_{PMI} can be used for the shift of the conduction and valence bands. A positive value of Λ_{PMI} means that the band shifts outwards, away from midgap (therefore, the band gap widens).

Dependencies

The apparent band-edge shift Λ_{PMI} may depend on:

n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]
t	Lattice temperature [K]
F	Absolute value of the electric field [Vcm^{-1}]

The PMI model must compute the following values:

shift	Apparent band-edge shift Λ_{PMI} [eV]
-------	--

41: Physical Model Interface

Apparent Band-Edge Shift

In the case of the standard interface, the following derivatives must be computed as well:

dshiftdn	Derivative of Λ_{PMI} with respect to n [eVcm^3]
dshiftdp	Derivative of Λ_{PMI} with respect to p [eVcm^3]
dshiftdt	Derivative of Λ_{PMI} with respect to t [eVK^{-1}]
dshiftdf	Derivative of Λ_{PMI} with respect to f [eVcmV^{-1}]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_ApparentBandEdgeShift : public PMI_Vertex_Interface {  
  
public:  
    PMI_ApparentBandEdgeShift (const PMI_Environment& env);  
    virtual ~PMI_ApparentBandEdgeShift ();  
  
    virtual void Compute_shift  
        (const double n, const double p,  
         const double t, const double f,  
         double& shift) = 0;  
  
    virtual void Compute_dshiftdn  
        (const double n, const double p,  
         const double t, const double f,  
         double& dshiftdn) = 0;  
  
    virtual void Compute_dshiftdp  
        (const double n, const double p,  
         const double t, const double f,  
         double& dshiftdp) = 0;  
  
    virtual void Compute_dshiftdt  
        (const double n, const double p,  
         const double t, const double f,  
         double& dshiftdt) = 0;  
  
    virtual void Compute_dshiftdf  
        (const double n, const double p,  
         const double t, const double f,  
         double& dshiftdf) = 0;  
};
```

The following virtual constructor must be implemented:

```
typedef PMI_ApparentBandEdgeShift* new_PMI_ApparentBandEdgeShift_func
(const PMI_Environment& env);
extern "C" new_PMI_ApparentBandEdgeShift_func new_PMI_ApparentBandEdgeShift;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_ApparentBandEdgeShift_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float n; // electron density
        pmi_float p; // hole density
        pmi_float t; // lattice temperature
        pmi_float f; // absolute value of electric field
    };

    class Output {
    public:
        pmi_float shift; // apparent band-edge shift
    };

    PMI_ApparentBandEdgeShift_Base (const PMI_Environment& env);
    virtual ~PMI_ApparentBandEdgeShift_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_ApparentBandEdgeShift_Base*
new_PMI_ApparentBandEdgeShift_Base_func (const PMI_Environment& env);
extern "C" new_PMI_ApparentBandEdgeShift_Base_func
new_PMI_ApparentBandEdgeShift_Base;
```

Multistate Configuration–dependent Apparent Band-Edge Shift

The multistate configuration (MSC)–dependent, apparent band-edge shift model is a variant of the apparent band-edge shift model (see [Apparent Band-Edge Shift on page 1061](#)). Besides dependencies on the solution variables, electron and hole densities, lattice temperature, and carrier temperatures, it allows dependencies on all state occupation rates of an arbitrary reference MSC defined by `MSConfig`. The remarks made for the apparent band-edge shift in connection with the density gradient model are valid here as well.

The model can be selected as arguments of the keywords `eBandEdgeShift`, `hBandEdgeShift`, and `BandEdgeShift` (see [Apparent Band-Edge Shift on page 435](#)) in the `MSConfig` specification. The optional model index parameter allows you to implement, in the same model, several variants that can be accessed from the command file.

Dependencies

The MSC apparent band-edge shift Λ_{PMI} may depend on:

<code>n</code>	Electron density [cm ⁻³]
<code>p</code>	Hole density [cm ⁻³]
<code>t</code>	Lattice temperature [K]
<code>ct</code>	Carrier temperature [K]
<code>s</code>	Vector of state occupation rates of the reference MSC [1]

The PMI model must compute the following values if the dependency is used:

<code>shift</code>	Apparent band-edge shift Λ_{PMI} [eV]
--------------------	--

In the case of the standard interface, the following derivatives must be computed as well:

<code>dshiftdn</code>	Derivative of Λ_{PMI} with respect to <code>n</code> [eVcm ³]
<code>dshiftdp</code>	Derivative of Λ_{PMI} with respect to <code>p</code> [eVcm ³]
<code>dshiftdt</code>	Derivative of Λ_{PMI} with respect to <code>t</code> [eVK ⁻¹]

dshiftdf Derivative of Λ_{PMI} with respect to ct [eVK^{-1}]

dshiftds Derivative of Λ_{PMI} with respect to s [eV]

Additional Functionality

The PMI model provides additional functionality.

Using Dependencies

You can use the function `set_dependency_used` to switch on or off the dependencies of this model explicitly (the default is on). For used dependencies, the function computing the corresponding derivative must be provided; for unused dependencies, the functions are not called.

Updating Actual Status

Before calling the computation functions (`compute_val` and `compute_dval_dX`), the simulator passes the actual values of the dependencies to the model using `set_actual_status`. The model parameters are updated by `init_parameter` before the actual status is updated.

Standard C++ Interface

The following base class (here, only an extract) is declared in the file `PMIModels.h`:

```
class PMI_MSC_ApparentBandEdgeShift : public PMI_MSC_Vertex_Interface {

public:
    enum e_var { var_n, var_p, var_T, var_eT, var_hT, var_s, var_undefined };

    class input_data {
    public:
        input_data ();
        ~input_data ();
        double& val ( e_var var, size_t ind );
        double val ( e_var var, size_t ind ) const;
    };

public:
    PMI_MSC_ApparentBandEdgeShift (const PMI_Environment& env,
        const std::string& msconfig_name, int model_index = 0);
```

41: Physical Model Interface

Multistate Configuration-dependent Apparent Band-Edge Shift

```
virtual ~PMI_MSC_ApparentBandEdgeShift ();

// get names of MSConfig and its states
const std::string& msconfig_name () const;
size_t nb_states () const;
const std::string& state ( size_t index ) const;

virtual void set_actual_status (
    const PMI_MSC_ApparentBandEdgeShift::input_data& id );

// compute value and derivatives
virtual void compute_val ( double& val );
virtual void compute_dval_dn ( double& val );
virtual void compute_dval_dp ( double& val );
virtual void compute_dval_dT ( double& val );
virtual void compute_dval_deT ( double& val );
virtual void compute_dval_dhT ( double& val );
virtual void compute_dval_ds ( std::vector<double>& val );

// support ramping of parameters
virtual void init_parameter ();

// handle dependencies
void set_dependency_used (
    PMI_MSC_ApparentBandEdgeShift::e_var var, bool flag );
bool dependency_used ( PMI_MSC_ApparentBandEdgeShift::e_var var ) const;
};
```

The following virtual constructor must be implemented:

```
typedef PMI_MSC_ApparentBandEdgeShift* new_PMI_MSC_ApparentBandEdgeShift_func
    (const PMI_Environment& env,
     const std::string& msconfig_name,
     int model_index);
extern "C" new_PMI_ApparentBandEdgeShift_func
    new_PMI_MSC_ApparentBandEdgeShift;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_MSC_ApparentBandEdgeShift_Base : public PMI_MSC_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float n;    // electron density
        pmi_float p;    // hole density
        pmi_float T;    // lattice temperature
        pmi_float eT;   // electron temperature
        pmi_float hT;   // hole temperature
        std::vector<pmi_float> s; // phase fraction
    };

    class Output {
    public:
        pmi_float val; // apparent band-edge shift
    };

    PMI_MSC_ApparentBandEdgeShift_Base (const PMI_Environment& env,
                                         const std::string& msconfig_name,
                                         const int model_index);

    virtual ~PMI_MSC_ApparentBandEdgeShift_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_MSC_ApparentBandEdgeShift_Base*
new_PMI_MSC_ApparentBandEdgeShift_Base_func
    (const PMI_Environment& env, const std::string& msconfig_name,
     const int model_index);
extern "C" new_PMI_MSC_ApparentBandEdgeShift_Base_func
new_PMI_MSC_ApparentBandEdgeShift_Base;
```

Electron Affinity

The electron affinity χ , that is, the energy separation between the conduction band and vacuum level, can be specified by using a PMI. The syntax in the command file is:

```
Physics {
    Affinity (pmi_model_name)
}
```

The default affinity model in Sentaurus Device can be selected explicitly by the keyword Default:

```
Physics {
    Affinity (Default)
}
```

Dependencies

The electron affinity χ may depend on:

`t` Lattice temperature [K]

The PMI model must compute:

`affinity` Electron affinity χ [eV]

In the case of the standard interface, the following derivative must be computed as well:

`affinitydt` Derivative of `affinity` with respect to `t` [eVK^{-1}]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_Affinity : public PMI_Vertex_Interface {
public:
    PMI_Affinity (const PMI_Environment& env);
    virtual ~PMI_Affinity ();

    virtual void Compute_affinity
```

```
(const double t, double& affinity) = 0;

virtual void Compute_daffinitydt
    (const double t, double& daffinitydt) = 0;
};
```

The prototype for the virtual constructor is:

```
typedef PMI_Affinity* new_PMI_Affinity_func
    (const PMI_Environment& env);
extern "C" new_PMI_Affinity_func new_PMI_Affinity;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_Affinity_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t; // lattice temperature
    };

    class Output {
    public:
        pmi_float affinity; // electron affinity
    };

    PMI_Affinity_Base (const PMI_Environment& env);
    virtual ~PMI_Affinity_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_Affinity_Base* new_PMI_Affinity_Base_func
    (const PMI_Environment& env);
extern "C" new_PMI_Affinity_Base_func new_PMI_Affinity_Base;
```

Example: Default Affinity Model

By default, Sentaurus Device uses this formula to compute χ :

$$\chi(t) = \chi(0) + 0.5 \frac{\alpha t^2}{t + \beta} \quad (1101)$$

$\chi(0)$ denotes the affinity at 0K.

```
#include "PMIModels.h"

class Default_Affinity : public PMI_Affinity {

private:
    double Affinity0, alpha, beta;

public:
    Default_Affinity (const PMI_Environment& env);

    ~Default_Affinity ();

    void Compute_affinity (const double t, double& affinity);

    void Compute_daffinitydt (const double t, double& daffinitydt);

};

Default_Affinity::
Default_Affinity (const PMI_Environment& env) :
    PMI_Affinity (env)
{ Affinity0 = InitParameter ("Affinity0", 4.05);
  alpha = InitParameter ("alpha", 4.73e-4);
  beta = InitParameter ("beta", 636);
}

Default_Affinity::
~Default_Affinity ()
{
}

void Default_Affinity::
Compute_affinity (const double t, double& affinity)
{ affinity = Affinity0 + 0.5 * alpha * t * t / (t + beta);
}

void Default_Affinity::
Compute_daffinitydt (const double t, double& daffinitydt)
```

```
{ daffinitydt = 0.5 * alpha * t * (t + 2.0 * beta) /
      ((t + beta) * (t + beta));
}
extern "C"
PMI_Affinity* new_PMI_Affinity
  (const PMI_Environment& env)
{ return new Default_Affinity (env);
}
```

Effective Mass

Sentaurus Device provides a PMI to compute the effective mass of electrons and holes. The effective mass is always expressed as a multiple of the electron mass in vacuum. The name of the PMI model must appear in the Physics section of the command file:

```
Physics {
    EffectiveMass (pmi_model_name)
}
```

Dependencies

The relative effective mass may depend on the following variables:

t	Lattice temperature [K]
bg	Band gap [eV]

The PMI model must compute the following results:

m	Relative effective mass (1)
---	-----------------------------

In the case of the standard interface, the following derivatives must be computed as well:

dmdt	Derivative of m with respect to t [K^{-1}]
dmdbg	Derivative of m with respect to bg [eV^{-1}]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_EffectiveMass : public PMI_Vertex_Interface {

public:
    PMI_EffectiveMass (const PMI_Environment& env);
    virtual ~PMI_EffectiveMass ();

    virtual void Compute_m
        (const double t, const double bg, double& m) = 0;

    virtual void Compute_dmdt
        (const double t, const double bg, double& dmdt) = 0;

    virtual void Compute_dmdbg
        (const double t, const double bg, double& dmdbg) = 0;
};
```

Two virtual constructors are necessary to compute the effective mass of electrons and holes:

```
typedef PMI_EffectiveMass* new_PMI_EffectiveMass_func
    (const PMI_Environment& env);
extern "C" new_PMI_EffectiveMass_func new_PMI_e_EffectiveMass;
extern "C" new_PMI_EffectiveMass_func new_PMI_h_EffectiveMass;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_EffectiveMass_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t;    // lattice temperature
        pmi_float bg;   // band gap
    };

    class Output {
    public:
        pmi_float m;    // effective mass
    };
};
```

```
PMI_EffectiveMass_Base (const PMI_Environment& env);
virtual ~PMI_EffectiveMass_Base ();

virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_EffectiveMass_Base* new_PMI_EffectiveMass_Base_func
    (const PMI_Environment& env);
extern "C" new_PMI_EffectiveMass_Base_func new_PMI_e_EffectiveMass_Base;
extern "C" new_PMI_EffectiveMass_Base_func new_PMI_h_EffectiveMass_Base;
```

Example: Linear Effective Mass Model

A simple, linear effective mass model is given by:

$$m = m_{300} + \frac{dm}{dt}(t - 300) \quad (1102)$$

m_{300} denotes the mass at 300 K. It can be implemented as follows:

```
#include "PMIModels.h"

class Linear_EffectiveMass : public PMI_EffectiveMass {
protected:
    double mass_300, dmass_dt;

public:
    Linear_EffectiveMass (const PMI_Environment& env);

    ~Linear_EffectiveMass ();

    void Compute_m (const double t, const double bg, double& m);

    void Compute_dmdt (const double t, const double bg, double& dmdt);

    void Compute_dmdbg (const double t, const double bg, double& dmdbg);
};

Linear_EffectiveMass::
Linear_EffectiveMass (const PMI_Environment& env) :
    PMI_EffectiveMass (env)
{
}
```

41: Physical Model Interface

Effective Mass

```
Linear_EffectiveMass::
~Linear_EffectiveMass ()
{
}

void Linear_EffectiveMass::
Compute_m (const double t, const double bg, double& m)
{ m = mass_300 + dmass_dt * (t - 300.0);
}

void Linear_EffectiveMass::
Compute_dmdt (const double t, const double bg, double& dmdt)
{ dmdt = dmass_dt;
}

void Linear_EffectiveMass::
Compute_dmdbg (const double t, const double bg, double& dmdbg)
{ dmdbg = 0.0;
}

class Linear_e_EffectiveMass : public Linear_EffectiveMass {
public:
    Linear_e_EffectiveMass (const PMI_Environment& env);

    ~Linear_e_EffectiveMass () {}
};

Linear_e_EffectiveMass::
Linear_e_EffectiveMass (const PMI_Environment& env) :
    Linear_EffectiveMass (env)
{ mass_300 = InitParameter ("mass_e_300", 1.09);
  dmass_dt = InitParameter ("dmass_e_dt", 1.6e-4);
}

class Linear_h_EffectiveMass : public Linear_EffectiveMass {
public:
    Linear_h_EffectiveMass (const PMI_Environment& env);

    ~Linear_h_EffectiveMass () {}
};

Linear_h_EffectiveMass::
Linear_h_EffectiveMass (const PMI_Environment& env) :
    Linear_EffectiveMass (env)
```



```
{ mass_300 = InitParameter ("mass_h_300", 1.15);
  dmass_dt = InitParameter ("dmass_h_dt", 9.2e-4);
}

extern "C"
PMI_EffectiveMass* new_PMI_e_EffectiveMass
  (const PMI_Environment& env)
{ return new Linear_e_EffectiveMass (env);
}

extern "C"
PMI_EffectiveMass* new_PMI_h_EffectiveMass
  (const PMI_Environment& env)
{ return new Linear_h_EffectiveMass (env);
}
```

Energy Relaxation Times

The model for the energy relaxation times τ in [Eq. 81](#) and [Eq. 82, p. 212](#) can be specified in the Physics section of the command file. The four available possibilities are:

```
Physics {
  EnergyRelaxationTimes (
    formula
    constant
    irrational
    pmi_model_name
  )
}
```

These entries have the following meaning:

formula	Use the value of formula in the parameter file (default)
constant	Use constant energy relaxation times (formula = 1)
irrational	Use the ratio of two irrational polynomials (formula = 2)
pmi_model_name	Call a PMI model to compute the energy relaxation times

Dependencies

The energy relaxation time τ may depend on the variable:

ct Carrier temperature [K]

NOTE The parameter *ct* represents the electron temperature during the calculation of τ_n and the hole temperature during the calculation of τ_p .

The PMI model must compute the following results:

tau Energy relaxation time τ [s]

In the case of the standard interface, the following derivative must be computed as well:

dtaudct Derivative of τ with respect to *ct* [sK^{-1}]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_EnergyRelaxationTime : public PMI_Vertex_Interface {  
  
public:  
    PMI_EnergyRelaxationTime (const PMI_Environment& env);  
  
    virtual ~PMI_EnergyRelaxationTime ();  
  
    virtual void Compute_tau  
        (const double ct, double& tau) = 0;  
  
    virtual void Compute_dtaudct  
        (const double ct, double& dtaudct) = 0;  
};
```

The following two virtual constructors must be implemented for electron and hole energy relaxation times:

```
typedef PMI_EnergyRelaxationTime* new_PMI_EnergyRelaxationTime_func  
    (const PMI_Environment& env);  
extern "C" new_PMI_EnergyRelaxationTime_func new_PMI_e_EnergyRelaxationTime;  
extern "C" new_PMI_EnergyRelaxationTime_func new_PMI_h_EnergyRelaxationTime;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_EnergyRelaxationTime_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float ct; // carrier temperature
    };

    class Output {
    public:
        pmi_float tau; // energy relaxation time
    };

    PMI_EnergyRelaxationTime_Base (const PMI_Environment& env);
    virtual ~PMI_EnergyRelaxationTime_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_EnergyRelaxationTime_Base* new_PMI_EnergyRelaxationTime_Base_func
    (const PMI_Environment& env);
extern "C" new_PMI_EnergyRelaxationTime_Base_func
    new_PMI_e_EnergyRelaxationTime_Base;
extern "C" new_PMI_EnergyRelaxationTime_Base_func
    new_PMI_h_EnergyRelaxationTime_Base;
```

Example: Constant Energy Relaxation Times

The following C++ code implements constant energy relaxation times:

```
#include "PMIModels.h"

class Const_EnergyRelaxationTime : public PMI_EnergyRelaxationTime {

protected:
    double tau_const;

public:
    Const_EnergyRelaxationTime (const PMI_Environment& env);
```

41: Physical Model Interface

Energy Relaxation Times

```
~Const_EnergyRelaxationTime ();

void Compute_tau
    (const double ct, double& tau);

void Compute_dtaudct
    (const double ct, double& dtaudct);

};

Const_EnergyRelaxationTime::
Const_EnergyRelaxationTime (const PMI_Environment& env) :
    PMI_EnergyRelaxationTime (env)
{
}

Const_EnergyRelaxationTime::
~Const_EnergyRelaxationTime ()
{
}

void Const_EnergyRelaxationTime::
Compute_tau (const double ct, double& tau)
{ tau = tau_const;
}

void Const_EnergyRelaxationTime::
Compute_dtaudct (const double ct, double& dtaudct)
{ dtaudct = 0.0;
}

class Const_e_EnergyRelaxationTime : public Const_EnergyRelaxationTime {

public:
    Const_e_EnergyRelaxationTime (const PMI_Environment& env);

    ~Const_e_EnergyRelaxationTime () {}

};

Const_e_EnergyRelaxationTime::
Const_e_EnergyRelaxationTime (const PMI_Environment& env) :
    Const_EnergyRelaxationTime (env)
{ tau_const = InitParameter ("tau_const_e", 0.3e-12);
}

class Const_h_EnergyRelaxationTime : public Const_EnergyRelaxationTime {
```

```
public:
    Const_h_EnergyRelaxationTime (const PMI_Environment& env);

    ~Const_h_EnergyRelaxationTime () {}

};

Const_h_EnergyRelaxationTime::
Const_h_EnergyRelaxationTime (const PMI_Environment& env) :
    Const_EnergyRelaxationTime (env)

{ tau_const = InitParameter ("tau_const_h", 0.25e-12);
}

extern "C"
PMI_EnergyRelaxationTime* new_PMI_e_EnergyRelaxationTime
    (const PMI_Environment& env)
{ return new Const_e_EnergyRelaxationTime (env);
}

extern "C"
PMI_EnergyRelaxationTime* new_PMI_h_EnergyRelaxationTime
    (const PMI_Environment& env)
{ return new Const_h_EnergyRelaxationTime (env);
}
```

Lifetimes

This PMI provides access to the electron and hole lifetimes, τ_n and τ_p , in the SRH recombination (see [Eq. 247, p. 313](#)) and the coupled defect level (CDL) recombination (see [Eq. 336, p. 365](#)). In the command file, the names of the lifetime models are given as arguments to the SRH or CDL keywords:

```
Physics {
    Recombination (SRH (pmi_model_name))
}
```

or:

```
Physics {
    Recombination (CDL (pmi_model_name))
}
```

NOTE A PMI model overrides all other keywords in an SRH or a CDL statement.

Dependencies

A PMI lifetime model may depend on the variable:

`t` Lattice temperature [K]

It must compute the following results:

`tau` Lifetime τ [s]

In the case of the standard interface, the following derivative must be computed as well:

`dtaudt` Derivative of τ with respect to lattice temperature [sK^{-1}]

Standard C++ Interface

The enumeration type `PMI_LifetimeModel` describes where the PMI lifetime is used:

```
enum PMI_LifetimeModel {
    PMI_SRH,
    PMI_CDL1,
    PMI_CDL2
};
```

The following base class is declared in the file `PMIModels.h`:

```
class PMI_Lifetime : public PMI_Vertex_Interface {

private:
    const PMI_LifetimeModel lifetimeModel;

public:
    PMI_Lifetime (const PMI_Environment& env,
                 const PMI_LifetimeModel model);

    virtual ~PMI_Lifetime ();

    PMI_LifetimeModel LifetimeModel () const { return lifetimeModel; }

    virtual void Compute_tau
        (const double t, double& tau) = 0;
```

```
virtual void Compute_dtaudt
    (const double t, double& dtaudt) = 0;
};
```

Two virtual constructors must be implemented for electron and hole lifetimes:

```
typedef PMI_Lifetime* new_PMI_Lifetime_func
    (const PMI_Environment& env, const PMI_LifetimeModel model);
extern "C" new_PMI_Lifetime_func new_PMI_e_Lifetime;
extern "C" new_PMI_Lifetime_func new_PMI_h_Lifetime;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_Lifetime_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t; // lattice temperature
    };

    class Output {
    public:
        pmi_float tau; // lifetime
    };

    PMI_Lifetime_Base (const PMI_Environment& env,
                      const PMI_LifetimeModel model);
    virtual ~PMI_Lifetime_Base ();

    PMI_LifetimeModel LifetimeModel () const;

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_Lifetime_Base* new_PMI_Lifetime_Base_func
    (const PMI_Environment& env, const PMI_LifetimeModel model);
extern "C" new_PMI_Lifetime_Base_func new_PMI_e_Lifetime_Base;
extern "C" new_PMI_Lifetime_Base_func new_PMI_h_Lifetime_Base;
```

Example: Doping- and Temperature-dependent Lifetimes

The following example combines doping-dependent lifetimes (Scharfetter) and temperature dependence (power law):

$$\tau = \left(\tau_{\min} + \frac{\tau_{\max} - \tau_{\min}}{1 + \left(\frac{N_{A,0} + N_{D,0}}{N_{\text{ref}}} \right)^\gamma} \right) \left(\frac{T}{300 \text{ K}} \right)^\alpha \quad (1103)$$

```
#include "PMIModels.h"

class Scharfetter_Lifetime : public PMI_Lifetime {

protected:
    const double T0;
    double taumin, taumax, Nref, gamma, Talpha;

public:
    Scharfetter_Lifetime (const PMI_Environment& env,
                          const PMI_LifetimeModel model);

    ~Scharfetter_Lifetime ();
    void Compute_tau
        (const double t, double& tau);

    void Compute_dtaudt
        (const double t, double& dtaudt);

};

Scharfetter_Lifetime::
Scharfetter_Lifetime (const PMI_Environment& env,
                      const PMI_LifetimeModel model) :
    PMI_Lifetime (env, model),
    T0 (300.0)
{
}

Scharfetter_Lifetime::
~Scharfetter_Lifetime ()
{
}

void Scharfetter_Lifetime::
Compute_tau (const double t, double& tau)
{ const double Ni = ReadDoping (PMI_Acceptor) + ReadDoping (PMI_Donor);
```



```

    tau = taumin + (taumax - taumin) / (1.0 + pow (Ni/Nref, gamma));
    tau *= pow (t/T0, Talpha);
}

void Scharfetter_Lifetime::
Compute_dtaudt (const double t, double& dtaudt)
{ const double Ni = ReadDoping (PMI_Acceptor) + ReadDoping (PMI_Donor);
  dtaudt = taumin + (taumax - taumin) / (1.0 + pow (Ni/Nref, gamma));
  dtaudt *= (Talpha/T0) * pow (t/T0, Talpha-1.0);
}

class Scharfetter_e_Lifetime : public Scharfetter_Lifetime {

public:
    Scharfetter_e_Lifetime (const PMI_Environment& env,
                           const PMI_LifetimeModel model);

    ~Scharfetter_e_Lifetime () {}

};

Scharfetter_e_Lifetime::
Scharfetter_e_Lifetime (const PMI_Environment& env,
                       const PMI_LifetimeModel model) :
    Scharfetter_Lifetime (env, model)
{ taumin = InitParameter ("taumin_e", 0.0);
  taumax = InitParameter ("taumax_e", 1.0e-5);
  Nref = InitParameter ("Nref_e", 1.0e16);
  gamma = InitParameter ("gamma_e", 1.0);
  Talpha = InitParameter ("Talpha_e", -1.5);
}

class Scharfetter_h_Lifetime : public Scharfetter_Lifetime {

public:
    Scharfetter_h_Lifetime (const PMI_Environment& env,
                           const PMI_LifetimeModel model);

    ~Scharfetter_h_Lifetime () {}

};

Scharfetter_h_Lifetime::
Scharfetter_h_Lifetime (const PMI_Environment& env,
                       const PMI_LifetimeModel model) :
    Scharfetter_Lifetime (env, model)
{ taumin = InitParameter ("taumin_h", 0.0);
  taumax = InitParameter ("taumax_h", 3.0e-6);
}

```

41: Physical Model Interface

Thermal Conductivity

```
Nref = InitParameter ("Nref_h", 1.0e16);
gamma = InitParameter ("gamma_h", 1.0);
Talpha = InitParameter ("Talpha_h", -1.5);
}

extern "C"
PMI_Lifetime* new_PMI_e_Lifetime
(const PMI_Environment& env, const PMI_LifetimeModel model)
{ return new Scharfetter_e_Lifetime (env, model);
}

extern "C"
PMI_Lifetime* new_PMI_h_Lifetime
(const PMI_Environment& env, const PMI_LifetimeModel model)
{ return new Scharfetter_h_Lifetime (env, model);
}
```

Thermal Conductivity

The PMI provides access to the lattice thermal conductivity κ in [Eq. 68, p. 208](#). To activate it, in the Physics section of the command file, specify:

```
Physics {
    ThermalConductivity (
        <string>
    )
}
```

where the string is the name of the PMI model.

The PMI supports anisotropic thermal conductivity, and the model can be evaluated along different crystallographic axes. The enumeration type `PMI_AnisotropyType` as defined in [Mobility Models on page 1022](#) determines the axis. The default is isotropic thermal conductivity. If anisotropic thermal conductivity is activated in the command file, the PMI class `PMI_ThermalConductivity` is also instantiated in the anisotropic direction.

Dependencies

The thermal conductivity κ may depend on the variable:

t	Lattice temperature [K]
---	-------------------------

The PMI model must compute the following results:

κ Thermal conductivity κ [$\text{Wcm}^{-1}\text{K}^{-1}$]

In the case of the standard interface, the following derivative must be computed as well:

dkappadt derivative of κ with respect to t [$\text{Wcm}^{-1}\text{K}^{-2}$]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_ThermalConductivity : public PMI_Vertex_Interface {
public:
    PMI_ThermalConductivity (const PMI_Environment& env,
                             const PMI_AnisotropyType anisotype);

    virtual ~PMI_ThermalConductivity ();

    PMI_AnisotropyType AnisotropyType () const { return anisoType; }

    virtual void Compute_kappa
        (const double t, double& kappa) = 0;

    virtual void Compute_dkappadt
        (const double t, double& dkappadt) = 0;
};
```

The following virtual constructor must be implemented:

```
typedef PMI_ThermalConductivity* new_PMI_ThermalConductivity_func
    (const PMI_Environment& env, const PMI_AnisotropyType anisotype);
extern "C" new_PMI_ThermalConductivity_func
    new_PMI_ThermalConductivity;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_ThermalConductivity_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t;    // lattice temperature
    };

    class Output {
    public:
        pmi_float kappa;    // thermal conductivity
    };

    PMI_ThermalConductivity_Base (const PMI_Environment& env,
                                   const PMI_AnisotropyType anisotype);

    virtual ~PMI_ThermalConductivity_Base ();

    PMI_AnisotropyType AnisotropyType () const;

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_ThermalConductivity_Base* new_PMI_ThermalConductivity_Base_func
    (const PMI_Environment& env, const PMI_AnisotropyType anisotype);
extern "C" new_PMI_ThermalConductivity_Base_func
    new_PMI_ThermalConductivity_Base;
```

Example: Temperature-dependent Thermal Conductivity

The following C++ code implements the temperature-dependent thermal conductivity:

$$\kappa(T) = \frac{1}{a + bT + cT^2} \quad (1104)$$

as given in [Eq. 971, p. 880](#).

```

#include "PMIModels.h"

class TempDep_ThermalConductivity : public PMI_ThermalConductivity {
private:
    double a, b, c;

public:
    TempDep_ThermalConductivity (const PMI_Environment& env, const
    PMI_AnisotropyType anisotype);
    ~TempDep_ThermalConductivity ();

    void Compute_kappa
        (const double t, double& kappa);

    void Compute_dkappadt
        (const double t, double& dkappadt);
};

TempDep_ThermalConductivity::
TempDep_ThermalConductivity (const PMI_Environment& env, const
PMI_AnisotropyType anisotype) :
    PMI_ThermalConductivity (env, anisotype)
{ // default values
    a = InitParameter ("a", 0.03);
    b = InitParameter ("b", 1.56e-03);
    c = InitParameter ("c", 1.65e-06);
}

TempDep_ThermalConductivity::
~TempDep_ThermalConductivity ()
{
}

void TempDep_ThermalConductivity::
Compute_kappa (const double t, double& kappa)
{ kappa = 1.0 / (a + b*t + c*t*t);
}

```

41: Physical Model Interface

Multistate Configuration–dependent Thermal Conductivity

```
void TempDep_ThermalConductivity::
Compute_dkappadt (const double t, double& dkappadt)
{ const double kappa = 1.0 / (a + b*t + c*t*t);
  dkappadt = -kappa * kappa * (b + 2.0*c*t);
}

extern "C"
PMI_ThermalConductivity* new_PMI_ThermalConductivity
(const PMI_Environment& env, const PMI_AnisotropyType anisotype)
{ return new TempDep_ThermalConductivity (env, anisotype);
}
```

Multistate Configuration–dependent Thermal Conductivity

This PMI provides access to the lattice thermal conductivity κ in [Eq. 68, p. 208](#) and allows it to depend on a multistate configuration (see [Chapter 18 on page 425](#)).

Command File

To activate a PMI of this type, in the Physics section, specify:

```
ThermalConductivity(
  PMIModel (
    Name = <string>
    MSConfig = <string>
    Index = <int>
    String = <string>
  )
)
```

Here, `Name` is the name of the PMI model; its specification is mandatory. `MSConfig` selects the name of the multistate configuration. `MSConfig` defaults to an empty string, which means the model does not depend on any multistate configuration. `Index` and `String` are optional; they determine the arguments `model_index` and `model_string` that are passed to the virtual constructor (see below); the interpretation of those arguments is up to the PMI model. `Index` defaults to zero, and `String` defaults to the empty string.

Dependencies

The thermal conductivity may depend on the variables:

n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]
T	Lattice temperature [K]
eT	Electron temperature [K]
hT	Hole temperature [K]
s	Multistate configuration occupation probabilities [1]

The model must compute the following quantities:

val	Thermal conductivity [$\text{Wcm}^{-1}\text{K}^{-1}$]
-----	---

In the case of the standard interface, the following derivatives must be computed as well:

dval_dn	Derivative with respect to electron density [$\text{Wcm}^2\text{K}^{-1}$]
dval_dp	Derivative with respect to hole density [$\text{Wcm}^2\text{K}^{-1}$]
dval_dT	Derivative with respect to lattice temperature [$\text{Wcm}^{-1}\text{K}^{-2}$]
dval_deT	Derivative with respect to electron temperature [$\text{Wcm}^{-1}\text{K}^{-2}$]
dval_dhT	Derivative with respect to hole temperature [$\text{Wcm}^{-1}\text{K}^{-2}$]
dval_ds	Derivative with respect to multistate configuration occupation probabilities [$\text{Wcm}^{-1}\text{K}^{-1}$]

Standard C++ Interface

The PMI offers a base class that presents the following interface:

```
class PMI_MSC_ThermalConductivity : public PMI_MSC_Vertex_Interface
{
public:
    class idata {
    public:
        double n () const;
```

41: Physical Model Interface

Multistate Configuration-dependent Thermal Conductivity

```
double p () const;
double T () const;
double eT () const;
double hT () const;
double s (size_t ind) const;
};

class odata {
public:
    double& val ();
    double& dval_dn ();
    double& dval_dp ();
    double& dval_dT ();
    double& dval_deT ();
    double& dval_dhT ();
    double& dval_ds ( size_t ind );
};

PMI_MSC_ThermalConductivity(const PMI_Environment& env,
    const std::string& msconfig_name,
    const int model_index,
    const std::string& model_string,
    const PMI_AnisotropyType anisotype);
virtual ~PMI_MSC_ThermalConductivity ();

PMI_AnisotropyType AnisotropyType () const;

virtual void compute
    (const idata* id,
     odata* od ) = 0;
};
```

The Compute function receives its input from id. It returns the results using od by assignment using the member functions of od. The PMI framework initializes the values of the derivatives to zero, so you do not have to do anything for derivatives with respect to the variables on which your model does not depend.

The following virtual constructor must be implemented:

```
typedef PMI_MSC_ThermalConductivity* new_PMI_MSC_ThermalConductivity_func
    (const PMI_Environment& env, const std::string& msconfig_name,
     int model_index, const std::string& model_string,
     const PMI_AnisotropyType anisotype);
extern "C" new_PMI_MSC_ThermalConductivity_func
    new_PMI_MSC_ThermalConductivity;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_MSC_ThermalConductivity_Base : public PMI_MSC_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        const pmi_float& n () const;    // electron density
        const pmi_float& p () const;    // hole density
        const pmi_float& T () const;    // lattice temperature
        const pmi_float& eT () const;   // electron temperature
        const pmi_float& hT () const;   // hole temperature
        const pmi_float& s (size_t ind) const; // phase fraction
    };

    class Output {
    public:
        pmi_float& val ();    // thermal conductivity
    };

    PMI_MSC_ThermalConductivity_Base (const PMI_Environment& env,
                                       const std::string& msconfig_name,
                                       const int model_index,
                                       const std::string& model_string,
                                       const PMI_AnisotropyType anisotype);

    virtual ~PMI_MSC_ThermalConductivity_Base ();

    PMI_AnisotropyType AnisotropyType () const;

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_MSC_ThermalConductivity_Base*
new_PMI_MSC_ThermalConductivity_Base_func
(const PMI_Environment& env, const std::string& msconfig_name,
 const int model_index, const std::string& model_string,
 const PMI_AnisotropyType anisotype);
extern "C" new_PMI_MSC_ThermalConductivity_Base_func
new_PMI_MSC_ThermalConductivity_Base;
```

Heat Capacity

The model for lattice heat capacity in [Eq. 68, p. 208](#) can be specified in the `Physics` section of the command file. The following two possibilities are available:

```
Physics {
  HeatCapacity (
    constant
    pmi_model_name
  )
}
```

These entries have the following meaning:

<code>constant</code>	Use constant heat capacity (default)
<code>pmi_model_name</code>	Call a PMI model to compute the heat capacity

Dependencies

The heat capacity c_L may depend on the variable:

<code>t</code>	Lattice temperature [K]
----------------	-------------------------

The PMI model must compute the following results:

<code>c</code>	Heat capacity c_L [$\text{JK}^{-1}\text{cm}^{-3}$]
----------------	--

In the case of the standard interface, the following derivative must be computed as well:

<code>dcdt</code>	Derivative of c_L with respect to <code>t</code> [$\text{JK}^{-2}\text{cm}^{-3}$]
-------------------	---

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_HeatCapacity : public PMI_Vertex_Interface {

public:
    PMI_HeatCapacity (const PMI_Environment& env);

    virtual ~PMI_HeatCapacity ();

    virtual void Compute_c
        (const double t, double& c) = 0;

    virtual void Compute_dcdt
        (const double t, double& dcdt) = 0
};
```

The following virtual constructor must be implemented:

```
typedef PMI_HeatCapacity* new_PMI_HeatCapacity_func
    (const PMI_Environment& env);
extern "C" new_PMI_HeatCapacity_func new_PMI_HeatCapacity;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_HeatCapacity_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t;    // lattice temperature
    };

    class Output {
    public:
        pmi_float c;    // heat capacity
    };
};
```

41: Physical Model Interface

Heat Capacity

```
    PMI_HeatCapacity_Base (const PMI_Environment& env);  
    virtual ~PMI_HeatCapacity_Base ();  
  
    virtual void compute (const Input& input, Output& output) = 0;  
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_HeatCapacity_Base* new_PMI_HeatCapacity_Base_func  
    (const PMI_Environment& env);  
extern "C" new_PMI_HeatCapacity_Base_func new_PMI_HeatCapacity_Base;
```

Example: Constant Heat Capacity

The following C++ code implements constant heat capacity:

```
#include "PMIModels.h"  
  
class Constant_HeatCapacity : public PMI_HeatCapacity {  
private:  
    double cv;  
  
public:  
    Constant_HeatCapacity (const PMI_Environment& env);  
    ~Constant_HeatCapacity ();  
  
    void Compute_c  
        (const double t, double& c);  
  
    void Compute_dcdt  
        (const double t, double& dcdt);  
};  
  
Constant_HeatCapacity::  
Constant_HeatCapacity (const PMI_Environment& env) :  
    PMI_HeatCapacity (env)  
{ // default values  
    cv = InitParameter ("cv", 1.63);  
}  
  
Constant_HeatCapacity::  
~Constant_HeatCapacity ()  
{  
}  
  
void Constant_HeatCapacity::  
Compute_c (const double t, double& c)
```

```
{ c = cv;
}

void Constant_HeatCapacity::
Compute_dcdt (const double t, double& dcdt)
{ dcdt = 0.0;
}

extern "C"
PMI_HeatCapacity* new_PMI_HeatCapacity
(const PMI_Environment& env)
{ return new Constant_HeatCapacity (env);
}
```

Multistate Configuration–dependent Heat Capacity

This PMI computes the lattice heat capacity and allows it to depend on a multistate configuration (see [Chapter 18 on page 425](#)).

Command File

To activate a PMI of this type, in the Physics section, specify:

```
HeatCapacity(
  PMIModel (
    Name = <string>
    MSConfig = <string>
    Index = <int>
    String = <string>
  )
)
```

The options of `PMIModel` are described in [Command File on page 1088](#).

Dependencies

The heat capacity may depend on the variables:

n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]

41: Physical Model Interface

Multistate Configuration-dependent Heat Capacity

T	Lattice temperature [K]
eT	Electron temperature [K]
hT	Hole temperature [K]
s	Multistate configuration occupation probabilities [1]

The model must compute the following quantities:

val	Heat capacity [$\text{Jcm}^{-3}\text{K}^{-1}$]
-----	--

In the case of the standard interface, the following derivatives must be computed as well:

dval_dn	Derivative with respect to electron density [JK^{-1}]
dval_dp	Derivative with respect to hole density [JK^{-1}]
dval_dT	Derivative with respect to lattice temperature [$\text{Jcm}^{-3}\text{K}^{-2}$]
dval_deT	Derivative with respect to electron temperature [$\text{Jcm}^{-3}\text{K}^{-2}$]
dval_dhT	Derivative with respect to hole temperature [$\text{Jcm}^{-3}\text{K}^{-2}$]
dval_ds	Derivative with respect to multistate configuration occupation probabilities [$\text{Jcm}^{-3}\text{K}^{-1}$]

Standard C++ Interface

The PMI offers a base class that presents the following interface:

```
class PMI_MSC_HeatCapacity : public PMI_MSC_Vertex_Interface
{
public:
    PMI_MSC_HeatCapacity (const PMI_Environment& env,
        const std::string& msconfig_name,
        const int model_index,
        const std::string& model_string);
    // otherwise, see Standard C++ Interface on page 1089
};
```

Apart from the base class name and the constructor, the explanations in [Standard C++ Interface on page 1089](#) apply here as well.

The following virtual constructor must be implemented:

```
typedef PMI_MSC_HeatCapacity* new_PMI_MSC_HeatCapacity_func
(const PMI_Environment& env, const std::string& msconfig_name,
 int model_index, const std::string& model_string);
extern "C" new_PMI_MSC_HeatCapacity_func new_PMI_MSC_HeatCapacity;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_MSC_HeatCapacity_Base : public PMI_MSC_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        const pmi_float& n () const;    // electron density
        const pmi_float& p () const;    // hole density
        const pmi_float& T () const;    // lattice temperature
        const pmi_float& eT () const;   // electron temperature
        const pmi_float& hT () const;   // hole temperature
        const pmi_float& s (size_t ind) const; // phase fraction
    };

    class Output {
    public:
        Output (NS_PMI_MSC::odata* odata);

        pmi_float& val ();    // heat capacity
    };

    PMI_MSC_HeatCapacity_Base (const PMI_Environment& env,
                               const std::string& msconfig_name,
                               const int model_index,
                               const std::string& model_string);
    virtual ~PMI_MSC_HeatCapacity_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_MSC_Mobility_Base* new_PMI_MSC_Mobility_Base_func
(const PMI_Environment& env, const std::string& msconfig_name,
 const int model_index, const std::string& model_string,
 const PMI_AnisotropyType anisotropy);
extern "C" new_PMI_MSC_HeatCapacity_Base_func new_PMI_MSC_HeatCapacity_Base;
```

Optical Quantum Yield

The quantum yield factors η_G , $\eta_{T_{\text{Eg}}}$, and η_{T_0} defined by [Eq. 516, p. 477](#) can be accessed through a PMI. In the command file, the PMI is specified in the QuantumYield section as follows:

```
Physics {
  Optics (
    OpticalGeneration (
      QuantumYield (
        ...
        pmiModel = "<pmi_model_name>"
      )
    )
  )
}
```

Dependencies

The quantum yield factors η_G , $\eta_{T_{\text{Eg}}}$, and η_{T_0} may depend on the variables:

n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]
T	Lattice temperature [K]
bg	Bandgap energy E_g [eV]
bg_eff	Effective bandgap energy (includes bandgap narrowing) $E_{g,\text{eff}}$ [eV]
wavelength	Wavelength of incident light λ [μm]
cplxRefIndex	Refractive index
cplxExtCoeff	Extinction coefficient
n_0	Base refractive index
k_0	Base extinction coefficient
d_n_lambda	Wavelength-dependent part of refractive index
d_k_lambda	Wavelength-dependent part of extinction coefficient
d_n_temp	Temperature-dependent part of refractive index

d_n_carr	Carrier-dependent part of refractive index
d_k_carr	Carrier-dependent part of extinction coefficient
d_n_gain	Gain-dependent part of refractive index

The PMI model must compute the following results:

eta_G	Quantum yield
eta_T_Eg	Thermalization yield (band gap)
eta_T0	Thermalization yield (vacuum)

Standard C++ Interface

The following base class (public interface) is declared in the file `PMIModels.h`:

```
class PMI_EXTERNAL PMI_OpticalQuantumYield : public PMI_Vertex_Interface
{
public:
    // the input data coming from the simulator

    class idata {
    public:
        idata(const void*);
        double n() const;
        double p() const;
        double T() const;
        double bg() const;
        double bg_eff() const;
        double wavelength() const;
        double cplxRefIndex() const;
        double cplxExtCoeff() const;
        double n_0() const;
        double k_0() const;
        double d_n_lambda() const;
        double d_k_lambda() const;
        double d_n_temp() const;
        double d_n_carr() const;
        double d_k_carr() const;
        double d_n_gain() const;
    };

    // the results computed by the PMI
    class odata {
    public:
```

41: Physical Model Interface

Optical Quantum Yield

```
        odata(void*);
        double& eta_G();
        double& eta_T_Eg();
        double& eta_T0();
    };

    // constructor and destructor
    PMI_OpticalQuantumYield(const PMI_Environment& env);
    virtual ~PMI_OpticalQuantumYield();

    // compute value and derivatives
    virtual void compute(const idata* id, odata* od ) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_OpticalQuantumYield* new_PMI_OpticalQuantumYield_func
(const PMI_Environment& env);
extern "C" new_PMI_OpticalQuantumYield_func new_PMI_OpticalQuantumYield;
```

Simplified C++ Interface

The following base class (public interface) is declared in the file `PMI.h`:

```
class PMI_OpticalQuantumYield_Base : public PMI_Vertex_Base {
public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        Input (const PMI_OpticalQuantumYield_Base* opticalquantumyield_base,
            const int vertex);
        pmi_float n;
        pmi_float p;
        pmi_float T;
        pmi_float bg;
        pmi_float bg_eff;
        pmi_float wavelength;
        pmi_float cplxRefIndex;
        pmi_float cplxExtCoeff;
        pmi_float n_0;
        pmi_float k_0;
        pmi_float d_n_lambda;
        pmi_float d_k_lambda;
        pmi_float d_n_temp;
        pmi_float d_n_carr;
        pmi_float d_k_carr;
        pmi_float d_n_gain;
    };
};
```

```
class Output {
public:
    pmi_float eta_G;
    pmi_float eta_T_Eg;
    pmi_float eta_T0;
};

PMI_OpticalQuantumYield_Base (const PMI_Environment& env);
virtual ~PMI_OpticalQuantumYield_Base ();
virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_OpticalQuantumYield_Base* new_PMI_OpticalQuantumYield_Base_func
    (const PMI_Environment& env);
extern "C" new_PMI_OpticalQuantumYield_Base_func
new_PMI_OpticalQuantumYield_Base;
```

Stress

Sentaurus Device supports a PMI for mechanical stress (see [Chapter 30 on page 687](#)). The name of the PMI model must appear in the `Piezo` section of the command file:

```
Physics {
    Piezo (
        Stress = pmi_model_name
    )
}
```

Dependencies

A PMI stress model has no explicit dependencies. However, it can depend on doping concentrations and mole fractions through the run-time support.

The PMI model must compute the following results:

stress_xx	XX component of stress tensor [Pa]
stress_yy	YY component of stress tensor [Pa]
stress_zz	ZZ component of stress tensor [Pa]

41: Physical Model Interface

Stress

stress_yz	YZ component of stress tensor [Pa]
stress_xz	XZ component of stress tensor [Pa]
stress_xy	XY component of stress tensor [Pa]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_Stress : public PMI_Vertex_Interface {  
  
public:  
    PMI_Stress (const PMI_Environment& env);  
    virtual ~PMI_Stress ();  
  
    virtual void Compute_StressXX  
        (double& stress_xx) = 0;  
  
    virtual void Compute_StressYY  
        (double& stress_yy) = 0;  
  
    virtual void Compute_StressZZ  
        (double& stress_zz) = 0;  
  
    virtual void Compute_StressYZ  
        (double& stress_yz) = 0;  
  
    virtual void Compute_StressXZ  
        (double& stress_xz) = 0;  
  
    virtual void Compute_StressXY  
        (double& stress_xy) = 0;  
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_Stress* new_PMI_Stress_func  
    (const PMI_Environment& env);  
extern "C" new_PMI_Stress_func new_PMI_Stress;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_Stress_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
    };

    class Output {
    public:
        pmi_float stress_xx; // xx component of stress
        pmi_float stress_yy; // yy component of stress
        pmi_float stress_zz; // zz component of stress
        pmi_float stress_yz; // yz component of stress
        pmi_float stress_xz; // xz component of stress
        pmi_float stress_xy; // xy component of stress
    };

    PMI_Stress_Base (const PMI_Environment& env);
    virtual ~PMI_Stress_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_Stress_Base* new_PMI_Stress_Base_func
    (const PMI_Environment& env);
extern "C" new_PMI_Stress_Base_func new_PMI_Stress_Base;
```

Example: Constant Stress Model

The following code returns constant values for the stress tensor:

```
#include "PMIModels.h"

class Constant_Stress : public PMI_Stress {

private:
    double xx, yy, zz, yz, xz, xy;

public:
    Constant_Stress (const PMI_Environment& env);
```

41: Physical Model Interface

Stress

```
~Constant_Stress ();

void Compute_StressXX (double& stress_xx);
void Compute_StressYY (double& stress_yy);
void Compute_StressZZ (double& stress_zz);
void Compute_StressYZ (double& stress_yz);
void Compute_StressXZ (double& stress_xz);
void Compute_StressXY (double& stress_xy);

};

Constant_Stress::
Constant_Stress (const PMI_Environment& env) :
    PMI_Stress (env)
{
    xx = InitParameter ("xx", 100);
    yy = InitParameter ("yy", -4e9);
    zz = InitParameter ("zz", 300);
    yz = InitParameter ("yz", 400);
    xz = InitParameter ("xz", 500);
    xy = InitParameter ("xy", 600);
}

Constant_Stress::
~Constant_Stress ()
{
}

void Constant_Stress::
Compute_StressXX (double& stress_xx)
{
    stress_xx = xx;
}

void Constant_Stress::
Compute_StressYY (double& stress_yy)
{
    stress_yy = yy;
}

void Constant_Stress::
Compute_StressZZ (double& stress_zz)
{
    stress_zz = zz;
}

void Constant_Stress::
Compute_StressYZ (double& stress_yz)
{
    stress_yz = yz;
}
}
```

```
void Constant_Stress::
Compute_StressXZ (double& stress_xz)
{ stress_xz = xz;
}

void Constant_Stress::
Compute_StressXY (double& stress_xy)
{ stress_xy = xy;
}

extern "C"
PMI_Stress* new_PMI_Stress
    (const PMI_Environment& env)
{ return new Constant_Stress (env);
}
```

Space Factor

The space distribution of metal workfunction (see [Metal Workfunction on page 242](#)), traps (see [Energetic and Spatial Distribution of Traps on page 408](#)), and piezoresistance enhancement factors (see [Additional Factor Model Options on page 723](#)) can be computed by a space factor PMI. The name of the PMI is specified in the appropriate Physics section as follows:

```
Physics (Material | Region = "<name>") {
    MetalWorkfunction (SFactor=pmi_model_name ...)
}
```

or:

```
Physics {
    Traps (SFactor=pmi_model_name ...)
}
```

or:

```
Physics {
    Piezo (
        Model (
            Mobility (
                Factor (SFactor=pmi_model_name ...)
            )
        )
    )
}
```

NOTE The name of the PMI model must not coincide with the name of an internal field of Sentaurus Device. Otherwise, Sentaurus Device takes the value of the internal field as the space factor.

Dependencies

A PMI space factor model has no explicit dependencies. The model must compute:

spacefactor Space factor (1) or [cm^{-3}]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_SpaceFactor : public PMI_Vertex_Interface {  
  
public:  
    PMI_SpaceFactor (const PMI_Environment& env);  
    virtual ~PMI_SpaceFactor ();  
  
    virtual void Compute_spacefactor  
        (double& spacefactor) = 0;  
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_SpaceFactor* new_PMI_SpaceFactor_func  
    (const PMI_Environment& env);  
extern "C" new_PMI_SpaceFactor_func new_PMI_SpaceFactor;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_SpaceFactor_Base : public PMI_Vertex_Base {  
  
public:  
    class Input : public PMI_Vertex_Input_Base {  
    public:  
    };  
  
    class Output {
```



```
public:
    pmi_float spacefactor; // space factor
};

PMI_SpaceFactor_Base (const PMI_Environment& env);
virtual ~PMI_SpaceFactor_Base ();

virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_SpaceFactor_Base* new_PMI_SpaceFactor_Base_func
    (const PMI_Environment& env);
extern "C" new_PMI_SpaceFactor_Base_func new_PMI_SpaceFactor_Base;
```

Example: PMI User Field as Space Factor

The following code reads the space factor from a PMI user field:

```
#include "PMIModels.h"

class pmi_spacefactor : public PMI_SpaceFactor {

public:
    pmi_spacefactor (const PMI_Environment& env);
    ~pmi_spacefactor ();

    void Compute_spacefactor (double& spacefactor);
};

pmi_spacefactor::
pmi_spacefactor (const PMI_Environment& env) :
    PMI_SpaceFactor (env)
{
}

pmi_spacefactor::
~pmi_spacefactor ()
{
}

void pmi_spacefactor::
Compute_spacefactor (double& spacefactor)
{
    spacefactor = ReadUserField (PMI_UserField1);
}
```

```
extern "C"  
PMI_SpaceFactor* new_PMI_SpaceFactor  
    (const PMI_Environment& env)  
{ return new pmi_spacefactor (env);  
}
```

Trap Capture and Emission Rates

The present PMI model can be used to define either capture and emission rates for traps (see c_C^n , c_V^p , e_C^n , and e_V^p in [Trap Occupation Dynamics on page 412](#)) or the transitions between states of multistate configurations (see [Specifying Multistate Configurations on page 427](#)). The model is an arbitrary function of n , p , T , T_n , T_p , and F .

Traps

In the command file, specify the model with the CBRate and VBRate options to Traps.

For example:

```
Traps((CBRate=("modelX",17) VBRate="modelY") ... )
```

uses the user-specified model modelX to compute c_C^n and e_C^n (see [Local Trap Capture and Emission on page 414](#)), and modelY to compute c_V^p and e_V^p . The 17 is passed as the second argument to the virtual constructor for modelX; modelY is passed as a default value of 0 instead. The interpretation of these integers is left to the user-specified models. They allow you to select among different parameters or model variants, without having to reimplement the full model for each different choice of parameters or each minor model variation.

Multistate Configurations

The present model is used for transitions of multistate configurations by the keyword CEModel (see [Specifying Multistate Configurations on page 427](#)), for example:

```
MSConfig { ...  
    Transition ( Name="t1" To = "c" From="a" CEModel ("modelX" 5) )  
}
```

where 5 is the optional model index parameter that defaults to 0.

Dependencies

The capture and emission rates may depend on the variables:

n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]
t	Lattice temperature [K]
tn	Electron temperature [K]
tp	Hole temperature [K]
f	Electric field [Vcm^{-1}]

The PMI model must compute the following results (note that the carrier type that is captured and emitted is determined by the band to which the model is applied):

capture	Capture rate [s^{-1}]
emission	Emission rate [s^{-1}]

In the case of the standard interface, the following derivatives must be computed as well:

dcapturedn	Derivative of capture rate with respect to n [s^{-1}cm^3]
dmissiondn	Derivative of emission rate with respect to n [s^{-1}cm^3]
dcapturedp	Derivative of capture rate with respect to p [s^{-1}cm^3]
dmissiondp	Derivative of emission rate with respect to p [s^{-1}cm^3]
dcapturedt	Derivative of capture rate with respect to t [$\text{s}^{-1}\text{K}^{-1}$]
dmissiondt	Derivative of emission rate with respect to t [$\text{s}^{-1}\text{K}^{-1}$]
dcapturedtn	Derivative of capture rate with respect to tn [$\text{s}^{-1}\text{K}^{-1}$]
dmissiondtn	Derivative of emission rate with respect to tn [$\text{s}^{-1}\text{K}^{-1}$]
dcapturedtp	Derivative of capture rate with respect to tp [$\text{s}^{-1}\text{K}^{-1}$]
dmissiondtp	Derivative of emission rate with respect to tp [$\text{s}^{-1}\text{K}^{-1}$]
dcapturedf	Derivative of capture rate with respect to f [$\text{s}^{-1}\text{V}^{-1}\text{cm}$]
dmissiondf	Derivative of emission rate with respect to f [$\text{s}^{-1}\text{V}^{-1}\text{cm}$]

Standard C++ Interface

The following base class is declared in `PMIModels.h`:

```
class PMI_TrapCaptureEmission : public PMI_Vertex_Interface {
public:
    PMI_TrapCaptureEmission(const PMI_Environment& env);
    virtual ~PMI_TrapCaptureEmission();

    virtual void Compute_rates
        (const double n, const double p, const double t,
         const double tn, const double tp, const double f,
         double& capture, double& emission) = 0;

    virtual void Compute_dratesdn
        (const double n, const double p, const double t,
         const double tn, const double tp, const double f,
         double& dcapturedn, double& demissiondn) = 0;

    virtual void Compute_dratesdp
        (const double n, const double p, const double t,
         const double tn, const double tp, const double f,
         double& dcapturedp, double& demissiondp) = 0;

    virtual void Compute_dratesdt
        (const double n, const double p, const double t,
         const double tn, const double tp, const double f,
         double& dcapturedt, double& demissiondt) = 0;

    virtual void Compute_dratesdtn
        (const double n, const double p, const double t,
         const double tn, const double tp, const double f,
         double& dcapturedtn, double& demissiondtn) = 0;

    virtual void Compute_dratesdtp
        (const double n, const double p, const double t,
         const double tn, const double tp, const double f,
         double& dcapturedtp, double& demissiondtp) = 0;

    virtual void Compute_dratesdf
        (const double n, const double p, const double t,
         const double tn, const double tp, const double f,
         double& dcapturedf, double& demissiondf) = 0;
};
```

The following virtual constructor must be implemented:

```
typedef PMI_TrapCaptureEmission* new_PMI_TrapCaptureEmission_func
(const PMI_Environment& env, int id);
extern "C" new_PMI_TrapCaptureEmission_func new_PMI_TrapCaptureEmission;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_TrapCaptureEmission_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float n;    // electron density
        pmi_float p;    // hole density
        pmi_float t;    // lattice temperature
        pmi_float tn;   // electron temperature
        pmi_float tp;   // hole temperature
        pmi_float f;    // absolute value of electric field
        pmi_float nc;   // lattice effective state density for electrons
        pmi_float nv;   // lattice effective state density for holes
        pmi_float egeff; // effective band gap
    };

    class Output {
    public:
        pmi_float capture; // capture rate
        pmi_float emission; // emission rate
    };

    PMI_TrapCaptureEmission_Base(const PMI_Environment& env);
    virtual ~PMI_TrapCaptureEmission_Base();

    virtual void compute(const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_TrapCaptureEmission_Base* new_PMI_TrapCaptureEmission_Base_func
(const PMI_Environment& env, int id);
extern "C" new_PMI_TrapCaptureEmission_Base_func
new_PMI_TrapCaptureEmission_Base;
```

Example: CEModel_ArrheniusLaw

The model has the structure of the Arrhenius law. It depends on the temperature.

Table 136 Model parameters

Symbol	Parameter name	Default	Unit	Description
δE	DeltaE	0.	eV	Energy difference
g	g	1.	1	Degeneracy factor
r_0	r0	1.	s ⁻¹	Maximal transition rate
E_{act}	Eact	0.	eV	Activation energy

Let δE and E_{act} be the energy difference and activation energy. The capture and emission rates are given by:

$$c = r_0 \exp(-E_{\text{act}}/kT) \quad (1105)$$

$$e = r_0 g \exp(-\langle E_{\text{act}} + \delta E \rangle / kT) \quad (1106)$$

The model is shipped with Sentaurus Device. A more flexible and general way to specify Arrhenius law transitions is provided by the transition model pmi_ce_msc (see [Arrhenius Law \(Formula=0\) on page 430](#)).

Trap Energy Shift

The present PMI determines a shift E_{shift} of trap energies that depends on the electric field and the lattice temperature at the location of the vertex or at a position given with ReferencePoint (see [Energetic and Spatial Distribution of Traps on page 408](#)).

Command File

The energy shift model is specified by EnergyShift=<model name> or EnergyShift=(<model name>, <int>) as an option to Traps in the Physics section. The optional integer defaults to zero and is passed as the argument id to the virtual constructor of the PMI (see below). The interpretation of id is dependent on the user-specified model.

Dependencies

The trap energy shift can depend on the variables:

f	Electric field vector [Vcm^{-1}], a vector with up to three components, for the field in the x-, y-, and z-direction
t	Lattice temperature [K]

The PMI model must compute the following results:

shift	Energy shift [eV]
-------	-------------------

In the case of the standard interface, the following derivatives must be computed as well:

dshiftdf	Derivative of energy shift with respect to each component of f [eVcmV^{-1}], a vector with up to three entries
dshiftdt	Derivative of energy shift with respect to t [eVK^{-1}]

Standard C++ Interface

The following base class is declared in `PMIModels.h`:

```
class PMI_TrapEnergyShift : public PMI_Vertex_Interface {
public:
    PMI_TrapEnergyShift(const PMI_Environment& env);
    virtual ~PMI_TrapEnergyShift();

    virtual void Compute_shift(
        const double f[3], const double t, double& shift) = 0;
    virtual void Compute_dshiftdf(
        const double f[3], const double t, double df[3]) = 0;
    virtual void Compute_dshiftdt(
        const double f[3], const double t, double& dt) = 0;
};
```

The following virtual constructor must be implemented:

```
typedef PMI_TrapEnergyShift* new_PMI_TrapEnergyShift_func
(const PMI_Environment& env, int id);
extern "C" new_PMI_TrapEnergyShift_func new_PMI_TrapEnergyShift;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_TrapEnergyShift_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float f[3]; // electric field vector
        pmi_float t;    // lattice temperature
    };

    class Output {
    public:
        pmi_float shift; // trap energy shift
    };

    PMI_TrapEnergyShift_Base (const PMI_Environment& env);
    virtual ~PMI_TrapEnergyShift_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_TrapEnergyShift_Base* new_PMI_TrapEnergyShift_Base_func
    (const PMI_Environment& env, int id);
extern "C" new_PMI_TrapEnergyShift_Base_func new_PMI_TrapEnergyShift_Base;
```

Piezoelectric Polarization

The effects of piezoelectric polarization can be modeled by adding the divergence of the piezoelectric polarization vector as an additional charge term:

$$q_{PE} = -\nabla P_{PE} \quad (1107)$$

to the right-hand side of the Poisson equation (see [Eq. 37, p. 191](#)):

$$\nabla \epsilon \cdot \nabla \phi = -q(p - n + N_D - N_A + q_{PE}) \quad (1108)$$

The quantity P_{PE} denotes the piezoelectric polarization vector, which may be defined by a PMI. The built-in models for piezoelectric polarization are discussed in [Dependency of Saturation Velocity on Stress on page 731](#).

The name of the PMI is specified in the Physics section of the command file as follows:

```
Physics {
    Piezoelectric_Polarization (pmi_polarization)
}
```

The piezoelectric polarization vector and the piezoelectric charge may be plotted by:

```
Plot {
    PE_Polarization/vector
    PE_Charge
}
```

Sentaurus Device assumes that the piezoelectric polarization vector P_{PE} is zero outside of the device. This boundary condition may lead to an unexpectedly large charge density if P_{PE} has a nonzero component orthogonal to the boundary (discontinuity in ∇P_{PE}).

Dependencies

The piezoelectric polarization model does not have explicit dependencies. However, it can use the run-time support. In particular, it has access to the stress fields.

The model must compute:

`pol` Piezoelectric polarization vector [Ccm^{-2}]

The resulting vector `pol` has the dimension 3. However, only the first *dim* components need to be defined, where *dim* is equal to the dimension of the problem.

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_Polarization : public PMI_Vertex_Interface {
public:
    PMI_Polarization (const PMI_Environment& env);
    virtual ~PMI_Polarization ();

    virtual void Compute_pol
        (double pol [3]) = 0;
};
```

41: Physical Model Interface

Piezoelectric Polarization

The prototype for the virtual constructor is given as:

```
typedef PMI_Polarization* new_PMI_Polarization_func
(const PMI_Environment& env);
extern "C" new_PMI_Polarization_func new_PMI_Polarization;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_Polarization_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
    };

    class Output {
    public:
        pmi_float pol [3]; // piezoelectric polarization
    };

    PMI_Polarization_Base (const PMI_Environment& env);
    virtual ~PMI_Polarization_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_Polarization_Base* new_PMI_Polarization_Base_func
(const PMI_Environment& env);
extern "C" new_PMI_Polarization_Base_func new_PMI_Polarization_Base;
```

Example: Gaussian Polarization Model

In this example, the piezoelectric polarization vector P_{PE} has a simple Gaussian shape in the x-direction:

```
#include "PMIModels.h"

class Gauss_Polarization : public PMI_Polarization {
private:
    double x0, c, a;

public:
```

```

Gauss_Polarization (const PMI_Environment& env);
~Gauss_Polarization ();

void Compute_pol (double pol [3]);
};

Gauss_Polarization::
Gauss_Polarization (const PMI_Environment& env) :
    PMI_Polarization (env)
{
    x0 = InitParameter ("x0", 0.0);
    c = InitParameter ("c", 1.0);
    a = InitParameter ("a", 1e-5);
}

Gauss_Polarization::
~Gauss_Polarization ()
{
}

void Gauss_Polarization::
Compute_pol (double pol [3])
{
    double x, y, z;
    ReadCoordinate (x, y, z);
    pol [0] = a * exp (-c * (x-x0) * (x-x0));
    pol [1] = 0.0;
    pol [2] = 0.0;
}

extern "C"
PMI_Polarization* new_PMI_Polarization
    (const PMI_Environment& env)
{
    return new Gauss_Polarization (env);
}

```

Incomplete Ionization

The ionization factors $G_D(T)$ and $G_A(T)$ (see [Incomplete Ionization Model on page 274](#)) can be defined by a PMI.

The name of the PMI should be specified in the Physics section of the command file as follows:

```

Physics {
    IncompleteIonization( Model( PMI_model_name("Species_name1
        Species_name2 ...") ) )
}

```

41: Physical Model Interface

Incomplete Ionization

In addition, it is possible to have a PMI for each species separately:

```
Physics {
    IncompleteIonization(
        Model(
            PMI_model_name1("Species_name1")
            PMI_model_name2("Species_name2")
        )
    )
}
```

The species PMI parameters should be defined in the parameter file (see [Parameter File of Sentaurus Device on page 1007](#)).

Dependencies

The ionization factors $G_D(T)$ and $G_A(T)$ may depend on the variable:

t Lattice temperature [K]

The PMI model must compute the following results:

g Ionization factor $G(T)$

In the case of the standard interface, the following derivative must be computed as well:

dgdt Derivative of $G(T)$ with respect to T

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
enum PMI_SpeciesType {
    PMI_acceptor,
    PMI_donor
};

class PMI_DistributionFunction : public PMI_Vertex_Interface {
private:
    const PMI_SpeciesType speciesType;
    const char* speciesName;
```

```
public:
    PMI_DistributionFunction (const PMI_Environment& env,
                             const char* name
                             const PMI_SpeciesType type = PMI_acceptor);

    virtual ~PMI_DistributionFunction ();

    PMI_SpeciesType SpeciesType () const { return speciesType; }
    const char* SpeciesName () const { return speciesName; }

    // read parameter from Sentaurus Device parameter file
    // (override for PMI_Vertex_Interface::ReadParameter)
    const PMIBaseParam* ReadParameter (const char* name) const;

    // initialize parameter from Sentaurus Device parameter file or from default
    // value (override for PMI_Vertex_Interface::InitParameter)
    double InitParameter (const char* name, double defaultvalue) const;

    virtual void Compute_g
        (const double T,          // lattice temperature
         double& g) = 0;          // g = G(T)

    virtual void Compute_dgdt
        (const double T,          // lattice temperature
         double& dgdt) = 0;       // dgdt = G'(T)

};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_DistributionFunction* new_PMI_DistributionFunction_func
    (const PMI_Environment& env);
extern "C" new_PMI_DistributionFunction_func new_PMI_DistributionFunction;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_DistributionFunction_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t;    // lattice temperature
    };
};
```

41: Physical Model Interface

Incomplete Ionization

```
class Output {
public:
    pmi_float g;    // ionization factor
};

PMI_DistributionFunction_Base (const PMI_Environment& env,
                              const char* name,
                              const PMI_SpeciesType type = PMI_acceptor);
virtual ~PMI_DistributionFunction_Base ();

PMI_SpeciesType SpeciesType () const;
const char* SpeciesName () const;

const PMIBaseParam* ReadParameter (const char* name) const;
double InitParameter (const char* name, double defaultvalue) const;

virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_DistributionFunction_Base* new_PMI_DistributionFunction_Base_func
(const PMI_Environment& env, const char* name, const PMI_SpeciesType type);
extern "C" new_PMI_DistributionFunction_Base_func
new_PMI_DistributionFunction_Base;
```

Example: Matsuura Incomplete Ionization Model

The following C++ code implements the Matsuura model [1] for dopant Al in SiC material:

$$G_A(T) = 4 \exp\left(\frac{\Delta E_A - E_{ex}}{kT}\right) \cdot \left(g_1 + \sum_{r=2} g_r \exp\left(\frac{\Delta E_r - \Delta E_A}{kT}\right)\right) \quad (1109)$$

where g_1 is the ground-state degeneracy factor, g_r is the $(r - 1)$ -th excited state degeneracy factor, and ΔE_r is the difference in energy between the $(r - 1)$ -th excited state level and E_V . ΔE_r is given by the hydrogenic dopant model [2]:

$$\Delta E_r = 13.6 \cdot \frac{m^*}{m_0 \cdot \epsilon_s^2} \cdot \frac{1}{r^2} \quad [\text{eV}] \quad (1110)$$

where m^* is the hole effective mass in SiC, and ϵ_s is the dielectric constant of SiC.

The acceptor level is described as [2]:

$$\Delta E_A = \Delta E_1 + E_{CCC} \quad (1111)$$

where E_{CCC} is the energy induced due to central cell corrections.

The ensemble average E_{ex} of the ground and excited state levels of the acceptor is given by [3]:

$$E_{ex} = \frac{\sum_{r=2} (\Delta E_A - \Delta E_r) g_r \exp\left(-\frac{\Delta E_A - \Delta E_r}{kT}\right)}{g_1 + \sum_{r=2} g_r \exp\left(-\frac{\Delta E_A - \Delta E_r}{kT}\right)} \quad (1112)$$

The Matsuura model can be implemented as follows:

```
class Matsuura_DistributionFunction : public PMI_DistributionFunction {
protected:
    const double kB_300; // Boltzmann constant * 300 [eV]
    int nb_item; // number of item in sum
    double *gr, *dEr;
    double Eex, dEA, Eccc;

public:
    Matsuura_DistributionFunction (const PMI_Environment& env,
                                   const char* name,
                                   const PMI_SpeciesType type = PMI_acceptor);

    ~Matsuura_DistributionFunction ();

    void Compute_g
        (const double T, // lattice temperature
         double& g); // g = G(T)

    void Compute_dgdt
        (const double T, // lattice temperature
         double& dgdt); // dgdt = G'(T)

    double Compute_Eex(double T); // compute Eex(T) Eq. 1112, p. 1121
    double Compute_dEexdT(double T); // compute dEex/dT

};

Matsuura_DistributionFunction::
Matsuura_DistributionFunction (const PMI_Environment& env,
                                const char* name,
```

41: Physical Model Interface

Incomplete Ionization

```
const PMI_SpeciesType type) :
PMI_DistributionFunction (env, name, type),
kB_300(1.380662e-23*300./1.602192e-19), // kB*T0/e0 = 0.02585199527 [eV]
Eex(0.)
{
    nb_item = InitParameter ("NumberOfItem", 1);
    Eccc     = InitParameter ("Eccc", 0);

    if(nb_item < 1) {
        printf("ERROR; PMI model Matsuura_DistributionFunction: parameter
NumberOfItem < 1 \n");
        exit(1);
    }
    gr = new double[nb_item];
    dEr = new double[nb_item];

    char str_r[6], name_gr[6];

    int r;
    for(r=0; r<nb_item; ++r) {
        name_gr[0] = 'g'; name_gr[1] = '\0';
        sprintf(str_r, "%d\0", r+1);
        strcat(name_gr, str_r);
        const PMIBaseParam* par = ReadParameter(name_gr);
        if(!par) {
            printf("ERROR; PMI model Matsuura_DistributionFunction: cannot read
parameter %s \n", name_gr);
            gr[r] = 2;
        } else
            gr[r] = *par;
    }

    const PMIBaseParam* par = ReadParameter("dE1");
    if(!par) {
        // dE[r] = 13.6 * m_eff/m0/eps/eps/r/r
        Compute_dEr(nb_item, dEr);
    } else {
        char name_dEr[6];
        for(r=0; r<nb_item; ++r) {
            strcpy(name_dEr, "dE");
            sprintf(str_r, "%d\0", r+1);
            strcat(name_dEr, str_r);
            const PMIBaseParam* par = ReadParameter(name_dEr);
            if(!par) {
                printf("ERROR; PMI model Matsuura_DistributionFunction: cannot read
parameter %s \n", name_dEr);
                exit(1);
            }
        }
    }
}
```



```

        dEr[r] = *par;
    }
}
dEA = dEr[0] + Eccc;
}

Matsuura_DistributionFunction::
~Matsuura_DistributionFunction ()
{
    delete[] gr;
    delete[] dEr;
}

void Matsuura_DistributionFunction::Compute_g
(const double T,                // lattice temperature
 double& g)                    // g = G(T)
{
    const double kT = kB_300*T/300;

    Eex = Compute_Eex(T);
    g = gr[0];

    for(int r=1; r<nb_item; ++r) {
        double delta = dEA - dEr[r];
        g += gr[r]*exp( -delta/kT );
    }
    g *= 4.*exp( (dEA-Eex)/kT );
}

void Matsuura_DistributionFunction::Compute_dgdt
(const double T,                // lattice temperature
 double& dgdt)                 // dgdt = G'(T)
{
    const double kT = kB_300*T/300;

    Eex = Compute_Eex(T);
    double s1, s2 = gr[0], s3, s4 = 0., delta, tmp;

    for(int r=1; r<nb_item; ++r) {
        delta = dEA - dEr[r];
        tmp = gr[r]*exp( -delta/kT );
        s2 += tmp;
        s4 += tmp*delta/kT/T;           // s4 = ds2/dT
    }

    delta = dEA - Eex;
    s1 = 4.*exp( delta/kT );
    double dEex_dT = Compute_dEexdT(T);

```

41: Physical Model Interface

Incomplete Ionization

```
s3 = s1*( -dEex_dT/kT - delta/kT/T );    // s3 = ds1/dT

dgdT = s3*s2 + s1*s4;                    // dgdT = d(s1*s2)/dT

return;
}

// Eex is given by Eq. 1112, p. 1121
double Matsuura_DistributionFunction::Compute_Eex(double T)
{
    const double kT = kB_300*T/300;

    double s1 = 0., s2 = gr[0];

    for(int r=1; r<nb_item; ++r) {
        double delta = dEA - dEr[r];
        double tmp   = gr[r]*exp( -delta/kT );
        s1 += delta*tmp;
        s2 += tmp;
    }

    return s1/s2;
}

double Matsuura_DistributionFunction::Compute_dEexdT(double T)
{
    const double kT = kB_300*T/300;

    double s1 = 0., s2 = gr[0], s3 = 0., s4 = 0.;

    for(int r=1; r<nb_item; ++r) {
        double delta = dEA - dEr[r];
        double tmp   = gr[r]*exp( -delta/kT );
        s1 += delta*tmp;
        s2 += tmp;
        s3 += delta*tmp*delta/kT/T;    // s3 = ds1/dT
        s4 += tmp*delta/kT/T;         // s4 = ds2/dT
    }

    return (s3*s2 - s1*s4)/s2/s2;
}

extern "C"
PMI_DistributionFunction* new_PMI_DistributionFunction
(const PMI_Environment& env,
 const char* name,
```

```

    const PMI_SpeciesType type)
{
    return new Matsuura_DistributionFunction (env, name, type);
}

void Compute_dEr(int nb_item, double* dEr)
{
    // dEr is given by Eq. 1110, p. 1120

    // data from file: 6H-SiC.par
    const double epsilon = 9.66;           // dielectric constant
    const double mh      = 1;              // hole effective mass in SiC

    const double E0 = 13.6*mh/epsilon/epsilon;

    for(int r=1; r<=nb_item; ++r) {
        dEr[r-1] = E0/r/r;
    }
}

```

Hot-Carrier Injection

Sentaurus Device provides a PMI for implementing hot-carrier injection models and computing the injection currents defined by the model. It is activated in the Physics interface section of the command file, GateCurrent subsection:

```

Physics(MaterialInterface="Silicon/Oxide"){
    GateCurrent( PMI_model(electron) )
}

```

The model can be used for both carrier types with either `PMI_model(electron hole)` or `PMI_model()`. The interface has access to the device mesh and device data (see [Vertex-based Run-Time Support for Multistate Configuration-dependent Models on page 995](#)).

Dependencies

A hot-carrier PMI model has no explicit dependencies. However, it can depend on any field at run-time using the access to device data. The model must compute:

gCurr	Vector of region/interface arrays with hot-carrier injection current densities; each region/interface array consists of the current density in each vertex of a region interface.
-------	---

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_HotCarrierInjection : public PMI_Device_Interface {

protected:
    const PMI_CarrierType cType;

public:
    PMI_HotCarrierInjection (const PMI_Device_Environment& env,
        const PMI_CarrierType cType);
    virtual ~PMI_HotCarrierInjection ();

    virtual void Compute_gCurr
        (const des_regioninterface_vector& regioninterfaces,
            // region interfaces associated with the model
        des_array_vector& gCurr) = 0; // gate injection current in each vertex
            // of specified region interfaces
};
```

Two virtual constructors are required for electron and hole hot injection:

```
typedef PMI_HotCarrierInjection* new_PMI_HotCarrierInjection_func
    (const PMI_Device_Environment& env, const PMI_CarrierType carType);
extern "C" new_PMI_HotCarrierInjection_func new_PMI_e_HotCarrierInjection;
extern "C" new_PMI_HotCarrierInjection_func new_PMI_h_HotCarrierInjection;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_HotCarrierInjection_Base : public PMI_Device_Base {

public:
    class Input : public PMI_Device_Input_Base {
    public:
        des_regioninterface_vector regioninterfaces; // region interfaces
                                                    // associated with model
                                                    // name
    };

    class Output {
    public:
        sdevice_array_vector gCurr; // gate injection current in each vertex
            // of specified region interfaces
    };
};
```

```
};

PMI_HotCarrierInjection_Base (const PMI_Device_Environment& env);
virtual ~PMI_HotCarrierInjection_Base ();

virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_HotCarrierInjection_Base* new_PMI_HotCarrierInjection_Base_func
(const PMI_Device_Environment& env);
extern "C" new_PMI_HotCarrierInjection_Base_func
new_PMI_e_HotCarrierInjection_Base;
extern "C" new_PMI_HotCarrierInjection_Base_func
new_PMI_h_HotCarrierInjection_Base;
```

Example: Lucky Model

The following example re-implements the built-in lucky injection model using the hot-carrier injection PMI:

```
#include "PMIModels.h"

class PMI_LuckyModel : public PMI_HotCarrierInjection {

private:
    const des_mesh* mesh; //device mesh
    des_data* data; //device data
    const double*const* measure;
    const double*const* surface_measure;

public:
    PMI_LuckyModel(const PMI_Device_Environment& env, const PMI_CarrierType
carType);
    ~PMI_LuckyModel();

    void Compute_gCurr(const des_regioninterface_vector regioninterfaces,
des_array_vector& gCurr);
};

void PMI_LuckyModel::Compute_gCurr(
const des_regioninterface_vector regioninterfaces, des_array_vector& gCurr)
{
    //compute current for each des_regioninterface associated with model
    for(int inter=0; inter < regioninterfaces.size(); inter++) {
        for(int k=0; k < regioninterfaces.at(inter)->size_vertex(); k++)
```

41: Physical Model Interface

Hot-Carrier Injection

```
gCurr[inter][k] = 0.0;

des_regioninterface* ri = regioninterfaces.at(inter);
des_bulk* b1 = ri->bulk1();
des_bulk* b2 = ri->bulk2();

//recognize regioninterface or regioninterface category
if(((b1->material() == "Silicon") && (b2->material() == "Oxide")) ||
    ((b1->material() == "Oxide") && (b2->material() == "Silicon"))) {
    des_bulk* semReg = (b1->material() == "Silicon") ? b1 : b2;
    des_bulk* insReg = (b1->material() == "Silicon") ? b2 : b1;;
    //read DataEntries used in computation
    const double* pot = data->ReadScalar(des_data::vertex,
    "ElectrostaticPotential");
    ...
    const double* eCurrent = (cType == PMI_Electron)
        ? data->ReadScalar(des_data::vertex, "eCurrentDensity") : NULL;

    //integrate over the corresponding semiconductor region
    for(size_t vi = 0; vi < semReg->size_vertex(); vi++) {
        des_vertex* rv = semReg->vertex(vi);
        //find the nearest interface vertex (distance to interface)
        des_vertex* NearestInterVertex;
        double NearestDistance;
        ...
        //find the nearest gate contact vertex (distance from NearestIntVertex
to gate contact)
        des_vertex* NearestContVertex;
        double NearestContDistance;
        ...
        //coordinates and distances are in um, transform to cm
        double OxThickn = NearestContDistance*1.0e-4;
        double DistToSurf = NearestDistance*1.0e-4;

        //find the corresponding oxide element and its epsilon
        ...
        double OxConst = Epsilon[xEl->index()];
        double OxImage0 = 1.6e-19/16/3.1452/8.85e-14/OxConst;

        //for electrons
        if(cType == PMI_Electron) {
            double eCur = 0.0;
            double eOxField = pot[NearestContVertex->index()]
                -pot[NearestInterVertex->index()];
            eOxField = eOxField > 0 ? OxField[NearestInterVertex->index()]
                : -OxField[NearestInterVertex->index()];

            double barrier;
```

```

        if(pot[NearestInterVertex->index()] < pot[NearestContVertex-
>index()])
            barrier = eBarrierHeight-eAlfa*sqrt(fabs(eOxField))-
eBeta*pow(fabs(eOxField),2.0/3.0);
        else
            barrier = eBarrierHeight+(pot[NearestInterVertex->index()]
-pot[NearestContVertex->index()])-
eAlfa*sqrt(fabs(eOxField))
            -eBeta*pow(fabs(eOxField),2.0/3.0);
        if(barrier < 0) barrier = 0.0;
        double eBarrierLoc = barrier;
        double p1 = 0.0;
        double eEnergy = eField[rv->index()*eLambd;
        if(eEnergy > 1.0e-30) {
            p1 = 0.25;
            if (eBarrierLoc > 1.0e-30) p1 = 0.25*eEnergy/eBarrierLoc*exp(-
eBarrierLoc/eEnergy);
        }
        double p2 = exp(-DistToSurf/eLambd);
        double eDistFromSurf = 1.0e30;
        if (eOxField > 1.0e-30) eDistFromSurf = sqrt(OxImage0/eOxField);
        double p3 = exp(-eDistFromSurf/eOxLambd);

        //compute the node measure
        double node_measure;

        eCur = eCurrent[rv->index()*p1*p2*p3*node_measure/eLambdR;
        gCurr[inter][NearestInterVertexRIind] += eCur/risurface;
    }

    //for holes
    if(cType == PMI_Hole) {
        ...
    }
} //end integration on semiconductor region
} //end model implementation
} //end loop over regioninterfaces associated with "PMI_HotCarrierInjection"
model
}
extern "C" {
    PMI_HotCarrierInjection* new_PMI_e_HotCarrierInjection
        (const PMI_Device_Environment& env, const PMI_CarrierType carType)
    {
        return new PMI_LuckyModel(env, carType);
    }
}
extern "C" {
    PMI_HotCarrierInjection* new_PMI_h_HotCarrierInjection

```

```
(const PMI_Device_Environment& env, const PMI_CarrierType carType)
{
    return new PMI_LuckyModel(env, carType);
}
}
```

Piezoresistive Coefficients

Sentaurus Device provides a PMI for implementing the dependencies of the piezoresistive prefactors over the normal electric field (see [Enormal- and MoleFraction-dependent Piezo Coefficients on page 723](#)). It is activated in the Piezo section of the command file, in the Tensor subsection:

```
Physics {
    ...
    Piezo(
        Model(Mobility(Tensor("pmi_model")))
    )
}
```

Dependencies

The piezoresistive prefactors e_{pij} and h_{pij} may depend on the following variable:

E_{normal}	Normal to interface semiconductor–dielectric electric field [Vcm^{-1}]
--------------	--

The PMI model must compute the following results:

p_{11} , p_{12} , p_{44}	Piezoresistive prefactors [1]
--------------------------------	-------------------------------

In the case of the standard interface, the following derivatives must be computed as well:

dp_{11} , dp_{12} , dp_{44}	Derivatives of p_{ij} with respect to E_{normal} [$V^{-1}cm$]
-----------------------------------	---

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_PiezoresistanceFactor : public PMI_Vertex_Interface {
public:
    PMI_PiezoresistanceFactor (const PMI_Environment& env);
    virtual ~PMI_PiezoresistanceFactor ();

    virtual void Compute_Pij(const double Enormal,
        double& p11, double& p12, double& p44) = 0;

    virtual void Compute_DerPij(const double Enormal,
        double& dp11, double& dp12, double& dp44) = 0;
};
```

Two virtual constructors are required for the calculation of the piezoresistive prefactors.

```
typedef PMI_PiezoresistanceFactor* new_PMI_PiezoresistanceFactor_func
    (const PMI_Environment& env);
extern "C" new_PMI_PiezoresistanceFactor_func new_PMI_ePiezoresistanceFactor;
extern "C" new_PMI_PiezoresistanceFactor_func new_PMI_hPiezoresistanceFactor;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_PiezoresistanceFactor_Base : public PMI_Vertex_Base {
public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float Enormal; // normal to interface electric field
    };

    class Output {
    public:
        pmi_float p11; // piezoresistive prefactor
        pmi_float p12; // piezoresistive prefactor
        pmi_float p44; // piezoresistive prefactor
    };

    PMI_PiezoresistanceFactor_Base (const PMI_Environment& env);
    virtual ~PMI_PiezoresistanceFactor_Base ();
```

41: Physical Model Interface

Current Plot File of Sentaurus Device

```
virtual bool IsPrefactor () = 0;  
  
virtual void compute (const Input& input, Output& output) = 0;  
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_PiezoresistanceFactor_Base*  
new_PMI_PiezoresistanceFactor_Base_func (const PMI_Environment& env);  
extern "C" new_PMI_PiezoresistanceFactor_Base_func  
new_PMI_ePiezoresistanceFactor_Base;  
extern "C" new_PMI_PiezoresistanceFactor_Base_func  
new_PMI_hPiezoresistanceFactor_Base;
```

Current Plot File of Sentaurus Device

The current plot PMI allows user-computed entries to be added to the current plot file. It is specified in the `CurrentPlot` section of the command file, for example:

```
CurrentPlot {  
    pmi_CurrentPlot  
}
```

The interface has access to the device mesh and device data (see [Mesh-based Run-Time Support on page 996](#)).

Structure of Current Plot File

A current plot file consists of a header section and a data section. For each function, the structure can be described as follows:

```
dataset name  
function name  
value0  
value1  
...
```

A dataset name denotes a dataset, for example:

```
time  
Tmin
```

If a dataset corresponds to a region or contact, it is customary to add the region or contact name:

gate Charge

The function name describes the function, for example:

ElectrostaticPotential
Temperature

Afterwards, a function value is added to the current plot file for each plot time point.

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_CurrentPlot : public PMI_Device_Interface {  
  
public:  
    PMI_CurrentPlot (const PMI_Device_Environment& env);  
    virtual ~PMI_CurrentPlot ();  
  
    virtual void Compute_Dataset_Names  
        (des_string_vector& dataset) = 0;  
  
    virtual void Compute_Function_Names  
        (des_string_vector& function) = 0;  
  
    virtual void Compute_Plot_Values  
        (des_double_vector& value) = 0;  
};
```

The methods `Compute_Dataset_Names()` and `Compute_Function_Names()` are used to generate the header in the current plot file (see [Structure of Current Plot File on page 1132](#)). `Compute_Plot_Values()` is called for each plot time point to compute the plot values. Use the `push_back()` function to add values to the arrays `dataset`, `function`, or `value`.

NOTE All three methods `Compute_Dataset_Names()`, `Compute_Function_Names()`, and `Compute_Plot_Values()` must always compute the same number of values. Otherwise, an inconsistent current plot file will be generated.

The prototype for the virtual constructor is given as:

```
typedef PMI_CurrentPlot* new_PMI_CurrentPlot_func  
    (const PMI_Device_Environment& env);  
extern "C" new_PMI_CurrentPlot_func new_PMI_CurrentPlot;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_CurrentPlot_Base : public PMI_Device_Base {

public:
    class Input : public PMI_Device_Input_Base {
    public:
    };

    class Output_Header {
    public:
        des_string_vector dataset;    // array of dataset names
        des_string_vector function;    // array of function names
    };

    class Output_Body {
    public:
        sdevice_pmi_float_vector value;    // array of plot values
    };

    PMI_CurrentPlot_Base (const PMI_Device_Environment& env);
    virtual ~PMI_CurrentPlot_Base ();

    virtual void compute_header (Output_Header& output) = 0;
    virtual void compute_body (const Input& input, Output_Body& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_CurrentPlot_Base* new_PMI_CurrentPlot_Base_func
    (const PMI_Device_Environment& env);
extern "C" new_PMI_CurrentPlot_Base_func new_PMI_CurrentPlot_Base;
```

Example: Average Electrostatic Potential

The following example computes regionwise averages for the electrostatic potential. This is the same functionality as provided by the built-in current plot command (see [Tracking Additional Data in the Current File on page 140](#)):

```
class CurrentPlot : public PMI_CurrentPlot {
private:
    typedef std::vector<des_bulk*> des_bulk_vector;

    const des_mesh* mesh;    // device mesh
```

```

    des_bulk_vector regions; // list of semiconductor bulk regions
    double scale;           // scaling factor

public:
    CurrentPlot (const PMI_Device_Environment& env);
    ~CurrentPlot ();

    void Compute_Dataset_Names (des_string_vector& dataset);
    void Compute_Function_Names (des_string_vector& function);
    void Compute_Plot_Values (des_double_vector& value);
};

CurrentPlot::
CurrentPlot (const PMI_Device_Environment& env) :
    PMI_CurrentPlot (env)
{
    mesh = Mesh ();
    // determine regions to process
    for (size_t ri = 0; ri < mesh->size_region (); ri++) {
        des_region* r = mesh->region (ri);
        if (r->type () == des_region::bulk) {
            des_bulk* b = dynamic_cast <des_bulk*> (r);
            if (b->material () != "Oxide") {
                // we found a semiconductor bulk region
                regions.push_back (b);
            }
        }
    }

    // read parameters
    scale = InitParameter ("scale", 0.0);
}

CurrentPlot::
~CurrentPlot ()
{
}

void CurrentPlot::
Compute_Dataset_Names (des_string_vector& dataset)
{
    for (size_t ri = 0; ri < regions.size (); ri++) {
        des_bulk* b = regions [ri];
        std::string name = "Average_";
        name += b->name ();
        name += "ElectrostaticPotential";
        dataset.push_back (name);
    }
}

```

```

void CurrentPlot::
Compute_Function_Names (des_string_vector& function)
{ for (size_t ri = 0; ri < regions.size (); ri++) {
    function.push_back ("ElectrostaticPotential");
  }
}

void CurrentPlot::
Compute_Plot_Values (des_double_vector& value)
{ des_data* data = Data ();
  const double*const* measure = data->ReadMeasure ();
  const double* pot = data->ReadScalar (des_data::vertex,
                                         "ElectrostaticPotential");

  for (size_t ri = 0; ri < regions.size (); ri++) {
    des_bulk* b = regions [ri];

    double sum_pot = 0.0;
    double sum_measure = 0.0;

    for (size_t ei = 0; ei < b->size_element (); ei++) {
      des_element* e = b->element (ei);
      for (size_t vi = 0; vi < e->size_vertex (); vi++) {
        des_vertex* v = e->vertex (vi);
        const double m = measure [e->index ()][vi];
        const double p = pot [v->index ()];
        sum_pot += m * p;
        sum_measure += m;
      }
    }

    value.push_back (scale * (sum_pot / sum_measure));
  }
}

extern "C" {
PMI_CurrentPlot* new_PMI_CurrentPlot (const PMI_Device_Environment& env)
{ return new CurrentPlot (env);
}
}

```

Postprocess for Transient Simulation

The postprocess PMI allows you to post-compute data during a transient simulation. The PMI is called after a transient time step has succeeded. It is specified in the Math section of the command file, for example:

```
Math {  
    PostProcess (  
        Transient = "pmi_postprocess"  
    )  
}
```

The interface provides access to the device mesh and device data (see [Mesh-based Run-Time Support on page 996](#)).

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_PostProcess : public PMI_Device_Interface {  
  
public:  
    PMI_PostProcess (const PMI_Device_Environment& env);  
    virtual ~PMI_PostProcess ();  
  
    virtual void Compute_PostProcess () = 0;  
}
```

The method `Compute_PostProcess()` is called after the transient time step has successfully completed.

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_PostProcess_Base : public PMI_Device_Base {  
  
public:  
    class Input : public PMI_Device_Input_Base {  
    public:  
    };  
};
```

41: Physical Model Interface

Postprocess for Transient Simulation

```
class Output {
public:
};

PMI_PostProcess_Base (const PMI_Device_Environment& env);
virtual ~PMI_PostProcess_Base ();

virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_PostProcess_Base* new_PMI_PostProcess_Base_func
(const PMI_Device_Environment& env);
extern "C" new_PMI_PostProcess_Base_func new_PMI_PostProcess_Base;
```

The method `compute()` is called after the transient time step has successfully completed.

Example: Postprocess User-Field

The following code modifies the user-field depending on the current temperature change and transient step size after the transient time step has succeeded:

```
#include "PMIModels.h"

class PostProcess : public PMI_PostProcess {

private:
    const des_mesh* mesh;

public:
    PostProcess (const PMI_Device_Environment& env);
    ~PostProcess ();
    void Compute_PostProcess ();
};

PostProcess::
PostProcess (const PMI_Device_Environment& env) :
    PMI_PostProcess (env)
{
    mesh = Mesh();
}

PostProcess::
~PostProcess ()
{
}
```



```
void PostProcess::
Compute_PostProcess ()
{
    des_data* data = Data();

    const double* T = data->ReadScalar(des_data::vertex,
        "LatticeTemperature");
    const double* T0 = data->ReadScalar(des_data::vertex, "PMIUserField0");

    double* delta_T = new double [mesh->size_vertex ()];
    for (int vi=0; vi<mesh->size_vertex (); vi++) {
        delta_T[vi] = (T[vi]-T0[vi])/ReadTransientStepSize();
    }
    data->WriteScalar(des_data::vertex, "PMIUserField0", T);
    data->WriteScalar(des_data::vertex, "PMIUserField1", delta_T);
    if (delta_T!=NULL) { delete [] delta_T; }
}
```

Special Contact PMI for Raytracing

The PMI for raytracing allows you to access and change the parameters of a ray at special contacts. These contacts are drawn in the same manner as electrodes and thermodes (see [Boundary Condition for Raytracing on page 520](#)). The raytrace PMI is specified in the RayTraceBC section of the command file:

```
RayTraceBC {...
    { Name = "pmi_contact"
      PMIModel = "pmi_modelname"
    }
}
```

The name of "pmi_contact" must match the contact name in the device. Any ray that hits this special contact invokes a call to this PMI. This raytrace PMI works only with the raytracer (see [Raytracer on page 510](#)) and the complex refractive index model (see [Complex Refractive Index Model on page 496](#)).

NOTE When using multithreading of the raytracer, you must carefully design the PMI code to be thread safe. This means that global variables should not be used since multiple threads may change the global variables at the same time, leading to confusion in the user PMI code at run-time.

Dependencies

The raytrace PMI depends on the following variables:

wavelength	Wavelength [cm]
incident_angle	Incident angle [radian]
*incident_dirvec	Direction vector of incident ray
*polarvec	Polarization vector of incident ray
*normalvec	Normal vector to surface of impingement from region 1 to region 2
*intersectpoint	Intersection position vector [cm]
*region1_name	Name of region 1 (string)
*region2_name	Name of region 2 (string)
n1_real	Real part of refractive index 1
n1_imag	Imaginary part of refractive index 1
n2_real	Real part of refractive index 2
n2_imag	Imaginary part of refractive index 2
reflected_angle	Reflected angle [radian]
transmitted_angle	Transmitted angle [radian]
*reflected_dirvec	Direction vector of reflected ray
*transmitted_dirvec	Direction vector of transmitted ray
*reflected_startposition	Starting position vector of reflected ray [cm]
*transmitted_startposition	Starting position vector of transmitted ray [cm]
R_TE	Power TE reflection coefficient
T_TE	Power TE transmission coefficient
R_TM	Power TM reflection coefficient
T_TM	Power TM transmission coefficient

To obtain the rate intensity (units of s^{-1}) carried by the ray, you need to compute the square of the length of `*polarvec`. This rate intensity includes all previous absorptions sustained by the ray when it traversed absorptive regions of the device, and it can be used directly to compute the amount of optical generation (with units of s^{-1}). However, for raytracing used in LED simulations, the square of the length of `*polarvec` gives only a relative intensity value because the polarization vector of the head ray of the raytree is initialized to a length of 1.0.

If the reflection or transmission coefficients are equal to zero, no reflected or transmitted rays are created in the raytracing process.

With this PMI model, you can change the following variables:

<code>reflected_angle</code>	Reflected angle [radian]
<code>transmitted_angle</code>	Transmitted angle [radian]
<code>*reflected_dirvec</code>	Direction vector of reflected ray
<code>*transmitted_dirvec</code>	Direction vector of transmitted ray
<code>*reflected_startposition</code>	Starting position vector of reflected ray [cm]
<code>*transmitted_startposition</code>	Starting position vector of transmitted ray [cm]
<code>R_TE</code>	Power TE reflection coefficient
<code>T_TE</code>	Power TE transmission coefficient
<code>R_TM</code>	Power TM reflection coefficient
<code>T_TM</code>	Power TM transmission coefficient

If you want to change the direction vector, angle, or position vector of the reflected or transmitted ray, the respective flags must be set to TRUE:

- `is_reflectedangle_changed`
- `is_reflecteddirvec_changed`
- `is_transmittedangle_changed`
- `is_transmitteddirvec_changed`
- `is_reflected_new_startposition`
- `is_transmitted_new_startposition`

However, no flags are needed if you want to change the power reflection or transmission coefficients.

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_EXTERNAL PMI-RayTraceBoundary : public PMI-Vertex-Interface {
public:
    PMI-RayTraceBoundary (const PMI-Environment& env);
    virtual ~PMI-RayTraceBoundary();

    // methods to be implemented by user
    virtual void Compute-BoundaryParameters
        (// Non-changeable quantities
         const double wavelength,           // wavelength [cm]
         const double incident_angle,       // incident angle
         const double* incident_dirvec,     // dir vector of incident ray
         const double* polarvec,           // polar vec of incident ray
         const double* normalvec,          // normal to impingement
         const double* intersectpoint,      // intersection point
         const char* region1_name,         // name of region 1
         const char* region2_name,         // name of region 2
         const double n1_real,             // real part of refr index 1
         const double n1_imag,             // imag part of refr index 1
         const double n2_real,             // real part of refr index 2
         const double n2_imag,             // imag part of refr index 2
         // User changeable quantities
         bool& is_reflectedangle_changed,  // is refl angle changed?
         bool& is_reflecteddirvec_changed, // is reflected dir changed?
         bool& is_transmittedangle_changed, // is transm angle changed?
         bool& is_transmitteddirvec_changed, // is transm dir changed?
         bool& is_reflected_new_startposition, // is refl pos changed?
         bool& is_transmitted_new_startposition, // is transm pos changed?
         double& reflected_angle,           // reflected angle
         double& transmitted_angle,         // transmitted angle
         double* reflected_dirvec,          // dir vec of reflected ray
         double* transmitted_dirvec,        // dir vec of transmitted ray
         double* reflected_startposition,   // start pos of reflected ray
         double* transmitted_startposition, // start pos of transm ray
         double& R_TE,                     // power TE reflection coeff.
         double& T_TE,                     // power TE transmission coeff.
         double& R_TM,                     // power TM reflection coeff.
         double& T_TM                      // power TM transmission coeff.
        ) = 0;

    // Auxiliary functions for users
    void ReadComplexRefractiveIndex(std::string materialname,
        double wavelength, // in microns
        double& n,
```

```
double& k);
private:
const PMI_Environment* thisenv;
};
```

An internal auxiliary function called `ReadComplexRefractiveIndex(...)` has been implemented to allow you to compute the complex refractive index of any material at a desired wavelength. The constraint is that the material must be defined in the parameter file.

Example: Assessing and Modifying a Ray

The following example shows how to access the information about a ray that intersects the special raytrace PMI contact, and how you can change the information of the ray:

```
class Dummy_RayTraceBoundary : public PMI_RayTraceBoundary {
private:
    // short ind_field; // field index for optical generation
    // double shape; // transient curve shape
    // double G1, G2, G3, T1, T2, T3, T4; // const for shapes
    int count;
    double d1;
public:
    Dummy_RayTraceBoundary (const PMI_Environment& env);
    ~Dummy_RayTraceBoundary ();

    void Compute_BoundaryParameters
        (const double wavelength,           // wavelength [cm]
         const double incident_angle,        // incident angle
         const double* incident_dirvec,      // dir vec of incident ray
         const double* polarvec,            // polar vec of incident ray
         const double* normalvec,           // normal to impingement
         const double* intersectpoint,      // intersection point
         const char* region1_name,          // name of region 1
         const char* region2_name,          // name of region 2
         const double n1_real,              // real part of refr index 1
         const double n1_imag,              // imag part of refr index 1
         const double n2_real,              // real part of refr index 2
         const double n2_imag,              // imag part of refr index 2
         // User changeable quantities
         bool& is_reflectedangle_changed,   // is refl angle changed?
         bool& is_reflecteddirvec_changed,  // is refl dir changed?
         bool& is_transmittedangle_changed,  // is transm angle changed?
         bool& is_transmitteddirvec_changed, // is transm dir changed?
         bool& is_reflected_new_startposition, // is refl pos changed?
         bool& is_transmitted_new_startposition, // is transm pos changed?
         double& reflected_angle,           // reflected angle
```

41: Physical Model Interface

Special Contact PMI for Raytracing

```
double& transmitted_angle,           // transmitted angle
double* reflected_dirvec,           // dir vec of reflected ray
double* transmitted_dirvec,         // dir vec of transmitted ray
double* reflected_startposition,     // start pos of reflected ray
double* transmitted_startposition,   // start pos of transm ray
double& R_TE,                       // power TE reflection coeff.
double& T_TE,                       // power TE transmission coeff.
double& R_TM,                       // power TM reflection coeff.
double& T_TM                       // power TM transmission coeff.
);
};

Dummy_RayTraceBoundary::
Dummy_RayTraceBoundary (const PMI_Environment& env) :
    PMI_RayTraceBoundary (env)
{
    printf("PMI: initializing ray trace PMI\n");
}

Dummy_RayTraceBoundary::
~Dummy_RayTraceBoundary ()
{
}

void Dummy_RayTraceBoundary::
Compute_BoundaryParameters (
    const double wavelength,
    const double incident_angle,
    const double* incident_dirvec,
    const double* polarvec,
    const double* normalvec,
    const double* intersectpoint,
    const char* region1_name,
    const char* region2_name,
    const double n1_real,
    const double n1_imag,
    const double n2_real,
    const double n2_imag,
    // User changeable quantities
    bool& is_reflectedangle_changed,
    bool& is_reflecteddirvec_changed,
    bool& is_transmittedangle_changed,
    bool& is_transmitteddirvec_changed,
    bool& is_reflected_new_startposition,
    bool& is_transmitted_new_startposition,
    double& reflected_angle,
    double& transmitted_angle,
    double* reflected_dirvec,
```

```

    double* transmitted_dirvec,
    double* reflected_startposition,
    double* transmitted_startposition,
    double& R_TE,
    double& T_TE,
    double& R_TM,
    double& T_TM
)
{
    // Ray goes from region 1 to region 2.
    // PMI contact is the interface between regions 1 and 2.
    printf("Region 1: Name=%s, Refractive Index = %e + i%e\n",
        region1_name, n1_real, n1_imag);
    printf("Angles: Incident=%lf, Reflected=%lf, Transmitted=%lf\n",
        incident_angle, reflected_angle, transmitted_angle);
    printf("Wavelength = %e [cm]\n", wavelength);
    printf("Incident Ray Direction=(%e,%e,%e)\n",
        incident_dirvec[0], incident_dirvec[1], incident_dirvec[2]);
    printf("Power Coefficients: R_TE=%e, T_TE=%e, R_TM=%e, T_TM=%e\n",
        R_TE, T_TE, R_TM, T_TM);

    // For example, change reflected direction to (1,2,3)
    is_reflecteddirvec_changed = TRUE;
    reflected_dirvec[0] = 1.0;
    reflected_dirvec[1] = 2.0;
    reflected_dirvec[2] = 3.0;
}

extern "C" {
    PMI_RayTraceBoundary* new_PMI_RayTraceBoundary (const PMI_Environment& env)
    { return new Dummy_RayTraceBoundary (env);
    }
}
}

```

Spatial Distribution Function

The spatial distribution function $R(w, l, E)$ can be defined by a PMI (see [Heavy Ions on page 574](#)).

The name of the PMI must be specified in the Physics section of the command file as follows:

```
Physics {  
    HeavyIon(  
        SpatialShape = PMI_shape_name  
    )  
}
```

or:

```
Physics {  
    HeavyIon(  
        SpatialShape = PMI_shape_name(Energy = value) # [eV]  
    )  
}
```

Dependencies

The spatial distribution function $R(w, l, E)$ depends on the following variables:

w	Radius defined as the perpendicular distance from the track [cm]
l	Coordinate along the track [cm]
E	Energy of heavy ion [eV]

The PMI model must compute the following result:

R	The value of the spatial distribution $R(w, l, E)$ [1]
-----	--

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_SpatialDistributionFunction: public PMI_Vertex_Interface {
// * The spatial distribution function:
// *
// *   R(w, l, E)
// *
// * where
// *   - l is the coordinate along the particle path [um];
// *   - w is radial coordinate orthogonal to l [um];
// *   - E is energy of heavy ion [eV];

public:
    PMI_SpatialDistributionFunction (const PMI_Environment& env,
                                     const char* IonType);

    virtual ~PMI_SpatialDistributionFunction ();

// user-defined name of heavy ion (see Using Alpha Particle Model on page 572)
const char* GetHeavyIonType () const;

// methods to be implemented by user
    virtual void Compute_R (double& R, const double w, const double l = -1.,
                           const double E = -1.) = 0;

};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_SpatialDistributionFunction*
    new_PMI_SpatialDistributionFunction_func
    (const PMI_Environment& env, const char* HeavyIonName);
new_PMI_SpatialDistributionFunction_func new_PMI_SpatialDistributionFunction;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_SpatialDistributionFunction_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float w; // radius (perpendicular distance from track)
```

41: Physical Model Interface

Spatial Distribution Function

```
    pmi_float l; // coordinate along track
    pmi_float E; // energy of heavy ion
};

class Output {
public:
    pmi_float R; // spatial distribution
};

PMI_SpatialDistributionFunction_Base (const PMI_Environment& env,
                                     const char* name);
virtual ~PMI_SpatialDistributionFunction_Base ();

const char* GetHeavyIonType () const;

virtual void compute (const Input& input, Output& output) = 0;
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_SpatialDistributionFunction_Base*
new_PMI_SpatialDistributionFunction_Base_func
(const PMI_Environment& env, const char* HeavyIonName);
extern "C" new_PMI_SpatialDistributionFunction_Base_func
new_PMI_SpatialDistributionFunction_Base;
```

Example: Gaussian Spatial Distribution Function

The built-in Gaussian spatial distribution function (see [Heavy Ions on page 574](#)) can also be implemented as a PMI model:

```
#include "PMIModels.h"

class Gaussian_SpatialDistributionFunction : public
PMI_SpatialDistributionFunction {

public:
    Gaussian_SpatialDistributionFunction (const PMI_Environment& env,
        const char* HeavyIonName);
    ~Gaussian_SpatialDistributionFunction ();

    void Compute_R
        (double& R, const double w, const double l, const double E);
};
```

```
Gaussian_SpatialDistributionFunction::
Gaussian_SpatialDistributionFunction (const PMI_Environment& env, const char*
HeavyIonName) :
    PMI_SpatialDistributionFunction (env, HeavyIonName)
{}

Gaussian_SpatialDistributionFunction::
~Gaussian_SpatialDistributionFunction ()
{}

void Gaussian_SpatialDistributionFunction::
Compute_R(double& R, const double w, const double l, const double E)
{
    // the unit w,l is [cm]
    // the unit E is [eV] (in this implementation not used)
    // R(w,l,E) = exp( -(w/wt(l))^2 )

    double wt = 1.e-4; // scaling factor 1um = 1.e-4cm
    double x = w/wt;

    R = exp(-x*x);
}

extern "C"
PMI_SpatialDistributionFunction* new_PMI_SpatialDistributionFunction
(const PMI_Environment& env, const char* HeavyIonName)
{
    return new Gaussian_SpatialDistributionFunction (env, HeavyIonName);
}
```

Metal Resistivity

The metal resistivity $\rho = 1/\sigma$ can be defined by a PMI (see [Transport in Metals on page 239](#)).

The name of the PMI must be specified in the Physics section of the command file as follows:

```
Physics {
    MetalResistivity (pmi_model)
}
```

Dependencies

The metal resistivity depends on the variables:

t	Lattice temperature [K]
f	Absolute value of the electric field [Vcm^{-1}]

The PMI model must compute the following result:

Resist	Metal resistivity [Ωcm]
--------	---

In the case of the standard interface, the following derivatives must be computed as well:

dResistdt	Derivative of Resist with respect to t [$\Omega\text{K}^{-1}\text{cm}$]
dResistdf	Derivative of Resist with respect to f [$\Omega\text{V}^{-1}\text{cm}^2$]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class MetalResistivity : public PMI_MetalResistivity {
public:
    MetalResistivity (const PMI_Environment& env);
    virtual ~MetalResistivity ();

    virtual void Compute_Resist
        (const double t,      // lattice temperature
         const double f,      // absolute value of electric field
         double& Resist);      // metal resistivity

    virtual void Compute_dResistdt
        (const double t,      // lattice temperature
         const double f,      // absolute value of electric field
         double& dResistdt);  // derivative of metal resistivity
                                // with respect to lattice temperature

    virtual void Compute_dResistdf
        (const double t,      // lattice temperature
         const double f,      // absolute value of electric field
```

```
double& dResistdf); // derivative of metal resistivity
// with respect to electric field
};
```

The following virtual constructor must be implemented:

```
typedef PMI_MetalResistivity* new_PMI_MetalResistivity_func
(const PMI_Environment& env);
extern "C" new_PMI_MetalResistivity_func new_PMI_MetalResistivity;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_MetalResistivity_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t;    // lattice temperature
        pmi_float f;    // absolute value of the electric field
    };

    class Output {
    public:
        pmi_float Resist; // metal resistivity
    };

    PMI_MetalResistivity_Base (const PMI_Environment& env);
    virtual ~PMI_MetalResistivity_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The following virtual constructor must be implemented:

```
extern "C"
PMI_MetalResistivity_Base* new_PMI_MetalResistivity_Base
(const PMI_Environment& env);
```

Example: Linear Metal Resistivity

The following C++ code implements linear metal resistivity:

```
#include "PMIModels.h"

class MetalResistivity : public PMI_MetalResistivity
{
private:
    double R0, AlphaT;

public:
    MetalResistivity (const PMI_Environment& env);
    ~MetalResistivity ();

    void Compute_Resist
        (const double t,      // lattice temperature
         const double f,      // absolute value of electric field
         double& Resist);     // metal resistivity

    void Compute_dResistdt
        (const double t,      // lattice temperature
         const double f,      // absolute value of electric field
         double& dResistdt);  // derivative of metal resistivity
                               // with respect to lattice temperature

    void Compute_dResistdf
        (const double t,      // lattice temperature
         const double f,      // absolute value of electric field
         double& dResistdf);  // derivative of metal resistivity
                               // with respect to electric field

};

MetalResistivity::
MetalResistivity (const PMI_Environment& env) :
    PMI_MetalResistivity (env)
{ // Gold values
    R0      = InitParameter ("R0",      2.0400e-06); // [ohm*cm]
    AlphaT = InitParameter ("AlphaT",  4.0000e-03); // [1/K]
}

MetalResistivity::
~MetalResistivity ()
{}

void MetalResistivity::
Compute_Resist (const double t, const double f, double& Resist)
```

```
{
    Resist = R0*( 1 + AlphaT*( t - 273 ) );
}

void MetalResistivity::
Compute_dResistdt (const double t, const double f, double& dResistdt)
{
    dResistdt = R0*AlphaT;
}

void MetalResistivity::
Compute_dResistdf (const double t, const double f, double& dResistdf)
{
    dResistdf = 0;
}

extern "C"
PMI_MetalResistivity* new_PMI_MetalResistivity
(const PMI_Environment& env)
{
    return new MetalResistivity(env);
}
```

Heat Generation Rate

The total heat generation rate is the term on the right-hand side of [Eq. 69, p. 209](#). You can specify an additional term `pmi_Heat` for a given material or region:

$$\text{Total_Heat} = \text{existing_Heat} + \text{pmi_Heat}$$

The name of the PMI must be specified in the new subsection `HeatSource(pmi_model)` in the Physics section of the command file as follows:

```
Physics (Material = "Silicon") {
    HeatSource (pmi_Heat_Si)
}
```

To plot the `pmi_Heat` values, specify the keyword `pmiHeat` in the Plot section of the command file.

Dependencies

The heat generation depends on the variables:

n	Electron density [cm^{-3}]
p	Hole density [cm^{-3}]
t	Lattice temperature [K]
f	Electric field vector [Vcm^{-1}]
g	Gradient temperature [Kcm^{-1}]

The PMI model must compute the following results:

Heat	Heat generation rate [Wcm^{-3}]
------	--

In the case of the standard interface, the following derivatives must be computed as well:

dHeat_dn	Derivative of Heat with respect to n [W]
dHeat_dp	Derivative of Heat with respect to p [W]
dHeat_dt	Derivative of Heat with respect to t [$\text{Wcm}^{-3}\text{K}^{-1}$]
dHeat_df	Derivative of Heat with respect to each component f [$\text{Wcm}^{-2}\text{V}^{-1}$]
dHeat_dg	Derivative of Heat with respect to each component g [$\text{Wcm}^{-2}\text{K}^{-1}$]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_Heat_Generation : public PMI_Vertex_Interface {  
  
public:  
    PMI_Heat_Generation (const PMI_Environment& env);  
    virtual ~PMI_Heat_Generation ();  
  
    // methods to be implemented by user  
    virtual void Compute_Heat  
        (const double n,           // electron density  
         const double p,           // hole density  
         const double t,           // lattice temperature
```



```

    const double f[3],          // electric field vector
    const double g[3],          // gradient of temperature
    double& heat) = 0;          // heat generation rate

virtual void Compute_dHeat_dn
(const double n,                // electron density
 const double p,                // hole density
 const double t,                // lattice temperature
 const double f[3],            // electric field vector
 const double g[3],            // gradient of temperature
 double& dHeat_dn) = 0;         // derivative of heat with respect to n

virtual void Compute_dHeat_dp
(const double n,                // electron density
 const double p,                // hole density
 const double t,                // lattice temperature
 const double f[3],            // electric field vector
 const double g[3],            // gradient of temperature
 double& dHeat_dp) = 0;         // derivative of heat with respect to p

virtual void Compute_dHeat_dt
(const double n,                // electron density
 const double p,                // hole density
 const double t,                // lattice temperature
 const double f[3],            // electric field vector
 const double g[3],            // gradient of temperature
 double& dHeat_dt) = 0;         // derivative of heat with respect to t

virtual void Compute_dHeat_df
(const double n,                // electron density
 const double p,                // hole density
 const double t,                // lattice temperature
 const double f[3],            // electric field vector
 const double g[3],            // gradient of temperature
 double dHeat_df[3]) = 0;       // derivative of heat with respect to f

virtual void Compute_dHeat_dg
(const double n,                // electron density
 const double p,                // hole density
 const double t,                // lattice temperature
 const double f[3],            // electric field vector
 const double g[3],            // gradient of temperature
 double dHeat_dg[3]) = 0;       // derivative of heat with respect to g
};

```

41: Physical Model Interface

Heat Generation Rate

The following virtual constructor must be implemented:

```
typedef PMI_Heat_Generation* new_PMI_Heat_Generation_func  
(const PMI_Environment& env);  
extern "C" new_PMI_Heat_Generation_func new_PMI_HeatGeneration;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_HeatGeneration_Base : public PMI_Vertex_Base {  
  
public:  
    class Input : public PMI_Vertex_Input_Base {  
    public:  
        pmi_float n;        // electron density  
        pmi_float p;        // hole density  
        pmi_float t;        // lattice temperature  
        pmi_float f[3];     // electric field vector  
        pmi_float g[3];     // gradient of temperature  
    };  
  
    class Output {  
    public:  
        pmi_float heat; // heat generation rate  
    };  
  
    PMI_HeatGeneration_Base (const PMI_Environment& env);  
    virtual ~PMI_HeatGeneration_Base ();  
  
    virtual void compute (const Input& input, Output& output) = 0;  
  
};
```

The prototype for the virtual constructor is given as:

```
typedef PMI_HeatGenerationFunction_Base*  
new_PMI_HeatGenerationFunction_Base_func  
(const PMI_Environment& env);  
extern "C" new_PMI_HeatGenerationFunction_Base_func  
new_PMI_HeatGenerationFunction_Base;
```

Example: Dependency on Electric Field and Gradient of Temperature

In the following example, heat generation has a linear dependency on the electric field and the gradient of temperature:

```
// Heat = kappa*(E, gradT)*1[1/V]
//   where
//   E - electric field vector,
//   gradT - gradient of temperature.
//
// The 1D equation
//   -div(kappa*grad(T(x))) = Heat, 0 <= x <= L
//   T(0) = T0,
//   T(L) = T1,
// has the exact solution:
//   T(x) = (T1-T0)*(exp(-E*x)-1)/(exp(-E*L)-1) + T0

class HeatGeneration : public PMI_HeatGeneration_Base
{
private:
    double kappa;

public:
    HeatGeneration (const PMI_Environment& env);
    ~HeatGeneration ();
    void compute (const Input& input, Output& output);
};

HeatGeneration::HeatGeneration (const PMI_Environment& env) :
    PMI_HeatGeneration_Base (env)
{
    kappa = InitParameter("kappa", 0.01); // [W/(K*cm)]
}

HeatGeneration::~HeatGeneration () {}

void HeatGeneration::compute (const Input& input, Output& output)
{
    const pmi_float (&f)[3] = input.f;
    const pmi_float (&g)[3] = input.g;

    output.heat = kappa*(f[0]*g[0] + f[1]*g[1] + f[2]*g[2]);
}
```

```
extern "C"
PMI_HeatGeneration_Base* new_PMI_HeatGeneration_Base
(const PMI_Environment& env)
{ return new HeatGeneration (env);}
```

Thermoelectric Power

The thermoelectric powers P_n and P_p in semiconductors can be defined by a PMI (see [Thermoelectric Power \(TEP\) on page 749](#)).

The name of the PMI can be specified regionwise, materialwise, or globally in the Physics section of the command file as follows:

```
Physics(Material="Silicon") {
    TEPower(pmi_model)
}
```

Dependencies

The semiconductor TEPs may depend on the following variables:

t	Lattice temperature [K]
dens	Carrier density [cm^{-3}]

The PMI model must compute the following results:

power	Thermoelectric power [VK^{-1}]
-------	---

In the case of the standard interface, the following derivatives must be computed as well:

dpowerdt	Derivative of power with respect to t [VK^{-2}]
dpowerddens	Derivative of power with respect to dens [$\text{VK}^{-1}\text{cm}^{-3}$]

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_ThermoElectricPower : public PMI_Vertex_Interface {

public:
    PMI_ThermoElectricPower(const PMI_Environment& env);
    virtual ~PMI_ThermoElectricPower ();

    virtual void Compute_power(
        const double t,          // lattice temperature
        const double dens,       // carrier density
        double& power);          // thermoelectric power

    virtual void Compute_dpowerdt(
        const double t,          // lattice temperature
        const double dens,       // carrier density
        double& dpowerdt);       // derivative of thermoelectric power
                                   // with respect to lattice temperature

    virtual void Compute_dpowerddens(
        const double t,          // lattice temperature
        const double dens,       // carrier density
        double& dpowerddens);    // derivative of thermoelectric power
                                   // with respect to carrier density
};
```

The following virtual constructor must be implemented:

```
typedef PMI_ThermoElectricPower* new_PMI_ThermoElectricPower_func
    (const PMI_Environment& env);
extern "C" new_PMI_ThermoElectricPower_func new_PMI_e_ThermoElectricPower;
extern "C" new_PMI_ThermoElectricPower_func new_PMI_h_ThermoElectricPower;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_ThermoElectricPower_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t;          // lattice temperature
        pmi_float dens;       // carrier density
    };
};
```

41: Physical Model Interface

Thermoelectric Power

```
};

class Output {
public:
    pmi_float power; // thermoelectric power
};

PMI_ThermoElectricPower_Base (const PMI_Environment& env);
virtual ~PMI_ThermoElectricPower_Base ();

virtual void compute (const Input& input, Output& output) = 0;
};
```

The following virtual constructor must be implemented:

```
extern "C"
PMI_ThermoElectricPower_Base* new_PMI_ThermoElectricPower_Base
(const PMI_Environment& env);
```

Example: Analytic TEP

The following C++ code implements an analytic TEP model (see [Thermoelectric Power \(TEP\) on page 749](#)):

```
#include "PMIModels.h"

namespace {
    // pi
    double pi = 3.141592654;
    // electron mass in kg
    double m0 = 9.109534e-31;
    // Planck's constant in J*s
    double h_planck = 6.626176e-34;
    // Boltzmann constant in J/K
    double kB = 1.380662e-23;
    // electron charge in C
    double e0 = 1.602192e-19;

    double pow_1_5 (double x) {
        return (x==1) ? 1 : x * sqrt (x);
    }

    double Compute_NB_By3_2 (double m_r) {
        double val = 2 * pow_1_5(2 * pi * kB / h_planck * m0 / h_planck) / 1e6;
        val *= pow_1_5(m_r);
        return val;
    }
}
```

```

}

class AnalyticalTEP_ThermoElectricPower : public PMI_ThermoElectricPower
{
private:
    double k_c, s_c;
    // relative effective mass
    double m_r;
    int sign;

public:
    AnalyticalTEP_ThermoElectricPower(const PMI_Environment& env);
    ~AnalyticalTEP_ThermoElectricPower ();

    void Compute_power
        (const double t,          // lattice temperature
         const double dens,       // carrier density
         double& power);          // thermoelectric power

    void Compute_dpowerdt
        (const double t,          // lattice temperature
         const double dens,       // carrier density
         double& dpowerdt);       // derivative of thermoelectric power
                                   // with respect to lattice temperature

    void Compute_dpowerddens
        (const double t,          // lattice temperature
         const double dens,       // carrier density
         double& dpowerddens);    // derivative of thermoelectric power
                                   // with respect to carrier density
};

AnalyticalTEP_ThermoElectricPower::
AnalyticalTEP_ThermoElectricPower (const PMI_Environment& env) :
    PMI_ThermoElectricPower (env)
{
}

AnalyticalTEP_ThermoElectricPower::
~AnalyticalTEP_ThermoElectricPower ()
{
}

void AnalyticalTEP_ThermoElectricPower::
Compute_power (const double t, const double dens, double& power)
{
    double NB = Compute_NB_By3_2(m_r) * pow_1_5(t);
    if(sign < 0)

```

41: Physical Model Interface

Thermoelectric Power

```
        power = k_c * (log(dens/NB) + s_c - 2.5);
    else
        power = k_c * (log(NB/dens) - s_c + 2.5);
    power *= kB/e0;
}

void AnalyticalTEP_ThermoElectricPower::
Compute_dpowerdt (const double t, const double dens, double& dpowerdt)
{
    dpowerdt = sign * kB/e0 * k_c * 1.5 / t;
}

void AnalyticalTEP_ThermoElectricPower::
Compute_dpowerddens (const double t, const double dens, double& dpowerddens)
{
    dpowerddens = -sign * kB/e0 * k_c / dens;
}

class AnalyticalTEP_e_ThermoElectricPower :
public AnalyticalTEP_ThermoElectricPower
{
    AnalyticalTEP_e_ThermoElectricPower(const PMI_Environment& env);
    ~AnalyticalTEP_e_ThermoElectricPower ();
};

AnalyticalTEP_e_ThermoElectricPower::
AnalyticalTEP_e_ThermoElectricPower(const PMI_Environment& env) :
AnalyticalTEP_ThermoElectricPower(env) {
    // default values
    k_c = InitParameter("k_c_e", 1);
    s_c = InitParameter("s_c_e", 1);
    m_r = InitParameter("m_r_e", 1);
    sign = -1;
}

AnalyticalTEP_h_ThermoElectricPower::
AnalyticalTEP_h_ThermoElectricPower(const PMI_Environment& env) :
AnalyticalTEP_ThermoElectricPower(env) {
    // default values
    k_c = InitParameter("k_c_h", 1);
    s_c = InitParameter("s_c_h", 1);
    m_r = InitParameter("m_r_h", 1);
    sign = 1;
}

extern "C"
PMI_ThermoElectricPower* new_PMI_e_ThermoElectricPower
(const PMI_Environment& env)
```



```
{
    return new AnalyticalTEP_e_ThermoElectricPower(env);
}

extern "C"
PMI_ThermoElectricPower* new_PMI_h_ThermoElectricPower
(const PMI_Environment& env)
{
    return new AnalyticalTEP_h_ThermoElectricPower(env);
}
```

Metal Thermoelectric Power

The metal thermoelectric power P in metals can be defined by a PMI (see [Thermoelectric Power \(TEP\) on page 749](#)).

The name of the PMI can be specified regionwise, materialwise, or globally in the Physics section of the command file as follows:

```
Physics(Material="Copper") {
    MetalTEPower(pmi_model)
}
```

Dependencies

The metal thermoelectric power may depend on the following variables:

t	Lattice temperature [K]
qf	Quasi-Fermi potential [V]
f	Electric field vector [Vcm^{-1}]

The PMI model must compute the following result:

power	Thermoelectric power [VK^{-1}]
-------	---

In the case of the standard interface, the following derivatives must be computed as well:

dpowerdt	Derivative of power with respect to t [VK^{-2}]
dpowerdqf	Derivative of power with respect to qf [K^{-1}]

41: Physical Model Interface

Metal Thermoelectric Power

`dpowerdf` Derivative of power with respect to each component of $\mathbf{f}$ [cmK^{-1}],
a vector with up to three entries

Standard C++ Interface

The following base class is declared in the file `PMIModels.h`:

```
class PMI_MetalThermoElectricPower : public PMI_Vertex_Interface {

public:
    PMI_MetalThermoElectricPower(const PMI_Environment& env);
    virtual ~MetalPMI_ThermoElectricPower ();

    virtual void Compute_power(
        const double t,          // lattice temperature
        const double qf,         // quasi-Fermi potential
        const double f[3]        // electric field vector
        double& power);          // thermoelectric power

    virtual void Compute_dpowerdt(
        const double t,          // lattice temperature
        const double qf,         // quasi-Fermi potential
        const double f[3]        // electric field vector
        double& dpowerdt);       // derivative of thermoelectric power
                                   // with respect to lattice temperature

    virtual void Compute_dpowerdqf(
        const double t,          // lattice temperature
        const double qf,         // quasi-Fermi potential
        const double f[3]        // electric field vector
        double& dpowerdqf);      // derivative of thermoelectric power
                                   // with respect to quasi-Fermi potential

    virtual void Compute_dpowerdf(
        const double t,          // lattice temperature
        const double qf,         // quasi-Fermi potential
        const double f[3]        // electric field vector
        double& dpowerdf[3]);    // derivative of thermoelectric power
                                   // with respect to electric field

};
```

The following virtual constructor must be implemented:

```
typedef PMI_MetalThermoElectricPower* new_PMI_MetalThermoElectricPower_func
(const PMI_Environment& env);
extern "C" new_PMI_MetalThermoElectricPower_func
new_PMI_MetalThermoElectricPower;
```

Simplified C++ Interface

The following base class is declared in the file `PMI.h`:

```
class PMI_MetalThermoElectricPower_Base : public PMI_Vertex_Base {

public:
    class Input : public PMI_Vertex_Input_Base {
    public:
        pmi_float t;           // lattice temperature
        pmi_float qf;          // quasi-Fermi potential
        pmi_float f[3];        // electric field vector
    };

    class Output {
    public:
        pmi_float power; // thermoelectric power
    };

    PMI_MetalThermoElectricPower_Base (const PMI_Environment& env);
    virtual ~PMI_MetalThermoElectricPower_Base ();

    virtual void compute (const Input& input, Output& output) = 0;
};
```

The following virtual constructor must be implemented:

```
extern "C"
PMI_MetalThermoElectricPower_Base* new_PMI_MetalThermoElectricPower_Base
(const PMI_Environment& env);
```

Example: Linear Field Dependency of Metal TEP

The following C++ code implements a metal TEP PMI depending linearly on two of the electric field components:

```
#include "PMIModels.h"

namespace {
    // Boltzmann constant in J/K
    double kB = 1.380662e-23;
    // electron charge in C
    double e0 = 1.602192e-19;
}

class MetalThermoElectricPower : public PMI_MetalThermoElectricPower
```

41: Physical Model Interface

Metal Thermoelectric Power

```
{
    private:
        double alpha, beta;

    public:
        MetalThermoElectricPower(const PMI_Environment& env);
        ~MetalThermoElectricPower ();

        void Compute_power
            (const double t,          // lattice temperature
             const double qf,         // quasi-Fermi potential
             const double f[3]        // electric field vector
             double& power);          // thermoelectric power

        void Compute_dpowerdt
            (const double t,          // lattice temperature
             const double qf,         // quasi-Fermi potential
             const double f[3]        // electric field vector
             double& dpowerdt);       // derivative of thermoelectric power
                                     // with respect to lattice temperature

        void Compute_dpowerdqf
            (const double t,          // lattice temperature
             const double qf,         // quasi-Fermi potential
             const double f[3]        // electric field vector
             double& dpowerdqf);      // derivative of thermoelectric power
                                     // with respect to quasi-Fermi potential

        void Compute_dpowerdf
            (const double t,          // lattice temperature
             const double qf,         // quasi-Fermi potential
             const double f[3]        // electric field vector
             double dpowerdf[3]);     // derivative of thermoelectric power
                                     // with respect to electric field
};

MetalThermoElectricPower::
MetalThermoElectricPower (const PMI_Environment& env) :
    PMI_ThermoElectricPower (env)
{
    // default values
    alpha = InitParameter("alpha",1);
    beta = InitParameter("beta",1e-2);
}

MetalThermoElectricPower::
~MetalThermoElectricPower ()
{
}
```

```

}

void MetalThermoElectricPower::
Compute_power (const double t, const double qf, const double f[3],
               double& power)
{
    power = 1e-6 * 1e-2 * (f[0] + f[1]);
}

void MetalThermoElectricPower::
Compute_dpowerdt (const double t, const double qf, const double f[3],
                  double& dpowerdt)
{
    dpowerdt = 0;
}

void MetalThermoElectricPower::
Compute_dpowerdqf (const double t, const double qf, const double f[3],
                   double& dpowerdqf)
{
    dpowerdqf = 0;
}

void MetalThermoElectricPower::
Compute_dpowerdf (const double t, const double qf, const double f[3],
                  double dpowerdf[3])
{
    dpowerdf[0] = 1e-6 * 1e-2;
    dpowerdf[1] = 1e-6 * 1e-2;
    dpowerdf[2] = 0;
}

extern "C"
PMI_MetalThermoElectricPower* new_PMI_MetalThermoElectricPower
(const PMI_Environment& env)
{
    return new MetalThermoElectricPower(env);
}

```

References

- [1] H. Matsuura, “Influence of Excited States of Deep Acceptors on Hole Concentration in SiC,” in *International Conference on Silicon Carbide and Related Materials (ICSCRM)*, Tsukuba, Japan, pp. 679–682, October 2001.
- [2] P. Y. Yu and M. Cardona, *Fundamentals of Semiconductors: Physics and Materials Properties*, Berlin: Springer, 2nd ed., 1999.
- [3] K. F. Brennan, *The Physics of Semiconductors: With applications to optoelectronic devices*, Cambridge: Cambridge University Press, 1999.

Part VI Appendices

This part of the *Sentaurus Device User Guide* contains the following appendices:

[Appendix A Mathematical Symbols on page 1171](#)

[Appendix B Syntax on page 1175](#)

[Appendix C File-naming Convention on page 1177](#)

[Appendix D Command-Line Options on page 1179](#)

[Appendix E Run-Time Statistics on page 1183](#)

[Appendix F Data and Plot Names on page 1185](#)

[Appendix G Command File Overview on page 1217](#)

APPENDIX A Mathematical Symbols

This appendix contains notational conventions and a list of symbols used in the Sentaurus Device User Guide.

They are listed alphabetically. Non-Latin characters are sorted according to their English translation.

$/$	Division, right binding: $a/bc = a/(bc)$
$\hat{a}$	Unit (3D) vector
$\vec{a}$	(3D) vector
$ a $	Absolute value of a scalar
$ \vec{a} $	Absolute value of a vector $a = \vec{a} $
a^T	Transpose of a vector or a matrix
a^{-1}	Inverse of function or matrix, reciprocal of a scalar
$\vec{a} \cdot \vec{b}$	Inner (dot) product
$\vec{a} \times \vec{b}$	Vector (cross) product
$\vec{a}\vec{b}$	Dyadic product: $(\vec{a}\vec{b})_{ij} = \vec{a}_i\vec{b}_j$
$\langle abc \bar{} \rangle$	Miller indices. a , b , and c are digits; a bar over a digit indicates negation applied to that particular digit.
χ	Electron affinity or thermal resistivity
c_L	Lattice heat capacity
e	Base of natural logarithm
$\vec{E}$	Electric field
E_{bgn}	Bandgap narrowing
E_C	Conduction band energy

A: Mathematical Symbols

$E_{F,n}$	Electron quasi-Fermi energy
$E_{F,p}$	Hole quasi-Fermi energy
E_g	Intrinsic band gap
$E_{g,eff}$	Effective band gap, $E_{g,eff} = E_g - E_{bgn}$
E_V	Valence band energy
ϵ	Absolute dielectric constant of a material
ϵ_0	Dielectric constant of vacuum
$\vec{F}$	Electric field
F_α	Integral of distribution function; for Fermi statistics, Fermi integral of order α
G	Generation rate (does not include recombination)
γ_n	Degeneracy factor for electrons
γ_p	Degeneracy factor for holes
$\hbar$	Planck's constant divided by 2π
i	Imaginary unit, $i^2 = -1$
Im	Imaginary part
$\vec{J}_D$	Displacement current density
$\vec{J}_M$	Current density in metals
$\vec{J}_n$	Electron current density
$\vec{J}_p$	Hole current density
k	Boltzmann constant
κ_L	Lattice thermal conductivity
κ_n	Electron thermal conductivity
κ_p	Hole thermal conductivity

Λ_n	Electron quantum potential
Λ_p	Hole quantum potential
$\ln$	Natural logarithm
m_0	Free electron mass
m_n	Electron density-of-states mass
m_p	Hole density-of-states mass
μ_n	Electron mobility
μ_p	Hole mobility
n	Electron density
$\hat{n}$	Unit normal vector
$N_{A,0}$	Chemically active acceptor concentration
N_A	Ionized acceptor concentration
N_C	Conduction band density-of-states
$N_{D,0}$	Chemically active donor concentration
N_D	Ionized donor concentration
n_i	Intrinsic density (not accounting for bandgap narrowing)
$n_{i,eff}$	Effective intrinsic density (accounting for bandgap narrowing)
N_i	Ionized dopant concentration, $N_i = N_A + N_D$
n_{se}	Single exciton density
N_{tot}	Total doping concentration, $N_{tot} = N_{A,0} + N_{D,0}$
N_V	Valence band density-of-states
p	Hole density
$\vec{P}$	Polarization
P_n	Electron thermoelectric power

A: Mathematical Symbols

P_p	Hole thermoelectric power
Φ_M	Metal Fermi potential
Φ_n	Electron quasi-Fermi potential
Φ_p	Hole quasi-Fermi potential
ϕ	Electrostatic potential
q	Elementary charge
r	Anisotropy factor
R	Recombination rate (does not include generation)
R_{net}	Net recombination rate, $R_{\text{net}} = R - G$
Re	Real part
$\vec{S}_L$	Lattice heat flux density
$\vec{S}_n$	Electron heat flux density
$\vec{S}_p$	Hole heat flux density
T	Lattice temperature
T_n	Electron temperature
T_p	Hole temperature
τ_n	Electron lifetime
τ_p	Hole lifetime
Θ	Unit step function (0 for negative arguments, 1 for positive arguments)
$\vec{v}_n$	Electron drift velocity
$\vec{v}_p$	Hole drift velocity
$\vec{v}_{\text{sat},n}$	Electron saturation velocity
$\vec{v}_{\text{sat},p}$	Hole saturation velocity

APPENDIX B Syntax

The syntax of the command file of Sentaurus Device, and the basic syntactical and lexical conventions are described here.

Sentaurus Device has a hierarchical input syntax. At the lowest level, **device**, **system**, and **solve** information is specified as well as the default and global parameters. Inside each **Device** section, the parameters specific to one device type can be specified.

Inside the **System** section, the real devices are specified or ‘instantiated.’ Here, parameters can be given that are specific to one instantiation of a device. The command file is a collection of specifications used to establish the simulation environment with actions describing which equations must be solved and how they must be solved. The syntax of the command file contains several entry types. All basic command file entries adhere to the syntactical and lexical rules described in [Table 137](#).

Table 137 Entry types in Sentaurus Device

Entry type	Description
Keyword	These are the known names of the command file. They are case insensitive. Therefore, the following keywords are all equivalent: Quasistationary , QuasiStationary , and quasistationary . Most keywords can be abbreviated. The above example can also be written as QuasiStat .
Integer	These are (possibly) signed decimal numbers. The following integers are valid: 123 , -73492 , 0 .
Float	Floating point numbers are compatible with the C language format for floating point numbers. The following floating point numbers are valid: 123 , 123.0 , 1.23e2 , -1.23E2 .
Vector	Vectors in real space are defined depending on the actual dimension. In 3D, a vector is specified by three floating point numbers; in 2D, by two floating point numbers enclosed in parentheses. The floats are separated by commas or spaces. In 1D, one floating point number without parentheses is sufficient. Valid vectors are (1,0,2) , (1e-4, -1e-3) , and 1 .
String	Strings are delimited by quotation marks. They are compatible with the C language format for strings. The following strings are valid: "Vdd" , "output/diode" .
Identifier	These are used to name objects such as nodes, devices, or attributes. They are compatible with the C language format for identifiers. The following identifiers are valid: Vdd , diode , bjt_345 .
Assignment	These are used to set values to keywords. Therefore, the following are valid assignments: Digits=4 , Save="output/diode" .

B: Syntax

Table 137 Entry types in Sentaurus Device

Entry type	Description
Signal	Signals are time dependent, piecewise, linear functions (not to be confused with UNIX signals) that are defined as inputs on the contacts of a device. They are specified as follows: (value0 at time0, value1 at time1, ...value_n at time_n). The following signal is valid: (0 at 0, 1 at 10.0e-9, 1 at 20.0e-9).
List	Lists are collections of keywords, assignments, and complex entries. They are delimited by "(...)" or "{...}". The following lists are valid: { Number=0 Voltage=0 Voltage=(0 at 0, 0 at 2e-8) } { Method=Super Digits=6 Numerically } (MinStep=1e-15 InitialStep=1e-10 Digits=3)
Structured entries	These are parameterized definitions or commands that can have the forms: <keyword> {<keywords>}, <keyword> (<keywords>), or <keyword> (<list>) {<list>}.

APPENDIX C File-naming Convention

The file-naming convention for TCAD tools relevant for Sentaurus Device are described here.

Overview

All strings that represent file names containing a dot (.) within their base name are taken literally. Otherwise, Sentaurus Device extends the given strings with the appropriate extension.

Sentaurus Device expands the extensions for output files by its tool extension `_des`, for example, the extension of a saved file is `_des.sav`. [Table 138](#) summarizes the *extensions* used in Sentaurus Device.

Table 138 Sentaurus Device file-naming convention

File	I/O	Extension
Command	I	<code>_des.cmd</code> , <code>.cmd</code>
Log	O	<code>_des.log</code>
Parameter	I	<code>.par</code>
Geometry/Doping	I	<code>.tdr</code>
Lifetime	I	<code>.tdr</code>
Save	O	<code>_des.sav</code>
Load	I	<code>.sav</code>
Device Plot (grid-based)	O	<code>_des.tdr</code>
Current Plot	O	<code>_des.plt</code>
AC Extraction	O	<code>_ac_des.plt</code>
Montecarlo	I/O	Refer to the <i>Sentaurus Device Monte Carlo User Guide</i> for more information.

During transient simulations, quasistationaries, and continuations, the plot and save files are numbered by a global index.

Compatibility with Old File-naming Convention

To be compatible with the old file-naming conventions, Sentaurus Device tries to find the files according to the current convention and, if the corresponding files are not found, it tries to read the files according to the old convention. [Table 139](#) summarizes the old convention.

Table 139 Old file-naming convention

File	I/O	Extension
Command	I	.in
Output or Log	O	.out
Parameter	I	.par
Geometry	I	.geo, .grd
Doping	I	.dop, .dat
Lifetime	I	.life, .dat
Save	O	.sav
Load	I	.sav
Device Plot (grid-based)	O	.prt
Current Plot	O	.cur
AC Extraction	O	.ac

APPENDIX D **Command-Line Options**

This appendix lists the most useful command-line options available in Sentaurus Device.

Starting Sentaurus Device

To start Sentaurus Device, enter:

```
sdevice [<options>] [<commandfile>]
```

Sentaurus Device appends automatically the corresponding extension to the given command file if necessary. If no command file is specified, Sentaurus Device reads from standard input.

Command-Line Options

Sentaurus Device interprets the following options:

-d	Prints debug information into the debug file. The information printed includes the numeric values of the Jacobian and RHS for each equation at each solution step.
-h	Lists these options and exits.
-i	Prints the initial solution in the save file, and print files specified in the File section of the command file, and exits without performing further computations.
-n	Does not include Newton information in the log file.
-P	Writes the silicon model parameters into a file <code>models.par</code> and exits. This file can be modified and reloaded into Sentaurus Device to make customized changes to physical models and parameters.
-P:<Material>	Writes the model parameters for the given material into a file <code>models.par</code> and exits.

D: Command-Line Options

Command-Line Options

-P:<Material>:<x>	Writes the model parameters for the given material and mole fraction into a file <code>models.par</code> and exits.
-P:<Material>:<x>:<y>	Writes the model parameters for the given material and mole fractions into a file <code>models.par</code> and exits.
-P:All	Writes the model parameters for all materials into a file <code>models.par</code> and exits.
-P:<Material>/<Material>	Writes the model parameters for the given material interface into a file <code>models.par</code> and exits.
-P <commandfile>	Writes the model parameters for the materials and interfaces used in <commandfile> into a file <code>models.par</code> and exits.
-L	Writes the silicon model parameters into the file <code>Silicon.par</code> and exits.
-L:<Material>	Writes a model parameter file <code><Material>.par</code> for the specified material and exits.
-L:<Material>:<x>	Writes the model parameters for the given material and mole fraction into a file <code><Material>.par</code> and exits.
-L:<Material>:<x>:<y>	Writes the model parameters for the given material and mole fractions into a file <code><Material>.par</code> and exits.
-L:All	Writes a separate model parameter file for all materials and exits.
-L:<Material>/<Material>	Writes a model parameter file <code><Material>%<Material>.par</code> for the specified material interface and exits.
-L <commandfile>	Writes model parameter files for all the materials, material interfaces, and electrodes used in <commandfile> and exits.
-M <commandfile>	Writes a parameter file <code>models-M.par</code> for regions with computed mole fraction dependencies.
-r	When used with -L or -P, reads parameters from the material library to generate output.
-q	Quiet mode for output.

-S	Writes the SiC model parameters into a <code>models_SiC.par</code> file and exits. This file can be modified and reloaded into Sentaurus Device to make customized changes to physical models and parameters.
-v	Prints header with version number of Sentaurus Device.
--parameter-names	Prints the names of the parameters from the parameter file that can be ramped. If a command file is also supplied, Sentaurus Device prints the parameters from the command file that can be ramped.
--exit-on-failure	Terminates immediately after a failed solve command.
--field-names	Prints fields and their numeric indices for use in the PMI.
--compiler-version	Prints the version of the C++ compiler that was used to compile Sentaurus Device.
--tcl	Invokes the Tcl interpreter to evaluate the command file.
--verbose	Prints additional diagnostic messages (alternatively, set the environment variable <code>SDEVICE_VERBOSITY</code> to high).

In addition, generic tool options can be found in the relevant section of the installation documentation, *Installing TCAD Products*.

D: Command-Line Options

Command-Line Options

APPENDIX E Run-Time Statistics

This appendix presents information about obtaining run-time statistics from Sentaurus Device.

The sdevicestat Command

The command `sdevicestat` displays some statistics of a previous run of Sentaurus Device based on the information found in its `log` file. For example, the command:

```
sdevicestat test_des.log
```

generates the following statistics:

```
Total number of Newton iterations      : 8
Number of restarts                     : 0
Rhs-time                              : 9.19 % ( 38.80 s )
Jacobian-time                          : 1.43 % ( 6.05 s )
Solve-time                             : 88.76 % ( 374.70 s )
Overhead                               : 0.62 % ( 2.61 s )
Total CPU time (sum of above times)    : 422.16 s
```

`sdevicestat` recognizes whether the `WallClock` keyword has been specified in the `Math` section of the Sentaurus Device command file (see [Parallelization on page 183](#)). In this case, the same simulation on a dual processor machine may produce the following output:

```
Total number of Newton iterations      : 8
Number of restarts                     : 0
Rhs-time                              : 8.57 % ( 20.68 s )
Jacobian-time                          : 1.96 % ( 4.73 s )
Solve-time                             : 88.33 % ( 213.07 s )
Overhead                               : 1.14 % ( 2.74 s )
Total wallclock time (sum of above times): 241.22 s
```

E: Run-Time Statistics

The sdevicestat Command

APPENDIX F Data and Plot Names

This appendix provides information about data and plot names.

Overview

Table 140 on page 1186, Table 141 on page 1212, and Table 142 on page 1216 list the plot names that are recognized in a Plot section of Sentaurus Device (see [Device Plots on page 144](#)) and the data names that are available in the current plot PMI (see [Current Plot File of Sentaurus Device on page 1132](#)). If the plot name is empty, the data name in quotation marks can be used, for example:

```
Plot {  
    "eTemperatureRelaxationTime"  
}
```

Vector data can be plotted by appending /Vector to the corresponding keyword, for example:

```
Plot {  
    ElectricField/Vector  
}
```

Element-based scalar data can be plotted by appending /Element to the corresponding keyword, for example:

```
Plot {  
    eMobility/Element  
}
```

Special vector data can be plotted by appending /SpecialVector to the corresponding keyword, for example:

```
Plot {  
    eSHEDistribution/SpecialVector  
}
```

Tensor data can be plotted by appending /Tensor to the corresponding keyword, for example:

```
Plot {  
    Stress/Tensor  
}
```

NOTE The location *rivertex* refers to region-interface vertices.

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
AbsorbedPhotonDensity	AbsorbedPhotonDensity	vertex	Quantum Yield Models on page 475	$\text{cm}^{-3} \text{s}^{-1}$
AbsorbedPhotonDensityDiffuse	AbsorbedPhotonDensity	vertex	Transfer Matrix Method on page 538	$\text{cm}^{-3} \text{s}^{-1}$
AbsorbedPhotonDensityFromMonochromaticSource	AbsorbedPhotonDensity	vertex	Quantum Yield Models on page 475	$\text{cm}^{-3} \text{s}^{-1}$
AbsorbedPhotonDensityFromSpectrum	AbsorbedPhotonDensity	vertex	Quantum Yield Models on page 475	$\text{cm}^{-3} \text{s}^{-1}$
AbsorbedPhotonDensitySpecular	AbsorbedPhotonDensity	vertex	Transfer Matrix Method on page 538	$\text{cm}^{-3} \text{s}^{-1}$
AccepMinusConcentration		vertex	Doping Specification on page 49	cm^{-3}
AcceptorConcentration	AcceptorConcentration	vertex		cm^{-3}
AlphaChargeDensity	AlphaCharge	vertex	Alpha Particles on page 572	cm^{-3}
AlphaGeneration		vertex	G^{Alpha} , Eq. 593, p. 573	$\text{cm}^{-3} \text{s}^{-1}$
AntimonyActiveConcentration		vertex	Doping Specification on page 49	cm^{-3}
AntimonyConcentration	AntimonyConcentration	vertex	Sb, Doping Specification on page 49	cm^{-3}
AntimonyPlusConcentration	sbPlus	vertex	Sb⁺, Chapter 13	cm^{-3}
ArsenicActiveConcentration		vertex	Doping Specification on page 49	cm^{-3}
ArsenicConcentration	ArsenicConcentration	vertex	As, Doping Specification on page 49	cm^{-3}
ArsenicPlusConcentration	AsPlus	vertex	As⁺, Chapter 13	cm^{-3}

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
AugerRecombination	AugerRecombination	vertex	R^A , Eq. 366, p. 376	$\text{cm}^{-3} \text{s}^{-1}$
AutoOrientationSmoothing	AutoOrientationSmoothing	vertex	Auto-Orientation Framework on page 75	1
AvalancheGeneration	AvalancheGeneration	vertex	$G^{\parallel}$, Eq. 369, p. 377	$\text{cm}^{-3} \text{s}^{-1}$
Band2BandGeneration	Band2Band	vertex	$G_{\text{net}}^{\text{bb}}$, Band-to-Band Tunneling Models on page 391	$\text{cm}^{-3} \text{s}^{-1}$
BandGap	BandGap	vertex	E_g , Bandgap and Electron-Affinity Models on page 250	eV
BandgapNarrowing	BandGapNarrowing	vertex	E_{bgn} , Bandgap and Electron-Affinity Models on page 250	eV
BeamGeneration	OptBeam	vertex	G^{opt} , Eq. 583, p. 558	$\text{cm}^{-3} \text{s}^{-1}$
BoronActiveConcentration		vertex	Doping Specification on page 49	cm^{-3}
BoronConcentration	BoronConcentration	vertex	B, Doping Specification on page 49	cm^{-3}
BoronMinusConcentration	bMinus	vertex	B', Chapter 13	cm^{-3}
BuiltinPotential		vertex	ϕ_0 , Eq. 96, p. 217	V
CDL1Recombination	CDL1	vertex	R_1 , Coupled Defect Level (CDL) Recombination on page 373	$\text{cm}^{-3} \text{s}^{-1}$
CDL2Recombination	CDL2	vertex	R_2 , Coupled Defect Level (CDL) Recombination on page 373	$\text{cm}^{-3} \text{s}^{-1}$

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
CDLcRecombination	CDL3	vertex	$R - R_1 - R_2$, Coupled Defect Level (CDL) Recombination on page 373	$\text{cm}^{-3} \text{s}^{-1}$
CDLRecombination	CDL	vertex	R , Coupled Defect Level (CDL) Recombination on page 373	$\text{cm}^{-3} \text{s}^{-1}$
	ComplexRefractiveIndex	vertex	Complex Refractive Index Model on page 496	1
ConductionBandEnergy	ConductionBandEnergy	vertex	E_C , Eq. 41, p. 193	eV
ConductionCurrentDensity	ConductionCurrent	vertex	$ J_n + J_p $, Eq. 53, p. 197 or $ J_M $ in metals, Eq. 129, p. 239	Acm^{-2}
	ConversePiezoelectricField	vertex	Chapter 30	1
ConversePiezoelectricFieldXX	ConversePiezoelectricFieldXX	vertex	Components of converse piezoelectric field tensor	1
ConversePiezoelectricFieldXY	ConversePiezoelectricFieldXY			
ConversePiezoelectricFieldXZ	ConversePiezoelectricFieldXZ			
ConversePiezoelectricFieldYY	ConversePiezoelectricFieldYY			
ConversePiezoelectricFieldYZ	ConversePiezoelectricFieldYZ			
ConversePiezoelectricFieldZZ	ConversePiezoelectricFieldZZ			
CurECImACGreenFunction		vertex	Table 104 on page 601	1
CurECReACGreenFunction		vertex	Table 104 on page 601	1
CurETImACGreenFunction		vertex	Table 104 on page 601	V^{-1}
CurETReACGreenFunction		vertex	Table 104 on page 601	V^{-1}
CurGeoGreenFunction		vertex	Table 104 on page 601	Acm^{-3}

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
CurHcImACGreenFunction		vertex	Table 104 on page 601	1
CurHcReACGreenFunction		vertex	Table 104 on page 601	1
CurHTImACGreenFunction		vertex	Table 104 on page 601	V^{-1}
CurHTReACGreenFunction		vertex	Table 104 on page 601	V^{-1}
CurLTImACGreenFunction		vertex	Table 104 on page 601	V^{-1}
CurLTReACGreenFunction		vertex	Table 104 on page 601	V^{-1}
CurPotImACGreenFunction		vertex	Table 104 on page 601	s^{-1}
CurPotReACGreenFunction		vertex	Table 104 on page 601	s^{-1}
CurrentPotential	CurrentPotential	vertex	W, Current Potential on page 201	Acm^{-1}
DelVorWeight	DelVorWeight	vertex	Weighted Voronoï Diagram on page 956	μm^2
DeepLevels	DeepLevels	vertex	Energetic and Spatial Distribution of Traps on page 408	cm^{-3}
DielectricConstant		element	ϵ , Eq. 37, p. 191	1
DielectricConstant		vertex	ϵ , Eq. 37, p. 191	1
DielectricConstantAniso		element	ϵ_{aniso} , Anisotropic Electrical Permittivity on page 677	1
DielectricConstantAniso		vertex	ϵ_{aniso} , Anisotropic Electrical Permittivity on page 677	1

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
DisplacementCurrentDensity	DisplacementCurrent	vertex	$ J_D $	Acm^{-2}
DonorConcentration	DonorConcentration	vertex	Doping Specification on page 49	cm^{-3}
DonorPlusConcentration		vertex		cm^{-3}
DopingConcentration	Doping	vertex		cm^{-3}
eAlphaAvalanche		vertex	α_n , Eq. 369, p. 377	cm^{-1}
eAmorphousRecombination	eGapStatesRecombination	vertex	Chapter 17	$\text{cm}^{-3}\text{s}^{-1}$
eAmorphousTrappedCharge	eTrappedCharge	vertex	Chapter 17	cm^{-3}
eAugerRecombination		vertex	R_n^A , Eq. 366, p. 376	$\text{cm}^{-3}\text{s}^{-1}$
eAvalancheGeneration	eAvalanche	vertex	G_n , Eq. 369, p. 377	$\text{cm}^{-3}\text{s}^{-1}$
eBand2BandGeneration	eBand2BandGeneration	vertex	Dynamic Nonlocal Path Band-to-Band Model on page 395	$\text{cm}^{-3}\text{s}^{-1}$
eCDL1Lifetime	eCDL1lifetime	vertex	τ_{n1} , Coupled Defect Level (CDL) Recombination on page 373	s
eCDL2Lifetime	eCDL2lifetime	vertex	τ_{n2} , Coupled Defect Level (CDL) Recombination on page 373	s
eCurrentDensity	eCurrent	vertex	$ J_n $, Eq. 53, p. 197	Acm^{-2}
eDensity	eDensity	vertex	n , Eq. 53, p. 197	cm^{-3}
eDifferentialGain	eDifferentialGain	vertex	Stimulated and Spontaneous Emission Coefficients on page 898	cm^2

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
eDiffusivityMobility	eDiffusivityMobility	vertex	$\mu_{n,diff}$ Non-Einstein Diffusivity on page 350	$\text{cm}^2 \text{V}^{-1} \text{s}^{-1}$
eDirectTunnelCurrent	eDirectTunneling	vertex	Direct Tunneling on page 608	Acm^{-2}
eDriftVelocity	eDriftVelocity	vertex	Electron drift velocity	cm s^{-1}
eeDiffusionLNS		vertex	Table 104 on page 601	$\text{C}^2 \text{s}^{-1} \text{cm}^{-1}$
eEffectiveField	eEffectiveField	vertex	E_n^{eff} , Eq. 395, p. 387	Vcm^{-1}
eEffectiveStateDensity		vertex	N_C , Effective Masses and Effective Density-of-States on page 261	cm^{-3}
eeFlickerGRLNS		vertex	Table 104 on page 601	$\text{C}^2 \text{s}^{-1} \text{cm}^{-1}$
eeMonopolarGRLNS		vertex	Table 104 on page 601	$\text{C}^2 \text{s}^{-1} \text{cm}^{-1}$
eEnormal	eEnormal	vertex	$F_{\perp}$, Eq. 293, p. 336 or $F_{n,\perp}$, Eq. 294, p. 336	Vcm^{-1}
eEparallel	eEparallel	vertex	F_n , Eq. 313, p. 348	Vcm^{-1}
eEquilibriumDensity	eEquilibriumDensity	vertex	n , Eq. 53, p. 197 at zero applied voltages (zero currents)	cm^{-3}
EffectiveBandGap	EffectiveBandGap	vertex	$E_g - E_{\text{bgn}}$, Bandgap and Electron-Affinity Models on page 250	eV
EffectiveIntrinsicDensity	EffectiveIntrinsicDensity	vertex	$n_{i,eff}$, Eq. 139, p. 249	cm^{-3}
eFreeCarrierAbsorption	eFreeCarrierAbsorption	vertex	$\alpha_n n \Psi ^2$, Eq. 900, p. 792	cm^{-3}

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
eGradQuasiFermi	eGradQuasiFermi	vertex	$ \nabla\Phi_n $, Eq. 314, p. 348	Vcm^{-1}
eHeatFlux	eHeatFlux	vertex	$ \vec{S}_n $, Eq. 74, p. 211	Wcm^{-2}
eInterfaceTrappedCharge		vertex	Chapter 17	cm^{-2}
eIonIntegral	eIonIntegral	vertex	Approximate Breakdown Analysis: Poisson Equation Approach on page 389	1
eJouleHeat	eJouleHeat	vertex	Table 22 on page 210	Wcm^{-3}
ElectricField	ElectricField	vertex	F	Vcm^{-1}
ElectronAffinity	ElectronAffinity	vertex	χ , Bandgap and Electron-Affinity Models on page 250	eV
ElectrostaticPotential	Potential	vertex	ϕ , Eq. 37, p. 191	V
eLifetime	eLifeTime	vertex	τ_n , Eq. 325, p. 360	s
eMobility	eMobility	element	μ_n , Chapter 15	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
eMobility	eMobility	vertex	μ_n , Chapter 15	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
eMobilityAniso		element	μ_n^{aniso} , Anisotropic Mobility on page 669	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
eMobilityAniso		vertex	μ_n^{aniso} , Anisotropic Mobility on page 669	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
eMobilityAnisoFactor		vertex	r_e , Eq. 732, p. 669	1

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
eMobilityStressFactorXX	eMobilityStressFactorXX	vertex	Using Piezoresistance Mobility Model on page 720	1
eMobilityStressFactorXY	eMobilityStressFactorXY	vertex		1
eMobilityStressFactorXZ	eMobilityStressFactorXZ	vertex		1
eMobilityStressFactorYY	eMobilityStressFactorYY	vertex		1
eMobilityStressFactorYZ	eMobilityStressFactorYZ	vertex		1
eMobilityStressFactorZZ	eMobilityStressFactorZZ	vertex		1
eNLLTunnelingGeneration	eBarrierTunneling	vertex	Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612	$\text{cm}^{-3} \text{s}^{-1}$
eNLLTunnelingPeltierHeat		vertex	Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612	Wcm^{-3}
eQuantumPotential	eQuantumPotential	vertex	Λ_n , Eq. 198, p. 279	eV
eQuasiFermiEnergy	eQuasiFermiEnergy	vertex	$E_{F,n}$ Quasi-Fermi Energy on page 201	eV
eQuasiFermiPotential	eQuasiFermi	vertex	Φ_n , Quasi-Fermi Potential with Boltzmann Statistics on page 193	V
EquilibriumPotential	EquilibriumPotential	vertex	ϕ , Eq. 37, p. 191 at zero applied voltages (zero currents)	V
eRelativeEffectiveMass		vertex	m_n , Effective Masses and Effective Density-of-States on page 261	1
eSaturationVelocity		vertex	$v_{\text{sat},n}$, Velocity Saturation Models on page 347	cm s^{-1}

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
eSaturationVelocityAniso		vertex	$v_{\text{sat},n}^{\text{aniso}}$, Anisotropic Mobility on page 669	cm s^{-1}
eSchenkBGN	eSchenkBGN	vertex	$-\Lambda_n$, Eq. 198, p. 279	eV
eSHEAvalancheGeneration	eSHEAvalancheGeneration	vertex	$G_{n,\text{SHE}}^{\text{ii}}$, Spherical Harmonics Expansion Method on page 634	$\text{cm}^{-3}\text{s}^{-1}$
eSHECurrentDensity	eSHECurrentDensity	vertex	$ \vec{J}_{n,\text{SHE}} $, Spherical Harmonics Expansion Method on page 634	Acm^{-2}
eSHEDensity	eSHEDensity	vertex	n_{SHE} , Spherical Harmonics Expansion Method on page 634	cm^{-3}
eSHEEnergy	eSHEEnergy	vertex	$T_{n,\text{SHE}}$, Spherical Harmonics Expansion Method on page 634	K
eSHEVelocity	eSHEVelocity	vertex	$ \vec{v}_{n,\text{SHE}} $, Spherical Harmonics Expansion Method on page 634	cm s^{-1}
eSRHRecombination	eSRHRecombination	vertex	Dynamic Nonlocal Path Trap-assisted Tunneling on page 367	$\text{cm}^{-3}\text{s}^{-1}$

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
eTemperature	eTemperature	vertex	T_n , Hydrodynamic Model for Temperatures on page 211	K
eTemperatureRelaxationTime		vertex	τ_{en} , Eq. 82, p. 212	s
eTensorMobilityFactorXX	eTensorMobilityFactorXX	vertex	Chapter 30	1
eTensorMobilityFactorYY	eTensorMobilityFactorYY	vertex	Chapter 30	1
eTensorMobilityFactorZZ	eTensorMobilityFactorZZ	vertex	Chapter 30	1
eTensorMobilityXX	eTensorMobilityXX	vertex	Chapter 30	$\text{cm}^2/(\text{Vs})$
eTensorMobilityYY	eTensorMobilityYY	vertex	Chapter 30	$\text{cm}^2/(\text{Vs})$
eTensorMobilityZZ	eTensorMobilityZZ	vertex	Chapter 30	$\text{cm}^2/(\text{Vs})$
eThermoElectricPower	eThermoelectricPower	vertex	P_n , Eq. 863, p. 750	V K^{-1}
eVelocity	eVelocity	vertex	$v_n = J_n/nq $	cm s^{-1}
f1BandOccupancy001	f1BandOccupancy001	vertex	Using Intel Mobility Model on page 716	1
f1BandOccupancy010	f1BandOccupancy010	vertex		1
f1BandOccupancy100	f1BandOccupancy100	vertex		1
f2BandOccupancy001	f2BandOccupancy001	vertex		1
f2BandOccupancy010	f2BandOccupancy010	vertex		1
f2BandOccupancy100	f2BandOccupancy100	vertex		1
FowlerNordheim	FowlerNordheim	vertex	j_{FN} , Eq. 629, p. 607	A cm^{-2}
Grad2PoECACGreenFunction		vertex	Table 104 on page 601	$\text{V}^2 \text{s}^2 \text{C}^{-2} \text{cm}^{-2}$
Grad2PoHCACGreenFunction		vertex	Table 104 on page 601	$\text{V}^2 \text{s}^2 \text{C}^{-2} \text{cm}^{-2}$
hAlphaAvalanche		vertex	α_p , Eq. 369, p. 377	cm^{-1}
hAmorphousRecombination	hGapStatesRecombination	vertex	Chapter 17	$\text{cm}^{-3} \text{s}^{-1}$
hAmorphousTrappedCharge	hTrappedCharge	vertex	Chapter 17	cm^{-3}

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
hAugerRecombination		vertex	R_p^A , Eq. 366, p. 376	$\text{cm}^{-3} \text{s}^{-1}$
hAvalancheGeneration	hAvalanche	vertex	G_p , Eq. 369, p. 377	$\text{cm}^{-3} \text{s}^{-1}$
hBand2BandGeneration	hBand2BandGeneration	vertex	Dynamic Nonlocal Path Band-to-Band Model on page 395	$\text{cm}^{-3} \text{s}^{-1}$
hCDL1Lifetime	hCDL1lifetime	vertex	τ_{p1} , Coupled Defect Level (CDL) Recombination on page 373	s
hCDL2Lifetime	hCDL2lifetime	vertex	τ_{p2} , Coupled Defect Level (CDL) Recombination on page 373	s
hCurrentDensity	hCurrent	vertex	$ J_p $, Eq. 53, p. 197	Acm^{-2}
hDensity	hDensity	vertex	p , Eq. 53, p. 197	cm^{-3}
hDifferentialGain	hDifferentialGain	vertex	Stimulated and Spontaneous Emission Coefficients on page 898	cm^2
hDiffusivityMobility	hDiffusivityMobility	vertex	$\mu_{p, \text{diff}}$ Non-Einstein Diffusivity on page 350	$\text{cm}^2 \text{V}^{-1} \text{s}^{-1}$
hDirectTunnelCurrent	hDirectTunneling	vertex	Direct Tunneling on page 608	Acm^{-2}
hDriftVelocity	hDriftVelocity	vertex	Hole drift velocity	cm s^{-1}
HeavyIonChargeDensity	HeavyIonChargeDensity	vertex	Heavy Ions on page 574	cm^{-3}
HeavyIonGeneration		vertex	G^{HeavyIon} , Eq. 598, p. 575	$\text{cm}^{-3} \text{s}^{-1}$
hEffectiveField	hEffectiveField	vertex	E_p^{eff} , Eq. 396, p. 387	Vcm^{-1}

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
hEffectiveStateDensity		vertex	N_V , Effective Masses and Effective Density-of-States on page 261	cm^{-3}
heiTemperature	HEItemperature	vertex	T_{hei} , hot-electron temperature, computed as postprocessing approach (Carrier TempPost), Chapter 25	K
hEnormal	hEnormal	vertex	$F_{\perp}$, Eq. 293, p. 336 or $F_{p,\perp}$, Eq. 294, p. 336	Vcm^{-1}
hEparallel	hEparallel	vertex	F_p , Eq. 313, p. 348	Vcm^{-1}
hEquilibriumDensity	hEquilibriumDensity	vertex	p , Eq. 53, p. 197 at zero applied voltages (zero currents)	cm^{-3}
hFreeCarrierAbsorption	hFreeCarrierAbsorption	vertex	$\alpha_p p \Psi ^2$, Eq. 900, p. 792	cm^{-3}
hGradQuasiFermi	hGradQuasiFermi	vertex	$ \nabla \Phi_p $, Eq. 314, p. 348	Vcm^{-1}
hhDiffusionLNS		vertex	Table 104 on page 601	$\text{C}^2 \text{s}^{-1} \text{cm}^{-1}$
hHeatFlux	hHeatFlux	vertex	$ \vec{S}_p $, Eq. 75, p. 211	Wcm^{-2}
hhFlickerGRLNS		vertex	Table 104 on page 601	$\text{C}^2 \text{s}^{-1} \text{cm}^{-1}$
hhMonopolarGRLNS		vertex	Table 104 on page 601	$\text{C}^2 \text{s}^{-1} \text{cm}^{-1}$
hInterfaceTrappedCharge		vertex	Chapter 17	cm^{-2}

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
hIonIntegral	hIonIntegral	vertex	Approximate Breakdown Analysis: Poisson Equation Approach on page 389	1
hJouleHeat	hJouleHeat	vertex	Table 22 on page 210	Wcm^{-3}
hLifetime	hLifeTime	vertex	τ_p , Eq. 325, p. 360	s
hMobility	hMobility	element	μ_p , Chapter 15	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
hMobility	hMobility	vertex	μ_p , Chapter 15	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
hMobilityAniso		element	μ_p^{aniso} , Anisotropic Mobility on page 669	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
hMobilityAniso		vertex	μ_p^{aniso} , Anisotropic Mobility on page 669	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
hMobilityAnisoFactor		vertex	r_h , Eq. 732, p. 669	1
hMobilityStressFactorXX	hMobilityStressFactorXX	vertex	Using Piezoresistance Mobility Model on page 720	1
hMobilityStressFactorXY	hMobilityStressFactorXY	vertex		1
hMobilityStressFactorXZ	hMobilityStressFactorXZ	vertex		1
hMobilityStressFactorYY	hMobilityStressFactorYY	vertex		1
hMobilityStressFactorYZ	hMobilityStressFactorYZ	vertex		1
hMobilityStressFactorZZ	hMobilityStressFactorZZ	vertex		1
hNLLTunnelingGeneration	hBarrierTunneling	vertex	Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612	$\text{cm}^{-3}\text{s}^{-1}$

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
hNLLTunnelingPeltierHeat		vertex	Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612	Wcm^{-3}
HotElectronInj	HotElectronInjection	vertex, rivertex	Hot-electron current density j_{he} at interface, Eq. 662, p. 630 , Eq. 668, p. 632	Acm^{-2}
HotHoleInj	HotHoleInjection	vertex, rivertex	Hot-hole current density j_{hh} at interface, Eq. 662, p. 630 , Eq. 668, p. 632	Acm^{-2}
hQuantumPotential	hQuantumPotential	vertex	Λ_p , Eq. 198, p. 279	eV
hQuasiFermiEnergy	hQuasiFermiEnergy	vertex	$E_{F,p}$ Quasi-Fermi Energy on page 201	eV
hQuasiFermiPotential	hQuasiFermi	vertex	Φ_p , Quasi-Fermi Potential with Boltzmann Statistics on page 193	V
hRelativeEffectiveMass		vertex	m_p , Effective Masses and Effective Density-of-States on page 261	1
hSaturationVelocity		vertex	$v_{\text{sat},p}$, Velocity Saturation Models on page 347	cm s^{-1}
hSaturationVelocityAniso		vertex	$v_{\text{sat},p}^{\text{aniso}}$, Anisotropic Mobility on page 669	cm s^{-1}
hSchenkBGN	hSchenkBGN	vertex	$-\Lambda_p$, Eq. 198, p. 279	eV

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
hSHEAvalancheGeneration	hSHEAvalancheGeneration	vertex	$G_{p, \text{SHE}}^{\text{ii}}$, Spherical Harmonics Expansion Method on page 634	$\text{cm}^{-3} \text{s}^{-1}$
hSHECurrentDensity	hSHECurrentDensity	vertex	$\vec{J}_{p, \text{SHE}}$, Spherical Harmonics Expansion Method on page 634	Acm^{-2}
hSHEDensity	hSHEDensity	vertex	p_{SHE} , Spherical Harmonics Expansion Method on page 634	cm^{-3}
hSHEEnergy	hSHEEnergy	vertex	$T_{p, \text{SHE}}$, Spherical Harmonics Expansion Method on page 634	K
hSHEVelocity	hSHEVelocity	vertex	$ \vec{v}_{p, \text{SHE}} $, Spherical Harmonics Expansion Method on page 634	cm s^{-1}
hSRHRecombination	hSRHRecombination	vertex	Dynamic Nonlocal Path Trap-assisted Tunneling on page 367	$\text{cm}^{-3} \text{s}^{-1}$
hTemperature	hTemperature	vertex	T_p , Hydrodynamic Model for Temperatures on page 211	K
hTemperatureRelaxationTime		vertex	τ_{ep} , Eq. 83, p. 212	s
hTensorMobilityFactorXX	hTensorMobilityFactorXX	vertex	Chapter 30	1
hTensorMobilityFactorYY	hTensorMobilityFactorYY	vertex	Chapter 30	1

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
hTensorMobilityFactorZZ	hTensorMobilityFactorZZ	vertex	Chapter 30	1
hTensorMobilityXX	hTensorMobilityXX	vertex	Chapter 30	$\text{cm}^2/(\text{Vs})$
hTensorMobilityYY	hTensorMobilityYY	vertex	Chapter 30	$\text{cm}^2/(\text{Vs})$
hTensorMobilityZZ	hTensorMobilityZZ	vertex	Chapter 30	$\text{cm}^2/(\text{Vs})$
hThermoElectricPower	hThermoelectricPower	vertex	P_p , Eq. 864, p. 750	V K^{-1}
hVelocity	hVelocity	vertex	$v_p = \left \vec{J}_p / pq \right $	cm s^{-1}
HydrogenAtom	HydrogenAtom	vertex	MSC–Hydrogen Transport Degradation Model on page 449	cm^{-3}
HydrogenIon	HydrogenIon	vertex	MSC–Hydrogen Transport Degradation Model on page 449	cm^{-3}
HydrogenMolecule	HydrogenMolecule	vertex	MSC–Hydrogen Transport Degradation Model on page 449	cm^{-3}
ImeDensityResponse		vertex	$\text{Im}(\tilde{n})$ AC Response on page 960	$\text{cm}^{-3}\text{V}^{-1}$
ImeeDiffusionLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ImeeFlickerGRLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ImeeLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ImeeMonopolarGRLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ImElectrostaticPotentialResponse		vertex	$\text{Im}(\tilde{\phi})$ AC Response on page 960	1

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
ImTemperatureResponse		vertex	$\text{Im}(\tilde{T}_n)$ AC Response on page 960	KV^{-1}
ImhDensityResponse		vertex	$\text{Im}(\tilde{p})$ AC Response on page 960	$\text{Acm}^{-2}\text{V}^{-1}$
ImhhDiffusionLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ImhhFlickerGRLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ImhhLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ImhhMonopolarGRLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ImhTemperatureResponse		vertex	$\text{Im}(\tilde{T}_p)$ AC Response on page 960	KV^{-1}
ImLatticeTemperatureResponse		vertex	$\text{Im}(\tilde{T})$ AC Response on page 960	KV^{-1}
ImLNISD		vertex	Table 104 on page 601	$\text{A}^2\text{scm}^{-3}$
ImLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ImTrapLNISD		vertex	Table 104 on page 601	$\text{A}^2\text{scm}^{-3}$
ImTrapLNVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
IndiumActiveConcentration		vertex	Doping Specification on page 49	cm^{-3}
IndiumConcentration	IndiumConcentration	vertex	In , Doping Specification on page 49	cm^{-3}
IndiumMinusConcentration	inMinus	vertex	In^- , Chapter 13	cm^{-3}

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
InterfaceNBTICharge		vertex	Two-Stage NBTI Degradation Model on page 457	cm^{-2}
InterfaceNBTIState1		vertex	Two-Stage NBTI Degradation Model on page 457	cm^{-2}
InterfaceNBTIState2		vertex	Two-Stage NBTI Degradation Model on page 457	cm^{-2}
InterfaceNBTIState3		vertex	Two-Stage NBTI Degradation Model on page 457	cm^{-2}
InterfaceNBTIState4		vertex	Two-Stage NBTI Degradation Model on page 457	cm^{-2}
InterfaceOrientation	InterfaceOrientation	vertex	Auto-Orientation Framework on page 75	1
IntrinsicDensity	IntrinsicDensity	vertex	n_i , Eq. 138, p. 249	cm^{-3}
JouleHeat	JouleHeat	vertex	Table 22 on page 210	Wcm^{-3}
LaserIntensity	LaserIntensity	vertex	Combined laser intensity for multimode lasing	Wcm^{-3}
LatticeHeatCapacity		vertex	c_L , Heat Capacity on page 745	$\text{JK}^{-1} \text{cm}^{-3}$
LatticeTemperature	LatticeTemperature, Temperature	vertex	T , Chapter 9, p. 205	K
LayerThickness		vertex	Thin-Layer Mobility Model on page 337	μm
lHeatFlux	lHeatFlux	vertex	$\vec{S}_L$, Eq. 76, p. 211	Wcm^{-2}

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
MeanIonIntegral	MeanIonIntegral	vertex	Approximate Breakdown Analysis: Poisson Equation Approach on page 389	1
MatGain	MatGain	vertex	Laser material gain, Stimulated and Spontaneous Emission Coefficients on page 898	m ⁻¹
MetalWorkfunction	MetalWorkfunction	vertex	Metal Workfunction on page 242	eV
MobilityAcceptorConcentration	MobilityAcceptorConcentration	vertex	Mobility Doping File on page 354 or Add2TotalDoping (ChargedTraps), Doping Specification on page 49	cm ⁻³
MobilityDonorConcentration	MobilityDonorConcentration	vertex	Mobility Doping File on page 354 or Add2TotalDoping (ChargedTraps), Doping Specification on page 49	cm ⁻³
NDopantActiveConcentration		vertex	Doping Specification on page 49	cm ⁻³
NDopantConcentration	NdopantConcentration	vertex	NDopant, Doping Specification on page 49	cm ⁻³
NDopantPlusConcentration	NdopantPlus	vertex	NDopant ⁺ , Chapter 13	cm ⁻³
NearestInterfaceOrientation	NearestInterfaceOrientation	vertex	Auto-Orientation Framework on page 75	1

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
NegInterfaceCharge	NegInterfaceCharge	vertex	Mobility Degradation Components due to Coulomb Scattering on page 332	cm^{-2}
NitrogenActiveConcentration		vertex	Doping Specification on page 49	cm^{-3}
NitrogenConcentration	NitrogenConcentration	vertex	N, Doping Specification on page 49	cm^{-3}
NitrogenPlusConcentration	NitrogenPlus	vertex	N⁺, Chapter 13	cm^{-3}
OneOverDegradationTime		vertex	Chapter 19	s^{-1}
OpticalAbsorption	OpticalAbsorptionHeat	vertex	Optical Absorption Heat on page 477	Wcm^{-3}
OpticalAbsorption(Bandgap)	OpticalAbsorptionHeat	vertex	Optical Absorption Heat on page 477	Wcm^{-3}
OpticalAbsorption(Vacuum)	OpticalAbsorptionHeat	vertex	Optical Absorption Heat on page 477	Wcm^{-3}
OpticalField	OpticalField	vertex	Plot optical intensity as well as real and imaginary parts of optical field.	W/m^{-3} V/m
OpticalGeneration	OpticalGeneration	vertex	G_0^{opt} , Eq. 571, p. 540	$\text{cm}^{-3}\text{s}^{-1}$
OpticalGenerationFromConstant	OpticalGeneration	vertex	Specifying the Type of Optical Generation Computation on page 470	$\text{cm}^{-3}\text{s}^{-1}$
OpticalGenerationFromFile	OpticalGeneration	vertex	Specifying the Type of Optical Generation Computation on page 470	$\text{cm}^{-3}\text{s}^{-1}$

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
OpticalGenerationFromMonochromaticSource	OpticalGeneration	vertex	Specifying the Type of Optical Generation Computation on page 470	$\text{cm}^{-3} \text{s}^{-1}$
OpticalGenerationFromSpectrum	OpticalGeneration	vertex	Specifying the Type of Optical Generation Computation on page 470	$\text{cm}^{-3} \text{s}^{-1}$
OpticalIntensity	OpticalIntensity	vertex	Solving the Optical Problem on page 480	Wcm^{-2}
OpticalIntensityDiffuse	OpticalIntensity	vertex	Transfer Matrix Method on page 538	Wcm^{-2}
OpticalIntensitySpecular	OpticalIntensity	vertex	Transfer Matrix Method on page 538	Wcm^{-2}
OpticalIntensityMode0 ... OpticalIntensityMode9	OpticalIntensityMode0 ... OpticalIntensityMode9	vertex	Current File and Plot Variables for Laser Simulation on page 773	Wcm^{-3}
OpticalPolarizationAngleMode0 ... OpticalPolarizationAngleMode9	OpticalPolarizationAngleMode0 ... OpticalPolarizationAngleMode9	vertex	Polarization-dependent Optical Matrix Element on page 910	1
ParallelToInterfaceInBoundaryLayerActive	ParallelToInterfaceInBoundaryLayerActive	element	Field Correction Close to Interfaces on page 349	1
PDopantActiveConcentration		vertex	Doping Specification on page 49	cm^{-3}
PDopantConcentration	pDopantConcentration	vertex	PDopant, Doping Specification on page 49	cm^{-3}
PDopantMinusConcentration	pDopantMinus	vertex	PDopant ⁻ , Chapter 13	cm^{-3}
PE_Charge	PE_Charge	vertex	q_{PE} , Eq. 1107, p. 1114	cm^{-3}

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
PeltierHeat	PeltierHeat	vertex	Table 22 on page 210	Wcm^{-3}
PhosphorusActiveConcentration		vertex	Doping Specification on page 49	cm^{-3}
PhosphorusConcentration	PhosphorusConcentration	vertex	P, Doping Specification on page 49	cm^{-3}
PhosphorusPlusConcentration	phPlus	vertex	P⁺, Chapter 13	cm^{-3}
PMIHeat	PMIHeat	vertex	Heat Generation Rate on page 1153	Wcm^{-3}
PMIRecombination	PMIRecombination	vertex	R^{PMI}, Generation-Recombination Model on page 1012	$\text{cm}^{-3}\text{s}^{-1}$
PMIUserField0	PMIUserField0	vertex	Command File of Sentaurus Device on page 989	1
PMIUserField1	PMIUserField1	vertex		1
...	...	vertex		1
PMIUserField99	PMIUserField99	vertex		1
PoECImACGreenFunction		vertex	Table 104 on page 601	VsC^{-1}
PoECreACGreenFunction		vertex	Table 104 on page 601	VsC^{-1}
PoETImACGreenFunction		vertex	Table 104 on page 601	A^{-1}
PoETReACGreenFunction		vertex	Table 104 on page 601	A^{-1}
PoGeoGreenFunction		vertex	Table 104 on page 601	Vcm^{-3}
PoHClmACGreenFunction		vertex	Table 104 on page 601	VsC^{-1}
PoHCreACGreenFunction		vertex	Table 104 on page 601	VsC^{-1}
PoHTImACGreenFunction		vertex	Table 104 on page 601	A^{-1}

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
PoHTReACGreenFunction		vertex	Table 104 on page 601	A^{-1}
PoLTIACGreenFunction		vertex	Table 104 on page 601	A^{-1}
PoLTReACGreenFunction		vertex	Table 104 on page 601	A^{-1}
PoPotImACGreenFunction		vertex	Table 104 on page 601	VC^{-1}
PoPotReACGreenFunction		vertex	Table 104 on page 601	VC^{-1}
Polarization	Polarization	vertex	$\frac{\vec{r}}{ P }$, Chapter 29	Ccm^{-2}
PosInterfaceCharge	PosInterfaceCharge	vertex	Mobility Degradation Components due to Coulomb Scattering on page 332	cm^{-2}
QuantumYield	QuantumYield	vertex	Quantum Yield Models on page 475	1
QuasiFermiPotential		vertex	Φ , Eq. 117, p. 227	V
QW_eLeakageCurrent	QW_eLeakageCurrent	vertex	Thermionic Emission on page 897	Acm^{-2}
QW_hLeakageCurrent	QW_hLeakageCurrent	vertex	Thermionic Emission on page 897	Acm^{-2}
QWeDensity	QWeDensity	vertex	n , Eq. 1009, p. 906	cm^{-3}
QWeQuasiFermi	QWeQuasiFermi	vertex	Φ_n , Quasi-Fermi Potential with Boltzmann Statistics on page 193	V
QWhDensity	QWhDensity	vertex	p , Eq. 1010, p. 906	cm^{-3}

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
QWhQuasiFermi	QWhQuasiFermi	vertex	Φ_p , Quasi-Fermi Potential with Boltzmann Statistics on page 193	V
RadiationGeneration		vertex	G_r , Eq. 582, p. 558	$\text{cm}^{-3}\text{s}^{-1}$
RadiativeRecombination	RadiativeRecombination	vertex	R , Eq. 365, p. 375	$\text{cm}^{-3}\text{s}^{-1}$
	RandomizedDoping	vertex	Statistical Impedance Field Method	cm^{-3}
RecombinationHeat	RecombinationHeat	vertex	Table 22 on page 210	Wcm^{-3}
ReeDensityResponse		vertex	$\text{Re}(\tilde{n})$ AC Response on page 960	$\text{Acm}^{-2}\text{V}^{-1}$
ReeDiffusionLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ReeFlickerGRLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ReeLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ReeMonopolarGRLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ReElectrostaticPotentialResponse		vertex	$\text{Re}(\tilde{\phi})$ AC Response on page 960	1
ReeTemperatureResponse		vertex	$\text{Re}(\tilde{T}_n)$ AC Response on page 960	KV^{-1}
RefractiveIndex		element	n , Complex Refractive Index Model on page 496	1
RefractiveIndex		vertex	n , Complex Refractive Index Model on page 496	1

F: Data and Plot Names

Scalar Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
RehDensityResponse		vertex	$\text{Re}(\tilde{p})$ AC Response on page 960	$\text{Acm}^{-2}\text{V}^{-1}$
RehhDiffusionLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
RehhFlickerGRLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
RehhLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
RehhMonopolarGRLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
RehTemperatureResponse		vertex	$\text{Re}(\tilde{T}_p)$ AC Response on page 960	KV^{-1}
ReLatticeTemperatureResponse		vertex	$\text{Re}(\tilde{T})$ AC Response on page 960	KV^{-1}
ReLNISD		vertex	Table 104 on page 601	$\text{A}^2\text{scm}^{-3}$
ReLNVXVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
ReTrapLNISD		vertex	Table 104 on page 601	$\text{A}^2\text{scm}^{-3}$
ReTrapLNVSD		vertex	Table 104 on page 601	$\text{V}^2\text{scm}^{-3}$
SpaceCharge	SpaceCharge	vertex	Eq. 37, p. 191	cm^{-3}
SpontaneousRecombination	SpontaneousRecombination	vertex	Spontaneous Recombination Rate on page 901	$\text{cm}^{-3}\text{s}^{-1}$
SRHRecombination	SRHRecombination	vertex	$R_{\text{net}}^{\text{SRH}}$, Shockley–Read–Hall Recombination on page 359	$\text{cm}^{-3}\text{s}^{-1}$
StimulatedRecombination	StimulatedRecombination	vertex	Stimulated Recombination Rate on page 900	$\text{cm}^{-3}\text{s}^{-1}$

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
	Stress	vertex	Stress tensor, Chapter 30	Pa
StressXX	StressXX	vertex	Components of stress tensor, Chapter 30	Pa
StressXY	StressXY			
StressXZ	StressXZ			
StressYY	StressYY			
StressYZ	StressYZ			
StressZZ	StressZZ			
SurfaceRecombination	SurfaceRecombination	vertex, rivertex	Surface SRH Recombination on page 372	$\text{cm}^{-2} \text{s}^{-1}$
ThermalConductivity	ThermalConductivity	vertex	κ , Eq. 858, p. 747	$\text{Wcm}^{-1} \text{K}^{-1}$
ThermalConductivityAniso		vertex	κ_{aniso} , Anisotropic Thermal Conductivity on page 678	$\text{Wcm}^{-1} \text{K}^{-1}$
ThermalizationYield(Bandgap)	ThermalizationYield	vertex	Optical Absorption Heat on page 477	1
ThermalizationYield(Vacuum)	ThermalizationYield	vertex	Optical Absorption Heat on page 477	1
ThomsonHeat	ThomsonHeat	vertex	Table 22 on page 210	Wcm^{-3}
TotalConcentration		vertex	Doping Specification on page 49	cm^{-3}
TotalCurrentDensity	Current	vertex	$ \vec{J}_n + \vec{J}_p + \vec{J}_D $	Acm^{-2}
TotalHeat	TotalHeat	vertex	Sum of all heat generation terms, Chapter 9, Temperature in Metals on page 244	Wcm^{-3}
TotalInterfaceTrapConcentration		vertex	Chapter 17	cm^{-2}

F: Data and Plot Names

Vector Data

Table 140 Scalar data

Data name	Plot name	Location	Description	Unit
TotalRecombination	TotalRecombination	vertex	Sum of all generation–recombination terms, Chapter 16	$\text{cm}^{-3} \text{s}^{-1}$
TotalTrapConcentration		vertex	Chapter 17	cm^{-3}
tSRHRecombination	tSRHRecombination	vertex	Dynamic Nonlocal Path Trap-assisted Tunneling on page 367	$\text{cm}^{-3} \text{s}^{-1}$
UserSpeciesConcentration	UserSpeciesConcentration	vertex	User-Defined Species on page 50	cm^{-3}
UserSpeciesIncomplete Concentration	UserSpeciesIncomplete Concentration	vertex		cm^{-3}
ValenceBandEnergy	ValenceBandEnergy	vertex	E_V , Eq. 42, p. 193	eV
xMoleFraction	xMoleFraction	vertex	Abrupt and Graded Heterojunctions on page 48	1
yMoleFraction	yMoleFraction	vertex		1

Vector Data

Table 141 Vector data

Data name	Plot name	Location	Description	Unit
ConductionCurrentDensity	ConductionCurrent	vertex	$\vec{J}_n + \vec{J}_p$, Eq. 53, p. 197 or J_M in metals, Eq. 129, p. 239	Acm^{-2}
DisplacementCurrentDensity	DisplacementCurrent	vertex	$\vec{J}_D$	Acm^{-2}
eCurrentDensity	eCurrent	vertex	$\vec{J}_n$, Eq. 53, p. 197	Acm^{-2}
eDriftVelocity	eDriftVelocity	vertex	Electron drift velocity	cm s^{-1}
eGradQuasiFermi	eGradQuasiFermi	vertex	$-\nabla \Phi_n$, Eq. 39, p. 193	Vcm^{-1}
eHeatFlux	eHeatFlux	vertex	$\vec{S}_n$, Eq. 74, p. 211	Wcm^{-2}
ElectricField	ElectricField	vertex	$\vec{F}$	Vcm^{-1}
EquilibriumElectricField		vertex	F_{eq} , Eq. 106, p. 221	Vcm^{-1}
eSHECurrentDensity	eSHECurrentDensity	vertex	$\vec{J}_{n, \text{SHE}}$, Spherical Harmonics Expansion Method on page 634	Acm^{-2}

Table 141 Vector data

Data name	Plot name	Location	Description	Unit
eSHEVelocity	eSHEVelocity	vertex	$\vec{v}_{n, \text{SHE}}$, Spherical Harmonics Expansion Method on page 634	cm s^{-1}
eVelocity	eVelocity	vertex	$\vec{v}_n = -\vec{J}_n/qn$	cm s^{-1}
GradPoECImACGreenFunction		vertex	Table 104 on page 601	$\text{VsC}^{-1}\text{cm}^{-1}$
GradPoEReACGreenFunction		vertex	Table 104 on page 601	$\text{VsC}^{-1}\text{cm}^{-1}$
GradPoETImACGreenFunction		vertex	Table 104 on page 601	$\text{A}^{-1}\text{cm}^{-1}$
GradPoETReACGreenFunction		vertex	Table 104 on page 601	$\text{A}^{-1}\text{cm}^{-1}$
GradPoHImACGreenFunction		vertex	Table 104 on page 601	$\text{VsC}^{-1}\text{cm}^{-1}$
GradPoHReACGreenFunction		vertex	Table 104 on page 601	$\text{VsC}^{-1}\text{cm}^{-1}$
GradPoHTImACGreenFunction		vertex	Table 104 on page 601	$\text{A}^{-1}\text{cm}^{-1}$
GradPoHTReACGreenFunction		vertex	Table 104 on page 601	$\text{A}^{-1}\text{cm}^{-1}$
hCurrentDensity	hCurrent	vertex	$\vec{J}_p$, Eq. 53, p. 197	Acm^{-2}
hDriftVelocity	hDriftVelocity	vertex	Hole drift velocity	cm s^{-1}
hGradQuasiFermi	hGradQuasiFermi	vertex	$-\nabla\Phi_p$, Eq. 314, p. 348	Vcm^{-1}
hHeatFlux	hHeatFlux	vertex	$\vec{S}_p$, Eq. 75, p. 211	Wcm^{-2}
hSHECurrentDensity	hSHECurrentDensity	vertex	$\vec{J}_{p, \text{SHE}}$, Spherical Harmonics Expansion Method on page 634	Acm^{-2}
hSHEVelocity	hSHEVelocity	vertex	$\vec{v}_{p, \text{SHE}}$, Spherical Harmonics Expansion Method on page 634	cm s^{-1}
hVelocity	hVelocity	vertex	$\vec{v}_p = \vec{J}_p/qp$	cm s^{-1}
ImConductionCurrentResponse		vertex	$\text{Im}(\vec{\tilde{J}}_n + \vec{\tilde{J}}_p)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$
ImDisplacementCurrentResponse		vertex	$\text{Im}(\vec{\tilde{J}}_D)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$

F: Data and Plot Names

Vector Data

Table 141 Vector data

Data name	Plot name	Location	Description	Unit
ImCurrentResponse		vertex	$\text{Im}(\vec{J}_n)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$
ImEnFluxResponse		vertex	$\text{Im}(\vec{S}_n)$ AC Current Density Responses on page 962	$\text{Wcm}^{-2}\text{V}^{-1}$
ImhCurrentResponse		vertex	$\text{Im}(\vec{J}_p)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$
ImhEnFluxResponse		vertex	$\text{Im}(\vec{S}_p)$ AC Current Density Responses on page 962	$\text{Wcm}^{-2}\text{V}^{-1}$
ImlEnFluxResponse		vertex	$\text{Im}(\vec{S}_L)$ AC Current Density Responses on page 962	$\text{Wcm}^{-2}\text{V}^{-1}$
ImTotalCurrentResponse		vertex	$\text{Im}(\vec{J}_n + \vec{J}_p + \vec{J}_D)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$
lHeatFlux	lHeatFlux	vertex	$\vec{S}_L$, Eq. 76, p. 211	Wcm^{-2}
NonLocalBackDirection	NonLocal	vertex	Visualizing Nonlocal Meshes on page 167	μm
NonLocalDirection	NonLocal	vertex		μm
OpticalField	OpticalField	vertex	Plot optical intensity as well as real and imaginary parts of optical field.	W/m^{-3} V/m
PE_Polarization	PE_Polarization	vertex	P_{PE} , Eq. 1107, p. 1114	Ccm^{-2}
Polarization	Polarization	element	$\vec{P}$, Chapter 29	Ccm^{-2}
Polarization	Polarization	vertex	$\vec{P}$, Chapter 29	Ccm^{-2}
ReConductionCurrentResponse		vertex	$\text{Re}(\vec{J}_n + \vec{J}_p)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$

Table 141 Vector data

Data name	Plot name	Location	Description	Unit
ReDisplacementCurrentResponse		vertex	$\text{Re}(\vec{J}_D)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$
ReeCurrentResponse		vertex	$\text{Re}(\vec{J}_n)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$
ReeEnFluxResponse		vertex	$\text{Re}(\vec{S}_n)$ AC Current Density Responses on page 962	$\text{Wcm}^{-2}\text{V}^{-1}$
RehCurrentResponse		vertex	$\text{Re}(\vec{J}_p)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$
RehEnFluxResponse		vertex	$\text{Re}(\vec{S}_n)$ AC Current Density Responses on page 962	$\text{Wcm}^{-2}\text{V}^{-1}$
RelEnFluxResponse		vertex	$\text{Re}(\vec{S}_L)$ AC Current Density Responses on page 962	$\text{Wcm}^{-2}\text{V}^{-1}$
ReTotalCurrentResponse		vertex	$\text{Re}(\vec{J}_n + \vec{J}_p + \vec{J}_D)$ AC Current Density Responses on page 962	$\text{Acm}^{-2}\text{V}^{-1}$
TotalCurrentDensity	Current	vertex	$\vec{J}_n + \vec{J}_p + \vec{J}_D$	Acm^{-2}

Special Vector Data

Table 142 Special vector data

Data name	Plot name	Location	Description	Unit
	eSHEDistribution	vertex	Electron energy distribution, SHE Distribution Hot-Carrier Injection on page 633	1
	hSHEDistribution	vertex	Hole energy distribution, SHE Distribution Hot-Carrier Injection on page 633	1

APPENDIX G Command File Overview

This appendix presents an overview of the command file of Sentaurus Device.

[Top Level of Command File on page 1219](#) presents the topmost level of the command file. Use this table to obtain a top-down overview of the command file. The remaining sections are ordered with respect to topics. The headings of the tables, therein, mostly start with a keyword. Use these tables to find details about keywords you already know.

Organization of Command File Overview

The tables in this appendix have two or three columns, and their rows are ordered alphabetically with respect to the first (and, as far as applicable, the middle) column. Placeholders (cross references, user-supplied values in <>) precede explicit keywords, irrespective of alphabetic order.

The left column contains keywords that can appear in the command file. In the topmost rows of some tables, the left column references another table. This means that all keywords in the referenced table can appear in the referring table as well. Some keywords are followed by an opening parenthesis or an opening brace to indicate that the keyword expects a selection of options listed in the middle column of the current and the following rows.

The middle column contains options or values to the keyword in the left column, one per row. For a selection of options, the last row indicates the closing parenthesis or the closing brace. For some tables, the middle column would be empty and is omitted.

The right column contains the description for keywords, options, and values of the row. As far as applicable, in the right column, the following additional information is given:

- The default value, which can be:
 - An explicit value. Those values are written in Courier font, as you would type them.
 - ‘+’ or ‘-’ to indicate the default (true or false, respectively) for keywords that support the ‘-’ prefix.
 - ‘on’ or ‘off’ for keywords that set and reset flags, but do not support the prefix ‘-’.
 - An asterisk (*) to indicate the default choice among mutually exclusive alternatives.
 - An exclamation mark (!) if no default exists and you must specify a value.

G: Command File Overview

Organization of Command File Overview

- The unit assumed for the given quantity. In unit specifications, d denotes the dimension of the mesh. For dimensionless quantities, no unit is specified.
- The location for which keywords should be specified, given as characters in parentheses:
 - ‘g’ stands for global parameters that must not appear in region-specific, interface-specific, or contact-specific sections.
 - ‘r’ stands for region-specific (or material-specific) parameters that usually do not make sense when specified for interfaces or contacts.
 - ‘i’ and ‘c’ stand for interface-specific or contact-specific parameters.

Furthermore, the tables use the following conventions:

- An ellipsis (...) denotes that the preceding item can be repeated an arbitrary number of times.
- An asterisk (*) followed by an integer, both typeset in Times font, denotes that the preceding item must appear the given number of times.
- Optional components of specifications are enclosed in brackets and typeset in Times font.
- Angle brackets (< >) indicate user-supplied values. [Table 143](#) summarizes the common types of specification. More specific notation is explained in the last column of the table where it is used.

Some tables use additional conventions as necessary. They are explained in the text that precedes the respective table.

Table 143 Notation for user-supplied values

Specification	Description
<(x,y)> <[x,y]> <(x,y)> <[x,y]>	Range of floating-point numbers from x to y . Parentheses are used when the limits are excluded from the range; brackets are used when they are included. To denote ranges unbound on one side, the corresponding limit is omitted.
<carrier>	A carrier type, Electron or Hole .
<float>	A floating-point number (integers are special cases thereof).
<ident>	An identifier. This is like a string, but is not enclosed in double quotation marks.
<int>	An integer.
<n..m>	A range of integers, from n to m , inclusively. Either n or m can be omitted, to denote ranges that are unbound on one side.
<string>	A sequence of characters (including digits and special characters) included in double quotation marks. Unlike most Sentaurus Device input, strings are case sensitive.
<vector>	A sequence of up to three floating-point numbers, enclosed in parentheses, and separated by spaces, for example: (1.0 3 0).

Top Level of Command File

[Table 144](#) lists the top levels in the command file of Sentaurus Device.

Table 144 Top level of command file

Table 145		Specification for single devices.
Device	<string> { Table 145 }	Device name and device specifications (mixed mode). Device Section on page 86
File{	Table 146 }	File specification. File Section on page 94
Math{	Table 170 }	Math parameters. Mixed-Mode Math Section on page 96
Physics{	Table 182 }	Global Physics specification.
Solve{	Table 147 }	The problem to be solved.
System{	Table 165 }	The circuit (mixed mode). System Section on page 87

Device

Table 145 Device{} or single-device specification [Chapter 2 on page 47](#)

CurrentPlot{	<ident>	Use current plot PMI. Current Plot File of Sentaurus Device on page 1132
	PMIModel (Table 255)	Use current plot PMI with parameters. Current Plot File of Sentaurus Device on page 1132
	Table 140 (Table 265)	Plot scalar data to current file. Tracking Additional Data in the Current File on page 140
	Table 141 /Vector(Table 265)}	Plot vectorial data to current file. Tracking Additional Data in the Current File on page 140
Electrode{	Table 167 }	Electrode specification. Specifying Electrical Boundary Conditions on page 99
eSHEDistributionPlot{	<vector>...}	Electron-energy distribution plot. Visualizing Spherical Harmonics Expansion Method on page 645
Extraction{	<ident> [=] <ident> ...}	Process parameters. Extraction File on page 149
FarfieldPlot{	Table 266 }	Parameters for far-field plots (laser).
File{	Table 146 }	Input and output files.

Table 145 Device{} or single-device specification [Chapter 2 on page 47](#)

GainPlot{	Table 267 }	Parameters for gain plot (laser and LED).
GridAdaptation({ Criteria { Table 172 } }	Table 174)	Perform grid adaptation. Adaptive Device Instances on page 938
hSHEDistributionPlot{	<vector>...}	Hole-energy distribution plot. Visualizing Spherical Harmonics Expansion Method on page 645
Math	[(Table 273)]	Math specification, potentially restricted to a location.
{	Table 170 }	
MonteCarlo{	options}	See the <i>Sentaurus Device Monte Carlo User Guide</i> .
NoisePlot{	Table 269 }	Noise output data. Noise Output Data on page 601
NonLocalPlot	(<vector>...){ Table 270 }	Nonlocal plot. Visualizing Data Defined on Nonlocal Meshes on page 167
Physics	[(Table 273)]	Physical models, potentially restricted to a location.
{	Table 182 }	
Plot{	Table 263 }	Plot data. Device Plots on page 144
RayTraceBC{	Table 168 }	Raytrace boundary conditions (photodetector and LED).
TensorPlot({	Table 271)	Tensor plot for beam propagation method. Visualizing Results on Native Tensor Grid on page 567
	ComplexRefractiveIndex	
	OpticalField	
	OpticalIntensity}	
Thermode{	Table 169 }	Thermode specification. Specifying Thermal Boundary Conditions on page 101
TrappedCarDistrPlot{	<vector>	Trapped carrier charge plot at position <vector>. Visualizing Traps on page 419
	Table 276 ={<vector>...}	Trapped carrier charge plot restricted to location.
VCSELNearFieldPlot{	Table 272 }	Parameters for VCSEL near-field plots (laser).

File

In the description of [Table 146](#), it is indicated whether a file is input or output, as well as the extensions (in Courier font) that Sentaurus Device appends to the user-supplied name to obtain the full name. The tags enclosed in at-signs (@) denote the default Sentaurus Workbench variables for the particular file name. (These defaults can be changed in `tooldb.tcl`, see [Sentaurus Workbench User Guide, Global Configuration Files on page 206](#)). ‘(g)’ denotes keywords that must be used in global File sections only, while ‘(d)’ denotes keywords that must be used in device-specific File sections only.

Table 146 File{} [Physical Models and the Hierarchy of Their Specification on page 57](#)

ACExtract	=<string>	(g) Small-signal and noise analysis (output, <code>_ac_des.plt, @acplot@</code>). Small-Signal AC Analysis on page 127
Bandstructure	=<string>	(d) Base name for plotting local band structure data in a laser or an LED simulation (output, <code>_kpbandstruc_vertexX_des.plt, _kpeigenfunc_vertexX_des.plt</code>).
CMIPath	=<string>	(g) Search path for compact circuit files (extension <code>.ccf</code>) and the corresponding shared object files (extension <code>.so.arch</code>). The files are parsed and added to the System section of the command file. If the environment variable <code>STROOT_ARCH_OS_LIB</code> is defined, the directory <code>\$STROOT_ARCH_OS_LIB/sdevice</code> is automatically added to <code>CMIPath</code> . System Section on page 87
Current	=<string>	Device currents, voltages, charges, temperatures, and times (output, <code>_des.plt, @plot@</code>). Current File on page 137
DephasingRates	=<string>	Directory for saving dephasing rates using the second Born approximation.
DevFields	=<string>	(d) Space distribution for trapped carrier charge density. Must match grid file (input <code>.tdr</code>). Energetic and Spatial Distribution of Traps on page 408
DevicePath	=<string>	(g) Load all files with the extension <code>.device</code> in the directory path <code><string></code> . The directory path has the format <code>dir1:dir2:dir3</code> . The devices found can be used in the System section. They are not overwritten by a definition with the same name in the command file. System Section on page 87
EmissionTable	=<string>	Tabulated emission data (output).
EMWgrid	=<string>	(d) Tensor grid for Sentaurus Device Electromagnetic Wave Solver (EMW) (output).

Table 146 File{} [Physical Models and the Hierarchy of Their Specification on page 57](#)

EMWinput	=<string>	(d) EMW command file.
eSHEDistribution	=<string>	Electron distribution versus kinetic energy (output, <code>_des.plt</code>). Visualizing Spherical Harmonics Expansion Method on page 645
Extraction	=<string>	(d) Extraction file (output, <code>extraction_des.xtr</code>). Extraction File on page 149
Gain	=<string>	(d) Modal stimulated and spontaneous emission spectra (output, <code>_gain_des.plt</code>), (d) Laser power spectra if specified (output, <code>_powspec_des.dat</code>) Plotting Gain and Power Spectra on page 919
Grid	=<string>	(d) Device geometry and mesh (input, <code>.tdr,@grid@</code> or <code>@tdr@</code>). Exception for grid adaptation: base name for output grid files. Specifying Grid Adaptations on page 937
hSHEDistribution	=<string>	Hole distribution versus kinetic energy (output, <code>_des.plt</code>). Visualizing Spherical Harmonics Expansion Method on page 645
IlluminationSpectrum	=<string>	Illumination spectrum. Illumination Spectrum on page 472
LifeTime	=<string>	(d) Lifetime profiles (input, <code>.tdr</code>). Lifetime Profiles from Files on page 361
Load	=<string>	Old simulation results (input, <code>.sav</code>). Save and Load on page 174
MesherInput	=<string>	Basename of boundary and command file to be used for calling Sentaurus Mesh. Specifying the Optical Solver on page 480
MobilityDoping	=<string>	(d) File from which donor and acceptor concentrations for mobility calculations are read. Mobility Doping File on page 354
NewtonPlot	=<string>	Convergence monitoring (output). NewtonPlot on page 173
NonLocalPlot	=<string>	Data defined on nonlocal line meshes (output). Visualizing Data Defined on Nonlocal Meshes on page 167
OptFarField	=<string>	(d) Optical far field (output, <code>_ff_<number>_des.plt</code> for 1D, <code>_ff_des.grd</code> for 2D observation angle grid, <code>_ff_Scalar_<number>_des.dat</code> for 2D scalar far-field, <code>_ff_Vector_<number>_des.dat</code> for 2D vector far-field). Far Field on page 878
OptField0 to OptField9	=<string>	(d) File from which optical field data is loaded in a laser simulation. Loading Optical Modes on page 876

Table 146 File{} Physical Models and the Hierarchy of Their Specification on page 57

OpticalGenerationFile	=<string>	Load optical generation from a file. Optical AC Analysis on page 568
OpticalGenerationInput	=<string>	File from which optical generation rate is loaded. Loading and Saving Optical Generation from and to File on page 474
OpticalGenerationOutput	=<string>	File to which optical generation rate is written. Loading and Saving Optical Generation from and to File on page 474
OpticalSolverInput	=<string>	Command file of optical solver. Specifying the Optical Solver on page 480
Output	=<string>	"output" (g) Run-time log (output, _des.log, @log@).
Parameters	=<string>	(d) Device parameters (input, .par, @parameter@). Physical Model Parameters on page 60
Piezo	=<string>	(d) Stress data for piezoelectric model (input). Chapter 30 on page 687
Plot	=<string>	Spatially distributed simulation results (output, _des.tdr, @dat@). Device Plots on page 144
PMIPath	=<string>	(g) Search path to load shared object files (extension .so.arch) for PMI models. Command File of Sentaurus Device on page 989
PMIUserFields	=<string>	(d) File containing any of the fields PMIUserField0 to PMIUserField99.
TensorPlot	=<string>	Base name for saving tensor plot files when using beam propagation method. Visualizing Results on Native Tensor Grid on page 567
Save	=<string>	Simulation results for retrieval with Load (output, _des.sav). Save and Load on page 174
SaveOptField	=<string>	Base name for saving optical field data in a laser or an LED simulation. Saving Optical Modes on Optical or Electrical Mesh on page 875
SPICEPath	=<string>	Search path for SPICE circuit files (extension .Scf). The files are parsed and added to the System section of the command file. If the environment variable \$STROOT_LIB is defined, the directory \$STROOT_LIB/sdevice/spice is automatically added to SPICEPath. SPICE Circuit Models on page 95
SponEmissionTable	=<string>	File containing tabulated spontaneous emission values to be used in a laser or an LED simulation (input). Loading Tabulated Stimulated and Spontaneous Emission on page 913

Table 146 File{} [Physical Models and the Hierarchy of Their Specification on page 57](#)

StimEmissionTable	=<string>	File containing tabulated stimulated emission values to be used in a laser or an LED simulation (input). Loading Tabulated Stimulated and Spontaneous Emission on
TrappedCarPlotFile	=<string>	Trapped carrier charge density, trap occupancy probability, and trap density versus energy (output, <code>_des.plt</code>). Visualizing Traps on page 419

Solve

Table 147 Solve{}

Table 160		Solve selected stand-alone problem.
[<ident>.] Table 152		Solve selected equation [for instance <ident>].
ACCoupled(	Table 148)	Perform small-signal and noise analysis. Small-Signal AC Analysis on page 127
{	[<ident>.] Table 152 ...	Equations to be solved consistently.
Continuation(	Table 149)	Continuation options.
{	Table 147 }	Problem to be solved.
Coupled(	Table 150)	Solve coupled equations consistently by Newton iteration.
{	[<ident>.] Table 152 ...	Equations to be solved.
CurrentPlot	as Load	Control current output. When to Write to the Current File on page 137
HBCoupled(	Table 154)	Perform harmonic balance analysis. Harmonic Balance on page 130
{	[<ident>.] Table 152 ...	Equations to be solved.
Load(	Table 155)	Load and continue with solution stored by Save . Save and Load on page 174
[{	Circuit	Load circuit.
	<ident>...}]	Devices to be loaded.
NewCurrentPrefix	=<string>	Use prefix <string> for current file. NewCurrentPrefix Statement on page 139
Plot(	Table 155)	Plot current solution. When to Plot on page 145
[{	<ident>...}]	Devices to be plotted.

Table 147 Solve{}

Plugin({	Table 157)	Solve equations consistently by Gummel iteration.
	Table 160	Solve selected stand-alone problem.
	[<ident>.] Table 152...	Solve selected equation [for example, <ident>].
	Coupled(Table 150){ [<ident>.] Table 152... ...}	Solve coupled equations.
Quasistationary({	Table 158)	Quasistationary simulation options.
	Table 147 }	Problem to be solved.
Save	as Load	Save current solution for retrieval with Load. Save and Load on page 174
Set([+]System	Table 159) (<string>)	Set status according to Table 159 . Execute UNIX command <string> [and use its return status to decide whether it was successful]. System Command on page 187
Transient({	Table 162)	Transient simulation options.
	Table 147 }	Problem to be solved.
Unset	(<ident>...)	Unset nodes. Mixed-Mode Electrical Boundary Conditions on page 101
	(TrapFilling)	Use the standard trap equations. Explicit Trap Occupation on page 421

Table 148 ACCoupled() [Small-Signal AC Analysis on page 127](#)

Table 150		Coupled() parameters.
ACCompute(ACExtract	Table 155) =<string>	Restrict AC or noise analysis to selected points in a Quasistationary command. Prefix for the name of the file to which the results of AC analysis are written (overrides the specification from the File section).
ACMethod	=Blocked	Use Blocked solver.
ACPlot	=<string>	Plot the responses of the solution variables to the AC signals. <string> is a prefix for the names of the files to which the responses are written.
ACSubMethod	(<ident>)= Table 176	Super Inner solver for device <ident>.

Table 148 ACCoupled() [Small-Signal AC Analysis on page 127](#)

CircuitNoise		Compute noise from circuit elements. Noise from SPICE Circuit Elements on page 591
Decade		Use logarithmic intervals between the frequencies.
EndFrequency	=<(0,)>	! Hz Upper frequency.
Exclude	(<ident>...)	Devices that will be removed from the circuit for AC analysis.
Extraction{	Table 15 }	List of frequency-dependent extraction curves. Extraction File on page 149
Linear		Use linear intervals between the frequencies.
Node	(<ident>...)	Nodes for which to perform AC analysis.
NoisePlot	=<string>	Prefix for a file name. Chapter 23 on page 581
NumberOfPoints	=<0..>	! Number of frequencies for which to perform the analysis.
ObservationNode	(<ident>...)	Nodes for which to perform noise analysis; subset of those in Node . Chapter 23 on page 581
Optical		Perform optical AC analysis. Optical AC Analysis on page 130
PhotonicAC		Perform intensity and photon phase AC analysis. Laser Small-Signal AC Analysis and Modulation Response on page 783
PhotonicIntensityNoise		Compute relative intensity noise. Modeling Relative Intensity Noise and Frequency Noise on page 785
PhotonicPhaseNoise		Compute frequency noise. Modeling Relative Intensity Noise and Frequency Noise on page 785
StartFrequency	=<(0,)>	! Hz Lower frequency.

Table 149 Continuation() [Continuation Command on page 116](#)

BreakCriteria{	Table 171 }	Break criteria. Break Criteria: Conditionally Stopping the Simulation on page 102
Decrement	=<float>	1.5 Divisor for step size on failure to solve.
DecrementAngle	=<float>	5 deg Angle for which step starts decreasing.
Digits	=<float>	3 Number of digits for relative error.
Error	=<float>	0.05 Absolute error target.
Iadapt	=<float>	! Lower current limit for adaptive algorithm.

Table 149 Continuation() [Continuation Command on page 116](#)

Increment	=<float>	2 Multiplier for step size on successful solve.
IncrementAngle	=<float>	2.5 deg Angle up to which step increases.
InitialVstep	=<float>	! Initial voltage step.
MaxCurrent	=<float>	! Upper current limit.
MaxIfactor	=<float>	! Maximum-allowed current relative to previous point as a multiplication factor.
MaxIstep	=<float>	! Maximum-allowed current step.
MaxLogIfactor		! Maximum-allowed current relative to previous point as a multiplication factor, in orders of magnitude.
MaxStep	=<float>	Maximum step for the internal arc length variable.
MaxVoltage	=<float>	! Upper voltage limit.
MaxVstep	=<float>	! Maximum-allowed voltage step.
MinCurrent	=<float>	! Lower current limit.
MinStep	=<float>	1e-5 Minimum step for the internal arc length variable.
MinVoltage	=<float>	! Lower voltage limit.
MinVoltageStep	=<float>	1e-2 Minimum voltage step controlling arc length step increase.
Name	=<ident>	Electrode to bias in continuation. It must be specified.
Normalized		Compute angles in a local scaled I-V coordinate system.
Rfixed	=<float>	0.001 Fixed resistor value.

Table 150 Coupled() [Coupled Command on page 158](#)

Digits	=<float>	Relative error target.
(	GridAdaptation	Perform grid adaptation. Adaptive Solve Statements on page 943
	CurrentPlot	- Plot data obtained on intermediate grids to current file.
	MaxCLoops=<int>	1e5 Maximum number of adaptation iterations.
	Plot)	- Plot device data on intermediate grids.

Table 150 Coupled() [Coupled Command on page 158](#)

IncompleteNewton		Incomplete Newton. Incomplete Newton Algorithm on page 161
(	RhsFactor=<float>	Maximum change in RHS to allow old Jacobian to be reused.
	UpdateFactor=<float>)	Maximum change in update to allow old Jacobian to be reused.
Iterations	=<0..>	Maximum number of iterations of the Newton algorithm.
LineSearchDamping	=<(0,1]>	Minimal coefficient for line search damping. 1 disables damping. Damped Newton Iterations on page 161
Method	= Table 176	Linear solver.
	=Blocked	Block decomposition solver.
NotDamped	=<0..>	Number of Newton iterations before Bank–Rose damping is applied. Damped Newton Iterations on page 161
RhsAndUpdateConvergence		Require both RHS and update error convergence.
RhsMin	=<float>	Upper bound for RHS norm convergence.
SubMethod	[(<ident>)]= Table 176	Super Inner solver [for device <ident>].
UpdateIncrease	=<float>	Maximum-allowed increase of update error per Newton step.
UpdateMax	=<float>	Maximum-allowed update error in Newton algorithm.

Table 151 Cyclic() [Large-Signal Cyclic Analysis on page 124](#)

Accuracy	=<float>	Tolerance ϵ_{cyc} .
Extrapolate(	Average	+ Use averaged factor r_{av}^o for each object. Factor r is defined separately for each vertex.
	Factor=<float>	1 Coefficient f for r_{av}^o estimation.
	Forward	- Proceed as in a standard transient, without cyclic extrapolation.
	MaxVal=<float>	25 Value of r_{max} .
	MinVal=<float>	1 Value of r_{min} .
	Print	+ - Print averaged factors r_{av}^o for each object.
	QFtraps)	- Apply extrapolation to ‘trap quasi-Fermi level’ Φ_T instead of trap occupation probabilities f_T .

Table 151 Cyclic() [Large-Signal Cyclic Analysis on page 124](#)

Period	=<float>	s Period of the cycle.
RelFactor	=<float>	Relaxation factor γ .
StartingPeriod	=<2..>	2 Period from which the extrapolation procedure starts.

In the description column of [Table 152](#), the default for ErrRef (first number; see [Table 170 on page 1243](#)) and its unit, and the default absolute error criterion (second number, see [Table 170](#)) are given.

Table 152 Equations

Charge	1.602192e-19 C 1e-3 Floating gate charge, available for TransientErrRef and TransientError only. Floating Gates on page 971
Circuit	0.0258 V 1e-3 Circuit equations.
CondInsulator	0.0258 V 1e-3 Conductive insulator equations.
Contact	0.0258 V 1e-3 Contact equations.
Electron	1e10 cm ⁻³ 1e-5 Electron continuity equation. Chapter 8 on page 197
eQuantumPotential	0.0258 V 1e-3 Electron quantum-potential equation. Density Gradient Quantization Model on page 288
eSHEDistribution	1 V 1e-3 Electron SHE distribution equation. Using Spherical Harmonics Expansion Method on page 638
eTemperature	300 K 1e-4 Electron temperature equation. Hydrodynamic Model for Temperatures on page 211
Hole	1e10 cm ⁻³ 1e-5 Hole continuity equation. Chapter 8 on page 197
hQuantumPotential	0.0258 V 1e-3 Hole quantum potential equation. Density Gradient Quantization Model on page 288
hSHEDistribution	1 V 1e-3 Hole SHE distribution equation. Using Spherical Harmonics Expansion Method on page 638
hTemperature	300 K 1e-4 Hole temperature equation. Hydrodynamic Model for Temperatures on page 211
HydrogenAtom	1e10 cm ⁻³ 1e-3 Hydrogen atom transport equation. Hydrogen Transport on page 450
HydrogenIon	1e10 cm ⁻³ 1e-3 Hydrogen ion transport equation. Hydrogen Transport on page 450
HydrogenMolecule	1e10 cm ⁻³ 1e-3 Hydrogen molecule transport equation. Hydrogen Transport on page 450
PhotonPhase	Photon phase equation. Photon Rate and Photon Phase Equations on page 780

Table 152 Equations

PhotonRate	Photon rate equation. Photon Rate and Photon Phase Equations on page 780
PhotonRecycle	Photon-recycling effect.
Poisson	0.0258 V 1e-3 Poisson equation. Electrostatic Potential on page 191
SingletExciton	Singlet exciton equation. Singlet Exciton Equation on page 235
TCircuit	Thermal circuit equations.
TContact	Thermal contact equations.
Temperature	300 K 1e-3 Temperature equation. Chapter 9 on page 205

Table 153 Goal{} Quasistationary Ramps on page 106

Charge	=<float>	C Target charge for contact.
Contact	=<string1>.<string2>	Name of instance and contact to be ramped.
Current	=<float>	$\text{A}\mu\text{m}^{d-3}$ Target current for contact.
Device	=<string>	Name of the device where the parameter will be ramped.
DopingWell	(<vector>)	Semiconductor well defined by point <vector> where quasi-Fermi potential will be ramped.
DopingWells	(Region=<string>)	Semiconductor wells in the region <string> where quasi-Fermi potential will be ramped.
	(Material=<string>)	Semiconductor wells in material <string> where quasi-Fermi potential will be ramped.
	(Semiconductor)	All device semiconductor wells where Fermi potential will be ramped.
eQuasiFermi	=<float>	V Target electron quasi-Fermi potential for semiconductor wells. Ramping Quasi-Fermi Potentials in Doping Wells on page 107
hQuasiFermi	=<float>	V Target hole quasi-Fermi potential for semiconductor wells. Ramping Quasi-Fermi Potentials in Doping Wells on page 107
Material	=<string>	Name of material where the parameter will be ramped.
MaterialInterface	=<string>	Name of material interface where parameter will be ramped.
Model	=<string>	Name of model for which the parameter will be ramped.

Table 153 Goal{} Quasistationary Ramps on page 106

ModelParameter	=<string>	Parameter path or name of parameter that is ramped when using Parameter Ramping on page 494 .
	=<string1>.<string2>	Name of instance and parameter path or name of parameter that is ramped when using Parameter Ramping on page 494 .
Name	=<string>	Name of contact to be ramped.
Node	=<string>	Name of node for which the voltage will be ramped.
Parameter	=<string>	Name of parameter that will be ramped.
	=<string1>.<string2>	Name of instance and parameter that will be ramped.
Power	=<float>	Target heat for contact.
Region	=<string>	Name of region where the parameter will be ramped.
RegionInterface	=<string>	Name of region interface where parameter will be ramped.
Temperature	=<float>	K Target temperature for contact.
Value	=<float>	Target value for parameter.
Voltage	=<float>	V Target voltage for node or contact.
WellContactName	=<string>	Name of contact defining the well where quasi-Fermi potential will be ramped.

Table 154 HBCoupled() [Harmonic Balance on page 130](#)

Table 150		Coupled() parameters.
CNormPrint		+ Print instance equation errors per Newton step (MDFT mode only).
Derivative		- Use complete Jacobian for HB Newton.
GMRES(		Use GMRES linear solver; requires Method=ILS .
	MaxIterations=<int>	200 Maximal number of iterations.
	Restart=<int>	20 Size of minimization subspace.
	Tolerance=<float>)	1e - 4 Required residuum reduction.
Initialize	=DCMode	0-th harmonic from DC solution; others, zero.
	=HBMode	All harmonics from previous harmonic balance (HB) solution.
	=MixedMode	0-th harmonic from DC; others, from previous HB solution.
Method	=ILS	Required for GMRES .
	=Pardiso	Use PARDISO to solve linear system (SDFT mode only).

Table 154 HBCoupled() [Harmonic Balance on page 130](#)

Name	=<string>	Name component of plot files. Default is Coupled_<number> , where <number> is the global index of all Coupled , ACCoupled , and HBCoupled solve entries.
RhsScale	(Table 152)=<float>	Scaling of RHS in Newton (MDFT mode only).
SolveSpectrum	=<string>	Referenced solve spectrum (MDFT mode only).
Tone(		Multiple specification for multitone (MDFT mode only).
	Frequency=<freqspec>	! Hz Base frequency. Performing Harmonic Balance Analysis on page 132
	NumberOfHarmonics=<int>)	! Number of harmonics H . Eq. 25, p. 131
UpdateScale	(Table 152)=<float>	Scaling of update in Newton (MDFT mode only).
ValueMin	(Table 152)=<float>	Lower bound for quantity in time domain (MDFT mode only).
ValueVariation	(Table 152)=<float>	Allowed variation of quantity in time domain (MDFT mode only).

Table 155 Load(), Plot(), Save() in Solve{} [When to Plot on page 145](#)

Current	(Difference=<float>)	A Only for Continuation simulations. Write save or plot files when the difference between the actual and previous plotting current values on the continuation contact is greater than the specified Difference .
	(Intervals=<int>)	A Only for Continuation simulations. Divide the range specified with MinCurrent and MaxCurrent of Continuation into <int> intervals, and write save or plot files every time the current enters one of these intervals. When LogCurrent is specified in Continuation section, logarithmic ($\text{asinh}(10^{100} \text{ current})$) current range is used. Write save or plot file when the current enters a new logarithmic interval.
	(LogDifference=<(0,)>)	A Only for Continuation simulations. LogCurrent must be specified in Continuation section. Write save or plot files when the difference between the actual and previous logarithmic plotting current values is greater than the specified Difference .
Explicit		- Write datasets specified in Plot section only.

Table 155 Load(), Plot(), Save() in Solve{} [When to Plot on page 145](#)

FilePrefix	=<string>	Prefix of file name. File names consist of the prefix, the instance name, an optional local number (depending on Overwrite and noOverwrite), and an extension. The default file prefix is save<globalsaveindex> for Save and plot<globalplotindex> for Plot .
Iterations	=(<int>;...)	(0;1; . . .) Save or plot at certain Plugin iterations.
IterationStep	=<int>	Save or plot all <int> steps of plugins, quasistationaries, continuations, or transients.
Loadable		+ Write additional information required to load a simulation from a .tdr file.
noOverwrite		off Give each new save or plot file a new name by numbering.
Number	=<int>	Number of the solution to be retrieved by Load .
Overwrite		on Rewrite the same file name at each loop.
Time=(	<float>	Time point for plot. When to Plot on page 145
	decade	Use logarithmic range subdivision. When to Plot on page 145
	intervals=<int>	Number of subdivisions. When to Plot on page 145
	range=(<float>*2); ...)	Plotting range. When to Plot on page 145
Voltage	(Difference=<float>)	V Only for Continuation simulations. Write save or plot files when the difference between current and previous plotting voltages is greater than the specified Difference .
	(Intervals=<int>)	Only for Continuation simulations. Divide the range specified with MinVoltage and MaxVoltage of Continuation into <int> intervals, and write save or plot files every time the voltage enters one of these intervals.
When(		Generate output whenever the target was crossed the between the previous and the current iteration i , $X_{i-1} < X_T \leq X_i$ or $X_{i-1} > X_T \geq X_i$. Available for Plot , Save , and CurrentPlot inside a Quasistationary , Transient , or Continuation .
	Contact=[<string>.]<string>	[Instance and] contact name for target. The instance name defaults to " ", appropriate for single-device simulation.
	Current=<float>	A Target current X_T .
	Node=<string>	Node name for target.
	Voltage=<float>)	V Target voltage X_T .

Table 156 MSConfigs() in Set() in Solve{} [Manipulating MSCs During Solve on page 437](#)

Frozen		+ Freeze MSCs.
MSConfig(		Set occupations for the specified MSC.
	Device=<string>	Device instance name to where the MSC belongs.
	Name=<string>	Name of MSC.
	State(Name=<string> Value=<float>)...)	Set occupation of MSC state <string> to given value.

Table 157 Plugin() [Plugin Command on page 164](#)

BreakOnFailure		Stop when an inner Coupled fails.
Digits	=<float>	Relative precision target.
Iterations	=<int>	Maximum number of iterations. 0 is used to perform one loop without error testing.

Table 158 Quasistationary() [Quasistationary Ramps on page 106](#)

AcceptNewtonParameter		Apply relaxed Newton parameter. Relaxed Newton Parameters on page 115
(	ReferenceStep=<float>)	1.e-9 Apply relaxed Newton parameter for smaller step of ramping parameter.
BreakCriteria{	Table 171 }	Break criteria. Break Criteria: Conditionally Stopping the Simulation on page 102
Decrement	=<float>	2 Divisor for the step size when last step failed.
DoZero		+ - The equations are solved for $t = 0$.
Extraction{	Table 15 }	List of voltage-dependent extraction curves. Extraction File on page 149
Extrapolate		Use extrapolation. Extrapolation on page 113
(	Order=<int>)	1 Order of extrapolation.
Goal{	Table 153 }	Goal for ramping.

Table 158 Quasistationary() [Quasistationary Ramps on page 106](#)

GridAdaptation (		Perform grid adaptation. Adaptive Solve Statements on page 943
	CurrentPlot	- Write currents obtained on intermediate grids.
	Iterations=(<i><int></i> ; <i>...</i>)	Adapt grid at given iterations.
	IterationStep= <i><int></i>	1 Adapt grid any <i><int></i> steps.
	MaxCLoops= <i><int></i>	1e5 Maximum number of adaptation iterations.
	Plot	- Plot device data on intermediate grids.
	Time=(Range=(<i><float></i> *2) ; <i>...</i>)	Adapt grid when time falls into a given range.
Increment	= <i><float></i>	2 Multiplier for the step size when last step was successful.
InitialStep	= <i><float></i>	0.1 Initial step size.
MaxStep	= <i><float></i>	1 Maximum step size.
MinStep	= <i><float></i>	0.001 Minimum step size.
NewtonPlotStep	= <i><float></i>	Upper limit for step size for which to write Newton plot files. NewtonPlot on page 173
Plot{	Intervals= <i><int></i>	! Number of intervals in Range . Saving and Plotting During a Quasistationary on page 112
	Range=(<i><float></i> *2)}	! Range of <i>t</i> for which to generate plots.
PlotBandstructure	as Plot	Plot band structure data in a laser or an LED simulation.
PlotFarField	as Plot	Optical far field. Far Field on page 878
PlotGain	as Plot	Plot stimulated and spontaneous emission data in laser and LED simulations. Modal Gain as Function of Energy/Wavelength on page 920
ReadExtrapolation		+ Try to use the extrapolation information from a previous Quasistationary if it is available and compatible. Extrapolation on page 113
SaveOptField	as Plot	Save optical field data in a laser or an LED simulation. Saving Optical Modes on Optical or Electrical Mesh on page 875
StoreExtrapolation		+ Store the extrapolation information internally at the end of the Quasistationary. Extrapolation on page 113

NOTE Multiple entries from [Table 159](#) must be specified in multiple `Set()` definitions, because `Set()` only takes a single option.

Table 159 `Set()` in `Solve{}`

<code><ident></code>	<code>=<float></code>	Set node. Mixed-Mode Electrical Boundary Conditions on page 101
<code><ident>.<string></code>	<code>=<float></code>	Set parameter <code><string></code> of compact circuit instance <code><ident></code> . Mixed-Mode Electrical Boundary Conditions on page 101
<code><ident></code>	<code>mode Charge</code>	Use charge boundary condition for contact <code><ident></code> . Changing Boundary Condition Type During Simulation on page 100
<code><ident></code>	<code>mode Current</code>	Use current boundary condition for contact <code><ident></code> . Changing Boundary Condition Type During Simulation on page 100
<code><ident></code>	<code>mode Voltage</code>	Use voltage boundary condition for contact <code><ident></code> . Changing Boundary Condition Type During Simulation on page 100
<code>MSConfigs(</code>	Table 156)	Set MSCs. Manipulating MSCs During Solve on page 437
<code>([Device=<string>] [MSConfig=<string>] [Transition=<string>] PreFactor</code>	<code>=<float>)...</code>	Set MSC transition prefactors for emission, capture, or both using <code>EPreFactor</code> , <code>CPreFactor</code> , or <code>PreFactor</code> , respectively. Manipulating Transition Dynamics on page 438
<code>eSHEDistributions(</code>	<code>[-]Frozen)</code>	Freeze or unfreeze the electron distribution function. Using Spherical Harmonics Expansion Method on page 638
<code>hSHEDistribution(</code>	<code>[-]Frozen)</code>	Freeze or unfreeze the hole distribution function. Using Spherical Harmonics Expansion Method on page 638
<code>HydrogenAtom</code>	<code>=<string></code>	Set the concentration of hydrogen atom. Hydrogen Transport on page 450
<code>HydrogenIon</code>	<code>=<string></code>	Set the concentration of hydrogen ion. Hydrogen Transport on page 450
<code>HydrogenMolecule</code>	<code>=<string></code>	Set the concentration of hydrogen molecule. Hydrogen Transport on page 450
<code>TrapFilling</code>	<code>=Table 164</code>	Set trap filling. Explicit Trap Occupation on page 421
<code>Traps(</code>	Table 164)	Set traps. Explicit Trap Occupation on page 421

Table 160 Standalone

DephasingRates	Dephasing rates for tabulated optical emission using second Born approximation.
EmissionTable	Tabulated stimulated and spontaneous emission.
Optics	Optical problem.
Wavelength	Update wavelength according to optical problem.

Table 161 Time conditions [Time-stepping on page 122](#), [When to Write to the Current File on page 137](#), [When to Plot on page 145](#)

<float>	s A time point.
Range=(<float>*2)	Interval on time axis.
Range=(<float>*2) Intervals=<int> [Decade Linear*]	Subdivided interval on time axis. Subdivision on logarithmic or linear scale.

Table 162 Transient() [Transient Command on page 119](#)

Table 180		Time step control.
AcceptNewtonParameter		Apply relaxed Newton parameter. Relaxed Newton Parameters on page 124
(	ReferenceStep =<float>)	1.e-9 s Apply relaxed Newton parameter for smaller time steps.
Cyclic(	Table 151)	Cyclic analysis. Large-Signal Cyclic Analysis on page 124
FinalTime	=<float>	s Final time.
InitialStep	=<float>	0.1 s Initial step size.
InitialTime	=<float>	0 s Start time.
MaxStep	=<float>	1 s Maximum step size.
MinStep	=<float>	0.001 s Minimum step size.
NewtonPlotStep	=<float>	Upper limit for step size for which to write Newton plot files. NewtonPlot on page 173
Plot{	Intervals=<int>	Number of intervals in Range .
	Range=(<float>*2)}	s Range of t for which to generate plots. Saving and Plotting During a Quasistationary on page 112

G: Command File Overview

Top Level of Command File

Table 162 Transient() [Transient Command on page 119](#)

ReadExtrapolation		+ Try to use the extrapolation information from a previous Transient if it is available and compatible. Extrapolation on page 113
StoreExtrapolation		+ Store the extrapolation information internally at the end of the Transient. Extrapolation on page 113
TurningPoints(	(<float1> <float2>)...	Time point <float1> and associated advancing time-step limit <float2>.
	(Condition (Time (Table 161 [; Table 161]...)) Value=<float>) ...)...	Time point conditions and associated advancing time-step limit Value.

Table 163 TrapFilling= in Set() in Solve {} [Explicit Trap Occupation on page 421](#)

0	Set trap occupation to be in equilibrium with zero electron and hole concentration.
-Degradation	Return trap concentrations to initial values. Device Lifetime and Simulation on page 446
Empty	Set all traps to empty.
Frozen	Keep the current trap occupation unchanged until the next Set or UnSet.
Full	Set all traps to fully occupied.
n	Set trap occupation to be in equilibrium with a very high electron and zero hole concentration.
p	Set trap occupation to be in equilibrium with a very high hole and zero electron concentration.

Table 164 Traps() in Set() in Solve {} [Explicit Trap Occupation on page 421](#)

[<string1>.]<string2>=<float>...	Set occupation of trap <string2> of device <string1> to specified value.
Frozen	+ Freeze traps.

System

Table 165 System{} [System Section on page 87](#)

<ident1>	<ident2> (<string>=<ident3>...)	Create device instance <ident2> from device <ident1> and connect electrode <string> to node <ident3>.
<ident1>	<ident2>(<ident3>...){ <ident4>=<pvalue>...}	Create instance <ident2> from parameter set <ident1>, connect terminals to nodes <ident3>, and override parameter <ident4> by <pvalue>, the type of which depends on the parameter.
ACPlot(	Table 166)	Define circuit quantities for AC analysis output. AC System Plot on page 93
Electrical	(<ident>...)	List of electrical nodes. System Section on page 87
HBPlot	[<string>](Table 166)	Define circuit quantities for HB analysis output. Harmonic Balance on page 130
Hint(	<ident>=<float>)	Set node only for the first solve. Set, Unset, Initialize, and Hint on page 92
Initialize(	<ident>=<float>)	Set node until first transient. Set, Unset, Initialize, and Hint on page 92 .
Plot	[<string>](Table 166)	Plot circuit quantities to file named <string>.
Set(	<ident>=<float>	Set node. Set, Unset, Initialize, and Hint on page 92
	<ident>.<string>=<float>)	Set parameter <string> of compact circuit instance <ident>.
Thermal	(<ident> ...)	List of thermal nodes. System Section on page 87
Unset(	<ident>)	Unset node. Set, Unset, Initialize, and Hint on page 92

Table 166 Plot(), ACPlot(), and HBPlot() in System{} [System Plot on page 93](#), [AC System Plot on page 93](#), [Harmonic Balance on page 130](#)

<ident>		Print the voltage at node <ident>.
freq	()	Print the current AC analysis frequency.
h	(<ident1> <ident2>)	Print the heat that exits device <ident1> through node <ident2>.
i	(<ident1> <ident2>)	Print the current that exits device <ident1> through node <ident2>.
p	(<ident1> <ident2>)	Print attribute <ident2> of circuit element <ident1>.
t	(<ident>)	Print the temperature at node <ident>.
	(<ident1> <ident2>)	Print the temperature difference between two given nodes.
time	()	Print the current time in transient analysis, or quasistationary t parameter for frequency-domain analysis.
v	(<ident>)	Print the voltage at node <ident>.
	(<ident1> <ident2>)	Print the voltage difference between two given nodes.

Boundary Conditions

Table 167 Electrode{} [Specifying Electrical Boundary Conditions on page 99](#)

AreaFactor	=<float>	1 Multiplier for electrode current. Reading a Structure on page 47
Barrier	=<float>	V Barrier voltage.
Charge	=<float>	C Charge for floating electrode. Voltage must not be specified. Floating Contacts on page 225
Current	=<float>	$A\mu m^{d-3}$ Current boundary condition. Voltage is used as initial guess only.
DistResist	=<float>	Ωcm^2 Distributed resistance. Resistive Contacts on page 222
	=SchottkyResist	Use Schottky contact resistance model. Resistive Contacts on page 222
eRecVelocity	=<[0,)>	2.573e6 cms^{-1} Electron recombination velocity. Schottky Contacts on page 219

Table 167 Electrode{} [Specifying Electrical Boundary Conditions on page 99](#)

Extraction{	bulk	Specify electrode as bulk for extraction purposes. Extraction File on page 149
	drain	Specify electrode as drain for extraction purposes. Extraction File on page 149
	gate	Specify electrode as gate for extraction purposes. Extraction File on page 149
	source}	Specify electrode as source for extraction purposes. Extraction File on page 149
FGcap=(	value=<float>	$F\mu m^{d-3}$ Additional capacitance for floating electrode. Floating Metal Contacts on page 225
	name=<string>)	Coupling to electrode <string>. Floating Metal Contacts on page 225
hRecVelocity	=<[0,)>	$1.93e6 \text{ cm s}^{-1}$ Hole recombination velocity. Schottky Contacts on page 219
Material	=<string>	Electrode material. Contacts on Insulators on page 218
	=<string> (N=<[0,)>)	cm^{-3} Electrode material with n-doping. Contacts on Insulators on page 218
	=<string> (P=<[0,)>)	cm^{-3} Electrode material with p-doping. Contacts on Insulators on page 218
Name	=<string>	Name of electrode.
Resist	=<float>	$\Omega\mu m^{3-d}$ Contact resistance. Resistive Contacts on page 222
Schottky		off Contact is a Schottky contact. Schottky Contacts on page 219
Voltage	=<float>	V Contact voltage.
Workfunction	=<float>	eV Electrode workfunction. Contacts on Insulators on page 218

Table 168 RayTraceBC{} [Boundary Condition for Raytracing on page 520](#)

Fresnel		Fresnel boundary condition.
Name	=<string>	Name of the reflectivity contact or photon-recycling contact.
LayerStructure{	<float> <string> [; <float> <string>]...}	Definition of multilayer structure used for TMM calculation. First column contains thickness of layer in μm . Second column contains material name of layer.

Table 168 RayTraceBC{} [Boundary Condition for Raytracing on page 520](#)

MapOptGenToRegions(	<string> <string> ...)	Specify list of regions to map the lumped TMM BC optical generation to.
PMIModel	=<ident> [(Table 278)]	Name of the PMI model associated with this BC contact.
QuantumEfficiency	=<float>	Define the quantum efficiency of the TMM BC optical generation.
ReferenceMaterial	=<string>	Definition of LayerStructure orientation. The topmost layer in the LayerStructure specification is connected to the region with material ReferenceMaterial .
ReferenceRegion	=<string>	Definition of LayerStructure orientation. The topmost layer in the LayerStructure specification is connected to the region with name ReferenceRegion .
Reflectivity	=<[0,1]>	0 Reflectivity.
Transmittivity	=<[0,1]>	0 Transmittivity.

Table 169 Thermode{} [Boundary Conditions for Lattice Temperature on page 228](#)

AreaFactor	=<float>	1 Multiplier for heat fluxes. Reading a Structure on page 47
Name	=<string>	Name of thermode.
Power	=<float>	Wcm^{-2} Heat flux boundary condition.
SurfaceConductance	=<float>	$\text{cm}^{-2}\text{K}^{-1}\text{W}$ Contact thermal conductance.
SurfaceResistance	=<float>	$\text{cm}^2\text{KW}^{-1}$ Contact thermal resistivity.
Temperature	=<float>	K Contact temperature.

Math

Table 170 Math{}

Table 180		Transient time-step control.
AcceptNewtonParameter		(g) Relaxed Newton parameters for Quasistationary (Relaxed Newton Parameters on page 115) and Transient (Relaxed Newton Parameters on page 124).
(	RhsAndUpdateConvergence	Require both RHS and update error convergence.
	RhsMin=<float>	Minimum norm of RHS.
	UpdateScale=<float>	Additional factor for the update error.
	)	
ACMethod	=Blocked	* Use block decomposition solver for ACCoupled .
ACSubMethod	= Table 176	Inner linear solver for Blocked method for ACCoupled .
AutomaticCircuitContact		on Poisson covers the circuit. Additional Equations Available in Mixed Mode on page 162
AutoOrientation	=(<int>...)	Miller indices for surface orientations supported by AutoOrientation . Changing Orientations Used with Auto-Orientation on page 76
AutoOrientationSmoothingDistance	=<float>	0.0 μm (r) Auto-orientation smoothing distance. Auto-Orientation Smoothing on page 76
AvalDerivatives		+ Compute analytic derivatives of avalanche generation. Derivatives on page 161
AverageBoxMethod		+ Use element-oriented element-intersection BM algorithm. If disabled, use quadrilateral BM algorithm. Box Method Coefficients on page 950
BoxCoefficientsFromFile	[(GrdNumbering)]	Try to read sections of the geometry file. Saving and Restoring Box Method Coefficients on page 957
BoxMeasureFromFile	[(GrdNumbering)]	Try to read sections of the geometry file. Saving and Restoring Box Method Coefficients on page 957

Table 170 Math{}

BoxMethodFromFile		+ Read Voronoï surface from this file if the grid file has a VoronoiFaces section. Box Method Coefficients on page 950
BreakAtIonIntegral	(<int> <float>)	1 1 Terminate the quasistationary simulation when the <int> largest ionization integrals are greater than <float>. Approximate Breakdown Analysis: Poisson Equation Approach on page 389
BreakCriteria{	Table 171 }	Break criteria. Break Criteria: Conditionally Stopping the Simulation on page 102
CDensityMin		3e-8 Acm ² (r) Current limit for parallel electric field computation. Driving Force Models on page 348
CheckUndefinedModels		+ (g) Check for undefined physical parameters. Undefined Physical Models on page 71
CNormPrint		(g) Convergence monitoring. CNormPrint on page 173
ComputeIonizationIntegrals		off Compute ionization integrals for paths that cross local field maxima in a semiconductor. Approximate Breakdown Analysis: Poisson Equation Approach on page 389
	(WriteAll)	off Output information for all computed paths.
ConstRefPot	=<float>	eV Value for ϕ_{ref} . Quasi-Fermi Potential with Boltzmann Statistics on page 193
CoordinateSystem{	Table 1 }	(g) Coordinate system of explicit coordinates in command file. Reading a Structure on page 47
cT_Range	=(<float*2)	10 80000 K (g) Lower and upper limit for carrier temperature. Numeric Parameters for Temperature Equations on page 215
CurrentPlot(	Digits=<float>	6 Number of digits in the names of quantities in the current plot file. CurrentPlot Options on page 143
	IntegrationUnit=<string>)	um Length unit for current plot integration. CurrentPlot Options on page 143
CurrentWeighting		off Compute contact currents using an optimal weighting scheme. Numeric Approaches for Contact Current Computation on page 201

Table 170 Math{}

Cylindrical	(<float>)	off Use the 2D mesh to simulate a 3D cylindrical device. The device is assumed to be rotationally symmetric around the vertical axis given by $x = \text{<float>} \mu\text{m}$. <float> must be less than or equal to the smallest horizontal device coordinate, and defaults to 0. Reading a Structure on page 47
	(xAxis=<float>)	This is same as (<float>).
	(yAxis=<float>)	The device is assumed to be rotationally symmetric around the horizontal axis given by $y = \text{<float>} \mu\text{m}$. <float> must be less than or equal to the smallest vertical device coordinate.
DensityIntegral	(<int>)	30 (g) Defines a number of Gauss–Laguerre quadrature integration points. Using Multivalley Band Structure on page 269
DensLowLimit	=<float>	$1\text{e}-100 \text{ cm}^{-3}$ (g) Lower limit for carrier densities. Numeric Parameters for Continuity Equation on page 200
Derivatives		+ Compute analytic derivatives of mobility and avalanche generation. Derivatives on page 161
Digits	=<float>	5 Approximate number of digits to which equations must be solved to be considered as converged. Coupled Error Control on page 159
	(NonLocal)=<float>	2 (ci) Accuracy for nonlocal tunneling currents. Nonlocal Tunneling Parameters on page 615
DirectCurrent		off Compute contact currents directly, using only contact nodes and their neighbors. Numeric Approaches for Contact Current Computation on page 201
eDrForceRefDens	=<[0,)>	0 cm^{-3} (r) Damping parameter for high-field mobility driving force. Driving Force Models on page 348
ElementEdgeCurrent		- (g) Use an alternative element-edge approximation of the current density compared to the default edge approximation.
eMobilityAveraging	=Element	* (r) Use element averaged electron mobility. Mobility Averaging on page 354
	=ElementEdge	(r) Use element-edge averaged electron mobility. Mobility Averaging on page 354

Table 170 Math{}

EnergyResolution	(NonLocal)=<(0,)>	0.005 eV (ci) Minimum energy resolution for integrals in computation of nonlocal tunneling current. Nonlocal Tunneling Parameters on page 615
EnormalInterface(	MaterialInterface=[<string>...]	Material interfaces for normal electric field computation. Normal to Interface on page 336
	RegionInterface=[<string>...])	Region interfaces for normal electric field computation. Normal to Interface on page 336
EquilibriumSolution(	Table 173)	(g) Parameters for equilibrium solution used with conductive insulators. Conductive Insulators on page 244
Error	(Table 152)=<float>	Value of ϵ_A . Coupled Error Control on page 159
ErrRef	(Table 152)=<float>	Value of x_{ref} . Coupled Error Control on page 159
ExitOnFailure		(g) off Terminate the simulation as soon as a Solve command fails.
ExitOnUnknownParameterRegion		+ (g) Exit when unknown region, region interface, or electrode appears in parameter file.
ExtendedPrecision	[(Table 20)]	(g) off Use extended precision floating-point arithmetic. Extended Precision on page 185
Extrapolate (		off Use extrapolation to obtain an initial guess for the next solution based on the previous solutions. Extrapolation on page 113 , Extrapolation on page 123
	Order=<int>)	1 Order of extrapolation in quasistationary command.
GeometricDistances		- (g) Use enhanced distance and normal to interface computation for mobility and MLDA. Normal to Interface on page 336
HB	{ Table 175 }	Harmonic Balance on page 130 ; not device specific.
hDrForceRefDens	=<[0,)>	0 cm ⁻³ (r) Damping parameter for high-field mobility driving force. Driving Force Models on page 348
hMobilityAveraging	=Element	* (r) Use element averaged hole mobility. Mobility Averaging on page 354
	=ElementEdge	(r) Use element-edge averaged hole mobility. Mobility Averaging on page 354

Table 170 Math{}

IgnoreTdrUnits		off (g) Ignore TDR units when loading data from a .tdr file. Reading a Structure on page 47
ILSrc	=<string>	ILS options. Linear Solvers on page 165
IncompleteNewton		- (g) Incomplete Newton. Incomplete Newton Algorithm on page 161
(	RhsFactor=<float>	10 Maximum change in RHS to allow old Jacobian to be reused.
	UpdateFactor=<float>)	0.1 Maximum change in update to allow old Jacobian to be reused.
Interrupt	=BreakRequest	Write .tdr or .sav file and abort actual solve statement after signal INT occurs. Snapshots on page 147
	=PlotRequest	Write .tdr or .sav file and continue simulation after signal INT occurs. Snapshots on page 147
Iterations	=<0..>	50 Maximum number of Newton iterations. Coupled Error Control on page 159
LineSearchDamping	=<(0,1]>	1 Smallest allowed damping coefficient for line search damping. Damped Newton Iterations on page 161
lT_Range	=(<float*2)	50 5000 K (g) Lower and upper limit for lattice temperature. Numeric Parameters for Temperature Equations on page 215
MeshDomain	<string> (<options>)...	Define a named mesh domain (see Table 177 for full description). Mesh Domains on page 942
MetalConductivity		+ Conductivity of metals. The thermal conductivity is simulated in any case. Transport in Metals on page 239
Method	= Table 176	Linear solver for Coupled .
	=Blocked	* Use block decomposition solver for Coupled . Linear Solvers on page 165
MLDAbox({	Table 178 }...)	A box within which the interface is confined for the calculation of the MLDA distance function. Modified Local-Density Approximation on page 293

Table 170 Math{}

MVMLDAcontrols (		Multivalley MLDA controlling options. Using MLDA on page 295
	AveDistanceFactor= <float>	Defines a factor that controls computation of an averaged distance from a vertex to the interface for the multivalley MLDA model.
	MaxDoping4Majority= <float>	Defines maximum doping concentration where the multivalley MLDA model is applied for majority carriers.
	MaxIntDistance= <float>)	Controls a distance from the interface where the multivalley MLDA model is applied.
NaturalBoxMethod		- Use edge-oriented element intersection BM algorithm. Box Method Coefficients on page 950
NewtonPlot (		(g) Convergence monitoring. NewtonPlot on page 173
	Error	Write the error of all solution variables.
	MinError	Write file only if error decreases.
	NewtonPlotStep= <float>	Upper limit for step size for which to write files.
	Plot	Write everything specified in the Plot section. Device Plots on page 144
	Residual	Write the residuals (right-hand sides) of all equations.
	Update)	Write the updates of all solution variables from the previous step.
NewWaveLengthSearch	[(InitialWaveLength= <float>)]	More robust wavelength search algorithm for laser.
NoAutomaticCircuitContact		off (g) Poisson excludes the circuit. Additional Equations Available in Mixed Mode on page 162
NonLocal	(Table 179)	(cir) Nonlocal mesh. Nonlocal Meshes on page 166
	<string> (Table 179)	(g) Named nonlocal mesh.
NonLocalLengthLimit	=<float>	1e-4 cm (g) Limit for nonlocal line length. Nonlocal Meshes on page 166
NormalFieldCorrection	=<float>	(r) Normal field correction factor for mobility on interface points. Field Correction on Interface on page 337

Table 170 Math{}

NoSRHperPotential		off Omit potential derivatives of SRH recombination rate. Using Field Enhancement on page 364
NoSRHperT		off Omit temperature derivatives of SRH recombination rate. Using Field Enhancement on page 364
NotDamped	=<0. .>	1000 (g) Number of iterations in each Newton iteration before Bank–Rose damping is activated. Damped Newton Iterations on page 161
Number_of_Assembly_Threads	=<int>	1 (g) Number of threads for linear solver. Parallelization on page 183
Number_of_Solver_Threads	=<int>	1 (g) Number of threads for assembly. Parallelization on page 183
Number_of_Threads	=<int>	1 (g) Number of threads for linear solver and assembly. Parallelization on page 183
Numerically	[(Table 152...)]	off (g) Use numeric derivatives. The optional list restricts the numeric computation to specific Jacobian blocks. Derivatives on page 161
ParallelLicense	(Table 19)	(g) Determine the behavior if insufficient parallel licenses are available.
ParallelToInterfaceInBoundaryLayer		+ (r) Controls the computation of driving forces for mobility and avalanche models along interfaces. Field Correction Close to Interfaces on page 349
(	ExternalBoundary	+ Include the external boundary in the interface for which this feature applies.
	ExternalXPlane	+ Include external x-planes in the interface for which this feature implies.
	ExternalYPlane	+ Include external y-planes in the interface for which this feature implies.
	ExternalZPlane	+ Include external z-planes in the interface for which this feature implies.
	FullLayer	off Apply switch to all elements that touch an interface either by a face, an edge, or a vertex.
	Interface	+ Include semiconductor–insulator region interfaces in the interface for which this feature applies.
	PartialLayer)	on Apply switch only to elements that touch an interface by an edge (2D) or a face (in 3D).

Table 170 Math{}

PeriodicBC ((		(g) Periodic boundary conditions (PBCs). Periodic Boundary Conditions on page 230
	Table 152	Equation for which to apply PBCs. If omitted, apply to all equations (RPBC only).
	Coordinates= (<float>*2)	μm Coordinate of Direction axis where the PBCs will be applied. Sentaurus Device replaces coordinates outside the device with coordinates at the outer boundary (RPBC only).
	Direction=<0..2>	Direction of periodicity: 0 for x-axis, 1 for y-axis, and 2 for z-axis.
	Factor=<float>	1e8 Tuning parameter α (RPBC only).
	MortarSide=<side>	XMin YMin ZMin Mortar side with <side> one of XMin, XMax, YMin, YMax, ZMin, or ZMax (MPBC only).
	Type=<type>...)	RPBC Select PBC mode where <type> is one of RPBC or MPBC.
PlotExplicit		- (g) All Plot statements write datasets specified in Plot section only.
PlotLoadable		+ (g) All Plot statements write additional information required to load a simulation from a .tdr file.
PostProcess	(Transient=<ident> [(Table 278)])	(g) Use PMI <ident> to postprocess data. Postprocess for Transient Simulation on page 1137
RandomizedVariation (		(g) Use statistical impedance field method. Statistical Impedance Field Method on page 592
	NumberOfSamples= <0..>	! Number of samples.
	Randomize=<int>)	0 Seed for random number generator.
RecBoxIntegr	(<float> <int> <int>)	(1e-2 10 1000) Maximum relative deviation of covered volume, maximum number of levels, maximum number of total rectangles per box.
RecomputeQFP		Keep density variables constant, and recompute quasi-Fermi potentials when the electrostatic potential changes and carrier equations are not solved. Introduction to Carrier Transport Models on page 197

Table 170 Math{}

RelErrControl		+ Use the unscaled expression Eq. 32, p. 160 for error control. Coupled Error Control on page 159
RelTermMinDensity	=<(0,)>	1e3 cm ⁻³ (g) Stabilization parameter for temperature relaxation term. Hydrodynamic Model Parameters on page 214
RelTermMinDensityZero	=<(0,)>	2e8 cm ⁻³ (g) Stabilization parameter for temperature relaxation term. Hydrodynamic Model Parameters on page 214
RhsAndUpdateConvergence		- Newton converged if both RHS and update are converged. Coupled Error Control on page 159
RhsFactor	=<float>	1e10 Maximum increase of the L_2 -norm of the RHS between Newton iterations. Coupled Error Control on page 159
RhsMax	=<float>	1e15 Maximum of L_2 -norm of the RHS in each Newton iteration. Used only during transient simulations. Coupled Error Control on page 159
RhsMin	=<float>	1e-5 Minimum of L_2 -norm of the RHS in each Newton iteration. Coupled Error Control on page 159
SHECutoff	=<[0,)>	5.0 (g) Maximum kinetic energy to be plotted in the SHE distribution model. Using Spherical Harmonics Expansion Method on page 638
SHEIterations	=<0..>	20 (g) Number of iterations in the SHE distribution model. Using Spherical Harmonics Expansion Method on page 638
SHEMethod	= Table 176	super (g) Linear solver for the SHE distribution model. Using Spherical Harmonics Expansion Method on page 638
SHERefinement	=<1..>	1 (g) Number of energy grid intervals inside the phonon energy spacing in the SHE distribution model. Using Spherical Harmonics Expansion Method on page 638
SHESOR		+ Use the successive over-relaxation method in the SHE distribution model. Using Spherical Harmonics Expansion Method on page 638
SHESORParameter	=<[1, 2)>	1.1 (g) Successive over-relaxation parameter in the SHE distribution model. Using Spherical Harmonics Expansion Method on page 638

Table 170 Math{}

SHETopMargin	=<[0,)>	1 . 0 (g) Top energy margin in the SHE distribution model. Using Spherical Harmonics Expansion Method on page 638
Smooth		off Keep mobility and recombination rates from the previous step to obtain better initial conditions for extreme nonlinear iterations.
Spice_gmin	=<float>	1e - 12 S (g) SPICE minimum conductance. Mixed-Mode Math Section on page 96
Spice_Temperature	=<float>	300 . 15 K (g) Temperature of SPICE circuit. Mixed-Mode Math Section on page 96
StackSize	=<int>	10000000 byte (g) Stack size per thread. Parallelization on page 183
StressLimit	=<float>	! Pa (g) Upper limit on stress values read from files or specified. Values exceeding the limit will be truncated to the limit.
StressMobilityDependence	=TensorFactor	Stress Tensor Applied to Low-Field Mobility on page 734
SubMethod	= Table 176	(g) Inner linear solver for B locked method. Linear Solvers on page 165
Surface	<string> (Table 275...)	(g) Define a surface. Mobility Degradation Components due to Coulomb Scattering on page 332 , Random Geometric Fluctuations on page 588
TensorGridAniso		off (g) Use tensor-grid approximation for anisotropic mobility. Tensor Grid Option on page 733
Transient	=BE	(g) Backward Euler method. Backward Euler Method on page 968
	=TRBDF	* (g) TRBDF method. TRBDF Composite Method on page 969
TrapDLN	=<int>	13 (g) Levels to approximate trap energy distribution. Energetic and Spatial Distribution of Traps on page 408
Traps(	Damping=<[0,)>	10 Damping for traps for the nonlinear Poisson equation. A value of 0 disables damping. Chapter 17 on page 407
	RegionWiseAssembly)	- Apply regionwise assembly for traps.
UpdateIncrease	=<float>	1.e100 (g) Maximum-allowed update error increase factor per Newton step.

Table 170 Math{}

UpdateMax	=<float>	1.e100 (g) Maximum-allowed update error in Newton algorithm.
VoronoiFaceBoxMethod	= Table 181	Use element-face intersection box method algorithm. Box Method Coefficients on page 950
Wallclock		- (g) Report wallclock times rather than CPU times after each step in the simulation.
WeightedVoronoiBox		off (g) Compute-weighted coefficients and measure. Weighted Box Method Coefficients on page 955

Table 171 BreakCriteria{ [Break Criteria: Conditionally Stopping the Simulation on page 102](#)

Current(	absval=<float>	$A \mu m^{d-3}$ Upper limit for absolute current value.
	contact=<string>	! Name of contact with break criterion.
	DevName=<ident>	Identifier of circuit device name (mixed mode only).
	maxval=<float>	$A \mu m^{d-3}$ Upper limit for current.
	minval=<float>	$A \mu m^{d-3}$ Lower limit for current.
	Node=<ident>)	Identifier of circuit node name (mixed mode only).
CurrentDensity(	DevName=<ident>	Identifier of device (mixed mode only).
	maxval=<float>)	$A cm^{-2}$ (r) Maximum current density.
DevicePower(	absval=<float>	$W \mu m^{d-3}$ Upper limit for absolute power value.
	DevName=<ident>	Identifier of circuit device name (mixed mode only).
	maxval=<float>	$W \mu m^{d-3}$ Upper limit for power value.
	minval=<float>)	$W \mu m^{d-3}$ Lower limit for power value.
ElectricField	DevName=<ident>	Identifier of circuit device name (mixed mode only).
	(maxval=<float>)	$V cm^{-1}$ (r) Maximum electric field.
LatticeTemperature	DevName=<ident>	Identifier of circuit device name (mixed mode only).
	(maxval=<float>)	K (r) Maximum lattice temperature.
Voltage	as Current	V Voltage break criteria.

Table 172 Criteria{} [Adaptation Criteria on page 940](#)

Element Residual		Parameters available for both E lement and R esidual criteria.
(	AbsError=<float>	Absolute error target.
	MaxElementSize=(<float>*d)	Maximal element size in axis directions.
	MeshDomain=<string>	Name of definition domain.
	MinElementSize=(<float>*d)	Minimal element size in axis directions.
	Name=<string>	Name of criterion for identification.
	RelError=<float>)	Relative error target.
Element		Parameters specific to E lement criterion.
(	DataSet="Table 140")	Dataset for criterion (must be defined on vertices).
	Logarithmic)	- Use logarithmic formula.

Table 173 EquilibriumSolution() [Conductive Insulators on page 244](#)

Iterations	=<0..>	50 Maximum number of Newton iterations. Coupled Error Control on page 159
LineSearchDamping	=<(0,1]>	1 Smallest allowed damping coefficient for line search damping. Damped Newton Iterations on page 161
NotDamped	=<0..>	1000 (g) Number of iterations in each Newton iteration before Bank–Rose damping is activated. Damped Newton Iterations on page 161

Table 174 GridAdaptation() in Device{} [Adaptive Device Instances on page 938](#)

AvaHomotopy (	Extrapolate	+ Use extrapolation during avalanche homotopy.
	Iterations=<int>	5 Number of Newton iterations in avalanche homotopy.
	LinearParametrization	+ Use linear parameterization of avalanche generation.
	Off)	- Disable avalanche homotopy.
MaxCloops	=<int>	1e5 Maximal number of iterations per adaptive coupled system.
Poisson		+ Perform EPC smoothing step.
Smooth		+ Perform NBJI smoothing step.
Weights(	Avalanche=<float>	1 Avalanche weight in AGM dissipation rate.
	eCurrent=<float>	1 Electron current weight in AGM dissipation rate.
	hCurrent=<float>	1 Hole current weight in AGM dissipation rate.
	Recombination=<float>)	1 Recombination weight in AGM dissipation rate.

Table 175 HB{} [Harmonic Balance on page 130](#)

MDFT		- Use MDFT mode.
RhsScale	(Table 152)=<float>	Scaling of RHS in Newton (MDFT mode only).
SolveSpectrum(	Name=<string>)	Solve spectrum with (mandatory) name.
{	(<int>* <i>n</i>)...}	List of spectrum multi-indices. <i>n</i> is the number of tones. Solve Spectrum on page 133
UpdateScale	(Table 152)=<float>	Scaling of update in Newton (MDFT mode only).
ValueMin	(Table 152)=<float>	Lower bound for quantity in time domain (MDFT mode only).
ValueVariation	(Table 152)=<float>	Allowed variation of quantity in time domain (MDFT mode only).

Table 176 Linear solvers ([Solvers User Guide](#))

ILS (Set=<int>)		Parallel, iterative linear solver, for single-device or multiple-device problems with over 8000 vertices.
	MultipleRHS	- Solve linear systems with multiple right-hand sides (only for AC analysis).
		Use ILS options from set <int>.
ParDiSo (IterativeRefinement IterativeRefinement=<int> MultipleRHS NonsymmetricPermutation RecomputeNonsymmetricPermutation)		Parallel, supernodal direct solver, for single-device problems with meshes up to 10 000 vertices.
	IterativeRefinement	- Perform up to two iterative refinement steps to improve the accuracy of the solution.
	IterativeRefinement=<int>	Perform up to <int> iterative refinement steps.
	MultipleRHS	- Solve linear systems with multiple right-hand sides (only for AC analysis).
	NonsymmetricPermutation	+ Compute an initial nonsymmetric matrix permutation and scaling, which places large matrix entries on the diagonal.
	RecomputeNonsymmetricPermutation)	- Compute a nonsymmetric matrix permutation and scaling before each factorization.
Super		Supernodal direct solver, for single-device problems with meshes up to 10000 vertices.

Table 177 MeshDomain{} in Math {} [Mesh Domains on page 942](#)

MeshDomain	<string> (Box((<float1>*d) (<float2>*d))... MeshDomain=<string>... Region=<string>... Type = Cap Cup*)	Define a named mesh domain.
		Domain covered by box given by minimum <float1> and maximum <float2> values in axis directions.
		Domain covered by referenced mesh domain.
		Domain covered by referenced region.
		Build intersection (Cap) or union (Cup).

Table 178 MLDAbox({}...) [Modified Local-Density Approximation on page 293](#)

MaxX	=<float>	μm Upper <i>x</i> -coordinate of the box.
MaxY	=<float>	μm Upper <i>y</i> -coordinate of the box.
MaxZ	=<float>	μm Upper <i>z</i> -coordinate of the box.
MinX	=<float>	μm Lower <i>x</i> -coordinate of the box.
MinY	=<float>	μm Lower <i>y</i> -coordinate of the box.
MinZ	=<float>	μm Lower <i>z</i> -coordinate of the box.

Table 179 Nonlocal() [Nonlocal Meshes on page 166](#)

Table 275		(g) Interface that is part of the reference surface.
Barrier	(Table 274...)	- (g) Regions that form the tunneling barrier.
Direction	=<vector>	(0 0 0) (cgi) If nonzero, suppress the construction of nonlocal mesh lines with a direction more than MaxAngle degrees different from <vector>.
Discretization	=<float>	1e100 cm (cgi) Maximum distance between nonlocal mesh points on a nonlocal line.
Electrode	=<string>	(g) Electrode that is part of the reference surface.
Endpoint		+ - (r) Allow nonlocal lines that end in the region. Default is Endpoint for semiconductors; otherwise, - Endpoint .
	(Table 274...)	+ - (g) Regions where nonlocal lines can or cannot end.
Length	=<float>	cm (cgi) Distance from the interface or contact up to which nonlocal mesh lines are constructed.
MaxAngle	=<float>	180 deg (cgi) Suppress construction of nonlocal mesh lines that enclose an angle of more than <float> degrees with the vector specified by Direction .
Outside		+ (cgi) Allow nonlocal mesh lines to leave the device.
Permeable		+ (r) Allow extension of nonlocal lines (as specified by the Permeation parameter) into or across the region.
	(Table 274...)	+ (g) Regions into or across which nonlocal lines can or cannot be extended.
Permeation	=<float>	0 cm (cgi) Length by which nonlocal mesh lines are extended across the interface or contact.
Refined		+ (r) Autorefine nonlocal lines inside the region.
	(Table 274...)	+ (g) Regions in which nonlocal lines are or are not autorefined.

Table 179 Nonlocal() [Nonlocal Meshes on page 166](#)

Transparent		+ (r) Allow nonlocal lines crossing the region.
	(Table 274...)	+ (g) Regions that nonlocal lines can or cannot cross.

Table 180 Transient time-step control [Numeric Control of Transient Analysis on page 121](#)

CheckTransientError		off Error control of transient integration method.
NoCheckTransientError		on No error control of transient integration method.
TransientDigits	=<float>	3 Relative accuracy for time-step control.
TransientError	(Table 152)= <float>	Absolute error for time-step control.
TransientErrRef	(Table 152)= <float>	Error reference for time-step control.
TrStepRejectionFactor	=<float>	Factor f_{rej} . Controlling Transient Simulations on page 970

Table 181 VoronoiFaceBoxMethod= [Box Method Coefficients on page 950](#)

MaterialBoundaryTruncated	Use truncated correction for obtuse elements that have an obtuse face on boundary of materials or that are not Delaunay.
RegionBoundaryTruncated	Use truncated correction for obtuse elements that have an obtuse face on boundary of regions or that are not Delaunay.
Truncated	Use truncated correction for all obtuse elements.
-Truncated	Use truncated correction only for non-Delaunay elements.

Physics

Table 182 Physics{} Part II on page 189

Affinity	(<ident> [(Table 278)])	(r) Use PMI model <ident> for electron affinity. Electron Affinity on page 1068
AlphaParticle(	Table 228)	(g) Generation by alpha particles. Alpha Particles on page 572
AnalyticTEP		(g) Analytic expression for thermoelectric power. Thermoelectric Power (TEP) on page 749
Aniso(	Table 231)	Anisotropic properties. Chapter 28 on page 665
AreaFactor	=<float>	1 (g) Multiplier for current and heat flux densities at electrodes and thermodes. Reading a Structure on page 47
BarrierLowering		off (c) Use barrier lowering for Schottky contact. Barrier Lowering at Schottky Contacts on page 220
ComplexRefractiveIndex(	)	off (g) Use complex refractive index model. Complex Refractive Index Model on page 496
CondInsulator		(r) Turn an insulator into a conductive insulator. Conductive Insulators on page 244
DefaultParametersFromFile		Initialize default parameters from parameter files instead of using built-in values. Default Parameters on page 73
DeterministicVariation(	Table 234)	(g) Deterministic variations. Deterministic Variations on page 595
Dipole(		(i) Use dipole interface model. Dipole Layer on page 192
	Reference=<string>	! Reference side <string> (either region or material name).
DistResist	=<float>	Ωcm^2 (i) Distributed resistance at metal–semiconductor interface. Transport in Metals on page 239
	=SchottkyResist	Emulate a Schottky interface. Transport in Metals on page 239
eBarrierTunneling	<string> [(Table 187)]	off (g) Nonlocal tunneling from and to conduction band. Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612

Table 182 Physics{} Part II on page 189

EffectiveIntrinsicDensity	(Table 236)	(r) Band gap and bandgap narrowing. Band Gap and Electron Affinity on page 249
EffectiveMass	(GaussianDOS)	(r) Use simplified Gaussian density-of-states model for organic semiconductors. Gaussian Density-of-States for Organic Semiconductors on page 264
	(<ident> [(Table 278)])	(r) Use PMI model <ident> for effective mass. Effective Mass on page 1071
eMLDA (	[()])	- (r) MLDA model for electrons. Modified Local-Density Approximation on page 293
	LambdaTemp)	+ (r) Use temperature-dependent λ for both electrons and holes.
eMobility(	Table 224)	(r) Electron mobility model. Chapter 15 on page 301
eMultiValley		off (r) Multivalley statistics. Multivalley Band Structure on page 267 , Multivalley Band Structure on page 699
eMultiValley()	MLDA	off (r) Multivalley MLDA quantization model. Modified Local-Density Approximation on page 293 , Using Multivalley Band Structure on page 699 , Inversion Layer on page 704
	Nonparabolicity	off (r) Band nonparabolicity. Nonparabolic Band Structure on page 268
	DensityIntegral	off (r) Numeric density computation. Using Multivalley Band Structure on page 269
EnergyRelaxationTimes	(<ident> [(Table 278)])	Use PMI model <ident> to compute energy relaxation times. Energy Relaxation Times on page 1075
	(constant)	Use constant energy relaxation times.
	(formula)	Use the value of formula in the parameter file.
	(irrational)	Use the ratio of two irrational polynomials. Energy-dependent Energy Relaxation Time on page 655
eQCvanDort		off (r) van Dort model for electrons. van Dort Quantization Model on page 280
eQuantumPotential	[(Table 238)]	- (r) Activate electron density gradient quantum correction. Density Gradient Quantization Model on page 288
eQuasiFermi	=<float>	V (r) Initial quasi-Fermi potential specification for electrons. Regionwise Specification of Initial Quasi-Fermi Potentials on page 196

Table 182 Physics{} Part II on page 189

eRecVelocity	=<[0,)>	2.573e6 cm s^{-1} (i) Electron recombination velocity. Electric Boundary Conditions for Metals on page 240
eSHEDistribution	(Table 239)	off (r) Specify the region where the electron SHE distribution is calculated. Using Spherical Harmonics Expansion Method on page 638
eThermionic		(i) Thermionic emission model for electrons. Conductive Insulators on page 244 , Thermionic Emission Current on page 651
	(HCI)	(i) Inject hot electrons locally. Destination of Injected Current on page 626
	(Organic_Gaussian)	(i) Thermionic-like Gaussian emission model at organic heterointerfaces for electrons. Gaussian Transport Across Organic Heterointerfaces on page 653
Fermi		off (g) Fermi statistics. Fermi Statistics on page 194
	(-WithJoyceDixon)	(g) Fermi statistics with old Wuensche approximation for Fermi integrals. Fermi Statistics on page 194
FloatCoef	=<float>	0 (g) Interpolation coefficient for initial guess in floating wells. Initial Guess for Electrostatic Potential and Quasi-Fermi Potentials in Doping Wells on page 195
GateCurrent(	Table 240)	(i) Gate currents (hot-carrier injection, some of the tunneling models).
GaussianDOS_full		(r) Use Gaussian density-of-states model for organic semiconductors. Gaussian Density-of-States for Organic Semiconductors on page 264
hBarrierTunneling	<string> [(Table 187)]	off (g) Nonlocal tunneling from and to valence band. Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612
HeatCapacity(	Table 242)	(r) Heat capacity.
HeatSource(	<ident>)	Name of the PMI model.
HeavyIon(	Table 229)	off (g) Generation by heavy ions. Heavy Ions on page 574
HeteroInterface		(i) Double points at interfaces. Abrupt and Graded Heterojunctions on page 48
hMLDA	[()]	- (r) MLDA model for holes. Modified Local-Density Approximation on page 293
(	LambdaTemp)	+ (r) Use temperature-dependent λ for both electrons and holes.

Table 182 Physics{} Part II on page 189

hMobility(	Table 224)	(r) Hole mobility model. Chapter 15 on page 301
hMultiValley		off (r) Multivalley statistics. Multivalley Band Structure on page 267 , Multivalley Band Structure on page 699
hMultiValley(	MLDA	off (r) Multivalley MLDA quantization model. Modified Local-Density Approximation on page 293 , Using Multivalley Band Structure on page 699
	Nonparabolicity	off (r) Band nonparabolicity. Nonparabolic Band Structure on page 268
	DensityIntegral)	off (r) Numeric density computation. Using Multivalley Band Structure on page 269
hQCvanDort		off (r) van Dort model for holes. van Dort Quantization Model on page 280
hQuantumPotential	[(Table 238)]	- (r) Activate hole density gradient quantum correction. Density Gradient Quantization Model on page 288
hQuasiFermi	=<float>	V (r) Initial quasi-Fermi potential specification for holes. Regionwise Specification of Initial Quasi-Fermi Potentials on page 196
hRecVelocity	=<[0,)>	1.93e6 cm s^{-1} (i) Hole recombination velocity. Electric Boundary Conditions for Metals on page 240
hSHEDistribution		off (r) Specify the region where the hole SHE distribution is calculated. Using Spherical Harmonics Expansion Method on page 638
	(Table 239)	
hThermionic		(i) Thermionic emission model for holes. Conductive Insulators on page 244 , Thermionic Emission Current on page 651
	(HCI)	(i) Inject hot holes locally. Destination of Injected Current on page 626
	(Organic_Gaussian)	(i) Thermionic-like Gaussian emission model at organic heterointerfaces for holes. Gaussian Transport Across Organic Heterointerfaces on page 653

Table 182 Physics{} Part II on page 189

Hydrodynamic	[()]	(g) Hydrodynamic model for electrons and holes. Hydrodynamic Model for Current Densities on page 200 , Hydrodynamic Model for Temperatures on page 211
	(eTemperature)	(g) Hydrodynamic model for electrons only. Hydrodynamic Model for Current Densities on page 200 , Hydrodynamic Model for Temperatures on page 211
	(hTemperature)	(g) Hydrodynamic model for holes only. Hydrodynamic Model for Current Densities on page 200 , Hydrodynamic Model for Temperatures on page 211
HydrogenDiffusion		Off (ri) Hydrogen transport model without reaction specification. Using MSC–Hydrogen Transport Degradation Model on page 456
	(Table 243 ...)	Off (ri) Hydrogen transport model with reaction specification. Using MSC–Hydrogen Transport Degradation Model on page 456
IncompleteIonization(	Table 244)	Incomplete ionization. Chapter 13 on page 273
Laser(	Table 212)	Laser. Lasers on page 777
LatticeTemperatureLimit	=<float>	K Maximum lattice temperature. Break Criteria: Conditionally Stopping the Simulation on page 102
LED(	Table 218)	LED. Lasers on page 777
Active (	Type = QuantumWell)	Activate the localized QW model to use in conjunction with QWLocal.
QWLocal (	Table 197)	Localized QW model parameters. Localized QW Model on page 820
MagneticField	=<vector>	T (g) Magnetic field. Chapter 31 on page 743
MetalResistivity(	<ident> [(Table 278)]	PMI for metal resistivity. Metal Resistivity on page 1149
MetalWorkfunction(	Table 245)	(r) Metal workfunction. Metal Workfunction on page 242
Mobility(	Table 224)	(r) Mobility model. Chapter 15 on page 301
MoleFraction(	Table 247)	Mole fractions. Mole-Fraction Specification on page 55
MSConfigs(	Table 248 ...)	(r) Multistate configurations. Specifying Multistate Configurations on page 427

Table 182 Physics{} Part II on page 189

MSPeltierHeat		(i) Peltier heat at metal–semiconductor interfaces or semiconductor contacts. Heating at Contacts, Metal–Semiconductor and Conductive Insulator–Semiconductor Interfaces on page 752
MultiValley		off (r) Multivalley statistics. Multivalley Band Structure on page 267 , Multivalley Band Structure on page 699
MultiValley()	MLDA	off (r) Multivalley MLDA quantization model. Modified Local-Density Approximation on page 293 , Using Multivalley Band Structure on page 699 , Inversion Layer on page 704
	Nonparabolicity	off (r) Band nonparabolicity. Nonparabolic Band Structure on page 268
	DensityIntegral	off (r) Numeric density computation. Using Multivalley Band Structure on page 269
NBTI()	Table 249	(i) NBTI degradation model. Two-Stage NBTI Degradation Model on page 457
Noise()	Table 250	(r) Noise sources. Noise Sources on page 585
Optics()	Table 252	Specifying the Type of Optical Generation Computation on page 470
Piezo()	Table 254	(g) Stress and strain models. Using Stress and Strain on page 689
Piezoelectric_Polarization	(<ident> [Table 278])	Use PMI model <ident> to compute piezoelectric polarization. Piezoelectric Polarization on page 1114
	(strain)	Use strain model to compute piezoelectric polarization. Strain Model on page 735
	(stress)	Use stress model to compute piezoelectric polarization. Stress Model on page 736
Polarization		off (r) Use ferroelectric model. Chapter 29 on page 681
	(Memory=<2..>)	10 Maximum nesting of minor loops. Using Ferroelectrics on page 681
PostTemperature		(g) Use simplified self-heating model. Uniform Self-Heating on page 206
		* Compute dissipated power as integral of Joule heat over entire device.
	(IV_diss)	(g) Compute dissipated power as $\sum IV$ over all electrodes.
	(IV_diss(<string>...))	(g) Compute dissipated power as $\sum IV$ over user-selected electrodes.

Table 182 Physics{} Part II on page 189

Radiation(	Table 230)	Radiation model. Chapter 22 on page 571
RandomizedVariation	<string> (Table 256)	(g) Set sIFM models. Statistical Impedance Field Method on page 592
RayTraceBC(	Table 199)	Physics material or region interface–based definition of boundary conditions. Boundary Condition for Raytracing on page 520
RecGenHeat		(g) Generation–recombination processes act as heat sources. Hydrodynamic Model for Temperatures on page 211
(	OptGenOffset= <float>	0.5 Divide contribution of optical generation rate into energy gain/loss terms H_n and H_p . Hydrodynamic Model for Temperatures on page 211
	OptGenWavelength= <float>)	μm Wavelength if optical generation is loaded from file. Hydrodynamic Model for Temperatures on page 211
Recombination(	Table 183)	Generation–recombination model. Chapter 16 on page 359
Schottky		off (i) Use Schottky boundary conditions. Electric Boundary Conditions for Metals on page 240
Schroedinger	(Table 257)	(i) Schrödinger solver. 1D Schrödinger Solver on page 281
	<string> (Table 257)	(g) Schrödinger solver on named nonlocal mesh.
SingletExciton (		off (ri) Activate region or interface for singlet exciton equation.
	FluxBC	off (i) Impose a zero flux boundary condition on specified interface. Using the Singlet Exciton Equation on page 237
	BarrierType(Table 232)	(i) Set the barrier type at organic heterointerface. Using the Singlet Exciton Equation on page 237
	Recombination(Table 201))	off (r) Switch on generation and recombination terms in singlet exciton equation. Using the Singlet Exciton Equation on page 237
Temperature	=<float>	300 K (g) Device (lattice) temperature.
ThermalConductivity(	Table 259)	(r) Thermal conductivity.

Table 182 Physics{} Part II on page 189

Thermionic		(i) Thermionic emission model for electrons and holes. Conductive Insulators on page 244 , Thermionic Emission Current on page 651
	(HCI)	(i) Inject hot carriers locally. Destination of Injected Current on page 626
	(Organic_Gaussian)	(i) Thermionic-like Gaussian emission model at organic heterointerfaces. Gaussian Transport Across Organic Heterointerfaces on page 653
Thermodynamic		off (g) Thermodynamic transport model. Thermodynamic Model for Current Densities on page 199 , Thermodynamic Model for Lattice Temperature on page 209
Traps((	Table 261)...)	Traps. Chapter 17 on page 407
UseNitrogenAsDopant		off (g) Recognize nitrogen as a dopant. Doping Specification on page 49

Generation and Recombination

Table 183 Recombination() Chapter 16 on page 359

<ident>	[(Table 278)]	Use PMI model <ident>. Generation-Recombination Model on page 1012
Auger		off (r) Auger recombination. Auger Recombination on page 376
	(WithGeneration)	off (r) Auger recombination and generation.
Avalanche(	Table 184)	off (r) Impact ionization. Avalanche Generation on page 377
Band2Band(	Table 185)	off (r) Band-to-Band Tunneling Models on page 391
CDL(	Table 205)	off (r) Coupled defect level recombination. Coupled Defect Level (CDL) Recombination on page 373
eAvalanche(	Table 184)	off (r) Electron impact ionization. Avalanche Generation on page 377
hAvalanche(	Table 184)	off (r) Hole impact ionization. Avalanche Generation on page 377
Radiative		- (r) Radiative recombination. Radiative Recombination on page 375

Table 183 Recombination() [Chapter 16 on page 359](#)

SRH(	Table 205)	off (r) Shockley–Read–Hall recombination. Shockley–Read–Hall Recombination on page 359
SurfaceSRH		off (i) Interface Shockley–Read–Hall recombination. Surface SRH Recombination on page 372
TrapAssistedAuger		off (r) Trap-assisted Auger recombination. Trap-assisted Auger Recombination on page 371

Table 184 Avalanche() [Avalanche Generation on page 377](#)

<ident> [(Table 278)]	Use PMI model <ident>. Avalanche Generation Model on page 1015
CarrierTempDrive	Use temperature driving force (default for hydrodynamic simulation). Avalanche Generation with Hydrodynamic Transport on page 387
ElectricField	Use plain electric field driving force. Approximate Breakdown Analysis: Poisson Equation Approach on page 389
Eparallel	Use current-parallel electric field driving force. Driving Force on page 387
GradQuasiFermi	* Use gradient quasi-Fermi potential driving force. For hydrodynamic simulation, default is CarrierTempDrive . Driving Force on page 387
Hatakeyama	Use Hatakeyama model. Hatakeyama Avalanche Model on page 384
Lackner	Use Lackner model. Lackner Model on page 380
Okuto	Use Okuto–Crowell model. Okuto–Crowell Model on page 379
UniBo	Use University of Bologna model. University of Bologna Impact Ionization Model on page 380
UniBo2	Use new University of Bologna impact ionization model. New University of Bologna Impact Ionization Model on page 382
vanOverstraeten	* Use van Overstraeten model. van Overstraeten – de Man Model on page 378

Table 185 Band2Band() [Band-to-Band Tunneling Models on page 391](#)

DensityCorrection	=Local	Use density correction. Schenk Density Correction on page 393
	=None	* Use plain densities. Schenk Density Correction on page 393
FranzDispersion		- Use Franz dispersion in the direct nonlocal path model. Using Nonlocal Path Band-to-Band Model on page 399
InterfaceReflection		+ Consider interface reflection in the nonlocal path model. Using Band-to-Band Tunneling on page 391
Model	=E1	Use simple model with $P = 1$. Simple Band-to-Band Models on page 393
	=E1_5	Use simple model with $P = 1.5$. Simple Band-to-Band Models on page 393
	=E2	Use simple model with $P = 2$. Simple Band-to-Band Models on page 393
	=Hurkx	Use Hurkx model. Hurkx Band-to-Band Model on page 394
	=NonlocalPath1	Use nonlocal path model with first tunnel path. Dynamic Nonlocal Path Band-to-Band Model on page 395
	=NonlocalPath2	Use nonlocal path model with first and second tunnel paths. Dynamic Nonlocal Path Band-to-Band Model on page 395
	=NonlocalPath3	Use nonlocal path model with first, second, and third tunnel paths. Dynamic Nonlocal Path Band-to-Band Model on page 395
	=Schenk	Use Schenk model. Schenk Model on page 392

Table 186 BPM() Beam Propagation Method on page 559

Bidirectional(	Error=<float>	! Relative error used as a break criterion for iterative algorithm in bidirectional beam propagation method.
	Iterations=<int>)	! Maximum number of iterations used as a break criterion for iterative algorithm in bidirectional beam propagation method.
Boundary(	GridNodes=<float>*2	! Number of PML boundary grid nodes to be inserted at left and right sides.
	Order=<1..2>	! Order of spatial variation of complex stretching parameter.
	Side=<string>	! "X", "Y", or "Z".
	StretchingParameterImag=(<float>*2)	! Minimum and maximum values of imaginary part of stretching parameter.
	StretchingParameterReal=(<float>*2)	! Minimum and maximum values of real part of stretching parameter.
	Type="PML"	! Use PML boundary conditions.
	VacuumGridNodes=(<float>*2))	! Number of vacuum grid nodes to be inserted at left and right sides.
Excitation(	CenterGauss=<vector>	μm Gaussian center. Size of vector is $d - 1$.
	SigmaGauss=<vector>	μm Gaussian half width. Size of vector is $d - 1$.
	TruncationPositionX=(<float>*2)	Left and right truncation positions of plane wave for x-axis.
	TruncationPositionY=(<float>*2)	Left and right truncation positions of plane wave for y-axis.
	TruncationSlope=<vector>	Plane-wave truncation slope. Size of vector is $d - 1$.
	Type="Gaussian"	Use Gaussian excitation.
	Type="PlaneWave"	Use truncated plane wave excitation.
GridNodes	=<vector>	! Number of grid nodes in each spatial dimension.

Table 186 BPM() [Beam Propagation Method on page 559](#)

ReferenceRefractiveIndex	=<float>	! Value for reference refractive index. General on page 561
	=average	Use average refractive index in propagation plane as reference refractive index.
	=fieldweighted	Use field-weighted refractive index in propagation plane as reference refractive index.
	=maximum	Use maximum refractive index in propagation plane as reference refractive index.
ReferenceRefractiveIndexDelta	=<float>	ReferenceRefractiveIndexDelta is added to ReferenceRefractiveIndex in the calculation. To be used for fine-tuning the numerics. General on page 561

Table 187 eBarrierTunneling() and hBarrierTunneling() [Nonlocal Tunneling at Interfaces, Contacts, and Junctions on page 612](#)

Band2Band	=None	* No band-to-band tunneling.
	=Full	Include band-to-band tunneling with consistent parallel momentum integral with the direct nonlocal path band-to-band tunneling model.
	=Simple	Include band-to-band tunneling with simple parallel momentum integral.
	=UpsideDown	Include band-to-band tunneling with unphysical parallel momentum integral.
BandGap		- Allow tunneling into the band gap at the interface for which tunneling is specified. Use this option for backward compatibility only.
PeltierHeat		- Include Peltier heating terms for tunneling carriers. Eq. 656, p. 623
Schroedinger		- Schrödinger equation-based model instead of WKB for tunneling probabilities. This option does not work with the TwoBand or Band2Band option. Schrödinger Equation-based Tunneling Probability on page 620
Transmission		- Use additional interface transmission coefficients. Eq. 647, p. 619
TwoBand		- Use two-band dispersion relation. Eq. 646, p. 618

Table 188 ElectricField() in SRH() and CDL() [SRH Field Enhancement on page 363](#)

DensityCorrection	=Local	(r) Use density correction. Schenk TAT Density Correction on page 366
	=None	* (r) Use local densities.
Lifetime	=Constant	* (r) No field-enhanced lifetime.
	=Hurkx	(r) Use Hurkx model for lifetime enhancement. Hurkx TAT Model on page 366
	=Schenk	(r) Use Schenk model for lifetime enhancement. Schenk Trap-assisted Tunneling (TAT) Model on page 364

Table 189 Farfield(...) in Physics{Optics(OpticalSolver(RayTracing(...)))} [Far Field and Sensors for Raytracing on page 533](#)

Origin	=Auto	Automatically compute origin as the center of the device. Auto is the default.
	=<vector>	User-specified origin as a vector in micrometers.
Discretization	=<integer>	Default is 360 for 2D, and 36 for 3D.
ObservationRadius	=<float>	Radius in micrometers. Default is 1e6 μm , that is, 1 m.
Sensor(	Table 203)	Specify a sensor detector.
SensorSweep(	Table 204)	Specify sensor sweep.

Table 190 FDTD() [Specifying the Optical Solver on page 480](#)

AccelewareOption		off Activate EMW interface to the Acceleware hardware-accelerated FDTD kernel.
EMWPlusOption		off Activate EMW interface to the FDTD kernel with 3D oblique periodic boundary conditions.
GenerateMesh(		Control of tensor mesh generation using Sentaurus Mesh.
	Once)	Generate tensor mesh once at the beginning of the simulation.
	ForEachWavelength)	Generate tensor mesh whenever the wavelength changes compared with the previous FDTD solution.
	Wavelength=(<float>*<0..m>))	Specify list of strictly monotonically increasing wavelengths between which the tensor mesh is generated only once.

Table 191 FromFile() [Loading Solution of Optical Problem from File on page 553](#)

DatasetName	=AbsorbedPhotonDensity OpticalGeneration	AbsorbedPhotonDensity Name of dataset to be loaded from file.
GridInterpolation	=Linear Conservative	Interpolation algorithm to be used if source and destination grid are different.
IdentifyingParameter	=(<string>*n)	! Name of identifying parameters or parameter paths.
ProfileIndex	=<int>	0 ID of loaded profiles after sorting with respect to leading IdentifyingParameter .
SpectralInterpolation	=Off PiecewiseConstant Linear	Off Type of interpolation between loaded profiles.

Table 192 Layers()... [Transfer Matrix Method on page 538](#) (deprecated transfer matrix method)

Absorption	=<float>	Constant value of absorption coefficient (overwrites the value from the TableODB section in the parameter file).
BottomReflectivity	=<[0,1]>	Bottom reflectivity of the current layer (overwrites the value from the TableODB section in the parameter file).
Left	=<vector>	'Upper-left' corner of the current layer illuminated by light wave. Needed for 1D simulation.
Middle	=<vector>	'Upper-middle' corner of current layer illuminated by light wave. Needed for 3D simulation along with Left and Right vectors.
Refract	=<float>	Constant value of refractive index (overwrites the value from the TableODB section in the parameter file).
Right	=<vector>	'Upper-right' corner of the current layer illuminated by light wave. Needed for 2D simulation along with Left vector.
SiLayer		Calculate and store generation rates for this layer. Must be used in one layer only.
Thickness	=<float>	μm Thickness of current layer.
TopReflectivity	=<[0,1]>	Top reflectivity of the current layer (overwrites the value from the TableODB section in the parameter file).

Table 193 Medium() [Transfer Matrix Method on page 538](#)

ExtinctionCoefficient	=<float>	1
Location	=top	! Position of medium with respect to extracted layer structure.
	=bottom	Position of medium with respect to extracted layer structure.
Material	=<string>	Name of material.
RefractiveIndex	=<float>	1

Table 194 NonlocalPath() in SRH() [Dynamic Nonlocal Path Trap-assisted Tunneling on page 367](#)

Fermi		- Use Fermi statistics.
Lifetime	=Hurkx	* Use Hurkx model for lifetime enhancement.
	=Schenk	Use Schenk model for lifetime enhancement.
TwoBand		- Use Two-band dispersion for the transmission coefficient.

Table 195 OptBeam()... [Using Optical Beam Absorption Method on page 558](#)

LayerStackExtraction(	ComplexRefractiveIndexThreshold=<float>)	0 Choose threshold value for layer creation when using ElementWise extraction mode.
	Mode=RegionWise ElementWise	RegionWise Specify whether layer stack is created on a per-element or per-region basis.
	Position=(<float>*3)	Specify starting point of extraction line.
	WindowName=<string>	Reference to illumination window in Excitation section.
	WindowPosition=<ident>	Center Specify starting point of extraction line in terms of a cardinal direction (North , South , East , West , NorthEast , SouthEast , NorthWest , SouthWest).

Table 196 OpticalGeneration{} [Specifying the Type of Optical Generation Computation on page 470](#)

AutomaticUpdate		+ Controls whether optical generation is recomputed if quantities on which it depends are not up-to-date.
ComputeFromMonochromaticSource(		Optical Generation from Monochromatic Source on page 472
	TimeDependence(Table 207)	
	Scaling=<float>)	1
ComputeFromSpectrum(		Illumination Spectrum on page 472
	TimeDependence(Table 207)	
	Select(Parameter = (<string>*n))	Active parameters of multidimensional spectrum file.
	Scaling=<float>)	1
	RefreshEveryTime	Force recomputation of respective optical generation contribution at every occasion.
QuantumYield	=<float>	1 Quantum Yield Models on page 475
QuantumYield(	StepFunction(Table 206)	Quantum Yield Models on page 475
ReadFromFile(		Loading and Saving Optical Generation from and to File on page 474
	DatasetName=Absorbed PhotonDensity OpticalGeneration	
	TimeDependence(Table 207)	
	Scaling=<float>)	1
	RefreshEveryTime	Force recomputation of respective optical generation contribution at every occasion.
	GridInterpolation= Linear Conservative	Interpolation algorithm to be used if source and destination grid are different.
Scaling	=<float>	1

Table 196 OpticalGeneration{} Specifying the Type of Optical Generation Computation on page 470

SetConstant(		Constant Optical Generation on page 475
	TimeDependence(Table 207)	
	Value=<float>)	$\text{s}^{-1} \text{cm}^{-3}$ Value for optical generation rate.
	RefreshEveryTime	Force recomputation of respective optical generation contribution at every occasion.
TimeDependence(	Table 207)	Specification of type of time dependency.

Table 197 QWLocal() Localized QW Model on page 820

-ElectricFieldDep		Switch off electric field dependency for the localized QW model.
NumberOfElectronSubbands	=<integer>	Set the maximum number of electron subbands.
NumberOfHeavyHoleSubbands	=<integer>	Set the maximum number of heavy-hole subbands.
NumberOfLightHoleSubbands	=<integer>	Set the maximum number of light-hole subbands.
WidthExtraction (	Table 210)	Set QW width extraction parameters.

Table 198 RayDistribution() Distribution Window of Rays on page 518

Dx	=<float>	Specify discretized x-size for Mode=Equidistant.
Dy	=<float>	Specify discretized y-size for Mode=Equidistant.
Mode	=AutoPopulate	Automatically populate rays within the excitation shape.
	=Equidistant	Equidistant distribution of rays within the excitation shape.
	=MonteCarlo	Monte Carlo distribution of rays within the excitation shape.
NumberOfRays	=<integer>	Set number of rays for Mode=MonteCarlo or Mode=AutoPopulate.

Table 199 RayTraceBC() in Physics material or region interface–based definition of boundary condition

Fresnel		Fresnel boundary condition.
PMIModel	=<ident> [(Table 278)]	Name of the PMI model associated with this BC contact.
Reflectivity	=<[0,1]>	0 Reflectivity.
TMM(	Table 209)	Specification of TMM multilayer structure.
Transmittivity	=<[0,1]>	0 Transmittivity.

Table 200 RayTracing()... in Physics{Optics(OpticalSolver(...))}, unified raytracing interface
[Raytracing on page 483](#)

CompactMemoryOption		Activate the compact memory model of raytracing.
DepthLimit	=<int>	Stop tracing a ray after passing through more than <int> material boundaries.
Farfield(	Table 189)	Activate the far field and sensor feature. Far Field and Sensors for Raytracing on page 533
MinIntensity	=<float>	Stop tracing a ray when its intensity becomes less than <float> times the original intensity. RelativeMinIntensity is an equivalent keyword.
MonteCarlo		Activate Monte Carlo raytracing.
NonSemiconductorAbsorption		Include optical generation calculation in non-semiconductor region or material.
OmitReflectedRays		Discard all reflected rays from raytracing process.
OmitWeakerRays		Discard the weaker ray at a material interface. This is chosen by comparing the reflectivity and transmittivity.
PlotInterfaceFlux		Activate plotting of interface fluxes on all BCs. Plotting Interface Flux on page 531
PolarizationVector	= Random	Default. Automatically choose a random polarization vector that is perpendicular to the starting ray direction.
	= <vector>	Set a fixed polarization vector for the starting ray.
Print		Create a .grd file with information about the raytree.

Table 200 RayTracing()... in Physics{Optics(OpticalSolver(...))}, unified raytracing interface
[Raytracing on page 483](#)

RayDistribution("string")	(Table 198)	Create a ray distribution to be used in conjunction with the excitation illumination window.
RectangularWindow{	Table 202 }	Create a rectangular window of starting rays.
RedistributeStoppedRays		Distribute the total power accumulated at terminated rays back into the raytree.
RetraceCRChange	=<float>	Fractional change of the complex refractive index to force retracing of rays.
UserWindow(	NumberOfRays= <int> RaysFromFile= <string>)	Input a set of starting rays from a file that is specified by the user.
VirtualRegions{	<string> <string> ...}	Specify virtual regions whereby the raytracer will ignore their existence.

Table 201 Recombination() in SingletExciton()

Bimolecular		off (r) Switch on bimolecular recombination in continuity and singlet exciton equations. Singlet Exciton Equation on page 235 , Bimolecular Recombination on page 401
Dissociation		off (i) Switch on interface exciton dissociation in continuity and singlet exciton equations. Singlet Exciton Equation on page 235 , Exciton Dissociation Model on page 402
eQuenching		off (r) Switch on quenching of singlet exciton due to free electrons. Singlet Exciton Equation on page 235
hQuenching		off (r) Switch on quenching of singlet exciton due to free holes. Singlet Exciton Equation on page 235
radiative		off (r) Switch on directly radiative decay of singlet exciton associated with light emission.
trappedradiative		off (r) Switch on trap-assisted radiative decay of singlet exciton associated with light emission.

Table 202 RectangularWindow(...) [Window of Starting Rays on page 517](#)

LengthDiscretization	=<int>	Number of discretized cells on longer side.
RayDirection	=<vector>	Specify the direction vector of the starting rays. If this keyword is absent, the ray direction is computed from Theta and Phi of the Excitation section of the unified interface for optical generation computation.
RectangleV1	=<vector>	Coordinates of rectangle vertex 1.
RectangleV2	=<vector>	Coordinates of rectangle vertex 2.
RectangleV3	=<vector>	Coordinates of rectangle vertex 3. Applies only to 3D.
WidthDiscretization	=<int>	Number of discretized cells on shorter side. Applies only to 3D.

Table 203 Sensor(...) in unified raytracing far field [Far Field and Sensors for Raytracing on page 533](#)

Angular (	Theta = (<float> <float>)	Define the range of θ for the angular sensor.
	Phi = (<float> <float>))	Define the range of ϕ for the angular sensor.
Line (	Corner1 = <vector>	Define first point of line sensor.
	Corner2 = <vector>	Define second point of line sensor.
	UseNormalFlux)	Compute the projected-to-normal flux.
Name	= "string"	Name of the sensor.
Rectangle (	Corner1 = <vector>	Define first corner of rectangle sensor.
	Corner2 = <vector>	Define second corner of rectangle sensor.
	Corner3 = <vector>	Define third corner of rectangle sensor.
	UseNormalFlux	Compute the projected-to-normal flux.
	AxisAligned)	Take Corner1 and Corner2 as opposing corners of the rectangle sensor.

Table 204 SensorSweep in unified raytracing far field [Far Field and Sensors for Raytracing on page 533](#)

Name	=<string>	Specify name of sensor sweep.
Ndivisions	=<integer>	Set number of subdivisions for the collection ring.
Phi	=(<float> <float>)	Specify range of ϕ for the sensor ring.
Theta	=(<float> <float>)	Specify range of θ for the sensor ring.
VaryPhi		Set the sensor sweep as a latitude ring.
VaryTheta		Set the sensor sweep as a longitudinal ring.

Table 205 SRH() and CDL() [Shockley–Read–Hall Recombination on page 359](#), [Coupled Defect Level \(CDL\) Recombination on page 373](#)

<ident>	[(Table 278)]	Use PMI model <ident> to compute lifetimes. Lifetimes on page 1079
ElectricField(	Table 188)	(r) Field enhancement. SRH Field Enhancement on page 363
DopingDependence		Doping dependence. SRH Doping Dependence on page 361
ExpTempDependence		Exponential temperature dependence. SRH Temperature Dependence on page 362
NonlocalPath(	Table 194)	(r) Nonlocal trap-assisted tunneling enhancement. Dynamic Nonlocal Path Trap-assisted Tunneling on page 367
TempDependence		Temperature dependence. SRH Temperature Dependence on page 362

Table 206 StepFunction() [Quantum Yield Models on page 475](#)

Bandgap		
EffectiveBandgap		
Energy	=<float>	eV
Unity		
Wavelength	=<float>	μm

Table 207 TimeDependence() [Specifying Time Dependency for Transient Simulations on page 478](#)

FromFile		off Reads the time dependency as a table from file.
Scaling	=<float>	1 Scaling factor for optical generation.
WaveTime	=(<float>*2)	s Time interval ($t_{\min}$, $t_{\max}$) when the optical generation rate is constant.
WaveTSigma	=<float>	s Standard deviation σ_t of the temporal Gaussian distribution that describes the decay of the optical generation rate outside the time interval WaveTime .
WaveTSlope	=<float>	s^{-1} Slope that characterizes the linear decay of the optical generation rate outside the time interval WaveTime .

Table 208 TMM() [Transfer Matrix Method on page 538](#)

IntensityPattern		(r) Choose type of intensity pattern.
	=StandingWave	* Compute the regular optical intensity without applying any algorithm that filters out oscillations on the wavelength scale.
	=Envelope	Compute the envelope of the optical intensity instead of the regular optical intensity.
LayerStackExtraction(	ComplexRefractiveIndexThreshold=<float>	0 Choose threshold value for layer creation when using ElementWise extraction mode.
	Medium(Table 193)	Specification of media surrounding the extracted layer structure.
	Mode=RegionWise ElementWise	RegionWise Specify whether layer stack is created on a per-element or per-region basis.
	Position=(<float>*3)	Specify starting point of extraction line.
	WindowName=<string>	Reference to illumination window in Excitation section.
	WindowPosition=<ident>)	Center Specify starting point of extraction line in terms of a cardinal direction (North , South , East , West , NorthEast , SouthEast , NorthWest , SouthWest).
NodesPerWavelength	=<float>	Number of nodes per wavelength used for computation of optical field.

Table 208 TMM() [Transfer Matrix Method on page 538](#)

PropagationDirection	=Perpendicular Refractive	Refractive Choose interpolation mode for 1D TMM solution.
RoughInterface		+ Flag interfaces as rough, that is, physics of rough interface scattering is applied.
Scattering()	AngularDiscretization=<int>	91 Angular discretization of interval $[-\pi/2, \pi/2]$ used in scattering solver.
	MaxNumberOfIterations=<int>	10 Break criterion for iterative scattering solver.
	Tolerance=<float>	1e-3 Break criterion for iterative scattering solver.

Table 209 TMM() in Physics(...){ RayTraceBC(...) }

LayerStructure{ ... }	<float> <string>; <float> <string>; ... <float> <string>}	Definition of multilayer structure used for TMM calculation. First column contains thickness of layer in μm . Second column contains material name of layer.
MapOptGenToRegions(...)	<string> <string> ...	Set list of regions to which the lumped optical generation of the TMM BC is mapped.
QuantumEfficiency	= <float>	Specify the quantum efficiency of the TMM BC optical generation.
ReferenceMaterial	=<string>	Definition of LayerStructure orientation. The topmost layer in the LayerStructure specification is connected to the region with material ReferenceMaterial .
ReferenceRegion	=<string>	Definition of LayerStructure orientation. The topmost layer in the LayerStructure specification is connected to the region with name ReferenceRegion .

Table 210 WidthExtraction() [Localized QW Model on page 820](#)

ChordWeight	=<float>	Specify chord weight for width extraction. Thickness Extraction on page 340
MinAngle	=(<float>, <float>)	Specify minimum angles for width extraction. Thickness Extraction on page 340
SideMaterial	=("mat1", ..., "matn")	Specify the materials adjoining the QW.
SideRegion	=("reg1", ..., "regn")	Specify the regions adjoining the QW.

Laser and LED

NOTE LED simulations present unique challenges that require problem-specific model and numerics setups. Contact TCAD Support for advice if you are interested in simulating LEDs (see [Contacting Your Local TCAD Support Team Directly on page xl](#)).

Table 211 Laser() or LED() [Chapter 35 on page 811](#)

Broadening(	Table 213)	Activate gain-broadening models or nonlinear gain saturation model. Gain-broadening Models on page 902 , Nonlinear Gain Saturation Effects on page 903
FreeCarr		Use free carrier absorption. Free Carrier Loss on page 792
GroupRefract	=<float>	Fixed effective index; ignore effective index computed by the optical solvers. Edge-Emitting Lasers on page 793
LateralLeakage		Activate nonperpendicular current flux of free carriers across quantum wells.
Optics(	Table 218)	Optics. Chapter 36 on page 853
QWExtension	=AutoDetect	Autodetect width of quantum wells. The quantum-well region must be specified by the keyword Active . Carrier Capture in Quantum Wells on page 896
QWTransport		Use ‘three-point’ QW model with thermionic emission. Carrier Capture in Quantum Wells on page 896
ReducedOutcoupling(	FreeCarrierAbsorption=<float>	Parameter for phenomenological reduction of VCSEL output power due to free carrier absorption. VCSEL Output Power on page 804
	OpticalLoss=<float>)	Parameter for phenomenological reduction of VCSEL output power due to distributed background losses. VCSEL Output Power on page 804
RefractiveIndex(	CarrierDep	Use carrier density–dependent refractive index. Carrier-Density Dependence of Refractive Index on page 790
	TempDep)	Use temperature-dependent refractive index. Temperature Dependence of Refractive Index on page 789
SplitOff	=<float>	eV Spin-orbit split-off energy. Strain Effects on page 908

Table 211 Laser() or LED() [Chapter 35 on page 811](#)

SponEmissionCoeff	(<ident>[(Table 278)])	Use PMI model <ident> for spontaneous emission. Importing Gain and Spontaneous Emission Data with PMI on page 914
	(AnalyticSponEmissionModel)	Activate use of simple model for spontaneous emission. Stimulated and Spontaneous Emission Coefficients on page 898
	(Tabulated)	Activate loading of tabulated spontaneous emission data. Loading Tabulated Stimulated and Spontaneous Emission on page 913
SponIntegration	(<float>,<int>)	eV Energy integration range and number of discretization intervals for numeric integration (for LED simulations only). Spontaneous Emission Rate and Power on page 813
SponScaling	=<float>	Scaling factor for matrix element of spontaneous emission. Radiative Recombination and Gain Coefficients on page 898
StimEmissionCoeff	(<ident>[(Table 278)])	Use PMI model <ident> for stimulated emission. Importing Gain and Spontaneous Emission Data with PMI on page 914
	(Tabulated)	Activate loading of tabulated stimulated emission data. Loading Tabulated Stimulated and Spontaneous Emission on page 913
StimScaling	=<float>	Scaling factor for matrix element of stimulated emission. Radiative Recombination and Gain Coefficients on page 898
Strain	(RefLatticeConst=<float>)	m Use strain model for quantum well with given reference lattice constant. Strain parameters are input in the parameter file. Strain Effects on page 908

Table 212 Laser() [Chapter 34 on page 777](#)

Table 218		Keywords for lasers and LEDs.
CavityLength	=<float>	μm Cavity length. Photon Rate and Photon Phase Equations on page 780
DFB(	DFBPeriod=<float>)	μm Use DFB feature with DFB grating period <float>. Edge-Emitting Lasers on page 793
GainShift	=<float>	eV Shift emission energy in a laser or an LED simulation. Fitting Stimulated and Spontaneous Emission Spectra on page 901
lFacetReflectivity	=<float>	Left-facet power reflectivity. Photon Rate and Photon Phase Equations on page 780
LinewidthEnhancementFactor	=<float>	Line width enhancement factor.
LongitudinalModes		Calculate multiple longitudinal optical modes if the number of modes is specified greater than one for the applied optical solver. By default, multiple transverse modes are calculated. Edge-Emitting Lasers on page 793
OpticalLoss	=<float>	cm^{-1} Background optical loss.
PhotonPhaseAlpha		Use line width enhancement model. This requires the specification of LinewidthEnhancementFactor .
rFacetReflectivity	=<float>	Right-facet power reflectivity.
SponEmiss	=<float>	Spontaneous emission factor.
TransverseModes		Calculate multiple transverse optical modes if the number of modes is specified greater than one for the applied optical solver. This is the default. Edge-Emitting Lasers on page 793
VCSEL(	)	VCSEL simulation. Vertical-Cavity Surface-Emitting Lasers on page 801
WaveguideLoss		off Activate the feedback of waveguide loss from optical solver to the photon rate equation.

Table 213 Broadening() [Gain-broadening Models on page 902](#)

Gamma	=<float>	eV Broadening coefficient Γ .
Type	=CosHyper	Use hyperbolic-cosine broadening. Hyperbolic-Cosine Broadening on page 903
	=Landsberg	Use Landsberg broadening. Landsberg Broadening on page 902
	=Lorentzian	Use Lorentzian broadening. Lorentzian Broadening on page 902

Table 214 DeterministicProblem() [Automatic Optical Mode Searching on page 889](#)

AzimuthalExpansion	=<int>	Order of the cylindrical harmonics, $\cos(m\varphi)$ (for cavity only).
Boundary	=Type1	(For waveguide only)
	=Type2	(For waveguide only)
Coordinates	=Cartesian	(For waveguide only)
	=Cylindrical	(For cavity only)
EquationType	=Cavity	Solve a cavity problem for the resonant frequency.
	=Waveguide	Solve a frequency-domain Helmholtz equation for the effective index.
LasingWavelength	=<float>	! nm Lasing wavelength (for waveguide only).
SourcePolarization	=phiDirection	(For cavity only)
	=rhoDirection	(For cavity only)
	=xDirection	(For waveguide only)
	=yDirection	(For waveguide only)
	=zDirection	
SourcePosition	(<float>*2)	Coordinates of the source dipole.
Sweep	(<float>*2 <int>)	For cavity, start and end wavelength, in nm, and number of points; for waveguide, start and end effective index, and number of points.
Symmetry	=NonSymmetric	(For waveguide only)
	=Periodic	(For waveguide only)
	=Symmetric	(For waveguide only)

Table 215 EffectiveIndex() [Effective Index Method for VCSELs on page 869](#)

Absorption	(ODB)	Use absorption from the TableODB section of the parameter file.
Coordinates	=Cartesian	
	=Cylindrical	
CoreWidth	=<float>	! μm Square aperture half-width d (for Cartesian) or the circular aperture radius R (for Cylindrical).
DiffractionLoss	=(<float>...)	0 cm^{-1} Diffraction loss. Alternative to DiffractionRate .
DiffractionRate	=(<float>...)	0 ps^{-1} Diffraction rate. Alternative to DiffractionLoss .
mModeIndex	=(<int>...)	0 Index m of the parameter specified with ModeType (cylindrical harmonic order if Coordinates=Cylindrical).
ModeNumber	=<int>	1 Number of modes to be calculated.
ModeType	=(<mt>...)	Mode types: HEmn , EHmn , TE0n , or TM0n .
nModeIndex	=(<int>...)	0 Index n of the parameter specified with ModeType .
OpticalDecayRate	=(<float>...)	0 ps^{-1} Optical loss.
PowerCouplingFactor	=(<float>...)	Scaling factor for optical output power in VCSELs. VCSEL Output Power on page 804
TargetWavelength	=(<float>...)	! nm Target value for lasing wavelength.

Table 216 FEScalar() [FE Scalar Solver on page 856](#)

Absorption	(ODB)	Use absorption from the TableODB section of the parameter file.
Boundary	=(<type>...)	Boundary condition types, "Type1" or "Type2".
EquationType	=Waveguide	Solve waveguide problem.
LasingWavelength	=<float>	! nm Lasing wavelength.
ModeNumber	=<int>	1 Number of modes to solve.
Polarization	=(<pt>...)	Polarization type, TE (default) or TM.
PowerCouplingFactor	=(<float>...)	Scaling factor for optical output power in VCSELs. VCSEL Output Power on page 804
Symmetry	=NonSymmetric	Boundary Conditions and Symmetry for Optical Solvers on page 861
	=Periodic	Boundary Conditions and Symmetry for Optical Solvers on page 861
	=Symmetric	* Boundary Conditions and Symmetry for Optical Solvers on page 861
TargetEffectiveIndex	=(<float>...)	! Initial guess for effective refractive index.
TargetLoss	=(<float>...)	0 cm ⁻¹ Initial guess for propagation loss.

Table 217 FEVectorial() [FE Vectorial Solver on page 857](#)

Absorption	(ODB)	Use absorption from the TableODB section of the parameter file.
AzimuthalExpansion	=(<int>...)	! Cylindrical harmonic order m , for cylindrical cavity problems only.
Boundary	=(<type>...)	Boundary condition types, "Type1" or "Type2".
Coordinates	=Cartesian	
	=Cylindrical	For cavity problems only.
EquationType	=Cavity	Solve cavity problem.
	=Waveguide	Solve waveguide problem.
LasingWavelength	=<float>	! nm Lasing wavelength, for Waveguide only.
LongitudinalWavevector	=<float>	! m^{-1} Longitudinal wavevector $2\pi/\lambda_z$, for Cartesian cavity problems only.
ModeNumber	=<int>	1 Number of modes to solve.
OpticalDecayRate	=(<float>...)	0 ps^{-1} Additional optical loss; for cavity problems only.
PowerCouplingFactor	=(<float>...)	Scaling factor for optical output power in VCSELs. VCSEL Output Power on page 804
Symmetry	=NonSymmetric	Boundary Conditions and Symmetry for Optical Solvers on page 861
	=Periodic	Boundary Conditions and Symmetry for Optical Solvers on page 861
	=Symmetric	* Boundary Conditions and Symmetry for Optical Solvers on page 861
TargetEffectiveIndex	=(<float>...)	! Initial guess for effective refractive index, for Waveguide only.
TargetLifetime	=(<float>...)	! ps Target value for photon lifetime; for Cavity problems only.
TargetLoss	=(<float>...)	0 cm^{-1} Initial guess for propagation loss; for Waveguide only.
TargetWavelength	=(<float>...)	! nm Target value for lasing wavelength; for Cavity problems only.

Table 218 Optics() in Laser() and LED() [Chapter 36 on page 853](#)

DeterministicProblem(	Table 214)	Automatic Optical Mode Searching on page 889
EffectiveIndex(	Table 215)	Effective index solver. Effective Index Method for VCSELs on page 869
FEScalar(	Table 216)	Scalar finite-element solver. FE Scalar Solver on page 856
FEVectorial(	Table 217)	Vectorial finite-element solver. FE Vectorial Solver on page 857
OptConfin	=(<float>)	Optical confinement factor. Waveguide Optical Modes and Fabry–Perot Cavity on page 793
OpticsFromFile(	Table 219)	Optical mode properties. Loading Optical Modes on page 876
RayTrace(	Table 221)	Raytracing. Far Field on page 878
TMM1D(	Table 223)	Transmission mode method. Transfer Matrix Method for VCSELs on page 867

Table 219 OpticsFromFile() [Loading Optical Modes on page 876](#)

LasingWavelength	=(<float>...)	nm Lasing wavelength, for edge-emitting laser only.
ModeNumber	=(<int>)	1 Number of modes to load.
OutcouplingLoss	=(<float>...)	ps ⁻¹ Decay rate through top mirror, for VCSEL only.
TargetEffectiveIndex	=(<float>...)	1 Real part of the complex propagation constant, for edge-emitting laser only.
TargetLoss	=(<float>...)	1 Imaginary part of the complex propagation constant, for edge-emitting laser. ps ⁻¹ Total decay rate, for VCSEL.
TargetWavelength	=(<float>...)	nm Lasing wavelength, for VCSEL only.

Table 220 PrintORArrays() in LED(Optics(RayTrace())) [Interface to LightTools on page 843](#)

FileName	=<string>	File name used to print the ray and spectrum data.
NumberOfSpectralPoints	=<int>	Number of discretized points to use for printing the spectrum.
RangeOfSpectrum	=(<float>*2)	eV Energy range of the spectrum to be printed.
RefractiveIndexDelta	=<float>	0.2 Expected range of refractive index change due to spectrum band width.
SplitSpectrumRays	=<int>	Number of extra rays into which each far-field ray will be split.
VectorUnits	=Millimeter	Output position vector of ray in millimeters.
	=Micron	Output position vector of ray in micrometers.

Table 221 RayTrace() in LED(Optics()) [LED Raytracing on page 823](#)

ClusterActive(	Table 222)	Activate the clustering option. Clustering Active Vertices on page 828
CompactMemoryOption		Activate the compact memory model for LED raytracing. Will not work with the full photon-recycling model.
Coordinates	=Cartesian	*
	=Cylindrical	
DebugLEDRadiation	(<string> <float1> <float2> <float3>)	off Trace the origin of the output rays that are within the angles [<float1>, <float2>] (in degrees) and of minimum intensity specified by the <float3> parameter. In 2D, the angle is defined in the regular polar coordinates. In 3D, the angles are defined from the z-axis, as in θ in regular spherical coordinates. The results are output to the file specified by <string>.
DepthLimit	=<int>	5 Stop tracing the ray after passing through more than <int> material boundaries.
Disable		off Disable raytracing but still run LED simulation.
EmissionType	(Anisotropic(Sine(<float>*3) Cosine(<float>*3))	
	(Isotropic)	*
ExcludeHorizontalSource	(<float>)	off Omit the source rays that are emitted within the horizontal angular zone specified by the <float> parameter (in degrees).

Table 221 RayTrace() in LED(Optics()) [LED Raytracing on page 823](#)

LEDRadiationPara	(<float>, <int>)	μm Observation radius and discretization of the observation circle or sphere.
LEDSpectrum	(<float>*2 <int>)	eV Starting and ending energy range, and number of subdivisions in that energy range.
MinIntensity	=<float>	1e-5 Stop tracing the ray when the intensity of a ray becomes less than <float> times the original intensity.
MonteCarlo		Activate Monte Carlo raytracing.
MoveBoundaryStartRays	= <float>	0 nm. Shift the starting ray position at device boundary inwards. Recommended values are 1 to 5 nm.
NonActiveAbsorptionOff		Do not add nonactive region absorption as generation rate to continuity equation. Nonactive Region Absorption (Photon Recycling) on page 843
ObservationCenter	=<vector>	Fixed observation center for LED radiation. By default, center of device.
OmitReflectedRays		Discard all reflected rays in raytracing process.
OmitWeakerRays		Discard the weaker ray at a material interface. This is decided by comparing the reflectivity and transmittivity.
OptGenScaling	= <float>	Set a multiplication factor to the nonactive optical generation computed.
PolarizationVector	= Random	Default. Automatically choose a random polarization vector that is perpendicular to the starting ray direction.
	= <vector>	Set a user-defined polarization vector.
Print		off Output all rays to a grid file.
	(ActiveVertex(<int>))	off Output only ray paths emitted from this active vertex.
	(Skip(<int>))	1 Output every other <int> ray to a grid file.
PrintORArrays(	Table 220)	Print ray and spectrum data into the input format of LightTools® for LED packaging design. Interface to LightTools on page 843
PrintRayInfo	(<string>)	off Print all indices and positions of starting rays into the file specified by <string>.
PrintSourceVertices	(<string>)	off Print the index and coordinates of all active vertices into the file specified by <string>.
ProgressMarkers	=<int>	5 Completion meter for raytracing.

Table 221 RayTrace() in LED(Optics()) [LED Raytracing on page 823](#)

RaysPerVertex	=<int>	10 Number of rays starting from each active source vertex. For 3D, the number of starting rays are constrained by 6, 18, 68, and so on. The number in the sequence is chosen such that RaysPerVertex is slightly larger or equal to it.
RaysRandomOffset		Randomize the angular shift of the starting rays.
	(RandomSeed=<int>)	Set a fixed random seed so that repeated simulations will yield the exact pseudorandom results.
RedistributeStoppedRays		Distribute the total accumulated power in terminated rays back into the raytree.
RetraceCRChange	=<float>	Fractional change of the complex refractive index to force retracing of rays.
SourceRaysFromFile (	<string>)	Import a set of starting ray directions from the file name specified by <string>. The number of imported rays are specified by RaysPerVertex .
Staggered3DFarfieldGrid		Use the staggered 3D far-field collection sphere. Staggered 3D Grid LED Radiation Pattern on page 839
TraceSource	()	Retrace the source of the output rays to produce a map of the source regions that give the strongest ray output.
TurnOffTreeNodeCount		Disable counting the total number of nodes in the raytree.
Wavelength	=<float>	nm Wavelength.
	=AutoPeak	Take wavelength at the peak of the spontaneous emission rate spectrum. LED Wavelength on page 817
	=AutoPeakPower	Take wavelength at the peak of the spontaneous emission power spectrum.
	=Effective	Wavelength computed such that total power = total rate x effective photon energy.

Table 222 ClusterActive() in LED(Optics(RayTrace())) [Clustering Active Vertices on page 828](#)

ClusterQuantity	=Nodes	Clustering by recursive grouping of active vertices.
	=OpticalGridElement	Clustering by grouping active vertices in each active optical element.
	=PlaneArea	Clustering by evenly dividing up QW plane area and grouping active vertices in each area element.
NumberOfClusters	=<integer>	Specify number of clusters, only for Nodes or PlaneArea clustering.

Table 223 TMM1D() [Transfer Matrix Method for VCSELS on page 867](#)

Absorption	(0DB)	Use absorption from the Table0DB section of the parameter file.
PowerCouplingFactor	=<float>	Scaling factor for optical output power in VCSELS. VCSEL Output Power on page 804
SigmaGauss	=<float>	μm Width of the Gaussian distribution.
TargetWavelength	=<float>	nm Target value for lasing wavelength. Only one value can be input because TMM1D solves the fundamental mode only.

Mobility

Table 224 Mobility(), eMobility(), hMobility() [Chapter 15 on page 301](#)

CarrierCarrierScattering	(BrooksHerring)	off Use Brooks–Herring carrier–carrier scattering model. Carrier–Carrier Scattering on page 309
	(ConwellWeisskopf)	off Use Conwell–Weisskopf carrier–carrier scattering model. Carrier–Carrier Scattering on page 309
ConstantMobility		+ Use constant mobility if neither PhuMob nor DopingDependence is specified. Mobility due to Phonon Scattering on page 302
Diffusivity(	Table 225)	off Use non-Einstein diffusivity. Non-Einstein Diffusivity on page 350

Table 224 Mobility(), eMobility(), hMobility() Chapter 15 on page 301

DopingDependence(	<ident> [(Table 278)]	off PMI model <ident>. Doping-dependent Mobility on page 1023
	Arora	off Use Arora doping-dependent mobility model. Doping-dependent Mobility Degradation on page 302
	Masetti	off Use Masetti doping-dependent mobility model. Doping-dependent Mobility Degradation on page 302
	PMIModel(Table 255)	off Use PMI for MSC-dependent mobility. Multistate Configuration-dependent Bulk Mobility on page 1029
	UniBo)	off Use University of Bologna doping-dependent mobility model. Doping-dependent Mobility Degradation on page 302
eDiffusivity(	Table 225)	off Use non-Einstein electron diffusivity. Non-Einstein Diffusivity on page 350
eHighFieldSaturation(	Table 225)	off Electron high-field saturation. High-Field Saturation on page 342
Enormal(	<ident> [(Table 278)]	off PMI model <ident>. Mobility Degradation at Interfaces on page 1032
	Coulomb2D	off ‘Two-dimensional’ ionized impurity mobility degradation. Mobility Degradation Components due to Coulomb Scattering on page 332
	IAlMob(Table 226)	off Inversion and accumulation layer mobility model. Mobility Degradation at Interfaces on page 315
	InterfaceCharge[(SurfaceName=<string>)]	off Negative and positive interface charge mobility degradation. Mobility Degradation Components due to Coulomb Scattering on page 332
	Lombardi(Table 227)	off Enhanced Lombardi model. Mobility Degradation at Interfaces on page 315
	Lombardi_highk	off Enhanced Lombardi model with high-k degradation. Mobility Degradation at Interfaces on page 315
	NegInterfaceCharge[(SurfaceName=<string>)]	off Negative interface charge mobility degradation. Mobility Degradation Components due to Coulomb Scattering on page 332
	PosInterfaceCharge[(SurfaceName=<string>)]	off Positive interface charge mobility degradation. Mobility Degradation Components due to Coulomb Scattering on page 332
	UniBo)	off University of Bologna surface mobility model. Mobility Degradation at Interfaces on page 315

Table 224 Mobility(), eMobility(), hMobility() Chapter 15 on page 301

hDiffusivity(	Table 225)	off Use non-Einstein hole diffusivity. Non-Einstein Diffusivity on page 350
hHighFieldSaturation(	Table 225)	off Hole high-field saturation. High-Field Saturation on page 342
HighFieldSaturation(	Table 225)	off High-field saturation. High-Field Saturation on page 342
IncompleteIonization		off Incomplete ionization-dependent mobility. Incomplete Ionization-dependent Mobility Models on page 352
PhuMob		off Philips unified mobility model. Philips Unified Mobility Model on page 310
(	Arsenic	* Use arsenic parameters. Table 46 on page 314
	Klaassen	* Use Klaassen parameters. Table 45 on page 314
	Meyer	Use Meyer parameters. Table 45 on page 314
	Phosphorus)	Use phosphorus parameters. Table 46 on page 314
ThinLayer		off Thin-layer mobility model. Thin-Layer Mobility Model on page 337
(	ChordWeight= <float>	0 Weight of chord length. Thickness Extraction on page 340
	MinAngle= (2*<float>)	0 0 Angle constraints. Thickness Extraction on page 340
	Thickness=<float>)	μm Explicit thickness value.
ToCurrentEnormal	as Enormal	off Dependence on electric field normal to current. Mobility Degradation at Interfaces on page 315
Tunneling		- Tunneling correction to mobility. Eq. 211, p. 289

Table 225 HighFieldSaturation(), Diffusivity() [High-Field Saturation on page 342](#)

<ident>	[(Table 278)]	PMI model <ident>. High-Field Saturation Model on page 1041
CarrierTempDrive		Temperature driving force (default for hydrodynamic simulation). Driving Force Models on page 348
CarrierTempDriveBasic		Basic temperature driving force. Basic Model on page 345
CarrierTempDriveME		Meinerzhagen–Engl temperature driving force. Meinerzhagen–Engl Model on page 345
CarrierTempDrivePolynomial		Energy-dependent mobility model using an irrational polynomial. Energy-dependent Mobility on page 658
CarrierTempDriveSpline		Energy-dependent mobility model using spline interpolation. Spline Interpolation on page 660
CaugheyThomas		* Use Canali/Hänsch model. Extended Canali Model on page 343
Eparallel		Use electric field parallel to current as driving force. Driving Force Models on page 348
EparallelToInterface		Use electric field parallel to the closest semiconductor–insulator interface as driving force. Driving Force Models on page 348
GradQuasiFermi		* Use gradient of quasi-Fermi potential as driving force. For hydrodynamic simulations, default is CarrierTempDrive . Driving Force Models on page 348
ParameterSetName	=<string>	Name of parameter set to use. Named Parameter Sets for High-Field Saturation on page 343
PFMob		Use Poole–Frenkel mobility model. Poole–Frenkel Mobility (Organic Material Mobility) on page 353
PMIModel(	Table 255)	PMI model dependent on two fields. High-Field Saturation with Two Driving Forces on page 1050
TransferredElectronEffect		Use transferred electron model. Transferred Electron Model on page 344

Table 226 IALMob() [Mobility Degradation at Interfaces on page 315](#)

AutoOrientation	=<0..1>	0 If = 1, use parameter set based on orientation of nearest interface. Auto-Orientation for IALMob on page 330
ClusteringEverywhere	=<0..1>	0 If = 1, use clustering formulas for all occurrences of N_A and N_D . Inversion and Accumulation Layer Mobility Model on page 323
FullPhuMob	=<0..1>	0 If = 1, use the full PhuMob mobility expressions. Inversion and Accumulation Layer Mobility Model on page 323
ParameterSetName	=<string>	Name of IALMob parameter set. Named Parameter Sets for IALMob on page 329
PhononCombination	=<0..2>	0 Selects how 2D and 3D phonon mobility are combined. Inversion and Accumulation Layer Mobility Model on page 323

Table 227 Lombardi() [Mobility Degradation at Interfaces on page 315](#)

AutoOrientation		off Use parameter set based on orientation of nearest interface. Orientation-dependent Lombardi Parameters on page 319
ParameterSetName	=<string>	Name of EnormalDependence parameter set. Named Parameter Sets for Lombardi Model on page 319

Radiation Models

Table 228 AlphaParticle() [Alpha Particles on page 572](#)

Direction	=<vector>	! Direction of motion of the particle.
Energy	=<float>	! eV Energy of the alpha particle.
Location	=<vector>	! μm Point where the alpha particle enters the device (bidirectional track).
StartPoint	=<vector>	! μm Point where the alpha particle enters the device (one-directional track).
Time	=<float>	! s Time at which the charge generation peaks.

Table 229 HeavyIon() [Heavy Ions on page 574](#)

Direction	=<vector>	Direction of motion of the ion.
Exponential		* Use exponential shape for the spatial distribution $R(w)$.
Gaussian		Use Gaussian shape for the spatial distribution $R(w)$.
Length	=[<float>, ...]	Length l where Wt_hi and Let_f are specified (in cm by default or μm if the PicoCoulomb option is selected).
LET_f	=[<float>, ...]	Linear energy transfer (LET) function (in pairs cm^{-3} by default or $\text{pC } \mu\text{m}^{-1}$ if the PicoCoulomb option is selected). Entries in the list match those in Length .
Location	=<vector>	μm Point where the heavy ion enters the device (bidirectional track).
PicoCoulomb		Switch units for LET_f , Wt_hi , and Length .
SpatialShape	=<ident> [(Table 278)]	PMI for spatial distribution function. Spatial Distribution Function on page 1146
StartPoint	=<vector>	μm Point where the heavy ion enters the device (one-directional track).
Time	=<float>	s Time at which the ion penetrates the device.
Wt_hi	=[<float>, ...]	Characteristic distance, $w_t(l)$ (in cm by default or μm if the PicoCoulomb option is selected). Entries in the list match those in Length .

Table 230 Radiation() [Chapter 22 on page 571](#)

Dose	=<float>	rad Total dose over exposure time.
DoseRate	=<float>	rads^{-1} Dose rate.
DoseTime	=(<float>*2)	! s Exposure time.
DoseTSigma	=<float>	! s Standard deviation of rise and fall of dose rate over time.

Various

Table 231 Aniso() [Chapter 28 on page 665](#)

eAvalanche		Use anisotropic electron avalanche generation. Anisotropic Avalanche Generation on page 675
eMobility		Use self-consistent anisotropic mobility for electrons. Self-Consistent Anisotropic Mobility on page 673
eMobilityFactor	(Total)=<float>	Total anisotropic mobility factor for electrons. Total Anisotropic Mobility on page 672
	(TotalDD)=<float>	Total direction-dependent mobility factor for electrons. Total Direction-dependent Anisotropic Mobility on page 673
hAvalanche		Use anisotropic hole avalanche generation. Anisotropic Avalanche Generation on page 675
hMobility		Use self-consistent anisotropic mobility for holes. Self-Consistent Anisotropic Mobility on page 673
hMobilityFactor	(Total)=<float>	Total anisotropic mobility factor for holes. Total Anisotropic Mobility on page 672
	(TotalDD)=<float>	Total direction-dependent mobility factor for holes. Total Direction-dependent Anisotropic Mobility on page 673
Poisson		Use anisotropic electrical permittivity. Anisotropic Electrical Permittivity on page 677
Temperature		Use anisotropic thermal conductivity. Anisotropic Thermal Conductivity on page 678

Table 232 BarrierType() in SingletExciton() [Singlet Exciton Equation on page 235](#)

Bandgap		* (i) Use bandgap difference as barrier.
CondBand		(i) Use conduction band-edge difference as barrier.
ValBand		(i) Use valence band-edge difference as barrier.

Table 233 ComplexRefractiveIndex() [Complex Refractive Index Model on page 496](#)

CarrierDep(	real	Use carrier dependency of refractive index. Carrier Dependency on page 497
	imag)	Use carrier dependency of extinction coefficient. Carrier Dependency on page 497
CRIModel(	Name=<string>)	Activate user-defined complex refractive index model. Complex Refractive Index Model Interface on page 504
GainDep	(real(lin))	Use linear gain dependency of refractive index. Gain Dependency on page 499
	(real(log))	Use logarithmic gain dependency of refractive index. Gain Dependency on page 499
TemperatureDep(	real)	Use temperature dependency of refractive index. Temperature Dependency on page 497
Wave lengthDep(	real	Use wavelength dependency of refractive index. Wavelength Dependency on page 497
	imag)	Use wavelength dependency of extinction coefficient. Wavelength Dependency on page 497

Table 234 DeterministicVariation() [Deterministic Variations on page 595](#)

DopingVariation	<string>(Table 235)	Deterministic Doping Variations on page 595
GeometricVariation	<string>(Table 241)	Deterministic Geometric Variations on page 596
ParameterVariation	<string>(Table 253)	Parameter Variations on page 597

Table 235 DopingVariation() [Deterministic Doping Variations on page 595](#)

Amplitude	=<vector>	μm Vectorial amplitude for gradient specification.
Amplitude_Iso	=<float>	μm Isotropic amplitude for gradient specification.
Conc	=<float>	cm^{-3} Normalization concentration.
Factor	=<float>	1 Multiplier for variation.
SFactor	=<string>	Dataset name for variation.
SpaceMid	=<vector>	μm Center of shape function.
SpaceSig	=<vector>	μm Width of shape function.
SpatialShape	=Gaussian	Use Gaussian shape function.
	=Uniform	Use window shape function.
Type	=Acceptor	Apply variation to acceptor concentration.
	=Donor	Apply variation to donor concentration.
	=Doping	* Apply variation according to sign to acceptor or donors.

Table 236 EffectiveIntrinsicDensity() [Band Gap and Electron Affinity on page 249](#)

BandGap	(<ident> [(Table 278)])	Use PMI model <ident> to compute band gap E_g . Band Gap on page 1054
BandGapNarrowing	(<ident> [(Table 278)])	Use PMI model <ident> for bandgap narrowing. Bandgap Narrowing on page 1058
	(BennettWilson)	* Bennett–Wilson model.
	(delAlamo)	del Alamo model.
	(JainRoulston)	Jain–Roulston model.
	(oldSlotboom)	Old Slotboom model.
	(Slotboom)	Slotboom model.
	(TableBGN)	Table-based model.
NoBandGapNarrowing		No bandgap narrowing.
NoFermi		off Omit correction Eq. 159, p. 259 even when using Fermi statistics. Bandgap Narrowing with Fermi Statistics on page 259

Table 237 eLucky(), hLucky(), eFiegna(), hFiegna() [Effective Field on page 630](#)

CarrierTempDrive		Use field derived from temperature.
CarrierTempPost		Use field derived from postprocessed temperature.
Eparallel		* Use field parallel to interface.

Table 238 eQuantumPotential() and hQuantumPotential() [Density Gradient Quantization Model on page 288](#)

AutoOrientation		off Use parameter set based on orientation of nearest interface. Auto-Orientation for Density Gradient on page 291
BoundaryCondition	=Dirichlet	(ci) Enforce homogeneous Dirichlet boundary conditions.
	=Neumann	(ci) Enforce homogeneous Neumann boundary conditions.
Density		- (r) Use Eq. 208, p. 288 instead of Eq. 209, p. 288 .
Ignore		- (r) Compute, but do not apply quantum correction.
LocalModel	=<ident> [(Table 278)]	(r) Name of PMI for apparent band-edge shift. Chapter 14 on page 279, Apparent Band-Edge Shift on page 1061
	=SchenkBGN_elec	(r) Schenk bandgap narrowing model for electrons. Schenk Bandgap Narrowing Model on page 255
	=SchenkBGN_hole	(r) Schenk bandgap narrowing model for holes. Schenk Bandgap Narrowing Model on page 255
ParameterSetName	=<string>	Name of QuantumPotentialParameters parameter set. Named Parameter Sets for Density Gradient on page 291
Resolve		- (r) Use more accurate discretization of non-heterointerfaces.

Table 239 eSHEDistribution(), hSHEDistribution() [Spherical Harmonics Expansion Method on page 634](#)

AdjustImpurityScattering		+ Use an option to adjust impurity scattering to match the low-field mobility specified in the Physics section.
FullBand		- Use the full band structure with the default band-structure file.
	=<string>	! File name of the user-defined band-structure file.
RTA		- Use relaxation time approximation.

Table 240 GateCurrent() [Chapter 24 on page 605](#), [Chapter 25 on page 625](#)

<ident>(	<carrier>... [Table 278])	(i) Use PMI model <ident> to compute hot-carrier injection. Hot-Carrier Injection on page 1125
DirectTunneling		(i) Schenk direct tunneling model. Direct Tunneling on page 608
eFiegna	[(Table 237)]	(i) Fiegna model for electrons. Fiegna Hot-Carrier Injection on page 631
eLucky	[(Table 237)]	(i) Lucky electron model. Classical Lucky Electron Injection on page 630
eSHEDistribution		(i) SHE distribution model for electrons. SHE Distribution Hot-Carrier Injection on page 633
Fowler		(i) Fowler–Nordheim tunneling. Fowler–Nordheim Tunneling on page 606
	(EVB)	(i) Fowler–Nordheim valence band tunneling of electrons. Fowler–Nordheim Tunneling on page 606
GateName	=<string>	(i) Electrode for monitoring gate current. Overview on page 625
hFiegna	[(Table 237)]	(i) Fiegna model for holes. Fiegna Hot-Carrier Injection on page 631
hLucky	[(Table 237)]	(i) Lucky hole model. Classical Lucky Electron Injection on page 630
hSHEDistribution		(i) SHE distribution model for holes. SHE Distribution Hot-Carrier Injection on page 633
InjectionRegion	=<string>	Regions to which hot carriers are injected. Destination of Injected Current on page 626
Interface		(i) Activate carrier injection with explicitly evaluated boundary conditions for continuity equations during a transient (Carrier Injection with Explicitly Evaluated Boundary Conditions for Continuity Equations on page 647) or if not in transient monitor interface current. Overview on page 625

Table 241 GeometricVariation() [Deterministic Geometric Variations on page 596](#)

Amplitude	=<vector>	μm Vectorial amplitude for displacement.
Amplitude_Iso	=<float>	μm Isotropic amplitude for displacement.
SpaceMid	=<vector>	μm Center of shape function.
SpaceSig	=<vector>	μm Width of shape function.
SpatialShape	=Gaussian	Use Gaussian shape function.
Surface	=<string>	! Name of varying surface.

Table 242 HeatCapacity() [Heat Capacity on page 745](#)

<ident>	[(Table 278)]	PMI model <ident> for lattice heat capacity. Heat Capacity on page 1092
Constant		Constant lattice heat capacity.
PMIModel(	Table 255)	Use multistate configuration-dependent model.
TempDep		* Temperature-dependent lattice heat capacity.

NOTE In [Table 243](#), d is 3 for bulk reactions and 2 for interface reactions.

Table 243 HydrogenReaction() [Reactions Between Mobile Elements on page 450](#)

FieldFromMaterial	=<string>	(i) Material where the electric field is obtained.
FieldFromRegion	=<string>	(i) Region where the electric field is obtained.
ForwardReactionCoef	=<float>	$0/\text{cm}^d\text{s}$ Forward reaction coefficient.
ForwardReactionEnergy	=<float>	0 eV Forward reaction activation energy.
ForwardReactionFieldCoef	=<float>	0 cm/V Forward reaction field coefficient.
LHSCoef (		Define particle numbers to be removed.
	Electron=<int>	0 Number of electrons to be removed.
	Hole=<int>	0 Number of holes to be removed.
	HydrogenAtom=<int>	0 Number of hydrogen atoms to be removed.
	HydrogenIon=<int>	0 Number of hydrogen ions to be removed.
	HydrogenMolecule=<int>)	0 Number of hydrogen molecules to be removed.
Material	=<string>	(i) Material where the recombination rate enters.
Region	=<string>	(i) Region where the recombination rate enters.
ReverseReactionCoef	=<float>	$0/\text{cm}^d\text{s}$ Reverse reaction coefficient.

Table 243 HydrogenReaction() [Reactions Between Mobile Elements on page 450](#)

ReverseReactionEnergy	=<float>	0 eV Reverse reaction activation energy.
ReverseReactionFieldCoef	=<float>	0 cm/V Reverse reaction field coefficient.
RHSCoef (		Define particle numbers to be created.
	Electron=<int>	0 Number of electrons to be created.
	Hole=<int>	0 Number of holes to be created.
	HydrogenAtom=<int>	0 Number of hydrogen atoms to be created.
	HydrogenIon=<int>	0 Number of hydrogen ions to be created.
	HydrogenMolecule=<int>)	0 Number of hydrogen molecules to be created.

Table 244 Incompletelionization() [Chapter 13 on page 273](#)

Dopants	=<string>	Restrict incomplete ionization to dopants named in <string>. Dopant names are separated by spaces.
Model(	<ident>(<string> [Table 278])	Use PMI model <ident> for species named in <string>. Incomplete Ionization on page 1117

Table 245 MetalWorkfunction() [Metal Workfunction on page 242](#)

Randomize(	AtInsulatorInterface	off (r) Randomize workfunction at metal–insulator vertices only. Metal Workfunction Randomization on page 243
	AverageGrainSize=<float>	! (r) Average metal grain size. Metal Workfunction Randomization on page 243
	GrainProbability=(<float>...)	! (r) Probabilities that grains will have a certain workfunction value. Metal Workfunction Randomization on page 243
	GrainWorkfunction=(<float>...)	! eV (r) Workfunction values corresponding to the above probabilities. Metal Workfunction Randomization on page 243
	RandomSeed=<int>	Seed for the random number generator. Metal Workfunction Randomization on page 243
	UniformDistribution)	off (r) Attempt to evenly distribute uniformly sized grains. Metal Workfunction Randomization on page 243

Table 245 MetalWorkfunction() [Metal Workfunction on page 242](#)

SFactor	=<string>	Dataset for the spatial distribution of metal workfunction. Metal Workfunction on page 242
	=<ident> [(Table 278)]	Name of model to be used by the PMI to compute the spatial distribution of metal workfunction. Space Factor on page 1105
	[Factor=<float>]	! Scale factor for normalized SFactor values. Metal Workfunction on page 242
	[Offset=<float>]	! Offset for raw or normalized SFactor values. Metal Workfunction on page 242
Workfunction	=<float>	eV Metal workfunction. Metal Workfunction on page 242
	=(<float>,<float>*3)...	eV , μm , μm , μm Metal workfunction–position quadruplets. Metal Workfunction on page 242

Table 246 Model(), options of stress-dependent models [Chapter 30 on page 687](#)

Deformation(		Linear deformation potential model for computing band structure. Deformation of Band Structure on page 691
	Minimum	Compute conduction and valence band edges using minimum band energies. Deformation of Band Structure on page 691
	ekp	k · p method for electron bands to account for shear strain components. Deformation of Band Structure on page 691
	hkp)	6x6 k · p method for hole bands. Deformation of Band Structure on page 691
DOS(	eMass	Strained electron effective mass and DOS model. Strained Electron Effective Mass and DOS on page 695
	hMass	Strained hole effective mass and DOS model. Strained Hole Effective Mass and DOS on page 697
	hMass(AnalyticLTFit)	Strained hole effective mass and DOS model with analytic lattice temperature fit. Strained Hole Effective Mass and DOS on page 697
	hMass(NumericalIntegration)	Strained hole effective mass and DOS model with numeric integrations. Strained Hole Effective Mass and DOS on page 697

Table 246 Model(), options of stress-dependent models [Chapter 30 on page 687](#)

Mobility(	eFactor[(Table 258)]	Piezoresistive factor for electrons. Isotropic Mobility Factors on page 720
	eMinorityFactor=<float>	1.0 Factor to scale stress effect for minority electrons. Stress Mobility Model for Minority Carriers on page 729
	eMinorityFactor(DopingThreshold=<float>)= <float>	1.0 Factor to scale stress effect for minority electrons with no doping dependency. Stress Mobility Model for Minority Carriers on page 729
	eSaturationFactor=<float>	1.0 Saturation factor for electrons. Dependency of Saturation Velocity on Stress on page 731
	eSubband(Doping)	Strain-induced subband model for electrons with doping dependency. Stress-induced Electron Mobility Model on page 700
	eSubband(EffectiveMass)	Stress-induced change of the electron effective mass. Effective Mass on page 704
	eSubband(Fermi)	Strain-induced subband model for electrons with carrier concentration (Fermi statistics) dependency. Stress-induced Electron Mobility Model on page 700
	eSubband(Scattering)	Strain-induced subband model for electrons with scattering. Intervalley Scattering on page 702
	eTensor[(Table 258)]	Piezoresistive tensor for electrons. Piezoresistance Mobility Model on page 717
	Factor[(Table 258)]	Piezoresistive factor for electrons and holes. Isotropic Mobility Factors on page 720
	hFactor[(Table 258)]	Piezoresistive factor for holes. Isotropic Mobility Factors on page 720
	hMinorityFactor=<float>	1.0 Factor to scale stress effect for minority holes. Stress Mobility Model for Minority Carriers on page 729
	hMinorityFactor(DopingThreshold=<float>)= <float>	1.0 Factor to scale stress effect for minority holes with no doping dependency. Stress Mobility Model for Minority Carriers on page 729
	hSaturationFactor=<float>	1.0 Saturation factor for holes. Dependency of Saturation Velocity on Stress on page 731

Table 246 Model(), options of stress-dependent models [Chapter 30 on page 687](#)

Mobility(	hSixband	Intel stress-induced model for holes. Intel Stress-induced Hole Mobility Model on page 713
	hSixband(Doping)	Intel stress-induced model for holes with doping dependency. Intel Stress-induced Hole Mobility Model on page 713
	hSixband(Fermi)	Intel stress-induced model for holes with carrier concentration (Fermi statistics) dependency. Intel Stress-induced Hole Mobility Model on page 713
	hSubband(Doping)	Strain-induced subband model for holes with doping dependency. Stress-induced Hole Mobility Model on page 708
	hSubband(EffectiveMass)	Stress-induced change of the holes effective mass. Effective Mass on page 708
	hSubband(Fermi)	Strain-induced subband model for holes with carrier concentration (Fermi statistics) dependency. Stress-induced Hole Mobility Model on page 708
	hSubband(Scattering)	Strain-induced subband model for holes with bulk scattering. Scattering on page 709
	hSubband(Scattering(MLDA))	Strain-induced subband model for holes with interface scattering. Scattering on page 709
	hTensor[(Table 258)]	Piezoresistive tensor for holes. Piezoresistance Mobility Model on page 717
	SaturationFactor=<float>	1.0 Saturation factor for electrons and holes. Dependency of Saturation Velocity on Stress on page 731
	Tensor[(Table 258)]	Piezoresistive tensor for electrons and holes. Piezoresistance Mobility Model on page 717

Table 247 MoleFraction() [Mole-Fraction Specification on page 55](#)

Grading((		Alternative way to specify grading; allows a nonzero mole fraction and different distance of grading from different parts of the boundaries.
	GrDistance=<float>	μm Distance in the direction normal to the specified interface, where linear interpolation of mole fraction(s) from the constant value to the specified boundary mole fraction(s) occurs.
	RegionInterface=(<string>*2)	Restrict this grading to the given interface (applied to all interfaces by default).
	xFraction=<[0,1]>	Boundary xMoleFraction.
	yFraction=<[0,1]>...)	Boundary yMoleFraction.
GrDistance	=<float>	μm Distance in the direction normal to the boundaries of the specified region(s) where linear interpolation of mole fraction(s) from the specified constant value to 0 occurs.
RegionName	= [<string>...]	List of regions where the mole-fraction specification will take effect.
	=<string>	Regions where the mole-fraction specification will take effect.
xFraction	=<[0,1]>	Constant value of xMoleFraction.
yFraction	=<[0,1]>	Constant value of yMoleFraction.

Table 248 MSConfig() [Chapter 18 on page 425](#)

BandEdgeShift(	<ident> [(Table 278)] [<int>])	Compute band-edge shifts for conduction and valence by PMI model <ident> with optional constructor argument <int>. Apparent Band-Edge Shift on page 435
Conc	=<float>	0 cm^{-d} Concentration of states ($d = 3$ for bulk; $d = 2$ for interface MSCs).
eBandEdgeShift(	<ident> [(Table 278)] [<int>])	Compute band-edge shift for conduction band by PMI model <ident> with optional constructor argument <int>. Apparent Band-Edge Shift on page 435
Elimination		+ Switch solving algorithm.
hBandEdgeShift(	<ident> [(Table 278)] [<int>])	Compute band-edge shift for valence band by PMI model <ident> with optional constructor argument <int>. Apparent Band-Edge Shift on page 435
Name	=<string>	! Identifier of MSConfig.

Table 248 MSConfig() [Chapter 18 on page 425](#)

State(		! Define a state (at least two required).
	Charge=<int>	0 Number of positive elementary charges.
	Hydrogen=<int>	0 Number of hydrogen atoms.
	Name=<string>)	! Identifier of state.
Transition(	Table 260)	! Define a transition.

Table 249 NBTI() [Two-Stage NBTI Degradation Model on page 457](#)

Conc	=<float>	0 cm ⁻² Concentration of NBTI traps.
NumberOfSamples	=<int>	0 Number of random samples.
hSHEDistribution		- Use hole-energy distribution function.

Table 250 Noise() [Chapter 23 on page 581](#)

DiffusionNoise(	e_h_Temperature	Diffusion noise; noise temperatures are the respective carrier temperatures. Diffusion Noise on page 585
	eTemperature	Noise temperature is electron temperature for electrons, lattice temperature for holes.
	hTemperature	Noise temperature is lattice temperature for electrons, hole temperature for holes.
	LatticeTemperature)	* Noise temperature is lattice temperature.
Doping(	BandGapNarrowing	Random dopant fluctuations, including their effect on bandgap narrowing. Random Dopant Fluctuations on page 587
	Mobility)	Random dopant fluctuations, including their effect on mobility.
FlickerGRNoise		Bulk flicker noise. Bulk Flicker Noise on page 586
GeometricFluctuations	<string>	Geometric fluctuations for surface <string>. Random Geometric Fluctuations on page 588
MonopolarGRNoise		Equivalent monopolar noise. Equivalent Monopolar Generation–Recombination Noise on page 586
TrapConcentration		Trap concentration fluctuations. Random Trap Concentration Fluctuations on page 590

Table 250 Noise() [Chapter 23 on page 581](#)

Traps		Trapping noise. Trapping Noise on page 586
WorkfunctionFluctuations	<string>	Workfunction fluctuations for surface <string>. Random Workfunction Fluctuations on page 590

Table 251 Optics() [Transfer Matrix Method on page 538](#) and [Beam Propagation Method on page 559](#)

Table 218		Optics stand-alone. Optics Stand-alone Option on page 887
BPMScalar (	Table 186)	Scalar beam propagation method (BPM) solver. Beam Propagation Method on page 559
TMM (	Table 208)	Transfer matrix method (TMM) solver. Transfer Matrix Method on page 538

Table 252 Optics() [Specifying the Type of Optical Generation Computation on page 470](#)

ComplexRefractiveIndex (	Table 233)	Complex refractive index models. Complex Refractive Index Model on page 496
Excitation (		Specification of excitation parameters. Setting the Excitation Parameters on page 486
	Intensity=<float>	W_{cm}^{-2}
	Phi=<float>	θ deg Angle between projection of propagation direction on xy plane and x-axis.
	Polarization=<float>	[0,1] Definition of polarization as in Using Transfer Matrix Method on page 543 .
	Polarization=<ident>	Transverse electric (TE) or transverse magnetic (TM) polarized light.
	PolarizationAngle=<float>	deg Angle between H-field and $\mathbf{z} \times \hat{\mathbf{k}}$ used for vectorial solvers only.
	Theta=<float>	θ deg Angle between propagation direction and z-axis.
	Wavelength=<float>)	μm
	Window(Table 262)	
OpticalGeneration (	Table 196)	Optical generation models. Specifying the Type of Optical Generation Computation on page 470

Table 252 Optics() [Specifying the Type of Optical Generation Computation on page 470](#)

OpticalSolver (		Specification of optical solver method. Specifying the Optical Solver on page 480
	BPM(Table 186)	
	FDTD(Table 190)	
	FromFile(Table 191)	
	OptBeam(Table 195)	
	RayTracing(Table 200)	Raytracer on page 510
	TMM(Table 208)	See Using Transfer Matrix Method on page 543 . Note that Excitation section must not be defined in TMM section.

Table 253 ParameterVariation() [Parameter Variations on page 597](#)

Factor	=<float>	1 1 Multiplier for original parameter value.
Material	=<string>	Material location of varied parameter.
MaterialInterface	=<string>	Material interface location of varied parameter.
Model	=<string>	! Model to which varied parameter belongs.
Parameter	=<string>	! Name of the varied parameter.
Region	=<string>	Region location of varied parameter.
RegionInterface	=<string>	Region interface location of varied parameter.
Summand=	=<float>	0 Summand to original parameter value.
Value=	=<float>	Modified parameter value.

Table 254 Piezo() [Chapter 30 on page 687](#)

Model(	Table 246)	Stress-dependent models. Deformation of Band Structure on page 691 to Stress-dependent Mobility Models on page 700
OriKddX	=<vector>	(1 0 0) Miller indices of the stress system relative to the simulation system.
OriKddY	=<vector>	(0 1 0) Miller indices of the stress system relative to the simulation system.
Stress	=(<float>*6)	Pa Components xx , yy , zz , yz , xz , xy of stress tensor.
	=<ident> [(Table 278)]	Use PMI model <ident> to compute stress. Stress on page 1101

Table 255 PMIModel() [Multistate Configuration-dependent Bulk Mobility on page 1029](#),
[High-Field Saturation with Two Driving Forces on page 1050](#), [Multistate Configuration-dependent Thermal Conductivity on page 1088](#), [Multistate Configuration-dependent Heat Capacity on page 1095](#)

Index	=<int>	0 Number passed to and interpreted by PMI model.
MSConfig	=<string>	"" Name of multistate configuration on which PMI depends.
Name	=<string>	! Name of PMI model.
String	=<string>	"" String passed to and interpreted by PMI model.
Table 278		PMI parameters.

Table 256 RandomizedVariation() in Physics{} [Statistical Impedance Field Method on page 592](#)

Doping		+ Doping variations. Doping Variations on page 593
TrapConcentration		- Trap concentration variation. Trap Concentration Variations on page 594
Workfunction(	AverageGrainSize=<float>	! μm Average metal grain size. Workfunction Variations on page 594
	GrainProbability=($\langle 0, \rangle \dots$)	! Probability for grain orientation. Workfunction Variations on page 594
	GrainWorkfunction=($\langle \text{float} \rangle \dots$)	! eV Grain workfunctions. Workfunction Variations on page 594
	Surface=<string>)	! Surface of variation. Workfunction Variations on page 594

Table 257 Schroedinger() 1D Schrödinger Solver on page 281

DensityTail	=Extrapolate	* Use estimate for lowest noncomputed subband energy as $E_{\max, v}$. Eq. 206, p. 287
	=MaxEnergy	Use highest computed subband energy as $E_{\max, v}$. Eq. 206, p. 287
Electron		- Solve Schrödinger equation for electrons.
EnergyInterval	[(<carrier>)]=<float>	0 eV Highest energy to which eigensolutions are computed, measured from the lowest interior potential point on the nonlocal line on which the Schrödinger equation is solved. When 0, only bound solutions are computed.
Error	[(<carrier>)]=<(0,)>	1e-5 eV Precision target for eigenenergies.
Hole		- Solve Schrödinger equation for holes.
MaxSolutions	[(<carrier>)]=<0..>	5 Maximum number of eigensolutions computed per ladder.
Smooth	=<float>	0 cm Length (measured from end of nonlocal line) over which to blend from classical to quantum density.

Table 258 Tensor(), eTensor, hTensor, Factor(), eFactor(), hFactor() Piezoresistance Mobility Model on page 717

<ident>	[Table 278]	Use PMI model <ident> to compute piezoresistive prefactors (piezoresistance tensor model only). Piezoresistive Coefficients on page 1130
AutoOrientation		off Use parameter set based on orientation of nearest interface. Auto-Orientation for Piezoresistance on page 722
ChannelDirection	=<1..3>	1 Channel direction (piezoresistance factor model only). Isotropic Mobility Factors on page 720
Enormal		off Use piezoresistive prefactors (piezoresistance tensor model only). Enormal- and MoleFraction-dependent Piezo Coefficients on page 723
FirstOrder		* Use the first-order piezoresistance model. Piezoresistance Mobility Model on page 717
Kanda		off Include temperature and doping dependency. Doping and Temperature Dependency on page 719
ParameterSetName	=<string>	Name of Piezoresistance parameter set. Named Parameter Sets for Piezoresistance on page 722

Table 258 Tensor(), eTensor, hTensor, Factor(), eFactor(), hFactor() [Piezoresistance Mobility Model on page 717](#)

SecondOrder		Use the second-order piezoresistance model. Piezoresistance Mobility Model on page 717
SFactor	=<string>	Dataset for the spatial distribution of the mobility enhancement factor. Additional Factor Model Options on page 723
	=<ident> [(Table 278)]	Name of model to be used by the PMI to compute the spatial distribution of the mobility enhancement factor. Space Factor on page 1105

Table 259 ThermalConductivity() [Thermal Conductivity on page 747](#)

<ident>	[(Table 278)]	Use PMI model <ident> for thermal conductivity. Thermal Conductivity on page 1084
Constant	Conductivity	Use constant conductivity.
	Resistivity	Use constant resistivity.
Formula		Use the built-in strategy for thermal conductivity. Thermal Conductivity on page 1084
PMIModel(	Table 255)	Use multistate configuration-dependent model.
TempDep	Conductivity	Use Eq. 860, p. 747 .
	Resistivity	Use Eq. 859, p. 747 .

Table 260 Transition() in MSConfig() [Chapter 18 on page 425](#)

CEModel(	<ident> [(Table 278)] [<int>])	! Use PMI_TrapCaptureEmission model <ident> with optional constructor argument <int>.
FieldFromInsulator		- Use the insulator electric field instead of the semiconductor electric field for the MSCs defined at the semiconductor-insulator interface.
From	=<string>	! Interacting state.
Name	=<string>	! Identifier of transitions.
Reservoirs(		List of reservoirs for particle conservation.
	<string>(Particles =<int>)...	Reservoir <string> and number of involved particles <int>.
To	=<string>	! Reference state.

NOTE In Table 261, $d = 3$ for bulk traps and $d = 2$ for interface traps.

Table 261 Traps()... Chapter 17 on page 407, Chapter 19 on page 441

Acceptor		Acceptor trap type. Trap Types on page 408
ActEnergy	=<float>	eV Equilibrium activation energy of hydrogen on Si-H bonds, ϵ_A^0 . Chapter 19 on page 441
Add2TotalDoping(		Include the trap concentration in the acceptor, donor, and total doping concentrations used to compute mobility, lifetimes, and so on. Doping Specification on page 49
	ChargedTraps)	Include the charged trap concentration in the acceptor or donor doping concentrations used to compute mobility. Doping Specification on page 49
BondConc	=<float>	cm^{-d} Total silicon dangling bond concentration N . Chapter 19 on page 441
CBRate	=<ident> [(Table 278)]	Use PMI <ident> to compute electron capture and emission rate for conduction band.
	=(<ident> [(Table 278)] <int>)	Use PMI <ident> with constructor argument <int> to compute electron capture and emission rate for conduction band. Trap Capture and Emission Rates on page 1108
Conc	=<float>	cm^{-d} or $\text{eV}^{-1}\text{cm}^{-d}$ Concentration or the peak density of the trap distribution. Energetic and Spatial Distribution of Traps on page 408
CritConc	=<float>	cm^{-d} Critical concentration N_{crit} , default $0.1N$. Chapter 19 on page 441
CurrentEnhan	(<float>*4)	$(0 \ 1 \ 0 \ 1) \text{ cm}^{2\rho_{\text{Tun}}}\text{A}^{-\rho_{\text{Tun}}}, 1, \text{ cm}^{2\rho_{\text{HC}}}\text{A}^{-\rho_{\text{HC}}}, 1$ Parameters of the tunneling and hot carrier-dependent terms of the depassivation constant, δ_{Tun} , ρ_{Tun} , δ_{HC} , and ρ_{HC} . Chapter 19 on page 441
Degradation		Use degradation model based on kinetic equation. Chapter 19 on page 441
	(PowerLaw)	Use degradation model based on power law. Chapter 19 on page 441
DePasCoef	=<float>	s^{-1} Depassivation coefficient V_0 at the passivation conditions (the equilibrium at the passivation temperature T_0). Chapter 19 on page 441
DiffusionEnhan	=(<float>*6)	$\text{cm cm}^2\text{s}^{-1} \text{ eV cm s}^{-1} \text{ cm}^{-3} 1$ Parameters of hydrogen diffusion in oxide, x_p , D_0 , ϵ_H , k_p , N_H^0 , and N_{ox} in Eq. 476, p. 443, Chapter 19 on page 441 .
Donor		Donor trap type. Trap Types on page 408

Table 261 Traps()... Chapter 17 on page 407, Chapter 19 on page 441

eBarrierTunneling(	NonLocal=<string>	Use nonlocal tunneling from the conduction band at reference surface of all connected unnamed, and all listed, connected named nonlocal meshes. Tunneling and Traps on page 417
	TwoBand)	Use two-band dispersion. WKB Tunneling Probability on page 618
eConstEmissionRate	=<float>	s^{-1} Constant electron emission rate term to the conduction band e_{const}^n . Local Trap Capture and Emission on page 414
eGfactor	=<float>	Electron degeneracy factor g_n . Trap Occupation Dynamics on page 412
eJfactor	=<float>	Electron J-model factor g_n^J . Local Trap Capture and Emission on page 414
ElectricField	[(<carrier>)]	Field-dependent model for cross sections. J-Model Cross Sections on page 415
EnergyMid	=<float>	eV Central energy E_0 of the trap distribution. Eq. 428, p. 408
EnergyShift	=<ident> [(Table 278)]	Use PMI <ident> to compute trap energy shift.
	=(<ident> [(Table 278)] <int>)	Use PMI <ident> with constructor argument <int> to compute trap energy shift. Energetic and Spatial Distribution of Traps on page 408 , Trap Energy Shift on page 1112
EnergySig	=(<0,>	eV Width E_S of the trap distribution. Eq. 428, p. 408
eNeutral		Electron trap type. Trap Types on page 408
eSHEDistribution	(<float>*3)	($0 \ 0 \ 0$) $\text{cm}^2 \text{A}^{-1}$, eV, eV Parameters of the electron energy-dependent terms of the depassivation constant, δ_{SHE} , ϵ_{th} , and ϵ_a . Trap Degradation Model on page 441
Exponential		Exponential energetic distribution. Energetic and Spatial Distribution of Traps on page 408 , Eq. 428, p. 408
eXsection	=<[0,>	cm^2 Electron capture cross section σ_n^0 . Local Trap Capture and Emission on page 414
FieldEnhan	(<float>*4)	($0 \ 1 \ 0 \ 1$) $\text{eV cm}^{\rho_{\parallel}} \text{V}^{-\rho_{\parallel}}$, 1, $\text{eV cm}^{\rho_{\perp}} \text{V}^{-\rho_{\perp}}$, 1 Parameters of the electric field-dependent terms of the Si-H bond energy and the activation energy, $\delta_{\parallel}$, $\rho_{\parallel}$, $\delta_{\perp}$, and $\rho_{\perp}$. Chapter 19 on page 441
FixedCharge		Fixed charge. Trap Types on page 408
fromCondBand		Zero point for E_0 is conduction band. Eq. 429, p. 409
fromMidBandGap		Zero point for E_0 is intrinsic energy. Eq. 429, p. 409
fromValBand		Zero point for E_0 is valence band. Eq. 429, p. 409

Table 261 Traps()... Chapter 17 on page 407, Chapter 19 on page 441

Gaussian		Gaussian energetic distribution. Energetic and Spatial Distribution of Traps on page 408 , Eq. 428, p. 408
hBarrierTunneling(	NonLocal=<string>	Use nonlocal tunneling from the valence band at reference surface of all connected unnamed, and all listed, connected named nonlocal meshes. Tunneling and Traps on page 417
	TwoBand)	Use two-band dispersion. WKB Tunneling Probability on page 618
hConstEmissionRate	=<float>	s^{-1} Constant hole emission rate to the valence band e_{Const}^p . Local Trap Capture and Emission on page 414
hGfactor	=<float>	Hole degeneracy factor g_p . Trap Occupation Dynamics on page 412
hJfactor	=<float>	Hole J-model factor g_p^J . Local Trap Capture and Emission on page 414
hNeutral		Hole trap type. Trap Types on page 408
hSHEDistribution	(<float>*3)	$(0 \ 0 \ 0) \text{ cm}^2 \text{ A}^{-1}$, eV, eV Parameters of the hole energy-dependent terms of the depassivation constant, δ_{SHE} , ϵ_{th} , and ϵ_a . Trap Degradation Model on page 441
HuangRhys	=<[0,]>	Huang–Rhys factor. Tunneling and Traps on page 417
hXsection	=<[0,]>	cm^2 Hole capture cross section σ_p^0 . Local Trap Capture and Emission on page 414
Level		Trap with single energy level. Energetic and Spatial Distribution of Traps on page 408 , Eq. 428, p. 408
Makram-Ebeid	[(<carrier> <simpleCapt>)]	Use Makram-Ebeid–Lannoo model. Local Capture and Emission Rates Based on Makram-Ebeid–Lannoo Phonon-assisted Tunnel Ionization Model on page 416
Material	=<string>	(i) Material to which energy specification refers. Energetic and Spatial Distribution of Traps on page 408
Name	=<string>	Identifier for trap.
PasCoef	=<float>	s^{-1} Passivation coefficient γ_0 . By default, computed automatically to provide the equilibrium, $v_0 N_{\text{hb}}^0 / (N - N_{\text{hb}}^0)$. Chapter 19 on page 441
PasTemp	=<float>	300 K Passivation temperature T_0 . Chapter 19 on page 441
PasVolume	=<float>	$0 \text{ cm}^2 \text{cm}^3$ Passivation volume Ω . Chapter 19 on page 441
PhononEnergy	=<[0,]>	eV Phonon energy for inelastic tunneling. Tunneling and Traps on page 417
PooleFrenkel	[(<carrier>)]	Use Poole–Frenkel model. Poole–Frenkel Model for Cross Sections on page 415

Table 261 Traps()... [Chapter 17 on page 407](#), [Chapter 19 on page 441](#)

PowerEnhan	(<float>*3)	(0 0 0) 1, $V^{-1}\text{cm}$, $V^{-1}\text{cm}$ Parameters in the chemical potential for kinetic equation degradation and the power for degradation by power law, β_0 , $\beta_{//}$, and $\beta_{\perp}$. Chapter 19 on page 441
Randomize	[=<int>]	Randomize traps. Trap Randomization on page 411
ReferencePoint	=<vector>	Coordinate where to take the data for EnergyShift .
Region	=<string>	(i) Region to which energy specification refers. Energetic and Spatial Distribution of Traps on page 408
SFactor	=<string>	Dataset for the spatial distribution of the traps. Energetic and Spatial Distribution of Traps on page 408
	=<ident> [(Table 278)]	Name of model to be used by the PMI to compute the spatial trap distribution. Space Factor on page 1105
SingleTrap		Use a single trap. Specifying Single Traps on page 410
SpaceMid	=<vector>	(0 0 0) μm Center of spatial distribution. Energetic and Spatial Distribution of Traps on page 408
SpaceSig	=<vector>	(1e100 1e100 1e100) μm Width of spatial distribution. Energetic and Spatial Distribution of Traps on page 408
SpatialShape	=Gaussian	Use Gaussian shape function. Energetic and Spatial Distribution of Traps on page 408
	=Uniform	* Use Uniform shape function. Energetic and Spatial Distribution of Traps on page 408
Table	=(<float>*2...)	eV $\text{eV}^{-1}\text{cm}^{-d}$ Pairs of energy and concentration of a tabular approximation to a trap distribution. Energetic and Spatial Distribution of Traps on page 408 , Eq. 428
TrapVolume	=<[0,)>	μm^3 Interaction volume for nonlocal tunneling. Tunneling and Traps on page 417
Tunneling(	Hurkx[(<carrier>)]	Use Hurkx trap-assisted tunneling model. Hurkx Model for Cross Sections on page 415
Uniform		Uniform energetic distribution. Energetic and Spatial Distribution of Traps on page 408 , Eq. 428, p. 408
VBRate	=<ident> [(Table 278)]	Use PMI <ident> to compute hole capture and emission rate for valence band.
	=(<ident> [(Table 278)] <int>)	Use PMI <ident> with constructor argument <int> to compute hole capture and emission rate for valence band. Trap Capture and Emission Rates on page 1108

Table 262 Window [Illumination Window on page 487](#)

Circle(	Radius=<float>)	
Line(	Dx=<float>	
	X1=<float>	
	X2=<float>)	
Origin	=(<float>*3)	Global location of window origin.
OriginAnchor	=<ident>	Center Origin anchor of illumination window in terms of cardinal direction (North , South , East , West , NorthEast , SouthEast , NorthWest , SouthWest).
Polygon(	(<float>*2)*n)	Specification of simple polygon using several vertices.
Polygon(	((<float>*2)*n)*m	Specification of complex polygon using several loops of vertices.
Rectangle(	Dx=<float>	
	Dy=<float>	
	Corner1=<float>*2)	
	Corner2=<float>*2))	
RotationAngles	=(<float>*3)	Angles specifying global orientation of window coordinate system.
XDirection	=(<float>*3)	(1, 0, 0) Direction of x-axis.
YDirection	=(<float>*3)	(0, 1, 0) Direction of y-axis.

Plotting

Table 263 Plot{} [Device Plots on page 144](#)

Table 140		Scalar plot data. Device Plots on page 144
Table 140/Element		Scalar plot datasets defined on elements.
Table 140/RegionInterface		Scalar plot data on region interfaces; supported for just a few datasets. Interface Plots on page 147
Table 140/Tensor		Tensorial plot data. Device Plots on page 144
Table 141/Vector		Vectorial plot data. Device Plots on page 144

Table 263 Plot{} Device Plots on page 144

Table 142/SpecialVector		Special vectorial plot data; supported for the SHE distribution data. SHE Distribution Hot-Carrier Injection on page 633
Table 268		Occupation rates of MSConfig states. Specifying Multistate Configurations on page 427
DatasetsFromGrid		Copy datasets from TDR grid file. What to Plot on page 144
[(	<ident>...)]	Datasets to be copied.

Table 264 Average(), Integrate(), Maximum(), and Minimum() Tracking Additional Data in the Current File on page 140

Table 276		Sample over location.
Coordinates		Print coordinates where maximum, minimum, or average occurs.
DopingWell	<vector>	Sample over well, defined by point in the well.
Everywhere		Sample over entire device.
Insulator		Sample over all insulator regions of the device.
Name	=<string>	Name for sampled data to be used in output.
Semiconductor		Sample over all semiconductor regions of the device.
Window[	<vector> <vector>]	Sample over the window, defined by two specified corners.

Table 265 CurrentPlot{} Tracking Additional Data in the Current File on page 140

<int>		Print data at vertex <int>.
<vector>		μm Print data at coordinate <vertex>.
Average(	Table 264)	Print average over given domain.
Integrate(	Table 264)	Print integral over given domain.
Maximum(	Table 264)	Print maximum in given domain.
Minimum(	Table 264)	Print minimum in given domain.
Model	=<string>	Select a model for printing of parameters.
ModelParameter	=<string>	Select model parameter for plotting when using unified interface for optical generation computation. Parameter Ramping on page 494
Parameter	=<string>	Print parameter <string>.

Table 266 FarfieldPlot{} [Far Field on page 878](#)

Interval	=<int>	Number of points per axis.
Range	=(<float>*2)	degree Horizontal and vertical angle ranges.
Scalar1D		* Type of far-field plot.
Scalar2D		Type of far-field plot.
Vector2D		Type of far-field plot.

Table 267 GainPlot{} [Modal Gain as Function of Energy/Wavelength on page 920](#)

Intervals	=<int>	Number of points to plot for each gain curve.
PlotLasingSpectrum		Enable plotting of laser power spectra. Plotting the Laser Power Spectrum on page 788
Range	=(<float>*2)	eV Energy range of the gain plot.
	= Auto	Automatically find the energy range (applies to LEDs only).

Table 268 MSConfig in Plot{} [Specifying Multistate Configurations on page 427](#)

MSConfig		All state occupations of all MSConfig.
	(<string>)	All state occupations of MSConfig <string>.
	(<string1> <string2>)	Occupation of state <string2> of MSConfig <string1>.

Table 269 NoisePlot{} [Noise Output Data on page 601](#)

Table 140	Scalar plot data.
Table 141 /Vector	Vector plot data.
AllLNS	All used local noise sources.
AllLNVSD	All used local noise voltage spectral densities.
AllLNVXVSD	All use local noise voltage cross-correlation spectral densities.
GreenFunctions	All used Green's functions and their gradients.

Table 270 NonLocalPlot{} Visualizing Data Defined on Nonlocal Meshes on page 167

Table 140		Scalar data.
EigenEnergy		Eigenenergies. 1D Schrödinger Solver on page 281
(	Electron[(Number=<int>)]	Restrict output to [<int> lowest] electron eigensolutions.
	Hole[(Number=<int>)]	Restrict output to [<int> lowest] hole eigensolutions.
WaveFunction		Wavefunctions. 1D Schrödinger Solver on page 281
(	Electron[(Number=<int>)]	Restrict output to [<int> lowest] electron eigensolutions.
	Hole[(Number=<int>)]	Restrict output to [<int> lowest] hole eigensolutions.

Table 271 TensorPlot(){} Visualizing Results on Native Tensor Grid on page 567

Name	=<string>	! Name of tensor plot. The file name for the TensorPlot given in the File section is appended by this name.
OutputLevel	=maximum	For bidirectional BPM, tensor plots are generated not only for the final solution, but also after every iteration if maximum is specified.
Xconst	=<float>	μm X-coordinate.
Xmax	=<float>	μm Maximum x-coordinate.
Xmin	=<float>	μm Minimum x-coordinate.
Yconst	=<float>	μm Y-coordinate.
Ymax	=<float>	μm Maximum y-coordinate.
Ymin	=<float>	μm Minimum y-coordinate.
Zconst	=<float>	μm Z-coordinate.
Zmax	=<float>	μm Maximum z-coordinate.
Zmin	=<float>	μm Minimum z-coordinate.

Table 272 VCSELNearFieldPlot{} [VCSEL Near Field and Far Field on page 885](#)

NumberOfPoints	=<int>	Discretization of the near-field plot.
Position	=<float>	y -coordinate of the near field.
Radius	=<float>	Radius of near field.

Various

Table 273 Locations, all

Table 274		Bulk location.
Table 275		Interface location.
Electrode	=<string>	Name of an electrode that must exist in the device.

Table 274 Locations, bulk (dimension is that of the device)

Material	=<string>	Name of a material.
Region	=<string>	Name of a region that must exist in the device.

Table 275 Locations, interface (dimension is one less than that of the device)

MaterialInterface	=<string>	Material interface of the form "<ident1>/<ident2>", with <ident1> and <ident2> as the names of materials.
RegionInterface	=<string>	Region interface of the form "<ident1>/<ident2>", with <ident1> and <ident2> as the names of regions that must exist in the device and must have a common interface.

Table 276 Locations, noncontact

Table 274	Bulk location.
Table 275	Interface location.

Table 277 Optics() in Physics{}

Table 218		Optics stand-alone. Optics Stand-alone Option on page 887
BPMScalar (	Table 186)	Scalar beam propagation method (BPM) solver. Beam Propagation Method on page 559
TMM (	Table 186)	Transfer matrix method (TMM) solver. Transfer Matrix Method on page 538

Table 278 PMI parameters [Command File of Sentaurus Device on page 989](#)

<ident>	=<float>	Scalar floating-point parameter.
	=<string>	Scalar string parameter.
	=(<float> ...)	Vector of floating-point parameters.
	=(<string> ...)	Vector of string parameters.

G: Command File Overview

Various